Algorithms From the Ground Up

A Deep, Conversational Reader Before CLRS

From Curiosity to Rigorous Algorithmics

June 2026

Preface

Welcome. If you are holding this book, you are probably about to study algorithms seriously for the first time — maybe for a course, maybe for an interview, maybe just because something in you wants to know how computers actually *think* their way through a problem. Whatever brought you here, I am glad you came, and I want to tell you on the very first page what this book does differently: it slows down.

Most algorithms books are written by experts, for people who are on their way to becoming experts. They are precise, they are correct, and they are often genuinely beautiful. But they move fast. They state a theorem, prove it in five tight lines, and trust you to fill in the ten lines of intuition that the author worked out years ago and no longer remembers needing. If you already see it, the brevity is a gift. If you do not yet see it, the same brevity feels like a wall.

This book supplies those missing lines. It draws the picture, walks you through a tiny numeric example by hand, points out the exact place where everyone gets confused, and only *then* writes down the formal statement — at which point the formal statement feels obvious, because you already understand the thing it is describing.

Who this book is for

You should read this book if any of the following sound like you.

- You are about to take an algorithms course, or you are in one right now and the textbook feels like it is written in a language you almost speak.
- You have done some programming — you can write a loop, you know what an array is, you have used a list or a dictionary — but nobody ever sat you down and explained *why* one way of solving a problem is a thousand times faster than another.
- You have heard words like "$O(n \log n)$", "recursion", "dynamic programming", and "NP-complete" and you can repeat them, but you could not yet derive them, defend them, or teach them to someone else.
- You tried to read CLRS — *Introduction to Algorithms* by Cormen, Leiserson, Rivest, and Stein — and you bounced off it. That is not a verdict on you. CLRS is a magnificent reference and a demanding first read. This book exists to make your *second* encounter with it go smoothly.

You do *not* need to be a strong mathematician. You need to be comfortable with high-school algebra, willing to count things carefully, and patient enough to trace an example by hand even when you

"sort of" see the answer. Every more advanced tool we need — summations, logarithms, a little probability, proof by induction — is built up gently right where we use it, and collected in the appendices for reference.

Why this matters

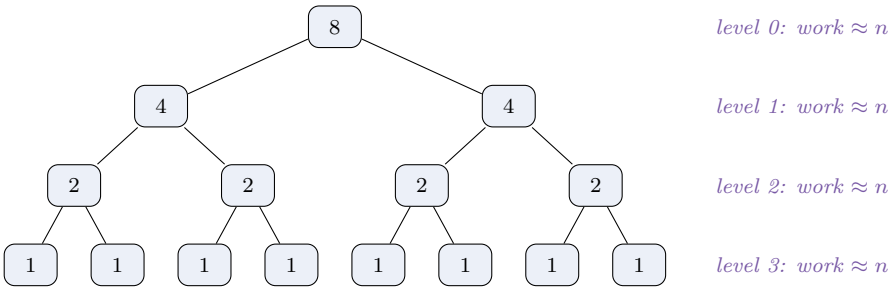
Why does any of this matter once you leave the classroom? Because the difference between a good algorithm and a careless one is the difference between a search that returns in 50 milliseconds and one that times out; between a map app that routes you across a country instantly and one that spins forever; between a system that scales to a billion users and one that falls over at a thousand. What separates them is *how many steps* the algorithm takes. Learning to count those steps — and to bring the count down — is one of the most durable, transferable skills in all of computing. Hardware gets faster every year; a better algorithm can make something possible that no amount of hardware ever would.

The one idea behind the whole book: intuition, then rigor

One principle shapes every chapter. **First we build a mental picture you can trust. Then we make it precise.** In that order, never the reverse.

Why that order? Because precision without a picture amounts to memorization, and memorized facts fade quickly. If I hand you the recurrence $T(n) = 2T(n/2) + \Theta(n)$ and tell you it solves to $\Theta(n \log n)$, you can nod and copy it down, and forget it by Thursday. But if you have *seen* a list of eight numbers split into two halves, then four, then eight singletons, and you have counted with your own finger that the splitting takes about $\log_2 8 = 3$ levels and each level does about n units of work — then the recurrence is no longer an arbitrary formula to memorize; it is a concise description of a picture you already understand.

We will draw exactly this kind of picture — a *recursion tree* — many times in the book. Look at one now, even before you can read every symbol on it. Each node is a piece of the problem; an arrow splits a piece into the two smaller pieces it asks for help with; the number inside a node is how big that piece is.



The tree has $\log_2 8 = 3$ levels of splitting beneath the root, so four rows in all. Look across any single row and add up the piece sizes: 8 on top; $4 + 4 = 8$ next; $2 + 2 + 2 + 2 = 8$; finally eight 1s. Every row sums to $n = 8$, which is why each level does about n units of work. Four levels, each costing about n, gives a total of about $n \cdot \log_2 n$, which is the $\Theta(n \log n)$ the recurrence predicted.

Intuition

Think of every algorithm in this book as a *story with a shape*. A loop is a story where the same small thing happens again and again until a finish line is crossed. Recursion is a story where a big problem politely asks two smaller copies of itself for help. A greedy algorithm is a story where you grab the best thing in front of you and never look back. Before we ever count operations or prove a bound, we will get the *shape of the story* clear in your head. The math is what we use to check that the story actually ends well, and to find out how long it takes.

So a typical section moves through four stages:

1. **A picture.** The core mental image, in plain language, often with a diagram. "Imagine the cards in your left hand are already sorted…"

2. **A tiny worked example.** Real numbers, traced step by step, so you can watch the idea move.

3. **The pseudocode.** Now that you know what it *does*, here is exactly *how* it does it, written carefully.

4. **The rigor.** A loop invariant or an induction that proves it is correct, and an analysis that derives — not asserts — how fast it is.

We will not skip the fourth stage. This is a book that respects proofs; "it seems to work" is never the end of a discussion here. But we earn the proof by understanding first, so that when the formal statement arrives it reads as a confirmation of something you already believe, not as a hurdle.

The colored boxes: a visual language

Throughout the book you will meet recurring colored boxes. Each one marks a particular *type of thinking*, and learning to recognize them on sight will make your reading faster and your studying sharper. Here is the full vocabulary, with one live example of each so you know exactly what to expect.

Focus

A **Focus** box holds the one or two things you must carry out of a section. If you remember nothing else, remember what is in here. Example: *Insertion sort is fast when the data is nearly sorted and slow when it is reversed — its cost is governed by the number of out-of-order pairs, not just the length.*

Intuition

An **Intuition** box is the napkin drawing — the mental model in everyday words, before any formality. This is where I try to make you *see* the idea.

Formal

A **Formal** box states something precisely: a definition, an invariant, a theorem. After the intuition has done its job, this is the version you could quote on an exam or cite in a proof. Example: *A sorting algorithm is stable if, whenever two elements have equal keys, their relative order in the output matches their relative order in the input.*

Common trap

A **Common trap** box names a mistake that almost everyone makes, so that you can make it on purpose, here, safely — instead of by accident on a test or in production. Example: *Off-by-one errors cluster around loop bounds. Writing $i < n$ when you meant $i \leq n$ is the single most common bug in all of the pseudocode you will write this year.*

Walking question

A **Walking question** is a prompt to close the book and think. Genuinely — go for a walk, do the dishes, let it simmer. The best learning happens slightly after you stop reading. Example: *If you could only sort by repeatedly swapping adjacent elements, what arrangement of n numbers would be the most expensive to sort, and exactly how many swaps would it take?*

Reading note

A **History and connection** box tells you where an idea came from — and *points directly to the matching place in CLRS* so you always know how this book maps onto the standard reference. Example: *Quicksort was invented by Tony Hoare in 1959 while he was a visiting student in Moscow, trying to sort words for machine translation. We treat it in Chapter 7, exactly mirroring CLRS Chapter 7.*

Why this matters

A **Why this matters** box connects the idea to the real world — to the systems, products, and engineering problems that rely on it in practice.

Cost summary

A **Cost summary** box collects the running times and space usage in one tidy place after we have derived them, so you can find them again at a glance. Example: *Best case* $\Theta(n)$; *average and worst case* $\Theta(n^2)$; *extra space* $\Theta(1)$.

Algorithm lab

An **Algorithm lab** is a hands-on "trace it yourself" exercise embedded in the text. You are handed a concrete input and asked to run the algorithm by hand, writing down the state at each step. There is no substitute for this. Reading an algorithm and running an algorithm are as different as reading sheet music and playing it.

Formal

There are a few more boxes you will meet occasionally. A **Reading note** gives advice on *how* to read a tricky passage. A **Pause** box is a gentle checkpoint inviting you to make sure the last idea landed before moving on. They work just as their names suggest.

Focus

You do not need to memorize this list. After two or three chapters the colors will do the work for you: green means "here is the picture," grey means "here is the precise truth," red means "careful here," orange means "go think," and so on. The boxes give every chapter the same predictable structure.

Where algorithms came from: from mechanical gears to universal recipes

Before we write down our first loop, it is worth asking why we have a discipline called "algorithms" at all. Humans have calculated for thousands of years. Why didn't our ancestors call long division or navigation tables an "algorithm"?

The answer lies in a long effort to separate *the recipe* from *the machine executing it*.

History and connection

In ninth-century Baghdad, at the House of Wisdom (*Bayt al-Hikma*), Muhammad ibn Musa al-Khwarizmi wrote a treatise on calculation that gave Europe its system of positional arithmetic with Hindu numerals; his Latinized name, *Algoritmi*, became our word *algorithm*. For centuries after, humanity imagined that performing calculations mechanically required building dedicated, bespoke machines out of brass gears: Blaise Pascal's 1642 Pascaline to add taxes, Gottfried Wilhelm Leibniz's 1673 Stepped Reckoner to multiply, and Charles Babbage's 1822 Difference Engine to crank out tables of polynomials. The false start of early computing was confusing the physical mechanism with the idea itself — believing that each new mathematical problem demanded an entirely new configuration of metal teeth.

The conceptual break came with Ada Lovelace in 1843 and Alan Turing in 1936. Translating Luigi Menabrea's description of Babbage's Analytical Engine, Lovelace saw that the proposed machine did not merely compute numbers; it could manipulate abstract symbols according to rules, remarking that "the Analytical Engine weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves." In Note G, she published the first true algorithm, a step-by-step trace to calculate Bernoulli numbers. Nearly a century later, with no electronic computers yet in existence, Alan Turing imagined a lone clerk manipulating discrete marks on paper tape according to a finite rulebook. Turing proved that a single Universal Machine could simulate any specialized calculating machine simply by reading the description of that machine encoded as instructions. This separated the algorithm from the hardware that runs it, which is the idea behind software.

Separating the algorithm from the hardware is what makes modern computing possible: a single slab of silicon can route global communications, render three-dimensional geometry, and sequence genomes purely by changing the symbolic recipe it runs. This abstraction has

a cost, though. Mathematical algorithm analysis ignores hardware realities: it treats every memory access as costing the exact same single instant ($\Theta(1)$), ignoring that reading from an on-chip cache takes a nanosecond while fetching from main memory takes a hundred. An algorithm with a theoretically superior step count can crawl on real silicon if its memory accesses scatter across the bus. In this book, we bridge that gap. Chapter 1 examines Euclid's ancient 300 BCE line-measuring algorithm; Chapter 2 develops loop invariants to prove that programs do what they claim; and Chapter 3 introduces Donald Knuth's asymptotic language (Θ, O, Ω) to quantify efficiency precisely.

How the parts map onto CLRS

This book deliberately follows the structure of CLRS so that the two can be read side by side. You can use this book as a standalone first course, or you can read each chapter here and then immediately read the corresponding CLRS chapter to see the same material at full speed and full rigor. The correspondence is close to one-to-one. Here is the map.

Ch.	Topic in this book	Matches CLRS
1	What is an algorithm?	Ch. 1 (the role of algorithms)
2	Getting started: insertion sort	Ch. 2
3	Asymptotic notation	Ch. 3
4	Divide and conquer	Ch. 4
5	Probability and randomized algorithms	Ch. 5
6	Heapsort	Ch. 6
7	Quicksort	Ch. 7
8	Sorting in linear time	Ch. 8
9	Medians and order statistics	Ch. 9
10	Elementary data structures	Ch. 10
11	Hash tables	Ch. 11
12	Binary search trees	Ch. 12
13	Red-black trees	Ch. 13
14	Augmenting data structures	Ch. 14
15	Dynamic programming	Ch. 15 (Ch. 14 in 4th ed.)
16	Greedy algorithms	Ch. 16 (Ch. 15 in 4th ed.)
17	Amortized analysis	Ch. 17 (Ch. 16 in 4th ed.)
18	B-trees	Ch. 18
19	Disjoint sets (union-find)	Ch. 19 / 21
20	Fibonacci heaps	Ch. 19 (3rd ed.)
21	van Emde Boas trees	Ch. 20
22	Elementary graph algorithms	Ch. 20 / 22
23	Minimum spanning trees	Ch. 21 / 23
24	Single-source shortest paths	Ch. 22 / 24
25	All-pairs shortest paths	Ch. 23 / 25
26	Maximum flow	Ch. 24 / 26
27	Matrix operations	Ch. 28
28	Linear programming	Ch. 29

Ch.	Topic in this book	Matches CLRS
29	Polynomials and the FFT	Ch. 30
30	Number-theoretic algorithms	Ch. 31
31	String matching	Ch. 32
32	Computational geometry	Ch. 33
33	NP-completeness	Ch. 34
34	Approximation algorithms	Ch. 35
35	Parallel algorithms	Ch. 26 (4th ed., multithreading)
36	Machine-learning algorithms	Ch. 20 (4th ed.) and beyond
37	Modern systems architecture & memory	Hardware hierarchy, caches, DRAM cycles
38	Compilers: from text to silicon	Lexing, parsing, SSA, register coloring
39	Quantum computing from the ground up	Qubits, interference, Grover, Shor

A note on the slashes: CLRS has gone through four editions, and the chapter numbers have drifted between them as topics were added, split, and reordered. Where a topic sits at different numbers in the third and fourth editions, the table shows both. Do not let the renumbering rattle you — the *ideas* are stable, and a **History and connection** box at the start of each chapter pins down exactly where to look in whichever edition you own. Chapters 35 through 39 reach past the classic CLRS core into material where modern practice and foundational systems live: parallelism, machine learning, hardware memory hierarchies and DRAM cycles, optimizing compilers, and quantum computation.

Reading note

You will notice the early chapters are densely interlinked: insertion sort (Ch. 2) sets up asymptotic notation (Ch. 3), which is the language used for everything afterward. If you are reading selectively, treat Chapters 1–4 as the non-negotiable foundation. After that, the sorting chapters (5–9), the data-structure chapters (10–14), and the graph chapters (22–26) each form a fairly self-contained arc you can take in whatever order your course demands.

How to actually use this book

Reading an algorithms book passively is like watching someone else exercise. You will enjoy it and learn nothing. Here is how to read this one so that it sticks.

Run the worked examples with a pencil

Every algorithm comes with at least one fully traced numeric example. *Do not merely read it.* Cover the worked trace with your hand, predict the next step, then uncover it and check. When you are wrong — and you will be, that is the point — stop and figure out exactly which assumption in your head was off. That moment of being wrong and finding out why is where almost all the learning actually happens.

Do the labs

The **Algorithm lab** boxes hand you a fresh input and ask you to be the computer. Get an index card or a sheet of scratch paper and genuinely write down the state of the array, the tree, or the queue at each step. It feels slow, and it is. It is also the most valuable activity in the whole book, because it forces you to hold the precise meaning of every variable in your head at once — which is exactly the skill that lets you later *design* algorithms instead of merely recognizing them.

Algorithm lab

Here is your very first lab, and it needs no algorithm at all. Take the seven numbers

$$5, \quad 2, \quad 9, \quad 1, \quad 5, \quad 6, \quad 3.$$

By hand, sort them into increasing order *using only swaps of adjacent elements* — you may only ever exchange two neighbors. Write down each swap you perform. Count them. Then ask yourself: did you find the smallest possible number of swaps? How could you be sure? Keep your answer; we will return to exactly this quantity (it is called the number of *inversions*) when we analyze insertion sort in Chapter 2.

Take the walking questions seriously

When you hit a **Walking question**, the best thing you can do is close the book. The orange boxes are placed at moments where a little independent struggle will pay off enormously. Your brain keeps working on a good question after you stop staring at it; some of your best insights will arrive in the shower, on a walk, or just before sleep. Give them the chance to.

Walking question

To break it in right now: you have a phone book with a million names in alphabetical order, and you want to find "Ramirez." One approach is to read from the first page until you hit it. Another is to flip to the middle, see whether "Ramirez" comes before or after, throw away half the book, and repeat. Roughly how many flips does the second method need in the worst case? Try to get an actual number before you read Chapter 1. The gap between "a million" and your answer is what this field is about.

Read the proofs, but read them gently

When you reach a loop invariant or an induction, do not brace yourself as if a storm is coming. A correctness proof in this book is just a careful retelling of the story you already understand, structured so that no case can sneak through unhandled. Read it once for the shape of the argument — "establish it is true at the start, show each step preserves it, conclude it holds at the end" — and then once more for the details. If a proof ever feels like it fell from the sky, that is a bug in my writing, not a deficiency in you; the intuition box right before it is where I owed you the picture.

A note on pseudocode

Throughout the book, algorithms are written in pseudocode — not in any real programming language. This is on purpose. Pseudocode lets us express *the idea* without the clutter of semicolons, type declarations, and library calls that differ from language to language. Our conventions are deliberately close to those of CLRS, so that moving between the two books costs you nothing. Here they are, all in one place.

- **Indentation shows block structure.** The body of a loop or an `if` is the indented text beneath it. There are no curly braces and no `end` keywords cluttering the page.
- **Assignment uses "←"** (or sometimes "=" inside the algorithm environments). The statement $x \leftarrow y$ means "copy the value of y into x." It is not a claim that x and y are equal; it is an action. Equality testing, when we need it, is written "=" inside a condition, and we will always make clear from context which we mean.
- **Arrays are indexed from** 1, the way CLRS does it, unless a chapter explicitly says otherwise. So $A[1]$ is the first element and $A[n]$ is the last, where $n = A.length$. (If you program in a language that counts from 0, just remember to shift by one when you translate; we will flag the places where this bites.)
- $A[i \,..\, j]$ **denotes a slice** — the elements $A[i], A[i+1], \ldots, A[j]$. If $i > j$, the slice is empty.
- **Objects have attributes accessed with a dot**, as in $A.length$ or $x.key$ or $node.left$. You can read $x.key$ as "the key field of x."
- **Variables are local** to the procedure they appear in unless we say otherwise. Parameters are passed by reference for arrays and objects (so a procedure can rearrange an array in place) and by value for plain numbers.
- **We freely use plain English** for steps whose detail would only distract — "*let q be the index of the median element*" — and then, if that step is the interesting part, we expand it into its own procedure.

Every algorithm appears inside a numbered, ruled box like the one below, with its inputs and outputs stated explicitly at the top. Read the **Input** and **Output** lines first — they are the contract. They tell you what you are allowed to assume on the way in and what you are promised on the way out, before you read a single line of the body.

Algorithm 1: A first taste of pseudocode: linear search

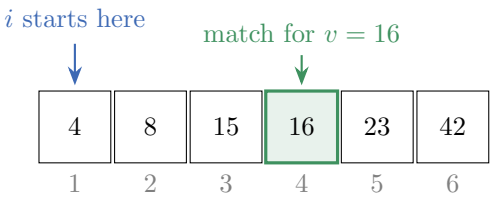
Input: An array $A[1 \,..\, n]$ and a target value v.
Output: An index i with $A[i] = v$, or the special value NIL if no such index exists.

```
1 for i ← 1 to n do
2     if A[i] = v then
3         return i ;                                  // found it; stop early
4 return NIL ;                                  // checked everything, v is absent
```

Take a moment with that little procedure — it is the simplest interesting algorithm there is, and it already shows the pattern every algorithm in the book follows. The **Input** and **Output** lines state the contract. The loop walks through the array one slot at a time. The `if` checks the current slot. The early `return` is a shortcut we have earned: the instant we find v there is no reason to

keep looking. And the final `return` NIL handles the case everyone forgets — what happens when the thing simply is not there.

We can picture the search as a single pointer sliding rightward along the array, pausing at each box just long enough to ask "are you the one?"



The pointer i begins at index 1 and steps right — past 4, past 8, past 15 — until at index 4 it finds 16 and stops. It never even looks at 23 or 42. If instead we had searched for a value that is not present, the pointer would have walked all the way off the right end and the final line would have returned NIL.

Common trap

A frequent bug in linear search lies after the loop rather than inside it: forgetting the final `return` statement. If you only return inside the `if`, then a search for an absent value falls off the bottom of the procedure and returns garbage (or crashes). Always ask of any algorithm: *what does it do when it finds nothing?* The empty case, the absent case, the zero-length case — these boundary inputs are where subtle bugs hide, so we will check them deliberately in every chapter.

To see why we will spend a whole book worrying about *how* we search and sort, look at the shape of even this trivial example. In the worst case — v is absent, or sits in the last slot — linear search inspects all n elements. The walking question above hinted that a cleverer search on *sorted* data can find the same value by inspecting only about $\log_2 n$ elements. For a million elements that is the difference between a million checks and about twenty. Repeated across millions of queries, that gap decides whether a system responds quickly or stalls.

Cost summary

Linear search (Algorithm 1) inspects at most n elements, so its worst-case running time is $\Theta(n)$, and it uses only $\Theta(1)$ extra space. We will make the symbols Θ, O, and Ω fully precise in Chapter 3; for now, read $\Theta(n)$ as "grows in step with n" and $\Theta(1)$ as "a fixed amount, no matter how big the input."

To make that promised gap concrete — and to give you a second worked example before you have even reached Chapter 1 — let us actually run the "flip to the middle" idea on a small sorted array, by hand, the way every section of this book will ask you to. This is the method the phone-book walking question was circling. It has a name, *binary search*, and we will meet it properly in Chapter 2; here we only want to *see* it move.

Algorithm lab

Take the sorted array

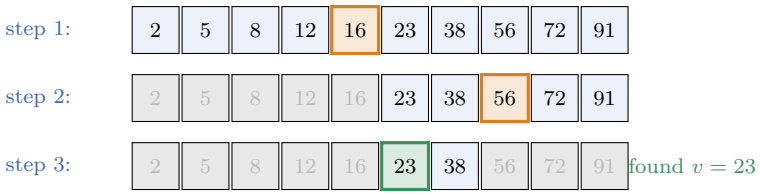
$$A = [\,2,\ 5,\ 8,\ 12,\ 16,\ 23,\ 38,\ 56,\ 72,\ 91\,],$$

ten elements, indexed 1 through 10, and search for $v = 23$. Keep two bookmarks, *lo* and *hi*, marking the part of the array that could still contain v; they start at 1 and 10. Repeat: look at the middle element $A[\,\lfloor (lo + hi)/2 \rfloor\,]$, and if it is too small move *lo* just past it, if too big move *hi* just before it, and if it is exactly v stop. Before reading the trace below, run those three steps yourself on paper and count how many middles you inspect.

Here is the trace. At each step the live range $[lo, hi]$ is the only part still in play; everything outside it has already been ruled out by a single comparison.

Step	*lo*	*hi*	**mid**	A[mid]	**action**
1	1	10	5	16	$16 < 23$, so search the right half: $lo \leftarrow 6$
2	6	10	8	56	$56 > 23$, so search the left half: $hi \leftarrow 7$
3	6	7	6	23	$23 = 23$, found at index 6; stop

Three comparisons, and we are done — against the ten that linear search would need in the worst case on this same array. The reason is visible in the *lo*/*hi* columns: each comparison throws away *half* of whatever remained. Ten candidates become five, then roughly two, then one. The number of halvings needed to grind a range of n down to a single element is exactly the quantity $\log_2 n$ that keeps reappearing.



The orange cell is the middle we test on each step; the grey cells have been discarded, never to be looked at again; the blue cells are still in play. Watch the blue region collapse from ten cells to five to two to the single green hit. Binary search is simply this repeated halving.

Intuition

This small example shows what the book is about. Linear search and binary search solve the *identical* problem — find v in an array — and a beginner might call them "basically the same, just two loops." But one inspects n cells and the other inspects $\log_2 n$, and on a billion-element array that is the difference between a billion steps and about thirty. Same problem, same hardware, a thousand-million-fold difference in work — all of it from a better algorithm. Learning to recognize and design such algorithms is the skill this book aims to build.

Cost summary

On a sorted array of n elements, binary search halves the live range each comparison, so it finishes after at most $\lfloor \log_2 n \rfloor + 1$ comparisons: worst-case time $\Theta(\log n)$, extra space $\Theta(1)$. Linear search, by contrast, costs $\Theta(n)$ time on the same data and does *not* require the array to be sorted. Sortedness is the price of admission for the logarithmic speed — a recurring bargain we will weigh again and again: pay once to organize the data, then query it cheaply forever after.

A few words of encouragement

Algorithms have a reputation for being hard, and parts of this subject genuinely are. But a great deal of the difficulty that students experience is not in the ideas themselves — it is in the *pace* at which the ideas are usually delivered. An algorithm that takes an expert thirty seconds to read took that expert, years ago, an afternoon of confusion to understand for the first time. Standard textbooks show you the thirty-second version. This book tries to give you back the afternoon.

So go slowly. Be willing to be confused for a while; confusion is a normal part of learning something new. Trace the examples. Do the labs. Take the walks. Work through the proofs until every step feels inevitable. If you do those things, then by the time you open CLRS for real, the ideas will already be familiar, and you will be able to read it at the pace it was written for.

Let us begin.

Focus

The contract of this book. You bring patience, a pencil, and a willingness to be temporarily confused. I bring the picture before the proof, a worked example for everything, and a clear pointer to CLRS at every turn. Together we get you ready to read the standard reference with confidence.

Exercises

These warm-ups need only careful counting and clear thinking, not any tools from later chapters. They exist to get you in the habit of *doing* rather than only reading.

Exercise 0.1. Go back and finish the first **Algorithm lab**: sort $5, 2, 9, 1, 5, 6, 3$ into increasing order using only swaps of adjacent elements, writing down every swap. Count your swaps. Now try to find an ordering of these same seven numbers that requires *more* adjacent swaps to sort, and one that requires *fewer*. What does the hardest such ordering look like in general?

Exercise 0.2. Answer the phone-book **Walking question**: for a sorted list of one million names, how many times can you halve the search range before only one name remains? Give an exact bound, and explain in one sentence why the answer is roughly $\log_2 n$ rather than n.

Exercise 0.3. Trace linear search (Algorithm 1) by hand on the array $A = [4, 8, 15, 16, 23, 42]$ with target $v = 16$. Write down the value of i and the result of the comparison $A[i] = v$ at every iteration, and state what the procedure returns.

Exercise 0.4. Run the same trace with $v = 17$ on the same array. How many comparisons does the procedure perform, and what does it return? Which line of the pseudocode is responsible for producing a sensible answer in this case?

Exercise 0.5. Suppose someone hands you a version of linear search that has its final `return` NIL line deleted. Describe precisely what goes wrong, and give a concrete input on which the buggy version behaves differently from the correct one.

Exercise 0.6. In your own words, and without looking back, explain the difference between an **Intuition** box and a **Formal** box, and why this book always presents them in that order. Then invent a one-sentence intuition and a one-sentence formal statement for the everyday task of "looking up a word in a dictionary."

Exercise 0.7. Pick any everyday process you do without thinking — making tea, getting dressed, finding your keys — and write it as pseudocode using the conventions from the "note on pseudocode" section: an explicit **Input** line, an explicit **Output** line, at least one loop or `if`, and a comment on at least one line. Then identify one **Common trap**: an input on which your pseudocode would misbehave.

Exercise 0.8. Run the binary-search trace from the **Algorithm lab** again on the same array $A = [\,2, 5, 8, 12, 16, 23, 38, 56, 72, 91\,]$, but this time search for $v = 12$ and for $v = 40$. For each, write down the (lo, hi, mid) triple at every step, the number of comparisons, and what the search concludes. Which search ends by reporting "absent," and how does the range tell you so?

Exercise 0.9. On the same ten-element array, what is the largest number of comparisons binary search could ever need, over all possible targets? Find a target that achieves it. Then check your count against the bound $\lfloor \log_2 n \rfloor + 1$ from the **Cost summary**.

Exercise 0.10 (Challenge)**.** Linear search inspects up to n elements; the phone-book method inspects about $\log_2 n$. Make a small table of $n = 10,\ 100,\ 1000,\ 10^6,\ 10^9$ and, for each, write down both n and $\lceil \log_2 n \rceil$. Looking at your table, write one sentence explaining why a programmer might reasonably treat $\log_2 n$ as "practically constant" for any input that fits on a real computer.

Contents

Preface iii

I Foundations 1

1 What an Algorithm Really Is 3

1.1 Recipes, and why precision matters . . . 3
1.2 Problems versus instances . . . 5
1.3 What "correct" actually means . . . 6
1.4 A first algorithm, written out properly . . . 7
1.4.1 Pseudocode conventions . . . 7
1.4.2 Tracing it on an instance . . . 8
1.5 Why correctness needs an argument . . . 9
1.6 Efficiency, and why it matters . . . 10
1.6.1 Counting steps, not seconds . . . 10
1.6.2 The shape of growth is what matters . . . 11
1.7 Comparing two approaches to one task . . . 12
1.7.1 Approach one: linear search . . . 12
1.7.2 Approach two: binary search . . . 13
1.7.3 Counting the halvings . . . 14
1.8 Algorithms as a technology . . . 15
1.9 What to carry forward . . . 15
Exercises . . . 16

2 Getting Started: Insertion Sort and Loop Invariants 19

2.1 The problem we are solving . . . 19

2.2 The card-player's idea . . . 21
2.3 Insertion sort, written down . . . 21
2.4 A full numeric trace . . . 22
2.5 Loop invariants: how we *know* it works . . . 23
2.5.1 The invariant for insertion sort . . . 24
2.6 How long does it take? . . . 25
2.6.1 Best case: already sorted . . . 26
2.6.2 Worst case: reverse-sorted . . . 26
2.6.3 Average case . . . 27
2.7 Order of growth . . . 28
2.8 Designing algorithms: two stances . . . 29
2.9 A first taste of merge sort . . . 29
2.10 What to carry forward . . . 31
2.11 Exercises . . . 32

3 The Growth of Functions: Asymptotic Notation 35
3.1 Why we throw away the constants . . . 35
3.2 The mental picture before the symbols . . . 37
3.3 Big-O: the upper bound . . . 37
3.3.1 Proving a Big-O bound: find c and n_0 . . . 38
3.4 Big-Omega: the lower bound . . . 39
3.5 Big-Theta: the tight bound . . . 40
3.5.1 A theorem you will reuse: polynomials are Theta of their top term . . . 41
3.6 The little ones: little-o and little-omega . . . 41
3.7 An analogy: comparing functions is comparing numbers . . . 42
3.8 A field guide to common growth rates . . . 43
3.9 Manipulating asymptotics: the algebra of growth . . . 45
3.10 Putting it to work: bounding an algorithm directly . . . 46
3.11 Exercises . . . 48

4 Divide and Conquer, Recurrences, and the Master Theorem 49
4.1 The shape of a divide-and-conquer algorithm . . . 50
4.1.1 A first, familiar example: binary search . . . 50

4.2 Merge sort, revisited slowly . . . 51
4.2.1 The idea . . . 51
4.2.2 A full numeric trace . . . 52
4.2.3 Why it is correct . . . 53
4.3 Counting the work: recurrences . . . 54
4.4 Method 1: the recursion tree (see the work) . . . 55
4.4.1 Merge sort's tree . . . 55
4.4.2 An unbalanced tree, to build instinct . . . 56
4.5 Method 2: substitution (guess, then prove) . . . 56
4.5.1 Proving an upper bound for merge sort . . . 56
4.5.2 The danger of a sloppy guess . . . 57
4.6 Method 3: the master theorem (the lookup table) . . . 57
4.6.1 The exponent that decides the answer . . . 58
4.6.2 The statement . . . 58
4.6.3 Why each case is true, in pictures . . . 59
4.7 Worked examples with the master theorem . . . 59
4.8 Two motivating algorithms beyond sorting . . . 60
4.8.1 Maximum subarray . . . 60
4.8.2 Strassen's matrix multiplication . . . 62
4.9 When the master theorem is not enough: Akra–Bazzi . . . 63
4.10 Cost summary . . . 64
4.11 Exercises . . . 65

5 Probability and Randomized Algorithms 67
5.1 Two Different Kinds of "Average" . . . 67
5.2 The Hiring Problem . . . 68
5.3 Random Variables and Expectation . . . 69
5.4 Indicator Random Variables: The Master Trick . . . 70
5.5 Solving the Hiring Problem with Indicators . . . 71
5.6 Randomly Permuting an Array . . . 73
5.7 A Recursion-Tree-Free Look at the Numbers . . . 75
5.8 A Second Helping: Expected Number of Inversions . . . 76
5.9 Las Vegas vs. Monte Carlo: Two Flavors of Randomized Algorithm . . . 77

5.10 What "Expected Running Time" Really Means . . . 77
5.11 Putting It Together: a Tiny End-to-End Example . . . 78
5.12 Summary . . . 80
5.13 Exercises . . . 81

II Sorting and Order Statistics 83

6 Heaps and Heapsort 85

6.1 The shape of a heap . . . 85
6.1.1 A tree hiding in an array . . . 85
6.1.2 The heap property: the ordering rule . . . 87
6.1.3 How tall is a heap? . . . 88
6.2 Max-heapify: repairing one broken spot . . . 88
6.2.1 A full numeric trace . . . 89
6.2.2 Why it is correct . . . 90
6.2.3 Cost of MAX-HEAPIFY . . . 91
6.3 Build-Max-Heap: a heap from an unordered array . . . 91
6.3.1 A full numeric trace . . . 92
6.3.2 Why it is correct: a loop invariant . . . 92
6.3.3 The surprising linear-time analysis . . . 93
6.4 Heapsort . . . 94
6.4.1 Watching it run . . . 95
6.4.2 Correctness via a loop invariant . . . 96
6.4.3 Cost of heapsort . . . 97
6.5 Priority queues . . . 97
6.5.1 Maximum and Extract-Max . . . 98
6.5.2 Increase-Key and Insert . . . 99
6.6 Putting it together . . . 102
6.7 Exercises . . . 102

7 Quicksort 105

7.1 The shape of the idea . . . 105
7.2 Lomuto partitioning, built by hand . . . 106

7.2.1 A complete numeric trace . . . 107
7.2.2 Why it is correct: a loop invariant . . . 108
7.3 The two recursion patterns: best and worst case . . . 110
7.3.1 The worst case: maximally lopsided splits . . . 110
7.3.2 The best case: perfectly even splits . . . 110
7.3.3 The reassuring middle: even bad balance is fine . . . 111
7.4 Defeating worst-case inputs with randomization . . . 112
7.5 Expected running time . . . 114
7.5.1 The key realization: each pair is compared at most once . . . 114
7.5.2 Indicator variables and the count . . . 115
7.5.3 Summing the series . . . 116
7.6 Hoare partitioning: the original, faster scheme . . . 116
7.7 Engineering the real thing . . . 118
7.8 Putting it all together . . . 119
7.9 Exercises . . . 120

8 Sorting in Linear Time **123**

8.1 Comparisons Are a Kind of Currency . . . 123
8.1.1 The Decision Tree: Drawing the Argument . . . 124
8.1.2 The Lower Bound, Stated and Proved . . . 125
8.2 Counting Sort: Sorting Without Comparing . . . 127
8.2.1 The Setup . . . 127
8.2.2 A Full Numeric Trace . . . 128
8.2.3 Stability, and Why We Care . . . 129
8.2.4 Cost, and the Catch . . . 130
8.3 Radix Sort: Many Small Sorts Make One Big Sort . . . 130
8.3.1 Least Significant Digit First – and Why That Order . . . 130
8.3.2 A Full Numeric Trace . . . 131
8.3.3 Cost . . . 132
8.4 Bucket Sort: Betting on the Distribution . . . 133
8.4.1 A Numeric Trace . . . 134
8.4.2 Why It Is Linear on Average . . . 135
8.5 So When Can You Actually Beat $n \log n$? . . . 136

Summary . . . 137
8.6 Exercises . . . 137

9 Medians and Order Statistics **139**
9.1 The selection problem . . . 139
9.1.1 The easy cases warm us up: minimum and maximum . . . 140
9.2 Selection by sorting: the baseline to beat . . . 141
9.3 Quickselect: partition, then recurse on one side . . . 141
9.3.1 Recalling PARTITION . . . 142
9.3.2 Randomized quickselect . . . 143
9.3.3 A full numeric trace . . . 144
9.3.4 Why quickselect is correct . . . 145
9.4 How fast is quickselect? . . . 146
9.4.1 The worst case is bad . . . 146
9.4.2 Expected linear time: the intuition first . . . 146
9.4.3 Expected linear time: the sharper analysis . . . 147
9.5 Median of medians: worst-case linear selection . . . 149
9.5.1 The five steps . . . 149
9.5.2 Why the pivot is guaranteed good . . . 150
9.5.3 The recurrence, and why groups of 5 work . . . 152
9.5.4 Comparing the two selection algorithms . . . 153
9.6 Exercises . . . 155

III Data Structures **157**

10 Elementary Data Structures **159**
10.1 Memory, Addresses, and the Array . . . 159
10.1.1 The price of insertion and deletion . . . 161
10.2 Matrices: arrays in two dimensions . . . 161
10.3 Stacks: last in, first out . . . 162
10.4 Queues: first in, first out . . . 164
10.5 Deques: open at both ends . . . 166
10.6 Pointers and objects: escaping contiguity . . . 167

10.7 Linked lists . . . 167
10.7.1 Searching a list . . . 168
10.7.2 Inserting into a list . . . 168
10.7.3 Deleting from a list . . . 169
10.7.4 The sentinel trick . . . 170
10.8 Representing rooted trees . . . 171
10.8.1 Binary trees . . . 171
10.8.2 When the number of children is unbounded . . . 172
10.8.3 Other tree representations . . . 173
10.9 Putting it together . . . 174
Exercises . . . 174

11 Hash Tables **177**
11.1 The dictionary problem . . . 177
11.2 Direct-address tables: the simple, wasteful idea . . . 178
11.3 From direct addressing to hashing . . . 179
11.4 Collision resolution by chaining . . . 179
11.4.1 The load factor . . . 180
11.5 Analyzing chaining: simple uniform hashing . . . 181
11.5.1 Unsuccessful search . . . 181
11.5.2 Successful search . . . 181
11.6 Hash functions: turning keys into slots . . . 183
11.6.1 The division method . . . 183
11.6.2 The multiplication method . . . 183
11.7 Open addressing: no lists at all . . . 184
11.7.1 Linear probing . . . 185
11.7.2 Quadratic probing . . . 186
11.7.3 Double hashing . . . 186
11.7.4 Cost of open addressing . . . 187
11.8 Universal hashing: removing the leap of faith . . . 188
11.8.1 A concrete universal family . . . 190
11.9 Perfect hashing: $O(1)$ worst case for static sets . . . 190
11.10Probabilistic Data Structures: Bloom Filters and Count-Min Sketch . . . 192

11.10.1 Bloom filters: trading perfection for compactness . . . 193
11.10.2 First-principles derivation of false positives and optimal parameters . . . 194
11.10.3 From set membership to stream frequency: Count-Min Sketch . . . 195
11.11 Choosing a scheme: a practical recap . . . 198
11.12 Exercises . . . 198

12 Binary Search Trees **201**
12.1 The shape of the idea . . . 201
12.1.1 From sorted arrays to branching decisions . . . 201
12.1.2 The in-order walk . . . 203
12.2 Searching, minimum, maximum . . . 204
12.2.1 Searching for a key . . . 204
12.2.2 Minimum and maximum . . . 205
12.3 The successor: finding the next key . . . 205
12.4 Insertion . . . 207
12.5 Deletion . . . 210
12.6 BSTs and quicksort are the same algorithm . . . 213
12.6.1 The first surprise . . . 213
12.6.2 Why we randomize . . . 214
12.7 The expected height of a random BST . . . 214
12.7.1 Why depth is just counting ancestors . . . 214
12.7.2 Counting with an indicator variable . . . 215
12.7.3 The harmonic numbers appear . . . 215
12.8 Putting it together: a worked lab . . . 217
12.9 Exercises . . . 217

13 Red-Black Trees **219**
13.1 Why Balance Matters . . . 219
13.2 The Red-Black Properties . . . 221
13.3 The Height Bound . . . 223
13.4 Rotations . . . 224
13.5 Insertion . . . 226
13.5.1 A fully worked insertion . . . 229

13.6 Deletion . . . 230
13.7 Putting It All Together . . . 234
Exercises . . . 236

14 Augmenting Data Structures **239**
14.1 Warming Up: What Question Are We Even Asking? . . . 239
14.2 Order-Statistic Trees: The Idea . . . 240
14.2.1 `SELECT`: finding the ith smallest . . . 241
14.2.2 `RANK`: where does this node sit? . . . 242
14.3 Keeping `size` Correct Through Insert and Delete . . . 244
14.3.1 Insertion and deletion: the downward pass . . . 244
14.3.2 Rotations: the local fix-up . . . 245
14.4 The General Recipe: A Theorem for Augmenting . . . 246
14.5 Interval Trees: Augmenting for Overlap . . . 248
14.5.1 `INTERVAL-SEARCH`: the one clever step . . . 250
14.5.2 Why the search never misses an overlap . . . 252
14.6 A Field Guide to Augmentation . . . 253
14.7 Exercises . . . 255

IV Advanced Design and Analysis **257**

15 Dynamic Programming **259**
15.1 A First Problem: Cutting a Rod . . . 259
15.1.1 Finding the recursive structure . . . 260
15.1.2 The naive recursion and why it explodes . . . 260
15.1.3 Memoization: the notebook fix . . . 261
15.1.4 Tabulation: filling the table bottom-up . . . 262
15.1.5 Reconstructing the actual cuts . . . 263
15.2 The Elements of Dynamic Programming . . . 265
15.2.1 Optimal substructure . . . 265
15.2.2 Overlapping subproblems . . . 266
15.2.3 Memoization versus tabulation . . . 266
15.3 Matrix-Chain Multiplication . . . 267

15.3.1 The recurrence . . . 268
15.3.2 Filling the table by chain length . . . 268
15.3.3 Reconstructing the parenthesization . . . 270
15.4 Longest Common Subsequence . . . 270
15.4.1 The recurrence . . . 271
15.4.2 Filling the table and tracking arrows . . . 271
15.4.3 Reading the subsequence backward . . . 272
15.5 Optimal Binary Search Trees . . . 274
15.5.1 The recurrence . . . 274
15.5.2 The algorithm . . . 275
15.6 The Shape of a DP . . . 276
15.6.1 The Dependency DAG and Table Fill Topologies . . . 277
Exercises . . . 279

16 Greedy Algorithms **281**
16.1 A first taste: choosing activities . . . 281
16.1.1 The greedy guess . . . 282
16.1.2 The algorithm . . . 283
16.1.3 Why is it correct? The exchange argument . . . 283
16.1.4 Breaking the wrong rules . . . 285
16.2 Greedy versus dynamic programming . . . 285
16.2.1 Two coins, one lesson . . . 285
16.2.2 A recipe for designing greedy algorithms . . . 287
16.3 Huffman codes: compression by greed . . . 288
16.3.1 Why variable-length codes . . . 288
16.3.2 Codes as binary trees . . . 288
16.3.3 Huffman's greedy idea . . . 289
16.3.4 A full build, traced . . . 290
16.3.5 Complexity . . . 291
16.3.6 Why Huffman is optimal . . . 291
16.4 Why does greed work? A glimpse of matroids . . . 293
16.4.1 The pattern behind the proofs . . . 293
16.4.2 The greedy theorem for matroids . . . 294

16.5 Summary . . . 295
Exercises . . . 295

17 Amortized Analysis **297**

17.1 Worst case per operation versus worst case per sequence . . . 297
17.2 The aggregate method: just add it all up . . . 298
17.2.1 Example: the stack with MULTIPOP . . . 299
17.2.2 Example: the binary counter . . . 300
17.3 The accounting method: prepay and bank the credit . . . 302
17.3.1 Accounting for the stack . . . 303
17.3.2 Accounting for the binary counter . . . 303
17.4 The potential method: bottle the credit into a single number . . . 305
17.4.1 Potential for the stack . . . 306
17.4.2 Potential for the binary counter . . . 306
17.5 The headline example: dynamic tables . . . 307
17.5.1 The setup and the naive disaster . . . 307
17.5.2 Aggregate analysis of doubling . . . 308
17.5.3 Potential analysis of doubling: choosing Φ . . . 309
17.5.4 Why constant-factor growth, and not additive . . . 311
17.5.5 A word on deletion and shrinking . . . 312
17.6 Choosing the right method . . . 313
17.7 Exercises . . . 314

V Advanced Data Structures **317**

18 B-Trees **319**

18.1 Why Secondary Storage Dictates Wide Trees . . . 319
18.2 What a B-Tree Is . . . 321
18.3 How Tall Can a B-Tree Get? . . . 323
18.4 Searching a B-Tree . . . 325
18.5 Splitting a Full Node . . . 326
18.6 Insertion in a Single Downward Pass . . . 328
18.6.1 A Fully Traced Insertion . . . 329

18.7 Why Insertion Is Correct . . . 331
18.8 Deletion . . . 331
18.9 B-Trees in Practice . . . 335
Exercises . . . 335

19 Disjoint Sets and Union-Find **337**
19.1 What We Are Actually Building . . . 338
19.1.1 A motivating use: connected components . . . 338
19.2 First Try: a Linked-List Representation . . . 339
19.2.1 How bad can it get? A worst-case sequence . . . 340
19.2.2 The weighted-union heuristic . . . 340
19.3 The Forest Representation . . . 341
19.4 Trick One: Union by Rank . . . 342
19.4.1 Why rank controls height . . . 344
19.5 Trick Two: Path Compression . . . 345
19.6 Putting It Together, and the Near-Constant Result . . . 346
19.6.1 Where does inverse Ackermann come from? . . . 346
19.6.2 A gentle sketch of why we win . . . 347
19.6.3 What each variant costs . . . 348
19.7 A Complete Worked Example . . . 348
19.8 Implementation Notes . . . 350
19.9 Summary . . . 350
19.10Exercises . . . 351

20 Fibonacci Heaps **353**
20.1 What problem are we actually solving? . . . 353
20.2 The idea: lazy bags of trees . . . 354
20.2.1 From one tree to a forest . . . 354
20.2.2 What each node carries . . . 355
20.3 The easy operations: make, insert, minimum, union . . . 356
20.3.1 Make-heap and minimum . . . 356
20.3.2 Insert: just drop it on the table . . . 356
20.3.3 Union: splice two circles together . . . 356

20.4 Extract-min: the one place we clean up . . . 357
20.4.1 The two-step plan . . . 357
20.4.2 Linking two trees . . . 358
20.4.3 The consolidation procedure . . . 358
20.4.4 A fully traced example . . . 359
20.5 Decrease-key: the operation we built all this for . . . 361
20.5.1 The naive idea, and why it is not enough . . . 361
20.5.2 The mark bit and cascading cuts . . . 362
20.5.3 A traced decrease-key . . . 363
20.6 The amortized analysis . . . 364
20.6.1 Choosing the potential . . . 364
20.6.2 Insert, minimum, union . . . 365
20.6.3 Extract-min . . . 365
20.6.4 Decrease-key . . . 365
20.6.5 Delete . . . 366
20.7 Why "Fibonacci"? The degree bound . . . 366
20.7.1 What the mark rule guarantees . . . 366
20.7.2 Fibonacci numbers fall out . . . 367
20.8 Putting it back in context: Dijkstra and Prim . . . 368
20.9 Summary . . . 369
Exercises . . . 370

21 van Emde Boas Trees **371**

21.1 The problem, stated carefully . . . 371
21.2 Idea zero: the bit vector . . . 372
21.3 Idea one: a tree of summaries (the layered approach) . . . 373
21.4 Idea two: split the key, build proto-vEB . . . 373
21.4.1 How tall is this recursion? . . . 375
21.5 Proto-vEB operations, and why they are not yet fast enough . . . 376
21.5.1 Member . . . 376
21.5.2 Minimum and Maximum – the slow ones . . . 376
21.5.3 Successor – the bottleneck diagnosed . . . 377
21.6 Two fixes: cached extremes and a floating minimum . . . 378

21.7 The fast operations . . . 379
21.7.1 Minimum, Maximum, Member . . . 379
21.7.2 Successor: the centerpiece . . . 380
21.7.3 A complete numeric trace of Successor . . . 380
21.7.4 Insert: one recursion via the doorman . . . 382
21.7.5 Delete: the bookkeeping is heavier but the bound holds . . . 382
21.8 Solving the recurrence, once and for all . . . 384
21.9 The cost of all this: space . . . 384
21.10Recap: the chain of ideas . . . 385
21.11Exercises . . . 386

VI Graph Algorithms 389

22 Elementary Graph Algorithms 391

22.1 What a Graph Is . . . 391
22.2 Storing a Graph: Lists versus Matrices . . . 392
22.2.1 The adjacency matrix . . . 392
22.2.2 The adjacency list . . . 393
22.3 Breadth-First Search: Exploring in Rings . . . 394
22.3.1 A full numeric trace . . . 395
22.3.2 Why BFS finds shortest paths . . . 397
22.3.3 The running time of BFS . . . 397
22.4 Depth-First Search: Plunge and Backtrack . . . 398
22.4.1 A full numeric trace . . . 399
22.4.2 The parenthesis structure . . . 400
22.4.3 Classifying the edges . . . 401
22.4.4 The running time of DFS . . . 403
22.5 Topological Sort: Untangling Dependencies . . . 403
22.5.1 A worked topological sort . . . 404
22.6 Strongly Connected Components . . . 405
22.6.1 Why the trick works . . . 406
22.6.2 A worked example . . . 407

22.7 Summary . . . 409
22.8 Exercises . . . 409

23 Minimum Spanning Trees **411**
23.1 What exactly are we looking for? . . . 411
23.2 A running example . . . 412
23.3 The cut property . . . 413
23.3.1 Proof of the cut property, slowly . . . 414
23.4 The generic MST algorithm . . . 415
23.5 Kruskal's algorithm . . . 416
23.5.1 The idea . . . 417
23.5.2 The mechanism: union-find . . . 417
23.5.3 The algorithm . . . 418
23.5.4 A full trace on our graph . . . 418
23.5.5 Why Kruskal is correct . . . 419
23.5.6 Running time of Kruskal . . . 419
23.6 Prim's algorithm . . . 420
23.6.1 The idea . . . 420
23.6.2 The mechanism: a priority queue . . . 421
23.6.3 A full trace on our graph . . . 421
23.6.4 Why Prim is correct . . . 422
23.6.5 Running time of Prim . . . 423
23.7 Kruskal vs. Prim: which, and why . . . 424
23.8 A few subtleties . . . 425
23.9 Exercises . . . 425

24 Single-Source Shortest Paths **427**
24.1 What is a shortest path, exactly? . . . 427
24.1.1 Why negative edges work and negative cycles break shortest paths . . . 428
24.2 The two structural facts everything rests on . . . 428
24.2.1 Optimal substructure: shortest paths contain shortest paths . . . 429
24.2.2 The triangle inequality . . . 429
24.3 Relaxation: the shared primitive . . . 430

24.4 Bellman–Ford: relax everything, repeatedly . . . 431
24.4.1 The negative-cycle detector, explained . . . 432
24.4.2 A complete numeric trace . . . 432
24.5 Shortest paths in a DAG: order is free . . . 434
24.6 Dijkstra's algorithm: greedy on nonnegative weights . . . 436
24.6.1 A complete numeric trace . . . 437
24.6.2 Why Dijkstra is correct . . . 438
24.6.3 The role of the priority queue . . . 439
24.6.4 The predecessor graph: reconstructing the actual path . . . 440
24.7 Putting the three together . . . 441
24.8 Exercises . . . 442

25 All-Pairs Shortest Paths 445
25.1 Setting Up: Inputs, Outputs, and Conventions . . . 446
25.2 First Idea: Shortest Paths as Matrix "Multiplication" . . . 446
25.2.1 The subproblem: shortest paths with a bounded number of edges . . . 447
25.2.2 When do we stop? . . . 448
25.2.3 Pseudocode and a first cost . . . 449
25.2.4 Repeated squaring: $\Theta(n^3 \log n)$ . . . 449
25.3 Floyd–Warshall: A Smarter Dynamic Program . . . 450
25.3.1 The new subproblem: allowed intermediate vertices . . . 450
25.3.2 The recurrence: "do I route through vertex k or not?" . . . 450
25.3.3 We only need two layers... actually one . . . 451
25.3.4 A full worked trace . . . 453
25.3.5 Why it is correct . . . 455
25.4 Transitive Closure: Reachability for the Price of Booleans . . . 455
25.5 Johnson's Algorithm: Fast on Sparse Graphs, Even with Negative Edges . . . 456
25.5.1 The reweighting trick . . . 457
25.5.2 Choosing h so every $\hat{w} \geq 0$ . . . 458
25.5.3 The full algorithm . . . 459
25.5.4 A worked example of the reweighting . . . 459
25.5.5 Why it is correct, and what it costs . . . 460
25.6 Choosing an Algorithm . . . 460

25.7 Summary . . . 462
Exercises . . . 462

26 Maximum Flow **465**
26.1 Flow Networks . . . 465
26.1.1 A network to keep on the table . . . 466
26.2 The Ford–Fulkerson Method . . . 467
26.2.1 Why naive greedy is not enough . . . 467
26.2.2 Residual networks . . . 468
26.2.3 Augmenting along a path . . . 469
26.2.4 A full numeric trace . . . 470
26.3 Cuts and the Max-Flow Min-Cut Theorem . . . 471
26.3.1 Why "method" and not "algorithm": termination . . . 474
26.4 Edmonds–Karp: Choosing Good Paths . . . 474
26.5 Application: Maximum Bipartite Matching . . . 476
26.5.1 The reduction . . . 477
26.5.2 Tracing the matching . . . 477
26.6 What to Carry Forward . . . 479
Exercises . . . 479

VII Selected Topics **481**

27 Matrix Operations **483**
27.1 Matrices from the ground up . . . 483
27.1.1 Why multiplication is defined the way it is . . . 484
27.1.2 The cost of the schoolbook product . . . 485
27.2 Strassen's algorithm, revisited . . . 486
27.2.1 A first attempt: blocking . . . 486
27.2.2 Seven products instead of eight . . . 487
27.2.3 Why seven beats eight: the recurrence . . . 488
27.3 Solving linear systems: $Ax = b$ . . . 489
27.3.1 Triangular systems are easy . . . 490
27.3.2 LU and LUP decomposition . . . 490

27.3.3 Why we never compute A^{-1} to solve $Ax = b$ . . . 493
27.4 Matrix inversion and its equivalence to multiplication . . . 493
27.4.1 Computing the inverse with LUP . . . 493
27.4.2 The equivalence, both directions . . . 494
27.5 Least squares: solving systems that have no solution . . . 495
27.5.1 The picture: a best-fit line . . . 495
27.5.2 The normal equations . . . 495
27.6 Pulling it together . . . 497
Exercises . . . 498

28 Linear Programming **501**
28.1 Modeling: From Words to Algebra . . . 502
28.1.1 A second model: shortest paths as an LP . . . 503
28.2 Standard and Slack Forms . . . 503
28.2.1 Standard form . . . 503
28.2.2 Slack form . . . 504
28.3 The Geometry of an LP . . . 505
28.4 The Simplex Method, by Example . . . 507
28.4.1 The idea in one paragraph . . . 507
28.4.2 Iteration 0: the starting dictionary . . . 508
28.4.3 Iteration 1: the pivot . . . 508
28.4.4 Iteration 2: the second pivot . . . 509
28.4.5 Simplex as pseudocode . . . 510
28.4.6 Two loose ends: starting and cycling . . . 511
28.5 Duality: Every LP Has a Shadow . . . 512
28.5.1 Where does the dual come from? . . . 512
28.5.2 Weak and strong duality . . . 513
28.5.3 What the dual variables mean: shadow prices . . . 515
28.6 LP Relaxation as a Design Tool . . . 515
28.7 Summary . . . 517
Exercises . . . 518

29 Polynomials and the FFT **521**

29.1 Two ways to describe a polynomial . . . 521
29.1.1 Point-value form . . . 522
29.1.2 A small worked conversion . . . 523
29.2 The operation we actually care about: multiplication . . . 523
29.3 Choosing the points: roots of unity . . . 525
29.3.1 Complex roots of unity . . . 525
29.3.2 Three facts that make everything work . . . 526
29.4 The Discrete Fourier Transform . . . 527
29.5 The FFT: divide and conquer on the coefficients . . . 528
29.5.1 The combine step, made explicit . . . 528
29.5.2 The algorithm . . . 529
29.5.3 Correctness, gently . . . 530
29.5.4 Running time . . . 530
29.6 Coming back: the inverse FFT . . . 531
29.7 The whole pipeline: fast polynomial multiplication . . . 532
29.7.1 A complete numerical trace . . . 532
29.8 Why the same code runs forward and backward . . . 533
29.9 A note on making it iterative and fast in practice . . . 534
29.10Exercises . . . 536

30 Number-Theoretic Algorithms **537**

30.1 Divisibility, GCD, and the Idea of a Remainder . . . 537
30.1.1 The greatest common divisor . . . 538
30.1.2 How fast is it? Enter Fibonacci . . . 539
30.2 The Extended Euclidean Algorithm . . . 540
30.3 Modular Arithmetic: Computing on a Clock . . . 542
30.3.1 Units and inverses . . . 544
30.4 Solving Modular Linear Equations . . . 544
30.5 The Chinese Remainder Theorem . . . 545
30.5.1 Constructing the solution . . . 546
30.6 Modular Exponentiation . . . 547
30.6.1 Fermat's and Euler's theorems . . . 549
30.7 The RSA Public-Key Cryptosystem . . . 549

30.7.1 Setting up the keys . . . 549
30.7.2 Encrypting and decrypting . . . 550
30.8 Primality Testing . . . 551
30.8.1 The Fermat test and its flaw . . . 551
30.8.2 Miller–Rabin: a better witness . . . 551
30.9 Putting It Together . . . 554
Exercises . . . 554

31 String Matching **557**
31.1 Setting the stage . . . 557
31.2 The naive matcher: just try every shift . . . 559
31.2.1 How slow can the naive matcher be? . . . 560
31.3 Rabin–Karp: matching by fingerprint . . . 561
31.3.1 Treating a string as a number . . . 561
31.3.2 Modular arithmetic and the spurious-hit check . . . 562
31.3.3 Why it is usually fast, and why it can be slow . . . 563
31.4 String matching with finite automata . . . 564
31.4.1 What a string-matching automaton is . . . 564
31.4.2 A worked machine . . . 565
31.4.3 Cost, and the one catch . . . 567
31.5 The Knuth–Morris–Pratt algorithm . . . 567
31.5.1 The insight: a mismatch already tells you a lot . . . 568
31.5.2 Computing the prefix function by hand . . . 568
31.5.3 Computing the prefix function efficiently . . . 569
31.5.4 KMP itself . . . 570
31.5.5 Why KMP is linear: an amortized argument . . . 571
31.6 Putting the four side by side . . . 573
31.6.1 From text streams to genomic search . . . 573
31.6.2 Practical trade-offs and engineering limits . . . 574
31.7 Exercises . . . 574

32 Computational Geometry **577**
32.1 The cross product: the only tool you really need . . . 577

32.1.1 The ORIENTATION primitive . . . 579
32.2 Do two segments intersect? . . . 580
32.2.1 Testing any pair: the sweep-line algorithm . . . 582
32.3 The convex hull . . . 583
32.3.1 Graham's scan . . . 584
32.3.2 Jarvis's march (gift wrapping) . . . 586
32.4 The closest pair of points . . . 587
32.4.1 The setup and the strip . . . 588
32.4.2 Why seven neighbors suffice . . . 588
32.4.3 The algorithm and its recurrence . . . 590
32.5 Looking back . . . 591
32.5.1 From planar primitives to spatial computing . . . 591
32.5.2 Higher dimensions and geometric boundaries . . . 592
Exercises . . . 592

33 NP-Completeness **595**
33.1 Decision problems: shrinking the question . . . 596
33.2 Computability Foundations: Turing Machines and the Undecidability of the Halting Problem . . . 597
33.2.1 Alan Turing's 1936 model: the pencil-and-paper clerk . . . 597
33.2.2 Code as data: the Universal Turing Machine . . . 598
33.2.3 The Halting Problem and Turing's diagonalization proof . . . 599
33.2.4 Cantor's diagonal and Rice's Theorem . . . 600
33.2.5 The hierarchy: from the impossible to the intractable . . . 601
33.3 The class P: problems you can actually solve . . . 602
33.4 The class NP: problems you can check . . . 602
33.4.1 The million-dollar question . . . 604
33.5 Reductions: "no harder than" . . . 604
33.6 NP-completeness: the hardest problems in NP . . . 605
33.6.1 The first domino: Cook–Levin . . . 606
33.6.2 The chain reaction: how to prove a problem NP-complete . . . 606
33.7 From SAT to 3-SAT: taming the formula . . . 608
33.8 Clique: hardness becomes a graph . . . 609

33.9 Vertex Cover: the same coin, flipped . . . 610
33.9.1 The direct gadget reduction: 3-SAT to Vertex Cover . . . 611
33.10 Hamiltonian Cycle and the Traveling Salesman . . . 611
33.10.1 TSP: putting numbers on the edges . . . 612
33.11 Subset-Sum: when small numbers hide big inputs . . . 613
33.12 A worked reduction, end to end . . . 615
33.13 A Plain-English Guide to P, NP, and Related Complexity Classes . . . 615
33.13.1 Why the names are confusing (and how to never mix them up again) . . . 616
33.13.2 What is actually at stake: a day in a P = NP world . . . 619
33.13.3 Why proving P ≠ NP is so hard: the three mathematical barriers . . . 620
33.14 What to do when your problem is NP-complete . . . 621
33.14.1 The practical dividend of hardness . . . 621
33.14.2 Coping with worst-case limits . . . 622
33.15 Exercises . . . 623

34 Approximation Algorithms 625

34.1 The one number that matters: the approximation ratio . . . 626
34.2 Vertex cover: a 2-approximation from a silly idea . . . 627
34.3 The traveling salesman with a triangle inequality . . . 629
34.3.1 Christofides: squeezing the ratio down to 1.5 . . . 631
34.4 Set cover: when the best you can do is logarithmic . . . 632
34.5 MAX-3-CNF: how a coin flip guarantees 7/8 . . . 634
34.6 Subset-sum: an FPTAS, the gold standard . . . 635
34.7 Stepping back: a field of guarantees . . . 638
34.7.1 The trap of uncertified heuristics and non-metric spaces . . . 638
34.7.2 Practical engineering with certified bounds . . . 638
34.8 Exercises . . . 639

35 Parallel and Multithreaded Algorithms 641

35.1 Two ways for work to happen at once . . . 642
35.2 The fork-join model . . . 643
35.2.1 The computation as a graph . . . 644
35.3 Work and span: the two numbers that matter . . . 644

35.3.1 The two laws . . . 645
35.3.2 Speedup and parallelism . . . 645
35.3.3 Worked example: the work and span of P-Fib . . . 646
35.4 Scheduling: turning the two numbers into a real running time . . . 647
35.5 Analyzing parallel loops . . . 649
35.5.1 Parallel prefix sums (scan): up-sweep and down-sweep . . . 650
35.6 A new hazard: races . . . 651
35.7 Parallel matrix multiplication . . . 651
35.7.1 The straightforward version . . . 652
35.7.2 Divide-and-conquer multiply, for a shorter span . . . 652
35.8 Parallel merge sort . . . 653
35.8.1 The naive parallelization, and its disappointing span . . . 654
35.8.2 Merging two sorted arrays in parallel . . . 654
35.8.3 Putting it together: a genuinely parallel merge sort . . . 656
35.9 Brent's theorem, and the right way to think about it . . . 657
35.10Summary . . . 658
Exercises . . . 659

36 Machine-Learning Algorithms **661**

36.1 Learning as optimization: the one idea . . . 662
36.2 Clustering with k-means . . . 663
36.2.1 The cost that k-means minimizes . . . 664
36.2.2 Lloyd's algorithm . . . 665
36.2.3 A fully worked numeric run . . . 665
36.2.4 Why does it stop? (Correctness as a decreasing potential) . . . 666
36.3 Gradient descent: walking downhill on purpose . . . 668
36.3.1 The gradient is the uphill arrow . . . 668
36.3.2 The update rule and the learning rate . . . 668
36.3.3 A fully worked one-variable run . . . 669
36.3.4 Convexity: when downhill is enough . . . 670
36.3.5 Batch vs. stochastic: the great speedup . . . 671
36.4 The perceptron: learning a straight-line boundary . . . 672
36.4.1 The setup: a linear classifier . . . 672

36.4.2 The learning rule . . . 672
36.4.3 A fully worked trace . . . 673
36.4.4 Why it works: the perceptron convergence theorem . . . 674
36.4.5 The perceptron as gradient descent . . . 676
36.5 Summary . . . 676
Exercises . . . 677

37 Modern Systems Architecture: From the CPU Pipeline to DRAM Cycles 679
37.1 Inside the Silicon: 6-Transistor SRAM versus 1T1C DRAM . . . 681
37.1.1 Static RAM (SRAM): The Fast, Expensive Bistable Seesaw . . . 681
37.1.2 Dynamic RAM (DRAM): Density via Dennard's 1T1C Simplification . . . 682
37.2 Anatomy of a DRAM Cycle: Banks, Row Buffers, and Latency Numbers . . . 683
37.2.1 Matrix Organization: Rows, Columns, and the Row Buffer . . . 683
37.2.2 The Three Scenarios: Hit, Empty, and Conflict . . . 684
37.2.3 Reading Memory Timings on Hardware Labels . . . 684
37.3 The CPU Cache Hierarchy and Working-Set Curves . . . 686
37.3.1 The Hierarchy Levels: L1, L2, and L3 . . . 686
37.3.2 The 64-Byte Cache Line: The Quantum of Memory Movement . . . 686
37.3.3 Set Associativity and the Power-of-Two Stride Disaster . . . 687
37.3.4 Hardware Prefetchers and Ulrich Drepper's Working-Set Curves . . . 687
37.4 Virtual Memory, Multi-Level Page Tables, and the TLB . . . 688
37.4.1 The x86-64 4-Level Page Table Walk . . . 688
37.4.2 The TLB and Huge Pages . . . 689
37.5 Inside the Core and Across Cores: Pipeline Execution and MESI False Sharing . . . 690
37.5.1 Out-of-Order Superscalar Execution and Dependency Chains . . . 690
37.5.2 Branch Prediction Penalties and Branchless Code . . . 690
37.5.3 Multicore Coherence and the MESI False Sharing Trap . . . 690
37.5.4 Store Buffers, Memory Ordering, and Memory Barriers . . . 691
37.6 Summary: Hardware-Conscious Algorithm Design . . . 693
Exercises . . . 693

38 Compilers: From Human Text to Optimizing Silicon 697
38.1 History and Overview . . . 698

38.2 Stage 1: Lexing — From Character Streams to Tokens via Finite Automata 700
38.2.1 The Maximal Munch Principle 700
38.2.2 Scanning via Deterministic Finite Automata 701
38.3 Stage 2: Parsing — From Flat Tokens to an Abstract Syntax Tree 701
38.3.1 Why Finite Automata Cannot Parse Code 701
38.3.2 The Trap of Grammar Ambiguity 702
38.3.3 Resolving Ambiguity: Recursive Descent and Operator Precedence Climbing . 702
38.3.4 The Abstract Syntax Tree (AST) 703
38.4 Stage 3: Semantic Analysis — Scoped Symbol Tables and Type Lowering 704
38.4.1 Scoped Symbol Tables as a Stack of Hash Tables 704
38.4.2 Type Checking and Lowering 705
38.5 Stage 4: Intermediate Representation, Control-Flow Graphs, and SSA Form 705
38.5.1 Three-Address Code and the Control-Flow Graph 706
38.5.2 The Mutable Variable Dilemma and Static Single Assignment (SSA) 706
38.5.3 The ϕ (Phi) Function at Control-Flow Joins 706
38.5.4 Four Classic Optimizations Simplified by SSA 706
38.6 Stage 5: The Backend — Instruction Selection, Liveness, and Register Allocation . . . 708
38.6.1 Liveness Analysis: Working Backward Through Time 709
38.6.2 The Register Interference Graph and Chaitin's Graph Coloring 709
38.6.3 The Emitted Machine Code 710
38.7 Summary: The Pipeline of Transformations 710
Exercises 711

39 Quantum Computing: Amplitudes, Interference, and What Qubits Actually Do 713
39.1 A Single Qubit as a 2D Unit Vector 715
39.1.1 Unwrapping Dirac's notation 715
39.1.2 Real amplitudes on the 2D unit circle 715
39.1.3 Measurement and the Born Rule 716
39.2 Quantum Gates as Rotations and Reflections 716
39.2.1 Conservation of probability and unitary matrices 716
39.2.2 Elementary quantum gates 717
39.2.3 Tracing the Hadamard gate by hand: The Two-Coin-Flip experiment 717
39.3 Multiple Qubits, Tensor Products, and Entanglement 719

39.3.1 Two qubits: A 4D vector space . . . 719
39.3.2 The CNOT Gate: Conditional interaction . . . 720
39.3.3 Creating the Bell State: An entangled pair . . . 720
39.3.4 What entanglement does, and what it cannot do . . . 721
39.4 Deutsch's Algorithm and the Phase-Kickback Trick . . . 722
39.4.1 The problem: Constant vs. balanced . . . 722
39.4.2 The quantum oracle and Phase Kickback . . . 722
39.4.3 Tracing Deutsch's algorithm step-by-step . . . 723
39.5 Grover's Search as 2D Plane Geometry . . . 725
39.5.1 The 2D Grover plane . . . 725
39.5.2 Two reflections make a rotation . . . 725
39.5.3 How many steps to find the needle? . . . 726
39.6 Shor's Algorithm, BQP Complexity, and Fault Tolerance . . . 727
39.6.1 Shor's algorithm and the Quantum Fourier Transform . . . 727
39.6.2 Does a quantum computer solve NP-complete problems? . . . 728
39.6.3 The Complexity Class BQP: Where Quantum Sits . . . 728
39.6.4 Quantum Error Correction and the No-Cloning Theorem . . . 729
39.7 Summary . . . 730
Exercises . . . 731

VIII Mathematical Background 733

40 Appendix: Summations 735

40.1 Notation: what $\sum$ actually says . . . 736
40.1.1 Two properties you will use constantly . . . 736
40.2 The arithmetic series . . . 737
40.2.1 Sums of squares and higher powers . . . 739
40.3 The geometric series . . . 739
40.3.1 The two regimes: shrinking vs. growing . . . 740
40.4 The harmonic series . . . 741
40.5 Bounding a sum by an integral . . . 741
40.6 Telescoping: when the middle cancels . . . 743

40.7 A field guide to the recurrences in this book . . . 744
40.7.1 Subtract-a-constant recurrences → arithmetic sums . . . 744
40.7.2 Halve-the-input recurrences → geometric sums . . . 745
40.7.3 Random-pivot and tree-build recurrences → harmonic sums . . . 745
40.8 Putting it together: a fully worked analysis . . . 746
40.9 Summary: Summation Reference and Bounds . . . 747
Exercises . . . 748

41 Appendix: Sets, Counting, and Probability 749
41.1 Sets: the language everything is written in . . . 750
41.1.1 Size, subsets, and the empty set . . . 750
41.1.2 Operations on sets . . . 751
41.1.3 Ordered pairs, tuples, and the Cartesian product . . . 751
41.2 Relations and functions . . . 752
41.2.1 Relations . . . 752
41.2.2 Functions . . . 753
41.3 Counting without enumeration . . . 754
41.3.1 Permutations: counting orderings . . . 754
41.3.2 Combinations: counting selections . . . 755
41.3.3 Identities to understand rather than memorize . . . 756
41.3.4 The binomial theorem . . . 756
41.4 Probability: reasoning about chance . . . 757
41.4.1 Sample spaces and events . . . 757
41.4.2 Conditional probability . . . 758
41.4.3 Independence . . . 759
41.5 Random variables and expectation . . . 760
41.5.1 What a random variable is . . . 760
41.5.2 Expectation: the average value . . . 760
41.5.3 Linearity of expectation . . . 761
41.5.4 Variance, briefly . . . 762
41.6 Useful inequalities . . . 762
41.6.1 Markov's inequality . . . 762
41.6.2 Chebyshev's inequality . . . 763

41.6.3 The union bound . . . 763
41.7 Summary: Sets, Counting, and Probability Reference . . . 764
Exercises . . . 765

42 Appendix: Matrices and Discrete Math 767
42.1 Matrices: grids of numbers that *do* something . . . 768
42.1.1 What a matrix is . . . 768
42.1.2 Adding matrices and scaling them . . . 769
42.1.3 The matrix product . . . 769
42.1.4 A picture of the handshake . . . 770
42.1.5 The identity matrix and the transpose . . . 770
42.1.6 The algebra rules . . . 771
42.1.7 The cost of multiplying matrices . . . 772
42.2 A pinch of linear algebra . . . 773
42.2.1 Vectors, linear combinations, and independence . . . 773
42.2.2 Determinant and inverse, briefly . . . 773
42.3 Graphs as formal objects . . . 774
42.3.1 The definition . . . 774
42.3.2 The vocabulary you actually need . . . 775
42.3.3 Two ways to store a graph . . . 776
42.4 Logarithms, factorials, and identities you will reuse . . . 777
42.4.1 What a logarithm really is . . . 777
42.4.2 The identities, with quick justifications . . . 778
42.4.3 Factorials and how fast they explode . . . 779
42.4.4 Two summation identities worth memorizing . . . 780
42.5 Putting the toolbox together . . . 781
Exercises . . . 781

Part I

Foundations

Chapter 1

What an Algorithm Really Is

You already know more algorithms than you think. When you sort a hand of playing cards, you are running one. When you look up a word in a dictionary by flipping to the middle and then deciding "earlier" or "later," you are running another – and a surprisingly good one. When you give a friend turn-by-turn directions to your house, you are writing one down. The word *algorithm* sounds technical and cold, but the thing it names is something you have been doing since childhood: a finite recipe of unambiguous steps that turns a question into an answer.

This chapter is about taking that everyday notion and sharpening it until it is precise enough to reason about. We are going to ask, slowly and carefully: what exactly *is* an algorithm? What does it mean for one to be *correct*? Why do we care how *fast* it is, when computers keep getting faster anyway? And how do we write algorithms down so that another human – or our future self – can understand them without drowning in the fiddly details of any one programming language?

We will not prove deep theorems yet. The goal is to build the vocabulary and the habits of mind that the rest of the book leans on. By the end you will be able to read a piece of pseudocode, say precisely what problem it solves, argue that it solves that problem, and estimate how its running time grows as the input grows. Those four skills – read, specify, justify, estimate – are the basis of everything that follows.

1.1 Recipes, and why precision matters

We start with a familiar analogy and then look at where it breaks down, because those failures show what precision actually requires.

Intuition

An algorithm is a recipe. It takes some *ingredients* (the input), follows a fixed sequence of *steps*, and produces a *dish* (the output). The two demands we place on a recipe are the two demands we place on an algorithm: every step must be something you can actually do, and the whole thing must eventually finish.

Suppose I hand you this "recipe" for finding the largest number in a list:

> *Look at all the numbers and write down the biggest one.*

Is that an algorithm? It has an input (a list of numbers) and an output (the biggest one), and a human can "run" it. But look closer. "Look at all the numbers" – in what order? All at once? A computer cannot glance at a thousand numbers simultaneously the way your eye sweeps a page. "Write down the biggest one" – how did you decide which is biggest? The instruction quietly assumes you already know how to do the very thing we are trying to describe. It is a wish, not a recipe.

Compare it with this:

> *Set a variable* `best` *to the first number. Then, going left to right through the rest of the numbers one at a time, whenever you meet a number larger than* `best`*, replace* `best` *with that number. When you reach the end,* `best` *holds the answer.*

This version never asks you to do anything you do not already know how to do: read a number, compare two numbers, copy a value. Every operation is explicit, and the order of steps is completely specified. That is the difference between a vague wish and an algorithm: replacing unstated leaps of intuition with concrete, mechanical steps.

Focus

An algorithm is a **finite**, **unambiguous**, **effective** sequence of steps that transforms an input into the desired output. *Finite*: it stops. *Unambiguous*: at every moment it is clear exactly what to do next. *Effective*: each step is a genuinely doable operation, not a restatement of the problem.

History and connection

Around 300 BCE in Alexandria, Euclid recorded the earliest non-trivial algorithm in surviving literature in his *Elements* (Book VII, Propositions 1 and 2). The practical problem was geometric and physical: given two rods of lengths a and b, how do you find the longest common measuring stick that measures both with zero remainder? The tools of Greek geometry were strictly compass and straightedge, executing the physical process of *anthyphairesis* (reciprocal subtraction) – repeatedly laying off the shorter segment along the longer until a remainder was left, and then using that remainder to measure the previous piece. Where earlier artisans guessed common divisors by trial subdivision, Euclid replaced guesswork with a deterministic recipe accompanied by humanity's first recorded termination proof: because positive whole numbers cannot decrease forever, the remainder must strictly shrink until it divides cleanly.
The same procedure also led to a philosophical crisis. When the early Pythagoreans attempted geometric subtraction on the diagonal and side of a unit square ($\sqrt{2}$), the process famously *never stopped* — it spun into an endless, self-similar loop, revealing the existence of incommensurable lengths and overturning the belief that all lengths could be expressed as ratios of integers. Euclid resolved this by strictly separating discrete whole numbers (Book VII), which are well-ordered and always halt, from continuous geometric lengths (Book X). Even in arithmetic, a naive subtraction approach ($\gcd(a, b) = \gcd(a - b, b)$) conceals an algorithmic trap: computing $\gcd(10^9, 1)$ would force a machine to tick through a billion individual subtractions! Compressing repeated subtraction into the modulo operation ($\gcd(a, b) = \gcd(b, a \bmod b)$) turned a process whose step count can grow linearly with the numbers' values into one that

needs only logarithmically many steps.
Euclid's method gave a guaranteed way to reduce fractions and established the template for algorithmic correctness: maintain an invariant while shrinking a quantity toward a base case. It also has computational limits. Finding the greatest common divisor does *not* reveal the underlying prime factorization of a composite number; factoring remains computationally hard, an asymmetry that underpins modern RSA cryptography. In Chapter 2, we formalize Euclid's termination insight into the Floyd-Hoare method of loop invariants for sorting, before returning to Euclid's algorithm in Chapter 30 to compute modular inverses for public-key encryption.

1.2 Problems versus instances

Beginners often blur the next distinction, so we pin it down early. There is a difference between a *problem* and an *instance* of that problem.

A **problem** is the general task, described in terms of *any* possible input. An **instance** is one specific input you might be handed.

Definition 1.1 (ch01:problem-instance)**.** A *computational problem* is specified by describing (i) the set of allowed *inputs*, often called *instances*, and (ii) for each input, what counts as a correct *output*. An *instance* of the problem is a single concrete input drawn from that set.

Take the sorting problem. As a *problem* it reads:

Formal

The Sorting Problem.
Input: a sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.
Output: a reordering $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \cdots \leq a'_n$.

That definition does not mention any particular list. It speaks of *every* list at once. An *instance* of the sorting problem is something concrete, like $\langle 31, 41, 59, 26, 41, 58 \rangle$, whose correct output is the single sequence $\langle 26, 31, 41, 41, 58, 59 \rangle$.

Intuition

Think of a problem as a giant lookup table you can never finish writing: down the left column is every conceivable input, and in the right column is the correct answer for that input. An *instance* is one row. An *algorithm* is a finite machine that, given any row's left entry, computes its right entry – without ever storing the impossible infinite table.

Figure 1.1 draws that mental picture for the sorting problem.

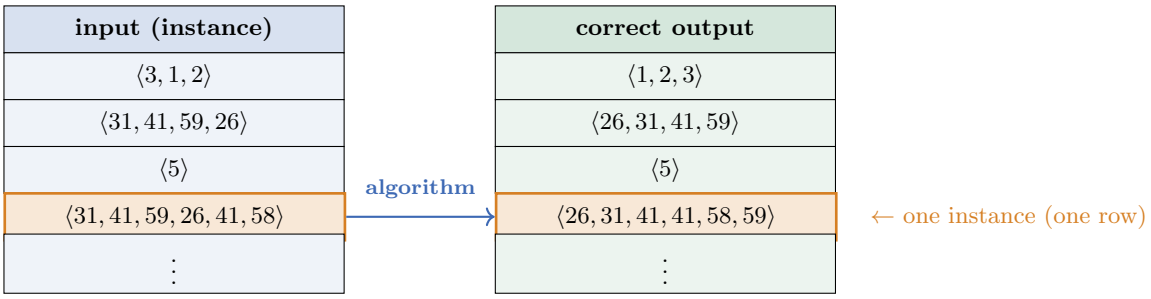


Figure 1.1: The sorting problem as an infinite lookup table: every possible input on the left, its unique correct output on the right. The highlighted row is one *instance*. An *algorithm* is a finite machine that computes the right entry from the left entry for *any* row, without ever storing the table.

This distinction gets its own section because many later confusions clear up once you ask, "Wait, am I talking about the problem or about one instance?" When we say an algorithm is *correct*, we mean it works on *every* instance, rather than merely the single test case in front of us. When we say an algorithm is *slow*, we usually mean there exists some family of instances on which it bogs down. And when we compare two algorithms, we compare them as solvers of the same *problem*, even though they may behave very differently on a given *instance*.

Common trap

"I ran it on $\langle 3, 1, 2\rangle$ and it gave $\langle 1, 2, 3\rangle$, so my sorting algorithm is correct." No – that shows it is correct on *one instance*. Correctness is a claim about *all* instances. A single successful example can never establish it; a single failing example can demolish it. Testing finds bugs; it does not prove their absence.

1.3 What "correct" actually means

We have been using the word *correct* freely. Let us make it sharp.

Definition 1.2 (ch01:correctness)**.** An algorithm is *correct* for a problem if, for *every* instance of that problem, it halts and returns a correct output for that instance. An algorithm that halts with a correct answer on some inputs but on others either halts with a wrong answer or never halts at all is *incorrect*.

Two words in there deserve attention: *every* and *halts*.

Every we have already met – correctness quantifies over all instances. The new word is *halts*. An algorithm that runs forever has not solved your problem, no matter how clever it looks, because you never get an answer. So correctness bundles two promises together: the algorithm *terminates*, and when it terminates the answer is *right*. We will return to both promises again and again, usually proving them with a tool called a *loop invariant*, which we preview at the end of this chapter and use in earnest from the next one onward.

Remark 1.1 (ch01:approximation). For some hard problems we deliberately relax "returns a correct output" to "returns an output provably close to correct." Those are *approximation algorithms*, a

whole subject of their own. And some algorithms flip a coin internally and are only correct *with high probability*; those are *randomized algorithms*. Both are legitimate and useful – but in each case the relaxation is stated openly and precisely rather than left vague.

Why this matters

Why be so fussy about "every instance"? Because software runs on inputs nobody imagined. A date library that works on every tested day except February 29 in a leap year is still incorrect, and that single edge case is enough to bring down a production system. The habit of asking "for which inputs might this fail?" is one of the most valuable habits this book can give you. Correctness separates code that works from code that works *until it doesn't*.

1.4 A first algorithm, written out properly

Now we turn the "find the maximum" recipe from earlier into something we can analyze. Along the way we introduce the way this book writes algorithms down – *pseudocode* – and explain the conventions as we go.

Algorithm 2: FIND-MAX: largest element of a list

Input: An array $A[1..n]$ of n numbers, with $n \geq 1$.
Output: The value of the largest element of A.

```
1 best ← A[1];
2 for i ← 2 to n do
3     if A[i] > best then
4         best ← A[i];
5 return best;
```

Step by step, the first line copies the first element into a variable named *best*, our running champion. The loop then walks the index i from 2 up to n, inspecting each remaining element exactly once. Whenever the current element beats the champion, it becomes the new champion. After the loop, *best* holds the answer, and we return it.

1.4.1 Pseudocode conventions

Pseudocode is not a programming language but a controlled form of English-plus-math whose only goal is to be unambiguous to a human reader. It leaves out details that do not matter to the *algorithm* – semicolons, type declarations, memory management – and keeps the ones that do. Here are the conventions this book uses; they match CLRS closely.

- **Assignment** is written $x \leftarrow v$, read "x gets v." It copies the value v into the variable x. We avoid the symbol $=$ for assignment so that $=$ can mean plain mathematical equality.
- **Arrays** are written $A[1..n]$ and indexed from 1, so $A[1]$ is the first element and $A[n]$ the last. (Many real languages index from 0; the ideas are identical, the arithmetic shifts by one.) The notation $A[i..j]$ means the slice from index i through j inclusive.

- **Blocks** are shown by indentation, not braces. The body of the **for** loop is everything indented beneath it.
- **Loops** come in **for** (a known number of repetitions) and **while** (repeat as long as a condition holds) flavors. After a **for** loop ends, the loop variable is considered to hold the value that first failed the test – a small convention worth remembering.
- **Conditionals** are **if** / **else**. **Return** immediately stops the algorithm and hands back a value.
- We freely use ordinary math – $\leq$, $\lfloor n/2 \rfloor$, "swap x and y" – whenever it is clearer than spelling out machine steps.

Focus

The point of pseudocode is communication, not execution. If a human reader could follow your steps and get the right answer by hand, your pseudocode is good – even if no compiler would accept it. Aim for the clarity of a well-written proof, not the syntax of a particular language.

1.4.2 Tracing it on an instance

Running the steps by hand makes them concrete. Trace FIND-MAX on the instance $A = \langle 4, 9, 3, 9, 7 \rangle$, with $n = 5$. The table records the state *after* each iteration of the loop.

Step	i	$A[i]$	*best* after this step	
initialize	–	–	4	
loop	2	9	9	(since $9 > 4$)
loop	3	3	9	(since $3 \not> 9$)
loop	4	9	9	(since $9 \not> 9$)
loop	5	7	9	(since $7 \not> 9$)

We return 9, which is indeed the largest element. Notice the tie at $i = 4$: the test is strictly $>$, so an equal value does not displace the champion. That choice does not matter for the maximum, but in other algorithms the difference between $>$ and $\geq$ decides whether a sort is *stable* or whether a loop terminates, so train yourself to notice it.

Algorithm lab

Trace FIND-MAX yourself on $A = \langle 5, 5, 5 \rangle$ and on $A = \langle 8, 1, 6, 9, 2 \rangle$. For each, write the four-column table above. Then answer: on an array where the maximum sits at the very first position, how many times does the body of the **if** (the assignment $best \leftarrow A[i]$) actually execute? On an array sorted strictly increasing, how many times? This gap between "never" and "every time" is your first encounter with *best case* versus *worst case*.

1.5 Why correctness needs an argument

FIND-MAX is simple enough that it feels intuitive at a glance. But that very feeling of simplicity is what often masks subtle bugs, so we will practice arguing correctness properly on an easy example, where the machinery is visible and the stakes are low.

The key idea is a *loop invariant*: a statement that is true every time we are about to test the loop condition, and that, once the loop ends, hands us the result we want. Think of it as a promise the loop keeps to itself at the top of every pass.

Focus

Loop invariant for Find-Max. Just before the loop runs with a given value of i, the variable *best* equals the maximum of the first $i-1$ elements, that is, $best = \max\{A[1], A[2], \ldots, A[i-1]\}$.

Figure 1.2 pictures the promise. At the top of the pass for a given i, the array is split into a settled prefix $A[1\,..\,i-1]$ – already scanned, with its maximum captured in *best* – and an unexamined suffix $A[i\,..\,n]$. Each pass folds $A[i]$ into the settled region and slides the boundary one step to the right.

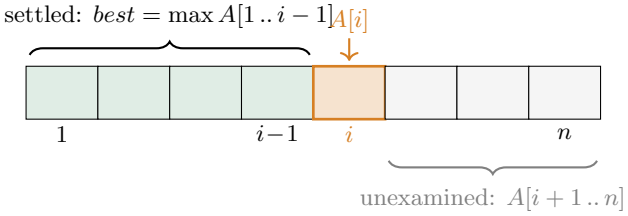


Figure 1.2: The loop invariant of FIND-MAX as a picture. Green: the settled prefix, whose maximum already lives in *best*. Orange: the current element $A[i]$, about to be compared against *best* and absorbed. Gray: the suffix not yet looked at. Each pass moves the green/orange boundary one cell right; when green covers the whole array, *best* is the answer.

To use an invariant we check three things – the same three every time, so they will soon feel routine.

- **Initialization.** The invariant holds before the first iteration. Here, before the loop, $best = A[1]$ and $i = 2$, so $i - 1 = 1$ and indeed $best = \max\{A[1]\}$. True.
- **Maintenance.** If the invariant holds before an iteration, it still holds before the next one. Suppose $best = \max\{A[1], \ldots, A[i-1]\}$ as we enter the body. The body compares $A[i]$ to *best* and keeps the larger. So afterwards $best = \max\{A[1], \ldots, A[i-1], A[i]\} = \max\{A[1], \ldots, A[i]\}$. Then i increases by 1, restoring the invariant's form with the new i. True.
- **Termination.** When the loop ends, the invariant plus the reason the loop stopped give the answer. The loop stops when $i = n + 1$. The invariant then says $best = \max\{A[1], \ldots, A[n]\}$ – exactly the maximum of the whole array, which is what we return. Done.

Formal

Claim. For every array $A[1\,..\,n]$ with $n \geq 1$, FIND-MAX halts and returns $\max\{A[1], \ldots, A[n]\}$.

Why it halts: the loop variable i starts at 2, rises by exactly 1 each pass, and the loop runs only while $i \leq n$; after at most $n - 1$ passes the condition fails.
Why it is right: the three invariant checks above.

This argument has the structure of an induction. *Initialization* is the base case; *maintenance* is the inductive step; and *termination* applies the invariant when the loop stops to establish the final result. A loop-invariant argument is simply mathematical induction applied step by step to a running loop. After a few chapters, running these three checks will become second nature.

Walking question

On your next walk, pick a kitchen task you do without thinking – making tea, folding laundry, washing a stack of dishes – and try to write it as careful pseudocode. Where are the loops? What is the invariant ("after washing k dishes, the first k are clean and stacked")? Where did you slip in an unstated step that quietly relies on human judgment? The discomfort you feel is exactly the gap between a wish and an algorithm.

1.6 Efficiency, and why it matters

Now for the question that drives most of this book. We have an algorithm that is correct. Why isn't that the end of the story?

Because correct is not the same as *usable*. Two algorithms can both be flawlessly correct, return identical answers on every instance, and yet one finishes in a blink while the other would still be running when the sun burns out. The resource we care about most is *time*: how many basic steps the algorithm performs before it stops. (We sometimes also care about *space* – how much memory it uses – but time leads the way.)

Why this matters

You might think: computers double in speed every couple of years, so won't slow algorithms simply become fast ones if we wait? No – and this is the central surprise of the subject. Hardware speed multiplies your throughput by a *constant* factor. A better algorithm can multiply it by a factor that *grows with the input size*. On a million-element problem, the right algorithm can beat the wrong one by a factor of fifty thousand on the very same machine. No amount of waiting for faster chips closes a gap like that. Algorithms are a technology, and often the cheapest, most decisive technology in the whole system.

1.6.1 Counting steps, not seconds

We deliberately do *not* measure efficiency in seconds. Seconds depend on the machine, the language, the compiler, the weather inside the chip. Instead we count *basic operations* – comparisons, additions, assignments, array accesses – as a function of the input size n. This abstracts away the machine and lets us compare algorithms on their merits.

For FIND-MAX, count the comparisons $A[i] > best$. The loop runs for $i = 2, 3, \ldots, n$, which is $n - 1$ iterations, doing one comparison each. So FIND-MAX uses exactly $n - 1$ comparisons, on *every*

instance of size n. The work grows in step with the input – double the array, roughly double the work. We call that *linear* growth, and it is about as good as you can hope for on a problem that must, at minimum, look at every element at least once.

1.6.2 The shape of growth is what matters

When we judge efficiency, we do not fuss over whether an algorithm does $n-1$ steps or $n+3$ or $2n$. We care about the *shape* of the growth – linear versus quadratic versus exponential – because for large inputs the shape swamps every constant.

The table below shows how different growth shapes balloon as n climbs. Imagine a machine doing a billion (10^9) basic steps per second.

n	$\log_2 n$	n	n^2	2^n
10	≈ 3	10	100	1024
100	≈ 7	100	10,000	$\approx 10^{30}$
1,000	≈ 10	1,000	1,000,000	astronomically huge
10^6	≈ 20	10^6	10^{12}	beyond comprehension

Read the bottom row as elapsed time on our billion-steps-per-second machine. A $\log_2 n$ algorithm finishes in about 20 steps – instant. A linear one does 10^6 steps – a millisecond. A quadratic one does 10^{12} steps – about 17 minutes. And a 2^n algorithm on $n = 10^6$? The number of steps has more digits than there are atoms in the observable universe; it will never finish, on any machine, ever. The shape of the curve, not the constant out front, decides whether your program returns before lunch or never.

Intuition

A constant factor is a one-time discount – "half off, forever." A better growth shape is a discount that gets *deeper the bigger your order*. For a small input the discount barely shows; for a large one it dominates all else. That is why, when we compare algorithms, we look past the constants and concentrate on the shape. This squinting has a precise name – *asymptotic analysis*, written with the O, Θ, Ω notation – and it gets its own chapter. For now, get a feel for the difference between linear and quadratic growth.

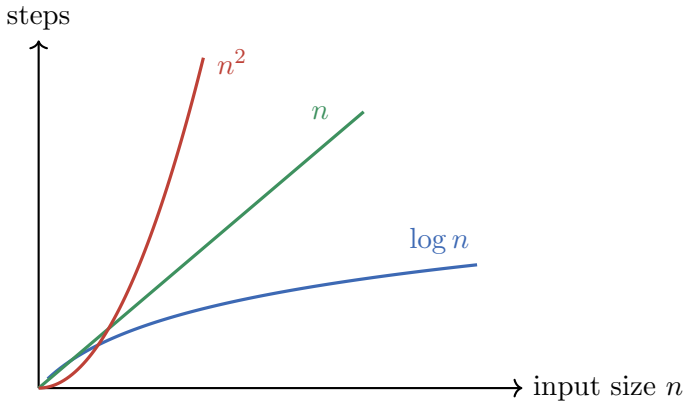


Figure 1.3: The same axes, three growth shapes. For small n the curves are tangled and the constants matter; as n grows the curves fan apart and the *shape* takes over completely. This fanning is why we judge algorithms by growth, not by a stopwatch on one input.

1.7 Comparing two approaches to one task

Talk of growth shapes stays abstract until you watch two correct algorithms solve the *same* problem with wildly different efficiency. The problem we will use: search a list for a target value.

Formal

The Searching Problem.
Input: an array $A[1..n]$ and a value v.
Output: an index i with $A[i] = v$, or the special value Nil if v does not appear in A.

1.7.1 Approach one: linear search

The obvious method: walk through the array from left to right, and stop the moment you find v.

Algorithm 3: Linear-Search

Input: An array $A[1..n]$ and a value v.
Output: An index i with $A[i] = v$, or Nil.

```
1 for i ← 1 to n do
2     if A[i] = v then
3         return i;
4 return Nil;
```

This is correct on any array – sorted or not – and uses at most n comparisons. If v is the first element we stop after one comparison (best case); if v is absent or last we make all n (worst case). Worst-case work is linear in n. Nothing wrong with that. But watch what happens when we are allowed to assume the array is *already sorted*.

1.7.2 Approach two: binary search

In the Preface, we asked what happens when data arrives already organized. Suppose A is sorted in increasing order. Now we can be cleverer. Look at the *middle* element. If it equals v, done. If it is *larger* than v, then v – if present at all – must lie in the left half, so we throw away the entire right half and repeat. If it is *smaller*, we throw away the left half. Each comparison cuts the region still worth searching in half.

Algorithm 4: BINARY-SEARCH (on a sorted array)

Input: A sorted array $A[1\,..\,n]$ with $A[1] \le \cdots \le A[n]$, and a value v.
Output: An index i with $A[i] = v$, or NIL.

```
1  lo ← 1;
2  hi ← n;
3  while lo ≤ hi do
4  |   mid ← ⌊(lo + hi)/2⌋;
5  |   if A[mid] = v then
6  |   |   return mid;
7  |   else if A[mid] < v then
8  |   |   lo ← mid + 1;
9  |   else
10 |   |   hi ← mid − 1;
11 return NIL;
```

The variables lo and hi fence off the slice $A[lo\,..\,hi]$ that could still contain v. Each pass examines its midpoint and discards half the slice. The loop ends either by finding v or by squeezing the slice empty ($lo > hi$), which proves v is absent.

Intuition

This is the dictionary trick in precise form. To find "algorithm" you do not read from "aardvark" onward; you flip to the middle, see you have overshot or undershot, and flip again into the surviving half. Each flip halves what remains, and that halving turns a million-step scan into about twenty comparisons.

Tracing binary search

Let $A = \langle 2, 5, 8, 12, 16, 23, 38, 56, 72, 91\rangle$ (so $n = 10$, sorted), and search for $v = 23$. The state is $(lo, hi, mid, A[mid])$ at the top of each pass.

Pass	lo	hi	mid	$A[mid]$	Decision
1	1	10	5	16	$16 < 23$, so go right: $lo \leftarrow 6$
2	6	10	8	56	$56 > 23$, so go left: $hi \leftarrow 7$
3	6	7	6	23	found it, **return** 6

Three comparisons to find 23 in a ten-element array. Linear search, looking for the same value, would have made six. The gap looks modest here, but it grows quickly with n because the halvings compound.

Figure 1.4 shows the same three passes as shrinking slices. Each shaded band is the live region $A[lo\,..\,hi]$ still worth searching; the boxed cell is the midpoint just examined. Watch the live region collapse by roughly half each time until it lands on the target.

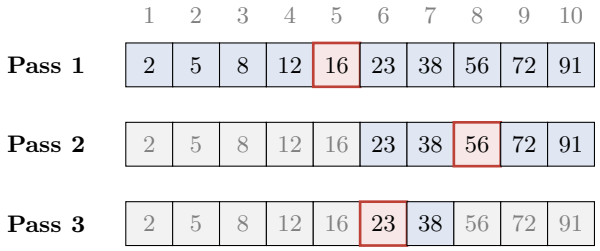


Figure 1.4: Binary search for $v = 23$ in a sorted array of 10 elements. Shaded cells are the live slice $A[lo\,..\,hi]$; the red-outlined cell is the midpoint probed that pass; gray cells have been discarded. Pass 1 probes $A[5] = 16 < 23$ and throws away the left half; pass 2 probes $A[8] = 56 > 23$ and throws away the right; pass 3 probes $A[6] = 23$ and returns. Three probes, not six.

1.7.3 Counting the halvings

How many passes can binary search make in the worst case? Each pass at least halves the size of the live slice. Start with n candidates; after one pass at most $n/2$ remain; after two, at most $n/4$; after k passes, at most $n/2^k$. The loop must stop once fewer than one candidate remains, i.e. once $n/2^k < 1$, which is $2^k > n$, which is $k > \log_2 n$. So the number of passes is at most about $\log_2 n + 1$.

Cost summary

Searching a sorted array of size n.

	worst-case comparisons	growth shape
LINEAR-SEARCH	n	linear
BINARY-SEARCH	$\approx \log_2 n + 1$	logarithmic

Binary search needs a *precondition* – the array must be sorted – that linear search does not. That precondition is the price of the speedup, and remembering it is part of using the algorithm correctly.

For $n = 1{,}000{,}000$, linear search may need a million comparisons; binary search needs about $\log_2(10^6) \approx 20$, for the identical answer on the identical machine. Faster hardware cannot buy back a gap like that, because it comes from a different *shape* of growth rather than a constant factor. Good algorithm design produces wins of this kind again and again.

Common trap

Binary search is famous for being easy to get *almost* right and maddening to get *exactly* right. The classic bugs: writing $lo < hi$ when you meant $lo \le hi$ (so you miss a one-element slice and report a present value as absent), or updating $lo \leftarrow mid$ instead of $lo \leftarrow mid + 1$ (so the slice never shrinks when $mid = lo$, and the loop spins forever). The loop's job is to *strictly shrink* $hi - lo$ every pass; if a bug lets the slice stay the same size, the loop can run forever. When

you write a loop, always ask: "what quantity must strictly decrease to guarantee I stop?"

Walking question

Here is a puzzle to carry on a walk. Binary search is exceptionally fast *but only on a sorted array*. Sorting an array, as you will see in the next chapters, costs roughly $n \log n$ work. So: if you are going to search the array just once, is it worth sorting it first? What if you will search it a thousand times? Ten thousand? Try to find the rough break-even point where "sort once, then binary-search many times" beats "linear-search every time." You have just stumbled onto the idea of *amortizing* a one-time setup cost across many later operations – a theme that runs through the whole field.

1.8 Algorithms as a technology

Compare what happened with these two searches. Both are correct, and both run on the same hardware. The jump from a million comparisons to twenty comes entirely from the algorithm: halving the search range instead of scanning element by element. That is what we mean when we say algorithms are a *technology*, every bit as real as faster processors or bigger memory, and frequently more decisive.

Why this matters

The same lesson applies well beyond search, to nearly everything computers do well: the route your map app finds across a continent in a heartbeat; the way a search engine ranks billions of pages faster than you can blink; the cryptography that keeps a payment secret; the compression that fits a movie down a thin wire; the model that recognizes a face. In each, a clever algorithm turns "impossible" into "instant." When a system is too slow, the seasoned engineer's first question is rarely "can we buy a faster machine?" It is "are we running the right algorithm?" More often than newcomers expect, the answer is no – and fixing it is cheaper, and wins bigger, than any hardware upgrade.

Connection to other fields

The framing of algorithms as a technology, on equal footing with hardware, opens **CLRS Chapter 1**. The companion idea – that we measure an algorithm's quality by how its resource use *grows* with input size, not by stopwatch readings on one machine – is developed in **CLRS Chapter 2 ("Getting Started")** and made rigorous with O, Θ, Ω in **CLRS Chapter 3 ("Characterizing Running Times")**. When you reach CLRS, you will find those chapters move fast; the slow, intuition-first version you are reading now is meant to make that speed feel comfortable rather than punishing.

1.9 What to carry forward

Before the exercises, here is the compressed version – the handful of ideas worth keeping in active memory as you read on.

Focus

The chapter in six sentences.

1. An algorithm is a finite, unambiguous, effective sequence of steps from input to output – a procedure with every step made completely explicit.
2. A *problem* describes the task for all inputs; an *instance* is one concrete input. Correctness, efficiency, and comparison all live at the level of the *problem*.
3. An algorithm is *correct* only if it *halts* with a *right* answer on *every* instance; one good test proves nothing, one bad test proves it broken.
4. We argue correctness with a loop invariant via three checks – initialization, maintenance, termination – which is mathematical induction applied across loop iterations.
5. We measure efficiency by counting basic operations as a function of input size n, not by timing seconds on one machine.
6. What matters is the *growth shape* – logarithmic, linear, quadratic, exponential – because for large n the shape overwhelms every constant factor.
7. A better algorithm can beat a worse one by a margin that grows with n, which no faster hardware can match: algorithms are a technology.

Everything ahead – sorting, data structures, graph algorithms, dynamic programming – repeats the same four steps: specify the problem precisely, design a method, prove it correct, and analyze how its cost grows.

In Chapter 2, we turn to the foundational challenge that makes binary search possible in the first place: *sorting*. We begin with the method you naturally use when picking up playing cards – *insertion sort* – and use it to put our new tools of loop invariants and step-counting to their first rigorous test.

Exercises

Exercise 1.1 (Wish or algorithm?)**.** For each "recipe" below, decide whether it is a genuine algorithm or a vague wish, and if it is a wish, identify the unstated step it assumes you already know how to do. (a) "To sort the list, put the numbers in increasing order." (b) "To sort the list, repeatedly find the smallest remaining number and move it to the end of a growing output list, until none remain." (c) "To find a friend in a crowd, look for them." (d) "Start at the left wall; walk forward scanning faces; stop when you see your friend or reach the right wall."

Exercise 1.2 (Problem versus instance)**.** Write down, in the formal Input/Output style of this chapter, the problem "find the *second*-largest element of an array of distinct numbers." Then give two specific instances of size 5 together with their correct outputs.

Exercise 1.3 (Trace it)**.** Trace Binary-Search on $A = \langle 2, 5, 8, 12, 16, 23, 38, 56, 72, 91 \rangle$ searching for $v = 12$, and again for the absent value $v = 40$. Produce the full $(lo, hi, mid, A[mid])$ table for each, and report how many passes each search takes.

Exercise 1.4 (Off-by-one autopsy)**.** Someone writes binary search with the loop condition `while` $lo < hi$ (strict) instead of $lo \leq hi$. Find a concrete sorted array and a target v *present* in it for which this buggy version returns Nil. Explain in one sentence which slice the bug fails to inspect.

Exercise 1.5 (Counting comparisons)**.** Exactly how many comparisons $A[i] = v$ does LINEAR-SEARCH perform, in the worst case, on an array of size n? In the best case? Describe an instance achieving each.

Exercise 1.6 (An invariant of your own)**.** Consider FIND-MIN, the obvious twin of FIND-MAX that returns the *smallest* element. Write its pseudocode, state a loop invariant for it, and carry out all three checks (initialization, maintenance, termination).

Exercise 1.7 (Sum of an array)**.** Write pseudocode for ARRAY-SUM($A[1\,..\,n]$) returning $A[1]+A[2]+\cdots+A[n]$. State a loop invariant about the running total, prove correctness with the three checks, and give the exact number of additions performed as a function of n.

Exercise 1.8 (Growth-shape intuition)**.** A quadratic algorithm does n^2 steps; a linear one does $100n$ steps – a hundred times more work *per element*. For which values of n is the linear algorithm actually faster? Solve $100n < n^2$ for n. What does your answer say about trusting small-input benchmarks when choosing between algorithms of different growth shapes?

Exercise 1.9 (Sort-then-search break-even)**.** Pursue the walking question from the chapter. Suppose sorting an n-element array costs about $6\,n\log_2 n$ basic operations, one linear search costs about n, and one binary search costs about $\log_2 n$. You will perform k searches on the same array. Write an inequality, in terms of n and k, for when "sort once, then do k binary searches" beats "do k linear searches." For $n = 1{,}000{,}000$, roughly how large must k be? (An estimate is fine.)

Exercise 1.10 (Halving, generalized – challenge)**.** Binary search throws away *half* the candidates per probe. Imagine a "ternary" variant that uses two probes per pass to throw away *two thirds* of the candidates. After k passes, at most how many candidates remain, as a function of n and k? How many passes until fewer than one remains? Compare the pass count to binary search's $\log_2 n$, and explain why doing more comparisons *per pass* does not necessarily mean fewer comparisons *overall*.

Exercise 1.11 (Find the missing number – challenge)**.** You are given an array containing $n-1$ distinct numbers, which are some $n-1$ of the n integers $\{1, 2, \ldots, n\}$ in arbitrary order; exactly one integer from that range is missing. Design an algorithm that finds the missing integer using a number of basic operations that grows only *linearly* in n, *without* sorting the array. (Hint: what is $1+2+\cdots+n$, and can you compute it with a formula?) Write the pseudocode and justify why it is correct.

Chapter 2

Getting Started: Insertion Sort and Loop Invariants

In Chapter 1, we saw how much depends on data being sorted: binary search collapses a million candidates to twenty, but only if the array is already in order. That leaves the foundational question: how do we put an unsorted array into order in the first place?

There is a reason almost every serious book on algorithms begins with sorting, and a reason this one begins with *insertion sort* in particular. Sorting is an ideal first example. The problem is easy to state, the answer is easy to check, and yet the act of sorting introduces, in miniature, almost every core question we will ask for the rest of the book: how to describe a process precisely, how to convince ourselves it is *correct*, and how to measure how *long* it takes without ever running it on a computer.

Insertion sort is the algorithm you already use, whether you know it or not, when you pick up a hand of playing cards and tidy them. We are going to take that everyday motion, slow it down, and turn it into something we can reason about with full mathematical confidence. By the end of this chapter you will have a tool — the *loop invariant* — that lets you prove a loop does what you claim, and a habit — counting *order of growth* — that lets you compare algorithms before you have written a single line of runnable code.

2.1 The problem we are solving

Let us be painfully precise about the goal, because precision here pays off everywhere later. We are given a list of numbers, and we want them rearranged into increasing order. "Increasing" will mean "non-decreasing": equal values are allowed to sit next to each other.

Formal

The sorting problem. *Input:* a sequence of n numbers $\langle a_1, a_2, \ldots, a_n \rangle$.
Output: a permutation (a reordering) $\langle a'_1, a'_2, \ldots, a'_n \rangle$ of the input such that

$$a'_1 \leq a'_2 \leq \cdots \leq a'_n.$$

The definition makes two separate demands.

Focus

A correct sorting algorithm must satisfy *two* conditions, not one.

1. **Ordered:** the output is sorted, $a'_1 \leq \cdots \leq a'_n$.
2. **Permutation:** the output contains exactly the same elements as the input, just possibly in a different order — nothing added, nothing lost, nothing duplicated.

An algorithm that returns $\langle 1, 2, 3, \ldots, n \rangle$ no matter what you feed it is perfectly ordered and completely wrong. Hold on to *both* conditions.

We will treat the numbers as living in an array, which is just a row of labeled boxes. We write $A[1\,..\,n]$ for an array with n slots, and $A[i]$ for the value in slot i. Throughout this book array indices start at 1, matching the way you would count cards in your hand and matching CLRS; later chapters will note where real programming languages start at 0 instead.

History and connection

Before stored-program computers, sorting was backbreaking clerical labor: millions of census records, banking ledgers, and punch cards had to be manually ordered by clerks or mechanical sorters. In 1945, at the Institute for Advanced Study, John von Neumann wrote his historic *First Draft of a Report on the EDVAC*. While early designers viewed electronic computers purely as giant calculators for artillery ballistics, von Neumann recognized that data processing — ordering, merging, and searching discrete records — would consume the lion's share of future computer cycles. On an experimental machine with acoustic mercury delay-line memory holding a mere 1,024 words, von Neumann wrote the first electronic sorting routines. Two decades later, Robert W. Floyd (1967) and C.A.R. (Tony) Hoare (1969) put program correctness on a rigorous footing by formulating the method of *loop invariants* (initialization, maintenance, and termination), allowing programmers to prove a loop correct for all possible inputs rather than merely hoping it works.

A common false start in early programming courses was Bubble Sort: its double loop of blind adjacent swaps looks simple on paper, but it repeatedly scans pairs that need no attention and lacks adaptability. Insertion sort, by contrast, mimics how human hands naturally sort playing cards, growing a sorted hand on the left and sliding each new card leftward into place. But intuition misleads here: because insertion sort feels instantaneous on thirteen cards, beginners assume it scales. On an array of 100,000 reversed numbers, it grinds through roughly five billion comparison and shift operations, which takes seconds even on a modern processor.

Insertion sort contributed two things: the prototype of formal inductive correctness via loop invariants, and an adaptive, cache-friendly method that runs in $\Theta(n)$ linear time on data that is already nearly sorted. Modern standard library sorts (such as Python's Timsort and C++'s Introsort) still rely on insertion sort to finish sorting small subarrays ($n \leq 32$) because its sequential memory access keeps CPU caches hot. For general data, though, its $\Theta(n^2)$ worst case cannot be avoided. In Chapter 3, we formalize this cost using Donald Knuth's asymptotic notation (Θ, O, Ω), before Chapter 4 introduces Merge Sort to bring the cost down to $\Theta(n \log n)$.

2.2 The card-player's idea

Picture yourself dealt a hand of cards, face down, on the table. You pick them up one at a time with your right hand and slot each new card into its correct place among the cards already arranged in your left hand. At every moment, the cards in your left hand are sorted; the cards still on the table are not yet looked at.

Intuition

The algorithm fits in one sentence: *keep a sorted region on the left, and repeatedly take the next unsorted item and slide it leftward, past everything bigger than it, until it lands in the right spot.*
The sorted region starts as just the first card: with only one item in hand, there are no adjacent pairs to be out of order. Each step grows the sorted region by exactly one. After you have inserted the last card, the whole hand is sorted. That "grow the sorted region by one each time" property is exactly what we will pin down with a loop invariant.

The only slightly clever part is *how* a card slides into place. You do not yank out a gap and drop it in. Instead you hold the new card (call it the *key*), compare it with the card to its left, and if that left card is bigger, you shove the left card one slot to the right to make room, then keep comparing with the next card further left. You stop when you meet a card that is not bigger than the key (or you run off the left end), and you drop the key into the gap you just opened.

Figure 2.1 shows the two regions at one frozen instant: a sorted left hand, an untouched pile on the right, and the single key being lifted across the boundary between them.

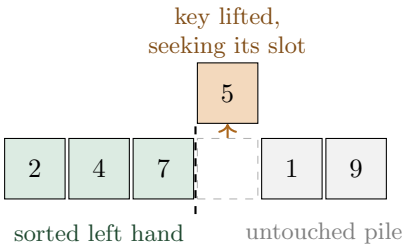


Figure 2.1: One frozen instant of insertion sort. The dashed line is the boundary; the key just left the unsorted pile and will slide left until it meets a card no bigger than itself.

2.3 Insertion sort, written down

Let us turn that motion into precise pseudocode. We process positions $j = 2, 3, \ldots, n$. At the start of step j, the subarray $A[1 \,..\, j-1]$ is already sorted (the cards in your left hand), and $A[j]$ is the new card we must insert.

Algorithm 5: INSERTION-SORT(A)

Input: An array $A[1 \,..\, n]$ of n numbers.
Output: The same array, rearranged so that $A[1] \le A[2] \le \cdots \le A[n]$.

```
1 for j ← 2 to n do
2     key ← A[j];
      // Insert A[j] into the sorted sequence A[1 .. j − 1].
3     i ← j − 1;
4     while i > 0 and A[i] > key do
5         A[i + 1] ← A[i];
6         i ← i − 1;
7     A[i + 1] ← key;
8 return A;
```

The outer **for** loop walks the boundary of the sorted region rightward. Inside, *key* is the card we are placing. The **while** loop does the shoving: as long as we have not fallen off the left edge ($i > 0$) and the card at $A[i]$ is strictly bigger than the key, we copy $A[i]$ one slot to the right (opening room) and step i leftward. When the loop stops, slot $i + 1$ is the empty gap, and that is exactly where the key belongs, so we drop it there.

Common trap

The comparison is $A[i] > key$, a *strict* inequality, and that strictness matters. It makes the sort *stable*: when two values are equal, the one that started earlier stays earlier, because we stop shoving the moment we meet an equal value rather than sliding past it. Write $A[i] \ge key$ by accident and the sort is still *correct* (still sorted, still a permutation) but no longer stable, and on a run of equal keys it does needless extra work. Stability will matter a great deal in Chapter 8 on sorting in linear time, where we lean on it to chain sorting passes together.

2.4 A full numeric trace

To make this concrete, let us sort

$$A = \langle 5,\ 2,\ 4,\ 6,\ 1,\ 3 \rangle.$$

We will show the array after each pass of the outer loop. The vertical bar "|" marks the boundary: everything to its left is the sorted region, everything to its right is untouched.

Algorithm lab

Trace of Insertion-Sort on $\langle 5, 2, 4, 6, 1, 3 \rangle$.

Start: $5 \mid 2\ \ 4\ \ 6\ \ 1\ \ 3$ (sorted region is just $A[1] = 5$)

$j = 2$, **key** $= 2$. Compare with 5: since $5 > 2$, shove 5 right. Now $i = 0$, run off the edge, drop 2 in slot 1.

$$2\ \ 5 \mid 4\ \ 6\ \ 1\ \ 3$$

$j = 3$, **key** $= 4$. Compare with 5: $5 > 4$, shove 5 right. Compare with 2: $2 \not> 4$, stop. Drop 4 after the 2.

$$2\ \ 4\ \ 5 \mid 6\ \ 1\ \ 3$$

$j = 4$**, key** $= 6$**.** Compare with 5: $5 \not> 6$, stop immediately. The key is already in place; drop it back where it was.

$$2\ \ 4\ \ 5\ \ 6\ |\ 1\ \ 3$$

$j = 5$**, key** $= 1$**.** Compare with $6, 5, 4, 2$ in turn; each is bigger, so all four shove right. Run off the edge, drop 1 in slot 1.

$$1\ \ 2\ \ 4\ \ 5\ \ 6\ |\ 3$$

$j = 6$**, key** $= 3$**.** Compare with $6, 5, 4$ (all bigger, shove right), then 2: $2 \not> 3$, stop. Drop 3 after the 2.

$$1\ \ 2\ \ 3\ \ 4\ \ 5\ \ 6\ |$$

The sorted region grew by one on every pass — $1, 2, 3, 4, 5, 6$ elements — and after the last pass the whole array is sorted. Two passes are worth contrasting: at $j = 4$ the key 6 caused *zero* shoves (it was already in place), while at $j = 5$ the key 1 caused *four* shoves (it had to travel all the way to the front). That gap between "barely any work" and "maximum work" is precisely what the running-time analysis later in the chapter will measure.

The hardest pass of this trace is $j = 5$, where the key 1 travels all the way to the front. Figure 2.2 captures that pass step by step: lifting the key creates a temporary gap; each element strictly greater than 1 slides one slot right to propagate the gap leftward; and finally 1 drops into the opened first slot.

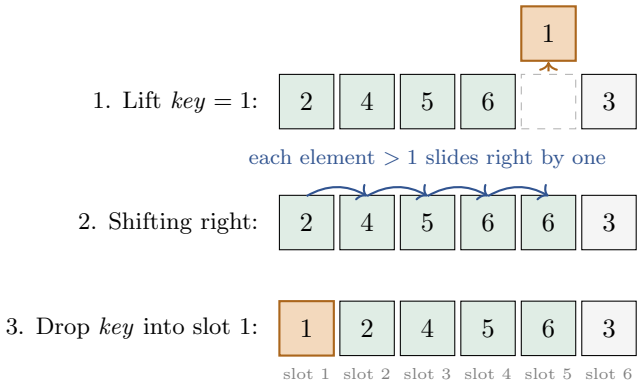


Figure 2.2: The mechanics of inserting $key = A[5] = 1$ during pass $j = 5$. Lifting the key opens an initial gap at position 5. In the inner loop, each sorted element strictly greater than 1 slides rightward by one position, walking the gap down to slot 1. Finally, the key drops into the opened slot, restoring sorted order on the prefix.

2.5 Loop invariants: how we *know* it works

Tracing one example builds confidence, but it does not build certainty. The array $\langle 5, 2, 4, 6, 1, 3 \rangle$ came out right — but what about the millions of arrays we did not try? We need an argument that covers *all* inputs at once. The standard tool for reasoning about loops is the *loop invariant*: a statement that is true every time the loop is about to start a new iteration, no matter what the input was.

Intuition

A loop invariant is like a promise you make about the state of the world at the top of the loop, and then keep, iteration after iteration. If the promise is true before the first iteration, and each iteration preserves it, then by the time the loop ends the promise still holds — and we choose the invariant so that, combined with the loop's stopping condition, it directly implies correctness. It is mathematical induction applied to a loop's state.

To use a loop invariant we check three things. The names are worth memorizing because you will run these checks for every loop in the book.

Focus

The three obligations of a loop invariant.

- **Initialization.** It is true before the first iteration of the loop. (This is the base case of the induction.)
- **Maintenance.** If it is true before an iteration, it remains true before the next iteration. (This is the inductive step.)
- **Termination.** When the loop ends, the invariant — together with the condition that ended the loop — gives us a useful property that proves the algorithm correct.

The first two are exactly induction: a base case and a step. The third is what distinguishes a loop invariant from plain induction — we deliberately choose the invariant so that its *ending* state tells us what we set out to prove. An ordinary induction proves a fact for all n; a loop invariant uses the stopping condition to turn that inductive fact into a guarantee about the final output.

2.5.1 The invariant for insertion sort

Here is the invariant for the outer **for** loop of Insertion-Sort.

Formal

Outer-loop invariant. At the start of each iteration of the **for** loop (the one indexed by j), the subarray $A[1\,..\,j-1]$ consists of the elements originally in $A[1\,..\,j-1]$, but in sorted order.

It claims two things — sorted, *and* the same elements — matching the two correctness conditions from the start of the chapter. Now we discharge the three obligations.

Formal

Initialization. Before the first iteration we have $j = 2$, so the subarray in question is $A[1\,..\,1]$, containing just the single element $A[1]$. With no pair of elements to be out of order, a single-element list is already sorted, and it contains precisely the original element $A[1]$. The invariant holds.

Maintenance. Assume the invariant holds at the start of an iteration: $A[1\,..\,j-1]$ is a sorted rearrangement of the original $A[1\,..\,j-1]$. The body grabs $key = A[j]$ and runs the **while** loop. That inner loop slides each element of $A[1\,..\,j-1]$ that is strictly greater than key one position to the right, and then deposits key into the freed slot. Because every element moved

was greater than key, and every element to the left of the landing slot was not greater than key, the result $A[1\,..\,j]$ is sorted; and since we only moved elements and re-inserted the very value we removed, it contains exactly the original elements of $A[1\,..\,j]$. Incrementing j for the next iteration restores the invariant's form "$A[1\,..\,j-1]$ is a sorted rearrangement" — now over a region one longer.

Termination. The loop ends when j exceeds n, i.e. when $j = n+1$. Substituting $j = n+1$ into the invariant, the subarray $A[1\,..\,n]$ is a sorted rearrangement of the original elements. But $A[1\,..\,n]$ is the entire array. So the whole array is sorted and is a permutation of the input — which is exactly the sorting problem. □

That is a complete correctness proof. Three short paragraphs cover every possible input array of every possible length, which no amount of testing could do.

Common trap

A subtlety swept under the rug above: maintenance leaned on the inner **while** loop "doing the right thing." To be fully rigorous you would give the **while** loop its *own* invariant — for example, "$A[i+1\,..\,j]$ all hold values $\geq key$, shifted from their original spots, and $A[1\,..\,i]$ is unchanged and still sorted." We will not write that one out in full here (it is an exercise), but you should notice that *nested* loops generally need *nested* invariants. Skipping the inner argument is a common way for a "proof" to end up with a gap.

Walking question

Take this on a walk. We observed that a single-element list has no out-of-order pairs and is therefore sorted. What about an *empty* list — is it sorted? Our loop never starts with an empty sorted region (it starts at $j = 2$ with one element). But many algorithms do. Convince yourself that the statement "$a'_1 \leq a'_2 \leq \cdots \leq a'_n$" is *vacuously true* when $n = 0$ or $n = 1$, because there are no adjacent pairs to violate it. Why is "vacuously true" the right default rather than a cop-out?

2.6 How long does it take?

We have an algorithm we trust. The next question is how *expensive* it is. We could time it on a laptop, but that answer would depend on the laptop, the programming language, the weather in the data center, and a hundred other things we do not care about. We want a measure of cost that depends only on the algorithm and the size of the input. The standard approach is to count *primitive steps*: each line of pseudocode is assumed to take some constant amount of time per execution, and we add up how many times each line runs.

Let n be the array length. Let t_j be the number of times the **while**-loop test is evaluated for that value of j (this is where all the input-dependence hides — some keys slide far, some not at all). Annotating each line with a per-execution cost c_k and an execution count gives the following.

Line	cost	times
`for` $j \leftarrow 2$ `to` n	c_1	n
$key \leftarrow A[j]$	c_2	$n-1$
$i \leftarrow j-1$	c_3	$n-1$
`while` $i > 0$ `and` $A[i] > key$	c_4	$\sum_{j=2}^{n} t_j$
$A[i+1] \leftarrow A[i]$	c_5	$\sum_{j=2}^{n}(t_j-1)$
$i \leftarrow i-1$	c_6	$\sum_{j=2}^{n}(t_j-1)$
$A[i+1] \leftarrow key$	c_7	$n-1$

The **for** test runs n times (the extra one is the final test that fails and exits). The body lines run $n-1$ times. The **while** test for a given j runs t_j times; the two lines *inside* the **while** run one fewer, $t_j - 1$ times, because the last test is the one that fails. Adding it all up, the total running time $T(n)$ is

$$T(n) = c_1 n + (c_2 + c_3 + c_7)(n-1) + c_4 \sum_{j=2}^{n} t_j + (c_5 + c_6) \sum_{j=2}^{n} (t_j - 1).$$

Everything except the t_j sums is fixed by n alone; all the input-dependent variation comes from how far each key has to slide, measured by t_j.

2.6.1 Best case: already sorted

If the input is already sorted, then for each j the key $A[j]$ is $\geq$ everything to its left, so the **while** test fails immediately: $A[i] = A[j-1] \not> A[j]$. That means $t_j = 1$ for every j. The two sliding lines never run. Substituting,

$$T(n) = c_1 n + (c_2 + c_3 + c_4 + c_7)(n-1) = an + b$$

for constants a, b. This is a *linear* function of n. The best case is fast: insertion sort barely glances at an already-sorted array.

2.6.2 Worst case: reverse-sorted

Now suppose the input is in strictly *decreasing* order. Every new key is smaller than everything to its left, so it must slide all the way to the front. For position j the **while** test runs $t_j = j$ times (it compares against all $j-1$ earlier elements and then once more to fall off the edge). We need two standard sums; the arithmetic series shows up constantly, so meet it now:

$$\sum_{j=2}^{n} j = \left(\sum_{j=1}^{n} j \right) - 1 = \frac{n(n+1)}{2} - 1, \qquad \sum_{j=2}^{n} (j-1) = \sum_{k=1}^{n-1} k = \frac{n(n-1)}{2}.$$

Where does $\sum_{k=1}^{m} k = \frac{m(m+1)}{2}$ come from? Pair the first and last numbers: $1 + m = m + 1$. Pair the second and second-to-last: $2 + (m-1) = m + 1$. Every pair adds to $m+1$, and with m numbers there are $m/2$ such pairs, giving $\frac{m(m+1)}{2}$.

Plugging these in, every term that involved $\sum t_j$ now contributes something proportional to n^2, and after collecting constants we get

$$T(n) = an^2 + bn + c$$

for some constants $a > 0$, b, c. This is a *quadratic* function of n. The worst case is slow: doubling the input roughly *quadruples* the work.

Intuition

Why n^2, intuitively? In the worst case, inserting the j-th key costs about j comparisons, and we sum that over all j from 2 to n. Summing $1 + 2 + 3 + \cdots + n$ gives about $n^2/2$. The "triangle" of work — a little for the early keys, a lot for the late ones — has area proportional to n^2. Any algorithm whose k-th step costs about k will show this same triangular, quadratic behavior.

Figure 2.3 makes the triangle literal: bar j has height $t_j = j$, and the bars together fill a right triangle of area about $\frac{1}{2}n^2$.

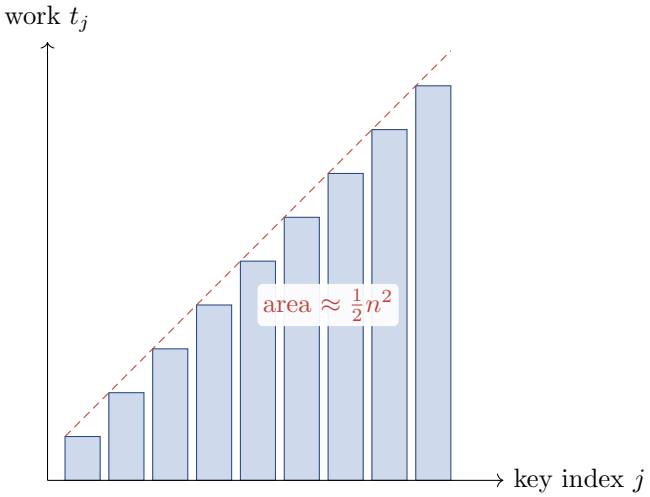


Figure 2.3: Worst-case work of insertion sort. The j-th key costs about j comparisons, so the bars climb linearly and their total area — the running time — grows like $\frac{1}{2}n^2$, hence $\Theta(n^2)$.

2.6.3 Average case

What about a "typical" random input? On average, when we insert $A[j]$ we expect it to land somewhere in the middle of the $j - 1$ already-sorted elements, so we expect to shove about half of them: $t_j \approx j/2$. Halving the worst-case sum still leaves a sum proportional to n^2. So the average case is *also* quadratic — about half the worst-case constant, but the same shape. This is a recurring and slightly disheartening lesson:

Focus

For insertion sort, best case is linear ($\Theta(n)$) but *both* the worst case *and* the average case are quadratic ($\Theta(n^2)$). Being lucky with the average does not save you here — "half of a lot" is still a lot. To beat n^2 on typical input we will need a genuinely different idea, which is where

merge sort comes in.

2.7 Order of growth

Our two formulas were $an+b$ and an^2+bn+c. When n is large, the leading term swamps everything else. For $n = 1000$, the term $n^2 = 1{,}000{,}000$ dwarfs any reasonable bn or c; and the constant a in front, while it affects the stopwatch, does not change the fundamental *shape* of the growth. So we throw away the lower-order terms and the leading constant, and we keep only the shape. We say insertion sort's worst-case running time is *of order* n^2, and we will write this $\Theta(n^2)$ ("theta of n squared").

Intuition

Order of growth answers the only question that survives changing computers: "if I make the input twice as big, roughly how much more work is it?" For a $\Theta(n)$ algorithm, double the input means double the work. For $\Theta(n^2)$, double the input means *four times* the work. For $\Theta(\log n)$, double the input means just *one more* step. The constants and small terms decide whether you wait 2 seconds or 5; the order of growth decides whether the algorithm is usable at all when n gets large.

We will make Θ, O, and Ω fully precise in the next chapter, with definitions and proofs. For now, carry this loose-but-correct intuition:

- $\Theta(f(n))$ means "grows like $f(n)$" — a tight, two-sided bound.
- $O(f(n))$ means "grows no faster than $f(n)$" — an upper bound, a ceiling.
- $\Omega(f(n))$ means "grows at least as fast as $f(n)$" — a lower bound, a floor.

So we may also say: insertion sort runs in $O(n^2)$ on *every* input (it is never worse than quadratic), and in $\Omega(n)$ on every input (it must at least look at each element once).

Cost summary

Insertion sort.

- Best case (sorted input): $\Theta(n)$ time.
- Worst case (reverse-sorted): $\Theta(n^2)$ time.
- Average case (random input): $\Theta(n^2)$ time.
- Extra space: $\Theta(1)$ — it sorts *in place*, using only a couple of variables beyond the array itself.
- Stable: yes (equal keys keep their original relative order).

Why this matters

You might think a quadratic sort is obsolete, but insertion sort is the fastest known method for *small* arrays — the constant factor a is tiny because each step is so cheap — so real

industrial sorting libraries (the ones inside your phone, your browser, your database) use a fancy $\Theta(n \log n)$ method for big inputs but *switch to insertion sort* once the subproblems shrink below a threshold of maybe 10 to 30 elements. It is also the natural choice when data arrives a little at a time and is "almost sorted" already, since then it runs in nearly linear time. As a result, insertion sort runs inside almost every program you use.

2.8 Designing algorithms: two stances

Insertion sort is an example of the *incremental* design stance: having sorted $A[1..j-1]$, we insert one more element to extend the solution by a single step. We grow the answer outward, one item at a time. Many algorithms fit this mold, and the loop-invariant method is tailor-made for analyzing them.

But there is a second, often far faster stance: *divide and conquer*. Instead of nibbling at the problem one element at a time, we split it into a few smaller subproblems of the same type, solve each (often recursively), and then stitch the sub-answers together. This stance frequently turns a $\Theta(n^2)$ algorithm into a $\Theta(n \log n)$ one — an enormous improvement at scale. The example to keep in mind, and the subject of this chapter's successor, is *merge sort*.

2.9 A first taste of merge sort

Merge sort applies divide and conquer in three steps.

Intuition

Divide: split the n-element array into two halves of about $n/2$ each.
Conquer: sort each half — by calling merge sort on it recursively. A one-element array is already sorted, so the recursion bottoms out there.
Combine: *merge* the two sorted halves into one sorted whole.

The key step is the *merge*. Merging two already-sorted lists is easy and fast: look at the front of each list, take the smaller of the two, and repeat. Each comparison commits one element to the output, so merging two lists of total length n costs only about n comparisons — it is $\Theta(n)$, linear.

Algorithm lab

Merging $\langle 2, 4, 5, 6 \rangle$ and $\langle 1, 3 \rangle$. Two fingers, one on each list's front. Smaller finger's element goes to the output; advance that finger.

fronts $(1, 2)$	take 1	out: 1
fronts $(3, 2)$	take 2	out: $1, 2$
fronts $(3, 4)$	take 3	out: $1, 2, 3$
right list empty	take rest $4, 5, 6$	out: $1, 2, 3, 4, 5, 6$

Six output elements, and we never looked back. That "one comparison per output element" is why merging is linear.

Figure 2.4 draws the same merge as two fingers walking down their lists. At each step the smaller of the two pointed-at values is emitted and only that finger advances; when one list runs dry, the rest of the other is copied wholesale.

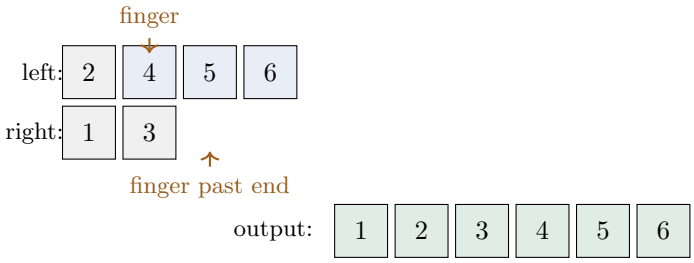


Figure 2.4: Merging $\langle 2, 4, 5, 6\rangle$ with $\langle 1, 3\rangle$. Greyed cells were already emitted; each comparison of the two finger cells sends one element to the output below. Once the right list is exhausted, its finger walks off the end and the remaining left cells $4, 5, 6$ are appended directly.

Now the cost. Let $T(n)$ be merge sort's running time on n elements. Dividing is cheap, $\Theta(1)$. Conquering is two recursive calls on inputs of size $n/2$, costing $2\,T(n/2)$. Combining (the merge) is $\Theta(n)$. Writing cn for the divide-plus-merge work, we get the *recurrence*

$$T(n) = 2\,T(n/2) + cn, \qquad T(1) = \Theta(1).$$

We will solve recurrences properly in the chapter on divide and conquer, but the recursion tree of Figure 2.5 already gives the answer. Picture the recursion as a tree: the root does cn work splitting and merging the whole array; its two children each do $c\,(n/2)$ work; their four children each do $c\,(n/4)$; and so on.

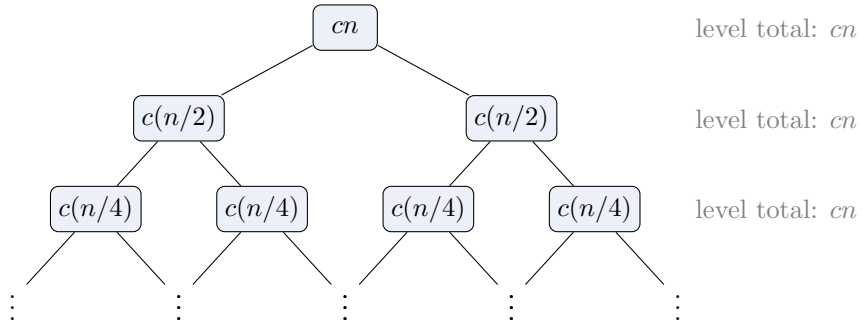


Figure 2.5: The merge-sort recursion tree for $T(n) = 2T(n/2) + cn$. The subproblem sizes on each level sum back to n, so every level costs cn. With about $\log_2 n$ levels, the total is $cn \log_2 n = \Theta(n \log n)$.

Each level of the tree does cn work in total, because the subproblem sizes at any level add back up to n (the halves recombine). And there are about $\log_2 n$ levels, because you can only halve n down to 1 about $\log_2 n$ times. Multiply: (work per level) $\times$ (number of levels) $= cn \cdot \log_2 n$, giving

$$T(n) = \Theta(n \log n).$$

Focus

Compare the two algorithms on $n = 1{,}000{,}000$. Insertion sort's worst case does about $n^2 = 10^{12}$ operations. Merge sort does about $n \log_2 n \approx 10^6 \times 20 = 2 \times 10^7$. That is a factor of roughly *fifty thousand*. No constant-factor speedup or faster laptop closes a gap like that, which is why "order of growth" is the first idea we introduced: it can decide whether an algorithm finishes at all.

Connection to other fields

Merge sort is one of the earliest algorithms designed for stored-program computers. We will develop it in full detail in Chapter 4 on divide and conquer, including a careful merge procedure, a correctness proof by loop invariant, and a rigorous solution to the recurrence $T(n) = 2T(n/2) + \Theta(n)$. Here it simply shows that insertion sort's quadratic cost can be beaten. In CLRS, this preview lives in Section 2.3.

Walking question

On your walk, weigh the trade. Merge sort is asymptotically far faster, but it needs $\Theta(n)$ *extra* memory to hold the merged output, whereas insertion sort sorts in place with $\Theta(1)$ extra space. And for tiny arrays, insertion sort's small constant actually wins. So "which sort is best?" has no single answer — it depends on n, on available memory, and on whether the data is nearly sorted already. When is each one the right tool? Good algorithm design is exactly this kind of judgment.

2.10 What to carry forward

Pause

Before the exercises, here is a summary.

- **The sorting problem** demands two things at once: a sorted output that is also a permutation of the input.
- **Insertion sort** grows a sorted region one element at a time, sliding each new key leftward into place — the incremental design stance.
- **Loop invariants** prove correctness via initialization, maintenance, and termination. It is induction, plus a deliberate payoff at the end.
- **Running time** is counted in primitive steps as a function of input size n; insertion sort is $\Theta(n)$ at best, $\Theta(n^2)$ on average and at worst.
- **Order of growth** discards constants and lower-order terms to expose how cost scales — the one feature that survives changing machines.
- **Divide and conquer**, previewed through merge sort, breaks $\Theta(n^2)$ down to $\Theta(n \log n)$, a difference that dwarfs any constant factor.

In this chapter we casually dropped notations like $\Theta(n)$ and $\Theta(n^2)$ to capture how runtimes scale. But

what exactly do those Greek letters mean, and how do we prove such bounds rigorously? Chapter 3 builds the formal language of asymptotic notation from the ground up.

2.11 Exercises

Exercise 2.1 (Warm-up trace)**.** Trace INSERTION-SORT on $A = \langle 31, 41, 59, 26, 41, 58 \rangle$, showing the array after each pass of the outer loop. There are two 41s; confirm from your trace that the original (earlier) 41 stays before the later one, illustrating stability.

Exercise 2.2 (Counting shoves)**.** For the array in the previous exercise, write down t_j (the number of **while** tests) for each j from 2 to 6, and use your table to state the total number of element shifts performed.

Exercise 2.3 (Best and worst by construction)**.** Give a 6-element array on which insertion sort performs the *fewest* possible shifts, and another on which it performs the *most*. State the shift count in each case, and connect them to the $\Theta(n)$ and $\Theta(n^2)$ analyses.

Exercise 2.4 (Descending order)**.** Modify the pseudocode of INSERTION-SORT so that it sorts into *non-increasing* order ($A[1] \geq A[2] \geq \cdots \geq A[n]$). Change as little as possible, and state precisely which line(s) you changed and why.

Exercise 2.5 (Linear search invariant)**.** Consider searching an array $A[1 \,..\, n]$ for a value v, scanning left to right and returning the index of the first match (or NIL if absent). Write the pseudocode, then state a loop invariant and use it to argue correctness via initialization, maintenance, and termination.

Exercise 2.6 (The inner-loop invariant)**.** The maintenance step in this chapter waved its hand at the **while** loop. State a precise loop invariant for the inner **while** loop of INSERTION-SORT and verify its initialization and maintenance. (Hint: describe what is true about $A[i+1 \,..\, j]$ and about the position of the gap.)

Exercise 2.7 (Why the strict inequality)**.** Suppose someone changes the while-test from $A[i] > key$ to $A[i] \geq key$. Argue that the algorithm is still correct (sorted + permutation), but exhibit a concrete input on which it does strictly more work, and explain why it is no longer stable.

Exercise 2.8 (Sum it carefully)**.** Prove by induction that $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$. Then use it to give the *exact* number of **while**-loop tests insertion sort performs on a strictly-decreasing array of length n (not merely the asymptotic order of growth).

Exercise 2.9 (Binary insertion)**.** In the inner loop we find the insertion point by scanning left one step at a time. Since $A[1 \,..\, j-1]$ is already sorted, we could instead find the insertion point by *binary search* in $\Theta(\log j)$ comparisons. Explain why this reduces the number of *comparisons* to $\Theta(n \log n)$ but does *not* reduce the overall running time below $\Theta(n^2)$. (Hint: think about the cost of the shifting, not the comparing.)

Exercise 2.10 (Merge by hand)**.** Merge the sorted lists $\langle 1, 4, 7, 9 \rangle$ and $\langle 2, 3, 5, 8, 10 \rangle$ using the two-finger method, writing out the output after each step. Count the comparisons and confirm the count is at most (length of first + length of second) -1.

Exercise 2.11 (Recursion-tree counting)**.** Merge sort on $n = 8$ elements. Draw the recursion tree, label the work at each node ($cn, c\frac{n}{2}, \ldots$), and verify directly that the total over all nodes equals $c \cdot 8 \cdot \log_2 8$. How many levels does the tree have?

Exercise 2.12 (Crossover point, a challenge)**.** Suppose insertion sort costs exactly $\frac{1}{2}n^2$ "time units" in the worst case and merge sort costs exactly $50\,n\log_2 n$ time units (merge sort's larger constant reflects its overhead). For which values of n is insertion sort actually *faster*? Solve the inequality $\frac{1}{2}n^2 < 50\,n\log_2 n$ approximately, and explain how this justifies the "switch to insertion sort for small subarrays" trick used by real libraries.

Chapter 3

The Growth of Functions: Asymptotic Notation

In the last chapter you counted the work that insertion sort does. You found that, in the worst case, the number of basic steps came out to something like $\frac{1}{2}n^2 + \frac{5}{2}n - 3$. That is an exact count, but it carries more detail than we usually need. The constant $\frac{1}{2}$ depends on how you decided to count comparisons. The $\frac{5}{2}n$ term depends on bookkeeping you might do differently tomorrow. The -3 is an accident of where the loops start. None of those details survive a change of computer, compiler, or counting convention.

What *does* survive is the shape of the function: it grows like n^2. Double the input and the running time roughly quadruples. That property does not depend on the details, and this chapter is about the language we use to state it precisely. The language is called **asymptotic notation**, and once it clicks you will be able to read running times at a glance.

Focus

The single idea of this chapter: when comparing algorithms, we care about *how the running time grows as the input grows*, not about its exact value on small inputs. Asymptotic notation (O, Ω, Θ, and their little cousins) is the vocabulary for talking about growth rate while deliberately ignoring constant factors and low-order terms.

3.1 Why we throw away the constants

Suppose Algorithm A runs in exactly $100n$ steps and Algorithm B runs in exactly $2n^2$ steps. Which is faster? The answer is: *it depends on* n.

Let us actually compute. For tiny inputs, B wins because A pays a large fixed toll:

n	$100n$ (A)	$2n^2$ (B)
1	100	2
10	1,000	200
50	5,000	5,000
100	10,000	20,000
1000	100,000	2,000,000

Up to $n = 50$ they are neck and neck, and below that B is the better deal. But look what happens after the crossover. At $n = 1000$, A does 100,000 steps while B does two million. Push to $n = 10^6$ and A does 10^8 steps while B does 2×10^{12} — B is now twenty thousand times slower. From then on the gap only widens.

Figure 3.1 shows this. The quadratic B starts below the linear A, but the two curves cross once and only once; past the crossover n^*, B is above A and pulls away for good.

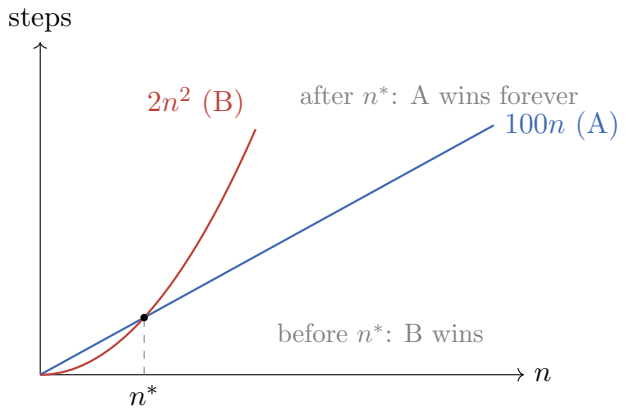


Figure 3.1: Linear A versus quadratic B. A pays a large constant toll and so loses for small n, but a single crossover at n^* flips the order permanently: the faster-growing curve eventually dominates regardless of the constants.

Intuition

The constant in front (100 versus 2) buys you an advantage on small inputs, but it is a *one-time* advantage that gets paid off. The exponent — linear versus quadratic — compounds. Asymptotic analysis is the decision to bet on the exponent, because for any fixed constant there is some input size past which the slower-growing function wins and never looks back. We care about "eventually and forever," not "on my laptop today with $n = 12$."

There is a second reason to drop constants, and it is humbler: *we usually do not know them.* The exact constant depends on the machine's clock, the cache, the compiler's cleverness, whether the data sits in registers. If we promised a precise step count we would be lying about precision we do not have. By contrast, the growth rate — n versus n^2 versus $\log n$ — is a property of the *algorithm*, not the machine. It is the part worth naming.

Why this matters

Working engineers use exactly this reasoning to triage. When a service slows to a crawl as data grows, the first question is never "what is the constant" but "what is the growth rate." An $O(n^2)$ routine that was invisible at a thousand records will melt the server at a million; an $O(n \log n)$ routine will cope. Choosing the right growth class decides whether a feature scales or has to be rewritten later. The notation in this chapter is the shorthand every performance discussion is conducted in.

3.2 The mental picture before the symbols

Before any Greek letters, fix the picture. We are comparing two functions of n, say your running time $T(n)$ and some clean reference function $g(n)$ like n^2 or $n \log n$. We ask one of three questions:

- Does T grow *no faster than* g? (An **upper** bound: O.)
- Does T grow *no slower than* g? (A **lower** bound: Ω.)
- Does T grow *at the same rate* as g? (A **tight** bound: Θ.)

Two ideas make these precise, and they recur in every single definition below, so let me name them now.

Idea one: a fudge constant. Because we refuse to care about constant factors, we are allowed to multiply g by any positive constant we like to make it fit. So "T grows no faster than g" really means "$T(n) \leq c\,g(n)$ for some constant c we get to choose." The c absorbs all the machine-dependent grime.

Idea two: eventually. Because small inputs are noise, we only require the inequality to hold once n is large enough — past some threshold n_0. Below n_0 the functions can cross, wobble, do whatever they like. We care about the tail.

Focus

Every asymptotic statement is of the form: *"there exist constants (a fudge factor c and a threshold n_0) such that for all $n \geq n_0$, some inequality between $T(n)$ and $g(n)$ holds."* Memorize that skeleton. The five notations differ only in *which* inequality and whether the constant is "some" or "every."

3.3 Big-O: the upper bound

We start with the one you will use most. Informally, $T(n) = O(g(n))$ means g is a *ceiling* on T's growth: past some point, T never rises above a constant multiple of g.

Formal

Definition 3.1 (Big-O)**.** Let $g : \mathbb{N} \to \mathbb{R}_{\geq 0}$. We write

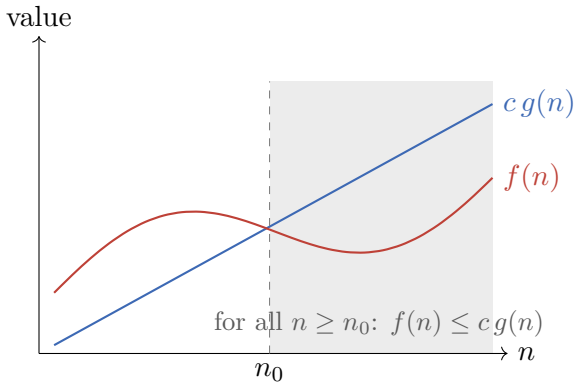
$$O\big(g(n)\big) = \big\{\, f(n) : \exists\, c > 0,\ n_0 > 0 \text{ such that } 0 \leq f(n) \leq c\, g(n) \text{ for all } n \geq n_0 \,\big\}.$$

We say "$f(n)$ is $O(g(n))$" or write $f(n) = O(g(n))$ to mean $f \in O(g(n))$.

Common trap

Strictly speaking $O(g(n))$ is a *set* of functions, so the cleanest reading of $f(n) = O(g(n))$ is "f is a member of the set $O(g(n))$." The equals sign here is a long-standing abuse of notation — it is *not* symmetric. You may write $2n^2 = O(n^2)$, but never $O(n^2) = 2n^2$. Read "=" in asymptotic statements as the word "is," and only ever from left to right. We follow CLRS's convention and keep the equals sign, but keep this one-way reading in your head.

Let us draw it. The claim "$f(n) = O(g(n))$" says that once we scale g up by the right constant c, the curve $c\,g(n)$ rises above $f(n)$ and stays there.



To the left of n_0, f is even allowed to poke above $c\,g$. We do not care. To the right — the shaded region, the tail that matters — f is trapped underneath.

3.3.1 Proving a Big-O bound: find c and n_0

A Big-O proof comes down to finding two numbers, c and n_0, that make the definition true. You do not need the *best* constants, just *some* that work.

Example 3.1. *Claim:* $\frac{1}{2}n^2 + \frac{5}{2}n - 3 = O(n^2)$ — our insertion-sort count is quadratic.

Find c and n_0. We want $\frac{1}{2}n^2 + \frac{5}{2}n - 3 \leq c\,n^2$ for all large n. A clean trick: when $n \geq 1$, every term is bounded by a multiple of the *largest* power, n^2, because $n \leq n^2$ and $1 \leq n^2$. So for $n \geq 1$,

$$\tfrac{1}{2}n^2 + \tfrac{5}{2}n - 3 \;\leq\; \tfrac{1}{2}n^2 + \tfrac{5}{2}n \;\leq\; \tfrac{1}{2}n^2 + \tfrac{5}{2}n^2 \;=\; 3n^2.$$

(We dropped the -3 because subtracting only helps an upper bound, then replaced $\frac{5}{2}n$ by the larger $\frac{5}{2}n^2$.) So $c = 3$ and $n_0 = 1$ work. Done. ■

Notice the strategy: *keep the dominant term, and bound every smaller term by it.* That single move proves almost every elementary Big-O bound you will ever need.

Example 3.2. *Claim:* $n^3 \neq O(n^2)$ — a cubic is genuinely not quadratic.

Why no constants can work. Suppose, for contradiction, that $n^3 \leq c\,n^2$ for all $n \geq n_0$. Divide both sides by n^2 (positive): $n \leq c$ for all $n \geq n_0$. But c is a fixed number and n marches to infinity, so as soon as $n > c$ the inequality breaks. No single c can cap n. Contradiction. ■

Walking question

Here is one to chew on while walking. Is $2^{n+1} = O(2^n)$? Is $2^{2n} = O(2^n)$? One of these is true and one is false. Try to write each as a factor times 2^n and check whether that factor is a fixed constant or grows with n. (Hint: $2^{n+1} = 2 \cdot 2^n$, but $2^{2n} = (2^n)^2$.)

3.4 Big-Omega: the lower bound

O gives a ceiling. **Big-Omega**, written Ω, gives a *floor*: it says T grows *at least* as fast as g. Flip the inequality in the definition of O and you have it.

Formal

Definition 3.2 (Big-Omega)**.**

$$\Omega\big(g(n)\big) = \big\{\, f(n) : \exists\, c > 0,\ n_0 > 0 \text{ such that } 0 \leq c\,g(n) \leq f(n) \text{ for all } n \geq n_0 \,\big\}.$$

Where O traps f *under* $c\,g$, Ω traps f *above* $c\,g$. O is for "no worse than"; Ω is for "no better than." A guarantee that an algorithm is *slow* (a lower bound on running time) is an Ω statement; a promise that it is *fast* (an upper bound) is an O statement.

Example 3.3. *Claim:* $\frac{1}{2}n^2 + \frac{5}{2}n - 3 = \Omega(n^2)$.

We need $c\,n^2 \leq \frac{1}{2}n^2 + \frac{5}{2}n - 3$ eventually. The $+\frac{5}{2}n$ only helps us (it makes the right side bigger), so it is enough to handle the $\frac{1}{2}n^2 - 3$ part. For $n \geq 4$ we have $\frac{1}{4}n^2 \geq \frac{1}{4} \cdot 16 = 4 \geq 3$, so $\frac{1}{4}n^2$ alone can soak up the -3. Thus for $n \geq 4$,

$$\tfrac{1}{2}n^2 + \tfrac{5}{2}n - 3 \;\geq\; \tfrac{1}{2}n^2 - 3 \;\geq\; \tfrac{1}{2}n^2 - \tfrac{1}{4}n^2 = \tfrac{1}{4}n^2.$$

Thus $c = \frac{1}{4}$, $n_0 = 4$ work. ■

The pattern mirrors Big-O: for a *lower* bound, *keep the dominant term and choose n_0 large enough that a fraction of the dominant term outweighs any subtracted lower-order terms.*

Common trap

Ω does *not* mean "the worst case." People conflate them constantly. "Worst case" picks which *input* to analyze; Ω and O describe how a function *grows*. You can put an O bound on the worst case ("never slower than …") and an Ω bound on the best case ("never faster than …"). Input choice and bound direction are two independent knobs. Mixing them up produces sentences that sound rigorous and mean nothing.

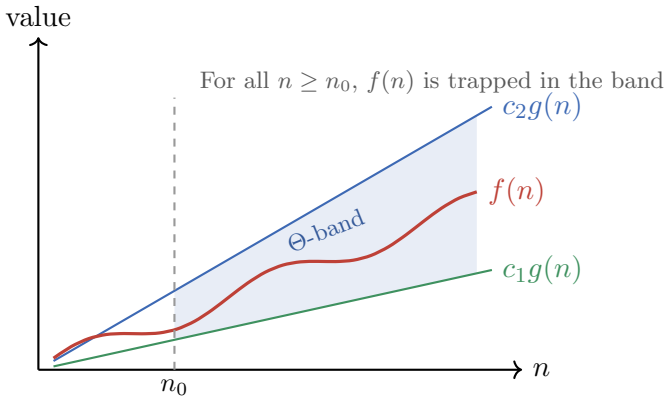
3.5 Big-Theta: the tight bound

$T(n) = \Theta(g(n))$ means T grows at the *same rate* as g — sandwiched between two constant multiples of g from above and below. It combines O and Ω into a single two-sided bound.

Formal

Definition 3.3 (Big-Theta)**.**

$$\Theta\big(g(n)\big) = \big\{\, f(n) : \exists\, c_1 > 0,\ c_2 > 0,\ n_0 > 0 \text{ s.t. } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0 \,\big\}.$$



As the picture shows, past n_0 the curve $f(n)$ is pinned between the two straight lines $c_1 g(n)$ and $c_2 g(n)$. It cannot escape upward (that is the O half) and cannot escape downward (that is the Ω half). The following theorem makes the "$\Theta = O$ and Ω" slogan official.

Formal

Theorem 3.1 (Theta as the conjunction of O and Ω)**.** *For any functions f and g,*

$$f(n) = \Theta(g(n)) \quad \textit{if and only if} \quad f(n) = O(g(n)) \textit{ and } f(n) = \Omega(g(n)).$$

Formal

Proof. ($\Rightarrow$) If $f = \Theta(g)$ then there are c_1, c_2, n_0 with $c_1 g(n) \le f(n) \le c_2 g(n)$ for $n \ge n_0$. The right inequality alone, with $c = c_2$, witnesses $f = O(g)$; the left inequality alone, with $c = c_1$, witnesses $f = \Omega(g)$.
($\Leftarrow$) Suppose $f = O(g)$ via (c_2, n_2) and $f = \Omega(g)$ via (c_1, n_1). Take $n_0 = \max(n_1, n_2)$. For every $n \ge n_0$ *both* witnessing inequalities hold simultaneously, so $c_1 g(n) \le f(n) \le c_2 g(n)$, which is exactly the Θ condition with constants c_1, c_2 and threshold n_0. ■

Combining Examples 3.1 and 3.3 through this theorem, we conclude $\frac{1}{2}n^2 + \frac{5}{2}n - 3 = \Theta(n^2)$. Insertion sort's worst case is *tightly* quadratic — bounded both from above and from below by constant multiples of n^2.

3.5.1 A theorem you will reuse: polynomials are Theta of their top term

The two examples generalize into a foundational fact. It justifies the move you have already been itching to make: just read off the highest power.

Formal

Theorem 3.2 (Degree dominates)**.** *Let* $p(n) = a_d n^d + a_{d-1} n^{d-1} + \cdots + a_1 n + a_0$ *be a polynomial of degree* d *with leading coefficient* $a_d > 0$*. Then* $p(n) = \Theta(n^d)$*.*

Formal

Proof sketch. **Upper bound.** For $n \geq 1$ each $n^k \leq n^d$, so

$$p(n) \leq \big(\,|a_d| + |a_{d-1}| + \cdots + |a_0|\,\big) n^d =: c_2\, n^d,$$

giving $p = O(n^d)$. **Lower bound.** Factor out n^d: $p(n) = n^d\big(a_d + a_{d-1}/n + \cdots + a_0/n^d\big)$. The parenthesized factor tends to $a_d > 0$ as $n \to \infty$, so for all n past some n_0 it exceeds $a_d/2 > 0$. Hence $p(n) \geq (a_d/2)\, n^d$, giving $p = \Omega(n^d)$. By Theorem 3.1, $p = \Theta(n^d)$. ■

Focus

Two rules you will use a hundred times. *Drop low-order terms* (they are out-grown by the leading term) and *drop constant factors* (they are absorbed by c). So $7n^3 - 100n^2 + 4n + 9999$ is just $\Theta(n^3)$, and $\frac{n \log n}{2} + 30n$ is just $\Theta(n \log n)$. The notation exists precisely so you may make these two moves without apology.

3.6 The little ones: little-o and little-omega

O and Ω are loose — they allow f and g to grow at the *same* rate. Indeed $n^2 = O(n^2)$. The *little* versions are strict: they insist that one function grows *strictly slower* (little-o) or *strictly faster* (little-omega) than the other, with room to spare.

The cleanest way to feel the difference: in O, *some* constant c works. In little-o, the bound must hold for *every* positive constant, no matter how tiny.

Formal

Definition 3.4 (little-o)**.** $f(n) = o(g(n))$ if for *every* constant $c > 0$ there exists $n_0 > 0$ such that $0 \leq f(n) < c\, g(n)$ for all $n \geq n_0$.

Definition 3.5 (little-omega)**.** $f(n) = \omega(g(n))$ if for *every* constant $c > 0$ there exists $n_0 > 0$ such that $0 \leq c\, g(n) < f(n)$ for all $n \geq n_0$.

That "for every c" is the decisive condition. Take $f(n) = 2n$ and $g(n) = n^2$. Is $2n = o(n^2)$? Pick *any* c, say $c = 0.001$. We need $2n < 0.001\, n^2$, i.e. $2 < 0.001\, n$, i.e. $n > 2000$. So $n_0 = 2001$ works for $c = 0.001$. Try $c = 10^{-9}$ and you would just need a bigger n_0 — but one always exists. The smaller the c, the longer you wait, but you always eventually win. That is little-o.

By contrast $2n^2 = O(n^2)$ but $2n^2 \neq o(n^2)$: choose $c = 1$ and you would need $2n^2 < n^2$, i.e. $2 < 1$, which never holds. A function is never little-o of itself.

Intuition

A clean way to remember the family, when the limit exists, is to look at the ratio $f(n)/g(n)$ as $n \to \infty$:

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0 & \Rightarrow\ f = o(g) \quad (f \text{ vanishes next to } g), \\ \text{a positive constant} & \Rightarrow\ f = \Theta(g) \quad (\text{same rate}), \\ \infty & \Rightarrow\ f = \omega(g) \quad (f \text{ runs away from } g). \end{cases}$$

Big-O is the union of the first two cases (0 or a constant); big-Ω is the union of the last two. The limit test is not the official definition (the limit need not exist), but when it exists it is the fastest way to classify a pair of functions.

Example 3.4. $\log n = o(n)$. The ratio $(\log_2 n)/n \to 0$: logarithms crawl while n strides. So any logarithm is little-o of n, hence of any positive power n^{ε}. Logs are, asymptotically, almost free.

Walking question

A genuinely surprising one to ponder on a walk: is $n^{100} = o(2^n)$? It feels like n^{100} should win — the exponent is huge. But 2^n is exponential and *every* polynomial is little-o of *every* exponential with base > 1. A clean way to see it: take $\log_2$ of the ratio, $\log_2\!\left(n^{100}/2^n\right) = 100\log_2 n - n$. The $-n$ term grows steadily while $100\log_2 n$ crawls, so the whole expression marches to $-\infty$; hence the ratio itself marches to 0. Even with the degree cranked to 100, the crossover merely waits until around $n = 1000$ and then 2^n pulls ahead and never trails again. Exponentials always win, no matter how large the polynomial's degree.

3.7 An analogy: comparing functions is comparing numbers

The five notations line up neatly against the relations you already know between plain numbers:

Functions	Numbers	Reading
$f = O(g)$	$a \leq b$	f grows no faster than g
$f = \Omega(g)$	$a \geq b$	f grows no slower than g
$f = \Theta(g)$	$a = b$	f grows at the same rate as g
$f = o(g)$	$a < b$	f grows strictly slower than g
$f = \omega(g)$	$a > b$	f grows strictly faster than g

This analogy is more than a memory aid; many properties carry straight over. *Transitivity* holds: if $f = O(g)$ and $g = O(h)$ then $f = O(h)$, just as $a \leq b$ and $b \leq c$ give $a \leq c$. *Reflexivity* holds for the big ones: $f = O(f)$, $f = \Theta(f)$, $f = \Omega(f)$ (but *not* for the little ones — nothing is o of itself, just as no number is $<$ itself). *Symmetry* holds only for Θ: $f = \Theta(g)$ iff $g = \Theta(f)$, mirroring $a = b$ iff $b = a$. And the little/big pair are *transposes*: $f = o(g)$ iff $g = \omega(f)$, exactly like $a < b$ iff $b > a$.

Common trap

The analogy has *one important caveat*: numbers obey *trichotomy* — any two numbers satisfy exactly one of $a < b$, $a = b$, $a > b$. Functions do not. Two functions can be *incomparable*: neither $f = O(g)$ nor $f = \Omega(g)$. For instance, $f(n) = n$ and $g(n) = n^{1+\sin n}$ keep overtaking each other forever, so neither bounds the other. In practice the functions you meet (polynomials, logs, exponentials) are all comparable, so the trichotomy intuition usually serves — but know that it can break.

3.8 A field guide to common growth rates

Here are the functions you will meet over and over, sorted slowest-growing (best) to fastest-growing (worst). Each line is little-o of the line below it.

Class	Name	Where it shows up
$\Theta(1)$	constant	array index, hash lookup (ideal)
$\Theta(\log n)$	logarithmic	binary search, balanced-tree height
$\Theta(n)$	linear	one scan of the input
$\Theta(n \log n)$	linearithmic	mergesort, heapsort, good comparison sorts
$\Theta(n^2)$	quadratic	insertion/selection sort, all-pairs naively
$\Theta(n^3)$	cubic	naive matrix multiply, Floyd–Warshall
$\Theta(2^n)$	exponential	brute-force subset search
$\Theta(n!)$	factorial	brute-force permutation search (TSP)

Figure 3.2 draws the most common classes on one set of axes. Once n moves away from the origin, the classes separate sharply: $\log n$ flattens almost to the floor, n and $n \log n$ rise gently, n^2 curls upward, and 2^n shoots off the top of the page almost immediately.

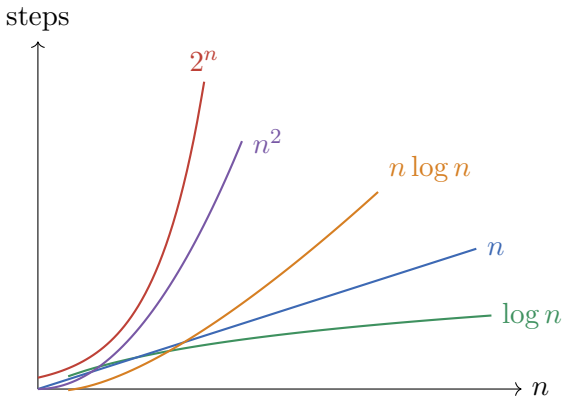


Figure 3.2: Common growth rates on shared axes. The vertical order at the right edge is the asymptotic order: slower-growing curves stay low, faster-growing curves climb away. Each curve is eventually little-o of every curve above it.

The gap between polynomial growth rates (everything down to n^3) and exponential ones is the single most important divide in algorithm design; Chapter 33 on NP-completeness studies which problems

admit polynomial-time algorithms at all. To see how quickly that gap widens, convert step counts into real time on a machine executing 10^9 basic steps per second:

n	$n \log_2 n$	n^2	2^n
10	33 ns	100 ns	1024 ns
20	86 ns	400 ns	1.0 ms
50	282 ns	2.5 µs	13 days
100	664 ns	10 µs	4×10^{13} yr
10^6	20 ms	17 min	— (heat death)

At $n = 100$, $n \log n$ finishes before you blink and 2^n outlasts the universe by many orders of magnitude. This is why growth rate, rather than the constant, is what matters most.

Why this matters

Logarithms deserve special attention. Because $\log n$ grows so slowly, an $O(\log n)$ operation is nearly constant for any input you will ever see: $\log_2$ of a billion is about 30, and $\log_2$ of the number of atoms in the observable universe is under 270. When you turn an $O(n)$ search into an $O(\log n)$ one — the leap from scanning a list to binary-searching a sorted array, or to walking a balanced tree — you can make the data a billion times bigger at the cost of only about thirty extra steps. Several data structures in later chapters exist to achieve exactly that logarithmic cost.

History and connection

Asymptotic notation did not originate in computer science but in nineteenth-century number theory. In 1894, German mathematician Paul Bachmann published *Die Analytische Zahlentheorie* in Leipzig, introducing the symbol O (from the German *Ordnung*, meaning order) to suppress messy error terms when estimating the distribution of prime numbers. In 1909, Edmund Landau popularized both Big-O and Little-o in his treatise at Göttingen, providing a clean shorthand for identities like $\pi(x) = \frac{x}{\ln x} + O\left(\frac{x}{\ln^2 x}\right)$. But when computer scientists adopted Landau's notation in the 1960s, they often used it loosely: researchers routinely wrote $O(n^2)$ when they meant that an algorithm's running time was *exactly* quadratic, blurring upper bounds with tight bounds. In 1976, Donald Knuth published a well-known letter in ACM SIGACT News, *Big Omicron and Big Omega and Big Theta for Computer Science*. Knuth advocated the Greek letters Ω (for lower bounds) and Θ (for tight two-sided bounds), establishing the notation we use today.

The most common misconception in asymptotic notation is the misleading equals sign in $f(n) = O(g(n))$. Mathematical equality is reflexive, symmetric, and transitive; if $=$ were a true equation, then $n = O(n^2)$ and $n^2 = O(n^2)$ would force the absurd conclusion that $n = n^2$. The equals sign is merely a historical one-way shorthand: $O(g(n))$ is really a *set of functions*, and $f(n) \in O(g(n))$ is the true relation. A companion trap is using O as a badge of exact performance. It is mathematically true that merge sort is $O(n^{100})$, but calling it that hides whether the algorithm could be faster. Only Θ pins the function between an upper and a lower bound.

Asymptotic notation provided a machine-independent way to rank algorithms, enabling the classification of problems into complexity classes like **P** and **NP** without drowning in CPU

clock speeds. It has limits, though: asymptotics is deliberately blind to constant factors and memory hierarchies. An algorithm requiring $10^{50}n$ operations is $\Theta(n)$, while one requiring $3n^2$ is $\Theta(n^2)$; for any input human beings will ever process, the quadratic algorithm is far faster. In Chapter 4, we put this asymptotic vocabulary to work solving recursive recurrences with the Master Theorem, and in Chapter 17 we expand it across operation sequences through amortized analysis.

3.9 Manipulating asymptotics: the algebra of growth

You rarely build a bound from c and n_0 in the wild. Instead you compose small facts with a few algebraic rules. Here are the ones that do the daily work; each follows directly from the definitions, and proving one or two yourself is the best exercise of the chapter.

Formal

Lemma 3.1 (Working rules for O)**.** *Let $f_1 = O(g_1)$ and $f_2 = O(g_2)$. Then:*

1. ***Sum / max:*** $f_1 + f_2 = O\big(\max(g_1, g_2)\big) = O(g_1 + g_2)$.
2. ***Product:*** $f_1 \cdot f_2 = O(g_1 \cdot g_2)$.
3. ***Scaling:*** *for any constant $a > 0$,* $a \cdot f_1 = O(g_1)$.
4. ***Transitivity:*** *if also $g_1 = O(h)$, then $f_1 = O(h)$.*

The same four rules hold verbatim with O replaced by Ω or by Θ.

Rule 1 is the one beginners trip over, so let us prove it and see why "the bigger one wins."

Formal

Proof of the sum rule. We are given $f_1(n) \le c_1 g_1(n)$ for $n \ge n_1$ and $f_2(n) \le c_2 g_2(n)$ for $n \ge n_2$. Let $n_0 = \max(n_1, n_2)$ and $c = c_1 + c_2$. For $n \ge n_0$ both bounds hold, so

$$f_1(n) + f_2(n) \;\le\; c_1 g_1(n) + c_2 g_2(n) \;\le\; (c_1 + c_2)\max\big(g_1(n), g_2(n)\big) = c \cdot \max(g_1, g_2)(n).$$

Thus $f_1 + f_2 = O(\max(g_1, g_2))$. ■

Focus

The sum rule is why you keep only the dominant term. A program that does an $O(n)$ phase, then an $O(n^2)$ phase, then an $O(n \log n)$ phase runs in $O(n + n^2 + n \log n) = O(n^2)$ — the slowest phase swallows the rest. When you read a running time, your eye should jump straight to the heaviest term and ignore the others.

Example 3.5. A doubly nested loop where the inner loop's length grows with the outer index:

$$\sum_{i=1}^{n} \sum_{j=1}^{i} 1 \;=\; \sum_{i=1}^{n} i \;=\; \frac{n(n+1)}{2} \;=\; \Theta(n^2).$$

This is the cost shape of insertion sort and of many "compare each pair" routines: the inner work is

$1+2+\cdots+n$, a triangle of area $\approx n^2/2$, which the notation cleanly reports as $\Theta(n^2)$. (The appendix on summations derives $\sum i = n(n+1)/2$ from scratch if you want the gory details.)

Figure 3.3 makes the "triangle" literal: stack one dot for the inner work at $i = 1$, two dots at $i = 2$, and so on. The dots fill half of an $n \times n$ square, so their count is about $n^2/2$ — a constant ($\frac{1}{2}$) times n^2, which is exactly what $\Theta(n^2)$ asserts.

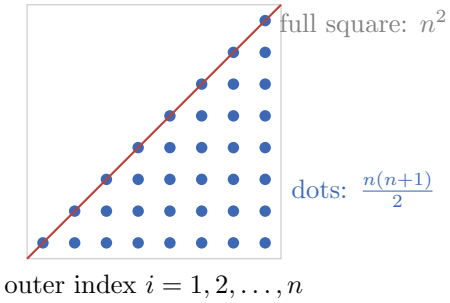


Figure 3.3: The work of nested loops as a triangle. Column i holds i units of inner work; the dots fill just over half the $n \times n$ square, so the total is $\frac{n(n+1)}{2} = \Theta(n^2)$.

Algorithm lab

Trace the growth yourself. Take the function $T(n) = 3n^2 + 50n + 200$.

1. Build a table of $T(n)$, $3n^2$, and the ratio $T(n)/n^2$ for $n = 1, 10, 100, 1000$. Watch the ratio slide toward 3.
2. That limiting ratio of 3 is your c_1/c_2 neighborhood: find a single c_2 with $T(n) \le c_2 n^2$ for all $n \ge 1$ (try $c_2 = 253$ bluntly, then sharpen it). Then find c_1 and n_0 with $c_1 n^2 \le T(n)$ (try $c_1 = 3$).
3. Conclude $T(n) = \Theta(n^2)$ by exhibiting your c_1, c_2, n_0. Notice how the constants 50 and 200 never appear in the answer — they only pushed n_0 a little to the right.

Finding such witnesses is how you turn a messy formula into a single asymptotic bound.

3.10 Putting it to work: bounding an algorithm directly

To finish, let us read a running time straight off pseudocode, the way you will for the rest of the book. Here is a routine that, given an array A of length n, checks whether it contains any duplicated value by brute force.

Algorithm 6: HASDUPLICATE(A, n)

Input: An array $A[1 \ldots n]$ of n values.
Output: TRUE if some value appears twice, else FALSE.

```
1 for i ← 1 to n − 1 do
2     for j ← i + 1 to n do
3         if A[i] = A[j] then
4             return TRUE;
5 return FALSE;
```

Counting the work. The body of the inner loop (the comparison) is the basic operation. For a fixed i, the inner loop runs j from $i+1$ to n, that is $n-i$ times. Summing over all outer iterations,

$$\sum_{i=1}^{n-1}(n-i) \;=\; (n-1)+(n-2)+\cdots+1 \;=\; \frac{(n-1)n}{2}.$$

By Theorem 3.2 this is $\Theta(n^2)$ — *in the worst case*, when no early RETURN fires (no duplicate, or the only duplicate is the very last pair). So the worst-case running time is $\Theta(n^2)$.

What about best case? If $A[1] = A[2]$, the routine returns after a single comparison: best-case cost is $\Theta(1)$. So we can say the running time is $O(n^2)$ (it is never worse than quadratic) and $\Omega(1)$ (it always does at least constant work) — but we *cannot* wrap the whole thing in a single Θ, because best and worst cases have different growth rates. Many algorithms sit in exactly this position, and the notation lets us say as much as is true and no more.

Common trap

"Θ of the worst case" and "Θ of the running time" are different claims. HASDUPLICATE's *worst case* is $\Theta(n^2)$ — a tight statement about one specific input family. Its *running time* overall is only $O(n^2)$ and $\Omega(1)$, because the cost genuinely varies with the input. Reserve a single Θ for when best and worst cases share a growth rate (as in mergesort, always $\Theta(n \log n)$). When they differ, say O for the ceiling and Ω for the floor, and resist the urge to collapse them.

Cost summary

The five notations at a glance.

Notation	Meaning	Number analogue
$f = O(g)$	$\le c\,g$ eventually, some c	$\le$
$f = \Omega(g)$	$\ge c\,g$ eventually, some c	$\ge$
$f = \Theta(g)$	$c_1 g \le f \le c_2 g$ eventually	$=$
$f = o(g)$	$< c\,g$ eventually, *every* c	$<$
$f = \omega(g)$	$> c\,g$ eventually, *every* c	$>$

Every bound is "$\exists$ constants, $\forall\, n \ge n_0$, inequality." For polynomials: Θ of the top power. For sums: keep the dominant term. For products: multiply the bounds. Logs $\prec$ powers $\prec$ exponentials $\prec$ factorials, each gap a little-o.

With our asymptotic vocabulary in hand, we are ready to analyze algorithms that call themselves. In Chapter 4, we turn to the design paradigm of *divide and conquer*, learning how to formulate recurrences for divide-and-conquer strategies and solve them with the Master Theorem.

3.11 Exercises

Exercise 3.1. For each pair, state whether $f = O(g)$, $f = \Omega(g)$, both (i.e. Θ), and whether either little version holds: (a) $f = 5n$, $g = n^2$; (b) $f = n^2 + n$, $g = 100n^2$; (c) $f = 2^n$, $g = 3^n$; (d) $f = \log_2 n$, $g = \log_{10} n$; (e) $f = n$, $g = n \log n$.

Exercise 3.2. Prove that $3n^2 + 10n + 7 = O(n^2)$ by exhibiting explicit constants c and n_0 and verifying the inequality for all $n \geq n_0$. (Any valid pair earns full marks; aim for small, clean numbers.)

Exercise 3.3. Prove that $\frac{n^2}{2} - 3n = \Theta(n^2)$. You must supply *both* an upper bound (c_2, n_0) and a lower bound (c_1, n_0); take care that the $-3n$ does not sink the lower bound.

Exercise 3.4. Show that $n^2 \neq O(n)$ by the contradiction method of Example 3.2: assume $n^2 \leq cn$ for $n \geq n_0$ and derive an impossibility.

Exercise 3.5. Using only the definition of O (constants c and n_0), prove the transitivity rule: if $f = O(g)$ and $g = O(h)$, then $f = O(h)$. Be explicit about how you combine the two sets of witnessing constants.

Exercise 3.6. Prove that $f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$ (the symmetry of Θ). Then give a concrete f, g showing that $f = O(g)$ does *not* imply $g = O(f)$, so O is *not* symmetric.

Exercise 3.7. Use the limit characterization to decide each: (a) is $n \log n = o(n^2)$? (b) is $\sqrt{n} = o(n)$? (c) is $n = o(n + \sqrt{n})$? For each, compute $\lim_{n\to\infty} f(n)/g(n)$ and read off the answer.

Exercise 3.8. A triple-nested loop runs i from 1 to n, j from 1 to i, and k from 1 to j, doing constant work in the innermost body. Write the total work as a double sum, evaluate it, and give the tight Θ bound. (You may use $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$ from the summations appendix.)

Exercise 3.9. *(Challenge.)* Prove that $n^k = o(2^n)$ for every fixed positive integer k. *Hint:* consider $\log_2$ of the ratio, $\log_2\big(n^k/2^n\big) = k \log_2 n - n$, and argue that this tends to $-\infty$, so the ratio tends to 0.

Exercise 3.10. *(Challenge.)* Give two functions $f, g : \mathbb{N} \to \mathbb{R}_{\geq 0}$ such that *neither* $f = O(g)$ *nor* $g = O(f)$ holds, and explain in one or two sentences why each O statement fails. (This shows asymptotic comparison is not a total order — trichotomy can fail.)

Exercise 3.11. *(Synthesis.)* Insertion sort runs in $\Theta(n^2)$ worst case; mergesort runs in $\Theta(n \log n)$ always. Suppose on your machine insertion sort costs exactly $2n^2$ nanoseconds and mergesort costs exactly $50n \log_2 n$ nanoseconds. Find, by hand or by a small table, the approximate input size n^* beyond which mergesort wins, and explain why the existence of such a crossover is guaranteed by the asymptotics no matter how the two constants (2 and 50) had been chosen.

Chapter 4

Divide and Conquer, Recurrences, and the Master Theorem

In Chapter 1, binary search halved an array at each probe. In Chapter 2, merge sort split an array into two halves and merged them in linear time. Both rely on the same core pattern: *cut a problem into smaller copies of itself, solve those, and stitch the answers together.* This is **divide and conquer**.

In this chapter, we turn that recursive instinct into a systematic design technique, learning how to express recursive running times as *recurrences* and solve them cleanly with the *master theorem*.

Focus

By the end of this chapter you should be able to: (1) recognize when a problem splits into independent subproblems; (2) write down a recurrence $T(n)$ that describes the running time of a recursive algorithm; (3) solve that recurrence three different ways — by guessing and proving (substitution), by drawing a recursion tree, and by plugging into the master theorem; and (4) read the answer the master theorem gives and *feel* why it is true.

History and connection

In the autumn of 1960 at Moscow State University, the mathematician Andrei Kolmogorov convened a seminar on cybernetics and put forward his famous n^2 conjecture: that multiplying two n-digit numbers inherently demands $\Omega(n^2)$ bit operations. For millennia, grade-school multiplication had been the standard method; it seemed self-evident that every digit of one number must touch every digit of the other. Sitting in the audience was a 23-year-old student named Anatoly Karatsuba. Within a week, Karatsuba discovered an algebraic identity: by expanding the product of two two-digit blocks with a clever grouping of terms, he could compute the product using only three half-sized multiplications instead of the expected four. Saving that one multiplication changed the recurrence to $T(n) = 3T(n/2) + O(n)$, reducing the cost from quadratic to $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$. Kolmogorov immediately canceled the remaining lectures on the conjecture, wrote up Karatsuba's result himself, and submitted it for publication.

Karatsuba's result showed that splitting a problem into smaller pieces pays off asymptotically only when it also cuts the number of recursive branches: four half-size products give back

$\Theta(n^2)$, while three give $\Theta(n^{1.585})$. In 1980, Jon Bentley, Dorothea Haken, and James B. Saxe unified the solution of such recursive equations into the *Master Theorem*, turning what once required page-long induction proofs into a clear comparison between recursive branching and recombination work.
Divide and conquer also has practical limits. Recursive calls create call-stack overhead and temporary buffers; on small inputs, the overhead dominates, which is why real systems switch back to elementary loops when subarrays drop below a dozen elements. Later in this chapter, we will see how this balance plays out in merge sort and Volker Strassen's 1969 sub-cubic matrix multiplication.

4.1 The shape of a divide-and-conquer algorithm

Every divide-and-conquer algorithm has the same three steps. Once you know them, most of these algorithms look like variations of one another.

Intuition

Divide. Break the problem into a few smaller subproblems that are *the same kind of problem*, just smaller.
Conquer. Solve each subproblem *recursively* — call the very same procedure on the smaller input. Once the subproblem is tiny enough (often size 1), solve it directly; that tiny case is the *base case*.
Combine. Glue the subproblem answers together into an answer for the original problem.

The word "recursively" in the conquer step carries most of the weight. We do not write three separate algorithms for the three sizes of the problem; we write *one* algorithm that calls itself. Each call shrinks the input, and because inputs cannot shrink forever, the chain of calls must eventually bottom out at the base case. That is the same logic as mathematical induction, and for good reason: we prove a divide-and-conquer algorithm correct by induction on the input size.

Focus

The two questions you must answer for *any* divide-and-conquer design: (1) *What is the base case, and why is its solution directly verifiable?* (2) *If the recursive calls return correct answers on the smaller inputs, does the combine step produce a correct answer on the whole input?* If both are "yes," induction guarantees the algorithm is correct on every size.

4.1.1 A first, familiar example: binary search

Here is the binary search of Chapter 1 expressed in the language of divide and conquer. The procedure looks for a value x in a *sorted* array $A[1 \ldots n]$.

Algorithm 7: BINARY-SEARCH(A, x, lo, hi)

Input: A sorted array A, a target x, and bounds $lo \le hi + 1$.
Output: An index i with $A[i] = x$, or NIL if x is absent.

```
1 if lo > hi then
2     return NIL;                                   // base case: empty range
3 mid ← ⌊(lo + hi)/2⌋;
4 if A[mid] = x then
5     return mid
6 else if A[mid] > x then
7     return BINARY-SEARCH(A, x, lo, mid − 1);      // left half
8 else
9     return BINARY-SEARCH(A, x, mid + 1, hi);      // right half
```

Divide: compare against the middle and pick a half. Conquer: search that half recursively. Combine: nothing to do — the half's answer *is* our answer. This is the friendliest kind of divide-and-conquer, where "divide" throws away work for free and "combine" is trivial. How long does it take? Each call does a constant amount of work (one comparison, some arithmetic) and then makes one call on a problem of half the size. If $T(n)$ is the time on a range of n elements,

$$T(n) = T(n/2) + c.$$

We will learn to read that line like a sentence. For now, recall our halving count from Chapter 1: about $\log_2 n$ halvings, each costing c, so $T(n) = \Theta(\log n)$.

4.2 Merge sort, revisited slowly

You may have met merge sort already. We meet it again here, more carefully, because it is the cleanest possible illustration of the paradigm and because its analysis sets up everything else.

4.2.1 The idea

Intuition

To sort a pile of n cards: if the pile is a single card, it is already sorted — done. Otherwise split the pile into two halves, sort each half (by the same method, recursively), and then *merge* the two sorted halves into one sorted pile. Merging is easy because both halves are already sorted: repeatedly take the smaller of the two top cards.

The clever part is *merge*, the combine step. Given two sorted lists, you can zip them together into one sorted list in a single pass. Keep a finger on the front of each list; compare the two fingered elements; copy the smaller one out and advance that finger. When one list empties, copy the rest of the other. Figure 4.1 shows a single merge step frozen mid-zip.

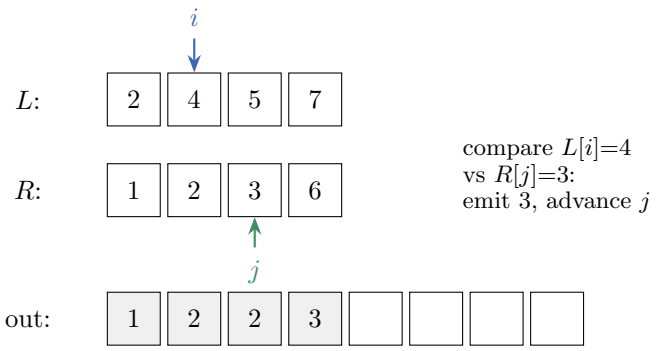


Figure 4.1: One step of MERGE. Two fingers i, j scan the sorted inputs L and R; the smaller fingered value is copied to the output and its finger advances. Here $4 > 3$, so 3 is emitted and j steps right.

Algorithm 8: MERGE(A, lo, mid, hi)

Input: $A[lo \ldots mid]$ and $A[mid + 1 \ldots hi]$ are each sorted.
Output: $A[lo \ldots hi]$ sorted, in place.

$L \leftarrow A[lo \ldots mid]$ with a $+\infty$ sentinel appended;
$R \leftarrow A[mid + 1 \ldots hi]$ with a $+\infty$ sentinel appended;
$i \leftarrow 1$; $\quad j \leftarrow 1$;
for $k \leftarrow lo$ **to** hi **do**
  if $L[i] \le R[j]$ **then**
    $A[k] \leftarrow L[i]$; $\quad i \leftarrow i + 1$
  else
    $A[k] \leftarrow R[j]$; $\quad j \leftarrow j + 1$

The $+\infty$ sentinels are a small trick so we never have to check "has this list run out?" inside the loop: once a real list is exhausted, its finger sits on $+\infty$, which loses every comparison, so the other list drains naturally.

Algorithm 9: MERGE-SORT(A, lo, hi)

Input: An array A and bounds $lo \le hi$.
Output: $A[lo \ldots hi]$ sorted in nondecreasing order.

if $lo \ge hi$ **then**
  return ; `// base case: 0 or 1 element, already sorted`
$mid \leftarrow \lfloor (lo + hi)/2 \rfloor$;
MERGE-SORT(A, lo, mid); `// conquer left`
MERGE-SORT$(A, mid + 1, hi)$; `// conquer right`
MERGE(A, lo, mid, hi); `// combine`

4.2.2 A full numeric trace

Let us sort $A = [5, 2, 4, 7, 1, 3, 2, 6]$ ($n = 8$). I will draw the recursion as a tree: splitting on the way down, merging on the way up.

Algorithm lab

Trace it yourself. Cover the answers below and run the splits in your head before peeking.
Splitting down:

- $[5,2,4,7,1,3,2,6] \to [5,2,4,7]$ and $[1,3,2,6]$
- $[5,2,4,7] \to [5,2]$ and $[4,7]$; $[1,3,2,6] \to [1,3]$ and $[2,6]$
- $[5,2] \to [5],[2]$; $[4,7] \to [4],[7]$; $[1,3] \to [1],[3]$; $[2,6] \to [2],[6]$

Merging up:

- $[5],[2] \to [2,5]$; $[4],[7] \to [4,7]$; $[1],[3] \to [1,3]$; $[2],[6] \to [2,6]$
- $[2,5],[4,7] \to [2,4,5,7]$; $[1,3],[2,6] \to [1,2,3,6]$
- $[2,4,5,7],[1,2,3,6] \to [1,2,2,3,4,5,6,7]$

Watch the last merge in slow motion. Fingers start at the fronts of $L = [2,4,5,7]$ and $R = [1,2,3,6]$.

step	compare	emit	L remaining	R remaining
1	2 vs 1	1	2, 4, 5, 7	2, 3, 6
2	2 vs 2	2	4, 5, 7	2, 3, 6
3	4 vs 2	2	4, 5, 7	3, 6
4	4 vs 3	3	4, 5, 7	6
5	4 vs 6	4	5, 7	6
6	5 vs 6	5	7	6
7	7 vs 6	6	7	—
8	(R empty)	7	—	—

Output: $[1,2,2,3,4,5,6,7]$. Notice step 2: when the two fingered elements tie, we took from L because the test is $L[i] \le R[j]$. That choice makes merge sort *stable* (equal elements keep their original relative order), a property that matters more than you would guess in real systems.

Common trap

The merge step needs *extra space* for the copies L and R — merge sort is not in place. A classic mistake is to assume you can merge two halves of one array back into that same array without a scratch buffer. You cannot do it in $O(1)$ extra space without a much fancier algorithm; the straightforward merge uses $\Theta(n)$ auxiliary memory. (We will revisit this trade-off in the cost summary.)

4.2.3 Why it is correct

Formal

Claim. MERGE-SORT(A, lo, hi) leaves $A[lo \ldots hi]$ sorted.
Proof by strong induction on $m = hi - lo + 1$, the number of elements.
Base case ($m \le 1$): a range of 0 or 1 element is sorted by definition, and the algorithm returns without touching it. True.

Inductive step ($m \geq 2$): assume the claim holds for all ranges of fewer than m elements. Since $m \geq 2$, the two recursive calls operate on ranges of sizes $\lceil m/2 \rceil$ and $\lfloor m/2 \rfloor$, both strictly less than m and both at least 1. By the inductive hypothesis, after the two calls $A[lo \ldots mid]$ and $A[mid + 1 \ldots hi]$ are each sorted. MERGE then combines two sorted ranges into one sorted range (this is the merge *lemma*; prove it as a loop invariant: "at the start of iteration k, $A[lo \ldots k-1]$ holds the $k - lo$ smallest of the merged elements, in order, and $L[i], R[j]$ are the smallest not-yet-copied elements of each list"). Hence $A[lo \ldots hi]$ is sorted. By induction the claim holds for all m. ■

That is the standard shape of a divide-and-conquer correctness proof, and you will reuse it constantly: base case obvious, recursive calls correct by hypothesis, combine step correct by a small lemma.

4.3 Counting the work: recurrences

Now the analysis. How long does MERGE-SORT take on n elements? The answer is itself recursive, because the algorithm is recursive. Let $T(n)$ denote the worst-case running time on an input of size n. Reading the code:

- If $n \leq 1$, we do a constant amount of work: $T(n) = \Theta(1)$.
- If $n > 1$, we make two recursive calls, each on half the input, costing $T(n/2)$ each. Computing *mid* is constant. MERGE on n elements runs a single loop of n iterations, so it costs $\Theta(n)$.

Putting those together gives the *recurrence*

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 1, \\ 2\,T(n/2) + \Theta(n) & \text{if } n > 1. \end{cases}$$

Definition 4.1 (Recurrence)**.** A *recurrence* is an equation that defines a function (here, the running time T) in terms of its own values on smaller inputs, together with one or more base cases. Solving a recurrence means finding a closed-form bound — usually a $\Theta(\cdot)$ bound — that does not refer to T on the right-hand side.

Focus

Two conventions we will lean on throughout, because they never change the asymptotic answer for the algorithms in this book. **(1) Floors and ceilings.** We write $T(n/2)$ even though the true sizes are $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$. **(2) Small base cases.** We treat $T(n) = \Theta(1)$ for n below some constant. Both simplifications are justified rigorously in CLRS; we lean on them so the algebra stays clean.

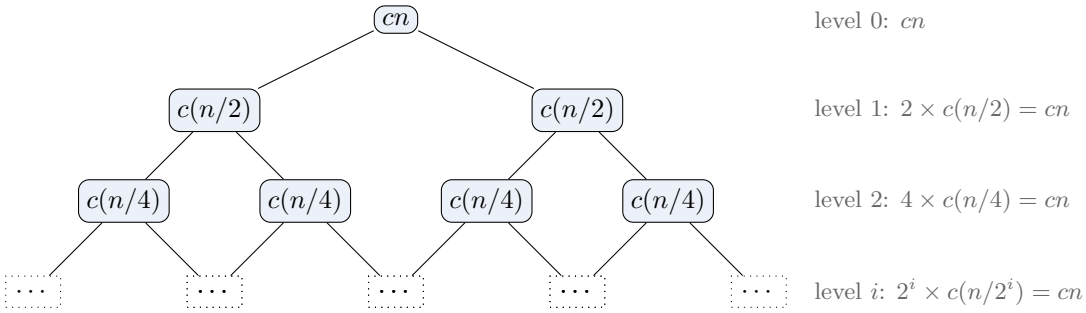
Three tools solve recurrences, and we will learn all three, because each teaches you something the others hide.

4.4 Method 1: the recursion tree (see the work)

The recursion tree turns a recurrence into a picture you can literally add up. Draw a node for the top-level call and label it with the *non-recursive* work that call does — for merge sort, that is the cn cost of merging, not the recursive calls. Then draw its children: the subproblems it spawns. Repeat until you hit base cases. Finally, sum the labels.

4.4.1 Merge sort's tree

For $T(n) = 2T(n/2) + cn$, the root does cn work and has two children of size $n/2$. Each of those does $c(n/2)$ work and has two children of size $n/4$, and so on.



Add up each *level*:

- Level 0 (root): cn.
- Level 1: two nodes of $c(n/2)$, total cn.
- Level 2: four nodes of $c(n/4)$, total cn.
- Level i: 2^i nodes of $c(n/2^i)$, total cn.

Intuition

Every level costs the same: cn. The subproblems get smaller, but there are proportionally more of them, and the two effects cancel exactly. So the only remaining question is: *how many levels are there?*

The size at level i is $n/2^i$. We reach the base case (size 1) when $n/2^i = 1$, i.e. $i = \log_2 n$. So there are $\log_2 n + 1$ levels, each costing cn, and the total is

$$T(n) = cn\,(\log_2 n + 1) = \Theta(n \log n).$$

Walking question

Take this on a walk: *why* does merge sort beat insertion sort's $\Theta(n^2)$? Insertion sort, in the worst case, does about n units of work for each of n elements — a full rectangle, $n \times n$. Merge sort does cn work on each of only $\log n$ levels — a much shorter stack of equal rows. The win is replacing the factor n by the factor $\log n$. For a million elements that is the difference between

10^{12} and roughly 2×10^7 operations: about fifty thousand times faster. Where exactly did the log come from, and what feature of the splitting produced it?

4.4.2 An unbalanced tree, to build instinct

The recursion tree is most useful when the levels do *not* all cost the same. Consider $T(n) = T(n/3) + T(2n/3) + cn$ (this shape shows up in some quicksort and selection analyses). The two children have sizes $n/3$ and $2n/3$; together they still hold all n elements, so each *full* level still sums to about cn — as long as the level is complete. But the two branches reach the bottom at different depths. The shallowest leaf is at depth $\log_3 n$ (following the /3 branch), and the deepest at depth $\log_{3/2} n$ (following the /(2/3) branch). Levels near the top all cost cn; below depth $\log_3 n$ some branches have already bottomed out, so those levels cost *less* than cn. The total is therefore at most $cn \cdot \log_{3/2} n = \Theta(n \log n)$ and at least $cn \cdot \log_3 n = \Omega(n \log n)$, pinning it to $\Theta(n \log n)$. The tree (Figure 4.2) makes the lopsidedness visible in a way a formula would not.

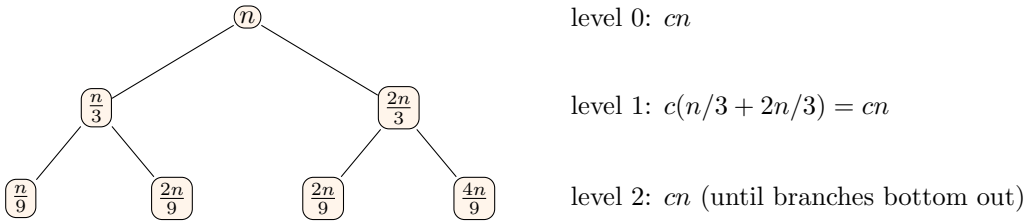


Figure 4.2: The lopsided split $T(n) = T(n/3) + T(2n/3) + cn$. Sizes are unequal, but each *full* level still sums to cn. The left (/3) branch reaches a leaf at depth $\log_3 n$; the right (/(2/3)) branch runs deeper, to $\log_{3/2} n$.

4.5 Method 2: substitution (guess, then prove)

The substitution method is exactly what it sounds like: *guess* the form of the answer, then *prove* it by induction, substituting the inductive hypothesis into the recurrence. It is the most general of the three — it can handle recurrences the master theorem cannot — but it asks you to guess well, which is a skill the recursion tree helps you develop.

4.5.1 Proving an upper bound for merge sort

Here is a rigorous proof that $T(n) = O(n \log n)$ for $T(n) = 2T(n/2) + cn$, worked out so you can see the machinery. We guess that there is a constant $d > 0$ with $T(n) \le d\, n \log_2 n$ for all $n \ge 2$, and we try to verify it by induction.

Formal

Inductive step. Assume the bound holds for all sizes below n; in particular $T(n/2) \le d\,\frac{n}{2}\log_2\frac{n}{2}$. Substitute into the recurrence:

$$\begin{aligned} T(n) &= 2\,T(n/2) + cn \\ &\le 2\Big(d\,\tfrac{n}{2}\log_2\tfrac{n}{2}\Big) + cn \\ &= d\,n\,(\log_2 n - 1) + cn \\ &= d\,n\log_2 n - dn + cn \\ &= d\,n\log_2 n - (d-c)\,n. \end{aligned}$$

As long as we choose $d \ge c$, the term $-(d-c)n \le 0$, so $T(n) \le d\,n\log_2 n$. The inductive step goes through.

Common trap

You must handle the *base case* of the *proof*, rather than assuming the algorithm's base case automatically suffices. Here the guess $d\,n\log_2 n$ is 0 at $n = 1$ (since $\log_2 1 = 0$), but $T(1) = \Theta(1) > 0$, so the bound *fails* at $n = 1$. The fix is standard: start the induction at $n = 2$ instead. Since asymptotic notation only cares about large n, we are free to ignore finitely many small cases, and we pick d large enough to cover $n = 2, 3$ directly. Forgetting to repair the base case is the single most common substitution-method error.

4.5.2 The danger of a sloppy guess

Common trap

Suppose you tried to prove $T(n) = O(n)$ for the same recurrence, guessing $T(n) \le dn$. Substituting: $T(n) \le 2(d\,\frac{n}{2}) + cn = dn + cn$. You might be tempted to write "$= O(n)$" and declare victory — but you have proved $T(n) \le dn + cn$, which is *not* $\le dn$. The constant grew. The "proof" must reproduce the *exact* guessed form with the *same* constant, and here it cannot, because the true answer $n\log n$ really is bigger than n. A guess that is too small leaks an extra additive term every level; that leak is the recurrence telling you to guess higher.

4.6 Method 3: the master theorem (the lookup table)

The recursion tree showed us something general. A huge family of divide-and-conquer recurrences has the shape

$$\boxed{T(n) = a\,T(n/b) + f(n)}$$

where $a \ge 1$ is the number of subproblems, $b > 1$ is the factor by which the input shrinks, and $f(n)$ is the cost of dividing and combining (the non-recursive work). Merge sort is $a = 2, b = 2, f(n) = cn$. Binary search is $a = 1, b = 2, f(n) = c$. The master theorem reads off the answer for almost all such recurrences without your having to draw the tree each time.

4.6.1 The exponent that decides the answer

Build the recursion tree in your head. Level i has a^i nodes, each of size n/b^i. The work at the leaves is governed by how many leaves there are. The tree bottoms out at depth $\log_b n$, where the number of leaves is

$$a^{\log_b n} = n^{\log_b a}.$$

(To see why $a^{\log_b n} = n^{\log_b a}$, take base-$b$ logarithms of both sides: $\log_b(a^{\log_b n}) = (\log_b n)(\log_b a)$, while $\log_b(n^{\log_b a}) = (\log_b a)(\log_b n)$. The two products are identical, so the expressions are equal.) That exponent, $\log_b a$, determines which case of the theorem applies. Give it a name and a comparison object:

$$n^{\log_b a} \quad \text{(the leaf/recursion cost)} \qquad \text{vs.} \qquad f(n) \quad \text{(the per-call combine cost).}$$

Intuition

The master theorem is a *tug of war* between two costs: the work piling up at the leaves, which totals about $n^{\log_b a}$, and the work done at the root, $f(n)$, which recurs in smaller amounts at every level below. Whichever cost is *polynomially bigger* wins and sets the answer. If they are the same size, every level contributes equally (as in merge sort) and you pay an extra logarithmic factor for the number of levels.

4.6.2 The statement

Theorem 4.1 (Master Theorem)**.** *Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a nonnegative function, and let $T(n) = a\,T(n/b) + f(n)$ (interpreting n/b as $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$). Let $E = \log_b a$. Then:*

Case 1 (leaves win). *If $f(n) = O(n^{E-\varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta\big(n^E\big)$.*

Case 2 (tie). *If $f(n) = \Theta(n^E)$, then $T(n) = \Theta\big(n^E \log n\big)$.*

Case 3 (root wins). *If $f(n) = \Omega(n^{E+\varepsilon})$ for some constant $\varepsilon > 0$,* and *the regularity condition $a\,f(n/b) \leq k\,f(n)$ holds for some constant $k < 1$ and all large n, then $T(n) = \Theta\big(f(n)\big)$.*

Focus

The recipe in practice:

1. Compute $E = \log_b a$. Write down n^E.
2. Compare $f(n)$ to n^E.
3. If f is polynomially *smaller* $\Rightarrow$ Case 1, answer n^E.
4. If f is the *same* order $\Rightarrow$ Case 2, answer $n^E \log n$.
5. If f is polynomially *larger* (and regular) $\Rightarrow$ Case 3, answer $f(n)$.

Common trap

"Polynomially smaller/larger" is a real requirement. Case 1 needs $f(n) = O(n^{E-\varepsilon})$ for a *fixed positive* ε — f must be smaller by a genuine factor of n^{ε}, not merely smaller by a log. There is a gap between the cases: if $f(n)$ sits just a logarithm away from n^E (for example $f(n) = n^E \log n$), *none* of the three cases applies, and you must fall back to a recursion tree or to Akra–Bazzi (Section 4.9).

4.6.3 Why each case is true, in pictures

Recall: level i contributes $a^i f(n/b^i)$, and there are $\log_b n$ levels with a leaf level of total cost $\Theta(n^E)$. Figure 4.3 draws the per-level cost for each case as a stack of bars, top of the tree at the left.

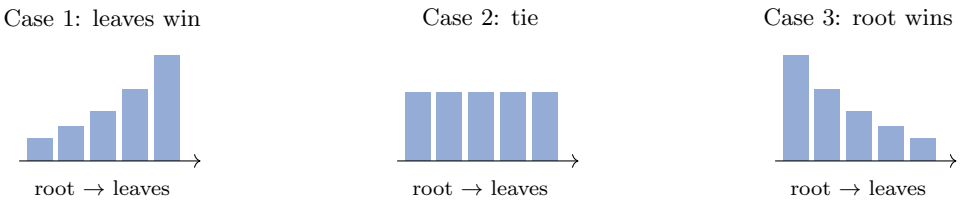


Figure 4.3: Per-level work down the recursion tree (root at left, leaves at right). Case 1: costs grow toward the leaves (a geometric series dominated by its last term, n^E). Case 2: every level costs the same (sum is $n^E \log n$). Case 3: costs shrink toward the leaves (dominated by the first term, $f(n)$).

- **Case 1.** f shrinks as you go down so slowly relative to the branching that the costs *grow* geometrically toward the leaves. A geometric series is dominated by its largest term — here the leaf level, n^E. So the total is $\Theta(n^E)$: *the leaves win.*
- **Case 2.** Every level costs the same, $\Theta(n^E)$ (this is exactly the merge-sort picture). With $\log_b n$ equal levels, the total is $\Theta(n^E \log n)$: *a tie, pay for the levels.*
- **Case 3.** f is so big at the top that the per-level costs *shrink* geometrically as you descend. A decreasing geometric series is dominated by its *first* term — the root, $f(n)$. The regularity condition is the fine print guaranteeing the series really does decrease geometrically. So the total is $\Theta(f(n))$: *the root wins.*

Why this matters

This theorem explains why so much of practical algorithm design reduces to "arrange to call yourself fewer times, or shrink the input faster, or do less combining." Each of those moves a, b, or f, and the master theorem turns the change instantly into an asymptotic verdict. Database query planners, numerical libraries (FFT, matrix multiply), and graphics geometry code are all tuned around exactly this trade-off.

4.7 Worked examples with the master theorem

The following examples drill all three cases until plugging in is automatic.

Example 4.1 (Merge sort — Case 2)**.** $T(n) = 2T(n/2)+cn$. Here $a = 2, b = 2$, so $E = \log_2 2 = 1$ and $n^E = n$. Compare $f(n) = cn = \Theta(n) = \Theta(n^E)$: a tie. Case 2 gives $T(n) = \Theta(n^E \log n) = \Theta(n \log n)$. Matches the tree.

Example 4.2 (Binary search — Case 2 with $E = 0$)**.** $T(n) = T(n/2) + c$. Here $a = 1, b = 2$, so $E = \log_2 1 = 0$ and $n^E = n^0 = 1$. Compare $f(n) = c = \Theta(1) = \Theta(n^E)$: a tie. Case 2 gives $T(n) = \Theta(n^0 \log n) = \Theta(\log n)$. This confirms the halving count from Chapter 1.

Example 4.3 (Leaves win — Case 1)**.** $T(n) = 8T(n/2) + cn^2$. Here $a = 8, b = 2$, so $E = \log_2 8 = 3$ and $n^E = n^3$. Compare $f(n) = cn^2$ to n^3: $n^2 = O(n^{3-\varepsilon})$ with, say, $\varepsilon = 1$. So f is polynomially smaller — Case 1 — and $T(n) = \Theta(n^3)$. Eight subproblems is a lot of branching; the leaves overwhelm the quadratic combine.

Example 4.4 (Root wins — Case 3)**.** $T(n) = 2T(n/2) + cn^2$. Here $E = \log_2 2 = 1$, $n^E = n$. Compare $f(n) = cn^2$ to n: $n^2 = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$. Check regularity: $af(n/b) = 2 \cdot c(n/2)^2 = \frac{cn^2}{2} = \frac{1}{2}f(n) \le kf(n)$ with $k = \frac{1}{2} < 1$. Both conditions hold — Case 3 — so $T(n) = \Theta(n^2)$. The expensive quadratic combine at the top dominates everything below it.

Example 4.5 (A gap case — master theorem does *not* apply)**.** $T(n) = 2T(n/2) + n \log n$. Here $E = 1$, $n^E = n$. Is $f(n) = n \log n$ polynomially bigger than n? No: $\log n$ grows slower than n^ε for *every* $\varepsilon > 0$, so there is no valid ε for Case 3, and f is not $\Theta(n)$ either. We are in the gap. A recursion tree (every level costs $n \log \frac{n}{2^i}$, summed over $\log n$ levels) gives $T(n) = \Theta(n \log^2 n)$ — a result the master theorem simply cannot reach.

Algorithm lab

Trace it yourself. Classify each recurrence by computing $E = \log_b a$ and comparing to f. Cover the answers.

1. $T(n) = 4T(n/2) + n$. ($E = 2$; $f = n = O(n^{2-\varepsilon})$; Case 1; $\Theta(n^2)$.)
2. $T(n) = 4T(n/2) + n^2$. ($E = 2$; tie; Case 2; $\Theta(n^2 \log n)$.)
3. $T(n) = 4T(n/2) + n^3$. ($E = 2$; $f = \Omega(n^{2+\varepsilon})$, regular; Case 3; $\Theta(n^3)$.)
4. $T(n) = 3T(n/4) + n \log n$. ($E = \log_4 3 \approx 0.79$; $n \log n = \Omega(n^{0.79+\varepsilon})$, regular; Case 3; $\Theta(n \log n)$.)
5. $T(n) = 2T(n/2) + \frac{n}{\log n}$. ($E = 1$; gap below n; master fails; tree gives $\Theta(n \log \log n)$.)

4.8 Two motivating algorithms beyond sorting

Divide and conquer extends far beyond sorting. Two famous examples show it producing genuinely surprising speedups, and both hand us nontrivial recurrences to chew on.

4.8.1 Maximum subarray

The problem. You are given an array of numbers, some positive, some negative — think daily changes in a stock price. Find the contiguous stretch (subarray) whose sum is largest. For

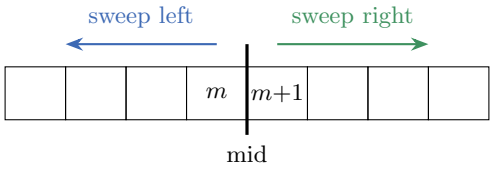
$$[13, -3, -25, 20, -3, -16, -23, 18, 20, -7, 12, -5, -22, 15, -4, 7]$$

the best stretch is $[18, 20, -7, 12]$, summing to 43.

The brute-force approach tries all $\Theta(n^2)$ subarrays — too slow. Divide and conquer does better. Split the array at the midpoint m. The maximum subarray must be one of exactly three things:

1. entirely in the *left* half $A[lo \dots m]$,
2. entirely in the *right* half $A[m+1 \dots hi]$, or
3. *crossing* the midpoint, using some of the left and some of the right.

Cases 1 and 2 are smaller copies of the same problem — solve them recursively. Case 3 is the new, combine work: a crossing subarray is forced to include $A[m]$ and $A[m+1]$, so we can find the best one by sweeping leftward from m for the best left-piece and rightward from $m+1$ for the best right-piece, in $\Theta(n)$ time. Take the best of the three.



Algorithm 10: MAX-SUBARRAY(A, lo, hi)

Input: Array A, bounds $lo \le hi$.
Output: The maximum sum of a contiguous subarray of $A[lo \dots hi]$.

```
if lo = hi then
    return A[lo];                                   // base case: one element
m ← ⌊(lo + hi)/2⌋;
left ← MAX-SUBARRAY(A, lo, m);
right ← MAX-SUBARRAY(A, m + 1, hi);
cross ← MAX-CROSSING(A, lo, m, hi);                 // Θ(n) sweep
return max(left, right, cross);
```

Two recursive calls on half the input, plus a $\Theta(n)$ crossing sweep:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

Same recurrence as merge sort. We turned an $\Theta(n^2)$ brute force into $\Theta(n \log n)$ with the same three steps.

Connection to other fields

The maximum-subarray problem has a famous twist: a clever *linear*-time dynamic-programming solution (Kadane's algorithm) beats the divide-and-conquer $\Theta(n \log n)$. We present the D&C version because it is the cleanest illustration of "the answer is left, right, or crossing." This maps to CLRS Chapter 4, Section 4.1, and you will meet Kadane-style thinking again in Chapter 15 on dynamic programming. Lesson: divide and conquer is a strong first instinct, but not always the last word.

4.8.2 Strassen's matrix multiplication

Multiplying two $n \times n$ matrices the obvious way costs $\Theta(n^3)$: each of the n^2 output entries is a dot product of length n. For decades people assumed n^3 was simply the cost of matrix multiply. In 1969 Volker Strassen showed it could be done faster, using divide and conquer.

Split each $n \times n$ matrix into four $\frac{n}{2} \times \frac{n}{2}$ blocks (Figure 4.4). The naive block formula needs *eight* multiplications of half-size blocks (plus some additions), giving

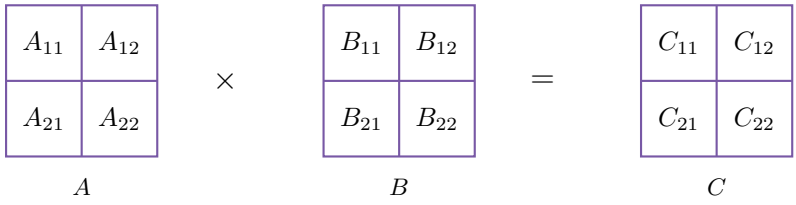


Figure 4.4: Block partition for matrix multiply. Each block is $\frac{n}{2} \times \frac{n}{2}$; e.g. $C_{11} = A_{11}B_{11} + A_{12}B_{21}$. The naive scheme computes the four C-blocks with eight block products; Strassen needs only seven.

$$T(n) = 8\,T(n/2) + \Theta(n^2).$$

Master theorem: $E = \log_2 8 = 3$, and $f(n) = n^2 = O(n^{3-\varepsilon})$, so Case 1 yields $\Theta(n^3)$ — no improvement, as expected. The bottleneck is the branching factor $a = 8$.

Strassen's insight: with a sequence of clever additions and subtractions, you can compute the four product blocks using only *seven* half-size multiplications instead of eight. That single saved multiplication changes a from 8 to 7:

$$T(n) = 7\,T(n/2) + \Theta(n^2).$$

Now $E = \log_2 7 \approx 2.807$, and $f(n) = n^2 = O(n^{2.807-\varepsilon})$, so Case 1 gives

$$T(n) = \Theta\big(n^{\log_2 7}\big) \approx \Theta(n^{2.807}).$$

Figure 4.5 visualizes why this happens by tracing Strassen's recursion tree. At each step down, the combine work per node drops by a factor of 4 (since $(n/2)^2 = n^2/4$), but the number of subproblems multiplies by 7. Thus, each level's total cost grows by a factor of $7/4 = 1.75$. The series of level costs is a geometric progression with ratio $r = 7/4 > 1$:

$$cn^2\left(1 + \frac{7}{4} + \left(\frac{7}{4}\right)^2 + \cdots + \left(\frac{7}{4}\right)^{\log_2 n}\right).$$

Any geometric sum $\sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1}$ with ratio $r > 1$ is dominated by its final term. Here, the final term is the leaf level at depth $\log_2 n$, where the $7^{\log_2 n} = n^{\log_2 7}$ base cases overwhelm all the combine work above them.

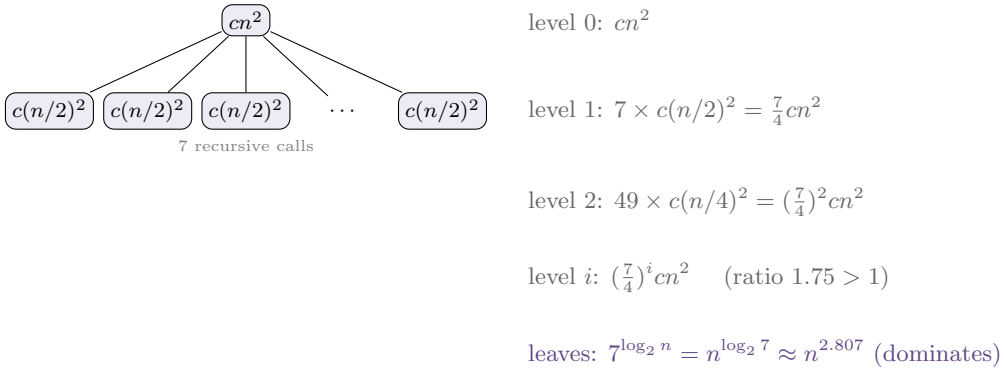


Figure 4.5: Strassen's recursion tree for $T(n) = 7T(n/2) + \Theta(n^2)$. Each level multiplies the number of calls by 7 while shrinking node cost by 4. Because $7/4 > 1$, work grows geometrically down the tree and the bottom (leaf) level dominates the runtime.

Why this matters

By trading a single block multiplication for extra additions at every level of recursion, Strassen broke the "obvious" n^3 barrier. For large matrices $n^{2.807}$ is dramatically less than n^3, and variants of this idea drive high-performance linear-algebra libraries to this day. The lesson that *reducing the branching factor a is worth a lot of constant-factor pain in f* is one of the most useful things to carry out of this chapter, and the master theorem makes the payoff easy to see.

Connection to other fields

Strassen's algorithm is CLRS Chapter 4, Section 4.2. It opened a still-active race: the exponent has since been pushed below 2.38 by Coppersmith–Winograd and successors, though those algorithms are galactic — their constants are so huge they only win on impossibly large inputs. Strassen itself, by contrast, is used in practice.

4.9 When the master theorem is not enough: Akra–Bazzi

The master theorem requires every subproblem to be the *same* size n/b. Real recurrences are not always so tidy. What about

$$T(n) = T(n/3) + T(2n/3) + cn,$$

the lopsided split we drew earlier? The subproblems have *different* sizes, so Theorem 4.1 does not apply at all.

The *Akra–Bazzi method* generalizes the master theorem to recurrences of the form

$$T(n) = \sum_{i=1}^{k} a_i\, T(n/b_i) + f(n),$$

with possibly different shrink factors b_i. You find the unique exponent p solving

$$\sum_{i=1}^{k} \frac{a_i}{b_i^p} = 1,$$

and then $T(n) = \Theta\Big(n^p\big(1 + \int_1^n \frac{f(u)}{u^{p+1}}\,du\big)\Big)$.

Example 4.6 (Akra–Bazzi on the lopsided split)**.** For $T(n) = T(n/3) + T(2n/3) + cn$ we have two terms with $a_1 = a_2 = 1$, $b_1 = 3$, $b_2 = 3/2$. Solve $\frac{1}{3^p} + \frac{1}{(3/2)^p} = 1$. Try $p = 1$: $\frac{1}{3} + \frac{2}{3} = 1$. It works, so $p = 1$. The integral term then contributes a $\log n$ factor (because $f(u)/u^{p+1} = cu/u^2 = c/u$ integrates to $c \ln n$), giving $T(n) = \Theta(n \log n)$ — exactly what our recursion tree predicted by hand.

Focus

You do not need to memorize the Akra–Bazzi integral for this book. Take away only this: when the master theorem stalls (unequal subproblem sizes, or a log in the gap), *the recursion tree never fails you*, and Akra–Bazzi exists as a heavier formal tool. Knowing the tool exists is enough; reach for it when you meet a recurrence the master theorem cannot classify.

4.10 Cost summary

Cost summary

The five recurrences worth knowing cold:

Recurrence	Solution	Example
$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$	binary search
$T(n) = 2T(n/2) + \Theta(1)$	$\Theta(n)$	tree traversal
$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$	merge sort, max-subarray
$T(n) = 2T(n/2) + \Theta(n^2)$	$\Theta(n^2)$	root-dominated combine
$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7})$	Strassen

Merge sort specifically:

- Time: $\Theta(n \log n)$ in *best, average, and worst* cases — its running time does not depend on the input order.
- Extra space: $\Theta(n)$ for the merge buffer (not in place).
- Stable: yes (equal keys keep their order).

Pause

If this chapter felt like a lot, here is the whole thing in four sentences. Divide and conquer splits a problem into smaller copies of itself, solves them recursively, and combines the results. The running time obeys a recurrence $T(n) = aT(n/b) + f(n)$. You solve such recurrences by drawing a recursion tree, by guessing-and-proving (substitution), or by the master theorem, which compares $f(n)$ against $n^{\log_b a}$ and reports which one wins. Everything else in this chapter is practice and proof of those four sentences.

Divide and conquer gave us deterministic guarantees: merge sort always runs in $\Theta(n \log n)$ time. But what happens when we introduce coin flips into an algorithm? In Chapter 5, we explore *randomized algorithms* and see how a little randomness can defeat worst-case inputs while keeping algorithms simple and fast.

4.11 Exercises

Exercise 4.1 (Warmup: classify by inspection)**.** For each recurrence, give $E = \log_b a$, the master-theorem case, and the Θ-bound. (a) $T(n) = 2T(n/4) + 1$. (b) $T(n) = 2T(n/4) + \sqrt{n}$. (c) $T(n) = 2T(n/4) + n$. (d) $T(n) = 16T(n/4) + n^2$.

Exercise 4.2 (Trace merge)**.** Run MERGE by hand on the two sorted runs $L = [1, 4, 4, 8]$ and $R = [2, 4, 6]$, writing the comparison table as we did in the chapter. How many comparisons did it take, and where did ties go?

Exercise 4.3 (Recursion tree from scratch)**.** Draw the recursion tree for $T(n) = 3T(n/2) + n$. Find the cost of level i, the number of levels, and sum them to get a Θ-bound. Then confirm your answer with the master theorem. (Hint: $E = \log_2 3 \approx 1.585$; the levels do *not* all cost the same.)

Exercise 4.4 (Substitution practice)**.** Use the substitution method to prove $T(n) = O(n^2)$ for $T(n) = 4T(n/2) + n$. Then, more sharply, prove $T(n) = \Theta(n^2)$. What goes wrong if you instead guess $T(n) = O(n^2)$ but try the form dn^2 without an extra lower-order term — and how does subtracting a cn term rescue the proof?

Exercise 4.5 (The gap, felt directly)**.** The master theorem cannot solve $T(n) = 2T(n/2) + n \log n$. Draw the recursion tree, show that level i costs $n \log(n/2^i) = n(\log n - i)$, and sum $\sum_{i=0}^{\log n} n(\log n - i)$ to prove $T(n) = \Theta(n \log^2 n)$.

Exercise 4.6 (Max-subarray crossing)**.** Write pseudocode for MAX-CROSSING(A, lo, m, hi) that finds the best subarray crossing the midpoint in $\Theta(n)$ time. Argue why it must include both $A[m]$ and $A[m + 1]$, and trace it on the 1-indexed array $A = [2, -5, 6, -2, 3, -1]$ with $lo = 1$, $hi = 6$, so $m = \lfloor (1 + 6)/2 \rfloor = 3$ (the left piece ends at index 3 holding 6, and the right piece starts at index 4 holding -2).

Exercise 4.7 (Counting inversions)**.** An *inversion* is a pair $i < j$ with $A[i] > A[j]$ — a measure of how unsorted A is. Modify merge sort so that, while merging, it also counts inversions in $\Theta(n \log n)$ total time. (Hint: when an element from the *right* list is emitted during a merge, every element still remaining in the *left* list forms an inversion with it. Why?) Trace your count on $[2, 4, 1, 3, 5]$ and verify it equals 3.

Exercise 4.8 (Strassen's bookkeeping)**.** Strassen replaces 8 multiplications by 7 at the cost of going from about 4 block additions to 18. Write the recurrence $T(n) = 7T(n/2) + cn^2$, classify it with the master theorem, and explain why the jump from 18 additions does *not* change the asymptotic exponent. For what rough range of n would a careful implementor expect Strassen to start beating the n^3 method, given its larger constant?

Exercise 4.9 (Akra–Bazzi)**.** Use Akra–Bazzi to solve $T(n) = T(n/4) + T(n/2) + n$. First find p from $4^{-p} + 2^{-p} = 1$ (show that $p = 1$ does *not* satisfy it, then argue $p < 1$), estimate p numerically, and state the resulting Θ-bound. Compare it to the master-theorem-style guess you would make if the two subproblems had been *equal* sized.

Exercise 4.10 (Challenge: closest pair of points)**.** Given n points in the plane, you want the two closest ones. Brute force is $\Theta(n^2)$. Sketch a divide-and-conquer algorithm: sort by x, split into left and right halves, recurse, and combine by checking pairs near the dividing line. Argue that the combine step can be done in $\Theta(n)$ time (the famous "only a constant number of points can be close across the strip" argument), and conclude $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. You need not prove the strip lemma; just state clearly what it claims.

Exercise 4.11 (Challenge: why polynomial gap?)**.** Construct a function $f(n)$ with $f(n) = \Omega(n^{\log_b a})$ but *not* $\Omega(n^{\log_b a+\varepsilon})$ for any $\varepsilon > 0$, so that Case 3 of the master theorem does not apply even though f "looks bigger." Explain in your own words why the theorem demands a *polynomial* gap rather than merely $f(n) > n^{\log_b a}$.

Chapter 5

Probability and Randomized Algorithms

So far, every algorithm you have met has been a predictable little machine: feed it the same input twice and it does exactly the same thing twice, step for step. In this chapter we hand the machine a coin. We let it flip that coin and decide what to do based on the result. That sounds reckless — who wants a sorting routine that behaves differently each time you run it? — but it is one of the most useful ideas in algorithmic design. Randomness, used deliberately, can make an algorithm *faster on average*, *simpler to reason about*, and *immune to the adversary who picks your worst-case input*.

To understand why, we need two things. First, a working vocabulary of probability — not a full course on probability, just the handful of tools that algorithm designers reach for constantly. Second, a way to talk about "how long does this take *on average*" that is just as rigorous as the worst-case analysis you already know. Both threads come together in a single device called the *indicator random variable*. With that one tool, much of average-case analysis becomes routine.

Focus

The destination of this chapter: you will be able to take an algorithm whose running time depends on chance (or on the order of its input), set up the expected cost with indicator random variables, and *compute* that expectation rather than guess at it. We build the probability tools first, then spend them on three concrete problems: the hiring problem, randomly permuting an array, and the famous birthday-paradox style counting that underlies hashing and quicksort.

5.1 Two Different Kinds of "Average"

Before any formulas, let us get the big picture straight, because beginners routinely confuse two genuinely different ideas that both get called "average-case."

Intuition

There are two distinct sources of uncertainty an analysis might average over.
(1) Random input. The algorithm is completely deterministic, but we *assume* the input arrives drawn from some probability distribution (say, "all $n!$ orderings of the array are equally

likely"). We average the running time over that distribution. This is *probabilistic analysis* of a deterministic algorithm.
(2) Random choices inside the algorithm. The input is whatever it is — possibly chosen by a malicious adversary — but the *algorithm itself* flips coins as it runs. We average over the algorithm's own coin flips. This is a *randomized algorithm*.
The second is far more satisfying, because *you* control the coins. You do not have to cross your fingers and hope the input is nicely shuffled; you shuffle it yourself. The guarantee then holds for *every* input.

The hiring problem below starts as case (1) and we will deliberately convert it into case (2). That conversion — turning an assumption about the input into a fact you enforce with your own randomness — is the single most important habit in this chapter.

5.2 The Hiring Problem

Here is a story with a clean cost structure. You are hiring an office assistant through an employment agency. Each day the agency sends you exactly one candidate. Interviewing a candidate is cheap (cost c_i per interview), but *hiring* is expensive: to hire someone you must first fire whoever currently holds the job and pay a large fee c_h for the new hire ($c_h \gg c_i$).

You want the best possible assistant, so you adopt a simple rule:

Algorithm 11: HIRE-ASSISTANT(n)

Input: a sequence of n candidates, interviewed one at a time
Output: the running staff is always the best candidate seen so far

```
1 best ← 0 ;                    // candidate 0 is a fictional least-qualified dummy
2 for i ← 1 to n do
3     interview candidate i;
4     if candidate i is better than candidate best then
5         best ← i;
6         hire candidate i;
```

You interview everyone: that costs $n\,c_i$, no surprises, it happens every time. The interesting cost is hiring. Every time you meet someone better than everyone before, you hire them (and eat the fee c_h). *How many times do you hire?*

Intuition

The worst case is easy and brutal. If the candidates arrive in strictly increasing order of quality — worst first, best last — then *every* candidate is better than all the predecessors, so you hire all n of them. Total hiring cost $n\,c_h$. Ouch.
But surely that exact adversarial order is rare. If candidates show up in a *random* order, you would expect to hire only occasionally: most new candidates are not record-breakers. The central question is: how many hires occur, on average?

The answer, which we will prove, is surprisingly small: about $\ln n$ hires. For $n = 1000$ candidates you expect to hire roughly 7 times, not 1000. To get there we need the right tool.

5.3 Random Variables and Expectation

Let us nail down the vocabulary. A *sample space* S is the set of all possible outcomes of a random experiment, and a *probability distribution* assigns to each outcome a number in $[0, 1]$ so that the numbers sum to 1. An *event* is a subset of S, and its probability is the sum of the probabilities of the outcomes inside it.

Definition 5.1 (Random variable)**.** A *random variable* X is a function from the sample space to the real numbers. It attaches a number to each outcome. We write $\Pr\{X = x\}$ for the probability of the event "X takes the value x," meaning the total probability of all outcomes mapped to x.

The number we care about most is the *average value* of a random variable, weighting each possible value by how likely it is.

Definition 5.2 (Expectation)**.** The *expected value* (or *expectation*, or *mean*) of a random variable X is

$$\mathbb{E}[X] \;=\; \sum_{x} x \cdot \Pr\{X = x\},$$

the sum over all values x that X can take, of the value times its probability.

Example 5.1. Roll a fair six-sided die; let X be the number shown. Each value has probability $\frac{1}{6}$, so

$$\mathbb{E}[X] = 1 \cdot \tfrac{1}{6} + 2 \cdot \tfrac{1}{6} + 3 \cdot \tfrac{1}{6} + 4 \cdot \tfrac{1}{6} + 5 \cdot \tfrac{1}{6} + 6 \cdot \tfrac{1}{6} = \frac{21}{6} = 3.5.$$

Notice that 3.5 is not a face of the die at all. The expectation is a weighted center of mass, not a value you ever literally observe. That is normal and important: expectations need not be "achievable" outcomes.

The single property of expectation that powers nearly everything in this chapter is its *linearity*.

Theorem 5.1 (Linearity of expectation)**.** *For any two random variables X and Y defined on the same sample space, and any constant a,*

$$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y], \qquad \mathbb{E}[aX] = a\,\mathbb{E}[X].$$

More generally, for any random variables $X_1, \ldots, X_n$,

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

Focus

Note that the last line has **no independence assumption**. Linearity of expectation holds even when the X_i depend on each other in complicated ways. This is what makes many hard problems easy: we get to break a tangled quantity into a sum of simple pieces, take the expectation of each piece separately, and add up — without ever worrying whether the pieces are correlated.

Common trap

Linearity is for *sums*, not products. In general $\mathbb{E}[XY] \neq \mathbb{E}[X]\,\mathbb{E}[Y]$; that product rule needs X and Y to be *independent*. Once you get used to linearity of expectation, it is easy to try factoring a product of random variables the same way — which fails whenever the variables are correlated. If you find yourself splitting a product, stop and check independence explicitly.

5.4 Indicator Random Variables: The Master Trick

Here is the device that simplifies average-case calculations. Suppose you want to count how many times some event happens — how many record-breaking candidates, how many collisions, how many inversions. Counting is hard to do directly because the events interact. The trick is to attach a tiny 0/1 variable to *each* possible occurrence, then add them up.

Definition 5.3 (Indicator random variable)**.** Given an event A, the *indicator random variable* $I\{A\}$ is defined by

$$I\{A\} = \begin{cases} 1 & \text{if event } A \text{ occurs,} \\ 0 & \text{if event } A \text{ does not occur.} \end{cases}$$

The link between probability and counting is one short lemma:

Lemma 5.1 (Expectation of an indicator)**.** *For any event A, the indicator $X_A = I\{A\}$ satisfies*

$$\mathbb{E}[X_A] = \Pr\{A\}.$$

Proof. X_A takes only the values 1 and 0. By the definition of expectation,

$$\mathbb{E}[X_A] = 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\overline{A}\} = \Pr\{A\}. \qquad \square$$

Intuition

The expected value of a yes/no variable is just the probability that the answer is yes. So an indicator is a bridge: it converts a *probability* (something you compute by reasoning about chance) into an *expectation* (something you can add up with linearity). The recipe, which we will run three times in this chapter, is:

1. Write the quantity you want to count as a **sum of indicators**, one per possible occurrence.
2. Take the expectation, and use **linearity** to push it inside the sum.
3. Replace each $\mathbb{E}$[indicator] by the corresponding **probability** (Lemma 5.1).
4. Compute those probabilities — usually each one is easy — and **sum**.

Figure 5.1 summarizes this pipeline: a hard count on the left, broken into easy probabilities on the right.

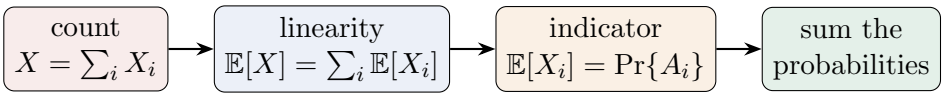


Figure 5.1: The indicator recipe as a pipeline. Each arrow is one of the four steps; the only place that needs real thought is the rightmost box, computing each $\Pr\{A_i\}$, and those are usually one-liners.

Warm-up: how many heads in n coin flips?

Flip a fair coin n times; let X be the total number of heads. You could work through the binomial distribution, but the indicator recipe is much shorter. Let $X_i = I\{\text{flip } i \text{ is heads}\}$. Then $X = \sum_{i=1}^{n} X_i$, and

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \Pr\{\text{flip } i \text{ heads}\} = \sum_{i=1}^{n} \tfrac{1}{2} = \frac{n}{2}.$$

No binomial coefficients or summation identities are needed; the whole calculation takes four lines.

Algorithm lab

Trace the recipe by hand for $n = 3$ flips. List the 8 equally likely outcomes (HHH, HHT, ..., TTT), count the heads in each, average the eight counts directly, and confirm you get $3/2 = 1.5$. Then redo it with the indicator method and compare how much shorter the second calculation is.

5.5 Solving the Hiring Problem with Indicators

Now we apply the indicator method. Assume the candidates arrive in a *uniformly random order*: all $n!$ orderings are equally likely (this is the probabilistic-analysis assumption; we will justify or replace it soon). Let X be the number of times we hire. We want $\mathbb{E}[X]$.

Step 1: write X as a sum of indicators. Define, for each candidate i,

$$X_i = I\{\text{candidate } i \text{ is hired}\}.$$

Hiring happens at step i exactly when candidate i is better than all of candidates $1, 2, \ldots, i-1$. Then $X = X_1 + X_2 + \cdots + X_n$.

Step 2: linearity.

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \Pr\{\text{candidate } i \text{ is the best of the first } i\}.$$

Step 3: compute the probability. Look only at the first i candidates. Because the order is uniformly random, each of these i candidates is equally likely to be the best *among those first i*. There are i of them, so the chance that the best one happens to sit in position i is exactly $1/i$. Hence $\mathbb{E}[X_i] = 1/i$.

Common trap

Notice we only needed the *relative* order of the first i candidates to be uniform, and by symmetry every candidate is equally likely to be the local maximum. A common error is to instead reason "candidate i must be the global best, probability $1/n$." That is wrong — we are not asking whether i is the best overall, only whether i beats the $i-1$ people who came before. Those are different events with different probabilities.

Step 4: sum.

$$\mathbb{E}[X] = \sum_{i=1}^{n} \frac{1}{i} = H_n,$$

the n-th *harmonic number*. And we know from calculus (the harmonic series is squeezed between two logarithms) that

$$\ln n \;<\; H_n \;\le\; \ln n + 1, \qquad \text{so} \qquad \mathbb{E}[X] = \ln n + O(1).$$

Formal

Theorem (expected hires). Assuming candidates are interviewed in a uniformly random order, Hire-Assistant hires an expected $H_n = \ln n + O(1)$ times, for a total expected hiring cost of $O(c_h \ln n)$.

Let us make that concrete.

Example 5.2. For $n = 1000$: $H_{1000} \approx \ln 1000 + 0.577 \approx 6.91 + 0.58 \approx 7.49$. So you expect to hire about 7 or 8 times, not 1000. For $n = 10^6$: $H_{10^6} \approx \ln 10^6 \approx 13.8$. A million applicants, fourteen hires. The worst case needs a million hires; the average needs fourteen. Gaps like this between worst-case and average-case cost are what this chapter is about.

Randomizing the algorithm

There is a catch hiding in the phrase "assuming a uniformly random order." Real candidates do not arrive in random order; the agency might, out of malice or laziness, send them worst-first and force n hires every single time. The probabilistic analysis rests on an assumption that nothing guarantees.

So we *enforce* the assumption. Before interviewing anyone, we collect all the applications and *randomly permute the list ourselves*, then interview in that scrambled order:

Algorithm 12: Randomized-Hire-Assistant(n)

Input: a list of n candidates in arbitrary (possibly adversarial) order
Output: expected number of hires is H_n, for *every* input

```
randomly permute the list of candidates;
best ← 0;
for i ← 1 to n do
    interview candidate i;
    if candidate i is better than candidate best then
        best ← i;    hire candidate i;
```

Now the random order is a *fact we created*, not an assumption about the world. The $\mathbb{E}[X] = H_n$ bound holds no matter how the adversary arranges the input, because we shuffle away their arrangement. This is the conversion promised at the start of the chapter: an assumption about the input becomes a property we enforce with our own random choices.

Why this matters

This pattern — shuffle the input, then run a simple deterministic procedure — is everywhere. It is exactly how randomized quicksort avoids the already-sorted-array worst case (Chapter 7), how randomized hashing defeats an adversary who tries to flood one bucket, and how load balancers spread work. The lesson generalizes far past hiring: *if a deterministic algorithm has a rare bad case that an adversary could trigger, inject your own randomness so that no input is reliably bad.*

5.6 Randomly Permuting an Array

That last algorithm leaned on a step we waved at: "randomly permute the list." This is its own little algorithm and it is surprisingly easy to get subtly wrong. Our goal is a *uniform random permutation*: a procedure that produces each of the $n!$ orderings with probability exactly $1/n!$.

The right way: Randomize-In-Place

The clean method shuffles the array in a single left-to-right pass. At step i it swaps element i with a randomly chosen element from positions i through n (so possibly with itself).

Algorithm 13: RANDOMIZE-IN-PLACE(A, n)

Input: an array $A[1 \dots n]$
Output: A holds a uniform random permutation of its original contents

```
for i ← 1 to n do
    j ← RANDOM(i, n) ;                    // uniform integer in {i, ..., n}
    swap A[i] ↔ A[j];
```

Here RANDOM(a, b) returns an integer in $\{a, a+1, \dots, b\}$, each equally likely, in $O(1)$ time. The whole routine is one loop of n iterations doing $O(1)$ work each, so it runs in $\Theta(n)$ time — you cannot do better, since you must touch every element.

The key picture is the *shrinking window*: once position i is filled it is frozen, and the random draw for the next position is taken only from the unfrozen tail. Figure 5.2 shows the window narrowing by one cell each step for $n = 5$.

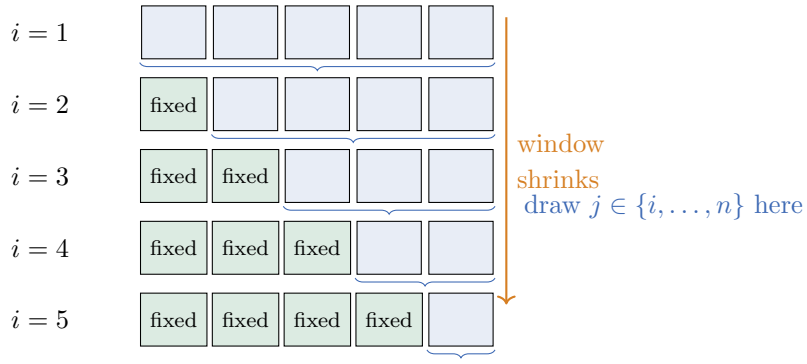


Figure 5.2: The shrinking pick-window of RANDOMIZE-IN-PLACE for $n = 5$. Green cells are frozen (already chosen and never touched again); the blue tail is the range $\{i, \ldots, n\}$ from which RANDOM draws at step i. Each step freezes one more cell, so the window loses exactly one position per row.

This is sometimes called the Fisher–Yates shuffle, or Knuth's shuffle. Let us see why it is uniform.

Lemma 5.2 (Uniformity of RANDOMIZE-IN-PLACE)**.** RANDOMIZE-IN-PLACE *produces a uniform random permutation: every ordering appears with probability* $1/n!$.

Proof by loop invariant. We use the invariant: *just before the iteration for index* i*, the subarray* $A[1 \ldots i-1]$ *contains any given sequence of* $i-1$ *of the original elements with probability* $(n-i+1)!/n!$. (Equivalently: the prefix of length $i-1$ is a uniformly random arrangement of some $(i-1)$-subset.)

Initialization $(i = 1)$**.** The prefix $A[1 \ldots 0]$ is empty, so there is exactly one "arrangement" of 0 elements, and the claimed probability is $(n-0)!/n! = 1$. True.

Maintenance. Assume the invariant holds before iteration i. Consider any specific desired arrangement of $A[1 \ldots i]$, say $(x_1, x_2, \ldots, x_i)$. For $A[1 \ldots i]$ to end up as this sequence, two things must happen: the prefix $A[1 \ldots i-1]$ already equaled $(x_1, \ldots, x_{i-1})$ *before* this iteration (probability $(n-i+1)!/n!$ by the invariant), *and* the swap put x_i into position i. Once the prefix is fixed, x_i still sits somewhere in $A[i \ldots n]$ — that is $n-i+1$ positions — and RANDOM(i, n) picks its position with probability $1/(n-i+1)$. These two are independent, so

$$\Pr\{A[1 \ldots i] = (x_1, \ldots, x_i)\} = \frac{(n-i+1)!}{n!} \cdot \frac{1}{n-i+1} = \frac{(n-i)!}{n!},$$

which is exactly the invariant with i in place of $i-1$. Maintenance holds.

Termination $(i = n+1)$**.** The invariant now describes the full array: each complete permutation $(x_1, \ldots, x_n)$ occurs with probability $(n-n)!/n! = 1/n!$. That is uniformity. □

Example 5.3. Trace the shuffle on $A = [a, b, c]$, $n = 3$. Suppose RANDOM returns: $j = 3$ at $i = 1$, then $j = 2$ at $i = 2$, then $j = 3$ at $i = 3$.

i	range for j	drawn j	array after swap	
start	—	—	$[a, b, c]$	
1	$\{1, 2, 3\}$	3	$[c, b, a]$	(swap $A[1] \leftrightarrow A[3]$)
2	$\{2, 3\}$	2	$[c, b, a]$	(swap with self)
3	$\{3\}$	3	$[c, b, a]$	(forced)

This run yields $[c, b, a]$. There are $3 \cdot 2 \cdot 1 = 6$ equally likely sequences of draws (3 choices at $i = 1$, then 2, then 1), and you can check that the six runs land on the six distinct permutations exactly once each — which is precisely uniformity.

A tempting wrong way

Common trap

A natural-looking "fix" is to let j range over *all* of $\{1, \ldots, n\}$ at every step instead of $\{i, \ldots, n\}$:

```
for i = 1 to n:  swap A[i] with A[RANDOM(1, n)]
```

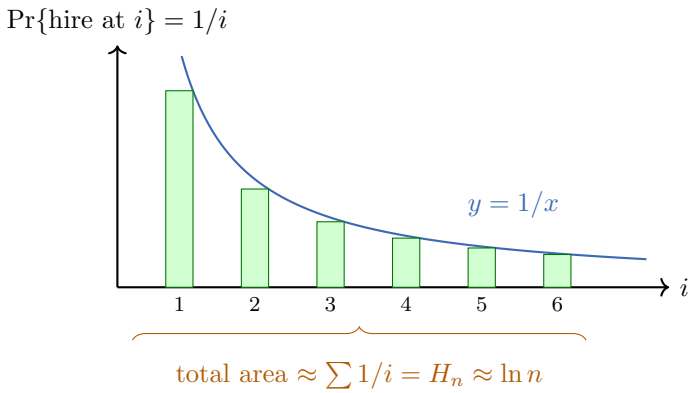
This looks more random — every position can move anywhere! — but it is *not* uniform. It performs n swaps, each with n equally likely choices, so it follows one of n^n equally likely paths. But there are only $n!$ permutations, and $n!$ does not divide n^n for $n \geq 3$. By pigeonhole, the permutations cannot all get equal probability: some orderings are slightly favored. For $n = 3$ the six permutations receive the path-counts $\{4, 4, 4, 5, 5, 5\}$ out of the 27 paths — not all $27/6 = 4.5$ — so three arrangements are over-represented and three are under-represented. The lesson: more visible randomness is not the same as uniform randomness. Use the proven RANDOMIZE-IN-PLACE with its shrinking range.

Walking question

Enumerate the broken shuffle above for $n = 3$. There are 27 equally likely (j_1, j_2, j_3) triples. Tally which of the 6 permutations each triple produces. You should find three permutations appearing 5 times and three appearing 4 times (so $3 \cdot 5 + 3 \cdot 4 = 27$ paths in all, as it must). Then ask yourself: *why* does the correct version, with its shrinking range $\{i, \ldots, n\}$, give exactly $3! = 6$ paths mapping one-to-one onto the 6 permutations? The contrast between "$n!$ equally likely paths" versus "n^n equally likely paths" explains why the bias arises.

5.7 A Recursion-Tree-Free Look at the Numbers

It helps to *see* the harmonic sum that controls the hiring problem. Each term $1/i$ is the probability that candidate i breaks the record. The early candidates almost always break it $(1/1, 1/2, 1/3, \ldots)$; the later ones rarely do. Plotting the terms makes the $\ln n$ growth visible: it is the area under the curve $1/x$.



The bars are the actual probabilities $1/i$; the smooth curve is $1/x$. The sum of the bar heights is H_n, and it tracks the area under $1/x$ from 1 to n, which is $\ln n$. This is why "a million applicants" costs only "fourteen hires."

5.8 A Second Helping: Expected Number of Inversions

To show that the indicator method is reusable and not a one-off trick, let us apply it to a different problem. An *inversion* in an array $A[1 \ldots n]$ of distinct numbers is a pair of indices (i, j) with $i < j$ but $A[i] > A[j]$ — a pair that is out of order. Inversions measure how unsorted an array is (insertion sort, from an earlier chapter, runs in time proportional to the number of inversions). *If the array is a uniform random permutation, how many inversions do we expect?*

Before the algebra, a picture of what we are counting. Figure 5.3 shows a small array; every line connects a pair (i, j) with $i < j$, and a pair is an inversion exactly when the left value is the larger one.

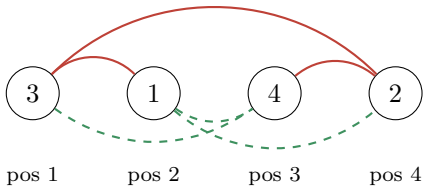


Figure 5.3: Inversions of $A = [3, 1, 4, 2]$. Solid red arcs above are the three inversions (3>1, 3>2, 4>2); dashed green arcs below are the three in-order pairs. There are $\binom{4}{2} = 6$ pairs total, and here exactly half are inverted — the average outcome the calculation below predicts.

Run the recipe. For each pair $i < j$ define the indicator

$$X_{ij} = I\{\, A[i] > A[j] \,\} = I\{(i, j) \text{ is an inversion}\}.$$

The total number of inversions is $X = \sum_{i<j} X_{ij}$. By linearity,

$$\mathbb{E}[X] = \sum_{1 \le i < j \le n} \mathbb{E}[X_{ij}] = \sum_{1 \le i < j \le n} \Pr\{A[i] > A[j]\}.$$

For any fixed pair of distinct values, in a uniform random permutation they are equally likely to appear in either order, so $\Pr\{A[i] > A[j]\} = \frac{1}{2}$. The number of pairs with $i < j$ is $\binom{n}{2} = \frac{n(n-1)}{2}$. Therefore

$$\mathbb{E}[X] = \binom{n}{2} \cdot \frac{1}{2} = \frac{n(n-1)}{4}.$$

Example 5.4. For $n = 10$: $\mathbb{E}[X] = \frac{10 \cdot 9}{4} = 22.5$ inversions on average. For $n = 100$: $\frac{100 \cdot 99}{4} = 2475$. Roughly a quarter of all pairs are inverted in a random array — which is why insertion sort, costing one unit per inversion, runs in expected $\Theta(n^2)$ time on random input. The indicator method just *told* us the average-case running time of an algorithm from two earlier chapters, almost for free.

Focus

Look back at the three problems — heads in coin flips, hires, inversions. The *same four-line recipe* solved all three. Decompose into indicators, apply linearity, swap each expectation for a probability, sum. When you see "expected number of *[somethings]*," start by defining one indicator per *something*.

5.9 Las Vegas vs. Monte Carlo: Two Flavors of Randomized Algorithm

Randomized algorithms come in two temperaments, and it is worth naming them.

Intuition

A **Las Vegas** algorithm always returns the *correct* answer; only its *running time* is random. Randomized quicksort and RANDOMIZED-HIRE are Las Vegas: the output is never wrong, but how long it takes depends on the coins. You gamble with time, never with correctness.

A **Monte Carlo** algorithm always finishes in a *bounded* time, but may occasionally return a *wrong* answer, with some controllable small probability. Many primality tests and approximate-counting schemes are Monte Carlo: fast and bounded, but right only "with high probability."

Mnemonic: in Las Vegas you are guaranteed to leave with the right answer, you just do not know *when*; in Monte Carlo you leave *on time*, but might leave with the wrong souvenir.

This chapter's algorithms are all Las Vegas. We will meet Monte Carlo algorithms later, especially in the number-theory chapter, where being wrong one time in a billion is a perfectly good trade for enormous speed.

5.10 What "Expected Running Time" Really Means

We should be careful about a phrase we have been using loosely. When we say a randomized algorithm has *expected running time* $T(n)$, we mean: *fix* any input of size n, then average the running time over the algorithm's internal coin flips. The expectation is taken over the randomness of the algorithm, with the input held fixed and adversarial.

Formal

For a randomized algorithm $\mathcal{A}$, the expected running time on an input x is

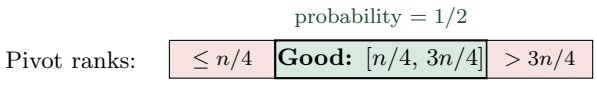
$$T_{\mathcal{A}}(x) = \mathbb{E}_r\big[\,\text{steps } \mathcal{A} \text{ takes on input } x \text{ using random bits } r\,\big],$$

where the expectation is over the algorithm's random bits r. The *worst-case expected running time* is then $\max_{|x|=n} T_{\mathcal{A}}(x)$: the worst this average gets over all inputs of size n.

Common trap

Do not confuse *expected* running time with *worst-case* running time. Randomized quicksort has expected running time $\Theta(n \log n)$ but worst-case running time $\Theta(n^2)$ — the bad case still exists, it has just become astronomically unlikely (the coins would all have to conspire). "Expected $\Theta(n \log n)$" is a promise about the average over coins, not a promise that n^2 can never happen.

Figure 5.4 shows this intuition through the probability tree of a randomized pivot choice. When a pivot is chosen uniformly at random, it lands in the central half of the sorted array with probability $1/2$. Whenever it lands there (a "good split"), neither recursive subarray exceeds $\frac{3}{4}n$ in size. Even if a pivot lands in the outer quarters (a "bad split"), the next random trial is an independent coin flip with success probability $1/2$. The expected number of trials until a good split is at most $1/(1/2) = 2$. On average, every two partition steps shrink the problem by a factor of at least $3/4$, capping the expected recursion depth at $O(\log n)$.



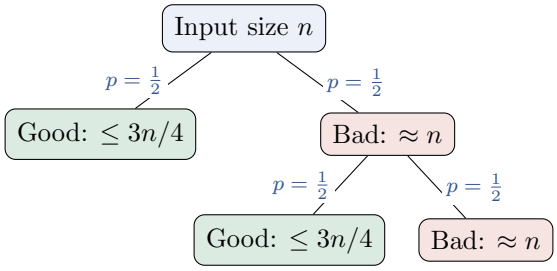


Figure 5.4: Randomized pivot partitioning represented as a probability tree. Half of all possible pivot choices fall in the central 50% of the ranks, guaranteeing that the larger subproblem has size at most $\frac{3}{4}n$. A geometric waiting time shows that a good split occurs within 2 steps in expectation, yielding $O(\log n)$ expected tree depth.

5.11 Putting It Together: a Tiny End-to-End Example

Let us run one complete numeric story through the machinery, hiring problem style, to leave no gap between the abstract recipe and concrete arithmetic.

Example 5.5 (Hiring with $n = 4$, fully enumerated)**.** Take four candidates with qualities $1 < 2 < 3 < 4$ (so 4 is best). With a uniform random arrival order there are $4! = 24$ equally likely orderings. We claim $\mathbb{E}[\text{hires}] = H_4 = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = \frac{25}{12} \approx 2.083$.

Let us sanity-check by counting hires for a few orders (candidate 0 is the dummy, so the first real candidate is always hired):

- Order $1, 2, 3, 4$: hire at every step. 4 hires.
- Order $4, 3, 2, 1$: hire only the first. 1 hire.
- Order $2, 1, 4, 3$: hire 2, then 4. 2 hires.
- Order $3, 1, 4, 2$: hire 3, then 4. 2 hires.

If you grind through all 24 orders, the hire counts sum to 50, giving average $50/24 = 25/12 \approx 2.083$ — exactly H_4. The indicator argument gave this in four lines, and the brute-force enumeration confirms it: linearity of expectation simply regroups the same sum by position rather than by permutation.

Algorithm lab

Reproduce Example 5.5 yourself. Write out all 24 permutations of $\{1, 2, 3, 4\}$ (or just generate them mentally in lexicographic order), count hires for each, and confirm the total is 50. Then group the 24 orders by "did candidate at position i get hired?" and verify that position i is a hire in exactly $24/i$ of the orders — 24 for $i = 1$, 12 for $i = 2$, 8 for $i = 3$, 6 for $i = 4$ — matching the $1/i$ probability of each indicator X_i.

Cost summary

Costs at a glance.

- RANDOMIZE-IN-PLACE: $\Theta(n)$ time, $O(1)$ extra space; produces each of $n!$ permutations with probability $1/n!$.
- HIRE-ASSISTANT (deterministic, random input): interviewing $\Theta(n)$ always; expected hires $H_n = \ln n + O(1)$; worst-case hires n.
- RANDOMIZED-HIRE-ASSISTANT: same expected hires H_n for *every* input (adversary cannot force the worst case); cost of the shuffle is $\Theta(n)$, dwarfed by interviewing.
- Expected inversions in a uniform random permutation: $\binom{n}{2}/2 = n(n-1)/4 = \Theta(n^2)$.

History and connection

In 1946, while recovering from encephalitis and playing Canfield solitaire, Polish-American mathematician Stanislaw Ulam wondered what fraction of solitaire deals are theoretically winnable. Working out the answer by pure combinatorics was hopeless. Ulam had a simpler idea: instead of solving the combinatorics exactly, why not simply deal one hundred random hands, count the wins, and observe the empirical frequency? He shared the idea with John von Neumann at Los Alamos, who immediately saw its value for simulating neutron transport and nuclear fission in the Manhattan Project. In 1947, von Neumann implemented the first large-scale automated simulation on the vacuum-tube ENIAC, codenaming it "Monte Carlo" after the Monaco gambling casino. Three decades later, in 1976, Michael Rabin published

his foundational paper "Probabilistic Algorithms," demonstrating that tossing coins inside an algorithm could tame notoriously hard discrete problems — from testing whether a thousand-digit number is prime to finding the closest pair of points in space.

The rise of randomized algorithms met fierce philosophical resistance. For generations, scientists believed an algorithm that could produce different running times or answers on the same input was broken or untrustworthy. It took decades to distinguish between *Monte Carlo* algorithms (which run in fixed time but carry an infinitesimal, tunable chance of error) and *Las Vegas* algorithms (which never return an incorrect answer, but whose running time is a random variable). Early implementations were also plagued by bad pseudo-random generators; von Neumann's middle-square method frequently decayed into zero, while naive array shuffles (swapping each $A[i]$ with $A[\textsc{Random}(1, n)]$) explored n^n paths, which cannot divide $n!$ evenly and thus quietly rigged card games in early software.

Randomization gives a strong defense against malicious adversaries: when an algorithm flips its own coins at runtime, no single worst-case input can ever be crafted in advance. It powers universal hashing to thwart denial-of-service attacks and enables polynomial-time prime generation for RSA encryption. It also has real downsides: software requires access to genuine physical entropy pools (such as thermal jitter), and tracking down a bug that only triggers once every ten million random coin flips can confound even the best engineers. In Chapter 7, we apply these coin flips directly to pivot selection in Randomized Quicksort; in Chapter 11, we use random hash families to build collision-resistant hash tables; and in Chapter 30, we study Rabin's primality test in full detail.

Why this matters

Why does any of this matter outside a textbook? Because the "shuffle, then run something simple" pattern is widely used in real systems. Randomized quicksort is the default sort in many language standard libraries precisely because randomization makes the $\Theta(n^2)$ case vanish in practice without any input assumptions. Hash tables randomize their hash function so that no adversary can craft keys that all collide — a real security concern ("hash-flooding" denial-of-service attacks against web servers were patched exactly this way). Distributed systems pick random servers to balance load. Cryptography is *built* on randomness. The humble indicator random variable is your first rigorous tool for predicting how all of these behave on average.

5.12 Summary

This chapter added randomness to the algorithm designer's toolkit. The main points:

- **Two averages.** Probabilistic analysis averages a deterministic algorithm over a random *input*; a randomized algorithm averages over its own *coin flips*. Prefer the second: it guarantees the bound for every input.
- **Expectation and its linearity.** $\mathbb{E}[X] = \sum_x x \Pr\{X = x\}$, and $\mathbb{E}[\sum X_i] = \sum \mathbb{E}[X_i]$ *always*, correlated or not. This unconditional linearity does most of the work in expectation calculations.
- **Indicator random variables.** $\mathbb{E}[I\{A\}] = \Pr\{A\}$. To find an expected count, write it as a sum of indicators, apply linearity, replace each expectation by a probability, and sum. We ran

this recipe on coin flips ($n/2$), hires (H_n), and inversions ($n(n-1)/4$).

- **The hiring problem.** Expected hires $= H_n = \ln n + O(1)$: tiny. Randomizing the input order makes this guarantee hold against any adversary.
- **Uniform shuffling.** RANDOMIZE-IN-PLACE swaps $A[i]$ with a random later (or equal) position, runs in $\Theta(n)$, and is provably uniform by a loop invariant. The all-positions variant looks more random but is provably *not* uniform.

Everything from here — quicksort, hashing, skip lists, primality testing — leans on these few ideas. Hold onto the indicator recipe most tightly; it is the single most reused tool in the analysis of algorithms.

Before we apply randomized analysis to quicksort in Chapter 7 and hashing in Chapter 11, we turn to the next major chapter in sorting: *heapsort* (Chapter 6). Unlike insertion sort ($\Theta(n^2)$) or merge sort (which requires $\Theta(n)$ extra memory), heapsort achieves optimal $\Theta(n \log n)$ worst-case sorting *in place*, using a compact data structure called the *binary heap*.

5.13 Exercises

Exercise 5.1 (Warm-up: expectation of a biased coin)**.** A coin comes up heads with probability p and tails with probability $1 - p$. Let X be the indicator that a single flip is heads. Compute $\mathbb{E}[X]$ directly from the definition of expectation, and confirm it equals $\Pr\{\text{heads}\}$ as Lemma 5.1 promises. Then, for n independent such flips, find the expected number of heads using indicators.

Exercise 5.2 (Expected maximum index of a hire)**.** In HIRE-ASSISTANT with random input, candidate 1 is always hired and candidate n is hired with probability $1/n$. Using indicators, what is the expected number of *interviews that result in a hire among the last half* of candidates, i.e. positions $\lceil n/2 \rceil$ through n? Express your answer as a difference of harmonic numbers and estimate it for large n.

Exercise 5.3 (Two dice)**.** Roll two fair dice and let X be the sum of the faces. Use linearity of expectation (write $X = X_1 + X_2$ for the two dice) to find $\mathbb{E}[X]$ in two lines, without summing over all 36 outcomes. Then find $\mathbb{E}[X]$ the long way and confirm they agree.

Exercise 5.4 (Tracing the shuffle)**.** Run RANDOMIZE-IN-PLACE on $A = [w, x, y, z]$ ($n = 4$) assuming RANDOM returns $j = 2, 4, 3, 4$ at steps $i = 1, 2, 3, 4$ respectively. Show the array after each swap. Then argue, using Lemma 5.2, why this particular output had probability exactly $1/24$ before the run started.

Exercise 5.5 (Why the broken shuffle fails, in miniature)**.** Consider the incorrect shuffle that, for each i from 1 to n, swaps $A[i]$ with $A[\text{RANDOM}(1, n)]$. For $n = 2$, enumerate all $2^2 = 4$ equally likely paths and compute the probability of each of the two permutations of $[a, b]$. Are they both $1/2$? Now do the same for the *correct* RANDOMIZE-IN-PLACE and confirm it gives $1/2$ each. (For $n = 2$ the broken version happens to be uniform; explain why it first breaks at $n = 3$, in terms of $n!$ dividing n^n.)

Exercise 5.6 (Expected inversions, restricted)**.** In a uniform random permutation of $\{1, \ldots, n\}$, what is the expected number of inversions (i, j) with $i < j$ that are *adjacent*, i.e. $j = i + 1$? Use one indicator per adjacent pair, and compare your answer to the total expected inversions $n(n-1)/4$. What fraction of inversions, on average, are adjacent?

Exercise 5.7 (Expected number of fixed points)**.** A *fixed point* of a permutation π of $\{1, \ldots, n\}$ is an index i with $\pi(i) = i$. For a uniform random permutation, define $X_i = I\{\pi(i) = i\}$ and use the indicator recipe to show the expected number of fixed points is exactly 1, *independent of* n. (This is a famous and counterintuitive fact: whether you shuffle 5 cards or 5 million, on average exactly one ends up in its original place.)

Exercise 5.8 (Coupon collector, gently)**.** You repeatedly draw a uniformly random coupon from n distinct types (with replacement). Let T be the number of draws until you have collected all n types. Split the process into n *phases*: phase k is the time spent holding exactly $k-1$ distinct types while waiting for a new one. Argue that the expected length of phase k is $\frac{n}{n-k+1}$ (a geometric waiting time), then use linearity to show $\mathbb{E}[T] = nH_n \approx n \ln n$. Compare this $n \ln n$ to the hiring problem's $\ln n$ and explain in words why one carries an extra factor of n.

Exercise 5.9 (Challenge: expected comparisons in a max-scan)**.** You scan an array of n distinct numbers in random order, keeping a running maximum (like HIRE-ASSISTANT, but you only update a variable rather than pay a hiring fee). Each time you see a new record you do one "update." You already know the expected number of updates is H_n. Now suppose each update costs i units when it happens at position i (later updates are more expensive). Using indicators *and* the fact that $\mathbb{E}[i \cdot X_i] = i \cdot \mathbb{E}[X_i]$ here (why is this valid even though i multiplies the indicator?), compute the expected total update cost $\sum_{i=1}^{n} i \cdot \Pr\{\text{record at } i\}$. Simplify it to a closed form and give its asymptotic order.

Exercise 5.10 (Challenge: Monte Carlo majority)**.** An array of n elements is promised to contain a *strict majority* element (one value occupying more than $n/2$ slots). Here is a Monte Carlo algorithm: pick a random index, read its value v, and *guess* that v is the majority. (a) What is the probability this single guess is correct? (b) If you repeat the guess k times independently and output any value that you guessed at least once, bound the probability that the true majority is never guessed. (c) How large must k be to make the failure probability below $1/1000$? This is the idea behind probability amplification — turning a coin-flip-quality algorithm into a reliable one by repetition.

Part II

Sorting and Order Statistics

Chapter 6

Heaps and Heapsort

So far you have met two sorting algorithms with very different trade-offs. Insertion sort is simple, works in place, and runs in $\Theta(n^2)$ time in the worst case. Merge sort divides and conquers in $\Theta(n \log n)$ time — but it pays for that speed with an extra array of scratch space the size of the input. A natural question follows: *can we sort in $\Theta(n \log n)$ time without borrowing a second array?*

The answer is yes, and the tool that gets us there is the **binary heap**. A heap is a tree that lives inside a plain array, with no pointers, no allocation, no fuss. Understanding it buys you two things at once. The first is **heapsort**: an in-place, worst-case $\Theta(n \log n)$ sorting algorithm. The second is the **priority queue**: a tiny, fast scheduler that answers the question "what is the most important thing waiting right now?" over and over, and which we will lean on heavily later when we build Dijkstra's algorithm and Prim's algorithm for graphs.

This chapter slows everything down. We will build the array-as-tree picture by hand, trace every operation on real numbers, prove the loop invariants, and derive — not merely state — the surprising fact that you can turn a random array into a heap in *linear* time.

6.1 The shape of a heap

6.1.1 A tree hiding in an array

Picture a binary tree. Not a search tree — forget everything you might know about "smaller on the left, bigger on the right." Just a tree where every node has at most two children. We are going to insist on one strict shape rule:

Focus

A **complete binary tree** is filled level by level, top to bottom, and within each level left to right. Every level is completely full except possibly the last, and the last level is packed as far to the left as it will go. There are no gaps.

That "no gaps" rule is what lets us drop the pointers. If the tree has no holes, we can lay the nodes out in an array just by reading them off level by level, and then a child's position is always a fixed arithmetic step away from its parent's. Figure 6.1 shows this dual perspective: a complete tree above, its flat array representation below, and the index mapping connecting them.

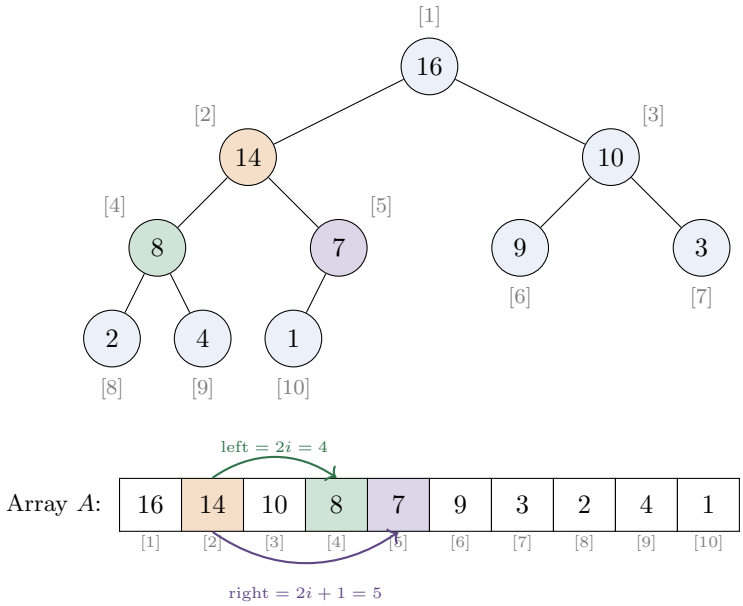


Figure 6.1: A complete binary tree mapped onto a flat array. Reading the tree level by level places node i into slot $A[i]$ (indicated by gray indices $[1 \dots 10]$). Node 2 (orange), its left child at $2i = 4$ (green), and right child at $2i + 1 = 5$ (purple) illustrate the direct indexing arithmetic: parent at $\lfloor i/2 \rfloor$, children at $2i$ and $2i + 1$. No pointer fields or dynamic allocations are needed.

The gray numbers underneath are array indices. Compare the tree and the array side by side. The root 16 is at index 1. Its children, 14 and 10, are at indices 2 and 3. The children of 14 (index 2) are 8 and 7, at indices 4 and 5. The children of 10 (index 3) are 9 and 3, at indices 6 and 7. A pattern is forming.

Focus

For a 1-indexed array, the node at index i has

$$\text{Parent}(i) = \lfloor i/2 \rfloor, \qquad \text{Left}(i) = 2i, \qquad \text{Right}(i) = 2i + 1.$$

Check it on index 2: its left child is $2 \cdot 2 = 4$ (the value 8) and its right child is $2 \cdot 2 + 1 = 5$ (the value 7). Its parent is $\lfloor 2/2 \rfloor = 1$ (the root). Every one of these is a single multiply, add, or shift. In binary arithmetic, multiplying by 2 shifts bits left by one position (appending a 0), while integer division by 2 shifts bits right by one position (dropping the lowest bit). Navigating up and down the heap tree costs no division hardware at all.

Intuition

A heap is a *tree you can think about* but an *array you can store*. You get the conceptual clarity of a tree — parents, children, levels, height — with the memory layout and cache-friendliness of a flat array. There are no nodes to allocate and no pointers to chase or break. The tree structure is entirely *implied* by the index arithmetic.

Common trap

The most common beginner stumble: doing this with 0-indexed arrays but using the 1-indexed formulas. If your language indexes from 0 (most do), the formulas shift to $\text{Left}(i) = 2i + 1$, $\text{Right}(i) = 2i + 2$, and $\text{Parent}(i) = \lfloor (i-1)/2 \rfloor$. Pick a convention and *write it at the top of your code*. In this chapter we follow CLRS and index from 1.

6.1.2 The heap property: the ordering rule

The shape rule (complete tree) tells us *where* things go. We now add an *ordering* rule that tells us how the values relate. There are two flavors.

Formal

An array $A[1 \ldots n]$ is a **max-heap** if, for every node i other than the root,

$$A[\text{Parent}(i)] \geq A[i].$$

Equivalently: every parent is at least as large as each of its children. It is a **min-heap** if instead $A[\text{Parent}(i)] \leq A[i]$ everywhere.

Look back at our example tree. The root is 16, the largest value. Below 16 sit 14 and 10, both smaller. Below 14 sit 8 and 7; below 10 sit 9 and 3. Walk down any path from the root to a leaf and the values only ever *decrease* (or stay equal). That is a max-heap.

Common trap

A heap is **not** a sorted array, and it is **not** a binary search tree. In a max-heap the only promise is parent $\geq$ children. There is *no* relationship enforced between a node and its sibling, or between an uncle and a niece, or between the two subtrees. In our example, 9 (deep on the right) is bigger than 8, 7, 2, 4, and 1 (all on the left), even though those left nodes sit higher in the tree. The heap property is purely local: parent versus child, and nothing more. This local-ness is exactly what makes heaps cheap to maintain.

For the rest of the chapter we work with max-heaps; min-heaps are the mirror image, obtained by flipping every $\geq$ to $\leq$.

Focus

A heap actually has two pieces of bookkeeping. The array A has some allocated length, written $A.\mathit{length}$. But only the first $A.\mathit{heapsize}$ elements are "in the heap" right now, with $0 \leq A.\mathit{heapsize} \leq A.\mathit{length}$. Elements past the heap size are still physically in the array but are not part of the tree. This split is what makes heapsort in-place: as we extract elements, the heap shrinks and the sorted region grows, all inside one array.

6.1.3 How tall is a heap?

The height of a tree is the number of edges on the longest path from the root down to a leaf. Because a heap is a complete binary tree, its height is tightly controlled by the number of elements.

Lemma 6.1 (Height of a heap)**.** *A heap with n elements has height $h = \lfloor \log_2 n \rfloor$.*

Intuition

Each new level of a binary tree can hold twice as many nodes as the one above it: level 0 holds 1 node, level 1 holds up to 2, level 2 up to 4, and in general level k holds up to 2^k nodes. To pile up n nodes you need about $\log_2 n$ levels, because doubling $\log_2 n$ times is what gets you to n. So the tree is short and wide: $n = 1{,}000{,}000$ items fit in a tree only 20 levels tall.

Formal

A complete binary tree of height h has at least 2^h nodes (the levels 0 through $h-1$ are completely full, contributing $1 + 2 + \cdots + 2^{h-1} = 2^h - 1$ nodes, plus at least one node on level h) and at most $2^{h+1} - 1$ nodes (every level full). Hence

$$2^h \le n \le 2^{h+1} - 1 < 2^{h+1}.$$

Taking $\log_2$ of $2^h \le n < 2^{h+1}$ gives $h \le \log_2 n < h + 1$, so $h = \lfloor \log_2 n \rfloor = \Theta(\log n)$.

This $\Theta(\log n)$ height, recorded in Lemma 6.1, is the source of every speed claim in the chapter. The basic heap operations all do work proportional to the height, so they all cost $O(\log n)$.

6.2 Max-heapify: repairing one broken spot

Here is the core operation. Suppose you have an array that is *almost* a heap. Specifically, you point at a node i, and you are promised that the left subtree of i and the right subtree of i are both already valid max-heaps. The only possible problem is at i itself: maybe $A[i]$ is smaller than one of its children, which would violate the heap property right there. The procedure Max-Heapify fixes that single violation by letting the value at i "sink" or "float down" to where it belongs.

Intuition

A too-small value sitting at the top of two good heaps does not belong up high. Max-Heapify lets it sink, one level at a time, swapping past the larger of its two children, until either it is bigger than both children or it has hit the bottom. As it sinks, the bigger child rises to take the parent's place — exactly what the heap property wants up top.

Algorithm 14: MAX-HEAPIFY(A, i) — sink the value at i into place

Input: Array A with heap size $A.heapsize$; index i whose two child subtrees are already max-heaps.

Output: The subtree rooted at i is now a valid max-heap.

```
1  l ← Left(i);
2  r ← Right(i);
3  if l ≤ A.heapsize and A[l] > A[i] then
4  |  largest ← l;
5  else
6  |  largest ← i;
7  if r ≤ A.heapsize and A[r] > A[largest] then
8  |  largest ← r;
9  if largest ≠ i then
10 |  exchange A[i] ↔ A[largest];
11 |  MAX-HEAPIFY(A, largest);
```

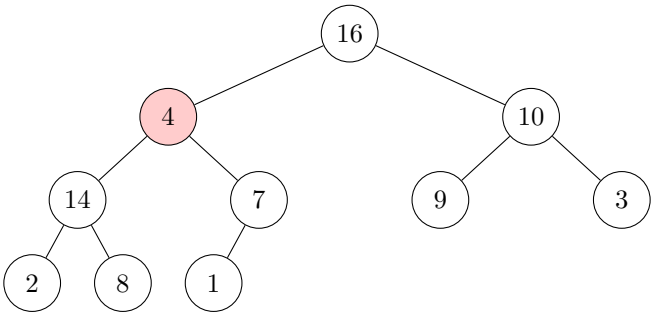
Line by line, we find the index of the largest among the node and its two children (carefully checking that each child actually exists, i.e. its index is within the heap size). If the node itself is already the largest, there is nothing to do — the heap property holds here and we stop. Otherwise we swap the node with its largest child, which fixes the violation at i, and then we recurse on the child we swapped into, because the swap may have created a new violation one level down.

6.2.1 A full numeric trace

Let us run MAX-HEAPIFY$(A, 2)$ on this array, where the subtrees are valid heaps but the value at index 2 is too small. Take

$$A = [\,16,\ \mathbf{4},\ 10,\ 14,\ 7,\ 9,\ 3,\ 2,\ 8,\ 1\,], \qquad heapsize = 10.$$

As a tree, with the offending 4 in red:



Call Max-Heapify$(A, 2)$. Here $i = 2$, so $l = 4$ and $r = 5$. The values are $A[2] = 4$, $A[4] = 14$, $A[5] = 7$. The largest of $\{4, 14, 7\}$ is 14 at index 4, so $largest = 4 \neq i$. We swap $A[2] \leftrightarrow A[4]$:

$$A = [\,16,\ 14,\ 10,\ \mathbf{4},\ 7,\ 9,\ 3,\ 2,\ 8,\ 1\,].$$

The 14 has floated up; the 4 has dropped to index 4. Now we recurse on index 4.

Call Max-Heapify$(A, 4)$. Now $i = 4$, so $l = 8$ and $r = 9$. Values: $A[4] = 4$, $A[8] = 2$, $A[9] = 8$. The largest of $\{4, 2, 8\}$ is 8 at index 9, so *largest* $= 9$. Swap $A[4] \leftrightarrow A[9]$:

$$A = [\,16,\ 14,\ 10,\ 8,\ 7,\ 9,\ 3,\ 2,\ \mathbf{4},\ 1\,].$$

Recurse on index 9.

Call Max-Heapify$(A, 9)$. Now $i = 9$, so $l = 18$ and $r = 19$. Both exceed *heapsize* $= 10$, so node 9 has no children — it is a leaf. Thus *largest* $= i$, the test *largest* $\neq i$ fails, and we stop.

The 4 has sunk all the way to a leaf, and the array is now a valid max-heap. Figure 6.2 shows the ripple path $2 \to 4 \to 9$: at each step, the smaller parent trades places with its larger child, rippling down one level at a time while promoting the larger values.

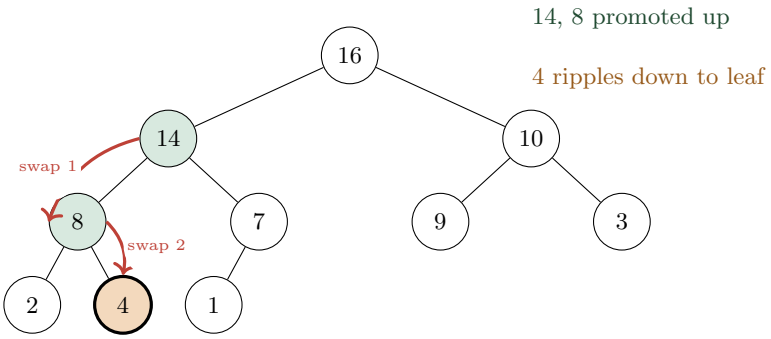


Figure 6.2: The sinking ripple of Max-Heapify$(A, 2)$. The key 4 starts at node 2, swaps with its larger child 14 (moving to node 4), then swaps with 8 (moving to node 9). At each step the displaced key ripples down exactly one level while a larger child floats up.

Notice the path the 4 took: index $2 \to 4 \to 9$, exactly one node per level, straight down the tree. That is the key observation for bounding the running time.

6.2.2 Why it is correct

Focus

The promise Max-Heapify relies on. It is only correct if, when you call it on i, the subtrees rooted at Left(i) and Right(i) are *already* valid max-heaps. The procedure fixes a violation *at* i; it cannot fix a mess scattered throughout the subtrees.

Given that promise, here is the gentle correctness argument, by induction on the height of node i.

- **Base case.** If i is a leaf, it has no children, so *largest* $= i$ and the procedure does nothing. A single node has no children to violate the heap property, so it is already a valid max-heap. Correct.
- **Inductive step.** Suppose Max-Heapify is correct on every node of height less than h, and let i have height h. We pick *largest* to be the index of the maximum of $A[i]$, $A[l]$, $A[r]$. If

largest $= i$, then $A[i]$ already dominates both children; combined with the promise that the subtrees are heaps, the whole subtree at i is a heap, and we correctly stop. If instead *largest* $\neq i$, we swap, which places the true maximum at i — so the heap property now holds between i and both of its children. The swap may have lowered a value into the child at *largest*, possibly breaking the property there; but the two subtrees of *largest* were untouched and are still heaps, so the promise holds for the recursive call. That child has height $h-1 < h$, so by the inductive hypothesis the recursive call repairs it. The subtree at i is now fully a max-heap.

6.2.3 Cost of Max-Heapify

Each call does a constant amount of work — a few comparisons and at most one swap — and then either stops or recurses on a node one level down. So the total work is proportional to the number of levels the value sinks, which is at most the height h of the starting node. Since the height of any node is at most $\lfloor \log_2 n \rfloor$ (Lemma 6.1), we get the following.

Cost summary

MAX-HEAPIFY on a node of height h runs in $O(h)$ time, hence $O(\log n)$ in the worst case on an n-element heap. It uses $O(1)$ extra space if written as a loop (the recursion is tail recursion and turns into a **while** loop with no stack growth).

Remark 6.1. You can also bound the recurrence directly. A subtree of n nodes has children subtrees of at most $2n/3$ nodes each (the worst case is when the bottom level is exactly half full). So the running time obeys $T(n) \leq T(2n/3) + \Theta(1)$, which by the master theorem (case 2 with $a = 1$, $b = 3/2$) solves to $T(n) = O(\log n)$. Same answer, arrived at two ways.

Walking question

Convince yourself that MAX-HEAPIFY really only walks one root-to-leaf path and never branches sideways. Why can it never need to "go back up"? Think about what is true above i before the call and whether any swap can disturb it.

6.3 Build-Max-Heap: a heap from an unordered array

We can fix one broken spot. How do we turn a completely arbitrary array into a heap? The clean idea: call MAX-HEAPIFY on every node, but in the *right order*. MAX-HEAPIFY demands that a node's subtrees already be heaps, so we must process nodes **bottom-up** — repair the small subtrees first, then larger ones built on top of them.

Focus

One simple fact cuts our work in half. In an array of size n, the elements indexed $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \ldots, n$ are exactly the *leaves* of the tree. A leaf has no children, so it is already a one-element max-heap — nothing to fix. We only need to run MAX-HEAPIFY on the internal nodes, indices 1 through $\lfloor n/2 \rfloor$, and we sweep them in *decreasing* order so children are handled before parents.

Algorithm 15: BUILD-MAX-HEAP(A) — heapify an arbitrary array

Input: Array $A[1 \dots n]$ in any order.
Output: A rearranged in place into a max-heap.

```
1 A.heapsize ← A.length;
2 for i ← ⌊A.length/2⌋ downto 1 do
3     MAX-HEAPIFY(A, i);
```

6.3.1 A full numeric trace

Take the unordered array

$$A = [\,4,\ 1,\ 3,\ 2,\ 16,\ 9,\ 10,\ 14,\ 8,\ 7\,], \qquad n = 10.$$

The internal nodes are indices 1 through $\lfloor 10/2 \rfloor = 5$. We process $i = 5, 4, 3, 2, 1$.

$i = 5$**:** node $A[5] = 16$, children $A[10] = 7$ only ($A[11]$ doesn't exist). $16 > 7$, no swap needed.

$i = 4$**:** node $A[4] = 2$, children $A[8] = 14$, $A[9] = 8$. Largest is 14; swap. Now $A[4] = 14$, $A[8] = 2$. The new node $A[8] = 2$ is a leaf; stop.

$$A = [\,4,\ 1,\ 3,\ 14,\ 16,\ 9,\ 10,\ 2,\ 8,\ 7\,].$$

$i = 3$**:** node $A[3] = 3$, children $A[6] = 9$, $A[7] = 10$. Largest is 10; swap. $A[3] = 10$, $A[7] = 3$. Node $A[7] = 3$ is a leaf; stop.

$$A = [\,4,\ 1,\ 10,\ 14,\ 16,\ 9,\ 3,\ 2,\ 8,\ 7\,].$$

$i = 2$**:** node $A[2] = 1$, children $A[4] = 14$, $A[5] = 16$. Largest is 16; swap. $A[2] = 16$, $A[5] = 1$. Recurse on index 5: $A[5] = 1$, child $A[10] = 7$; largest is 7, swap, $A[5] = 7$, $A[10] = 1$. Node 10 is a leaf; stop.

$$A = [\,4,\ 16,\ 10,\ 14,\ 7,\ 9,\ 3,\ 2,\ 8,\ 1\,].$$

$i = 1$**:** node $A[1] = 4$, children $A[2] = 16$, $A[3] = 10$. Largest is 16; swap. $A[1] = 16$, $A[2] = 4$. Recurse on index 2: this is exactly the MAX-HEAPIFY$(A, 2)$ we traced earlier! The 4 sinks $2 \to 4 \to 9$, giving

$$A = [\,16,\ 14,\ 10,\ 8,\ 7,\ 9,\ 3,\ 2,\ 4,\ 1\,].$$

That is a valid max-heap — the very tree we drew at the start of the chapter.

6.3.2 Why it is correct: a loop invariant

Formal

Loop invariant for Build-Max-Heap. At the start of each iteration of the **for** loop with counter i, each of the nodes $i + 1, i + 2, \dots, n$ is the root of a valid max-heap.

Initialization. Before the first iteration, $i = \lfloor n/2 \rfloor$, so the nodes $i + 1, \dots, n$ are exactly the leaves. Every leaf is a one-node heap, so the invariant holds.

Maintenance. Consider an iteration with counter i. The children of i are indexed $2i$ and $2i+1$, both *strictly greater* than i, so by the invariant they are already roots of valid max-heaps. That is exactly the promise MAX-HEAPIFY(A, i) requires, so after it runs, node i is the root of a max-heap too. The higher-indexed nodes were not touched, so they remain heaps. Decrementing i reestablishes the invariant for the next iteration.

Termination. The loop ends when $i = 0$. The invariant then says nodes $1, \ldots, n$ are all roots of max-heaps. In particular node 1, the root of the whole tree, is a max-heap — which is exactly what we wanted.

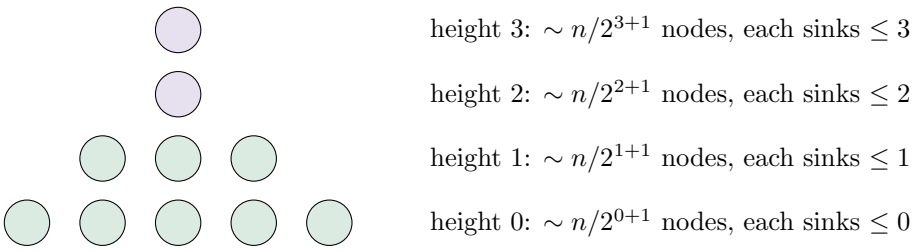
6.3.3 The surprising linear-time analysis

Here is the part people get wrong. The lazy estimate says: we call MAX-HEAPIFY on $O(n)$ nodes, and each call costs $O(\log n)$, so the total is $O(n \log n)$. That bound is *correct but loose*. The truth is better: BUILD-MAX-HEAP runs in $O(n)$ time. Let us see why.

Intuition

The $O(n \log n)$ estimate pretends every node sinks all the way down through $\log n$ levels. But most nodes are near the bottom and barely sink at all! Half the nodes are leaves (sink 0), a quarter are one level up (sink at most 1), an eighth sink at most 2, and so on. The cheap, numerous nodes vastly outnumber the expensive, rare ones near the root. Add up the real costs and the total collapses to linear.

The picture below makes the imbalance visible. Each row is a level of the tree; the rows near the bottom are crowded with cheap nodes, while the expensive nodes near the top are few. The arrow on the right of each row is the most a node on that row can sink.



This is a 10-node heap, so the counts are small; in a large heap the doubling is exact and each row down holds twice as many nodes as the one above. Notice how the green bottom rows — the cheap, numerous nodes near height 0 — dwarf the purple top rows near the root. This imbalance is why the bound is linear.

Let us count carefully. A MAX-HEAPIFY call on a node of height h costs $O(h)$. So we need to know: how many nodes have each height? In a heap with n elements, the number of nodes at height h is at most $\lceil n/2^{h+1} \rceil$.

Focus

Why $\lceil n/2^{h+1} \rceil$? Height-0 nodes are the leaves: there are about $n/2$ of them ($\lceil n/2 \rceil$ exactly). Height-1 nodes are about $n/4$, height-2 are about $n/8$. Each step up the tree halves the count,

so height-h nodes number about $n/2^{h+1}$.

The total cost of BUILD-MAX-HEAP is therefore at most

$$\sum_{h=0}^{\lfloor \log_2 n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log_2 n \rfloor} \frac{h}{2^{h+1}} \right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right).$$

Everything now hinges on that last infinite sum, $\sum_{h=0}^{\infty} h/2^h$. This is a standard series. Using the identity $\sum_{h=0}^{\infty} hx^h = x/(1-x)^2$ for $|x| < 1$, plug in $x = 1/2$:

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = \frac{1/2}{1/4} = 2.$$

The sum is a *constant*, namely 2. Therefore

$$\text{cost of BUILD-MAX-HEAP} = O(n \cdot 2) = O(n).$$

Cost summary

BUILD-MAX-HEAP converts an arbitrary n-element array into a max-heap in $\Theta(n)$ time — **linear**, not $n \log n$. It works in place, using $O(1)$ extra space.

Intuition

The intuition behind "the sum is just 2": the nodes that are expensive to sink (high up, large h) are exponentially rare, while the cheap nodes (low down, small h) are exponentially common. The geometric decay of the counts overwhelms the linear growth of the per-node cost. This tension between a linearly growing weight and a geometrically shrinking population, resolving to a constant, is a pattern you will meet again and again in algorithm analysis.

Common trap

If you build a heap by *inserting* elements one at a time instead (a method we will meet under priority queues), each insertion costs $O(\log n)$ and the total really is $\Theta(n \log n)$ in the worst case. The clever bottom-up BUILD-MAX-HEAP is faster precisely because it does most of its work near the cheap bottom of the tree. Do not confuse the two construction methods.

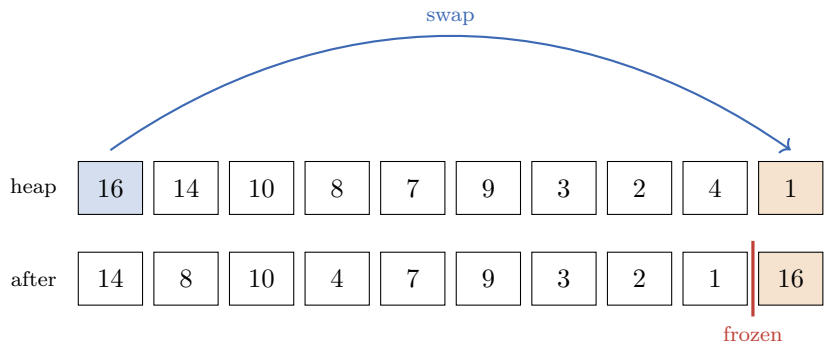
6.4 Heapsort

Now the payoff. We have a max-heap, so the largest element of the whole array is sitting right at the root, index 1. Notice where that wants to end up: in a sorted array (ascending), the largest element belongs at the *end*. So:

Intuition

The heapsort move. The maximum is at the front (index 1). The slot it ultimately belongs in is the back. So swap them: put the max where it belongs at the end, and bring the back element up to the root. Now shrink the heap by one so that the parked maximum is "out of the game," and call MAX-HEAPIFY on the root to fix the one violation we just created. The new maximum bubbles up to the root, and we repeat. Each round freezes one more element into its correct final place at the growing sorted tail.

Here is a single step as a picture. The largest element lives at the front of the heap; the sorted tail grows at the back. One step swaps them, shrinks the heap boundary leftward by one, and re-heapifies the front:



The blue root and the orange last slot trade places; the red bar then walls off the now-frozen maximum, and MAX-HEAPIFY repairs the shrunken heap on its left.

Algorithm 16: HEAPSORT(A) — sort an array ascending, in place

Input: Array $A[1 \ldots n]$ in any order.
Output: A sorted in nondecreasing order.

```
1 BUILD-MAX-HEAP(A);
2 for i ← A.length downto 2 do
3 |   exchange A[1] ↔ A[i];
4 |   A.heapsize ← A.heapsize − 1;
5 |   MAX-HEAPIFY(A, 1);
```

6.4.1 Watching it run

Start from the heap we built, $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, heap size 10. We will track the array; a vertical bar | marks the boundary between the shrinking heap (left) and the sorted tail (right).

step	array
start	16 14 10 8 7 9 3 2 4 1
swap $1 \leftrightarrow 10$, shrink, heapify	14 8 10 4 7 9 3 2 1 | 16
swap $1 \leftrightarrow 9$, shrink, heapify	10 8 9 4 7 1 3 2 | 14 16
swap $1 \leftrightarrow 8$, shrink, heapify	9 8 3 4 7 1 2 | 10 14 16
swap $1 \leftrightarrow 7$, shrink, heapify	8 7 3 4 2 1 | 9 10 14 16
swap $1 \leftrightarrow 6$, shrink, heapify	7 4 3 1 2 | 8 9 10 14 16
swap $1 \leftrightarrow 5$, shrink, heapify	4 2 3 1 | 7 8 9 10 14 16
swap $1 \leftrightarrow 4$, shrink, heapify	3 2 1 | 4 7 8 9 10 14 16
swap $1 \leftrightarrow 3$, shrink, heapify	2 1 | 3 4 7 8 9 10 14 16
swap $1 \leftrightarrow 2$, shrink, heapify	1 | 2 3 4 7 8 9 10 14 16
done	1 2 3 4 7 8 9 10 14 16

Watch the sorted tail on the right grow leftward, one frozen element per step, always pulling the current maximum out of the heap and dropping it just to the left of the previously frozen values. When the heap is down to a single element, that element is automatically the smallest, already in place, and we are done.

Algorithm lab

Trace it yourself. Take $A = [5, 13, 2, 25, 7, 17, 20, 8, 4]$ (nine elements). First run BUILD-MAX-HEAP by hand: process indices $4, 3, 2, 1$ in order, drawing the tree after each MAX-HEAPIFY. You should get a heap with 25 at the root. Then run the heapsort loop and confirm the array marches to $[2, 4, 5, 7, 8, 13, 17, 20, 25]$. Check your work by verifying after every MAX-HEAPIFY that no parent is smaller than a child — if you ever spot one, you slipped.

6.4.2 Correctness via a loop invariant

Formal

Loop invariant for the heapsort loop. At the start of each iteration with counter i:

1. the subarray $A[i+1 \ldots n]$ is sorted and contains the $n - i$ largest elements of the whole array, each in its final position; and
2. the subarray $A[1 \ldots i]$ is a valid max-heap on the current heap size i.

Initialization. Before the first iteration $i = n$, so the sorted tail $A[n+1 \ldots n]$ is empty (vacuously sorted), and $A[1 \ldots n]$ is a max-heap because BUILD-MAX-HEAP just made it one.

Maintenance. In the iteration we swap $A[1]$ (the maximum of the current heap) with $A[i]$. Because the heap held the i largest unsorted elements and $A[1]$ was their maximum, $A[1]$ is the largest element not yet placed — so moving it to position i, just before the sorted tail, keeps $A[i \ldots n]$ sorted and correct. Shrinking the heap to size $i - 1$ removes that frozen element from play. The swap put a possibly-small value at the root, breaking the heap property only at index 1; the two subtrees

are untouched and still heaps, so MAX-HEAPIFY$(A, 1)$ restores a valid max-heap on $A[1 \dots i-1]$. Decrementing i reestablishes the invariant.

Termination. The loop ends at $i = 1$. Then $A[2 \dots n]$ is sorted and holds the $n-1$ largest elements, while $A[1]$ is the single remaining smallest element. So the whole array $A[1 \dots n]$ is sorted. Done.

6.4.3 Cost of heapsort

Cost summary

HEAPSORT runs in $\Theta(n \log n)$ time in the worst case and uses $O(1)$ extra space (it sorts in place). The build phase is $\Theta(n)$; the extract phase does $n-1$ calls to MAX-HEAPIFY, each $O(\log n)$, for $O(n \log n)$. The $\Theta(n \log n)$ term dominates the $\Theta(n)$ build.

The $\Omega(n \log n)$ lower bound (so the running time is Θ, not merely O) comes from the fact that many inputs force the maximum element, after each extraction, to sink a full $\Theta(\log n)$ levels back down. We will not belabor it, but the upper and lower bounds meet at $\Theta(n \log n)$.

Why this matters

Heapsort is the algorithm you reach for when memory is precious and you must guarantee $n \log n$ behavior on *every* input. Quicksort (next chapter) is usually faster in practice on real machines because of better cache behavior, but it can degrade to $\Theta(n^2)$ on adversarial inputs. Merge sort guarantees $\Theta(n \log n)$ but needs $\Theta(n)$ scratch space. Heapsort offers both: worst-case $\Theta(n \log n)$ *and* in-place. That is why it shows up in places like the Linux kernel and in introsort (the hybrid behind many standard-library sorts), which starts with quicksort and falls back to heapsort if recursion goes too deep — using heapsort precisely as the worst-case safety net.

Common trap

Heapsort is **not stable**: equal keys can be reordered relative to their original positions, because the swaps fling elements across the array. If you need stability (e.g. sorting records by one field while preserving an earlier ordering on another), heapsort alone will not give it to you without extra tie-breaking keys.

6.5 Priority queues

Beyond sorting, a heap directly implements a **priority queue** — the role where binary heaps are used most heavily in real systems. A priority queue maintains a set S of elements, each with an associated *key* (its priority), and supports operations that always let you get at the most important element fast.

Focus

A max-priority-queue supports:

- INSERT(S, x) — add element x to the set.
- MAXIMUM(S) — return the element with the largest key.
- EXTRACT-MAX(S) — remove and return the element with the largest key.
- INCREASE-KEY(S, x, k) — raise element x's key to the new, larger value k.

A min-priority-queue is the mirror image (MINIMUM, EXTRACT-MIN, DECREASE-KEY) and is what shortest-path and minimum-spanning-tree algorithms actually use.

Why this matters

Priority queues are everywhere a system must repeatedly ask "what next?" An operating-system scheduler picks the highest-priority runnable process. A network router forwards the highest-priority packet. Event-driven simulations pull the next event by time. Dijkstra's shortest-path algorithm repeatedly extracts the closest unfinished vertex. Huffman coding repeatedly extracts the two lowest-frequency symbols. In every case the heap gives you EXTRACT and INSERT in $O(\log n)$, which is what makes those algorithms efficient.

6.5.1 Maximum and Extract-Max

Reading the maximum is trivial: it is the root.

Algorithm 17: HEAP-MAXIMUM(A)

Input: Nonempty max-heap A.
Output: The maximum element.

```
1 return A[1];
```

Removing it reuses the heapsort move: take the root, plug the last leaf into the root's slot, shrink, and heapify.

Algorithm 18: HEAP-EXTRACT-MAX(A)

Input: Max-heap A with $A.heapsize \geq 1$.
Output: The maximum element, removed from the heap.

```
1 if A.heapsize < 1 then
2     error "heap underflow";
3 max ← A[1];
4 A[1] ← A[A.heapsize];
5 A.heapsize ← A.heapsize − 1;
6 MAX-HEAPIFY(A, 1);
7 return max;
```

We save the root to return it, move the last element of the heap up to the root, shrink the heap, and let MAX-HEAPIFY sink that moved element to its proper place. One root-to-leaf path of work: $O(\log n)$.

Quick trace. On $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, HEAP-EXTRACT-MAX saves 16, moves the last element 1 into the root, shrinks to size 9, giving $[1, 14, 10, 8, 7, 9, 3, 2, 4]$, then heapifies the root: 1 sinks $1 \to 2 \to 4 \to 9$, landing among the leaves, and the array becomes the valid heap $[14, 8, 10, 4, 7, 9, 3, 2, 1]$ with 16 returned. Compare this to the first heapsort step — it is the same operation.

6.5.2 Increase-Key and Insert

The remaining operations move a value *up* the tree instead of down. If you raise a node's key, it may now exceed its parent, so it should "float up" toward the root — the opposite of sinking.

Algorithm 19: HEAP-INCREASE-KEY(A, i, key) — raise $A[i]$ and float it up

Input: Max-heap A, index i, and a new $key \geq A[i]$.
Output: Heap property restored after raising $A[i]$ to key.

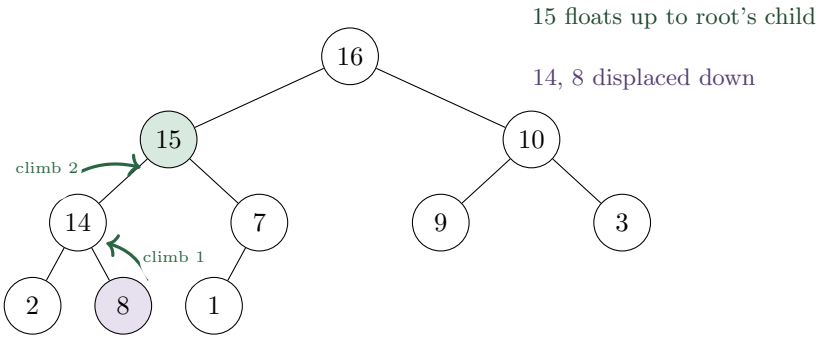
```
1 if key < A[i] then
2     error "new key smaller than current key";
3 A[i] ← key;
4 while i > 1 and A[Parent(i)] < A[i] do
5     exchange A[i] ↔ A[Parent(i)];
6     i ← Parent(i);
```

The **while** loop is the floating-up: as long as the node has a parent and is bigger than that parent, swap them and climb. It is the structural twin of MAX-HEAPIFY, walking a leaf-to-root path instead of a root-to-leaf one.

Trace. In $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, call HEAP-INCREASE-KEY$(A, 9, 15)$ — raise the value at index 9 (it is 4) up to 15. Set $A[9] = 15$. Its parent is index $\lfloor 9/2 \rfloor = 4$, value 8. Since $15 > 8$, swap: now 15 is at index 4, 8 at index 9. The parent of index 4 is index 2, value 14. Since $15 > 14$, swap: 15 at index 2, 14 at index 4. The parent of index 2 is index 1, value 16. Since $15 < 16$, stop. The 15 has floated up two levels to index 2, and the heap is valid again:

$$A = [\,16,\ 15,\ 10,\ 14,\ 7,\ 9,\ 3,\ 2,\ 8,\ 1\,].$$

The path the raised key climbs is shown below: from index 9 up through index 4 to its resting place at index 2, stopping only when it meets a parent (16) at least as large.



The green node is the raised key 15 in its final spot; the purple node holds the 8 that the climb pushed down into the vacated slot at index 9. Float-up is the exact mirror of the sink in Max-Heapify: one node per level, along a single path, comparing against just one neighbor (here the parent).

Insertion is then a small trick: append a sentinel of $-\infty$ as a new leaf (growing the heap), then immediately Increase-Key it up to the real value, which floats it to the correct spot.

Algorithm 20: Max-Heap-Insert(A, key)

Input: Max-heap A and a *key* to insert.
Output: A with *key* added, heap property intact.

```
1 A.heapsize ← A.heapsize + 1;
2 A[A.heapsize] ← −∞;
3 Heap-Increase-Key(A, A.heapsize, key);
```

Intuition

Max-Heapify sinks values *down* (used by extract and build); Heap-Increase-Key floats values *up* (used by insert and key raises). Down-heap walks parent-to-children comparing against the larger child; up-heap walks child-to-parent comparing against the one parent. Both operations rely on the same local repair: fix a single violation by swapping along the one path where that change can propagate.

Cost summary

On an n-element heap, all priority-queue operations are fast:

Heap-Maximum	$\Theta(1)$
Heap-Extract-Max	$O(\log n)$
Heap-Increase-Key	$O(\log n)$
Max-Heap-Insert	$O(\log n)$
Build-Max-Heap	$\Theta(n)$

because each path-following operation does $O(1)$ work per level and there are $O(\log n)$ levels.

Common trap

A heap is a *terrible* data structure if your main job is to *search* for an arbitrary element or to find the *minimum* of a max-heap. Finding a specific value, or the smallest value, can require scanning $\Theta(n)$ elements, because the heap only promises fast access to the *one* extreme at the root. For two-ended access (both min and max fast), or for searching, reach for a balanced search tree instead. Likewise, INCREASE-KEY needs you to *already know the index i* of the element. Real implementations keep a side table mapping elements to their current heap positions, kept up to date on every swap, so that algorithms like Dijkstra can find the node to update.

History and connection

In 1964, British computer scientist J. W. J. Williams published Algorithm 232 in the *Communications of the ACM*, introducing both the binary heap and heapsort. At the time, programmers faced an unsatisfying dilemma: insertion sort was simple and worked in-place but crawled in $\Theta(n^2)$ time; merge sort achieved $\Theta(n \log n)$ speed but demanded a second auxiliary array the size of the entire input; and quicksort was fast on average but risked catastrophic quadratic slowdowns on unfavorable data. Williams saw that a complete binary tree could be embedded directly into a contiguous array without storing a single pointer: the children of node i live at $2i$ and $2i + 1$, and the parent lives at $\lfloor i/2 \rfloor$. Later that same year, Robert W. Floyd published Algorithm 245 (*Treesort 3*), showing that an unordered array can be turned into a heap in strictly *linear* time ($\Theta(n)$) by sifting down from the leaves up, rather than inserting elements one by one.

When heapsort first appeared, many theorists believed it would make Tony Hoare's quicksort obsolete. On paper, heapsort was strictly superior: it guaranteed $\Theta(n \log n)$ performance in the absolute worst case, used $\Theta(1)$ auxiliary space, and had no vulnerable pivot choices. Yet in practice on actual computers, quicksort consistently ran twice as fast! The explanation lay in computer hardware: cache locality. While quicksort sweeps through contiguous array elements sequentially, heapsort constantly leaps across memory addresses — jumping from index i to $2i$, $4i$, and $8i$ — which causes modern CPU cache lines to miss repeatedly. Extracting the maximum also repeatedly moves an arbitrary leaf to the root and sifts it all the way back down, performing roughly twice as many key comparisons as quicksort on average.

More broadly, Williams's heap introduced the *priority queue*: a dynamic data structure supporting insertion, maximum extraction, and priority updates in $O(\log n)$ time, which now drives operating system task schedulers, network packet buffers, and discrete-event simulations. In Chapter 7, we investigate Quicksort to see why its sequential memory access often matters more than its worse worst case. Later, in Chapter 20, we explore Fibonacci Heaps, which accelerate key decreases to $O(1)$ amortized time to speed up Prim's Minimum Spanning Tree algorithm (Chapter 23) and Dijkstra's Shortest Paths algorithm (Chapter 24).

6.6 Putting it together

Pause

Let us collect the whole chapter in one place. A **heap** is a complete binary tree stored implicitly in an array, where every parent dominates its children (max-heap). Because the tree has no gaps, children live at indices $2i$ and $2i+1$ and parents at $\lfloor i/2 \rfloor$ — no pointers needed. The tree has height $\Theta(\log n)$.
The one primitive that does all the work downward is **Max-Heapify**, which sinks an out-of-place value along a single root-to-leaf path in $O(\log n)$. Sweeping MAX-HEAPIFY bottom-up over the internal nodes gives **Build-Max-Heap**, which — thanks to the geometric series in the analysis — runs in $\Theta(n)$.
From there, **heapsort** repeatedly swaps the root maximum to the end, shrinks the heap, and re-heapifies, sorting in place in $\Theta(n \log n)$. And the same machinery, plus an upward float (**Heap-Increase-Key**), gives a **priority queue** with $O(\log n)$ insert, extract-max, and key changes — used inside schedulers, simulations, Dijkstra, Prim, and Huffman.

Walking question

Here is a question to chew on during a walk. Heapsort and merge sort are both $\Theta(n \log n)$, yet they feel completely different — one needs a second array, the other works in a single array. Merge sort builds order by *combining* pieces; heapsort builds order by repeatedly *pulling out the extreme*. Are these two faces of one deeper idea, or genuinely different strategies? And where would quicksort, which you are about to meet, fit on that map?

6.7 Exercises

Exercise 6.1. Using 1-based indexing, what are the indices of the parent, left child, and right child of the node at index 13? Of index 1? Which indices in a heap of size $n = 20$ are leaves?

Exercise 6.2. Is the array $[23, 17, 14, 6, 13, 10, 1, 5, 7, 12]$ a max-heap? If not, identify every parent-child pair that violates the heap property, then run BUILD-MAX-HEAP on it by hand and show the resulting heap as both an array and a tree.

Exercise 6.3. Trace MAX-HEAPIFY$(A, 3)$ on $A = [27, 17, 3, 16, 13,\ 10, 1, 5, 7, 12,\ 4, 8, 9, 0]$ with $\mathit{heapsize} = 14$. Write the array after each swap, and state which index the displaced value finally lands at.

Exercise 6.4. Run the full HEAPSORT on $A = [5, 3, 17, 10, 84, 19, 6, 22, 9]$. Show the heap after BUILD-MAX-HEAP, then the array after each iteration of the extraction loop, marking the boundary between the heap and the sorted tail.

Exercise 6.5. Explain in one or two careful sentences why the elements at indices $\lfloor n/2 \rfloor + 1$ through n are exactly the leaves of a heap with n elements. *Hint:* a node i is internal exactly when it has a left child, i.e. when $2i \le n$.

Exercise 6.6. The text proves $\sum_{h=0}^{\infty} h/2^h = 2$ using a closed-form identity. Give an alternative derivation: let $S = \sum_{h \ge 1} h/2^h$, write down $2S$, subtract S from $2S$, and recognize the resulting series as a plain geometric series. Confirm you get $S = 2$.

Exercise 6.7. Show how to implement MAX-HEAPIFY as an iterative **while** loop with no recursion and $O(1)$ extra space. Argue that it visits exactly the same sequence of indices as the recursive version.

Exercise 6.8. Describe how to adapt every operation in this chapter to a *min*-heap and a *min*-priority queue (HEAP-MINIMUM, HEAP-EXTRACT-MIN, and HEAP-DECREASE-KEY). What exactly changes in MAX-HEAPIFY? Write the pseudocode for MIN-HEAPIFY.

Exercise 6.9. Give an array of 7 distinct numbers that is simultaneously a valid max-heap and *not* sorted in descending order, demonstrating concretely that "heap" and "sorted" are different properties. Then explain why the root nonetheless always holds the maximum.

Exercise 6.10 (Challenge)**.** You are given k sorted lists containing n elements in total, and you want to merge them into one sorted list. Describe an $O(n \log k)$ algorithm using a min-heap of size k. Explain why the heap holds at most k elements at any time and why each of the n output steps costs $O(\log k)$.

Exercise 6.11 (Challenge)**.** The operation HEAP-DELETE(A, i) removes the element at an arbitrary index i from a max-heap. Design it to run in $O(\log n)$ time. *Careful:* after you move the last element into slot i, the new value might be too big *or* too small for that position — so a single MAX-HEAPIFY may not suffice. Explain what you must check and write the pseudocode.

Exercise 6.12 (Challenge)**.** Argue that heapsort is not stable by exhibiting a small input of records with equal keys whose relative order is reversed by the algorithm. Then describe a general fix that makes *any* comparison sort stable by augmenting the keys, and state the extra space it costs.

Chapter 7

Quicksort

Chapter 6 ended with a question on how sorting strategies divide the territory: merge sort builds order by combining sorted halves, while heapsort builds order by repeatedly extracting the maximum element. Quicksort stakes out a third position. Instead of spending effort on combining or extracting, it does all its work during the split: pick one element, rearrange the array around it in place, and leave the combine step completely free.

On paper, this algorithm has an alarming worst case: quadratic, $\Theta(n^2)$, the same as insertion sort on a reversed list. And yet on real hardware, quicksort usually outpaces heapsort and merge sort. Standard libraries choose it as their default sort because its sequential memory sweeps keep CPU caches warm. This chapter explains how an algorithm with a quadratic worst case earns the name "quick."

7.1 The shape of the idea

Start with the picture, before any code. Suppose someone hands you an array of numbers and asks you to sort it. Quicksort's strategy is simple. Pick one element. Call it the *pivot*. Now shove everything smaller than the pivot to its left and everything larger to its right. The pivot has now landed in its *final, sorted position* – it will never move again. Then recursively do the same thing to the left chunk and the right chunk.

Intuition

Imagine you are sorting a deck of cards by the value 7. You sweep through the deck and toss every card below 7 into a left pile and every card above 7 into a right pile, then set the 7 down in the middle. The 7 is now exactly where it belongs in the final sorted deck: every card to its left is smaller, every card to its right is larger. You have not finished sorting – the two piles are still messes – but you have made real progress, because that one card is locked in. Repeat on each pile. When the piles shrink to a single card, everything is sorted.

This is divide and conquer, but with a twist. In merge sort (Chapter 4) the *divide* step is trivial – you just cut the array in half – and all the real work happens in the *combine* step, where you merge two sorted halves. Quicksort flips this. The divide step, called *partitioning*, does all the work: it rearranges the elements so that the pivot is correctly placed. And the combine step is *nothing at*

all. Once the two halves are sorted, the whole array is already sorted, because everything left of the pivot is smaller than everything right of it. There is no merge.

Focus

Hold onto this contrast. Merge sort: easy split, hard combine. Quicksort: hard split (partition), free combine. Both are $\Theta(n \log n)$ on average, but they spend their effort in opposite places. Quicksort's free combine is exactly why it sorts *in place* – no scratch array needed.

Here is the top-level routine. It is short – almost all the work happens inside PARTITION, which we have not written yet.

Algorithm 21: QUICKSORT(A, p, r)

Input: Array A, and indices $p \le r$ marking the subarray $A[p\,..\,r]$.
Output: $A[p\,..\,r]$ rearranged into nondecreasing order.

```
1 if p < r then
2     q ← PARTITION(A, p, r) ;          // pivot lands at index q
3     QUICKSORT(A, p, q − 1) ;           // sort the left part
4     QUICKSORT(A, q + 1, r) ;           // sort the right part
```

To sort an entire array A of length n (with indices 1 through n), you call QUICKSORT$(A, 1, n)$. The guard $p < r$ handles the base case: a subarray of size 0 or 1 ($p \ge r$) is already sorted, so we do nothing. Everything now rides on PARTITION, so we build it next.

7.2 Lomuto partitioning, built by hand

There are two famous ways to partition. We will start with *Lomuto's* scheme, named for Nico Lomuto, because it is the easier of the two to reason about. It is also the version CLRS presents first.

The plan: we choose the *last* element $A[r]$ as the pivot. We then walk a finger from left to right across the subarray, maintaining a growing region of "small" elements at the front. To make the proof painless later, be precise about what that region promises at every step.

Intuition

Picture the subarray split into four zones as you scan. From left to right: (1) a region of elements known to be $\le$ pivot; (2) a region of elements known to be $>$ pivot; (3) elements you have not looked at yet; (4) the pivot itself, sitting at the far right. Your scanning finger j marks the boundary between "looked at" and "not yet looked at." A second marker i tracks the end of the small region. Each step you examine $A[j]$ and decide which of the first two zones it belongs to. Figure 7.1 freezes this picture at one moment in the middle of the scan.

Here is the code. Read it once, then we will trace it.

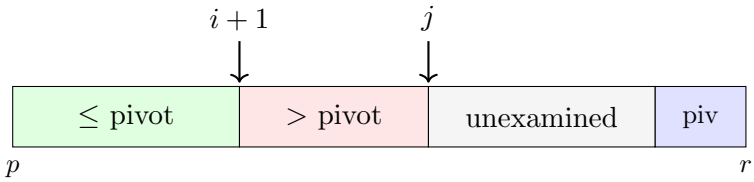


Figure 7.1: The Lomuto invariant. Everything before index $i+1$ is $\leq$ the pivot; everything from $i+1$ up to $j-1$ is $>$ the pivot; $A[j\,..\,r-1]$ is not yet examined; $A[r]$ is the pivot.

Algorithm 22: PARTITION(A, p, r) – Lomuto scheme

Input: Subarray $A[p\,..\,r]$ with $p \leq r$.
Output: Returns an index q; afterward $A[p\,..\,q-1] \leq A[q] < A[q+1\,..\,r]$.

```
1 x ← A[r] ;                                   // the pivot is the last element
2 i ← p − 1 ;                                  // i = end of the ‘‘≤ pivot’’ region
3 for j ← p to r − 1 do
4     if A[j] ≤ x then
5         i ← i + 1;
6         exchange A[i] with A[j];
7 exchange A[i + 1] with A[r] ;                // drop the pivot into place
8 return i + 1;
```

7.2.1 A complete numeric trace

Let us partition the subarray

$$A = [\,2,\ 8,\ 7,\ 1,\ 3,\ 5,\ 6,\ 4\,],$$

with $p = 1$, $r = 8$. The pivot is $x = A[8] = 4$. We start with $i = 0$ (meaning "the small region is empty"). We will show the array after each iteration of j, marking the pivot in **bold** and using a vertical bar to show where i sits (everything up to and including position i is the $\leq$-pivot region).

Algorithm lab

Trace it with me, but also keep a finger on the page and predict each step before you read it. The mechanical rule is: *whenever $A[j] \leq 4$, bump i up by one and swap $A[i]$ with $A[j]$; otherwise do nothing and move on.*

step	j, $A[j]$, decision	array (bar after position i)
start	—	| 2 8 7 1 3 5 6 **4**
$j = 1$	$A[1] = 2 \le 4$: i=1, swap $A[1], A[1]$	2 | 8 7 1 3 5 6 **4**
$j = 2$	$A[2] = 8 > 4$: skip	2 | 8 7 1 3 5 6 **4**
$j = 3$	$A[3] = 7 > 4$: skip	2 | 8 7 1 3 5 6 **4**
$j = 4$	$A[4] = 1 \le 4$: i=2, swap $A[2], A[4]$	2 1 | 7 8 3 5 6 **4**
$j = 5$	$A[5] = 3 \le 4$: i=3, swap $A[3], A[5]$	2 1 3 | 8 7 5 6 **4**
$j = 6$	$A[6] = 5 > 4$: skip	2 1 3 | 8 7 5 6 **4**
$j = 7$	$A[7] = 6 > 4$: skip	2 1 3 | 8 7 5 6 **4**
final swap	swap $A[i+1]$=$A[4]$ with $A[8]$	2 1 3 **4** | 8 7 5 6

The function returns $q = i + 1 = 4$. Look at the result: $[2, 1, 3]$ are all ≤ 4, the pivot 4 sits at index 4, and $[8, 7, 5, 6]$ are all > 4. The pivot is home. Quicksort now recurses on $[2, 1, 3]$ and $[8, 7, 5, 6]$.

Figure 7.2 freezes this execution at each critical swap, showing how the colored regions expand and how the boundary pointer i tags along.

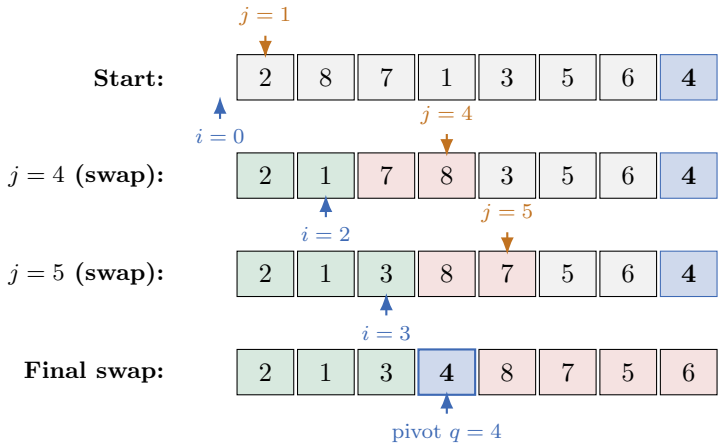


Figure 7.2: Key snapshots during the Lomuto partition of $[2, 8, 7, 1, 3, 5, 6, 4]$. Green cells form the $\le x$ region ending at index i, red cells form the $> x$ region up to $j - 1$, gray cells are unexamined, and blue is the pivot. At the end, the pivot swaps into index $q = i + 1 = 4$.

In Figure 7.2, notice the two swaps that actually *moved* things: at $j = 4$ the value 1 (small) traded places with 8 (large), and at $j = 5$ the value 3 (small) traded with the 7 that 8 had been swapped past. Each time we find a small element, we swap it into the small region, pushing one large element out into the large region. The pattern is: *small elements bubble forward, large elements get shoved back,* and at the very end the pivot jumps over the entire large region to land right after the small region.

7.2.2 Why it is correct: a loop invariant

We claimed the output satisfies $A[p\,..\,q-1] \le A[q] < A[q+1\,..\,r]$. The gentle way to prove it is with a loop invariant: a statement that is true before the loop starts and stays true after every iteration.

If it survives to the end, it pins down the final state.

Formal

Loop invariant (Lomuto). At the start of each iteration of the **for** loop with loop variable j, the following hold:

1. For every index k with $p \le k \le i$, we have $A[k] \le x$.
2. For every index k with $i+1 \le k \le j-1$, we have $A[k] > x$.
3. $A[r] = x$ (the pivot has not moved).

In words: everything in the small region is $\le$ the pivot, everything in the large region is $>$ the pivot, and the pivot is still parked at the right end.

Lemma 7.1 (Partition correctness)**.** *After* PARTITION(A, p, r) *returns* $q = i+1$*, every element of* $A[p\,..\,q-1]$ *is* $\le A[q]$ *and every element of* $A[q+1\,..\,r]$ *is* $> A[q]$*, where* $A[q]$ *equals the original pivot value* x*.*

Proof. We verify the invariant in three steps.

Initialization. Before the first iteration, $j = p$ and $i = p-1$. Region 1 spans $p \le k \le p-1$, which is empty, so condition 1 holds vacuously. Region 2 spans $p \le k \le p-1$, also empty, so condition 2 holds vacuously. And $A[r] = x$ because we have not touched it. The invariant holds.

Maintenance. Suppose the invariant holds at the start of an iteration with loop variable j. We look at $A[j]$.

- If $A[j] > x$: we only advance j. The element at the old position j now falls into region 2 (indices $i+1$ through the new $j-1$), and it satisfies $A[j] > x$, so condition 2 still holds. Conditions 1 and 3 are untouched.
- If $A[j] \le x$: we increment i and swap $A[i]$ with $A[j]$. The slot $A[i]$ used to be the first element of region 2, so it held a value $> x$; now it receives the value $A[j] \le x$, extending region 1 correctly. The displaced large value lands at the old position j, which becomes the new tail of region 2 as j advances, so condition 2 still holds. Condition 3 is untouched because $j \le r-1 < r$, so we never swap the pivot.

In both cases the invariant survives.

Termination. The loop ends when $j = r$. By condition 1, all of $A[p\,..\,i]$ is $\le x$; by condition 2, all of $A[i+1\,..\,r-1]$ is $> x$. The final swap exchanges $A[i+1]$ with $A[r] = x$. (When the large region is nonempty, $A[i+1]$ held a value $> x$; when it is empty, $i+1 = r$ and the swap is harmless.) After it, $A[i+1] = x$, the values in $A[p\,..\,i]$ are $\le x$, and the values in $A[i+2\,..\,r]$ are $> x$ (the displaced large value joined them). Returning $q = i+1$ makes the claim exact. □

Common trap

A subtle point that trips people up: the inequality on the left is $\le$, not $<$. Elements *equal* to the pivot go to the *left*. That is fine for correctness, but it matters a great deal for performance. If your array is all duplicates – say n copies of the number 5 – Lomuto sends *every* element into the left region, so the partition is maximally unbalanced ($n-1$ on the left, 0 on the

right) every single time, triggering the $\Theta(n^2)$ worst case. We will fix this danger later with randomization, and we will see that Hoare's scheme handles equal keys more gracefully.

7.3 The two recursion patterns: best and worst case

Now that partition is trustworthy, the running time of quicksort is decided entirely by *how balanced the splits are*. Partitioning a subarray of size n costs $\Theta(n)$ – one scan, a constant amount of work per element. The question is what sizes the two recursive calls get.

7.3.1 The worst case: maximally lopsided splits

Suppose every single partition is as unbalanced as possible: the pivot is always the smallest (or largest) element, so one side gets $n-1$ elements and the other gets 0. When does this happen? Precisely when the array is already sorted (or reverse sorted) and we keep picking the last element as the pivot. Then the recurrence for the running time $T(n)$ is

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n).$$

This unrolls into a sum (the chain in Figure 7.3):

$$T(n) = \Theta(n) + \Theta(n-1) + \cdots + \Theta(2) + \Theta(1) = \Theta\left(\sum_{k=1}^{n} k\right) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

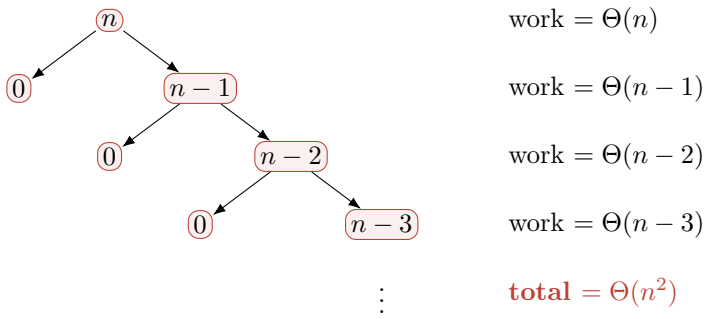


Figure 7.3: The worst-case recursion "tree" for quicksort is really a chain of depth n. Each level removes one element and does linear work on the rest, so the total work is $n+(n-1)+\cdots+1 = \Theta(n^2)$.

So the worst case is genuinely $\Theta(n^2)$. This is no better than insertion sort. If that were the whole story, nobody would use quicksort.

7.3.2 The best case: perfectly even splits

Now suppose we get the opposite luck: every partition splits the subarray into two halves of size $n/2$ (give or take one for the pivot). Then

$$T(n) = 2\,T(n/2) + \Theta(n).$$

This is the merge-sort recurrence, and by the master theorem (Chapter 4) its solution is $T(n) = \Theta(n \log n)$. The recursion tree has depth $\log_2 n$, each level does $\Theta(n)$ total work, and $\log_2 n$ levels times $\Theta(n)$ gives $\Theta(n \log n)$. Figure 7.4 shows why the work per level stays flat at $\Theta(n)$ even as the tree fans out.

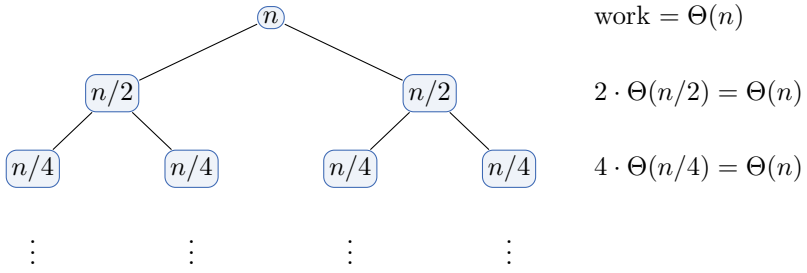


Figure 7.4: The best-case recursion tree. Each level holds subproblems whose sizes sum to n, so each level costs $\Theta(n)$. Balanced halving makes the tree only $\log_2 n$ levels deep, for a total of $\Theta(n \log n)$. Compare the worst-case chain in Figure 7.3, which is n levels deep.

Intuition

Why does balance matter so much? Think about *depth*. Every level of recursion does about $\Theta(n)$ work in total (the partitions at that level collectively scan roughly all n elements). So the running time is essentially $\Theta(n)$ times the *number of levels*. Balanced splits halve the problem each level, giving only $\log_2 n$ levels. Lopsided splits shave off one element per level, giving n levels. That is the entire difference between $n \log n$ and n^2: how fast the subproblems shrink.

7.3.3 The reassuring middle: even bad balance is fine

The following fact explains why quicksort works well in practice. You do not need *perfect* balance to get $\Theta(n \log n)$. You just need the split ratio to be bounded away from the extreme. Suppose every partition is a lopsided 9-to-1 split – that sounds awful – so that

$$T(n) = T\left(\tfrac{9n}{10}\right) + T\left(\tfrac{n}{10}\right) + \Theta(n).$$

Draw the recursion tree (Figure 7.5). The shallowest leaf is reached by repeatedly taking the $\frac{1}{10}$ branch, at depth $\log_{10} n$. The deepest leaf is reached by repeatedly taking the $\frac{9}{10}$ branch, at depth $\log_{10/9} n$. Every *full* level does at most $\Theta(n)$ work, and there are at most $\log_{10/9} n = \Theta(\log n)$ levels before everything bottoms out. So the total is at most $\Theta(n \log n)$. A 99-to-1 split would still give $\Theta(n \log n)$; only the constant inside the log changes.

Focus

The lesson: quicksort is $\Theta(n \log n)$ unless the splits are *consistently, catastrophically* lopsided – close to 0-to-n almost every time. A merely "unlucky" split here and there costs you nothing asymptotically. This robustness is why randomization works so well for quicksort: random pivots make consistently bad splits extremely unlikely.

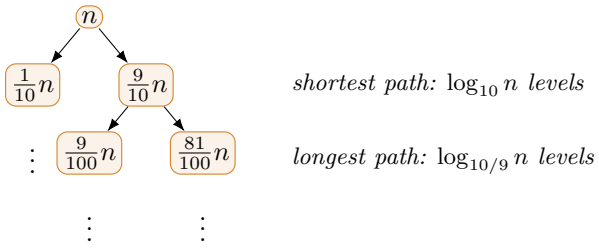


Figure 7.5: A consistently 9-to-1 recursion tree. The lightly loaded $\frac{1}{10}$ branches bottom out quickly ($\log_{10} n$ deep); the heavy $\frac{9}{10}$ branches form the longest root-to-leaf path ($\log_{10/9} n$ deep). Both depths are $\Theta(\log n)$, and each level still costs $\Theta(n)$, so the total stays $\Theta(n \log n)$.

7.4 Defeating worst-case inputs with randomization

So far we have chosen the pivot as $A[r]$, the last element. That choice has a fatal flaw: it makes the running time depend on the *arrangement of the input*. An already-sorted array – one of the most common inputs in the real world – triggers the $\Theta(n^2)$ worst case every time. Imagine a database that re-sorts an already-sorted log file and grinds to a halt.

Common trap

The naive fix people reach for first is "pick the middle element" or "pick the first element." These help against sorted inputs but do not solve the problem: for *any* fixed deterministic rule for choosing the pivot, an adversary who knows your rule can construct an input that forces the worst case. There is always a bad input. The only escape is to stop letting the input decide.

The cure is a classic move in algorithm design: make the pivot *random*. Instead of always taking $A[r]$, pick a uniformly random index between p and r, swap that element into position r, and proceed exactly as before. The partition logic is unchanged; only the pivot's identity is now a coin flip.

Algorithm 23: RANDOMIZED-PARTITION(A, p, r)

Input: Subarray $A[p\,..\,r]$.
Output: A partition around a uniformly random pivot; returns the pivot index.

```
1 s ← RANDOM(p, r) ;                          // uniform in {p, ..., r}
2 exchange A[r] with A[s] ;                   // move random pivot to the end
3 return PARTITION(A, p, r);
```

Algorithm 24: RANDOMIZED-QUICKSORT(A, p, r)

```
1 if p < r then
2     q ← RANDOMIZED-PARTITION(A, p, r);
3     RANDOMIZED-QUICKSORT(A, p, q − 1);
4     RANDOMIZED-QUICKSORT(A, q + 1, r);
```

Why this matters

This shifts what "worst case" even means. With deterministic quicksort, the worst case is triggered by specific *inputs*, such as an already-sorted array. With randomized quicksort, *no input is reliably bad*. For any fixed array, the running time becomes a random variable, and we will prove its *expected* value is $\Theta(n \log n)$ regardless of how the input was arranged. The randomness lives in the algorithm's coin flips, not in the data. An adversary can choose the array but cannot see your coins. The same trick – randomize to defeat worst-case inputs – shows up all over computer science: in hashing (Chapter 11), in load balancing, in cryptography.

History and connection

Between 1959 and 1961, a young British postgraduate named C.A.R. (Tony) Hoare was studying at Moscow State University, working on machine translation of Russian scientific articles into English. The practical problem was severe: Russian sentences were streamed off magnetic tape on an Elliott 803 computer with just 4,096 words of core memory. To look up each word in a Russian-English dictionary stored on a second tape reel, the words in each sentence needed to be alphabetized first; otherwise, the computer spent minutes endlessly rewinding magnetic tape spools. Frustrated by the slowness of insertion sort and Shell sort, Hoare conceived the partition idea: pick an arbitrary pivot, swap elements smaller than the pivot to the left and larger to the right so the pivot lands in its permanent home, and recurse on both sides. When Hoare returned to the UK and excitedly showed the algorithm to his colleagues, they told him it was impossible to implement because Elliott 803 Autocode did not support recursion! Hoare had to wait until Algol 60 introduced recursive subroutines to publish his discovery in 1961 (Algorithm 64 in *Communications of the ACM*), which eventually earned him the Turing Award.

Quicksort's history includes several well-known pitfalls. The most obvious pivot choice is the first or last element of the array. But on an array that is *already sorted* – precisely the condition most commonly encountered when refreshing database tables – picking an endpoint pivot produces maximally lopsided splits of 0 and $n - 1$ elements. Instead of halving the problem, the recursion tree becomes a chain of depth n, the running time degrades to $\Theta(n^2)$, and the deep recursion can overflow the call stack. Nico Lomuto's elegant one-finger partition scheme, while simple to teach, degrades to quadratic time when all elements are identical. Hoare's original bidirectional two-finger scheme handles duplicate keys far better and executes roughly three times fewer swaps on average, while modern implementations rely on Dutch-national-flag three-way partitioning and median-of-three pivot heuristics.

The result was one of the fastest general-purpose comparison sorts in practice. Because its combine step is completely free, quicksort sorts in-place, and its sequential memory sweeps keep CPU cache lines continuously filled, usually beating heapsort and merge sort on real hardware. Today, quicksort variants are the default sorts in the C standard library (`qsort`), C++ (`std::sort`), and Java. It still has real downsides: quicksort is inherently unstable (equal keys can swap relative positions), and an adversarial input can theoretically trigger quadratic time unless protected by random coin flips or an Introsort fallback to heapsort. In Chapter 8, we discover how counting sort and radix sort avoid comparisons entirely to achieve $\Theta(n)$ linear time, and in Chapter 9, we adapt quicksort's partition step into Quickselect to find medians and percentiles in expected linear time.

7.5 Expected running time

We now prove the central theorem. This is the analysis the chapter has been building toward.

Theorem 7.1 (Expected time of randomized quicksort)**.** *On any input of n distinct elements,* RANDOMIZED-QUICKSORT *runs in expected time $O(n \log n)$, where the expectation is over the random choices of pivots.*

The whole running time of quicksort is dominated by one operation: *comparing an element to a pivot* inside PARTITION. Every other operation (swaps, index bookkeeping, recursive call overhead) is bounded by a constant times the number of partition calls, and there are at most n partition calls total. So bounding the *total number of comparisons* X across the entire run bounds the running time, which is $O(n + X)$.

Focus

The entire analysis reduces to one question: *across the whole sort, how many times does the algorithm compare two elements?* Count the comparisons and you have the running time. We will count them with a technique called *indicator random variables*.

7.5.1 The key realization: each pair is compared at most once

Rename the elements by their *sorted order*, not their positions in the input. Let $z_1 < z_2 < \cdots < z_n$ be the elements in sorted order, so z_i is the i-th smallest. Define $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$, the set of elements whose sorted ranks fall between i and j inclusive.

Intuition

The key observation is this. When are two elements z_i and z_j ever *directly compared*? Comparisons only happen between a pivot and a non-pivot. So z_i and z_j are compared if and only if one of them is chosen as a pivot *while they are still in the same subarray*. Once two elements are compared, one of them was the pivot, which then leaves the recursion – so **any pair is compared at most once** in the entire run.

Now ask *which* element, chosen as a pivot, would either compare z_i with z_j or else separate them forever. Consider the set Z_{ij}. As long as no element of Z_{ij} has been chosen as a pivot, all of Z_{ij} sits together in one subarray (because any pivot *outside* Z_{ij} is either smaller than every z in Z_{ij}, sending them all right, or larger than every one, sending them all left – never splitting them). The first pivot chosen *from within* Z_{ij} decides their fate:

- If that first pivot is z_i or z_j itself, then z_i and z_j *are* compared (the pivot is compared against every other element in the subarray, including the other one).
- If that first pivot is some *middle* element z_k with $i < k < j$, then z_i goes to the left subarray and z_j goes to the right subarray. They are now separated forever and will *never* be compared.

So z_i and z_j are compared if and only if the *first* element chosen as a pivot from the set Z_{ij} is either z_i or z_j. Figure 7.6 lays out the three cases on a number line of sorted ranks.

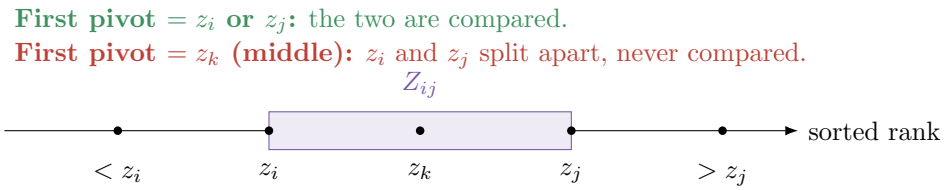


Figure 7.6: The fate of the pair (z_i, z_j) is decided by the first pivot drawn from the contiguous rank-block Z_{ij}. A pivot from *outside* the block (rank $< z_i$ or $> z_j$) pushes the whole block to one side without splitting it, so it never decides anything. An endpoint pivot compares the pair; a strictly interior pivot z_k separates them forever.

Walking question

Before reading the probability calculation, take this on a walk. The set Z_{ij} has exactly $j-i+1$ elements. Each pivot is chosen uniformly at random, so before any of Z_{ij} has been a pivot, each of its $j-i+1$ members is *equally likely* to be the first one chosen from the set. We want the chance that the first one is one of the *two* endpoints, z_i or z_j. Two favorable outcomes out of $j-i+1$ equally likely ones. Can you guess the probability before we derive it?

7.5.2 Indicator variables and the count

Define the indicator random variable

$$X_{ij} = \mathbb{I}\{z_i \text{ is compared with } z_j \text{ at some point during the sort}\} = \begin{cases} 1 & \text{if they are compared,} \\ 0 & \text{otherwise.} \end{cases}$$

Since every pair is compared at most once, the total number of comparisons is exactly

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}.$$

We want $\mathbb{E}[X]$, the expected number of comparisons. By *linearity of expectation* – which holds whether or not the variables are independent –

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \mathbb{E}[X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \Pr[z_i \text{ compared with } z_j].$$

The last equality is the defining property of an indicator: its expectation equals the probability of the event it indicates.

Now we compute that probability using the realization above. Among the $j-i+1$ elements of Z_{ij}, each is equally likely to be the first chosen as a pivot. The comparison happens exactly when that first pivot is z_i or z_j:

$$\Pr[z_i \text{ compared with } z_j] = \frac{2}{j-i+1}.$$

Formal

Why the probability is $\frac{2}{j-i+1}$. Restrict attention to the moment when the first pivot *from* Z_{ij} is selected. By symmetry of the uniform random pivot choice, each of the $j-i+1$ elements of Z_{ij} is equally likely to be that first one, each with probability $\frac{1}{j-i+1}$. The event "compared" is the union of the two disjoint events "z_i is first" and "z_j is first," so its probability is $\frac{1}{j-i+1} + \frac{1}{j-i+1} = \frac{2}{j-i+1}$.

7.5.3 Summing the series

Substitute, and let $k = j - i$ so that as j ranges from $i+1$ to n, k ranges from 1 to $n-i$:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k}.$$

The inner sum $\sum_{k=1}^{n} \frac{1}{k}$ is the n-th *harmonic number* H_n, and a fact we will lean on (proved in the summations appendix, and in CLRS) is that $H_n = \ln n + O(1) = O(\log n)$. Therefore

$$\mathbb{E}[X] < \sum_{i=1}^{n-1} 2H_n = \sum_{i=1}^{n-1} O(\log n) = O(n \log n).$$

Since the running time is $O(n + X)$ and $\mathbb{E}[X] = O(n \log n)$, the expected running time is $O(n \log n)$. This proves Theorem 7.1. □

Intuition

We did not solve a recurrence or draw a recursion tree. We *counted comparisons one pair at a time*, and the only thing we needed about each pair was the chance that the two ever meet. Distant pairs (large $j-i$) almost never meet – the probability $\frac{2}{j-i+1}$ shrinks toward zero – while adjacent pairs (small $j-i$) meet often. Summing those probabilities gives the harmonic series, which is $\Theta(\log n)$. That logarithm, which comes from the harmonic numbers, is why quicksort is fast.

Remark 7.1. Why the harmonic number? Because the probability $\frac{2}{j-i+1}$ depends only on the *gap* $j-i$ between the ranks. For each fixed gap d, there are about n pairs at that gap, each contributing $\frac{2}{d+1}$, giving roughly $\frac{2n}{d+1}$. Summing over gaps $d = 1, 2, \ldots, n-1$ produces $2n$ times the harmonic series – exactly the $\Theta(n \log n)$ we found.

7.6 Hoare partitioning: the original, faster scheme

Lomuto is easy to reason about, but Hoare's original 1961 partition is typically *faster in practice* because it does fewer swaps – on average about three times fewer. It works by sending two fingers toward each other from the two ends, swapping out-of-place pairs as they meet.

Algorithm 25: HOARE-PARTITION(A, p, r)

Input: Subarray $A[p\,..\,r]$ with $p < r$.
Output: An index $q \in \{p, \ldots, r-1\}$ splitting A so that $A[p\,..\,q]$ all $\leq A[q+1\,..\,r]$.

```
1  x ← A[p] ;                                   // pivot = first element
2  i ← p − 1;
3  j ← r + 1;
4  while true do
5      repeat
6          j ← j − 1
7      until A[j] ≤ x;
8      repeat
9          i ← i + 1
10     until A[i] ≥ x;
11     if i < j then
12         exchange A[i] with A[j];
13     else
14         return j;
```

Intuition

The two fingers i and j march toward each other. The left finger i stops when it finds an element that is too big (it belongs on the right). The right finger j stops when it finds an element too small (it belongs on the left). Then they swap, fixing two misplaced elements at once. They keep going until the fingers cross. Where they cross is the boundary: everything at or left of j is $\leq$ the pivot, everything right of j is $\geq$ it.

Figure 7.7 contrasts the pointer mechanics of the two partitioning schemes side by side.

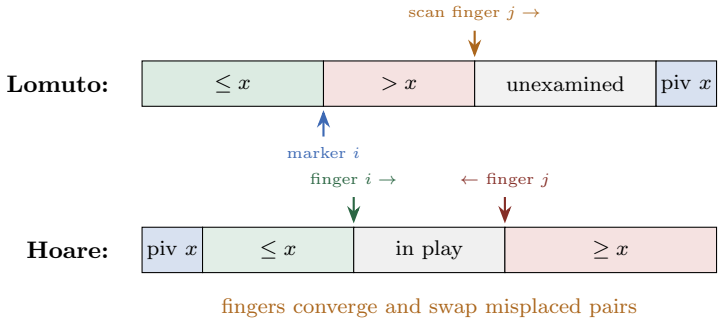


Figure 7.7: Pointer sweeps in Lomuto versus Hoare partitioning. Lomuto advances a single forward-scanning finger j, dragging pointer i behind it to grow the small region. Hoare converges two fingers i and j from opposite ends toward the middle, exchanging opposing out-of-place pairs in each step until the fingers cross.

Common trap

Hoare partition has two gotchas that Lomuto does not. First, the returned index q is *not* the pivot's final resting place – the pivot may still be loose somewhere in the left part. So the recursive calls differ: you call QUICKSORT(A, p, q) and QUICKSORT$(A, q+1, r)$ – note q is *included* in the left call, and there is no "$q-1$" or "$q+1$" skipping a pivot. Second, you must choose the pivot as $A[p]$ (the first element), not $A[r]$; choosing $A[r]$ can cause the loop to run off the end of the array. These off-by-one and boundary subtleties are exactly why textbooks teach Lomuto first.

Algorithm lab

Trace HOARE-PARTITION on $A = [5,\ 3,\ 8,\ 1,\ 9,\ 2,\ 7]$, $p = 1$, $r = 7$, pivot $x = A[1] = 5$.

fingers stop at	comparison	action
j: 7, 2 stop at $A[6]$=2 ≤ 5; i: stop at $A[1]$=5 ≥ 5	i=1 $< j$=6	swap $A[1], A[6]$: $[2,3,8,1,9,5,7]$
j: 9, 1 stop at $A[4]$=1; i: 3, 8 stop at $A[3]$=8	i=3 $< j$=4	swap $A[3], A[4]$: $[2,3,1,8,9,5,7]$
j: stop at $A[3]$=1; i: stop at $A[4]$=8	i=4 $\not< j$=3	return $q = 3$

Final array $[2,3,1,\ 8,9,5,7]$ with $q = 3$. Check: $A[1\,..\,3] = [2,3,1]$ are all ≤ 5, and $A[4\,..\,7] = [8,9,5,7]$ are all ≥ 5. The split is valid even though the pivot 5 is sitting at index 6, not at the boundary. That is why the recursion includes index q on the left.

Why this matters

Why does industry care which partition scheme is used if both are $\Theta(n \log n)$ on average? Because the constant factor shows up on the clock. A swap moves data in memory; fewer swaps mean fewer cache misses and fewer writes. Hoare's scheme does about $\frac{1}{3}$ as many swaps as Lomuto's, which is why high-performance library sorts (and hybrids like *introsort*, used in C++'s `std::sort`) build on Hoare-style partitioning. The asymptotics give the growth rate; the constant factors decide which implementation is faster in practice.

7.7 Engineering the real thing

A production quicksort is rarely the textbook version verbatim. A handful of refinements turn the idea into a fast library sort:

- **Cut over to insertion sort for tiny subarrays.** When a subarray drops below, say, 10–16 elements, insertion sort (Chapter 2) beats quicksort because its constant factors are tiny and it has no recursion overhead. Many implementations stop recursing at small sizes and finish with a single insertion-sort pass over the whole nearly-sorted array.
- **Median-of-three pivoting.** Instead of a random or fixed pivot, take the median of $A[p]$, $A[r]$, and the middle element. This makes a balanced split far more likely on real-world data and is cheap.
- **Three-way partitioning for duplicates.** Recall the trap: arrays with many equal keys destroy basic quicksort. A *three-way* partition ("Dutch national flag") splits into $< x$, $= x$,

and $> x$ regions, so all copies of the pivot are placed at once and never recursed on. This turns the all-duplicates worst case into linear time.

- **Bounded recursion depth (introsort).** To guarantee $O(n \log n)$ even in the worst case, monitor the recursion depth; if it exceeds $\approx 2 \log_2 n$, bail out to heapsort (Chapter 6) for that subarray. You keep quicksort's speed on typical inputs and heapsort's worst-case guarantee. This hybrid is *introsort*, the basis of the C++ standard library sort.

Cost summary

Quicksort at a glance (sorting n elements).

Quantity	Value	When
Best-case time	$\Theta(n \log n)$	balanced splits
Average / expected time	$\Theta(n \log n)$	random pivots, distinct keys
Worst-case time	$\Theta(n^2)$	always-lopsided splits
Auxiliary space	$\Theta(\log n)$ expected	recursion stack
In place?	yes (no scratch array)	—
Stable?	no (swaps reorder equal keys)	—
Comparisons (expected)	$\approx 2n \ln n \approx 1.39\, n \log_2 n$	from $2H_n$ analysis

The expected comparison count $2n \ln n \approx 1.39\, n \log_2 n$ is only about 39% above the information-theoretic floor of $\log_2(n!) \approx n \log_2 n$ comparisons that *any* comparison sort must pay (Chapter 8). Quicksort comes within a small factor of the comparison lower bound while doing very little data movement per step, which explains its speed in practice.

Common trap

"In place" but watch the stack. Quicksort uses no auxiliary *array*, but it does use the call stack for recursion, $\Theta(\log n)$ deep when balanced and $\Theta(n)$ deep in the worst case – enough to overflow the stack on a large adversarial input. The fix is *tail-recursion elimination*: always recurse on the *smaller* side first and loop on the larger side, which caps the stack at $\Theta(\log n)$ even in the worst case. Library implementations do this.

7.8 Putting it all together

Focus

The five things to carry out of this chapter:

1. **Partition is the engine.** It places one pivot in its final spot in $\Theta(n)$ time; the combine step is free. That is what makes quicksort sort in place.
2. **Balance is everything.** Running time is $\Theta(n)$ times the recursion depth. Balanced splits give depth $\log n$; lopsided splits give depth n. Even 99-to-1 splits stay $\Theta(n \log n)$.
3. **Fixed pivot rules have bad inputs.** Any deterministic pivot choice can be forced into quadratic time; last-element pivoting fails on sorted data.

4. **Randomization protects every input.** Choosing pivots at random gives $\Theta(n \log n)$ expected time regardless of how the input is arranged. The randomness is in your coins, not the data.
5. **The proof counts comparisons.** Each pair z_i, z_j is compared with probability $\frac{2}{j-i+1}$; summing gives harmonic numbers, hence $\Theta(n \log n)$.

We have now examined three comparison sorts – merge sort, heapsort, and quicksort – all clustering near the $\Theta(n \log n)$ boundary. Is $\Theta(n \log n)$ an unbreakable barrier, or an artifact of comparing elements two by two? In Chapter 8, we will prove that any algorithm using only comparisons must take $\Omega(n \log n)$ comparisons in the worst case, and then discover how counting sort and radix sort get around that limit by not using comparisons at all.

7.9 Exercises

Exercise 7.1 (Trace Lomuto)**.** Run PARTITION (Lomuto) by hand on

$$A = [\,13,\ 19,\ 9,\ 5,\ 12,\ 8,\ 7,\ 4,\ 11,\ 2,\ 6,\ 21\,]$$

with pivot $A[12] = 21$, then again on the same array with pivot $A[12]$ replaced by 6 (swap the 6 into the last slot first). Show the array after every iteration of j and give the returned index q.

Exercise 7.2 (Counting in the worst case)**.** Exactly how many comparisons does Lomuto PARTITION make on a single call to a subarray of size n? Using this, write and solve the recurrence for the total number of comparisons quicksort makes when every split is maximally unbalanced, and confirm it is $\Theta(n^2)$.

Exercise 7.3 (Balanced partitioning)**.** Suppose PARTITION always returned an index q exactly in the middle of the subarray. Write the recurrence and use the master theorem (Chapter 4) to solve it. Then suppose instead that q always lands at the $\frac{1}{4}$ point (a 1-to-3 split). Draw the recursion tree, bound its depth, and argue the total is still $\Theta(n \log n)$.

Exercise 7.4 (The duplicate-key disaster)**.** Let A be an array of n identical elements. Trace what Lomuto PARTITION does, and determine the exact split sizes it produces. Conclude that basic quicksort runs in $\Theta(n^2)$ on all-equal input even with a random pivot. Then describe, in words, how three-way (Dutch-flag) partitioning fixes this.

Exercise 7.5 (Hoare hands-on)**.** Trace HOARE-PARTITION on $A = [\,4,\ 7,\ 2,\ 9,\ 1,\ 6,\ 3\,]$ with pivot $A[1] = 4$. Show the fingers i, j and the array after each swap, and give the returned q. Verify that everything in $A[1\,..\,q]$ is $\leq$ everything in $A[q+1\,..\,r]$. Where did the pivot end up?

Exercise 7.6 (Why the indicator trick works)**.** In the expected-time proof we claimed that two elements z_i and z_j are compared if and only if the first pivot chosen from $Z_{ij} = \{z_i, \ldots, z_j\}$ is one of the two endpoints. Explain carefully why a pivot chosen from *outside* Z_{ij} never separates z_i from z_j, and why a middle element z_k $(i < k < j)$ chosen first guarantees they are never compared.

Exercise 7.7 (Probability of comparing extremes)**.** Using the formula $\Pr[z_i \text{ compared with } z_j] = \frac{2}{j-i+1}$, compute the probability that the smallest and the largest elements (z_1 and z_n) are ever directly compared, as a function of n. What is this probability for $n = 10$, $n = 100$, $n = 1000$? What does the trend tell you intuitively?

Exercise 7.8 (Expected comparisons, exactly)**.** Show that the expected number of comparisons can be written exactly as $\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$. For $n = 4$, evaluate this sum by hand to a fraction. (You should get $\frac{2}{2} + \frac{2}{3} + \frac{2}{4} + \frac{2}{2} + \frac{2}{3} + \frac{2}{2}$ once you list all pairs – organize them and add.)

Exercise 7.9 (Tail recursion and stack depth)**.** Explain why recursing on the *smaller* subarray first and looping on the larger one bounds the recursion-stack depth to $\Theta(\log n)$ even in the $\Theta(n^2)$-time worst case. Hint: each time you push a stack frame, the remaining problem is at most half the size. (Challenge.)

Exercise 7.10 (Randomized vs. shuffle-then-sort)**.** Some people achieve the benefits of a random pivot by first randomly *permuting* the entire input array and then running plain (deterministic, last-element-pivot) quicksort. Argue that this gives the same expected running time $\Theta(n \log n)$. What is one practical advantage of choosing random pivots *during* the recursion instead of shuffling up front? (Challenge.)

Exercise 7.11 (Median-of-three by hand)**.** For the array $A = [\,8,\ 1,\ 4,\ 9,\ 6,\ 3,\ 5,\ 2,\ 7\,]$, compute the median-of-three pivot using $A[1]$, the middle element $A[5]$, and $A[9]$. Then argue, in a sentence or two, why median-of-three makes the already-sorted worst case essentially impossible to trigger by accident. (Warmup-to-medium.)

Chapter 8

Sorting in Linear Time

So far, every sorting algorithm you have met – insertion sort, merge sort, heapsort, quicksort – shares one habit. To decide where an element belongs, the algorithm *compares* it against another element: "is a less than b?" It never peeks inside the values; it only asks, over and over, which of two things is smaller. The very best of these algorithms run in $\Theta(n \log n)$ time, and you might reasonably start to suspect that $\Theta(n \log n)$ is an unavoidable barrier for sorting.

This chapter is going to do two surprising things. First, it will *prove* that suspicion partly right: any algorithm that sorts using only comparisons must, in the worst case, take at least $\Theta(n \log n)$ time. This limit comes from the comparison model itself, not from a lack of cleverness. Second, it will show you how to bypass that lower bound when keys have extra structure. If you are allowed to look inside the values, to use them as more than opaque tokens you can only compare, you can sort genuinely faster: in *linear* time, $\Theta(n)$. We will meet three algorithms that take advantage of that structure: **counting sort**, **radix sort**, and **bucket sort**.

Focus

The main idea of this chapter is that pairwise *comparisons* extract very little information per step. Each comparison gives you only one bit of information (less, or not less), and you need a lot of bits to pin down the right ordering, so comparison sorting cannot beat $\Theta(n \log n)$. But if the values themselves carry usable structure – they are small integers, or fixed-width keys, or uniformly spread reals – you can extract information *in bulk* instead of one comparison at a time, and sorting drops to $\Theta(n)$.

8.1 Comparisons Are a Kind of Currency

We go slowly here, because the lower bound is the central idea of the chapter and it is easy to follow the steps without seeing why it holds.

Suppose I hand you three numbers in three sealed boxes, labelled A, B, C. You cannot see the numbers. Your only allowed move is to point at two boxes and ask me "is the number in this one smaller than the number in that one?" I answer yes or no, truthfully. Your job is to tell me the correct sorted order of the three boxes.

How many questions might you need? Let us count the possible answers you might have to produce.

With three distinct values, the sorted order could be any of the $3! = 6$ permutations:

$$ABC, \quad ACB, \quad BAC, \quad BCA, \quad CAB, \quad CBA.$$

Each of these is a genuinely different answer, and before you ask anything you have no idea which one is correct. Every question you ask gets a yes/no answer, which at best splits the still-possible answers into two groups. The best you can hope for from one question is to cut the number of surviving possibilities in half.

Intuition

Think of it as a guessing game. There are 6 secret answers on the table. Each yes/no question can, at most, cut the pile in half. After one question: at least 3 left. After two: at least $\lceil 3/2 \rceil = 2$ left. After three: at least 1. You cannot guarantee you have narrowed it to a single answer in fewer than $\lceil \log_2 6 \rceil = 3$ questions, because two questions can only distinguish $2^2 = 4$ outcomes, and $4 < 6$. There simply are not enough yes/no patterns to go around.

The same argument works for any n. To sort n elements you must be able to produce any of $n!$ possible orderings. Each comparison is one yes/no question. A sequence of k comparisons produces a pattern of k bits, and there are only 2^k distinct bit-patterns. If your algorithm always finishes in k comparisons, then those 2^k patterns must cover all $n!$ answers – otherwise two different correct orderings would produce the same pattern of answers, and the algorithm could not tell them apart. So we need $2^k \geq n!$, which forces $k \geq \log_2(n!)$.

8.1.1 The Decision Tree: Drawing the Argument

The clean way to make this rigorous is to draw the algorithm as a tree, called a *decision tree*. Forget the algorithm's code for a moment and watch only its comparisons. The first comparison it makes is the root. Depending on the yes/no answer, it goes left or right to its next comparison, and so on. Every internal node is a comparison; every leaf is a final answer (a permutation).

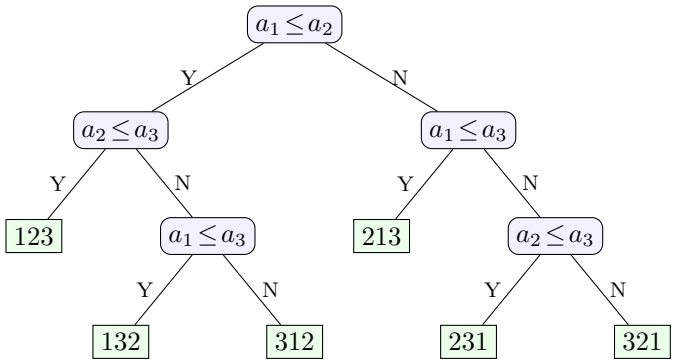


Figure 8.1: A decision tree for sorting three elements. Each internal node is a comparison; "Y"/"N" edges record the answer. A leaf labelled ijk means the sorted order is $a_i \leq a_j \leq a_k$. There are $3! = 6$ leaves, one per permutation, and the deepest leaf is 3 comparisons down.

In Figure 8.1, *running* the algorithm on a particular input means starting at the root and walking down to a leaf, turning left or right according to each comparison's answer. The number of comparisons that run makes on that input is exactly the length of the path it walks – the *depth* of the leaf

it reaches. The *worst-case* number of comparisons is the depth of the deepest leaf: the *height* of the tree.

So the question "how few comparisons can a sorting algorithm guarantee?" becomes a question about trees: how *short* can a tree be if it must have at least $n!$ leaves?

Formal

Two facts about binary trees we will lean on. A binary tree of height h has at most 2^h leaves – because each level doubles the count of nodes available, and the leaves all live at depth $\le h$. Turned around: a binary tree with L leaves has height at least $\log_2 L$.

8.1.2 The Lower Bound, Stated and Proved

Theorem 8.1 (Comparison-sort lower bound)**.** *Any comparison sort, in the worst case, performs $\Omega(n \log n)$ comparisons to sort n elements. Concretely, its decision tree has height at least $\log_2(n!) = \Theta(n \log n)$.*

Proof. Consider any comparison sort and its decision tree T on inputs of length n. Because the algorithm must be *correct*, every one of the $n!$ possible orderings of the input must appear as the label of at least one reachable leaf; if some permutation π labelled no leaf, then the input whose correct answer is π would be sorted wrongly. Hence T has at least $n!$ leaves.

A binary tree with at least $n!$ leaves has height $h \ge \log_2(n!)$. The worst-case comparison count is exactly h, so the algorithm makes at least $\log_2(n!)$ comparisons on some input.

It remains to see that $\log_2(n!) = \Theta(n \log n)$. The upper side is easy: $n! \le n^n$, so $\log_2(n!) \le n \log_2 n$. For the lower side, keep only the top half of the factors,

$$n! \;=\; n \cdot (n-1) \cdots 2 \cdot 1 \;\ge\; \underbrace{n \cdot (n-1) \cdots \tfrac{n}{2}}_{n/2 \text{ factors, each} \ge n/2} \;\ge\; \left(\frac{n}{2}\right)^{n/2},$$

so $\log_2(n!) \ge \frac{n}{2} \log_2 \frac{n}{2} = \frac{n}{2}(\log_2 n - 1) = \Omega(n \log n)$. Both bounds together give $\log_2(n!) = \Theta(n \log n)$. □

Corollary 8.1. *Merge sort and heapsort are asymptotically optimal comparison sorts: their $O(n \log n)$ worst-case running time matches the $\Omega(n \log n)$ lower bound to within a constant factor.*

Common trap

The lower bound is about *comparison* sorts, and only about them. It says nothing about algorithms that do other things with the keys – adding them up, indexing an array with them, slicing them into digits. People sometimes over-read Theorem 8.1 as "sorting is $\Omega(n \log n)$, period." It is not. It is "*comparison-based* sorting is $\Omega(n \log n)$." The rest of this chapter shows what becomes possible once you allow operations other than comparisons.

Walking question

Here is something to chew on while you walk. The proof only needs the tree to have $\geq n!$ leaves and to be *binary*. Where exactly did "binary" come from? It came from each comparison having two outcomes. Now imagine a machine whose every probe has *three* outcomes (less, equal, greater). Redo the counting. Does three-way branching break the $\Omega(n \log n)$ barrier, or does it only change the base of the logarithm – and why does changing the base of a logarithm not change a $\Theta(n \log n)$ bound at all?

History and connection

In 1890, the United States Census Bureau faced a serious problem: the 1880 census had taken eight years to tabulate by hand, and officials projected that the 1890 count would take more than a decade – meaning the results would not be ready before the 1900 census arrived! A young engineer named Herman Hollerith devised an electromechanical tabulating machine using punched paper cards, with holes representing demographic variables like age, occupation, and birthplace. To sort millions of cards efficiently, Hollerith built mechanical sorting boxes that sorted cards into bins one punch-column at a time, moving counterintuitively from the least significant digit to the most significant. His machines tabulated the records of 62 million Americans in mere months, and his Tabulating Machine Company eventually merged to become IBM. Sixty-four years later, in 1954, Harold H. Seward at MIT wrote his landmark Master's thesis adapting Hollerith's mechanical punch-card techniques to digital stored-program computers on the NBS SEAC, formally inventing both Counting Sort and digital Radix Sort. Meanwhile, at Bell Labs in 1948, Claude Shannon published *A Mathematical Theory of Communication*, defining information entropy and establishing that each binary question extracts at most one bit of information. Applied to sorting, this idea gives a hard limit: because n distinct elements can arrive in any of $n!$ possible permutations, an algorithm must distinguish among $n!$ leaves in a decision tree. Since each two-way comparison produces at most one bit of information, reaching one specific leaf out of $n!$ requires at least $\log_2(n!) = \Omega(n \log n)$ comparisons. This bound is often misread as a speed limit for all sorting. The methods of Hollerith and Seward show that the $\Omega(n \log n)$ bound applies only to algorithms that treat elements as opaque keys compared pairwise. If an algorithm is permitted to inspect the internal numerical digits of keys, it can use the digits directly as array addresses and sort in $\Theta(n)$ time.

Yet linear-time sorting comes with strict preconditions. Counting sort and radix sort are not general-purpose sorts. They require keys with known, bounded discrete ranges ($k = O(n)$); attempting to run counting sort on 64-bit integers would require an auxiliary allocation array of 2^{64} slots, far more memory than any machine has. Radix sort's correctness depends on the stability of each intermediate pass, and scattering elements into memory bins often incurs cache-miss penalties on modern CPUs. In Chapter 9, we ask the natural next question: if we only want to find the single median or k-th percentile rather than sorting the whole array, can we also find it in linear time? Later, in Chapter 11, we generalize direct-address binning into universal Hash Tables, and in Chapter 31 we employ radix-style passes to build suffix arrays for rapid genomic string matching.

8.2 Counting Sort: Sorting Without Comparing

Counting sort is the first way around the lower bound: it sorts integers without ever comparing two of them. Instead it asks a different question about each element: "*what* are you?" – and uses the answer as an array index.

Intuition

Picture a classroom of students, each holding a card with a single digit 0–9. You want them lined up in sorted order. Instead of comparing cards pairwise, you put ten labelled bins on the floor – bin 0 through bin 9 – and shout "put your card in the bin that matches your digit." Now just sweep the bins left to right and read the cards out. They come out sorted, and you never compared a single pair. The cost was one pass to fill bins and one pass to drain them: linear. That is counting sort. The catch, already visible, is that you needed one bin per possible value, so this only works when the values come from a small, known range.

8.2.1 The Setup

Counting sort assumes the n input values are integers in a known range $\{0, 1, \ldots, k\}$. The plan:

1. **Count.** Make an array $C[0\,..\,k]$ and set $C[v]$ to the number of input elements equal to v. (One pass over the input.)
2. **Accumulate.** Turn C into a running total, so that $C[v]$ becomes the number of elements *less than or equal to* v. Now $C[v]$ tells you the last position in the sorted output where a value v belongs.
3. **Place.** Walk the input and drop each element into its computed output slot, decrementing $C[v]$ as you go so equal elements stack up neatly.

Algorithm 26: COUNTING-SORT

Input: Array $A[1\,..\,n]$ of integers, each in $\{0, \ldots, k\}$.
Output: Array $B[1\,..\,n]$ holding the elements of A in sorted order.

```
1  let C[0..k] be a new array;
2  for v ← 0 to k do
3      C[v] ← 0;
4  for j ← 1 to n do
5      C[A[j]] ← C[A[j]] + 1;                    // C[v] now counts how many equal v
6  for v ← 1 to k do
7      C[v] ← C[v] + C[v − 1];                   // C[v] now counts how many ≤ v
8  for j ← n to 1 do
9      B[ C[A[j]] ] ← A[j];
10     C[A[j]] ← C[A[j]] − 1;
11 return B;
```

The single subtle choice is the *last* loop running *backwards*, from n down to 1. That reverse-order pass is what makes counting sort *stable*, and stability is exactly what radix sort will demand of it in

the next section.

8.2.2 A Full Numeric Trace

Let us sort

$$A = (2,\ 5,\ 3,\ 0,\ 2,\ 3,\ 0,\ 3), \qquad n = 8, \quad k = 5.$$

Step 1 – count. Sweep A and tally each value:

v	0	1	2	3	4	5
$C[v]$	2	0	2	3	0	1

(Two 0s, no 1s, two 2s, three 3s, no 4s, one 5. They sum to 8, good.)

Step 2 – accumulate into running totals "how many $\leq v$":

v	0	1	2	3	4	5
$C[v]$	2	2	4	7	7	8

Read $C[3] = 7$ as "there are 7 elements ≤ 3, so the last 3 belongs in slot 7 of the output."

Step 3 – place, scanning A from right ($j = 8$) to left ($j = 1$). At each step we read $A[j]$, look up $C[A[j]]$ to get the slot, write into B, and decrement.

j	$A[j]$	slot $C[A[j]]$	write	then C becomes
8	3	7	$B[7] = 3$	$C[3]: 7 \to 6$
7	0	2	$B[2] = 0$	$C[0]: 2 \to 1$
6	3	6	$B[6] = 3$	$C[3]: 6 \to 5$
5	2	4	$B[4] = 2$	$C[2]: 4 \to 3$
4	0	1	$B[1] = 0$	$C[0]: 1 \to 0$
3	3	5	$B[5] = 3$	$C[3]: 5 \to 4$
2	5	8	$B[8] = 5$	$C[5]: 8 \to 7$
1	2	3	$B[3] = 2$	$C[2]: 3 \to 2$

Collecting the writes by slot index gives

$$B = (\,0,\ 0,\ 2,\ 2,\ 3,\ 3,\ 3,\ 5\,),$$

which is exactly A in sorted order. Notice we never asked "is this bigger than that?" even once.

Figure 8.2 shows what the placement step is really doing: the accumulated array C hands each value a *block* of output slots, and the contiguity of those blocks is what makes the writes land in sorted order with no comparisons.

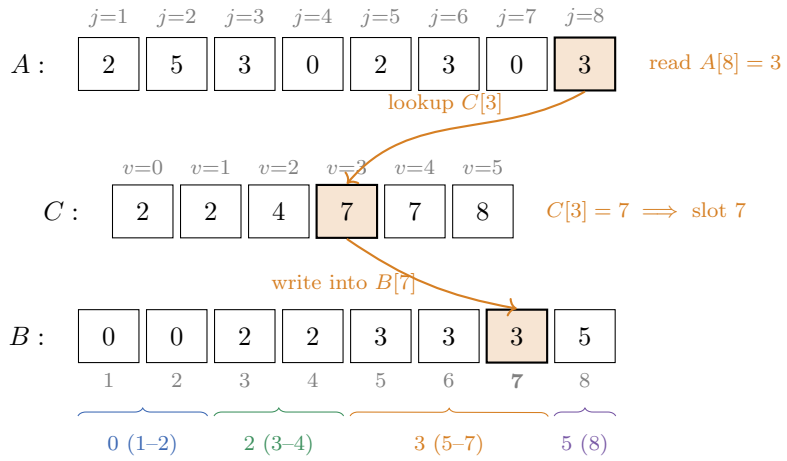


Figure 8.2: Counting Sort: from accumulation to placement. The accumulated count array C reserves a contiguous block of output slots for each value: value v owns slots $C[v-1]+1$ through $C[v]$. As the algorithm scans A right-to-left, each element $A[j]$ consults $C[A[j]]$ for its target index in B and decrements the count. Here $A[8]=3$ looks up $C[3]=7$, writes into $B[7]$, and $C[3]$ becomes 6 for the next 3.

Algorithm lab

Trace it yourself, but flip the final loop to run *forwards*, j from 1 to n, with everything else unchanged. You will find it still sorts! So why do we insist on going backwards? Re-run both directions on an input where you can *tell the equal elements apart* – say, tag them: $3_a, 3_b, 3_c$ in their original left-to-right order. With the backward loop, do $3_a, 3_b, 3_c$ come out in their original relative order in B? With the forward loop, what happens to that order? The next subsection explains the answer.

8.2.3 Stability, and Why We Care

Definition 8.1 (Stable sort)**.** A sort is *stable* if elements that compare equal keep their original relative order in the output. If 3_a appeared before 3_b in the input, a stable sort guarantees 3_a still appears before 3_b in the output.

Counting sort as written is stable. Here is why. The accumulated $C[v]$ marks the *last* output slot for value v, and we fill from the back. So the *rightmost* input element of value v grabs the highest slot, and as we move leftward through the input, each earlier element of value v grabs the next-lower slot. Earlier-in-input lands earlier-in-output: that is exactly stability. Run the loop forward instead and the order of equal elements reverses – still sorted, but no longer stable.

Focus

Two properties of counting sort to carry forward: it is **linear** in $n+k$, and it is **stable**. Radix sort, coming next, leans on *both* – it would simply not work without stability.

8.2.4 Cost, and the Catch

Cost summary

Counting sort.

- Initialize C: $\Theta(k)$.
- Count: $\Theta(n)$.
- Accumulate: $\Theta(k)$.
- Place: $\Theta(n)$.

Total time $\Theta(n+k)$. Extra space $\Theta(n+k)$ (the output B and the counter C). When $k = O(n)$ – the range is no bigger than the count of elements – this is $\Theta(n)$: *linear* sorting, beating the $n \log n$ comparison lower bound.

Common trap

The catch is the "$+k$" term. If your values are 32-bit integers, then k could be about 4,000,000,000, and counting sort would allocate a four-billion-entry array to sort, say, ten numbers. That wastes both time and memory. Counting sort is only a good idea when the range k is comparable to (or smaller than) the number of elements n. The moment $k \gg n$, you want a different tool – which is precisely the gap radix sort fills.

8.3 Radix Sort: Many Small Sorts Make One Big Sort

Counting sort works well, but only for a small range. What if your keys are big – 32-bit integers, or words, or dates? The trick of *radix sort* is to chop each key into small *digits* and sort by one digit at a time, using a stable sort (counting sort!) for each pass. The digit's range is small, so each pass is cheap; and by ordering the passes cleverly, the small sorts compose into a full sort.

8.3.1 Least Significant Digit First – and Why That Order

The counterintuitive rule is: sort by the *least* significant digit first, then the next, and finish with the *most* significant digit.

Intuition

Sorting most-significant-digit-first feels natural – it is how you would alphabetize by hand – but it forces you to recurse into ever-smaller groups and keep them separated. Sorting from the least significant digit upward avoids all that bookkeeping: because each pass is *stable*, sorting on a higher digit never disturbs the relative order already established by the lower digits. Each pass refines the previous one. After the last (most significant) pass, ties on the top digit are broken by the order from the previous pass, whose ties are broken by the pass before, all the way down. The whole number ends up sorted.

Algorithm 27: RADIX-SORT

Input: Array $A[1..n]$ of d-digit numbers; digit 1 is least significant.
Output: A sorted into nondecreasing order.

```
1 for i ← 1 to d do
2     use a stable sort to sort array A on digit i;
3 return A;
```

8.3.2 A Full Numeric Trace

Sort these seven three-digit numbers:

$$329,\ 457,\ 657,\ 839,\ 436,\ 720,\ 355.$$

That is $n = 7$; I will line them up in columns and sort digit by digit, using a stable sort on each column. The base is 10, so each digit ranges over 0–9.

input	**by 1's**	**by 10's**	**by 100's**
329	72$\underline{0}$	7$\underline{2}$0	$\underline{3}$29
457	35$\underline{5}$	3$\underline{2}$9	$\underline{3}$55
657	43$\underline{6}$	4$\underline{3}$6	$\underline{4}$36
839	45$\underline{7}$	8$\underline{3}$9	$\underline{4}$57
436	65$\underline{7}$	3$\underline{5}$5	$\underline{6}$57
720	32$\underline{9}$	4$\underline{5}$7	$\underline{7}$20
355	83$\underline{9}$	6$\underline{5}$7	$\underline{8}$39

Read it left to right. Column 2 sorts by the units digit only (underlined); note 457 and 657 both end in 7 and, because the sort is stable, 457 stays ahead of 657 exactly as it was in the units-sorted column. Column 3 sorts that result by the tens digit; again ties (like 355 vs. 457, both tens digit 5) keep the order the units pass gave them. Column 4 sorts by the hundreds digit, and we are done:

$$329,\ 355,\ 436,\ 457,\ 657,\ 720,\ 839.$$

Here is exactly where stability mattered. After the tens pass, the order among the 5-tens group is $355, 457, 657$ – and that came purely from their units digits: 355 leads because its units digit 5 is smallest, while 457 and 657 tie on units digit 7 and so keep the order the units pass already gave them. When the hundreds pass scatters them, that internal order survives wherever the hundreds digits agree.

Figure 8.3 isolates that single moment: the tens pass moves 355, 457, and 657 around, but because they share the same tens digit, the order the units pass established travels with them untouched.

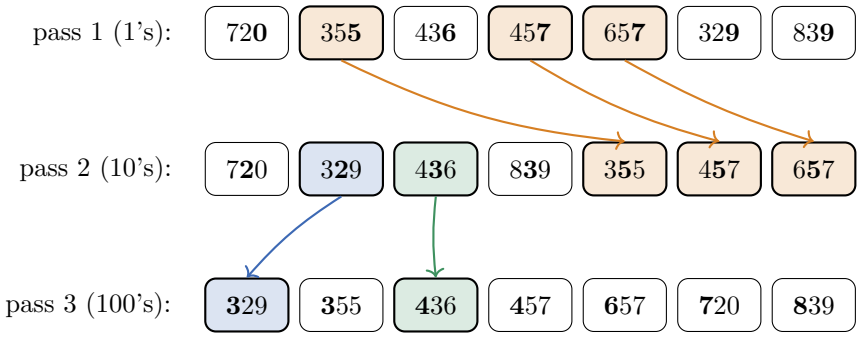


Figure 8.3: The three digit passes of Radix Sort and stability in action. In pass 1, numbers are ordered by their units digit (bold). In pass 2, numbers are sorted by their tens digit; the three orange-shaded elements $(355, 457, 657)$ all share tens digit 5, and stability guarantees their relative order from pass 1 is preserved. In pass 3, sorting on the hundreds digit completes the sort; ties such as 329 vs. 355 (hundreds digit 3) and 436 vs. 457 (hundreds digit 4) maintain their pass-2 relative order, yielding the fully sorted array.

Focus

The invariant that makes radix sort correct: *after the pass on digit i, the array is sorted according to the low-order i digits treated as a single number.* The base case ($i = 0$) holds because an empty sequence of digits defines no ordering to violate. Each stable pass on digit $i + 1$ preserves it. After d passes, the array is sorted on all d digits – i.e., fully sorted.

Formal

Why the inductive step holds. Assume after pass i the array is sorted on the low i digits. Now stably sort on digit $i+1$. Take any two elements x, y. *(a)* If they differ on digit $i+1$, the new pass orders them correctly by that digit, which dominates all lower digits – correct. *(b)* If they agree on digit $i + 1$, stability keeps their pre-pass relative order, which by the inductive hypothesis already reflects the low i digits – correct. Either way the array is now sorted on the low $i + 1$ digits.

Common trap

Radix sort is correct *only because each pass is stable.* Swap in an unstable per-digit sort and it breaks: a later pass on a higher digit would scramble the ordering established by the lower digits. This is the reason we built counting sort to be stable. If you find yourself "optimizing" radix sort by dropping in quicksort per digit, you have quietly broken it.

8.3.3 Cost

Each of the d passes is a counting sort over a digit, whose range is the base b (for decimal digits, $b = 10$). One pass costs $\Theta(n + b)$, and there are d passes:

$$T(n) = \Theta\big(d\,(n + b)\big).$$

The digit width is a tuning parameter. Suppose the keys are w bits wide. You may slice them into

digits of any width r bits you like; then each digit ranges over $b = 2^r$ values and there are $d = \lceil w/r \rceil$ digits. Substituting:

$$T(n) = \Theta\Big(\frac{w}{r}\,(n + 2^r)\Big)\,.$$

Choosing $r = \log_2 n$ (so $2^r = n$) makes each pass cost $\Theta(n)$ and gives

$$T(n) = \Theta\left(\frac{w}{\log_2 n}\,n\right)\,.$$

If the word width w is a constant number of machine words – as it is for ordinary 32- or 64-bit integers – then $w = O(\log n)$ on inputs large enough to address, and this collapses to $\Theta(n)$. *Linear* again.

Cost summary

Radix sort on n keys of d digits in base b: time $\Theta\big(d(n+b)\big)$, stable, extra space $\Theta(n+b)$. With digit width tuned to $r = \log_2 n$ for w-bit keys: $\Theta\big(\frac{w}{\log n}\,n\big)$, which is $\Theta(n)$ whenever $w = O(\log n)$.

Why this matters

Radix sort is heavily used in practice to sort huge files of fixed-width keys: billions of 64-bit integers, network packets by IP address, strings of bounded length, database columns. GPU and large-scale sorting routines very often *are* radix sorts, because each pass is a simple, cache-friendly, embarrassingly parallel counting pass with no branchy comparisons. Record-setting large-scale sorting runs often use radix sort.

Walking question

A puzzle for the walk. Radix sort's cost has that d factor, and merge sort is $\Theta(n \log n)$, so which one wins comes down to d versus $\log n$. When you sort n distinct integers, how many bits w do you *at minimum* need just to give them distinct values? What does that say about d relative to $\log n$, and does radix sort's "linear" bound quietly reintroduce a $\log n$ factor through the required word width w? (This is a real and much-debated subtlety.)

8.4 Bucket Sort: Betting on the Distribution

The last linear-time sorter changes the assumption entirely. Counting and radix sort require integer (or integer-like) keys. Bucket sort instead assumes the inputs are *real numbers drawn uniformly at random* from an interval, say $[0, 1)$. Under that assumption it runs in linear time *on average*.

Intuition

If n numbers are sprinkled uniformly over $[0, 1)$, then chopping the interval into n equal slots – $[0, \frac{1}{n}), [\frac{1}{n}, \frac{2}{n}), \ldots$ – should catch roughly one number per slot. So: scatter every number into its slot (cheap, just multiply by n and take the floor), sort each tiny slot with any simple method (they are nearly empty, so this is cheap), then read the slots out in order. Because uniform inputs spread evenly, no slot gets crowded, and the total work stays linear. The method relies

on the input distribution; when that assumption holds, it is fast.

Algorithm 28: BUCKET-SORT

Input: Array $A[1..n]$ with each $A[j] \in [0, 1)$.
Output: The elements of A in sorted order.

```
1 let Bk[0..n−1] be an array of empty lists;
2 for j ← 1 to n do
3     insert A[j] into list Bk[⌊n · A[j]⌋];
4 for i ← 0 to n − 1 do
5     sort list Bk[i] with insertion sort;
6 return the lists Bk[0], Bk[1], ..., Bk[n − 1] concatenated in order;
```

8.4.1 A Numeric Trace

Take $n = 10$ and the inputs

$$.78,\ .17,\ .39,\ .26,\ .72,\ .94,\ .21,\ .12,\ .23,\ .68.$$

With $n = 10$, the bucket index of value x is $\lfloor 10x \rfloor$ – just the first decimal digit. Scatter:

bucket	contents (in arrival order)
0	–
1	.17, .12
2	.26, .21, .23
3	.39
4	–
5	–
6	.68
7	.78, .72
8	–
9	.94

Now insertion-sort each bucket (most have 0 or 1 element, so there is nothing to do; bucket 1 becomes .12, .17; bucket 2 becomes .21, .23, .26; bucket 7 becomes .72, .78). Concatenate buckets 0 through 9 in order:

$$.12,\ .17,\ .21,\ .23,\ .26,\ .39,\ .68,\ .72,\ .78,\ .94.$$

Sorted. The buckets did the coarse sorting (which tenth are you in?) and insertion sort did the fine sorting inside each tenth. Figure 8.4 draws the same scatter, with each bucket's (already sorted) contents stacked beneath it.

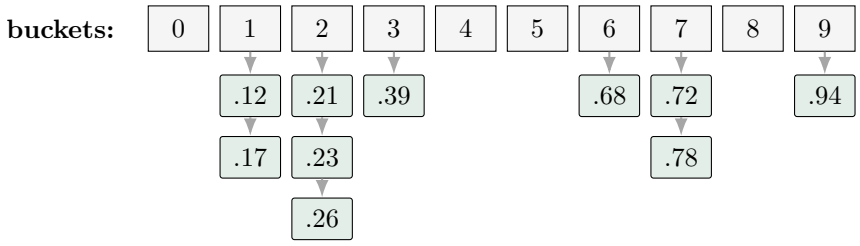


Figure 8.4: The ten buckets after scattering, with each bucket's contents chained below it (already insertion-sorted). Reading the chains left to right, top to bottom within each bucket, yields the sorted sequence. Most buckets hold zero or one element – the uniform spread keeps them shallow.

8.4.2 Why It Is Linear on Average

The danger is obvious: if every element fell into one bucket, insertion sort would face all n of them and cost $\Theta(n^2)$. The uniform-distribution assumption is precisely what makes that outcome extremely unlikely. Let us see the expected cost.

Let n_i be the number of elements landing in bucket i. Scattering is $\Theta(n)$, and concatenation is $\Theta(n)$. The interesting cost is the per-bucket insertion sort, which on n_i elements costs $O(n_i^2)$. So the total expected time is

$$\Theta(n) + \sum_{i=0}^{n-1} O\big(\mathbb{E}[\, n_i^2 \,]\big).$$

We need $\mathbb{E}[n_i^2]$. Each of the n elements lands in bucket i independently with probability $\frac{1}{n}$, so n_i is a Binomial$(n, \frac{1}{n})$ count. A standard fact (or a short calculation with indicator variables) gives, for such a binomial,

$$\mathbb{E}[n_i] = n \cdot \tfrac{1}{n} = 1, \qquad \mathrm{Var}(n_i) = n \cdot \tfrac{1}{n} \cdot \big(1 - \tfrac{1}{n}\big) = 1 - \tfrac{1}{n},$$

and therefore, using $\mathbb{E}[n_i^2] = \mathrm{Var}(n_i) + (\mathbb{E}[n_i])^2$,

$$\mathbb{E}[n_i^2] = \Big(1 - \tfrac{1}{n}\Big) + 1^2 = 2 - \tfrac{1}{n} < 2.$$

Each bucket contributes an expected *constant*. Summing over n buckets:

$$\Theta(n) + \sum_{i=0}^{n-1} O(2) = \Theta(n) + O(n) = \Theta(n).$$

Formal

Bucket sort runs in $\Theta(n)$ expected time when inputs are independent and uniform on $[0, 1)$, because each bucket holds a constant number of elements *in expectation*, so the quadratic insertion sorts are quadratic over tiny inputs and sum to linear. Its *worst case* – all elements in one bucket – is still $\Theta(n^2)$; the linearity is an average over the random inputs, not a guarantee on every input.

Cost summary

Bucket sort. Expected time $\Theta(n)$ on uniform $[0, 1)$ inputs; worst-case $\Theta(n^2)$. Extra space $\Theta(n)$. Stable if the per-bucket sort is stable and buckets are read in order.

Common trap

Bucket sort's linearity *depends on the input distribution.* Feed it skewed data – everything clustered in $[0, 0.01)$ – and it degrades toward $\Theta(n^2)$ as one bucket swallows the input. Counting sort and radix sort, by contrast, are linear on *every* input within their key assumptions; their bet is on the *keys* (small range, fixed width), which you can check, not on the *distribution*, which you often cannot. Know which kind of assumption your algorithm is making.

8.5 So When Can You Actually Beat $n \log n$?

Putting the lower bound alongside the three linear-time sorts clarifies what each algorithm assumes about its keys. The lower bound and the three fast sorts are not in conflict; they operate under different rules for *what the algorithm is allowed to know about the keys.*

Algorithm	Assumption on keys	Time
Merge / heapsort	only comparisons allowed	$\Theta(n \log n)$ worst case
Counting sort	integers in $\{0, \ldots, k\}$, $k = O(n)$	$\Theta(n + k)$
Radix sort	d digits, base b (e.g. fixed-width ints)	$\Theta(d(n + b))$
Bucket sort	reals, uniform on $[0, 1)$	$\Theta(n)$ expected

Focus

The linear-time sorts do not refute the lower bound; they *escape its hypothesis.* Theorem 8.1 constrains algorithms that learn about keys *only* by comparing them. Counting and radix sort learn by *indexing with* the keys; bucket sort learns by *computing a bucket from* the key. None of them is a comparison sort, so none of them is bound by $\Omega(n \log n)$. The cost is a usable assumption about the keys: small integer range, bounded width, or a known distribution.

Why this matters

The practical lesson generalizes far past sorting. Whenever you meet a proven lower bound, ask: *what assumption is the bound resting on, and can my real problem quietly drop it?* "Comparison-based" was the key assumption here, and dropping it saved a $\log n$ factor. The same move – find the hypothesis, then exploit special structure to sidestep it – recurs everywhere in algorithm design: hashing beats balanced-tree lower bounds by abandoning worst-case guarantees, and so on. A lower bound tells you precisely which assumption you have to relax if you want a faster algorithm.

Summary

- Any sort that touches keys *only* through comparisons needs $\Omega(n \log n)$ comparisons in the worst case. The proof models the algorithm as a binary *decision tree* that must have $\geq n!$ leaves, hence height $\geq \log_2(n!) = \Theta(n \log n)$.
- **Counting sort** sorts integers in $\{0, \ldots, k\}$ in $\Theta(n + k)$ time and is *stable*, by counting occurrences and placing from the back. Linear when $k = O(n)$.
- **Radix sort** sorts d-digit keys in $\Theta(d(n + b))$ time by stably sorting one digit at a time, *least significant first*. Correct precisely because each pass is stable.
- **Bucket sort** sorts uniform reals in $\Theta(n)$ *expected* time by scattering into n buckets and sorting each; $\Theta(n^2)$ worst case.
- All three beat the lower bound by *not being comparison sorts* – they exploit structure in the keys. The cost is a real assumption you must be able to justify.

Sorting an entire array gives you every order statistic at once: the minimum, the maximum, and every percentile in between. But what if you only need a single value – say, the exact median of an unsorted list? Sorting in $\Theta(n \log n)$ time feels like overkill when you only care about one rank. Can we find the median in purely linear time, $\Theta(n)$, using comparisons alone? In Chapter 9, we turn to selection and order statistics to find out.

8.6 Exercises

Exercise 8.1 (Warmup: count the leaves)**.** Draw a decision tree, like Figure 8.1, for sorting *two* elements a_1, a_2. How many leaves does it have, and what is its height? Confirm that the height equals $\lceil \log_2(2!) \rceil$.

Exercise 8.2 (The bound is tight-ish for small n)**.** Compute $\lceil \log_2(n!) \rceil$ for $n = 1, 2, 3, 4, 5$. These are the *minimum possible* worst-case comparison counts. It is known that 5 elements can be sorted using exactly 7 comparisons in the worst case. Does your value of $\lceil \log_2(5!) \rceil$ match 7? (It should – a rare case where the information-theoretic bound is achievable.)

Exercise 8.3 (Counting sort by hand)**.** Run Counting-Sort on $A = (6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2)$ with $k = 6$. Write out the count array, the accumulated array, and then trace the final placement loop step by step (as in the table in the text). State the sorted output.

Exercise 8.4 (Why backwards?)**.** Tag the equal elements of $A = (3_a, 1, 3_b, 1, 3_c)$ and trace Counting-Sort once with the final loop going backwards and once going forwards. Report the order of the 3's and the 1's in each output, and explain in one sentence why the backward version is stable and the forward version is not.

Exercise 8.5 (Radix by hand)**.** Use radix sort (base 10, least significant digit first) to sort

$$170,\ 045,\ 075,\ 090,\ 002,\ 802,\ 024,\ 066.$$

Show the array after each of the three digit passes, and point out one place where stability of the per-digit sort mattered.

Exercise 8.6 (Choosing the radix)**.** You must sort $n = 2^{20}$ keys, each $w = 64$ bits wide. Using the analysis $T(n) = \Theta\big(\frac{w}{r}(n + 2^r)\big)$, evaluate the cost (up to constants) for digit widths $r = 8$, $r = 16$, and $r = 20$. Which r does the "$r = \log_2 n$" rule recommend here, and does your arithmetic agree that it is the best of the three?

Exercise 8.7 (Bucket sort, skewed input)**.** Bucket sort assumes uniform inputs. Construct an input of n numbers in $[0, 1)$ that forces *every* element into a single bucket, and state the resulting running time. Then describe, in words, a preprocessing step that could rescue performance if you knew the inputs were uniform on $[a, b)$ rather than $[0, 1)$.

Exercise 8.8 (Expected bucket size)**.** Redo the bucket-sort analysis but with m buckets instead of n, where m is a parameter you choose. Show $\mathbb{E}[n_i] = n/m$ and find $\mathbb{E}[n_i^2]$ for a Binomial$(n, 1/m)$ count. For which choice of m (in terms of n) does the total expected time stay $\Theta(n)$, and what goes wrong if m is far too small?

Exercise 8.9 (Sorting strings)**.** You have n strings over the lowercase alphabet, all of length exactly L. Explain how to sort them in $\Theta(L\,(n+26))$ time using radix sort, and say which end (first letter or last letter) you sort on first and why. Then sketch why comparison-sorting them costs $\Theta(nL \log n)$ in the worst case, so radix sort genuinely wins for large n.

Exercise 8.10 (Challenge: a three-way decision tree)**.** Suppose each probe returns one of three answers: "$<$", "$=$", or "$>$". Rebuild the lower-bound argument with ternary decision trees. Show the height is at least $\log_3(n!)$, and then argue that $\log_3(n!) = \Theta(n \log n)$ all the same – so three-way comparisons do *not* break the asymptotic barrier. What constant factor do you save, if any?

Exercise 8.11 (Challenge: stability is not optional)**.** Give a concrete small input ($n = 4$ suffices) and a deliberately *unstable* per-digit sort for which radix sort produces the *wrong* final order. Trace it to show exactly which pass scrambles which earlier ordering. This is the cleanest way to convince yourself that the correctness of radix sort really does depend on stability.

Chapter 9

Medians and Order Statistics

Sorting, as you have seen across the previous chapters, provides a complete answer to every ranking question at once. If you have an unsorted array and want to know which element is largest, smallest, or sitting right in the middle, sorting the entire list gives you the answer. But sorting an entire list just to find the median is like alphabetizing a whole bookshelf to find a single volume. If all you want is one item — the middle one, the third smallest, or the 90th percentile — you should not have to put every element in order first.

That intuition is sound. In this chapter, we develop algorithms to find the k-th smallest element of an unsorted list in strictly linear time — $\Theta(n)$, the same order of growth as scanning through the elements once. Because reading the input already costs $\Theta(n)$, we find the desired element for the same asymptotic cost as reading the data.

Intuition

Sorting answers *every* "what is in position i?" question at once, and pays $\Theta(n \log n)$ for the privilege. **Selection** answers just *one* such question. This chapter shows that one such question can be answered in $\Theta(n)$ time if you are careful to throw away the parts of the input that cannot possibly contain the answer.

9.1 The selection problem

Let us pin down what we are after. Imagine a list (an array) of n numbers. They are not sorted; they sit in whatever order they came. We will assume, to keep things clean, that the numbers are *distinct* — no ties. (Ties can be handled with a little care, but they cloud the ideas, so set them aside for now.)

Definition 9.1 (Order statistic)**.** The **i-th order statistic** of a set of n numbers is the i-th smallest of them. So the 1st order statistic is the minimum, the n-th order statistic is the maximum, and the i-th order statistic for a general i is the value that would land in position i if you sorted the list into increasing order.

Definition 9.2 (Median)**.** The **median** is the "halfway point" of the set. When n is odd there is a single middle element: the median is the i-th order statistic for $i = (n+1)/2$. When n is even there are *two* middles; the **lower median** is at $i = \lfloor (n+1)/2 \rfloor = n/2$ and the **upper median** is at

$i = \lceil (n+1)/2 \rceil = n/2 + 1$. In this book, when we say "the median" without qualification we mean the lower median, so that the formula $i = \lfloor (n+1)/2 \rfloor$ always works.

Focus

The **selection problem**: given an array A of n distinct numbers and an integer i with $1 \le i \le n$, return the element of A that is larger than exactly $i-1$ of the others — the i-th smallest. The median is the special case $i = \lfloor (n+1)/2 \rfloor$.

Let us make the indices concrete, because this is where beginners stumble. Take the array

$$A = [\,7,\ 3,\ 9,\ 1,\ 5,\ 8,\ 2\,], \qquad n = 7.$$

If we sorted it we would get $[\,1, 2, 3, 5, 7, 8, 9\,]$. So:

- the 1st order statistic (minimum) is 1;
- the 7th order statistic (maximum) is 9;
- the 3rd order statistic is 3;
- the median, $i = \lfloor (7+1)/2 \rfloor = 4$, is 5.

Notice the i-th order statistic is a *value*, not a position in the original array. In the unsorted A, the median value 5 happens to live at index 5, but that coincidence means nothing. We care about its rank, not its address.

Common trap

"i-th smallest" is a **1-based rank**. The 1st smallest is the minimum, not the second-smallest, and not whatever sits at array index 1. A huge fraction of off-by-one bugs in selection code come from sliding between "rank i" and "array index i." Keep them separate in your head: rank is a property of the *sorted order*; index is a property of *where the value currently sits*.

9.1.1 The easy cases warm us up: minimum and maximum

Before the general problem, look at the very simplest order statistics, $i = 1$ and $i = n$. Finding the minimum is the canonical first algorithm everyone writes: walk through the array holding onto the smallest thing seen so far.

Algorithm 29: MINIMUM(A)

Input: An array $A[1\,..\,n]$ of numbers, $n \ge 1$.
Output: The smallest element of A.

```
1 min ← A[1];
2 for j ← 2 to n do
3     if A[j] < min then
4         min ← A[j];
5 return min;
```

This does exactly $n-1$ comparisons, one per element after the first. Can we do better? No — and there is a clean argument why. Think of each comparison as a little tournament match between two

values; the loser is the larger one (it cannot be the minimum). To *know* a particular element is the minimum, every other element must have lost at least one match, because an element that never lost might still be smaller than our claimed winner. There are $n-1$ "other" elements, each needing to lose at least once, and each comparison produces exactly one loss. So you need at least $n-1$ comparisons. MINIMUM is optimal.

Walking question

Suppose I ask for the minimum *and* the maximum of the same array. The lazy way runs MINIMUM and MAXIMUM separately: $2(n-1)$ comparisons. Can you do it in roughly $\frac{3}{2}n$ comparisons instead? Hint: process the elements in *pairs*. Compare the two members of a pair to each other first; the smaller is a candidate for the min, the larger a candidate for the max. Count how many comparisons that costs per pair, and you will see where the $\frac{3}{2}$ comes from.

The minimum and maximum are gentle because i is fixed at an extreme. The interesting trouble is the *general* i, and in particular the median, which sits as far from both ends as possible. That is what the rest of the chapter attacks.

9.2 Selection by sorting: the baseline to beat

The first algorithm is the one we already grumbled about: sort, then index.

Algorithm 30: SELECTBYSORTING(A, i)

Input: An array $A[1..n]$ of distinct numbers; a rank i, $1 \le i \le n$.
Output: The i-th smallest element of A.

```
1 B ← SORT(A) ;                          // B is A in increasing order
2 return B[i];
```

It is correct by definition: the i-th order statistic is, literally, the element in position i of the sorted array. With a good comparison sort like merge sort or heapsort, it costs $\Theta(n \log n)$. That is our baseline. Everything we build from here is an attempt to beat it — to get the single answer without paying for the full ordering.

Why this matters

Why fight for the difference between $n \log n$ and n? Because selection is a *subroutine*, called over and over inside bigger jobs. Order statistics power the "median-of-medians" pivots that make quicksort robust; they let databases answer "the 95th percentile latency" without sorting a billion rows; they sit inside k-nearest-neighbor search, robust statistics (the median is far more outlier-resistant than the mean), and load balancers picking a cutoff. When a routine is called n times, shaving a $\log n$ factor off each call can decide whether the system scales.

9.3 Quickselect: partition, then recurse on one side

Here is the key shift in thinking. In Chapter 7 you met PARTITION, the engine inside quicksort. It picks a *pivot*, then rearranges the array so that everything smaller than the pivot sits to its left and

everything larger sits to its right, with the pivot itself landing in its final sorted position. Quicksort then recurses on *both* sides.

But for selection we do not need both sides. After partitioning, the pivot is sitting at its true sorted rank. If that rank happens to be exactly i, we are done — we found it. If the pivot's rank is bigger than i, the answer must lie to the *left* of the pivot, so we recurse there and forget the right half entirely. If the pivot's rank is smaller than i, the answer is to the *right*, and we recurse there, forgetting the left.

Intuition

Quicksort recurses on both halves because it wants *everything* sorted. Selection recurses on *one* half because it wants *one* element. Throwing away half the array on each step is what makes the difference: it turns the $T(n) = 2T(n/2) + \Theta(n)$ recurrence of quicksort (which gives $n \log n$) into something closer to $T(n) = T(n/2) + \Theta(n)$ (which gives $\Theta(n)$).

9.3.1 Recalling Partition

Let me restate the partition routine so this chapter stands on its own. We use the Lomuto scheme: take the last element as the pivot, sweep a finger across the array, and keep a wall such that everything left of the wall is $\leq$ pivot.

Algorithm 31: PARTITION(A, p, r)

Input: Subarray $A[p\,..\,r]$.
Output: Rearranges $A[p\,..\,r]$ around the pivot $A[r]$; returns the pivot's final index q.

```
1 x ← A[r] ;                                              // the pivot value
2 j ← p − 1 ;                          // j is the wall:  A[p..j] are all ≤ x
3 for k ← p to r − 1 do
4     if A[k] ≤ x then
5         j ← j + 1;
6         swap A[j] ↔ A[k];
7 swap A[j + 1] ↔ A[r] ;                  // drop the pivot just past the wall
8 return j + 1;
```

After PARTITION returns q, three facts hold: $A[q]$ is the original pivot; every element in $A[p\,..\,q-1]$ is less than $A[q]$; and every element in $A[q+1\,..\,r]$ is greater than $A[q]$. So $A[q]$ is in its final sorted slot *within this subarray*. Its rank *within the subarray* is $q - p + 1$ (the count of elements at positions p through q). Figure 9.1 pictures the three regions and how selection chooses which one to keep.

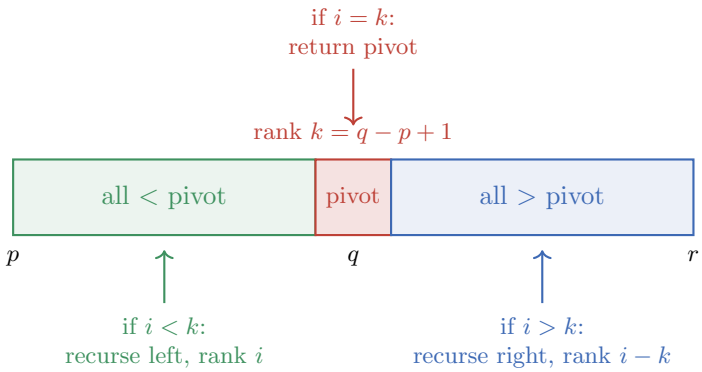


Figure 9.1: After PARTITION, the pivot sits at its true rank $k = q - p + 1$. Selection keeps only the one shaded region that can hold rank i and discards the rest — including the pivot's whole other side.

Common trap

The rank of the pivot *inside the current subarray* is $k = q - p + 1$, not q itself. When you recurse, you must convert between "rank within the whole problem" and "rank within the subarray." Forgetting the $-p + 1$ shift, or forgetting to subtract k when you recurse right, is the most common bug in quickselect. The numeric trace below works through this conversion.

9.3.2 Randomized quickselect

We will use a *randomized* pivot: before partitioning, swap a uniformly random element into the last slot. This is the same idea you saw stabilize quicksort — it defends against adversarial inputs and makes the analysis about randomness in *our* coin flips rather than luck in the input.

Algorithm 32: RANDOMIZEDPARTITION(A, p, r)

Input: Subarray $A[p..r]$.
Output: A partition around a randomly chosen pivot; returns the pivot's index.

```
1 t ← RANDOM(p, r) ;                          // uniform integer in [p, r]
2 swap A[r] ↔ A[t];
3 return PARTITION(A, p, r);
```

Algorithm 33: RandomizedSelect(A, p, r, i)

Input: Subarray $A[p..r]$; rank i with $1 \le i \le r - p + 1$, the rank *within this subarray*.
Output: The i-th smallest element of $A[p..r]$.

```
if p = r then
    return A[p] ;                                   // one element, must be the answer
q ← RandomizedPartition(A, p, r);
k ← q − p + 1 ;                                     // rank of the pivot within A[p..r]
if i = k then
    return A[q] ;                                   // the pivot is the answer
else if i < k then
    return RandomizedSelect(A, p, q − 1, i) ;       // answer is on the left
else
    return RandomizedSelect(A, q + 1, r, i − k) ; // answer on the right; shift the
     rank
```

Look carefully at the last line, because that shift is the subtle part. When we recurse *right*, the new subarray $A[q+1..r]$ has had k elements chopped off its front (positions p through q). An element that was the i-th smallest in the old subarray is now the $(i-k)$-th smallest in the new one, because k smaller elements are no longer in the picture. That is why we pass $i - k$, not i. When we recurse *left*, no elements smaller than the target were removed, so the rank i is unchanged.

Focus

Call RandomizedSelect$(A, 1, n, i)$ to find the i-th smallest of the whole array. The procedure never sorts; it only partitions and then dives into the *one* side that can contain the answer, adjusting the rank as it goes.

9.3.3 A full numeric trace

Let us run it on

$$A = [\,7,\ 3,\ 9,\ 1,\ 5,\ 8,\ 2\,], \qquad n = 7,$$

asking for the median, $i = 4$. To make the trace reproducible I will pretend the random pivot choices come out the way I describe; the point is to see the *logic*, not the dice.

Step 1. Call RandomizedSelect$(A, 1, 7, 4)$. Say the random pivot is $A[5] = 5$. Swap it to the end: $A = [7, 3, 9, 1, 2, 8, 5]$. Now partition around $x = 5$. Sweeping $k = 1 \ldots 6$: $7 > 5$ skip; $3 \le 5$ advance wall, A becomes $[3, 7, 9, 1, 2, 8, 5]$; $9 > 5$ skip; $1 \le 5$ advance wall, $[3, 1, 9, 7, 2, 8, 5]$; $2 \le 5$ advance wall, $[3, 1, 2, 7, 9, 8, 5]$; $8 > 5$ skip. Finally drop the pivot just past the wall (position 4): $A = [3, 1, 2, 5, 9, 8, 7]$, and $q = 4$.

The pivot 5 sits at index $q = 4$. Its rank within $A[1..7]$ is $k = q - p + 1 = 4 - 1 + 1 = 4$. We wanted $i = 4$. Since $i = k$, **we are done**: the median is 5. One partition, no recursion at all. That was lucky — the random pivot happened to be the median itself. Let us run a less lucky example to see the recursion fire.

A second trace, $i = 6$ (the 6th smallest). Same starting $A = [7, 3, 9, 1, 5, 8, 2]$, call RandomizedSelect$(A, 1, 7, 6)$. Suppose the random pivot is $A[2] = 3$. Swap to end: $A = [7, 2, 9, 1, 5, 8, 3]$, partition around $x = 3$. Sweep: $7 > 3$ skip; $2 \le 3$ wall to index 1, $[2, 7, 9, 1, 5, 8, 3]$; $9 > 3$ skip; $1 \le 3$

wall to index 2, $[2, 1, 9, 7, 5, 8, 3]$; $5 > 3$ skip; $8 > 3$ skip. Drop pivot at index 3: $A = [2, 1, 3, 7, 5, 8, 9]$, $q = 3$. Rank $k = 3 - 1 + 1 = 3$.

We want $i = 6$; since $6 > 3$, recurse right on $A[4\,..\,7] = [7, 5, 8, 9]$ with new rank $i - k = 6 - 3 = 3$. Call RandomizedSelect$(A, 4, 7, 3)$. Suppose the pivot is $A[6] = 8$. Swap to end ($A[7]$): $A[4\,..\,7]$ becomes $[7, 5, 9, 8]$, partition around $x = 8$. Sweep over $[7, 5, 9]$: $7 \le 8$ wall, then $5 \le 8$ wall, then $9 > 8$ skip; drop pivot: $A[4\,..\,7] = [7, 5, 8, 9]$, $q = 6$. Rank within this subarray is $k = 6 - 4 + 1 = 3$. We wanted $i = 3$ here, and $i = k = 3$, so **return** $A[6] = 8$. And indeed the sorted array is $[1, 2, 3, 5, 7, 8, 9]$, whose 6th smallest is 8. Correct.

Figure 9.2 visualizes this two-stage search, highlighting how the search window shrinks to the right subarray while the target rank adjusts from $i = 6$ to $i' = 3$.

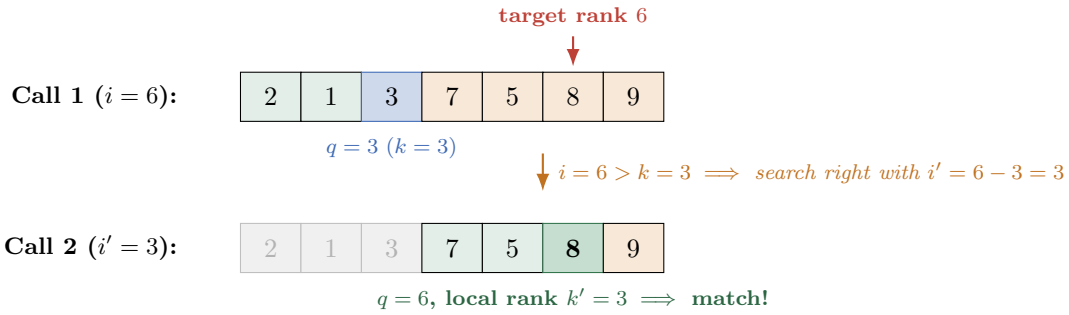


Figure 9.2: State snapshot of RandomizedSelect searching for the 6-th smallest element. After the first partition with pivot 3 ($q = 3$), the left subarray is pruned. The second call searches only $A[4\,..\,7]$ for rank $i' = 6 - 3 = 3$; pivot 8 lands at local rank 3, terminating the search.

Algorithm lab

Trace RandomizedSelect$([7, 3, 9, 1, 5, 8, 2], 1, 7, 2)$ by hand, choosing the pivots however you like, and confirm you reach 2 (the 2nd smallest). Do it a second time with *different* pivot choices and confirm you still reach 2. The pivots change the *path*; the answer never moves. That invariance — same output regardless of the coin flips — is exactly what makes a randomized algorithm trustworthy.

9.3.4 Why quickselect is correct

Correctness rests on a single invariant, proved by induction on the size of the subarray $n' = r - p + 1$.

Formal

Claim. RandomizedSelect(A, p, r, i), called with $1 \le i \le r - p + 1$, returns the i-th smallest element of $A[p\,..\,r]$.
Base case. If $p = r$ the subarray has one element, $n' = 1$, and the only legal rank is $i = 1$. The procedure returns $A[p]$, which is indeed the 1st (and only) smallest. True.
Inductive step. Assume the claim for all subarrays smaller than $A[p\,..\,r]$. After RandomizedPartition returns q, the pivot $A[q]$ is greater than exactly the $k - 1$ elements in $A[p\,..\,q - 1]$ and less than the elements in $A[q + 1\,..\,r]$, so the pivot's true rank in $A[p\,..\,r]$ is $k = q - p + 1$.

- If $i = k$, the i-th smallest is precisely the pivot; we return $A[q]$.
- If $i < k$, the i-th smallest of $A[p\,..\,r]$ lies among the $k-1$ elements strictly below the pivot, i.e. in $A[p\,..\,q-1]$, and at the *same* rank i (no smaller element was removed). That subarray is smaller, so by induction the recursive call returns the right value.
- If $i > k$, the target lies in $A[q+1\,..\,r]$. Exactly k elements (the pivot and the $k-1$ below it) were removed from the front, all of them smaller than the target, so the target's rank in the new subarray is $i-k$. Again by induction the recursive call is correct.

In all three cases the value returned is the i-th smallest of $A[p\,..\,r]$. □

9.4 How fast is quickselect?

Now to the cost. Each call does $\Theta(n)$ work to partition, then recurses on *one* side. The size of that side depends on where the pivot fell — and that is random. So we get a range of behaviors.

9.4.1 The worst case is bad

Imagine the pivot is, every single time, the smallest remaining element. Then the partition peels off just one element and recurses on the other $n-1$. The sizes go $n, n-1, n-2, \ldots, 1$, and the total work is

$$\Theta(n) + \Theta(n-1) + \cdots + \Theta(1) = \Theta\left(\sum_{j=1}^{n} j\right) = \Theta(n^2).$$

So in the worst case, quickselect is *quadratic* — no better than the naive bubble-style approaches. With random pivots this worst case is extremely unlikely (it requires the random choice to land on the extreme every time), but it is possible, and a worst-case-conscious engineer will not be satisfied. We will fix it in the next section. First, let us see why the *expected* case is linear.

9.4.2 Expected linear time: the intuition first

Intuition

A random pivot splits the array somewhere. On average it lands near the middle — not exactly at the median, but "balanced enough." A pivot is what we will call **good** if it falls in the middle half of the values, i.e. its rank is between $n/4$ and $3n/4$. The chance of that is $1/2$ (half the positions qualify). A good pivot guarantees the side we recurse into has at most $3/4$ of the elements. So *at least half the time*, the problem shrinks to at most three-quarters its size. Shrinking by a constant factor a constant fraction of the time is exactly the condition for a geometric series — and a geometric series of linear-work terms sums to linear total work.

Let us make that intuition into a clean bound, then sketch the sharper expectation argument.

A clean bound via "good" pivots. Call a partition step *lucky* if the pivot's rank lands in the middle half, between $\lceil n/4 \rceil$ and $\lfloor 3n/4 \rfloor$. There are about $n/2$ such ranks out of n, so a uniformly

random pivot is lucky with probability $\geq 1/2$. When a step is lucky, whichever side we recurse on has size at most $3n/4$ (because at least $n/4$ elements sit on each side of a middle-half pivot). Now think of the sequence of subproblem sizes. Each step does $\leq c\,n$ work for the current n. On average we need at most 2 tries to get a lucky step (a geometric random variable with success probability $1/2$ has mean 2). So, in expectation, the size shrinks by a factor of $3/4$ every 2 steps. Writing $T(n)$ for the expected work,

$$T(n) \leq T\left(\tfrac{3}{4}n\right) + (\text{expected work to reach a lucky step}).$$

The expected work to reach a lucky step is at most $2cn$ (two partitions, each $\leq cn$, since sizes only shrink). So

$$T(n) \leq T\left(\tfrac{3}{4}n\right) + 2cn.$$

Unrolling this geometric recurrence,

$$T(n) \leq 2cn\left(1 + \tfrac{3}{4} + \left(\tfrac{3}{4}\right)^2 + \cdots\right) = 2cn \cdot \frac{1}{1 - 3/4} = 2cn \cdot 4 = 8cn = \Theta(n).$$

The geometric series $\sum(3/4)^j$ converges to 4, a *constant*, so the whole thing is linear.

Focus

The reason quickselect is linear and quicksort is $n \log n$ comes down to one line: selection recurses on *one* side, so its sizes form a *geometric* (shrinking) sequence that sums to $\Theta(n)$; sorting recurses on *both* sides, so it must touch every level fully, paying $\Theta(n)$ per level across $\Theta(\log n)$ levels.

Figure 9.3 makes the contrast visual: quicksort's tree is full and fat, while quickselect walks a single shrinking path down one branch.

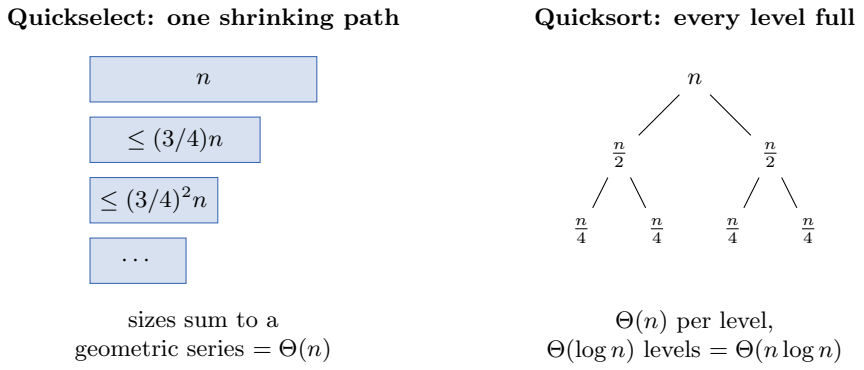


Figure 9.3: Selection descends a single branch whose sizes shrink by a constant factor, so the total work is a convergent geometric series and stays linear. Sorting must fill out the entire tree, paying linear work on each of its $\Theta(\log n)$ levels.

9.4.3 Expected linear time: the sharper analysis

The "good pivot" bound is correct and gives the right answer, $\Theta(n)$. CLRS derives the constant more precisely with indicator random variables. It is worth following, because it applies the tools from Chapter 5.

When we partition a subarray of size n, the pivot is equally likely to have any rank from 1 to n, each with probability $1/n$. If the pivot rank is k, the recursion goes left into a subarray of size $k-1$ or right into a subarray of size $n-k$ (or stops, if we got lucky and $i = k$). To get an *upper* bound on the expected time, assume the adversary always sends us into the *larger* of the two sides — that can only slow us down. The larger side has size $\max(k-1,\, n-k)$.

Let $T(n)$ be the maximum, over all i, of the expected running time on an array of n elements. Then

$$T(n) \le \frac{1}{n}\sum_{k=1}^{n} T\big(\max(k-1,\, n-k)\big) + \Theta(n).$$

As k ranges over $1, \ldots, n$, the quantity $\max(k-1, n-k)$ runs through the values $\lfloor n/2 \rfloor, \ldots, n-1$, and each such size is hit *at most twice*. So

$$T(n) \le \frac{2}{n}\sum_{m=\lfloor n/2 \rfloor}^{n-1} T(m) + \Theta(n).$$

Now guess $T(n) \le c\,n$ for a constant c to be chosen, and substitute:

$$T(n) \le \frac{2}{n}\sum_{m=\lfloor n/2 \rfloor}^{n-1} c\,m + \Theta(n) = \frac{2c}{n}\sum_{m=\lfloor n/2 \rfloor}^{n-1} m + \Theta(n).$$

The inner sum is a stretch of consecutive integers. Bounding it from above by the full sum minus the small head,

$$\sum_{m=\lfloor n/2 \rfloor}^{n-1} m = \sum_{m=1}^{n-1} m - \sum_{m=1}^{\lfloor n/2 \rfloor - 1} m \le \frac{(n-1)n}{2} - \frac{(n/2-1)(n/2)}{2} \le \frac{3n^2}{8}.$$

(You can verify the $3n^2/8$ by a little algebra; it is the cleanest constant that survives the floors.) Plugging back in,

$$T(n) \le \frac{2c}{n}\cdot\frac{3n^2}{8} + \Theta(n) = \frac{3cn}{4} + \Theta(n).$$

Write the $\Theta(n)$ term as an for a constant a. Then

$$T(n) \le \frac{3cn}{4} + an = cn - \left(\frac{cn}{4} - an\right).$$

If we choose $c \ge 4a$, the parenthesized quantity is ≥ 0, so $T(n) \le cn$, completing the induction. Hence $T(n) = \Theta(n)$: quickselect runs in **expected linear time**.

Formal

Theorem. On an array of n distinct numbers, RandomizedSelect runs in expected time $\Theta(n)$, where the expectation is over the algorithm's own random pivot choices and holds for every input and every rank i.

Cost summary

Randomized quickselect.

- Expected time: $\Theta(n)$ — linear, for *any* input.
- Worst-case time: $\Theta(n^2)$ — but only on a vanishingly unlikely sequence of pivot choices, not on any fixed input.
- Extra space: $\Theta(1)$ if you turn the tail recursion into a loop; $\Theta(\log n)$ expected stack depth otherwise. The array is permuted in place.
- Comparisons in the expected case: about $3.4n$ for the median with a careful implementation — genuinely cheaper than sorting.

Common trap

"Expected linear" is *not* "always linear." For most applications the expectation, plus the fact that bad luck is exponentially rare, is plenty. But if you are building something where a rare quadratic blowup is unacceptable — a real-time system, or code facing an adversary who can see your random seed — you need a *worst-case* guarantee. The next section gives one.

9.5 Median of medians: worst-case linear selection

Now for a classic result in algorithms: you can find the i-th order statistic in $\Theta(n)$ time *in the worst case*, with no randomness at all. This is the 1973 algorithm of Blum, Floyd, Pratt, Rivest, and Tarjan, often called SELECT or simply "median of medians."

The problem with quickselect's worst case was bad pivots. A random pivot is good *on average*, but a single unlucky run can pick the extreme every time. The fix is to stop gambling and instead *compute* a pivot that is guaranteed to be reasonably central — guaranteed to throw away a constant fraction of the array no matter what. The hard part is computing such a pivot cheaply enough to keep the total linear.

Intuition

We cannot afford to find the *true* median to use as a pivot — that is the very problem we are trying to solve, so it would be circular. But we do not need the true median. We need a pivot that is merely "okay" — guaranteed to split off at least, say, 30% of the array on each side. The median-of-medians method finds such a pivot by taking the median of a small *sample* of medians, chosen so that its quality can be proven.

9.5.1 The five steps

The algorithm SELECT(A, i) does the following.

1. **Group.** Divide the n elements into groups of 5 (the last group may have fewer). There are $\lceil n/5 \rceil$ groups.

2. **Sort each group.** Find the median of each group of 5. Since each group is tiny, you can sort it with a fixed number of comparisons — a constant amount of work per group, $\Theta(n)$ total.

3. **Recurse for the pivot.** Collect the $\lceil n/5 \rceil$ group medians into a list and *recursively call* SELECT to find *their* median. Call this value x, the "median of medians."

4. **Partition around x.** Use x as the pivot in a partition of the whole array. Let k be x's rank in A.

5. **Recurse into one side.** Exactly as in quickselect: if $i = k$ return x; if $i < k$ recurse left; if $i > k$ recurse right with rank $i - k$.

Steps 4 and 5 are pure quickselect. The new ideas are steps 1–3, which replace "pick a random pivot" with "compute a provably central pivot."

Algorithm 34: SELECT(A, p, r, i) – deterministic, worst-case linear

Input: Subarray $A[p..r]$ of distinct numbers; rank i, $1 \le i \le r - p + 1$.
Output: The i-th smallest element of $A[p..r]$.

```
1  n ← r − p + 1;
2  if n ≤ 5 then
3  |  sort A[p..r] directly and return A[p + i − 1];
4  divide A[p..r] into ⌈n/5⌉ groups of 5 (last group may be smaller);
5  for each group do
6  |  sort it and record its median;
7  let M be the array of the ⌈n/5⌉ group medians;
8  x ← Select(M, 1, |M|, ⌊(|M| + 1)/2⌋) ;                    // median of medians
9  q ← PartitionAround(A, p, r, x) ;      // partition A[p..r] using value x as pivot
10 k ← q − p + 1;
11 if i = k then
12 |  return x;
13 else if i < k then
14 |  return Select(A, p, q − 1, i);
15 else
16 |  return Select(A, q + 1, r, i − k);
```

Here PARTITIONAROUND(A, p, r, x) is just PARTITION modified to use a specified pivot *value* x (find x in the subarray, swap it to the end, and run the usual Lomuto sweep). Everything else mirrors quickselect.

9.5.2 Why the pivot is guaranteed good

This is the key step, and a picture makes it clear. Lay out the $\lceil n/5 \rceil$ groups as columns, each column sorted from smallest (bottom) to largest (top), and arrange the columns left to right by their median. The median-of-medians x sits in the middle column, at the middle height. Figure 9.4 draws a small case.

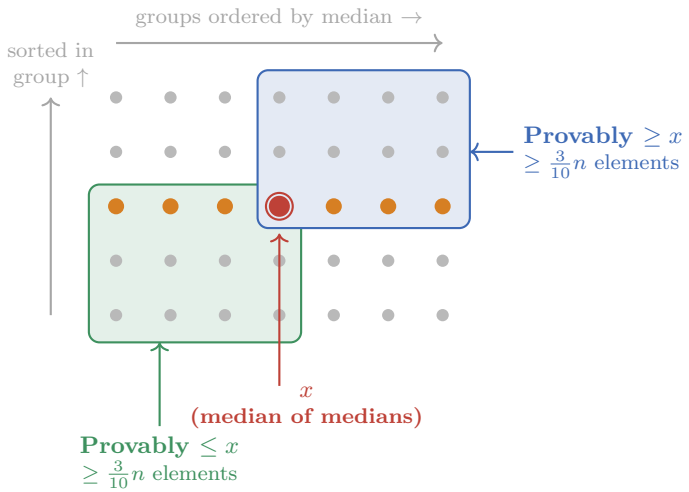


Figure 9.4: The median-of-medians 5-element column grouping. Elements are partitioned into $\lceil n/5 \rceil$ groups of 5, each column sorted upward, and columns ordered left to right by their median (orange). The median-of-medians x (circled in red) is greater than or equal to at least half the column medians, each of which is ≥ 3 elements in its group, guaranteeing $\geq \frac{3}{10}n$ elements $\leq x$ (green region). Symmetrically, $\geq \frac{3}{10}n$ elements are $\geq x$ (blue region). Consequently, any recursive call faces at most $\frac{7}{10}n$ elements.

Read the picture carefully. Each column is a group of 5, sorted so larger elements are higher. The orange dots are the group medians; columns are arranged so their medians increase left to right. The red dot x is the median of those medians.

Now look at the green block: the lower-left region. Take any column *to the left* of x's column (its median is $\leq x$). In such a column, the median and the two elements below it — that is 3 of the 5 — are all $\leq$ the column's median, hence $\leq x$. The same holds for x's own column. So in roughly half the columns, at least 3 elements are guaranteed $\leq x$. By a symmetric argument (the blue block), in roughly half the columns at least 3 elements are guaranteed $\geq x$.

Formal

How many elements are guaranteed $\leq x$? At least half of the $\lceil n/5 \rceil$ groups contribute 3 elements each that are $\leq x$. Being careful with the group containing x and a possibly-undersized last group, the count of elements guaranteed $\leq x$ is at least

$$3\left(\left\lceil \tfrac{1}{2}\left\lceil \tfrac{n}{5}\right\rceil\right\rceil - 2\right) \geq \frac{3n}{10} - 6.$$

By symmetry, at least $\frac{3n}{10} - 6$ elements are guaranteed $\geq x$. Therefore the side we recurse into in step 5 has size at most

$$n - \left(\frac{3n}{10} - 6\right) = \frac{7n}{10} + 6.$$

Focus

In one line: the pivot x is *guaranteed* to throw away at least about $\frac{3}{10}n$ elements, so the recursion never faces a subproblem bigger than $\frac{7}{10}n + 6$. No randomness, no luck — a worst-case promise. (Why groups of 5? See the box after the analysis.)

9.5.3 The recurrence, and why groups of 5 work

Now we tally the work. Let $T(n)$ be the worst-case running time of SELECT on n elements.

- Grouping into fives and finding each group's median: $\Theta(n)$, since each of the $\lceil n/5 \rceil$ groups costs a constant (sort 5 items in ≤ 7 comparisons).
- The recursive call to find the median of medians runs on $\lceil n/5 \rceil$ elements: $T(\lceil n/5 \rceil)$.
- Partitioning around x: $\Theta(n)$.
- The recursive call in step 5 runs on at most $\frac{7n}{10} + 6$ elements: $T(\frac{7n}{10} + 6)$.

Putting it together, for n above some small threshold,

$$T(n) \leq T\big(\lceil \tfrac{n}{5} \rceil\big) + T\big(\tfrac{7n}{10} + 6\big) + \Theta(n).$$

This is an unusual recurrence: it has *two* recursive calls, but on subproblems whose sizes sum to strictly less than n: $\frac{1}{5} + \frac{7}{10} = \frac{2}{10} + \frac{7}{10} = \frac{9}{10} < 1$. Because the combined size of the subproblems is only 9/10 of the parent, the total work across recursion levels contracts geometrically, summing to $\Theta(n)$.

Let us verify by the substitution method. Guess $T(n) \leq c\,n$ for all n past the threshold, with c to be fixed and the $\Theta(n)$ term written as an:

$$T(n) \leq c\lceil \tfrac{n}{5} \rceil + c\left(\tfrac{7n}{10} + 6\right) + an \leq \frac{cn}{5} + c + \frac{7cn}{10} + 6c + an.$$

Combine the n terms: $\frac{cn}{5} + \frac{7cn}{10} = \frac{2cn+7cn}{10} = \frac{9cn}{10}$. So

$$T(n) \leq \frac{9cn}{10} + 7c + an = cn - \left(\frac{cn}{10} - 7c - an\right) = cn - \left(\left(\tfrac{c}{10} - a\right)n - 7c\right).$$

We need the bracket ≥ 0, i.e. $(\frac{c}{10} - a)n \geq 7c$. Choose $c = 20a$, say. Then $\frac{c}{10} - a = 2a - a = a$, and the condition becomes $an \geq 7c = 140a$, i.e. $n \geq 140$. So for all $n \geq 140$ (handle smaller n by the brute-force base case), $T(n) \leq cn = 20an$. Hence

$$\boxed{T(n) = \Theta(n)} \qquad \text{worst case.}$$

Formal

Theorem (Blum–Floyd–Pratt–Rivest–Tarjan, 1973). The deterministic SELECT algorithm finds the i-th order statistic of n distinct numbers in $\Theta(n)$ time in the *worst case*.

Walking question

Why groups of 5, and not 3 or 7? Redo the recurrence with groups of g. The sample has n/g medians, so the pivot call is $T(n/g)$; the guaranteed discard is about $\frac{1}{2} \cdot \lceil g/2 \rceil$ per group, giving a surviving side of about $(1 - \frac{1}{2} \cdot \frac{\lceil g/2 \rceil + 1}{g})\, n$. For the two children to sum to less than n you need $\frac{1}{g}$ + (survivor fraction) < 1. Check $g = 3$: the fractions come out to $\frac{1}{3} + \frac{2}{3} = 1$ — exactly 1, *not* less, so the proof breaks and you do *not* get linear time. Check $g = 7$: it works, but the constant is worse and sorting groups of 7 is fiddlier. Five is the smallest odd group size that makes the two subproblem fractions sum strictly below 1. That is why the algorithm chooses groups of five.

Common trap

A tempting "simplification" is to use the median of medians *without* the recursion in step 3 — say, pick the median of a fixed-size sample. That gives a fast *expected* bound but loses the worst-case guarantee, because a small fixed sample can be unrepresentative. The recursive median-of-medians is what makes the pivot's quality *provable for every input*. Without the recursion, the theorem no longer holds.

9.5.4 Comparing the two selection algorithms

You now have two linear-time selection algorithms with very different practical trade-offs.

	Randomized quickselect	**Median of medians**
Expected time	$\Theta(n)$	$\Theta(n)$
Worst-case time	$\Theta(n^2)$	$\Theta(n)$
Randomness	yes	none (deterministic)
Hidden constant	small ($\approx 3.4n$ cmps)	large (often $> 20n$)
In practice	usually faster	a safety net / pivot oracle

In practice, randomized quickselect is the one you reach for; its constants are small and its bad case almost never happens. Median of medians is worth its overhead when you cannot tolerate *any* quadratic risk, or as the *pivot oracle inside* an otherwise-randomized quicksort, giving a sort that is $\Theta(n \log n)$ in the worst case ("introselect" and "introsort" use exactly this hybrid idea).

Why this matters

The broader lesson applies beyond this algorithm. Median of medians is the textbook example of **spending a little work to guarantee good structure, then proving the guarantee pays for itself**. The recurrence $T(n) \le T(n/5) + T(7n/10) + \Theta(n)$ — two children summing to under 1 — is a pattern you will meet again whenever a clever preprocessing step shrinks two pieces that together are smaller than the whole. Recognizing "the children sum to less than the parent, so it is linear" is more useful to remember than the specific constant 5.

History and connection

In 1961, C.A.R. (Tony) Hoare published Algorithm 65 (`find`) in the *Communications of the ACM*, showing that his quicksort partitioning technique could select the k-th smallest element in $\Theta(n)$ expected time simply by recursing into only the one partition containing the target rank. But just like quicksort, Hoare's Quickselect had a weakness: on an adversarial or unfortunate input, it could degenerate into $\Theta(n^2)$ quadratic time. For over a decade, computer scientists debated whether it was mathematically possible to find the median of an unsorted list deterministically in guaranteed $O(n)$ worst-case time. In 1973, Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan (BFPRT) solved the open problem with their median-of-medians algorithm.

The key choice in BFPRT is the group size. Why divide into groups of 5? If you try groups of 3, the recursive recurrence becomes $T(n) \leq T(n/3) + T(2n/3) + O(n)$. Because $1/3 + 2/3 = 1$, the subproblem sizes do not diminish geometrically; each level of the recursion tree does $c \cdot n$ work across $\log n$ levels, summing to $\Theta(n \log n)$ rather than linear time. But with groups of 5, the median-of-medians guarantees that at least $3/10$ of the elements are smaller than the pivot and at least $3/10$ are larger, leaving at most $7n/10$ in the recursive step. The recurrence becomes $T(n) \leq T(n/5) + T(7n/10) + O(n)$. Because $1/5 + 7/10 = 9/10 < 1$, the total work across levels shrinks like a decaying geometric series: $n + \frac{9}{10}n + (\frac{9}{10})^2 n + \cdots \leq 10n$, giving a deterministic $\Theta(n)$ bound.

In practice, however, BFPRT is almost never used in production software. The bookkeeping of grouping into fives, copying subarrays, and recursive function calls creates a large constant factor. In practice, randomized Quickselect (or David Musser's Introselect, which runs Quickselect and falls back to median-of-medians only if recursion depth exceeds $2 \log n$) runs two to three times faster on real hardware. BFPRT proved that linear-time selection is possible deterministically, providing the theoretical guarantee for worst-case $O(n \log n)$ quicksort. In Chapter 12, we explore Binary Search Trees to see how maintaining subtree counts enables dynamic rank queries; in Chapter 14, we augment Red-Black Trees into Order-Statistic Trees that select elements in $O(\log n)$ time even as data is inserted and deleted; and in Chapter 32, we use median finding to split geometric planes for spatial divide-and-conquer.

Algorithm lab

Implement both algorithms and *count comparisons* on the same inputs.

1. Generate 10,000 random permutations of $1 \,..\, 1001$ and select the median with quickselect. Plot a histogram of the comparison counts. You should see them cluster tightly just above $3n$, with no extreme outliers — concrete evidence of how rarely random pivots produce bad splits.

2. Feed quickselect a *sorted* array with the "always pick the last element" (non-randomized) pivot rule and watch it crawl to $\Theta(n^2)$. Then turn the randomization back on and watch the problem vanish. This is the most direct demonstration of why we randomize.

3. Run median of medians on the same inputs. Confirm it never spikes — but also confirm its constant is large, so for everyday work quickselect wins on wall-clock time despite its scarier worst case.

So far, our algorithms have operated on static arrays: we loaded an array, sorted it or selected from it, and produced an answer. But in running systems, items arrive unpredictably, priorities shift, and

records must be inserted, queried, and deleted on the fly. In Part 3, beginning with Chapter 10, we introduce data structures that support these dynamic operations efficiently: stacks, queues, linked lists, and rooted trees.

9.6 Exercises

Exercise 9.1. For $A = [\,12, 4, 17, 9, 1, 20, 7, 15\,]$, write the sorted order and give: the minimum, the maximum, the 3rd order statistic, the 6th order statistic, and the (lower) median. For each, state both its *value* and the *index* it occupies in the original (unsorted) A.

Exercise 9.2. Trace RandomizedSelect($[\,8, 2, 11, 6, 4, 9, 1, 7\,], 1, 8, 5$) by hand. At each partition, choose the pivot to be the current *first* element of the subarray (so the trace is deterministic and reproducible). Show the array after each partition, the value q, the rank k, and which branch you take. Confirm the final answer equals the 5th smallest element.

Exercise 9.3. Flesh out the walking-question pairing trick: write pseudocode that finds both the minimum and the maximum of n numbers using at most $\frac{3}{2}n$ comparisons. Prove the comparison count carefully, handling the cases n even and n odd separately. (Hint: the parity of n changes how you seed the running min and max.)

Exercise 9.4. You can find the minimum of n numbers in $n - 1$ comparisons. Show that you can find the *second*-smallest in only $n + \lceil \log_2 n \rceil - 2$ comparisons. (Hint: run a knockout tournament to find the minimum; the second-smallest must be one of the $\lceil \log_2 n \rceil$ elements that lost directly to the eventual minimum.)

Exercise 9.5. Describe an input and a (non-randomized) pivot rule under which quickselect performs $\Theta(n^2)$ comparisons while searching for the median. Then explain in one or two sentences why RandomizedPartition makes such an input harmless.

Exercise 9.6. Repeat the median-of-medians analysis with groups of **3** instead of 5. Show that the recurrence becomes $T(n) \le T(\lceil n/3 \rceil) + T(2n/3 + c) + \Theta(n)$ for a constant c, that the two child sizes sum to about n, and argue informally that this fails to guarantee linear time. (This is the precise reason the algorithm uses 5, not 3.)

Exercise 9.7. The **weighted (lower) median** of n distinct elements $x_1, \ldots, x_n$ with positive weights $w_1, \ldots, w_n$ summing to 1 is the element x_k such that the total weight of elements smaller than x_k is less than $1/2$ and the total weight of elements larger than x_k is at most $1/2$. Design a $\Theta(n)$ worst-case algorithm to find it. (Hint: partition around the median-of-medians pivot, but base the recurse-left-or-right decision on accumulated *weight*, not count.)

Exercise 9.8. Generalize selection to the "quantiles" problem: given A and an integer $q \ge 2$, output the $q - 1$ order statistics that divide the sorted data into q groups of as-equal-as-possible size. Give an algorithm running in $\Theta(n \log q)$ time and justify the bound. (Hint: select the overall median to split the work, then recurse on each half for the quantiles that fall in it; write and solve the resulting recurrence.)

Exercise 9.9. Given n distinct numbers, a target value t, and an integer k, give an $\Theta(n)$-time algorithm that outputs the k numbers closest to t (ties broken arbitrarily). The output need not be sorted. (Hint: "closeness to t" is just $|x - t|$; selection works on any key, not only the values themselves.)

Exercise 9.10. Two sorted arrays $X[1\,..\,n]$ and $Y[1\,..\,n]$ together hold $2n$ distinct numbers. Give an $\Theta(\log n)$-time algorithm to find the median of the combined $2n$ elements *without merging them.* (Hint: compare the medians of X and Y; whichever is smaller, you can safely discard its lower half and the other array's upper half, halving the problem each step.) Explain why this beats the $\Theta(n)$ selection algorithms for this special, already-sorted case.

Part III

Data Structures

Chapter 10

Elementary Data Structures

So far in this book you have spent most of your energy on *algorithms*: recipes that take some input and grind out an answer. Insertion sort, merge sort, quicksort, the order-statistic selection trick — all of them assumed that the data was just *sitting there*, in an array, ready to be poked at. We never asked the obvious follow-up question:

How is the data actually stored, and what does that storage let us do quickly?

That question is the subject of this chapter. A **data structure** is a specific way of organizing data in a computer's memory so that certain operations — inserting a new item, deleting an old one, finding the item you want, asking "what is next?" — become cheap. Choosing the right one is often the difference between a program that finishes in a blink and one that never finishes at all. The structures here are called *elementary* not because they are unimportant, but because nearly every fancier structure later in the book (hash tables, balanced trees, heaps, graphs) is built out of these few primitives. Master these and you have the vocabulary for everything that follows.

Here, we build the basic vocabulary of dynamic storage: arrays and matrices, stacks and queues, linked lists, and rooted trees.

Focus

A data structure combines two things: a *memory layout* (how bytes are arranged in memory) and an *interface contract* (which operations it supports, and at what cost). When you compare data structures, you are really comparing the costs of their contracts. Keep a running table in your head: for each structure, what does insert cost? Delete? Search? Access-by-position?

10.1 Memory, Addresses, and the Array

Before any data structure, there is memory. Picture your computer's memory as an enormously long street of identical houses, numbered $0, 1, 2, 3, \ldots$ up into the billions. Each house holds one small fixed-size value (think of it as one byte, or for our purposes one "cell"). The house number is called its **address**. The single most important fact about this street is:

Intuition

Given an address, the hardware can fetch the value living there in *constant time* — one step, no matter how large the address. The hardware does *not* scan down the street counting houses; address-decoding circuitry jumps directly to that cell. This is why it is called *random-access* memory: any address is as cheap to reach as any other.
Every $\Theta(1)$ lookup in this chapter relies on computing an address directly; every $\Theta(n)$ scan happens when we have to follow elements one by one.

An **array** is the most direct possible use of that capability. You ask for a contiguous block of n cells — n houses in a row — and you get back the address of the first one, call it *base*. Now suppose each element of your array takes s cells of space (an integer might be $s = 4$ cells, say). Then element number i lives at address

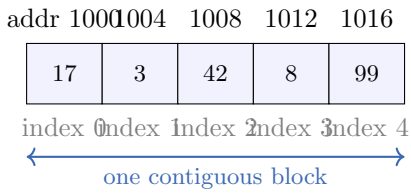
$$base + i \cdot s.$$

That is a single multiplication and a single addition — a constant amount of arithmetic — followed by one direct memory fetch. So reading or writing $A[i]$ costs $\Theta(1)$, *regardless of which i you ask for and regardless of how big the array is.*

Focus

The defining strength of an array is **random access by index in $\Theta(1)$ time**, via the address formula $base + i \cdot s$. This is why we have been able to write $A[i]$ in pseudocode all book long without worrying about cost. Now you know *why* it is free.

A concrete picture nails down the address arithmetic.



Here $base = 1000$ and each element takes $s = 4$ cells (a typical 32-bit integer). The element $A[3] = 8$ lives at $1000 + 3 \cdot 4 = 1012$, exactly where the picture says. Notice index 3 was no harder to reach than index 0: both are one multiply, one add, one fetch.

Example 10.1 (Addressing by hand)**.** An array of 64-bit floating point numbers ($s = 8$ cells) starts at address $base = 200000$. Where does $A[10]$ live? Plug in:

$$200000 + 10 \cdot 8 = 200000 + 80 = 200080.$$

And $A[11]$ is at 200088, just 8 cells further along. Consecutive elements sit s cells apart — that adjacency is what "contiguous" means.

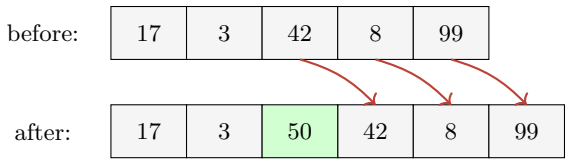
Common trap

Zero-based versus one-based indexing. CLRS and much of this book write arrays as $A[1..n]$, starting at 1. Most real programming languages (C, Java, Python) start at 0, so the

first element is $A[0]$ and the last is $A[n-1]$. The address formula adapts simply: for one-based, element i is at $base + (i-1)\cdot s$. The danger is the *off-by-one* bug, the most common mistake in all of programming: writing $\leq n$ where you meant $< n$, or forgetting to subtract 1. Whenever you write a loop over an array, pause and check both endpoints explicitly: "Does it touch the very first element? Does it touch the very last? Does it run one too far?"

10.1.1 The price of insertion and deletion

Instant random access is fast, but arrays have a hard limit baked into that contiguity. Suppose you have the array $[17, 3, 42, 8, 99]$ and you want to *insert* the value 50 so that it sits at index 2 (pushing $42, 8, 99$ rightward). There is no spare house between index 1 and index 2; the block is packed solid. So you must *shove* every element from index 2 onward one house to the right to make a gap:



In the worst case — inserting at the very front — you have to shift all n elements. So array insertion costs $\Theta(n)$ in the worst case. Deletion is the mirror image: to remove the element at index i, you slide everything after it one house to the left to close the gap, again $\Theta(n)$ in the worst case. The one happy exception is inserting or deleting at the *end*: nothing needs to move, so it is $\Theta(1)$ (provided there is room).

Cost summary

Array, costs. Let n be the number of elements.

Operation	Worst-case time
Access / update $A[i]$	$\Theta(1)$
Search for a value (unsorted)	$\Theta(n)$
Insert / delete at the end	$\Theta(1)$
Insert / delete in the middle	$\Theta(n)$

The array trades flexibility for raw access speed. It is the perfect choice when your data is fixed in size and you mostly read it, and a poor choice when items are constantly being inserted and removed in the middle.

10.2 Matrices: arrays in two dimensions

A **matrix** is a grid of values with m rows and n columns; we write $A[i, j]$ for the entry in row i, column j. But memory is a one-dimensional street — there is no two-dimensional memory. So we must *flatten* the grid into a single line of houses. There are two standard ways to do it, and they differ only in the order they visit the cells.

Row-major order lays down the entire first row, then the entire second row, and so on:

$$A[1,1],\ A[1,2],\ \ldots,\ A[1,n],\quad A[2,1],\ A[2,2],\ \ldots$$

Column-major order lays down the first column, then the second, and so on. C, Python (NumPy by default), and most languages use row-major; Fortran and MATLAB use column-major. The choice changes only the address formula. For an $m \times n$ matrix in row-major order with one-based indices, the entry $A[i,j]$ sits at offset

$$\mathit{base} + \big[(i-1)\cdot n + (j-1)\big]\cdot s.$$

Example 10.2 (Locating a matrix entry)**.** Take a 3×4 matrix (so $m = 3$, $n = 4$), stored row-major with $s = 1$ cell per entry and $\mathit{base} = 0$. Where is $A[2,3]$? The full row 1 uses indices $0, 1, 2, 3$ (that is $n = 4$ cells). Then within row 2 we step to column 3. The formula gives offset $(2-1)\cdot 4 + (3-1) = 4 + 2 = 6$. Lay out the flattened order to check:

$$\underbrace{A_{11}, A_{12}, A_{13}, A_{14}}_{\text{offsets } 0..3},\ \underbrace{A_{21}, A_{22}, \mathbf{A_{23}}, A_{24}}_{\text{offsets } 4..7},\ \ldots$$

Indeed $A[2,3]$ lands at offset 6. The formula and the picture agree.

Why this matters

The choice between row-major and column-major order can make real programs run several times faster or slower. Modern CPUs fetch memory in chunks ("cache lines"), so reading cells that are *adjacent in memory* is far cheaper than jumping around. Loop over a row-major matrix row by row and you stride through memory in order, which the cache rewards. Loop column by column over that same matrix and you leap n cells on every step, defeating the cache and slowing the loop dramatically. The math is identical; the running time is not. This is why numerical libraries care intensely about memory layout.

10.3 Stacks: last in, first out

Now we take an array and impose a *discipline* on it. A **stack** is a collection where you may only add and remove from *one end*, called the top. The rule is **LIFO**: last in, first out. Think of a stack of dinner plates: you add a clean plate to the top, you take the next plate from the top, and you never reach into the middle. The last plate placed is the first one taken.

Intuition

A stack remembers things in reverse order of when they arrived. That sounds restrictive, but it is exactly the memory you want whenever you are working on something, get interrupted by a sub-task, and need to come back to where you were. Reading a book and following a footnote, which itself has a footnote? Your brain is running a stack. Pending function calls in a running program? A stack (literally called *the* call stack). Matching brackets in an expression? A stack.

A stack supports three operations:

- PUSH(S, x) — put x on top.
- POP(S) — remove and return the top element.
- STACK-EMPTY(S) — is the stack empty?

Figure 10.1 illustrates the pointer mechanics. The array stays put; only $S.top$ moves.

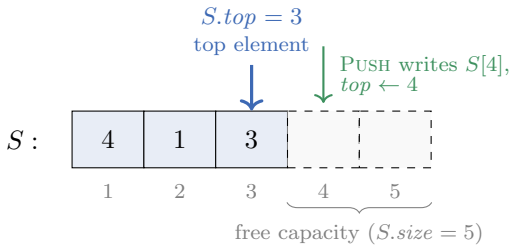


Figure 10.1: A stack implemented on a fixed array $S[1\,..\,5]$. Live elements occupy slots 1 through $S.top = 3$. PUSH increments $S.top$ and writes into the next free slot (green arrow); POP reads $S[S.top]$ and decrements $S.top$. The array elements never shift, giving $\Theta(1)$ worst-case time for all operations.

In Figure 10.1, the stack holds $[4, 1, 3]$ with $top = 3$; the next PUSH writes into slot 4 and bumps top to 4, while the next POP returns slot 3 and drops top to 2. The dashed cells are reserved but unused.

Algorithm 35: Stack operations on an array

```
1  STACK-EMPTY(S)
2      if S.top == 0 then return TRUE
3      else return FALSE

4  PUSH(S, x)
5      if S.top == S.size then
6          return overflow                          // array is full
7      S.top ← S.top + 1
8      S[S.top] ← x

9  POP(S)
10     if STACK-EMPTY(S) then
11         return underflow                         // nothing to pop
12     S.top ← S.top − 1
13     return S[S.top + 1]
```

Every line here is constant-time arithmetic and a single array access, so **push, pop, and empty-test all cost $\Theta(1)$.** That is the main appeal of a stack: its operations are about as cheap as operations get.

Algorithm lab

Trace a stack by hand. Start empty and perform, in order: PUSH(4), PUSH(1), PUSH(3), POP, PUSH(8), POP, POP.

We track the array contents and the value of *top* after each step. Bold is the current top.

Operation	Contents (bottom → top)	*top*
PUSH(4)	[**4**]	1
PUSH(1)	[4, **1**]	2
PUSH(3)	[4, 1, **3**]	3
POP → 3	[4, **1**]	2
PUSH(8)	[4, 1, **8**]	3
POP → 8	[4, **1**]	2
POP → 1	[**4**]	1

Notice the popped values come out $3, 8, 1$ — exactly the reverse of the most recent pushes. Convince yourself that after the last step the stack still holds 4 at the bottom, untouched since the very first push.

Common trap

Two errors to guard against. **Underflow**: popping an empty stack. Always check STACK-EMPTY first, or you will read garbage at $S[0]$. **Overflow**: pushing onto a full fixed-size array. In the pseudocode above we return an error; in practice you would grow the array (more on that in the amortized-analysis chapter, where "doubling" makes the average push cost $\Theta(1)$ even with growth). For now, just remember that a fixed array stack has a ceiling.

10.4 Queues: first in, first out

A **queue** flips the discipline around. You add at one end (the **tail**) and remove from the *other* end (the **head**). The rule is **FIFO**: first in, first out — exactly like a line of people waiting for coffee. The person who has waited longest is served next; newcomers join the back.

- ENQUEUE(Q, x) — add x at the tail.
- DEQUEUE(Q) — remove and return the element at the head.

The implementation is subtler than a stack, because the queue "crawls" down the array: every dequeue advances the head, every enqueue advances the tail, so the live region drifts rightward and would eventually fall off the end of a finite array. The fix is to make the array **circular** (or "wrap around"): when an index reaches the end, it wraps back to the start, as if the array were a ring. We keep two indices, $Q.head$ (where the next dequeue happens) and $Q.tail$ (where the next enqueue will write). Figure 10.2 illustrates the ring buffer and pointer movement.

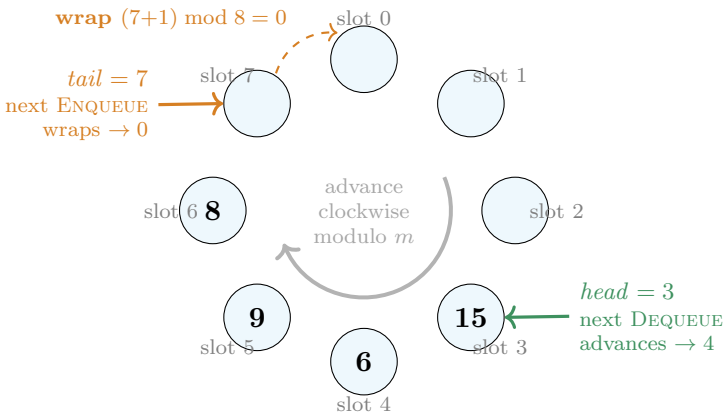


Figure 10.2: The queue as a circular ring buffer ($m = 8$). Live elements $(15, 6, 9, 8)$ occupy contiguous slots 3 through 6. Dequeue reads from $Q[head]$ and advances $head$ clockwise; Enqueue writes into $Q[tail]$ and advances $tail$. Modular arithmetic $(tail + 1) \bmod m$ wraps around from slot 7 back to 0 (dashed orange arrow), allowing the buffer to reuse space continuously in $\Theta(1)$ time without shifting elements.

In Figure 10.2, the live elements $15, 6, 9, 8$ sit at indices $3, 4, 5, 6$; the next dequeue returns 15 (at $head = 3$), and the next enqueue writes at $tail = 7$. The next index after 7 wraps around to 0. We do the wrap with modular arithmetic: "the index after i" in an array of size m is $(i \bmod m) + 1$ in one-based terms, or simply $(i + 1) \bmod m$ in zero-based terms.

Algorithm 36: Queue operations on a circular array $Q[1..m]$

```
1 Enqueue(Q, x)
2     Q[Q.tail] ← x
3     if Q.tail == Q.size then Q.tail ← 1
4     else Q.tail ← Q.tail + 1
5 Dequeue(Q)
6     x ← Q[Q.head]
7     if Q.head == Q.size then Q.head ← 1
8     else Q.head ← Q.head + 1
9     return x
```

Both operations are pure constant-time index arithmetic: **enqueue and dequeue cost $\Theta(1)$.**

Algorithm lab

Trace a circular queue of size $m = 4$ (one-based indices 1..4), starting with $head = 1$, $tail = 1$ (empty). Perform: Enq(7), Enq(2), Enq(5), Deq, Enq(9), Enq(1).

Operation	Array (slots 1..4)	head	tail
start	[_,_,_,_]	1	1
ENQ(7)	[7,_,_,_]	1	2
ENQ(2)	[7,2,_,_]	1	3
ENQ(5)	[7,2,5,_]	1	4
DEQ $\to$ 7	[7,2,5,_]	2	4
ENQ(9)	[7,2,5,9]	2	**1 (wrapped!)**
ENQ(1)	[**1**,2,5,9]	2	2

Watch the wrap: after writing 9 into slot 4, *tail* jumps back to 1. The final ENQ(1) overwrites slot 1 — which is legal because slot 1's old value (7) had already been dequeued. The live queue is now $2, 5, 9, 1$ reading from head. Notice slot 1 got reused without ever moving any data: that is what the circular layout buys you.

Common trap

Full versus empty ambiguity. A subtle classic: in a circular array of size m, both a completely full queue and a completely empty queue can have $head == tail$. You cannot distinguish them from the indices alone! The standard fixes are (a) keep a separate count of live elements, or (b) never let the queue hold more than $m - 1$ elements so $head == tail$ unambiguously means "empty." Pick one and be consistent, or you will dequeue from an empty queue or silently overwrite live data.

10.5 Deques: open at both ends

A **deque** (pronounced "deck," short for *double-ended queue*) is the natural generalization: you may insert and delete at *either* end. It subsumes both the stack and the queue — use only one end and you have a stack; insert at one end and remove at the other and you have a queue. With a circular array and two indices it supports all four operations — push-front, push-back, pop-front, pop-back — in $\Theta(1)$ time each. Deques return when we need a "sliding window" over a sequence; the point to remember is that a structure open at both ends can still run in constant time per operation.

Walking question

You are given only a stack (push/pop) as a black box, and you want to build a queue (enqueue/dequeue) out of it. A single stack reverses order, which is the *opposite* of what a queue does — so one stack alone cannot work. But *two* stacks can. Can you figure out the trick? Hint: reversing a reversal gives you back the original order. What happens if you pour elements from one stack into another, one pop-then-push at a time? (This "two stacks make a queue" construction is a famous interview question, and it has a beautiful amortized cost of $\Theta(1)$ per operation — which you will be able to prove after the amortized-analysis chapter.)

10.6 Pointers and objects: escaping contiguity

Every structure so far has leaned on the array's contiguous block. That block is exactly what makes random access fast — and exactly what makes middle-insertion slow, because there is no slack between neighbors. What if we gave up on contiguity? What if each element could live *anywhere* in memory, and simply held a note saying "the next element is over there"?

That note is called a **pointer** (or reference): a value that is itself an address. An **object** is a small bundle of cells holding several named fields, one or more of which may be pointers to other objects. Given pointers, data structures stop being rigid blocks and become *webs*: objects scattered across memory, wired together by pointers.

Intuition

A pointer is an arrow. An object is a box with labeled slots, and some slots hold arrows pointing at other boxes. Drawing a data structure becomes literally drawing boxes and arrows. When you "follow a pointer," you teleport (constant time!) to wherever it aims. The cost is that you can no longer compute where the i-th element lives by arithmetic — there is no $base + i \cdot s$ formula across a web of scattered boxes. To reach the i-th element you must *follow* i *arrows*, one box at a time.

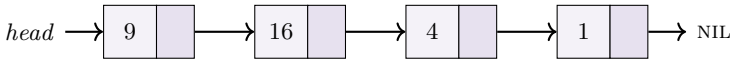
We will write `x.next` for "the field named *next* inside the object that x points to," and we use the special value **nil** (sometimes written $\perp$ or `null`) for "a pointer that points at nothing." NIL is how we mark the end of a chain or an empty structure — it is the period at the end of the sentence.

Common trap

The **null pointer** (NIL) is the single most prolific source of crashes in real software — so notorious its inventor, Tony Hoare (who also gave us quicksort), called it his "billion-dollar mistake." The trap: trying to follow a pointer that is NIL, i.e. asking for `x.next` when x itself is NIL. There is no box there to look inside. Every single time you follow a pointer in a loop, you must first ask: "Could this be NIL?" The loop condition `while x != NIL` exists precisely to guard against this.

10.7 Linked lists

A **linked list** is the simplest pointer structure: a chain of objects, each called a **node**, where every node holds a *key* (the actual data) and a pointer *next* to the following node. The list as a whole is identified by a pointer *head* to its first node. The last node's *next* is NIL, marking the end.



This is a **singly linked list**: each arrow points only forward. There is also the **doubly linked list**, where each node additionally carries a *prev* pointer to its predecessor, letting you walk backward as well as forward. Doubly linked lists make deletion much cleaner, as we will see, at the cost of one extra pointer per node.

10.7.1 Searching a list

How do you find a key in a linked list? There is no address arithmetic to jump to the middle — you have no choice but to start at the head and follow *next* pointers one node at a time until you find the key or hit NIL.

Algorithm 37: LIST-SEARCH(L, k) — find the first node with key k

Input: a linked list L and a key k
Output: a pointer to a node with key k, or NIL if none exists

```
1 x ← L.head
2 while x ≠ NIL and x.key ≠ k do
3     x ← x.next
4 return x
```

In the worst case (the key is absent, or sits at the very end) you visit all n nodes, so search costs $\Theta(n)$ — the same as an unsorted array, but *slower in practice* because you follow scattered pointers instead of striding through contiguous cache-friendly memory. Linked lists earn their advantage elsewhere, in insertion and deletion.

Formal

Loop invariant for List-Search. *At the start of each iteration of the while loop, none of the nodes strictly before x in the list have key k.*
Initialization. Before the first iteration $x = L.head$, so there are no nodes before x at all; the claim holds vacuously.
Maintenance. Suppose the invariant holds at the top of an iteration and the loop body runs. The body runs only when $x \neq$ NIL and $x.key \neq k$, so the current node also fails to hold k. We then set $x \leftarrow x.next$, advancing past the current node. Now *every* node before the new x — those before the old x (by hypothesis) plus the old x itself (just checked) — fails to hold k. The invariant is restored.
Termination. The loop ends either because $x =$ NIL (we walked off the end; by the invariant no earlier node held k, and there are no later ones, so k is absent — correctly returning NIL) or because $x.key = k$ (we found it, and return that node). Either way the output is correct.

10.7.2 Inserting into a list

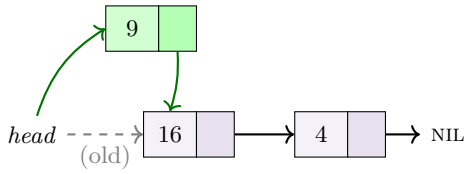
Here is where linked lists shine. To insert a new node at the *front* of the list, you do not shift anything — you rewire two pointers. Make the new node point at the old head, then make *head* point at the new node:

Algorithm 38: LIST-PREPEND(L, x) — insert node x at the front

Input: a linked list L and a fresh node x to insert

```
1 x.next ← L.head
2 L.head ← x
```

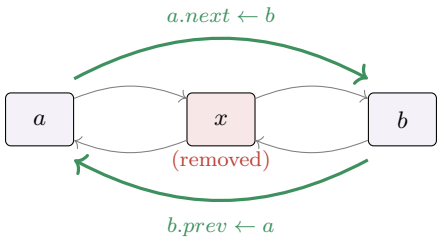
Two pointer assignments, no shifting, no matter how long the list is: **front insertion is $\Theta(1)$.** Contrast that with an array, where front insertion was $\Theta(n)$ because everything had to slide. This is the main reason to use a linked list.



The dashed gray arrow shows where *head* used to point; the two green arrows are the rewiring. Nothing else in the list moved a single byte.

10.7.3 Deleting from a list

To delete a node, we splice it out: make its predecessor's *next* skip over it and point straight at its successor (and, in a doubly linked list, make the successor's *prev* point back at the predecessor). In a **doubly** linked list this is wonderfully direct, because every node knows its own predecessor through *prev* — no searching required.



The gray arrows are the original links through x; the two bold green arrows are the new links that bypass x. Once nothing points at x, it is gone from the list.

Algorithm 39: LIST-DELETE(L, x) — remove node x from a doubly linked list

Input: a doubly linked list L and a pointer to the node x to remove

```
1 if x.prev ≠ NIL then x.prev.next ← x.next
2 else L.head ← x.next                                  // x was the head
3
4 if x.next ≠ NIL then
5     x.next.prev ← x.prev
```

Given the pointer x to the doomed node, deletion is a handful of pointer rewires — $\Theta(1)$ **time**, again independent of list length. (The IF/ELSE branches just handle the boundary cases where x is the first node, which has no predecessor, or the last, which has no successor.) Compare the array, where deletion in the middle forced an $\Theta(n)$ slide.

Common trap

That last paragraph hid an assumption: "*given the pointer* x." If you only know the *key* you want to delete, you must first LIST-SEARCH for it — which is $\Theta(n)$ — and *then* splice it out in $\Theta(1)$. So delete-by-key is $\Theta(n)$ overall; only delete-by-pointer is truly $\Theta(1)$. People love to claim "linked list deletion is $O(1)$" and forget the fine print. In a *singly* linked list it is worse still: even with a pointer to x you cannot reach $x.prev$ to rewire it, so you must walk from the

head to find the predecessor — $\Theta(n)$ again. This single fact is the best argument for paying the extra pointer per node and going doubly linked.

10.7.4 The sentinel trick

All those NIL-checking IF/ELSE branches in LIST-DELETE are ugly and bug-prone. A beautiful simplification is the **sentinel**: a single dummy node, $L.nil$, that has no real key but sits permanently in the list as both the node *after* the last real node and the node *before* the first. The list becomes *circular*, with the sentinel stitching the two ends together. Now there are no boundaries: every real node always has a genuine predecessor and successor (possibly the sentinel itself), so the special cases vanish and LIST-DELETE collapses to just two lines:

$$x.prev.next \leftarrow x.next; \qquad x.next.prev \leftarrow x.prev.$$

Figure 10.3 shows the layout: the circular sentinel structure eliminates boundary checks because every node, including the head and tail, always has valid neighbors.

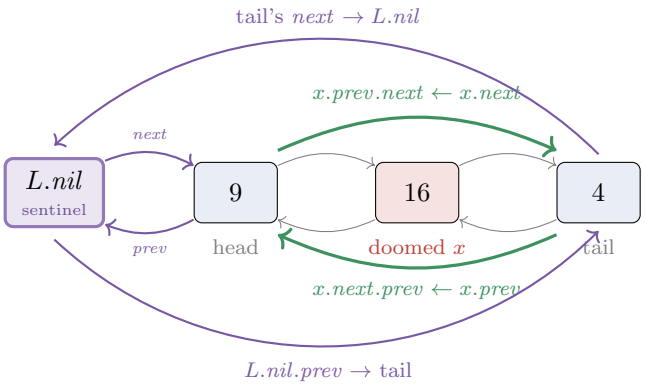


Figure 10.3: The sentinel trick in a circular doubly linked list. The dummy node $L.nil$ stitches the head and tail together into a closed ring: $L.nil.next$ points to the head, and $L.nil.prev$ points to the tail. Deleting any node x — whether in the middle, at the head, or at the tail — executes the exact same two pointer assignments ($x.prev.next \leftarrow x.next$ and $x.next.prev \leftarrow x.prev$, bold green arrows), completely eliminating null-pointer checks and edge-case branching.

The cost is one wasted node's worth of memory and a little care during initialization. For lists where you do many insertions and deletions, the cleaner, branch-free code is well worth it.

Cost summary

Doubly linked list, costs. Let n be the number of nodes.

Operation	Worst-case time
Access i-th element	$\Theta(n)$
Search for a key	$\Theta(n)$
Insert at front / back†	$\Theta(1)$
Delete a node (given its pointer)	$\Theta(1)$
Delete by key	$\Theta(n)$

†Back insertion is $\Theta(1)$ if you keep a *tail* pointer; otherwise you must walk to the end, $\Theta(n)$. The list is the array's complement: it gives up $\Theta(1)$ random access to gain $\Theta(1)$ structural edits.

Why this matters

When should you actually reach for a linked list over an array? When your workload is dominated by inserting and deleting at known positions and you rarely need to jump to the i-th element. The textbook real-world example is a music or video *playlist* with constant reordering, or the list of free memory blocks an operating system splices in and out as programs allocate and free memory. But heed a modern caveat: because array elements sit contiguously and linked-list nodes are scattered, arrays are so much friendlier to the CPU cache that in practice a simple array often beats a linked list *even for tasks the list is supposedly better at*, until n gets quite large. Asymptotics tell you the trend; they do not always tell you the winner at the sizes you care about. Measure.

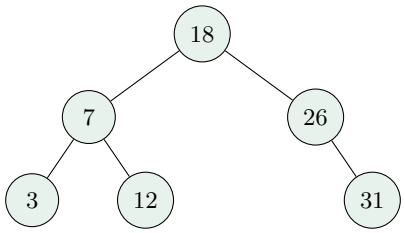
10.8 Representing rooted trees

We close with the structure that dominates the rest of the book: the **rooted tree**. A tree is a collection of nodes connected so that there is exactly one special node, the **root**, at the top, and every other node has exactly one *parent* (the node just above it) and any number of *children* (the nodes just below). No node is its own ancestor; there are no cycles. A node with no children is a **leaf**.

The pointer-and-object idea generalizes perfectly: a tree node is just an object holding a key plus pointers to its relatives. The only question is *which* relatives to store pointers to, and that choice depends on how many children a node may have.

10.8.1 Binary trees

The cleanest case is the **binary tree**, where every node has at most two children, named *left* and *right*. Each node x stores three pointers: $x.p$ to its parent, $x.left$ to its left child, $x.right$ to its right child. Any missing relative is NIL — the root's parent is NIL, and a leaf's children are NIL.



Here 18 is the root. Its left child is 7, its right child is 26. Node 7 has two children, 3 and 12 (both leaves). Node 26 has *no* left child (that pointer is NIL) and one right child, 31. Reading the tree from the top, each node points down to up to two children and up to one parent.

10.8.2 When the number of children is unbounded

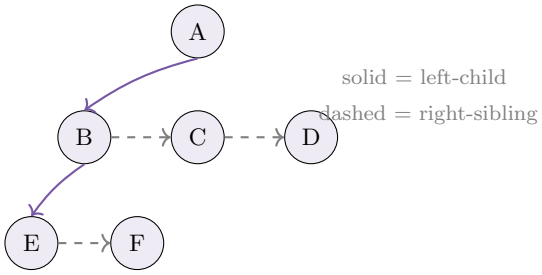
Binary trees are tidy because "at most two children" means a fixed two pointer slots. But many trees let a node have *arbitrarily* many children — think of a file system folder that may contain hundreds of items, or a company org chart. We cannot give each node a fixed pointer for "child 1, child 2, child 3, ..." because we do not know in advance how many there will be, and reserving a huge fixed number would waste enormous memory on nodes with few children.

The space-frugal answer is the **left-child, right-sibling** representation. Each node stores only *two* pointers, regardless of how many children it has:

- *left-child*: a pointer to its *first* child (or NIL if it is a leaf);
- *right-sibling*: a pointer to its *next sibling* — the next child of the same parent (or NIL if it is the last child).

Intuition

Picture each parent reaching down with a single arrow to its *eldest* child. The siblings then hold hands left to right in a chain. To visit *all* the children of a node, you drop down to the first child, then walk the sibling chain to the right until it runs out. You have turned "a list of children of unknown length" into exactly the linked list from the previous section — so it costs only two pointers per node, no matter how lopsided or bushy the tree.



Read it as a conventional tree: A is the root with three children B, C, D; B in turn has two children E, F. But in memory, A holds only one downward pointer (to B, its first child), and B, C, D are linked left-to-right by sibling pointers. Likewise E points sideways to F. To list A's children you go down to B, then follow dashed arrows: $B \to C \to D \to$ NIL. A node with 500 children still needs only its two pointers; the 500 children form a sibling chain among themselves.

Focus

The big idea of this section: **any rooted tree, no matter how many children per node, can be stored with just two pointers per node** via left-child/right-sibling. Trees do not require fancy variable-length nodes. This same two-pointer node is exactly the binary-tree node from before — so an arbitrary tree and a binary tree can share one representation. That equivalence is one reason binary trees appear everywhere in computer science: any general rooted tree can be represented as one.

10.8.3 Other tree representations

Two more variations are worth naming. If you know an upper bound k on the number of children (a "k-ary tree"), you can give each node an array $child[1..k]$ of child pointers — direct and fast, at the cost of wasting slots on nodes with fewer than k children. And for the special case of a *complete* binary tree — one filled level by level with no gaps — you can drop pointers entirely and store the whole tree in a plain array, using arithmetic to find relatives: the children of the node at index i live at indices $2i$ and $2i + 1$, and its parent at $\lfloor i/2 \rfloor$. That index arithmetic is how the *heap* is stored, which we studied in Chapter 6.

History and connection

In 1956 at the RAND Corporation, Allen Newell, J.C. Shaw, and Herbert Simon were attempting to build the Logic Theorist — the earliest artificial intelligence program, designed to discover automated proofs for theorems in Russell and Whitehead's *Principia Mathematica*. They quickly ran into trouble with standard contiguous arrays: the branching search trees of symbolic logic and chess move exploration expanded and contracted unpredictably. If they allocated fixed-size arrays, memory constantly ran out or sat mostly empty. In response, they designed the Information Processing Language (IPL) and invented the *linked list*, which removed the need for contiguous memory by letting each item point to the next wherever space was free. Concurrently in Munich, Friedrich L. Bauer and Klaus Samelson (1957) patented the stack principle (the *Kellerprinzip*, or cellar principle) for translating mathematical formulas. Soon after, Edsger W. Dijkstra (1960–1961) invented the Shunting-Yard algorithm, using an operator stack and an output queue to parse nested parenthetical expressions for the world's first Algol 60 compiler, while early operating system architects designed circular ring buffers to connect asynchronous hardware interrupts and I/O devices with CPU consumers.

These structures have also been the subject of practical disagreement. For decades, computer science textbooks taught that linked lists were strictly superior to arrays for dynamic collections because they offer $O(1)$ insertion and deletion without moving neighbors. Modern hardware has changed that advice. To traverse a linked list, a processor must "chase pointers" across randomly scattered heap addresses, stalling the CPU on repeated cache misses. An array, though requiring $O(n)$ shifts in the worst case, sits in contiguous memory where hardware prefetchers can load whole cache lines at once, making contiguous vectors (like C++'s `std::vector`) dramatically faster for almost all practical workloads. The ubiquitous null pointer — introduced by Tony Hoare in 1965 for ALGOL W, which he later dubbed his "billion-dollar mistake" — plagued programmers with endless null-dereference crashes and messy special cases; sentinel nodes remove some of those special cases by joining list boundaries into a circular ring.

Arrays, stacks, queues, and linked lists appear throughout systems software: compilers use stacks to parse, operating systems use FIFO queues to schedule tasks, and memory allocators use linked lists to track free space. Their limits should guide your choices: lists sacrifice $\Theta(1)$ random access, and on 64-bit architectures, pointer overhead can double or triple memory consumption. In Chapter 11, we combine contiguous arrays with linked-list chaining to build Hash Tables with $O(1)$ expected search; in Chapter 12, we extend pointer chains into Binary Search Trees to achieve $O(\log n)$ search and insertion; and in Chapter 22, we use queues and stacks to drive Breadth-First and Depth-First Graph Search.

10.9 Putting it together

Everything in this chapter rests on *two* primitive ideas:

1. The **contiguous block** (the array), which gives $\Theta(1)$ random access by index via address arithmetic, but pays $\Theta(n)$ to insert or delete in the middle because neighbors have no slack.
2. The **pointer-linked object** (the node), which gives $\Theta(1)$ structural edits by rewiring a few arrows, but pays $\Theta(n)$ to reach the i-th element because there is no arithmetic shortcut — you must walk.

Stacks, queues, and deques are arrays (or lists) under a discipline about *which end* you may touch, and they are all $\Theta(1)$ per operation. Linked lists are pure pointer webs. Trees are pointer webs with a parent-child shape, and even the bushiest tree collapses to two pointers per node. Every advanced structure later in the book — hash tables, balanced search trees, heaps, graphs — is some combination of these two primitives, chosen to make the operations *you* care about cheap.

Pause

Before moving on, make sure you can answer these without peeking. Why is array random access $\Theta(1)$ but array middle-insertion $\Theta(n)$? Why is linked-list insertion $\Theta(1)$ but linked-list random access $\Theta(n)$? What does a stack's LIFO discipline buy you, and a queue's FIFO? Why does a circular array stop a queue from crawling off the end? And how do you store a tree where a node might have a thousand children using only two pointers per node? If those five answers feel solid, you are ready for what comes next.

Exercises

Exercise 10.1 (Address arithmetic)**.** An array of 64-bit integers ($s = 8$ cells per element) begins at base address 4096 and uses zero-based indexing. (a) At what address does $A[0]$ live? $A[5]$? $A[5]$ if the array were instead one-based? (b) An element is found at address 4216; what is its index (zero-based)? Show the arithmetic in both directions.

Exercise 10.2 (Stack trace)**.** Starting from an empty stack, perform: PUSH(5), PUSH(8), PUSH(2), POP, PUSH(11), POP, POP, POP. List the value returned by each POP, in order, and state the contents of the stack after every operation. What goes wrong on the final operation, and what should the implementation do about it?

Exercise 10.3 (Balanced brackets)**.** Describe an algorithm using a single stack that decides whether a string of brackets such as `([]{()})` is properly balanced (every opener has a matching closer, correctly nested). Trace your algorithm on `([)]` and explain precisely where and why it reports an error. What is the running time on a string of length n?

Exercise 10.4 (Circular queue)**.** Using a circular array of size $m = 5$ (one-based) with $head = tail = 1$, trace: ENQ(1), ENQ(2), ENQ(3), DEQ, DEQ, ENQ(4), ENQ(5), ENQ(6), ENQ(7). Show the array contents and both indices after each step, and mark exactly where a wrap-around occurs. Then explain the "full vs. empty" ambiguity in your own words and propose a fix.

Exercise 10.5 (Reverse a singly linked list)**.** Write pseudocode that reverses a singly linked list *in place*, returning the new head, using only a constant number of extra pointers (no second list, no recursion). Hint: walk the list maintaining three pointers — *prev*, *curr*, *next* — and at each node flip its *next* pointer to aim at *prev*. Trace it on the list $9 \to 16 \to 4 \to$ NIL, drawing the pointers after each step. Argue that it runs in $\Theta(n)$ time and $\Theta(1)$ extra space.

Exercise 10.6 (Two stacks make a queue)**.** Implement a queue using two stacks S_{in} and S_{out}, supporting only PUSH and POP on each. ENQUEUE pushes onto S_{in}. DEQUEUE: if S_{out} is empty, pop *everything* from S_{in} and push it onto S_{out}, then pop from S_{out}. Trace ENQ(1), ENQ(2), ENQ(3), DEQ, ENQ(4), DEQ, DEQ and verify that values come out in FIFO order $1, 2, 3, 4$. Why does pouring from S_{in} to S_{out} produce the correct order? (You will analyze its amortized cost in a later chapter.)

Exercise 10.7 (Doubly linked delete)**.** Given a doubly linked list with a sentinel, write the two-line LIST-DELETE and argue it correctly handles deleting (a) the only node, (b) the first node, (c) the last node, (d) a middle node — all without any IF statements. Why does the sentinel make all four cases identical?

Exercise 10.8 (Array vs. list, in practice)**.** For each scenario, decide whether a dynamic array or a doubly linked list is the better fit, and justify in one or two sentences: (a) storing the pixels of an image you will read millions of times but never resize; (b) a text editor's buffer where the user constantly inserts and deletes characters in the middle; (c) a to-do list where you mostly append at the end and occasionally read the k-th item; (d) the "undo history" of an editor.

Exercise 10.9 (Left-child, right-sibling traversal)**.** A rooted tree is stored in left-child, right-sibling form: each node x has $x.child$ (first child or NIL) and $x.sibling$ (next sibling or NIL). Write a recursive procedure COUNT-NODES(x) that returns the number of nodes in the subtree rooted at x (counting x itself). Then explain why the procedure, despite the tree being arbitrary-arity, only ever follows two kinds of pointer. What is its running time on a tree of n nodes?

Exercise 10.10 (Heap-as-array preview, challenge)**.** A complete binary tree is stored in a one-based array, where the node at index i has children at $2i$ and $2i + 1$ and parent at $\lfloor i/2 \rfloor$. (a) Draw the tree for the array $[16, 14, 10, 8, 7, 9, 3]$. (b) Which array index holds the parent of the node at index 6? Its sibling? (c) If the array has n elements, which indices are leaves? (Hint: a node is a leaf exactly when it has no left child, i.e. when $2i > n$.) Verify your answer on the 7-element example.

Chapter 11

Hash Tables

Chapter 10 established the fundamental trade-off of elementary data structures: contiguous arrays offer instant $\Theta(1)$ indexing but expensive $\Theta(n)$ insertions, while linked lists offer cheap $\Theta(1)$ pointer splices but force $\Theta(n)$ linear scans to find an element by key. Neither structure can search for an arbitrary key in constant time.

Imagine you run a small library, and someone asks for book number 4173. You could walk the shelves one by one, inspecting every spine – that is searching a linked list. Or you could build the library so that book 4173 lives at position 4173 on a very long shelf, letting you walk straight to it using array index arithmetic. The catch is that a shelf with a slot for every possible catalog number wastes immense space if most numbers never appear. Hash tables combine the strengths of both: they compute memory locations directly from keys to achieve $\Theta(1)$ average search, insertion, and deletion, while compressing a vast universe of possible keys into an array proportional to the records actually stored.

By the end of this chapter you will understand why a well-built hash table answers "is x in here?", "insert x", and "delete x" in *constant expected time*, what "expected" is doing in that sentence, and how to build hash functions that earn that guarantee instead of just hoping for it.

Focus

A hash table is a dictionary: it stores a set of *(key, value)* pairs and supports Insert, Search, and Delete. Its trick is a *hash function* h that maps each key into a slot of a small array. The central design problem is to scatter keys evenly enough that every slot holds only a few elements, so lookups stay fast while memory stays proportional to the number of keys actually stored.

11.1 The dictionary problem

First, let us specify the exact operations we need. We have a universe U of possible keys – maybe all 32-bit integers, maybe all strings up to length 100, maybe all possible employee IDs. At any moment we are storing some *subset* of those keys, a dynamic set we will call K, and K is usually *tiny* compared to U. We want three operations:

- Search(T, k): return the element with key k, or report that it is absent.

- INSERT(T, x): add element x (whose key is $x.key$) to the set.
- DELETE(T, x): remove element x from the set.

We do *not* need the keys sorted, we do *not* need to ask "what is the smallest key larger than k", we do *not* need predecessors or successors. We just need membership and retrieval. That modest wish list is exactly why hash tables can be so fast: by giving up order, they buy speed.

Why this matters

This is one of the most-used data structures on Earth. Python's `dict` and `set`, Java's `HashMap`, C++'s `unordered_map`, JavaScript objects, Ruby hashes, the symbol tables inside every compiler, the routing tables inside databases, deduplication, caching, and the "have I seen this before?" check in a hundred algorithms – all hash tables underneath. When people say a program "uses a hash map to make it fast," they mean: it turned an $O(n)$ scan into an $O(1)$ lookup.

11.2 Direct-address tables: the simple, wasteful idea

Let us start with the version that has no cleverness at all, just to see the shape of the answer before we economize.

Suppose the universe of keys is small: $U = \{0, 1, 2, \ldots, m-1\}$ for some manageable m, and no two elements share a key. Then we can do the library-shelf thing literally. Allocate an array $T[0\,..\,m-1]$, called a *direct-address table*. Slot $T[k]$ holds the element whose key is k (or a pointer to it), and holds NIL if no such element is present.

Algorithm 40: Direct-address operations

Input: A direct-address table $T[0\,..\,m-1]$, a key k, an element x

1 DIRECT-ADDRESS-SEARCH(T, k): **return** $T[k]$;
2 DIRECT-ADDRESS-INSERT(T, x): $T[x.key] \leftarrow x$;
3 DIRECT-ADDRESS-DELETE(T, x): $T[x.key] \leftarrow$ NIL;

Each operation is three lines, no loops, no comparisons. Every one runs in $O(1)$ time in the *worst* case – not expected, not amortized, just flatly constant. This is the performance we want to keep.

Example 11.1. Say employee IDs run from 0 to 99, and right now we employ people with IDs 4, 17, and 42. The direct-address table is an array of 100 slots. $T[4]$, $T[17]$, $T[42]$ point at their records; the other 97 slots hold NIL. "Is employee 17 here?" is just "look at $T[17]$" – one memory access.

The catch is memory: when the universe of possible keys U is large, allocating one slot per possible key is impractical or impossible.

Common trap

Direct addressing wastes space when U is large. If keys are 32-bit integers, U has about 4.3 billion values, so T needs 4.3 billion slots even to store ten keys. That is gigabytes of mostly-NIL array. And if keys are strings or 64-bit numbers, a direct-address table exceeds any physical memory. Hashing exists to avoid this.

Intuition

Direct addressing pays $|U|$ memory to get $O(1)$ time. We are about to pay only $\Theta(|K|)$ memory – proportional to the number of keys we actually store – and *keep* $O(1)$ time, on average. The trade is that "on average" replaces "always." For almost every real program that is a good trade.

11.3 From direct addressing to hashing

We have a huge universe U but only ever store $|K| = n$ keys, and we are willing to allocate an array $T[0\,..\,m-1]$ of just m slots, where m is on the order of n, not $|U|$. The problem: a key k might be a nine-digit number, but our array only has, say, $m = 701$ slots. Where does key k go?

We *compute* a slot from the key, using a *hash function*

$$h : U \to \{0, 1, \ldots, m-1\}.$$

The key k "hashes to" slot $h(k)$, and we say $h(k)$ is the *hash value* of k. Instead of storing element x at $T[x.key]$ (impossible – the array is too small), we store it at $T[h(x.key)]$. The array shrinks from $|U|$ slots to m slots.

There is a catch. We are cramming $|U|$ possible keys into $m \ll |U|$ slots. By the pigeonhole principle, *many keys must map to the same slot.* When $h(k_1) = h(k_2)$ for $k_1 \neq k_2$, we call it a *collision*, and we now have two elements wanting the same parking spot.

Why do collisions happen so much faster than intuition suggests? If you have $m = 365$ slots and begin inserting keys, naive intuition might suggest that collisions remain rare until the table is mostly full. Yet, by the classic birthday paradox, the probability of at least one collision crosses 50% after inserting just $n \approx \sqrt{2m \ln 2} \approx 23$ keys. Because the number of possible colliding pairs grows quadratically as $\binom{n}{2} = n(n-1)/2$, collisions appear long before the table approaches capacity. Every hash table design therefore needs a plan for handling collisions.

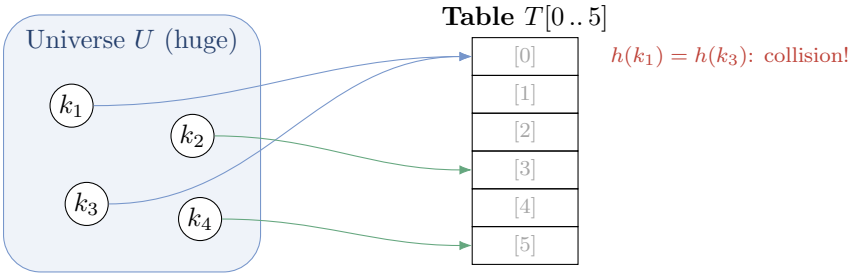


Figure 11.1: A hash function squashes a huge universe into a small table. Keys k_1 and k_3 collide in slot 0.

11.4 Collision resolution by chaining

The simplest, most robust answer to collisions is: if several keys land in the same slot, just *keep a list there.* Slot $T[j]$ holds not a single element but a pointer to a linked list of all elements whose keys hash to j. This is *chaining.*

Intuition

Picture a coat-check with m hooks. The hash function tells you which hook your coat goes on. If two coats get the same hook, you just hang the second one in front of the first – the hook holds a little chain of coats. To find your coat, you go to its hook and look through the (hopefully short) chain hanging there.

The three operations become:

Algorithm 41: Chained hash table operations

Input: Hash table T with m slots, hash function h, element x, key k

1 CHAINED-HASH-INSERT(T, x): insert x at the head of list $T[h(x.key)]$;
2 CHAINED-HASH-SEARCH(T, k): search list $T[h(k)]$ for an element with key k;
3 CHAINED-HASH-DELETE(T, x): delete x from list $T[h(x.key)]$;

INSERT is $O(1)$ worst case: hash the key, splice the new node onto the front of the list – no scanning, because we assume x is not already present (or we do not care if it duplicates). SEARCH must walk the list at slot $h(k)$, so its cost is proportional to that list's length. DELETE, if we are handed a pointer to x and use doubly linked lists, is also $O(1)$: we splice x out without searching for it.

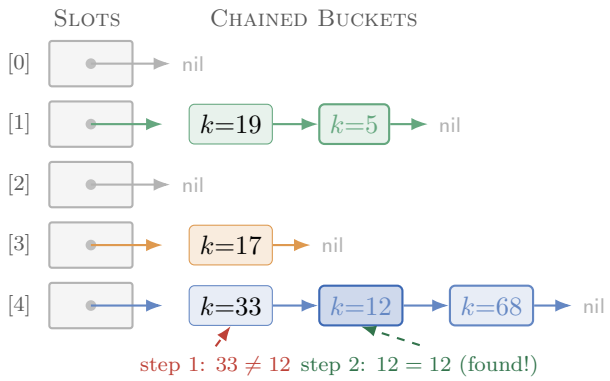


Figure 11.2: Collision resolution by chaining with $m = 5$ slots. Keys 19 and 5 map to slot 1; keys 33, 12, and 68 map to slot 4. Searching for $k = 12$ computes $h(12) = 4$, jumps to list head $T[4]$, and walks 2 nodes before terminating successfully.

11.4.1 The load factor

How long are these lists, on average? If the table has m slots and currently holds n elements, define the *load factor*

$$\alpha = \frac{n}{m}.$$

It is the average number of elements per slot. If $n = 1000$ and $m = 500$, then $\alpha = 2$: on average two elements per list. A load factor of $\alpha = 0.75$ means the table is about three-quarters full in the "one element per slot" sense. The load factor is the single most important dial on a hash table, and a great deal of our analysis is really analysis of α.

Focus

$\alpha = n/m$ is the average list length under chaining. We will see that search costs $\Theta(1+\alpha)$. So if we keep α bounded by a constant – say, never let it exceed 1 or 2 – searches are $\Theta(1)$. Keeping α small is why hash tables *resize* (rehash into a bigger array) as they fill.

11.5 Analyzing chaining: simple uniform hashing

We want to prove that search is fast on average. But "on average" over what? There is no randomness in a fixed set of keys and a fixed hash function – if an adversary hands you keys that all collide, every search is slow, period. To get a meaningful average we make an idealizing assumption about *how keys distribute*.

Formal

Simple uniform hashing. Any given key is equally likely to hash into any of the m slots, independently of where the other keys have hashed. Formally, for the keys we store, $\Pr[h(k) = j] = 1/m$ for every slot j, and the slots chosen by distinct keys are independent.

This is an assumption about the *behavior* of h on our particular keys, not something we can prove about an arbitrary function. Later, with *universal hashing*, we will replace this leap of faith with a real probabilistic guarantee. For now, accept it and see where it leads.

Let n_j denote the length of the list at slot j, so $n = n_0 + n_1 + \cdots + n_{m-1}$. Under simple uniform hashing, each of the n keys lands in slot j with probability $1/m$, so the *expected* length of any list is

$$\mathbb{E}[n_j] = \frac{n}{m} = \alpha.$$

11.5.1 Unsuccessful search

An *unsuccessful* search – looking for a key k that is *not* in the table – walks the entire list at slot $h(k)$ to the end (it has to, to be sure k is absent). Under simple uniform hashing, k is equally likely to hash to any slot, and the expected length of that slot's list is α. Add the constant cost of computing $h(k)$ and indexing into the array, and we get expected time

$$\Theta(1+\alpha).$$

11.5.2 Successful search

A *successful* search is a touch subtler, because the key we are looking for is somewhere in its list, and on average we walk only *partway* down. The careful count – which we do gently below – also gives $\Theta(1+\alpha)$.

Theorem 11.1 (Expected cost of search under chaining)**.** *In a hash table with chaining, under the assumption of simple uniform hashing, both a successful and an unsuccessful search take expected time $\Theta(1+\alpha)$, where $\alpha = n/m$ is the load factor.*

Sketch for successful search. Suppose we search for each of the n stored keys equally often. Number the keys $x_1, x_2, \ldots, x_n$ in the order they were *inserted* (each at the head of its list). When we successfully search for x_i, the number of elements we examine is 1 (for x_i itself) plus the number of elements that appear *before* x_i in its list – which are exactly the keys inserted *after* x_i that happened to land in the same slot.

For $j > i$, let the indicator $X_{ij} = 1$ if x_i and x_j hash to the same slot. Under simple uniform hashing, $\Pr[X_{ij} = 1] = 1/m$, so $\mathbb{E}[X_{ij}] = 1/m$. The expected number of elements examined when searching for x_i is $1 + \sum_{j>i} \mathbb{E}[X_{ij}]$. Averaging over all i,

$$\frac{1}{n}\sum_{i=1}^{n}\left(1 + \sum_{j=i+1}^{n}\frac{1}{m}\right) = 1 + \frac{1}{nm}\sum_{i=1}^{n}(n-i) = 1 + \frac{1}{nm}\cdot\frac{n(n-1)}{2} = 1 + \frac{n-1}{2m}.$$

Since $\frac{n-1}{2m} < \frac{n}{2m} = \frac{\alpha}{2}$, the expected number of elements examined is at most $1 + \alpha/2$, giving total expected time $\Theta(1+\alpha)$ once we add the constant cost of computing the hash. □ □

Intuition

The two searches give the same big-O for a satisfying reason. An unsuccessful search marches to the end of a length-α list, costing about α. A successful search, on average, stops about halfway, costing about $\alpha/2$. Same order, $\Theta(\alpha)$, differing only by the constant 1 versus 1/2. And the leading "1+" in $\Theta(1+\alpha)$ is just the unavoidable cost of computing the hash and touching the array, which we pay even when a list is empty.

Corollary 11.1. *If $m = \Omega(n)$ – that is, the number of slots grows at least linearly with the number of keys, so $\alpha = O(1)$ – then all dictionary operations take $O(1)$ expected time.*

This is the headline result of the chapter. Keep the load factor bounded by a constant, and a chained hash table does Insert, Search, and Delete in expected constant time. Insert and (pointered) delete are already $O(1)$ in the *worst* case; only search carries the "expected."

Cost summary

Chaining, with $\alpha = n/m$:

Operation	Expected	Worst case
Insert	$O(1)$	$O(1)$
Search	$\Theta(1+\alpha)$	$\Theta(n)$
Delete (pointer to x)	$O(1)$	$O(1)$

With $\alpha = O(1)$, search is $O(1)$ expected. Worst-case $\Theta(n)$ happens only if every key collides into one slot – which good hashing makes astronomically unlikely.

Algorithm lab

Build a tiny chained table by hand with $m = 5$ and $h(k) = k \bmod 5$. Insert the keys $12, 7, 33, 20, 18, 25, 2$ in that order, always at the *head* of each list. After all insertions, write out the five lists. (Check, reading each list from its head: slot 0 should hold $25, 20$; slot 2 should hold $2, 7, 12$; slot 3 should hold $18, 33$. Slots 1 and 4 empty.) Then count how many

nodes a successful search for 7 examines, and an unsuccessful search for 9.

11.6 Hash functions: turning keys into slots

Everything above assumed simple uniform hashing – an idealized, perfectly uniform spread across slots. Real hash functions are concrete arithmetic, and a bad one clumps keys together and destroys the guarantee. What makes a good hash function?

Focus

A good hash function should (a) be fast to compute, (b) scatter keys so that any clustering or pattern in the keys does *not* translate into clustering in the slots, and (c) ideally satisfy simple uniform hashing. Since we rarely know the exact distribution of keys, we want functions that behave well across many plausible key sets.

Most hash functions first interpret the key as a natural number. A string like `"pt"` becomes a number by treating its characters as digits in base 128: `p`= 112, `t`= 116, so the key is $112 \cdot 128 + 116 = 14452$. Once a key is a non-negative integer k, two classic methods turn it into a slot.

11.6.1 The division method

Pick a table size m and define

$$h(k) = k \bmod m.$$

The hash value is the remainder of k divided by m. It is wonderfully fast – one modulo operation. But the choice of m matters enormously.

Common trap

Do not let m be a power of 2 in the division method. If $m = 2^p$, then $k \bmod m$ is just the low p bits of k – it ignores all the higher bits entirely. If your keys share patterns in their low bits (common with structured data, pointers, aligned addresses), they collide catastrophically. Powers of 10 are similarly dangerous for decimal-structured keys.

The folklore fix: choose m to be a *prime* not too close to a power of 2. A prime modulus mixes all the bits of k into the remainder, so accidental patterns in the keys are far less likely to survive. For example, to store about 2000 keys with load factor near 3, you want $m \approx 700$; the prime 701 is a fine choice.

Example 11.2. Let $m = 701$ and key $k = 1{,}234{,}567$. Then $h(k) = 1{,}234{,}567 \bmod 701$. Compute: $701 \cdot 1761 = 1{,}234{,}461$, and $1{,}234{,}567 - 1{,}234{,}461 = 106$. So the key hashes to slot 106.

11.6.2 The multiplication method

This method does not care about the exact value of m, which frees you to pick $m = 2^p$ for fast indexing. Choose a constant A with $0 < A < 1$. Then

$$h(k) = \lfloor\, m\,(kA \bmod 1)\,\rfloor,$$

where kA mod 1 means "the fractional part of kA." In words: multiply the key by A, throw away the integer part to keep a fraction in $[0, 1)$, then scale that fraction up to $[0, m)$ and floor it.

Intuition

Think of the fractional parts $\{kA\}$ for $k = 1, 2, 3, \ldots$ as points dropped on the circle of circumference 1. If A is a nicely irrational-looking constant, these points spread out evenly around the circle, never bunching up. We then chop the circle into m equal arcs and read off which arc each key landed in. Even, well-mixed arcs mean even, well-mixed slots.

Knuth suggested $A \approx (\sqrt{5} - 1)/2 = 0.6180339887\ldots$, the reciprocal golden ratio, as a value that spreads keys especially well.

Example 11.3. Let $m = 16$ (so $p = 4$), $A = 0.6180339887$, and $k = 100$. Then $kA = 61.80339887$, the fractional part is 0.80339887, times $m = 16$ gives $12.854\ldots$, and the floor is 12. So $h(100) = 12$.

Reading note

This material parallels CLRS Chapter 11. Direct addressing and chaining correspond to sections 11.1–11.2; hash functions (division, multiplication) to 11.3; open addressing to 11.4; and universal and perfect hashing to 11.3.3 and 11.5. Knuth's *The Art of Computer Programming*, Volume 3, provides the classic mathematical treatment.

11.7 Open addressing: no lists at all

Chaining stores collisions in side lists, which means extra pointers and extra memory beyond the array. *Open addressing* takes a different, leaner stance: **every element lives in the array itself**, one element per slot, no lists, no external pointers. The load factor $\alpha = n/m$ therefore never exceeds 1 – you cannot store more elements than you have slots.

But if a key's natural slot is already occupied by a different key, where does it go? It *probes* a sequence of slots until it finds an empty one. The hash function now takes a second argument, the probe number:

$$h : U \times \{0, 1, \ldots, m-1\} \to \{0, 1, \ldots, m-1\}.$$

For a key k, we try slots $h(k, 0)$, $h(k, 1)$, $h(k, 2)$, $\ldots$ in order. We require that this *probe sequence* $\langle h(k, 0), h(k, 1), \ldots, h(k, m-1)\rangle$ be a permutation of all m slots, so that probing eventually examines every slot and never gives up while empty slots remain.

Algorithm 42: Open-addressing Insert and Search

Input: Hash table $T[0\,..\,m-1]$, key k (Insert) or sought key k (Search)
Output: Insert: slot index, or table-full error. Search: slot index or NIL.

```
1  HASH-INSERT(T, k):;
2  for i ← 0 to m − 1 do
3      j ← h(k, i);
4      if T[j] is NIL or DELETED then
5          T[j] ← k;
6          return j;
7  return "table overflow";
8  HASH-SEARCH(T, k):;
9  for i ← 0 to m − 1 do
10     j ← h(k, i);
11     if T[j] = k then return j;
12     if T[j] is NIL then return NIL;
13 return NIL;
```

Notice the search-stopping rule: we stop and report "absent" the moment we hit a genuinely empty (NIL) slot, because the key, had it been inserted, would have claimed that slot or an earlier one in its probe sequence. We do *not* stop on an occupied-by-someone-else slot; we keep probing.

Common trap

Deletion in open addressing is tricky. You cannot simply set a deleted slot back to NIL – that would create a false "empty" in the middle of some other key's probe sequence, causing searches for those keys to stop early and wrongly report "absent." The standard fix is a special DELETED marker (a "tombstone"): search treats it as occupied (keep probing past it), but insert treats it as free (reuse it). The cost: with many tombstones, search times no longer depend only on α, which is one reason chaining is often preferred when deletions are frequent.

11.7.1 Linear probing

The simplest probe sequence: if your slot is taken, try the next one, then the next, wrapping around. Given an ordinary hash function $h'(k)$, define

$$h(k, i) = \big(h'(k) + i\big) \bmod m.$$

The probe sequence is $h'(k)$, $h'(k)+1$, $h'(k)+2, \ldots$ marching forward (mod m).

Example 11.4. Let $m = 11$, $h'(k) = k \bmod 11$, and insert $20, 31, 9, 42$ in order.

- 20: $h' = 20 \bmod 11 = 9$. Slot 9 empty → place at 9.
- 31: $h' = 31 \bmod 11 = 9$. Slot 9 taken; try 10 – empty → place at 10.
- 9: $h' = 9$. Slot 9 taken; 10 taken; try 0 (wrap) – empty → place at 0.
- 42: $h' = 42 \bmod 11 = 9$. Slots $9, 10, 0$ taken; try 1 – empty → place at 1.

Four keys that all wanted slot 9 spilled forward into $9, 10, 0, 1$ – a run of four consecutive filled slots, shown in Figure 11.3.

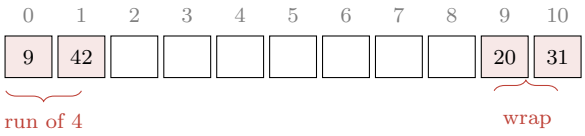


Figure 11.3: Linear probing after inserting $20, 31, 9, 42$ (all with $h' = 9$) into $m = 11$ slots. They form one contiguous run wrapping past the end: slots $9, 10, 0, 1$.

That run is an example of *primary clustering*: long runs of occupied slots build up, and they grow faster than you would guess, because any key hashing anywhere into the run gets appended to its far end, lengthening it. Long runs mean long probes.

Common trap

Primary clustering. Linear probing creates long contiguous runs of full slots. A slot just after a long run is very likely to be chosen next (any of the run's keys, plus that slot's own keys, feed it), so runs snowball. Linear probing is cache-friendly – it walks contiguous memory – but its clustering makes probe sequences long as α approaches 1.

11.7.2 Quadratic probing

To break up those runs, jump by a quadratically growing offset instead of 1:

$$h(k, i) = \big(h'(k) + c_1 i + c_2 i^2\big) \bmod m,$$

for constants c_1, c_2 (with $c_2 \neq 0$). Now the probe sequence spreads out: $h', h' + c_1 + c_2, h' + 2c_1 + 4c_2, \ldots$. Two keys with the *same* initial slot h' still follow the same probe sequence – a milder defect called *secondary clustering* – but you avoid the long contiguous runs of linear probing. The constants c_1, c_2 and m must be chosen carefully (a common safe choice is m a power of 2 with $c_1 = c_2 = 1/2$) so that the probe sequence actually visits all m slots.

11.7.3 Double hashing

The best of the simple open-addressing schemes uses a *second* hash function to determine the step size, so that different keys with the same starting slot march off in *different* directions:

$$h(k, i) = \big(h_1(k) + i\, h_2(k)\big) \bmod m.$$

The first probe is at $h_1(k)$; thereafter each step jumps by $h_2(k)$, an amount that *depends on the key*. Two keys sharing a starting slot almost surely have different step sizes, so they diverge immediately – no secondary clustering. For the probe sequence to hit every slot, $h_2(k)$ must be relatively prime to m; a clean way to guarantee this is to make m a power of 2 and design h_2 to always produce odd values, or to make m prime and let h_2 return values in $1..m-1$.

Example 11.5. Let $m = 13$, $h_1(k) = k \bmod 13$, and $h_2(k) = 1 + (k \bmod 11)$. Insert key $k = 14$. $h_1(14) = 1$, $h_2(14) = 1 + (14 \bmod 11) = 1 + 3 = 4$. Probe sequence: $1, 5, 9, 0, 4, 8, 12, 3, 7, 11, 2, 6, 10$ – step 4, wrapping mod 13, visiting all thirteen slots. If slot 1 is taken, we try 5; if that is taken, 9; and so on.

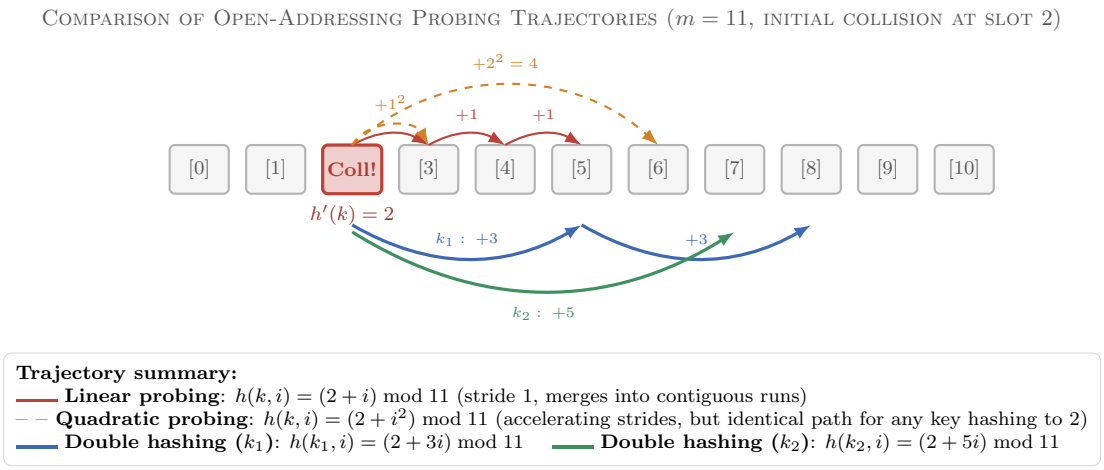


Figure 11.4: Probing trajectories following a collision at slot $h'(k) = 2$ in a table of size $m = 11$. Linear probing steps through immediate neighbors, causing primary clusters. Quadratic probing accelerates away, but all keys sharing initial slot 2 trace identical sequences (secondary clustering). Double hashing assigns distinct strides $h_2(k_1) = 3$ and $h_2(k_2) = 5$, decoupling trajectories across the table.

Intuition

Linear probing: everybody who collides walks the same single-file line, so jams pile up. Double hashing: each key, even those starting at the same slot, walks at its own stride, so they fan out across the table instead of queuing behind one another. Double hashing uses $\Theta(m^2)$ distinct probe sequences (one per (h_1, h_2) pair), versus only m for linear or quadratic probing, which is why it comes closest to the idealized behavior.

11.7.4 Cost of open addressing

The clean analysis assumes *uniform hashing*: each key's probe sequence is equally likely to be any of the $m!$ permutations of the slots. (Double hashing approximates this; linear and quadratic probing do not, but the bound is still a useful guide.) Under this assumption one can prove:

Theorem 11.2 (Expected probes in open addressing)**.** *In an open-address table with load factor $\alpha = n/m < 1$, under uniform hashing, the expected number of probes is at most $\dfrac{1}{1-\alpha}$ for an unsuccessful search (and for an insertion), and at most $\dfrac{1}{\alpha} \ln \dfrac{1}{1-\alpha}$ for a successful search.*

Intuition for the unsuccessful bound. A probe sequence of length at least 1 always happens. It continues to a second probe only if the first slot was occupied, which has probability about α (the

fraction of full slots). It reaches a third probe only if the first two were both occupied, probability about α^2, and so on. The expected number of probes is therefore about

$$1 + \alpha + \alpha^2 + \alpha^3 + \cdots = \frac{1}{1-\alpha},$$

a geometric series, valid because $\alpha < 1$. □ □

Put some numbers to that formula. They are the reason hash tables resize before they get too full.

α	table is	unsuccessful $\frac{1}{1-\alpha}$	successful $\frac{1}{\alpha}\ln\frac{1}{1-\alpha}$
0.5	half full	2.0	1.39
0.75	3/4 full	4.0	1.85
0.9	9/10 full	10.0	2.56
0.99	nearly full	100	4.65

Cost summary

Open addressing. Expected probes blow up like $\frac{1}{1-\alpha}$ as $\alpha \to 1$: at $\alpha = 0.9$ an unsuccessful search averages 10 probes; at $\alpha = 0.99$, a hundred. Practical implementations cap α (often around 0.5–0.75) and *rehash* into a larger table when the cap is hit. Compared to chaining, open addressing uses no pointer overhead and is cache-friendly, but degrades sharply near full and handles deletion awkwardly.

Walking question

Under chaining, the expected search cost is $\Theta(1+\alpha)$ and grows *linearly* in α – doubling the load factor roughly doubles the work. Under open addressing it grows like $\frac{1}{1-\alpha}$, which is fine for small α but *explodes* as $\alpha \to 1$. Why the difference? What is fundamentally happening at $\alpha = 1$ in each scheme? (Hint: in chaining, $\alpha = 1$ just means "one element per list on average" – totally healthy. In open addressing, $\alpha = 1$ means the array is *completely full*, and an unsuccessful search must probe everything.)

11.8 Universal hashing: removing the leap of faith

Everything so far rested on simple uniform hashing – an *assumption* that our keys happen to spread evenly. But that assumption can fail spectacularly, and not merely through bad luck. If your hash function is fixed and publicly known, an adversary (or simply an unlucky real-world dataset) can hand you n keys that *all* hash to the same slot. Then chaining degenerates to one list of length n, and every search is $\Theta(n)$. This is a practical concern: it has been used to mount real denial-of-service attacks on web servers, by sending requests whose parameters all collide in the server's hash table.

Intuition

The fix is simple and effective: *do not commit to a single fixed hash function.* At program start, pick a hash function *at random* from a carefully designed family $\mathcal{H}$. Now an adversary,

choosing keys, cannot know which function you will use, so they cannot engineer collisions. The randomness lives in *our* choice of h, not in the keys – which means we get a guarantee that holds for *any* input keys, even adversarial ones, in expectation over our coin flips.

What property must the family $\mathcal{H}$ have? Exactly this: any two distinct keys should be unlikely to collide, over the random choice of h.

Formal

Universal family. A finite collection $\mathcal{H}$ of hash functions, each mapping $U \to \{0, \ldots, m-1\}$, is *universal* if for every pair of distinct keys $k \neq \ell$,

$$\Pr_{h \in \mathcal{H}} \left[h(k) = h(\ell) \right] \leq \frac{1}{m},$$

where the probability is over choosing h uniformly at random from $\mathcal{H}$.

In words: two distinct keys collide with probability at most $1/m$ – exactly the collision probability you would get from a perfectly random function. And we get that guarantee while still using a small, efficiently-computable family.

Theorem 11.3 (Universal hashing gives short chains)**.** *If h is chosen from a universal family $\mathcal{H}$ and we store n keys with chaining in a table of m slots, then for any fixed key k, the expected length of the list containing k's slot is at most $1 + \alpha$ if k is in the table, and at most α if it is not. Hence expected search time is $O(1 + \alpha)$* – for any set of keys*, no assumption required.*

Proof. Fix a key k and any other stored key $\ell \neq k$. Define the indicator random variable $X_{k\ell} \in \{0, 1\}$ to be 1 if $h(k) = h(\ell)$ and 0 otherwise. Because $X_{k\ell}$ takes only values in $\{0, 1\}$, its expectation is exactly the collision probability:

$$\mathbb{E}[X_{k\ell}] = 1 \cdot \Pr[h(k) = h(\ell)] + 0 \cdot \Pr[h(k) \neq h(\ell)] = \Pr[h(k) = h(\ell)] \leq \frac{1}{m},$$

by the universality of $\mathcal{H}$. The total number of stored keys colliding with k is the sum of these indicators: $Y_k = \sum_{\ell \in K,\, \ell \neq k} X_{k\ell}$. By linearity of expectation – which holds even though pairwise collision events are not necessarily mutually independent:

$$\mathbb{E}[Y_k] = \sum_{\ell \in K,\, \ell \neq k} \mathbb{E}[X_{k\ell}] \leq \sum_{\ell \in K,\, \ell \neq k} \frac{1}{m}.$$

If $k \notin K$, the sum contains exactly n terms, yielding $\mathbb{E}[Y_k] \leq n/m = \alpha$. If $k \in K$, the sum excludes k itself and has $n - 1$ terms, giving $\mathbb{E}[Y_k] \leq (n-1)/m < \alpha$. Adding 1 for the step of inspecting k itself yields an expected chain traversal of at most $1 + \alpha$. □ □

This is a stronger guarantee: **the same $\Theta(1+\alpha)$ average search cost holds for every possible set of input keys**, because the probability distribution is governed entirely by our own randomized algorithm rather than by assumptions about user inputs.

11.8.1 A concrete universal family

It is not obvious such families exist, so here is one you can implement. Pick a prime p larger than every possible key, so $U \subseteq \{0, 1, \ldots, p-1\}$. For any $a \in \{1, 2, \ldots, p-1\}$ and $b \in \{0, 1, \ldots, p-1\}$, define

$$h_{a,b}(k) = \big((ak + b) \bmod p\big) \bmod m.$$

The family $\mathcal{H}_{p,m} = \{\, h_{a,b} : a \in \{1, \ldots, p-1\},\ b \in \{0, \ldots, p-1\} \,\}$ – one function per choice of (a, b) – is universal. We choose a and b at random when the table is created and keep them fixed thereafter. The proof that collisions happen with probability $\leq 1/m$ rests on the fact that, modulo a prime, the map $k \mapsto ak + b$ shuffles the keys around in a way that decouples distinct keys; CLRS section 11.3.3 carries it out in full, and it is a lovely little argument with $\mathbb{Z}_p$ arithmetic.

Example 11.6. Take $p = 17$, $m = 6$, and suppose we randomly drew $a = 3$, $b = 4$. Then $h_{3,4}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = (28 \bmod 17) \bmod 6 = 11 \bmod 6 = 5$. And $h_{3,4}(15) = ((45 + 4) \bmod 17) \bmod 6 = (49 \bmod 17) \bmod 6 = 15 \bmod 6 = 3$. A *different* random draw, say $a = 5, b = 2$, would send key 8 to $((42) \bmod 17) \bmod 6 = (8) \bmod 6 = 2$ instead – different function, different slot. That dependence on the secret (a, b) is precisely what foils an adversary.

Why this matters

Universal hashing has direct practical uses. Around 2003 and again around 2011, researchers showed that fixed, predictable hash functions in popular language runtimes let attackers craft HTTP requests whose keys all collided, turning an $O(1)$ lookup into $O(n)$ and freezing servers under a trickle of traffic – "hash-flooding" denial of service. The defense deployed across Python, Ruby, and others was exactly this chapter's idea: seed the hash function with a per-process random key, so collisions cannot be predicted. The math you just saw is the reason that defense works.

11.9 Perfect hashing: $O(1)$ worst case for static sets

Universal hashing gives $O(1)$ *expected* time. Can we ever get $O(1)$ *worst-case* search – a guarantee with no "expected" attached? Yes, in an important special case: when the set of keys is *static* – fixed and known in advance, never changing afterward. This is the situation for, say, reserved keywords in a programming language, or a fixed dictionary of words. We call the result *perfect hashing*.

Formal

A hash function is *perfect* for a set K of keys if it maps them with *no collisions at all* – every key gets its own slot. Then a search is a single probe: $O(1)$ worst case, guaranteed.

How do we find a perfect, collision-free function? The clever construction is *two levels of universal hashing*.

1. **Top level.** Use a universal hash function h to spread the n keys into $m = n$ slots. There will still be collisions – some slots get several keys – but we don't store them in lists. Instead…
2. **Second level.** For each slot j that received n_j keys, build a *secondary* hash table of size $m_j = n_j^2$ – the square of the number of keys in that slot – using its own universal hash function. Squaring the size is the trick that makes the secondary table collision-free.

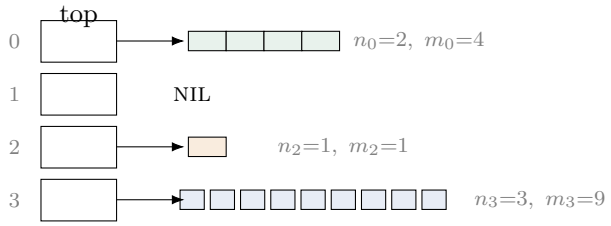


Figure 11.5: Perfect hashing: a top-level table whose slots point to small secondary tables of size n_j^2, each collision-free.

Two questions should be bothering you. First, why does squaring make a slot collision-free? And second, isn't all that squared space enormous?

Lemma 11.1 (Squaring kills collisions, in expectation)**.** *If you hash n_j keys into a table of size $m_j = n_j^2$ using a universal hash function, the expected number of colliding pairs is less than $1/2$. So a single random choice of function is collision-free with probability greater than $1/2$, and after a few tries you are guaranteed to find one.*

Why. There are $\binom{n_j}{2} < n_j^2/2$ pairs of keys. Each pair collides with probability at most $1/m_j = 1/n_j^2$ by universality. By linearity of expectation, the expected number of colliding pairs is less than $\frac{n_j^2}{2} \cdot \frac{1}{n_j^2} = \frac{1}{2}$. A non-negative quantity with expectation below $1/2$ must be 0 more than half the time, so a random universal function gives zero collisions with probability $> 1/2$. Keep drawing until it does. □ □

That answers the first question: each secondary table, sized as the square of its occupancy, is collision-free, so a search there is a single probe. A perfect-hash search is thus: one probe at the top level to find the right secondary table, one probe inside it – two probes total, $O(1)$ *worst case*.

Now the space. Surely $\sum_j n_j^2$ could be huge – if all n keys piled into one slot, it would be n^2. The saving grace is that, by choosing the top-level function from a universal family, that pile-up almost never happens.

Theorem 11.4 (Perfect hashing uses linear space)**.** *If we choose the top-level universal hash function with $m = n$, then the expected total size of all secondary tables is less than $2n$. Hence perfect hashing stores n keys in $O(n)$ space while guaranteeing $O(1)$ worst-case search.*

The key fact behind this is that $\mathbb{E}\big[\sum_j n_j^2\big] < 2n$ when $m = n$ – the expected sum of squared bucket sizes is linear, not quadratic, precisely because universality keeps the buckets balanced. (CLRS section 11.5 proves it; it again comes down to counting colliding pairs.) As with the secondary tables, we simply redraw the top-level function if the total turns out too large; on average a couple of tries suffice.

Focus

Perfect hashing in brief: use two levels of universal hashing. The top level spreads n keys over n slots; each slot's n_j keys get a private secondary table of size n_j^2, which universality makes collision-free. Result: $O(1)$ *worst-case* lookup using $O(n)$ space – but only for a *static* key set, because rebuilding on every insert would be too slow.

History and connection

The dictionary problem arose in the earliest commercial computers at IBM. In 1953, Hans Peter Luhn was tasked with indexing chemical formulas and bibliographic records on punch cards. Direct addressing was impossible: the universe of chemical compounds was astronomical, yet RAM was measured in tiny magnetic core words. Luhn proposed computing a numerical address from the key by folding and division, inventing hash addressing. Simultaneously, A. D. Linh (1953) developed separate chaining with linked lists to resolve collisions, and Arnold I. Dumey (1956) formulated open addressing with linear probing. Two decades later, J. Lawrence Carter and Mark N. Wegman (1979) introduced universal hashing, providing the first mathematically rigorous defense against worst-case collision attacks. Two-level perfect hashing was achieved by Michael Fredman, János Komlós, and Endre Szemerédi in 1984 (the FKS scheme).

Early practitioners treated hash functions as ad-hoc engineering hacks, assuming real-world inputs would distribute uniformly like independent random variables. This naive assumption repeatedly failed: systematic patterns in input data (such as zero-padded customer IDs or common compiler identifier prefixes) produced severe clustering. In open addressing, linear probing suffered from primary clustering, where contiguous blocks of filled slots acted as magnets for subsequent keys. Worse, fixed public hash functions created severe denial-of-service vulnerabilities: web servers running PHP, Java, and Python could be brought to a halt by sending HTTP requests with thousands of distinct parameter names crafted to collide into a single hash bucket, degenerating $O(1)$ lookups into $\Theta(n)$ linked-list traversals. Universal hashing solved this by drawing a randomized hash function fresh at runtime from a provably collision-bounded family.

Expected $O(1)$ dictionary search, insertion, and deletion made hash tables a standard tool. Hash tables power compiler symbol tables, database indices, associative arrays in modern scripting languages (Python dicts, JavaScript objects), deduplication caches, and distributed key-value stores. The trade-off is that hash tables discard key ordering: they cannot support Minimum, Maximum, Predecessor, Successor, or sorted range queries ($[\text{low}, \text{high}]$) without scanning all m slots ($\Theta(m)$ time). Dynamic resizing causes latency spikes when tables double, and linked-list chaining incurs cache misses from pointer chasing. To restore order without sacrificing dynamic efficiency, we turn to tree-based structures. In **Chapter 12 (Binary Search Trees)**, we replace pseudo-random scattering with the binary-search-tree property, which supports sorted in-order traversals and fast predecessor searches.

11.10 Probabilistic Data Structures: Bloom Filters and Count-Min Sketch

Every hash table we have analyzed up to this point shares an implicit contract: it stores the keys themselves, or pointers to where those keys live. If an item is present, the table returns its record; if it is absent, the table reports absence with absolute certainty.

In large-scale systems, that contract can be too expensive.

Imagine an LSM-tree (Log-Structured Merge-tree) powering a distributed storage engine like Google Bigtable, Apache Cassandra, or RocksDB. The system stores billions of records across hundreds of immutable files on disk, known as SSTables (Sorted String Tables). When a client requests a read for

key k, the database might need to consult forty different SSTables. Must it perform forty separate disk reads just to discover that key k is absent from thirty-nine of them? A single solid-state drive or spinning disk read takes anywhere from tens of microseconds to several milliseconds. Multiplying that latency by forty disk accesses per read would cripple query throughput.

A natural first reaction is: keep an in-memory hash table of all keys contained in each SSTable. But compute the arithmetic. If a storage node hosts 10^9 keys and each key is a 64-bit integer, storing the raw keys alone requires:

$$10^9 \times 8 \text{ bytes} = 8 \text{ gigabytes of RAM.}$$

Once you include bucket arrays, collision pointers, and load-factor padding to keep lookups fast, the memory footprint easily balloons to 16 to 32 gigabytes. For an embedded database, a network router line-card, or a web browser validating visited URLs against a blacklist of hundreds of millions of malicious phishing domains, keeping all keys in RAM is not affordable.

The way out is to ask whether we really need 100% certainty in both directions.

11.10.1 Bloom filters: trading perfection for compactness

In 1970, Burton Howard Bloom proposed a compromise. Suppose we do not demand complete certainty when an item is reported present, but we keep a firm guarantee when an item is reported absent:

- If the structure reports **"Definitely Absent"**, the item is guaranteed not to be in the set. We can skip the disk seek entirely, with zero risk of missing real data.
- If the structure reports **"Probably Present"**, the item might be in the set, but there is a tiny, tunable probability ε (such as 1%) that it is a *false positive*.

This asymmetric contract guarantees **zero false negatives** ($\P(\text{report absent} \mid \text{present}) = 0$) in exchange for a small, controlled rate of **false positives** ($\P(\text{report present} \mid \text{absent}) = \varepsilon$). If $\varepsilon = 0.01$, ninety-nine out of every one hundred queries for non-existent keys are eliminated in RAM without touching disk. The one remaining query incurs a disk read that returns empty, a small performance cost.

How can a data structure provide this guarantee without storing any keys? Instead of storing the keys, a *Bloom filter* stores only a single flat array $B[0\,..\,m-1]$ of m bits, initially all set to 0, accompanied by k independent, uniform hash functions:

$$h_1, h_2, \ldots, h_k : U \to \{0, 1, \ldots, m-1\}.$$

The mechanics are shown in Figure 11.6(a):

- **Insertion of key x:** Compute the k hash values $h_1(x), h_2(x), \ldots, h_k(x)$ and set every corresponding bit to 1:
$$B[h_i(x)] \leftarrow 1 \quad \text{for each } i \in \{1, 2, \ldots, k\}.$$
- **Query for key y:** Inspect the k probe positions $h_1(y), h_2(y), \ldots, h_k(y)$ in array B:
  - If *at least one* bit $B[h_i(y)] = 0$, then y was never inserted. Had y been inserted, that bit would have been flipped to 1. The query immediately short-circuits and reports False ("Definitely Absent").

– If *all* k bits $B[h_i(y)] = 1$, the query reports TRUE ("Probably Present").

Deletion is impossible in a standard Bloom filter: clearing a bit back to 0 could erase the presence of other keys whose hash functions happened to land on that same slot.

11.10.2 First-principles derivation of false positives and optimal parameters

We now derive the false-positive probability ε.

Suppose we insert n distinct keys into an array of m bits using k independent hash functions that scatter each key uniformly across the m slots.

Consider a single fixed bit in the array during one hash computation of one key. The probability that this specific bit is *not* chosen by the hash function is:

$$1 - \frac{1}{m}.$$

Because each of the n keys is hashed by k independent hash functions, there are kn total bit selections made during the insertion phase. Assuming independence across hash choices, the probability that our chosen bit remains 0 after all n insertions have taken place is:

$$p_0 = \left(1 - \frac{1}{m}\right)^{kn} = \left[\left(1 - \frac{1}{m}\right)^{m}\right]^{kn/m}.$$

Using the fundamental limit $\lim_{m\to\infty}(1 - 1/m)^m = e^{-1}$, we obtain the clean approximation:

$$p_0 \approx e^{-kn/m}.$$

The probability that this bit has been flipped to 1 is therefore:

$$p_1 = 1 - p_0 \approx 1 - e^{-kn/m}.$$

Now consider querying a key y that was never inserted into the set. What is the probability that the filter mistakenly answers that y is present? A false positive occurs if and only if all k bits examined during the query of y happen to be 1. Assuming the k probe locations behave independently, the false-positive probability ε is:

$$\varepsilon = (p_1)^k \approx \left(1 - e^{-kn/m}\right)^k. \tag{11.1}$$

In equation (11.1), for a fixed ratio of bits per key, m/n, what value of k minimizes the error rate ε?

- If k is very small (say $k = 1$), we test only a single bit. An absent key easily lands on a bit that was set by some other key, yielding a high false-positive rate.
- If k is very large, each inserted key sets many bits. The array quickly fills with ones, so $p_1 \to 1$, and again almost every query returns a false positive.

Between these two extremes lies a unique minimum, visible in Figure 11.6(b). To calculate this minimum, take the natural logarithm of ε:

$$g(k) = \ln \varepsilon = k \ln\left(1 - e^{-kn/m}\right).$$

Let $p = e^{-kn/m}$, the fraction of bits that remain 0. Taking the natural logarithm gives $\ln p = -kn/m$, so:

$$k = -\frac{m}{n} \ln p.$$

Substituting this expression for k into $g(k)$:

$$g(k) = -\frac{m}{n} \ln p \cdot \ln(1-p).$$

Because $-m/n$ is a negative constant, minimizing $g(k)$ is equivalent to maximizing the product:

$$f(p) = \ln p \cdot \ln(1-p) \quad \text{for } p \in (0,1).$$

Notice the symmetry of $f(p)$: swapping p with $1-p$ leaves the product unchanged ($f(p) = f(1-p)$). The unique maximum on the open interval $(0,1)$ occurs at the midpoint:

$$p = \frac{1}{2}.$$

So **a Bloom filter is at its optimum when half the bits are zeros and half are ones.**

Setting $p = e^{-kn/m} = 1/2$, we solve for the optimal number of hash functions $k^\star$:

$$-\frac{kn}{m} = \ln\left(\frac{1}{2}\right) = -\ln 2 \implies k^\star = \frac{m}{n} \ln 2 \approx 0.6931 \left(\frac{m}{n}\right).$$

When operating at $k^\star$, the minimized false-positive probability is:

$$\varepsilon = \left(1 - \frac{1}{2}\right)^{k^\star} = 2^{-k^\star} = 2^{-(m/n)\ln 2} = e^{-(m/n)(\ln 2)^2} \approx (0.6185)^{m/n}.$$

Rearranging this equation gives the required memory ratio m/n as a function of the desired error rate ε:

$$\frac{m}{n} = -\frac{\ln \varepsilon}{(\ln 2)^2} \approx -1.4427 \ln \varepsilon \approx 2.081 \log_2\left(\frac{1}{\varepsilon}\right).$$

Consider what this means in practice:

- For a 1% false-positive rate ($\varepsilon = 0.01$), the formula requires $m/n \approx 1.4427 \times \ln(100) \approx 9.57 \approx 10$ bits per key, with $k^\star = 10 \ln 2 \approx 7$ hash functions.
- For a 0.1% false-positive rate ($\varepsilon = 0.001$), we need $m/n \approx 1.4427 \times \ln(1000) \approx 14.35 \approx 15$ bits per key, with $k^\star = 15 \ln 2 \approx 10$ hash functions.

At 10 bits per key (just 1.25 bytes), a billion keys require only 1.2 gigabytes of memory, fitting comfortably in RAM on standard servers while filtering out 99% of unnecessary disk reads.

11.10.3 From set membership to stream frequency: Count-Min Sketch

A Bloom filter answers a binary question about set membership: has key x appeared at least once?

In high-throughput distributed systems – such as network switches inspecting packet headers, search engines tracking surging queries, and telemetry pipelines aggregating metrics over millions of events per second – we face a quantitative question:

> "In a streaming sequence of billions of items, how many times has item x appeared?"

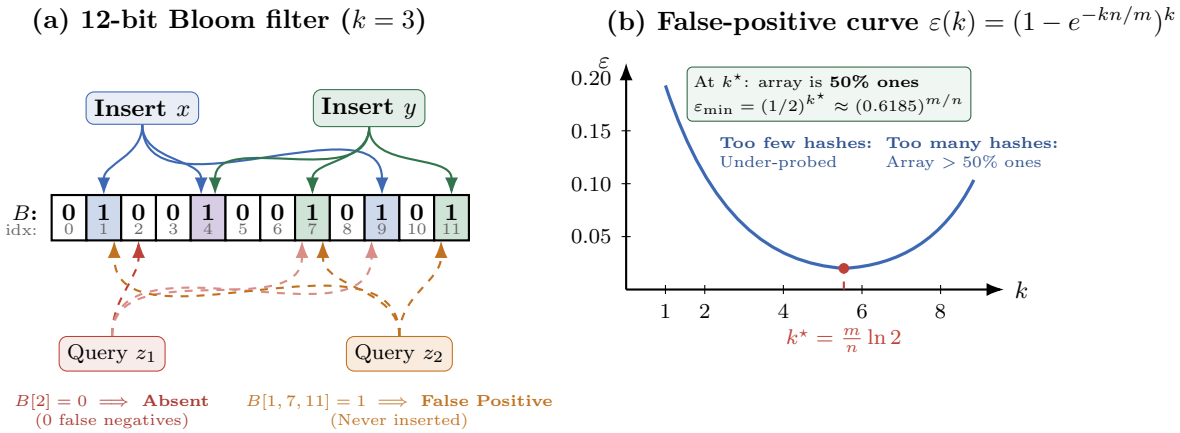


Figure 11.6: The mechanics and mathematics of a Bloom filter. (a) A 12-bit array populated by keys x and y with $k = 3$. Querying z_1 immediately reveals it is absent because $B[2] = 0$, guaranteeing zero false negatives. Querying z_2 yields a false positive because its three probe positions $(1, 7, 11)$ happen to have been set by other elements. (b) The false-positive probability ε as a function of the number of hash functions k. The curve attains its global minimum at $k^\star = (m/n) \ln 2$, the operating point where exactly half the array bits are ones.

Storing an exact counter for every unique item in a hash table requires memory proportional to the number of distinct keys. When the universe of keys contains millions of IP addresses or URLs, an exact table exceeds cache limits.

In 2005, Graham Cormode and S. Muthukrishnan introduced the *Count-Min Sketch*, extending the Bloom filter's bit-array idea to integer frequency estimation.

Instead of a single one-dimensional bit array, a Count-Min Sketch maintains a two-dimensional grid of integer counters $C[1\,..\,d, 1\,..\,w]$ with depth d and width w:

$$\begin{pmatrix} C[1,1] & C[1,2] & \dots & C[1,w] \\ C[2,1] & C[2,2] & \dots & C[2,w] \\ \vdots & \vdots & \ddots & \vdots \\ C[d,1] & C[d,2] & \dots & C[d,w] \end{pmatrix}$$

Each row $i \in \{1, \dots, d\}$ is paired with its own independent hash function $h_i : U \to \{1, \dots, w\}$. All counters are initialized to zero.

- **Update on arrival of item x:** Whenever an occurrence of item x arrives in the stream (or its count increases by $c \geq 1$), we increment the hashed counter in every row:

$$C[i, h_i(x)] \leftarrow C[i, h_i(x)] + c \quad \text{for each row } i \in \{1, \dots, d\}.$$

- **Point query for estimated frequency $\hat{f}(x)$:** To estimate the true frequency $f(x)$ of item x, inspect the counter at column $h_i(x)$ across all d rows, and return the **minimum**:

$$\hat{f}(x) = \min_{1 \leq i \leq d} C[i, h_i(x)]. \tag{11.2}$$

Why does taking the minimum work so well?

Because counters are only ever incremented by non-negative amounts, hash collisions can only ever add positive excess to a counter. If another item $y \neq x$ collides with x in row i ($h_i(y) = h_i(x)$), y's occurrences inflate $C[i, h_i(x)]$. Collisions never subtract. Consequently, every single cell is an upper bound on the true count:

$$C[i, h_i(x)] \geq f(x) \quad \text{for every row } i \in \{1, \ldots, d\}.$$

Because every counter satisfies $C[i, h_i(x)] \geq f(x)$, their minimum also satisfies $\hat{f}(x) \geq f(x)$. The estimate never underestimates. By taking the minimum over d rows hashed independently, we pick the row whose counter suffered the least collision contamination from other elements.

Theorem 11.5 (Count-Min Sketch accuracy guarantee)**.** *Let $N = \sum_y f(y)$ be the total number of items processed in the stream. If we choose width $w = \lceil e/\varepsilon \rceil$ and depth $d = \lceil \ln(1/\delta) \rceil$, the estimate $\hat{f}(x)$ satisfies:*

$$f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N$$

with probability at least $1 - \delta$.

Proof. Fix an item x and consider row i. The counter $C[i, h_i(x)]$ equals the true frequency $f(x)$ plus the contributions from all colliding items $y \neq x$:

$$C[i, h_i(x)] = f(x) + \sum_{y \neq x} f(y) \cdot \mathbb{I}\{h_i(y) = h_i(x)\}.$$

Define the random indicator variable $X_{i,y} = \mathbb{I}\{h_i(y) = h_i(x)\}$. Under uniform hashing into w slots, $\mathbb{E}[X_{i,y}] = ¶(h_i(y) = h_i(x)) = 1/w$. By linearity of expectation, the expected overcount in row i is:

$$\mathbb{E}\big[C[i, h_i(x)] - f(x)\big] = \sum_{y \neq x} f(y) \cdot \frac{1}{w} \leq \frac{N}{w}.$$

With $w = \lceil e/\varepsilon \rceil$, this expected error is at most $\varepsilon N/e$. Because the overcount $C[i, h_i(x)] - f(x)$ is non-negative, Markov's inequality implies:

$$¶\Big(C[i, h_i(x)] - f(x) \geq \varepsilon N\Big) \leq \frac{\mathbb{E}[C[i, h_i(x)] - f(x)]}{\varepsilon N} \leq \frac{\varepsilon N/e}{\varepsilon N} = \frac{1}{e}.$$

For the minimum $\hat{f}(x)$ to exceed $f(x) + \varepsilon N$, *every single row* $i \in \{1, \ldots, d\}$ must independently fail to stay within the error bound. Because the d rows use independent hash functions:

$$¶\Big(\hat{f}(x) > f(x) + \varepsilon N\Big) \leq \left(\frac{1}{e}\right)^d = e^{-d}.$$

Setting $d = \lceil \ln(1/\delta) \rceil$ yields $e^{-d} \leq \delta$. Therefore, with probability at least $1 - \delta$, the estimate is bounded by $f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N$. □ □

The defining property of both Bloom filters and Count-Min sketches is that their memory footprint depends only on the desired accuracy parameters (ε and δ), independent of the length of the data stream or the size of the key universe. By accepting a small, bounded probability of error, they track very large streams in small, predictable memory.

11.11 Choosing a scheme: a practical recap

You now have several collision strategies. Which to reach for?

- **Chaining** is the safe default. It tolerates load factors above 1, degrades gracefully, and handles deletions cleanly. Its cost is pointer overhead and worse cache behavior. Pair it with a *universal* hash function and you get $O(1)$ expected time with no assumptions on the keys – and protection against hash-flooding.
- **Open addressing** (especially *double hashing*) is leaner and cache-friendlier, with no pointer overhead. Use it when memory is tight and deletions are rare, and keep the load factor well under 1 (resize at, say, 0.7). Watch out for tombstones and clustering.
- **Perfect hashing** is the specialist: a *static*, known-in-advance key set where you want a guaranteed single-probe lookup – compiler keyword tables, immutable lookup sets.
- **Bloom filters and sketches** are the probabilistic specialists. When datasets contain billions of keys that cannot physically fit in RAM, or when monitoring high-velocity data streams, trade a small, tunable probability of false positives for zero false negatives and large memory savings (≈ 10 bits per key for 1% error with a Bloom filter, and counter grids for frequency estimation with Count-Min sketches).

Pause

If the chapter feels like a lot, here is the summary to hold onto. (1) Direct addressing is $O(1)$ but needs $|U|$ space – too much. (2) Hashing shrinks $|U|$ down to m slots, creating collisions. (3) Chaining resolves collisions with lists; expected search is $\Theta(1+\alpha)$. (4) Open addressing stores everything in the array and probes; expected search is about $\frac{1}{1-\alpha}$. (5) Universal hashing makes the expectation hold for *any* keys by randomizing the function. (6) Perfect hashing achieves $O(1)$ worst-case for static sets via two universal levels. (7) Bloom filters trade a tunable false-positive rate for zero false negatives, compressing set membership into ≈ 10 bits per key; Count-Min sketches generalize this to estimate stream frequencies using counter grids. Everything else is detail and proof.

The price hash tables pay for expected $O(1)$ speed is the loss of key order. By scattering keys uniformly across an array, a hash table cannot answer questions like "what is the minimum key?", "which key comes next after x?", or "list all elements between 10 and 20" without scanning all m slots. When your workload requires dynamic insertions and deletions while keeping keys in sorted order, you need a different structure. In Chapter 12, we turn to **Binary Search Trees**, where hierarchical branching preserves key order to support searches, successors, and sorted range queries in logarithmic time.

11.12 Exercises

Exercise 11.1. A direct-address table $T[0\,..\,9]$ starts empty. Show its contents (which slots are non-NIL) after INSERT of keys 3, 7, 1, 7 in order, then DELETE of key 7. (Treat re-inserting 7 as overwriting its slot.) What is the worst-case time of each operation?

Exercise 11.2. A chained hash table has $m = 7$ slots and uses $h(k) = k \bmod 7$. Insert keys

$15, 22, 8, 1, 29, 36$ in order, always at the head of the list. Draw all seven lists. What is the load factor? For a successful search of 22, how many list nodes are examined? For an unsuccessful search of 9?

Exercise 11.3. You expect to store about $n = 10{,}000$ keys and want an average chain length of about 2. What table size m should you choose, and why should it be a prime not near a power of 2 if you use the division method? Give a concrete suitable prime.

Exercise 11.4. Using the multiplication method with $A = (\sqrt{5} - 1)/2 \approx 0.6180339887$ and $m = 8$, compute $h(k)$ for $k = 50$ and $k = 51$. Show the intermediate fractional part each time. Do consecutive keys land in nearby slots or scattered ones?

Exercise 11.5. A table of size $m = 11$ uses linear probing with $h'(k) = k \bmod 11$. Insert the keys $26, 37, 15, 48, 4$ in order and show the final array. Identify the longest run of consecutive occupied slots, and explain in your own words why such runs form (primary clustering).

Exercise 11.6. With $m = 13$, $h_1(k) = k \bmod 13$, and $h_2(k) = 1 + (k \bmod 11)$, write out the first five probe positions for double hashing of key $k = 27$. Verify that $h_2(27)$ is between 1 and 11 and therefore coprime to 13.

Exercise 11.7. Using Theorem 11.2, compute the expected number of probes for an *unsuccessful* search when $\alpha = 0.8$ and when $\alpha = 0.95$. Then explain, in terms of these numbers, why production hash tables resize once the load factor crosses a threshold like 0.7.

Exercise 11.8. Consider the universal family $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$ with $p = 17$ and $m = 6$. With $a = 2$, $b = 3$, compute $h_{2,3}(k)$ for $k = 0, 5, 11$. Then pick a *different* pair (a, b) and recompute $h(5)$; observe that the slot changes. Why does the randomness in (a, b) defend against an adversary who chooses the keys?

Exercise 11.9. Prove that if a hash function h maps a set of n keys into m slots and $n > m$, then there must be at least one collision. (This is the pigeonhole principle. State it carefully and connect it to why collision resolution is unavoidable when $|K| > m$.)

Exercise 11.10. In perfect hashing, suppose the top level sends $n_j = 4$ keys into some slot j. What size m_j does the secondary table get? Using the collision-pair argument of Lemma 11.1, give an upper bound on the expected number of colliding pairs in that secondary table, and explain why this means a collision-free secondary function is easy to find.

Exercise 11.11. (Challenge.) Explain the "hash-flooding" denial-of-service attack: how can an adversary who knows your fixed hash function force a chained hash table's performance to degrade from $O(1)$ to $O(n)$ per operation? Describe precisely how switching to a randomly chosen universal hash function defeats the attack, referencing Theorem 11.3.

Exercise 11.12. (Challenge.) Compare chaining and open addressing on *deletion*. Explain why deletion takes direct $O(1)$ pointer operations in a doubly-linked chained table but requires tombstones in open addressing, and describe one concrete failure that occurs if you delete by simply setting the slot to NIL. Then argue why a table full of tombstones can make searches slow even when n is small.

Exercise 11.13 (Bloom filter dimensioning and zero false negatives)**.** A storage engine stores $n = 5 \times 10^7$ keys and allocates an array of $m = 5 \times 10^8$ bits ($m/n = 10$). (a) Compute the optimal number of hash functions $k^\star$ to minimize false positives, rounding to the nearest integer. (b) Using that $k^\star$, calculate the theoretical false-positive probability ε. (c) Explain physically why a query that observes a single 0 bit at any of its k probed locations provides an absolute guarantee that the key is absent (0 false negatives), regardless of how many other keys have been inserted into the array.

Exercise 11.14 (Count-Min Sketch under collisions)**.** A network router uses a Count-Min Sketch with depth $d = 3$ and width $w = 5$. Counters $C[i, j]$ are initially zero. Three packet streams A, B, and C arrive with true volumes $f(A) = 10$, $f(B) = 4$, and $f(C) = 2$. The hash functions map the keys as follows: $h_1(A) = 2$, $h_2(A) = 4$, $h_3(A) = 1$; $h_1(B) = 2$, $h_2(B) = 1$, $h_3(B) = 5$; $h_1(C) = 3$, $h_2(C) = 4$, $h_3(C) = 1$. (a) Trace the counter matrix $C[1..3, 1..5]$ after processing all arrivals. (b) Compute the estimated frequencies $\hat{f}(A)$, $\hat{f}(B)$, and $\hat{f}(C)$ using the minimum rule $\hat{f}(x) = \min_{1 \le i \le 3} C[i, h_i(x)]$. (c) Verify that $\hat{f}(x) \ge f(x)$ for all three keys, and identify which row provided the exact count for each key. Why can hash collisions never cause undercounting?

Chapter 12

Binary Search Trees

Chapter 11 showed how hash tables deliver expected $\Theta(1)$ search by scattering keys across an array. But hashing scrambles key order, so sorted walks, range queries, and successor searches become expensive. A sorted array keeps keys in order and allows fast $\Theta(\log n)$ binary search, but inserting a new item means shifting half the array over to make room.

In this chapter, we build a structure that achieves both: fast lookups *and* fast dynamic updates, all while maintaining strict sorted order. It is called the *binary search tree* (BST). The idea is to build the basic step of binary search – "compare, then go left or right" – directly into the branching shape of the data. Instead of repeatedly cutting a sorted array in half, we build a tree whose every fork already encodes a comparison.

By the end you will know how a BST is shaped, how to search it, how to find the smallest and largest elements, how to walk from any element to the "next" one, and how to insert and delete while keeping the shape valid. Along the way, we will see a direct connection to Chapter 7: building a BST by inserting items in random order performs the exact same comparisons as randomized quicksort. That observation gives us the expected height of a random BST with little extra work.

12.1 The shape of the idea

12.1.1 From sorted arrays to branching decisions

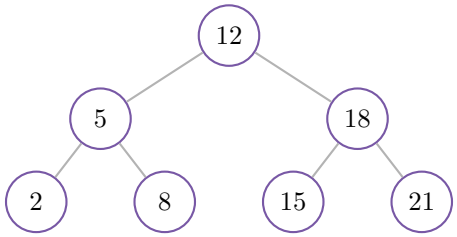
Picture binary search on the sorted array $[2, 5, 8, 12, 15, 18, 21]$. To find 15, you look at the middle element 12, decide 15 is bigger, and throw away the left half. Now you look at the middle of what remains, 18, decide 15 is smaller, and throw away the right half. One more comparison and you are done. Every step is a single comparison that sends you either left (toward smaller things) or right (toward larger things).

A binary search tree is what you get if you write that decision structure down once and reuse it. Each element becomes a little box called a *node*. Every node holds a key and has at most two children: a *left child* pointing to a subtree of smaller keys and a *right child* pointing to a subtree of larger keys. Searching is then just walking down from the top, going left or right at each node exactly as binary search would.

Intuition

A BST turns the comparisons of binary search into explicit pointers. In a sorted array, you recompute midpoints from array indices on every query. In a BST, each node already separates smaller keys into its left pointer and larger keys into its right pointer, so searching simply follows those links down from the root.

Here is a tree holding those same seven keys. Read it top to bottom.



Notice what is true at *every* node. Look at 12: everything in its left subtree $(5, 2, 8)$ is less than 12, and everything in its right subtree $(18, 15, 21)$ is greater. Look at 5: its left subtree (2) is smaller, its right (8) is larger. The same rule holds at every node, and that rule is the definition.

Formal

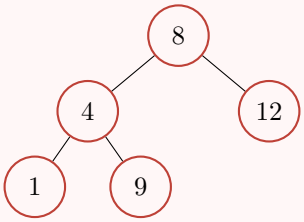
Definition 12.1 (Binary search tree property)**.** A *binary search tree* is a binary tree in which each node x stores a key $x.key$ and pointers $x.left$, $x.right$, $x.parent$. The tree satisfies the *binary-search-tree property*: for every node x,

- if y is a node in the **left** subtree of x, then $y.key \leq x.key$; and
- if y is a node in the **right** subtree of x, then $y.key \geq x.key$.

The topmost node is the *root*; a node with no children is a *leaf*. A missing child is represented by the special value NIL.

Common trap

The BST property is about *entire subtrees*, rather than merely immediate children. A common beginner error is to check only that a node's left child is smaller and its right child is larger, and declare victory. That is not enough. In the tree below, every parent-child pair looks locally fine, yet 9 sits in the left subtree of 8 where only keys ≤ 8 are allowed:



The local check at 4 passes ($9 > 4$, fine for a right child). But $9 > 8$, so 9 must not be anywhere left of 8. This is *not* a valid BST.

12.1.2 The in-order walk

The BST property pays off immediately. Suppose you visit the whole tree in a particular rhythm: for each node, first fully visit its *left* subtree, then print the node's own key, then fully visit its *right* subtree. This rhythm is called an *in-order traversal*.

Algorithm 43: IN-ORDER-WALK(x)

Input: A node x (possibly NIL) of a BST.
Output: Prints the keys of x's subtree in sorted order.

```
1 if x ≠ NIL then
2     IN-ORDER-WALK(x.left);
3     print x.key;
4     IN-ORDER-WALK(x.right);
```

Run it on the seven-key tree above. We start at 12, but before printing 12 we must finish its left subtree rooted at 5. Before printing 5 we must finish its left subtree, which is just 2. So 2 prints first. Then back up: 5 prints, then 5's right child 8. Now the left subtree of 12 is done, so 12 prints. Then we descend into 12's right subtree at 18: first its left child 15, then 18, then 21. The output is

$$2,\ 5,\ 8,\ 12,\ 15,\ 18,\ 21,$$

which is *exactly* sorted order. That sorted output follows directly from the BST property at every node.

Figure 12.1 shows the visiting order as small superscripts: each node is printed only after its entire left subtree, so the keys come out smallest-first.

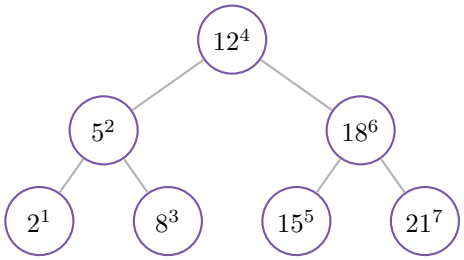


Figure 12.1: The in-order walk of the seven-key tree. Superscripts give the print order: left subtree, then the node, then the right subtree. Reading the superscripts $1, 2, \ldots, 7$ visits the keys $2, 5, 8, 12, 15, 18, 21$ in sorted order.

Focus

An in-order walk of a BST prints the keys in sorted order. So a BST is, among other things, a way of *storing a sorted sequence* that you can also splice into and delete from cheaply. Keep the picture in mind: "left subtree, me, right subtree" = "everything smaller, me, everything larger" = sorted.

Lemma 12.1 (Correctness of the in-order walk)**.** IN-ORDER-WALK(x) *prints the keys of the subtree rooted at x in nondecreasing order.*

Proof. We argue by induction on the size n of the subtree rooted at x. If $n = 0$ then $x =$ NIL and

nothing is printed; an empty sequence is vacuously sorted, so the base case holds. Now suppose the claim holds for all subtrees of size less than n, and let x have a subtree of size $n \geq 1$. Its left and right subtrees each have fewer than n nodes, so by the induction hypothesis the call In-Order-Walk($x.left$) prints the left subtree's keys in sorted order, and likewise on the right. By the BST property (Definition 12.1), every left-subtree key is $\leq x.key$ and every right-subtree key is $\geq x.key$. The walk prints (left keys, sorted) then $x.key$ then (right keys, sorted), and these three blocks are already in the correct relative order. The concatenation is therefore sorted. □

Since the walk touches each of the n nodes exactly once and does a constant amount of work per node, it runs in $\Theta(n)$ time.

Walking question

You have just seen that a BST can be flattened into a sorted list in linear time. So here is a question to chew on while you walk: *given* a sorted list, how many different BSTs produce it as their in-order walk? The seven-key tree we drew was nicely balanced, but is it the only tree whose in-order walk is $2, 5, 8, 12, 15, 18, 21$? Try to build a wildly different one by hand. What does that tell you about how much the *shape* of a BST can vary while the *contents* stay fixed?

12.2 Searching, minimum, maximum

12.2.1 Searching for a key

Searching is the operation a BST is named for, and it is exactly the binary-search rhythm. Start at the root. If the node's key equals what you want, stop. If your target is smaller, go to the left child; if larger, go to the right. Repeat until you find the key or fall off the bottom of the tree into a NIL.

Algorithm 44: Tree-Search(x, k)

Input: Root x of a (sub)tree, search key k.
Output: A node with key k, or NIL if absent.

```
while x ≠ NIL and k ≠ x.key do
    if k < x.key then
        x ← x.left
    else
        x ← x.right
return x;
```

Let us trace a search for $k = 15$ in the seven-key tree. We start at 12. Since $15 > 12$ we go right to 18. Since $15 < 18$ we go left to 15. Now $k = x.key$, so we stop and return that node. Three comparisons, one per level. A search for $k = 7$ would go: 12 (go left, $7 < 12$) $\rightarrow$ 5 (go right, $7 > 5$) $\rightarrow$ 8 (go left, $7 < 8$) $\rightarrow$ NIL, and we return NIL – 7 is not in the tree. Figure 12.2 highlights both paths: the successful hunt for 15 ending on a node, and the failed hunt for 7 running off the bottom into a missing child.

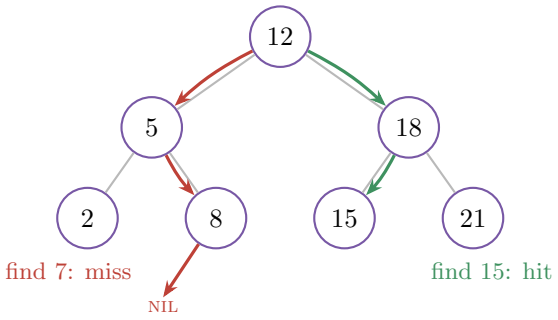


Figure 12.2: Two searches in the seven-key tree. Green: searching for 15 goes right then left and lands on a node. Red: searching for 7 goes left, right, left, then falls off the bottom into NIL, so 7 is absent.

Intuition

A search never wanders. At each node it either finishes or commits to exactly one child and never looks back. So a search traces a single *path* from the root downward. The number of comparisons is at most the length of that path, which is at most the *height* of the tree. This is why height is the number that controls everything in this chapter.

12.2.2 Minimum and maximum

Where is the smallest key? The BST property says everything smaller than a node lives in its left subtree. So to find the minimum, keep following left children until you cannot anymore. The last node you reach has no left child, which means nothing in the tree is smaller. The maximum is the mirror image: follow right children to the end.

Algorithm 45: TREE-MINIMUM(x)

Input: Root $x \neq$ NIL of a (sub)tree.
Output: The node holding the smallest key.

```
1 while x.left ≠ NIL do
2     x ← x.left
3 return x;
```

In our tree the minimum walk is $12 \to 5 \to 2$, and 2 has no left child, so 2 is the minimum. The maximum walk is $12 \to 18 \to 21$, giving 21. Both touch one node per level, so both run in $O(h)$ time where h is the height.

12.3 The successor: finding the next key

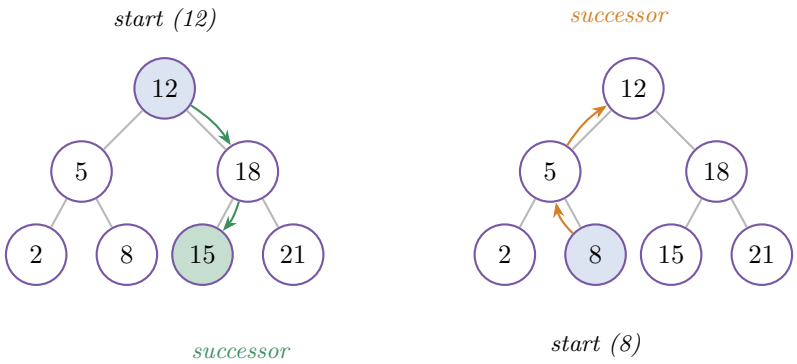
Suppose you are standing at a node and you want the node with the *next-largest* key – the one that would come immediately after it in the sorted in-order list. This is the *successor*, and it is the trickiest of the basic queries, so let us build it carefully.

There are two cases, and the picture makes them obvious. Think about where the "next" element hides relative to the in-order walk.

Case 1: the node has a right subtree. Then everything in that right subtree is larger than our node, and the successor is the *smallest* thing in there. So the answer is TREE-MINIMUM of the right child. For example, the successor of 12 is the minimum of the right subtree rooted at 18, which is 15. Check the sorted list: $\ldots, 12, 15, \ldots$ – correct.

Case 2: the node has no right subtree. Then there is nothing larger "below" it, so the next-largest key must be "above" it. We climb upward. As long as we are coming up from a *right* child, our parent is smaller than us, so it cannot be the successor; keep climbing. The first time we come up from a *left* child, that parent is larger than us and is the successor. (If we climb all the way off the top, our node was the maximum and has no successor.)

Figure 12.3 shows both cases on the seven-key tree. On the left, the successor of 12 is found by descending to the minimum of its right subtree. On the right, the successor of 8 (which has no right child) is found by climbing: up through 5 as a right child, then stopping at 12, the first ancestor reached from a left child.



Case 1: go right, then all the way left Case 2: climb until you rise from a left child

Figure 12.3: The two cases of TREE-SUCCESSOR. Left: when a right subtree exists, the successor is its minimum. Right: with no right subtree, the successor is the lowest ancestor whose left subtree contains the node.

Algorithm 46: TREE-SUCCESSOR(x)

Input: A node x in a BST.
Output: The node with the next-larger key, or NIL if x is the maximum.

```
1 if x.right ≠ NIL then
2     return TREE-MINIMUM(x.right);
3 y ← x.parent;
4 while y ≠ NIL and x = y.right do
5     x ← y;
6     y ← y.parent;
7 return y;
```

Algorithm lab

Trace TREE-SUCCESSOR on node 8 in the seven-key tree. Node 8 has no right child, so we are in Case 2. Set $y = 5$ (the parent of 8). Is 8 a right child of 5? Yes, so we climb: now $x = 5$, $y = 12$. Is 5 a right child of 12? No, 5 is the *left* child of 12. The loop stops and we return $y = 12$. And indeed, in $2, 5, 8, 12, \ldots$ the key right after 8 is 12. Now trace it yourself on node 21 (the maximum): you should climb all the way up, fall off the top, and return NIL.

Both cases follow a single path – either straight down the right subtree or straight up the ancestor chain – so TREE-SUCCESSOR runs in $O(h)$ time. The mirror algorithm, TREE-PREDECESSOR, finds the next-*smaller* key by swapping "left" and "right."

Why this matters

Successor and predecessor are what make a BST behave like a navigable, ordered collection rather than a mere lookup table. They are why the ordered maps and sets in standard libraries (C++ `std::map`, Java's `TreeMap`) can answer "give me the smallest key $\geq x$" or iterate keys in sorted order. A hash table, fast as it is, cannot do any of that: it has no notion of "next." When you need range queries or ordered iteration, you reach for a balanced BST.

12.4 Insertion

To insert a new key, we want to find the spot where a *search* for that key would have fallen off the tree into a NIL, and hang the new node there. That placement automatically preserves the BST property, because we walked down using the very comparisons the property demands.

We keep two pointers as we descend: x races ahead looking for the empty slot, and a trailing pointer y remembers x's parent so we know what to attach the new node to.

Algorithm 47: TREE-INSERT(T, z)

Input: Tree T and a new node z with $z.key$ set, $z.left = z.right =$ NIL.
Output: T with z inserted.

```
1  y ← NIL;
2  x ← T.root;
3  while x ≠ NIL do
4  |   y ← x;
5  |   if z.key < x.key then
6  |   |   x ← x.left
7  |   else
8  |   |   x ← x.right
9  z.parent ← y;
10 if y = NIL then
11 |   T.root ← z ;                                   // tree was empty
12 else if z.key < y.key then
13 |   y.left ← z
14 else
15 |   y.right ← z
```

Example 12.1 (Building a tree by hand)**.** Start with an empty tree and insert keys $12, 5, 18, 2, 8, 15, 21$ in order.

- Insert 12: tree is empty, so 12 becomes the root.
- Insert 5: at root 12, $5 < 12$, go left into NIL. Attach 5 as the left child of 12.
- Insert 18: at 12, $18 > 12$, go right into NIL. Attach 18 as right child.
- Insert 2: $12 \to$ left to $5 \to (2 < 5)$ left into NIL. Attach under 5.
- Insert 8: $12 \to$ left to $5 \to (8 > 5)$ right into NIL. Attach under 5.
- Insert 15: $12 \to$ right to $18 \to (15 < 18)$ left into NIL. Attach under 18.
- Insert 21: $12 \to$ right to $18 \to (21 > 18)$ right into NIL. Attach under 18.

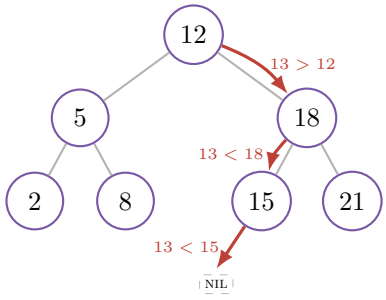
The result is exactly the balanced seven-key tree from the start of the chapter. Notice that the root is whatever you inserted *first*. We will lean on that fact heavily later.

Insertion walks one root-to-leaf path and then does constant work to attach the node, so it costs $O(h)$.

Common trap

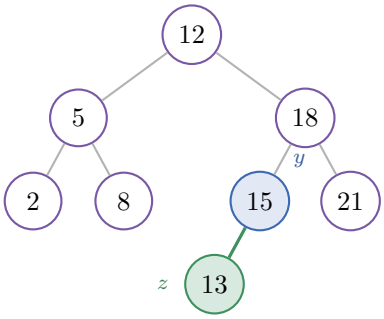
The order you insert keys determines the shape you get. Insert $2, 5, 8, 12, 15, 18, 21$ – already sorted – and every key is larger than the last, so each one becomes the right child of the previous one. You get a "tree" that is really a straight line slanting to the right:

(A) Descent path for $z = 13$



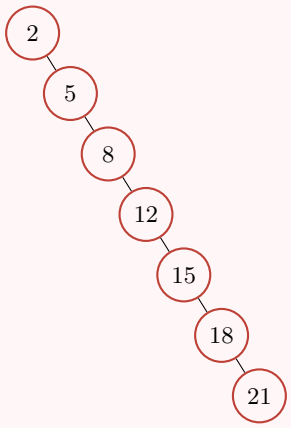
Trailing pointer y stops at 15

(B) Attaching new leaf z



BST property: $12 < 13 < 15 < 18$
$y.left \leftarrow z$, $z.p \leftarrow y$

Figure 12.4: The mechanics of TREE-INSERT(T, z) with $z.key = 13$. In (a), pointer x walks down the tree guided by key comparisons, while trailing pointer y lags one step behind. In (b), when x falls off the tree at NIL, z is spliced in as y's left child.



This is a valid BST, but its height is $n - 1$, so searching it is no better than scanning a linked list: $\Theta(n)$ per operation. The keys are the same, but the shape is far worse. The next chapter (red-black trees) shows how to prevent this.

Intuition

How many BST shapes can n keys form? (The Catalan numbers.) If the insertion order determines the tree's shape, how many shapes are possible? Suppose we have n distinct keys, which we can label $1, 2, \ldots, n$ in increasing order. Any key $i \in \{1, \ldots, n\}$ could end up as the root. Because every key smaller than i must go into the left subtree and every key larger than i into the right subtree, choosing key i as the root leaves exactly $i - 1$ keys on the left and $n - i$ keys on the right.

The shapes of the left and right subtrees can be chosen independently. If C_n denotes the

number of distinct binary tree structures with n nodes, we obtain the recurrence:

$$C_n = \sum_{i=1}^{n} C_{i-1} C_{n-i}, \quad \text{with } C_0 = 1.$$

For small values: $C_1 = 1$, $C_2 = C_0C_1 + C_1C_0 = 2$, $C_3 = C_0C_2 + C_1C_1 + C_2C_0 = 2+1+2 = 5$. These combinatorial quantities are the *Catalan numbers*, with the exact closed-form formula:

$$C_n = \frac{1}{n+1}\binom{2n}{n}.$$

As n grows, Stirling's approximation reveals $C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$. For $n = 10$, there are 16,796 distinct tree shapes; for $n = 20$, over 6.5×10^9. Of these shapes, only a tiny fraction are perfectly balanced, while 2^{n-1} of them are degenerate linear sticks. Fortunately, when keys arrive in random order, the probability concentrates heavily on trees with height $O(\ln n)$ – as we will prove in Section 12.7.

12.5 Deletion

Deletion is the fussiest BST operation because removing a node can leave a hole in the middle of the structure, and we have to fill that hole without breaking the BST property. The good news is there are only three cases, and the third reduces to the others.

Suppose we want to delete node z.

Case A: z has no children. It is a leaf. Just detach it – replace the pointer from its parent with NIL. Nothing else can break.

Case B: z has exactly one child. Lift that child up to take z's place, reconnecting it to z's parent. Because the single child subtree was already entirely on the correct side of z's ancestors, sliding it up one level keeps everything ordered.

Case C: z has two children. This is the interesting case. We cannot simply promote a single child, because z has two branches requiring a parent slot. Instead we find a key that can sit in z's position without disturbing the order: the *successor* of z, call it y. The successor is the smallest key in z's right subtree, so it is larger than everything in z's left subtree and smaller than everything else in z's right subtree – precisely the property z's key had. We splice y out of its current spot and let it stand in for z.

Intuition

Why the successor specifically? Because we need a replacement that is $\geq$ all of z's left subtree and $\leq$ all the rest of z's right subtree. The successor – the next key up in sorted order – is the unique tightest fit on the right side. Because the successor of a two-child node lives in its right subtree and is the minimum there, it has *no left child*, so plucking it out is easy: it falls under Case A or B.

To keep the code clean, we use a small helper, TRANSPLANT, that replaces the subtree rooted at u with the subtree rooted at v by fixing up exactly one parent pointer.

Algorithm 48: TRANSPLANT(T, u, v)

Input: Tree T; node u to be replaced; node v (possibly NIL) to put in its place.

```
1 if u.parent = NIL then
2 |   T.root ← v
3 else if u = u.parent.left then
4 |   u.parent.left ← v
5 else
6 |   u.parent.right ← v
7 if v ≠ NIL then
8 |   v.parent ← u.parent
```

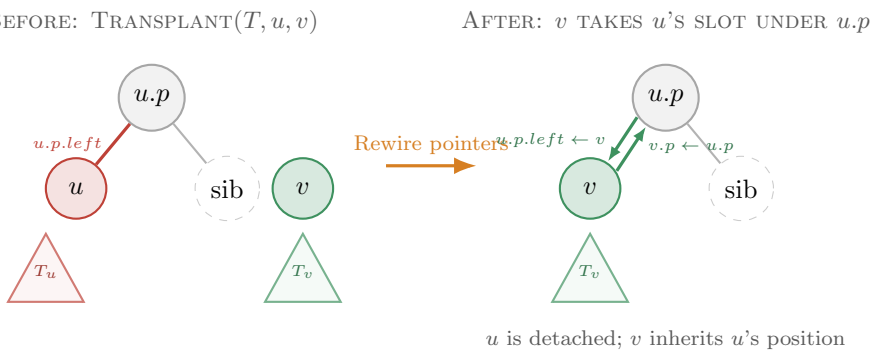


Figure 12.5: The TRANSPLANT(T, u, v) subroutine replaces the subtree rooted at u with the subtree rooted at v. It updates u's parent to point to v and sets v's parent pointer to $u.p$, leaving u's former children undisturbed for caller-specific handling.

Algorithm 49: TREE-DELETE(T, z)

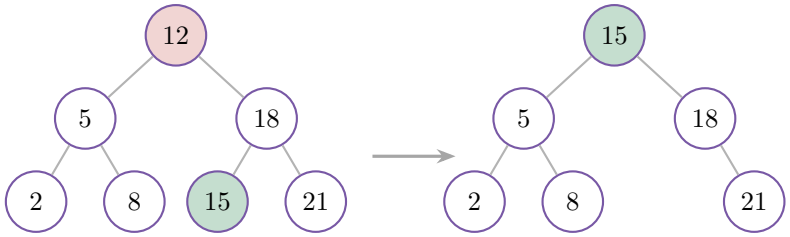
Input: Tree T and a node z to remove.

```
1  if z.left = NIL then
2  |   TRANSPLANT(T, z, z.right) ;          // 0 or 1 child:  lift the right
3  else if z.right = NIL then
4  |   TRANSPLANT(T, z, z.left) ;           // exactly 1 child:  lift the left
5  else
6  |   y ← TREE-MINIMUM(z.right) ;                    // y = successor of z
7  |   if y.parent ≠ z then
8  |   |   TRANSPLANT(T, y, y.right);
9  |   |   y.right ← z.right;
10 |   |   y.right.parent ← y;
11 |   TRANSPLANT(T, z, y);
12 |   y.left ← z.left;
13 |   y.left.parent ← y;
```

The first two branches cover Cases A and B together (if the left child is NIL, lifting the right child handles both a leaf and a one-right-child node; the *else if* handles a lone left child). The final branch is Case C: we find the successor y, detach it from its old location, and slot it in where z was, inheriting both of z's subtrees.

Example 12.2 (Deleting a two-child node)**.** Delete 12 (the root) from the balanced seven-key tree. Node 12 has two children, so we are in Case C. Its successor is the minimum of the right subtree rooted at 18: follow left from 18 to 15, which has no left child. So $y = 15$.

Since y's parent is 18, not 12, we first transplant y out: 15 has no right child, so 18's left pointer becomes NIL, and we set 15's right child to be z's old right subtree (18). Then we transplant 15 into 12's place at the root and give it 12's left subtree (5). Figure 12.6 shows the tree just before and just after the surgery.



before: delete 12 (red), successor 15 (green)

after: 15 rises to the root, keeping both old subtrees

Figure 12.6: Deleting the two-child node 12. Its successor 15 (the minimum of the right subtree) is spliced out of its old spot and lifted into 12's place, inheriting both of 12's subtrees. The in-order order is preserved.

Read the in-order walk of the result: $2, 5, 8, 15, 18, 21$. It is still sorted, with 12 gone, so the BST property still holds.

Like every operation in this chapter, deletion does a constant amount of pointer surgery plus at most one TREE-MINIMUM call, so it runs in $O(h)$.

Cost summary

Every dynamic-set operation on a BST – search, minimum, maximum, successor, predecessor, insert, delete – runs in $O(h)$ time, where h is the height of the tree.

Operation	Balanced	Worst (degenerate)
SEARCH/MIN/MAX	$\Theta(\log n)$	$\Theta(n)$
SUCCESSOR/PREDECESSOR	$\Theta(\log n)$	$\Theta(n)$
INSERT/DELETE	$\Theta(\log n)$	$\Theta(n)$
IN-ORDER-WALK (all keys)	$\Theta(n)$	$\Theta(n)$

Everything hinges on h. A balanced tree has $h = \Theta(\log n)$; a degenerate one has $h = \Theta(n)$. The rest of the chapter is about which one you should *expect*.

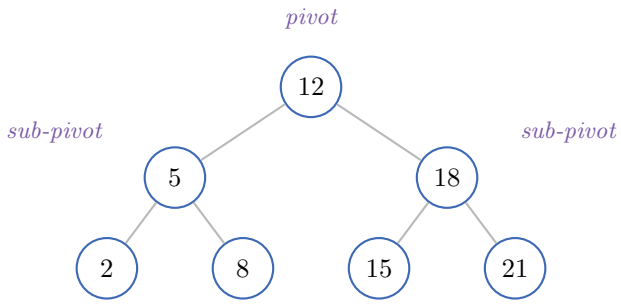
12.6 BSTs and quicksort are the same algorithm

12.6.1 The first surprise

Look again at Example 12.1, where we built a tree by inserting $12, 5, 18, 2, 8, 15, 21$. The very first key, 12, became the root. Then every later key got compared against 12 on its way down: the smaller ones $(5, 2, 8)$ drifted into the left subtree and the larger ones $(18, 15, 21)$ into the right subtree.

Now recall quicksort. You pick a *pivot*, then partition the rest of the array into "everything smaller than the pivot" and "everything larger," and recurse on each side.

The root of a BST plays the exact role of the first pivot in quicksort. The first inserted key splits all the others into a smaller group and a larger group, just like a pivot. The root of the left subtree is then the pivot for the smaller group, the root of the right subtree is the pivot for the larger group, and so on, recursively. The set of comparisons quicksort makes is, key for key, the set of comparisons that BST insertion makes.



Focus

Building a BST by inserting keys in some order, and running quicksort using those same keys as pivots in that same order, perform *exactly the same comparisons between the same pairs of keys*. The BST is a recording of quicksort's recursion tree; the in-order walk of the BST replays quicksort's final sorted output.

History and connection

The binary search tree was independently formulated around 1960 by P. F. Windley, A. D. Booth, and A. J. T. Colin to bridge the gap between sorted arrays (which allow $O(\log n)$ binary search but require $\Theta(n)$ shifts on insertion) and linked lists (which allow $O(1)$ insertion once located but require $\Theta(n)$ sequential scans). By structuring nodes as a linked hierarchy preserving left-smaller and right-larger invariants, BSTs aimed to deliver logarithmic operations dynamically. In 1962, Thomas N. Hibbard published the classic deletion algorithm using the in-order successor. Donald Knuth (1971) later developed the dynamic programming formulation for optimal static search trees with known access probabilities.

For years, practitioners assumed that if keys arrived in random order, a BST would perpetually maintain its $O(\log n)$ average depth even as elements were repeatedly inserted and deleted. Empirical simulations contradicted this intuition, and Jeffrey Culberson and J. Ian Munro (1989) later gave a proof: Hibbard deletion is asymmetric. Replacing a deleted node with its in-order successor systematically unbalances subtrees, causing the average path length to drift

away from $O(\log n)$ toward $\Theta(\sqrt{n})$ after long sequences of random updates. Unaugmented BSTs are also vulnerable to natural sorted or nearly sorted input streams, which degenerate tree height to $n-1$, reducing the structure to an ordinary linked list.
Binary search trees made it possible to maintain sorted order dynamically without pre-allocating contiguous blocks. Unlike hash tables, a BST efficiently supports sorted order traversals, minimum/maximum queries, predecessors, successors, and range lookups in time proportional to tree height h. The trade-off is that, without rebalancing, BST worst-case time is $\Theta(n)$. Query cost depends on insertion history, and Hibbard deletion slowly erodes tree balance over time. To guarantee $O(\log n)$ worst-case bounds regardless of input distribution, we must actively enforce tree balance. In **Chapter 13 (Red-Black Trees)**, we introduce node coloring invariants and rotations, bounding tree height strictly by $2\log_2(n+1)$ under arbitrary dynamic updates.

12.6.2 Why we randomize

We saw in the deletion section's trap that inserting keys in sorted order yields a height-$(n-1)$ disaster. The quicksort connection explains *why*: feeding sorted keys to BST insertion is exactly feeding quicksort the worst possible pivots (always the smallest remaining element), the case that makes quicksort take $\Theta(n^2)$ time.

So we do what we did for quicksort: we randomize. A *randomly built binary search tree* on n distinct keys is the tree you get by starting empty and inserting the keys in an order chosen uniformly at random from all $n!$ orderings. We now ask the natural question: how tall should we *expect* such a tree to be?

12.7 The expected height of a random BST

12.7.1 Why depth is just counting ancestors

The height of a tree is the depth of its deepest node, so let us understand the depth of a single node first. The *depth* of a node is the number of edges from the root down to it – equivalently, the number of *ancestors* it has. So if we can count, on average, how many ancestors a key ends up with, we can get a handle on the tree's shape.

The analysis rests on one fact. Sort the keys and rename them $1, 2, \ldots, n$ by rank (so key 1 is the smallest, key n the largest). Consider two keys i and j with $i < j$.

Lemma 12.2 (Ancestor criterion)**.** *In a BST built by insertions, keys i and j end up in an ancestor relationship (one is an ancestor of the other) if and only if, among all the keys with rank between i and j inclusive – that is, $i, i{+}1, \ldots, j$ – the* first one inserted *is i or j itself.*

Intuition

Think about who gets to be the "pivot" separating i and j. Consider the block of keys $i, i{+}1, \ldots, j$. The first of these to be inserted becomes the highest of them in the tree; it is the pivot that all the others in the block fall around. If that first-inserted key is some k strictly

between i and j, then i slides into k's left subtree and j into k's right subtree – they are separated forever and neither can be the other's ancestor. But if the first-inserted key of the block is i or j itself, then no key ever comes between them to split them, and one of them ends up as an ancestor of the other. Only the keys *between* i and j can separate them, because any key outside the block is too small or too large to land between them in any comparison.

12.7.2 Counting with an indicator variable

The block $\{i, i+1, \ldots, j\}$ contains exactly $j - i + 1$ keys. Because the insertion order is uniformly random, every one of these keys is equally likely to be the first of the block to be inserted. Two of them (i and j) are "good." So

$$\Pr[\, i \text{ and } j \text{ are in an ancestor relationship}\,] \;=\; \frac{2}{j-i+1}.$$

Let us turn this into a count of ancestors. Fix a key j and define, for each $i \neq j$, the indicator

$$A_{ij} = \begin{cases} 1 & \text{if key } i \text{ is an ancestor of key } j, \\ 0 & \text{otherwise.} \end{cases}$$

The depth of key j – its number of ancestors – is exactly $\sum_{i \neq j} A_{ij}$. By linearity of expectation, the *expected* depth of key j is $\sum_{i \neq j} \Pr[A_{ij} = 1]$. For any pair, at most one of "i ancestor of j" or "j ancestor of i" can hold, and by symmetry each is equally likely; so the probability that i is specifically the ancestor of j is half of $\frac{2}{|i-j|+1}$, namely $\frac{1}{|i-j|+1}$.

12.7.3 The harmonic numbers appear

Now sum over all $i \neq j$ to get the expected depth of key j. The keys ranked at distance d from j each contribute $\frac{1}{d+1}$, and there are at most two such keys (one of smaller rank, $j - d$, and one of larger rank, $j + d$), with d ranging from 1 up to at most $n - 1$. Splitting the sum into the smaller-ranked ancestors and the larger-ranked ancestors, each half is bounded by a harmonic sum:

$$\mathbb{E}[\text{depth of } j] \;=\; \sum_{i \neq j} \frac{1}{|i-j|+1} \;\leq\; 2 \sum_{d=1}^{n-1} \frac{1}{d+1} \;<\; 2 \sum_{k=1}^{n} \frac{1}{k} \;=\; 2H_n,$$

where H_n is the *n-th harmonic number*,

$$H_n \;=\; \sum_{k=1}^{n} \frac{1}{k} \;=\; 1 + \tfrac{1}{2} + \tfrac{1}{3} + \cdots + \tfrac{1}{n}.$$

Harmonic numbers appear whenever pivots are chosen at random, and H_n is the same quantity that governs quicksort's expected comparison count. And H_n is famously close to a logarithm:

$$\ln n \;<\; H_n \;\leq\; \ln n + 1, \qquad \text{so} \qquad H_n = \Theta(\log n).$$

Focus

Each pairwise probability $\frac{2}{j-i+1}$ is a piece of a harmonic sum, and harmonic sums grow like $\ln n$. That is why a random BST is short: the same $H_n \approx \ln n$ that makes quicksort fast makes the random tree's nodes shallow on average.

So the expected depth of every individual node is $O(\log n)$, and summing over all n nodes, the expected total path length of a random BST is $\Theta(n \log n)$; the *average* node sits at depth $\Theta(\log n)$. But the *height* is the depth of the single deepest node, which could in principle be larger than the average. Controlling it needs one more idea.

Theorem 12.1 (Expected height of a random BST)**.** *The expected height of a randomly built binary search tree on n distinct keys is $O(\log n)$. More precisely, as $n \to \infty$ it is asymptotic to $\alpha \ln n$ with $\alpha \approx 4.311$ (equivalently about $2.99 \log_2 n$), and in particular it is $\Theta(\log n)$.*

The proof in CLRS uses a clever trick: instead of analyzing the height h directly, it analyzes the *exponential height* $Y_n = 2^h$, because expectations of products behave nicely under the random split into left and right subtrees. One sets up a recurrence for $\mathbb{E}[Y_n]$ – the root falls at a uniformly random rank, splitting the keys into a left subtree and a right subtree – solves it to get $\mathbb{E}[Y_n] = O(n^3)$, and then uses Jensen's inequality to pull the conclusion back down to h itself. We will not go through every line here; what matters is the *structure* of the argument and where $\log n$ comes from.

Formal

Lemma 12.3 (The convexity step)**.** *Because $f(x) = 2^x$ is convex, for any random variable h we have $2^{\mathbb{E}[h]} \le \mathbb{E}\big[2^h\big]$ (this is Jensen's inequality). So if we can bound the expected exponential height $\mathbb{E}[2^h]$ by a polynomial cn^3, then $2^{\mathbb{E}[h]} \le cn^3$, and taking logarithms,*

$$\mathbb{E}[h] \;\le\; \log_2(cn^3) \;=\; 3\log_2 n + O(1) \;=\; O(\log n).$$

Why this matters

Theorem 12.1 is good news for practice, with one important caveat. "Expected $O(\log n)$" assumes the insertion order is random. Real data is rarely random – it arrives sorted by timestamp, alphabetized, or clustered – and an adversary who feeds you sorted input gets the $\Theta(n)$ disaster every time. That is why production systems do not bet on randomness; they use *balanced* trees (red-black trees, AVL trees, B-trees) that *guarantee* $O(\log n)$ height no matter the input. The next chapter builds the most famous of these. Think of the random-BST result as the reason balanced trees are worth the trouble: it shows $O(\log n)$ is achievable, and balancing makes it dependable.

Walking question

We proved the random tree is short *on average*. But "on average" hides the spread. Could a random BST occasionally be tall – say height $\Theta(\sqrt{n})$ – with non-negligible probability, even though the average is $\Theta(\log n)$? If you inserted a million random keys, would you ever expect to see a path of length, say, 200? (Hint: the expected height is about $4.31 \ln(10^6) \approx 60$, while the average *depth* is only about $\ln(10^6) \approx 14$. How far above the mean do random heights stray?) What would you have to prove to sleep soundly using an unbalanced BST on random

data?

12.8 Putting it together: a worked lab

Algorithm lab

Work this entire scenario with pencil and paper before reading the answers, then check yourself. Start with an empty BST and insert, in order: 50, 30, 70, 20, 40, 60, 80, 35.

1. Draw the tree. (You should get 50 at the root with two full levels below it, and 35 dangling as the left child of 40.)
2. Write the in-order walk. (Answer: $20, 30, 35, 40, 50, 60, 70, 80$ – check it is sorted.)
3. Find the successor of 40 by the algorithm. (40 has no right child, so climb: 40 is the right child of 30, climb to 30; 30 is the left child of 50, stop – successor is 50.)
4. Find the successor of 50. (50 has a right child, so take the minimum of the right subtree rooted at 70: that is 60.)
5. Delete 30 (a two-child node). Its successor is 35 (minimum of the subtree rooted at 40). Splice 35 into 30's place; 35 inherits 30's children 20 and 40. Redraw and confirm the in-order walk is now $20, 35, 40, 50, 60, 70, 80$.
6. This insertion order gave a short tree. Now imagine inserting the *same eight keys* in sorted order $20, 30, 35, 40, 50, 60, 70, 80$. What height do you get, and why? (A right-leaning chain of height 7 – the worst case.)

Chapter summary

A binary search tree stores keys so that, at every node, the left subtree holds smaller keys and the right subtree holds larger ones. That single invariant makes an in-order walk emit the keys in sorted order, and it makes search, minimum, maximum, successor, predecessor, insertion, and deletion all run by tracing a single root-to-leaf (or leaf-to-root) path – hence all cost $O(h)$, the height. Insertion hangs a new node where a search would have failed; deletion has three cases, the hardest replacing a doomed two-child node with its successor. The shape, and thus the height, depends entirely on insertion order: sorted input degenerates to a height-n chain. Building the tree from a random insertion order is identical to running quicksort with those keys as pivots, and that correspondence – through harmonic numbers and a convexity argument – yields an expected height of $O(\log n)$. Because real input is not random, the next chapter trades expectation for a guarantee with balanced trees.

12.9 Exercises

Exercise 12.1 (Warmup: build and walk)**.** Insert the keys $15, 6, 18, 3, 7, 17, 20, 2, 4, 13, 9$ into an initially empty BST in the given order. Draw the resulting tree and write out its in-order walk.

Confirm that the walk is sorted.

Exercise 12.2 (Search paths)**.** Using your tree from the previous exercise, list the exact sequence of nodes visited by TREE-SEARCH when searching for (a) 13, (b) 9, and (c) 16 (which is absent). How many comparisons does each search make?

Exercise 12.3 (Successor and predecessor by hand)**.** In the same tree, find the successor of 7 and the predecessor of 13 using the algorithms, showing each climb or descent. Verify your answers against the in-order walk.

Exercise 12.4 (Deletion cases)**.** Starting from your tree, delete 6 (a two-child node), then delete 2 (a leaf), then delete 18. Redraw after each deletion and confirm the in-order walk stays sorted throughout.

Exercise 12.5 (Why local checking fails)**.** Give a binary tree with at least five nodes in which every node's left child is smaller and every node's right child is larger than the node itself, yet the tree is *not* a valid BST. Explain precisely which node violates Definition 12.1 and why the local check missed it.

Exercise 12.6 (Counting tree shapes)**.** How many *structurally different* BSTs can hold the three keys $\{1, 2, 3\}$? Draw them all. Then argue that the count for n keys is the n-th Catalan number $C_n = \frac{1}{n+1}\binom{2n}{n}$. (Hint: which key is the root, and how does that split the rest?)

Exercise 12.7 (The pivot correspondence)**.** Take the keys $4, 1, 3, 2, 5$ and insert them in that order into a BST. Separately, run quicksort on the array $[4, 1, 3, 2, 5]$ choosing the *first* element of each subarray as the pivot. Show that the set of pairwise comparisons made by quicksort equals the set of comparisons made by BST insertion, pair for pair.

Exercise 12.8 (Expected depth of the smallest key)**.** Using the ancestor criterion (Lemma 12.2), show that in a randomly built BST the expected number of ancestors of the smallest key (rank 1) is exactly $H_n - 1 = \sum_{k=2}^{n} \frac{1}{k}$. (Hint: key 1 has key j as an ancestor exactly when j is the first inserted among ranks $1, \ldots, j$.) Conclude that the expected depth of the minimum is $\Theta(\log n)$.

Exercise 12.9 (Sorting via a BST)**.** Describe an algorithm that sorts n numbers by inserting them all into a BST and then doing an in-order walk. What is its worst-case running time, and on what kind of input is that worst case achieved? What is its expected running time on a random permutation, and which classic sorting algorithm does this expected cost match – and why do those two recurrences match?

Exercise 12.10 (Challenge: in-order without recursion or parent pointers)**.** The TREE-SUCCESSOR algorithm needs parent pointers. Suppose your nodes have only *left* and *right* (no *parent*). Design an in-order traversal that uses an explicit stack and prints all keys in sorted order in $\Theta(n)$ time and $O(h)$ extra space. Trace it on the seven-key tree from the chapter opening.

Exercise 12.11 (Challenge: tightening the height intuition)**.** We claimed the expected height is about $4.31 \ln n$, yet the average *depth* is about $\ln n$ (roughly H_n). Explain in words why the *maximum* depth (the height) should be a constant factor larger than the *average* depth, and why that constant is finite rather than growing with n. (You do not need the full proof; argue from the fact that very deep paths require an unlucky chain of pivot choices, each independently unlikely.)

Chapter 13

Red-Black Trees

In the previous chapter you met the binary search tree (BST): a simple idea where every key sits to the left of bigger keys and to the right of smaller ones, so searching feels like a game of higher-or-lower. The catch was that a plain BST is only fast *if it stays bushy*. Feed it keys in sorted order and it quietly degenerates into a linked list, and your $\Theta(\log n)$ search becomes $\Theta(n)$.

This chapter solves that problem permanently. A red-black tree augments each node of a binary search tree with a single color bit – red or black – governed by five local invariants that keep the height at most $2\log_2(n+1)$. Every search, insertion, and deletion runs in $\Theta(\log n)$ time in the worst case, regardless of the input order.

It can also feel daunting at first glance. There are cases, sub-cases, and rotations that swing whole subtrees around. We will step through each case with clear pictures, making sure that by the end you do not merely *believe* red-black trees work – you can *see* why they must.

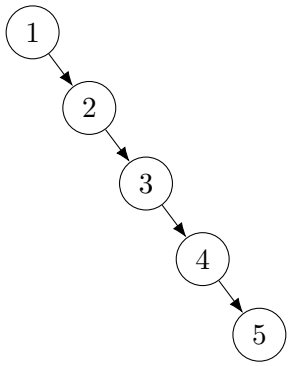
13.1 Why Balance Matters

We start with the problem itself, since the rest of the chapter is about avoiding it.

Intuition

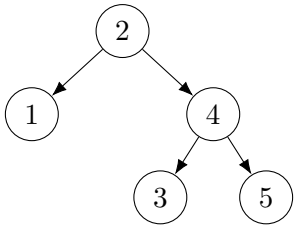
A binary search tree is a sorted structure you walk *down* to find things. Each step of the walk should cut the remaining possibilities roughly in half – that is what makes search logarithmic. But "cut in half" only happens if, at each node, the two subtrees hanging below are about the same size. If one side is always tiny and the other always huge, you barely make progress per step. The speed of a BST depends on this one question: *is it balanced?*

Concretely, suppose you insert the keys $1, 2, 3, 4, 5$ in that order into an ordinary BST. The first key, 1, becomes the root. Then 2 is bigger, so it goes right. Then 3 is bigger than both, so it goes right again. And so on. You get this:



Structurally, such a tree degenerates into a linked list. Searching for 5 visits all five nodes. With a million sorted keys, a single lookup visits all one million nodes, degrading from $\Theta(\log n)$ back to $\Theta(n)$.

Now compare it to the *balanced* arrangement of the same five keys:



Here finding 5 takes two steps. Finding anything takes at most three. The keys are identical; only the *shape* differs, and the shape determines the cost of a search.

Focus

The height of a BST is the length of the longest root-to-leaf path, and the cost of search, insert, and delete is $\Theta(\text{height})$. A balanced tree on n keys has height $\Theta(\log n)$; a degenerate one has height $\Theta(n)$. The entire goal of this chapter is to keep the height $O(\log n)$ *automatically*, after every single insertion and deletion, without ever rebuilding the tree.

Why this matters

Why should you care beyond passing a course? Because balanced search trees are *everywhere* the moment a system needs ordered, dynamic data with hard worst-case guarantees. The C++ `std::map` and `std::set`, Java's `TreeMap`, the Linux kernel's process scheduler (which once used a red-black tree to pick the next task to run), the data structures inside databases that answer range queries, virtual-memory region trackers – all lean on red-black trees or close cousins. The reason engineers reach for them over a hash table is precisely the *ordering*: a hash table can tell you whether 42 is present, but a red-black tree can also tell you the smallest key larger than 42, walk the keys in sorted order, and do it all with a guaranteed worst case rather than a hopeful average.

The naive fix would be to rebalance from scratch whenever the tree gets lopsided, but rebuilding costs $\Theta(n)$ and we cannot afford that on every insert. We need *local* surgery: small, $O(1)$ adjustments

near the point of change that keep the whole tree balanced. That surgery has two ingredients – a set of color rules that *certify* balance, and an operation called a rotation that lets us *restore* the rules cheaply. Both are coming up.

13.2 The Red-Black Properties

A red-black tree is a binary search tree in which every node carries one extra attribute: a color, either red or black. We also adopt one small but important convention that will save us endless case-fiddling later.

Focus

The sentinel. Instead of having children pointers that are sometimes `NIL`, every "missing" child and the parent of the root all point to a single shared dummy node we call *the sentinel*, written $T.nil$. The sentinel is colored *black* and its key is irrelevant. Treating every leaf as this one black node means we never have to ask "but what if this pointer is null?" in the middle of a delicate argument. Every real (internal) node thus has exactly two children, each either a real node or the sentinel.

Now the rules. A binary search tree is a *red-black tree* if it satisfies all five of the following *red-black properties*.

Formal

The five red-black properties.

1. Every node is either red or black.
2. The root is black.
3. Every leaf (the sentinel $T.nil$) is black.
4. If a node is red, then both of its children are black. (No red node has a red child – equivalently, "no two reds in a row" down any path.)
5. For each node, all simple paths from that node down to descendant leaves contain the same number of black nodes.

Properties 1, 2, and 3 are mostly housekeeping. The two that do the real work are 4 and 5, and they deserve some intuition.

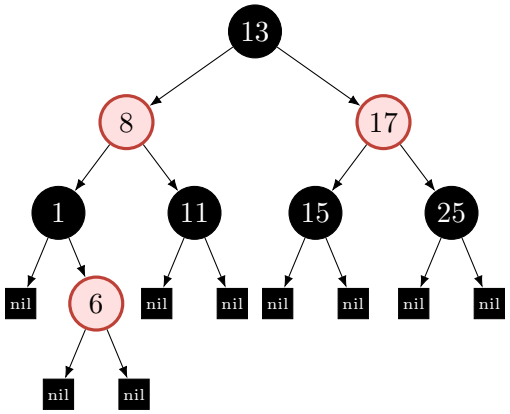
Intuition

Think of black nodes as setting the vertical height budget while red nodes sit singly between them. Property 5 says: *measured in black nodes only, the tree is perfectly balanced* – every path from a node to a leaf crosses exactly the same number of black nodes. That is a very strong balance condition. On its own, though, property 5 would let you pad one path with a long run of red nodes and make it much longer than another. Property 4 forbids that: since no two reds may touch, the reds on any path can at most *double* its length compared to the blacks alone. Put the two together and you get: the longest path is at most twice the shortest, and that factor of two is what yields the logarithmic height bound.

This motivates a piece of vocabulary we will use constantly.

Definition 13.1 (Black-height)**.** The *black-height* of a node x, written $\mathrm{bh}(x)$, is the number of black nodes on any simple path from x down to a leaf, *not counting* x *itself* but counting the leaf sentinel. Property 5 guarantees this number is the same no matter which downward path you take, so it is well-defined.

Here is a small valid red-black tree. Black nodes are drawn as solid dark circles, red nodes as lighter circles with a red border. The little squares are the sentinel leaves.



Walk through the properties on this tree to convince yourself it is legal. The root 13 is black (property 2). No red node has a red child – 8 is red and its children 1 and 11 are both black; 17 is red and its children 15 and 25 are both black; 6 is red and both its children are the black sentinel (property 4). And property 5: pick the root and count black nodes (including sentinels) on each downward path. Down through $8, 1, 6$ to a sentinel you cross blacks $\{1, \text{sentinel}\}$ – wait, let us count carefully, since this is exactly the kind of counting people botch.

Algorithm lab

Lab: count the black-height by hand. Take the root 13. Trace the path $13 \to 8 \to 1 \to 6 \to$ nil. The nodes *below* 13 on this path are 8 (red), 1 (black), 6 (red), and the sentinel (black). Black nodes, not counting 13 itself: 1 and the sentinel, so 2. Now trace $13 \to 17 \to 15 \to$ nil: below 13 we have 17 (red), 15 (black), sentinel (black) – again 2 blacks. Try every downward path from 13 you can find; you will always get 2. So $\mathrm{bh}(13) = 2$. Do the same starting from node 8: every path below it hits exactly 1 black plus its sentinel, giving $\mathrm{bh}(8) = 2$ as well (paths through 1 cross 1 then sentinel; path to 11 crosses 11 then sentinel). Counting the sentinels is what keeps property 5 working out – skip them and you will miscount.

Common trap

Three beginner snares with the properties:

- **Forgetting the sentinel counts.** Property 5 counts the black leaf at the bottom of every path. Many off-by-one errors come from omitting it.
- **Thinking "balanced" means equal subtree sizes.** It does not. Red-black balance

is about *black-height equality*, not node counts. The left subtree can hold more nodes than the right; that is fine.

- **Reading property 4 as "red children must be black."** It is the other direction: a *red parent* forces *black children*. A black node is free to have children of either color.

13.3 The Height Bound

Before we tackle insertion or deletion, let us use the five properties to prove the main height bound: a red-black tree with n internal nodes has height at most $2\log_2(n+1)$. This is the theorem that makes the whole structure worth its trouble, and the proof is a short induction.

Lemma 13.1 (Subtree size from black-height)**.** *The subtree rooted at any node x contains at least $2^{\mathrm{bh}(x)} - 1$ internal nodes.*

Intuition

The claim says: "a node with black-height b is sitting on top of at least a full black tree of height b." A perfectly full binary tree of height b has $2^b - 1$ internal nodes, and red nodes can only *add* to that count, never subtract. So the black-height alone forces a minimum population. The bigger the black-height, the more nodes you are guaranteed to have crammed underneath.

Proof. We induct on the height of x (the ordinary height, longest path down to a leaf).

Base case. If x has height 0, then x is the sentinel itself. Its black-height is 0, and the formula demands at least $2^0 - 1 = 0$ internal nodes. The sentinel's subtree has zero internal nodes. True.

Inductive step. Let x be an internal node of positive height with two children. Each child has height strictly less than x, so the induction hypothesis applies to both. Now, what is each child's black-height? A child is either red or black. If the child is black it contributes 1 to the black count on paths through it, so its own black-height is $\mathrm{bh}(x) - 1$. If the child is red it contributes 0, so its black-height is $\mathrm{bh}(x)$. Either way, each child has black-height *at least* $\mathrm{bh}(x) - 1$. By the induction hypothesis, each child's subtree holds at least $2^{\mathrm{bh}(x)-1} - 1$ internal nodes. Adding the two children's subtrees plus x itself,

$$\left(2^{\mathrm{bh}(x)-1} - 1\right) + \left(2^{\mathrm{bh}(x)-1} - 1\right) + 1 = 2 \cdot 2^{\mathrm{bh}(x)-1} - 1 = 2^{\mathrm{bh}(x)} - 1,$$

which is exactly what we wanted to show. □

Theorem 13.1 (Height of a red-black tree)**.** *A red-black tree with n internal nodes has height at most $2\log_2(n+1)$.*

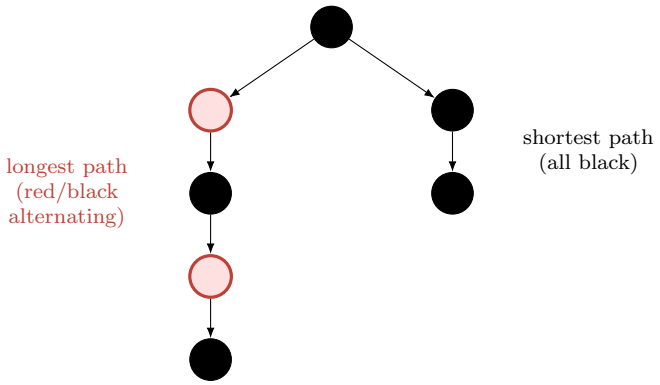
Proof. Let h be the height of the tree, that is, the length of the longest path from the root down to a leaf. Property 4 (no two reds in a row) means that at least half the nodes on that longest path are black: as you walk down, you cannot take two red steps back to back, so reds are at most every other node. Hence the black-height of the root is at least $h/2$:

$$\mathrm{bh}(\mathrm{root}) \geq \frac{h}{2}.$$

By Lemma 13.1 applied at the root, the whole tree has at least $2^{\mathrm{bh(root)}} - 1 \geq 2^{h/2} - 1$ internal nodes. So

$$n \geq 2^{h/2} - 1 \quad \Longrightarrow \quad n + 1 \geq 2^{h/2} \quad \Longrightarrow \quad \log_2(n+1) \geq \frac{h}{2} \quad \Longrightarrow \quad h \leq 2\log_2(n+1). \qquad \square$$

The picture below makes the "black-height pins the height" idea concrete. Two paths leave the same root. The left one is the densest a path can be: black and red strictly alternating, which property 4 permits but cannot exceed. The right one is the sparsest: all black. Both must carry the *same* number of black nodes (property 5), so the alternating path can be at most twice as long.



both paths cross the same number of *black* nodes (property 5);
the alternating path is at most twice as long (property 4).

Focus

Because the height is $O(\log n)$, every operation that runs in time proportional to the height – search, minimum, maximum, successor, predecessor, insert, delete – runs in $O(\log n)$ *worst case*. Searching a red-black tree is identical to searching an ordinary BST (the colors are invisible to a search); it just so happens that the tree you are searching is guaranteed to be short.

Walking question

The bound says the longest path is at most twice the shortest. Could you build a *valid* red-black tree where the longest root-to-leaf path is exactly twice the shortest? What would it have to look like – where do the red nodes go, and where are they forbidden? Try to sketch the smallest such tree. (Hint: alternate red and black down the long path, and keep the short path all black.)

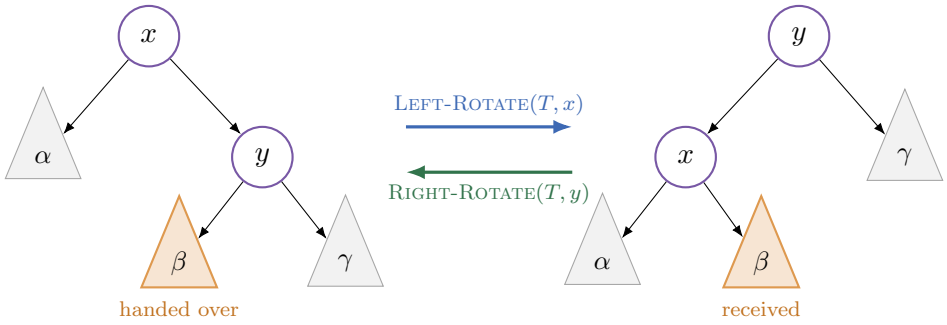
13.4 Rotations

Searching never changes the tree, so it never breaks a property. Insertion and deletion *do* change the tree, and they will temporarily break the rules. To repair them we need a tool that restructures the tree locally while preserving the binary-search-tree ordering. That tool is the *rotation*.

Intuition

A rotation pivots an edge between a parent and a child, swapping which node sits higher in the tree. Importantly, this local pointer change preserves the global binary-search-tree order, because there is exactly one subtree caught in the middle that has to change hands, and it lands in a spot that respects the ordering. A left rotation makes the right child rise; a right rotation makes the left child rise. Each touches only a constant number of pointers, so it costs $O(1)$.

Picture a left rotation around node x, whose right child is y. We want y to rise above x. The subtree β (which sits between x and y in key order) is the one that has to move.



Read the key ordering off both pictures. Before: $\alpha < x < \beta < y < \gamma$. After: $\alpha < x < \beta < y < \gamma$. *Identical.* The rotation slid β from being y's left child to being x's right child, which is exactly where it still belongs, because everything in β is greater than x and less than y in both pictures. The in-order traversal is unchanged; only the shape moved.

A right rotation is the mirror image: it is the inverse operation, taking the right picture back to the left. Here is the code for a left rotation, written against the sentinel convention so we never test for null.

Algorithm 50: LEFT-ROTATE(T, x)

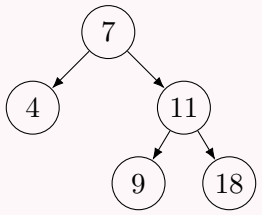
Input: A red-black tree T and a node x whose right child is not the sentinel.
Output: The tree, restructured so x's former right child rises above it.

```
1  y ← x.right;
2  x.right ← y.left ;                          // β becomes x's right subtree
3  if y.left ≠ T.nil then
4  |  y.left.parent ← x;
5  y.parent ← x.parent ;                       // link y to x's old parent
6  if x.parent = T.nil then
7  |  T.root ← y;
8  else if x = x.parent.left then
9  |  x.parent.left ← y;
10 else
11 |  x.parent.right ← y;
12 y.left ← x ;                                // put x on y's left
13 x.parent ← y;
```

RIGHT-ROTATE is obtained by swapping every *left* with *right* in the code above. Notice there are no loops: a fixed handful of pointer writes, so each rotation is firmly $O(1)$.

Algorithm lab

Lab: rotate by hand. Take this little tree (all colors ignored for the moment; we are just practicing the mechanics):



Left-rotate around the root 7. Here $x = 7$, $y = 11$, and the subtree β is the node 9 (it lies between 7 and 11). After the rotation, 11 becomes the root, 7 becomes its left child, 9 moves to be the *right* child of 7, and 18 stays as 11's right child. Draw it and read the in-order traversal of both: you should get $4, 7, 9, 11, 18$ both times. If your before-and-after in-orders disagree, you moved the wrong subtree.

Common trap

The single most common rotation bug is forgetting to reattach *three* parent pointers: the moved subtree's parent (β now points up to x), y's parent (now x's old parent), and x's parent (now y). Miss any one and the tree's downward pointers will look fine while the upward ones lie – and the fixup code that walks *up* the tree will wander off into the weeds.

13.5 Insertion

We now have everything we need. Insertion happens in two acts. In Act One we drop the new key in exactly where an ordinary BST would put it, and we paint it *red*. In Act Two we walk back up the tree fixing any property the red paint job may have violated.

Intuition

Why color the new node red rather than black? Because red is the "cheap" color: a red node does not change anybody's black-height. If we inserted a black node, we would instantly break property 5 along that one path (it now has one more black than its siblings) and the repair would be ugly. By inserting red, the only property we can possibly violate is property 4 (a red node with a red parent) – and property 2 if the new node happens to be the root. Those are far gentler to fix. So we always start red and only darken nodes as the fixup demands.

The first act is just BST insertion with the sentinel bookkeeping; here it is.

Algorithm 51: RB-Insert(T, z)

Input: Tree T and a fresh node z holding the key to insert.
Output: T with z inserted and all five properties restored.

```
1  y ← T.nil;   x ← T.root;
2  while x ≠ T.nil do
3  |   y ← x;
4  |   if z.key < x.key then
5  |   |   x ← x.left;
6  |   else
7  |   |   x ← x.right;
8  z.parent ← y;
9  if y = T.nil then
10 |   T.root ← z;
11 else if z.key < y.key then
12 |   y.left ← z;
13 else
14 |   y.right ← z;
15 z.left ← T.nil;   z.right ← T.nil;   z.color ← RED;
16 RB-Insert-Fixup(T, z);
```

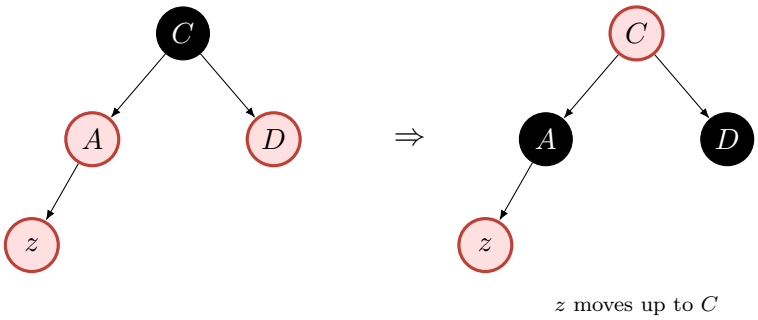
Everything interesting is in the fixup, which involves a few nodes near z: its parent, and its parent's sibling, which we call the *uncle*. The case analysis depends on the uncle's color.

Focus

After Act One, the only thing that can be wrong is a *red-red* violation: z is red, and its parent might also be red. (If the parent is black we are already done; if z is the root we just paint it black.) So the fixup loop's job is: "as long as z has a red parent, push the violation upward or rotate it away." Because each pass either moves the problem two levels closer to the root or ends with a rotation, the loop runs $O(\log n)$ times.

There are three cases, and they come in a left-leaning flavor and a mirror-image right-leaning flavor. We describe the version where z's parent is a *left* child of the grandparent; the other side is symmetric (swap left and right throughout). Let $z.p$ be the parent, $z.p.p$ the grandparent, and y the uncle.

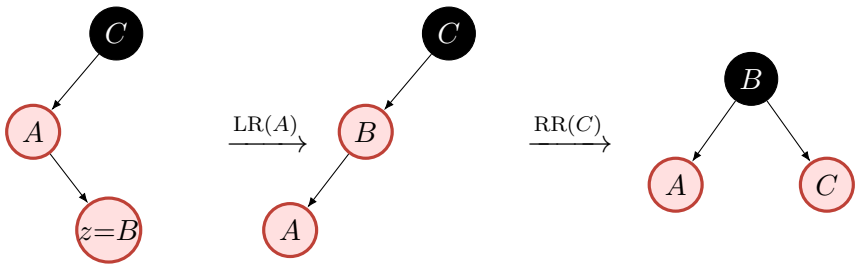
Case 1: the uncle y is red. Both the parent and the uncle are red, sitting under a (necessarily black) grandparent. The fix is pure recoloring, no rotation: paint the parent and the uncle black, paint the grandparent red, then move our attention up to the grandparent and continue. We have "pushed" the red-red conflict up two levels.



Why does this preserve property 5? The grandparent went from black to red and its two children went from red to black. Any path passing through this little cluster used to cross the black grandparent and then one of the now-black children – the total black count along every such path is unchanged, because we removed one black (the grandparent) and added one back (a child). The black-height is conserved exactly. We may have created a new red-red conflict higher up (if the grandparent's own parent is red), which is why we loop with z now pointing at the grandparent.

Case 2: the uncle y is black, and z is a "zig-zag" (an inner grandchild). That is, z is the right child of a left-child parent. A single rotation will not straighten this; we first do a *left rotation on the parent* to turn the zig-zag into a straight line, converting it into Case 3. After this rotation we relabel so that z points at what is now the lower node.

Case 3: the uncle y is black, and z is a straight line (an outer grandchild). Now z is the left child of a left-child parent – parent and grandparent and z all lean the same way. Recolor the parent black and the grandparent red, then *right-rotate the grandparent*. This lifts the parent into the grandparent's old slot, with the (formerly red, now black) parent on top and the grandparent and z as its red children – no two reds touch, and the loop terminates.



Here is the whole fixup, both sides.

Algorithm 52: RB-INSERT-FIXUP(T, z)

Input: Tree T where the only possible violation is a red-red edge at z.
Output: T with all five properties restored.

```
1  while z.parent.color = RED do
2      if z.parent = z.parent.parent.left then
3          y ← z.parent.parent.right ;                          // the uncle
4          if y.color = RED then                                  // Case 1
5              z.parent.color ← BLACK;
6              y.color ← BLACK;
7              z.parent.parent.color ← RED;
8              z ← z.parent.parent;
9          else
10             if z = z.parent.right then                         // Case 2
11                 z ← z.parent;
12                 LEFT-ROTATE(T, z);
13             z.parent.color ← BLACK ;                           // Case 3
14             z.parent.parent.color ← RED;
15             RIGHT-ROTATE(T, z.parent.parent);
16     else
17         same as above with LEFT and RIGHT exchanged;
18 T.root.color ← BLACK;
```

Focus

The very last line – forcibly painting the root black – quietly handles two situations at once: the new node was inserted into an empty tree and became the root, or Case 1 propagated a red all the way up to the root. Painting the root black can never violate property 5 (it adds one black to *every* path equally) and it restores property 2.

13.5.1 A fully worked insertion

We will build a red-black tree from empty by inserting $10, 20, 30, 15, 25, 5, 35$ and watch the fixups fire, narrating the interesting moments.

1. **Insert** 10**.** Empty tree, so 10 becomes the root, inserted red, then the final line paints it black. Tree: black 10.

2. **Insert** 20**.** Goes right of 10, red. Its parent 10 is black, no violation. Tree: black 10 with red right child 20.

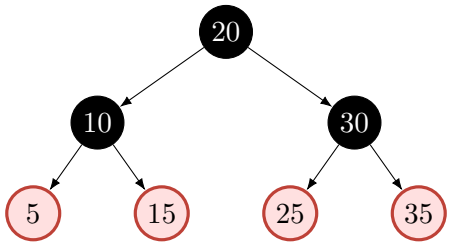
3. **Insert** 30**.** Goes right of 20, red. Now 20 (red) has a red child 30: a red-red violation, and 20's uncle is the sentinel (black). $z = 30$ is an outer (right-right) grandchild, so this is Case 3 on the right side: recolor 20 black, 10 red, and left-rotate the root 10. Now 20 is the black root with red children 10 and 30. Balanced and legal.

4. **Insert** 15**.** From 20 go left to 10, then right of 10, red. Parent 10 is red – violation. The uncle here is 30, which is *red*. Case 1: paint 10 and 30 black, paint 20 red, move z up to 20. But 20

is the root, and its parent (the sentinel) is black, so the loop stops. The final line paints the root 20 black again. Tree now: black 20; black 10 (with red right child 15); black 30.

5. **Insert 25.** From 20 right to 30, then left of 30, red. Parent 30 is black – no violation. Easy.
6. **Insert 5.** From 20 left to 10, left of 10, red. Parent 10 is black – no violation.
7. **Insert 35.** From 20 right to 30, right of 30, red. Parent 30 is black – no violation.

The final tree:



Notice how the sorted-order insertion that destroyed a plain BST in Section 13.1 would, here, have been rebalanced at every step.

Cost summary

Insertion cost. Act One (the BST descent) is $O(\log n)$ because the tree's height is $O(\log n)$. The fixup loop runs $O(\log n)$ iterations in the worst case (Case 1 climbs two levels per iteration), and it performs *at most two rotations* total – Cases 2 and 3 each end the loop. So insertion is $O(\log n)$ time with $O(1)$ rotations. The low rotation count is what makes red-black trees attractive in practice: restructuring is cheap.

13.6 Deletion

Deletion is harder. Searching never breaks anything and insertion can only create a red-red violation; but deletion can *remove* a black node, and removing a black node punches a hole in property 5 – suddenly one path has fewer black nodes than its peers. Repairing that hole is the delicate part.

Intuition

Removing a *red* node is harmless: red nodes do not count toward black-height, so plucking one out leaves every path's black count unchanged, and no red-red pair can be newly created in the spot it vacated. The trouble is removing a *black* node. Imagine the missing black node leaves behind a "debt" of one black on its path. We model this with a colorful fiction: the node that moves into the gap is said to carry an *extra black* – it becomes "doubly black" – and the entire fixup is the process of *discharging that extra black*, either by shoving it onto a red node nearby (which can simply absorb it by turning black) or by sliding it upward until it reaches the root or a red node that can swallow it.

As with ordinary BST deletion, the node we physically remove is not always the node holding the key we asked to delete. If the target has two children, we splice out its *successor* (the smallest key

in its right subtree, which has at most one child) and move that key up. So we track two things: the node y actually spliced out, and the node x that moves into y's position. The rule is: *the fixup is needed exactly when the spliced-out node y was black.*

We will lean on a helper, RB-Transplant, that replaces one subtree with another by rewiring the parent's pointer – the same plumbing used in plain BST deletion, adjusted for the sentinel.

Algorithm 53: RB-Transplant(T, u, v)

Input: Replace the subtree rooted at u with the subtree rooted at v.

```
1  if u.parent = T.nil then
2  |  T.root ← v;
3  else if u = u.parent.left then
4  |  u.parent.left ← v;
5  else
6  |  u.parent.right ← v;
7  v.parent ← u.parent;
```

Algorithm 54: RB-Delete(T, z)

Input: Tree T and the node z to delete.
Output: T with z removed and all five properties restored.

```
1  y ← z;   y-orig-color ← y.color;
2  if z.left = T.nil then
3  |  x ← z.right;   RB-Transplant(T, z, z.right);
4  else if z.right = T.nil then
5  |  x ← z.left;   RB-Transplant(T, z, z.left);
6  else
7  |  y ← Tree-Minimum(z.right) ;                          // the successor
8  |  y-orig-color ← y.color;
9  |  x ← y.right;
10 |  if y.parent = z then
11 |  |  x.parent ← y;
12 |  else
13 |  |  RB-Transplant(T, y, y.right);
14 |  |  y.right ← z.right;   y.right.parent ← y;
15 |  RB-Transplant(T, z, y);
16 |  y.left ← z.left;   y.left.parent ← y;   y.color ← z.color;
17 if y-orig-color = BLACK then
18 |  RB-Delete-Fixup(T, x);
```

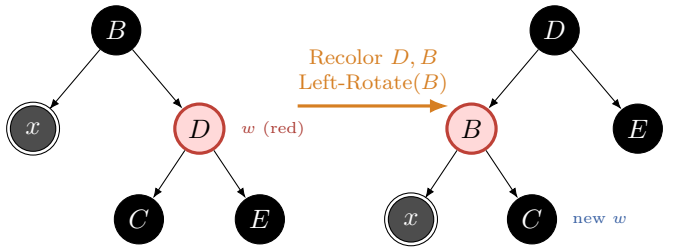
Focus

The key bookkeeping: y is the node that physically leaves the tree's structure (either z itself, or z's successor when z has two children). x is the node that moves into y's old spot. We remember y's *original* color. If it was red, no property broke and we are done. If it was black, x inherits an *extra black* and we call the fixup to discharge it.

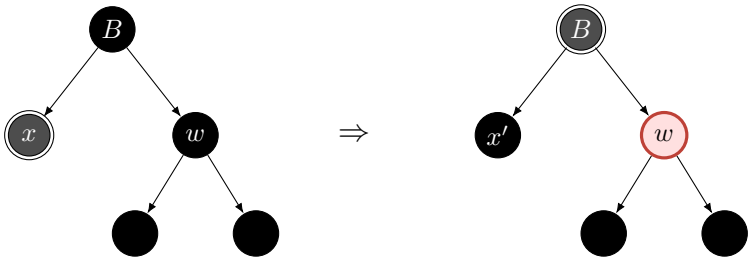
Now the fixup. We describe the case where x is a *left* child; the right-child cases are mirror images.

Let w be x's sibling. Because x carries an extra black (its path is one black short), w's subtree must be "black-heavy" relative to x's – and that asymmetry is what we exploit. There are four cases.

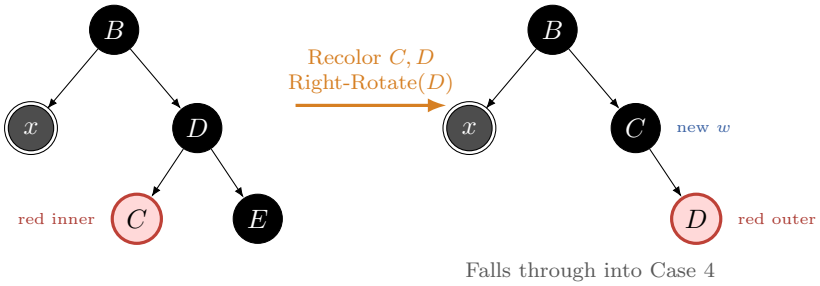
Case 1: the sibling w is red. We cannot reason cleanly while w is red, so we first convert it away. Recolor w black and x's parent red, then left-rotate the parent. Now x has a new, black sibling, and we have reduced to one of the next three cases.



Case 2: w is black and both of w's children are black. Here neither side of w can absorb the extra black by rotation, so we "move the debt up." Recolor w red – this lightens w's side by one black, rebalancing w's subtree against x's – and pass the extra black up to the parent by setting $x \leftarrow x.parent$. The loop continues from there. If we ever reach a red node (or the root), it absorbs the extra black and we stop.

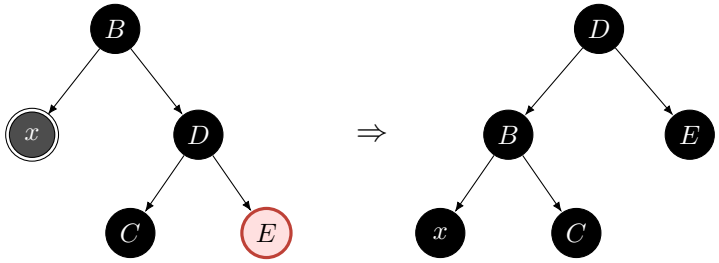


Case 3: w is black, w's left child is red, and w's right child is black. This is the "wrong-side red" situation; we rotate it into Case 4. Recolor w's left child black and w red, then right-rotate w. Now x's new sibling is black with a red *right* child, which is Case 4.



Case 4: w is black and w's right child is red. This is the terminal case that actually discharges the debt. Color w with the parent's color, color the parent black, color w's red right child black, and

left-rotate the parent. This single rotation restores the missing black on x's path without disturbing any other path's count, so we set $x \leftarrow T.root$ to end the loop.



The doubly-black x on the left has shed its extra black on the right; the red node E paid the debt by turning black, and the rotation rebalanced the black-heights.

Algorithm 55: RB-DELETE-FIXUP(T, x)

Input: Tree T where x carries an extra black.
Output: T with the extra black discharged and all properties restored.

```
1  while x ≠ T.root and x.color = BLACK do
2      if x = x.parent.left then
3          w ← x.parent.right;
4          if w.color = RED then                                   // Case 1
5              w.color ← BLACK;   x.parent.color ← RED;
6              LEFT-ROTATE(T, x.parent);   w ← x.parent.right;
7          if w.left.color = BLACK and w.right.color = BLACK then  // Case 2
8              w.color ← RED;   x ← x.parent;
9          else
10             if w.right.color = BLACK then                       // Case 3
11                 w.left.color ← BLACK;   w.color ← RED;
12                 RIGHT-ROTATE(T, w);   w ← x.parent.right;
13             w.color ← x.parent.color ;                          // Case 4
14             x.parent.color ← BLACK;   w.right.color ← BLACK;
15             LEFT-ROTATE(T, x.parent);   x ← T.root;
16     else
17         same as above with LEFT and RIGHT exchanged;
18 x.color ← BLACK;
```

Focus

A map of the four cases, so they stop feeling arbitrary:

- **Case 1** is a *setup* move: it gets rid of a red sibling so the others can reason about a black one. It always falls through into 2, 3, or 4.
- **Case 2** is the only case that *loops*. It cannot fix the debt locally (both nephews are black), so it pushes the extra black up one level and tries again. Every other case terminates.
- **Case 3** is a *setup* move for Case 4: it rotates a red nephew to the far side.

- **Case 4** is the *terminating* fix: a red nephew on the far side lets one rotation pay off the debt for good.

Only Case 2 repeats, and it climbs one level each time, so the loop is $O(\log n)$. The total number of rotations is at most *three* (Case 1, then 3, then 4 in the worst chain).

Common trap

The notorious confusion in deletion is the "extra black" fiction. The node x does not literally hold two colors; the doubling is a way of saying "this path is one black short, and we are pinning the deficit to x until we find a place to pay it." If you try to picture a node that is somehow black-and-a-half, you will tie yourself in knots. Picture instead a debt token sitting on x, which the fixup either hands to a red node (paid) or carries one step up the tree.

Walking question

On your next walk, reason about *why Case 2 is the only one that loops.* Cases 1, 3, and 4 each either end the loop or set up a strictly later case, but Case 2 fires when both of the sibling's children are black – the one situation where there is no red node anywhere nearby to absorb the debt. With nothing to pay the debt locally, the only move is to carry it up to the parent and try the whole analysis again one level higher. Convince yourself that the debt can climb at most to the root, where the final line of the algorithm simply forgives it.

Cost summary

Deletion cost. Finding z and its successor is $O(\log n)$. The fixup loop iterates only via Case 2, which ascends one level per iteration, so at most $O(\log n)$ iterations, with at most three rotations total. Hence deletion is $O(\log n)$ time, $O(1)$ rotations – matching insertion.

13.7 Putting It All Together

Here is a summary of what that single color bit per node guarantees:

Cost summary

Red-black tree operations, all worst case.

Operation	Time	Rotations
Search / Minimum / Maximum	$O(\log n)$	0
Successor / Predecessor	$O(\log n)$	0
Insert	$O(\log n)$	≤ 2
Delete	$O(\log n)$	≤ 3

The height is at most $2\log_2(n+1)$, so nothing is ever worse than logarithmic – no matter the order of operations.

Why this matters

Red-black trees are the standard balanced BST of modern systems libraries, but they are not the only option. AVL trees keep an even tighter height bound (about $1.44 \log_2 n$) and so search a touch faster, at the cost of doing more rotations on update – a good trade when you read far more than you write. B-trees (Chapter 18) generalize the idea to fat nodes that hold many keys, which is exactly what you want when each node access is a slow disk read. Skip lists reach the same guarantees with randomization and simpler code. The red-black tree sits at a sweet spot: a strict worst-case guarantee, few rotations per update, and a structure that maps neatly onto a single color bit you can often hide inside an already-present pointer.

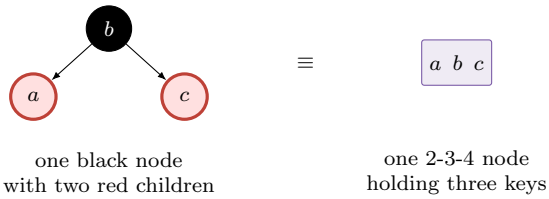
History and connection

Self-balancing binary search trees began with Georgy Adelson-Velsky and Evgenii Landis (1962, AVL trees), which enforced rigid balance ($|h_{\text{left}} - h_{\text{right}}| \leq 1$). In 1972, Rudolf Bayer introduced "symmetric binary B-trees" to simulate multi-key B-trees within binary pointer nodes. Leonidas J. Guibas and Robert Sedgewick (1978) recognized that Bayer's structure could be reframed using a single color bit (red or black), replacing cumbersome multi-node representations with clean local rotation rules. Red-black trees provide a 1-to-1 binary isomorphism for 2-3-4 trees (B-trees of order 4): a black node together with its red children models a single fat B-tree node.

A long debate centered on whether strict height balance (AVL: height $\leq 1.44 \log_2 n$) was superior to relaxed balance (Red-Black: height $\leq 2 \log_2(n+1)$). While AVL trees offer slightly faster read lookups due to shallower depth, their deletion routine can trigger up to $O(\log n)$ rotations rippling all the way to the root. Red-black trees bound this: insertion requires at most 2 rotations, and deletion requires at most 3 rotations, bounding the structural pointer adjustments to $O(1)$ worst-case per update.

Red-black trees provide $O(\log n)$ worst-case dictionaries, even under adversarial update sequences. They are used widely in production systems: the Linux kernel Completely Fair Scheduler (CFS), virtual memory range tracking (`vm_area_struct`), high-performance memory allocators, and standard template libraries (C++ `std::map`, Java `TreeMap`). The cost is high per-node pointer overhead (left, right, parent, and color metadata consumes 32 bytes per node on 64-bit architectures before payload). Traversal is dominated by cache misses as nodes are scattered arbitrarily across the heap. Having mastered balance, we can now make tree nodes carry additional metadata. In **Chapter 14 (Augmenting Data Structures)**, we prove that red-black trees can be systematically augmented with summary statistics – such as subtree sizes for dynamic order selection or interval endpoints for range overlap queries – without increasing the $O(\log n)$ cost of rotations.

The correspondence is worth seeing once. A black node with one or two red children collapses into a single 2-3-4 node holding two or three keys; a lone black node is a 2-node holding one key. Property 5 (equal black-height) is just the statement that a B-tree keeps all its leaves on the same level, and property 4 (no two reds in a row) is what stops a B-tree node from overflowing past three keys.



Focus

What to hold onto from this chapter.

1. BST operations cost $\Theta(\text{height})$, so a tree that stays $O(\log n)$ tall stays fast.
2. Five properties enforce balance, and the two that matter are "no two reds in a row" (property 4) and "equal black-height on all paths" (property 5). Together they force the longest path to be at most twice the shortest.
3. Rotations are $O(1)$ local restructurings that preserve search order; they are the only tool we have for changing shape.
4. Insert red, then fix up the at-most-one red-red violation with three cases and ≤ 2 rotations.
5. Delete, and if a black node left, discharge the "extra black" with four cases and ≤ 3 rotations.

Red-black trees give us guaranteed logarithmic searches, insertions, and deletions by maintaining a strict balance invariant through local rotations. But what if we want a balanced tree to do more than store keys? For example, can we determine the k-th smallest key or count how many keys lie in a range $[a, b]$ in $O(\log n)$ time, without sacrificing the logarithmic update guarantee? In Chapter 14, we learn how to **augment** red-black trees by attaching summary fields to each node and updating them during rotations at zero extra asymptotic cost.

Exercises

Exercise 13.1 (Warmup: legal or not)**.** For each of the following, decide whether it can be a valid red-black tree and justify with the specific property that fails if not. (a) A tree whose root is red. (b) A black root with two red children, each of which has two black children. (c) A path from the root that reads black, red, red, black going downward.

Exercise 13.2 (Black-height counting)**.** In the worked example tree at the end of Section 13.5.1 (root 20), compute the black-height $\text{bh}(x)$ for every node x, remembering to count the sentinel leaves. Verify property 5 holds at the root by listing the black count of three distinct root-to-leaf paths.

Exercise 13.3 (Rotations preserve order)**.** Take the tree from the rotation lab in Section 13.4. Perform a left rotation around the root, then a right rotation around the new root. You should recover the original tree. Write out the in-order traversal after each step to confirm the key order never changes, and explain in one sentence why RIGHT-ROTATE is the exact inverse of LEFT-ROTATE.

Exercise 13.4 (Insert and trace)**.** Starting from an empty tree, insert the keys $41, 38, 31, 12, 19, 8$ one at a time. For each insertion, name which case of RB-INSERT-FIXUP fires (if any) and draw the resulting tree with colors. (Several of these will trigger rotations – this sequence is designed to.)

Exercise 13.5 (Why red, not black?)**.** Suppose you modified RB-Insert to color the new node *black* instead of red. Which of the five properties would be violated immediately upon insertion into a non-empty tree, and why would the repair be more involved than in the red case? Argue in terms of black-height.

Exercise 13.6 (Maximum and minimum reds)**.** Among all valid red-black trees on $n = 7$ internal nodes, what is the largest possible number of red nodes? The smallest? Sketch a tree achieving each extreme. (Hint: think about property 4 for the maximum and property 5 for the minimum.)

Exercise 13.7 (Delete and trace)**.** Build the red-black tree by inserting $5, 10, 15, 20, 25, 30, 35$ (this gives a nicely balanced tree). Then delete 10. Identify the spliced-out node y, its original color, the node x that moves up, and which deletion-fixup case (if any) runs. Draw the final tree.

Exercise 13.8 (The extra-black invariant)**.** Explain, in your own words and in at most six sentences, the loop invariant maintained by RB-Delete-Fixup: what is true about the tree at the top of each iteration, in what sense is property 5 "violated only at x," and why does terminating the loop (either reaching a red x or the root) restore all five properties when we finally paint x black?

Exercise 13.9 (AVL versus red-black, conceptually)**.** An AVL tree guarantees height at most about $1.44 \log_2 n$, tighter than the red-black bound of $2 \log_2(n + 1)$. Without looking anything up, argue from the balance conditions why an AVL tree should generally perform *more* rotations per insertion/deletion than a red-black tree, and describe one workload where each structure would be the better choice.

Exercise 13.10 (Challenge: join two trees)**.** Suppose you have two red-black trees T_1 and T_2 together with a key k such that every key in T_1 is less than k and every key in T_2 is greater than k. Describe an $O(\log n)$ algorithm to combine them into a single red-black tree containing all the keys of T_1, all of T_2, and k. (Hint: the black-heights of the two trees are the lever. Walk down the taller tree's appropriate spine until you reach a node whose black-height matches the shorter tree, then splice in k as a red node joining the two and run a single insert-style fixup.)

Chapter 14

Augmenting Data Structures

Over the last few chapters you have built the red-black tree. It gives you `INSERT`, `DELETE`, `SEARCH`, `MINIMUM`, `MAXIMUM`, `SUCCESSOR`, and `PREDECESSOR`, every one of them in $O(\lg n)$ time, even in the worst case, because the tree stays balanced. That is a versatile set of operations at $O(\lg n)$ cost per query.

Real applications, however, rarely ask the exact question a standard textbook data structure answers out of the box. A bank wants to know, "how many of my customers have a balance below this one?" A scheduler wants to know, "does any meeting on the calendar overlap the slot 2:00–3:00?" A median-tracking dashboard wants to know, "what is the 90th-percentile latency *right now*, as requests stream in?" None of these is plain `SEARCH`.

You have two options. You could invent a brand-new data structure from the ground up for each new question — exhausting, error-prone, and wasteful. Or you could take a structure you already trust and *teach it a new trick* by storing a little extra information at each node. This chapter is about the second approach, which is called *augmenting* a data structure.

Intuition

Augmenting a data structure is like adding a mileage counter to a car you already own. You do not rebuild the engine. You bolt on one small gauge, wire it to something the car already does (turning the wheels), and make sure it stays accurate as you drive. The hard part is keeping it correct every time the car moves.

We will work through two flagship examples — *order-statistic trees*, which answer ranking and percentile questions, and *interval trees*, which answer overlap questions — and between them we extract the general recipe: a short theorem that tells you exactly when augmentation is cheap and how to do it without breaking what you already had.

14.1 Warming Up: What Question Are We Even Asking?

Suppose you have n distinct numbers and you store them in a balanced binary search tree. The tree already answers "is x present?" and "what is the smallest element?" Now imagine someone hands you these two new requests:

- `SELECT`(i)**:** return the element with the ith smallest key. If $i = 1$ you want the minimum; if $i = n$ you want the maximum; if $i = \lceil n/2 \rceil$ you want the median.

- `RANK`(x)**:** given a node x in the tree, return its position in sorted order. If x holds the smallest key, its rank is 1.

These two are inverses of each other. `SELECT` goes from a *position* to an *element*; `RANK` goes from an *element* to its *position*. Together they are called *order-statistic* queries, because the ith smallest element is also called the ith *order statistic* of the set. (You met order statistics back in Chapter 9; there we found a single one in linear time on a static array. Here we want a *dynamic* set that supports insert and delete *and* answers order-statistic queries fast.)

Focus

Hold onto this distinction: `SELECT`(i) = "give me the ith smallest" (position $\to$ element), `RANK`(x) = "where does x sit in sorted order?" (element $\to$ position). Everything in the first half of this chapter is about answering both in $O(\lg n)$ time on a set that keeps changing.

The crude attempt: walk the tree in sorted order (an in-order traversal), counting as you go, and stop at the ith node. That works, but it costs $\Theta(n)$ in the worst case — you might walk the whole tree. We did all that work to get $O(\lg n)$ operations, and now a single query throws it away. We can do much better by storing one extra number per node.

14.2 Order-Statistic Trees: The Idea

Take an ordinary red-black tree. At every node x, add one new field, x`.size`, defined as the number of nodes in the subtree rooted at x, *including x itself*. To make the bookkeeping uniform, we let the sentinel `NIL` (the stand-in leaf at the bottom) have size 0. Then for every real node the field obeys a clean local rule:

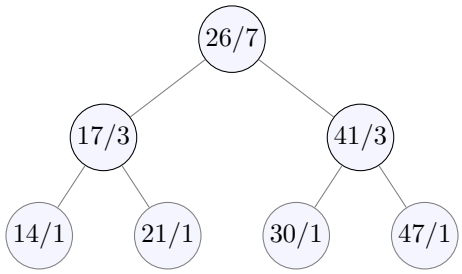
$$x.size \;=\; x.left.size \;+\; x.right.size \;+\; 1.$$

In words: the size of your subtree is the size of your left subtree, plus the size of your right subtree, plus one for yourself.

Intuition

Think of `size` as each node knowing the headcount of the family living beneath it (itself included). The root knows the whole population, n. A leaf knows just itself, 1. Importantly, a node can compute its own headcount knowing *only* the headcounts of its two children — it never has to look further down. That "I only need my kids" property is what will make updates cheap.

Here is a small tree of seven keys, with each node labelled *key / size*. Spend a moment checking the rule at every node.



Check the root: its left subtree (rooted at 17) has size 3, its right subtree (rooted at 41) has size 3, plus itself makes $3+3+1=7$. Good. Check node 17: $1+1+1=3$. Good. Every leaf has two `NIL` children of size 0, so $0+0+1=1$. The rule holds everywhere.

14.2.1 `SELECT`: finding the ith smallest

Now watch how that one number lets you home in on the ith smallest element without ever scanning. Stand at a node x. Let

$$r \;=\; x.left.size + 1.$$

This r is the rank of x *within the subtree rooted at* x: everything in x's left subtree is smaller than x, and there are $x.left.size$ of them, so x itself is the rth smallest in its own subtree. Three cases:

- If $i = r$, then x *is* the element you want. Done.
- If $i < r$, the element you want is smaller than x, so it lives in the left subtree. Recurse left, still looking for the ith smallest (the count is unchanged — everything to the left is still to the left).
- If $i > r$, the element is bigger than x. Recurse right. But now you must *subtract* the r elements you have skipped over (everything in the left subtree, plus x itself): in the right subtree you are now hunting for the $(i-r)$th smallest.

Algorithm 56: OS-Select(x, i) — the ith smallest in the subtree at x

Input: a subtree root x, and an index i with $1 \le i \le x.size$
Output: the node holding the ith smallest key in x's subtree

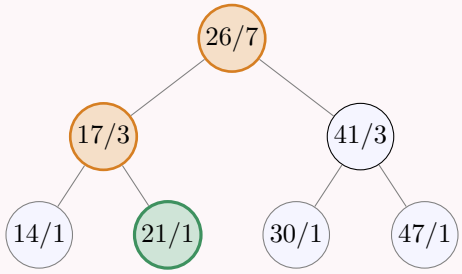
```
r ← x.left.size + 1;
if i = r then
    return x;
if i < r then
    return OS-Select(x.left, i);
else
    return OS-Select(x.right, i − r);
```

To answer a query on the whole tree, you call OS-Select$(root, i)$.

Algorithm lab

Let us find the **3rd smallest** in the seven-node tree above, by hand. Call OS-Select$(root = 26,\ i = 3)$.

Step 1. At node 26: left subtree (rooted at 17) has size 3, so $r = 3 + 1 = 4$. We want $i = 3$. Since $3 < 4$, the answer is to the left. Recurse into node 17 with $i = 3$.
Step 2. At node 17: left subtree (node 14) has size 1, so $r = 1 + 1 = 2$. We want $i = 3$. Since $3 > 2$, go right and subtract: recurse into node 21 with $i = 3 - 2 = 1$.
Step 3. At node 21: left subtree is `NIL`, size 0, so $r = 0 + 1 = 1$. We want $i = 1$. Since $1 = 1$, **return node** 21.
Sanity check: the sorted keys are $14, 17, 21, 26, 30, 41, 47$. The 3rd smallest is indeed 21. We touched exactly 3 nodes — the height of the tree — not all seven.



The shaded path is the only part of the tree OS-Select ever looks at: 26 (steer left), then 17 (steer right), then 21 (the green answer). The four unshaded nodes are never visited.

Common trap

The single most common bug is forgetting to subtract when you go right. If you recurse right still asking for the ith smallest, you are now asking for the ith smallest *of the right subtree*, which is a completely different element. You must shrink the target to $i - r$, because by stepping right you have left r smaller elements (the whole left subtree, plus x) behind you. Trace the example again and watch where the $3 \to 1$ happens.

Because the tree is balanced, its height is $O(\lg n)$, and OS-Select descends one level per call, so a query costs $O(\lg n)$.

14.2.2 `RANK`: where does this node sit?

Now the inverse. Given a pointer to a node x, what is its position in the sorted order of the *whole* tree? Start with the rank of x inside its own subtree, namely $x.left.size + 1$, and then climb toward the root. Each time you step up, ask: did I just come up from a right child? If so, everything in that sibling-and-parent's left side, plus the parent itself, is smaller than x and has not yet been counted — so add it in. If you came up from a left child, you are still on the small side of that parent, so add nothing.

Algorithm 57: OS-RANK(T, x) — the position of node x in sorted order

Input: an order-statistic tree T and a node x in it
Output: the rank of x among all keys of T (so 1 means smallest)

```
1 r ← x.left.size + 1;
2 y ← x;
3 while y ≠ T.root do
4     if y = y.parent.right then
5         r ← r + y.parent.left.size + 1;
6     y ← y.parent;
7 return r;
```

Algorithm lab

Let us compute OS-RANK of node 21 in the same tree, and confirm we get 3 back (the inverse of the previous lab).
Start. $x = 21$. Its left child is `NIL` (size 0), so $r = 0 + 1 = 1$. Set $y = 21$.
Climb 1. $y = 21$ is the *right* child of its parent 17. So add the parent's left subtree (node 14, size 1) plus 1 for the parent: $r \leftarrow 1 + 1 + 1 = 3$. Move up: $y = 17$.
Climb 2. $y = 17$ is the *left* child of its parent 26. Left child means add nothing. Move up: $y = 26 = root$, loop ends.
Return $r = 3$. It matches: 21 is the 3rd smallest. `SELECT` and `RANK` really are inverses.

Figure 14.1 visualizes this upward climb: starting at node 21, each step up from a right child sweeps the skipped left subtree and parent into the accumulator, leaving unrelated subtrees untouched.

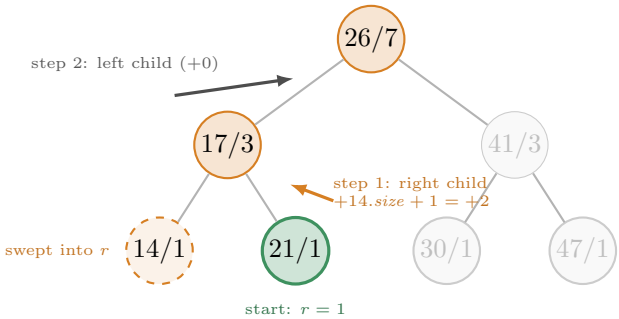


Figure 14.1: The upward climb of OS-RANK for key 21. Starting with local rank $r = 1$ (its left child is empty), stepping up to parent 17 from the right child adds the parent and its left subtree $(1 + 1 = 2)$, updating r to 3. Stepping up to root 26 from the left child adds nothing. Total work is $O(h) = O(\log n)$.

The `while` loop runs once per level on the path from x to the root, so OS-RANK is $O(\lg n)$ as well.

Walking question

`SELECT` starts at the root and walks *down*; `RANK` starts at a node and walks *up*. Why the opposite directions? (Hint: `SELECT` is given a number and must find a place, so it can only

steer using information at the current node on the way down. `RANK` is given a place and must accumulate a number, gathering up all the smaller elements that hang off to its left as it rises.) Convince yourself that you could not easily swap the directions.

14.3 Keeping `size` Correct Through Insert and Delete

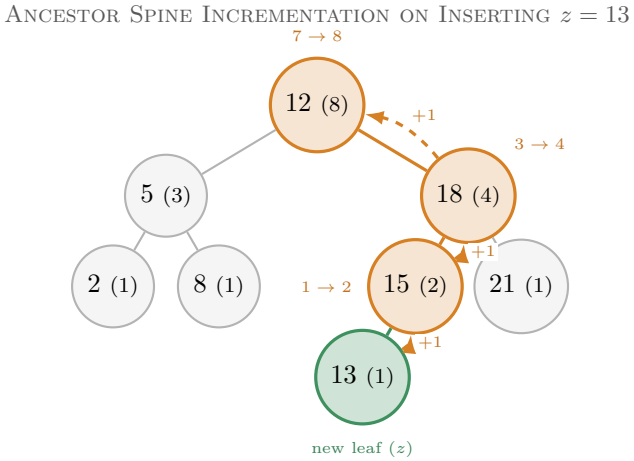
The whole scheme depends on every `size` field being exactly right at all times — including the instant *after* an insertion or deletion, when red-black trees do their rotations and recolorings. So we must check two things:

1. When we add or remove a key, we update the `size` fields cheaply.

2. When the tree rebalances itself with rotations, the `size` fields survive.

14.3.1 Insertion and deletion: the downward pass

A red-black insertion happens in two phases. First you walk down from the root to a leaf to find the spot for the new key. Every node on that path gains exactly one descendant — the new node — so as you walk down, just do $x.size \mathrel{+}= 1$ at each step. That is $O(\lg n)$ extra work on a walk you were doing anyway. Deletion is symmetric: every node on the path from the root to the spliced-out node loses exactly one descendant, so decrement along the way.



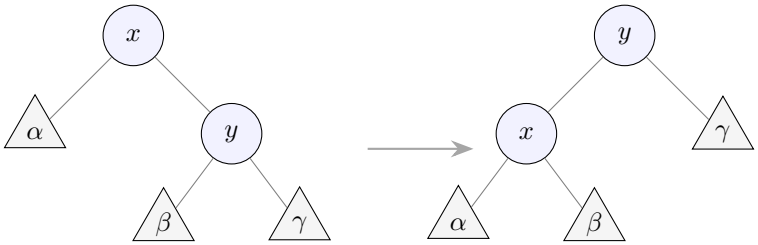
Local property: Only ancestors along the search path gain a descendant.
Subtrees outside this spine (e.g., node 5's branch) are unchanged. Total update cost: $O(h) = O(\log n)$.

Figure 14.2: Updating augmented `size` fields during insertion. When new node $z = 13$ is added, only its direct ancestors along the insertion spine (15, 18, 12) have their subtree counts incremented. All sibling subtrees remain untouched, guaranteeing that augmentation maintenance requires only $O(h)$ work.

14.3.2 Rotations: the local fix-up

The subtle part is the *second* phase. After the simple insert, a red-black tree may perform rotations to restore balance. A rotation rearranges three nodes and a couple of subtrees. It changes which subtrees hang where, so it can change `size` fields — but only at the two nodes directly involved, and only using information those nodes can already see.

Look at a left rotation about x, which promotes its right child y:



before: left-rotate at x after

Before the rotation, y was the root of a subtree containing x. After it, y sits where x was, so y inherits x's old size, and x must recompute its own from its (new) children. Only x and y change, and each fix uses only the sizes of its immediate children:

Algorithm 58: Size maintenance inside LEFT-ROTATE(T, x)

Input: a node x whose right child y is being rotated up

```
// ...  perform the ordinary pointer surgery of a left rotation first ...
```

1 $y.size \leftarrow x.size$; `// y now governs x's old subtree`

2 $x.size \leftarrow x.left.size + x.right.size + 1$;

Algorithm lab

Let us watch the numbers move. Suppose before a left rotation at x the subtree sizes are: α has 4 nodes, β has 2, γ has 3.
Before. $y.size = \beta + \gamma + 1 = 2 + 3 + 1 = 6$. And $x.size = \alpha + y.size + 1 = 4 + 6 + 1 = 11$.
After. Now y is on top with children x and γ. Apply the two lines: first $y.size \leftarrow x.size = 11$ (the whole subtree still has 11 nodes — a rotation never adds or drops a node). Then $x.size \leftarrow \alpha + \beta + 1 = 4 + 2 + 1 = 7$.
Verify independently. x's new subtree is α, β, and x: $4 + 2 + 1 = 7$. ✓ And y's subtree is x's subtree, γ, and y: $7 + 3 + 1 = 11$. ✓ The total stayed 11, which is exactly what should happen — you only moved nodes around, you did not create or destroy any.

Because a red-black insert or delete performs at most a constant number of rotations ($O(1)$ of them, as you proved last chapter), the rotation fix-ups add only $O(1)$ total work. The downward size bumps add $O(\lg n)$. So the grand total is still $O(\lg n)$: augmenting cost us nothing asymptotically.

Cost summary

Order-statistic tree (red-black base, n keys):

Operation	Worst-case time
`INSERT`, `DELETE`	$O(\lg n)$ (size bumps absorbed)
`SEARCH`, `MIN`, `MAX`	$O(\lg n)$
`OS-SELECT`(i)	$O(\lg n)$
`OS-RANK`(x)	$O(\lg n)$
extra space	$O(n)$ (one integer per node)

The augmentation is "free": same asymptotics as the plain tree, brand-new queries.

Why this matters

Where does this show up in practice? Anywhere a system needs *live percentiles*. A trading system ranks an incoming order against millions of resting orders to decide priority. A leaderboard in a game must answer "what is my global rank?" the instant your score changes. A monitoring dashboard reports the median and 99th-percentile request latency as a stream of measurements pours in. Each of these is `INSERT` plus `RANK`/`SELECT` on a set that never stops changing — exactly the workload an order-statistic tree is built for.

14.4 The General Recipe: A Theorem for Augmenting

We just augmented a red-black tree with a `size` field and got order statistics almost for free. The natural question is: *when* is it that cheap, and what do we actually have to check? Augmentation follows the same four steps each time.

Formal

Augmenting in four steps.

1. *Choose* an underlying data structure (here, a red-black tree).
2. *Decide* what extra information to store (here, `size`).
3. *Verify* the extra information can be *maintained* during the structure's basic modifying operations (insert, delete, rotate).
4. *Develop* the new operations the extra information enables.

The steps are a checklist, not a straitjacket — in practice you iterate, often discovering at step 3 that you must enrich what you store in step 2. The work that needs real care is step 3, and the next theorem makes step 3 almost mechanical for red-black trees.

What matters is *locality*. Our `size` field had one important property: a node could compute its field from *only its own data and the fields of its two children*. It never needed to look down into the depths or out across the tree. That is exactly the property that makes rotations cheap, because a rotation only rearranges a node and its children — so if a field depends only on children, only the rotated nodes can possibly need a fresh value.

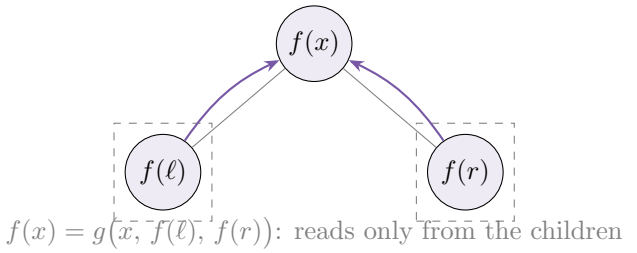


Figure 14.3: The locality condition. A node's augmented field $f(x)$ is a function of the node itself and the f values of its two children only — never of anything deeper. The whole subtrees below ℓ and r are invisible to x except through those two summaries.

Theorem 14.1 (Maintaining augmentation, CLRS Theorem 14.1)**.** *Let f be a field added to every node of a red-black tree of n nodes. Suppose that for each node x, the value $f(x)$ can be computed using only the information in x itself together with the f values of x's two children, $x.left$ and $x.right$ (Figure 14.3). Then we can maintain f at every node during* `INSERT` *and* `DELETE` *without changing their $O(\lg n)$ asymptotic running time.*

Remark 14.1 (Why the locality condition matters)*.* Read the hypothesis slowly: $f(x)$ depends only on x, $f(x.left)$, and $f(x.right)$. Our `size` field qualifies, because $x.size = x.left.size + x.right.size + 1$ — only the children's f values appear. So does, say, a field storing the *minimum* key in a subtree, or the *sum* of subtree keys, or (as we are about to see) the *maximum endpoint* of a set of intervals.

The proof is the argument we already ran for `size`, stated in general.

Sketch, with all the moving parts named. A red-black `INSERT` or `DELETE` changes the tree in two ways: a *structural change* on the root-to-leaf path (one node added or removed), and a constant number of *rotations*.

The path. When a node is added or removed, the only subtrees whose contents changed are those on the path from the root down to the affected spot. By hypothesis, each node's f depends only on its children's f. So after fixing the bottom of the path, we recompute f one node at a time as we walk back up: each recomputation is $O(1)$, and there are $O(\lg n)$ nodes on the path. Total: $O(\lg n)$.

The rotations. A single rotation changes the parent-child relationships of only $O(1)$ nodes (the two pivots and the one subtree that swaps sides). Every *other* node still has the same children with the same f values, so its f is automatically still correct — nothing to do. Only the two pivot nodes need their f recomputed, each in $O(1)$ time using the formula. Since a red-black operation does $O(1)$ rotations, the rotation fix-ups cost $O(1)$ total.

Adding $O(\lg n)$ for the path and $O(1)$ for the rotations leaves the running time at $O(\lg n)$. □

Intuition

The theorem is really saying: *if your gauge reads off of its own children and nothing farther, then rebalancing can never invalidate more than the handful of nodes it physically touched.* This is why augmentation is usually free, and it is the reason you should always reach for a field defined by "my value = some combination of my children's values."

Common trap

The condition is a real restriction. Suppose you tried to store at each node its *rank* in the whole tree (its global position in sorted order) directly. Inserting one tiny element at the front shifts *every* node's rank by one — that field depends on the entire tree, rather than merely on immediate children, so it fails the hypothesis and costs $\Theta(n)$ to maintain. The fix is exactly what we did: store the *local* `size` instead, and *derive* rank on demand by combining sizes along a path. Always prefer a field that reads only from your children.

History and connection

As computer graphics, VLSI layout verification (checking that millions of microscopic transistors on a chip do not cross illegally), and database query planners matured in the late 1970s and 1980s, computer scientists encountered problems requiring more than key membership: engineers needed to query geometric overlaps, point stabbing (finding all intervals containing a given number), and dynamic percentile ranks. Herbert Edelsbrunner (1980s), Franco P. Preparata, and Michael I. Shamos pioneered segment and interval trees for computational geometry. Thomas H. Cormen, Charles E. Leiserson, and Ronald L. Rivest formalized the general four-step methodology for augmenting existing balanced structures, proving that local child-computable fields never increase the asymptotic cost of tree rotations.

A tempting false start in data structure design is attempting to augment nodes with global properties. For instance, directly storing a node's global index or rank in sorted order appears convenient for $O(1)$ percentile queries. However, inserting a single element at the minimum shifts the rank of every existing node, requiring $\Theta(n)$ updates. The fix is to store *local subtree summaries* (such as `size` or `max`) and synthesize global answers dynamically along the $O(\log n)$ search path.

Dynamic rank selection and interval overlap queries run in $O(\log n)$ worst-case time amidst continuous insertions and deletions. This powers calendar scheduling collision detection, IP address routing tables, and VLSI circuit trace clearance checks. The limitation is that the methodology strictly requires that the augmented attribute $f(x)$ depends only on x, $x.left$, and $x.right$. Higher-dimensional range queries (such as finding rectangles overlapping in two dimensions) fail this simple locality test and require multi-level structures like range trees or segment trees that incur $O(n \log^{d-1} n)$ memory overhead. The boundary between one-dimensional local augmentation and multi-dimensional geometric index structures remains an active area of investigation in computational geometry.

14.5 Interval Trees: Augmenting for Overlap

To show that the recipe is reusable, we apply it to a different problem. Instead of numbers, our data are now *closed intervals* on the real line, like $[15, 23]$ or $[6, 10]$. Think of them as time slots on a calendar, or as ranges of IP addresses, or as line segments. We want a dynamic set of intervals supporting:

- `INTERVAL-INSERT` / `INTERVAL-DELETE`, and
- `INTERVAL-SEARCH`(i): given a query interval i, return *some* interval in the set that overlaps i, or report that none does.

Definition 14.1 (Overlap)**.** Two closed intervals $i = [i.low, i.high]$ and $i' = [i'.low, i'.high]$ *overlap* when they share at least one point, that is, when

$$i.low \le i'.high \quad \text{and} \quad i'.low \le i.high.$$

Focus

That double inequality is worth memorizing, because it is cleaner than the mistake people reach for first. *Two intervals overlap iff each one starts before the other ends.* The negation is just as clean and often easier to reason about: they *miss* each other exactly when one ends before the other starts, i.e. $i.high < i'.low$ or $i'.high < i.low$.

Figure 14.4 shows the two ways intervals can sit relative to each other: sharing at least one point (overlap) or sitting wholly apart (miss).



overlap: $i.low \le i'.high$ *and* $i'.low \le i.high$ miss: $i.high < i'.low$ (a gap between them)

Figure 14.4: Two closed intervals overlap exactly when neither ends before the other begins. On the left they share the shaded band; on the right a gap separates them.

Step 1 (choose the base): a red-black tree, keyed on the *low* endpoint of each interval. So the tree is sorted by where intervals start.

Step 2 (extra info): at each node x store

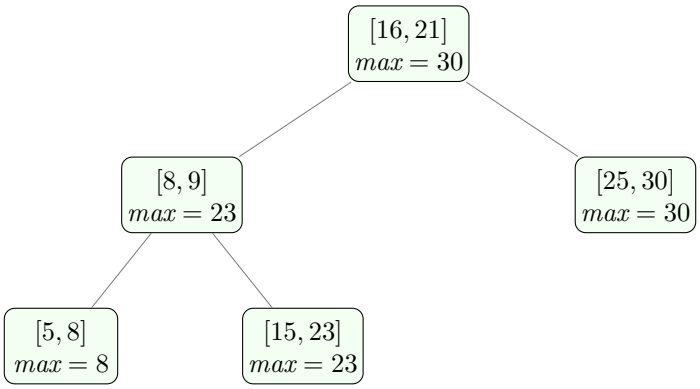
$$x.max \;=\; \text{the largest } high \text{ endpoint of any interval in } x\text{'s subtree.}$$

Does this field satisfy the locality condition of Theorem 14.1? Yes:

$$x.max \;=\; \max\big(x.int.high,\; x.left.max,\; x.right.max\big).$$

The maximum high endpoint in your subtree is the biggest of three things: your own interval's high endpoint, and the maxes already computed by your two children. Only children's f values appear — so by the theorem, `INTERVAL-INSERT` and `INTERVAL-DELETE` maintain `max` for free, in $O(\lg n)$, including through rotations. **Step 3** is done before we even start, courtesy of the theorem. (We let the sentinel `NIL` have $max = -\infty$ so the formula needs no special cases.)

Here is a set of five intervals in the tree, each node labelled $[low, high]$ on top and its `max` below.



Check the root: the high endpoints in the whole tree are $21, 9, 30, 8, 23$; the biggest is 30, and indeed the root's `max` is 30. Check node $[8, 9]$: its subtree holds $[8, 9], [5, 8], [15, 23]$, high endpoints $9, 8, 23$, max $= 23$. ✓

14.5.1 `INTERVAL-SEARCH`: the one clever step

Now **step 4**, the new operation. We want to find any interval overlapping a query i. Start at the root and, at each node x, decide whether to go left or right. The rule is:

Algorithm 59: INTERVAL-SEARCH(T, i) — find an interval overlapping i

Input: interval tree T and a query interval i
Output: a node whose interval overlaps i, or `NIL` if none does

```
x ← T.root;
while x ≠ T.NIL and i does not overlap x.int do
    if x.left ≠ T.NIL and x.left.max ≥ i.low then
        x ← x.left;                          // a left interval might still reach i
    else
        x ← x.right;
return x;
```

Read the branch carefully. At node x, if x's own interval overlaps i we are done and the loop stops. Otherwise we go *left* exactly when the left subtree exists and its `max` (the farthest-right endpoint anywhere on the left) is at least $i.low$ — meaning some interval down the left *might* reach far enough right to touch i. If even the leftmost, farthest-reaching interval cannot reach $i.low$, there is no point looking left; go right instead.

Intuition

The `max` field is acting like a periscope. Standing at x, you cannot see the individual intervals buried in the left subtree, but `max` lets you see the *rightmost reach* of the entire left subtree in one glance. If that rightmost reach falls short of where your query begins ($x.left.max < i.low$), the entire left subtree is hopeless and you can discard it without looking. That is how one number prunes half the tree.

Algorithm lab

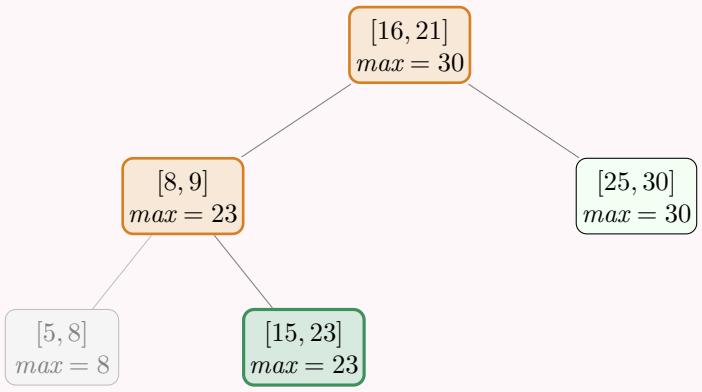
Let us run INTERVAL-SEARCH for the query $i = [22, 25]$ on the tree above. (i starts at 22, so $i.low = 22$, $i.high = 25$.)

At root $[16, 21]$**.** Overlap? We need $16 \leq 25$ (yes) *and* $22 \leq 21$ (no). Not overlapping. Now the branch: the left child is $[8, 9]$ with $max = 23$. Is $23 \geq i.low = 22$? Yes — so something on the left *might* reach 22. Go left to $[8, 9]$.

At node $[8, 9]$**.** Overlap with $[22, 25]$? Need $8 \leq 25$ (yes) and $22 \leq 9$ (no). No overlap. Branch: left child is $[5, 8]$ with $max = 8$. Is $8 \geq 22$? No. So abandon the left subtree — nothing there reaches 22 — and go right to $[15, 23]$.

At node $[15, 23]$**.** Overlap with $[22, 25]$? Need $15 \leq 25$ (yes) and $22 \leq 23$ (yes). **Overlap! Return** $[15, 23]$**.**

And indeed $[15, 23]$ and $[22, 25]$ share the points 22 and 23. Notice the periscope at work: at node $[8, 9]$ we threw away the entire left subtree using a single comparison $8 \geq 22$, never inspecting $[5, 8]$ directly.



Orange marks the nodes the search visits for $i = [22, 25]$; green is the overlap it returns; the grayed-out $[5, 8]$ is pruned by the periscope ($max = 8 < 22$) and never touched.

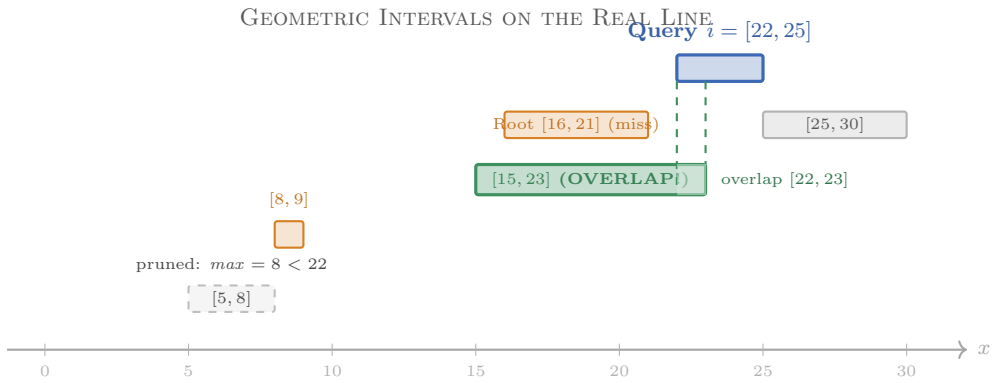


Figure 14.5: The geometry of an interval overlap query. For query $i = [22, 25]$, the search starts at root $[16, 21]$ (which misses i as $21 < 22$). The left child has $max = 23 \geq 22$, proving that the left branch can reach $i.low$. Following the left branch discovers the overlapping interval $[15, 23]$ in $O(\log n)$ time, safely bypassing $[5, 8]$ whose reach $max = 8$ falls short.

14.5.2 Why the search never misses an overlap

It is not obvious that this greedy left/right choice can never miss an existing overlap. What if we go left and the only overlapping interval was actually on the right? The following lemma rules that out.

Lemma 14.1 (Interval search is correct, CLRS Theorem 14.2)**.** *In* INTERVAL-SEARCH*, if the algorithm steps to a child, then that child's subtree contains an interval overlapping* i whenever the whole tree does*. Consequently, if some interval in* T *overlaps* i*, the algorithm returns one; otherwise it returns* `NIL`*.*

Gentle proof. Each iteration that does not stop sends us either left or right. We show that in each case, we never abandon an overlap that exists.

Case 1: we go right. We only go right when the left child is missing, or $x.left.max < i.low$. Take the interesting subcase, $x.left.max < i.low$. By definition, $x.left.max$ is the largest high endpoint anywhere in the left subtree. So *every* interval i' in the left subtree has $i'.high \leq x.left.max < i.low$, i.e. $i'.high < i.low$ — it ends strictly before i begins. By Definition 14.1, no such i' can overlap i. The left subtree is genuinely empty of overlaps, so discarding it loses nothing. Going right is safe.

Case 2: we go left. We go left when $x.left.max \geq i.low$. So the left subtree contains *some* interval i' with $i'.high \geq i.low$ (the one achieving the max). That is half of the overlap condition. We must rule out the danger that, despite a promising left subtree, the only true overlap was sitting on the *right*, and by going left we walk away from it.

Here is where the tree's sorting saves us. The tree is keyed on *low* endpoints. Consider that left-subtree interval i' with $i'.high \geq i.low$. If i' overlaps i, we are home — the left subtree contains an overlap and going left is correct. If i' does *not* overlap i, then since $i'.high \geq i.low$ already holds, the other half of overlap must fail: $i.high < i'.low$. But i' is in the left subtree, and the tree is sorted by low endpoint, so *every* interval i'' to the right of i' (which includes the entire right subtree of x, and x itself) has $i''.low \geq i'.low > i.high$. Such an i'' starts *after* i ends, so it cannot overlap i either (Figure 14.6). Hence no overlap lives on the right; if one exists at all it must be in the left subtree. Going left is safe.

In both cases we move toward a subtree that still contains an overlap whenever one exists. Since the loop also stops the moment x itself overlaps, by the time we reach `NIL` we have correctly concluded no overlap exists. □

Walking question

Take the decisive line of Case 2 on your walk: "if i' does not overlap but $i'.high \geq i.low$, then $i.high < i'.low$, and so everything to the right of i' starts even later and also misses i." Draw two intervals on a number line where $i'.high \geq i.low$ yet they do not overlap. What must be true of their starts? Now slide a third interval even farther right. See why it cannot rescue the overlap? That picture contains the whole argument.

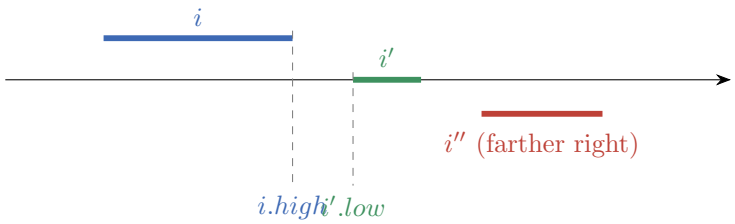


Figure 14.6: Case 2 of the correctness proof. If i' does not overlap i yet $i'.high \geq i.low$, then $i.high < i'.low$: i' begins to the right of where i ends. Any i'' sorted after i' begins even farther right, so it cannot overlap i either. No overlap can hide on the right.

Since each iteration descends one level and the tree has height $O(\lg n)$, INTERVAL-SEARCH runs in $O(\lg n)$ time. Insert and delete are $O(\lg n)$ by the maintenance theorem. We have built a second, completely different data structure with the same four-step recipe and the same locality trick.

Cost summary

Interval tree (red-black base, n intervals):

Operation	Worst-case time
`INTERVAL-INSERT`, `INTERVAL-DELETE`	$O(\lg n)$
`INTERVAL-SEARCH`(i) (find one overlap)	$O(\lg n)$
report *all* k overlaps	$O((k+1)\lg n)$
extra space	$O(n)$ (one `max` per node)

"Find all k overlaps" is just: find one, delete it from a copy, repeat — each round is $O(\lg n)$, there are k successful rounds plus one final round that reports "no more," giving $O((k+1)\lg n)$; or, better, recurse using the `max` fields to prune (Exercise 14.8).

Why this matters

Interval trees are everywhere there is a notion of "range" and a need to detect clashes fast. Calendar apps check whether a proposed meeting collides with any existing one. Network routers and firewalls match a packet's address against ranges of allowed addresses. Genome browsers find which genes overlap a region of a chromosome — a query that runs millions of times over datasets with hundreds of millions of features, where the difference between $O(\lg n)$ and $O(n)$ per query is the difference between interactive and useless.

14.6 A Field Guide to Augmentation

Both examples worked for the same reason. In each case we found a field that summarizes a subtree and is computable from a node plus its children's fields. Once you have that, the theorem hands you maintenance for free, and your job shrinks to designing the clever query (`SELECT`/`RANK`, or the left/right periscope). Here is a small catalog of fields that pass the locality test, to train your eye:

Field at node x	Local formula	Buys you
`size`	$\ell.size + r.size + 1$	rank, select, percentile
`sum`	$\ell.sum + r.sum + x.key$	subtree / prefix sums
`min`	$\min(x.key, \ell.min, r.min)$	range minimum
`max-high`	$\max(x.high, \ell.max, r.max)$	interval overlap
`height`	$1 + \max(\ell.height, r.height)$	subtree height

Each one reads only off the node and its two children, so each one is free to maintain. The design work is choosing a field that answers the question.

Common trap

A subtle failure: not every useful field is local, and you must check rather than hope. "Number of nodes within distance d of x in key-value" or "the median of my subtree" do *not* obey the locality condition — a child's median tells you almost nothing about the parent's median. When a field fails the test, you have two moves: enrich it into something that *is* local (store a sorted summary, a count, a histogram), or accept that this particular field will cost more than $O(\lg n)$ to keep current. Never assume; write down the formula and see whether only the children appear.

Pause

If your head is full, here is the one sentence to keep. **To augment a balanced tree, add a field that each node can compute from itself and its two children; the rebalancing machinery will then maintain it for free, and you spend your cleverness only on the query.** Order-statistic trees and interval trees are two instances of that rule.

A Closing Picture

In this chapter we did not invent two new data structures. We took *one* structure we already trusted — the red-black tree — and, twice, taught it to answer a new family of questions by storing a single extra number per node and writing one clever query. The maintenance theorem is the reason this is a repeatable method rather than luck each time: as long as your extra information is *local* — readable from a node and its children — the expensive part (keeping it correct under insertion, deletion, and rebalancing) is handled for you, automatically, at no asymptotic cost.

The same approach applies in the rest of the book and in real systems. When a problem looks like it needs a bespoke structure, pause and ask: *is there a balanced tree underneath, and a small local summary I could bolt on to answer this?* Often the answer is yes.

Up to this point in the book, our main tool for speed has been data structure design: organizing elements so that queries can drill straight down to the answer in $O(\lg n)$ time. In Part IV, we turn from organizing stored data to the structure of algorithmic decisions themselves. In **Chapter 15 (Dynamic Programming)**, we encounter problems where subproblems overlap and repeat, learning how to record subproblem solutions in a table to collapse exponential search spaces into fast polynomial-time algorithms.

14.7 Exercises

Exercise 14.1 (Warm-up: trace by hand)**.** Build an order-statistic tree (you may draw any balanced shape) on the keys $\{3, 8, 12, 19, 22, 27, 35\}$, labelling each node with its `size`. Then run OS-SELECT$(root, 5)$ step by step, and separately compute OS-RANK of the node holding key 22. Confirm the two answers are consistent (i.e. `SELECT` and `RANK` invert each other on this key).

Exercise 14.2 (The forgotten subtraction)**.** A classmate writes OS-SELECT but recurses right with the unchanged index i instead of $i - r$. On the seven-node tree from Section 14.2, trace their buggy code for the query $i = 6$ and show that, because the inflated index is never shrunk on a right turn, the recursion walks off the right end of the tree (it reaches a `NIL` child asking for an index larger than any subtree can hold). What is the correct answer? Explain in one sentence why dropping the subtraction breaks the meaning of i.

Exercise 14.3 (Maintaining size through a rotation)**.** Suppose a left rotation is performed at a node x whose subtrees have sizes $\alpha = 6$ (left of x), $\beta = 3$, $\gamma = 5$ (with β, γ the subtrees of x's right child y, as in the figure). Compute every `size` field before and after the rotation, and verify the total node count is unchanged.

Exercise 14.4 (Counting smaller elements)**.** Using an order-statistic tree, give an $O(\lg n)$ algorithm `COUNT-LESS`(k) that returns how many keys in the tree are strictly less than a given value k (where k need not be present in the tree). Sketch the algorithm and argue its running time.

Exercise 14.5 (Does the field stay local?)**.** For each proposed node field, write down its formula in terms of the node and its children, and state whether it satisfies the locality hypothesis of Theorem 14.1. (a) the number of *red* nodes in the subtree; (b) the depth of the node (its distance from the root); (c) the sum of all keys in the subtree; (d) the second-smallest key in the subtree.

Exercise 14.6 (Overlap by hand)**.** On the five-interval tree of Section 14.5, run INTERVAL-SEARCH for the query $i = [11, 14]$. Show which node you visit at each step, which branch you take and why (cite the `max` comparison), and the final result.

Exercise 14.7 (The point-stabbing query)**.** Specialize interval search to a single point: given a number p, find any interval in the tree that contains p. Show that this is just INTERVAL-SEARCH with the degenerate query interval $[p, p]$, and rewrite the overlap test for this case. What does the `max` pruning rule become?

Exercise 14.8 (Report all overlaps)**.** Modify interval search to return *all* k intervals overlapping a query i, rather than a single match. Argue that a careful recursion (descending left only when $x.left.max \geq i.low$, and right only when $x.int.low \leq i.high$) finds all of them, and that its running time is $O((k + 1) \lg n)$.

Exercise 14.9 (A new augmentation: subtree sum)**.** Augment a balanced BST so that each node stores the `sum` of all keys in its subtree. (a) Give the local formula and confirm it passes Theorem 14.1. (b) Give an $O(\lg n)$ operation `SUM-LESS`(k) returning the sum of all keys strictly less than k. (c) Explain how `sum` is fixed up during a left rotation.

Exercise 14.10 (Challenge: dynamic minimum-gap)**.** You maintain a dynamic set of distinct numbers in a balanced tree and want to support `MIN-GAP`, returning the smallest difference between any two adjacent numbers (in sorted order) currently in the set. Design an augmentation that supports `INSERT`, `DELETE`, and `MIN-GAP` in $O(\lg n)$. *Hint: at each node store three local summaries — the minimum key, the maximum key, and the minimum adjacent gap within the subtree — and write the formula combining a node with its two children. Check it is local.*

Exercise 14.11 (Challenge: why not store rank directly?)**.** Argue carefully that storing each node's global rank (its position in sorted order) as a field cannot be maintained in $o(n)$ time per insertion in the worst case. Then explain in one paragraph how the `size` field sidesteps this exact difficulty while still answering rank queries in $O(\lg n)$.

Part IV

Advanced Design and Analysis

Chapter 15

Dynamic Programming

In the previous chapter, we augmented trees to query dynamic data. But many optimization problems require choosing an optimal sequence of decisions rather than storing data. Divide and conquer chops a problem into *independent* pieces, solves each, and glues the answers together. Greedy methods (Chapter 16) make one locally best choice at a time and never look back. Dynamic programming sits between them. It is the technique you reach for when a problem can be broken into subproblems, but the subproblems *overlap* – the same little question gets asked again and again – and when the best overall answer is built out of best answers to those smaller questions.

The name is historical and a little misleading; we will untangle it later. In one sentence, dynamic programming is the technique of *not solving the same subproblem twice*.

Intuition

Imagine you are climbing a staircase and someone asks, "how many distinct ways can you reach step 30?" You realize that the number of ways to reach step 30 is just the number of ways to reach step 29 plus the number of ways to reach step 28 (you arrive by a final hop of one or two). The same is true for step 29, which leans on 28 and 27, and so on down. If you computed this by blind recursion you would recompute the answer for step 25 thousands of times. Dynamic programming says: compute the answer for each step *once*, write it in a little notebook, and look it up forever after. That notebook – a table of solved subproblems – is the core mechanism of dynamic programming.

We will develop the method on four classic problems: cutting a steel rod into pieces, multiplying a chain of matrices, finding the longest common subsequence of two strings, and building an optimal binary search tree. By the end you will recognize the shape of a dynamic-programming problem on sight, and you will know the two mechanical ways to turn that recognition into working code.

15.1 A First Problem: Cutting a Rod

A company buys long steel rods and cuts them into shorter rods to sell. A rod of integer length n inches can be cut into pieces of integer lengths, and a piece of length i sells for a known price p_i. The prices are not proportional to length – there are wholesale and scarcity effects – so a short piece can be surprisingly valuable. The question is simple to state and genuinely not obvious: *given a rod*

of length n and the price table, what is the maximum revenue r_n you can earn by cutting it up (or leaving it whole)?

Here is the price table we will use for the entire section.

length i	1	2	3	4	5	6	7	8	9	10
price p_i	1	5	8	9	10	17	17	20	24	30

Let us warm up with tiny cases, by hand. A rod of length 1 can only be left alone: $r_1 = 1$. A rod of length 2 can be left whole for $p_2 = 5$, or cut into $1 + 1$ for $1 + 1 = 2$; the best is $r_2 = 5$. Length 3: whole gives 8; $1 + 2$ gives $1 + 5 = 6$; $1 + 1 + 1$ gives 3. Best is $r_3 = 8$. Length 4 is the first surprise: whole gives $p_4 = 9$, but $2 + 2$ gives $5 + 5 = 10$. So $r_4 = 10$, and the optimal solution is *not* to leave it whole even though a length-4 price exists. This is what makes the problem interesting.

15.1.1 Finding the recursive structure

How would you reason about a rod of length n in general? Picture making the cuts left to right. Whatever the optimal cutting is, it begins with some *first* piece, of length i for some $1 \le i \le n$. That first piece earns exactly p_i. The rest of the rod, length $n - i$, is then a smaller copy of *the very same problem*: cut a rod of length $n - i$ for maximum revenue. We do not know which first length i is best, so we try them all and keep the winner.

Focus

This gives the central recurrence for rod cutting:

$$r_n \;=\; \max_{1 \le i \le n} \big(p_i + r_{n-i} \big), \qquad r_0 = 0.$$

In words: the best revenue for length n equals the best, over all choices of first-piece length i, of "sell the first piece for p_i, then optimally cut the remaining $n - i$ inches." The base case $r_0 = 0$ says an empty rod earns nothing.

This recurrence already exposes the two properties that make a problem suitable for dynamic programming. The first we call *optimal substructure*: the optimal solution to the whole problem contains within it optimal solutions to subproblems (the leftover rod is cut optimally, never wastefully). The second is *overlapping subproblems*: when you expand the recurrence you find that r_{n-i} for the same small value shows up over and over. We will make both ideas precise in Section 15.2.

15.1.2 The naive recursion and why it explodes

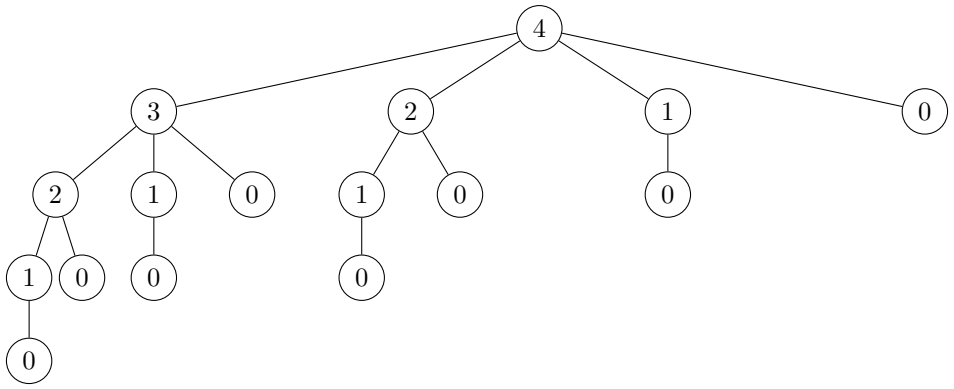
If you translate the recurrence directly into code, you get this.

Algorithm 60: CUT-ROD(p, n) – naive recursion

Input: price array $p[1\,..\,n]$, rod length n
Output: maximum revenue r_n

```
if n = 0 then
    return 0
q ← −∞;
for i ← 1 to n do
    q ← max(q, p[i] + CUT-ROD(p, n − i));
return q;
```

This is correct, and for $n = 4$ it is fine. But watch what happens for larger n. To solve length 4 we call lengths 3, 2, 1, 0. To solve length 3 we again call 2, 1, 0. The same subproblems are re-solved in completely separate branches of the recursion. The recursion tree below shows the calls for $n = 4$; notice how often each node reappears.



Let $T(n)$ count the number of calls CUT-ROD makes (including itself) when solving length n. The root makes one call to itself plus one call for each of $n-0, n-1, \ldots, 1$, i.e. to lengths 0 through $n-1$:

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j), \qquad T(0) = 1.$$

A little unrolling shows $T(n) = 2^n$. (Check: $T(0) = 1$, $T(1) = 1+T(0) = 2$, $T(2) = 1+T(0)+T(1) = 4$, $T(3) = 8$.) The work doubles with every extra inch. That is catastrophic – length 40 would take a trillion calls – and the cause is pure waste: we keep re-deriving answers we already found.

Common trap

A natural first reaction is "this exponential blowup is fundamental to the problem." In fact the blowup comes from the *recursion*, not from the *problem*. There are only $n + 1$ distinct subproblems (r_0 through r_n). Any algorithm that solves each distinct subproblem once is already polynomial. Dynamic programming is precisely the discipline of arranging that.

15.1.3 Memoization: the notebook fix

A small change fixes this. Keep an array $r[0\,..\,n]$, initialized to "unknown" (we use $-\infty$ as a sentinel). Before solving a subproblem, check the array; if the answer is already there, return it. After solving

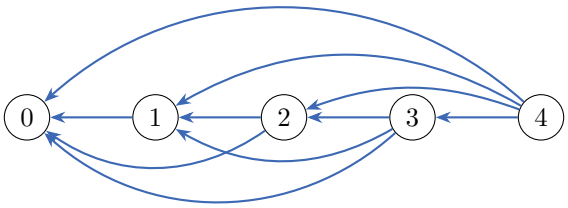
it, store it. This top-down caching is called *memoization* (yes, no "r" – as in writing a *memo*).

Algorithm 61: MEMOIZED-CUT-ROD(p, n)

```
Input: price array p[1..n], length n
Output: maximum revenue r_n
let r[0..n] be a new array, all entries ← −∞;
return MCR-AUX(p, n, r);
procedure MCR-AUX(p, n, r);
if r[n] ≥ 0 then
    return r[n] ;                                   // already solved
if n = 0 then
    q ← 0
else
    q ← −∞;
    for i ← 1 to n do
        q ← max(q, p[i] + MCR-AUX(p, n − i, r));
r[n] ← q;
return q;
```

Now each subproblem is genuinely solved once; subsequent requests are one-line lookups. There are $n+1$ subproblems, and solving subproblem j does $O(j)$ work in its loop, so the total is $\sum_{j=0}^{n} O(j) = O(n^2)$. We went from 2^n to n^2 by adding one array and one early-return.

The picture below makes the saving visible. On the left is the exploding recursion tree of the naive call; on the right is what memoization (or tabulation) actually computes – the *subproblem graph*, with one node per distinct length and an edge $j \to j'$ whenever solving j consults j'. The tree has 2^n nodes; the graph has only $n + 1$.



The subproblem graph for $n = 4$: five nodes, a handful of edges. Each node is solved once. Compare the 16-node recursion tree above.

15.1.4 Tabulation: filling the table bottom-up

There is a second, often tidier way to get the same effect. Instead of recursing top-down and caching, go *bottom-up*: solve the smallest subproblem first, then the next, building up. This is called *tabulation*. For rod cutting, solve r_0, then r_1, then r_2, and so on; by the time you reach r_n every value you need is already sitting in the table.

Algorithm 62: BOTTOM-UP-CUT-ROD(p, n)

Input: price array $p[1\,..\,n]$, length n
Output: maximum revenue r_n

```
let r[0..n] be a new array;
r[0] ← 0;
for j ← 1 to n do
    q ← −∞;
    for i ← 1 to j do
        q ← max(q, p[i] + r[j − i]);
    r[j] ← q;
return r[n];
```

The two nested loops make the $O(n^2)$ cost transparent: the inner loop runs j times for each j, and $\sum_{j=1}^{n} j = \frac{n(n+1)}{2} = \Theta(n^2)$.

Algorithm lab

Let us run BOTTOM-UP-CUT-ROD on the price table, by hand, for $n = 8$. We fill $r[0]$ through $r[8]$, each time trying every first-piece length i from 1 to j and recording the best $p_i + r[j-i]$.

j	best split tried	value	$r[j]$
0	—	—	0
1	$p_1 + r_0 = 1 + 0$	1	1
2	$p_2 + r_0 = 5 + 0$	5	5
3	$p_3 + r_0 = 8 + 0$	8	8
4	$p_2 + r_2 = 5 + 5$	10	10
5	$p_2 + r_3 = 5 + 8$	13	13
6	$p_6 + r_0 = 17 + 0$	17	17
7	$p_1 + r_6 = 1 + 17$ or $p_6 + r_1 = 17 + 1$	18	18
8	$p_2 + r_6 = 5 + 17$	22	22

Trace $j = 7$ carefully, since it has a tie. We try $p_1 + r_6 = 18$, $p_2 + r_5 = 5 + 13 = 18$, $p_3 + r_4 = 8 + 10 = 18$, $p_4 + r_3 = 9 + 8 = 17$, $p_5 + r_2 = 10 + 5 = 15$, $p_6 + r_1 = 17 + 1 = 18$, $p_7 + r_0 = 17$. The maximum is 18, and several splits achieve it. The table does not care which – it stores only the value 18. Reconstructing *a* set of cuts is the next subsection.

15.1.5 Reconstructing the actual cuts

The table gives you the best *revenue*, but a customer wants to know *where to cut*. The standard trick is to record, alongside each $r[j]$, the first-piece length that achieved the maximum. We store it in a second array $s[j]$.

Algorithm 63: EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

Input: price array $p[1..n]$, length n
Output: revenue array r and choice array s

```
let r[0..n] and s[1..n] be new arrays;   r[0] ← 0;
for j ← 1 to n do
    q ← −∞;
    for i ← 1 to j do
        if q < p[i] + r[j − i] then
            q ← p[i] + r[j − i];
            s[j] ← i;
    r[j] ← q;
return (r, s);
```

To print the cuts for a rod of length n, peel off $s[n]$, then recurse on the remainder $n - s[n]$:

Algorithm 64: PRINT-CUT-ROD-SOLUTION(p, n)

```
let (r, s) ← EXTENDED-BOTTOM-UP-CUT-ROD(p, n);
while n > 0 do
    print s[n];
    n ← n − s[n];
```

For our table, s comes out as $s = [1, 2, 3, 2, 2, 6, 1, 2]$ for $j = 1..8$. For $n = 8$: $s[8] = 2$, print 2, go to 6; $s[6] = 6$, print 6, go to 0. So the cut list is $2 + 6$, earning $5 + 17 = 22$. Confirmed against the table.

Figure 15.1 traces this reconstruction visually: each entry $s[j]$ points backward to the remaining subproblem $j - s[j]$ until reaching the base case $j = 0$.

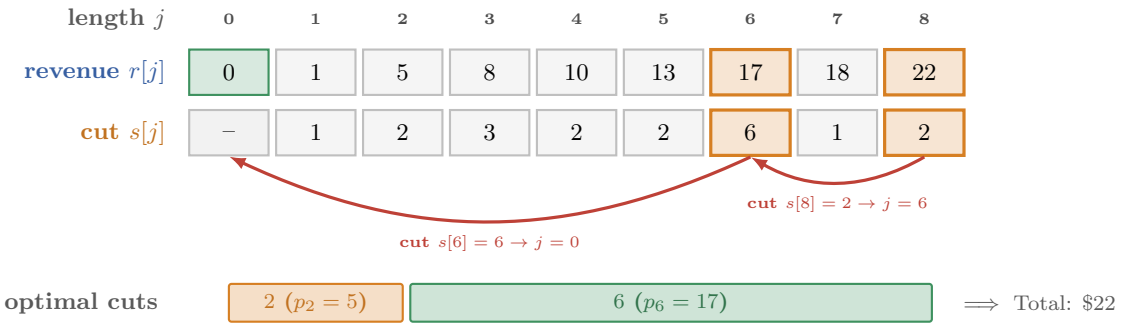


Figure 15.1: Reconstructing the optimal cut of a length-8 rod using the choice table s. Starting at $j = 8$, entry $s[8] = 2$ selects a first piece of length 2 (revenue $p_2 = 5$) and jumps to remaining length $8 - 2 = 6$. At $j = 6$, entry $s[6] = 6$ selects a piece of length 6 (revenue $p_6 = 17$) and jumps to remainder 0, producing the optimal partition $2 + 6$ with total revenue \$22.

Why this matters

Rod cutting is a toy, but its skeleton – "the best first move plus the best solution to what remains" – is exactly the structure of resource-allocation problems that come up constantly in

practice: how to slice a fixed advertising budget across channels, how to pack a knapsack of cargo, how to break a paragraph into lines of justified text (the algorithm behind TEX's line breaking is dynamic programming), and how to schedule jobs for maximum profit.

15.2 The Elements of Dynamic Programming

Now that we have solved one problem end to end, let us name its recurring parts so we can spot them quickly in new problems. Every dynamic-programming algorithm is built from the same four steps.

Focus

The four-step recipe for designing a DP:

1. **Characterize the structure** of an optimal solution: what is the "last choice" that splits a problem into subproblems?
2. **Define the value** of an optimal solution recursively (write the recurrence, including base cases).
3. **Compute that value**, usually bottom-up into a table (or top-down with memoization).
4. **Reconstruct** an optimal solution from recorded choices (only if you need the solution itself rather than merely its value).

For step 3 to be possible at all, the problem must have two properties. They are the litmus test for "can I use DP here?"

15.2.1 Optimal substructure

Definition 15.1 (Optimal substructure)**.** A problem has *optimal substructure* if an optimal solution to the problem contains within it optimal solutions to subproblems.

In rod cutting, once you commit to a first piece of length i, the rest of the rod *must* be cut optimally – if it were not, you could replace that part with a better cutting and improve the whole, contradicting optimality. This cut-and-paste argument is how you prove optimal substructure in general: *assume* the subpart is suboptimal, *paste* in a better subpart, and derive a better global solution than the supposedly optimal one. A contradiction means the subpart was optimal all along.

Common trap

Not every problem has optimal substructure, and the failures are instructive. Consider *unweighted longest simple path* in a graph. A longest simple path from u to v need not decompose into a longest simple path from u to some intermediate w plus one from w to v – stitching two longest paths can revisit a vertex and break simplicity. (Shortest paths, by contrast, do have optimal substructure, which is why Dijkstra's algorithm and Bellman–Ford work.) Always *check* the property; do not assume it.

15.2.2 Overlapping subproblems

Definition 15.2 (Overlapping subproblems)**.** A problem has *overlapping subproblems* if a recursive solution revisits the same subproblem repeatedly, rather than always generating fresh ones.

This is what divide-and-conquer problems usually *lack*: when you split an array for merge sort, the left and right halves are disjoint and never share sub-instances, so caching buys nothing. Dynamic programming pays off precisely when the subproblem space is small but the recursion keeps wandering back into it. For rod cutting the space had size $n+1$ yet the naive recursion made 2^n calls; that gap of "few distinct subproblems, many repeated visits" is what to look for.

Intuition

A clean way to see whether DP applies: draw the *subproblem graph*. Put one node per distinct subproblem, and an edge from A to B if solving A directly needs the answer to B. For rod cutting this is a little chain $r_n \to r_{n-1} \to \cdots \to r_0$ with extra forward edges. The number of nodes is the number of distinct subproblems; the number of edges bounds the total work. Bottom-up tabulation is just a topological-order sweep of this graph. Memoization is a depth-first traversal that caches. If the graph is small and acyclic, DP is efficient.

15.2.3 Memoization versus tabulation

We have now seen both implementation styles. They compute the same answers and have the same asymptotic cost; the choice is about taste and a few practical trade-offs.

	Memoization (top-down)	**Tabulation (bottom-up)**
Control flow	recursion + cache	loops fill a table
Order solved	on demand, lazily	every subproblem, in order
Skips unused subproblems	yes	no (unless you are clever)
Recursion overhead	yes (stack frames)	none
Easy to add "window" space saving	harder	easier
Best when	subproblem space sparse, or recurrence awkward	all subproblems needed, order clear

Focus

A useful default: *think* with the top-down recurrence (it mirrors how you reason), then *write* bottom-up tabulation when every subproblem is needed (it is faster and avoids deep recursion). Reach for memoization when only a sparse slice of the table is actually visited, or when figuring out the correct bottom-up filling order is genuinely fiddly.

History and connection

The foundations of dynamic programming were laid by Richard Bellman in the 1950s at the RAND Corporation while analyzing multistage decision processes for military logistics.

"Programming" referred to tabular scheduling (as in mathematical programming), not writing computer code. Bellman recounted that he chose the term "dynamic" partly because it had an active, unimpeachable connotation that protected foundational mathematical research from budget cuts by Secretary of Defense Charles Wilson. Bellman formulated the *Principle of Optimality*: an optimal policy has the property that whatever the initial state and decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.
The fundamental misconception in early optimization was treating all recursive problems as divide-and-conquer. When subproblems are independent (as in mergesort), divide-and-conquer excels; but when subproblems overlap heavily (as in rod-cutting or shortest paths), naive branching recursion explodes exponentially, re-evaluating identical states trillions of times. Memoization and bottom-up tabulation replace an exponential recursion tree with a polynomial-size directed acyclic graph (DAG) of distinct subproblems, and this became a standard technique of computer science.
The technique brought polynomial-time solutions for problems once deemed intractable: genomic sequence alignment (Needleman–Wunsch and Smith–Waterman in bioinformatics), hidden Markov model decoding in speech recognition and communications (the Viterbi algorithm), context-free grammar parsing in compilers (the Cocke–Younger–Kasami algorithm), and shortest-path network routing. Dynamic programming has a well-known limitation: the "curse of dimensionality." When a problem requires tracking multiple interacting resource constraints, the dimensions of the table multiply, causing memory and execution time to grow exponentially with the number of state variables. Developing space-efficient algorithms and approximation schemes for high-dimensional dynamic programs remains an active research area.

15.3 Matrix-Chain Multiplication

Here the DP runs not over "length" but over *parenthesizations*, and optimal substructure shows up in a particularly clean form.

You are given a chain of matrices $A_1, A_2, \ldots, A_n$ to multiply together. Matrix multiplication is associative – you may group the product any way you like and get the same matrix – but the *cost* of computing it depends enormously on the grouping. Multiplying a $p \times q$ matrix by a $q \times r$ matrix using the standard algorithm costs $p \cdot q \cdot r$ scalar multiplications. So the question is: *how should you parenthesize the chain to minimize the total number of scalar multiplications?*

Example 15.1 (Why grouping matters)**.** Take three matrices with dimensions $A_1 : 10 \times 100$, $A_2 : 100 \times 5$, $A_3 : 5 \times 50$. Two groupings:

$$((A_1A_2)A_3): \quad 10 \cdot 100 \cdot 5 + 10 \cdot 5 \cdot 50 = 5000 + 2500 = 7500,$$

$$(A_1(A_2A_3)): \quad 100 \cdot 5 \cdot 50 + 10 \cdot 100 \cdot 50 = 25000 + 50000 = 75000.$$

Same answer matrix, but one grouping is *ten times* cheaper. With a long chain the gap can be astronomical.

We describe the chain by its dimension sequence $p_0, p_1, \ldots, p_n$, where matrix A_k has dimensions $p_{k-1} \times p_k$. (Consecutive matrices share a dimension, which is what makes them multipliable.) In the example, $p = \langle 10, 100, 5, 50 \rangle$.

15.3.1 The recurrence

What is the "last choice" here? Any full parenthesization of $A_i \cdots A_j$ splits, at the very top level, into a left group $A_i \cdots A_k$ and a right group $A_{k+1} \cdots A_j$ for some split point k with $i \le k < j$. The two groups produce a $p_{i-1} \times p_k$ matrix and a $p_k \times p_j$ matrix, and the final multiply costs $p_{i-1}\, p_k\, p_j$. Let $m[i, j]$ be the minimum number of scalar multiplications to compute $A_i \cdots A_j$. Then:

Focus

$$m[i,j] = \begin{cases} 0 & \text{if } i = j, \\ \min\limits_{i \le k < j} \Big(m[i,k] + m[k+1,j] + p_{i-1}\, p_k\, p_j \Big) & \text{if } i < j. \end{cases}$$

A single matrix costs nothing. Otherwise, try every split point k, pay for the best way to build each side plus the cost of the final multiply, and keep the minimum.

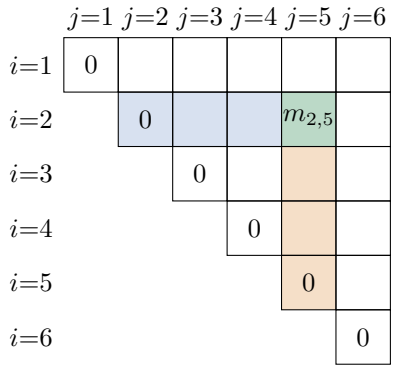
Optimal substructure: if the optimal parenthesization splits at k, then its left subchain must itself be optimally parenthesized (cut-and-paste again: a cheaper left grouping would cheapen the whole, contradiction). And the subproblems overlap massively – $m[2, 4]$, say, is needed by many different larger chains.

How many distinct subproblems are there? One for each pair $i \le j$, which is $\binom{n}{2} + n = \Theta(n^2)$. Solving each tries up to n split points, so the total is $\Theta(n^3)$. (For comparison, the number of distinct parenthesizations is the Catalan number, which grows like $4^n/n^{1.5}$ – so brute force is hopeless, and DP turns it polynomial.)

15.3.2 Filling the table by chain length

The base cases ($i = j$) sit on the diagonal of the table. Each $m[i, j]$ depends on entries with *shorter* chains ($m[i, k]$ and $m[k + 1, j]$ each span fewer matrices). So the safe filling order is by increasing chain length $\ell = j - i + 1$, from 2 up to n.

Geometrically, we fill the upper-triangular table one *diagonal* at a time, starting from the main diagonal of zeros and moving toward the lone top-right corner that holds the final answer. The diagram shows the order for $n = 6$: the shaded cell $m[i, j]$ pulls from the row to its left ($m[i, k]$) and the column below it ($m[k + 1, j]$), both of which lie on already-completed diagonals.



Computing $m_{2,5}$ (green): the minimum scans split points $k = 2, 3, 4$, each combining a blue cell from row 2 ($m_{2,k}$) with an orange cell from column 5 ($m_{k+1,5}$). Both lie on shorter-chain diagonals, hence already

known.

Algorithm 65: MATRIX-CHAIN-ORDER(p)

Input: dimension sequence $p[0\,..\,n]$ (so there are n matrices)
Output: cost table m and split table s

```
for i ← 1 to n do
    m[i, i] ← 0
for ℓ ← 2 to n do
    for i ← 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;
        m[i, j] ← ∞;
        for k ← i to j − 1 do
            q ← m[i, k] + m[k + 1, j] + p[i − 1] · p[k] · p[j];
            if q < m[i, j] then
                m[i, j] ← q;
                s[i, j] ← k;
return (m, s);
```

The three nested loops each run $O(n)$ times, giving $O(n^3)$ time, and the two tables use $\Theta(n^2)$ space.

Algorithm lab

Let us trace a real chain of six matrices, the classic CLRS instance, with

$$p = \langle 30,\ 35,\ 15,\ 5,\ 10,\ 20,\ 25 \rangle,$$

so $A_1 : 30 \times 35$, $A_2 : 35 \times 15$, $A_3 : 15 \times 5$, $A_4 : 5 \times 10$, $A_5 : 10 \times 20$, $A_6 : 20 \times 25$. We fill $m[i, j]$ diagonal by diagonal.
Length 1 (the diagonal): all $m[i, i] = 0$.
Length 2: each is a single multiply.

$$\begin{aligned} m[1, 2] &= p_0 p_1 p_2 = 30 \cdot 35 \cdot 15 = 15750, \\ m[2, 3] &= 35 \cdot 15 \cdot 5 = 2625, \\ m[3, 4] &= 15 \cdot 5 \cdot 10 = 750, \\ m[4, 5] &= 5 \cdot 10 \cdot 20 = 1000, \\ m[5, 6] &= 10 \cdot 20 \cdot 25 = 5000. \end{aligned}$$

Length 3: try both split points. For $m[1, 3]$: k=1 gives $0 + 2625 + 30 \cdot 35 \cdot 5 = 2625 + 5250 = 7875$; k=2 gives $15750 + 0 + 30 \cdot 15 \cdot 5 = 15750 + 2250 = 18000$. Minimum 7875 at k=1. Working out the rest of this diagonal the same way gives $m[2, 4] = 4375$ (k=3), $m[3, 5] = 2500$ (k=3), $m[4, 6] = 3500$ (k=5).
Length 4 and up: continuing the sweep produces $m[1, 4] = 9375$, $m[2, 5] = 7125$, $m[3, 6] = 5375$, then $m[1, 5] = 11875$, $m[2, 6] = 10500$, and finally the answer $m[1, 6] = 15125$. The full upper-triangular table:

	j=1	2	3	4	5	6
i=1	0	15750	7875	9375	11875	**15125**
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

The optimal cost is $m[1, 6] = 15125$ scalar multiplications.

15.3.3 Reconstructing the parenthesization

The split table $s[i, j]$ records the best top-level split for each subchain. For the trace above, $s[1, 6] = 3$, meaning the best top split of $A_1 \cdots A_6$ is $(A_1A_2A_3)(A_4A_5A_6)$. Recursing with s inside each side ($s[1, 3] = 1$, $s[4, 6] = 5$) yields the full grouping

$$\big((A_1(A_2A_3))\,((A_4A_5)A_6)\big).$$

Algorithm 66: PRINT-OPTIMAL-PARENS(s, i, j)

```
if i = j then
    print "A_i"
else
    print "(";
    PRINT-OPTIMAL-PARENS(s, i, s[i, j]);
    PRINT-OPTIMAL-PARENS(s, s[i, j] + 1, j);
    print ")";
```

Walking question

We filled the matrix-chain table by increasing chain length. Could we instead have filled it by, say, increasing i then increasing j? Try to articulate exactly which earlier entries $m[i, j]$ depends on, and convince yourself that *any* ordering that computes all shorter chains before longer ones works – but column-by-column left-to-right does *not*. The lesson generalizes: in a DP, the legal filling orders are exactly the topological orders of the subproblem dependency graph.

15.4 Longest Common Subsequence

Switch domains entirely: from numbers to strings. A *subsequence* of a string is what you get by deleting zero or more characters without reordering the rest. So `ace` is a subsequence of `abcde` (delete `b` and `d`), but `aec` is not (wrong order). Given two sequences X and Y, the *longest common subsequence* (LCS) problem asks for the longest string that is a subsequence of *both*.

Why this matters

The Unix `diff` utility, the "changes" view in every code review tool, and Git's merge machinery all rest on a close cousin of LCS: to show how two files differ, find their longest common subsequence of lines and call everything else an insertion or a deletion. Bioinformatics uses the same idea to align DNA or protein sequences and measure how related two organisms are. When you see a green-and-red diff, you are looking at dynamic programming.

15.4.1 The recurrence

Let $X = \langle x_1, \ldots, x_m \rangle$ and $Y = \langle y_1, \ldots, y_n \rangle$. Write X_i for the prefix $\langle x_1, \ldots, x_i \rangle$. Define $c[i, j]$ as the length of an LCS of the prefixes X_i and Y_j. Think about the *last* characters x_i and y_j.

- If $x_i = y_j$, that shared character can end an LCS. Use it, and the problem reduces to the LCS of the shorter prefixes X_{i-1} and Y_{j-1}, plus one. So $c[i, j] = c[i-1, j-1] + 1$.

- If $x_i \neq y_j$, the two last characters cannot both be the final character of a common subsequence. At least one of them is unused, so the answer is the better of dropping x_i (use $c[i-1, j]$) or dropping y_j (use $c[i, j-1]$): $c[i, j] = \max(c[i-1, j],\ c[i, j-1])$.

Focus

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i-1, j-1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max\big(c[i-1, j],\ c[i, j-1]\big) & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

The base case says: the LCS of anything with an empty prefix is empty.

The subproblems are all pairs (i, j) with $0 \leq i \leq m$, $0 \leq j \leq n$: that is $(m+1)(n+1) = \Theta(mn)$ of them, each solved in $O(1)$, giving $\Theta(mn)$ time and space.

15.4.2 Filling the table and tracking arrows

We fill c row by row. Alongside, we keep a direction array $b[i, j]$ recording which case won – a diagonal arrow $\nwarrow$ when characters matched, an up arrow $\uparrow$ when we took $c[i-1, j]$, a left arrow $\leftarrow$ when we took $c[i, j-1]$. These arrows let us walk back and read off the actual subsequence.

Algorithm 67: LCS-Length(X, Y)

Input: sequences $X[1..m]$ and $Y[1..n]$
Output: length table c and direction table b

```
for i ← 0 to m do
    c[i, 0] ← 0
for j ← 0 to n do
    c[0, j] ← 0
for i ← 1 to m do
    for j ← 1 to n do
        if x_i = y_j then
            c[i, j] ← c[i − 1, j − 1] + 1;    b[i, j] ← " ↖ ";
        else
            if c[i − 1, j] ≥ c[i, j − 1] then
                c[i, j] ← c[i − 1, j];    b[i, j] ← " ↑ ";
            else
                c[i, j] ← c[i, j − 1];    b[i, j] ← " ← ";
return (c, b);
```

Algorithm lab

Let us compute the LCS of $X =$ `ABCBDAB` and $Y =$ `BDCABA` ($m = 7$, $n = 6$), the textbook example. Here is the completed c table with the winning arrows shown inside matched cells. Rows are indexed by X, columns by Y; row/column 0 are the all-zero borders.

		B	D	C	A	B	A
	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
A 1	0	0	0	0	↖1	1	↖1
B 2	0	↖1	1	1	1	↖2	2
C 3	0	1	1	↖2	2	2	2
B 4	0	↖1	1	2	2	↖3	3
D 5	0	1	↖2	2	2	3	3
A 6	0	1	2	2	↖3	3	↖4
B 7	0	↖1	2	2	3	↖4	4

The bottom-right cell $c[7, 6] = 4$ is the length of the LCS. Let us read *one* such subsequence by tracing arrows backward from $c[7, 6]$.

15.4.3 Reading the subsequence backward

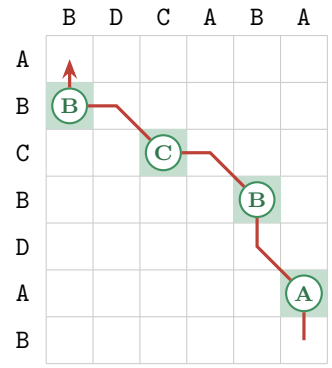
Starting at (m, n), follow b: on a ↖ (match) record the character and step diagonally to $(i-1, j-1)$; on ↑ step to $(i-1, j)$; on ← step to $(i, j-1)$. Stop when you fall off the table. Because we collect characters from the back, reverse the collected string at the end.

Algorithm 68: PRINT-LCS(b, X, i, j)

```
if i = 0 or j = 0 then
    return
if b[i, j] = "↖" then
    PRINT-LCS(b, X, i − 1, j − 1);
    print x_i;
else if b[i, j] = "↑" then
    PRINT-LCS(b, X, i − 1, j);
else
    PRINT-LCS(b, X, i, j − 1);
```

Tracing our table from $(7, 6)$ with the $\geq$ tie-break baked into LCS-LENGTH: the cell $(7, 6)$ is an $\uparrow$, so we step to $(6, 6)$, which is a $\nwarrow$ match emitting $x_6 =$ `A`; continuing, the $\nwarrow$ cells we meet emit, in back-to-front order, $x_6 =$ `A`, then $x_4 =$ `B`, then $x_3 =$ `C`, then $x_2 =$ `B`. Reversed, that is `BCBA` – a length-4 common subsequence of both strings.

The backward walk is easiest to see as a staircase through the grid. Diagonal steps (on a $\nwarrow$) emit a character; up and left steps just move without emitting. The four diagonal steps below are the four characters of the LCS.



Reconstructing the LCS: the red path walks backward from the bottom-right corner; green cells are the $\nwarrow$ matches. Collected back-to-front they give `A`,`B`,`C`,`B`; reversed into forward order this forms `BCBA`.

(Other LCSs of length 4 exist, e.g. `BDAB` and `BCAB`; ties in the max are broken by our $\geq$ choice, so the algorithm prints one specific witness.)

Common trap

A subsequence is not a substring. `BCBA` need not appear as a contiguous block in either string – and indeed it does not. Students routinely conflate the two. "Subsequence" allows gaps; "substring" (also called factor) does not. The longest common *substring* problem is a different, also-DP problem with a slightly different recurrence.

Cost summary

Longest common subsequence.

- Subproblems: $(m + 1)(n + 1)$, one per prefix pair.

- Time: $\Theta(mn)$ (constant work per cell).
- Space: $\Theta(mn)$ for the full table; reducible to $\Theta(\min(m, n))$ if you only need the *length* (keep two rows), though reconstructing the subsequence itself in linear space needs the cleverer Hirschberg divide-and-conquer trick.

15.5 Optimal Binary Search Trees

Our last example combines DP with a data structure you met earlier, and it introduces a twist: we optimize not for the worst case but for the *expected* case, given how often each key is searched.

Suppose you are building a binary search tree to hold n keys $k_1 < k_2 < \cdots < k_n$ that will be searched many times. You know the *probability* p_i that a search is for key k_i. (We will also allow "misses" – searches for values between the keys – with probabilities $q_0, \ldots, q_n$, where q_j is the chance of a search landing in the gap just after k_j; the d_j "dummy" leaves represent these gaps. All the p's and q's together sum to 1.) A search for a key costs one comparison per node visited, i.e. its depth plus one. So the *expected search cost* of a tree T is

$$E[\text{cost}] = \sum_{i=1}^{n} \big(\text{depth}_T(k_i) + 1\big) p_i \; + \; \sum_{j=0}^{n} \big(\text{depth}_T(d_j) + 1\big) q_j.$$

We want the tree – the shape – that minimizes this. A balanced tree is *not* always best: if one key is searched 90% of the time, you want it at the root even if that unbalances everything.

Intuition

Frequency should pull popular keys toward the root, the way a well-organized toolbox keeps the tools you grab constantly right on top, even if that means a lopsided layout. A balanced BST minimizes the worst-case depth; an *optimal* BST minimizes the average depth weighted by how often you actually reach for each key. Those are different goals, and the gap between them is exactly what this section computes.

15.5.1 The recurrence

What is the last choice? The *root*. In an optimal BST for keys $k_i, \ldots, k_j$, some key k_r (with $i \le r \le j$) sits at the root; its left subtree holds $k_i, \ldots, k_{r-1}$ and its right subtree holds $k_{r+1}, \ldots, k_j$, and – optimal substructure! – each subtree must itself be an optimal BST for its keys.

One difference from matrix-chain needs care. When you make k_r the root, every key and dummy in both subtrees drops *one level deeper*, so each of their costs increases by its own probability. The total extra cost when you combine subtrees over the key range $i..j$ is the sum of all probabilities in that range. Define

$$w(i, j) = \sum_{\ell=i}^{j} p_\ell + \sum_{\ell=i-1}^{j} q_\ell,$$

the total weight of subtree $i..j$. Let $e[i, j]$ be the expected cost of an optimal BST built on keys $k_i, \ldots, k_j$. Then:

Focus

$$e[i,j] = \begin{cases} q_{i-1} & \text{if } j = i-1 \text{ (dummy leaf)}, \\ \min\limits_{i \le r \le j} \Big(e[i,r-1] + e[r+1,j] + w(i,j) \Big) & \text{if } i \le j. \end{cases}$$

Choose each possible root r; pay for the optimal left and right subtrees and add $w(i,j)$ for sinking every node one level. The $w(i,j)$ term does *not* depend on r, but it must be added once for whichever root you pick.

Notice w obeys an easy recurrence of its own: $w(i,j) = w(i,j-1) + p_j + q_j$, so we can tabulate all the weights cheaply rather than re-summing.

15.5.2 The algorithm

The structure mirrors matrix-chain exactly: fill by increasing range length, try every root, keep the minimum, record the chosen root in $root[i,j]$ for reconstruction.

Algorithm 69: OPTIMAL-BST(p, q, n)

Input: key probabilities $p[1\,..\,n]$, dummy probabilities $q[0\,..\,n]$
Output: expected-cost table e and $root$ table

```
for i ← 1 to n + 1 do
    e[i, i − 1] ← q_{i−1};   w[i, i − 1] ← q_{i−1};
for ℓ ← 1 to n do
    for i ← 1 to n − ℓ + 1 do
        j ← i + ℓ − 1;
        e[i, j] ← ∞;
        w[i, j] ← w[i, j − 1] + p_j + q_j;
        for r ← i to j do
            t ← e[i, r − 1] + e[r + 1, j] + w[i, j];
            if t < e[i, j] then
                e[i, j] ← t;   root[i, j] ← r;
return (e, root);
```

Three nested loops over $O(n)$ ranges and $O(n)$ roots give $\Theta(n^3)$ time and $\Theta(n^2)$ space – same budget as matrix-chain. (A clever observation by Knuth that the optimal root moves monotonically lets you shave this to $O(n^2)$, but we leave that as a pointer.)

Algorithm lab

Take the small CLRS instance with $n = 5$ keys:

$$p = \langle 0.15,\ 0.10,\ 0.05,\ 0.10,\ 0.20 \rangle, \quad q = \langle 0.05,\ 0.10,\ 0.05,\ 0.05,\ 0.05,\ 0.10 \rangle.$$

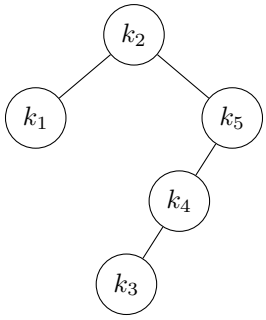
(Check: the p's sum to 0.60, the q's to 0.40, total 1.00.)
Start with the empty ranges: $e[i, i-1] = q_{i-1}$, so $e[1,0]$=0.05, $e[2,1]$=0.10, $e[3,2]$=0.05, $e[4,3]$=0.05, $e[5,4]$=0.05, $e[6,5]$=0.10.

Length 1. For a single key k_i the only root is $r = i$, so $e[i, i] = e[i, i-1] + e[i+1, i] + w[i, i]$ with $w[i, i] = p_i + q_{i-1} + q_i$. For k_1: $w[1, 1] = 0.15 + 0.05 + 0.10 = 0.30$, and $e[1, 1] = 0.05 + 0.10 + 0.30 = 0.45$. Doing all five: $e[1, 1]$=0.45, $e[2, 2]$=0.40, $e[3, 3]$=0.25, $e[4, 4]$=0.30, $e[5, 5]$=0.50.

Longer ranges. Sweeping $\ell = 2, 3, 4, 5$, trying every root and keeping the minimum, fills out the upper triangle. The final answer for the whole set of keys is

$$e[1, 5] = 2.75,$$

the expected number of comparisons per search in the optimal tree, with $root[1, 5] = 2$. Recursing on the *root* table ($root[1, 1] = 1$, $root[3, 5] = 5$, $root[3, 4] = 4$, $root[4, 4] = 4$, ...) reconstructs the optimal shape: k_2 at the root, with k_1 as its left child and k_5 heading the right subtree.



The optimal BST for the lab instance: heavy keys k_2, k_5 ride high.

Formal

Correctness of the optimal-BST recurrence. We claim $e[i, j]$ as defined equals the minimum expected cost over all BSTs on keys $k_i, \ldots, k_j$. Proof sketch by induction on the range length $j - i + 1$. *Base:* an empty range is a single dummy leaf d_{i-1} at depth 0 within the subtree, contributing q_{i-1}; the formula gives exactly that. *Step:* any BST on $k_i, \ldots, k_j$ has *some* root k_r; its subtrees are BSTs on $k_i, \ldots, k_{r-1}$ and $k_{r+1}, \ldots, k_j$. By optimal substructure (cut-and-paste), an optimal overall tree uses optimal subtrees, whose costs are $e[i, r-1]$ and $e[r+1, j]$ by the induction hypothesis. Rooting at k_r deepens every node of both subtrees by one, adding $w(i, j)$ to the cost. Minimizing over all valid roots r gives exactly the recurrence, so the computed $e[i, j]$ is optimal. ■

15.6 The Shape of a DP

You have now seen the main archetypes of dynamic programming, and they all share the same structure. In each, you found a "last choice" (first piece, top-level split, last character, root, or including an item) that broke the problem into independent optimal subproblems; you wrote a recurrence over a small, structured space of subproblems; you filled a table in an order respecting the dependencies; and you stored side information to reconstruct an actual solution. Once you know this pattern, most such optimization problems come down to finding the right last choice.

15.6.1 The Dependency DAG and Table Fill Topologies

Every dynamic program can be viewed geometrically as a directed acyclic graph (DAG) where vertices represent distinct subproblems and directed edges lead from each subproblem to the smaller subproblems it queries. Computing a solution bottom-up by tabulation is simply a *reverse topological traversal* of this DAG – guaranteeing that every prerequisite subproblem is solved and waiting in memory before it is ever consulted.

Figure 15.2 summarizes four common dependency patterns:

1. **1D Linear Prefix (Rod Cutting):** Each subproblem $r[j]$ depends on all shorter prefixes $r[0 \dots j-1]$. The table is a 1D array filled sequentially from $j = 1$ to n.

2. **2D Grid Neighborhood (LCS):** Each cell $c[i, j]$ depends strictly on three local neighbors: directly north ($c[i-1, j]$), directly west ($c[i, j-1]$), and northwest ($c[i-1, j-1]$). This allows row-major or column-major evaluation.

3. **2D Diagonal Intervals (Matrix-Chain Multiplication & Optimal BST):** Subproblem $m[i, j]$ spans an interval from index i to j. A split at pivot k queries $m[i, k]$ (in row i) and $m[k+1, j]$ (in column j). Because dependencies have shorter chain lengths, the table must be filled *diagonally* by increasing length $\ell = j - i + 1$.

4. **2D Subset / Capacity Step (0-1 Knapsack):** Given n items with weights w_i and values v_i, and total capacity W, let $K[i, w]$ denote the maximum value using a subset of items $1 \dots i$ within capacity weight w. Deciding whether to take item i yields:

$$K[i,w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0, \\ K[i-1, w] & \text{if } w_i > w, \\ \max\big(K[i-1, w],\ K[i-1, w-w_i] + v_i\big) & \text{if } w_i \le w. \end{cases}$$

   Each cell $K[i, w]$ depends only on row $i-1$: straight above at weight w, and offset to the left at weight $w - w_i$. The table is filled row-by-row ($i = 1 \dots n$). By iterating w backward from W down to w_i, memory compresses to a single 1D buffer of size $W + 1$.

Cost summary

Costs at a glance.

Problem	Subproblems	Time	Space
Rod cutting	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$
Matrix-chain	$\Theta(n^2)$	$\Theta(n^3)$	$\Theta(n^2)$
LCS	$\Theta(mn)$	$\Theta(mn)$	$\Theta(mn)$
Optimal BST	$\Theta(n^2)$	$\Theta(n^3)$	$\Theta(n^2)$
0-1 Knapsack	$\Theta(nW)$	$\Theta(nW)$	$\Theta(nW)$ or $\Theta(W)$

In every row the time is (number of subproblems) $\times$ (work per subproblem), where "work per subproblem" is the cost of evaluating the min/max over all choices.

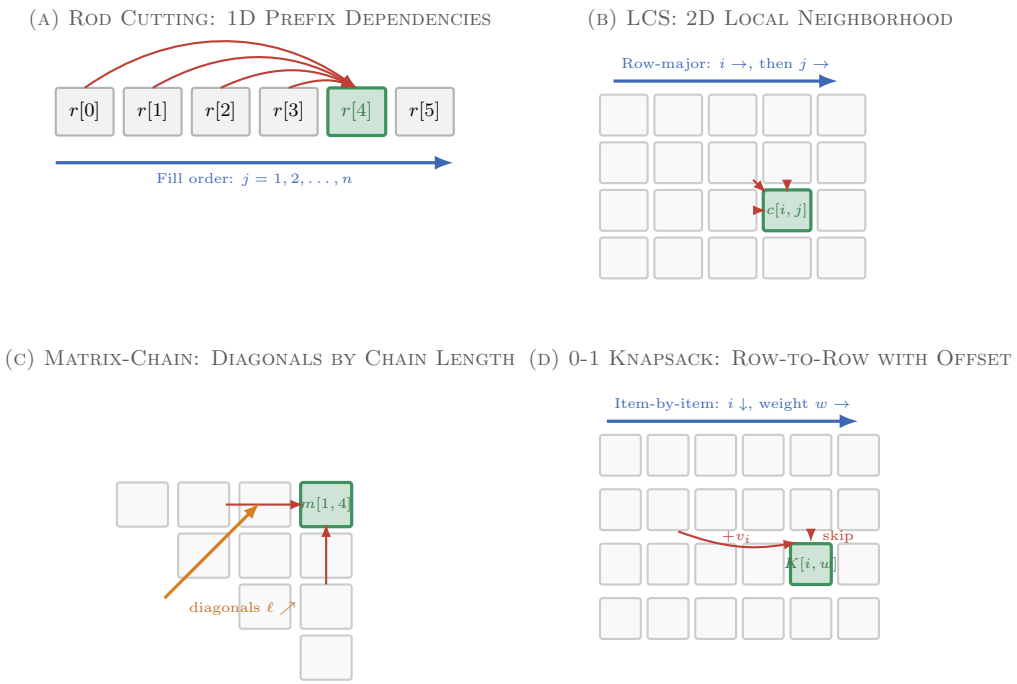


Figure 15.2: Comparison of dynamic programming dependency DAGs and table fill topologies. In (a) Rod Cutting, each subproblem depends on all smaller lengths in a 1D prefix. In (b) LCS, cells depend strictly on three immediate grid neighbors. In (c) Matrix-Chain Multiplication, cells depend on cross-sections along its row and column, mandating a diagonal-by-diagonal fill by chain length ℓ. In (d) 0-1 Knapsack, each cell in row i depends on two cells in row $i-1$ (directly above or offset left by w_i), enabling row-by-row tabulation.

Walking question

One last walk. Pick any optimization puzzle you have met – making change with fewest coins, the longest increasing subsequence of a list, the minimum number of edits to turn one word into another. For each, ask the three diagnostic questions: (1) What is the last choice that splits it? (2) Does an optimal solution force its subparts to be optimal (optimal substructure)? (3) Do the subproblems overlap? If all three answer yes, you can write the recurrence and the table on the spot. Try it for "fewest coins to make 47 cents with denominations $\{1, 5, 10, 25\}$" before you read the exercises.

Dynamic programming is thorough: it explores overlapping subproblems and tabulates their solutions to guarantee a global optimum. But evaluating every choice takes time and requires maintaining a table in memory.

When can we make a choice immediately without solving subproblems first? In **Chapter 16 (Greedy Algorithms)**, we study problems that possess the *greedy-choice property*: problems where making the locally best choice at each step never locks us out of the globally optimal solution.

Exercises

Exercise 15.1 (Rod cutting by hand)**.** Using the price table of Section 15.1, compute r_9 and r_{10} by extending the lab table. For each, give both the maximum revenue and an optimal list of cuts. (Hint: you already have r_0 through r_8.)

Exercise 15.2 (A cheaper-is-better price table)**.** Suppose every price equals the length: $p_i = i$ for all i. Argue that for any n the optimal revenue is exactly n and that *every* way of cutting is optimal. What does the choice array s look like, and why does that match your argument?

Exercise 15.3 (Memoization order)**.** In MEMOIZED-CUT-ROD, suppose you call it once for $n = 6$. List the subproblems in the order their final values are *stored* in r. How does this order compare to the bottom-up order $0, 1, 2, \ldots, 6$? Explain the difference in one sentence.

Exercise 15.4 (Matrix-chain grouping)**.** For the dimension sequence $p = \langle 5, 10, 3, 12, 5, 50, 6\rangle$ (six matrices), fill the m and s tables and state the optimal cost and parenthesization. Show at least the length-2 and length-3 diagonals in full.

Exercise 15.5 (Why not greedy?)**.** A tempting greedy heuristic for matrix-chain is "always first multiply the adjacent pair whose product is cheapest." Find a chain of three or four matrices where this greedy rule produces a *suboptimal* parenthesization, and explain which DP principle it violates.

Exercise 15.6 (LCS trace)**.** Compute an LCS of $X =$ `10010101` and $Y =$ `010110110` (treat each digit as a character). Build the full c table, then read off one LCS by tracing arrows. State its length.

Exercise 15.7 (LCS and edit distance)**.** The *edit distance* between two strings is the minimum number of single-character insertions and deletions (no substitutions) to turn one into the other. Show that if X and Y have lengths m and n and an LCS of length L, then this edit distance equals $m + n - 2L$. (Hint: characters not in the common subsequence must be deleted from X or inserted into Y.)

Exercise 15.8 (Counting subsequences)**.** Modify the LCS recurrence to instead *count* the number of distinct longest common subsequences. Write the recurrence for a table $N[i, j]$ that stores this count alongside $c[i, j]$, being careful about the case $c[i - 1, j] = c[i, j - 1]$.

Exercise 15.9 (Optimal BST sanity check)**.** For the lab instance in Section 15.5, verify by hand that the single-key cost $e[3, 3] = 0.25$ is correct: write out $w[3, 3]$ and the empty-range costs it combines, and confirm the arithmetic. Then explain in words why the optimal tree places k_2 (probability 0.10) at the root rather than the more probable k_5 (probability 0.20).

Exercise 15.10 (Balanced is not optimal)**.** Construct probabilities p_1, p_2, p_3 (with all $q_j = 0$) for three keys so that the optimal BST is a degenerate "stick" (each node has at most one child) rather than the balanced tree with k_2 at the root. State your probabilities and the expected costs of both trees to justify it.

Exercise 15.11 (Subproblem graph)**.** Draw the subproblem dependency graph for LCS of two length-3 strings (a 4×4 grid of nodes $c[i, j]$). Put an edge from each node to every node it directly depends on. How many edges leave a typical interior node, and how does that number explain the $\Theta(mn)$ running time?

Exercise 15.12 (Coins, challenge)**.** Given coin denominations $d_1 < d_2 < \cdots < d_k$ and a target amount A, design a DP that computes the fewest coins summing to exactly A (or reports impossibility). State the recurrence, the table size, the running time, and how to reconstruct which coins to use. Then run it for $A = 47$ and denominations $\{1, 5, 10, 25\}$.

Chapter 16

Greedy Algorithms

In the previous chapter you learned to be patient. Dynamic programming never commits to a choice until it has considered all the alternatives, stored their answers in a table, and compared them. It is thorough, careful, and a little slow. This chapter takes the opposite approach. A *greedy* algorithm is impatient. At every step it grabs the choice that looks best *right now*, locks it in permanently, and moves on without building a table of alternatives.

For certain problems this purely local strategy produces a provably *optimal* global solution – matching what an exhaustive dynamic program would find, in a fraction of the time and memory. The central challenge of this chapter is telling *when* a greedy rule is provably optimal and when it fails, because a plausible greedy heuristic is also one of the easiest ways to write an algorithm that is fast, simple, and wrong on subtle inputs.

We solve a scheduling problem twice – once the greedy way and once the dynamic-programming way – and watch them agree, then distill the two structural properties every greedy proof leans on. From there we build Huffman's optimal compression code from scratch, byte by byte. The chapter closes with *matroids*, the abstraction that explains, in one theorem, an entire family of problems where greed is guaranteed to win.

16.1 A first taste: choosing activities

Start with a concrete scheduling problem. You have a single shared resource – a lecture hall, a tennis court, a telescope – and a pile of requests to use it. Each request is an *activity* with a fixed start time and a fixed finish time. Two activities are *compatible* if they do not overlap in time; you cannot run two things in the hall at once. Your goal is to schedule as *many* activities as possible.

That last word matters: you are maximizing the *count* of activities, not the total time used, not anyone's preference. Just the number of non-overlapping talks you can squeeze in.

Here is a concrete set. I will give each activity an index i, a start s_i, and a finish f_i, and – this will matter shortly – I have sorted them by *finish time*.

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

This is the exact instance CLRS uses, and it is worth keeping on a sticky note. Activity 1 runs during the time interval $[1, 4)$, activity 2 during $[3, 5)$, and so on. Activities 1 and 2 overlap (both want the room around time 3.5), so they are *not* compatible. Activities 1 and 4 do not overlap ($[1, 4)$ and $[5, 7)$), so they are.

Intuition

Picture the day as a long horizontal timeline and each activity as a horizontal bar laid across it from its start to its finish. "Compatible" just means "these two bars do not touch." You want to lay down as many bars as you can without any two overlapping. Stack them on a single shelf; how many fit?

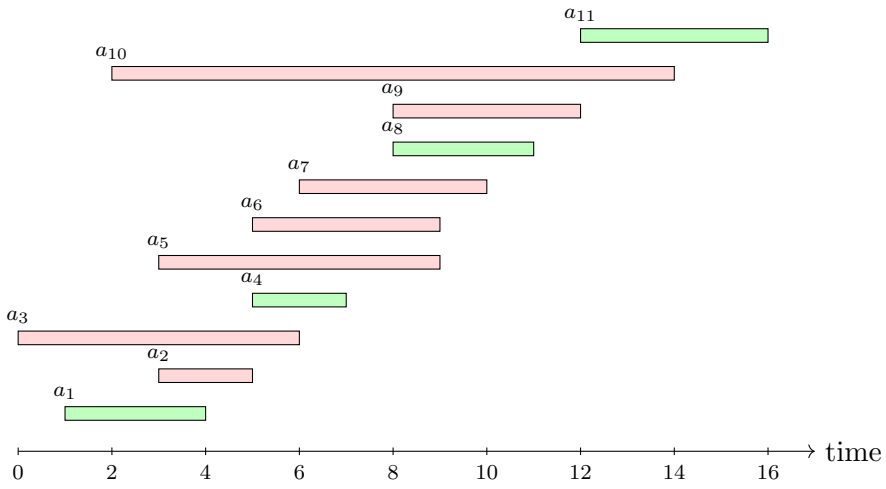


Figure 16.1: The eleven activities as bars. Highlighted in green is one optimal selection $\{a_1, a_4, a_8, a_{11}\}$ of four mutually non-overlapping activities. No selection of five exists.

16.1.1 The greedy guess

Faced with this, your instinct might be to try several rules of thumb. "Pick the shortest activity first, to leave room for others." "Pick the activity that overlaps the fewest others." "Pick the one that starts earliest." Some of these sound reasonable. Most of them are *wrong* – we will break them in a moment. But one rule, almost suspiciously simple, is provably right:

Focus

The greedy rule for activity selection. Among all activities still compatible with what you have chosen, always pick the one that *finishes earliest*. Then throw away everything that conflicts with it and repeat.

Why finish-earliest? Because finishing early is exactly the property that leaves *the most room* for everything that comes after. When you commit to the activity with the smallest finish time, you

free up the resource as soon as possible, maximizing the remaining open timeline for future choices. The rest of this section makes this precise.

Because we sorted by finish time, the greedy rule becomes almost trivial to execute: scan left to right, and grab an activity whenever it does not collide with the last one you grabbed.

Algorithm lab

Let us trace it on the table. We pick a_1 (finish 4) – it is first, so take it. Now scan for the next activity whose *start* is ≥ 4. Activity 2 starts at 3: too early, skip. Activity 3 starts at 0: skip. Activity 4 starts at $5 \geq 4$: **take it**. Last finish is now 7. Scan on: a_5 starts 3, a_6 starts 5, a_7 starts 6 – all < 7, skip. a_8 starts $8 \geq 7$: **take it**. Last finish 11. a_9 starts 8, a_{10} starts 2 – skip. a_{11} starts $12 \geq 11$: **take it**. We end with $\{a_1, a_4, a_8, a_{11}\}$ – four activities. Go back to Figure 16.1 and confirm the green bars never touch.

16.1.2 The algorithm

Here is the routine. We assume the activities arrive already sorted so that $f_1 \leq f_2 \leq \cdots \leq f_n$; if they are not, sorting first costs an extra $O(n \log n)$, which we will fold into the analysis.

Algorithm 70: GREEDY-ACTIVITY-SELECTOR(s, f, n)

Input: Start times $s[1..n]$ and finish times $f[1..n]$, with f nondecreasing.
Output: A maximum-size set A of mutually compatible activities.

```
1 A ← {a_1} ;                          // the first to finish is always safe to take
2 k ← 1 ;                              // k = index of the last activity we added
3 for m ← 2 to n do
4     if s[m] ≥ f[k] then
5         A ← A ∪ {a_m} ;              // a_m starts after a_k finishes
6         k ← m ;                      // a_m is now the most recent
7 return A
```

The loop runs once per activity and does a constant amount of work each time, so the loop is $\Theta(n)$. The only state it carries between iterations is k, the index of the last chosen activity – a single integer. That economy of state is a hallmark of greedy algorithms.

Cost summary

If the input is already sorted by finish time, GREEDY-ACTIVITY-SELECTOR runs in $\Theta(n)$ time and $\Theta(1)$ extra space (beyond the output). If you must sort first, the total is $\Theta(n \log n)$, dominated by the sort. Contrast this with a dynamic-programming solution to the same problem, which fills an $n \times n$ table in $\Theta(n^3)$ or, carefully, $\Theta(n^2)$ time. Greed is simpler here and also asymptotically faster.

16.1.3 Why is it correct? The exchange argument

A fast wrong answer is worthless, so we owe you a proof. The standard tool for greedy correctness is the *exchange argument*: take any optimal solution, and show you can edit it – swap one element for

the greedy choice – without making it worse. If the greedy choice can always be substituted in, then there is an optimal solution that *contains* the greedy choice, so greed loses nothing by taking it.

Theorem 16.1 (Greedy is optimal for activity selection)**.** *Let a_1 be the activity with the earliest finish time. Then some maximum-size set of mutually compatible activities includes a_1.*

Formal

Proof. Let A be any maximum-size compatible set, and let a_j be the activity in A with the earliest finish time. If $a_j = a_1$ we are done. Otherwise, form $A' = (A \setminus \{a_j\}) \cup \{a_1\}$: throw out a_j, throw in a_1. We claim A' is still a valid set of the same size.
Same size: we removed one activity and added one, so $|A'| = |A|$.
Still compatible: every activity in A other than a_j starts no earlier than a_j finishes (since a_j finishes earliest in A and they are all pairwise compatible). But a_1 finishes *no later* than a_j, because a_1 has the globally earliest finish time, i.e. $f_1 \le f_j$. So everything that did not conflict with a_j cannot conflict with a_1 either – a_1 vacates the room even sooner. Hence A' is compatible.
So A' is a maximum-size compatible set containing a_1. ■

Figure 16.2 illustrates this substitution: because a_1 finishes at or before a_j ($f_1 \le f_j$), swapping a_1 in place of a_j vacates the room even earlier, leaving every subsequent activity in A completely conflict-free.

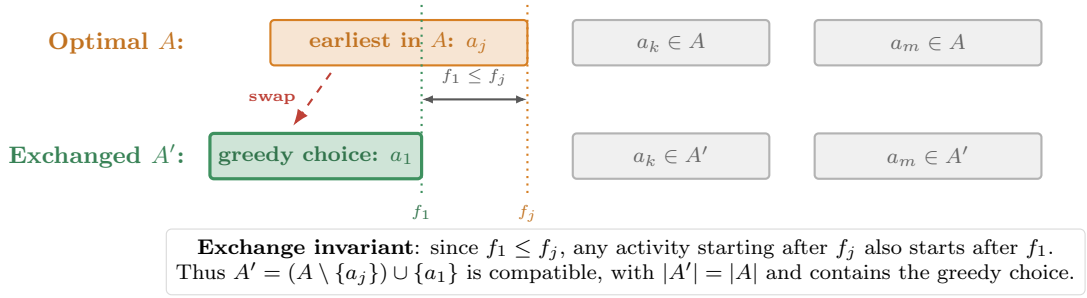


Figure 16.2: The greedy exchange argument. Given an optimal solution A whose first finishing activity is a_j, replacing a_j with the greedy choice a_1 preserves compatibility because $f_1 \le f_j$. All subsequent activities in A finish after $f_j \ge f_1$, so A' remains compatible and optimal while containing a_1.

The theorem handles the *first* choice. What about the rest? Once a_1 is fixed, every remaining activity that conflicts with it is useless, and the problem shrinks to "select a maximum set among the activities that start at or after f_1." That is a smaller instance of the *exact same problem*. So we apply the theorem again to the subproblem, and again, and again. This is an induction: each greedy choice is safe, and what remains is a smaller version of the original. That structure – *a greedy choice plus one subproblem* – is the skeleton of every argument in this chapter.

Common trap

It is tempting to "prove" greedy works by checking it on a few examples and declaring victory. Do not. The danger is that wrong greedy rules *also* pass small examples. The shortest-first rule, for instance, gives the right answer on plenty of inputs – and then quietly fails. Only an exchange argument or an explicit counterexample settles the question.

16.1.4 Breaking the wrong rules

To see how sensitive greedy correctness is to the exact rule, let us rule out two plausible alternatives with small counterexamples.

Shortest-duration-first. Consider three activities: $[1, 5)$, $[4, 6)$, $[5, 9)$. The shortest is $[4, 6)$ (length 2), so a shortest-first greedy grabs it – and now *both* other activities conflict with it, so it ends with one activity. But $[1, 5)$ and $[5, 9)$ are compatible: the optimal answer is two. Shortest-first loses.

Fewest-conflicts-first. You can build instances (CLRS Exercise 16.1-3 walks you through one) where always taking the activity that overlaps the fewest others also fails. The earliest-finish rule is special; near-misses are not good enough.

Walking question

Take a walk and try to *break* the earliest-finish rule. Sketch activities on a timeline and hunt for a case where finishing earliest leads you astray. You will not find one – but the act of failing to find one, and feeling *why* it always leaves the most room behind, will cement the proof better than reading it twice.

16.2 Greedy versus dynamic programming

You have now met two paradigms that both solve optimization problems by combining solutions to subproblems. They look similar on the page. When do you reach for which? Greedy is really a *special case* of dynamic programming that you are allowed to use only when the problem is unusually well-behaved. Let us pin down what "well-behaved" means.

Both paradigms require **optimal substructure**: an optimal solution to the whole contains optimal solutions to its subproblems. That is shared. The thing greedy demands *on top* is the **greedy-choice property**.

Focus

Greedy-choice property. You can assemble a globally optimal solution by making a *locally* optimal (greedy) choice and then solving the *one* subproblem that remains. Key point: the greedy choice is made *before* the subproblem is solved – it does not depend on the answers to any subproblems.

That last sentence is the key difference. In dynamic programming, the choice at the top *depends* on having already computed the subproblem answers – you compare alternatives and pick the best, which is why you need the table. In a greedy algorithm, you can decide the top-level move *up front*, with no table at all, because a simple local rule ("earliest finish") is guaranteed to be part of some optimal solution.

16.2.1 Two coins, one lesson

The cleanest way to feel the boundary is the *coin-change* problem. You owe someone n cents and want to pay with the fewest coins.

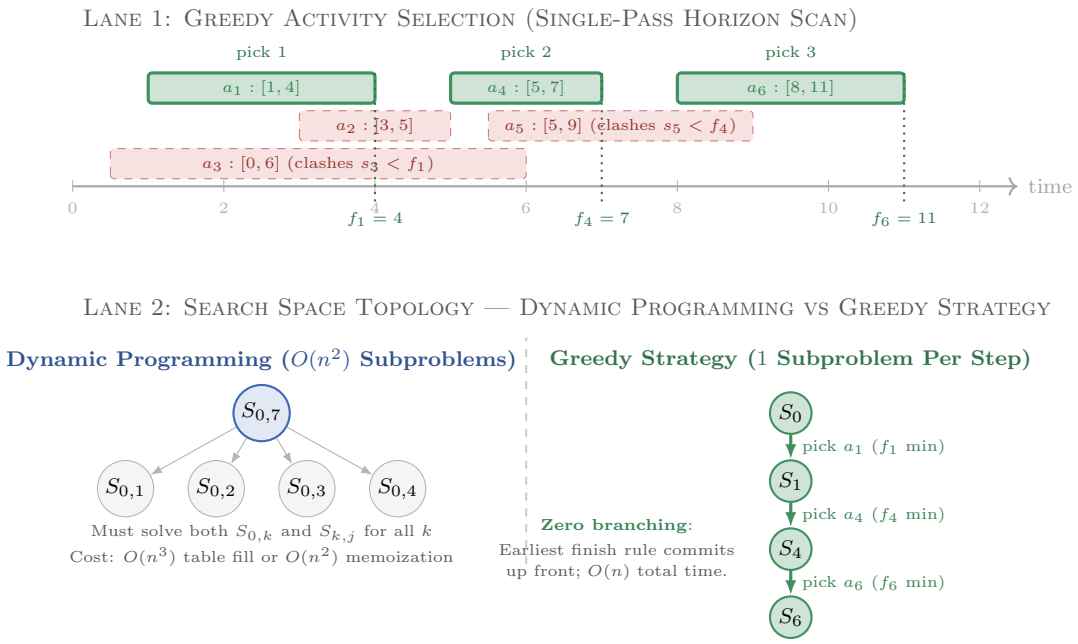


Figure 16.3: Contrasting the decision structure of Greedy Activity Selection with Dynamic Programming. In Lane 1, sorting activities by finish time allows the greedy choice (a_1, then a_4, then a_6) to sweep left-to-right, eliminating conflicting activities without looking ahead. In Lane 2, whereas a full dynamic program must evaluate choices across a branching web of $O(n^2)$ subproblems $S_{i,j}$, the greedy-choice property collapses the search space into a single linear descent path.

With U.S. denominations $\{1, 5, 10, 25\}$, the greedy rule – "hand over the largest coin that fits, repeat" – is optimal. To make 30 cents: a quarter (25), then a nickel (5). Two coins. You cannot do better.

Now change the coin system to $\{1, 3, 4\}$ and make 6 cents. Greedy grabs the 4, then needs 2 more, which costs two 1s: total *three* coins ($4 + 1 + 1$). But the optimal answer is $3 + 3$: *two* coins. Greed failed. Nothing about the algorithm changed – only the data did – and the greedy-choice property no longer held.

Common trap

The coin-change lesson is the most important warning in this chapter: whether greed is correct depends on the *structure of the input*, not on the shape of the algorithm. The same greedy code is optimal on $\{1, 5, 10, 25\}$ and wrong on $\{1, 3, 4\}$. You must *prove* the greedy-choice property for your specific problem; you cannot inherit it.

For the bad coin system, dynamic programming saves you. Let $C(n)$ be the fewest coins to make n cents. Then

$$C(n) = \min_{d \in \{1,3,4\},\ d \le n} \big(1 + C(n - d)\big), \qquad C(0) = 0,$$

which tries *every* first coin and keeps the best. Tracing $\{1, 3, 4\}$ at $n = 6$: $C(6) = \min(1 + C(5), 1 + C(3), 1 + C(2))$. With $C(3) = 1$ (one 3-coin), the middle option gives $1 + 1 = 2$. DP finds the $3 + 3$ answer that greed missed.

Filling the table from $n = 0$ upward makes the rescue concrete. Each cell is one plus the smallest reachable cell a single coin away, and Figure 16.4 shows the row in full.

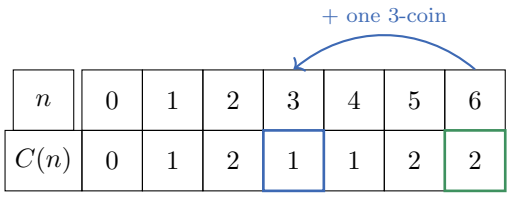


Figure 16.4: The coin-change DP table for the system $\{1, 3, 4\}$. Cell $C(6) = 2$ (green) is reached from $C(3) = 1$ (blue) by adding a single 3-coin, recovering the $3 + 3$ solution that greedy never considers.

Dynamic programming	**Greedy**
Choice depends on subproblem solutions; must compute them first.	Choice is made first, by a local rule, before any subproblem is solved.
Considers many choices, keeps the best (a min or max).	Commits to one choice and never revisits it.
Needs a table of size proportional to the subproblems.	Needs $O(1)$ extra state.
Always correct when there is optimal substructure.	Correct only when the greedy-choice property *also* holds.

Table 16.1: Greedy is DP with the freedom to choose first and never look back – a freedom you only earn by proving the greedy-choice property.

Why this matters

This distinction is not academic. A router deciding which packet to forward next, an operating system picking which job to run, a cache deciding what to evict – these systems make *millions* of decisions per second and cannot afford to build a DP table each time. They run greedy rules. Knowing when a greedy rule is provably optimal (and when it is merely a fast heuristic you must test) is the difference between a system that is correct by design and one that is correct by luck.

16.2.2 A recipe for designing greedy algorithms

CLRS distills the design process into a sequence of steps; here it is in plain language.

1. **Cast the problem as a choice.** Reframe "find the optimal solution" as "make one choice now, then you are left with a single subproblem to solve."
2. **Guess a greedy choice.** Propose a local rule – earliest finish, largest coin, smallest weight – that seems to make the most progress.
3. **Prove the greedy-choice property.** Show, usually by an exchange argument, that there is an optimal solution containing your greedy choice.
4. **Prove optimal substructure.** Show that after the greedy choice, an optimal solution to the rest combines with the choice to give a global optimum.
5. **Stitch it together by induction.** Greedy choice plus optimal subproblem, repeated, yields

a global optimum.

Most of the work is in steps 3 and 4. Skip them and you have a fast algorithm of unknown correctness.

16.3 Huffman codes: compression by greed

Huffman coding is a greedy algorithm you have used today without knowing it – it sits inside ZIP, JPEG, MP3, PNG, and the deflate format used throughout the web. It answers a simple question: given how often each symbol appears in a file, what is the *shortest* possible way to encode it?

16.3.1 Why variable-length codes

Suppose a file uses only six characters, a through f, with these frequencies (in thousands of occurrences):

character	a	b	c	d	e	f
frequency (thousands)	45	13	12	16	9	5

A *fixed-length* code gives every character the same number of bits. With six characters you need 3 bits each ($2^3 = 8 \geq 6$). Total cost: $(45 + 13 + 12 + 16 + 9 + 5) \times 3 = 100{,}000 \times 3 = 300{,}000$ bits.

But that is wasteful. The character a appears nine times as often as f, yet both get three bits. Surely a deserves a *shorter* codeword and f can afford a longer one. This is the idea of a *variable-length* code: frequent symbols get short codes, rare symbols get long ones. The catch is that the decoder, reading a stream of bits with no separators, must never get confused about where one codeword ends and the next begins.

Focus

Prefix-free codes. A code is *prefix-free* (CLRS just says "prefix code") if no codeword is a prefix of another. This single property lets the decoder parse the stream unambiguously: read bits until they form a codeword, emit that character, and start fresh. No codeword can be the start of another, so there is never a fork in the road.

For example, if $a =$ `0` and $b =$ `101` and $c =$ `100`, then reading `0101100` is unambiguous: `0` is a, then `101` is b, then `100` is c. There was never a moment of doubt, because no codeword began with another.

16.3.2 Codes as binary trees

The clean way to picture a prefix-free code is as a *binary tree*. Each leaf is a character. The path from the root to a leaf forms its codeword: go left, write `0`; go right, write `1`. Because characters live only at *leaves*, no character's path can pass *through* another character – which is exactly the prefix-free condition, drawn as a picture.

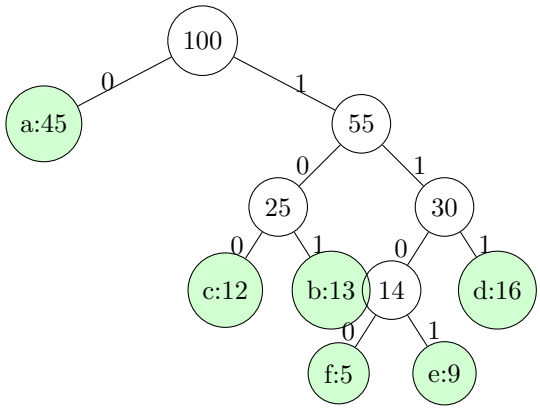


Figure 16.5: The optimal Huffman tree for the six-character file. Internal nodes show the combined frequency of their subtree; leaves (green) show character and frequency. Codewords: $a = \texttt{0}$, $c = \texttt{100}$, $b = \texttt{101}$, $f = \texttt{1100}$, $e = \texttt{1101}$, $d = \texttt{111}$.

The number of bits to encode the whole file is the sum, over all characters x, of (frequency of x) $\times$ (depth of x's leaf), because the depth is exactly the codeword length. Define the cost of a tree T as

$$B(T) = \sum_{x} f(x)\, d_T(x),$$

where $f(x)$ is the frequency and $d_T(x)$ is the leaf depth. Minimizing $B(T)$ over all prefix-free codes is precisely the compression problem. For the tree in Figure 16.5:

$$B = 45(1) + 13(3) + 12(3) + 16(3) + 9(4) + 5(4) = 45 + 39 + 36 + 48 + 36 + 20 = 224.$$

So this code uses 224,000 bits versus the fixed code's 300,000 – a 25% saving, achieved simply by assigning a one bit and giving f four.

16.3.3 Huffman's greedy idea

How do we *find* the cheapest tree? Huffman's insight, from a 1951 term paper, is to build the tree greedily *bottom-up*. The two rarest characters should sit deepest in the tree (longest codewords), so *merge them first*: make them siblings under a new internal node whose frequency is the sum of theirs. Now treat that merged node as a single "character" with the combined frequency, and repeat. Each step fuses the two lowest-frequency items into one, until a single tree remains.

Intuition

Think of it as a tournament run in reverse. The two *weakest* (rarest) players are knocked out first – buried deepest in the bracket – by being paired off and replaced by a team carrying their combined weight. You keep pairing the two weakest survivors. The last team standing is the root, and the depth at which a character was eliminated is the length of its codeword. Rare characters get eliminated early (deep), frequent ones survive late (shallow), which is what you want.

The natural data structure for "repeatedly extract the two smallest" is a *min-priority queue* (a min-heap, from Chapter 6). Here is the algorithm.

Algorithm 71: HUFFMAN(C)

Input: A set C of n characters, each with a frequency $f(c)$.
Output: The root of an optimal prefix-free coding tree.

```
1  n ← |C|;
2  Q ← a min-priority queue on C, keyed by frequency;
3  for i ← 1 to n − 1 do
4  |   allocate a new node z;
5  |   x ← EXTRACT-MIN(Q) ;                          // rarest remaining
6  |   y ← EXTRACT-MIN(Q) ;                          // second rarest
7  |   z.left ← x;   z.right ← y;
8  |   f(z) ← f(x) + f(y);
9  |   INSERT(Q, z) ;                  // the merged node rejoins the pool
10 return EXTRACT-MIN(Q) ;                  // the one node left is the root
```

16.3.4 A full build, traced

Run HUFFMAN on our six characters by hand. Start with the queue holding all six, sorted by frequency:

$$Q = [\, f{:}5,\ e{:}9,\ c{:}12,\ b{:}13,\ d{:}16,\ a{:}45 \,].$$

Step 1. Extract the two smallest: f:5 and e:9. Merge them under a new node z_1 with frequency $5 + 9 = 14$. Reinsert:

$$Q = [\, c{:}12,\ b{:}13,\ z_1{:}14,\ d{:}16,\ a{:}45 \,].$$

Step 2. Extract c:12 and b:13. Merge under z_2 with frequency 25. Reinsert:

$$Q = [\, z_1{:}14,\ d{:}16,\ z_2{:}25,\ a{:}45 \,].$$

Step 3. Extract z_1:14 and d:16. Merge under z_3 with frequency 30. Reinsert:

$$Q = [\, z_2{:}25,\ z_3{:}30,\ a{:}45 \,].$$

Step 4. Extract z_2:25 and z_3:30. Merge under z_4 with frequency 55. Reinsert:

$$Q = [\, a{:}45,\ z_4{:}55 \,].$$

Step 5. Extract a:45 and z_4:55. Merge under z_5 with frequency 100. This is the root.

$$Q = [\, z_5{:}100 \,].$$

Five merges for six leaves – exactly the $n - 1 = 5$ iterations the loop predicts. And the tree we just built, with z_5 as root, a hanging off it at depth 1, and f, e buried at depth 4, is *precisely* Figure 16.5. The greedy "merge the two rarest" rule reconstructed the optimal code without ever comparing it to an alternative.

Figure 16.6 lays the five steps out as a queue that shrinks by one item per merge: read each row left to right, watch the two smallest keys disappear and their sum reappear, until a single root key of 100 remains.

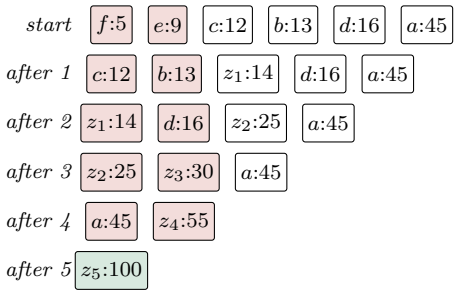


Figure 16.6: The priority queue across the five Huffman merges, one row per state. In each row the two red nodes are the smallest keys; they are extracted and merged, and their sum appears in the next row in sorted position (e.g. f:5 and e:9 become z_1:14). The queue shrinks by one each step until only the root z_5:100 (green) remains.

Algorithm lab

Re-trace the build but keep a running list of codewords. After step 1, f and e each have a tentative last bit ($f = \texttt{0}\ldots$, $e = \texttt{1}\ldots$); every later merge *prepends* a bit to everything in the merged subtree. Watch a stay a lone leaf until the very last merge, earning it the single bit `0`. This "prepend on merge" view is why the rarest characters accumulate the most bits.

16.3.5 Complexity

The loop runs $n - 1$ times. Each iteration does two Extract-Mins and one Insert on a heap of size at most n, and each heap operation costs $O(\log n)$. Building the initial heap costs $O(n)$. So the recurrence is just a sum:

$$T(n) = \underbrace{O(n)}_{\text{build heap}} + (n-1) \cdot \underbrace{O(\log n)}_{\text{per merge}} = O(n \log n).$$

Cost summary

Huffman runs in $O(n \log n)$ time using a binary min-heap, where n is the number of distinct characters. With a more clever structure (keeping the input pre-sorted and using two queues), it can be done in $O(n \log n)$ for the sort plus $O(n)$ for the merging – but the heap version is the one to remember. The output tree has n leaves and $n - 1$ internal nodes, so it uses $O(n)$ space.

16.3.6 Why Huffman is optimal

We must prove that greed is optimal here too. The proof rests on two lemmas, both using the same kind of exchange argument that worked for activity selection.

Lemma 16.1 (The greedy choice is safe)**.** *Let x and y be the two characters with lowest frequency. Then there is an optimal prefix-free code in which x and y have codewords of the same length that differ only in the last bit – that is, x and y are sibling leaves at the* maximum depth *of the tree.*

Formal

Proof sketch. Take any optimal tree T. Let b and c be two sibling leaves at maximum depth (a deepest internal node has two leaf children – it must, or the tree could be improved). Without loss of generality $f(b) \le f(c)$, and since x, y are the two least frequent overall, $f(x) \le f(b)$ and $f(y) \le f(c)$.
Now *swap* x with b to form T', then swap y with c to form T''. The change in cost from the first swap is

$$B(T) - B(T') = \big(f(b) - f(x)\big)\big(d_T(b) - d_T(x)\big) \ge 0,$$

because $f(b) \ge f(x)$ (the swapped-in character is rarer) and $d_T(b) \ge d_T(x)$ (b was at maximum depth). So the swap does not *increase* cost. The same holds for the second swap. Hence $B(T'') \le B(T)$, but T was optimal, so $B(T'') = B(T)$ and T'' is also optimal – and in it, x and y are deepest siblings. ■

The intuition behind that inequality drives the whole algorithm: *putting a rarer character deeper never costs more.* If a frequent character sits deep and a rare one sits shallow, swapping them can only help, because you pay the deep (long) codeword on fewer occurrences. The least frequent characters *belong* at the bottom.

Lemma 16.2 (Optimal substructure)**.** *Let z be the merged node replacing siblings x and y, with $f(z) = f(x) + f(y)$. If T' is an optimal tree for the reduced alphabet (with z in place of x, y), then the tree T obtained from T' by re-splitting z into its children x and y is optimal for the original alphabet.*

Formal

Proof idea. The two trees' costs differ by a fixed, choice-independent amount. When we split z at depth d into x and y at depth $d+1$, the cost changes by

$$B(T) - B(T') = \big(f(x) + f(y)\big)(d{+}1) - \big(f(x) + f(y)\big)d = f(x) + f(y),$$

a constant. So minimizing $B(T')$ is the same as minimizing $B(T)$: an optimal reduced tree gives an optimal full tree. Combining this with Lemma 16.1 (the merge was a safe choice) and inducting on the alphabet size proves HUFFMAN optimal. ■

History and connection

David Huffman invented optimal prefix coding (a code where no character's bit string begins another character's bit string) in 1951 as a graduate student at MIT. His professor, Robert Fano, offered students a choice between a final examination and writing a term paper on finding the most efficient binary code for an alphabet with known frequencies. Claude Shannon and Fano had developed a top-down heuristic (Shannon–Fano coding) that split probabilities as equally as possible, but it was not always optimal. After weeks of failed attempts, Huffman was on the verge of throwing his notes into the trash to study for the exam when the breakthrough struck him: instead of splitting top-down, build bottom-up by repeatedly pairing the two least frequent characters. Two decades later, Jack Edmonds (1971) unified greedy algorithms through matroid theory (an algebraic framework formalizing the concept of independence, building on Hassler Whitney's 1935 foundations), proving that greedy optimization succeeds across all weight functions if and only if the underlying structure forms a matroid.
The main trap in greedy design is that greedy algorithms look simple. Because greedy al-

gorithms produce correct, intuitive solutions on certain problems (like fractional knapsack or minimum spanning trees), engineers frequently apply them to problems where they fail catastrophically. In 0-1 knapsack, greedily picking items with the highest value-to-weight ratio can yield arbitrarily poor solutions. Proving greedy correctness strictly requires either an exchange argument or verification of the matroid axioms; intuition alone is unreliable.
Huffman coding provided optimal data compression that became foundational to formats like JPEG, MP3, ZIP, and DEFLATE, while greedy scheduling and spanning tree algorithms power network routing. Yet greedy algorithms are brittle: small variations in problem constraints (such as shifting from unit-time tasks to arbitrary deadlines or adding capacities) almost always destroy the greedy-choice property, forcing a fallback to dynamic programming or exponential search. Determining tight approximation bounds when constraints intersect multiple matroids remains an active area of research in combinatorial optimization.

16.4 Why does greed work? A glimpse of matroids

You now have two greedy success stories and one greedy failure (the $\{1, 3, 4\}$ coins). It would be wonderful to have a *single* theorem that says, "If your problem has this abstract shape, greed is guaranteed optimal – proof not required." Such a theorem exists for a large family of problems. The abstraction is called a *matroid*, and it is a central idea in combinatorics.

16.4.1 The pattern behind the proofs

Look back at activity selection and Huffman. Both proofs had the same two moving parts: a notion of a partial solution you can *extend* (a compatible set you can add to; a tree you can keep merging), and an *exchange* step (swap the greedy choice in without loss). A matroid is an abstract structure that guarantees both parts work.

Definition 16.1 (Matroid)**.** A *matroid* is a pair $M = (S, \mathcal{I})$ where S is a finite set and $\mathcal{I}$ is a collection of subsets of S – the *independent* sets – satisfying:

1. **(Nonempty)** $\emptyset \in \mathcal{I}$.
2. **(Hereditary)** If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$. Every subset of an independent set is independent.
3. **(Exchange)** If $A, B \in \mathcal{I}$ and $|A| < |B|$, then there is some element $x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$. A smaller independent set can always be grown using an element from any larger one.

The word "independent" is borrowed from linear algebra, and that is the canonical example: let S be a set of vectors and let $\mathcal{I}$ be the collection of *linearly independent* subsets. Property 2 holds (a subset of independent vectors is independent), and property 3 is a classic fact of linear algebra (a smaller independent set can be extended toward any basis). That is where the whole vocabulary – "independent," "rank," "basis" – comes from.

Intuition

Informally, a matroid is a setting where "a valid partial solution" (an independent set) has two friendly properties: you can always shrink it and stay valid (hereditary), and if your partial solution is smaller than someone else's, you can always borrow one of their elements to grow yours (exchange). Those are exactly the two things an exchange-argument correctness proof needs, so checking the axioms once gives you the proof.

16.4.2 The greedy theorem for matroids

In a *weighted* matroid, each element $x \in S$ carries a positive weight $w(x)$, and we want a *maximum-weight* independent set. The greedy algorithm is the obvious one: sort elements by weight, heaviest first, and add each to your set if it keeps the set independent.

Algorithm 72: GREEDY(M, w)

Input: A matroid $M = (S, \mathcal{I})$ and a weight function w.
Output: A maximum-weight independent set A.

```
A ← ∅;
sort S into nonincreasing order by weight w;
for each x ∈ S taken in that order do
    if A ∪ {x} ∈ I then
        A ← A ∪ {x};
return A
```

Theorem 16.2 (Greedy is optimal on any matroid)**.** *If $M = (S, \mathcal{I})$ is a matroid with positive weights w, then* GREEDY(M, w) *returns a maximum-weight independent set.*

We will not grind through the full proof – it is essentially the exchange argument from the activity-selection section, lifted to the abstract setting, and CLRS gives it in full. But notice what this theorem *buys* you. Activity selection, minimum spanning trees (Chapter 23's Kruskal algorithm is greedy on the *graphic matroid* of a graph), and a host of scheduling problems are all *instances* of weighted matroids. For each, you do not need to invent a new exchange argument: you check the three matroid axioms once, and Theorem 16.2 hands you optimality for free.

Why this matters

This is the payoff of abstraction. The first time you prove a greedy algorithm correct, it feels like a clever, problem-specific trick. The matroid theorem shows that it is an instance of a general structural result. When you later meet Kruskal's minimum-spanning-tree algorithm and someone says "it's just greedy on the graphic matroid," you will understand both *why* it is correct and why you do not have to re-derive its proof from scratch. Good abstractions turn many proofs into one.

Remark 16.1. Not every greedy success is a matroid. Huffman coding, for instance, is not most naturally phrased as a weighted matroid; its optimality genuinely needs the two lemmas we proved. Matroids capture a *large* class of greedy-friendly problems, not *all* of them. A broader generalization (greedoids, and the theory of submodular functions) pushes further, but those are beyond this book. In short, matroids explain a great deal of greed, and where they apply, they apply cleanly.

Walking question

Here is a matroid to verify on your walk. Let S be the edges of a connected graph, and let $\mathcal{I}$ be all subsets of edges that contain *no cycle* (the "forests"). Convince yourself the hereditary property holds – remove an edge from a forest and it is still a forest. The exchange property is subtler: if forest A has fewer edges than forest B, why must some edge of B extend A without making a cycle? Sketch a small graph and feel it out. You will have discovered the *graphic matroid*, the structure that makes Kruskal's algorithm correct when we study minimum spanning trees in Chapter 23.

16.5 Summary

Greed is the strategy of committing to the locally best choice and never looking back. It is much faster and simpler than dynamic programming *when it works* – and it works exactly when a problem has the greedy-choice property on top of optimal substructure. The recurring proof technique is the *exchange argument*: show any optimal solution can be edited to contain the greedy choice without getting worse.

We saw greed succeed for activity selection (earliest finish wins, $\Theta(n)$ after sorting) and for Huffman coding (merge the two rarest, $O(n \log n)$), and we saw it fail for coin change under the $\{1, 3, 4\}$ system, where dynamic programming was required. Finally, matroids gave us a glimpse of *why* greed works at all: for a whole family of problems, three abstract axioms guarantee optimality, turning many separate exchange proofs into a single theorem you can reuse. Keep that lens in mind; we will see the graphic matroid in action when we study minimum spanning trees in Chapter 23.

Greedy algorithms and dynamic programming both tackle optimization problems by exploiting optimal substructure. But whether we commit greedily or evaluate subproblems across a table, we have so far measured the cost of an algorithm by its worst-case performance on a single operation.

In many real systems, however, individual operations vary wildly. A dynamic array might append in $O(1)$ time for a thousand steps, then suddenly freeze for an $O(n)$ resizing pass before returning to $O(1)$ appends. Calling every append $O(n)$ because of that rare spike is overly pessimistic: over any sequence of n operations, the total work is only $O(n)$, averaging $O(1)$ per operation. In **Chapter 17 (Amortized Analysis)**, we develop three formal methods – the aggregate method, the accounting method, and the potential method – to analyze operations that are individually erratic but collectively cheap.

Exercises

Exercise 16.1. Run Greedy-Activity-Selector by hand on the activities (given as (s, f) pairs) $(1, 3)$, $(2, 5)$, $(4, 7)$, $(1, 8)$, $(5, 9)$, $(8, 10)$, $(9, 11)$. List the chosen set and its size. (Remember to sort by finish time first.)

Exercise 16.2. Give a concrete counterexample showing that the "pick the activity with the earliest *start* time" greedy rule can be suboptimal. Three activities are enough.

Exercise 16.3. For U.S. coins $\{1, 5, 10, 25\}$, prove that the greedy "largest coin first" rule is optimal for making any amount n. (Hint: argue about how many of each smaller coin an optimal solution

can contain before it would be cheaper to use a larger one.)

Exercise 16.4. Find the smallest amount n for which greedy coin change fails under the system $\{1, 4, 6\}$, and exhibit both the greedy answer and a strictly better one.

Exercise 16.5. Build the Huffman tree for the frequencies a:10, b:15, c:30, d:16, e:29. Show the priority queue after each merge, draw the final tree, and compute the total encoded length $B(T)$.

Exercise 16.6. Suppose all n characters have *equal* frequency. What shape does the Huffman tree take, and what are the codeword lengths? Relate your answer to the fixed-length code. (Consider separately the cases where n is and is not a power of two.)

Exercise 16.7. Prove that in any optimal prefix-free code, the character with the highest frequency has a codeword of length 1 *or* shares the minimum codeword length with at least one other character. (Hint: exchange argument, mirroring Lemma 16.1.)

Exercise 16.8. A Huffman tree is built greedily; suppose instead you (wrongly) always merge the two *most* frequent items. On the six-character file of this chapter, build that tree and compute its cost. By how many bits does it lose to the optimal 224,000?

Exercise 16.9 (Matroid check)**.** Let S be a finite set and let $\mathcal{I}$ consist of all subsets of S of size at most k, for a fixed integer k. Verify that $(S, \mathcal{I})$ satisfies the three matroid axioms. (This is the *uniform matroid*; greedy on it just picks the k heaviest elements.)

Exercise 16.10 (Challenge)**.** You have n tasks; task i takes time t_i on a single machine. If you run them in some order, the *completion time* of a task is when it finishes, and you want to minimize the *sum* of all completion times. Propose a greedy rule, and prove it optimal with an exchange argument. (Hint: think about which task you would never want to schedule last.)

Exercise 16.11 (Challenge)**.** Prove the optimal-substructure direction for activity selection rigorously: if A_{ij} denotes a maximum-size compatible set drawn entirely from the activities that start after a_i finishes and finish before a_j starts, and if $a_k \in A_{ij}$, show that $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ where A_{ik} and A_{kj} are themselves optimal for their sub-ranges. Use a cut-and-paste argument.

Chapter 17

Amortized Analysis

Across our study of dynamic programming and greedy choices, we analyzed algorithms where each step was bounded by a uniform worst-case cost. But when designing dynamic data structures, a different pattern emerges. You build a data structure. You measure one operation on it and find that, every so often, that operation is *slow* – it does a big burst of work, touching a large chunk of the structure. Naively, you write down "worst case per operation: expensive" and conclude the whole sequence of operations must be expensive. And yet, when you actually run a long sequence of these operations and divide the total time by the number of operations, the average comes out *cheap* – often constant. The slow operations are real, but they are so rare, and the cheap ones so common, that the expensive bursts are paid for by everything around them.

Amortized analysis is the mathematical framework for making that intuition precise. It lets you charge a sequence of operations fairly – not by pretending the slow ones do not happen, but by spreading their cost over the many cheap operations that surround them. The payoff is that you can prove a tight bound on the *total* cost of a sequence without ever invoking probability. There is no "on average over random inputs" here. The bound holds for *every* sequence, in the worst case, which is what makes it a guarantee you can actually rely on.

We will meet three techniques – the *aggregate*, *accounting*, and *potential* methods – and run each of them on the same three examples: a stack with a bulk-pop operation, a binary counter you increment, and a dynamic table that doubles itself when it fills up. By the end you will be able to look at a bursty data structure and say, calmly, "yes, but amortized it's constant time, and here is the proof."

17.1 Worst case per operation versus worst case per sequence

Let us be very careful about what question we are asking, because the entire subject rests on this distinction.

When we analyzed sorting, we asked: what is the worst-case running time of *one call* to this algorithm on an input of size n? That is a perfectly good question. But data structures are different. You rarely call a data structure once. You hammer it with a long *sequence* of operations – push, push, pop, push, search, insert, delete – and what you actually care about is the total time for the whole sequence.

Intuition

Think about your monthly expenses. Most days you spend almost nothing. But once a month the rent comes due, and that single day is enormous. If a visitor watched you on rent day, they would scream "you can't afford this lifestyle!" But you can, because you have all those cheap days to balance it out. Your *average daily spend* is modest. Amortized cost is exactly this average daily spend, computed across the whole month – and computed so that it is a true *upper bound* no matter which days the visitor happens to look at.

Here is the key definition, stated plainly.

Definition 17.1 (Amortized cost)**.** Suppose a sequence of n operations runs on a data structure and the total actual cost of the whole sequence is $T(n)$. We say each operation has *amortized cost* at most $\hat{c}$ if, for *every* such sequence,

$$T(n) \;\le\; \hat{c} \cdot n.$$

More generally we may assign different amortized costs $\hat{c}_i$ to different operations i, and the requirement is that the sum of amortized costs is an upper bound on the total actual cost: $\sum_{i=1}^{n} c_i \le \sum_{i=1}^{n} \hat{c}_i$, where c_i is the actual cost of the ith operation.

That last inequality is the core of the method. The amortized costs are *bookkeeping numbers we invent*. We are free to choose them however we like, as long as one rule holds: their running sum must always be at least the running sum of the real costs. If we can do that, then the total amortized cost is a genuine upper bound on the total real cost, and we have permission to use the amortized cost as if it were the per-operation cost.

Focus

Amortized cost is not a measured quantity. It is an *accounting choice* that you must *justify*. The justification always reduces to one inequality: across any prefix of the sequence, total amortized cost $\ge$ total actual cost. Get that inequality and you may quote the amortized cost freely.

Common trap

Amortized is *not* average-case, and it is *not* expected. There is no randomness anywhere in this chapter. An amortized bound is a worst-case guarantee about a *sequence*; an average-case bound is a probabilistic statement about a *single* operation over random inputs. A randomized algorithm can have a great expected cost and still, on one unlucky run, blow up. An amortized bound never blows up over the sequence: it holds for every sequence, every time, with no luck involved.

We will now develop three methods that all establish bounds of this form. They prove the same kinds of theorems but use three different *styles of argument*, and different examples feel natural in different styles. A good engineer keeps all three in the toolbox.

17.2 The aggregate method: just add it all up

The aggregate method is the bluntest of the three. You do not try to be clever about individual operations. You ask one question: across the whole sequence of n operations, what is the total work

$T(n)$? Then you divide by n and call the result the amortized cost per operation. That is it.

Intuition

The aggregate method is the "total the receipt, then split the bill evenly" approach. You do not care who ordered the lobster and who ordered the salad. You sum the entire tab and divide by the number of diners. If the total is small, the per-person share is small, even if one dish was expensive.

It sounds too simple to be useful, but for many structures the total $T(n)$ has a clean closed form that is much smaller than (worst-case-per-operation) $\times n$. Let us see this on our first running example.

17.2.1 Example: the stack with Multipop

Start with an ordinary stack. It supports two cheap operations: PUSH(S, x) puts x on top, and POP(S) removes and returns the top. Each costs 1 unit (one object moved). Now we add a third, bulkier operation.

Algorithm 73: MULTIPOP(S, k)

Input: A stack S and a count $k \geq 0$.
Output: The top $\min(k, |S|)$ elements are popped off.

```
1 while S is not empty and k > 0 do
2     POP(S);
3     k ← k − 1;
```

MULTIPOP just pops repeatedly until either it has popped k items or the stack runs dry. Its actual cost is the number of POP calls it makes, which is $\min(k, |S|)$.

Now here is the trap a naive analysis falls into. A single MULTIPOP can be expensive: if the stack holds n items and you call MULTIPOP(S, n), it does n units of work. So the worst-case cost of one operation is $\Theta(n)$. A sequence of n operations, naively bounded, looks like it could cost $n \cdot \Theta(n) = \Theta(n^2)$.

Common trap

That $\Theta(n^2)$ bound is correct – it is a valid upper bound – but it is wildly loose. It assumes *every* operation can be the expensive one. It cannot. A MULTIPOP that pops 100 items requires that 100 items were pushed earlier, and pushing them already cost 100 units. You cannot pop what you never pushed. The naive bound double-counts a constraint that the structure enforces for free.

Let us total it up properly. Consider any sequence of n operations, each one a PUSH, POP, or MULTIPOP, starting from an empty stack. The decisive observation is this:

Focus

Every object can be popped *at most once for each time it is pushed.* The total number of pops (counting each pop inside a MULTIPOP individually) can therefore never exceed the total number of pushes.

How many pushes can there be in a sequence of n operations? At most n, since each operation is a single PUSH at most. So the total number of pushes is $\leq n$, and therefore the total number of pops – across all POP and MULTIPOP calls combined – is also $\leq n$. Each push costs 1 and each pop costs 1, so:

$$T(n) \;=\; (\text{total pushes}) + (\text{total pops}) \;\leq\; n + n \;=\; 2n \;=\; O(n).$$

Divide by n: the amortized cost per operation is at most $2n/n = 2 = O(1)$.

Why this matters

Notice how much the bound improves: from a naive $\Theta(n^2)$ down to a tight $\Theta(n)$ total, just by counting pushes and pops globally instead of bounding each operation separately. The expensive MULTIPOPs are real, but they are *funded* by the pushes that came before them. This same "you can only take out what you put in" accounting underlies the analysis of countless real systems: memory allocators, log compaction, garbage collectors. Whenever work is bounded by some resource that had to be created earlier, amortized analysis usually applies.

Algorithm lab

Trace it yourself. Start with an empty stack and run this exact sequence of 8 operations, writing down the stack and the actual cost after each: PUSH(a), PUSH(b), PUSH(c), MULTIPOP(2), PUSH(d), PUSH(e), MULTIPOP(5), PUSH(f). Add the actual costs (you should get 3 pushes then a 2-pop, 2 pushes, a 3-pop because only 3 items are present, then 1 push: $1+1+1+2+1+1+3+1 = 11$). Confirm $11 \leq 2 \cdot 8 = 16$. Notice the 5-multipop only cost 3: it could not pop what was not there.

17.2.2 Example: the binary counter

Our second running example will follow us through all three methods, so meet it now. We keep a k-bit binary counter in an array $A[0\,..\,k-1]$, where $A[0]$ is the lowest-order bit. The only operation is INCREMENT, which adds 1.

Algorithm 74: INCREMENT(A)

Input: A binary counter stored in bits $A[0\,..\,k-1]$, low bit first.
Output: A holds the value increased by 1 (modulo 2^k).

```
1 i ← 0;
2 while i < k and A[i] = 1 do
3 |   A[i] ← 0 ;                          // a carry:  flip a 1 down to 0
4 |   i ← i + 1;
5 if i < k then
6 |   A[i] ← 1 ;                          // set the lowest 0 bit to 1
```

This is just grade-school carrying in base two. To add 1, you flip the trailing run of 1s to 0s, then set the next 0 to a 1. The cost of one INCREMENT is the number of bits it flips.

How bad can one INCREMENT be? When the counter reads $0111\cdots1$ (all low bits set), incrementing flips *all* of them: up to k bit-flips. So the worst case of a single operation is $\Theta(k)$, and a naive bound

on n increments is $n \cdot \Theta(k) = \Theta(nk)$. Let us see how loose that is.

The aggregate insight is to ask not "how many bits does one increment flip?" but "how many times, total, does each *individual bit* flip across n increments?" Watch the bits.

- Bit $A[0]$ flips on *every* increment: $1, 0, 1, 0, \ldots$. So it flips n times.
- Bit $A[1]$ flips on every *second* increment: it changes once for every two ticks of $A[0]$. So it flips $\lfloor n/2 \rfloor$ times.
- Bit $A[2]$ flips every *fourth* increment: $\lfloor n/4 \rfloor$ times.
- In general, bit $A[i]$ flips $\lfloor n/2^i \rfloor$ times.

Figure 17.1 makes this halving-per-column pattern visible for a small counter.

Intuition

A binary counter is a row of toggle switches geared like an odometer. The ones digit spins fast and flips constantly. The tens digit barely moves. The highest-order switch flips only on the rare grand rollover. Most of the bits rarely move, so the total flipping work is dominated by the low bits, and each of their flips is *cheap*.

value	A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0

Figure 17.1: A 4-bit counter climbing $0 \to 8$. Shaded cells are the bits that flipped on that increment. The low bit A_0 flips every step; A_1 every two steps; A_2 every four; A_3 only on the big rollover at $7 \to 8$. Each column flips half as often as the one to its right, which gives the geometric series $n + n/2 + n/4 + \cdots < 2n$.

Now add up the total number of flips across all bits:

$$T(n) \;=\; \sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor \;<\; \sum_{i=0}^{\infty} \frac{n}{2^i} \;=\; n \sum_{i=0}^{\infty} \frac{1}{2^i} \;=\; n \cdot 2 \;=\; 2n.$$

That geometric series $1 + \frac{1}{2} + \frac{1}{4} + \cdots = 2$ gives the bound. The total flip count over n increments is less than $2n$, so:

$$T(n) = O(n), \qquad \text{amortized cost per } \textsc{Increment} = \frac{T(n)}{n} < 2 = O(1).$$

Why this matters

The naive bound was $\Theta(nk)$; the truth is $\Theta(n)$. The factor of k disappears, because the high bits almost never move. This is the same reason that incrementing a 64-bit loop counter is effectively free even though, once in a great while, an increment ripples through dozens of carries. Because flip frequencies halve with each higher bit, the geometric series ensures that all the rare carries combined cost no more than the common low-bit flips.

Algorithm lab

Build the table by hand. Start at 0000 and increment a 4-bit counter 16 times back to 0000. After each increment, write the counter value and the number of bits that flipped. You should see the cost pattern $1, 2, 1, 3, 1, 2, 1, 4, 1, 2, 1, 3, 1, 2, 1, 4$. Sum it: you get 30. Check that this is the total flips $\lfloor 16/1 \rfloor + \lfloor 16/2 \rfloor + \lfloor 16/4 \rfloor + \lfloor 16/8 \rfloor = 16+8+4+2 = 30$, and that $30 < 2 \cdot 16 = 32$. Notice the "4" costs appear only at the rollovers $7 \to 8$ and $15 \to 0$ – exactly twice.

The aggregate method works well when a clean global total exists. But sometimes you want to assign *different* amortized costs to different operations, or you want a more local, more flexible argument. That is where the next two methods come in.

17.3 The accounting method: prepay and bank the credit

The accounting method uses a bookkeeping device: we charge each operation an amortized cost that may differ from its real cost, and we treat the difference as *currency*. If we charge an operation more than it actually costs, the surplus is stored in the data structure as *credit*. If an operation costs more than we charge it, it pays the difference by spending stored credit.

Intuition

Imagine each object in your structure carries a little prepaid debit card. When you do a cheap operation, you overcharge it slightly and load the extra onto the card. Later, when an expensive operation happens, it does not reach into your wallet – it swipes the cards that were prepaid earlier. As long as no card ever goes negative, you know the prepaid charges covered all the real work. The total you charged is then a valid upper bound on the total real cost.

The single rule that makes this rigorous:

Focus

The bank balance – total credit stored in the structure – must *never go negative*. Equivalently, at every prefix of the sequence, the total amount charged (amortized) must be at least the total real cost. If credit can never go negative, then total charged $\geq$ total spent, which is exactly Definition 17.1. The credit you store represents prepaid future work, so it makes physical sense that it cannot be negative – you cannot prepay a debt you have not funded.

The hard part of the accounting method is *choosing the charges*. You want simple, constant charges that nevertheless keep the bank solvent. Let us pick charges for our two examples.

17.3.1 Accounting for the stack

We assign amortized charges as follows:

- PUSH: charge 2. Real cost is 1. The extra 1 stays as credit *on the pushed object*.
- POP: charge 0. Real cost is 1. Pay the 1 with the credit sitting on the object being popped.
- MULTIPOP: charge 0. Real cost is the number of items it pops; each pop is paid by the credit on the object it removes.

Here is the invariant that makes the scheme work: *every object currently on the stack carries exactly* \$1 *of credit*. Why is this maintained? When you push an object, you charged \$2: \$1 pays the real push cost, and \$1 rides onto the object. So freshly pushed objects satisfy the invariant. When you pop an object (whether via POP or inside MULTIPOP), the object carries exactly the \$1 needed to pay for its own removal. The credit is consumed precisely when the object leaves. Figure 17.2 shows the coins in action across a push burst followed by a MULTIPOP.

Intuition

Every object walks onto the stack already holding the coin it will need to walk back off. PUSH hands it that coin. Whenever it is popped – now, later, in bulk, it does not matter – it pays its own way out. No pop ever finds an empty pocket, so the bank never goes negative.

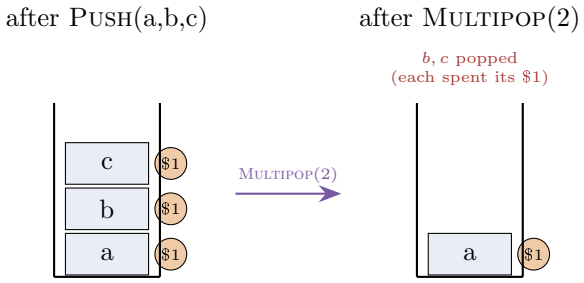


Figure 17.2: The accounting view of the stack. Each PUSH overcharges by \$1 and parks that coin on the object (left). When MULTIPOP(2) removes c and b, each pays for its own removal out of the coin it has been carrying (right). The bank – the total of all parked coins – equals the number of items on the stack and never dips below zero.

Since the bank never goes negative, the total charged is an upper bound on total real cost. The amortized cost of each operation is at most 2 (the most we ever charge), so n operations cost at most $2n = O(n)$. Same bound as the aggregate method, reached by a completely different argument – and notice this argument is more *local*: it never needed to know the length of the sequence.

17.3.2 Accounting for the binary counter

For the counter, the trick is to ask: which bit-flips are expensive, and which can prepay them? An INCREMENT flips a run of 1s down to 0, then sets one 0 up to 1. The flips $1 \to 0$ can be arbitrarily numerous (that is the carry chain), but a $0 \to 1$ flip – setting a bit – happens *exactly once* per INCREMENT. That asymmetry is what makes the accounting work.

We charge:

- Setting a bit ($0 \to 1$): charge \$2. One dollar pays the real flip; the other dollar rides onto the bit as credit.

- Clearing a bit ($1 \to 0$): charge \$0. Pay its real flip with the \$1 credit that the bit has been carrying since it was set.

Invariant: *every bit that currently equals* 1 *carries exactly* \$1 *of credit.* When we set a bit to 1, we loaded \$1 onto it. When a carry later clears that bit to 0, the stored dollar pays for the clear. Because every INCREMENT sets at most one bit to 1, every INCREMENT is charged at most \$2. Therefore n increments cost at most $2n = O(n)$, and the amortized cost per INCREMENT is at most $2 = O(1)$.

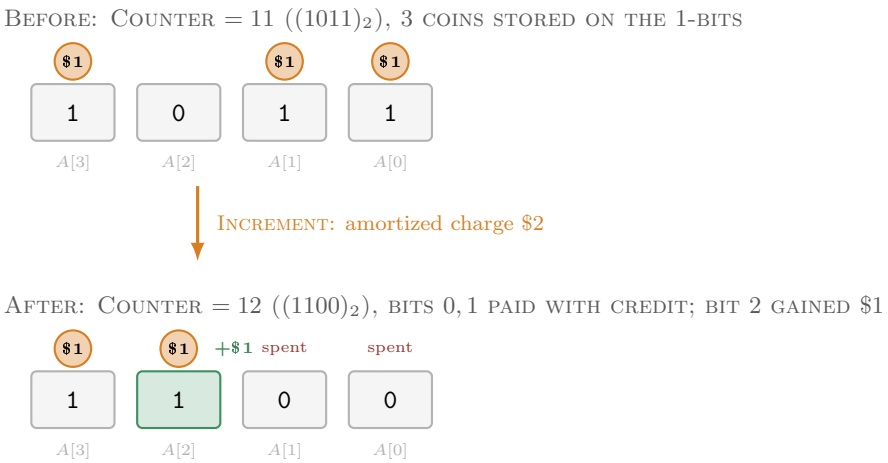


Figure 17.3: The accounting method on a binary counter. Every bit equal to 1 maintains exactly \$1 of stored credit. Setting a bit ($0 \to 1$) incurs an amortized charge of \$2: \$1 pays for the physical flip, and \$1 is banked on the bit. Clearing a bit ($1 \to 0$) is charged \$0, paid for by releasing the banked dollar. The bank balance equals the number of 1-bits, preventing the bank from ever going negative.

Focus

The same pattern appears in both: charge \$2 for the one cheap, guaranteed, once-per-operation action (a push, a bit-set), and let the prepaid dollar fund the unbounded-looking burst later (a multipop, a carry chain). The structure of the argument is identical even though the structures are different.

Walking question

Take this on a walk. In the counter argument we charged the *set* and let the *clears* be free. Could we instead charge each clear and make sets free? Try to define such a scheme and find where the bank goes negative. What is special about "at most one set per increment" that makes the set the right thing to charge? When you see why one direction works and the other fails, you understand the accounting method.

17.4 The potential method: bottle the credit into a single number

The accounting method spread credit across individual objects – a dollar on each stacked item, a dollar on each set bit. The *potential method* is the same idea, viewed globally. Instead of tracking who holds which coin, we summarize the *entire* stored credit as one number: the *potential* Φ of the whole data structure. As the structure evolves, Φ rises and falls, and that rise and fall is exactly the bookkeeping.

Intuition

Think of potential energy in physics. Lift a weight and you store energy; the storing costs you effort now. Let it drop and that stored energy does work for free later. The data structure's potential Φ is precisely this: a measure of "stored-up future work." Cheap operations that build up structure raise Φ (you pay a little extra now); the rare expensive operation that tears that structure down releases Φ (it pays for itself out of the reservoir).

Here is the precise machinery. Let D_0 be the starting state and D_i the state after the ith operation. We pick a *potential function* Φ mapping each state to a real number. The amortized cost of the ith operation is *defined* as its actual cost plus the change in potential it causes:

$$\boxed{\hat{c}_i \;=\; c_i \;+\; \Phi(D_i) - \Phi(D_{i-1}).}$$

Why is this the right definition? Sum the amortized costs over the whole sequence and watch the potentials telescope:

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} \big(c_i + \Phi(D_i) - \Phi(D_{i-1})\big) = \left(\sum_{i=1}^{n} c_i\right) + \Phi(D_n) - \Phi(D_0).$$

Every intermediate $\Phi(D_i)$ cancels: $+\Phi(D_1)$ from the first term against $-\Phi(D_1)$ from the second, and so on. Only the endpoints survive. Rearranging, the total *actual* cost is

$$\sum_{i=1}^{n} c_i \;=\; \sum_{i=1}^{n} \hat{c}_i \;-\; \Phi(D_n) + \Phi(D_0).$$

Formal

If we choose Φ so that $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i, then $-\Phi(D_n) + \Phi(D_0) = -\Phi(D_n) \leq 0$, and therefore

$$\sum_{i=1}^{n} c_i \;\leq\; \sum_{i=1}^{n} \hat{c}_i.$$

So the total amortized cost is an upper bound on the total actual cost – exactly the requirement of Definition 17.1. The two conditions "Φ starts at 0" and "Φ never goes negative" are the potential-method analogue of "the bank never goes negative."

Focus

The potential is just the bank balance from the accounting method, packaged as a single function of the state. Conditions $\Phi(D_0) = 0$ and $\Phi \geq 0$ say exactly "start with no credit, and never go into debt." In other words, the potential method is the accounting method with the entire credit ledger summed into a single number.

17.4.1 Potential for the stack

Define $\Phi(D)$ = the number of objects currently on the stack. Initially the stack is empty so $\Phi(D_0) = 0$, and a count of objects is never negative, so $\Phi \geq 0$ always. Both conditions hold. Now compute the amortized cost of each operation as "actual cost plus change in Φ."

Push. Actual cost 1. The stack grows by one object, so Φ goes up by 1. Amortized cost $= 1+1 = 2$.

Pop. Actual cost 1. The stack shrinks by one, so Φ drops by 1. Amortized cost $= 1 + (-1) = 0$.

Multipop popping $m = \min(k, |S|)$ items. Actual cost m. The stack shrinks by m, so Φ drops by m. Amortized cost $= m + (-m) = 0$.

Every operation has amortized cost at most 2. So n operations cost at most $2n = O(n)$. Notice this reproduces the accounting result – "\$1 on each stacked object" is literally "Φ = number of objects." The potential method just did the same accounting without ever naming an individual coin.

17.4.2 Potential for the binary counter

Define $\Phi(D) = b_i$, the number of 1-bits currently in the counter after operation i. Start at 0, so $\Phi(D_0) = 0$, and a bit count is never negative, so $\Phi \geq 0$. Both conditions hold.

Now analyze a single INCREMENT. Suppose it clears a run of t trailing 1-bits to 0 (those are the t carries) and then sets one 0-bit to 1 – unless the counter overflowed, but let us take the typical, non-overflow case first.

- **Actual cost** $c_i = t + 1$: it flipped t ones down and one zero up.
- **Change in potential**: we lost t ones (the cleared bits) and gained 1 one (the set bit), so $b_i = b_{i-1} - t + 1$, giving $\Phi(D_i) - \Phi(D_{i-1}) = (b_{i-1} - t + 1) - b_{i-1} = 1 - t$.
- **Amortized cost** $\hat{c}_i = c_i + (\Phi(D_i) - \Phi(D_{i-1})) = (t + 1) + (1 - t) = 2$.

The t – the entire unbounded carry chain – cancels. The amortized cost is a flat 2 no matter how long the carry ripples. The expensive carries are paid for exactly by the drop in potential they cause: each cleared bit was a unit of stored potential, released to fund its own flip.

Intuition

Φ counts the 1-bits – the "loaded springs" of the counter. A long carry chain is a long line of springs releasing at once: lots of actual work, but each release hands back the potential that was stored when that bit was set. The set bit at the end is a new spring loaded, costing the one unit that does not cancel. Amortized: 2, always.

Therefore n increments cost at most $2n + \Phi(D_0) - \Phi(D_n) \leq 2n = O(n)$, matching the aggregate and accounting analyses a third time.

Remark 17.1. What about overflow – when the counter is all 1s and INCREMENT wraps it to all 0s? Then it clears all k bits and sets none, so $c_i = k$ and Φ drops from k to 0, a change of $-k$. The amortized cost is $k + (-k) = 0 \leq 2$, still bounded. The telescoping bound still holds because $\Phi(D_n) \geq 0$ regardless. If you forbid overflow (start at 0, do at most $2^k - 1$ increments), Φ never reaches the wrap and the clean "$\leq 2n$" holds outright.

Common trap

A potential function can be *wrong* – it just has to satisfy $\Phi(D_0) = 0$ and $\Phi \geq 0$ to give a *valid* bound, but it gives a *tight* bound only if you chose it well. Picking $\Phi \equiv 0$ always satisfies the conditions and makes amortized cost equal actual cost – useless. The skill is choosing a Φ that rises during cheap operations (so their amortized cost absorbs a little extra) and plunges during expensive ones (so the plunge pays the bill). If your Φ does not drop when the expensive operation fires, you chose the wrong Φ.

17.5 The headline example: dynamic tables

Everything so far has been a warm-up for the example that matters most in practice. You want an array-backed structure – a list, a hash table, a stack – that you can keep inserting into, but you do not know in advance how big it will get. Arrays have fixed size. So what do you do when the array fills up?

You allocate a bigger one, copy everything over, and keep going. The question is: *how much bigger?* The answer, and the reason it is the right answer, is one of the cleanest results amortized analysis delivers.

17.5.1 The setup and the naive disaster

A *dynamic table* stores items in an array of some capacity. It tracks two numbers: *size* (how many items are stored) and *capacity* (how many the current array can hold). The *load factor* is $\alpha = size/capacity$. We INSERT one item at a time. When the array is full ($size = capacity$), an insert must first *expand*: allocate a new array, copy all existing items in, free the old one, and only then store the new item.

Common trap

The tempting, terrible idea: when the table fills, grow it by a *fixed* amount – say one extra slot, or a constant c slots. Then *every* insert (or every cth insert) triggers a full copy. If you grow by one slot each time, inserting n items copies $0 + 1 + 2 + \cdots + (n-1) = \Theta(n^2)$ items total. That is quadratic total work from what looks like a harmless "just make it a bit bigger." Growing additively is the classic performance bug.

The fix is to grow *multiplicatively*: when the table fills, *double* its capacity. This is called *table doubling*. Here is the insert operation.

Algorithm 75: TABLE-INSERT(T, x)

Input: A dynamic table T with fields $T.size$, $T.cap$, and array $T.A$.
Output: x appended; the table is doubled first if it was full.

```
1 if T.size = T.cap then
2     newcap ← max(1, 2 · T.cap);
3     allocate new array B of capacity newcap;
4     for j ← 1 to T.size do
5         B[j] ← T.A[j] ;            // copy every existing item:  the expensive part
6     free T.A;   T.A ← B;   T.cap ← newcap;
7 T.size ← T.size + 1;
8 T.A[T.size] ← x ;                          // store the new item:  always cheap
```

We charge 1 unit per item stored or copied. A non-expanding insert costs 1 (it just stores x). An expanding insert that copies m existing items costs $m + 1$. Let us watch the cost of the ith insert, starting from an empty table.

The expansions happen exactly when the table is full and one more item arrives. With doubling from capacity 1, the table expands on inserts numbered $1, 2, 3, 5, 9, 17, \ldots$. After the first three (which walk the capacity through $1, 2, 4$), the pattern settles down: for $j \geq 2$, insert number $2^j + 1$ is the one that fills capacity 2^j and forces the doubling to 2^{j+1}. (Insert 1 fills capacity 1; insert 2 forces capacity to 2; insert 3 forces capacity to 4; insert 5 forces capacity to 8; insert 9 to 16; and so on.)

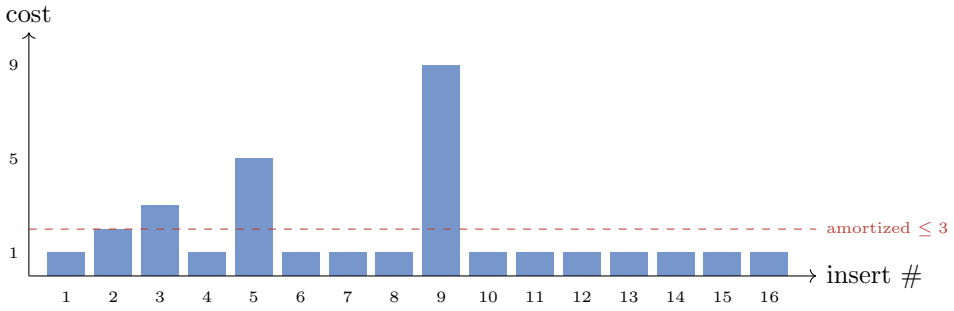


Figure 17.4: Cost of each insert into a doubling table, $i = 1 \ldots 16$. Almost every insert costs 1. The spikes at $i = 2^j + 1$ (heights $2, 3, 5, 9, \ldots$) are the doublings. Tall but rare – and as we will prove, their total is bounded by a geometric series.

17.5.2 Aggregate analysis of doubling

Consider n inserts into an initially empty table. Each insert pays 1 for storing its own item: that is n total. On top of that, the expanding inserts pay for *copies*. How many copies happen in total?

The jth expansion copies 2^{j-1} items (it copies everything in the table just before doubling capacity from 2^{j-1} to 2^j). The expansions that occur during n inserts copy

$$1 + 2 + 4 + 8 + \cdots + 2^{\lfloor \log_2(n-1) \rfloor} \;<\; 2n.$$

Let us be exact rather than hand-wavy. The copies form the geometric sum $\sum_{j=0}^{J} 2^j$ where $2^J < n$,

and a geometric sum of powers of two is just under twice its largest term:

$$\sum_{j=0}^{J} 2^j \;=\; 2^{J+1} - 1 \;<\; 2 \cdot 2^J \;<\; 2n.$$

So total copies $< 2n$, and the total cost is

$$T(n) \;=\; \underbrace{n}_{\text{stores}} + \underbrace{(\text{copies})}_{<\,2n} \;<\; n + 2n \;=\; 3n \;=\; O(n).$$

The amortized cost per insert is at most $3n/n = 3 = O(1)$.

Why this matters

This is the result that makes growable arrays – C++ `vector`, Java `ArrayList`, Python `list`, Go slices – practical. Each one of those doubles (or grows by a constant factor like $1.5\times$) precisely so that the amortized cost of appending is $O(1)$. Without amortized analysis you might believe appending is sometimes $O(n)$ and panic; with it, you can promise users "`append` is constant time on average over any sequence," and mean it as a hard guarantee. Once again, the geometric series bounds the total work: all the rare doublings combined cost at most a constant multiple of the cheap appends.

Intuition

The accounting view of table doubling. We can also understand the \$3 amortized cost directly through credit tokens. When we insert an item into a table of capacity m that was just doubled (so it currently holds $m/2$ elements), we charge an amortized fee of \$3:

- \$1 pays for inserting the element into its empty array slot.
- \$1 is stored as a credit token on the new element, to pay for moving itself during the next doubling.
- \$1 is deposited as a credit token onto one of the older elements that was copied in during the previous doubling without credit.

By the time the table becomes completely full (m elements), exactly $m/2$ new elements have been inserted, contributing $(m/2) \times \$2 = \$m$ in surplus tokens. Each of the m elements now holds \$1 of credit, exactly funding the m copy operations needed to relocate to the new $2m$ array. Figure 17.5 illustrates this accumulation of credit.

17.5.3 Potential analysis of doubling: choosing Φ

The aggregate argument is clean, but the potential method is more useful here because it gives a per-operation account and extends naturally to deletion. We need a Φ that is 0 on an empty table, never negative, and that has steadily built up to a large value exactly when the table is full, so that the expansion can drain it.

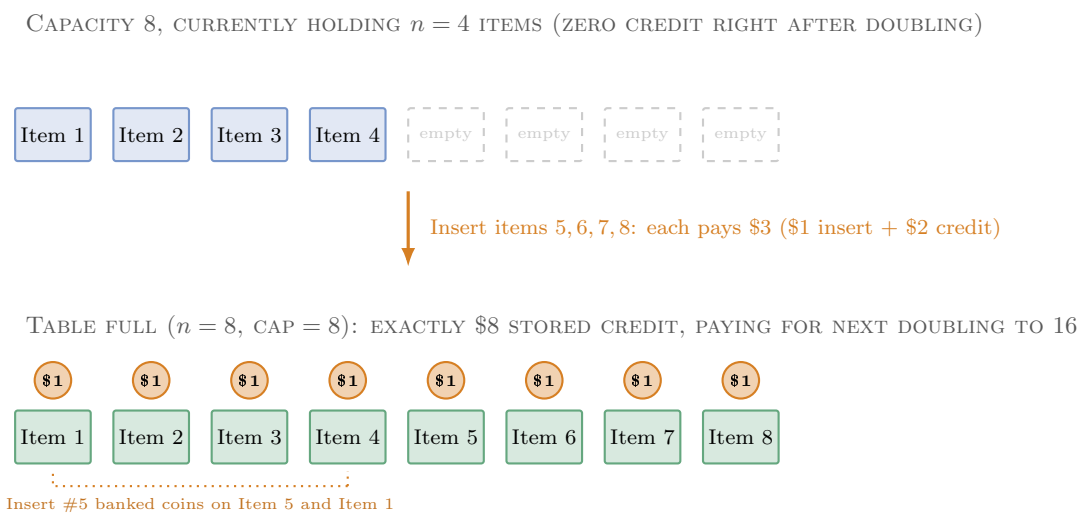


Figure 17.5: The accounting method for dynamic table doubling. Each append pays an amortized charge of \$3: \$1 performs the immediate insertion into an empty slot, \$1 stays on the new item to fund its future relocation, and \$1 is deposited onto an earlier item that lacks credit. By the time the table fills from size $m/2$ to m, all m items hold \$1, generating precisely the \$$m$ credit required to copy all elements into the new $2m$-capacity array.

Intuition

When is a doubling table about to do expensive work? When it is *full*. So we want potential to be near zero right after a doubling (the table is half empty, lots of room) and to climb to its peak right before the next doubling (the table is packed). The potential should measure "how dangerously full are we?"

The classic choice that achieves this:

$$\Phi(T) \;=\; 2 \cdot T.\mathit{size} \;-\; T.\mathit{cap}.$$

Let us sanity-check the two required conditions. Right after a genuine doubling (capacity at least 2), the table is half full: $\mathit{size} = \mathit{cap}/2$, so $\Phi = 2(\mathit{cap}/2) - \mathit{cap} = 0$. As we insert without expanding, *size* rises while *cap* stays fixed, so Φ climbs. When the table becomes full, $\mathit{size} = \mathit{cap}$, so $\Phi = 2\mathit{cap} - \mathit{cap} = \mathit{cap}$ – a big reservoir, exactly equal to the number of items the next expansion will have to copy. The potential has saved up precisely enough.

We also need $\Phi \geq 0$ throughout. Because doubling keeps the load factor $\alpha = \mathit{size}/\mathit{cap}$ at least $\frac{1}{2}$ at all times after the first item (right after a double it is exactly $\frac{1}{2}$, and it only rises until the next double), we have $\mathit{size} \geq \mathit{cap}/2$, hence $2\,\mathit{size} - \mathit{cap} \geq 0$. And the empty table has $\Phi = 0$. Good – both conditions hold.

Now compute the amortized cost of the ith insert. Write $s = T.\mathit{size}$ and $c = T.\mathit{cap}$ for the values *just before* this insert. Figure 17.6 previews the arithmetic: watch Φ climb on cheap inserts and crash on the doubling.

Case 1: no expansion ($s < c$). Actual cost $c_i = 1$. After insert, size is $s + 1$, capacity unchanged at c. So

$$\Delta\Phi = \big(2(s+1) - c\big) - \big(2s - c\big) = 2.$$

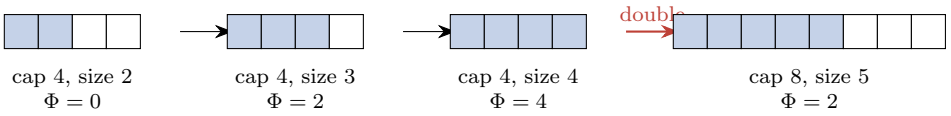


Figure 17.6: Potential $\Phi = 2\,size - cap$ filling and draining. Right after a doubling the array is half full and Φ is small. Each cheap insert raises Φ by 2, building a reservoir. When the array fills, $\Phi = cap$ – exactly the number of items the next doubling must copy. The expensive copy then drains Φ back down, and the cycle repeats.

Amortized cost $\hat{c}_i = 1 + 2 = 3$.

Case 2: expansion ($s = c$, with $c \geq 1$). The insert copies all c items and stores 1 new one, so actual cost $c_i = c + 1$. Capacity becomes $2c$; size becomes $c + 1 = s + 1$. The potential goes from $\Phi_{\text{before}} = 2c - c = c$ to $\Phi_{\text{after}} = 2(c + 1) - 2c = 2$. So

$$\Delta\Phi = 2 - c.$$

Amortized cost $\hat{c}_i = (c + 1) + (2 - c) = 3$.

Focus

Both cases give amortized cost exactly **3**. In Case 2 the $+c$ of copying is cancelled by the $-c$ drop in potential: the reservoir built up during the cheap inserts is spent all at once to pay for the copy. Amortized cost per insert is $3 = O(1)$, so n inserts cost $\leq 3n = O(n)$, confirming the aggregate result with a tidy per-operation story.

Remark 17.2. There is one edge case at the very start. The Case 2 algebra assumed $c \geq 1$. The first insert into a fresh table has $s = c = 0$, and our algorithm grows the capacity to $\max(1, 2 \cdot 0) = 1$ rather than to $2c = 0$. That insert copies nothing, so its actual cost is 1, and the potential moves from 0 to $2(1) - 1 = 1$, giving amortized cost $1 + 1 = 2$ – not 3. This is harmless: 2 is still at most 3, so the bound $\leq 3n$ holds for every n. From the second insert onward the clean "always exactly 3" story takes over. (You can see the first insert costing 2 in the lab below.)

Algorithm lab

Verify the telescoping by hand. Insert 5 items into an empty doubling table and track, after each insert: actual cost c_i, size, capacity, $\Phi = 2\,size - cap$, and amortized $\hat{c}_i = c_i + \Delta\Phi$. You should find: insert 1 (expand to cap 1): $c = 1$, $\Phi = 1$, $\hat{c} = 2$; insert 2 (expand to 2): $c = 2$, $\Phi = 2$, $\hat{c} = 3$; insert 3 (expand to 4): $c = 3$, $\Phi = 2$, $\hat{c} = 3$; insert 4: $c = 1$, $\Phi = 4$, $\hat{c} = 3$; insert 5 (expand to 8): $c = 5$, $\Phi = 2$, $\hat{c} = 3$. Total actual $= 1 + 2 + 3 + 1 + 5 = 12$. Total amortized $= 2 + 3 + 3 + 3 + 3 = 14$. Confirm $12 \leq 14$, and that the gap $14 - 12 = 2$ equals the final $\Phi = 2$ – exactly the leftover reservoir, as the telescoping formula predicts.

17.5.4 Why constant-factor growth, and not additive

We can now say precisely why doubling – or any constant-factor growth – works and additive growth fails. With multiplicative growth by a factor $r > 1$, the copy costs across all expansions form a geometric series with ratio r, which sums to a constant times its largest term, giving $O(n)$ total. With additive growth, the copy costs form an arithmetic series $0 + c + 2c + \cdots$, which sums to $\Theta(n^2)$.

Cost summary

Dynamic table, doubling growth (factor 2):

- Single INSERT, worst case: $\Theta(n)$ (when it triggers a copy).
- INSERT, *amortized*: $\Theta(1)$ (we proved ≤ 3).
- n inserts, total: $\Theta(n)$.
- Wasted space: at most a factor of 2 (load factor $\geq \frac{1}{2}$).

Additive growth (by c each time): single insert $\Theta(n)$, but amortized $\Theta(n)$ as well – n inserts cost $\Theta(n^2)$. Never do this.

Walking question

On your walk, weigh the trade-off. Growing by $2\times$ wastes up to half the array but appends in amortized $O(1)$. Growing by $1.5\times$ wastes at most a third and is still amortized $O(1)$ – the geometric series still converges. Where is the line? Show yourself that *any* growth factor $r > 1$ gives amortized $O(1)$, with the constant depending on r as roughly $r/(r-1)$. What happens to that constant as $r \to 1^+$? That limit is exactly why additive growth (morally $r \to 1$) explodes.

17.5.5 A word on deletion and shrinking

If a table can grow, you may want it to shrink when it empties out, to reclaim memory. The naive idea – halve the array as soon as it drops below half full – contains a trap.

Common trap

Suppose you double when full and halve when the table falls to exactly half full. Now sit right at the boundary $size = cap/2$ and alternate: insert (table fills, double, copy $\Theta(n)$ items), delete (table drops to half, halve, copy $\Theta(n)$ items), insert (double again), delete (halve again)... *Every single operation* now triggers a full copy. The amortized cost rockets back up to $\Theta(n)$. The structure "thrashes" at the threshold.

The cure is *hysteresis*: separate the grow and shrink thresholds so the table must change substantially before either fires (Figure 17.7). The standard fix is to double when full but only halve when the load factor drops to $\frac{1}{4}$ (not $\frac{1}{2}$). After either resize the table sits at load factor $\frac{1}{2}$, comfortably between the $\frac{1}{4}$ and 1 triggers, so $\Omega(cap)$ operations must occur before the next resize. A suitable potential – something like $\Phi = |\,2\,size - cap\,|$ when $\alpha \geq \frac{1}{2}$ and $\Phi = \frac{1}{2}\,cap - size$ when $\alpha < \frac{1}{2}$ – then yields amortized $O(1)$ for *both* insert and delete. We leave the careful case analysis to the exercises and to CLRS, but the idea carries over to other structures.

Focus

Whenever you have a structure that resizes at a threshold, separate the grow and shrink triggers so a resize leaves you far from *both*. The gap (hysteresis) guarantees many cheap operations between expensive ones, which is precisely the condition amortization needs. Resize-at-the-same-threshold is a recurring real-world bug – in hash tables, buffers, thread pools.

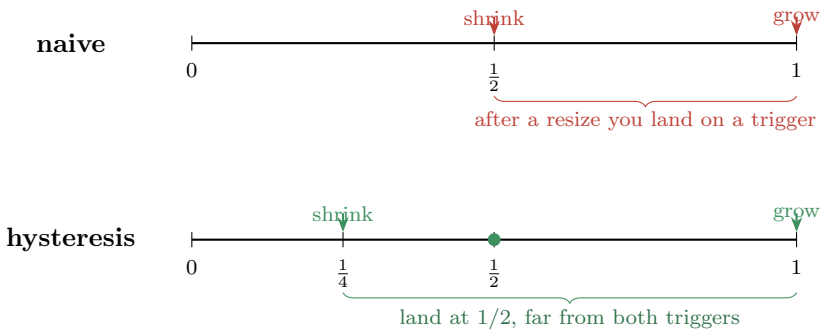


Figure 17.7: Load-factor thresholds. Top: shrinking at $\frac{1}{2}$ and growing at 1 leaves the table sitting on the shrink trigger right after a doubling, so a single insert–delete pair thrashes. Bottom: shrinking at $\frac{1}{4}$ instead leaves every resize at load factor $\frac{1}{2}$, a comfortable distance from both triggers, so $\Omega(cap)$ operations must pass before the next resize.

17.6 Choosing the right method

You now have three methods for the same kind of bound. They are interchangeable in power – anything one proves, the others can prove – but they differ in convenience.

- **Aggregate** is best when a clean global total $T(n)$ has an obvious closed form (a geometric series, a count of bit-flips). It is the least flexible: it gives every operation the *same* amortized cost, namely $T(n)/n$.
- **Accounting** is best when you can point at a concrete, local unit of "stored work" – a coin on each stacked object, a dollar on each set bit – and you want different operations to carry different charges.
- **Potential** is best for the trickiest structures, and it is the method that scales: it gives a per-operation bound, it composes when a structure supports several operations, and choosing a clever Φ often cracks problems the other two struggle with (Fibonacci heaps, splay trees, and union-find by rank all lean on potential functions).

History and connection

Amortized analysis was formalized by Robert E. Tarjan in his 1985 paper *Amortized Computational Complexity*. Before that paper, theorists and systems programmers used ad-hoc financial bookkeeping or physics analogies without a rigorous unifying framework. Tarjan systematized the three classic approaches – the aggregate method, the accounting method, and the potential method – drawing directly upon physical analogies of stored potential energy from classical mechanics. Together with Daniel Sleator, Tarjan used these techniques to analyze splay trees, pairing heaps, and the inverse-Ackermann bounds of disjoint sets.
In the 1970s, strict worst-case analysis was the dominant paradigm in computer science. Under that lens, a resizable array (such as C++ `std::vector` or Python list) was judged as a slow $\Theta(n)$ operation because doubling the table requires copying all elements. This pessimistic assessment contradicted actual practice, where dynamic arrays routinely outperformed linked lists. The key insight of amortized analysis was that worst-case per-operation bounds are often misleading: an expensive operation can only occur if preceded by a long sequence of cheap

operations that deposited sufficient credit or potential to fund it.
Amortized analysis provided a rigorous mathematical foundation for modern memory allocators, dynamic tables, splay trees, and Fibonacci heaps. It allows systems designers to build structures that occasionally perform massive housekeeping without violating sequence-wide $O(1)$ efficiency. The trade-off is that amortized guarantees are sequence-wide averages and give no latency bound for individual operations. In hard real-time systems (such as medical equipment, automotive engine controls, or real-time audio synthesis), a single 10ms table reallocation pause can cause catastrophic buffer underflows or safety violations. Designing de-amortized algorithms that spread expensive resizing work incrementally across individual steps without sacrificing asymptotic bounds remains an active area of system and algorithm design.

Why this matters

Why does a working programmer care? Because amortized bounds are the only truthful way to advertise the performance of growable, self-adjusting structures, and those are everywhere: every dynamic array, every hash table that rehashes, every rope or gap buffer in a text editor, every memory allocator that occasionally compacts. "Worst case per call" would force you to quote terrifying, useless numbers. Amortized analysis lets you make a true, strong promise – "constant time over any sequence" – and back it with a proof that no adversary can break.

With amortized analysis in hand, we complete Part IV on algorithmic design and analysis techniques. Throughout our study of sorting, dynamic programming, and greedy choices, we have quietly assumed a uniform-cost random access machine (RAM) model: reading any word in memory costs the same unit of time.

In Part V (Advanced Data Structures), that assumption meets the reality of physical hardware. When datasets grow too large to fit in main memory, storage shifts to magnetic disks or solid-state drives, where fetching a block of data takes thousands or millions of CPU clock cycles. In **Chapter 18 (B-Trees)**, we meet multi-way balanced search trees whose large branching factors match the page sizes of secondary storage, minimizing disk accesses while maintaining logarithmic dictionary operations.

17.7 Exercises

Exercise 17.1 (Multipop by hand)**.** Starting from an empty stack, execute: Push(1), Push(2), Push(3), Push(4), Multipop(3), Push(5), Multipop(10). Write the stack contents and actual cost after each operation, sum the actual costs, and verify the total is at most 2 times the number of operations. Why did the final Multipop(10) cost only 2? (Warmup.)

Exercise 17.2 (Aggregate the counter exactly)**.** For a 5-bit counter starting at 0, list the cost (number of bit-flips) of each of the first 16 increments. Sum them, and check your sum against the formula $\sum_{i \geq 0} \lfloor 16/2^i \rfloor$. By what factor does the true total beat the naive bound $16 \cdot 5$? (Warmup.)

Exercise 17.3 (Accounting charges must stay solvent)**.** A student proposes charging Push a flat \$1 and Pop a flat \$1, with Multipop charged \$1 total (not per item). Exhibit a short sequence of operations on which this charging scheme lets the "bank balance" go negative – i.e. total charged falls below total real cost – thereby proving the scheme invalid. (Medium.)

Exercise 17.4 (A reset operation)**.** Augment the binary counter with a RESET operation that sets the entire counter back to 0 (cost equal to the number of 1-bits cleared). Using the potential $\Phi =$ (number of 1-bits), show that a sequence of n operations, each an INCREMENT or a RESET, still costs $O(n)$. Identify the amortized cost of RESET under this potential. (Medium.)

Exercise 17.5 (Telescoping leftover)**.** Prove, directly from the telescoping identity $\sum c_i = \sum \hat{c}_i - \Phi(D_n) + \Phi(D_0)$, that for the doubling table the difference between total amortized and total actual cost after n inserts equals exactly $\Phi(D_n) = 2\,\mathit{size} - \mathit{cap}$ of the final table. Verify this against the 5-insert lab earlier in the chapter, where the gap was 2. (Medium.)

Exercise 17.6 (Tripling instead of doubling)**.** Suppose the dynamic table *triples* its capacity when full instead of doubling. Redo the aggregate analysis: show the total copy cost over n inserts is still $O(n)$, and find the constant (the amortized cost per insert). Then find a potential function Φ analogous to $2\,\mathit{size} - \mathit{cap}$ that proves the same bound per operation. (Medium-to-hard.)

Exercise 17.7 (General growth factor)**.** Show that growing the table by a factor $r > 1$ on each expansion gives amortized insert cost at most $1 + \frac{r}{r-1}$. Evaluate this for $r = 2$, $r = 1.5$, and $r = 1.1$. Explain, using your formula, why the amortized cost diverges as $r \to 1^+$, connecting this to why additive growth is $\Theta(n^2)$. (Hard.)

Exercise 17.8 (Why halving at $\frac{1}{2}$ thrashes)**.** Consider a table that doubles when full and halves the moment size drops to $\mathit{cap}/2$. Construct an explicit sequence of n insert/delete operations on which *every* operation triggers a full copy, so the total cost is $\Theta(n^2)$ and the amortized cost is $\Theta(n)$. Then explain precisely how moving the shrink threshold to $\mathit{cap}/4$ breaks your construction. (Hard.)

Exercise 17.9 (Insert and delete with $\frac{1}{4}$ hysteresis)**.** For the dynamic table that doubles when full and halves when the load factor drops to $\frac{1}{4}$, consider the potential

$$\Phi(T) = \begin{cases} 2\,\mathit{size} - \mathit{cap}, & \alpha \geq \frac{1}{2}, \\ \frac{1}{2}\,\mathit{cap} - \mathit{size}, & \alpha < \frac{1}{2}. \end{cases}$$

Verify $\Phi \geq 0$ and $\Phi = 0$ on an empty table, then show that both INSERT and DELETE have amortized cost $O(1)$ by checking the four cases: insert with/without expansion, delete with/without contraction. (Challenge.)

Exercise 17.10 (Designing a potential from scratch)**.** You maintain a sorted dynamic structure that occasionally performs a global "rebuild" costing $\Theta(m)$ when m items have accumulated since the last rebuild, and rebuilds happen every time the count of pending items reaches the current *size*. Propose a potential function and prove the amortized cost per insert is $O(1)$. (This is the seed of the "rebuild when you double" idea behind many real structures.) (Challenge.)

Part V

Advanced Data Structures

Chapter 18

B-Trees

Up to now, every search structure in this book has relied on a simplifying assumption: all your data fits in main memory, and reaching any node costs roughly the same as reaching any other. In RAM, counting each pointer-follow as one unit of work is a reasonable approximation. Once your dataset outgrows memory and spills onto secondary storage – a magnetic disk or a solid-state drive – that uniform-cost assumption breaks down. Suddenly *where* a node lives matters enormously, and the cost of a search is dominated not by how many comparisons you do but by how many times you have to go fetch a chunk of data from far away.

This chapter is about the search tree that takes that cost asymmetry seriously. A B-tree generalizes a binary search tree by trading height for width: instead of tall, narrow two-way branching, it uses wide, shallow multi-way nodes. Instead of one key and two children per node, a B-tree node holds dozens or hundreds of keys and hundreds or thousands of children. The result is a tree so shallow that you can index billions of records and still reach any one of them in a handful of disk visits. Most relational databases and many modern filesystems organize their data with a B-tree or a close relative. We will build one from the ground up.

18.1 Why Secondary Storage Dictates Wide Trees

The design of the B-tree follows from one physical fact about storage hardware.

Intuition

Imagine your data is a giant library. Fast memory (RAM) is the desk in front of you: tiny, but anything on it is instantly available. The disk is the warehouse across town: enormous, but every trip to fetch a box takes ages compared to reading something already on your desk. Importantly, the warehouse does not hand you *one page* per trip – it hands you a whole *box*, because the trip itself is the expensive part, not the box. So the smart move is: design your data so that each warehouse trip brings back a box stuffed with exactly the things you will need next, and so that you need as few trips as possible.

That "box" has a name. Disks (and the operating system on top of them) move data in fixed-size blocks called *pages* – typically something like 4 kilobytes, sometimes much larger. You cannot read a single byte from disk; you read a whole page. And the cost of reading that page is dominated by

the *seek*: physically moving a read head, or waiting out the latency of a flash lookup. Once the page is on its way, the difference between grabbing 8 bytes from it and grabbing 4000 bytes from it is negligible.

The numbers make the asymmetry clear.

Why this matters

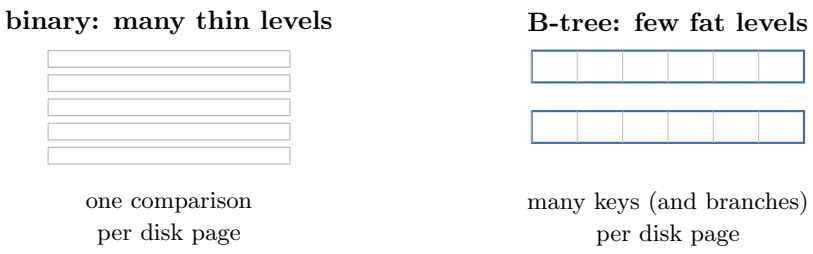
A modern CPU performs a few billion simple operations per second, so one comparison costs on the order of a nanosecond (10^{-9} seconds). A single random read from a spinning disk costs on the order of 10 milliseconds (10^{-2} seconds). That is a factor of roughly *ten million*. One disk access is worth about ten million in-memory comparisons. Solid-state drives shrink the gap, but even there a random read costs tens of microseconds – still tens of thousands of comparisons. The lesson is the same: when your data lives on disk, the number you must minimize is *disk accesses*, and you should be willing to do a ridiculous amount of in-memory computation to save even one of them.

Now reconsider a balanced binary search tree holding $n = 10^9$ keys (a billion). Its height is about $\log_2(10^9) \approx 30$. If the tree lives on disk and each node sits on its own page, a single search touches about 30 different pages, and in the worst case that is 30 disk seeks. At 10 milliseconds each, that is about a third of a second to find *one* record. A database answering thousands of queries per second cannot possibly afford this.

Focus

Two-way branching is the bottleneck here. A binary tree spends a whole expensive disk access just to make *one* comparison and then branch one of two ways. We have already paid for an entire page; we should extract far more than one bit of decision from it. The fix is to cram many keys into a single page so that one disk access lets us branch one of *many* ways. Fewer, fatter nodes give a shorter tree, and a shorter tree means fewer disk accesses.

Here is the picture of the trade we are making. On the left, the binary tree: deep, and each level is a separate disk trip. On the right, a B-tree: each node is a full page packed with keys, so the tree is only a few levels tall.



History and connection

B-trees were invented by Rudolf Bayer and Edward M. McCreight in 1970 while working at Boeing Scientific Research Laboratories. Boeing's engineering operations required indexing massive flight test datasets and manufacturing components stored on mechanical magnetic disks and drums. At the time, RAM was prohibitively expensive, and mechanical disk seeks

took 10 to 20 milliseconds – a lifetime compared to microsecond CPU cycles. Bayer and McCreight designed B-trees specifically to minimize the number of disk page accesses. (The meaning of "B" has never been settled – Bayer, Boeing, balanced, or broad; Bayer himself remained famously coy.)

The natural early instinct was to store an in-memory balanced binary tree (such as an AVL tree) directly on secondary storage. This was disastrous: traversing a depth-20 binary search tree required 20 sequential disk seeks, each fetching a 4KB disk page only to inspect a single 32-byte node, taking over 200 milliseconds per lookup. Bayer and McCreight realized that once a 4KB disk sector is read into RAM, scanning dozens or hundreds of keys in memory is essentially free. Widening the tree to have branching factor $t \approx 1000$ shrunk tree height to 3 or 4, slashing disk reads by 80% and making multi-gigabyte databases practical.

B-trees became the standard index structure in modern systems: relational database indices (PostgreSQL, MySQL InnoDB, SQLite, Oracle), document databases, and high-performance filesystems (ext4, NTFS, macOS APFS, and Btrfs). The main trade-off is concurrency overhead. Because insertions and deletions split or merge nodes along an entire root-to-leaf path, concurrent multi-threaded writes require complex latch crabbing (locking parent nodes until child operations finish). On modern solid-state flash drives (SSDs), random in-place page updates lead to severe write amplification (rewriting entire flash blocks for small updates), which is why log-structured merge (LSM) trees often replace B-trees in write-heavy workloads. Developing cache-oblivious B-trees and write-optimized learned index structures remains an active area of database systems research.

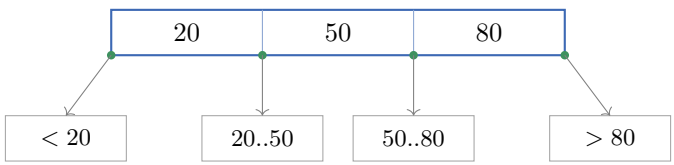
18.2 What a B-Tree Is

A B-tree is a rooted tree, but unlike a binary search tree its nodes are wide. Each node stores a sorted run of keys, and between and around those keys hang child pointers, one more child than there are keys.

Intuition

Picture a single B-tree node as a row of k keys laid out in increasing order, with $k+1$ "gaps" – one before the first key, one between each adjacent pair, and one after the last key. Each gap holds a pointer to a child subtree, and that subtree contains exactly the keys that belong in that gap. So if a node has keys $[20, 50, 80]$, it has four children: one for keys < 20, one for keys strictly between 20 and 50, one for keys strictly between 50 and 80, and one for keys > 80. The keys in the node are the *signposts* that tell you which gap – which child – to descend into next.

Here is a node with three keys and its four children drawn explicitly. Notice how each child's allowed range is fenced in by the keys on either side.



To keep the tree shallow and balanced we impose limits on how empty or how full a node may be. The key parameter is a fixed integer $t \geq 2$ called the *minimum degree* of the tree. It is chosen once, when you build the tree, usually so that a full node exactly fills one disk page.

Formal

Definition (B-tree). Let $t \geq 2$ be a fixed integer, the *minimum degree*. A *B-tree* is a rooted tree with the following properties.

1. Every node x stores a number of keys $x.n$, held in nondecreasing sorted order $x.key_1 \leq x.key_2 \leq \cdots \leq x.key_{x.n}$, and a boolean $x.leaf$ saying whether x is a leaf.
2. An internal (non-leaf) node with $x.n$ keys has exactly $x.n + 1$ children, pointed to by $x.c_1, \ldots, x.c_{x.n+1}$. Leaves have no children.
3. The keys separate the subtrees: if k_i is any key stored in the subtree rooted at child $x.c_i$, then
$$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \cdots \leq x.key_{x.n} \leq k_{x.n+1}.$$
4. **Every leaf is at the same depth**, which equals the tree's height h. (B-trees grow and shrink only at the root, which is how they stay perfectly height-balanced.)
5. **Occupancy bounds.** Every node other than the root has at least $t - 1$ keys, and therefore at least t children if it is internal. Every node has at most $2t - 1$ keys, and therefore at most $2t$ children if it is internal. The root may have as few as one key (or zero, if the tree is empty), but obeys the same upper bound.

A node with the maximum $2t - 1$ keys is called *full*.

That occupancy rule – between $t - 1$ and $2t - 1$ keys – is what keeps the tree balanced. Why are those the right bounds, and not arbitrary ones?

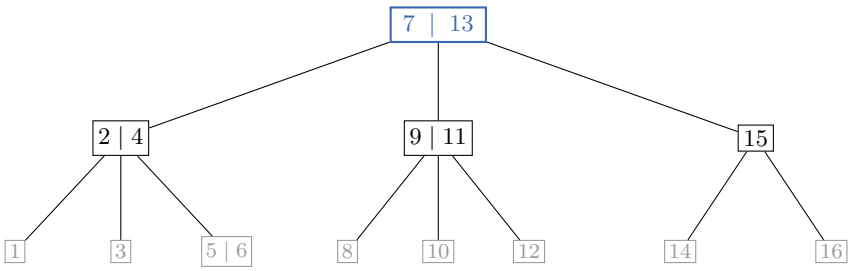
Intuition

The upper bound ($2t-1$ keys) is so a node fits in one disk page; you pick t to make a full node fill the page. The lower bound ($t - 1$ keys) is so nodes are never too *empty*. If we let nodes dwindle to one key each, the tree would drift back toward being binary and tall, defeating the whole purpose. Forcing every node at least half-full guarantees the branching factor stays large, which keeps the height tiny. The two bounds together say: *every node is between half-full and completely full.* The factor-of-two slack between $t - 1$ and $2t - 1$ is exactly the room we need to absorb insertions and deletions cheaply, as you will see.

Common trap

A frequent confusion: "minimum degree" t counts *children*, not keys. A non-root node has *at least* t children and *at most* $2t$ children; correspondingly it has between $t - 1$ and $2t - 1$ keys (always one fewer key than children, for internal nodes). Do not mix up t (a per-node minimum) with the *order* of a B-tree, a competing piece of terminology some textbooks use for the maximum number of children. We will stick exclusively with CLRS's minimum degree t to avoid the muddle. The smallest legal B-tree has $t = 2$: nodes hold 1 to 3 keys and have 2 to 4 children. Such a tree is sometimes called a *2-3-4 tree*, and it is the close cousin of the red-black tree from Chapter 13.

Example 18.1. Let $t = 2$. Then nodes may hold 1, 2, or 3 keys and (if internal) have 2, 3, or 4 children. Here is a legal B-tree of height 2 on the keys $1, 2, \ldots, 16$. The root holds two keys; each leaf is at the same depth.



Read the separators: the root's keys 7 and 13 split everything into three ranges – "below 7," "between 7 and 13," "above 13" – and the three child subtrees hold exactly those ranges. Within the leftmost child, its keys 2 and 4 split its range further. Every path from root to a leaf has the same length.

18.3 How Tall Can a B-Tree Get?

The point of a B-tree is that it is short, and we can prove exactly how short. We want an upper bound on the height h in terms of the number of keys n, and we want that bound to be logarithmic with a *large base* – the larger the base, the shorter the tree.

The strategy is the one you always use to bound the height of a balanced tree: count how few nodes a tree of height h can possibly contain. If even the *sparsest* legal tree of height h must hold a lot of keys, then a tree holding only n keys cannot be very tall.

Intuition

To make a B-tree as tall as possible for a given number of keys, you make every node as *empty* as the rules allow – because emptier nodes branch less, so it takes more levels to hold the same keys. The emptiest legal tree has the root with a single key (its minimum) and every other node with exactly $t - 1$ keys and t children (their minimum). So the number of nodes at least *multiplies by* t at each level down. That geometric growth is what forces the height to be only about $\log_t n$.

Count the nodes level by level in this emptiest tree. The root is 1 node. The root has at least 2 children (one key forces two children). Each of those, and everything below, has at least t children. So the node count is 1 at the root, then 2, and after that it multiplies by at least t at every step down. Figure 18.1 shows that geometric growth; the table beside it records the per-level minimum.

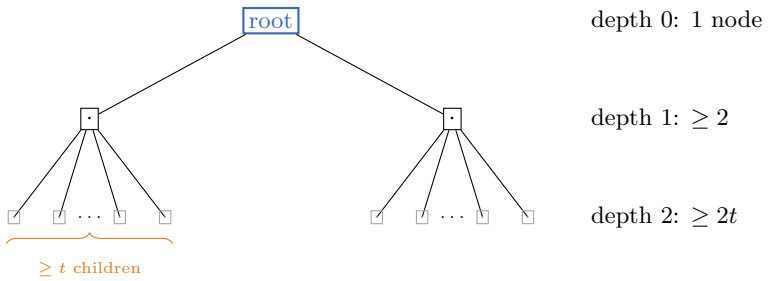


Figure 18.1: The sparsest (tallest) B-tree: the root has ≥ 2 children, and every node below fans out to $\geq t$ children, so each level holds at least t times as many nodes as the level above.

depth	at least this many nodes
0	1
1	2
2	$2t$
3	$2t^2$
$\vdots$	$\vdots$
h	$2t^{h-1}$

Every node except the root holds at least $t-1$ keys. The number of non-root nodes is at least the number of nodes at depths 1 through h, which is at least

$$2 + 2t + 2t^2 + \cdots + 2t^{h-1} = 2\sum_{i=0}^{h-1} t^i = 2 \cdot \frac{t^h - 1}{t-1}.$$

So the total number of keys satisfies

$$n \;\geq\; \underbrace{1}_{\text{root}} + (t-1) \cdot 2 \cdot \frac{t^h - 1}{t-1} \;=\; 1 + 2(t^h - 1) \;=\; 2t^h - 1.$$

Rearranging, $t^h \leq \dfrac{n+1}{2}$, and taking $\log_t$ of both sides gives the result.

Formal

Theorem (Height of a B-tree). If $n \geq 1$, then any B-tree of minimum degree $t \geq 2$ holding n keys has height

$$h \;\leq\; \log_t \frac{n+1}{2}.$$

Focus

The height is logarithmic in n with base t. The large base is what matters. Compared to a red-black tree, whose height is $\log_2 n$, the B-tree's height is $\log_t n = \dfrac{\log_2 n}{\log_2 t}$ – smaller by a factor of $\log_2 t$. With a realistic t of, say, 1000, that factor is about 10, so the B-tree is roughly ten times shorter, meaning ten times fewer disk accesses per search.

Example 18.2. Suppose each disk page holds a node with up to 1000 children, so $2t = 1000$ and $t = 500$. How tall is the tree for a billion keys, $n = 10^9$?

$$h \le \log_{500} \frac{10^9 + 1}{2} \approx \log_{500}(5 \times 10^8) = \frac{\ln(5 \times 10^8)}{\ln 500} \approx \frac{20.03}{6.21} \approx 3.2,$$

so $h \le 3$. Three levels! A tree of height 3 has its root, one level of internal nodes, and the leaves. If you keep the root permanently cached in memory (databases always do – it is just one page), then a search costs at most *two* disk accesses to reach any of a billion records. Compare that with the ≈ 30 accesses a binary tree would demand. In practice that is the difference between a usable database and an unusable one.

18.4 Searching a B-Tree

Searching is the simplest operation and a good warm-up, because it does not change the tree at all – it just walks down it.

Intuition

At each node you have a sorted row of keys. Scan along it (or binary-search it) to find where your target key would sit. Either you land exactly on it – done – or you fall into one of the gaps between two keys, and that gap's child pointer tells you which subtree to descend into. Repeat until you find the key or hit a leaf with no further child to follow, at which point the key simply is not present. It is the same higher-or-lower game as a binary search tree, except at each node you choose among many doors instead of just two.

Algorithm 76: B-Tree-Search(x, k) – find key k in the subtree rooted at x

Input: a node x, a search key k
Output: a pair (y, i) such that $y.key_i = k$, or NIL if absent

```
1  i ← 1;
2  while i ≤ x.n and k > x.key_i do
3  |  i ← i + 1;
4  if i ≤ x.n and k = x.key_i then
5  |  return (x, i);                                  // found it in this node
6  if x.leaf then
7  |  return NIL;                                     // nowhere left to go
8  else
9  |  Disk-Read(x.c_i);                  // the only line that costs a disk access
10 |  return B-Tree-Search(x.c_i, k);
```

The single line that matters for cost is Disk-Read: it is the only place we pay to fetch a page. We perform exactly one disk read per level of descent, so a search does at most $h + 1 = O(\log_t n)$ disk reads. The in-memory work per node is the linear scan over up to $2t - 1$ keys, so the CPU time is $O(t)$ per node and $O(t\,h) = O(t \log_t n)$ in total. We are happily trading a lot of in-memory comparison work for very few disk reads – exactly the bargain disks demand.

Algorithm lab

Trace B-TREE-SEARCH for $k = 12$ on the tree of Example 18.1.

- *At the root* $[7, 13]$: start $i = 1$. Is $12 > 7$? Yes, advance to $i = 2$. Is $12 > 13$? No, stop. Is $12 = 13$? No. The root is not a leaf, so descend into c_2, the child for the range between 7 and 13 – the node $[9, 11]$.
- *At* $[9, 11]$: $12 > 9$, advance; $12 > 11$, advance to $i = 3$. Now $i > x.n = 2$, stop. Not equal to anything here. Not a leaf, so descend into c_3, the child for keys above 11 – the leaf $[12]$.
- *At the leaf* $[12]$: $12 = 12$ at $i = 1$. **Found**, return this node and index 1.

Three nodes visited, one per level – exactly the height plus one. Now repeat the trace for $k = 5$ (you should find it in the leaf $[5, 6]$) and for the absent $k = 10\frac{1}{2}$, i.e. a key between 10 and 11 (you should fall off the bottom and return NIL).

18.5 Splitting a Full Node

Before we can insert, we need one surgical tool. Inserting a new key always ultimately squeezes it into some leaf, but a leaf might already be full ($2t - 1$ keys) with no room. The cure is *splitting*: take a full node and break it in two, pushing its middle key up into the parent as a new separator.

Intuition

A full node has $2t - 1$ keys. That is an odd number, so it has a unique middle key, the one at position t. Splitting cleaves the node down that middle key: the $t - 1$ keys to its left become a new left node, the $t - 1$ keys to its right become a new right node, and the middle key itself *floats up* to the parent, slotting in as a brand-new separator between those two new children. Each of the two halves has exactly $t - 1$ keys – the legal minimum – so both are valid nodes, and the parent gains one key and one child. Think of it as a cell dividing: one full cell becomes two half-cells plus a signpost handed to the level above.

Figure 18.2 illustrates the geometry of the split operation for general parameter t as well as the special case $t = 2$ (2-3-4 tree).

Comparing the parent before and after the split, it gains one key (the promoted median k_t) and one child pointer. For this to be safe, **the parent must not itself be full** when we ask it to absorb a new key – otherwise it would overflow. That single requirement is what shapes the entire insertion algorithm. The procedure assumes its caller hands it a non-full parent x and the index i of the full child to split.

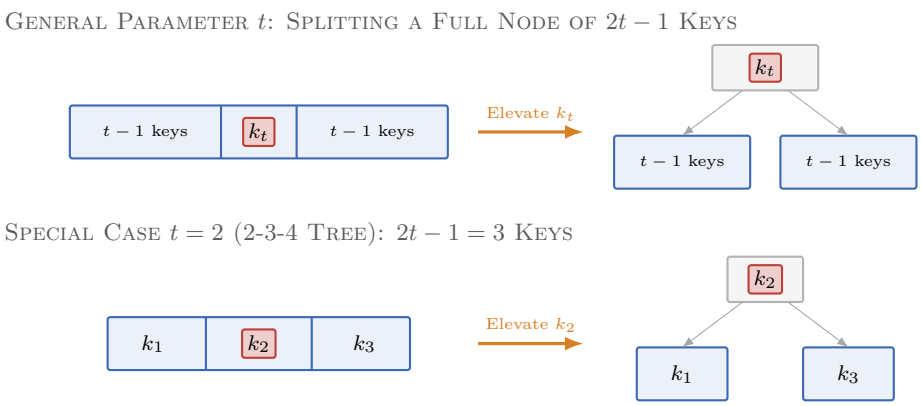


Figure 18.2: B-tree node splitting for general parameter t (top) and the $t = 2$ case (bottom). In both configurations, a full node holding $2t - 1$ keys splits at its median element k_t, which is promoted to the parent. The remaining $2t - 2$ keys split symmetrically into two legal minimal nodes of $t - 1$ keys each.

Algorithm 77: B-Tree-Split-Child(x, i) – split the full child $x.c_i$

Input: a non-full internal node x and an index i with $x.c_i$ full
Output: $x.c_i$ split in two, its median key moved up into x

```
1  y ← x.c_i;                                   // the full child, with 2t − 1 keys
2  z ← a new node;   z.leaf ← y.leaf;   z.n ← t − 1;
3  for j ← 1 to t − 1 do
4      z.key_j ← y.key_{j+t};                   // copy y's upper t − 1 keys into z
5  if not y.leaf then
6      for j ← 1 to t do
7          z.c_j ← y.c_{j+t};                   // and y's upper t children
8  y.n ← t − 1;                                 // y keeps only its lower t − 1 keys
9  for j ← x.n + 1 to i + 1 (decreasing) do
10     x.c_{j+1} ← x.c_j;                       // shift x's children right to open a slot for z
11 x.c_{i+1} ← z;
12 for j ← x.n to i (decreasing) do
13     x.key_{j+1} ← x.key_j;                   // shift x's keys right to open a slot
14 x.key_i ← y.key_t;                           // the median floats up into x
15 x.n ← x.n + 1;
16 Disk-Write(y); Disk-Write(z); Disk-Write(x);
```

Every loop runs at most t times, so the whole split is $O(t)$ in-memory work plus three disk writes – a tidy $O(1)$ in disk accesses. Splitting never touches anything below y and z; it is purely local surgery at one parent and its one full child.

Common trap

Two off-by-one traps lurk here. First, the median sits at index t, *not* $t-1$ or $t+1$; with $2t-1$ keys indexed $1 \ldots 2t-1$, position t has exactly $t-1$ keys on each side. Second, z receives keys $y.key_{t+1}$ through $y.key_{2t-1}$ – that is the loop "$z.key_j \leftarrow y.key_{j+t}$ for $j = 1 \ldots t-1$." The

lower keys $y.key_1 \dots y.key_{t-1}$ stay in y, and we simply set $y.n = t - 1$ to forget the rest. If you accidentally leave the median in one of the halves, that half ends up with t keys and the parent never receives its separator, and the whole structure quietly corrupts.

18.6 Insertion in a Single Downward Pass

Now the clever part. The naive plan would be: walk down to the right leaf, insert there, and if the leaf overflows, split it and push the median up, possibly triggering a cascade of splits all the way back up to the root. That works, but it walks the path *twice* – once down, once back up – which on disk means re-reading pages we already paid for. B-trees use a slicker, *single-pass* strategy that fixes problems on the way down, so it never has to come back up.

Focus

The single-pass insight: split full nodes *before* you enter them. As you descend toward the target leaf, whenever you are about to step into a child that is full, split it first. Splitting it pushes a key up into the current node – but you guaranteed the current node was non-full before you arrived, so it can safely absorb that key. By induction, every node you stand on during the descent is non-full, so every split you ever need to do has a safe place to put its median. You never overflow, you never recurse upward, and you touch each page on the path exactly once.

There is one node this trick cannot help with: the root, which has no parent to absorb a pushed-up key. So the root gets special handling. If the root is full when we begin, we grow a brand-new empty root above it and split the old root into two children of the new one (Figure 18.3). This is the *only* way a B-tree gains height, and it explains property 4 of the definition – since growth happens at the top, all leaves stay at the same depth automatically.

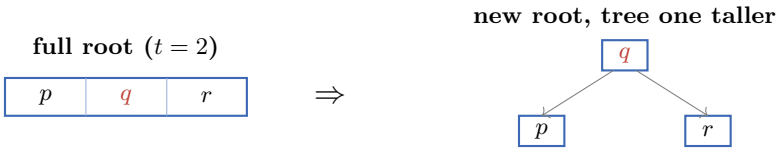


Figure 18.3: Growing the tree at the root. A full root splits about its median q, which becomes the sole key of a fresh root sitting one level higher; the two halves p and r become its children. Because the new level appears above everything, all leaves still share the same depth.

Algorithm 78: B-Tree-Insert(T, k) – insert key k into B-tree T

Input: a B-tree T and a key k to insert
Output: T updated to contain k, all invariants preserved

```
1 r ← T.root;
2 if r.n = 2t − 1 then
3 |  s ← a new node;   s.leaf ← FALSE;   s.n ← 0;
4 |  s.c_1 ← r;   T.root ← s;                    // new empty root above the old one
5 |  B-Tree-Split-Child(s, 1);                   // split old root; tree grows one level
6 |  B-Tree-Insert-Nonfull(s, k);
7 else
8 |  B-Tree-Insert-Nonfull(r, k);
```

Algorithm 79: B-Tree-Insert-Nonfull(x, k) – insert k into the non-full node x or below it

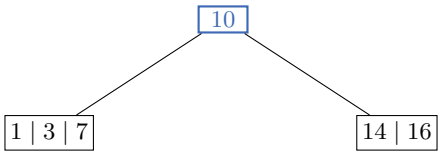
Input: a node x that is *not* full, and a key k

```
1  i ← x.n;
2  if x.leaf then
3  |  while i ≥ 1 and k < x.key_i do
4  |  |  x.key_{i+1} ← x.key_i;                  // shift keys right to make room
5  |  |  i ← i − 1;
6  |  x.key_{i+1} ← k;   x.n ← x.n + 1;
7  |  Disk-Write(x);                             // drop k into the leaf, done
8  else
9  |  while i ≥ 1 and k < x.key_i do
10 |  |  i ← i − 1;
11 |  i ← i + 1;                                // x.c_i is the child we must descend into
12 |  Disk-Read(x.c_i);
13 |  if x.c_i.n = 2t − 1 then
14 |  |  B-Tree-Split-Child(x, i);              // full child!  split it before descending
15 |  |  if k > x.key_i then
16 |  |  |  i ← i + 1;                          // the pushed-up median may shift which half we want
17 |  |  B-Tree-Insert-Nonfull(x.c_i, k);
```

That little "if $k > x.key_i$ then $i \leftarrow i + 1$" is easy to overlook but essential. After a split, the median key has just landed at position i in x, creating a new child at $i + 1$. If our key k is bigger than that median, it belongs in the right half, so we must redirect into $x.c_{i+1}$ instead of $x.c_i$.

18.6.1 A Fully Traced Insertion

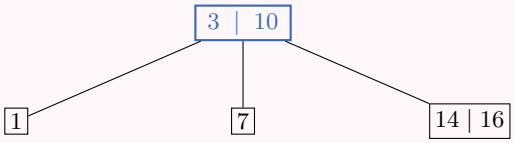
Watch the machinery run. Take $t = 2$ (so nodes hold 1 to 3 keys, and full means 3 keys). We start from this tree and insert the key 4.



Algorithm lab

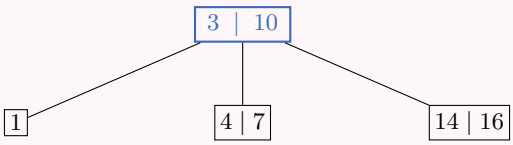
Inserting 4 into the tree above ($t = 2$).

1. *Check the root.* The root [10] has 1 key, which is not $2t - 1 = 3$, so the root is not full. We call INSERT-NONFULL on it.
2. *At the root, choose a child.* Scanning, $4 < 10$, so we head for the left child $c_1 = [1, 3, 7]$. But that child has 3 keys – it is *full*! We must split it before descending.
3. *Split the full child.* The median of $[1, 3, 7]$ is 3 (position $t = 2$). It floats up into the root, which becomes $[3, 10]$. The child splits into [1] (left) and [7] (right). The tree is now:



4. *Redirect after the split.* Back in the root, the median 3 now sits at position $i = 1$. Is $k = 4 > 3$? Yes, so bump i to 2: we descend into $c_2 = [7]$, the right half.
5. *At the leaf.* Node [7] is a non-full leaf. We slide 4 into place: $4 < 7$, so 4 goes before 7, giving the leaf $[4, 7]$. Disk-write it. Done.

The final tree:



Notice we proactively split $[1, 3, 7]$ even though, in hindsight, it had room for 4 after the split. The single-pass discipline does not gamble: it splits any full node it might pass through, guaranteeing the leaf at the bottom always has room.

Walking question

Here is something to chew on while you stretch your legs. The single-pass insert sometimes splits a node that, it turns out, did not strictly *need* splitting – the eventual insertion happened elsewhere, or the node had spare room. Is that wasteful? Now think about the alternative: an insert that descends first and splits on the way back up only when forced. That avoids "unnecessary" splits but must keep the whole root-to-leaf path of pages in memory and re-walk it. On disk, which cost dominates – a few extra splits, or re-reading the path? And does your answer change if the tree lives entirely in RAM? Ask which resource is actually scarce in each setting; that question is the main lesson of this chapter.

Cost summary

B-tree operations, minimum degree t, n keys, height $h = O(\log_t n)$.

operation	disk accesses	CPU time
search	$O(h) = O(\log_t n)$	$O(t\,h) = O(t \log_t n)$
insert	$O(h) = O(\log_t n)$	$O(t\,h) = O(t \log_t n)$
delete	$O(h) = O(\log_t n)$	$O(t\,h) = O(t \log_t n)$

Each operation visits $O(h)$ nodes, one disk access per node, and does $O(t)$ in-memory work per node (a scan plus at most one split or merge). Since $t \log_t n = t \frac{\log_2 n}{\log_2 t}$, increasing t shortens the tree but adds per-node work; you choose t to make a node fill exactly one page, which minimizes the thing that actually hurts – disk accesses.

18.7 Why Insertion Is Correct

It is worth pausing to convince ourselves the single-pass insert really does preserve every B-tree property. The argument rests on one clean invariant.

Formal

Descent invariant. Every call B-Tree-Insert-Nonfull(x, k) is made on a node x that is *not full*, i.e. $x.n \leq 2t - 2$.

Why the invariant holds. B-Tree-Insert establishes it for the first call: if the original root was full, we split it so the new root s holds a single key (non-full); if it was not full, we call directly on it. Now suppose we are at a non-full node x and about to recurse into child $x.c_i$. If $x.c_i$ is full, we split it first. After the split, $x.c_i$ (and its new sibling) each hold $t-1$ keys – well below $2t-1$ – so whichever half we descend into is non-full. If $x.c_i$ was not full to begin with, we descend into it unchanged, still non-full. Either way the next call receives a non-full node. By induction the invariant holds all the way down.

Why that makes everything correct. Because the node we land on is always non-full, the leaf at the bottom always has room to absorb k without overflowing, so we never violate the upper bound $2t - 1$. Every split we perform takes a full node to two half-full ($t - 1$-key) nodes and lifts one key into a guaranteed-non-full parent, so no node ever drops below $t - 1$ keys or exceeds $2t - 1$. The sorted order (property 3) is maintained because we always insert k into the gap where it belongs and because a split's median is exactly the correct separator between its two halves. Finally, height grows only when we split the root, and that adds one level uniformly above everything, so all leaves stay at equal depth (property 4). Every property survives. □

18.8 Deletion

Deletion is the fussiest operation, and it is where most first-time implementers get tangled. The guiding idea, though, mirrors insertion exactly, so hold onto that symmetry and the cases will feel less like a thicket.

Intuition

Insertion's enemy was *overflow*: a node getting too full, cured by splitting. Deletion's enemy is the opposite, *underflow*: a node dropping below $t-1$ keys, cured by either *borrowing* a key from a sibling or *merging* with a sibling. And just as insertion split full nodes on the way *down* so the leaf always had room, deletion will *fatten* thin nodes on the way down so that whenever we finally remove a key, the node we remove it from has a key to spare. We maintain the mirror-image invariant: *never step into a node with only the minimum $t-1$ keys; first top it up to at least t.*

Focus

The deletion descent invariant. Whenever B-Tree-Delete is called recursively on a node, that node has at least t keys (one more than the minimum $t-1$). The root is exempt – it is allowed to be thin – which is why deletion can shrink the tree's height only by emptying the root.

To enforce that invariant we need two repair moves. Suppose we want to descend into a child c that has only $t-1$ keys (the minimum). Before descending, we fatten it:

- **Borrow (rotate).** If an *adjacent sibling* of c has at least t keys (a key to spare), rotate one key across. The separator key in the parent that sits between c and that sibling moves *down* into c, and the sibling's extreme key (and a child pointer, if internal) moves *up* to take the separator's place. Now c has t keys and the sibling still has at least $t-1$. Everybody is legal.
- **Merge.** If *both* adjacent siblings have only $t-1$ keys (nobody can spare a key), merge c with one of them. Pull the separator key down from the parent and fuse c, that separator, and the sibling into one node. The merged node has $(t-1)+1+(t-1)=2t-1$ keys – a perfectly full but legal node – and the parent loses one key and one child.

Figure 18.4 shows a borrow and Figure 18.5 a merge; in both, the parent's separator is the hinge the keys rotate around.

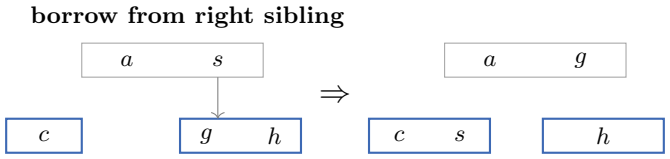


Figure 18.4: Borrowing. The thin child c takes the parent separator s, and the donor sibling's smallest key g rises to become the new separator.

With those two moves in hand, the deletion itself splits into a few cases depending on where the key to remove lives and whether it is in a leaf.

Formal

Deleting key k from the subtree rooted at x (sketch). First ensure x has at least t keys (borrow/merge as above, unless x is the root). Then:

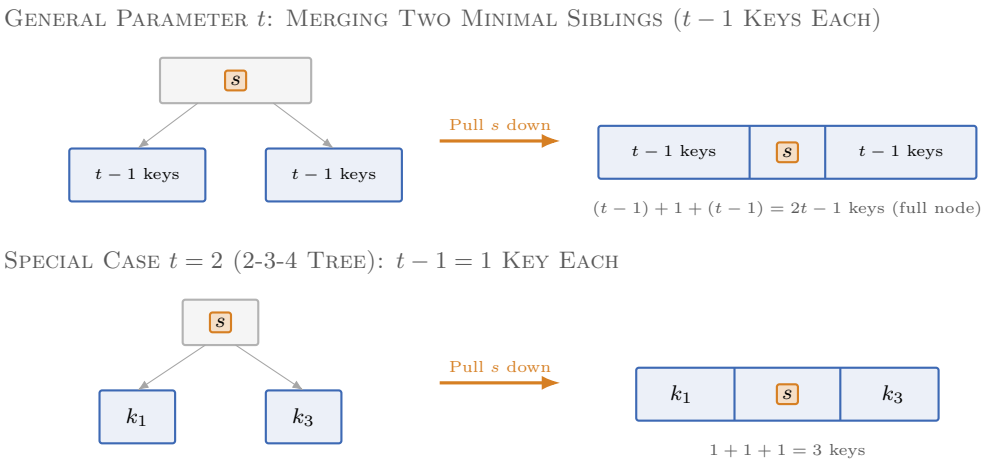


Figure 18.5: B-tree node merging for general parameter t (top) and the $t = 2$ case (bottom). When two adjacent siblings both hold the minimum $t-1$ keys, their intervening parent separator s is pulled down to fuse all three components into a single valid node containing $(t-1)+1+(t-1) = 2t-1$ keys.

Case 1 – k is in x and x is a leaf. Just remove k from x. Since x had $\geq t$ keys it still has $\geq t-1$. Done.

Case 2 – k is in x and x is internal. We cannot leave a hole, so we replace k with a neighboring key that can fill its slot.

2a. If the child y *left* of k has $\geq t$ keys, find k's *predecessor* k' (the largest key in y's subtree), recursively delete k' from that subtree, and overwrite k with k'.

2b. Symmetrically, if the child z *right* of k has $\geq t$ keys, use k's *successor*.

2c. If both flanking children have only $t-1$ keys, *merge* k and the right child into the left child (a full $2t-1$-key node), then recursively delete k from that merged child.

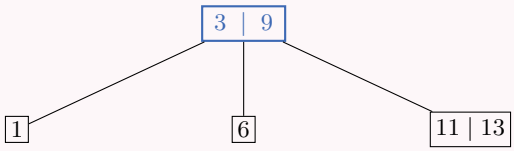
Case 3 – k is not in x. Find the child $x.c_i$ whose subtree must contain k. If $x.c_i$ has only $t-1$ keys, fatten it first (borrow from a sibling, or merge), then recurse into the appropriate child to delete k.

Intuition

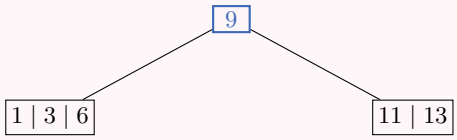
Case 2 is the only genuinely new idea, and it is the same trick as deleting from an ordinary binary search tree: to remove an internal key without leaving a gap, replace it with its *predecessor* or *successor*, then delete *that* key from where it lives (always lower down, eventually in a leaf). The predecessor is the rightmost key in the left subtree; the successor is the leftmost key in the right subtree. We prefer whichever side has a key to spare so the recursion lands on a node that is already fat enough – preserving the invariant. If neither side can spare a key, we merge them and the problem shrinks to one level down.

Algorithm lab

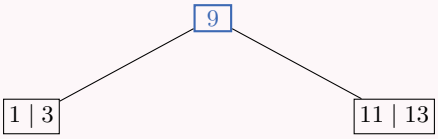
A deletion that triggers a merge ($t = 2$). Start from this tree and delete the key 6.



1. *At the root* $[3, 9]$: 6 is not here. It lies between 3 and 9, so it belongs in the middle child $c_2 = [6]$.
2. *Fatten before descending.* Child [6] has only $t - 1 = 1$ key – the minimum. Can it borrow? Its left sibling [1] has 1 key (nothing to spare); its right sibling $[11, 13]$ has 2 keys, but that is across the separator 9, and to keep the example pointed, suppose we look left first and, finding no donor, *merge* [6] with its left sibling [1]. We pull the separator 3 down: the merged node is $[1, 3, 6]$, and the root loses a key, becoming [9].



3. *Delete from the fattened leaf.* Now recurse into $[1, 3, 6]$, a leaf with 3 keys. It is Case 1: simply remove 6, leaving $[1, 3]$. Done.



The merge fixed the underflow *on the way down*, so by the time we removed 6 the leaf had keys to spare. Had the root dropped to zero keys during such a merge, we would discard it and make the merged node the new root – that is the lone way a B-tree loses height.

Common trap

The classic deletion bug is forgetting to fatten *before* you descend, and instead deleting first and patching up underflow afterward. The patch-up-afterward style does work, but it forces you to walk back up the path – exactly the double-pass cost we worked so hard to avoid in insertion – and it is far easier to get the cascade of merges wrong. The disciplined recipe is always: *guarantee the node you are about to enter has $\geq t$ keys, then enter it.* If you maintain that one invariant religiously, every removal you finally perform leaves a legal node behind, and you never recurse upward.

Like search and insert, deletion visits one node per level and does $O(t)$ in-memory work (one borrow or merge) at each, so it costs $O(h) = O(\log_t n)$ disk accesses and $O(t \log_t n)$ CPU time – the same budget as everything else.

18.9 B-Trees in Practice

Why this matters

You now understand the data structure behind most large-scale data storage. When PostgreSQL, MySQL/InnoDB, Oracle, or SQLite build an index on a column, that index is a B-tree (often the B^+-tree variant, which stores all real data in the leaves and chains the leaves together for fast range scans – a small twist on what you have just learned). When you query "`WHERE id = 91823`" and it returns in a millisecond from a table of a billion rows, you are watching a height-3 B-tree do its two disk reads. Filesystems use them to map filenames to locations. Key-value stores, time-series databases, and even some in-memory systems use them because the same width-over-height logic that beats disk latency also beats CPU-cache-miss latency: a cache line is just a very small, very fast "page," and packing many keys into one fetched block pays off there too.

Focus

The one idea to carry away: **a B-tree is a balanced search tree re-tuned for a world where each access is expensive, so it maximizes the work done per access by making nodes wide.** Width ⇒ low height ⇒ few accesses. Every detail – the $t-1$-to-$2t-1$ occupancy band, splitting full nodes on the way down, fattening thin nodes on the way down, growing and shrinking only at the root – serves that one goal.

B-trees solve the dynamic dictionary problem at large scale, balancing multi-way branching against the physical page transfers of secondary storage. Every query navigates an ordered key space to search, insert, or delete individual records.

Many algorithmic problems, however, require managing collections of items where order does not matter at all. Instead, we need to track dynamic partitions: maintaining disjoint pools of connected elements, merging pools together, and querying whether two items belong to the same component. In **Chapter 19 (Disjoint Sets and Union-Find)**, we study a compact forest data structure that implements these set operations so efficiently that any sequence of m operations runs in practically linear time – bounded by the inverse of the fast-growing Ackermann function.

Exercises

Exercise 18.1. For a B-tree with minimum degree $t = 4$, state: (a) the minimum and maximum number of keys in a non-root node; (b) the minimum and maximum number of children of an internal non-root node; (c) the maximum number of keys in any node. Then state how many keys the *root* is allowed to have at minimum.

Exercise 18.2. Using the height theorem, compute an upper bound on the height of a B-tree with $t = 100$ holding $n = 10^6$ keys, then $n = 10^{12}$ keys. Roughly how many disk accesses does a search cost in each case if the root is kept cached in memory?

Exercise 18.3. Starting from an empty B-tree with $t = 2$, insert the keys F, S, Q, K, C, L, H, T, V, W, M, R, N, P in this order (compare them alphabetically). Draw the tree after every

insertion that causes a split. (This is the classic CLRS build-up exercise; doing it by hand cements the single-pass insert.)

Exercise 18.4. Show that a B-tree of minimum degree t and height h holding n keys must satisfy $n \le (2t)^{h+1} - 1$ as well (the *maximum* number of keys, from the fullest possible tree), and combine it with the chapter's lower bound to sandwich n. What does the upper bound tell you about the *smallest* possible height for n keys?

Exercise 18.5. Let $t = 3$. First check yourself: how many keys does a full node hold? Write that full node's keys as $10, 20, 30, 40, 50$, identify the median that floats up under B-Tree-Split-Child, and write out the two resulting child nodes and the key handed to the parent.

Exercise 18.6. B-Tree-Split-Child requires that the parent be non-full before it is called. Explain concretely what goes wrong if you split a child whose parent is already full, and explain how B-Tree-Insert-Nonfull's descent guarantees the parent is never full at the moment of a split.

Exercise 18.7. Take the final tree from the deletion lab in this chapter (the one with root [9], left child $[1, 3]$, right child $[11, 13]$, with $t = 2$). Delete the key 9. Which deletion case applies, and what does the tree look like afterward? (Hint: 9 lives in an internal node, so you are in Case 2 – decide whether to use its predecessor or successor based on which flanking child can spare a key.)

Exercise 18.8. Argue carefully why the single-pass insertion never needs to revisit a node it has already left on its way down. Phrase your answer in terms of the descent invariant. Then describe a scenario in which a *two-pass* (descend, then split upward) insertion would re-read the same disk page multiple times, and count how many extra disk reads it incurs in the worst case for a tree of height h.

Exercise 18.9. A disk page holds 4096 bytes. Each key plus its associated child pointer occupies 32 bytes, and a node needs an extra 8-byte header. Roughly what is the largest minimum degree t such that a full node still fits in one page? Show your arithmetic, and explain why you would choose t this large rather than, say, $t = 2$.

Exercise 18.10. *(Reading ahead.)* Look up the B^+-tree variant. In a B^+-tree, all real records live in the leaves and internal nodes hold only separator keys, and the leaves are linked in a chain. Explain in one paragraph why this layout makes *range queries* ("give me every key between 40 and 90 in sorted order") faster than in the plain B-tree of this chapter.

Chapter 19

Disjoint Sets and Union-Find

In the previous chapter, B-trees organized massive ordered dictionaries across secondary storage pages, where queries drilled down an ordered key space. But many computational problems care nothing about key order. You are given a pile of objects – people, computers on a network, pixels in a photograph, cells in a maze – and from time to time someone tells you that two of them belong together. "Alice and Bob are friends." "Cable runs from router 3 to router 7." "This pixel touches that pixel and they are the same color." As these facts arrive, the objects clump up into groups. And at any moment, someone might stop you and ask: *are these two particular objects currently in the same group?*

That is the entire problem. You maintain a collection of groups, you occasionally merge two groups into one, and you occasionally ask whether two objects share a group. It is called the *disjoint-set* or *union-find* problem, and its best solution is one of the most surprising results in data structures: an implementation so fast that bounding its per-operation cost requires the extraordinarily slow-growing inverse Ackermann function.

Intuition

Picture a room full of strangers. Each begins alone, a group of one. You walk around announcing introductions: "you two know each other now." When two people meet, their entire friend-groups silently merge – if Alice knew Carol and Bob knew Dave, then after Alice meets Bob, all four are one group. At any point I can tap two people on the shoulder and ask, "same group?" To answer instantly, each group elects a single *leader*, and everyone in the group knows (perhaps after a short chain of pointers) who their leader is. Two people are in the same group exactly when they share a leader. Everything in this chapter builds on that idea: *a group is named by its leader.*

We will start with the slowest reasonable version, watch exactly where it hurts, and add two small tricks. Those two tricks together turn a structure that could cost $\Theta(n)$ per operation into one that costs, for all practical purposes, a constant. Then we will meet the strange, slow-growing function that measures precisely how close to constant we got.

19.1 What We Are Actually Building

Let us pin the problem down. We have a universe of n elements. They are partitioned into *disjoint sets* – "disjoint" meaning no element lives in two sets at once, so every element belongs to exactly one set. We want to support three operations.

Formal

A **disjoint-set data structure** maintains a collection $\mathcal{S} = \{S_1, S_2, \ldots, S_k\}$ of disjoint dynamic sets. Each set is identified by a *representative*, which is some fixed member of the set. Three operations are supported:

- MAKE-SET(x): create a new set whose only member (and representative) is x. Since x joins no other set, the sets stay disjoint.
- UNION(x, y): replace the two sets containing x and y, say S_x and S_y, by their union $S_x \cup S_y$, and destroy the two originals. The new set gets one representative (often a member of one of the two old sets).
- FIND-SET(x): return the representative of the set containing x.

Notice how cleanly FIND-SET answers our shoulder-tap question. "Are x and y in the same set?" is just the test FIND-SET(x) = FIND-SET(y). We never need a separate "same-set" operation; representatives give it to us for free. The only promise the representative must keep is *stability*: if you call FIND-SET twice on the same element with no intervening UNION, you must get the same answer both times. Which element serves as representative is up to us, and we will make good use of that freedom.

Focus

Hold these three operations in your head as the contract. Everything that follows – linked lists, forests, ranks, path compression – is just *implementation* behind that contract. The user only ever sees MAKE-SET, UNION, and FIND-SET.

19.1.1 A motivating use: connected components

Before we build anything, here is why the contract is worth keeping. A *graph* is a set of vertices and a set of edges joining them; two vertices are in the same *connected component* if you can walk from one to the other along edges. Suppose edges arrive one at a time and, after each, you want to know how many components remain, or whether two given vertices are connected yet. Union-find answers both questions directly.

Algorithm 80: CONNECTED-COMPONENTS(G)

Input: A graph $G = (V, E)$.
Output: A disjoint-set structure where u, v share a set iff they are connected.

```
foreach vertex v ∈ V do
    MAKE-SET(v);
foreach edge (u, v) ∈ E do
    if FIND-SET(u) ≠ FIND-SET(v) then
        UNION(u, v);
return the disjoint-set structure;
```

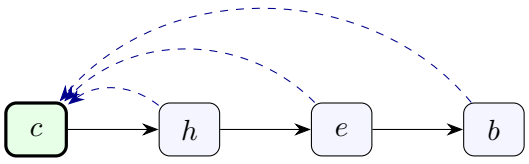
After this runs, the query procedure SAME-COMPONENT(u, v) is just one line: return whether FIND-SET(u) = FIND-SET(v). We will meet this exact pattern again in Chapter 23, where Kruskal's minimum-spanning-tree algorithm leans entirely on union-find to avoid creating cycles.

Why this matters

Union-find is used in a lot of real software. Kruskal's minimum-spanning-tree algorithm uses it to detect whether adding an edge would form a cycle. Image-processing tools use it for *connected-component labeling* – finding the blobs in a black-and-white image. Compilers use it for *unification* in type inference, merging type variables that must be equal. Maze generators carve passages by unioning cells. Even physics simulations of percolation (does water seep through a porous rock?) reduce to union-find.

19.2 First Try: a Linked-List Representation

Let us build the simplest thing that could possibly work, so we can see exactly where its weaknesses bite. Represent each set as a *linked list*. The first object in the list is the representative. Each object stores a pointer to the next object in its list, and – this is the clever part – also a pointer straight back to the representative.



dashed = "back to representative" pointer

The solid arrows chain the list together (so we can walk it); the dashed arrows let every member point home to the representative c. With this layout, MAKE-SET and FIND-SET are both fast.

- MAKE-SET(x): build a one-element list. Constant time.
- FIND-SET(x): follow x's back-pointer to the representative. Constant time.

The trouble is UNION. To union two lists, we append one onto the other. Appending the list is cheap – splice one tail onto the other. But every object on the appended list now has the *wrong* back-pointer: it still points to its old representative. We must walk the appended list and repoint each one to the new representative. That walk costs time proportional to the length of the list we appended.

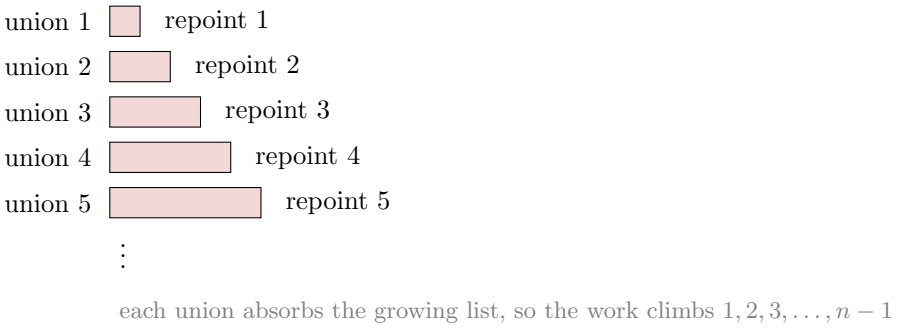
Common trap

The expensive part of the linked-list union is not joining the lists – that is one pointer splice. It is *updating every back-pointer* on the list that got absorbed. Beginners often assume union is cheap because "it's just a list append." The back-pointer updates are what make this representation slow.

19.2.1 How bad can it get? A worst-case sequence

Naively, UNION(x, y) might always append y's list onto x's. An adversary can exploit this. Suppose we make n singleton sets $x_1, \ldots, x_n$, then perform $n - 1$ unions, always appending the long accumulated list onto a fresh singleton:

$$\textsc{Union}(x_2, x_1),\ \textsc{Union}(x_3, x_2),\ \ldots,\ \textsc{Union}(x_n, x_{n-1}).$$



Watch the cost if each union repoints the *growing* side. The first union repoints 1 object, the second repoints 2, the third 3, and so on, so the total update work across all $n - 1$ unions is

$$1 + 2 + 3 + \cdots + (n-1) = \frac{(n-1)\,n}{2} = \Theta(n^2).$$

Averaged over the $2n - 1$ operations in the sequence, that is $\Theta(n)$ per operation. For a million elements, a single union near the end could touch a million back-pointers. Now let us fix it.

19.2.2 The weighted-union heuristic

The fix is embarrassingly simple, and it is the first appearance of a theme that will repeat: *when you merge two things, always absorb the smaller into the larger.* Keep, with each list, a count of its length. When you union, append the shorter list onto the longer one, and repoint only the shorter list's back-pointers. This is the *weighted-union heuristic.*

Why does this help so dramatically? Think about a single object x, and ask: how many times can x's back-pointer ever be updated? Its back-pointer is updated only when x is on the *smaller* of two lists being merged. But the moment that happens, x lands in a merged list at least *twice* as big as the one it left. So each time x gets repointed, the size of x's set at least doubles.

Intuition

A quantity that at least doubles each time something happens cannot have that something happen very often. Start at size 1; after k doublings the set has size at least 2^k. Since no set exceeds n, we need $2^k \le n$, so $k \le \log_2 n$. Therefore *any single object's back-pointer is updated at most* $\log_2 n$ *times over the entire history of the structure.*

Summing across all n objects, the total back-pointer work over any sequence of operations is at most $n \log_2 n$. The earlier quadratic blow-up is gone.

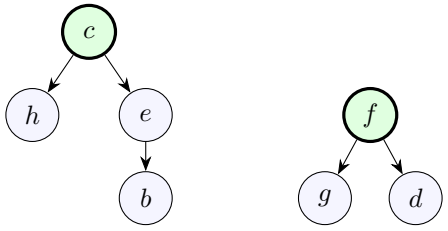
Formal

Theorem (weighted union). Using the linked-list representation with the weighted-union heuristic, any sequence of m MAKE-SET, UNION, and FIND-SET operations, of which n are MAKE-SET, takes $O(m + n \log n)$ time in total.

This is a good result – it is exactly the bound CLRS proves for this representation. But we can do better still. A second way to picture sets opens the door to near-constant time: we trade lists for trees.

19.3 The Forest Representation

Represent each set not as a list but as a *rooted tree*. Each element has a single pointer to its *parent*. The root of a tree points to itself (a convenient way to say "I am the representative"). The whole structure is therefore a *forest* – a collection of these trees, one per set. The representative of a set is the root of its tree.



Each arrow points from a child *up* to its parent. These are not binary trees and not search trees. There is no ordering, no balance condition yet, and a node may have any number of children. The only thing that matters is *which root you reach by walking up.*

The three operations become:

- MAKE-SET(x): make a one-node tree. Set $x.p = x$ (its parent is itself). Constant time.
- FIND-SET(x): walk up parent pointers from x until you hit a node that is its own parent. That node is the root; return it.

- UNION(x, y): find the two roots, then make one root point to the other. One pointer change (plus two finds).

Here is the bare-bones pseudocode, before any cleverness.

Algorithm 81: Naive forest operations

```
1  function MAKE-SET(x)
2      x.p ← x;
3  function FIND-SET(x)
4      while x ≠ x.p do
5          x ← x.p;
6      return x;
7  function UNION(x, y)
8      LINK(FIND-SET(x), FIND-SET(y));
9  function LINK(u, v)
       // u and v are roots
10     u.p ← v;
```

The cost of everything is the cost of FIND-SET, which is the cost of walking from a node to its root – that is, the *height* of the tree. And here is the catch: nothing yet stops the trees from growing tall and stringy. If LINK always hangs the first root under the second, the same adversarial union chain from before builds a single path of n nodes, and FIND-SET on the deepest node walks all n of them. We are right back to $\Theta(n)$.

Walking question

Before reading on, take this on a walk. The linked-list version had a weighted-union trick: absorb the smaller into the larger. The forest version is stringy because we attach roots carelessly. What is the analogous "attach the smaller thing under the larger thing" rule for trees – and what should "smaller" even mean when the thing is a tree, not a list? There are two natural answers, and they behave almost identically. See if you can invent at least one of them.

19.4 Trick One: Union by Rank

The trees got tall because we attached roots blindly. The cure is the same philosophy as weighted union: *when you link two roots, put the shorter tree under the taller one's root.* If you always hang the shorter tree beneath the taller, the combined tree's height does not grow – the shorter tree's nodes are now one level deeper, but they were shallower to begin with, so they still fit under the old height. The total height only increases when you link two trees of *equal* height, and even then it goes up by just one.

We could track exact heights, but maintaining exact heights becomes awkward once we add the second trick (path compression shortens paths during queries without recomputing subtree heights). So instead we track an *upper bound* on the height called the *rank*. Every node carries an integer $x.rank$. A fresh singleton has rank 0. When we link two roots:

- if their ranks differ, hang the lower-rank root under the higher-rank root, and *no rank changes*;
- if their ranks are equal, pick either as the new root and increment its rank by one.

Algorithm 82: Forest with union by rank

```
1  function MAKE-SET(x)
2      x.p ← x;
3      x.rank ← 0;
4  function UNION(x, y)
5      u ← FIND-SET(x);
6      v ← FIND-SET(y);
7      if u = v then return // already in the same set; nothing to do;
8      LINK(u, v);
9  function LINK(u, v)
       // u, v are distinct roots
10     if u.rank > v.rank then
11         v.p ← u;
12     else
13         u.p ← v;
14         if u.rank = v.rank then
15             v.rank ← v.rank + 1;
```

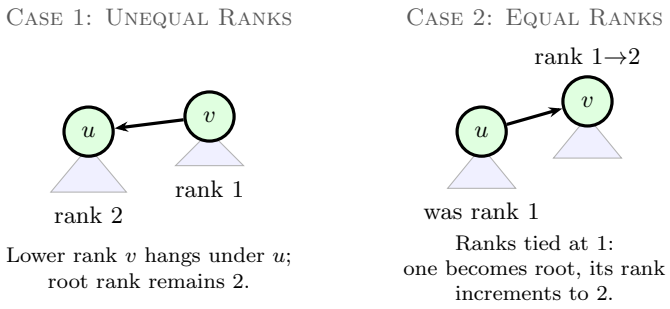


Figure 19.1: The two cases of LINK(u, v) in union by rank. When ranks differ, the smaller rank tree is attached under the larger with no rank change. When ranks match, either root may be chosen as parent, and its rank increases by exactly 1.

The two cases of LINK: a triangle is the rest of each root's tree. Only the equal-rank case ever raises a rank, and only by one.

19.4.1 Why rank controls height

The following guarantee, and its proof, explain *why* rank works.

Lemma 19.1. *With union by rank (and no path compression), any root of rank r is the root of a tree containing at least 2^r nodes.*

Why it is true. We argue by induction on the number of links performed. A fresh node has rank 0 and a tree of $1 = 2^0$ node, so the base case holds. Now consider a link. If two roots of *different* rank are joined, the surviving root's rank does not change, and its tree only grew, so its size still beats 2^r. The only interesting case is linking two roots of *equal* rank r. By the inductive hypothesis each had at least 2^r nodes, so the merged tree has at least $2^r + 2^r = 2^{r+1}$ nodes – and its new rank is exactly $r + 1$. The bound holds. □

Corollary 19.1. *Every rank is at most $\lfloor \log_2 n \rfloor$, where n is the number of elements. Hence every tree has height at most $\lfloor \log_2 n \rfloor$, and* Find-Set *runs in $O(\log n)$ time.*

Why it is true. A root of rank r owns at least 2^r nodes, and no tree can own more than n nodes, so $2^r \le n$, giving $r \le \log_2 n$. Since rank is an upper bound on height, the height is at most $\log_2 n$ too. □

So union by rank alone already matches the $O(\log n)$ per-operation behavior of weighted lists – but now in a tree, which sets up the second trick. A trace makes it concrete.

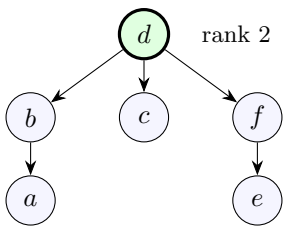
Algorithm lab

Trace union by rank. Start with singletons a, b, c, d, e, f, g, h, each rank 0. Execute, in order:

$$\textsc{Union}(a, b),\ \textsc{Union}(c, d),\ \textsc{Union}(a, c),\ \textsc{Union}(e, f),\ \textsc{Union}(a, e).$$

1. $\textsc{Union}(a, b)$: equal ranks 0. Hang a under b; now $b.\mathit{rank} = 1$. Tree: root b, child a.
2. $\textsc{Union}(c, d)$: equal ranks 0. Hang c under d; $d.\mathit{rank} = 1$. Tree: root d, child c.
3. $\textsc{Union}(a, c)$: roots are b (rank 1) and d (rank 1). Equal! Hang b under d; $d.\mathit{rank} = 2$. Now d roots all of a, b, c, d.
4. $\textsc{Union}(e, f)$: equal ranks 0. Hang e under f; $f.\mathit{rank} = 1$.
5. $\textsc{Union}(a, e)$: roots are d (rank 2) and f (rank 1). Different! Hang the lower, f, under d. No rank changes; d stays rank 2.

Final tree rooted at d: children c, b, f; under b is a; under f is e. The deepest node (a or e) is 2 steps from the root – and indeed $\mathit{rank}(d) = 2$. Check that this 8-node tree has height $2 \le \log_2 8 = 3$, as the corollary promises.



The tree the trace builds. Its height is 2, matching $rank(d) = 2$, and comfortably under the $\log_2 8 = 3$ ceiling.

19.5 Trick Two: Path Compression

Now the second idea. Every time you call FIND-SET(x), you walk a path from x up to the root. That walk is free information: along the way you visited a whole chain of nodes, and you *now know the root for every one of them*. So why throw that away? On the way back, repoint every node you visited directly at the root. The next FIND-SET on any of them is then a single hop. This is *path compression*.

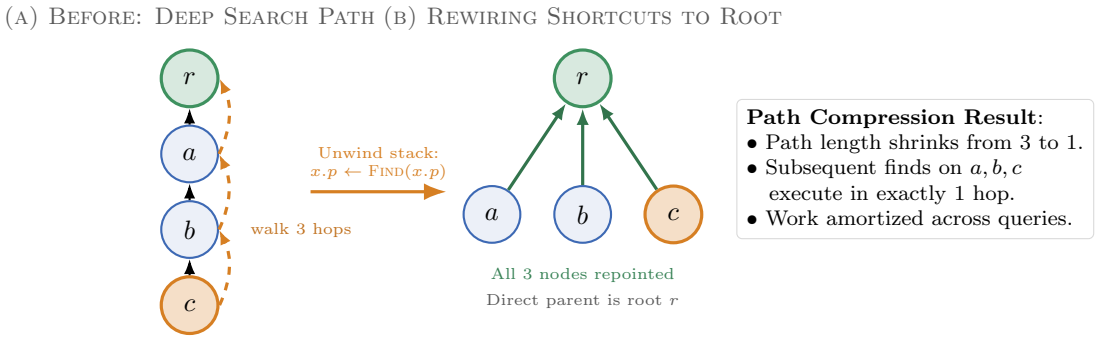


Figure 19.2: Path compression in FIND-SET(c). In (a), the search traverses the path $c \to b \to a \to r$. As the recursive calls return in (b), each visited node updates its parent pointer directly to root r. The tree is flattened, turning future lookups into instantaneous $O(1)$ operations.

A single FIND(c) walked $c \to b \to a \to r$; afterward a, b, and c *all* point straight at r. The tree got dramatically flatter, and it cost nothing extra – we were walking that path anyway. The implementation is a two-line recursion.

Algorithm 83: FIND-SET with path compression

```
1 function FIND-SET(x)
2     if x ≠ x.p then
3         x.p ← FIND-SET(x.p);
          // recurse to the root, then repoint x straight at it
4     return x.p;
```

Read that recursion carefully. The call FIND-SET($x.p$) returns the root and, as a side effect, compresses everything above x. Then the line $x.p \leftarrow$ (that root) snaps x itself onto the root. By the time the outermost call returns, every node on the original path points directly at the root.

Common trap

Path compression changes node *depths* but we deliberately do *not* update ranks when it happens. After compression a node's rank may be larger than its actual height. That is fine and intentional: rank stays a valid *upper bound* on height, which is all our analysis ever needed, and recomputing exact heights after every find would be slow and pointless.

Walking question

Path compression makes FIND-SET *change the structure* even though it is, semantically, a read-only query – you only asked "who is the representative?" yet the tree was rearranged. This is the same flavor of self-improvement you saw in amortized analysis: an operation pays a little extra now to make all future operations cheaper. Convince yourself that the answer it returns is never wrong, no matter how much it rearranges – the root of x's tree is the same before and after, because compression never moves a node to a *different* tree.

19.6 Putting It Together, and the Near-Constant Result

Use both tricks – union by rank *and* path compression – on the forest, and the bound on a sequence of operations drops from $O(m \log n)$ to something almost, but not quite, linear. The result is due to Tarjan; afterward we will explain the unusual function that appears in it.

Formal

Theorem (Tarjan). On a forest with union by rank and path compression, any sequence of m disjoint-set operations, of which n are MAKE-SET, runs in total time

$$O\big(m\,\alpha(n)\big),$$

where α is the *inverse Ackermann function*.

The function $\alpha(n)$ grows so slowly that for every n you will ever encounter – every n that could be written down with atoms in the observable universe – $\alpha(n) \le 4$. So the average cost per operation is bounded by a constant that, in any real program, never exceeds 4 or 5. The structure is *effectively* constant-time per operation, even though, famously, it is provably *not* constant in the strict asymptotic sense.

19.6.1 Where does inverse Ackermann come from?

You do not need the full proof – it is intricate, and CLRS devotes a careful section to it. But you should know *why* such a bizarre function shows up at all. It is the inverse of a function that *grows* extremely fast, so α itself grows extremely slowly.

Start with a function that grows absurdly fast. Define a tower of operations where each level is the iterate of the level below:

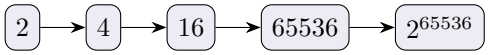
- Repeated addition is multiplication: $a \cdot b = a + a + \cdots + a$ (b times).
- Repeated multiplication is exponentiation: $a^b = a \cdot a \cdots a$ (b times).
- Repeated exponentiation is *tetration*: a tower $a^{a^{\cdot^{\cdot^{a}}}}$ of height b.

Keep climbing this ladder of operations – each one iterating the previous – and you get the *Ackermann function* $A(k, j)$, which grows faster than any primitive-recursive function. For a sense of scale, the

iterated power of two,

$$2, \quad 2^2 = 4, \quad 2^{2^2} = 16, \quad 2^{2^{2^2}} = 65536, \quad 2^{2^{2^{2^2}}} = 2^{65536},$$

reaches a number with about 20,000 digits in just *five* steps. The inverse Ackermann function $\alpha(n)$ asks the reverse question: *how many steps up this tower do I need before I exceed n?* Because the tower explodes so fast, the answer stays tiny forever.



each step raises 2 to the previous value: explosive up, glacial inverse

Reading this strip *left to right* is the tower exploding; reading it *right to left* – "how many boxes back from n to reach 2?" – is α. Five boxes already carry you past anything you could store, so $\alpha(n)$ never climbs past a tiny handful.

Intuition

Think of $\alpha(n)$ as the answer to: "how many times must I apply this fast-growing tower-building before I pass n?" Since one application can leap from 65536 to 2^{65536}, you essentially never need more than a handful of applications. The single-variable inverse Ackermann $\alpha(n) \leq 4$ for all n below roughly $2^{2^{2^{16}}}$ – a number so large it has no name and no physical meaning. For your laptop, your cluster, and every dataset humanity will ever assemble, $\alpha(n)$ is a small constant.

19.6.2 A gentle sketch of why we win

We sketch the argument without the full proof. Union by rank alone guarantees that ranks along any root path strictly increase, and that ranks never exceed $\log_2 n$. Path compression then attacks those paths: every time a node is found, it leaps closer to the root, so a node can only be "far from the root" a limited number of times before it is essentially adjacent to the root.

The accounting trick (this is amortized analysis, the previous chapter's tool) groups nodes into rank *blocks* and charges the cost of each step of a find either to the operation itself or to a node whose rank is about to jump it into a higher block. A node can only be charged this way a number of times related to how many rank blocks exist – and because ranks grow only to $\log_2 n$ while the block boundaries are spaced by the tower function, the number of blocks is about $\alpha(n)$. Multiply m operations by $\alpha(n)$ charges each and you get the $O(m\,\alpha(n))$ bound.

Focus

You do not need to reproduce the proof. You need three things: **(1)** both heuristics are needed for the α bound – either one alone gives only $O(\log n)$ per operation; **(2)** α is the inverse of a function that grows faster than any tower, so it is bounded by a tiny constant for all realistic n; **(3)** therefore union-find is, in every practical sense, constant time per operation.

History and connection

The disjoint-set problem was formalized in 1964 by Bernard A. Galler and Michael J. Fischer while designing compiler algorithms for FORTRAN's `EQUIVALENCE` statement, which aliased variables in shared memory. Galler and Fischer introduced the disjoint-set forest representation and the union-by-rank and path-compression heuristics. In 1975, Robert E. Tarjan published his celebrated analysis establishing the tight $\Theta(m\,\alpha(m,n))$ bound, which he later refined with Jan van Leeuwen (1984).

For over a decade, theorists assumed that path compression together with union by rank ran in strictly linear $O(m)$ time, suspecting that the exotic inverse-Ackermann factor $\alpha(n)$ was merely an artifact of an imperfect upper-bound proof. Tarjan then proved a matching lower bound: on any pointer machine where operations must follow pointers, *no* disjoint-set algorithm can achieve $o(m\,\alpha(m,n))$ worst-case time. So the inverse-Ackermann factor is inherent to the problem in this model, not an artifact of the analysis.

Tarjan's analysis showed that disjoint-set forests answer connectivity queries in near-constant amortized time. This enables near-linear running time for Kruskal's minimum spanning tree algorithm, connected-component labeling in computer vision, Hindley–Milner type unification in programming language compilers, and physical percolation modeling. The structure has one real limitation: it only supports union operations; it cannot split sets, undo previous merges, or delete individual elements without reconstructing the forest from scratch. Designing fully dynamic connectivity algorithms that support both insertions and deletions of edges in polylogarithmic time is still an active research area.

19.6.3 What each variant costs

Cost summary

For a sequence with n Make-Sets and m operations total:

Representation	Find-Set (worst)	Sequence cost
Linked list, arbitrary union	$O(1)$	$O(n^2)$ in the bad case
Linked list, weighted union	$O(1)$	$O(m + n \log n)$
Forest, naive link	$O(n)$	$O(mn)$ in the bad case
Forest, union by rank only	$O(\log n)$	$O(m \log n)$
Forest, path compression only	—	$O(m \log n)$ (amortized)
Forest, rank *and* compression	—	$\boxed{O(m\,\alpha(n))}$

The last row is what you should actually implement. The two heuristics together cost two extra fields per node (a parent pointer and a small rank) and a few lines of code, and they buy you effectively constant time.

19.7 A Complete Worked Example

Now run the finished structure – forest, union by rank, path compression – on a small connected-components problem so you see all the pieces move together.

Vertices: $1, 2, 3, 4, 5, 6$. Edges arrive in this order:

$$(1,2),\ (3,4),\ (5,6),\ (1,3),\ (2,5).$$

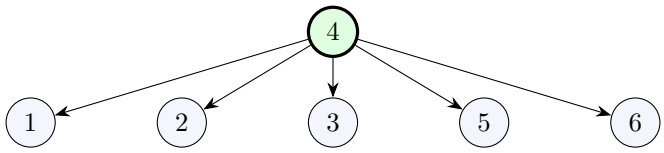
Every vertex starts as a singleton: parent itself, rank 0.

Algorithm lab

Trace. We write $p[\,]$ for the parent array and $r[\,]$ for ranks. Initially $p[i] = i$ and $r[i] = 0$ for all i.

1. **Edge** $(1,2)$**.** FIND$(1) = 1$, FIND$(2) = 2$. Equal rank, link 1 under 2: $p[1] = 2$, $r[2] = 1$.
2. **Edge** $(3,4)$**.** Roots $3, 4$, equal rank: $p[3] = 4$, $r[4] = 1$.
3. **Edge** $(5,6)$**.** Roots $5, 6$, equal rank: $p[5] = 6$, $r[6] = 1$. Now three components: $\{1,2\}$, $\{3,4\}$, $\{5,6\}$.
4. **Edge** $(1,3)$**.** FIND(1) walks $1 \to 2$, returns 2 (rank 1). FIND(3) walks $3 \to 4$, returns 4 (rank 1). Equal rank, link 2 under 4: $p[2] = 4$, $r[4] = 2$. Component $\{1,2,3,4\}$ rooted at 4.
5. **Edge** $(2,5)$**.** FIND(2) walks $2 \to 4$, returns 4 (rank 2). FIND(5) walks $5 \to 6$, returns 6 (rank 1). Different rank: hang the smaller, 6, under 4: $p[6] = 4$. No rank change. Everything is now one component, rooted at 4.

Now watch path compression flatten the tree. Call FIND(5). The path is $5 \to 6 \to 4$. Compression repoints: $p[5] = 4$ and $p[6] = 4$ (it already was). Next call FIND(1): path $1 \to 2 \to 4$; afterwards $p[1] = 4$ and $p[2] = 4$. After a few finds, *every* vertex points directly at 4, and all subsequent finds are a single hop. The tree has been ironed flat.



after compression: a flat star, every FIND is one hop

19.8 Implementation Notes

In real code you almost never use explicit node objects with pointers. You use two integer arrays indexed $0 \dots n-1$: a `parent` array and a `rank` array. MAKE-SET(i) sets `parent[i] = i` and `rank[i] = 0`. This is cache-friendly and executes in direct $O(1)$ time.

A common and excellent variant of FIND avoids recursion (and its stack depth) using *path halving*: as you walk up, make every node point to its grandparent. It compresses almost as well as full compression and runs in a single tight loop.

Algorithm 84: FIND-SET with path halving (iterative)

```
1 function FIND-SET(x)
2     while x ≠ parent[x] do
3         parent[x] ← parent[parent[x]];
          // point x at its grandparent, then step up
4         x ← parent[x];
5     return x;
```

Common trap

A subtle but classic bug: comparing the *representatives* but unioning the *original elements* correctly. In UNION(x, y) you must call FIND-SET on both *first*, link the two roots, and never link an element to itself. Linking a root to itself ($u.p \leftarrow u$ when $u = v$) is a silent no-op that wastes a rank increment in some buggy variants; always check that the two roots are distinct before incrementing any rank.

Many implementations replace rank with *size* (the number of nodes in the tree) and link the smaller tree under the larger – "union by size." It achieves the same $O(\alpha(n))$ bound, and size is sometimes useful on its own ("how big is this component?"). Rank and size are two flavors of the same "small under big" idea you have now seen three times: weighted lists, union by rank, union by size. Whenever you merge structures, absorbing the small into the large is almost always the right choice.

19.9 Summary

You started with a problem so plain it barely seemed to need an algorithm: group objects, merge groups, ask if two objects share a group. You built it four ways, each better than the last.

- **Linked lists** give instant FIND but slow UNION; the weighted-union heuristic (small list onto big) tames it to $O(m + n \log n)$.
- **Forests** name each set by a tree root. Naive linking can build a stringy path of height n.
- **Union by rank** attaches the shorter tree under the taller, keeping height $O(\log n)$.
- **Path compression** flattens the find path on every query, so the structure improves itself as it is used.

Together, rank and compression give $O(m\,\alpha(n))$ – effectively constant time per operation, where α,

the inverse Ackermann function, is bounded by a tiny constant for every input you will ever meet. The implementation needs only two integer arrays and about a dozen lines of code.

Why this matters

Both heuristics follow the same design pattern: each operation invests a small amount of work up front to make *future* operations cheaper. Weighted union bounds how often any element can change leaders, and path compression flattens pointer chains lazily during find queries, postponing structural cleanup until someone actually visits those nodes.

In **Chapter 20 (Fibonacci Heaps)**, we apply this philosophy of lazy postponement to priority queues. Instead of reorganizing heaps on every key update, a Fibonacci heap does almost nothing during insertions and key decreases, letting roots accumulate in a loose collection. Only when EXTRACT-MIN demands the smallest item does the structure perform a consolidating cleanup. Using the potential method of amortized analysis, we will prove that this procrastination pays off: insertions, merges, and key decreases all run in $O(1)$ amortized time, which speeds up graph algorithms such as Dijkstra's and Prim's.

19.10 Exercises

Exercise 19.1. (Warmup: trace it.) Start with singletons 1 through 8. Using the forest with union by rank (no path compression), perform UNION$(1,2)$, UNION$(3,4)$, UNION$(5,6)$, UNION$(7,8)$, UNION$(1,3)$, UNION$(5,7)$, UNION$(1,5)$. Draw the final tree and give the rank of the root. What is the height of the tree?

Exercise 19.2. (Same set?) In one or two sentences, explain why the test FIND-SET(x) = FIND-SET(y) correctly decides whether x and y are in the same set, and why no separate operation is needed. What property of representatives makes this work?

Exercise 19.3. (The weighted-union doubling argument.) In the linked-list representation with weighted union, prove carefully that a single fixed object's back-pointer is updated at most $\lfloor \log_2 n \rfloor$ times over any sequence of operations. Where exactly does the "set at least doubles" step come from?

Exercise 19.4. (Rank versus height.) Give a concrete sequence of operations (using both union by rank and path compression) after which some node's rank is strictly greater than the actual height of its tree. Explain why this does not break the analysis.

Exercise 19.5. (Path compression alone.) Suppose you use path compression but *not* union by rank – LINK always hangs the first root under the second. Argue informally that the amortized cost per operation is still $O(\log n)$, not the bad $O(n)$ of the naive forest. (Hint: compression keeps ironing the trees flat even without rank.)

Exercise 19.6. (Union by size.) Rewrite MAKE-SET, LINK, and the rank logic to use "union by size" instead: store the number of nodes in each tree at its root, and on a link hang the smaller-size tree under the larger. Prove the analogue of Lemma 19.1: a tree of height h contains at least 2^h nodes, so heights stay $O(\log n)$.

Exercise 19.7. (Counting components.) Augment the structure with a global counter so that after any sequence of operations you can report, in $O(1)$ time, how many disjoint sets currently exist.

Specify exactly where the counter is initialized, incremented, and decremented, and prove it stays correct.

Exercise 19.8. (Kruskal preview.) You are given a graph whose edges arrive one at a time, and you must print "CYCLE" the first time an edge connects two vertices already in the same component, and "ADD" otherwise. Write pseudocode using union-find, and argue its total running time over m edges and n vertices is $O((m+n)\,\alpha(n))$. You have just written the core of Kruskal's algorithm.

Exercise 19.9. (Scale of α, really?) The two-argument Ackermann-style tower satisfies (roughly) $A(2) = 4$, $A(3) = 16$, $A(4) = 65536$, and $A(5) = 2^{65536}$. Using the intuition that $\alpha(n)$ is "the least k with $A(k) \geq n$," explain why $\alpha(n) \leq 4$ for every n that could be stored in the memory of any physically possible computer. Why does this make the "not constant time" result a mathematical curiosity rather than a practical worry?

Exercise 19.10. (Challenge: offline minimum / interval merging.) You process a stream of integers from a fixed range $1 \ldots n$. Some are "insert k" and some are "extract the current minimum." Design a way to use union-find so that each extract finds the next still-present value quickly, by unioning each removed value with its successor. Explain why each value is removed at most once and the whole process is $O((n+q)\,\alpha(n))$ for q operations. (This is the classic "union-find as a pointer to the next free slot" trick.)

Chapter 20

Fibonacci Heaps

In the previous chapter, union-find postponed work lazily until queries required it. We can push this principle of lazy postponement into priority queues. You have met binary heaps back in Chapter 6: a heap lives in an array, it keeps the smallest (or largest) thing on top, and it lets you insert and extract-min in $O(\log n)$ time. For most problems that is plenty.

Some graph algorithms, however, have a heavily skewed workload: they call one priority-queue operation *thousands* of times for every call to another. When that happens, we want to make the frequent operation cheaper even if a rarer operation has to do more cleanup later. That is the exact trade-off a *Fibonacci heap* is designed for. It is the data structure you reach for when you want `decrease-key` to be, on average, nearly free.

This chapter feels different from the binary-heap chapter. There, the structure was rigid and the analysis was direct. Here, the structure is loose, almost messy, and all the cleverness sits in the *amortized* analysis. We will go slowly. By the end you will understand both the mechanics of each operation and the structural reasons why the looseness is deliberate, including why Fibonacci numbers govern the tree sizes.

Focus

The chapter rests on a single trade-off. We let the heap get structurally lazy and messy so that `insert` and `decrease-key` can finish in $O(1)$ amortized time. We pay for that laziness later, all at once, during `extract-min`, which does a big cleanup. In short: cheap deposits now, one expensive cleanup later.

20.1 What problem are we actually solving?

First, who needs a fancier heap?

Think about Dijkstra's shortest-path algorithm, or Prim's minimum-spanning-tree algorithm (we will meet both properly in Chapters 23 and 24; for now just trust the shape of their work). Both algorithms keep a priority queue of vertices keyed by a tentative distance. The main loop does this:

- Pull out the closest unfinished vertex. That is one `extract-min`.
- For each edge leaving that vertex, maybe *lower* the key of the neighbor on the other end,

because we just found a shorter way to reach it. That is a `decrease-key`.

Now count. Over the whole run of the algorithm on a graph with V vertices and E edges, you do at most V extract-mins (one per vertex) but up to E decrease-keys (one per edge, in the worst case). In a dense graph, E can be as large as V^2. So decrease-key is the operation that gets hammered, and extract-min is comparatively rare.

Intuition

A binary heap charges $O(\log n)$ for *both* extract-min and decrease-key. Dijkstra then costs $O((V + E) \log V)$. The $E \log V$ term dominates, and it is entirely the fault of decrease-key being logarithmic.
A Fibonacci heap charges $O(\log V)$ *amortized* for extract-min but only $O(1)$ *amortized* for decrease-key. Now Dijkstra costs $O(V \log V + E)$. We traded a log factor off the busiest operation. On dense graphs that is the difference between $V^2 \log V$ and V^2 work for the relaxation step.

So, before we design anything, here is the target:

Focus

We want a *mergeable* priority queue where:

operation	we will achieve	(binary heap was)
`insert`	$O(1)$ amortized	$O(\log n)$
`minimum`	$O(1)$ worst case	$O(1)$
`extract-min`	$O(\log n)$ amortized	$O(\log n)$
`decrease-key`	$O(1)$ amortized	$O(\log n)$
`union`	$O(1)$ worst case	$O(n)$

That `union` row deserves a word. "Mergeable" means you can take two whole heaps and splice them into one. With an array-based binary heap that is expensive — you essentially rebuild. Fibonacci heaps make it trivial, and the rest of the design builds on that cheap merge.

20.2 The idea: lazy bags of trees

20.2.1 From one tree to a forest

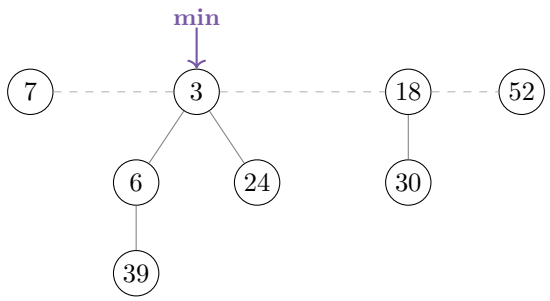
A binary heap is a single tree, kept perfectly balanced inside an array. The balance is what makes it $O(\log n)$ — but maintaining balance is also what makes every operation *pay* that $O(\log n)$.

A Fibonacci heap drops that strict balance requirement. Instead of one carefully shaped tree, it keeps a *collection* of heap-ordered trees. "Heap-ordered" means the same local rule you already know: in every tree, each node's key is $\leq$ the keys of its children. So the minimum of any one tree is at that tree's root. The minimum of the *whole* heap is therefore the smallest root among all the trees — and we keep a pointer straight to it.

Intuition

Picture a tabletop covered with little trees, each one a heap. The roots are laid out in a circle (a doubly linked list joined at the ends), so you can walk from any root to the next. One special pointer, *min*, always aims at the smallest root. That is a Fibonacci heap: a circle of root-tops, with *min* pointing at the lowest.

Here is a small forest. Each tree is heap-ordered, and *min* points at the overall smallest root, 3.



Two things to notice right away. First, the trees can have wildly different shapes and sizes — nobody is enforcing balance. Second, finding the minimum is instant: just read the *min* pointer. We never search for it.

20.2.2 What each node carries

Every node in the forest stores a handful of fields. Keep this list nearby; the operations are just careful manipulations of these pointers.

- **key** — the priority value.
- **parent** — pointer up, or null if this node is a root.
- **child** — pointer to *one* of its children (the rest are reachable from there).
- **left, right** — the node's neighbors in a *circular, doubly linked list*. Roots live in one such list (the root list); siblings under a common parent live in another.
- **degree** — how many children the node currently has.
- **mark** — a single boolean, true or false. This flag is what makes the analysis work; we explain it in Section 20.5.

Focus

Children of a node are kept in a *circular doubly linked list*. That is what makes splicing a node in or out an $O(1)$ pointer update: no shifting, no searching. Every structural move in this chapter is "unhook here, hook in there," and circular doubly linked lists make both ends of that move constant time.

20.3 The easy operations: make, insert, minimum, union

Three of the five core operations are simple because we allow the structure to be lazy.

20.3.1 Make-heap and minimum

`make-heap` returns an empty heap: $min = \text{null}$ and a node count $n = 0$. Done.

`minimum` reads and returns the key at min. One pointer dereference, $O(1)$ worst case, no surprises.

20.3.2 Insert: just drop it on the table

Insert is the laziest operation of all. To insert a new key, we do *not* find where it "belongs." We make a one-node tree out of it and toss it into the root list. If the newcomer is smaller than the current minimum, we swing the min pointer to it. That is the entire operation.

Algorithm 85: FibHeap-Insert(H, x)

Input: a Fibonacci heap H and a new node x with a key
Output: H with x inserted

```
1 x.degree ← 0;  x.parent ← null;  x.child ← null;  x.mark ← false;
2 if H.min = null then
3 |   create a root list containing just x;
4 |   H.min ← x;
5 else
6 |   splice x into H's root list;
7 |   if x.key < H.min.key then
8 |   |   H.min ← x
9 H.n ← H.n + 1;
```

Splicing into a circular doubly linked list is a fixed number of pointer writes, so insert is $O(1)$ in the *worst* case here — even better than the amortized bound we promised. We will see the structure get untidy, and pay for it later.

Common trap

Do not picture insert as "adding a leaf to a tree." There is no single tree. Insert literally creates a brand-new one-node tree sitting beside all the others. After ten inserts into an empty heap, you have ten separate one-node trees in the root list — not one tree of ten nodes. The cleanup that consolidates them happens only when an extract-min forces it.

20.3.3 Union: splice two circles together

Two circular lists can be joined into one in $O(1)$ time — you cut each circle open and stitch the loose ends together. So to union two heaps H_1 and H_2, we concatenate their root lists and keep whichever min is smaller.

Algorithm 86: FibHeap-Union(H_1, H_2)

Input: two Fibonacci heaps
Output: a single heap containing all their nodes

```
H ← a new empty heap;
concatenate the root lists of H1 and H2 into H's root list;
if H1.min = null or (H2.min ≠ null and H2.min.key < H1.min.key) then
    H.min ← H2.min
else
    H.min ← H1.min
H.n ← H1.n + H2.n;
return H;
```

This is why we call the structure *mergeable*. A binary heap cannot do this in $O(1)$; a Fibonacci heap does it efficiently, because the root list is a circular doubly linked list and circles splice together in $O(1)$ pointer operations.

Why this matters

Cheap union matters in practice. Whenever an algorithm builds many small queues and then merges them — think of certain graph algorithms, event-driven simulations, or some computational-geometry sweeps — the $O(1)$ merge can be the deciding factor. It is the same reason linked lists beat arrays for splicing: you pay in pointers, not in copying.

20.4 Extract-min: the one place we clean up

We have been making a mess. Every insert and every union just dumps trees into the root list and walks away. Sooner or later the bill comes due, and it comes due here, in `extract-min`. This is the only operation that is allowed to be slow, and it spends its time doing the housekeeping everyone else skipped.

20.4.1 The two-step plan

Extract-min has a small first step and a big second step.

Step 1 — remove the minimum. The minimum is a root, so its children are a clump of heap-ordered subtrees hanging off it. We snip those children loose and promote them to be roots in their own right (they are still heap-ordered, just no longer under a parent). Then we delete the old minimum node. After this, the root list may be long and lumpy.

Step 2 — consolidate. Now we tidy. We repeatedly merge trees until no two roots have the same *degree* (same number of children). When two roots have equal degree, we *link* them: compare their keys, make the larger-keyed root a child of the smaller-keyed root. That link raises the survivor's degree by one and removes one tree from the root list. We keep linking until all surviving roots have distinct degrees. Finally we scan the survivors to find the new minimum.

Intuition

Consolidation is like making change with coins, where a "coin" is a tree and its denomination is its degree. You are not allowed to keep two coins of the same denomination: whenever you have two degree-d trees, you fuse them into one degree-$(d{+}1)$ tree (a bigger coin). At the end, you hold at most one tree of each degree — so at most $\lfloor \log_\phi n \rfloor + 1$ trees total. That bound is where the $O(\log n)$ comes from.

20.4.2 Linking two trees

The atomic move inside consolidation is LINK. It takes two roots of equal degree and makes one the child of the other, preserving the heap order by demoting whichever root has the larger key.

Algorithm 87: LINK(H, y, x) // assumes $x.key \le y.key$

Input: two roots x, y of equal degree, with x the smaller key
Output: y becomes a child of x

1 remove y from the root list of H;
2 make y a child of x; insert y into x's child list;
3 $x.degree \leftarrow x.degree + 1$;
4 $y.mark \leftarrow$ false;

(We clear $y.mark$ because y is now a freshly attached child with no history of loss yet — again, that flag will make sense in Section 20.5.)

20.4.3 The consolidation procedure

To consolidate, keep an array A indexed by degree. Walk the root list; for each root w, look at its degree d. If $A[d]$ is empty, park w there. If $A[d]$ already holds some root x, link them (the smaller key wins), clear $A[d]$, and try again with the merged tree at degree $d+1$ — cascading upward just like carrying in binary addition. Figure 20.1 pictures the carry: two degree-d trees collide in slot d, fuse into one degree-$(d{+}1)$ tree, and that bigger tree then tries slot $d{+}1$ — which may itself be full, propagating the carry.

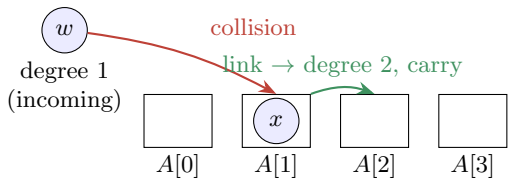


Figure 20.1: Consolidation as binary-addition carry. An incoming degree-1 root w finds slot $A[1]$ already occupied by x. The two are linked into one degree-2 tree, which then carries into slot $A[2]$; if that slot is full too, the carry keeps propagating upward.

Algorithm 88: FibHeap-Extract-Min(H)

Input: a Fibonacci heap H
Output: the node with the smallest key, removed from H

```
z ← H.min;
if z ≠ null then
    foreach child c of z do
        add c to the root list of H;   c.parent ← null;
    remove z from the root list of H;
    if z = z.right (it was the only root) then
        H.min ← null
    else
        H.min ← z.right;   Consolidate(H)
    H.n ← H.n − 1;
return z;
```

Algorithm 89: Consolidate(H)

```
let A[0 .. D(n)] be an array of nulls, where D(n) = ⌊log_φ n⌋;
foreach root w in the root list of H do
    x ← w;   d ← x.degree;
    while A[d] ≠ null do
        y ← A[d];
        if x.key > y.key then
            exchange x ↔ y
        Link(H, y, x);
        A[d] ← null;   d ← d + 1;
    A[d] ← x;
H.min ← null;
foreach slot i with A[i] ≠ null do
    add A[i] to a fresh root list; update H.min if A[i] is smaller;
```

20.4.4 A fully traced example

Let us watch consolidation happen. Suppose extract-min has just removed the old minimum and promoted its children, leaving this root list (degrees written underneath):

root key	23	7	21	3	17
degree	0	1	0	2	0

We process roots left to right, parking them in A by degree.

1. **23**, degree 0. $A[0]$ empty $\Rightarrow$ park: $A[0] = 23$.
2. **7**, degree 1. $A[1]$ empty $\Rightarrow$ park: $A[1] = 7$.

3. **21**, degree 0. $A[0] = 23$ is occupied. Link: compare 21 vs 23; 21 is smaller, so 23 becomes a child of 21. Now 21 has degree 1. Clear $A[0]$, move up to degree 1. But $A[1] = 7$ is occupied. Link: 7 vs 21; 7 is smaller, so 21 (carrying 23 under it) becomes a child of 7. Now 7 has degree 2. Clear $A[1]$, move to degree 2. $A[2]$ empty $\Rightarrow$ park: $A[2] = 7$.

4. **3**, degree 2. $A[2] = 7$ occupied. Link: 3 vs 7; 3 is smaller, so 7 becomes a child of 3. Now 3 has degree 3. Clear $A[2]$, move to degree 3. $A[3]$ empty $\Rightarrow$ park: $A[3] = 3$.

5. **17**, degree 0. $A[0]$ empty $\Rightarrow$ park: $A[0] = 17$.

The survivors are $A[0] = 17$ (degree 0) and $A[3] = 3$ (degree 3). All degrees distinct, as promised. The new minimum is 3. Figure 20.2 illustrates the three operational stages of this consolidation: the snipped minimum releasing promoted roots, the cascading links in degree array A, and the final survivor list. Figure 20.3 then details the internal subtree structure of the resulting degree-3 winner. Note that 3 already had two children of its own before this extract-min (it entered with degree 2); the consolidation then hung the whole degree-2 tree rooted at 7 underneath it, pushing 3 to degree 3. The newly built part — 7 over 21 over 23 — is fully determined by the trace; the two children 3 carried in from before (drawn as □) had keys we were never told, so we leave them unlabelled.

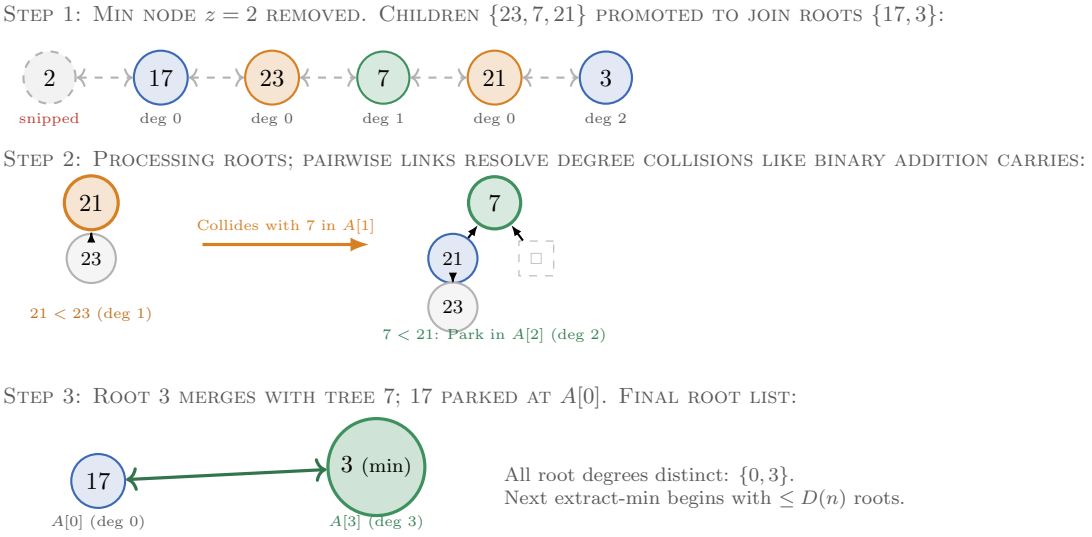


Figure 20.2: The mechanics of root list consolidation during EXTRACT-MIN. Roots are inspected sequentially. Collisions in degree array A trigger tree links that propagate like binary addition carries until each surviving root has a distinct degree.

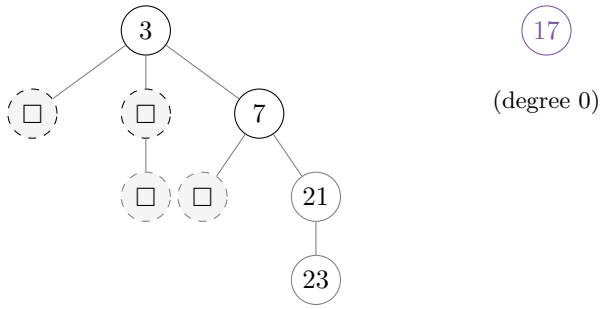


Figure 20.3: Result of the traced consolidation: two survivors, 3 (degree 3) and 17 (degree 0), of distinct degree. Dashed □ nodes are children whose keys the trace never pinned down; the solid nodes $7, 21, 23$ are exactly what the linking built. The new minimum is 3.

(The exact shapes of the unlabelled children depend on the heap's earlier history; what matters is the *degree* structure. A degree-3 root ends up with three children of degrees $0, 1, 2$, the same clean "binomial" shape you get from repeated equal-degree linking. That shape follows directly from the linking rule, and it is the next thing we will explain.)

Algorithm lab

Trace it yourself. Start with an empty heap and insert the keys $5, 2, 9, 1, 7, 4$ in that order. You now have six one-node trees with $min = 1$. Call extract-min once. Carry out CONSOLIDATE by hand using a degree array $A[0..2]$. Track A after each root you process, and draw the final forest. You should finish with $min = 2$ and a small handful of trees with distinct degrees. Then call extract-min a second time and repeat. Watching the degree array "carry" like binary addition is the best way to understand consolidation.

20.5 Decrease-key: the operation we built all this for

Now we turn to the operation that motivates the entire structure. We are handed a pointer to a node x and a new, smaller key. We must update x's key and restore heap order — and we want it done in $O(1)$ amortized time.

20.5.1 The naive idea, and why it is not enough

When we lower x's key, the only thing that can break is the relationship with x's *parent*: maybe x is now smaller than its parent. Heap order between x and its children is still fine (we only made x smaller). So the fix is local: if x violates heap order with its parent, *cut* x away from its parent and move x (with its whole subtree) up to the root list, where it has no parent to violate.

That single cut is $O(1)$. But if we allow unlimited cutting, a parent can lose child after child and degenerate into a sparse, spindly tree. Spindly trees ruin our degree bound: the $O(\log n)$ bound for extract-min rested on trees being "bushy" — a degree-d root having close to ϕ^d descendants. Lose that, and consolidation no longer runs in $O(\log n)$.

Common trap

The tempting bug is to cut a violating node and stop. That keeps decrease-key fast, but over many decrease-keys a single parent can be stripped down to almost nothing, destroying the size-versus-degree guarantee. The trees thin out, and extract-min degrades toward linear time. You must limit how much a node is allowed to lose *before* it too gets cut loose.

20.5.2 The mark bit and cascading cuts

The fix is a simple rule. Each non-root node carries a **mark** bit. The marking discipline is:

- A node's mark is cleared whenever it *becomes* a child (during a link).
- The first time a node *loses* a child (because that child was cut away by a decrease-key), we *mark* it. It is now "on notice."
- The *second* time a marked node loses a child, we do not merely mark it; we cut *it* loose from its own parent too, sending it up to the root list. And since cutting it counts as its parent losing a child, this can trigger the same rule one level up: a *cascading cut* that ripples toward the root.

Intuition

The mark bit is a "you have lost a child once; lose another and you are out" warning. A node may shed at most one child while staying put. On the second loss it gets evicted to the root list along with everything still hanging off it. This caps how lopsided any tree can get: a node can be missing at most a bounded fraction of its children, so trees stay bushy and the degree bound survives.

Algorithm 90: FibHeap-Decrease-Key(H, x, k)

Input: heap H, node x, new key $k \le x.key$
Output: H with x's key lowered to k and heap order restored

```
1 x.key ← k;
2 y ← x.parent;
3 if y ≠ null and x.key < y.key then
4 |   Cut(H, x, y);
5 |   Cascading-Cut(H, y);
6 if x.key < H.min.key then
7 |   H.min ← x
```

Algorithm 91: Cut(H, x, y)

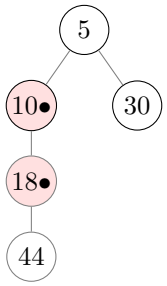
```
1 remove x from y's child list;   y.degree ← y.degree − 1;
2 add x to the root list of H;
3 x.parent ← null;   x.mark ← false;
```

Algorithm 92: CASCADING-CUT(H, y)

```
1 z ← y.parent;
2 if z ≠ null then
3 |   if y.mark = false then
4 |   |   y.mark ← true
5 |   else
6 |   |   CUT(H, y, z);   CASCADING-CUT(H, z)
```

20.5.3 A traced decrease-key

Take a tree rooted at 5 with this shape (marks shown as a •):



Both 10 and 18 are marked — each has already lost one child in the past. Now we call decrease-key on 44, lowering it to 2.

1. $44 \to 2$. Its parent is 18, and $2 < 18$, so heap order is violated. CUT 2 from 18: node 2 flies up to the root list, unmarked. Now 18 has lost a child.
2. CASCADING-CUT(18): 18 is already marked, so we cut *it* too. Node 18 flies up to the root list (still carrying nothing now, since 44/2 already left), unmarked. Now 10 has lost a child.
3. CASCADING-CUT(10): 10 is already marked, so we cut *it* too. Node 10 flies up to the root list, unmarked. Now 5 has lost a child.
4. CASCADING-CUT(5): 5 is a root (no parent), so we stop.

The single decrease-key triggered a chain of three cuts. Figure 20.4 shows the before-and-after. The root list grew by three trees (2, 18, 10), each now unmarked. One decrease-key, several cuts — and yet we are about to prove that decrease-key still costs only $O(1)$ *amortized*. The reason is that all those cuts were "pre-paid" by the marks.

Walking question

Before you read the analysis, go for a walk with this puzzle. A decrease-key can, in the worst single case, trigger a long cascade of cuts — so its *actual* cost is not constant at all; it can be $\Theta(\log n)$ for one unlucky call. Yet we are going to call the cost $O(1)$ amortized. Where could the "saved up" money for a long cascade have come from? Hint: think about what had to happen, earlier and cheaply, for all those nodes to become marked in the first place.

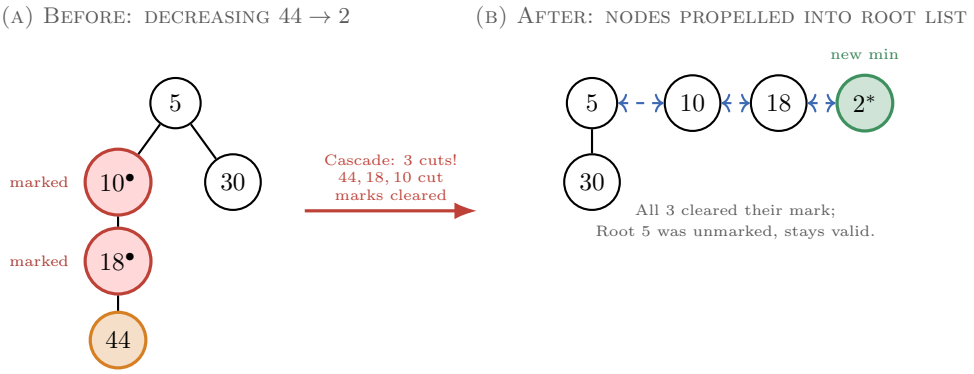


Figure 20.4: A cascading cut during Decrease-Key. Key 44 is decreased to $2 < 18$, prompting an initial cut. Because parent 18 and grandparent 10 were already marked (each having previously lost a child), the loss triggers a chain reaction of cascading cuts. Both 18 and 10 are severed, stripped of their marks, and placed in the root list. The potential $\Phi = t + 2m$ drops by $\Delta\Phi \leq -1$, paying for the cascade.

20.6 The amortized analysis

Now we make the $O(1)$ and $O(\log n)$ claims precise using the *potential method* you met in Chapter 17. If that method is hazy, here is the one-line reminder: we invent a "bank balance" Φ that depends only on the current state, and we define the *amortized* cost of an operation as its real cost plus the change in the balance, $\hat{c} = c + \Delta\Phi$. If the balance never goes below its starting value, the total amortized cost is a valid upper bound on the total real cost.

20.6.1 Choosing the potential

We need a potential that goes *up* cheaply when we are being lazy (so future cleanups are pre-funded), and goes *down* when we finally clean up (refunding the cleanup). Two things accumulate during laziness: extra trees in the root list, and marked nodes. So we charge for both.

Formal

For a Fibonacci heap H, let $t(H)$ be the number of trees in the root list and $m(H)$ the number of marked nodes. Define the potential

$$\Phi(H) \;=\; t(H) \;+\; 2\,m(H).$$

An empty heap has $\Phi = 0$, and Φ is never negative, so amortized costs sum to an upper bound on real costs.

Why $2m$ rather than simply m? Because a cascading cut does two jobs at once when it fires: it removes a mark *and* it creates a new tree. We need each mark to have stored enough to pay for both. Two units per mark is exactly the right deposit, as the decrease-key analysis will show.

20.6.2 Insert, minimum, union

Insert. Real cost $O(1)$. It adds one tree, so $\Delta t = +1$; marks unchanged. Thus $\Delta\Phi = +1$ and the amortized cost is $O(1) + 1 = O(1)$.

Minimum and Union. Neither changes t or m in any meaningful way (union just concatenates root lists; the tree count of the combined heap equals the sum). $\Delta\Phi = 0$, amortized cost $O(1)$.

20.6.3 Extract-min

Let $D(n)$ be the maximum possible degree of any node in an n-node Fibonacci heap. We will prove in Section 20.7 that $D(n) = O(\log n)$; take it on loan for now.

Real cost. Step 1 promotes the children of the minimum — at most $D(n)$ of them — to roots: $O(D(n))$. Step 2, consolidation, walks the root list. After step 1 the root list has at most $D(n) + t(H) - 1$ trees (the original trees, minus the extracted root, plus its promoted children). Each LINK reduces the number of roots by one and costs $O(1)$, and the final scan to rebuild the root list and find the new min touches each surviving slot once, of which there are at most $D(n) + 1$. So the real cost is $O(D(n) + t(H))$.

Potential change. Before consolidation, $t(H)$ trees; afterward, at most $D(n)+1$ trees (one per distinct degree). Marks only ever decrease here. So

$$\Delta\Phi \;\le\; \big[(D(n) + 1) + 2m(H)\big] - \big[t(H) + 2m(H)\big] \;=\; D(n) + 1 - t(H).$$

Amortized cost.

$$\hat{c} = O(D(n) + t(H)) + \big(D(n) + 1 - t(H)\big) = O(D(n)) = O(\log n).$$

The $t(H)$ terms cancel: the more trees there were to clean up (high real cost), the more potential we release (big refund). That cancellation is exactly what the potential was designed to produce. Extract-min is $O(\log n)$ amortized.

20.6.4 Decrease-key

Now let us see how the stored marks pay for cascading cuts. Suppose a single decrease-key triggers c cuts in total (one direct cut, then $c - 1$ cascading cuts).

Real cost. The direct cut plus $c - 1$ cascading cuts, each $O(1)$: total $O(c)$.

Potential change. Track the two pieces of $\Phi = t + 2m$:

- **Trees.** Every one of the c cuts moves a node to the root list, so t increases by c. That is $\Delta t = +c$.
- **Marks.** Each of the $c - 1$ cascading cuts was performed *on a marked node*, and CUT clears that mark: that removes $c - 1$ marks. The cascade stops at one node that was unmarked and now becomes marked: that adds back at most 1 mark. Net: m changes by at most $-(c - 1) + 1 = -(c - 2) = 2 - c$. So $2\,\Delta m \le 2(2 - c) = 4 - 2c$.

Adding up,

$$\Delta\Phi \;\le\; c + (4 - 2c) \;=\; 4 - c.$$

Amortized cost.

$$\hat{c} \;=\; O(c) + (4 - c).$$

Write the $O(c)$ as $a\,c$ for some constant a by choosing the potential units large enough that each cut's real work is at most one unit of potential ($\hat{c} = c + (4 - c) = 4 = O(1)$). The c terms cancel exactly: the real cost of a long cascade is paid, unit for unit, by the marks that the cascade releases. What remains is a constant.

Focus

A single decrease-key might really do $\Theta(\log n)$ work via a long cascade. But every cut in that cascade spends a mark that was *deposited earlier*, one cheap unit at a time, by previous decrease-keys. The bank balance $\Phi = t + 2m$ exactly funds the cascade. So no matter how the work clusters, decrease-key is $O(1)$ amortized. The factor of 2 on m is what lets each mark pay for both the cut's work and the new tree the cut creates.

20.6.5 Delete

To delete an arbitrary node x: decrease its key to $-\infty$ (which floats it up to become the minimum), then extract-min. Amortized cost $O(1) + O(\log n) = O(\log n)$.

Cost summary

With potential $\Phi = t + 2m$, every bound is amortized except where noted:

MAKE-HEAP	$O(1)$	worst case
MINIMUM	$O(1)$	worst case
INSERT	$O(1)$	
UNION	$O(1)$	worst case
EXTRACT-MIN	$O(\log n)$	
DECREASE-KEY	$O(1)$	
DELETE	$O(\log n)$	

Here n is the number of nodes and $D(n) = O(\log n)$ is the maximum degree.

20.7 Why "Fibonacci"? The degree bound

We have leaned twice on the claim $D(n) = O(\log n)$: a node with n descendants cannot have huge degree. Now we prove it; the proof is also where the data structure gets its name.

20.7.1 What the mark rule guarantees

The mark-and-cascade rule has a precise structural consequence. Consider any node x and list its current children in the order they were linked to x: $y_1, y_2, \ldots, y_k$, where y_i was the i-th child attached.

Lemma 20.1 (The size each child must carry)**.** *With the children ordered by time of attachment, the i-th child y_i has degree at least $i - 2$ (for $i \geq 2$); and y_1 has degree at least 0.*

Intuition

Here is why. When y_i was linked to x, that link happened during a consolidation between two roots of *equal* degree. At that moment x already had children $y_1, \ldots, y_{i-1}$, so x's degree was $i-1$, and therefore y_i's degree was also $i-1$. Since then, y_i is allowed to lose *at most one* child before it would have been cut away from x entirely (the mark rule!). So y_i's degree now is at least $(i-1)-1 = i-2$. The mark bit is what limits the loss to that one child.

20.7.2 Fibonacci numbers fall out

Let s_d be the *minimum* possible number of nodes in a subtree whose root has degree d. We want a lower bound on s_d. The root itself is one node. Its children, by Lemma 20.1, have degrees at least $0, 0, 1, 2, \ldots, d-2$. So the smallest such subtree satisfies

$$s_d \;\geq\; 1 + 1 + \sum_{i=2}^{d} s_{i-2} \;=\; 2 + \sum_{j=0}^{d-2} s_j,$$

with $s_0 = 1$ and $s_1 = 2$. If you have seen the Fibonacci recurrence $F_k = F_{k-1} + F_{k-2}$, the next line will feel familiar. One can show by induction that

$$s_d \;\geq\; F_{d+2},$$

where F_k are the Fibonacci numbers ($F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, \ldots$). The sum-of-all-previous form above collapses, via the identity $F_{d+2} = 1 + \sum_{i=0}^{d} F_i$, into the familiar two-term recurrence. This is where the data structure gets its name: the worst-case bushiness of its trees is governed by exactly the Fibonacci sequence.

Figure 20.5 draws the smallest possible trees of each degree and counts their nodes; the counts are exactly $s_0, s_1, s_2, s_3 = 1, 2, 3, 5$ — the Fibonacci numbers F_2, F_3, F_4, F_5.

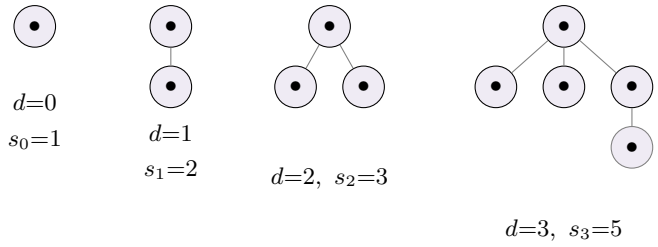


Figure 20.5: The leanest trees a degree-d root can be reduced to under the mark rule, for $d = 0, 1, 2, 3$. Their node counts $1, 2, 3, 5$ are the Fibonacci numbers F_{d+2}. The children are forced to have degrees $0, 0, 1, \ldots, d-2$, and feeding those minimal sizes into $s_d \geq 2 + \sum_{j=0}^{d-2} s_j$ makes the counts satisfy $s_d = s_{d-1} + s_{d-2}$ — the Fibonacci recurrence.

Formal

The size–degree bound. A node of degree d in a Fibonacci heap has at least $F_{d+2} \geq \phi^d$ descendants (including itself), where $\phi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the golden ratio. Consequently, if a

heap has n nodes, the maximum possible degree is

$$D(n) \leq \log_\phi n = O(\log n).$$

History and connection

Fibonacci heaps were invented by Michael L. Fredman and Robert E. Tarjan in 1984 (published in 1987). Their explicit motivation was breaking the asymptotic bottleneck of Dijkstra's single-source shortest-path algorithm and Prim's minimum-spanning-tree algorithm. On a dense graph with V vertices and $E \approx V^2$ edges, binary heaps spend $O(E \log V)$ total time, with almost all time spent repeatedly lowering priority keys during edge relaxation (updating a neighbor's tentative distance when a newly explored edge offers a shorter route). Fredman and Tarjan designed a priority queue that reduced `decrease-key` from $O(\log n)$ to $O(1)$ amortized, achieving the landmark $O(V \log V + E)$ running time.

The natural temptation when making `decrease-key` fast is to cut any violating child and immediately move it to the root list. This naive cut is $O(1)$ worst-case, but it rapidly ruins the heap: over thousands of cuts, a parent can be stripped of nearly all its children, degenerating trees into tall, spindly chains. Without a bound on tree bushiness, a node of degree d might contain only $O(d)$ descendants rather than exponential descendants, blowing up the degree bound $D(n)$ to $\Theta(n)$ and causing `extract-min` consolidation to degrade to $\Theta(n)$ time. Fredman and Tarjan solved this with the mark bit: allowing a node to lose at most one child before being evicted itself guarantees that tree sizes grow as $s_d \geq F_{d+2} \geq \phi^d$, maintaining $D(n) \leq \log_\phi n = O(\log n)$.

Fibonacci heaps became the theoretical gold standard for priority queues, delivering $O(V \log V + E)$ shortest paths and minimum spanning trees, along with $O(1)$ amortized insertion and merging. The price is high constant factors and pointer overhead. Each node requires four pointers (parent, left sibling, right sibling, child), an integer degree, and a boolean mark (amounting to dozens of bytes per element). On modern CPUs with deep cache hierarchies, the pointer-chasing overhead of Fibonacci heaps makes them slower in practice than 4-ary heaps or self-adjusting pairing heaps except on extremely large graphs. Designing a priority queue that achieves $O(1)$ decrease-key with strict worst-case bounds, minimal pointer overhead, and cache locality is still an open research problem.

Why this matters

The golden ratio ϕ has a concrete role here. Because each node of degree d is forced to hide at least ϕ^d descendants, degree grows only *logarithmically* in the number of nodes, which is exactly what makes consolidation's tidy-up cost $O(\log n)$. Loosen the mark rule and the base of that logarithm degrades; tighten it and you would pay more per decrease-key. Fredman and Tarjan found the balance point, and the golden ratio is what falls out of it.

20.8 Putting it back in context: Dijkstra and Prim

Now we can return to the question from Section 20.1. Recall Dijkstra on a graph with V vertices and E edges performs:

- one `insert` per vertex: V inserts, $O(V)$ amortized total;
- one `extract-min` per vertex: V extracts, $O(V \log V)$ amortized total;
- up to one `decrease-key` per edge during relaxation: E decrease-keys, $O(E)$ amortized total.

Add them: $O(V \log V + E)$. Compare with the binary-heap implementation's $O((V + E) \log V) = O(E \log V)$ on a connected graph. On a *dense* graph where $E = \Theta(V^2)$, the Fibonacci version does $O(V^2)$ work for relaxation instead of $O(V^2 \log V)$ — a clean $\log V$ saved, exactly where the algorithm spends most of its time.

Why this matters

Textbooks sometimes oversell Fibonacci heaps, so a caveat is in order. The constant factors and pointer-chasing overhead are real, and on the *sparse* graphs that dominate practice ($E = O(V)$), a plain binary heap or a d-ary heap is usually faster on the wall clock despite its worse asymptotics. So in everyday code, Fibonacci heaps are rare. Their importance is twofold: they prove what is theoretically *possible* (the best known asymptotic bound for Dijkstra in the comparison model for a long time), and the amortized-analysis techniques they teach — lazy structure plus a potential that pre-funds cleanup — recur all over algorithm design.

20.9 Summary

Focus

- A Fibonacci heap is a *forest* of heap-ordered trees with a pointer to the minimum root. It is deliberately lazy.
- `insert` and `union` just splice into the root list: $O(1)$. `minimum` reads a pointer: $O(1)$.
- `extract-min` is the one cleanup: promote the min's children, then *consolidate* so that no two roots share a degree. This is $O(\log n)$ amortized.
- `decrease-key` cuts a violating node up to the root list and uses the *mark bit* to bound how lopsided trees can get, via *cascading cuts*. It is $O(1)$ amortized.
- The potential $\Phi = t(H) + 2m(H)$ (trees plus twice the marked nodes) funds the cleanups: long cascades are paid for by marks deposited earlier.
- The mark rule forces a degree-d node to hide at least $F_{d+2} \geq \phi^d$ descendants, so $D(n) = O(\log n)$ — which is where "Fibonacci" comes from.
- Net effect: Dijkstra and Prim drop to $O(V \log V + E)$, a real win on dense graphs.

Fibonacci heaps push comparison-based priority queues to their theoretical limits: as long as keys can only be compared pairwise, an $\Omega(\log n)$ lower bound governs sorting and priority queue extractions. To beat this bound, we have to leave the comparison-only model.

What if our keys are not arbitrary opaque objects, but integers drawn from a bounded universe $\{0, 1, \ldots, u - 1\}$? In **Chapter 21 (van Emde Boas Trees)**, we drop the comparison model entirely. By decomposing an integer universe recursively into clusters of size $\sqrt{u}$, van Emde Boas trees achieve $O(\log \log u)$ time for search, predecessor, successor, minimum, maximum, insert, and

delete — so even for a universe of four billion 32-bit integers, every operation takes fewer than six recursive steps.

Exercises

Exercise 20.1 (Warmup: insert only)**.** Start from an empty Fibonacci heap and insert the keys $8, 3, 11, 1, 6$ in order. Draw the heap after each insert, showing the root list and the *min* pointer. How many trees are in the root list at the end, and what is Φ?

Exercise 20.2 (Consolidation by hand)**.** Continue from the previous exercise and call EXTRACT-MIN once. Carry out CONSOLIDATE using a degree array, writing down the contents of A after each root is processed. Draw the resulting forest and give the new *min*.

Exercise 20.3 (Cascading cut)**.** Build (or accept as given) a tree rooted at 4 with a chain of children $4 \to 9 \to 15 \to 26 \to 40$, where 9, 15, and 26 are all marked. Call decrease-key on 40, lowering it to 1. List every CUT and CASCADING-CUT that fires, and state how many trees the root list gains.

Exercise 20.4 (Potential bookkeeping)**.** For the decrease-key in the previous exercise, compute the real cost (count the cuts), the change $\Delta\Phi$ in the potential $\Phi = t + 2m$, and the amortized cost $\hat{c} = c + \Delta\Phi$. Confirm it is a constant.

Exercise 20.5 (Why twice the marks?)**.** Redo the decrease-key amortized analysis of Section 20.6.4 using the potential $\Phi' = t(H) + m(H)$ (only *one* unit per mark). Show that the c terms no longer cancel, and identify the exact step that breaks. Explain in one sentence why the coefficient 2 is necessary.

Exercise 20.6 (Degree bound, small cases)**.** Using $s_0 = 1$, $s_1 = 2$, and $s_d \geq 2 + \sum_{j=0}^{d-2} s_j$, compute lower bounds s_2, s_3, s_4, s_5. Compare each to the Fibonacci number F_{d+2} and to ϕ^d. At what degree does a node first guarantee at least 20 descendants?

Exercise 20.7 (Worst-case versus amortized)**.** Construct a sequence of operations on a Fibonacci heap in which a *single* decrease-key really performs $\Theta(\log n)$ cuts. (Hint: you must first arrange a long chain of marked nodes; describe how a sequence of decrease-keys and extract-mins can set that up.) Explain why this does not contradict the $O(1)$ amortized bound.

Exercise 20.8 (Dijkstra accounting)**.** On a graph with $V = 1000$ vertices and $E = 400{,}000$ edges, write the total amortized cost of Dijkstra using (a) a binary heap and (b) a Fibonacci heap, as expressions in V and E. Which wins asymptotically here, and roughly by what factor in the relaxation term? Would your answer change for a sparse graph with $E = 3V$?

Exercise 20.9 (Mergeable advantage)**.** Describe a scenario — a concrete algorithm or workload — where the $O(1)$ UNION of a Fibonacci heap is the deciding advantage over a binary heap. Estimate the asymptotic saving for k merges of heaps with n total elements.

Exercise 20.10 (Challenge: no decrease-key?)**.** Some applications never call decrease-key (for example, certain event-simulation queues). In that restricted setting, the mark bit and cascading cuts are never exercised. Argue what the amortized bounds become, and explain why, in this decrease-key-free world, a simpler "lazy binomial heap" suffices and the Fibonacci machinery buys you nothing.

Chapter 21

van Emde Boas Trees

In Chapter 20, Fibonacci heaps demonstrated how amortized accounting drives insertion and decrease-key down to $O(1)$ time, yet extracting the minimum remained bounded by $\Omega(\log n)$ comparisons. Every comparison-based data structure for maintaining an ordered set—balanced binary search trees, red-black trees, B-trees, or heaps—must charge $\Omega(\log n)$ to locate or extract a key, bounded by the information-theoretic floor on pairwise comparisons.

What happens if keys are not abstract objects, but integers drawn from a known, bounded range $\{0, 1, \ldots, u-1\}$—such as 32-bit network addresses, millisecond timestamps within a day, or memory offsets? In that domain, we can bypass the comparison barrier entirely: we can *look at the bits* and index directly into memory. Once key bits become array addresses, an exponential speedup becomes possible. The van Emde Boas tree (written vEB) supports Insert, Delete, Member, Minimum, Maximum, Successor, and Predecessor all in

$$O(\lg \lg u)$$

time, where u is the size of the *universe* of possible keys – not the number of keys you happen to be storing. For a 32-bit universe, $u = 2^{32}$, and $\lg \lg u = \lg 32 = 5$. Exactly five levels of descent suffice for any query, regardless of whether the structure holds ten keys or ten million, and regardless of how keys are distributed.

This chapter builds the structure step by step. We begin with a flat bit vector and refine it; each refinement removes the bottleneck of the previous design, until we reach the $\lg \lg u$ running time.

21.1 The problem, stated carefully

Fix a universe size u. For this whole chapter we assume u is a power of two; write $u = 2^k$. The keys we store are integers from the set $\{0, 1, 2, \ldots, u-1\}$. We want a *dynamic set* $S \subseteq \{0, \ldots, u-1\}$ supporting these operations:

- Member(S, x): is $x \in S$?
- Insert(S, x), Delete(S, x): add/remove a key.
- Minimum(S), Maximum(S): the smallest/largest key, or nil if S is empty.

- Successor(S, x): the smallest key in S strictly greater than x (or NIL). Note x itself need not be in S.
- Predecessor(S, x): the largest key strictly less than x.

The hard operations are Successor and Predecessor. Membership and min/max are easy to make fast in isolation; the difficulty is making *successor* fast *at the same time*. A hash table (Chapter 11) achieves Member and Insert in $O(1)$ expected time but is ill-suited for successor: the keys are scattered across buckets, so locating the next key above x requires scanning. A balanced BST (Chapters 12–13) supports successor in $O(\log n)$ but pays $O(\log n)$ for every operation. We want all operations to be fast simultaneously, taking advantage of the integer universe.

Focus

The cost is measured against u, the *universe size*, not n, the number of stored keys. We trade a dependence on key count n for a mild double-logarithmic dependence on the universe size u.

Why this matters

Where does a bounded-integer universe with fast successor actually show up? Network routers perform *longest-prefix matching* on 32- or 128-bit IP addresses – essentially repeated successor and predecessor queries on integer keys. Event simulators and operating system schedulers keep queues of integer timestamps and repeatedly ask "what is the next scheduled event?" Memory allocators track free blocks by integer addresses. In each case, n can be large while u is fixed by the machine word size, and shaving $\log n$ down to $\lg \lg u$ enables guaranteed real-time response.

21.2 Idea zero: the bit vector

The simplest structure that can store a subset of $\{0, \ldots, u-1\}$ is a single array of u bits. Call it A, with $A[i] = 1$ exactly when $i \in S$.

Intuition

Picture a long row of u light switches, one per possible key, all initially off. Inserting x flips switch x on; deleting flips it off; membership is just "is switch x on?" To find the successor of x, you stand at switch x and walk to the right until you hit the next lit switch.

Member, Insert, and Delete run in immediate $O(1)$ time by indexing straight to bit x. But Successor is slow: to find the next key after x, we must scan $A[x+1], A[x+2], \ldots$ sequentially until we encounter a 1 or reach the end. In the worst case – say $S = \{0, u-1\}$ and we ask for the successor of 0 – we scan nearly all u bits. Thus:

Cost summary

Bit vector. Member/Insert/Delete: $O(1)$. Successor/Predecessor/Minimum/Maximum: $O(u)$ worst case. Space: $O(u)$. Eliminating

that $O(u)$ successor scan is the goal of the rest of the chapter.

21.3 Idea one: a tree of summaries (the layered approach)

The bit vector's failing is that successor has to scan a long, flat row with no shortcuts. The classic fix for "I need shortcuts over a flat array" is to build a summary on top. Lay a complete binary tree over the u bits, with the bits as leaves. Each internal node stores the OR of its two children: a node is 1 if *any* leaf beneath it is 1 (i.e. that subtree contains a key). Figure 21.1 shows such a tree for a tiny universe.

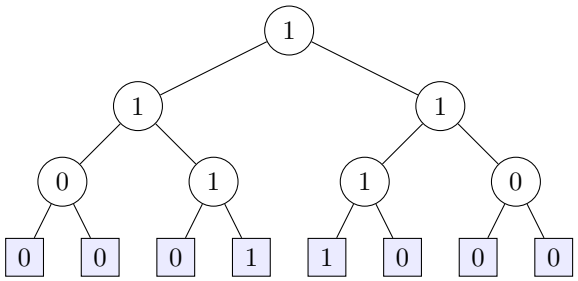


Figure 21.1: A summary tree over $u = 8$ for the set $S = \{3, 4\}$. Leaves are the bit vector; each internal node is the OR of its children. Successor of $x = 3$: climb until you find a node whose right subtree is nonempty, cross over, descend to the leftmost 1.

How does this help successor? To find the successor of x, start at leaf x and *climb*. At each step, look at whether the sibling subtree to your right contains a 1 (its summary node is 1). The moment it does, cross into it and *descend*, always taking the leftmost child that is a 1, until you reach a leaf. That leaf is the successor. Climbing is $O(\log u)$ and descending is $O(\log u)$, so successor is now $O(\log u)$.

That already matches a balanced BST, and we did it without comparisons. But $O(\log u)$ is not our target; we want $O(\lg \lg u)$. The summary tree has $\log u$ levels and we walk all of them. To go faster we need a tree that is *much shallower* – and the way to get a shallow tree is to give each node many children instead of two.

Walking question

Before reading on, consider this question: the binary summary tree has branching factor 2 and height $\log_2 u$. If instead each node had $\sqrt{u}$ children, how tall would the tree be? Work out the height for $u = 2^{16}$ under branching factor 2 versus branching factor $\sqrt{u} = 2^8$. The gap you find is exactly the gap between $\log u$ and the answer we are chasing.

21.4 Idea two: split the key, build proto-vEB

To achieve a tree depth of $\lg \lg u$, we change the branching factor: instead of branching by 2, we branch by $\sqrt{u}$. Take a key $x \in \{0, \ldots, u-1\}$ represented by $k = \lg u$ bits, and partition its bit representation into two equal halves. The *high half* – the more significant $k/2 = \frac{1}{2} \lg u$ bits – indexes

one of $\sqrt{u}$ "clusters." The *low half* – the lower $k/2$ bits – indexes a position *within* that cluster, selecting one of $\sqrt{u}$ relative slots.

Formally, with $u = 2^k$ and $\sqrt{u} = 2^{k/2}$, define

$$\text{high}(x) = \left\lfloor \frac{x}{\sqrt{u}} \right\rfloor, \qquad \text{low}(x) = x \bmod \sqrt{u}, \qquad \text{index}(h, \ell) = h\sqrt{u} + \ell.$$

In hardware and modern programming languages, these operations require no arithmetic division or multiplication. Because $\sqrt{u} = 2^{k/2}$, they correspond to direct bit-level operations:

$$\text{high}(x) = x \gg (k/2), \qquad \text{low}(x) = x \;\&\; (2^{k/2} - 1), \qquad \text{index}(h, \ell) = (h \ll (k/2)) \mid \ell.$$

The index operation reassembles the two halves into the original integer, so $x = \text{index}(\text{high}(x), \text{low}(x))$ for all x. Each operation executes in $O(1)$ time with single machine instructions.

Example 21.1 (Splitting a key)**.** Let $u = 16$, so $\sqrt{u} = 4$ and keys are 4-bit numbers. Take $x = 13$, which in binary is 1101_2. The high two bits are $11_2 = 3$ and the low two bits are $01_2 = 1$. Indeed

$$\text{high}(13) = \lfloor 13/4 \rfloor = 3, \quad \text{low}(13) = 13 \bmod 4 = 1, \quad \text{index}(3, 1) = 3 \cdot 4 + 1 = 13. \checkmark$$

Key 13 resides in cluster 3 at internal offset 1.

Intuition

Think of the u keys as the cells of a $\sqrt{u} \times \sqrt{u}$ grid, read row by row (Figure 21.2). The high half of a key is its *row* (which cluster); the low half is its *column* (where in the cluster). Key 13 in a 4×4 grid sits in row 3, column 1. Successor now has a two-part flavor: either the next key is later in the *same row*, or it is the first key of some *later nonempty row*.

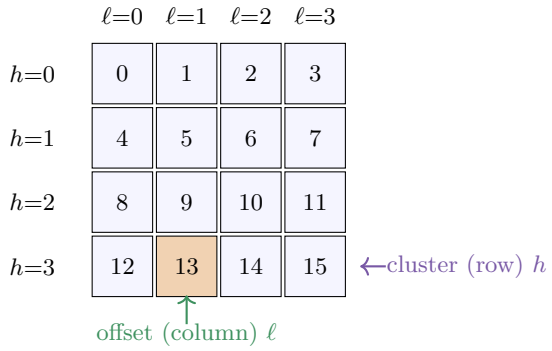


Figure 21.2: The $u = 16$ universe drawn as a $\sqrt{u} \times \sqrt{u} = 4 \times 4$ grid, read row by row. A key's value is high $\cdot \sqrt{u} +$ low. Key 13 (highlighted) sits in row $h = 3$, column $\ell = 1$. Clusters are rows; "successor" means "next filled cell scanning left-to-right, top-to-bottom."

We now build a recursive structure to match. A *proto-vEB* structure of size u, written proto-vEB(u), contains:

- if $u = 2$ (the base case), a tiny 2-bit array $A[0..1]$ holding the set directly;
- otherwise, an array $cluster[0..\sqrt{u} - 1]$ of $\sqrt{u}$ pointers, each to a proto-vEB($\sqrt{u}$) holding one cluster's worth of keys, plus one extra proto-vEB($\sqrt{u}$) called *summary*.

The *summary* is the recursive analog of the layered tree's OR-nodes: *summary* records *which clusters are nonempty*. Bit i of the summary is 1 exactly when *cluster*$[i]$ contains at least one key. The summary is itself a proto-vEB over a universe of size $\sqrt{u}$ – with one slot per cluster – so it answers "which is the next nonempty cluster?" by its *own* recursive successor query. Figure 21.3 illustrates this two-level recursive structure.

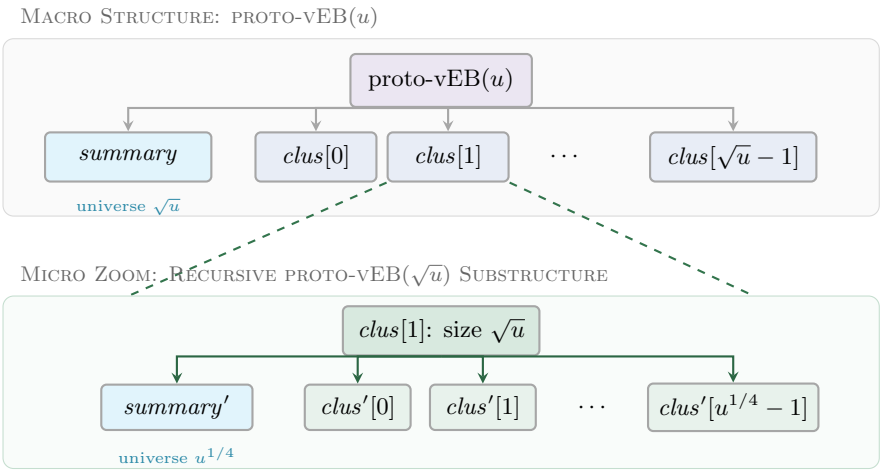


Figure 21.3: The recursive partition of the universe. Top lane: a proto-vEB over universe u contains $\sqrt{u}$ clusters (each of size $\sqrt{u}$) and a summary structure over universe $\sqrt{u}$ recording which clusters are nonempty. Bottom lane: zooming into any cluster shows the same recursive pattern over universe size $u^{1/2}$, branching into $u^{1/4}$ clusters, until the base case $u = 2$ is reached.

21.4.1 How tall is this recursion?

Each level shrinks the universe from u to $\sqrt{u}$. So the universe sizes go

$$u, \quad u^{1/2}, \quad u^{1/4}, \quad u^{1/8}, \quad \dots$$

and we stop when the size reaches 2. How many square roots does it take to drive u down to 2? Write $u = 2^k$. One square root sends 2^k to $2^{k/2}$; that is, it *halves the exponent* k. We are done when the exponent reaches 1. Halving k down to 1 takes $\lg k = \lg \lg u$ steps.

Focus

The depth of the recursion is $\lg \lg u$, not $\lg u$. Taking the square root of the universe at each level halves the *exponent*, and halving repeatedly is a logarithm. Every $\lg \lg u$ in this chapter comes from this observation, so check it before continuing.

Example 21.2 (Counting the levels)**.** Let $u = 2^{32}$, so $k = 32$. The universe sizes down the recursion are

$$2^{32} \to 2^{16} \to 2^8 \to 2^4 \to 2^2 \to 2^1 = 2.$$

That is 5 square-roots, and indeed $\lg \lg u = \lg 32 = 5$. A 32-bit universe is only five levels deep. Figure 21.4 shows this descent beside the contrast: a binary tree of the same universe is 32 levels deep.

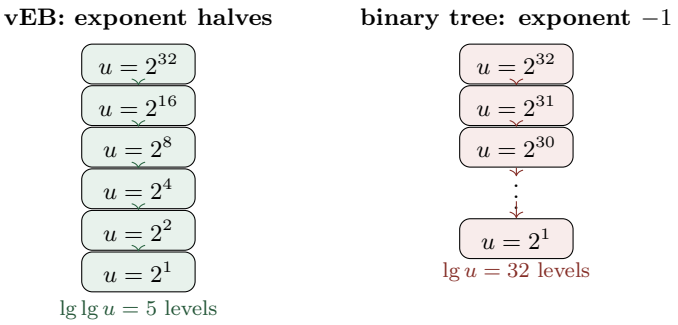


Figure 21.4: Why the recursion is shallow. Square-rooting the universe (left) *halves the exponent* each level, reaching the base in $\lg \lg u$ steps. A binary tree (right) only *decrements* the exponent, taking $\lg u$ steps. For $u = 2^{32}$ that is 5 levels versus 32.

21.5 Proto-vEB operations, and why they are not yet fast enough

Let us write the operations on proto-vEB and watch carefully where the time goes. We will find that MEMBER already achieves $O(\lg \lg u)$, whereas SUCCESSOR and MINIMUM still degrade to $\Theta(\lg u)$. Seeing exactly why tells us how to fix the structure.

21.5.1 Member

Algorithm 93: PROTO-VEB-MEMBER(V, x)

Input: A proto-vEB V of size u, key x.
Output: TRUE iff $x \in S$.

```
1 if V.u = 2 then
2     return V.A[x] ;                          // base case: read the bit directly
3 return PROTO-VEB-MEMBER(V.cluster[high(x)], low(x));
```

To test membership you peel off the high half to pick the cluster and recurse on the low half. The recursion descends one level each call, so it makes $\lg \lg u$ calls, each $O(1)$. Let $M(u)$ be the time. Then

$$M(u) = M(\sqrt{u}) + O(1).$$

Substitute $m = \lg u$ (so $\lg \sqrt{u} = m/2$) to get $M(2^m) = M(2^{m/2}) + O(1)$, i.e. a function that halves m each step and adds a constant – exactly the merge-sort-style recurrence whose answer is $O(\lg m) = O(\lg \lg u)$. MEMBER is already at the target.

21.5.2 Minimum and Maximum – the slow ones

To find the minimum, you find the first nonempty cluster (using the summary), then recurse to find the minimum within it. Watch the cost.

Algorithm 94: PROTO-VEB-MINIMUM(V)

Input: A proto-vEB V of size u.
Output: The smallest stored key, or NIL if empty.

```
1 if V.u = 2 then
2     if V.A[0] = 1 then return 0;
3     else if V.A[1] = 1 then return 1;
4     else return NIL;
5 c ← PROTO-VEB-MINIMUM(V.summary) ;          // first nonempty cluster
6 if c = NIL then return NIL;
7 o ← PROTO-VEB-MINIMUM(V.cluster[c]) ;              // min within it
8 return index(c, o);
```

The problem is that this routine makes *two* recursive calls of size $\sqrt{u}$ on the path to the answer – one into the summary, one into a cluster. The recurrence is

$$T(u) = 2\,T(\sqrt{u}) + O(1).$$

With $m = \lg u$ this is $T(2^m) = 2\,T(2^{m/2}) + O(1)$, and substituting $S(m) = T(2^m)$ gives $S(m) = 2\,S(m/2) + O(1)$, which by the master theorem (Chapter 4) solves to $S(m) = \Theta(m)$. So $T(u) = \Theta(\lg u)$, no better than logarithmic: two calls per level multiply out to $\lg u$ and cancel the benefit of the shallow recursion.

Common trap

The problem is making *more than one* recursive call per level. With one call per level you pay (depth)$\times O(1) = O(\lg \lg u)$. With two calls per level the work *branches*, and a branching recursion of depth $\lg \lg u$ with branching factor 2 does $\Theta(\lg u)$ total work. Every fast operation in the final structure will obey one rule: **at most one recursive call into a $\sqrt{u}$-sized substructure.** The rest of the design exists to satisfy it.

21.5.3 Successor – the bottleneck diagnosed

Successor has the same problem: branching recursive calls. Tracing the call path shows why we need to cache the extreme keys. To find the successor of x:

1. Look *within x's own cluster* for something bigger than low(x). If found, splice it back with high(x) and return. (One recursive call.)

2. Otherwise the answer is in a *later* cluster. Ask the summary for the next nonempty cluster after high(x). (A second recursive call.) Then take the *minimum* of that cluster. (A *third* recursive call!)

Algorithm 95: PROTO-VEB-SUCCESSOR(V, x)

Input: A proto-vEB V of size u, key x.
Output: Smallest stored key $> x$, or NIL.

```
1  if V.u = 2 then
2      if x = 0 and V.A[1] = 1 then return 1;
3      else return NIL;
4  o ← PROTO-VEB-SUCCESSOR(V.cluster[high(x)], low(x));
5  if o ≠ NIL then
6      return index(high(x), o) ;                    // found in same cluster
7  c ← PROTO-VEB-SUCCESSOR(V.summary, high(x));
8  if c = NIL then return NIL;
9  o' ← PROTO-VEB-MINIMUM(V.cluster[c]);
10 return index(c, o');
```

In the worst case this makes recursive calls into the cluster *and* the summary *and* a minimum. With MINIMUM itself costing $\Theta(\lg u)$ as we just found, the whole procedure costs $\Theta(\lg u)$ or more, which is too slow.

Cost summary

proto-vEB. MEMBER: $O(\lg \lg u)$ – already perfect. MINIMUM/MAXIMUM: $\Theta(\lg u)$ – too slow (two calls/level). SUCCESSOR/PREDECESSOR: $\Theta(\lg u)$ – too slow. INSERT/DELETE: $\Theta(\lg u)$ for the same reason (they recurse into a cluster *and* update the summary). The structure is correct but the speed is no better than a BST. Diagnosis: multiple recursive calls per level.

21.6 Two fixes: cached extremes and a floating minimum

Consider where the extra recursive calls came from. MINIMUM made a call just to find its answer. SUCCESSOR made a call to MINIMUM. INSERT made a call into a cluster *and* a call to update the summary. If we could make MINIMUM cost $O(1)$, and stop INSERT from always touching both the cluster and the summary, the budget of "one recursive call per level" would be within reach. Two modifications do exactly this.

Fix 1: store the minimum and maximum explicitly. Give every vEB node two extra fields, $V.min$ and $V.max$, caching the smallest and largest keys in that subtree. Now MINIMUM and MAXIMUM are $O(1)$ – just read the field. No recursion at all.

Fix 2: the minimum does not live in any cluster. This is the subtle part. We adopt the convention that the element stored in $V.min$ is *not* also stored down in the clusters. It floats at the top, held only in the *min* field. (The maximum *is* kept down in its cluster as well; only the min is special.)

Why do that? Because it lets INSERT into an *empty* vEB stop instantly, and – more importantly – it ensures that whenever INSERT has to recurse into a cluster, that cluster was already nonempty, so the summary does *not* also need a recursive update. We will see in a moment how this convention reduces two recursive calls to one. The same applies to SUCCESSOR: pulling the min out lets it answer most queries by reading *min* and *max* fields without a second descent.

Intuition

The *min* field is a doorman standing at the entrance of each vEB node. Inserting into an empty node? The doorman just remembers you, no need to disturb anyone inside (the clusters stay untouched, the summary stays untouched). Inserting a second element? The doorman is already busy, so *now* you go inside – but because the node is no longer empty, the summary already knows this cluster is occupied, so you only ever need to recurse *one* level deeper, never two. This is what guarantees one recursive call per level.

Formal

The real vEB tree. A vEB(u) structure stores a set $S \subseteq \{0, \ldots, u-1\}$. It has fields *min*, *max* (each a key or NIL). If $u > 2$ it also has $cluster[0..\sqrt{u}-1]$, each a vEB($\sqrt{u}$), and a *summary*, also a vEB($\sqrt{u}$). Invariants:

- *min* holds the smallest key of S and is *not* stored in any cluster.
- *max* holds the largest key of S (and, unless it equals *min*, *is* also stored in its cluster).
- $cluster[i]$ stores exactly $\{\text{low}(x) : x \in S, \text{high}(x) = i, x \neq min\}$.
- *summary* stores exactly $\{i : cluster[i] \text{ is nonempty}\}$.
- An empty S has $min = max =$ NIL. A singleton $S = \{x\}$ has $min = max = x$ and *all clusters empty.*

When $u = 2$ there are no clusters or summary; the set is fully described by *min* and *max* over $\{0, 1\}$.

21.7 The fast operations

Now we rewrite the operations to exploit the two fixes. The recurring pattern to watch for: *do as much as possible with min and max in $O(1)$, and make at most one genuine recursive call.*

21.7.1 Minimum, Maximum, Member

MINIMUM and MAXIMUM simply return $V.min$ and $V.max$: $O(1)$, done. MEMBER is almost as before, but must first check whether x is the floating minimum or maximum (since those are not in the clusters):

Algorithm 96: vEB-MEMBER(V, x)

```
1 if x = V.min or x = V.max then
2     return TRUE
3 else if V.u = 2 then return FALSE;
4 else return vEB-MEMBER(V.cluster[high(x)], low(x));
```

One recursive call per level $\Rightarrow O(\lg \lg u)$, as before.

21.7.2 Successor: the centerpiece

Algorithm 97: vEB-Successor(V, x)

Input: vEB V of size u, key x (need not be in S).
Output: Smallest key in S that is $> x$, or NIL.

```
1  if V.u = 2 then
2      if x = 0 and V.max = 1 then return 1;
3      else return NIL;
4  if V.min ≠ NIL and x < V.min then
5      return V.min ;                          // the floating min is the answer
6  maxlow ← vEB-Maximum(V.cluster[high(x)]);
7  if maxlow ≠ NIL and low(x) < maxlow then
8      o ← vEB-Successor(V.cluster[high(x)], low(x));
9      return index(high(x), o) ;               // successor is in same cluster
10 else
11     c ← vEB-Successor(V.summary, high(x));
12     if c = NIL then return NIL;
13     return index(c, vEB-Minimum(V.cluster[c]));
```

There are three exits, and only one of them recurses:

1. *Cheap shortcut.* If x is smaller than the floating minimum, the minimum itself is the successor. $O(1)$, no recursion.

2. *Same cluster.* We peek at the *maximum* of x's own cluster in $O(1)$ (it is just a cached field). If $\text{low}(x)$ is below that max, the successor lives in this cluster, so we make one recursive Successor call into it – and *stop*.

3. *Later cluster.* Otherwise we recurse once into the *summary* to find the next nonempty cluster c, then read that cluster's Minimum – but Minimum is now $O(1)$! So this branch also makes exactly *one* recursive call (into the summary).

The $O(1)$ Minimum (Fix 1) is what lets branch (3) avoid a second recursion. So no matter which exit we take, we recurse at most once into a $\sqrt{u}$-sized substructure. The recurrence is finally

$$T(u) = T(\sqrt{u}) + O(1),$$

and we already solved this for Member: $T(u) = O(\lg \lg u)$. Predecessor is the mirror image (swap min/max, swap successor/maximum) and is the same cost.

21.7.3 A complete numeric trace of Successor

Let $u = 16$ (so $\sqrt{u} = 4$) and $S = \{2, 3, 4, 5, 7, 14, 15\}$. We ask for Successor$(S, 5)$; the answer should be 7. Recall the grid view: $\text{high}(x) = \lfloor x/4 \rfloor$ is the row, $\text{low}(x) = x \bmod 4$ is the column. Figure 21.5 lays out the structure for this set.

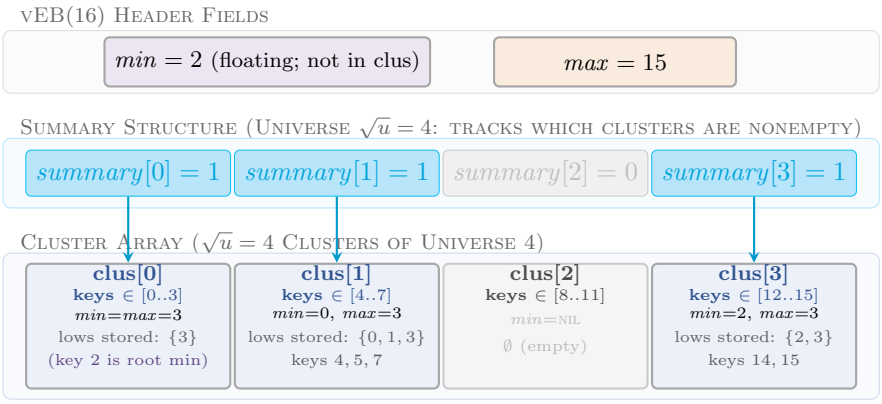


Figure 21.5: The vEB(16) for $S = \{2, 3, 4, 5, 7, 14, 15\}$ displayed with dedicated horizontal lanes. The overall minimum key, 2, floats in *min* and is *not* stored in any cluster. Keys $\{3, 4, 5, 7, 14, 15\}$ map to clusters by their high half, where they are stored as low offsets. The summary structure tracks nonempty clusters $0, 1, 3$. Cluster 2 contains no keys.

Algorithm lab

Trace. The top vEB has $min = 2$ (floating, not in any cluster). The remaining keys $\{3, 4, 5, 7, 14, 15\}$ distribute into clusters by their high half:

cluster i	keys with high $= i$	stored lows
0	3	$\{3\}$ (since low(3) = 3)
1	4, 5, 7	$\{0, 1, 3\}$
3	14, 15	$\{2, 3\}$

Cluster 2 is empty. The summary stores $\{0, 1, 3\}$ (the nonempty clusters), with its own floating $min = 0$.

Now run vEB-Successor$(V, 5)$:

1. $u = 16 \neq 2$. Is $5 < V.min = 2$? No.
2. high(5) = 1, low(5) = 1. Peek at Maximum$(cluster[1])$. Cluster 1 holds lows $\{0, 1, 3\}$ so its max is 3. Is low(5) = $1 < 3$? Yes — the successor is in the same cluster.
3. Recurse: vEB-Successor$(cluster[1], 1)$. Inside, the next low after 1 among $\{0, 1, 3\}$ is 3. Return 3.
4. Splice back: index(high $= 1, o = 3) = 1 \cdot 4 + 3 = 7$.

Answer: 7. ✓ Exactly one recursive descent was made.

Second trace, crossing clusters: vEB-Successor$(V, 7)$, answer should be 14.

1. $7 < 2$? No. high(7) = 1, low(7) = 3.
2. Maximum$(cluster[1]) = 3$. Is low(7) = $3 < 3$? No. So look in a *later* cluster.
3. Recurse into summary: vEB-Successor$(summary, 1)$. Summary holds $\{0, 1, 3\}$; the next after 1 is 3. So $c = 3$.
4. Minimum$(cluster[3])$: cluster 3 holds lows $\{2, 3\}$, min is 2 ($O(1)$ field read). Splice: index$(3, 2) = 3 \cdot 4 + 2 = 14$.

Answer: 14. ✓

21.7.4 Insert: one recursion via the doorman

Algorithm 98: vEB-INSERT(V, x)

```
1  if V.min = NIL then
2      V.min ← V.max ← x ;                  // empty: just seat the doorman, O(1), stop
3      return
4  if x < V.min then
5      swap x ↔ V.min ;                     // new smallest: displace the old min downward
6  if V.u > 2 then
7      if vEB-MINIMUM(V.cluster[high(x)]) = NIL then
8          vEB-INSERT(V.summary, high(x)) ;     // cluster was empty: update summary
9          set cluster[high(x)].min = cluster[high(x)].max = low(x);
10     else
11         vEB-INSERT(V.cluster[high(x)], low(x)) ;   // cluster nonempty: recurse into
               it
12 if x > V.max then V.max ← x;
```

This mutual exclusion between the summary update and the cluster descent is what makes the floating minimum convention work. The two recursive calls – one into the summary, one into the cluster – live in *opposite* branches of an `if/else`. They never both fire on the same level:

- If the target cluster is *empty*, we recurse into the summary to record that it just became nonempty, and we seat the cluster's doorman in $O(1)$ *without* recursing into the cluster. Inserting into an empty vEB is $O(1)$ (the first `if`), so the summary call's own deeper recursion also terminates immediately at its next level.
- If the target cluster is *nonempty*, the summary already knows about it, so we skip the summary entirely and recurse only into the cluster.

Figure 21.6 contrasts the two execution paths: in both branches, exactly one operation recurses, while the other is either omitted or finishes in $O(1)$ time.

So along the actual chain of nested calls, only one "real" (non-$O(1)$) recursive call happens per level. The recurrence is again $T(u) = T(\sqrt{u}) + O(1) = O(\lg \lg u)$.

Common trap

A tempting misreading: "INSERT makes two recursive calls, so it must be $\Theta(\lg u)$ like proto-vEB." In fact the two calls are in mutually exclusive branches, and the branch that recurses into the summary does so only when the cluster is empty – which makes the *paired* cluster operation $O(1)$. The "$\leq$ one expensive recursive call per level" invariant holds. Always check *which* calls can fire *together on a path*, not how many appear in the source.

21.7.5 Delete: the bookkeeping is heavier but the bound holds

DELETE is the fiddliest operation because removing the floating minimum forces us to promote a new one from the clusters. The structure of the argument is identical, though: only one nontrivial

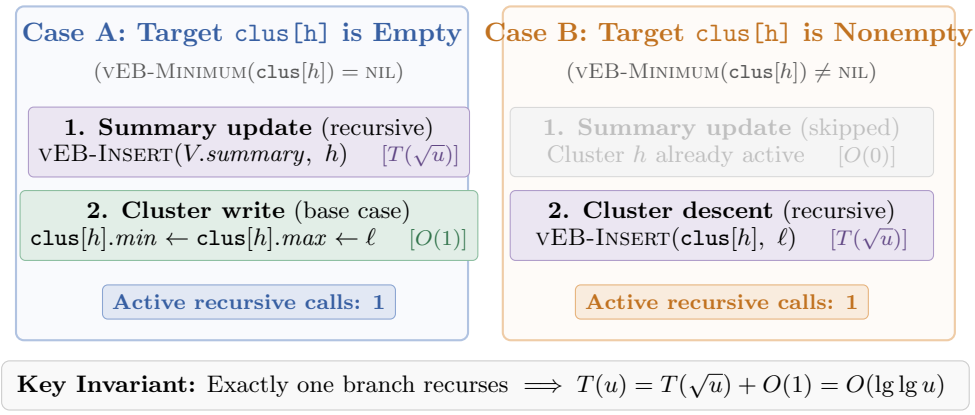


Figure 21.6: The mutual-exclusion invariant in vEB-Insert. When cluster h is empty (left), the recursive call goes to the summary while the cluster itself is initialized in $O(1)$ as a doorman base case. When cluster h is nonempty (right), the summary requires no update, and the single recursive call descends into cluster h. In both cases, exactly one non-$O(1)$ recursive call is issued per level.

recursive call fires per level.

Algorithm 99: vEB-Delete(V, x)

```
1  if V.min = V.max then
2      V.min ← V.max ← NIL ;                          // only one element; now empty
3      return
4  if V.u = 2 then
5      if x = 0 then V.min ← 1;
6      else V.min ← 0;
7      V.max ← V.min;
8      return
9  if x = V.min then
10     c ← vEB-Minimum(V.summary) ;                   // first nonempty cluster
11     x ← index(c, vEB-Minimum(V.cluster[c]));
12     V.min ← x ;    // promote:  this key will be removed from its cluster below
13 vEB-Delete(V.cluster[high(x)], low(x));
14 if vEB-Minimum(V.cluster[high(x)]) = NIL then
15     vEB-Delete(V.summary, high(x)) ; // cluster emptied:  clear its summary bit
16     if x = V.max then
17         recompute V.max from summary/clusters (O(1) field reads);
18 else if x = V.max then  V.max ← index(high(x), vEB-Maximum(V.cluster[high(x)])) ;
```

There appear to be two recursive Delete calls (cluster, then possibly summary), so we must argue carefully that they do not both genuinely recurse on the same path. The second one fires only when the cluster became *empty* after the first deletion. But deleting the last element of a cluster (Delete on a one-element vEB) hits the very first `if` and costs $O(1)$ – it does not recurse deeper. So the expensive deletion happens in *either* the cluster *or* the summary, never both at full depth. Once more $T(u) = T(\sqrt{u}) + O(1) = O(\lg \lg u)$.

Cost summary

vEB (final). Every operation – MEMBER, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR – runs in $O(\lg \lg u)$ worst-case time, governed by the single recurrence $T(u) = T(\sqrt{u}) + O(1)$. The price is space (next section) and a fixed integer universe.

21.8 Solving the recurrence, once and for all

We keep invoking "$T(u) = T(\sqrt{u}) + O(1) = O(\lg \lg u)$." Let us solve it, because the same substitution handles several other recurrences in this book.

The difficulty is the $\sqrt{u}$, which is awkward to iterate. A change of variables turns the square root into a halving. Let

$$m = \lg u, \qquad \text{equivalently } u = 2^m.$$

Then $\sqrt{u} = u^{1/2} = 2^{m/2}$, so its log is $m/2$. Define $S(m) = T(2^m)$. The recurrence $T(u) = T(\sqrt{u}) + O(1)$ becomes

$$S(m) = T(2^m) = T(2^{m/2}) + O(1) = S(m/2) + O(1).$$

That is a recurrence that *halves* m and adds a constant – precisely the shape of binary search. By the master theorem (Chapter 4, case 2 with $a = 1, b = 2, f = O(1)$), it solves to

$$S(m) = O(\lg m).$$

Unwinding the substitution, $m = \lg u$, so

$$T(u) = S(\lg u) = O(\lg \lg u).$$

Walking question

On your walk, prove the $O(\lg m)$ from scratch by unrolling, no master theorem. Write $S(m) = S(m/2) + c = S(m/4) + 2c = S(m/8) + 3c = \cdots$. After t steps the argument is $m/2^t$ and the accumulated cost is tc. The recursion stops when $m/2^t$ reaches the base (1), i.e. when $t = \lg m$. So $S(m) = \Theta(\lg m) \cdot c = O(\lg m)$. This rederives the depth $\lg \lg u$.

21.9 The cost of all this: space

A vEB(u) recursively contains $\sqrt{u}$ cluster substructures plus one summary substructure, each a vEB($\sqrt{u}$), and all of them are *allocated up front* – even the clusters that hold nothing. Let $P(u)$ be the space. Then

$$P(u) = (\sqrt{u} + 1)\, P(\sqrt{u}) + O(1).$$

This solves to $P(u) = \Theta(u)$. The structure occupies space proportional to the *entire universe*, whether you store one key or a million.

Common trap

$\Theta(u)$ space is the catch that limits where vEB is usable. For $u = 2^{32}$ that is billions of cells – gigabytes – even to store ten keys. The remedy in practice is to replace the array of $\sqrt{u}$ cluster pointers with a *hash table* (Chapter 11) keyed by cluster index, allocating a cluster only when it first becomes nonempty. That cuts space to $O(n)$ (proportional to the keys actually present) while keeping the $O(\lg \lg u)$ time *expected*. CLRS develops this as the related "y-fast trie" direction in its problems and chapter notes.

Why this matters

The general lesson: *when your keys are not abstract, exploit their structure.* The comparison lower bound only binds when all you can do is ask "is $a < b$?". Once you can *index by a key* – use the key as an address – faster algorithms become possible. This is the same insight behind counting sort and radix sort (Chapter 8) and behind direct-address tables (Chapter 11). vEB is the most sophisticated member of that family: it indexes *recursively*, by half the key at a time.

History and connection

Peter van Emde Boas presented the first version of this data structure in 1975 at the IEEE Symposium on Foundations of Computer Science (with full journal versions co-authored with Roman Kaas and Erik Zijlstra appearing in 1977). His goal was to determine whether a priority queue on integers could break the $\Omega(\log n)$ comparison bound on a realistic machine model. By viewing integers as hierarchical coordinate pairs through square-root decomposition ($\sqrt{u}$ clusters of capacity $\sqrt{u}$), caching extremes, and using bitwise shifting to locate clusters in constant time, he showed that all priority-queue and predecessor queries can run in $O(\log \log u)$ time.

The structure had a serious practical limitation: the tree required $\Theta(u)$ words of memory even when storing only a handful of keys, rendering it prohibitive on large universes such as 64-bit address spaces. This motivated decades of research into integer data structures. In 1983, Dan Willard introduced x-fast and y-fast tries, which combine prefix trees with dynamic hashing to shrink the space requirement to $O(n)$ while preserving $O(\log \log u)$ expected time. In 1993, Michael Fredman and Dan Willard introduced fusion trees, which pack multiple keys into a single machine word to achieve $O(\log n / \log w)$ time per operation on a w-bit word RAM, proving that integers can be sorted faster than comparison trees without bounding the universe size. Later cell-probe lower bounds by Mihai Pătraşcu and Mikkel Thorup proved that the double-logarithmic query time of van Emde Boas trees is optimal for any data structure occupying space polynomial in n. Modern research focuses on cache-oblivious layouts and closing constant-factor instruction penalties on real CPUs. In CLRS (3rd edition, Chapter 20), the structure is developed through the same progression: bit vector to binary summary tree, proto-vEB, and finally the full recursive vEB with cached extremes.

21.10 Recap: the chain of ideas

Each step below fixes the flaw in the step before it.

Focus

1. **Bit vector** ($O(u)$ successor): $O(1)$ membership but successor scans a flat array.
2. **Binary summary tree** ($O(\lg u)$): add OR-summaries to get shortcuts, but the tree is $\lg u$ tall.
3. **Square-root split / proto-vEB** (depth $\lg \lg u$): branch by $\sqrt{u}$ so the recursion is only $\lg \lg u$ deep – but operations make ≥ 2 recursive calls per level, so they still cost $\Theta(\lg u)$.
4. **Stored** *min*/*max* **+ floating min** ($O(\lg \lg u)$): cache min/max for $O(1)$ access, and keep the min out of the clusters, so every operation makes *exactly one* recursive call per level. Recurrence $T(u) = T(\sqrt{u}) + O(1) = O(\lg \lg u)$.

The rule behind every step: *one recursive call into a $\sqrt{u}$-substructure per level.* The rest is bookkeeping that maintains that rule.

This chapter ends our study of advanced data structures. In Part 6, we turn to graphs and network algorithms, beginning in Chapter 22 with representations of graphs and the two basic ways to traverse them—breadth-first and depth-first search.

21.11 Exercises

Exercise 21.1 (Counting the levels)**.** For each universe size $u \in \{4,\ 16,\ 256,\ 2^{16},\ 2^{64}\}$, write out the sequence of universe sizes down the vEB recursion ($u \to \sqrt{u} \to \cdots \to 2$) and confirm the number of levels equals $\lg \lg u$. Which of these would have operations costing more than 6 "levels" of work? (Warmup.)

Exercise 21.2 (Splitting keys)**.** Let $u = 256$, so $\sqrt{u} = 16$. Compute high(x), low(x), and verify index(high(x), low(x)) $= x$ for $x \in \{0, 17, 200, 255\}$. Describe high and low purely as bit operations (shift / mask) on the 8-bit value x. (Warmup.)

Exercise 21.3 (Why two calls per level is fatal)**.** Solve each recurrence by the $m = \lg u$ substitution and state the closed form: (a) $T(u) = T(\sqrt{u}) + O(1)$; (b) $T(u) = 2\,T(\sqrt{u}) + O(1)$; (c) $T(u) = \sqrt{u}\,T(\sqrt{u}) + O(1)$. Match each to one of the structures in the chapter (final vEB, proto-vEB minimum, total space). (Medium.)

Exercise 21.4 (Trace Successor)**.** Let $u = 16$ and $S = \{1, 2, 6, 9, 13\}$. Lay out the clusters and the summary (recall that the floating minimum is not stored in any cluster). Trace vEB-Successor(V, x) for $x = 2$, $x = 6$, and $x = 9$, naming which of the three exits fires each time and how many recursive descents occur. (Medium.)

Exercise 21.5 (Trace Insert)**.** Starting from an empty vEB(16), insert the keys $5, 2, 8, 5, 0$ in that order (the second 5 is a no-op; explain why). After each insertion, write down *min*, *max*, the contents of each cluster, and the summary. Point out every insertion that took $O(1)$ time because it hit the empty-node fast path. (Medium.)

Exercise 21.6 (The floating minimum, examined)**.** Explain in your own words why vEB-Insert into an *empty* node is $O(1)$, and why this single fact is what keeps Insert's recurrence at $T(u) = T(\sqrt{u}) + O(1)$ rather than $T(u) = 2\,T(\sqrt{u}) + O(1)$. What would break if we instead stored the minimum down in its cluster like every other element? (Medium.)

Exercise 21.7 (Predecessor by mirror)**.** vEB-PREDECESSOR is the mirror of SUCCESSOR, but there is one asymmetry: it has a special case that SUCCESSOR does not, involving *min*. Write vEB-PREDECESSOR in full pseudocode and explain the extra case. Hint: the floating *min* is below everything in the clusters, so when the cluster search fails, you must still check whether x is greater than *min*. (Medium-hard.)

Exercise 21.8 (Trace Delete, including a promotion)**.** Take $u = 16$ and $S = \{0, 3, 4, 11\}$ with the structure correctly built. Trace vEB-DELETE$(V, 0)$. Since 0 is the floating minimum, follow the promotion: which key becomes the new *min*, and from which cluster is it then removed? Show the clusters and summary before and after. (Hard.)

Exercise 21.9 (Space recurrence)**.** Set up and solve the space recurrence $P(u) = (\sqrt{u}+1)P(\sqrt{u}) + O(1)$ to show $P(u) = \Theta(u)$. Then estimate the space, in cells, for $u = 2^{20}$ holding a set of just 3 keys, and contrast with a balanced BST holding the same 3 keys. What does this tell you about when to reach for a vEB tree? (Hard.)

Exercise 21.10 (From $\Theta(u)$ space to $O(n)$ space)**.** Sketch how replacing the array $cluster[0..\sqrt{u}-1]$ with a hash table (Chapter 11), allocating a cluster only on its first nonempty insert, reduces space to $O(n)$. Which time bounds become *expected* rather than worst-case, and why? (Challenge.)

Exercise 21.11 (Successor without stored max)**.** Suppose you kept the floating *min* trick but did *not* cache *max* – you would have to compute a cluster's maximum by a recursive call. Show that SUCCESSOR then makes two genuine recursive calls in its "same cluster" branch (the max query plus the successor query), and that its running time degrades to $\Theta(\lg u)$. This isolates exactly why caching *max* in addition to *min* is essential. (Challenge.)

Part VI

Graph Algorithms

Chapter 22

Elementary Graph Algorithms

In earlier chapters, we developed data structures to organize collections of independent items: trees, heaps, hash tables, and priority queues. Many problems, though, are about relationships between items. Cities joined by roads. Web pages joined by links. People joined by friendships. Tasks joined by "must happen before" arrows. Courses joined by prerequisites. The abstraction that covers all of these is the *graph*. A graph is just a set of dots and a set of lines between dots, yet a wide range of algorithmic problems can be modeled directly as questions on graphs.

This chapter is the foundation for everything graph-related that follows – minimum spanning trees, shortest paths, network flow. The starting question is how to *store* a graph in a computer, because that choice affects the cost of everything else. With a graph in hand, there are two fundamental ways to *explore* it: breadth-first search, which fans outward in rings, and depth-first search, which plunges forward and backtracks. Depth-first search also leads to two important tools. One is *topological sort*, which linearizes a tangle of dependencies. The other is the discovery of *strongly connected components*, the tightly knit clusters inside a directed graph. By the end you will be able to look at almost any "things and connections" problem and reach for the right tool.

22.1 What a Graph Is

First the vocabulary. A *graph* $G = (V, E)$ has a set of *vertices* V (the dots, also called *nodes*) and a set of *edges* E (the lines). Each edge joins two vertices.

The first distinction: are the edges *directed* or *undirected*? In an *undirected* graph, an edge $\{u, v\}$ is a two-way street – if u is connected to v, then v is connected to u, with no preferred direction. Think of friendships, or roads you can drive both ways. In a *directed* graph (often called a *digraph*), an edge (u, v) is a one-way arrow from u *to* v, and it says nothing about whether you can get back. Think of one-way streets, web links, or "task u must finish before task v starts."

Intuition

A useful mental test: if you can imagine standing on a vertex and an edge "points" somewhere, it is directed. If the edge is just a wire connecting two pegs with no arrowhead, it is undirected. Undirected graphs are really a special case of directed ones where every arrow comes with a

matching arrow pointing the other way. We keep them separate because so many real situations are genuinely symmetric, and pretending otherwise wastes space.

A few more words you will need constantly. Two vertices joined by an edge are *adjacent* (or *neighbors*). The number of edges touching a vertex is its *degree*; in a directed graph we split this into *out-degree* (arrows leaving) and *in-degree* (arrows arriving). A *path* is a sequence of vertices where each consecutive pair is joined by an edge. A *cycle* is a path that returns to where it started. A graph with no cycles is *acyclic*; a directed acyclic graph appears so often it has its own three-letter name, *DAG*, which we will meet again in topological sort.

Focus

Throughout the chapter we write V for the number of vertices and E for the number of edges. (Strictly $|V|$ and $|E|$, but everyone abuses notation and writes V and E inside $O(\cdot)$, and so will we.) Almost every running time in graph algorithms is some combination of V and E, and learning to read those formulas – to feel whether $O(V + E)$ is fast and $O(V^2)$ is wasteful for your particular graph – is a core skill.

How big can E be relative to V? In an undirected graph with no self-loops or repeated edges, every pair of vertices contributes at most one edge, so $E \le \binom{V}{2} = \Theta(V^2)$. A graph that has nearly this many edges is called *dense*; one with only a handful per vertex, so $E = O(V)$, is *sparse*. Most real graphs – road networks, social networks, the web – are very sparse, and this fact drives the main storage decision we are about to make.

22.2 Storing a Graph: Lists versus Matrices

You cannot run an algorithm on a picture. We need to encode those vertices and edges in memory so an algorithm can traverse them efficiently. There are two classic encodings, and the trade-off between them matters.

22.2.1 The adjacency matrix

The most literal encoding is a grid. Number the vertices $1, 2, \ldots, V$. Build a $V \times V$ table A where $A[i][j] = 1$ if there is an edge from i to j, and 0 otherwise. That's the *adjacency matrix*.

Example 22.1. Take an undirected graph on four vertices $\{1, 2, 3, 4\}$ with edges $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$, and $\{3, 4\}$. Its adjacency matrix is

$$A = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 1 & 0 \\ 2 & 1 & 0 & 1 & 0 \\ 3 & 1 & 1 & 0 & 1 \\ 4 & 0 & 0 & 1 & 0 \end{array}$$

Notice the matrix is *symmetric* across the diagonal: $A[i][j] = A[j][i]$. That symmetry is exactly what "undirected" means in matrix form. For a directed graph the matrix would generally not be symmetric.

The matrix has two appealing properties. You can answer "is there an edge from i to j?" in $O(1)$ time – one array lookup, no searching. And it is simple and supports algebraic methods (you will see in a later chapter that squaring the matrix counts paths of length two). But it has one serious drawback for large sparse graphs: it always uses $\Theta(V^2)$ space, even if the graph has only a dozen edges. A social network with a billion users would need a matrix with 10^{18} entries, almost all of them zero, which is infeasible.

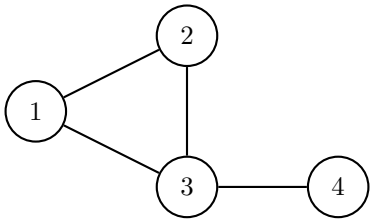
22.2.2 The adjacency list

The adjacency list throws away all those zeros. For each vertex u, keep a little list $Adj[u]$ naming exactly the vertices u connects to. Nothing more.

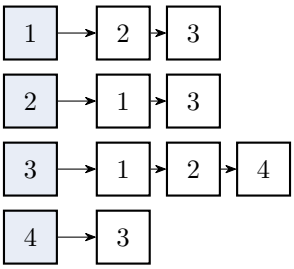
Example 22.2. The same graph from Example 22.1 as adjacency lists:

$$Adj[1] = [\,2,\ 3\,]$$
$$Adj[2] = [\,1,\ 3\,]$$
$$Adj[3] = [\,1,\ 2,\ 4\,]$$
$$Adj[4] = [\,3\,]$$

In an undirected graph each edge shows up *twice* – edge $\{1, 2\}$ puts a 2 in $Adj[1]$ and a 1 in $Adj[2]$. So the total length of all the lists is exactly $2E$. In a directed graph each edge appears once, in the list of its source vertex, and the total length is E. Either way, the space used is $\Theta(V + E)$: one slot per vertex for the list header, plus one entry per edge-endpoint.



The same four lists, drawn as the linked structure a machine actually holds – a header array on the left, each pointing to a little chain of its neighbors:



Now the trade-off is clear. The list uses $\Theta(V + E)$ space, which for a sparse graph is enormously smaller than $\Theta(V^2)$. The price is that checking "is there an edge from i to j?" now means scanning i's list, which can take up to $O(\deg(i))$ time instead of $O(1)$. For most algorithms – including every one in this chapter – you do not ask that question; instead you ask "what are all of u's neighbors?", and the list answers that directly. That is why adjacency lists are the default.

Cost summary

For a graph with V vertices and E edges:

	Adjacency list	Adjacency matrix
Space	$\Theta(V + E)$	$\Theta(V^2)$
Edge query (i, j)?	$O(\deg i)$	$O(1)$
List u's neighbors	$\Theta(\deg u)$	$\Theta(V)$
Best for	sparse graphs	dense graphs

The rule of thumb: *lists for sparse, matrix for dense.* Since real graphs are usually sparse, lists win in practice almost every time.

Common trap

A classic beginner mistake is to reach for the adjacency matrix because it "feels simpler," then watch the program run out of memory on a graph with a million vertices. A million-by-million matrix of bytes is a terabyte. The same graph as lists, if it averages ten edges per vertex, is about ten million entries – a few tens of megabytes. Always ask: is my graph sparse? If yes, use lists.

22.3 Breadth-First Search: Exploring in Rings

Imagine you drop a pebble into a still pond at vertex s (the *source*). Ripples spread outward in perfect rings: first the vertices one edge away, then the vertices two edges away, then three, and so on. *Breadth-first search* (BFS) turns this ripple into an algorithm. It visits vertices in order of their distance *in number of edges* from s, and along the way it computes the length of the shortest such path to every reachable vertex.

Intuition

Picture a fire spreading through dry grass from one spot. At minute 1 every cell touching the start is alight. At minute 2 every cell touching *those*. The fire never skips ahead – it cannot reach a cell three steps away before it has burned through the cells two steps away. BFS is that fire. The "distance" it computes is the minute at which each cell catches.

How do we make a computer fan out in rings? We use a *queue* – a first-in-first-out line. We process vertices in the exact order we discover them. Because a vertex at distance d is always discovered before any vertex at distance $d+1$, and the queue hands them back in arrival order, we are guaranteed to finish *all* of ring d before touching ring $d + 1$.

To keep the bookkeeping straight, we color every vertex one of three colors as the search runs:

- **white** – not yet discovered (still untouched grass),
- **gray** – discovered but we have not yet looked at all its neighbors (the burning frontier),
- **black** – discovered and fully explored (burnt out).

A vertex goes white → gray when it first enters the queue, and gray → black when it leaves the

queue after we have scanned its neighbors. We also record, for each vertex u, a distance $d[u]$ (rings from s) and a *predecessor* $\pi[u]$ (the vertex from which we discovered u). Those predecessors, chained together, form the shortest-path tree.

Algorithm 100: BFS(G, s)

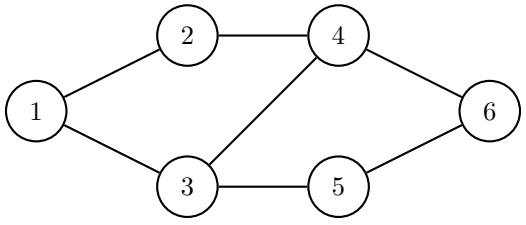
Input: Graph $G = (V, E)$ as adjacency lists; source vertex s.
Output: For each v: distance $d[v]$ in edges from s, and predecessor $\pi[v]$.

```
1  for each vertex u ∈ V do
2      color[u] ← white;   d[u] ← ∞;   π[u] ← NIL;
3  color[s] ← gray;   d[s] ← 0;   π[s] ← NIL;
4  Q ← empty queue;   ENQUEUE(Q, s);
5  while Q is not empty do
6      u ← DEQUEUE(Q);
7      for each v ∈ Adj[u] do
8          if color[v] = white then
9              color[v] ← gray;
10             d[v] ← d[u] + 1;
11             π[v] ← u;
12             ENQUEUE(Q, v);
13     color[u] ← black;
```

Read it once for the shape. We seed the queue with the source. Then, over and over, we pull the front vertex u, look at each neighbor v, and – only if v is still white (never seen) – we color it gray, set its distance to one more than u's, point its predecessor at u, and put it in line. Testing for a white vertex prevents redundant processing: it guarantees that each vertex enters the queue at most once.

22.3.1 A full numeric trace

Let us run BFS by hand. Here is an undirected graph on six vertices; we will start from $s = 1$.



The adjacency lists (with neighbors listed in increasing order) are $Adj[1] = [2, 3]$, $Adj[2] = [1, 4]$, $Adj[3] = [1, 4, 5]$, $Adj[4] = [2, 3, 6]$, $Adj[5] = [3, 6]$, $Adj[6] = [4, 5]$.

We track the queue contents and the distances after each dequeue.

Step	Dequeue	Newly discovered (distance)	Queue after
0	–	1 (0)	[1]
1	1	2 (1), 3 (1)	[2, 3]
2	2	4 (2)	[3, 4]
3	3	5 (2)	[4, 5]
4	4	6 (3)	[5, 6]
5	5	(nothing new)	[6]
6	6	(nothing new)	[ ]

Walk through the interesting moments. At step 2 we dequeue 2 and look at its neighbors 1 and 4. Vertex 1 is already black, so we skip it; vertex 4 is white, so it becomes gray at distance $d[2]+1=2$, with $\pi[4]=2$. At step 3 we dequeue 3; among its neighbors 1 (black) and 4 (already gray – we discovered it a moment ago via 2) are skipped, but 5 is white and gets distance 2 with $\pi[5]=3$. Notice that 4 was *not* re-discovered through 3: the white check protected it. At step 4 vertex 4 finally gives us 6 at distance 3. By the time we reach 5 and 6, everything is discovered, so they add nothing.

Final distances from $s=1$:

$$d[1]=0, \quad d[2]=1, \quad d[3]=1, \quad d[4]=2, \quad d[5]=2, \quad d[6]=3.$$

It helps to see the same result organized by distance rings (Figure 22.1). Each band collects vertices at a specific distance from source $s=1$. The bold edges represent predecessor (tree) edges kept by BFS; notice every bold edge connects a vertex in ring d to a vertex in ring $d+1$, never skipping across rings.

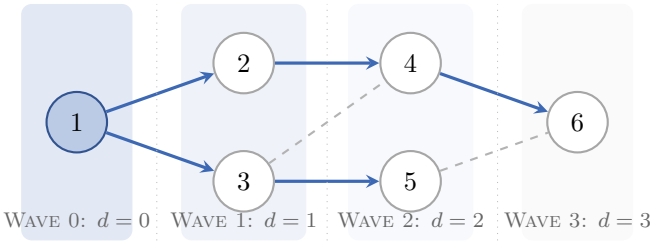


Figure 22.1: Breadth-first search as expanding concentric wavefronts. Vertices partition into frontier layers $d=0,1,2,3$. Bold arrows show the predecessor tree edges discovered during exploration; non-tree edges (dashed) connect vertices either within the same wave or between adjacent waves, confirming that BFS never skips a layer.

And the predecessor tree, $\pi[2]=\pi[3]=1$, $\pi[4]=2$, $\pi[5]=3$, $\pi[6]=4$, encodes one shortest path to each vertex. To recover the path to 6, follow predecessors backward: $6 \leftarrow 4 \leftarrow 2 \leftarrow 1$, i.e. $1 \rightarrow 2 \rightarrow 4 \rightarrow 6$, three edges – exactly $d[6]$.

Algorithm lab

Re-run the trace, but this time start from $s=6$ and write out the queue at every step yourself. Predict the distances before you check them: you should get $d[6]=0$, the neighbors of 6 at distance 1, and so on. Then notice something: because this graph is undirected, $d_{s=1}[6]=d_{s=6}[1]=3$. Distance is symmetric in undirected graphs. Is it symmetric in directed

graphs? (No; find a small directed example that shows why.)

22.3.2 Why BFS finds shortest paths

We claimed BFS computes the shortest distance in edges. Time to make that precise and prove it. Define $\delta(s, v)$ to be the *true* minimum number of edges on any path from s to v (and ∞ if no path exists).

Formal

Theorem (BFS correctness). After BFS(G, s) terminates, $d[v] = \delta(s, v)$ for every vertex v. The predecessor edges also form a tree in which the path from s to v is a shortest path.

The argument rests on a loop invariant about the queue: the queue is always a snapshot of the frontier, and the frontier is never more than one ring deep.

Formal

Queue invariant. At every point during BFS, if the queue holds the vertices $\langle v_1, v_2, \ldots, v_r \rangle$ from front to back, then $d[v_r] \leq d[v_1]+1$, and the distances are non-decreasing: $d[v_1] \leq d[v_2] \leq \cdots \leq d[v_r]$.

Why does this hold? Initially the queue is just $\langle s \rangle$ with $d[s] = 0$, so both conditions hold immediately for the base case. Now suppose they hold and we dequeue the front v_1. Whatever new vertices we enqueue get distance $d[v_1] + 1$, and they go to the *back*. The new front becomes v_2, whose distance is at least $d[v_1]$; the new back has distance $d[v_1] + 1 \leq d[v_2] + 1$. A short check shows both parts survive. So the queue stays sorted by distance and spans at most two adjacent rings.

From this invariant the theorem follows by induction on distance. Every vertex v first becomes gray when some neighbor u with $d[u] = \delta(s, v) - 1$ dequeues; at that instant $d[v]$ is set to $d[u]+1 = \delta(s, v)$, and it is never changed again. The one subtlety is ruling out that v gets a *smaller* (impossible – you cannot beat the true shortest path) or *larger* value; the invariant, by keeping the queue ordered, guarantees v is discovered by a vertex on the correct ring. CLRS Section 20.2 gives the full five-line proof; the picture to keep is the ordered queue.

Walking question

Go for a walk and convince yourself, without any algebra, why a queue and not a stack is what makes BFS find shortest paths. What goes wrong if you swap the queue for a stack – if you always explore the *most recently* discovered vertex instead of the oldest? (Hint: you get a different, equally famous algorithm, and it does *not* respect rings. That is the next section.)

22.3.3 The running time of BFS

Rather than quote the cost, let us derive it. Two things happen during BFS: queue operations, and scanning adjacency lists.

Queue operations. Each vertex is colored gray exactly once – the moment it is enqueued – and the

white check forbids enqueuing it again. So each vertex is enqueued once and dequeued once: $O(V)$ total queue operations, each $O(1)$.

Scanning lists. When a vertex u is dequeued, we scan its entire adjacency list $Adj[u]$ once, doing $O(1)$ work per neighbor. Since each vertex is dequeued once, each adjacency list is scanned once. The total length of all adjacency lists is $\Theta(E)$ (counting each edge once if directed, twice if undirected). So all the scanning costs $\Theta(E)$.

Adding the one-time initialization of $\Theta(V)$, we get:

Cost summary

BFS runs in $O(V + E)$ time and uses $O(V)$ extra space for the queue, colors, distances, and predecessors. This is *linear in the size of the graph* – you cannot do better, because you must at least look at every vertex and edge. The + matters: you pay for vertices *and* edges separately, never their product. This is the payoff of adjacency lists; the same algorithm on an adjacency matrix would cost $O(V^2)$, because scanning a vertex's neighbors means walking a whole matrix row of length V.

22.4 Depth-First Search: Plunge and Backtrack

Breadth-first search is cautious: it never steps into ring $d+1$ until ring d is fully mapped. *Depth-first search* (DFS) does the opposite: from wherever it stands, it follows the first unexplored edge it sees, then down the first edge from *there*, plunging ever deeper until it hits a dead end – a vertex whose neighbors are all already visited. Only then does it *backtrack* one step and try the next edge.

Intuition

DFS is how you explore a maze with a ball of string and a piece of chalk. You walk forward, chalking each junction so you know you have been there, always taking some unmarked corridor. When every corridor from your current spot is already chalked, you reel in the string one junction and try another corridor there. You only stop when the string is fully reeled in and you are back at the start with no fresh corridors anywhere. The string is the recursion stack; the chalk is the color.

Because DFS backtracks, it is naturally *recursive*: "explore u" means "for each neighbor, if it is white, explore it." The recursion stack remembers the trail of string for us. We keep the same white/gray/black coloring as BFS, but now we also stamp each vertex with two *timestamps* from a global clock that ticks once per event:

- $d[u]$, the *discovery time*, when u turns gray (we first arrive),
- $f[u]$, the *finish time*, when u turns black (we leave for good, having explored everything reachable below it).

These timestamps record the nested structure of the search, and they are what make topological sort and strongly connected components possible. Every vertex gets a discovery and a finish time, so the clock runs from 1 to $2V$.

Unlike BFS, DFS does not need a source: it sweeps over *all* vertices, launching a fresh exploration

from each white one, so that even a disconnected graph is fully covered. The result is a *forest* of trees, one per launch.

Algorithm 101: DFS(G)

Input: Graph $G = (V, E)$ as adjacency lists.
Output: Discovery/finish times $d[u], f[u]$ and predecessors $\pi[u]$.

```
for each vertex u ∈ V do
    color[u] ← white;   π[u] ← NIL;
time ← 0;
for each vertex u ∈ V do
    if color[u] = white then
        DFS-Visit(G, u);
```

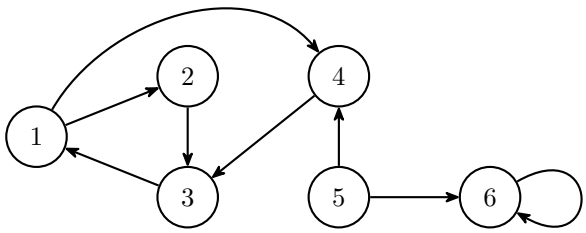
Algorithm 102: DFS-Visit(G, u)

```
time ← time + 1;   d[u] ← time;
color[u] ← gray;
for each v ∈ Adj[u] do
    if color[v] = white then
        π[v] ← u;
        DFS-Visit(G, v);
color[u] ← black;
time ← time + 1;   f[u] ← time;
```

22.4.1 A full numeric trace

Let us run DFS on a *directed* graph – the timestamps are most interesting there. Consider six vertices with arrows:

$$1 \to 2, \quad 1 \to 4, \quad 2 \to 3, \quad 3 \to 1, \quad 4 \to 3, \quad 5 \to 4, \quad 5 \to 6, \quad 6 \to 6.$$



We process vertices in numeric order and, within each list, neighbors in increasing order. The clock starts at 0.

We launch from 1 (time becomes 1, $d[1] = 1$). Its first white neighbor is 2 ($d[2] = 2$). From 2 we go to 3 ($d[3] = 3$). From 3 the only neighbor is 1, which is *gray* (still on the stack) – skip it. So 3 has no white neighbor; it finishes: $f[3] = 4$. Back at 2: no more neighbors, finish $f[2] = 5$. Back

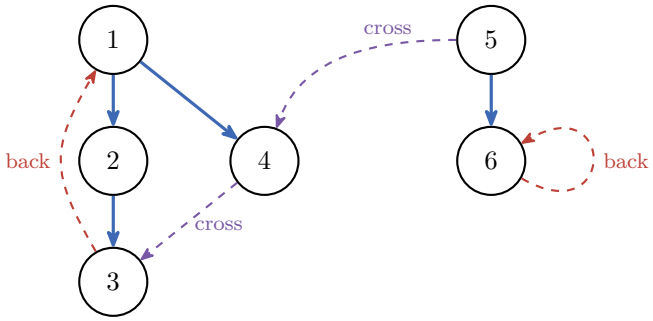
at 1: next neighbor 4 is white ($d[4] = 6$). From 4 the neighbor 3 is now *black* – skip. So 4 finishes $f[4] = 7$. Back at 1: no more neighbors, finish $f[1] = 8$.

The outer loop now looks for the next white vertex. Vertices $2, 3, 4$ are black; 5 is white. Launch from 5 ($d[5] = 9$). Neighbor 4 is black – skip. Neighbor 6 is white ($d[6] = 10$). From 6, the only neighbor is 6 itself, which is gray – skip. Finish $f[6] = 11$. Back at 5: done, $f[5] = 12$.

Collecting the stamps as (d, f) pairs:

Vertex	1	2	3	4	5	6
d	1	2	3	6	9	10
f	8	5	4	7	12	11

The DFS forest has two trees: one rooted at 1 (containing $1, 2, 3, 4$) and one rooted at 5 (containing $5, 6$). It is worth redrawing the graph as that forest, with each non-tree edge labeled by its kind (we classify the edges in the next subsection); the solid edges are the tree, and the dashed ones are the back and cross edges that DFS *walked but did not build the tree from*.



22.4.2 The parenthesis structure

Treat the intervals $[d[u], f[u]]$ as if each vertex opens a parenthesis at $d[u]$ and closes it at $f[u]$. Writing them out in time order gives a nested string:

$$(_1\ (_2\ \underbrace{(_3\ )_3}_{[3,4]}\ )_2\ \underbrace{(_4\ )_4}_{[6,7]}\ )_1\ \underbrace{(_5\ \underbrace{(_6\ )_6}_{[10,11]}\ )_5}_{[9,12]}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad}_{[2,5]}$$

$$\underbrace{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad}_{[1,8]}$$

They never *cross*. Vertex 3's interval $[3, 4]$ sits entirely inside 2's $[2, 5]$, which sits inside 1's $[1, 8]$. This nesting holds in every DFS run, and it is the most useful structural property of depth-first search. Figure 22.2 plots these intervals against the clock.

Formal

Parenthesis theorem. In any DFS, for any two vertices u and v, exactly one of these holds:

- the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are completely **disjoint**, and neither is a descendant of the other in the forest;

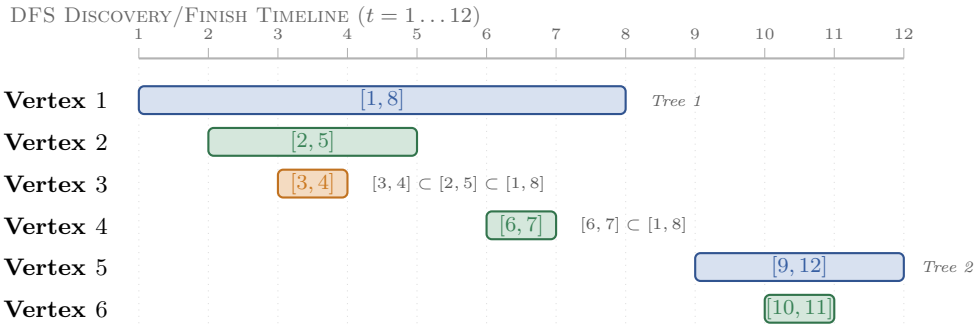


Figure 22.2: The Parenthesis Theorem visualized as interval nesting across the discrete DFS clock. Each interval $[d[u], f[u]]$ represents the active lifetime of vertex u on the recursive call stack. Descendant intervals ($[3, 4]$ inside $[2, 5]$ inside $[1, 8]$) nest strictly within their ancestors. Disjoint intervals ($[1, 8]$ and $[9, 12]$) correspond to separate trees in the DFS forest. No two intervals ever partially overlap without containment.

- one interval is entirely **contained** in the other, and the inner vertex is a descendant of the outer in the DFS forest.

The forbidden case – intervals that overlap but neither contains the other – **never happens.**

Why must this be true? Because the recursion stack enforces nesting. If u is discovered while v is still gray (on the stack), then u is being explored *inside* v's call, so u must finish before control returns to v – forcing $[d[u], f[u]]$ inside $[d[v], f[v]]$. If instead u is discovered after v has already finished, the two calls never overlap in time at all. There is no third option: a vertex is either still on the stack or already off it.

Focus

The slogan: *descendant intervals nest inside ancestor intervals; unrelated intervals are disjoint.* Equivalently, v is a descendant of u in the DFS forest *if and only if* $d[u] < d[v] < f[v] < f[u]$. Most DFS applications – topological sort, strongly connected components, finding articulation points – rely on this nesting fact.

22.4.3 Classifying the edges

This nesting structure lets us classify every edge of G during the search. As DFS walks an edge (u, v) – meaning we are at gray vertex u and looking at neighbor v – the *color of* v tells us exactly what kind of edge it is. There are four kinds:

- **Tree edge**: v is white. We are discovering v for the first time through this edge. Tree edges are the solid edges of the DFS forest.
- **Back edge**: v is gray. Then v is still on the stack – it is an *ancestor* of u. The edge points backward, up toward the root. Back edges are the exact witnesses to cycles.
- **Forward edge**: v is black and $d[u] < d[v]$. Then v is a *descendant* of u that we already finished exploring by another route. The edge skips forward to an already-explored descendant.

- **Cross edge**: v is black and $d[u] > d[v]$. Then v is in a different subtree (or an earlier-finished sibling). The edge crosses between branches.

Intuition

The color of the vertex you are looking at almost *determines* the edge type: white means tree, gray means back, black means forward-or-cross, and you split the last case by comparing discovery times. You never have to do anything extra to classify edges; the coloring the search already maintains gives the answer.

Figure 22.3 illustrates the geometric meaning of each classification relative to the DFS tree.

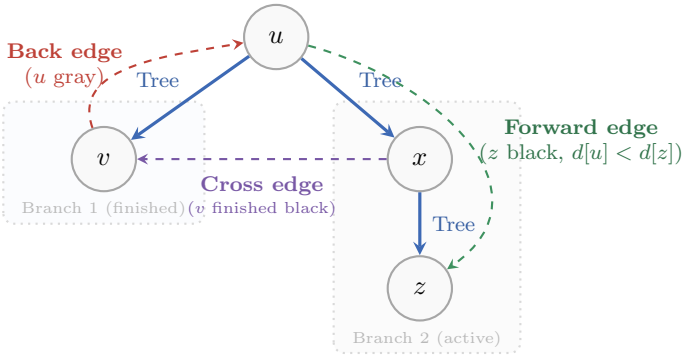


Figure 22.3: Classification of edges during Depth-First Search. Solid blue lines are tree edges forming the DFS forest. Non-tree edges: *back edges* return up to a gray ancestor on the active call stack (detecting a cycle); *forward edges* skip down to an already-finished descendant; and *cross edges* connect across to an already-finished vertex in an earlier-explored branch.

In our trace above, walk the edges: $1 \to 2$, $2 \to 3$, $1 \to 4$ are tree edges (each found a white vertex). $3 \to 1$ is a *back edge* – when we looked at 1 from 3, vertex 1 was gray (an ancestor). $4 \to 3$ is a *cross edge* – when we looked at 3 from 4, vertex 3 was already black and $d[4] = 6 > d[3] = 3$. The self-loop $6 \to 6$ is a back edge (a vertex is its own ancestor in the degenerate sense). The edge $5 \to 4$ is also a cross edge: 4 was black with $d[4] = 6 < d[5] = 9$.

Formal

Cycle test. A directed graph has a cycle *if and only if* some DFS of it produces a back edge.

This is the key to detecting cycles, and it has a one-line proof both ways. If there is a back edge (u, v), then v is an ancestor of u, so the tree path from v down to u plus the edge (u, v) is a cycle. Conversely, if there is a cycle, consider the first vertex v on it that DFS discovers; following the cycle, every later vertex is reachable from v through white vertices, so they all become descendants of v, and the cycle's edge that returns to v is a back edge. We will lean on exactly this fact in topological sort.

Common trap

Edge classification depends on the *order* in which DFS happens to visit vertices and neighbors. The same graph, explored in a different order, can label the same edge differently (a forward edge in one run might not appear as such in another). But two things never change: tree and

back edges in an *undirected* graph are the only two kinds that can appear, and a back edge appears if and only if the graph has a cycle. Do not over-read a specific classification as a property of the graph; read it as a property of *this* traversal.

22.4.4 The running time of DFS

The accounting is the same shape as BFS. The initialization loop is $\Theta(V)$. DFS-Visit is called exactly once per vertex (the white check guards it), contributing $\Theta(V)$ across all calls. Inside those calls, we scan each adjacency list exactly once, and the lists total $\Theta(E)$ in length.

Cost summary

DFS runs in $\Theta(V+E)$ time, linear in the graph, using $O(V)$ space for colors, timestamps, predecessors, and the recursion stack (whose depth is at most V). Like BFS, it touches every vertex and edge a constant number of times and never their product. If your DFS ever seems to cost more, you are probably re-scanning lists or testing edges through a matrix – check that.

22.5 Topological Sort: Untangling Dependencies

Now an application. Suppose you must do a list of tasks, and some tasks depend on others: you cannot put your socks on after your shoes, cannot compile a file before its headers, cannot take Algorithms before Discrete Math. Model each task as a vertex and draw an arrow $u \to v$ meaning "u must come before v." The dependencies form a *directed graph*, and if your requirements are sane – no task transitively requires itself – that graph is a *DAG*, a directed acyclic graph.

The question: *can you line all the tasks up in a single row so that every arrow points forward*? Such a linear order is a *topological sort*. It tells you a valid sequence in which to actually do everything.

Intuition

Think of the famous "getting dressed" graph: underwear before pants, pants before belt, socks before shoes, shirt before tie, tie before jacket. There is no single "correct" order – you might do socks-then-underwear or underwear-then-socks, it doesn't matter – but *any* order you pick must respect every arrow. A topological sort produces one such legal dressing sequence. There can be many; you just need one.

DFS already computes this at almost no extra cost. Recall the finish times $f[u]$. The method is:

Formal

Topological sort via DFS. Run DFS on the DAG. As each vertex finishes (turns black), prepend it to the front of a list. When DFS ends, the list is a valid topological order. Equivalently: output the vertices in *decreasing order of finish time* $f[u]$.

Algorithm 103: Topological-Sort(G)

Input: A directed acyclic graph $G = (V, E)$.
Output: A list of vertices in topological order.

```
1 Call DFS(G), computing finish times f[u] for all u;
2 (as each vertex finishes, push it onto the front of a linked list L);
3 return L;
```

Why does ordering by finish time work? The proof rests on one claim:

Formal

Claim. If G is a DAG and DFS is run on it, then for every edge (u, v) we have $f[u] > f[v]$.

The argument is short. Consider any edge (u, v). When DFS explores u and looks at v, v has some color.

- v cannot be *gray*. If it were, (u, v) would be a back edge, and a back edge means a cycle – but G is acyclic. Contradiction. (This is the only place we use the no-cycles assumption, and it is essential.)
- If v is *white*, then v becomes a descendant of u, so by the parenthesis theorem v finishes before u: $f[v] < f[u]$. Good.
- If v is *black*, then v already finished while u is still gray, so $f[v] < d[u] < f[u]$. Good.

In every possible case, $f[v] < f[u]$. So listing vertices by decreasing finish time puts u before v for *every* edge (u, v) – which is exactly the definition of a topological order. Done.

22.5.1 A worked topological sort

Take this small DAG of morning tasks:

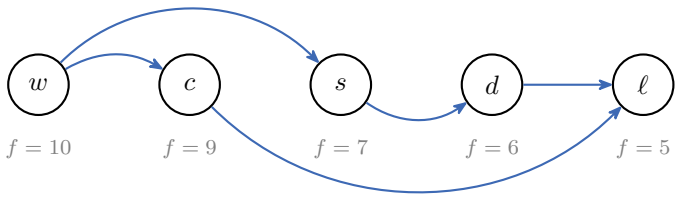
$$\text{wake} \to \text{shower}, \quad \text{shower} \to \text{dress}, \quad \text{wake} \to \text{coffee}, \quad \text{dress} \to \text{leave}, \quad \text{coffee} \to \text{leave}.$$

Label them w, s, d, c, ℓ. Run DFS starting at w, neighbors in the order listed. We discover w ($d[w] = 1$), then s ($d = 2$), then d ($d = 3$), then ℓ ($d = 4$). From ℓ there are no out-edges, so ℓ finishes $f[\ell] = 5$. Back up: d finishes $f[d] = 6$, s finishes $f[s] = 7$. Back at w, next neighbor c is white: discover c ($d = 8$); its neighbor ℓ is black, skip; c finishes $f[c] = 9$; finally w finishes $f[w] = 10$.

Finish times: $f[w] = 10$, $f[c] = 9$, $f[s] = 7$, $f[d] = 6$, $f[\ell] = 5$. Sorting by *decreasing* finish time:

$$w\ (10), \quad c\ (9), \quad s\ (7), \quad d\ (6), \quad \ell\ (5).$$

So a valid order is: **wake, coffee, shower, dress, leave**. Drawn on a line, with every vertex placed left-to-right by decreasing finish time, the point of the claim becomes visible: *every* edge points rightward.



Check every edge: w before s and c (yes), s before d (yes), d before ℓ (yes), c before ℓ (yes). Every arrow points forward in the list. Note this is not the *only* valid order – "wake, shower, dress, coffee, leave" also works – but DFS handed us one legal answer with no extra effort.

Why this matters

Topological sort appears in many systems. `make` and every modern build system topologically sorts your source files so headers compile before the code that includes them. Spreadsheet engines topologically sort cell formulas so a cell is computed after the cells it references. Package managers (`apt`, `npm`, `pip`) topologically sort dependencies to decide install order. Course-planning tools sort prerequisites. Every time a program must do things "in a valid order given constraints," it is usually doing a topological sort. And because it is just one DFS, it costs only $O(V + E)$.

Walking question

On your walk, ponder: what happens if you run Topological-Sort on a directed graph that actually contains a cycle? Trace what the DFS does – it produces a back edge. Could you modify Topological-Sort to *detect* the cycle and report "these tasks have a circular dependency" instead of silently producing garbage? (You can: it is exactly the back-edge test from the previous section.)

22.6 Strongly Connected Components

Our last application. In a *directed* graph, two vertices u and v are *strongly connected* if you can get from u to v *and* from v back to u, each by following arrows. This "mutual reachability" is an equivalence relation, so it carves the vertices into clusters: a *strongly connected component* (SCC) is a maximal set of vertices that are all pairwise mutually reachable.

Intuition

Picture a city of one-way streets. Two intersections are in the same SCC if you can legally drive from either to the other and back. An SCC is a "neighborhood" you can fully circulate within. Between neighborhoods, traffic might flow one way only – you can leave downtown for the suburbs but never drive back. If you shrink each neighborhood to a single point, the city of one-way streets becomes a *DAG*: a clean hierarchy with no loops. That shrunken DAG, the *component graph*, is often what you really wanted to understand.

Focus

The component graph (contract each SCC to a single super-vertex) is *always* a DAG. Reason: if it had a cycle among components, all those components would be mutually reachable and would therefore be one single larger component – a contradiction with "maximal." So finding the SCCs of a directed graph exposes its DAG of components.

How do we find these components? A standard answer is *Kosaraju's algorithm*, which uses DFS *twice* and a trick involving the *transpose* graph. The transpose G^T is G with every edge reversed: if G has $u \to v$, then G^T has $v \to u$. The key fact is that G and G^T have *exactly the same strongly connected components* – reversing all arrows does not change whether two vertices can reach each other both ways.

Algorithm 104: Strongly-Connected-Components(G)

Input: A directed graph $G = (V, E)$.
Output: The strongly connected components of G.

1 Call DFS(G) to compute finish times $f[u]$ for every vertex u;
2 Compute the transpose G^T (reverse every edge);
3 Call DFS(G^T), but in the main loop consider the vertices in order of *decreasing* $f[u]$ (from step 1);
4 Output the vertices of each tree of the second DFS forest as one SCC;

The algorithm is just two depth-first searches and an edge reversal, and it runs in $\Theta(V + E)$. Why it works needs a careful explanation.

22.6.1 Why the trick works

Think about the component graph – the DAG you get by shrinking each SCC to a point. Because it is a DAG, it has "source" components (nothing flows in) and "sink" components (nothing flows out).

The key fact is about finish times *of whole components*. Define the finish time of a component C to be the largest finish time among its vertices. Then:

Formal

Lemma. If C and C' are distinct SCCs and there is an edge from *some* vertex of C to *some* vertex of C' in G, then the largest finish time in C is greater than the largest finish time in C'.

In words: in the first DFS, edges between components always run from *later*-finishing to *earlier*-finishing components. This is the component-level version of the DAG fact we proved for topological sort ($f[u] > f[v]$ across an edge). So if we sort components by their largest finish time, we get a topological order of the component DAG – the source components come first.

Now see what reversing every edge accomplishes. In the second DFS we start from the vertex of overall largest finish time – which lives in a *source* component C of G. In G^T all edges are reversed, so C becomes a *sink*: in G^T there are no edges *leaving* C to other components. Therefore the DFS-Visit launched from that first vertex can reach exactly the vertices of C and no further – it is trapped inside the component, which is precisely what we want. It scoops up all of C as one tree,

then halts. We move to the next-highest finish time still white, which heads the next component, and so on. Each tree of the second DFS forest is exactly one SCC.

Intuition

In short, the first DFS *ranks the components* by finish time so that sources rank highest. Reversing the edges turns every source into a sink, so when the second DFS starts at the highest-ranked vertex, the reversed edges fence it inside its own component. Peel off the top component, repeat. Each peel is one DFS tree, each tree is one SCC.

22.6.2 A worked example

Reuse the directed graph from the DFS trace in Section 22.4.1: edges $1 \to 2$, $1 \to 4$, $2 \to 3$, $3 \to 1$, $4 \to 3$, $5 \to 4$, $5 \to 6$, $6 \to 6$. We already computed the first-DFS finish times:

$$f[1] = 8,\ f[2] = 5,\ f[3] = 4,\ f[4] = 7,\ f[5] = 12,\ f[6] = 11.$$

Sorting vertices by decreasing finish time gives the order $5\,(12),\ 6\,(11),\ 1\,(8),\ 4\,(7),\ 2\,(5),\ 3\,(4)$.

Now form the transpose G^T by flipping every arrow:

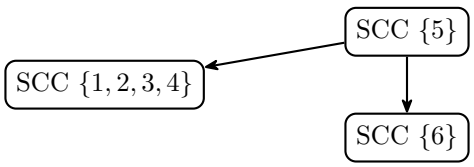
$$2 \to 1,\ 4 \to 1,\ 3 \to 2,\ 1 \to 3,\ 3 \to 4,\ 4 \to 5,\ 6 \to 5,\ 6 \to 6.$$

Adjacency lists of G^T: $Adj^T[1] = [3]$, $Adj^T[2] = [1]$, $Adj^T[3] = [2, 4]$, $Adj^T[4] = [1, 5]$, $Adj^T[5] = [\,]$, $Adj^T[6] = [5, 6]$.

Run the second DFS, taking roots in the order $5, 6, 1, 4, 2, 3$:

- Start at 5. In G^T, vertex 5 has *no* out-edges, so the tree is just $\{5\}$. **SCC:** $\{5\}$**.**
- Next white root is 6. From 6: neighbor 5 is black (skip), neighbor 6 is itself. So the tree is just $\{6\}$. **SCC:** $\{6\}$**.**
- Next white root is 1. From $1 \to 3 \to 2 \to 1$(gray, skip) and $3 \to 4 \to 1$(black, skip), $4 \to 5$(black, skip). So this DFS reaches $1, 3, 2, 4$ and stops. **SCC:** $\{1, 2, 3, 4\}$**.**
- Vertices $4, 2, 3$ are now black, so the loop ends.

The strongly connected components are $\{1, 2, 3, 4\}$, $\{5\}$, and $\{6\}$. Sanity check the big one: in the original G, can we get from 1 to 4 and back? $1 \to 4$ directly; and $4 \to 3 \to 1$ returns. From 1 to 2? $1 \to 2$, and $2 \to 3 \to 1$ returns. Yes – $1, 2, 3, 4$ are all mutually reachable, forming one tight cluster. Vertex 5 can reach others but nothing returns to it, so it stands alone; same for 6.



That last picture is the *component graph*: shrink each SCC to a point and the arrows between them form a DAG (here, $\{5\}$ points to both $\{1, 2, 3, 4\}$ and $\{6\}$, and nothing points back).

Cost summary

Kosaraju's algorithm runs in $\Theta(V+E)$: the first DFS is $\Theta(V+E)$, building the transpose is $\Theta(V+E)$ (one pass over all edges), and the second DFS is $\Theta(V+E)$. That is linear time, even though the problem sounds as if it should require checking reachability between all pairs of vertices.

History and connection

The systematic study of graph traversal began with concrete routing puzzles. In 1959 at Bell Labs, Edward F. Moore formulated breadth-first search to find paths with the fewest switching hops in telephone networks and navigate mazes via concentric wavefronts; C. Y. Lee independently rediscovered the method in 1961 for routing printed-circuit board traces. Depth-first search originated in the 19th century with Pierre Trémaux as a rule for exploring physical labyrinths by tracking visited corridors with markings. In 1972, Robert E. Tarjan systematized depth-first search by introducing discovery and finishing timestamps, establishing the parenthesis structure of recursive calls and proving that topological sorts, biconnectivity, bridges, and strongly connected components can all be determined in linear $\Theta(V+E)$ time.
For strongly connected components, textbooks usually present one of two algorithms: S. Rao Kosaraju's 1978 algorithm (simplified by Micha Sharir in 1981) and Tarjan's 1972 algorithm. Kosaraju computes two full DFS passes and inverts the graph via edge transposition, which simplifies correctness proofs via the parenthesis theorem but incurs additional memory traffic. Tarjan's single-pass method tracks a recursion stack and low-link numbers (the earliest ancestor reachable via back edges) without reversing edges, giving better cache performance at the cost of a more intricate invariant.
Much of later graph algorithms builds on these linear-time traversals: unweighted BFS inspired Dijkstra's shortest-path algorithm, while DFS trees became central to compiler control-flow analysis, garbage collection, and planarity testing. A fundamental theoretical question remains open in parallel and dynamic settings: while BFS parallelizes naturally across frontier levels, DFS is P-complete under log-space reductions, meaning that depth-first search is believed to be inherently sequential and cannot run in polylogarithmic depth with work-efficient processor counts. In dynamic settings, maintaining connectivity and components in sub-polynomial update time under arbitrary edge insertions and deletions remains an active research area.
In CLRS, this material corresponds to Chapter 20 ("Elementary Graph Algorithms"), with strongly connected components treated in Section 20.5.

Why this matters

Why care about SCCs? They expose hidden structure. In a web graph, an SCC is a cluster of pages that all link to each other – a tightly knit community. In a package-dependency graph, an SCC bigger than one vertex is a *circular dependency*, a bug you must break. In program analysis, the SCCs of a call graph reveal mutual recursion. In social networks they find groups where influence circulates. And contracting SCCs turns any messy directed graph into a DAG, on which topological sort and dynamic programming apply. SCC decomposition is often the first step in analyzing a directed graph.

22.7 Summary

This chapter covered the two main graph-traversal strategies and three of their most important consequences.

- **Representation.** Store sparse graphs as *adjacency lists* ($\Theta(V + E)$ space); reserve the *adjacency matrix* ($\Theta(V^2)$) for dense graphs or $O(1)$ edge queries.
- **BFS** explores in rings using a *queue*, computes shortest-path distances *in edges* from a source, and runs in $O(V + E)$.
- **DFS** explores by *plunging and backtracking* using recursion, stamps each vertex with discovery/finish times, classifies edges, and runs in $\Theta(V + E)$. Its *parenthesis structure* is the key structural property.
- **Topological sort** of a DAG falls out of DFS for free: list vertices by *decreasing finish time*. $O(V + E)$.
- **Strongly connected components** come from two DFS passes plus an edge reversal (Kosaraju), also $O(V + E)$.

Focus

If you remember one structural idea from this chapter, make it the parenthesis theorem: DFS intervals nest, never cross. Topological sort, cycle detection, and SCCs all rely on that nesting fact. And if you remember one practical idea, make it $O(V + E)$: every algorithm here is linear in the graph, and that is the standard to aim for in a graph algorithm.

Everything ahead – weighted shortest paths (Dijkstra, Bellman–Ford), minimum spanning trees (Prim, Kruskal), maximum flow – builds directly on this foundation. They are BFS and DFS with weights, priorities, and cleverer bookkeeping added. In Chapter 23, we explore what happens when edges carry weights and we seek the cheapest network connecting all vertices: the minimum spanning tree.

22.8 Exercises

Exercise 22.1. A graph has $V = 1000$ vertices and $E = 4000$ edges. Estimate the storage (in number of entries) for (a) an adjacency matrix and (b) adjacency lists, assuming the graph is undirected. Which uses less, and by roughly what factor?

Exercise 22.2. Run BFS by hand on the undirected graph of Section 22.3.1 starting from $s = 4$. Give $d[v]$ and $\pi[v]$ for every vertex, and write out the shortest path from 4 to 1 that your predecessor tree encodes.

Exercise 22.3. On the same undirected graph, run DFS from vertex 1 (neighbors in increasing order) and record discovery and finish times. Then explain in one or two sentences why the DFS tree and the BFS tree differ even though the graph is the same.

Exercise 22.4. Prove carefully: an *undirected* graph contains a cycle if and only if a DFS of it produces a back edge. (Be careful with the edge that leads straight back to your DFS parent – why does that one not count as a cycle?)

Exercise 22.5. Consider the DAG with edges $a \to b$, $a \to c$, $b \to d$, $c \to d$, $d \to e$, $c \to e$. Run DFS (vertices and neighbors in alphabetical order), report all finish times, and produce a topological order. Then exhibit a *second*, different valid topological order, proving the answer is not unique.

Exercise 22.6. A DAG can have many topological orders. Give a small DAG on 4 vertices that has exactly 2 valid topological orders, and a different DAG on 4 vertices that has $4! = 24$ valid orders. (Hint: the second has no edges at all – what does that allow?)

Exercise 22.7. For the directed graph with edges $a \to b$, $b \to c$, $c \to a$, $c \to d$, $d \to e$, $e \to d$, list all strongly connected components by inspection (no full algorithm needed). Then draw the component graph and confirm it is a DAG.

Exercise 22.8. Run Kosaraju's algorithm in full on the graph of the previous exercise: give the first-DFS finish times, the transpose's adjacency lists, the order of roots in the second DFS, and the resulting components. Check they match your by-inspection answer.

Exercise 22.9. Prove that G and its transpose G^T have exactly the same strongly connected components. (Hint: u and v are mutually reachable in G if and only if they are mutually reachable in G^T – show this directly from the definition of a path.)

Exercise 22.10. (Challenge.) A graph is *bipartite* if you can 2-color its vertices so that every edge joins two different colors. Describe how to modify BFS so that it either produces a valid 2-coloring or reports that the graph is not bipartite, and argue your method runs in $O(V + E)$. (Hint: color each vertex the opposite of its BFS parent, and watch what happens at an edge between two same-distance vertices.)

Exercise 22.11. (Challenge.) Given a DAG and a source s, describe an $O(V + E)$ algorithm to count the number of distinct paths from s to every other vertex. (Hint: process vertices in topological order; the number of paths to v is the sum over its in-neighbors u of the number of paths to u. Why does topological order make this work in one pass?)

Chapter 23

Minimum Spanning Trees

In Chapter 22, breadth-first and depth-first search traversed unweighted graphs by counting hops. But real-world networks rarely treat every connection equally: laying fiber across a mountain pass costs vastly more than stringing cable down a city street.

Imagine you have been handed a map of a small mountain region. There are a dozen villages scattered across the slopes, and the regional council wants to lay fiber-optic cable so that every village can reach every other village over the network. Cable is expensive, and the cost of connecting two particular villages depends on the terrain between them: some pairs are a short flat hop, others require boring a tunnel through a ridge. You are given, for each pair of villages that *could* be connected directly, the cost of that link. Your job is to choose a set of links so that the whole region is connected — you can get from any village to any other by hopping along chosen links — while spending as little money as possible.

That is the *minimum spanning tree* problem. The simple greedy rule — "keep selecting the cheapest link that does not form a cycle" — is provably optimal. The reason *why* is a result called the *cut property*. Once you understand the cut property, the two classical algorithms in this area, Kruskal's and Prim's, are two different ways of applying it repeatedly.

Focus

The minimum spanning tree problem asks for an acyclic subgraph connecting all vertices with the smallest possible total edge weight. The cut property ensures that purely local greedy choices yield a globally optimal network.

23.1 What exactly are we looking for?

Let us pin down the vocabulary before we chase any algorithm. We model the region as a *graph*. The villages are *vertices*, gathered into a set V. The possible direct links are *edges*, gathered into a set E; each edge joins two vertices. Every edge e carries a number $w(e)$, its *weight* — here, the cost of building that link. We write the graph as $G = (V, E)$ with weight function w. The graph is *undirected* (a cable from u to v is also a cable from v to u) and *connected* (there is at least one way to get between any two villages using available links — otherwise no choice of edges could ever connect everyone, and the problem has no solution).

Definition 23.1 (Spanning tree)**.** A *spanning tree* of a connected undirected graph $G = (V, E)$ is a subset $T \subseteq E$ of the edges such that the graph (V, T) is (i) *connected* — every vertex can reach every other using only edges of T — and (ii) *acyclic* — T contains no cycle. A set of edges that is connected and acyclic is called a *tree*; "spanning" means it touches all of V.

Intuition

A spanning tree is the leanest possible connected skeleton of the graph. It is *connected*, so the network does its job: everyone can reach everyone. It is *acyclic*, so no cable is wasted: a cycle would mean two different routes between some pair of villages, and you could snip one edge of that cycle and still keep everyone connected — so a cycle is, by definition, an avoidable expense.

Trees obey an exact counting identity that we will rely on throughout: a tree on n vertices has *exactly* $n - 1$ edges. Fewer than $n - 1$ edges cannot connect n vertices; more than $n - 1$ edges on n vertices must contain a cycle. So every spanning tree of a graph with $n = |V|$ vertices uses precisely $n - 1$ edges — no algorithm can do better or worse on the *count*. The only thing left to optimize is *which* $n - 1$ edges.

Definition 23.2 (Minimum spanning tree)**.** The *weight* of a spanning tree T is $w(T) = \sum_{e \in T} w(e)$. A *minimum spanning tree* (MST) is a spanning tree of smallest possible total weight. (There may be several MSTs tied for the minimum; we want any one of them.)

Focus

Hold these three facts: (1) A spanning tree is connected and acyclic and touches every vertex. (2) Every spanning tree of an n-vertex graph has exactly $n - 1$ edges. (3) We want one whose total weight is as small as possible. The rest of the chapter shows how to *find* that tree without checking the enormous number of spanning trees one by one.

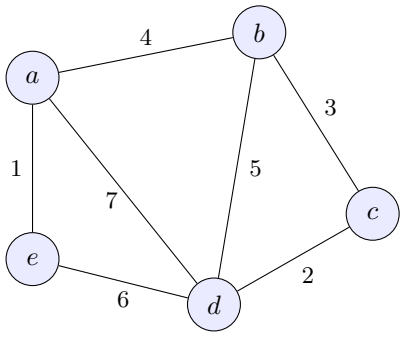
Checking every possible tree is out of the question: a complete graph on n vertices has n^{n-2} different spanning trees (a classical result of Cayley). For a mere 30 villages that is already 30^{28}, a number with 42 digits, so brute force is hopeless. Instead we use the structure of the problem, expressed by the cut property.

Why this matters

Minimum spanning trees are foundational tools of network design: laying water pipes, electrical grids, road and rail networks, and fiber-optic links. They appear inside clustering algorithms (delete the most expensive edges of an MST and the tree falls into natural clusters), inside image segmentation, inside the approximation algorithm for the traveling salesman problem you will meet much later, and even in the wiring of computer chips. When "connect everything as cheaply as possible" is the question, an MST is usually the answer.

23.2 A running example

We use one small graph for the rest of the chapter. Five villages, a through e, with the following links and costs.



Read off the edges and weights: ae=1, cd=2, bc=3, ab=4, bd=5, de=6, ad=7. There are 5 vertices, so every spanning tree will use exactly 4 edges. Spend a moment eyeballing it: the cheap edges are ae=1, cd=2, bc=3, ab=4. Do those four edges form a spanning tree? They connect a–e, c–d, b–c, a–b. Following the chain: e–a–b–c–d. That is all five villages, no cycle, four edges. Total cost $1 + 2 + 3 + 4 = 10$. We will prove later that 10 is optimal, but you can already sense the strategy: grab cheap edges, refuse any that would close a loop.

23.3 The cut property

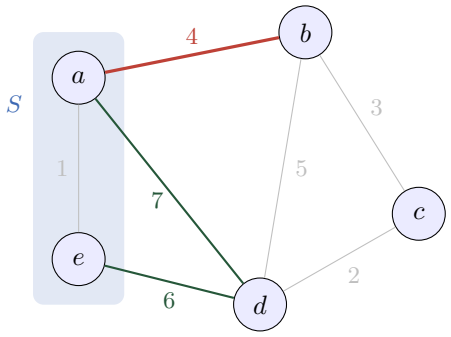
Every algorithm in this chapter rests on one idea. Suppose you split the vertices into two non-empty groups — call this split a *cut*. Some edges have both endpoints inside the same group; those edges stay clear of the divide. But some edges have one endpoint in each group; those edges *cross* the cut.

Definition 23.3 (Cut and crossing edge)**.** A *cut* of $G = (V, E)$ is a partition of V into two non-empty sets S and $V \setminus S$. An edge *crosses* the cut if it has one endpoint in S and the other in $V \setminus S$. A *light edge* crossing the cut is a crossing edge of minimum weight among all edges that cross that cut.

Intuition

Picture a river slicing your map into a north bank (S) and a south bank ($V \setminus S$). Some links are entirely on one bank; the river does not touch them. The links that span the river are the *crossing* edges. Now ask: if the final network must connect north to south, it must include *at least one* bridge across the river. Which bridge should it be? Intuitively, the cheapest one. The cut property proves that choosing the cheapest crossing bridge is always safe.

Here is a concrete cut on our five-village graph. Take $S = \{a, e\}$ (shaded) and $V \setminus S = \{b, c, d\}$. The edges *inside* a side (ae, bc, cd, bd) stay gray; the edges that *cross* the cut are drawn in color with their weights: ab=4, ad=7, de=6. The lightest crossing edge is ab=4, so the cut property promises ab is in some MST.



Theorem 23.1 (Cut property)**.** *Let $G = (V, E)$ be a connected, undirected, weighted graph. Take any cut $(S, V \setminus S)$, and let e be a light edge crossing that cut (a crossing edge of minimum weight). Then e belongs to* some *minimum spanning tree of G.*

If, in addition, all edge weights are distinct, then e belongs to every *minimum spanning tree — the MST is unique and e is forced.*

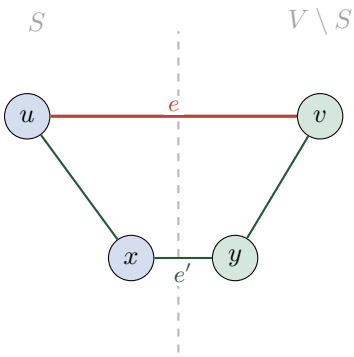
The theorem does not require you to know anything about the rest of the graph. Pick *any* cut you like; the cheapest edge across it is safe to commit to. So a purely *local* decision — look at one cut, grab its cheapest crossing edge — is guaranteed to be part of a *globally* optimal solution. Greedy algorithms rarely get a guarantee this strong.

23.3.1 Proof of the cut property, slowly

We will use an *exchange argument*, the standard technique for proving greedy choices safe. The plan: assume someone hands you an MST that does *not* contain our light edge e, and show you can swap e into it without making it worse. If you can never make it worse by adding e, then some MST contains e.

Proof. Let $e = (u, v)$ be a light edge crossing the cut $(S, V \setminus S)$, with $u \in S$ and $v \in V \setminus S$. Let T be *any* minimum spanning tree. If $e \in T$ we are already done, so assume $e \notin T$.

Now T is a spanning tree, so it connects u to v by *some* path P inside T. (A spanning tree connects every pair of vertices, and does so by a unique path, since trees have no cycles.) This path starts at u, which is in S, and ends at v, which is in $V \setminus S$. Walking along P from u to v, you begin on the S side and finish on the other side, so *somewhere* along the way the path must step across the cut. Let $e' = (x, y)$ be an edge of P that crosses the cut, with $x \in S$ and $y \in V \setminus S$.



Both e and e' cross the same cut. Since e is a *light* crossing edge, $w(e) \le w(e')$.

Here is the exchange. Form a new edge set $T' = T - \{e'\} + \{e\}$: throw out e', throw in e. I claim T' is again a spanning tree. Why? Adding e to the tree T creates exactly one cycle, and that cycle consists of the new edge e together with the path P that already joined u and v. The edge e' lies on P, hence on that cycle. Deleting any one edge of a cycle leaves the graph connected, so removing e' breaks the cycle without disconnecting anything. After the swap we still have $|V| - 1$ edges, still connected, hence still a spanning tree.

What is its weight? We removed e' and added e, so

$$w(T') = w(T) - w(e') + w(e) \le w(T),$$

because $w(e) \le w(e')$. But T was a *minimum* spanning tree, so its weight cannot be beaten: $w(T') \ge w(T)$. The only way to have both $w(T') \le w(T)$ and $w(T') \ge w(T)$ is $w(T') = w(T)$. Therefore T' is also a minimum spanning tree — and it contains e. So *some* MST contains e, which is the first claim.

For the second claim, suppose all weights are distinct. Then unless $e' = e$ we have strict inequality $w(e) < w(e')$, which would give $w(T') < w(T)$, contradicting that T is minimum. So in fact $e' = e$, meaning e was in T all along. Since T was an arbitrary MST, *every* MST contains e. □

Focus

The exchange argument is useful well beyond this theorem. The pattern — "assume an optimal solution lacks my greedy choice; find a cycle/path that lets me swap my choice in for free; conclude an optimal solution *can* contain my choice" — recurs throughout greedy algorithm design and combinatorial optimization.

Common trap

The cut property does **not** say "the globally cheapest edge of the whole graph is in every MST." It says "the cheapest edge *across a given cut* is safe." These coincide for the very cheapest edge (it is light across the cut separating one of its endpoints from everything else), but for a general edge you must name the cut. Beginners often try to argue "this edge is the second cheapest, so it must be in the MST" — false in general. Always exhibit a cut.

23.4 The generic MST algorithm

The cut property hands us a strategy so general that it is barely an algorithm yet — more like a recipe with a blank to fill in. We grow a set of edges A that we have committed to. We maintain one invariant: *A is always a subset of some MST.* At each step we find an edge that is "safe" to add — meaning $A \cup \{e\}$ is still a subset of some MST — and add it. When A has $|V| - 1$ edges, it is a spanning tree, and since it is a subset of some MST and has the right size, it *is* that MST.

Algorithm 105: GENERIC-MST(G, w)

Input: A connected weighted undirected graph $G = (V, E)$ with weights w.
Output: A minimum spanning tree, returned as an edge set A.

```
A ← ∅;
while A does not yet span all of V (fewer than |V| − 1 edges) do
    find an edge (u, v) that is safe for A;
    A ← A ∪ {(u, v)};                                   // commit to it
return A;
```

How do we find a safe edge? The cut property gives a sufficient condition.

Formal

Call an edge e *safe* for A if $A \cup \{e\}$ is a subset of some MST. **Sufficient condition (from the cut property).** Suppose A is a subset of some MST. Choose any cut $(S, V \setminus S)$ that *no edge of A crosses*. Then a light edge across that cut is safe for A.

The condition "no edge of A crosses the cut" is what guarantees the new edge can be added to the partial solution without conflict: A lives entirely within the two sides, and we are adding the first, cheapest bridge between them. The proof is the cut property applied to the MST that contains A — the light edge joins A rather than disturbing it.

Focus

Every MST algorithm in this chapter is just GENERIC-MST with a specific rule for choosing the cut (and therefore the safe edge):

- **Prim** fixes one growing blob S and always cuts "blob vs. the rest," adding the cheapest edge leaving the blob.
- **Kruskal** keeps a forest of blobs and adds the globally cheapest edge that joins two *different* blobs — which is a light edge for the cut separating one of those blobs from everything else.

Both algorithms apply the same cut-property invariant to different families of cuts.

23.5 Kruskal's algorithm

Kruskal's algorithm follows the village engineer's instinct: sort all the candidate links from cheapest to most expensive, then walk down the list and build each one *unless* it would create a redundant loop.

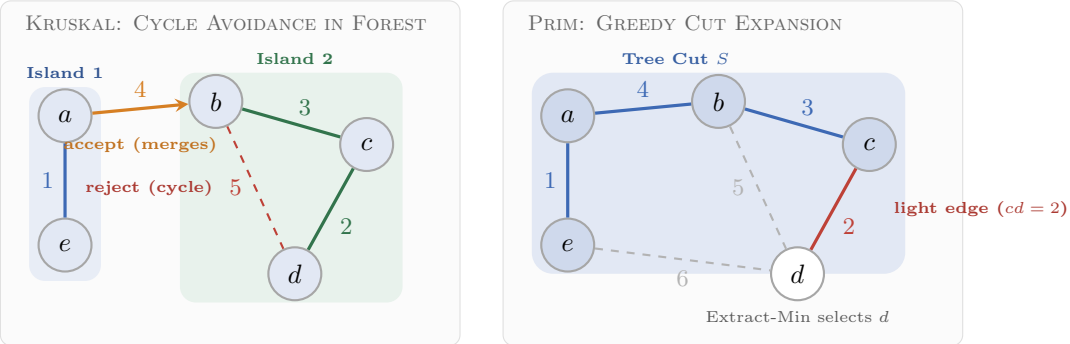


Figure 23.1: Kruskal cycle avoidance versus Prim greedy cut expansion. Left: Kruskal maintains a forest of connected islands; when considering edge ab ($w = 4$), it bridges distinct components and is accepted, whereas edge bd ($w = 5$) joins vertices already in the same component and is rejected to avoid a cycle. Right: Prim grows a single tree S; the cut property guarantees that the lightest edge crossing the boundary into the exterior (here cd with $w = 2$) safely extends the tree without cycles.

23.5.1 The idea

Intuition

Start with n villages and no cable at all — n separate islands. Sort the links by cost. Consider them cheapest first. For each link, ask: do its two endpoints already sit on the same island (already connected by cable laid so far)? If yes, skip it — building it would only close a wasteful loop. If no, build it; this fuses two islands into one. Stop when a single island remains.

Why is each accepted edge safe? Suppose we are about to accept edge (u, v) because u and v are on different islands. Let S be the island containing u. No edge of A crosses the cut $(S, V \setminus S)$ — because S is a *whole* connected island, every edge already built either stays inside S or stays outside it. And (u, v) is the cheapest edge crossing that cut that we have seen, because we are processing edges in increasing weight order and this is the first one we have encountered that joins S to the outside. So (u, v) is a light edge for a cut that A does not cross: safe, by the cut property.

23.5.2 The mechanism: union-find

The one operation we keep performing is "are u and v on the same island, and if not, fuse their islands." This is precisely what a *disjoint-set* (union-find) data structure is built for. You met it in Chapter 19; here is the interface we need.

- Make-Set(x): create a one-element island $\{x\}$.
- Find-Set(x): return a canonical name (the "representative") of the island containing x. Two elements are on the same island iff their Find-Set answers are equal.
- Union(x, y): fuse the islands of x and y into one.

With *union by rank* and *path compression*, a sequence of m of these operations on n elements costs $O(m\,\alpha(n))$, where α is the inverse-Ackermann function — so slow-growing it is at most 4 for any

input size that fits in this universe. For our purposes you may treat each operation as "essentially constant time."

23.5.3 The algorithm

Algorithm 106: MST-KRUSKAL(G, w)

Input: Connected weighted undirected graph $G = (V, E)$.
Output: An MST as an edge set A.

```
A ← ∅;
for each vertex v ∈ V do
    MAKE-SET(v);                                   // each village its own island
sort the edges of E into nondecreasing order by weight w;
for each edge (u, v) ∈ E, taken in that sorted order do
    if FIND-SET(u) ≠ FIND-SET(v) then
        A ← A ∪ {(u, v)};                  // endpoints on different islands:  safe
        UNION(u, v);                                              // fuse the islands
return A;
```

23.5.4 A full trace on our graph

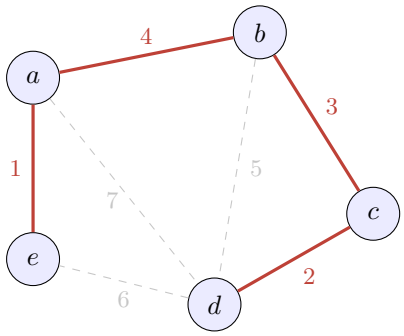
Sort the edges of the five-village graph by weight:

$$ae{=}1, \quad cd{=}2, \quad bc{=}3, \quad ab{=}4, \quad bd{=}5, \quad de{=}6, \quad ad{=}7.$$

Start with five islands: $\{a\}, \{b\}, \{c\}, \{d\}, \{e\}$.

edge	weight	same island?	action / islands after
ae	1	no	**accept**; islands $\{a, e\}, \{b\}, \{c\}, \{d\}$
cd	2	no	**accept**; islands $\{a, e\}, \{b\}, \{c, d\}$
bc	3	no	**accept**; islands $\{a, e\}, \{b, c, d\}$
ab	4	no	**accept**; islands $\{a, b, c, d, e\}$
bd	5	yes	skip (would close cycle b–c–d–b)
de	6	yes	skip
ad	7	yes	skip

We accepted $\{ae, cd, bc, ab\}$, four edges, total weight $1+2+3+4 = 10$. Once the fourth edge merged everything into one island, every later edge had both endpoints in that single island and was skipped. The result matches the tree we eyeballed earlier.



Algorithm lab

Trace Kruskal yourself on this graph. Vertices $\{1, 2, 3, 4\}$; edges with weights 12=1, 13=4, 14=3, 23=2, 34=5. Sort the edges, walk the list, and record the islands after each decision. You should accept three edges of total weight $1 + 2 + 3 = 6$. Which edges? Confirm that each edge you skip would have closed a cycle.

23.5.5 Why Kruskal is correct

Theorem 23.2. MST-KRUSKAL *returns a minimum spanning tree.*

Proof. We show the loop invariant: *at every point, A is a subset of some MST.* Initially $A = \emptyset$, which is a subset of any MST, so the invariant holds.

Suppose it holds and we are about to accept an edge (u, v) because FIND-SET$(u) \neq$ FIND-SET(v). Let S be the set of vertices currently in the island (the union-find tree) of u. Two things are true. First, no edge of A crosses the cut $(S, V \setminus S)$: every edge already in A joined two vertices that ended up in the *same* island, so it lies wholly inside one side. Second, (u, v) is a light edge across this cut. To see this, notice that no crossing edge of $(S, V \setminus S)$ can have been examined before (u, v). Indeed, take any edge (x, y) that crosses the cut now, with $x \in S$ and $y \notin S$. The island S only grew over time, so at any earlier moment x sat in some sub-island of S while y sat outside it; the two endpoints were therefore in *different* islands, and had we examined (x, y) back then we would have *accepted* it, fusing y's side into S — contradicting that y is still outside S. So every crossing edge is examined no earlier than (u, v). Because we scan edges in nondecreasing weight order, (u, v) is no heavier than any crossing edge, i.e. it is a minimum-weight crossing edge. By the cut property, (u, v) is safe: $A \cup \{(u, v)\}$ is still a subset of some MST. The invariant is preserved.

Edges we *skip* (same island) do not change A, so they cannot break the invariant. When the loop ends, A is acyclic (we never added an edge inside an island) and connected (the graph is connected, so processing all its edges must have fused every vertex into one island; an unfused vertex would witness an edge that crosses between islands and was wrongly skipped). Hence A is a spanning tree, and being a subset of some MST with the full $|V| - 1$ edges, it *is* that MST. □

23.5.6 Running time of Kruskal

Let $n = |V|$ and $m = |E|$. Walk through the costs:

- n calls to MAKE-SET: $O(n)$.
- Sorting the m edges: $O(m \log m)$. Since $m \leq n^2$, $\log m \leq 2 \log n$, so this is $O(m \log n)$.
- The main loop runs m times, each doing two FIND-SETs and at most one UNION. That is $O(m)$ union-find operations, costing $O(m\,\alpha(n))$ in total.

Cost summary

Kruskal's algorithm. Sorting dominates.

$$T(n, m) = O(m \log m) + O(m\,\alpha(n)) = O(m \log n).$$

Because $\alpha(n)$ grows slower than every log, the union-find work is lost in the noise; the bottleneck is putting the edges in sorted order. If the edges arrive *already* sorted, or can be sorted in linear time (small integer weights), Kruskal runs in $O(m\,\alpha(n))$, essentially linear.

Common trap

Spanning tree cost vs. shortest-path distance. An MST minimizes total global construction cost (such as total fiber cable or copper wire), but it does *not* minimize point-to-point travel distance between individual vertices. Two neighboring towns might be connected in an MST only through a circuitous path visiting ten other villages. When path length from a designated source matters rather than overall edge weight, we must turn to shortest-path algorithms.

23.6 Prim's algorithm

Kruskal grows many little trees at once and merges them. Prim takes the opposite tack: grow *one* tree, outward from a single seed, always reaching for the cheapest edge that extends it.

23.6.1 The idea

Intuition

Plant a flag on any one village; call it the start of your network. You now have a set $S = \{\text{start}\}$ and everyone else outside. Repeat: among all links with exactly one endpoint inside S, pick the cheapest, and build it. That link adds one new neighbor to S. Keep going $n - 1$ times until S spans every village. The links you built form the tree.

This is GENERIC-MST with the cut fixed as "the blob S versus the rest" at every step. The cheapest edge leaving the blob is a light edge across that cut, and no edge of our growing tree crosses the cut (the tree lives entirely inside S). So every edge Prim adds is safe.

23.6.2 The mechanism: a priority queue

The repeated question is now "which outside village is cheapest to attach to the blob, and by which edge?" For each vertex v still outside, keep a number $key[v]$ = the weight of the cheapest edge connecting v to a vertex already inside, and $\pi[v]$ = the inside vertex at the other end of that edge. We want to repeatedly extract the outside vertex of smallest key, then, when a vertex joins the blob, update its outside neighbors in case it offers them a cheaper hookup. "Repeatedly extract the smallest, and decrease keys" is exactly the job of a *min-priority queue*.

Algorithm 107: MST-PRIM(G, w, r)

Input: Connected weighted undirected graph $G = (V, E)$; a start vertex r.
Output: For each v, its parent $\pi[v]$; the tree edges are $\{(v, \pi[v])\}$.

```
for each vertex u ∈ V do
    key[u] ← ∞;   π[u] ← Nil
key[r] ← 0;
Q ← V;                                   // min-priority queue keyed by key[·]
while Q ≠ ∅ do
    u ← Extract-Min(Q);                  // cheapest vertex to attach now
    for each neighbor v of u do
        if v ∈ Q and w(u, v) < key[v] then
            π[v] ← u;
            key[v] ← w(u, v);            // Decrease-Key
```

Note that "$v \in Q$" means v is still outside the blob. Once a vertex is extracted it is inside the blob forever, and its key is frozen — that frozen key is the weight of the edge that pulled it in.

23.6.3 A full trace on our graph

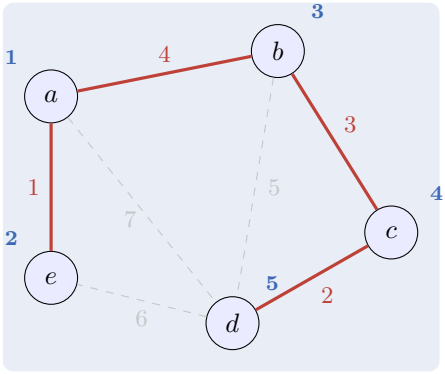
Start at $r = a$. Keys begin at $key[a] = 0$, all others ∞. We will write the key vector as (a, b, c, d, e) and underline the vertex we extract.

step	extract u	edge added	keys after relaxing u
1	$\underline{a}$ (0)	— (root)	b=4 (πa), d=7 (πa), e=1 (πa)
2	$\underline{e}$ (1)	$a-e$	d: de=6 $<$ 7 $\Rightarrow$ d=6 (πe)
3	$\underline{b}$ (4)	$a-b$	c=3 (πb); d: bd=5 $<$ 6 $\Rightarrow$ d=5 (πb)
4	$\underline{c}$ (3)	$b-c$	d: cd=2 $<$ 5 $\Rightarrow$ d=2 (πc)
5	$\underline{d}$ (2)	$c-d$	queue empty

Let us narrate the interesting steps. After extracting a, its neighbors get keys: b=4, d=7, e=1, each pointing back at a. The smallest key now belongs to e, so we extract e and add the edge $a-e$ (weight 1). From e the only outside neighbor is d via de=6, which beats d's current 7, so $key[d]$ drops to 6 with $\pi[d] = e$. Next smallest outside is b at 4; extract it, add $a-b$. Relaxing b: neighbor c gets key=3 ($\pi c = b$), and d improves from 6 to 5 via bd. Next we extract c at 3, add $b-c$; relaxing c drops d from 5 to 2 via cd. Finally extract d at 2, adding $c-d$.

Edges added: $a{-}e\,(1)$, $a{-}b\,(4)$, $b{-}c\,(3)$, $c{-}d\,(2)$. Total $1+4+3+2=10$. Same tree, same cost as Kruskal — as it must be, since the MST here is unique.

The figure below shows the same run as a *growing blob*. The shaded region is S, the set of villages already pulled in; the small number on each tree edge is its weight, and the number in a circle marks the step at which that vertex was extracted. At each step S absorbs one more vertex along the cheapest crossing edge.



Common trap

Notice that $key[d]$ was lowered *four* times — from ∞ to 7, then 6, then 5, then 2 — before d was finally extracted. This is what DECREASE-KEY is for: a vertex's cheapest connection keeps improving as the blob grows and gets closer to it. The value frozen at extraction time (2, via cd) is the one that lands in the tree. A common bug is to "lock in" a vertex's parent the first time you see it; you must keep relaxing until it is extracted.

23.6.4 Why Prim is correct

Theorem 23.3. MST-PRIM *computes a minimum spanning tree.*

Proof. Let $A = \{(v, \pi[v]) : v \neq r,\ v \notin Q\}$ be the set of tree edges collected so far — the edges of vertices already extracted. We claim A is always a subset of some MST. Initially $A = \emptyset$: fine.

Consider the moment we extract a vertex u (other than r) and thereby add edge $(u, \pi[u])$ to A. Let S be the set of vertices already inside the blob just before this extraction (the previously extracted vertices). No edge of A crosses the cut $(S, V \setminus S)$, since every edge of A joins two already-extracted vertices, both in S. We must check that $(u, \pi[u])$ is a light edge across this cut. Each outside vertex v has $key[v]$ equal to the weight of the cheapest edge from v into S (that is the invariant the relaxation loop maintains), and EXTRACT-MIN chose u as the outside vertex with smallest such key. So $w(u, \pi[u]) = key[u]$ is the minimum over all outside vertices of their cheapest edge into S — precisely the minimum-weight edge crossing the cut. By the cut property, $(u, \pi[u])$ is safe, and the invariant survives. When Q empties, every vertex is in the blob, A has $|V| - 1$ edges, and being a subset of an MST of the right size, it *is* an MST. □

23.6.5 Running time of Prim

The cost depends entirely on the priority queue. The algorithm performs:

- $|V|$ Extract-Min operations (one per vertex), and
- at most $|E|$ Decrease-Key operations (each edge (u, v) can trigger at most one key decrease for v when u is extracted; summed over the graph, the relaxation loop runs $\sum_v \deg(v) = 2|E|$ times).

So the running time is

$$|V| \cdot t_{\text{extract}} + |E| \cdot t_{\text{decrease}},$$

and we just plug in the queue's costs.

priority queue	Extract-Min	Decrease-Key	total time
unsorted array	$O(n)$	$O(1)$	$O(n^2 + m) = O(n^2)$
binary heap	$O(\log n)$	$O(\log n)$	$O((n + m) \log n) = O(m \log n)$
Fibonacci heap	$O(\log n)\,*$	$O(1)\,*$	$O(m + n \log n)$

(The Fibonacci-heap bounds are *amortized*, marked $*$.) With a binary heap Prim matches Kruskal at $O(m \log n)$. With a Fibonacci heap Prim reaches $O(m + n \log n)$, which is asymptotically better on *dense* graphs where m is close to n^2. And for a truly dense graph the plain unsorted array, at $O(n^2)$, can beat the heap, since $O(n^2) = O(m)$ when $m = \Theta(n^2)$ and there is no log factor to pay.

Cost summary

Prim's algorithm.

- Binary heap: $O(m \log n)$ — the standard, all-purpose choice.
- Fibonacci heap: $O(m + n \log n)$ — best for dense graphs, in theory.
- Array: $O(n^2)$ — simplest, and best when $m \approx n^2$.

Kruskal's algorithm. $O(m \log n)$, dominated by sorting the edges. On sparse graphs the two are tied; on dense graphs the right Prim variant wins.

History and connection

The minimum spanning tree problem originated in 1926 with Otakar Borůvka, who was commissioned to design an efficient electrical power network connecting forty rural towns across south-western Moravia (Czech Republic) with minimal copper wire. Borůvka introduced the first greedy cut algorithm: every component simultaneously selects its cheapest incident edge, shrinking components in logarithmic rounds. In 1930, Vojtěch Jarník formulated the single-tree growing method using a priority queue, later rediscovered independently by Robert Prim (1957) and Edsger Dijkstra (1959). Joseph Kruskal published his edge-sorting and cycle-checking algorithm in 1956 in a succinct two-page note.

A long-standing question concerned the necessity of sorting: both Kruskal and binary-heap Prim require $O(E \log V)$ time, leading researchers to ask whether finding an MST fundamen-

tally requires sorting all edge weights or if a strictly linear-time algorithm is possible. In 1984, Fredman and Tarjan's Fibonacci heaps lowered Prim's running time to $O(E + V \log V)$, which achieves linear time on dense graphs where $E = \Omega(V \log V)$. In 1995, David Karger, Philip Klein, and Robert E. Tarjan discovered a randomized algorithm that finds an MST in linear $O(V + E)$ expected time by interleaving Borůvka contraction steps with random edge sampling. Bernard Chazelle (2000) established an almost-linear deterministic algorithm running in $O(E \cdot \alpha(V, E))$ time using soft heaps.
The major question that remains open is whether a deterministic, linear-time $O(V + E)$ algorithm exists in the comparison model. In 2002, Seth Pettie and Vijaya Ramachandran proved that an optimal deterministic comparison-based algorithm exists whose runtime matches the unknown decision-tree complexity of the problem, yet whether that complexity is $O(E)$ or requires super-linear work remains unsolved. Note also the parallel between Prim's algorithm and Dijkstra's shortest-path algorithm (Chapter 24): both grow a set of vertices by extracting the minimum-key element from a priority queue, differing only in whether the key is the single connecting edge weight or the accumulated path distance from the source. (In CLRS, this is Chapter 21, with disjoint sets in Chapter 19.)

23.7 Kruskal vs. Prim: which, and why

Intuition

Kruskal thinks globally about edges: sort everything, take the cheapest that does not loop. It grows a *forest* of many trees that gradually merge. It is the natural choice when the edges are few (sparse graph) or already sorted.
Prim thinks locally about one tree: keep extending a single blob along its cheapest exit. It grows *one* tree from a seed. It does well on dense graphs, especially with a good priority queue, because it never has to sort all the edges at once — it only ever looks at edges on the current frontier.

Both rest on the same foundation — the cut property — and both return a correct MST. They differ only in *which cut* they apply the property to at each step. One theorem thus gives rise to a family of algorithms, each making the abstract "find a safe edge" step concrete in its own way.

Walking question

Consider a dual question on your walk. The cut property guarantees the cheapest edge across any cut is in *some* MST. Is there a matching "cycle property"? Try to convince yourself that the *most expensive* edge on any cycle can be safely *excluded* from some MST. (Hint: mirror the exchange argument — if an MST contained that heaviest cycle edge, swap it out for the edge that closes the cycle.) If both properties hold, you have two ways to think about every MST: include light cut edges, exclude heavy cycle edges. Together they give complementary ways to certify any minimum spanning tree.

23.8 A few subtleties

Remark 23.1 (Ties and uniqueness). If all edge weights are distinct, the MST is unique (the cut property, second half, forces every light edge). When weights tie, several MSTs may exist; both algorithms still return a valid one, but *which* one can depend on tie-break order. Nothing about correctness changes — all of them have the same minimum total weight.

Remark 23.2 (Disconnected graphs). We assumed G is connected. If it is not, no spanning tree exists. Both algorithms then produce a *minimum spanning forest*: Kruskal naturally yields one tree per connected component; Prim from a single seed only reaches the seed's component, so you would restart it on an unreached vertex.

Remark 23.3 (Negative weights are fine). Unlike some shortest-path algorithms, MST algorithms do not care if weights are negative. The cut property never assumed positivity — "cheapest crossing edge" makes perfect sense whether costs are $+5$ or -5. You could add a huge constant to every weight; the same tree stays minimal, since every spanning tree has the same number of edges, so all their weights shift by the same amount.

Looking back

We started this chapter laying fiber between villages and reduced the problem to a single invariant. One theorem, the cut property, says local greed is globally safe. One generic procedure repeatedly grabs a safe edge. Two famous algorithms, Kruskal and Prim, are just two ways of choosing the cut to look at. The analysis rests on two data structures you already know: union-find for Kruskal, a priority queue for Prim.

An MST minimizes total connection cost, but does not guarantee that point-to-point paths are short: two neighboring vertices may be separated by a long chain. In Chapter 24, we shift from minimum-weight trees to shortest paths, using edge relaxation, Dijkstra's algorithm, and Bellman–Ford.

23.9 Exercises

Exercise 23.1. Run both Kruskal and Prim (start Prim at vertex 1) on this graph and confirm they produce the same tree and total weight. Vertices $\{1, 2, 3, 4, 5\}$; edges 12=2, 13=3, 24=5, 34=4, 35=6, 45=1, 25=7. Show your island merges (Kruskal) and your key vector at each extraction (Prim).

Exercise 23.2. A connected graph has 100 vertices. Exactly how many edges does *any* of its spanning trees contain? Explain in one sentence why the count cannot depend on which spanning tree you pick.

Exercise 23.3. Prove that the single cheapest edge of a connected graph (assume a unique minimum-weight edge) belongs to every MST. (Hint: find a cut for which it is the light edge.)

Exercise 23.4. Cycle property. Let C be any cycle in G, and let e be a strictly-heaviest edge on C (assume it is the unique heaviest edge of C). Prove that e belongs to *no* MST. Use an exchange argument that mirrors the proof of the cut property.

Exercise 23.5. Suppose you wanted a *maximum*-weight spanning tree instead (the most expensive connected acyclic skeleton). Describe the smallest change to Kruskal's algorithm that solves it, and argue briefly why your change is correct.

Exercise 23.6. Prove that if every edge weight in G is distinct, then G has exactly one minimum spanning tree. (You may use the second half of the cut property.)

Exercise 23.7. For a complete graph on n vertices ($m = \binom{n}{2}$), compare the running time of Prim with a binary heap, $O(m \log n)$, against Prim with an unsorted array, $O(n^2)$. Which is asymptotically faster here, and by what factor? When would you still prefer the heap?

Exercise 23.8. Challenge (bottleneck spanning tree). The *bottleneck* of a spanning tree is the weight of its single heaviest edge. Prove that every minimum spanning tree is also a *minimum-bottleneck* spanning tree — that is, no spanning tree has a smaller heaviest edge. (Is the converse true? Find a small counterexample showing a minimum-bottleneck tree need not be an MST.)

Exercise 23.9. You have already computed an MST T of a graph G. Now a single new edge (u, v) with weight w is added to G. Describe an $O(n)$-time method to update the MST, and justify it using the cycle property from Exercise 23.4. (Hint: adding (u, v) to T creates one cycle.)

Exercise 23.10. In Kruskal's algorithm, why is the union-find structure the right tool rather than, say, re-running a graph search after each edge to test connectivity? State the running time Kruskal would have with the naive "search every time" test, and compare it to the union-find version.

Chapter 24

Single-Source Shortest Paths

In Chapter 23, minimum spanning trees found the cheapest global network connecting all vertices, but offered no guarantee on point-to-point travel: two nodes in an MST may be linked only through a circuitous path.

Suppose you are standing at a street corner in an unfamiliar city, holding a phone that routes you across town. In the fraction of a second before the blue path appears, software evaluates countless walking routes and returns the shortest one—not by trying every route, but by exploiting how shortest paths decompose into subpaths.

This chapter is about those properties and the algorithms built upon them. We are given a graph whose edges carry *weights* (distances, traversal times, costs) and a designated *source* vertex. We seek the cheapest path from the source to every other vertex: the *single-source shortest-paths* problem. Every algorithm in this chapter rests on a single primitive, *relaxation*, which we implement across three methods: Bellman–Ford (for general weights and negative cycle detection), DAG shortest paths (linear time on acyclic graphs), and Dijkstra's algorithm (for non-negative weights).

24.1 What is a shortest path, exactly?

We set up the notation carefully, because loose notation here causes confusion later.

We have a directed graph $G = (V, E)$ with $|V| = n$ vertices and $|E| = m$ edges. (We write n and m throughout; CLRS often writes $|V|$ and $|E|$.) Each edge $(u, v) \in E$ has a real-valued *weight* $w(u, v)$. A *path* from u to v is a sequence of vertices $\langle v_0, v_1, \ldots, v_k \rangle$ with $v_0 = u$, $v_k = v$, and every consecutive pair (v_{i-1}, v_i) an edge of G. The *weight* of that path is just the sum of its edge weights:

$$w(p) \;=\; \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Definition 24.1 (Shortest-path weight)**.** The *shortest-path weight* from u to v, written $\delta(u, v)$, is

$$\delta(u, v) = \begin{cases} \min\{\, w(p) : p \text{ is a path from } u \text{ to } v \,\} & \text{if such a path exists,} \\ \infty & \text{otherwise.} \end{cases}$$

A *shortest path* from u to v is any path p with $w(p) = \delta(u, v)$.

Notice the word *any*. There can be many shortest paths tied for the minimum, and that is fine; we want one of them (and its weight). And since δ is a *minimum*, a minimum over an infinite set of paths only makes sense if that set is bounded below. Negative-weight cycles break exactly this condition.

Intuition

Picture the graph as a road network. The vertices are intersections, the directed edges are one-way street segments, and the weight on an edge is how many minutes that segment takes. A path is a legal driving route; its weight is the total driving time. $\delta(u, v)$ is the fastest possible time to get from u to v. Our whole job, given a starting intersection s, is to fill in a little table: "fastest time from s to here" for every intersection.

24.1.1 Why negative edges work and negative cycles break shortest paths

Weights are not always distances, and they are not always positive. If an edge weight is a financial gain or loss, or an elevation change, it can be negative. Negative *edges* are perfectly fine and meaningful. The trouble is negative *cycles*.

Suppose there is a cycle – a path that returns to where it started – whose total weight is negative. Then for any vertex on that cycle, you can lower your path weight by looping around once more, and again, and again, forever. The minimum in Definition 24.1 is then $-\infty$: there is no shortest path. We say $\delta(u, v) = -\infty$ for any v reachable from such a cycle (reachable from u).

Common trap

A negative *edge* is not a problem. A negative-weight *cycle that is reachable from the source* is. The two get confused constantly. Dijkstra's algorithm forbids negative edges entirely (we will see precisely why). Bellman–Ford tolerates negative edges and, as a bonus, *detects* negative cycles and tells you they exist instead of silently returning garbage.

There is also the gentler cousin: a *positive* or *zero*-weight cycle. Those never help – looping around a cycle of nonnegative weight can only keep your weight the same or make it worse. So we can always assume a shortest path has *no repeated vertices*: it is a *simple* path, with at most n vertices and hence at most $n-1$ edges. Keep that bound of $n-1$ edges in mind – it is the exact number of passes Bellman–Ford will need.

24.2 The two structural facts everything rests on

Before any algorithm, we extract the facts that make shortest paths tractable. There are two, and both are intuitive.

24.2.1 Optimal substructure: shortest paths contain shortest paths

Intuition

Consider the natural substructure of optimal routing: suppose the fastest route from your home to the airport happens to pass through a particular gas station. Then the piece of that route from home to the gas station must itself be the fastest route from home to the gas station. Why? Because if there were a faster way to reach the gas station, you could splice it in and beat your supposedly-fastest airport route – contradiction. *Subpaths of shortest paths are shortest paths.*

Lemma 24.1 (Optimal substructure)**.** *Let $p = \langle v_0, v_1, \ldots, v_k \rangle$ be a shortest path from v_0 to v_k. For any $0 \le i \le j \le k$, the subpath $p_{ij} = \langle v_i, \ldots, v_j \rangle$ is a shortest path from v_i to v_j.*

Formal

Proof (cut-and-paste). Decompose p into three pieces: $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$, so $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk})$. Suppose, for contradiction, that p_{ij} is *not* a shortest path from v_i to v_j; then there is a path p'_{ij} from v_i to v_j with $w(p'_{ij}) < w(p_{ij})$. Replace p_{ij} by p'_{ij} inside p to get a new v_0-to-v_k path of weight $w(p_{0i}) + w(p'_{ij}) + w(p_{jk}) < w(p)$. That contradicts p being shortest. Hence p_{ij} is shortest. □

This optimal substructure is exactly what licenses dynamic programming and greedy strategies on shortest paths. It is the same shape of argument you saw in Chapter 15.

24.2.2 The triangle inequality

The second fact is the shortest-path version of "a detour is never shorter than going straight."

Lemma 24.2 (Triangle inequality)**.** *For any edge $(u, v) \in E$ and any source s,*

$$\delta(s, v) \;\le\; \delta(s, u) + w(u, v).$$

Formal

Proof. One particular way to get from s to v is: take a shortest path from s to u (weight $\delta(s, u)$), then traverse the single edge (u, v) (weight $w(u, v)$). This is *a* path to v, of weight $\delta(s, u) + w(u, v)$. The shortest-path weight $\delta(s, v)$ is the minimum over *all* paths to v, so it cannot exceed this particular one. (If $\delta(s, u) = \infty$ the inequality holds directly; if a negative cycle makes $\delta(s, u) = -\infty$, then $\delta(s, v) = -\infty$ too and again it holds.) □

Focus

Two facts. (1) **Optimal substructure**: chop a shortest path anywhere and each piece is itself shortest. (2) **Triangle inequality**: $\delta(s, v) \le \delta(s, u) + w(u, v)$ for every edge. The first tells us shortest paths can be *built up* from smaller shortest paths. The second is the yardstick our algorithms will constantly check estimates against. Everything else in this chapter is bookkeeping on top of these two.

24.3 Relaxation: the shared primitive

Every algorithm in this chapter – Bellman–Ford, the DAG algorithm, Dijkstra – is the same single operation applied to edges, over and over, in different orders. That operation is called *relaxation*. Understand it, and the three algorithms become three ways of scheduling it.

We keep two arrays. For each vertex v:

- $d[v]$ – our current *estimate* of $\delta(s, v)$. It starts at ∞ ("no idea, assume unreachable") and only ever goes *down*. It is always an *upper bound*: $d[v] \geq \delta(s, v)$ at all times.
- $\pi[v]$ – the *predecessor* of v on the best path we have found so far. Following π pointers backward from v reconstructs the path. Starts at NIL.

Initialization sets everyone to "unknown" except the source, which is zero distance from itself.

Algorithm 108: INITIALIZE-SINGLE-SOURCE(G, s)

Input: Graph $G = (V, E)$, source vertex s.
Output: Arrays d and π initialized.

```
1 for each vertex v ∈ V do
2 |   d[v] ← ∞;
3 |   π[v] ← NIL;
4 d[s] ← 0;
```

To *relax* an edge (u, v) is to ask one question: "Can I reach v more cheaply by going through u?" If yes, we improve the estimate.

Algorithm 109: RELAX(u, v, w)

Input: Edge (u, v) with weight $w(u, v)$; current d, π.
Output: $d[v]$ and $\pi[v]$ possibly improved.

```
1 if d[v] > d[u] + w(u, v) then
2 |   d[v] ← d[u] + w(u, v) ;          // found a cheaper route to v via u
3 |   π[v] ← u;
```

Intuition

Relaxation is a rumor spreading through the graph. Vertex u currently believes it can be reached from s in $d[u]$ time. It turns to its neighbor v and says, "I can get to you in $d[u] + w(u, v)$. Is that better than your current best?" If so, v updates its belief and remembers that the good news came from u. The word "relax" comes from physics/optimization: we are loosening an over-tight upper bound, letting it slip down toward the truth.

Why does this single move suffice? Because of the triangle inequality. The true distances satisfy $\delta(s, v) \leq \delta(s, u) + w(u, v)$ for every edge. An algorithm has finished only when its estimates likewise satisfy $d[v] \leq d[u] + w(u, v)$ for *every* edge – when no edge can be relaxed any further. At that point, as we will prove, the estimates are correct. So every algorithm here does the same thing: relax edges, in some order, until none can be improved.

Two small lemmas make this precise. Every correctness proof that follows uses them.

Lemma 24.3 (Relaxation never undershoots the truth)**.** *If we run* INITIALIZE-SINGLE-SOURCE *and*

then perform any sequence of relaxation steps, then $d[v] \geq \delta(s, v)$ holds for every vertex v at all times; once $d[v]$ reaches $\delta(s, v)$, it never changes again.

Formal

Proof sketch (induction on the number of relaxations). Initially $d[s] = 0 = \delta(s, s)$ and $d[v] = \infty \geq \delta(s, v)$ for $v \neq s$. Now suppose the invariant holds and we relax (u, v), possibly setting $d[v] = d[u] + w(u, v)$. By the inductive hypothesis $d[u] \geq \delta(s, u)$, so $d[u] + w(u, v) \geq \delta(s, u) + w(u, v) \geq \delta(s, v)$, the last step by the triangle inequality (Lemma 24.2). Thus the new $d[v]$ is still $\geq \delta(s, v)$. Since $d[v]$ can never drop below $\delta(s, v)$ and relax only ever decreases it, once it equals $\delta(s, v)$ it is stuck there. □

Lemma 24.4 (Convergence)**.** *Let $s \rightsquigarrow u \to v$ be a shortest path to v, so that $\delta(s, v) = \delta(s, u) + w(u, v)$. Suppose at some moment $d[u] = \delta(s, u)$, and at some later moment we relax the edge (u, v). Then from that point on $d[v] = \delta(s, v)$.*

Formal

Proof. By Lemma 24.3, once $d[u] = \delta(s, u)$ it stays there, so at the moment we relax (u, v) we have $d[u] = \delta(s, u)$. Relaxation guarantees afterward $d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$. But also $d[v] \geq \delta(s, v)$ always. Hence $d[v] = \delta(s, v)$, and it cannot change afterward. □

Every algorithm in this chapter relies on the convergence lemma. It says: *if you relax the edges of a shortest path in order, the estimate at the end of that path becomes correct.* Each algorithm is just a strategy for making sure the edges of every shortest path get relaxed in the right order.

Walking question

Before reading on, take a walk with this question. The convergence lemma needs the edges of a shortest path relaxed *in path order* – $d[u]$ correct first, *then* relax (u, v). If you relax edges in a bad order, you might relax (u, v) while $d[u]$ is still too big, miss the improvement, and never revisit it. How could you guarantee good order? Two ideas: relax *everything* many times so order stops mattering (Bellman–Ford), or process vertices in an order where d-values are correct by the time you use them (topological order for DAGs, increasing-distance order for Dijkstra). The rest of the chapter develops those two ideas.

24.4 Bellman–Ford: relax everything, repeatedly

We start with the simplest idea. If we are not clever enough to figure out a good relaxation order, we can be *thorough* instead: relax *every* edge, then do it again, and again, enough times that no order could have mattered. How many times is enough? Exactly $n - 1$, and we can see why from the convergence lemma.

A shortest (simple) path has at most $n-1$ edges. Walk along such a path $s = v_0 \to v_1 \to \cdots \to v_k = v$ with $k \leq n - 1$. After the first full pass over all edges, the edge (v_0, v_1) certainly got relaxed at some point, and since $d[v_0] = \delta(s, v_0) = 0$ from the start, the convergence lemma makes $d[v_1] = \delta(s, v_1)$. After the second pass, (v_1, v_2) got relaxed with $d[v_1]$ already correct, so $d[v_2]$ becomes correct. In

general, after pass i, the first i vertices on the path have correct d-values. So after $k \le n-1$ passes, $d[v] = \delta(s, v)$. Since this holds for every vertex's shortest path simultaneously, $n-1$ passes settle everyone.

Algorithm 110: BELLMAN-FORD(G, w, s)

Input: Weighted digraph $G = (V, E)$ with weights w (may be negative), source s.
Output: TRUE with correct d, π if no negative cycle is reachable; else FALSE.

```
1 INITIALIZE-SINGLE-SOURCE(G, s);
2 for i ← 1 to n − 1 do
3 |   for each edge (u, v) ∈ E do
4 |   |   RELAX(u, v, w);
5 for each edge (u, v) ∈ E do
6 |   if d[v] > d[u] + w(u, v) then
7 |   |   return FALSE ;                  // an edge still improves:  negative cycle!
8 return TRUE;
```

24.4.1 The negative-cycle detector, explained

Look at that final loop. After $n-1$ full passes, if the graph had no reachable negative cycle, *every* edge would satisfy $d[v] \le d[u] + w(u, v)$ – nothing left to relax. So if even one edge still says "I can improve you," $d[v] > d[u] + w(u, v)$, the only explanation is a negative cycle: the relaxations never converged because there is no finite minimum. The check is a single extra pass, and it turns a fragile algorithm into a robust one that *tells you when the question has no answer.*

Formal

Why the detector is correct. Suppose a negative-weight cycle $c = \langle v_0, v_1, \ldots, v_k = v_0 \rangle$ reachable from s existed and the algorithm wrongly returned TRUE. Returning TRUE means every edge on c passed the test: $d[v_i] \le d[v_{i-1}] + w(v_{i-1}, v_i)$ for $i = 1, \ldots, k$. Sum all k of these inequalities:

$$\sum_{i=1}^{k} d[v_i] \;\le\; \sum_{i=1}^{k} d[v_{i-1}] \;+\; \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Because c is a cycle, the vertices on the left and the vertices in the first sum on the right are the same multiset, so those two sums are equal and cancel. We are left with $0 \le \sum_{i=1}^{k} w(v_{i-1}, v_i) = w(c)$. But c is a negative cycle, $w(c) < 0$, a contradiction. So a reachable negative cycle always trips the detector. □

24.4.2 A complete numeric trace

Let us run Bellman–Ford on a small graph with a negative edge but *no* negative cycle, so we can watch relaxation grind its way to the answer. The vertices are s, t, x, y, z and the edges (with weights) are:

$$(s,t) = 6,\ (s,y) = 7,\ (t,x) = 5,\ (t,y) = 8,\ (t,z) = -4,$$
$$(x,t) = -2,\ (y,x) = -3,\ (y,z) = 9,\ (z,s) = 2,\ (z,x) = 7.$$

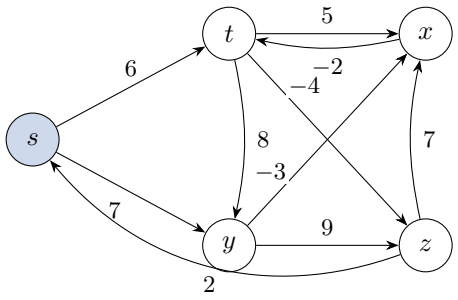


Figure 24.1: The Bellman–Ford example graph. Source s is shaded. Note the negative weights on (t, z), (x, t), (y, x), but no negative cycle.

We fix the edge-relaxation order once and reuse it every pass:

$$(t, x),\ (t, y),\ (t, z),\ (x, t),\ (y, x),\ (y, z),\ (z, x),\ (z, s),\ (s, t),\ (s, y).$$

Start with $d[s] = 0$ and everyone else ∞.

Algorithm lab

Trace each pass with me. In a pass, walk the edge list above and apply RELAX. An edge does nothing if its tail u still has $d[u] = \infty$.

Pass 1. Most edges have an infinite tail and do nothing. The two edges out of s fire: (s, t) sets $d[t] = 0 + 6 = 6$; (s, y) sets $d[y] = 0 + 7 = 7$. Everyone else is still ∞.

$$d = (s{:}0,\ t{:}6,\ x{:}\infty,\ y{:}7,\ z{:}\infty).$$

Pass 2. Now t and y are finite. (t, x): $d[x] = 6 + 5 = 11$. (t, z): $d[z] = 6 + (-4) = 2$. (y, x): $7 + (-3) = 4 < 11$, so $d[x] = 4$. (y, z): $7 + 9 = 16$, not better than 2. After the pass:

$$d = (s{:}0,\ t{:}6,\ x{:}4,\ y{:}7,\ z{:}2).$$

Pass 3. (x, t): $4 + (-2) = 2 < 6$, so $d[t] = 2$. Other edges find no improvement this pass.

$$d = (s{:}0,\ t{:}2,\ x{:}4,\ y{:}7,\ z{:}2).$$

Pass 4. With $d[t] = 2$ now, (t, z): $2 + (-4) = -2 < 2$, so $d[z] = -2$. Nothing else improves.

$$d = (s{:}0,\ t{:}2,\ x{:}4,\ y{:}7,\ z{:}-2).$$

That is $n - 1 = 4$ passes for $n = 5$ vertices. The final pass over all edges finds *no* improvable edge, so the detector returns TRUE. These are the true shortest-path weights: $\delta(s, t) = 2$, $\delta(s, x) = 4$, $\delta(s, y) = 7$, $\delta(s, z) = -2$.

Following π pointers, the shortest path to z is $s \to y \to x \to t \to z$ with weight $7 - 3 - 2 - 4 = -2$. Note that the *shortest* route to z goes the "long way" through four edges, exploiting the negative weights. A greedy nearest-first strategy would have grabbed $s \to t \to z$ for weight 2 and missed it. This is exactly why Dijkstra cannot handle negative edges, a point we return to once we meet it.

In this particular example the final predecessor pointers happen to line up into a single chain: $\pi[y] = s$, $\pi[x] = y$, $\pi[t] = x$, $\pi[z] = t$. That chain *is* the shortest-paths tree (Figure 24.2); reading it backward from any vertex recovers that vertex's shortest path from s.

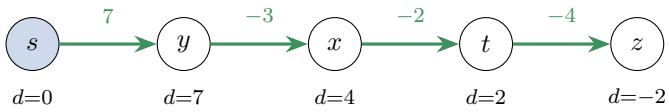


Figure 24.2: The shortest-paths tree the π pointers encode after Bellman–Ford finishes on Figure 24.1. Here it degenerates to a single path; following the arrows from s gives each vertex its δ-value (shown below).

Cost summary

Bellman–Ford. The main loop runs $n - 1$ passes, each relaxing all m edges in $\Theta(m)$ time, plus one detector pass. Total:

$$\Theta\big((n-1)\cdot m + m\big) = \Theta(nm).$$

With m up to $\Theta(n^2)$ in a dense graph, that is $\Theta(n^3)$ in the worst case. Space is $\Theta(n)$ for the d and π arrays. It is slow, but it is the price of handling negative edges and detecting negative cycles.

Common trap

A common implementation bug: people "optimize" by stopping early when a pass makes no changes (a fine and correct optimization) but then *forget the detector pass*, or they run only $n - 2$ passes off a fencepost error. If the longest shortest path has exactly $n - 1$ edges, $n - 2$ passes leave one vertex wrong. Always $n - 1$ relaxation passes, then one detector pass. Count carefully.

24.5 Shortest paths in a DAG: order is free

Now a special case where we can be genuinely clever about order. Suppose the graph is a *directed acyclic graph* (DAG) – no cycles at all. Then there are no negative cycles to worry about (no cycles, period), and we have a useful extra tool: a *topological order*.

Recall from Chapter 22 that a topological sort lists the vertices so that every edge (u, v) goes from an earlier vertex to a later one. In a DAG such an ordering always exists and is computable in $\Theta(n+m)$ time, giving a relaxation order that satisfies the convergence lemma in a single left-to-right pass.

Intuition

Imagine the vertices lined up left to right in topological order, every edge pointing rightward. Process them left to right. When you arrive at a vertex u, *every* vertex that could possibly feed into u – everything that could lower $d[u]$ – sits to its left and has already been processed. So $d[u]$ is already final the moment you reach u. You relax u's outgoing edges once, push

its now-correct estimate rightward, and never look back. A single left-to-right sweep suffices, with no repeated passes and no priority queue (Figure 24.3).

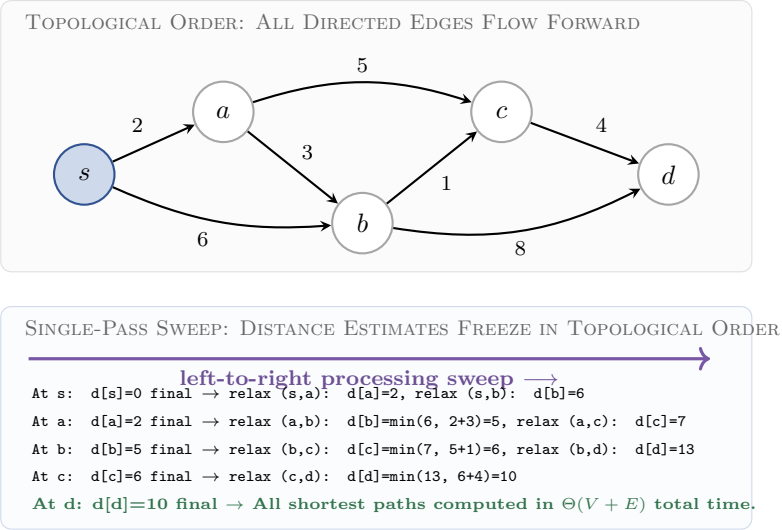


Figure 24.3: A directed acyclic graph processed in topological order. Top: every edge points strictly rightward, ensuring that when the sweep reaches vertex u, all potential predecessors have already transmitted their final distances. Bottom: step-by-step relaxation timeline showing how a single linear sweep locks in optimal distances without any priority queue overhead.

Algorithm 111: DAG-Shortest-Paths(G, w, s)

Input: Weighted DAG $G = (V, E)$, weights w (any sign), source s.
Output: Correct d and π.

```
1 topologically sort the vertices of G;
2 Initialize-Single-Source(G, s);
3 for each vertex u in topological order do
4     for each vertex v in Adj[u] do
5         Relax(u, v, w);
```

Formal

Why it is correct. We claim $d[u] = \delta(s, u)$ for every u, by induction along the topological order. Consider a shortest path $s = v_0 \to v_1 \to \cdots \to v_k = u$. Because every edge goes forward in topological order, the vertices $v_0, v_1, \ldots, v_k$ appear in increasing topological position. The algorithm processes them in that very order, and when it processes v_{i-1} it relaxes (v_{i-1}, v_i). By induction $d[v_{i-1}]$ is already $\delta(s, v_{i-1})$ at that moment, so by the convergence lemma (Lemma 24.4) $d[v_i]$ becomes $\delta(s, v_i)$. Marching i from 1 to k gives $d[u] = \delta(s, u)$. □

Cost summary

DAG shortest paths. Topological sort is $\Theta(n + m)$. The main loop touches each vertex once and each edge exactly once (every edge is relaxed a single time), which is again $\Theta(n+m)$.

Total: $\Theta(n + m)$ – *linear*. This is the fastest single-source algorithm in the chapter, and it works with negative edges, because a DAG can never contain a negative cycle.

Why this matters

DAG shortest paths is the basis of *critical-path analysis* in project scheduling (PERT charts). Tasks are vertices, dependencies are edges, durations are weights, and the *longest* path – found by negating weights and running this same algorithm – is the minimum time to finish the whole project. The same linear sweep also underlies many dynamic programs: dynamic programming over an acyclic graph of subproblems evaluates the same recurrence in topological order, which is why Chapter 15's tables and this chapter's relaxations have the exact same structure.

24.6 Dijkstra's algorithm: greedy on nonnegative weights

Now the famous one. Edsger Dijkstra's algorithm assumes *all edge weights are nonnegative* and, in return, is fast and elegant. It does not need a DAG, and it does not relax every edge $n - 1$ times. Instead it discovers the correct order on the fly using a greedy rule.

Intuition

Think of a wavefront expanding outward from the source at speeds inversely proportional to edge weights. Each vertex has a current best-known time $d[v]$. Maintain a set S of vertices whose true distance is already *finalized*. Repeatedly do this: among all vertices *not* yet finalized, pick the one with the smallest current estimate $d[u]$ – call it the "closest frontier vertex" – declare its estimate final, add it to S, and relax all its outgoing edges to update its neighbors. Repeat until every vertex is finalized. The closest unfinished vertex is always safe to finalize, because (with nonnegative weights) no cheaper route to it could possibly exist through a vertex that is even farther away.

That last sentence is the algorithm's key claim, and it is exactly where nonnegativity is needed. We prove it below.

Algorithm 112: DIJKSTRA(G, w, s)

Input: Weighted digraph $G = (V, E)$ with $w(u, v) \geq 0$ for all edges, source s.
Output: Correct d and π.

```
1 INITIALIZE-SINGLE-SOURCE(G, s);
2 S ← ∅;
3 Q ← V ;                                   // min-priority queue keyed on d
4 while Q ≠ ∅ do
5     u ← EXTRACT-MIN(Q) ;                  // closest unfinalized vertex
6     S ← S ∪ {u};
7     for each vertex v ∈ Adj[u] do
8         RELAX(u, v, w) ;                  // may call DECREASE-KEY on v
```

Each vertex is extracted from Q exactly once, and at the moment of extraction we claim its estimate is already the true distance. The greedy choice – always pull the smallest – is what makes the order

come out right.

24.6.1 A complete numeric trace

Let us run Dijkstra on a small *nonnegative* graph. Vertices s, t, x, y, z, edges:

$$(s,t) = 10,\ (s,y) = 5,\ (t,x) = 1,\ (t,y) = 2,$$
$$(y,t) = 3,\ (y,x) = 9,\ (y,z) = 2,\ (x,z) = 4,\ (z,x) = 6,\ (z,s) = 7.$$

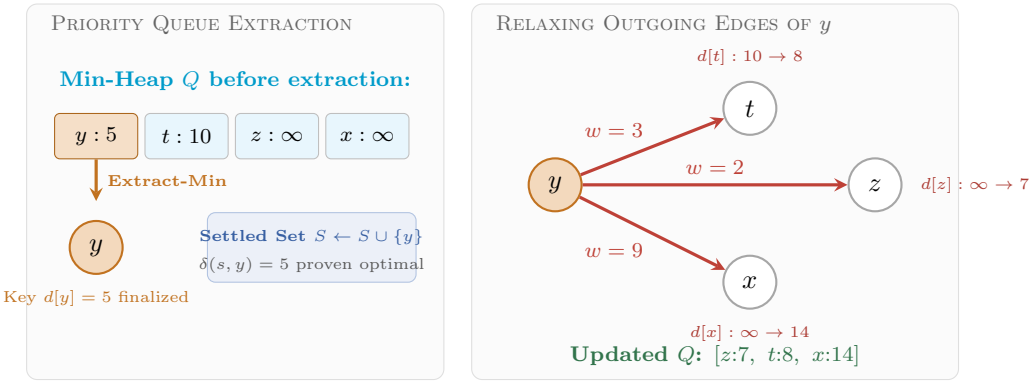


Figure 24.4: Dijkstra priority queue extraction versus outgoing edge relaxation. Left: the min-priority queue Q yields the vertex with smallest tentative distance estimate, y with $d[y] = 5$, which moves into the settled set S. Right: the outgoing edges of y fire relaxation updates to its neighbors, triggering Decrease-Key operations in Q for t (lowered to 8), z (lowered to 7), and x (lowered to 14).

Algorithm lab

Initialize: $d[s] = 0$, rest ∞; $S = \emptyset$. At each step I pick the non-finalized vertex of smallest d, finalize it, and relax its out-edges.

Step 1. Smallest d is s ($d = 0$). Finalize s. Relax: (s,t) gives $d[t] = 10$; (s,y) gives $d[y] = 5$.

$$S = \{s\}, \quad d = (t{:}10,\ y{:}5,\ x{:}\infty,\ z{:}\infty).$$

Step 2. Smallest non-final is y ($d = 5$). Finalize y. Relax: (y,t): $5 + 3 = 8 < 10$, so $d[t] = 8$. (y,x): $5 + 9 = 14$, so $d[x] = 14$. (y,z): $5 + 2 = 7$, so $d[z] = 7$.

$$S = \{s, y\}, \quad d = (t{:}8,\ x{:}14,\ z{:}7).$$

Step 3. Smallest non-final is z ($d = 7$). Finalize z. Relax: (z,x): $7 + 6 = 13 < 14$, so $d[x] = 13$. (z,s): $7 + 7 = 14$, but s is finalized at 0 – no change.

$$S = \{s, y, z\}, \quad d = (t{:}8,\ x{:}13).$$

Step 4. Smallest non-final is t ($d = 8$). Finalize t. Relax: (t,x): $8 + 1 = 9 < 13$, so $d[x] = 9$. (t,y): y already finalized – no change.

$$S = \{s, y, z, t\}, \quad d = (x{:}9).$$

Step 5. Only x remains ($d = 9$). Finalize it. Done.

$$\delta(s,s) = 0,\ \delta(s,y) = 5,\ \delta(s,z) = 7,\ \delta(s,t) = 8,\ \delta(s,x) = 9.$$

Notice the finalization order: s, y, z, t, x – exactly increasing d-value, $0, 5, 7, 8, 9$. That monotonicity holds in general. Dijkstra finalizes vertices in nondecreasing order of true distance. That monotonic extraction order is what makes the correctness proof work.

24.6.2 Why Dijkstra is correct

Theorem 24.1 (Dijkstra's correctness)**.** *Run on a graph with all nonnegative edge weights,* DIJKSTRA *terminates with $d[u] = \delta(s,u)$ for every vertex u.*

The key is the invariant that the vertex we extract is already correct. The picture to hold in mind is the cut between the finalized set S and the rest (Figure 24.5): a shortest path to the soon-to-be-extracted u must cross that cut somewhere, and we zero in on the very first crossing edge.

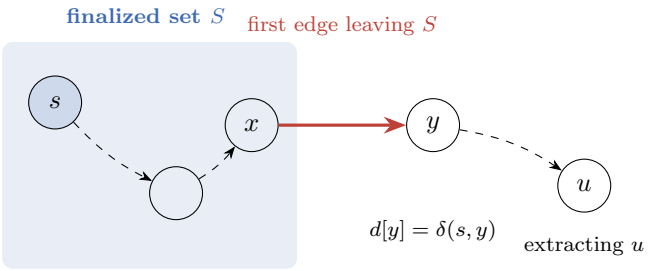


Figure 24.5: The cut argument in Dijkstra's proof. A shortest path from s (inside S) to u (outside S) crosses the boundary; (x, y) is its first crossing edge. We show $d[y]$ is already correct and $d[y] \le d[u]$, forcing $d[u] = \delta(s,u)$.

Formal

Proof. We show: *at the moment a vertex u is extracted (added to S), $d[u] = \delta(s,u)$.* Since d never changes after extraction (Lemma 24.3, $d[u]$ already at its floor), this gives the theorem. Suppose not. Let u be the *first* vertex extracted with $d[u] \neq \delta(s,u)$; by Lemma 24.3 this means $d[u] > \delta(s,u)$, so in particular $u \neq s$ (since s is extracted first with $d[s] = 0 = \delta(s,s)$) and u is reachable (else $\delta = \infty = d$).

Consider a shortest path p from s to u. It starts inside S (at s) and ends outside S (at u, the vertex we are about to add). Walk along p and let (x, y) be the *first* edge that leaves S: $x \in S$, $y \notin S$, with y the first off-S vertex on p (possibly $y = u$).

Because x was extracted before u and u is the *first* bad extraction, x was correct when extracted: $d[x] = \delta(s,x)$. When x was extracted we relaxed the edge (x, y), so by the convergence lemma (Lemma 24.4) $d[y] = \delta(s,y)$ now.

Here is the key inequality, and the exact step where nonnegativity is required. Since y lies on

a shortest path to u and all weights from y onward to u are *nonnegative*, the subpath from y to u has weight ≥ 0, so $\delta(s, y) \leq \delta(s, u)$. Therefore

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u],$$

the last step because $d[u] \geq \delta(s, u)$ always. But u was chosen by EXTRACT-MIN as having the *smallest* d among non-S vertices, and y is also outside S, so $d[u] \leq d[y]$. Squeezing, $d[y] = d[u] = \delta(s, u)$ – which says $d[u] = \delta(s, u)$, contradicting our assumption that u was bad. No bad extraction exists. □

Common trap

Examine the step "$\delta(s, y) \leq \delta(s, u)$ because the subpath from y to u has nonnegative weight." If a *negative* edge lay on that subpath, the inequality could flip: a vertex y with a larger current estimate might still reach u for less total weight along a negative-weight edge. Then finalizing the smallest d is wrong, and Dijkstra returns incorrect answers *silently*. This is the precise, single reason Dijkstra requires nonnegative weights. Our earlier Bellman–Ford trace, where the best path to z snaked through negatives, is a concrete graph on which Dijkstra would have failed.

24.6.3 The role of the priority queue

Dijkstra's skeleton – "repeatedly grab the closest unfinalized vertex, finalize it, relax its edges" – demands a data structure that supports three operations: build the queue, EXTRACT-MIN (remove the smallest-d vertex), and DECREASE-KEY (lower a vertex's d when a relaxation improves it). The *entire* running time of Dijkstra is determined by how fast that priority queue is. The algorithm performs exactly n EXTRACT-MIN operations (one per vertex) and at most m DECREASE-KEY operations (at most one per edge). So the cost is

$$n \cdot T_{\text{extract}} \; + \; m \cdot T_{\text{decrease}} \; + \; T_{\text{build}}.$$

Different queues give different bounds:

- **Unsorted array / linear scan.** EXTRACT-MIN scans all remaining vertices: $\Theta(n)$ each. DECREASE-KEY is $\Theta(1)$ (just write the value). Total $\Theta(n^2 + m) = \Theta(n^2)$. Best for *dense* graphs where $m \approx n^2$.

- **Binary min-heap** (Chapter 6). EXTRACT-MIN and DECREASE-KEY are each $\Theta(\log n)$. Total $\Theta((n + m) \log n) = \Theta(m \log n)$ for a connected graph. Best for *sparse* graphs where $m = O(n^2/\log n)$. This is the version most libraries ship.

- **Fibonacci heap** (Chapter 20). EXTRACT-MIN is $O(\log n)$ amortized, but DECREASE-KEY is $O(1)$ amortized. Total $\Theta(n \log n + m)$, the theoretical best. The constants are large, so it wins only on very large, edge-heavy graphs.

Cost summary

Dijkstra, by priority queue.

array	$\Theta(n^2)$	dense graphs
binary heap	$\Theta(m \log n)$	sparse graphs, the practical default
Fibonacci heap	$\Theta(n \log n + m)$	asymptotically optimal

Across all three the count of operations is identical – n extractions and $\leq m$ decrease-keys – only the per-operation cost differs. This is a textbook example of an algorithm whose performance is dictated by its data structure, not its control flow.

History and connection

The single-source shortest path problem was solved in the late 1950s by several researchers working independently. In 1956, Edsger W. Dijkstra designed his greedy exploration algorithm while having coffee at a terrace in Amsterdam, aiming to showcase the newly built ARMAC computer by calculating the shortest road route between 64 Dutch cities. Dijkstra published the three-page paper in 1959 without a single code listing, presenting it as an exercise in minimal calculation. Independently, the dynamic programming and relaxation framework for general edge weights was developed by Alfonso Shimbel (1955), Lester R. Ford Jr. (1956), Richard Bellman (1958), and Edward F. Moore (1959).

A long-standing computational tension arose between edge weights and algorithmic efficiency: Dijkstra's greedy priority queue achieves near-linear $O(m + n \log n)$ time but fails if even a single edge is negative, whereas Bellman–Ford tolerates negative edges and detects negative cycles at the steep cost of $O(nm)$ time. For over six decades, researchers wondered whether negative weights fundamentally required quadratic time or if near-linear search was possible.

Shortest-path algorithms are widely used in networking and finance: Dijkstra became the routing engine for OSPF (Open Shortest Path First) and vehicle navigation systems, while Bellman–Ford became the routing information protocol (RIP) and the mathematical detector for currency arbitrage loops in financial markets. In 2022, Aaron Bernstein, Danupon Nanongkai, and Christian Wulff-Nilsen proved that negative-weight single-source shortest paths can be solved in near-linear $O(m^{1+o(1)} \log W)$ time, breaking the $O(nm)$ barrier. The remaining open questions center on whether deterministic near-linear negative-weight algorithms exist and how to maintain shortest-path trees efficiently under continuous dynamic edge updates. In CLRS (4th edition Chapter 22; 3rd edition Chapter 24), the algorithms are unified under the same sequence of relaxations.

24.6.4 The predecessor graph: reconstructing the actual path

We have been computing *weights* $d[v]$, but the user usually wants the *route*. That is what π is for. After any of these algorithms finishes, the π pointers form a *shortest-paths tree* rooted at s: follow π backward from any vertex v and you walk a shortest path from v to s in reverse. Reversing that list gives the path from s to v.

Algorithm 113: PRINT-PATH(G, s, v)

Input: Source s, target v, predecessor array π.
Output: The vertices of a shortest $s \to v$ path, in order.

```
1 if v = s then
2     print s;
3 else if π[v] = NIL then
4     print "no path from s to v";
5 else
6     PRINT-PATH(G, s, π[v]);
7     print v;
```

Focus

The shape of a shortest-paths tree: s is the root; each vertex v has exactly one parent $\pi[v]$; and the tree path from s to v is *a* shortest path. There may be other shortest paths of equal weight, but the tree commits to one. The relaxation steps build this tree edge by edge: every time RELAX updates $\pi[v] \leftarrow u$, it is re-parenting v under a better-positioned u.

24.7 Putting the three together

The table below summarizes the three algorithms, which all use the same relaxation operation.

Algorithm	Graph	Negative edges?	Order of relaxation	Time
Bellman–Ford	any	yes (detects neg. cycles)	all edges, $n-1$ times	$\Theta(nm)$
DAG	acyclic only	yes	topological order, once	$\Theta(n+m)$
Dijkstra	any	*no*	greedy, closest-first	$\Theta(m \log n)$

Intuition

A decision tree for choosing. *Is the graph a DAG?* If yes, use the linear DAG algorithm – nothing beats $\Theta(n+m)$. *Are there negative edges (and the graph is not a DAG)?* You must use Bellman–Ford, and you get negative-cycle detection for free. *Otherwise* – general graph, nonnegative weights, the overwhelmingly common case – use Dijkstra with a binary heap. It is faster and simpler, and it is the basis of real routing engines (which then add tricks like A* and contraction hierarchies on top).

Why this matters

These three algorithms appear in many real systems. Internet routing protocols (OSPF) run Dijkstra to compute forwarding tables. GPS navigation runs Dijkstra variants over road graphs. Currency-arbitrage detection runs Bellman–Ford: take logs of exchange rates, negate, and a reachable negative cycle is a sequence of trades that prints money. Build systems and spreadsheet recalculation run the DAG algorithm over dependency graphs. All of them use the same five-line RELAX.

Pause

Here is the chapter in one paragraph. A shortest-path estimate $d[v]$ starts at ∞ and only ever decreases. *Relaxing* an edge (u, v) checks whether routing through u beats v's current estimate, and if so improves it. Relax the edges of a shortest path in order and its endpoint's estimate becomes exactly correct. The three algorithms are three disciplines for guaranteeing good relaxation order: brute repetition (Bellman–Ford), topological order (DAG), and greedy closest-first (Dijkstra).

What happens when we need shortest paths between *every* pair of vertices, not just from a single source? In Chapter 25, we expand to all-pairs shortest paths, developing matrix-multiplication formulations, the Floyd–Warshall algorithm, and Johnson's reweighting technique.

24.8 Exercises

Exercise 24.1. Consider the graph with vertices s, a, b, c and edges $(s, a) = 4$, $(s, b) = 1$, $(b, a) = 2$, $(a, c) = 1$, $(b, c) = 5$. Initialize and then relax the edges in the order (s, a), (a, c), (b, c), (s, b), (b, a), (a, c). Show d after each relaxation. Did $d[c]$ reach $\delta(s, c)$? Explain how the order helped or hurt.

Exercise 24.2. Run Bellman–Ford by hand on the graph s, a, b, c with edges $(s, a) = 3$, $(s, b) = 5$, $(a, b) = -2$, $(b, c) = 2$, $(a, c) = 6$, $(c, a) = 1$. Use any fixed edge order, show the d-array after each of the $n - 1$ passes, and state whether the detector pass returns TRUE or FALSE. (Check first: the cycle $a \to b \to c \to a$ has weight $-2 + 2 + 1 = 1 > 0$, so there is no negative cycle and the detector should return TRUE.)

Exercise 24.3. Modify edge (c, a) in the previous exercise to weight -6. Now the cycle $a \to b \to c \to a$ becomes a negative-weight cycle reachable from s. Identify it, compute its total weight, and confirm that the Bellman–Ford detector would return FALSE (no $n - 1$ passes will settle the d-values, and the detector pass finds an edge it can still relax).

Exercise 24.4. Give a topological order for the DAG with edges $(s, a), (s, c), (a, b), (c, a), (c, d), (b, d)$ and weights $2, 4, 1, 3, 1, 2$ respectively. Run DAG-Shortest-Paths from s and report d and π for every vertex. How many times is each edge relaxed?

Exercise 24.5. Run Dijkstra from s on the graph with edges $(s, a) = 2$, $(s, b) = 6$, $(a, b) = 3$, $(a, c) = 5$, $(b, c) = 1$, $(c, d) = 4$, $(b, d) = 8$ (all nonnegative). List the order in which vertices are finalized and the final d-values. Verify that the finalization order is nondecreasing in d.

Exercise 24.6. Construct a graph with *one* negative edge and no negative cycle on which Dijkstra returns a wrong distance for some vertex. Trace Dijkstra to show the wrong answer, then run Bellman–Ford to show the right one. (Three vertices suffice.)

Exercise 24.7. Argue carefully that Dijkstra performs exactly n Extract-Min operations and at most m Decrease-Key operations, regardless of which priority queue is used. Now specialize to a d-ary heap (a heap where each node has d children), where Extract-Min costs $\Theta\big(d \frac{\log n}{\log d}\big)$ and Decrease-Key costs $\Theta\big(\frac{\log n}{\log d}\big)$. Write the total running time as a function of n, m, and d, and show that choosing $d \approx \max(2, m/n)$ minimizes it. What total time does that choice give for a dense graph with $m = \Theta(n^2)$? (Binary heap, $d = 2$, and the dense-graph optimum should both fall out of your formula.)

Exercise 24.8. You are given a DAG of tasks with positive durations on the edges and want the *longest* path (the critical path / minimum project completion time). Explain precisely how to adapt DAG-Shortest-Paths to compute longest paths, and why the same trick would *not* work for longest paths in a general graph with cycles.

Exercise 24.9. Prove the upper-bound invariant (Lemma 24.3) in full, filling in every inductive step, including the base case and the case where a relaxation does *not* change $d[v]$. Where exactly does the triangle inequality get used?

Exercise 24.10. Explain, in your own words and with a worst-case example graph, why Bellman–Ford needs $n - 1$ passes and not fewer. Construct a graph on $n = 5$ vertices where some vertex's d-value only becomes correct on the very last (fourth) pass. (Hint: a single long path $s \to v_1 \to v_2 \to v_3 \to v_4$, and pick an edge order that relaxes the path edges in reverse each pass.)

Chapter 25

All-Pairs Shortest Paths

In the previous chapter you learned how to stand at one fixed vertex – call it the *source* – and compute the shortest distance from that single source to every other vertex in a graph. Dijkstra's algorithm did this when all the edge weights were nonnegative; the Bellman–Ford algorithm did it even when some weights were negative, while also detecting when a reachable negative-weight cycle makes shortest paths undefined.

But sometimes one source is not enough. A road atlas in the back of an old travel guide does not give you distances from a single city; it gives you a big triangular table where you can look up the distance between *any* two cities at once. A router wants to know the best next hop toward every possible destination, rather than a single target. A program that evaluates how "central" each person is in a social network needs the distance between every pair of people. In all of these you want the shortest path between *every* ordered pair of vertices. That is the all-pairs shortest-paths problem, and it is the subject of this chapter.

Intuition

You already have a way to solve this: run a single-source algorithm once from each vertex. With n vertices, run Dijkstra n times, or Bellman–Ford n times. That genuinely works, and for some graphs it is the best thing to do. But running n separate searches feels wasteful – each run rediscovers structure the others also needed. The new idea in this chapter is to drop "search from a source" entirely and instead think about the answer as one big table that we refine, all n^2 entries at once, in a handful of sweeps. That table-refinement view turns out to be pure dynamic programming, and it produces two clean dynamic programs: the matrix-multiplication method and the Floyd–Warshall algorithm.

Focus

Hold on to three numbers as you read. The input is a graph with n vertices and m edges. The output is an $n \times n$ matrix D where $D[i][j]$ is the weight of a shortest path from i to j. The repeated-Dijkstra baseline costs $O(nm \log n)$ (or $O(nm + n^2 \log n)$ with a Fibonacci heap); Floyd–Warshall costs a clean $\Theta(n^3)$ no matter what; and Johnson's algorithm gets you down to roughly $O(nm \log n)$ *even with negative edges*, which makes it the best choice for sparse graphs.

25.1 Setting Up: Inputs, Outputs, and Conventions

Let us fix notation once and reuse it for the whole chapter. We are given a directed, weighted graph $G = (V, E)$ with $V = \{1, 2, \ldots, n\}$. Each edge $(i, j) \in E$ carries a real weight $w(i, j)$, which may be negative. We will represent the graph as an $n \times n$ *weight matrix* W, defined by

$$W[i][j] = \begin{cases} 0 & \text{if } i = j, \\ w(i,j) & \text{if } i \neq j \text{ and } (i,j) \in E, \\ \infty & \text{if } i \neq j \text{ and } (i,j) \notin E. \end{cases}$$

The diagonal is 0 because the empty path from a vertex to itself has weight zero (assuming no negative-weight cycles, which we discuss below). A missing edge is ∞ because you cannot get there in one hop. Every algorithm in this chapter starts from this matrix.

The output is the *distance matrix* D, where $D[i][j] = \delta(i, j)$, the weight of a shortest path from i to j, with the usual convention that $\delta(i, j) = \infty$ if j is unreachable from i.

Common trap

A negative-weight *cycle* reachable on a path from i to j makes $\delta(i, j) = -\infty$: you can loop the cycle forever, shaving off weight each time, so there is no shortest path at all. Negative edges by themselves are fine; negative *cycles* make shortest paths undefined. Every algorithm in this chapter assumes there is no negative-weight cycle, and we will note how each one can be made to *detect* one if it is present.

We will also want a second output: in addition to the distances, we reconstruct the actual paths. For that we keep a *predecessor matrix* Π, where $\Pi[i][j]$ is the vertex that comes just before j on a shortest path from i to j (or NIL if no path exists or if $i = j$). Reading off the path then means walking backwards: from j to $\Pi[i][j]$ to $\Pi[i][\Pi[i][j]]$ and so on until you reach i. The i-th row of Π is exactly a shortest-paths tree rooted at i, the same object Dijkstra and Bellman–Ford produced.

Intuition

Think of the whole chapter as maintaining two matrices in lockstep. D holds the best distances we currently believe; Π holds the "how did I get here?" breadcrumbs. Every time we improve a D entry we also update its Π entry to remember the move that made the improvement. If you ever lose the thread, ask: *what does $D[i][j]$ mean right now, and what does $\Pi[i][j]$ point at?*

25.2 First Idea: Shortest Paths as Matrix "Multiplication"

We will warm up with an algorithm that is a little slower than what we ultimately want, but that shows the dynamic-programming structure clearly. The idea is to organize shortest paths by *how many edges they use*.

25.2.1 The subproblem: shortest paths with a bounded number of edges

Let $\ell_{ij}^{(r)}$ denote the minimum weight of any path from i to j that uses *at most* r edges. This is our subproblem. The base case is the zero-edge case:

$$\ell_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ \infty & \text{if } i \neq j. \end{cases}$$

With zero edges you can only stay put. Now the recursive step. Suppose we already know $\ell_{ij}^{(r-1)}$ for all pairs – the best you can do with at most $r-1$ edges. How do we get the best path with at most r edges from i to j? Either we are happy with a path of at most $r-1$ edges (which is already covered), or we take a path of at most $r-1$ edges from i to some intermediate vertex k and then add the single final edge (k, j). We try every possible last-but-one vertex k and keep the best:

$$\ell_{ij}^{(r)} = \min\left(\ell_{ij}^{(r-1)},\ \min_{1 \le k \le n}\ \left(\ell_{ik}^{(r-1)} + W[k][j]\right)\right).$$

Because $W[j][j] = 0$, the term $k = j$ inside the inner min reproduces $\ell_{ij}^{(r-1)}$ exactly. So the outer min with $\ell_{ij}^{(r-1)}$ is redundant, and we can write the whole thing as one tidy minimization:

$$\ell_{ij}^{(r)} = \min_{1 \le k \le n}\ \left(\ell_{ik}^{(r-1)} + W[k][j]\right).$$

Compare that formula with ordinary matrix multiplication, $C[i][j] = \sum_k A[i][k] \cdot B[k][j]$. The two have the same form: we have replaced "+" with "min" and "×" with "+". In this *min-plus* algebra (also called the tropical semiring), computing the next layer of distances *is* a matrix product. If we name the operation $\odot$, then $L^{(r)} = L^{(r-1)} \odot W$.

Formal

Define the min-plus product of two $n \times n$ matrices A and B by

$$(A \odot B)[i][j] = \min_{1 \le k \le n}\ \left(A[i][k] + B[k][j]\right).$$

Let $L^{(1)} = W$ (at most one edge) and $L^{(r)} = L^{(r-1)} \odot W$. Then $L^{(r)}[i][j] = \ell_{ij}^{(r)}$, the minimum weight of an i-to-j path using at most r edges.

Figure 25.1 shows the shape of one entry of the product: to fill $L'[i][j]$ we slide across row i of L and down column j of W, add the paired entries, and keep the smallest sum – the same row-against-column pairing as an ordinary dot product, with min replacing summation and + replacing multiplication.

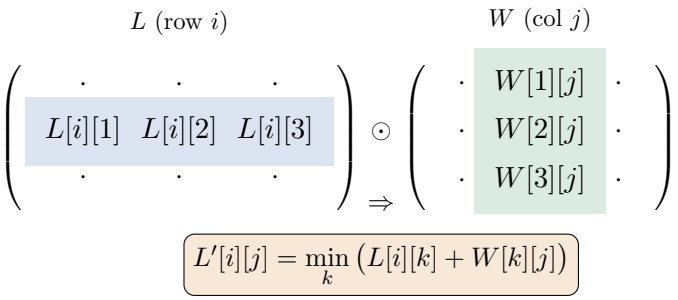


Figure 25.1: One entry of the min-plus product $L' = L \odot W$. Slide row i of L against column j of W, add paired cells, and keep the minimum.

25.2.2 When do we stop?

In a graph with no negative-weight cycles, any shortest path is *simple*: it never repeats a vertex, because cutting out a loop can only keep the weight the same or lower it. A simple path among n vertices has at most $n-1$ edges. So once r reaches $n-1$, the table stops changing: $L^{(n-1)} = L^{(n)} = \cdots$, and $L^{(n-1)}[i][j] = \delta(i, j)$. We have the answer.

Algorithm lab

Let us trace it on a tiny graph. Vertices $\{1, 2, 3\}$ with edges $1 \to 2$ (weight 3), $2 \to 3$ (weight 4), and $1 \to 3$ (weight 9). The weight matrix (rows = from, columns = to; "$\cdot$" is ∞) is

$$W = L^{(1)} = \begin{pmatrix} 0 & 3 & 9 \\ \cdot & 0 & 4 \\ \cdot & \cdot & 0 \end{pmatrix}.$$

Now compute $L^{(2)} = L^{(1)} \odot W$. The interesting entry is $L^{(2)}[1][3] = \min_k (L^{(1)}[1][k] + W[k][3])$. Try each k: $k = 1$ gives $0 + 9 = 9$; $k = 2$ gives $3 + 4 = 7$; $k = 3$ gives $9 + 0 = 9$. The minimum is 7, found via $k = 2$. So routing through vertex 2 beats the direct edge:

$$L^{(2)} = \begin{pmatrix} 0 & 3 & 7 \\ \cdot & 0 & 4 \\ \cdot & \cdot & 0 \end{pmatrix}.$$

With $n = 3$ we stop at $r = n - 1 = 2$, and indeed $D = L^{(2)}$. The shortest path from 1 to 3 has weight 7, going $1 \to 2 \to 3$.

25.2.3 Pseudocode and a first cost

Algorithm 114: EXTEND-SHORTEST-PATHS(L, W) – one min-plus product

Input: An $n \times n$ matrix L (best distances so far) and weight matrix W.
Output: The matrix $L' = L \odot W$.

```
let L' be a new n × n matrix;
for i ← 1 to n do
    for j ← 1 to n do
        L'[i][j] ← ∞;
        for k ← 1 to n do
            L'[i][j] ← min(L'[i][j], L[i][k] + W[k][j]);
return L';
```

One product costs $\Theta(n^3)$, exactly like grade-school matrix multiplication. We need products $L^{(2)}$, $L^{(3)}$, all the way up to $L^{(n-1)}$, which is $n-2$ of them, so the naive method costs $\Theta(n^4)$. That is worse than running Bellman–Ford n times ($\Theta(n^2 m)$, which is $\Theta(n^4)$ on a dense graph but better on a sparse one), but the method admits a simple speed-up.

25.2.4 Repeated squaring: $\Theta(n^3 \log n)$

We do not actually need every intermediate $L^{(r)}$; we only need $L^{(n-1)}$. And once the table stabilizes at $L^{(n-1)}$, multiplying by W again changes nothing. So $L^{(r)} = L^{(n-1)}$ for every $r \geq n-1$. That means we can leap ahead by *squaring*: $L^{(2r)} = L^{(r)} \odot L^{(r)}$.

Intuition

This is the same trick as fast exponentiation. To compute x^{16} you do not multiply x by itself sixteen times; you square four times: $x \to x^2 \to x^4 \to x^8 \to x^{16}$. Here we square the distance matrix instead. Each squaring doubles the number of edges our paths may use, so after $\lceil \log_2(n-1) \rceil$ squarings we have surpassed $n-1$ edges and the answer has converged.

Algorithm 115: FASTER-APSP(W) – min-plus repeated squaring

Input: Weight matrix W of an n-vertex graph with no negative cycle.
Output: Distance matrix D.

```
L ← W;
r ← 1;
while r < n − 1 do
    L ← EXTEND-SHORTEST-PATHS(L, L);
    r ← 2r;
return L;
```

Each squaring is one $\Theta(n^3)$ product, and we do $\lceil \log_2(n-1) \rceil$ of them, for a total of $\Theta(n^3 \log n)$. Notice we feed L as *both* arguments now (squaring), not L and W.

Cost summary

Matrix-multiplication APSP.

- Naive layer-by-layer: $\Theta(n^4)$ time, $\Theta(n^2)$ space.
- Repeated squaring: $\Theta(n^3 \log n)$ time, $\Theta(n^2)$ space.
- Handles negative edges; assumes no negative-weight cycle.
- Sub-cubic "fast" matrix multiplication does *not* directly help, because min-plus has no subtraction, so Strassen-style cancellation tricks do not apply. Whether min-plus multiplication admits a truly subcubic $O(n^{3-\varepsilon})$ algorithm is a central open question in fine-grained complexity.

Walking question

We argued $L^{(r)}$ stops changing at $r = n - 1$ because shortest paths are simple. Suppose the graph *did* contain a negative-weight cycle. Trace what would happen to a $D[i][j]$ for some i, j that can reach and leave that cycle as you keep squaring. Does the table ever stabilize? What does that suggest about how to *detect* a negative cycle with this method?

25.3 Floyd–Warshall: A Smarter Dynamic Program

The matrix method organized paths by their *number of edges*. Robert Floyd and Stephen Warshall organized them by a completely different feature, and it buys us a factor of $\log n$, landing at a clean $\Theta(n^3)$ with code so short you can write it from memory.

25.3.1 The new subproblem: allowed intermediate vertices

Fix an ordering of the vertices $1, 2, \ldots, n$. Consider a path from i to j. Its *intermediate vertices* are all the vertices on it except the two endpoints i and j. The new idea: build up the set of intermediate vertices we are *allowed to use*, one vertex at a time.

Define $d_{ij}^{(k)}$ to be the weight of a shortest path from i to j whose intermediate vertices all lie in the set $\{1, 2, \ldots, k\}$. (The endpoints i and j are always allowed; the restriction is only on what we may pass *through*.) The base case uses no intermediate vertices at all:

$$d_{ij}^{(0)} = W[i][j].$$

With $k = 0$ the only allowed paths are direct edges, which is exactly W.

25.3.2 The recurrence: "do I route through vertex k or not?"

Now suppose we know $d_{ij}^{(k-1)}$ for all i, j – shortest paths allowed to pass through $\{1, \ldots, k-1\}$. We want $d_{ij}^{(k)}$, which additionally permits passing through vertex k. A shortest such path either uses k as an intermediate vertex or it does not. These two cases tile every possibility:

- **Does not use k.** Then it only passes through $\{1, \ldots, k-1\}$, so its weight is $d_{ij}^{(k-1)}$.
- **Uses k exactly once.** (At most once, since the path is simple when there are no negative cycles.) Then it splits at k into a piece from i to k and a piece from k to j, and *neither* piece uses k as an intermediate vertex – they only pass through $\{1, \ldots, k-1\}$. So the weight is $d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Take the better of the two (Figure 25.2):

$$\boxed{d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ \ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right).}$$

After we have allowed all n vertices as intermediates, no restriction remains, so $d_{ij}^{(n)} = \delta(i, j)$. Nothing else is needed.

Intuition

Picture the sentence: "I now permit myself to drive through city k. For each pair of cities (i, j), is it worth detouring through k?" Routing through k is worth it exactly when the trip $i \to k$ plus the trip $k \to j$ (each using only cities I had already cleared) is shorter than my current best $i \to j$. Sweep k from 1 to n, opening up one city at a time, and after the last sweep every city is open and every distance is final.

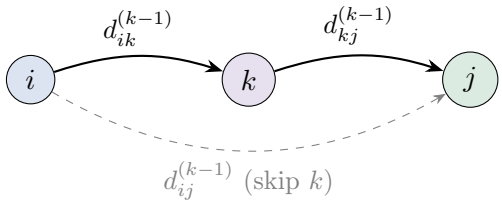


Figure 25.2: The Floyd–Warshall choice for the pair (i, j) at step k: either keep the best path that avoids k (dashed) or detour through k (solid), gluing together the best i-to-k and k-to-j pieces that already avoid k.

25.3.3 We only need two layers... actually one

The recurrence reads layer $k-1$ to produce layer k. Naively that suggests storing all $n+1$ layers, costing $\Theta(n^3)$ space. But you can do better: *overwrite the matrix in place.* Use a single matrix D and the assignment $D[i][j] \leftarrow \min(D[i][j],\ D[i][k] + D[k][j])$.

Why is reading possibly-already-updated values safe? Look at the two cells we read: $D[i][k]$ and $D[k][j]$. In the row k and column k entries, $d_{ik}^{(k)} = d_{ik}^{(k-1)}$ and $d_{kj}^{(k)} = d_{kj}^{(k-1)}$, because a shortest path to or from k that is allowed to use k as an intermediate gains nothing – using k in the middle would just repeat the endpoint. So the k-th row and k-th column do not change during pass k, and reading them "early" returns the right old value. Single matrix, $\Theta(n^2)$ space.

Figure 25.3 captures the picture: during pass k, the gray cross (row k and column k) is frozen, and the update to any cell (i, j) reads exactly the two cross cells $D[i][k]$ and $D[k][j]$. Because those two never move during this pass, it does not matter whether they have already been visited.

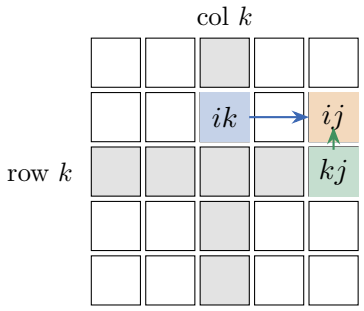


Figure 25.3: Pass k of Floyd–Warshall on a 5×5 table. The frozen cross is row k and column k (gray). Updating cell (i, j) (orange) reads only $D[i][k]$ (blue) and $D[k][j]$ (green), both on the frozen cross – which is why the in-place overwrite is safe.

Algorithm 116: FLOYD-WARSHALL(W) with predecessor reconstruction

Input: Weight matrix W of an n-vertex graph with no negative cycle.
Output: Distance matrix D and predecessor matrix Π.

```
D ← W;
for i ← 1 to n do
    for j ← 1 to n do
        if i ≠ j and W[i][j] < ∞ then Π[i][j] ← i;
        else Π[i][j] ← NIL;
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            if D[i][k] + D[k][j] < D[i][j] then
                D[i][j] ← D[i][k] + D[k][j];
                Π[i][j] ← Π[k][j];
return (D, Π);
```

The predecessor update deserves a word. When routing through k improves the path from i to j, the new path ends with the k-to-j portion, so the vertex just before j is whatever was just before j on the best k-to-j path: that is $\Pi[k][j]$. Copying it gives correct breadcrumbs.

Focus

The loop nesting order is essential: k *outermost*, then i, then j. The k loop is the dynamic-programming dimension (which intermediates are allowed); i and j just sweep all pairs within one k-layer. Swap k to the inside and you compute nonsense, because you would be allowing each pair a different, inconsistent set of intermediates.

25.3.4 A full worked trace

Algorithm lab

Take a four-vertex directed graph with these edges (and weights), which we will carry through: $1\to2=3$, $1\to4=7$, $2\to1=8$, $2\to3=2$, $3\to1=5$, $3\to4=1$, $4\to1=2$, $4\to3=6$. The starting matrix $D=D^{(0)}=W$ ("$\cdot$" is ∞):

$$D^{(0)}=\begin{pmatrix}0&3&\cdot&7\\8&0&2&\cdot\\5&\cdot&0&1\\2&\cdot&6&0\end{pmatrix}.$$

Pass $k=1$ (route through vertex 1). We improve $D[i][j]$ whenever $D[i][1]+D[1][j]<D[i][j]$. For instance $D[2][4]$: currently ∞; $D[2][1]+D[1][4]=8+7=15$, so it drops to 15. Also $D[3][2]$: $D[3][1]+D[1][2]=5+3=8$, new. And $D[4][2]$: $D[4][1]+D[1][2]=2+3=5$, new.

$$D^{(1)}=\begin{pmatrix}0&3&\cdot&7\\8&0&2&15\\5&8&0&1\\2&5&6&0\end{pmatrix}.$$

Pass $k=2$ (now also allow vertex 2). Check $D[1][3]$: $D[1][2]+D[2][3]=3+2=5<\infty$, so it becomes 5. Check $D[1][4]$: $D[1][2]+D[2][4]=3+15=18$, not better than 7. $D[3][3]$ stays 0.

$$D^{(2)}=\begin{pmatrix}0&3&5&7\\8&0&2&15\\5&8&0&1\\2&5&6&0\end{pmatrix}.$$

Pass $k=3$ (allow vertex 3). $D[1][4]$: $D[1][3]+D[3][4]=5+1=6<7$, improves to 6! $D[2][4]$: $D[2][3]+D[3][4]=2+1=3<15$, big improvement to 3. $D[2][1]$: $D[2][3]+D[3][1]=2+5=7<8$, drops to 7. $D[4][1]$ stays 2.

$$D^{(3)}=\begin{pmatrix}0&3&5&6\\7&0&2&3\\5&8&0&1\\2&5&6&0\end{pmatrix}.$$

Pass $k=4$ (allow vertex 4). $D[3][2]$: $D[3][4]+D[4][2]=1+5=6<8$, improves to 6. $D[3][1]$: $D[3][4]+D[4][1]=1+2=3<5$, drops to 3. $D[1][3]$: $D[1][4]+D[4][3]=6+6=12$, no. $D[2][1]$: $D[2][4]+D[4][1]=3+2=5<7$, drops to 5.

$$D^{(4)}=D=\begin{pmatrix}0&3&5&6\\5&0&2&3\\3&6&0&1\\2&5&6&0\end{pmatrix}.$$

That final matrix is the answer: every entry is a true shortest distance. For example, the shortest way from 3 to 1 has weight 3, achieved by $3\to4\to1$ $(1+2)$, which the direct edge (5) could not match.

25.3.5 Why it is correct

Formal

Loop invariant (Floyd–Warshall). Just before the iteration of the outer loop with index k, every entry satisfies

$$D[i][j] = d_{ij}^{(k-1)} = (\text{weight of a shortest } i\text{-to-}j \text{ path through intermediates in } \{1, \dots, k-1\}).$$

Initialization. Before $k = 1$ we have $D = W = D^{(0)}$: paths through the empty set of intermediates are exactly direct edges. The invariant holds.

Maintenance. Assume it holds entering pass k. The update sets $D[i][j] \leftarrow \min(D[i][j], D[i][k] + D[k][j])$. By the in-place argument above, the values read are the old layer-$(k-1)$ values. By the case analysis (route through k or not), this min is exactly $d_{ij}^{(k)}$. So after the pass the invariant holds for $k + 1$.

Termination. After $k = n$ finishes, $D[i][j] = d_{ij}^{(n)}$, paths allowed to use *any* intermediate, which is the unrestricted shortest distance $\delta(i, j)$. Done.

Common trap

If a negative-weight cycle exists, Floyd–Warshall does not loop forever – it runs the same $\Theta(n^3)$ steps – but the answers become meaningless, and some diagonal entry $D[i][i]$ ends up *negative*. That is your detector: after running, scan the diagonal; if any $D[i][i] < 0$, vertex i lies on a negative-weight cycle and the affected distances are really $-\infty$.

Cost summary

Floyd–Warshall. Three nested loops of length n: $\Theta(n^3)$ time, flat, best case equals worst case. Space $\Theta(n^2)$ (single in-place matrix, plus Π if you want paths). Handles negative edges; detects negative cycles via the diagonal. No graph is too dense for it – in fact it is the method of choice precisely *when* the graph is dense.

25.4 Transitive Closure: Reachability for the Price of Booleans

Sometimes you do not care about distances at all – you only want to know *whether* a path exists from i to j. The transitive closure of G is the graph G^* with an edge (i, j) exactly when j is reachable from i in G. We adopt the convention that every vertex reaches itself (the empty path), so the diagonal is always `true`; off the diagonal, (i, j) means there is a directed path of one or more edges. Think of "can task i eventually trigger task j?" in a dependency graph, or "can person i reach person j through any chain of follows?"

You could just run Floyd–Warshall with weights 0 and ∞ and then mark "reachable" wherever $D[i][j] < \infty$. That works. But if all you need is a yes/no answer, you can run the *same skeleton* on booleans and avoid arithmetic entirely, which is faster in practice and uses single bits.

Define $t_{ij}^{(k)}$ to be `true` iff there is a path from i to j with all intermediate vertices in $\{1, \dots, k\}$. The

base case is $t_{ij}^{(0)} = \texttt{true}$ iff $i = j$ or $(i, j) \in E$. The recurrence mirrors Floyd–Warshall, with min becoming logical OR and + becoming logical AND:

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \ \textbf{or} \ \left(t_{ik}^{(k-1)} \ \textbf{and} \ t_{kj}^{(k-1)}\right).$$

Intuition

The translation is exact. "Shortest distance" lived in the $(\min, +)$ world; "can you get there" lives in the (OR, AND) world. Both are semirings, and the Floyd–Warshall update is really a generic semiring algorithm. Change the two operations and the same triple loop computes a different fact about the graph. Distances, reachability, maximum-bottleneck capacity, most-reliable paths – all the same code, four different pairs of operators.

Algorithm 117: TRANSITIVE-CLOSURE(G)

Input: Adjacency structure of an n-vertex graph G.
Output: Boolean matrix T with $T[i][j] = \texttt{true}$ iff j is reachable from i.

```
for i ← 1 to n do
    for j ← 1 to n do
        if i = j or (i, j) ∈ E then T[i][j] ← true;
        else T[i][j] ← false;
for k ← 1 to n do
    for i ← 1 to n do
        for j ← 1 to n do
            T[i][j] ← T[i][j] or (T[i][k] and T[k][j]);
return T;
```

Same $\Theta(n^3)$ shape, but each operation is a single bit op. If you pack rows into machine words, the inner loop becomes a bitwise-OR of word-sized chunks, and you get a real constant-factor speed-up of roughly the word size w (commonly 64), giving $\Theta(n^3/w)$ in practice – which is why transitive closure on bitsets is genuinely fast.

Why this matters

Many static analyses reduce to reachability. Compilers ask "can the value defined here reach this use?"; type checkers and build systems ask "does target A transitively depend on target B?"; security tools ask "can tainted input flow to this dangerous sink?" All of these are transitive closures of a graph, and large-scale tools run bitset versions of this loop.

25.5 Johnson's Algorithm: Fast on Sparse Graphs, Even with Negative Edges

Floyd–Warshall works well on dense graphs, but its $\Theta(n^3)$ is wasteful when the graph is *sparse* – few edges, $m \ll n^2$. On a sparse graph you would love to run Dijkstra from each vertex, since one Dijkstra costs $O(m \log n)$ and n of them cost $O(nm \log n)$, far below n^3 when m is small. There is

just one problem: Dijkstra requires *nonnegative* edge weights, and we want to handle negative ones.

Donald Johnson's idea is to *reweight* the graph so that all weights become nonnegative, *without changing which paths are shortest*, and then run Dijkstra n times on the reweighted graph. The reweighting itself is done with one run of Bellman–Ford. Here is how.

25.5.1 The reweighting trick

We want to assign each vertex v a number $h(v)$ – think of it as an "elevation" or potential – and replace every edge weight by

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v).$$

Watch what happens to the weight of an entire path $p = v_0 \to v_1 \to \cdots \to v_k$. The new weight telescopes:

$$\hat{w}(p) = \sum_{i=1}^{k} \big(w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)\big) = w(p) + h(v_0) - h(v_k).$$

Every interior h cancels with its neighbor; only the endpoints survive. So the new weight of *any* path from s to t is the old weight plus the same constant $h(s) - h(t)$, regardless of which path you took, so the ranking of paths is unchanged. Figure 25.4 shows the cancellation on a three-edge path: each $+h(v_i)$ from one edge meets the $-h(v_i)$ of the next.

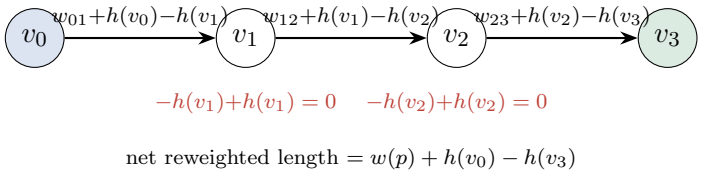


Figure 25.4: Reweighting telescopes. Each interior potential $h(v_i)$ appears once with a $+$ and once with a $-$ and cancels; only the endpoint terms $+h(v_0)$ and $-h(v_3)$ survive, so every v_0-to-v_3 path shifts by the same amount.

Focus

Because the additive correction $h(s)-h(t)$ depends only on the endpoints, *the ranking of paths from s to t is unchanged.* A path that was shortest in w is still shortest in $\hat{w}$. We have changed all the numbers but preserved every shortest path. To recover the true distance afterward we just undo the shift: $\delta(s, t) = \hat{\delta}(s, t) - h(s) + h(t)$.

Intuition

Think of $h(v)$ as the height of vertex v and an edge's reweight as "how much you climb minus how much you descend" added to its length. Walking a whole route, all the intermediate ups and downs cancel; only your net elevation change between start and finish matters. Reweighting is just adding a conservative "gravity" field. Gravity does not change which route between

two fixed points is shortest – it only re-labels the costs.

25.5.2 Choosing h so every $\hat{w} \geq 0$

We need an h that makes $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$ for every edge, i.e. $h(v) - h(u) \leq w(u, v)$. That inequality should look familiar: it is exactly the *triangle / relaxation* condition that shortest-path distances satisfy. So if we let $h(v)$ be a shortest-path distance from some fixed source to v, the condition holds automatically.

Here is the construction. Add a brand-new vertex s to the graph, with a zero-weight edge $s \to v$ to every original vertex v (and no edges into s). This new vertex can reach everything and reaches it with finite distance (at worst 0). Run Bellman–Ford from s. Set $h(v) = \delta(s, v)$, the shortest distance from s to v in the augmented graph.

Why does this satisfy $h(v) - h(u) \leq w(u, v)$? Because shortest distances obey the triangle inequality: the best way to reach v is no worse than the best way to reach u followed by the edge (u, v), so $\delta(s, v) \leq \delta(s, u) + w(u, v)$, which rearranges to exactly what we need. And Bellman–Ford from s is also where we *detect a negative cycle*: if it reports one, we stop and announce that all-pairs shortest paths are undefined.

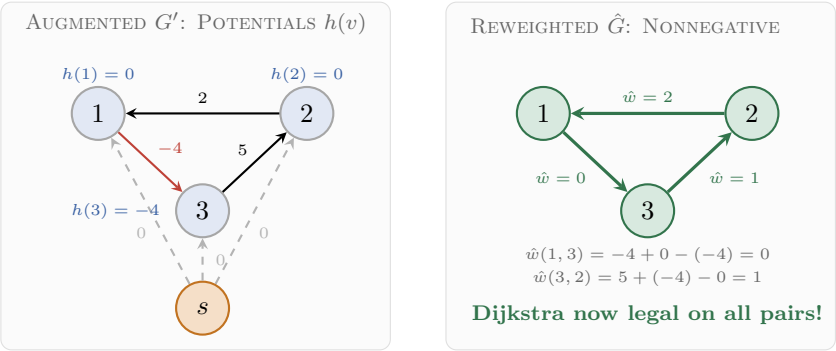


Figure 25.5: Johnson's reweighting technique. Left: the augmented graph G' introduces a synthetic source s with 0-weight edges to all vertices. Running Bellman–Ford once from s computes vertex potentials $h(v) = \delta(s, v)$. Right: replacing edge weights with $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ yields nonnegative weights without altering shortest-path trees, enabling Dijkstra to run safely from every vertex.

The dashed 0-weight edges from the synthetic source s let Bellman–Ford compute one h value per vertex in a single pass over the whole graph (Figure 25.5).

25.5.3 The full algorithm

Algorithm 118: JOHNSON(G, w)

Input: Directed weighted graph $G = (V, E)$ with weights w, possibly negative.
Output: Distance matrix D, or a report that a negative-weight cycle exists.

form G' by adding a new vertex s and edges (s, v) of weight 0 for all $v \in V$;
if BELLMAN-FORD*(G', w, s) finds a negative-weight cycle* **then**
  return "negative-weight cycle";
foreach $v \in V$ **do**
  $h(v) \leftarrow \delta(s, v)$; `// from Bellman--Ford`
foreach *edge* $(u, v) \in E$ **do**
  $\hat{w}(u, v) \leftarrow w(u, v) + h(u) - h(v)$
foreach $u \in V$ **do**
  run DIJKSTRA(G, $\hat{w}$, u) to get $\hat{\delta}(u, v)$ for all v;
  foreach $v \in V$ **do**
    $D[u][v] \leftarrow \hat{\delta}(u, v) - h(u) + h(v)$;
return D;

25.5.4 A worked example of the reweighting

Algorithm lab

Use the three-vertex graph from the diagram: edges $2 \to 1$ (weight 2), $1 \to 3$ (weight -4), $3 \to 2$ (weight 5). Add source s with 0-weight edges to $1, 2, 3$ and run Bellman–Ford. The cheapest way to reach each vertex from s:

$$h(1) = \delta(s, 1) = 0, \quad h(2) = \delta(s, 2) = 0, \quad h(3) = \delta(s, 3) = -4.$$

(Vertex 3 is reached via $s \to 1 \to 3 = 0 + (-4) = -4$; everything else is reached directly at cost 0, since the synthetic edges already cost 0.) Now reweight each real edge with $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$:

$$\hat{w}(2, 1) = 2 + 0 - 0 = 2, \quad \hat{w}(1, 3) = -4 + 0 - (-4) = 0, \quad \hat{w}(3, 2) = 5 + (-4) - 0 = 1.$$

Every reweighted edge is nonnegative – exactly as promised – so Dijkstra is now legal. And the -4 edge, which would have broken Dijkstra, became a harmless 0. To read a true distance back out, say from 1 to 2: Dijkstra on $\hat{w}$ gives $\hat{\delta}(1, 2) = \hat{w}(1, 3) + \hat{w}(3, 2) = 0 + 1 = 1$; then $\delta(1, 2) = \hat{\delta}(1, 2) - h(1) + h(2) = 1 - 0 + 0 = 1$, while the true path $1 \to 3 \to 2$ costs $-4 + 5 = 1$. They match.

25.5.5 Why it is correct, and what it costs

Formal

Claim. With $h(v) = \delta(s, v)$, every reweighted edge satisfies $\hat{w}(u, v) \geq 0$, and for every pair (u, v), $\delta(u, v) = \hat{\delta}(u, v) - h(u) + h(v)$.
Nonnegativity. The triangle inequality gives $\delta(s, v) \leq \delta(s, u) + w(u, v)$, i.e. $h(v) - h(u) \leq w(u, v)$, i.e. $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.
Distance recovery. For any path p from u to v, telescoping gives $\hat{w}(p) = w(p) + h(u) - h(v)$. The additive term is the same for all p, so the $\hat{w}$-shortest path is a w-shortest path; taking the minimum over p on both sides yields $\hat{\delta}(u, v) = \delta(u, v) + h(u) - h(v)$, which rearranges to the stated recovery formula.

Cost summary

Johnson's algorithm.

- One Bellman–Ford from s: $O(nm)$.
- Reweighting all edges: $O(m)$.
- n runs of Dijkstra with a binary heap: $O\big(n\,(m \log n)\big) = O(nm \log n)$; with a Fibonacci heap, $O\big(n\,(m + n \log n)\big) = O(nm + n^2 \log n)$.
- Total: $O(nm \log n)$ (binary heap), dominated by the Dijkstra runs.
- Space $O(n^2)$ for the output, $O(n + m)$ working.
- Handles negative edges; detects negative cycles in the Bellman–Ford phase.

Intuition

Johnson's algorithm splits the work between two algorithms. Bellman–Ford is slow but robust: it tolerates negative edges, so we use it *once* to neutralize them by reweighting. Dijkstra is fast but fragile: it cannot tolerate negative edges, so we run it n times only *after* the graph has been made nonnegative. Johnson's algorithm combines Bellman–Ford's handling of negative edges with Dijkstra's speed on sparse graphs.

25.6 Choosing an Algorithm

You now have several tools. Which one should you reach for?

Method	Time	Best when
Repeated Dijkstra	$O(nm \log n)$	sparse, weights ≥ 0
Johnson	$O(nm \log n)$	sparse, some negative edges
Floyd–Warshall	$\Theta(n^3)$	dense, any weights, want simplicity
Matrix (squaring)	$\Theta(n^3 \log n)$	mainly of theoretical interest
Transitive closure	$\Theta(n^3/w)$	only reachability needed

The rough rule: **dense graph ⇒ Floyd–Warshall**; **sparse graph ⇒ Johnson** (or repeated Dijkstra if all weights are already nonnegative). The crossover is around $m \approx n^2/\log n$. Below that density, $nm \log n < n^3$ and Johnson wins; above it, Floyd–Warshall's tiny constant factor and cache-friendly triple loop make it the better choice in practice.

History and connection

The main all-pairs shortest path algorithms were developed in the 1950s, 1960s, and 1970s. Alfonso Shimbel (1953) first formulated shortest paths in the min-plus semiring (where addition plays the role of multiplication and minimization plays the role of addition). In 1959, French mathematician Bernard Roy published the triple-loop dynamic programming algorithm. Stephen Warshall independently published the boolean transitive-closure version in 1962 for compiler syntax analysis, and Robert W. Floyd generalized Warshall's construction that same year to real-valued distances and predecessor reconstruction, yielding the Roy–Floyd–Warshall algorithm. In 1977, Donald B. Johnson discovered that a single run of Bellman–Ford could compute node potentials to eliminate negative edge weights, enabling n efficient priority-queue runs of Dijkstra on sparse graphs.

A prominent theoretical question centers on whether all-pairs shortest paths can break the cubic barrier: while Floyd–Warshall achieves a simple $O(n^3)$ bound with tight cache locality, researchers long asked whether fast matrix multiplication techniques could yield a truly subcubic $O(n^{3-\epsilon})$ algorithm for general real-weighted graphs.

This work led to the algebraic path problem, which shows that all-pairs shortest paths, transitive closure, Kleene's regular expressions in formal languages, and parsing algorithms share the same underlying semiring structure. Today, whether a truly subcubic $O(n^{3-\epsilon})$ algorithm exists for real-weighted APSP remains one of the central open questions in fine-grained computational complexity (the APSP conjecture). In 2014, Ryan Williams designed an algorithm running in $O(n^3/2^{\Omega(\sqrt{\log n})})$ time using circuit complexity techniques, yet whether the exponent 3 can be strictly reduced remains unsolved. In CLRS (Chapter 25), matrix multiplication and Floyd–Warshall are covered in Sections 25.1–25.2, and Johnson's algorithm in Section 25.3.

Why this matters

All-pairs shortest paths appears in many everyday systems. Route planners precompute distance tables over regions so a query is a lookup, not a fresh search. The classic distance-vector routing protocols (RIP and friends) are distributed cousins of Bellman–Ford run continuously across the internet. Network-analysis tools compute *betweenness centrality* – how often a node sits on shortest paths between others – which needs all-pairs distances and is used to find influential people, fragile links, and bottleneck routers. And in program analysis, the transitive-closure form decides the reachability questions that compilers and security scanners depend on.

25.7 Summary

Pause

The main points of the chapter:

- The problem: a full $n \times n$ table of shortest distances between all ordered pairs, plus optional predecessors for path reconstruction.
- Organize paths by *number of edges* and you get a min-plus matrix product; squaring gives $\Theta(n^3 \log n)$.
- Organize paths by *which intermediate vertices are allowed* and you get Floyd–Warshall, a clean in-place $\Theta(n^3)$ triple loop.
- Swap $(\min, +)$ for (OR, AND) and the same loop computes transitive closure (reachability) on bits.
- For sparse graphs with negative edges, Johnson reweights once with Bellman–Ford to remove negative edges, then runs Dijkstra n times for $O(nm \log n)$.
- Negative cycles make distances $-\infty$; watch the Floyd–Warshall diagonal, or trust Bellman–Ford in Johnson, to detect them.

Shortest-path algorithms assume edges have unlimited capacity. But real networks have throughput limits: pipes, wires, and roads can only carry so much traffic. In Chapter 26, we explore maximum flow, the max-flow min-cut theorem, and augmenting path algorithms.

Exercises

Exercise 25.1. Run FASTER-APSP (repeated squaring) by hand on the four-vertex graph from the Floyd–Warshall lab. Write $L^{(1)} = W$, then $L^{(2)}$, then $L^{(4)}$. Confirm that $L^{(4)}$ equals the final matrix $D^{(4)}$ we obtained with Floyd–Warshall. Why is computing $L^{(4)}$ (rather than stopping at $L^{(3)}$) the right call for $n = 4$?

Exercise 25.2. Carry the predecessor matrix Π through the full Floyd–Warshall trace in the lab. Then use it to print the actual shortest path from vertex 2 to vertex 4. (You should recover $2 \to 3 \to 4$ with total weight 3.)

Exercise 25.3. Explain precisely why the loop order in Floyd–Warshall must have k outermost. Construct a tiny graph (three vertices is enough) on which putting the k loop innermost produces a wrong answer, and show the wrong number it computes.

Exercise 25.4. The in-place version of Floyd–Warshall reuses one matrix. We justified this by arguing the k-th row and k-th column do not change during pass k. Prove that claim carefully from the recurrence $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ by specializing to $j = k$ and to $i = k$.

Exercise 25.5. Modify Floyd–Warshall so that instead of *shortest* paths it computes, for every pair (i, j), the maximum-bottleneck path: the path that maximizes the minimum edge weight along it (useful for finding the widest channel between two network nodes). What two operations replace min and +? What is the base case? This is another semiring – name the operations.

Exercise 25.6. A negative-weight cycle leaves a fingerprint on the Floyd–Warshall diagonal. Run Floyd–Warshall on the two-vertex graph with edges $1 \to 2$ (weight 1) and $2 \to 1$ (weight -3). Show that after the algorithm finishes, $D[1][1]$ and $D[2][2]$ are negative, and explain how you would scan for this.

Exercise 25.7. In Johnson's algorithm we added a synthetic source s with 0-weight edges to every vertex. Suppose instead you used edges of weight $+5$. Would the algorithm still be correct? What about weight -5? Argue from the role $h(v) = \delta(s, v)$ plays, and state exactly which property of h you must preserve.

Exercise 25.8. Work out the density crossover between Johnson and Floyd–Warshall more carefully. Setting $nm \log n = n^3$, solve for m in terms of n. For $n = 1000$, roughly how many edges does a graph need before Floyd–Warshall becomes the faster choice? (Use $\log_2 1000 \approx 10$.)

Exercise 25.9. (*Challenge.*) Show how to use the predecessor information from a single run of Floyd–Warshall to detect, for a graph *with* a negative cycle, an actual vertex sequence forming a negative cycle, rather than only reporting that one exists. Hint: a negative diagonal entry $D[i][i] < 0$ means i is on such a cycle; follow predecessors from i back to i.

Exercise 25.10. (*Challenge.*) The min-plus product has no known truly sub-cubic *combinatorial* algorithm, but there is a clever $O(n^3/\log n)$-style speed-up and connections to the famous APSP-vs-min-plus equivalence. Without looking it up, explain why Strassen's $O(n^{2.81})$ matrix multiplication cannot simply be dropped in to speed up min-plus APSP. (Hint: what algebraic operation does Strassen rely on that min does not have?)

Chapter 26

Maximum Flow

In Chapters 24 and 25, we computed shortest paths under the assumption that edges could carry unbounded traffic without congestion. In real networks, however, physical connections have finite capacities.

Picture a network of pipes. Water enters at one special place, sloshes through a tangle of junctions and pipes of various widths, and leaves at another special place. Each pipe has a limit – a maximum rate it can carry before it bursts. The question of this chapter is easy to state but harder to answer: *how much water per second can you push from the entrance to the exit?*

That is the maximum-flow problem. It does not look like sorting, or shortest paths, or anything you have met so far, and the first reaction most people have is "surely I just send as much as I can down every pipe." But the pipes share junctions, and water that piles up at a junction has nowhere to go – whatever flows in must flow out. The constraints interlock. Greedily flooding one path can block a better global arrangement, and the key technique is a way to *undo* a greedy mistake without starting over.

The central result is the max-flow min-cut theorem: the maximum rate of flow from source to sink equals the minimum capacity required to sever all connections between them. The same model also solves bipartite matching, project scheduling, and network connectivity.

Intuition

The physical intuition is straightforward: flow models a conserved fluid volume, where success depends on leaving yourself the freedom to *reroute* volume already committed. The central technique – the residual network – is a bookkeeping device that says, "if I previously sent five units down this pipe, I am allowed to cancel up to five of them later by pretending to send flow backward." That permission to take back a decision is what turns a hopeless greedy idea into an exact algorithm.

26.1 Flow Networks

Let us pin down the objects. A *flow network* is a directed graph $G = (V, E)$ in which every edge $(u, v) \in E$ carries a nonnegative *capacity* $c(u, v) \geq 0$. Two vertices are singled out: a *source* s, where material is produced, and a *sink* t, where it is consumed. We assume every vertex lies on some path

from s to t, so the graph is connected in the relevant sense and $|E| \geq |V| - 1$.

For convenience we adopt a small convention: if (u, v) is not an edge, we set $c(u, v) = 0$, and we forbid *antiparallel* edges, meaning we never have both (u, v) and (v, u) in E at once. (If a real problem has both, split one of them by inserting a dummy vertex in the middle; nothing important changes. We will explain why this convention is convenient when residual networks appear.)

Definition 26.1 (Flow)**.** A *flow* in G is a function $f : V \times V \to \mathbb{R}$ satisfying two properties:

- **Capacity constraint:** for all $u, v \in V$, $0 \leq f(u, v) \leq c(u, v)$.
- **Flow conservation:** for every vertex $u \in V \setminus \{s, t\}$,

$$\sum_{v \in V} f(v, u) \;=\; \sum_{v \in V} f(u, v).$$

The *value* of the flow, written $|f|$, is the net amount leaving the source,

$$|f| \;=\; \sum_{v \in V} f(s, v) \;-\; \sum_{v \in V} f(v, s).$$

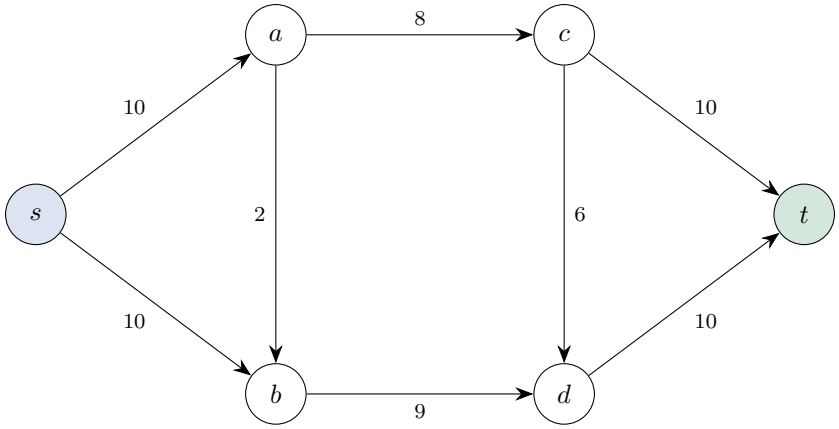
Read the two constraints carefully, because the rest of the chapter works with them. The capacity constraint says no pipe carries more than its width allows, and (in this nonnegative formulation) no pipe carries a negative amount. Flow conservation is the law of the junction: at every vertex other than the source and sink, total inflow equals total outflow. Water is neither created nor destroyed inside the network; it only gathers at s and drains at t.

Focus

A flow is a *global* object that must be *locally consistent* everywhere. Maximizing each pipe independently fails, because altering flow in one pipe ripples through conservation at both of its endpoints. The skill you are building is reasoning about a quantity ($|f|$) that lives at the source and sink, while honoring constraints that live on every edge and at every interior vertex.

26.1.1 A network to keep on the table

We will trace one small network through the whole chapter. It has six vertices s, a, b, c, d, t and the capacities drawn below. Memorize nothing; just keep this picture handy.



Take a guess right now, before any theory: what do you think the maximum flow from s to t is? Many people eyeball s's two outgoing pipes, see capacity $10 + 10 = 20$, and guess 20. Hold that guess. We will see it is too optimistic, and the reason is the main idea of this chapter.

Why this matters

Flow networks model a wide range of capacity-constrained systems. The same model computes how much data a computer network can route between two machines, how many trucks a road system can carry between two cities, how to schedule airline crews, how to segment an image into foreground and background, how to assign students to schools, and how baseball teams get mathematically eliminated from a pennant race. Whenever a resource must travel through a shared, capacity-limited structure, maximum flow probably applies. Because so many problems reduce to it, it is one of the most useful algorithms an engineer can know.

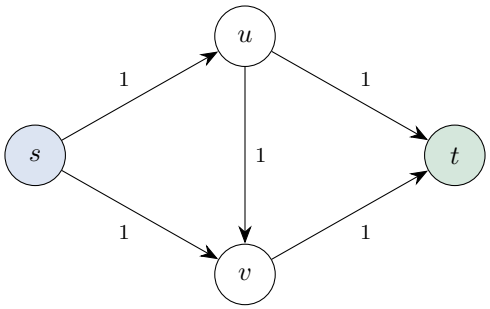
26.2 The Ford–Fulkerson Method

We attack maximum flow with a natural idea: *start with zero flow, then keep finding a path from s to t along which you can push a little more, and push it.* A path with spare room all the way from source to sink is called an *augmenting path*. As long as one exists, you are not done; when none exists, the claim (to be proved) is that you have the maximum.

This is called a *method* rather than an algorithm because it leaves one choice unspecified: *which* augmenting path do you pick when several exist? Different answers give different algorithms with wildly different running times. We will nail down a good choice in Section 26.4. For now, we focus on the basic method.

26.2.1 Why naive greedy is not enough

Let us try the naive version on a tiny graph and watch it fail, to see why a fix is needed. Consider this network, with all capacities equal to 1 except where noted.



Suppose your first greedy path is $s \to u \to v \to t$, and you push 1 unit along it. Now $s \to u$, $u \to v$, and $v \to t$ are all saturated. Look for another augmenting path with spare capacity: $s \to v$ has room, but $v \to t$ is full; $u \to t$ has room, but $s \to u$ is full. There is no path of strictly positive spare capacity from s to t. Naive greedy stops, declaring the answer 1.

But the true answer is 2: push 1 unit along $s \to u \to t$ and 1 unit along $s \to v \to t$, using no middle edge at all. The greedy mistake was committing flow to the middle edge $u \to v$. We need

a mechanism to *retract* that unit – to send flow "backward" along $u \to v$ to cancel it, freeing u to serve t and v to be served by s. That mechanism is the residual network, and the figure below shows the step it allows: a path that uses the *dashed backward copy* of the saturated middle edge.

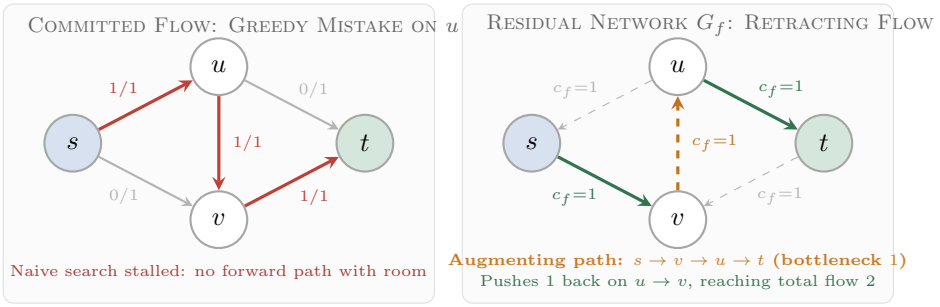


Figure 26.1: The residual network G_f resolving a greedy suboptimal trap. Left: after pushing 1 unit along $s \to u \to v \to t$, the network appears saturated to simple forward search. Right: the residual network introduces forward edges with spare capacity and backward edges (dashed) with capacity equal to existing flow. The augmenting path $s \to v \to u \to t$ traverses the backward edge $v \to u$, canceling the unit on $u \to v$ and routing 2 full units from s to t.

26.2.2 Residual networks

Given a flow f, the *residual network* G_f tells you, for every ordered pair of vertices, how much *additional* flow you may push there. It has two kinds of edges, and the second kind is the clever one.

For each edge $(u, v) \in E$:

- a *forward residual edge* (u, v) with residual capacity $c_f(u, v) = c(u, v) - f(u, v)$, the unused room in the pipe;
- a *backward residual edge* (v, u) with residual capacity $c_f(v, u) = f(u, v)$, equal to the flow currently in the pipe.

Edges of residual capacity 0 are simply omitted. The backward edge encodes exactly the "permission to cancel" from our intuition box: if $f(u, v)$ units are flowing forward, you may push up to $f(u, v)$ units backward, which has the net effect of reducing the forward flow.

Formal

Formally, the residual capacity is

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise.} \end{cases}$$

The residual network is $G_f = (V, E_f)$ where $E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$. Since $|E_f| \leq 2|E|$, G_f is no more than twice the size of G. This is exactly why we banned antiparallel edges: it guarantees that a forward residual edge and a backward residual edge never collide on the same ordered pair, so c_f is well-defined by a single formula.

Common trap

The backward edge is not a real pipe and carries no real water. It is a *bookkeeping* edge that exists only inside G_f. When an augmenting path uses a backward residual edge (v, u), you are not sending new water from v to u; you are *reducing* the existing real flow $f(u, v)$. Mixing up "residual flow" with "actual flow" is the single most common confusion in this chapter. Actual flow f lives in G; residual capacities live in G_f; the algorithm reads G_f to decide how to update f.

26.2.3 Augmenting along a path

An *augmenting path* p is any simple path from s to t in the residual network G_f. Because every edge on it has $c_f > 0$, we can push flow along the whole path. How much? Exactly the smallest residual capacity on the path – the bottleneck:

$$c_f(p) \;=\; \min\{\, c_f(u, v) : (u, v) \text{ is on } p \,\}.$$

To augment, we walk the path and, for each residual edge (u, v), push $c_f(p)$ units. If (u, v) is a forward edge of G, we *increase* $f(u, v)$; if it is a backward edge (meaning $(v, u) \in E$), we *decrease* $f(v, u)$. In symbols, the updated flow f' satisfies, for each (u, v) on p,

$$f'(u, v) = f(u, v) + c_f(p) \quad \text{(forward)}, \qquad f'(v, u) = f(v, u) - c_f(p) \quad \text{(backward)}.$$

Lemma 26.1 (Augmentation produces a valid flow)**.** *If f is a flow in G and p is an augmenting path in G_f, then f' defined above is a flow in G, and $|f'| = |f| + c_f(p) > |f|$.*

Why this is true, gently. Capacities first. On a forward edge we add at most $c_f(u, v) = c(u, v) - f(u, v)$, so $f'(u, v) \le c(u, v)$, and since $c_f(p) > 0$ we keep $f'(u, v) \ge 0$. On a backward edge we subtract at most $c_f(v, u) = f(u, v)$, so $f'(u, v) \ge 0$, and it only decreased so it stays $\le c(u, v)$. Every edge remains legal.

Conservation next. Take any interior vertex w on the path. The path enters w once and leaves w once, so exactly two residual edges at w change. Walk through the four cases (the entering edge forward or backward, likewise the leaving edge) and in each case the net inflow at w changes by $+c_f(p)$ from one edge and $-c_f(p)$ from the other – conservation is preserved. (Pushing $c_f(p)$ "backward" on a real edge reduces both that edge's contribution to its head and to its tail by the same amount, which is exactly what keeps the books balanced.) Vertices not on p are untouched.

Finally the value. The path leaves s along exactly one residual edge. If it is a forward edge out of s, then $f(s, \cdot)$ increases by $c_f(p)$; if it is a backward edge into s, then some $f(\cdot, s)$ decreases by $c_f(p)$. Either way the net out-of-s quantity $|f|$ rises by exactly $c_f(p)$. □

This lemma is the key result. It says: *find any s-t path in G_f, push its bottleneck, repeat.* Each augmentation strictly increases the value. We state the method as pseudocode.

Algorithm 119: FORD-FULKERSON(G, s, t)

Input: flow network G with capacities c, source s, sink t
Output: a maximum flow f

```
for each edge (u, v) ∈ E do
    f(u, v) ← 0
while there exists an augmenting path p from s to t in G_f do
    c_f(p) ← min{c_f(u, v) : (u, v) ∈ p};
    for each edge (u, v) on p do
        if (u, v) ∈ E then
            f(u, v) ← f(u, v) + c_f(p)
        else
            f(v, u) ← f(v, u) − c_f(p)
return f;
```

26.2.4 A full numeric trace

Let us run the method on the running network of Section 26.1.1. We start with $f = 0$ everywhere, so initially G_f is just G with every edge at its full capacity.

Augmentation 1. Find a path; say we find $s \to a \to c \to t$. The residual capacities are $10, 8, 10$, so the bottleneck is $c_f(p) = 8$. Push 8. Now $f(s, a) = 8$, $f(a, c) = 8$, $f(c, t) = 8$, and $|f| = 8$. The edge $a \to c$ is saturated.

Augmentation 2. Find $s \to b \to d \to t$. Residual capacities $10, 9, 10$; bottleneck 9. Push 9. Now $f(s, b) = 9$, $f(b, d) = 9$, $f(d, t) = 9$, and $|f| = 8 + 9 = 17$. The edge $b \to d$ is saturated.

Augmentation 3. Is there any path left? Let us search the residual network from s and see how far we get. From s we can still reach a (residual $10 - 8 = 2$) and b (residual $10 - 9 = 1$). From a the forward edge $a \to c$ is saturated, but $a \to b$ has residual 2 – so a only takes us to b, which we already have. From b the lone out-edge $b \to d$ is saturated. So the search from s reaches exactly $\{s, a, b\}$ and stops dead.

You might hope a *backward* residual edge rescues us, the way it did in the diamond. The only real edge with spare capacity past the waist is $c \to d$ (residual 6), which would let us try

$$s \;\to\; a \;\to\; c \;\to\; d \;\to\; t.$$

But this path is blocked at its second step: $c_f(a, c) = 0$, because $a \to c$ is saturated. We can never *reach* c from s in G_f, so its outgoing room is useless. So after pushing 8 and 9, *there is no augmenting path*, and the maximum flow of this network is

$$|f| = 17,$$

not the 20 you might have guessed from the source's two outgoing pipes of capacity 10 each. The bottleneck is downstream: edges $a \to c$ (cap 8) and $b \to d$ (cap 9) together can pass at most $8 + 9 = 17$, and everything from s must funnel through them.

Walking question

Go look at the running network again and find the "narrowest waist" of the whole graph – a set of edges you could cut to completely separate s from t whose total capacity is as small as possible. Did you find $\{a \to c,\, b \to d\}$ with total $8 + 9 = 17$? Notice that this number is exactly the maximum flow we computed, and that this waist is the very cut drawn in Figure 26.2 below. The next section proves that this equality holds in every flow network.

26.3 Cuts and the Max-Flow Min-Cut Theorem

The trace ended with an observation: the maximum flow equaled the capacity of the cheapest "waist" separating s from t. Let us make "waist" precise and then prove the equality, which is the central result of the chapter.

Definition 26.2 (Cut)**.** An *s-t cut* of G is a partition of V into two sets S and $T = V \setminus S$ with $s \in S$ and $t \in T$. The *net flow across the cut* is

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u),$$

and the *capacity of the cut* is

$$c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v).$$

Notice the asymmetry: the *net flow* subtracts the flow coming back from T to S, but the *capacity* only counts edges going from S to T (forward), with no subtraction. That asymmetry is exactly where the slack between flow and capacity will live.

Lemma 26.2 (Flow across any cut equals the flow value)**.** *For any flow f and any s-t cut (S,T), we have $f(S,T) = |f|$.*

The bookkeeping. Start from $|f| = \sum_v f(s,v) - \sum_v f(v,s)$, the net flow out of s alone. Now *add zero* cleverly: for every other vertex $u \in S$, conservation says its net outflow $\sum_v f(u,v) - \sum_v f(v,u)$ is 0, so we may add all of these in without changing the total. Summing over all $u \in S$,

$$|f| = \sum_{u \in S} \Big(\sum_{v \in V} f(u,v) - \sum_{v \in V} f(v,u) \Big).$$

Split the inner sum over v into $v \in S$ and $v \in T$. The $v \in S$ terms cancel in pairs (each $f(u,v)$ with $u, v \in S$ appears once with a $+$ and once, as $f(v,u)$, with a $-$). What survives is precisely the S-to-T part, which is $f(S,T)$. Done. □

This identity has an important consequence. The flow value is the same no matter which cut you measure it across – pick any line separating s from t, and the net flow crossing it equals $|f|$. Combined with capacities, it gives an instant upper bound.

Corollary 26.1 (Weak duality)**.** *For any flow f and any cut (S,T), $|f| \le c(S,T)$.*

Proof. $|f| = f(S,T) = \sum f(u,v) - \sum f(v,u) \le \sum f(u,v) \le \sum c(u,v) = c(S,T)$, where the first inequality drops the nonnegative back-flow term and the second uses the capacity constraint edge by edge. □

So *every* flow value is at most *every* cut capacity. In our running example, the cut $S = \{s, a, b\}$, $T = \{c, d, t\}$ has forward edges $a \to c$ (cap 8) and $b \to d$ (cap 9), capacity 17. Weak duality already guarantees the maximum flow cannot exceed 17. In fact, this bound is always *tight*: some flow actually achieves the smallest cut.

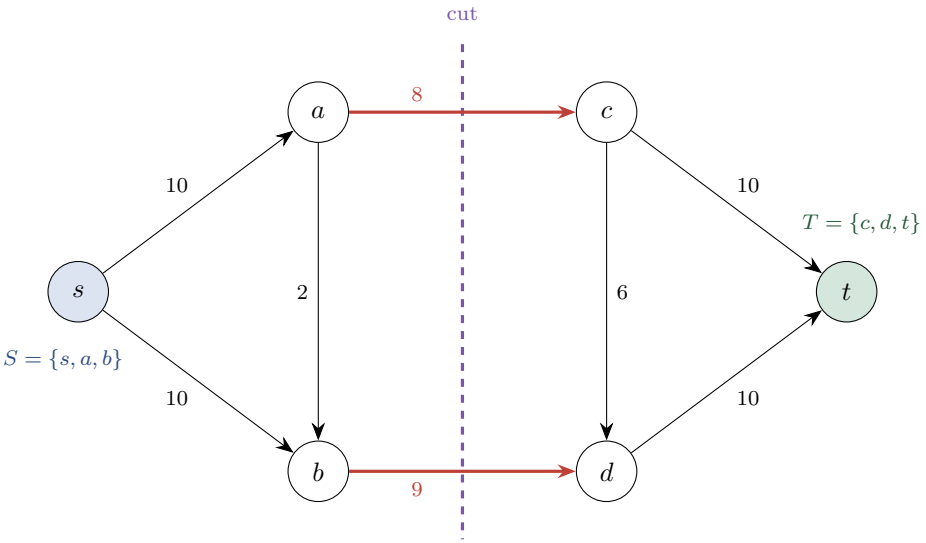


Figure 26.2: The minimum cut of the running network. The dashed purple line separates $S = \{s, a, b\}$ from $T = \{c, d, t\}$. Only the two thick red edges $a \to c$ and $b \to d$ cross forward from S to T, so the cut capacity is $8 + 9 = 17$. By max-flow min-cut this equals the maximum flow value – the network's true bottleneck lies at this downstream waist, not at the source's two pipes of capacity 10.

Theorem 26.1 (Max-Flow Min-Cut)**.** *For a flow f in a network G with source s and sink t, the following three statements are equivalent:*

1. *f is a maximum flow in G.*
2. *The residual network G_f contains no augmenting path from s to t.*
3. *$|f| = c(S,T)$ for some s-t cut (S,T).*

Consequently the value of a maximum flow equals the capacity of a minimum cut.

The three-way cycle. We show (1) ⇒ (2) ⇒ (3) ⇒ (1).

(1) ⇒ (2). Contrapositive. If G_f *has* an augmenting path p, then by Lemma 26.1 we can push $c_f(p) > 0$ more flow, so f was not maximum.

(2) ⇒ (3). This is the key step. Suppose G_f has no s-t path. Define

$$S = \{\, v \in V : \text{there is a path from } s \text{ to } v \text{ in } G_f \,\}, \qquad T = V \setminus S.$$

Then $s \in S$ (since s connects to itself by an empty path) and $t \in T$ (because no path reaches it in G_f), so (S,T) is a genuine s-t cut. Now look at any real edge (u,v) with $u \in S$, $v \in T$. If it had

spare capacity, $c_f(u,v) = c(u,v) - f(u,v) > 0$, then v would be reachable from s in G_f, contradicting $v \in T$. So every such edge is *saturated*: $f(u,v) = c(u,v)$. Symmetrically, any real edge (v,u) with $v \in T$, $u \in S$ must carry *zero* flow, for otherwise the backward residual edge (u,v) would have $c_f(u,v) = f(v,u) > 0$ and again reach into T. Plug these into the cut identity:

$$|f| = f(S,T) = \underbrace{\sum_{u\in S, v\in T} f(u,v)}_{=\sum c(u,v) = c(S,T)} - \underbrace{\sum_{u\in S, v\in T} f(v,u)}_{=0} = c(S,T).$$

(3) $\Rightarrow$ (1). If $|f| = c(S,T)$ for some cut, then by weak duality every flow f' satisfies $|f'| \le c(S,T) = |f|$, so f is maximum. The final sentence follows because the maximum flow value equals this particular cut capacity, and no cut can be cheaper (weak duality again), so it is the minimum cut. □

Focus

The proof gives more than the theorem: once Ford–Fulkerson halts, the set S of vertices reachable from s in the final residual network *is* a minimum cut. So the same run that computes the maximum flow also hands you the bottleneck – the exact edges you would need to upgrade (or sabotage) to change the answer. "Run a max-flow, then read off the reachable set" is a standard technique.

History and connection

The maximum flow problem was formulated in 1954 by T. E. Harris and F. S. Ross at the RAND Corporation to evaluate transportation bottleneck capacity on rail networks. In 1956, Lester R. Ford Jr. and Delbert R. Fulkerson established the augmenting path method and proved the max-flow min-cut theorem. Independently in that same year, Peter Elias, Amiel Feinstein, and Claude Shannon published an equivalent max-flow min-cut theorem in the context of information theory and communication channels. In 1970, Ukrainian mathematician Yefim Dinitz (and independently Jack Edmonds and Richard Karp in 1972) turned Ford and Fulkerson's general method into a polynomial-time algorithm: Edmonds and Karp showed that selecting augmenting paths via breadth-first search guarantees termination in $O(VE^2)$ operations, while Dinitz introduced layered networks and blocking flows to achieve $O(V^2E)$ time.

Push-relabel methods later emerged as an alternative to augmenting paths. In 1986, Andrew V. Goldberg and Robert E. Tarjan abandoned augmenting paths entirely, introducing preflows that allow intermediate nodes to violate flow conservation temporarily. Their push-relabel algorithms achieve $O(V^3)$ or $O(V^2\sqrt{E})$ time and often run much faster in practice on dense networks.

Max-flow min-cut is a central result in combinatorial optimization: it provides constructive proofs for Hall's marriage theorem, Menger's graph connectivity theorems, and Kőnig's bipartite matching theorem, and serves as an archetype of strong duality in linear programming. For over fifty years, the question of whether maximum flow could be solved in nearly linear time $O(m^{1+o(1)})$ remained open. In 2022, Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva resolved this problem affirmatively, presenting an almost-linear time algorithm running in $O(m^{1+o(1)})$ time using interior-point methods and continuous optimization techniques. Modern research focuses on finding

purely combinatorial near-linear algorithms and developing dynamic flow algorithms that handle continuous edge capacity modifications. In CLRS, this is Chapter 24 in the 4th edition (Chapter 26 in the 3rd edition), covering flow networks (§24.1), the Ford–Fulkerson method (§24.2), and bipartite matching (§24.3).

26.3.1 Why "method" and not "algorithm": termination

We have been careful to call Ford–Fulkerson a *method*, and now we can say why precisely. If all capacities are integers, then every residual capacity stays an integer, every bottleneck $c_f(p)$ is a positive integer, and so each augmentation increases $|f|$ by at least 1. The value can rise at most up to $|f^*| = c(S, T)$ for the min cut, so the method halts after at most $|f^*|$ augmentations. Good – it terminates, and as a bonus we get:

Corollary 26.2 (Integrality theorem)**.** *If every capacity is an integer, then some maximum flow is integer-valued (every $f(u, v)$ is an integer). Ford–Fulkerson, started from $f = 0$, produces such a flow.*

Integrality connects flows to combinatorics. It is why a problem about "half a matching" never arises – if capacities are 1, the flow on every edge is forced to be 0 or 1, which is precisely a yes/no decision. We exploit this directly in Section 26.5.

Common trap

With *irrational* capacities, the bad choice of augmenting paths can make Ford–Fulkerson run forever – and worse, converge to a flow value strictly *below* the true maximum. There is a classic five-vertex example using the golden ratio $\phi = (\sqrt{5} - 1)/2$ where naive path selection produces an infinite sequence of ever-smaller augmentations summing to a value well short of optimal. The lesson is that the path choice matters. Choosing *shortest* augmenting paths – the Edmonds–Karp rule – cures this completely, regardless of the capacities.

26.4 Edmonds–Karp: Choosing Good Paths

The fix requires changing only how each augmenting path is chosen. When several augmenting paths exist, always take a *shortest* one – fewest edges – which you find with a breadth-first search from s in G_f. That single rule turns the method into a polynomial-time algorithm whose running time does not depend on the capacity values at all.

Algorithm 120: EDMONDS-KARP(G, s, t)

Input: flow network G, source s, sink t
Output: a maximum flow f

```
f ← 0 on every edge;
while BFS from s in G_f reaches t do
    let p be the BFS tree path s ⇝ t (fewest edges);
    c_f(p) ← min{c_f(u, v) : (u, v) ∈ p};
    augment f along p by c_f(p) ;                    // as in Ford--Fulkerson
return f;
```

Why does "shortest path" help? The key is a monotonicity fact about BFS distances in the residual network. Write $\delta_f(s, v)$ for the *breadth-first distance* (number of edges) from s to v in G_f.

Lemma 26.3 (Distances never decrease)**.** *Running Edmonds–Karp, for every vertex $v \neq s, t$ the distance $\delta_f(s, v)$ in the residual network is monotonically nondecreasing across augmentations.*

The proof (we sketch it rather than belabor it) is a tidy argument by contradiction: suppose some augmentation strictly decreased $\delta(s, v)$, and take the v closest to s for which this happens. Augmenting along a *shortest* path can only *add* residual edges that go "backward" one BFS level, never a shortcut that jumps forward, so no distance can drop – contradiction. The discipline of always using a shortest path is exactly what forbids the creation of shortcuts (Figure 26.3).

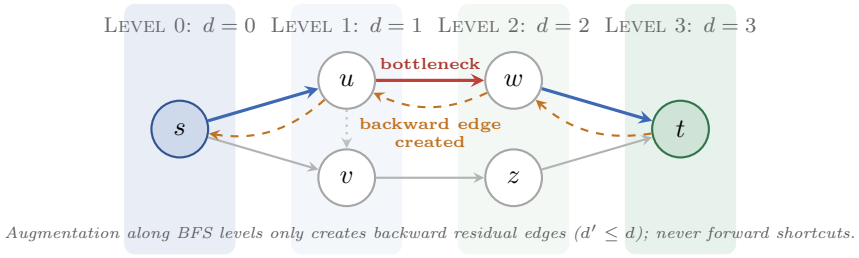


Figure 26.3: The BFS level graph structure in the Edmonds–Karp algorithm. Augmenting along a shortest path ($s \to u \to w \to t$) strictly traverses adjacent levels from left to right. Any new residual edges created by this flow (dashed orange) point backward to previous levels ($w \to u$), proving that BFS distances $\delta_f(s, \cdot)$ can never decrease across augmentations.

Lemma 26.4 (At most $O(VE)$ augmentations)**.** *Edmonds–Karp performs $O(|V||E|)$ augmentations before halting.*

The critical-edge counting argument. Call an edge (u, v) *critical* on an augmentation if it is the bottleneck, i.e. $c_f(u, v) = c_f(p)$; this edge disappears from G_f (its residual capacity drops to 0). On a shortest augmenting path through (u, v) we have $\delta_f(s, v) = \delta_f(s, u) + 1$. For (u, v) to become critical *again* later, the flow on it must first increase from 0, which happens only when (u, v) lies on an augmenting path in the other direction, i.e. when (v, u) is used; at that moment $\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1$. Because distances never decrease (Lemma 26.3), between the two times (u, v) is critical, $\delta(s, u)$ must have grown by at least 2. Since δ ranges in $\{0, 1, \ldots, |V|\}$ (or is ∞), each edge can be critical at most $|V|/2 = O(|V|)$ times. There are $O(|E|)$ residual edges, so the total number of critical events – and hence augmentations, since each augmentation has at least one critical edge – is $O(|V||E|)$. □

Each augmentation costs one BFS, which is $O(|V| + |E|) = O(|E|)$ on a connected network. Multiply.

Theorem 26.2 (Edmonds–Karp running time)**.** *Edmonds–Karp computes a maximum flow in $O(|V||E|^2)$ time, regardless of the edge capacities.*

Cost summary

Ford–Fulkerson (any augmenting path, integer capacities): up to $|f^*|$ augmentations, each $O(E)$, giving $O(E \cdot |f^*|)$. This is *pseudo-polynomial* – it depends on the numeric value of the answer, so a single capacity of 1,000,000 can make it crawl.

Edmonds–Karp (shortest augmenting path via BFS): $O(VE^2)$, independent of capacities. For reference, more advanced algorithms beat this: Dinic's algorithm runs in $O(V^2E)$; push–relabel methods reach $O(V^2\sqrt{E})$ or $O(V^3)$; and modern results (Orlin; and the 2022 almost-linear-time breakthrough) push the frontier far lower. Edmonds–Karp is the right first algorithm to *know cold*, because it is Ford–Fulkerson plus one line: "use BFS."

Algorithm lab

Re-run the running network of Section 26.1.1 *by hand* under the Edmonds–Karp rule, and keep a table.

step	shortest path p	bottleneck $c_f(p)$	$\lvert f\rvert$ after
0	—	—	0
1	$s \to a \to c \to t$	$\min(10, 8, 10) = 8$	8
2	$s \to b \to d \to t$	$\min(10, 9, 10) = 9$	17
3	no s-t path in G_f	—	17 (max)

Now do the verification the theorem promises: after step 2, run a BFS from s in G_f and list which vertices you can reach. You should reach $\{s, a, b\}$ and no further ($a \to c$ and $b \to d$ are saturated, blocking the way). That set is S; the cut (S, T) has capacity $c(a, c) + c(b, d) = 8 + 9 = 17 = |f|$. You have just *certified* optimality with a min cut, by hand. Try inventing a third path that increases the flow; convince yourself it is impossible.

26.5 Application: Maximum Bipartite Matching

We now solve a problem that has nothing to do with water – matchmaking – by building a flow network and pushing ones and zeros through it.

A *bipartite graph* $G = (L \cup R, E)$ has its vertices split into two sides L and R, with every edge joining a vertex of L to a vertex of R. Think of L as job applicants and R as positions, with an edge (ℓ, r) meaning "applicant ℓ is qualified for position r." A *matching* is a set of edges $M \subseteq E$ no two of which share a vertex: each applicant gets at most one job, each job at most one applicant. The *maximum matching* problem asks for a matching of greatest size. How many people can be hired at once?

Intuition

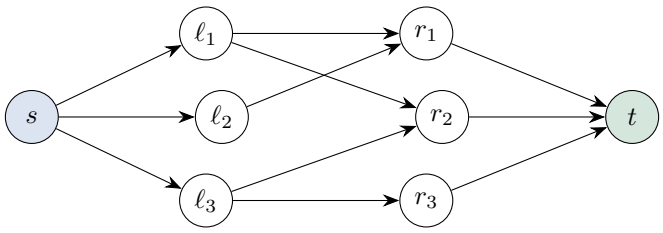
A matching is just a pairing-off with no double-booking. The trouble is local greed: hire applicant 1 for the only job she shares with applicant 2, and you may strand applicant 2 with nothing – when a smarter pairing could employ both. Here too, an early greedy choice can block a larger global matching, and backward residual edges provide the exact mechanism needed to reassign earlier pairs.

26.5.1 The reduction

Build a flow network $G' = (V', E')$ from the bipartite graph as follows.

- Add a new source s and sink t.
- Add an edge $s \to \ell$ of capacity 1 for every $\ell \in L$.
- Add an edge $r \to t$ of capacity 1 for every $r \in R$.
- For every original edge $(\ell, r) \in E$, add a directed edge $\ell \to r$ of capacity 1 (any capacity ≥ 1 works; we use 1).

Every capacity is 1. Here is the picture, with three applicants and three jobs.



Theorem 26.3 (Matching equals integer flow)**.** *The maximum number of edges in a matching of G equals the value of a maximum flow in G'. Given an integer maximum flow f, the set of middle edges $\ell \to r$ carrying flow 1 is also a maximum matching.*

Why the correspondence works, both directions. Matching → flow. Given a matching M, route one unit along $s \to \ell \to r \to t$ for each edge $(\ell, r) \in M$. Because M has no two edges sharing an ℓ, each $s \to \ell$ carries at most 1; because no two share an r, each $r \to t$ carries at most 1; capacities are respected and conservation holds. This is a valid flow of value $|M|$.

Flow → matching. By the integrality theorem (Corollary 26.2), a maximum flow may be taken integer-valued, so every middle edge carries 0 or 1. Let M be the middle edges carrying 1. Each ℓ has only one incoming unit (from s, capacity 1), so by conservation it sends out at most one unit – ℓ touches at most one matched edge. Each r sends only one unit to t, so receives at most one – r touches at most one matched edge. Thus M is a matching, and $|M| = |f|$ because every unit out of s passes through exactly one middle edge.

Since the two constructions preserve value and are inverse in size, the maxima coincide. □

So to find a maximum matching, build G' and run Edmonds–Karp. Because every capacity is 1, the analysis even sharpens: the flow value is at most $\min(|L|, |R|) \leq |V|$, so Ford–Fulkerson alone makes at most $|V|$ augmentations, each an $O(E)$ search, giving a clean $O(VE)$ bound for bipartite matching.

26.5.2 Tracing the matching

Run it on the three-by-three example. First augmenting path $s \to \ell_1 \to r_1 \to t$, push 1: ℓ_1 matched to r_1. Second path $s \to \ell_3 \to r_2 \to t$, push 1: ℓ_3 matched to r_2. Now try ℓ_2: from $s \to \ell_2$ we reach

r_1, but $r_1 \to t$ is saturated. Dead end? No – the residual network offers the backward edge $r_1 \to \ell_1$ (because $f(\ell_1, r_1) = 1$). The BFS finds

$$s \to \ell_2 \to r_1 \xrightarrow{\text{backward}} \ell_1 \to r_2 \xrightarrow{\text{backward}} \ell_3 \to r_3 \to t.$$

Pushing 1 unit along it *re-routes* the earlier matches: ℓ_1 drops r_1 and takes r_2; ℓ_3 drops r_2 and takes r_3; and ℓ_2 grabs the freed r_1. Final flow value 3 – a *perfect* matching $\{\ell_1 r_2, \ell_2 r_1, \ell_3 r_3\}$ that naive greed would have missed. Figure 26.4 traces that zig-zag.

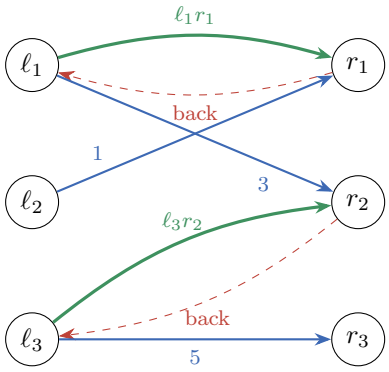


Figure 26.4: The augmenting (alternating) path that grows the matching from 2 to 3. Thick green edges are the current matches $\ell_1 r_1$ and $\ell_3 r_2$. The path enters along blue "add" edges ($\ell_2 r_1$, $\ell_1 r_2$, $\ell_3 r_3$, numbered in traversal order) and crosses two dashed red backward edges that *undo* the old matches. Flipping every edge on the path turns the two matches into three: $\{\ell_1 r_2,\ \ell_2 r_1,\ \ell_3 r_3\}$.

Focus

That backward-edge zig-zag has a classical name in matching theory: an *augmenting path* for matchings – a path that alternates between unmatched and matched edges, starting and ending at unmatched vertices. Flipping it increases the matching by one. The flow algorithm rediscovers this idea for free: matching's augmenting paths are exactly s-t paths in the residual network.

Why this matters

This reduction is a main reason maximum flow is taught in algorithms courses. The same approach handles assigning taxis to riders, scheduling exams to rooms, detecting whether a sports team is mathematically eliminated, segmenting medical images (foreground/background as a min cut), and the entire theory of bipartite assignment that underlies kidney-exchange matching programs and ad auctions. The pattern – "model the decisions as edges of capacity 1, route flow, read the integer answer" – is one of the most reusable tools in applied computing.

26.6 What to Carry Forward

Pause

Five sentences to keep:

1. A flow obeys *capacity* on every edge and *conservation* at every interior vertex; its *value* is the net amount leaving s.
2. The *residual network* adds backward edges that grant permission to cancel earlier flow; this is what lets a greedy method become exact.
3. Ford–Fulkerson repeatedly pushes the bottleneck of an augmenting path; with integer capacities it terminates and yields an integer flow.
4. *Max-flow equals min-cut*: when no augmenting path remains, the set reachable from s in G_f is a minimum cut certifying optimality.
5. *Edmonds–Karp* = "always use the shortest augmenting path (BFS)," giving $O(VE^2)$ time, independent of capacities; bipartite matching reduces to a unit-capacity flow.

With maximum flow and minimum cuts, we conclude Part 6 on Graph Algorithms. In Part 7, we turn to numerical and algebraic algorithms, beginning in Chapter 27 with matrix operations: solving linear systems, matrix inversion, and LUP decomposition.

Exercises

Exercise 26.1 (Warmup: read a flow)**.** In the running network of Section 26.1.1, suppose someone hands you the assignment $f(s,a) = 8$, $f(a,c) = 8$, $f(c,t) = 8$, $f(s,b) = 9$, $f(b,d) = 9$, $f(d,t) = 9$, and 0 on the remaining edges. Verify that it satisfies the capacity constraint and flow conservation at a, b, c, d, and compute $|f|$.

Exercise 26.2 (Build a residual network)**.** For the flow in Exercise 26.1, draw the residual network G_f: list every residual edge and its residual capacity (remember both forward and backward edges). Then argue from your drawing that no s-t path exists.

Exercise 26.3 (Find the min cut)**.** Continuing the previous exercise, compute the set S of vertices reachable from s in G_f, write down the cut (S, T), and verify that its capacity equals $|f|$. Which two edges form the bottleneck?

Exercise 26.4 (Greedy can fail)**.** Recall the five-vertex network of Section 26.2 (the s, u, v, t diamond with the middle edge $u \to v$, all capacities 1). Trace Ford–Fulkerson starting with the bad first path $s \to u \to v \to t$. Show explicitly the augmenting path through the backward residual edge that recovers the optimal flow of value 2, and give the resulting flow on each real edge.

Exercise 26.5 (Why no antiparallel edges)**.** Suppose a network genuinely has both (u, v) and (v, u) as edges with positive capacities. Explain in one or two sentences why the single-formula definition of c_f in Section 26.2.2 would become ambiguous. Then describe the fix (split one edge with a dummy vertex w: replace (u, v) by $u \to w \to v$, each of capacity $c(u, v)$) and argue it does not change the maximum flow value.

Exercise 26.6 (Integrality is essential for matching)**.** Show by a small example that if you allowed *fractional* flow on the middle edges of the bipartite construction, a maximum flow could split a unit of an applicant across two jobs. Explain why Corollary 26.2 guarantees we can always avoid this, so the flow really does encode a valid matching.

Exercise 26.7 (Counting augmentations)**.** Construct a network on which *naive* Ford–Fulkerson, making the worst possible path choices, performs about 2,000,000 augmentations, while Edmonds–Karp performs only a handful. *(Hint: the classic "diamond" with two outer edges of capacity* 1,000,000 *and a middle edge of capacity* 1*; alternately augment through the middle.)* What does this say about the difference between pseudo-polynomial and polynomial time?

Exercise 26.8 (Vertex capacities)**.** Some problems enforce throughput limits on vertices in addition to edges: at most $\mathrm{cap}(v)$ units may pass through v. Show how to model this with ordinary edge capacities by splitting v into v_{in} and v_{out} joined by an edge of capacity $\mathrm{cap}(v)$, routing all in-edges to v_{in} and all out-edges from v_{out}. Argue correctness.

Exercise 26.9 (Multiple sources and sinks)**.** A factory network has three sources s_1, s_2, s_3 (each able to produce any amount) and two sinks t_1, t_2. Explain how to compute the maximum total flow from all sources to all sinks by adding a single *super-source* and a single *super-sink* with appropriate infinite-capacity edges, reducing to the single-source single-sink problem of this chapter.

Exercise 26.10 (Edge-disjoint paths – a challenge)**.** Menger's theorem states that the maximum number of *edge-disjoint* paths from s to t in a directed graph equals the minimum number of edges whose removal disconnects t from s. Show how to prove this using max-flow min-cut: assign every edge capacity 1, and interpret an integer flow of value k as k edge-disjoint paths. Where exactly does the integrality theorem enter the argument?

Exercise 26.11 (Uniqueness – a challenge)**.** The *value* of the maximum flow is unique, but the flow itself and the minimum cut need not be. Construct a small network that has (a) two different maximum flows of the same value, and (b) two different minimum cuts of the same capacity. Then prove the general fact that if (S_1, T_1) and (S_2, T_2) are both minimum cuts, so are $(S_1 \cap S_2,\, \cdot)$ and $(S_1 \cup S_2,\, \cdot)$ – the minimum cuts form a lattice.

Part VII

Selected Topics

Chapter 27

Matrix Operations

In Part 6, we used adjacency matrices and distance tables to capture network connectivity. Here in Part 7, we shift from combinatorial graph traversals to numerical and algebraic computation.

Somewhere in your past a teacher drew a grid of numbers, called it a "matrix," and made you multiply two of them by a procedure that felt like a secret handshake: go across one, down the other, add up the products. You did it, but you may never have paused to ask the deeper question: *why is multiplication defined this way, and what geometric transformation does it represent?*

This chapter answers both. We will rebuild matrices as a tool for computation. A matrix, it turns out, is the natural language for talking about *many things changing at once*: a thousand linear equations, a rotation in space, a step of a neural network, the flow of populations between cities. The operations we study — multiply, decompose, solve, invert — are the verbs of that language, and they are central to scientific computing, graphics, statistics, optimization, and machine learning.

Focus

Matrix operations translate systems of linear equations and geometric transformations into fast algorithms. Through LUP decomposition, triangular substitution, Strassen's subcubic multiplication, and matrix inversion, we solve linear systems in $O(n^3)$ time and demonstrate that matrix inversion is asymptotically equivalent to matrix multiplication.

27.1 Matrices from the ground up

A *matrix* is a rectangular array of numbers. We write A for the whole array and a_{ij} for the entry in row i, column j. A matrix with m rows and n columns is an "$m \times n$ matrix" (always rows first, columns second — say it out loud a few times so the order sticks). A matrix with one column is a *column vector*; one row, a *row vector*. When $m = n$ we call the matrix *square*.

$$A = \begin{pmatrix} 2 & 0 & 1 \\ 3 & 5 & 4 \end{pmatrix} \quad \text{is } 2 \times 3, \qquad a_{13} = 1, \quad a_{21} = 3.$$

Two matrices of the same shape *add* entrywise: $(A+B)_{ij} = a_{ij} + b_{ij}$. You scale a matrix by a number c entrywise too: $(cA)_{ij} = c\,a_{ij}$. Both cost $\Theta(mn)$ time — one operation per entry, no surprises. The

interesting verb is multiplication. (See Appendix 42 for further foundations on vector spaces and matrix properties.)

27.1.1 Why multiplication is defined the way it is

One idea explains the "secret handshake." Think of an $m \times n$ matrix not just as a static grid of numbers, but as a linear map that takes an input vector of length n and produces an output vector of length m. Feed it the vector $x = (x_1, \dots, x_n)$; out comes $y = Ax$, where

$$y_i = \sum_{k=1}^{n} a_{ik}\, x_k.$$

Row i of A is a recipe: "to make output i, take a_{i1} of the first input plus a_{i2} of the second, and so on." This is exactly how a system of linear equations looks, and it is exactly how a layer of a neural network mixes its inputs. A matrix *is* a linear transformation written down in coordinates.

Now suppose you have two machines. Machine B ($n \times p$) turns a length-p vector into a length-n vector; machine A ($m \times n$) turns that into a length-m vector. Chain them: first B, then A. The combined machine takes length p to length m. *What single matrix represents the chained machine?* Call it $C = AB$. Push a vector x through: $z = Bx$ has entries $z_k = \sum_j b_{kj} x_j$, and then $y = Az$ has entries

$$y_i = \sum_k a_{ik} z_k = \sum_k a_{ik} \sum_j b_{kj} x_j = \sum_j \Big(\underbrace{\sum_k a_{ik} b_{kj}}_{c_{ij}} \Big) x_j.$$

The thing in the parentheses must be c_{ij}, because $y = Cx$ means $y_i = \sum_j c_{ij} x_j$. So the definition is forced on us:

$$\boxed{c_{ij} = \sum_{k=1}^{n} a_{ik}\, b_{kj}}$$

Intuition

Matrix multiplication is *composition of machines*. The reason you go "across a row of A and down a column of B" is that c_{ij} collects every path from input j, through some middle coordinate k, to output i: the coefficient b_{kj} on the way in, the coefficient a_{ik} on the way out, multiplied, then summed over all middles k. This is the only rule that makes "do B, then do A" equal "do AB."

Two shapes must match for the product to exist: the *inner* dimensions. An $m \times n$ times an $n \times p$ gives an $m \times p$. The n's must agree because the middle coordinate k runs over them. Figure 27.1 shows which slice of each factor produces a single output entry.

Example 27.1 (A product, traced by hand)**.** Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Both are 2×2, so $C = AB$ is 2×2. Compute each entry by "row of A dotted with column of B":

$$c_{11} = 1 \cdot 5 + 2 \cdot 7 = 5 + 14 = 19, \qquad c_{12} = 1 \cdot 6 + 2 \cdot 8 = 6 + 16 = 22,$$
$$c_{21} = 3 \cdot 5 + 4 \cdot 7 = 15 + 28 = 43, \qquad c_{22} = 3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50.$$

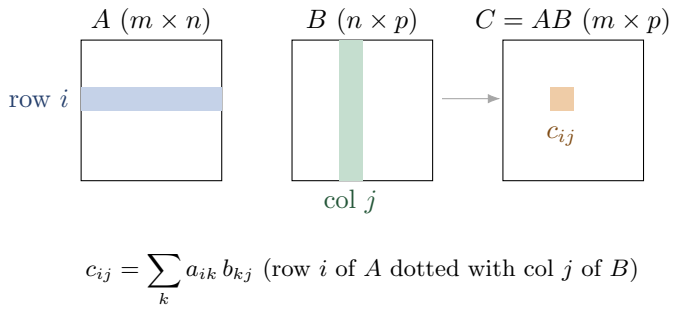


Figure 27.1: The entry c_{ij} of the product gathers row i of A against column j of B. The shared length n is the "inner" dimension both factors must agree on.

So $C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$. Notice BA is *different*: $BA = \begin{pmatrix} 23 & 34 \\ 31 & 46 \end{pmatrix}$. Matrix multiplication does *not* commute — of course not: putting on socks then shoes is not the same as shoes then socks.

Common trap

Three traps that bite everyone at least once. **(1) Order matters.** $AB \neq BA$ in general; you may not silently swap. **(2) Shapes must line up.** You can multiply A $(m \times n)$ by B $(n \times p)$ only when A's column count equals B's row count. "2×3 times 2×3" is undefined. **(3) Indices.** c_{ij} uses row i of A and column j of B. Going "down a column of A" by accident is the single most common bug in handwritten matrix code.

A few special matrices earn names. The *identity* I_n has 1's on the diagonal and 0's elsewhere; it is the "do nothing" machine, so $AI = IA = A$. The *transpose* A^{T} flips rows and columns: $(A^{\mathsf{T}})_{ij} = a_{ji}$. A matrix is *symmetric* if $A = A^{\mathsf{T}}$. A *diagonal* matrix is zero off the diagonal; *lower triangular* means zero above the diagonal, *upper triangular* means zero below. Keep triangular matrices in mind — the second half of this chapter relies on them, because systems involving them can be solved directly by single-pass substitution.

27.1.2 The cost of the schoolbook product

How long does it take to multiply two $n \times n$ matrices by the definition? The output has n^2 entries. Each entry is a sum of n products. So we do $n^2 \cdot n = n^3$ scalar multiplications and about as many additions.

Algorithm 121: SQUARE-MATRIX-MULTIPLY(A, B)

Input: Two $n \times n$ matrices A and B.
Output: Their product $C = AB$.

```
let C be a new n × n matrix;
for i ← 1 to n do
    for j ← 1 to n do
        c_ij ← 0;
        for k ← 1 to n do
            c_ij ← c_ij + a_ik · b_kj;
return C;
```

Three nested loops, each running n times, with constant work inside: the running time is $\Theta(n^3)$. For decades people assumed that was the end of the story — surely you cannot multiply two matrices without touching every pair of entries the cubic number of times. In 1969 Volker Strassen proved that assumption wrong, and we turn to his trick next.

Cost summary

Schoolbook $n \times n$ matrix multiply. Scalar multiplications: n^3. Additions: $n^3 - n^2$. Time: $\Theta(n^3)$. Extra space: $\Theta(n^2)$ for the output (the inputs are read-only). Simple, cache- friendly, and for small or medium n it is genuinely the fastest thing you can do.

27.2 Strassen's algorithm, revisited

You met divide-and-conquer earlier in the book. Applied to matrix multiplication, it gives a surprising result.

27.2.1 A first attempt: blocking

Split each $n \times n$ matrix into four $n/2 \times n/2$ *blocks* (assume n is a power of two so the halving stays clean):

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

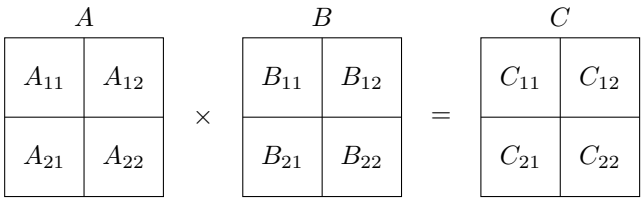


Figure 27.2: Each $n \times n$ matrix is split into four $n/2 \times n/2$ blocks. The blocks then multiply by the same "rows times columns" rule as scalar entries, e.g. $C_{11} = A_{11}B_{11} + A_{12}B_{21}$.

Blocks multiply by the same rule as scalars, as long as you keep the order straight (see Figure 27.2). So

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22},$$
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}, \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}.$$

Each of the four output blocks is a sum of two products of half-size matrices: 8 multiplications of $n/2 \times n/2$ matrices, plus a handful of $n/2 \times n/2$ additions (each costing $\Theta(n^2)$). That gives the recurrence

$$T(n) = 8\,T(n/2) + \Theta(n^2).$$

By the master theorem (here $a = 8$, $b = 2$, so $n^{\log_b a} = n^{\log_2 8} = n^3$, and the $\Theta(n^2)$ combine is polynomially smaller), $T(n) = \Theta(n^3)$. We split the work cleverly and *got nothing*: still cubic. The eight multiplications are the problem. Strassen's insight is to do the same job with *seven*.

27.2.2 Seven products instead of eight

Strassen discovered that you can compute the four output blocks using only *seven* multiplications of half-size matrices, at the cost of a few extra additions and subtractions. The additions are cheap ($\Theta(n^2)$); the multiplications are the expensive recursive calls, so trading one multiplication for several additions is a clear win. First form ten sums:

$$S_1 = B_{12} - B_{22}, \quad S_2 = A_{11} + A_{12}, \quad S_3 = A_{21} + A_{22},$$
$$S_4 = B_{21} - B_{11}, \quad S_5 = A_{11} + A_{22}, \quad S_6 = B_{11} + B_{22},$$
$$S_7 = A_{12} - A_{22}, \quad S_8 = B_{21} + B_{22}, \quad S_9 = A_{11} - A_{21},$$
$$S_{10} = B_{11} + B_{12}.$$

Then the seven products (these are the recursive multiplications):

$$P_1 = A_{11}\,S_1, \quad P_2 = S_2\,B_{22}, \quad P_3 = S_3\,B_{11}, \quad P_4 = A_{22}\,S_4,$$
$$P_5 = S_5\,S_6, \quad P_6 = S_7\,S_8, \quad P_7 = S_9\,S_{10}.$$

Finally assemble the answer with additions and subtractions only:

$$C_{11} = P_5 + P_4 - P_2 + P_6, \quad C_{12} = P_1 + P_2,$$
$$C_{21} = P_3 + P_4, \quad C_{22} = P_5 + P_1 - P_3 - P_7.$$

Intuition

Do not try to *memorize* those formulas, and do not expect them to be immediately apparent — they are not. The structural point is that by allowing yourself to add and subtract input blocks *before* multiplying, you can arrange for the products to overlap so much that seven of them carry all the information eight used to. It is the same spirit as the fast Fourier transform: spend cheap operations (adds) to save expensive ones (multiplies).

Let us check one entry explicitly to see how the cancellation works.

Example 27.2 (Verifying $C_{12} = P_1 + P_2$)**.** $P_1 = A_{11}S_1 = A_{11}(B_{12} - B_{22}) = A_{11}B_{12} - A_{11}B_{22}$, and $P_2 = S_2B_{22} = (A_{11} + A_{12})B_{22} = A_{11}B_{22} + A_{12}B_{22}$. Adding,

$$P_1 + P_2 = A_{11}B_{12} - A_{11}B_{22} + A_{11}B_{22} + A_{12}B_{22} = A_{11}B_{12} + A_{12}B_{22} = C_{12}.$$

The two $A_{11}B_{22}$ terms cancel exactly. Strassen's formulas work this way throughout: a term you do not want is created in one product and cancelled in another.

27.2.3 Why seven beats eight: the recurrence

The recurrence is now

$$T(n) = 7\,T(n/2) + \Theta(n^2).$$

Here $a = 7$, $b = 2$, so $n^{\log_b a} = n^{\log_2 7}$. Since $\log_2 7 \approx 2.807 > 2$, the recursive term dominates the $\Theta(n^2)$ combine, and the master theorem gives

$$T(n) = \Theta\big(n^{\log_2 7}\big) = \Theta\big(n^{2.807\ldots}\big).$$

That single dropped multiplication moved the exponent from 3.000 down to about 2.807. The change looks small, but for $n = 1024$, $n^3 \approx 1.07 \times 10^9$ while $n^{2.807} \approx 2.0 \times 10^8$, roughly a fivefold reduction, and the gap widens without bound as n grows.

Algorithm lab

Trace the recursion tree. At depth d from the top there are 7^d subproblems, each of size $n/2^d$, and each does $\Theta((n/2^d)^2)$ addition work. So level d costs $7^d \cdot c\,(n/2^d)^2 = c\,n^2(7/4)^d$. The levels form a geometric series with ratio $7/4 > 1$, so the *bottom* levels dominate. There are $\log_2 n$ levels, and the last level's cost is $c\,n^2(7/4)^{\log_2 n} = c\,n^{\log_2 7}$. Sum the geometric series and the total is $\Theta(n^{\log_2 7})$. Draw this tree on paper for $n = 8$: a root, 7 children, 49 grandchildren, and watch the per-level cost climb by $7/4$ each step.

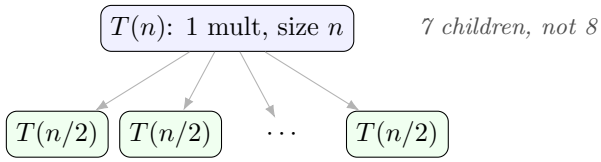


Figure 27.3: One level of Strassen's recursion: *seven* subproblems of half the size. Eight would give $\Theta(n^3)$; seven gives $\Theta(n^{2.807})$.

History and connection

The computational study of matrices traces back to Carl Friedrich Gauss, who around 1801 used systematic row elimination to compute the celestial orbit of the asteroid Ceres from sparse astronomical observations. In 1948, Alan Turing formalized matrix decompositions and analyzed the propagation of roundoff errors in Gaussian elimination on digital computers. For decades, it was widely presumed that multiplying two $n \times n$ matrices inherently required $\Theta(n^3)$ operations. In 1969, Volker Strassen disproved this assumption in his paper "Gaussian Elimination is not Optimal," presenting a clever decomposition that multiplies 2×2 block matrices using only 7 multiplications instead of 8, lowering the asymptotic complexity to $O(n^{\log_2 7}) \approx O(n^{2.807})$.

Strassen's discovery triggered intense debate between numerical analysts and theoreticians. Practical implementers observed that Strassen's algorithm exhibits less numerical stability than standard elimination under finite-precision floating-point arithmetic and incurs significant memory allocation overhead. Consequently, modern high-performance libraries (such as BLAS and LAPACK) continue to rely on block-partitioned Gaussian elimination with partial pivoting for moderate matrix dimensions.

On the theoretical side, Strassen's work led to the result that matrix multiplication, matrix inversion, and determinant computation are computationally equivalent, all bounded by the matrix multiplication exponent ω. Don Coppersmith and Shmuel Winograd achieved $O(n^{2.376})$ in 1990 using the laser method, and ongoing refinements by Virginia Vassilevska Williams, Josh Alman, and others have reduced the upper bound near 2.371. The fundamental open question remains whether $\omega = 2$ is achievable—whether multiplying two $n \times n$ matrices can be done in essentially $O(n^2)$ time. In 2014, Ambainis, Filmus, and Le Gall proved structural barrier limits on the laser method, indicating that reaching $\omega = 2$ will require entirely new mathematical techniques. In CLRS (4th edition), Strassen's algorithm appears in Chapter 4 and matrix operations in Chapter 28.

Common trap

Strassen is *not* a free lunch in practice. The extra additions, the recursive bookkeeping, and worse numerical stability mean that on real hardware Strassen only pays off for fairly large n (often several hundred and up), and serious libraries switch to the cache-tuned cubic algorithm once the blocks get small. "Asymptotically faster" and "faster on your laptop today" are different claims; respect the constant factors.

27.3 Solving linear systems: $Ax = b$

Now we arrive at the operation that matrices were arguably invented for: solving *simultaneous linear equations*. You want x such that $Ax = b$, where A is a known $n \times n$ matrix, b a known vector, and x the unknown vector. Spelled out, that is n equations in n unknowns:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\ &\vdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n. \end{aligned}$$

You learned to do this with Gaussian elimination: scale and subtract rows until the system is triangular, then read off the answers. We are going to repackage exactly that process into a reusable *factorization*, which is both cleaner to reason about and far more useful when you must solve many systems with the same A.

Intuition

Prerequisites: rank, determinant, and invertibility. When does a linear system $Ax = b$ have a unique solution? Linear algebra provides three equivalent perspectives:

- **Rank**: The rank of A is the maximum number of linearly independent columns (or rows). A square $n \times n$ matrix has *full rank* n precisely when no row is a redundant combination of others, meaning each equation provides genuinely independent information.
- **Determinant**: The determinant $\det(A)$ measures the signed n-dimensional volume scaling factor under the transformation $x \mapsto Ax$. If $\det(A) = 0$, the matrix squashes space into a lower dimension, destroying information and preventing a unique pre-image. If $\det(A) \neq 0$, the transformation is non-degenerate and volume is preserved with finite non-zero scaling. In an LUP decomposition, $\det(A) = (-1)^s \prod_{i=1}^{n} u_{ii}$, where s is the

number of row swaps in P and u_{ii} are the diagonal pivots of U.

- **Inverse**: If A has full rank ($\det(A) \neq 0$), there exists a unique two-sided inverse matrix A^{-1} satisfying $AA^{-1} = A^{-1}A = I_n$.

27.3.1 Triangular systems are easy

Start with the easy case, because the whole strategy is "reduce the hard case to the easy one." Suppose A is *lower triangular* (zeros above the diagonal). Then the first equation involves only x_1, so you solve it immediately. The second involves x_1 (now known) and x_2, so you solve for x_2. Marching down, each new equation introduces exactly one new unknown. This is *forward substitution*:

$$x_i = \frac{1}{\ell_{ii}}\Big(b_i - \sum_{j=1}^{i-1} \ell_{ij}\, x_j\Big).$$

For an *upper* triangular system you start from the *last* equation and march up — *back substitution*. Either way the cost is $\Theta(n^2)$: roughly one multiply-add per nonzero entry below (or above) the diagonal.

Example 27.3 (Forward substitution by hand)**.** Solve $Lx = b$ with

$$L = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ 4 & 5 & 6 \end{pmatrix}, \qquad b = \begin{pmatrix} 4 \\ 11 \\ 40 \end{pmatrix}.$$

Row 1: $2x_1 = 4 \Rightarrow x_1 = 2$. Row 2: $1 \cdot 2 + 3x_2 = 11 \Rightarrow 3x_2 = 9 \Rightarrow x_2 = 3$. Row 3: $4 \cdot 2 + 5 \cdot 3 + 6x_3 = 40 \Rightarrow 8 + 15 + 6x_3 = 40 \Rightarrow 6x_3 = 17$, so $x_3 = 17/6$. Each step uses only already-known values — no system-solving, just arithmetic.

27.3.2 LU and LUP decomposition

Here is the plan. Most matrices are not triangular, but *every* (nonsingular) matrix can be *factored* into a product of triangular matrices. If we can write

$$A = LU, \qquad L \text{ unit lower triangular, } U \text{ upper triangular,}$$

then $Ax = b$ becomes $L(Ux) = b$. Let $y = Ux$. First solve $Ly = b$ by forward substitution (easy, triangular), then solve $Ux = y$ by back substitution (easy, triangular). Two easy solves replace one hard one. "Unit" lower triangular means L has 1's on its diagonal — a normalization that makes the factorization unique.

Where does L come from? It is the *record of Gaussian elimination*. When you eliminate x_1 from row i by subtracting a multiple ℓ_{i1} of row 1, you *store* that multiplier ℓ_{i1}. Collect all the multipliers into L and the eliminated (upper-triangular) result into U, and you can check that $LU = A$. No information is lost: the multipliers in L record every step.

Intuition

LU decomposition is Gaussian elimination that *records its row multipliers*. The upper-triangular U is where elimination leaves the matrix. The lower-triangular L records, in entry ℓ_{ij}, exactly how much of row j you subtracted from row i along the way. Multiply L back into U and you recover the original A. The factorization stores the result of the algorithm so that it can be reused.

One catch makes the plain $A = LU$ recipe fragile: at some step the entry we want to divide by (the *pivot*) might be zero, or merely tiny. Dividing by a tiny pivot amplifies rounding error catastrophically. The fix is *pivoting*: before each elimination step, swap rows to bring the largest-magnitude available entry into the pivot position. Swapping rows is itself a matrix operation — multiplication by a *permutation matrix* P — and recording all the swaps gives the more robust LUP *decomposition*:

$$PA = LU.$$

Now solving $Ax = b$ becomes: permute b to get Pb, forward-solve $Ly = Pb$, back-solve $Ux = y$. Same two easy solves, just with the rows reordered first.

Focus

The LUP strategy. Factor once: $PA = LU$, costing $\Theta(n^3)$. Then *each* right-hand side b is solved for $\Theta(n^2)$ by a forward solve and a back solve. If you must solve $Ax = b$ for many different b's with the same A, you pay the cubic cost a single time and the cheap quadratic cost over and over.

Example 27.4 (A full LU factorization)**.** Factor (no pivoting needed here)

$$A = \begin{pmatrix} 2 & 4 & -2 \\ 4 & 9 & -3 \\ -2 & -3 & 7 \end{pmatrix}.$$

Step 1: eliminate column 1. Pivot is $a_{11} = 2$. Multiplier for row 2: $\ell_{21} = 4/2 = 2$; subtract $2\times$row 1 from row 2: $(4, 9, -3) - 2(2, 4, -2) = (0, 1, 1)$. Multiplier for row 3: $\ell_{31} = -2/2 = -1$; subtract $-1\times$row 1 from row 3: $(-2, -3, 7) - (-1)(2, 4, -2) = (0, 1, 5)$. The matrix is now $\begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 1 & 5 \end{pmatrix}$.

Step 2: eliminate column 2. Pivot is 1. Multiplier $\ell_{32} = 1/1 = 1$; subtract $1\times$row 2 from row 3: $(0, 1, 5) - (0, 1, 1) = (0, 0, 4)$. Now the matrix is upper triangular:

$$U = \begin{pmatrix} 2 & 4 & -2 \\ 0 & 1 & 1 \\ 0 & 0 & 4 \end{pmatrix}, \qquad L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 1 & 1 \end{pmatrix}.$$

Check: LU should rebuild A. Row 3 of LU is $(-1)(2, 4, -2) + (1)(0, 1, 1) + (1)(0, 0, 4) = (-2, -3, 7)$, the original row 3. It works.

Algorithm 122: LUP-Decomposition(A)

Input: A nonsingular $n \times n$ matrix A.
Output: A permutation π and matrices L, U (stored in A) with $PA = LU$.

```
let π[1...n] be a new array; for i ← 1 to n do
    π[i] ← i
for k ← 1 to n do
    p ← 0;
    for i ← k to n do
        if |a_ik| > p then
            p ← |a_ik|;  k' ← i;                          // find largest pivot
    if p = 0 then error "singular matrix";
    exchange π[k] ↔ π[k'] and row k ↔ row k' of A;
    for i ← k + 1 to n do
        a_ik ← a_ik / a_kk;                          // store multiplier in L's slot
        for j ← k + 1 to n do
            a_ij ← a_ij − a_ik · a_kj;                                  // eliminate
return π;
```

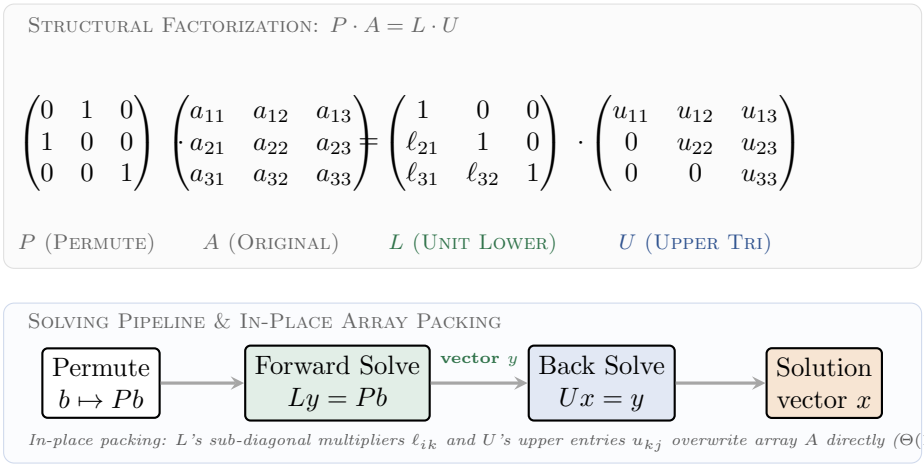


Figure 27.4: Visual structure of LUP matrix decomposition. Top: row permutation P stabilizes elimination, decomposing PA into unit lower-triangular matrix L and upper-triangular matrix U. Bottom: solving $Ax = b$ reduces to a two-step triangular pipeline (forward substitution $Ly = Pb$ followed by back substitution $Ux = y$) taking $\Theta(n^2)$ time per right-hand side b. In-place execution packs both L and U within the original array A.

A space-saving detail (Figure 27.4): L and U never overlap (off-diagonal L below, U on and above the diagonal, and L's diagonal 1's are implicit), so both fit inside the original array A. No extra $\Theta(n^2)$ storage needed.

Cost summary

LUP decomposition. The outer loop runs n times; step k does about $(n-k)^2$ multiply-adds in its double inner loop. Summing, $\sum_{k=1}^{n}(n-k)^2 \approx n^3/3$, so the time is $\Theta(n^3)$ — asymp-

totically the same as one matrix multiply, with a smaller constant. **Each solve** afterward (forward + back substitution) is $\Theta(n^2)$. Pivot search adds only $\Theta(n^2)$ total, lost in the noise.

Walking question

You need to solve $Ax = b_1$, $Ax = b_2$, …, $Ax = b_{1000}$ — a thousand right-hand sides, same A ($n = 500$). Estimate the work two ways: (a) solve each from scratch with full elimination; (b) factor $A = LU$ once, then do a thousand cheap solves. Which is faster, and by roughly what factor? (Hint: compare $1000 \cdot n^3/3$ against $n^3/3 + 1000 \cdot n^2$.) This is why linear-algebra libraries hand you a reusable *factorization object* rather than an ad-hoc solve routine.

27.3.3 Why we never compute A^{-1} to solve $Ax = b$

Beginners reach for the inverse: "$Ax = b$, so $x = A^{-1}b$ — just invert and multiply!" This is correct mathematically but a poor way to compute. Computing A^{-1} costs more than an LUP factorization, then you still must multiply $A^{-1}b$. And the inverse is numerically worse: it tends to be denser and to amplify rounding error. The factor-and-substitute route is faster *and* more accurate. Compute inverses only when you genuinely need the inverse *matrix* itself, not merely the solution to a system.

27.4 Matrix inversion and its equivalence to multiplication

The *inverse* of a square matrix A is the matrix A^{-1} with $AA^{-1} = A^{-1}A = I$. It exists exactly when A is *nonsingular* (its columns are linearly independent; equivalently $\det A \neq 0$). The inverse is the "undo" machine: if A turns x into b, then A^{-1} turns b back into x.

27.4.1 Computing the inverse with LUP

You can read A^{-1} off of solves. The j-th column of A^{-1} is the solution x to $Ax = e_j$, where e_j is the j-th standard basis vector (a 1 in position j, zeros elsewhere) — because $AA^{-1} = I$ says "A times column j of A^{-1} equals column j of I." So: factor $A = LU$ once ($\Theta(n^3)$), then solve n systems $Ax = e_1, \ldots, Ax = e_n$ (n solves at $\Theta(n^2)$ each, totaling $\Theta(n^3)$). Stack the n answer vectors as columns and you have A^{-1}. Total time: $\Theta(n^3)$.

Focus

Inversion costs the same order as multiplication: $\Theta(n^3)$ by the schoolbook route. In fact, inversion and multiplication are *equivalent* problems — if you can do either in $\Theta(n^c)$ time (for any reasonable $c \geq 2$), you can do the other in $\Theta(n^c)$ too. Speed one up and the other automatically speeds up to match.

27.4.2 The equivalence, both directions

Multiplication is no harder than inversion. Suppose you have a fast inverter. Given $n \times n$ matrices A and B, build the $3n \times 3n$ block matrix

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}.$$

This D is upper triangular with 1's on the diagonal, so it is invertible, and a short computation shows

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix}.$$

The product AB sits right there in the top-right block of D^{-1}! So one inversion of a matrix three times the size — still $\Theta(n^c)$ since the size only tripled — hands you the product. Inverting is *at least as hard* as multiplying.

Example 27.5 (Reading AB out of D^{-1})**.** Verify the middle of the claim: multiply D by the proposed D^{-1} and check you get I_{3n}. The $(1, 3)$ block of DD^{-1} is

$$I_n \cdot (AB) + A \cdot (-B) + 0 \cdot I_n = AB - AB = 0,$$

which is exactly what the identity's top-right block should be (zero). The product AB appears and then cancels precisely because we placed A and B in adjacent super-diagonal blocks. Every block of DD^{-1} checks out the same tidy way.

Inversion is no harder than multiplication. The other direction is more involved (CLRS proves it carefully for symmetric positive-definite matrices and then reduces the general case to that one), but the main result is this: a divide-and-conquer scheme inverts a matrix using a *constant number* of multiplications, inversions of half-size matrices, additions, and subtractions. Unrolling the recursion gives $I(n) = \Theta(M(n))$, where $M(n)$ is the cost of multiplying $n \times n$ matrices. Plug in Strassen's $M(n) = \Theta(n^{2.807})$ and you get an inverter that beats $\Theta(n^3)$ for free.

Formal

Theorem (equivalence of inversion and multiplication). Let $M(n)$ be the time to multiply two $n \times n$ matrices and $I(n)$ the time to invert a nonsingular $n \times n$ matrix, where both M and I are well-behaved (satisfy a mild regularity condition and are $\Omega(n^2)$). Then

$$M(n) = \Theta\big(I(n)\big).$$

Concretely: any algorithm multiplying in $\Theta(n^c)$ yields one that inverts in $\Theta(n^c)$, and vice versa. The two problems have the same complexity exponent.

Why this matters

Why care about an *equivalence*? Because it focuses the entire research program. You do not need to separately chase a fast inverter, a fast determinant, a fast linear-system solver, a fast LU factorizer — they are all tied, by reductions like the one above, to the single quantity

$M(n)$, the cost of multiplying. Improve matrix multiplication and many numerical algorithms speed up with it. That is why the exponent ω is so heavily studied in computer science.

27.5 Least squares: solving systems that have no solution

So far A has been square and the system $Ax = b$ has had exactly one answer. Real data is messier. Often you have *more equations than unknowns*: A is $m \times n$ with $m > n$, and there is simply *no* x satisfying all m equations at once. The system is *overdetermined*. This is the everyday situation of fitting a model to noisy measurements. Instead of demanding $Ax = b$ exactly, we ask for the x that comes *closest*.

27.5.1 The picture: a best-fit line

Imagine you measured a quantity y at several inputs t and suspect a linear law $y \approx c_0 + c_1 t$. Each data point (t_i, y_i) gives one equation $c_0 + c_1 t_i = y_i$. With, say, 50 data points and only 2 unknowns (c_0, c_1), you have 50 equations in 2 unknowns. Because the data is noisy, no straight line hits all 50 points. So we minimize the total squared miss:

$$\min_x \|Ax - b\|^2 = \min_x \sum_{i=1}^{m} \big((Ax)_i - b_i\big)^2,$$

where row i of A is $(1,\ t_i)$, the unknown is $x = (c_0, c_1)$, and $b_i = y_i$. We square the errors so that positive and negative misses cannot cancel, and because squaring makes the calculus clean. The vector $r = Ax - b$ is the *residual* — what is left over — and we are shrinking its length.

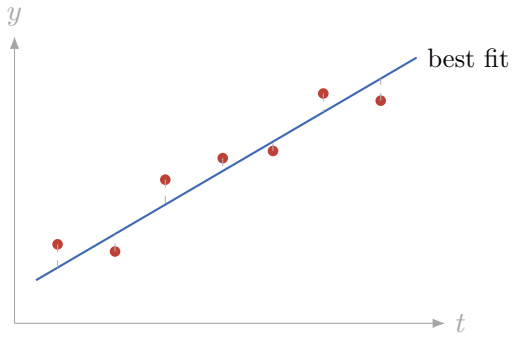


Figure 27.5: Least squares fits the line minimizing the sum of squared vertical residuals (dashed). No line passes through every point; we pick the one that misses by the least, measured in squared distance.

27.5.2 The normal equations

Calculus (or a clean geometric argument) gives a simple answer. The minimizing x satisfies the *normal equations*:

$$\boxed{(A^{\mathsf{T}}A)\,x = A^{\mathsf{T}}b}$$

The geometry behind that boxed line explains it better than the calculus does. The achievable outputs $\{Ax : x \in \mathbb{R}^n\}$ form an n-dimensional *subspace* (the column space of A) sitting inside $\mathbb{R}^m$. The target b usually lies outside it. The closest point in the subspace to b is the *perpendicular projection* of b onto the subspace. "Closest" and "perpendicular" mean the residual $r = Ax - b$ must be orthogonal to *every* column of A, i.e. $A^\mathsf{T} r = 0$, i.e. $A^\mathsf{T}(Ax - b) = 0$, which rearranges to exactly the normal equations.

Intuition

Picture standing in a room (your 3-D world) trying to reach a point b floating near the ceiling, but you may only stand on the *floor* (the column space). Where on the floor are you closest to b? Directly beneath it — the foot of the perpendicular dropped from b. Least squares amounts to dropping a perpendicular; the normal equations are that perpendicularity condition written in matrix form.

Figure 27.6 illustrates this orthogonal projection: the vector b hovers outside the column space $\mathrm{Col}(A)$, and the optimal approximation $p = Ax$ is the unique point on the floor where the residual vector $e = b - Ax$ drops at a right angle.

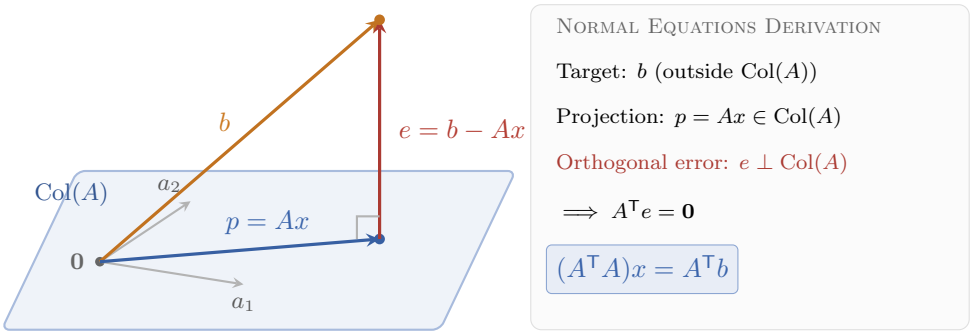


Figure 27.6: The geometric foundation of least squares and the normal equations. The vector b lies outside the column space $\mathrm{Col}(A)$ of achievable outputs. The closest point in $\mathrm{Col}(A)$ is the orthogonal projection $p = Ax$, where the error vector $e = b - Ax$ drops perpendicular to every column a_j of A. Demanding $A^\mathsf{T} e = \mathbf{0}$ yields the normal equations $(A^\mathsf{T} A)x = A^\mathsf{T} b$.

Because the error e is orthogonal to every vector in the column space, $A^\mathsf{T}(Ax - b) = \mathbf{0}$ holds uniquely at the minimizer x.

The matrix $A^\mathsf{T} A$ is $n \times n$ and (when A's columns are independent) symmetric positive-definite, so the normal equations are an ordinary square system you already know how to solve: factor $A^\mathsf{T} A$ and forward/back-substitute. The whole least-squares recipe is therefore: form $A^\mathsf{T} A$ and $A^\mathsf{T} b$, then run an LUP (or, exploiting symmetry, a Cholesky) solve.

Example 27.6 (Fitting a line to three points)**.** Fit $y = c_0 + c_1 t$ to $(t, y) \in \{(1,1), (2,2), (3,2)\}$. Then

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}, \quad A^\mathsf{T} A = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}, \quad A^\mathsf{T} b = \begin{pmatrix} 5 \\ 11 \end{pmatrix}.$$

(Check: $(A^\mathsf{T} A)_{11} = 1 + 1 + 1 = 3$, $(A^\mathsf{T} A)_{12} = 1 + 2 + 3 = 6$, $(A^\mathsf{T} A)_{22} = 1 + 4 + 9 = 14$; $(A^\mathsf{T} b)_1 = 1 + 2 + 2 = 5$, $(A^\mathsf{T} b)_2 = 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 2 = 11$.) Solve $\begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$.

The determinant is $3 \cdot 14 - 6 \cdot 6 = 6$, so

$$c_0 = \frac{14 \cdot 5 - 6 \cdot 11}{6} = \frac{70 - 66}{6} = \frac{2}{3}, \qquad c_1 = \frac{3 \cdot 11 - 6 \cdot 5}{6} = \frac{33 - 30}{6} = \frac{1}{2}.$$

The best-fit line is $y = \frac{2}{3} + \frac{1}{2}t$. Sanity check at $t = 2$: $y = \frac{2}{3} + 1 = \frac{5}{3} \approx 1.67$, sitting sensibly between the data values 1, 2, 2.

Common trap

Forming and solving the normal equations is the textbook method and it is great for understanding, but on a computer $A^\mathsf{T}A$ can be numerically nasty: it *squares* the condition number, so it amplifies rounding error. Production least-squares solvers usually avoid $A^\mathsf{T}A$ and instead factor A directly with a *QR decomposition* (or an SVD). The answer is the same in exact arithmetic, but far more stable in floating point. Learn the normal equations for the geometry; reach for QR when accuracy matters.

Why this matters

Least squares has many uses. It underlies linear regression in statistics, the calibration of GPS positions from satellite ranges, the "line of best fit" in every spreadsheet, polynomial curve fitting, and the closed-form solution of a linear neural-network layer. The single idea — when you cannot solve a system, project onto what you *can* reach — carries over to modern machine learning.

27.6 Pulling it together

Focus

The arc of this chapter.

- A matrix is a linear machine; multiplication is *composing* machines, which forces the "row dot column" rule. Schoolbook multiply: $\Theta(n^3)$.
- **Strassen** trades one multiplication for several additions per recursion level, dropping the exponent to $\log_2 7 \approx 2.807$. It started a fifty-year hunt for the true exponent ω.
- **LUP decomposition** records the elimination multipliers of Gaussian elimination: $PA = LU$. Factor once in $\Theta(n^3)$, then solve any $Ax = b$ in $\Theta(n^2)$ via forward and back substitution. Pivoting buys numerical stability.
- **Inversion** costs $\Theta(M(n))$: it is *equivalent* to multiplication, so a faster multiplier is automatically a faster inverter. Never invert just to solve a system.
- **Least squares** handles overdetermined systems by projecting b onto the column space; the normal equations $A^\mathsf{T}Ax = A^\mathsf{T}b$ are the perpendicularity condition, solvable with the same factor-and-substitute tools.

The thread running through all of it: *a hard matrix problem is solved by reducing it to an easy one.* Hard multiplication reduces to seven half-size multiplications. A general linear system reduces to two triangular ones. Inversion reduces to multiplication. An unsolvable overdetermined system

reduces to a small, solvable square system. Reducing a general problem to a structured special case is a recurring strategy across algorithm design, and matrix factorizations make each step of that reduction explicit.

Linear systems of equations solve $Ax = b$ under exact equality constraints. In Chapter 28, we extend these matrix formulations to Linear Programming, where linear *inequalities* model optimization problems over high-dimensional polyhedra using the simplex algorithm and duality.

Exercises

Exercise 27.1 (Warmup: by hand)**.** Let $A = \begin{pmatrix} 1 & -1 \\ 2 & 0 \end{pmatrix}$ and $B = \begin{pmatrix} 3 & 1 \\ 4 & 2 \end{pmatrix}$. Compute AB and BA entry by entry and confirm they differ. Then compute A^{T} and verify $(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}}$ (note the reversed order).

Exercise 27.2 (Identity and powers)**.** Show that for any $n \times n$ matrix A, $AI_n = I_nA = A$ directly from the definition $c_{ij} = \sum_k a_{ik}b_{kj}$. Then compute M^2 and M^3 for $M = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ and explain what the entries of M^k have to do with Fibonacci numbers.

Exercise 27.3 (Strassen's count)**.** The blocked 8-multiplication scheme has recurrence $T(n) = 8T(n/2) + \Theta(n^2)$ and Strassen's has $T(n) = 7T(n/2) + \Theta(n^2)$. For each, identify a, b, and $n^{\log_b a}$, then state the master-theorem case and the resulting Θ. Why does the $\Theta(n^2)$ combine term not change either answer?

Exercise 27.4 (Verify a Strassen formula)**.** Using the P_i definitions in the text, verify algebraically that $C_{21} = P_3 + P_4$ equals $A_{21}B_{11} + A_{22}B_{21}$. (Expand $P_3 = S_3B_{11}$ and $P_4 = A_{22}S_4$ and watch the unwanted terms cancel.)

Exercise 27.5 (Forward and back substitution)**.** Solve $Lx = b$ where $L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 3 & -1 & 1 \end{pmatrix}$ and $b = \begin{pmatrix} 1 \\ 4 \\ 6 \end{pmatrix}$ by forward substitution. Then, *using the same numbers*, solve $Uy = x$ where $U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & 3 & 2 \\ 0 & 0 & 5 \end{pmatrix}$ by back substitution.

Exercise 27.6 (An LU factorization by hand)**.** Factor $A = \begin{pmatrix} 4 & 3 \\ 6 & 3 \end{pmatrix}$ as $A = LU$ with L unit lower triangular (no pivoting). Then use it to solve $Ax = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ by a forward solve followed by a back solve.

Exercise 27.7 (Why pivot?)**.** Consider $A = \begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix}$ for a tiny $\varepsilon > 0$. Attempt $A = LU$ without pivoting and observe that the multiplier $\ell_{21} = 1/\varepsilon$ and the resulting U entry $1 - 1/\varepsilon$ are enormous. Now swap the two rows first and redo it. Explain in one or two sentences why pivoting prevents the blow-up. (This is the trap behind "tiny pivots.")

Exercise 27.8 (Cost of many solves)**.** You must solve $Ax = b$ for r different right-hand sides b, all with the same $n \times n$ matrix A. Write the total cost of (a) running full Gaussian elimination r

separate times, and (b) factoring $PA = LU$ once and doing r forward/back solves. For $n = 1000$ and $r = 200$, estimate the speedup of (b) over (a) using $n^3/3$ and n^2 as your unit costs.

Exercise 27.9 (Inversion via solves)**.** Explain why solving $Ax = e_j$ for $j = 1, \dots, n$ and stacking the answers as columns produces A^{-1}. Using this, invert $A = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}$ by solving $Ax = e_1$ and $Ax = e_2$, and check your answer by computing AA^{-1}.

Exercise 27.10 (Multiplication from inversion)**.** Take $A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}$ (so $n = 2$). Build the 6×6 block matrix D from the text, write down the claimed D^{-1}, and confirm that its top-right 2×2 block equals AB.

Exercise 27.11 (Least squares from scratch)**.** Fit a line $y = c_0 + c_1 t$ to the four points $(0, 1)$, $(1, 3)$, $(2, 4)$, $(3, 4)$. Write down A and b, form $A^{\mathsf{T}}A$ and $A^{\mathsf{T}}b$, solve the 2×2 normal equations, and report the best-fit line. What is the residual at $t = 1$?

Exercise 27.12 (Challenge: projection is idempotent)**.** When $A^{\mathsf{T}}A$ is invertible, the best-fit *prediction* is $\hat{b} = A(A^{\mathsf{T}}A)^{-1}A^{\mathsf{T}}b = Hb$, where H is the "hat matrix." Prove that $H^2 = H$ (projecting twice is the same as projecting once) and that $H^{\mathsf{T}} = H$. Explain geometrically why both facts *must* hold for an orthogonal projection onto the column space.

Chapter 28

Linear Programming

In Chapter 27, we studied matrix operations and solved systems of linear equations ($Ax = b$), finding exact balances when constraints are rigid equalities. But real-world resources are rarely exact equalities: you have at most twelve hours of labor, at least five tons of raw material, and a goal to maximize profit. Replacing equalities with linear inequalities and seeking the best possible solution turns linear algebra into *linear programming*.

Linear programming is best understood as a *language*: a single, expressive way to write down an enormous family of optimization problems, paired with general solvers that can answer any problem posed in that language. Once you know the language, shortest paths, maximum flow, and many real scheduling, blending, and resource-allocation problems can all be written as instances of the same geometric problem.

The phrase "linear programming" is, like "dynamic programming," a historical accident. "Programming" here means *planning* – arranging a schedule or an allocation – not writing code. And "linear" means exactly what it says: every quantity we care about is built only out of additions and constant multiples. No squares, no products of two unknowns, no `if` branches. Restricting every constraint and objective to straight lines and flat hyperplanes is what makes general, efficient solvers possible.

Intuition

Picture a factory that makes two products. Each product earns some profit per unit, and each consumes some of your limited labor hours and raw material. You want to choose how much of each to make to maximize profit, without using more labor or material than you have. Every constraint ("labor used $\leq$ labor available") is a straight-line boundary cutting through the plane of possible production plans. Together those boundaries fence off a region of *legal* plans, and "maximize profit" means sliding a ruler across that region until it just touches the farthest corner. The rest of the chapter makes this picture precise and shows how to find the best corner quickly.

We begin with the algebra and geometry of two-variable programs, trace how the simplex algorithm walks along the edges of multi-dimensional polytopes, uncover the duality theorem that certifies optimality, and conclude by using linear programming to approximate hard discrete problems.

28.1 Modeling: From Words to Algebra

Let us make the factory concrete. You run a small workshop that builds two kinds of furniture, chairs and tables. The numbers:

	chair	table	available
profit per unit ($)	3	5	—
wood (board-feet)	1	2	8
labor (hours)	2	2	12

Each chair earns $3 and each table earns $5. A chair needs 1 board-foot of wood and 2 hours of labor; a table needs 2 board-feet and 2 hours. You have 8 board-feet of wood and 12 hours of labor this week. How many chairs and tables should you build to maximize profit?

The first move in every modeling problem is to name the things you get to *decide*. Call them *decision variables*. Let x_1 be the number of chairs and x_2 the number of tables. Now everything else is forced.

- **Objective.** Total profit is $3x_1 + 5x_2$, and you want it as large as possible. A linear function of the variables that you push up or down is called the *objective function*.
- **Constraints.** You cannot use more wood than you have: $x_1 + 2x_2 \le 8$. You cannot use more labor than you have: $2x_1 + 2x_2 \le 12$. And you cannot build a negative number of anything: $x_1 \ge 0$ and $x_2 \ge 0$. Those last two are the *nonnegativity constraints*, and they appear so often that we usually group them on a single line.

Put it all together and you have a *linear program*:

$$\begin{aligned} \text{maximize} \quad & 3x_1 + 5x_2 \\ \text{subject to} \quad & x_1 + 2x_2 \le 8 \\ & 2x_1 + 2x_2 \le 12 \\ & x_1, x_2 \ge 0. \end{aligned}$$

Focus

An LP has exactly three ingredients, and the word "linear" applies to all of them:

1. **Decision variables** $x_1, \dots, x_n$ – the quantities you choose.
2. A **linear objective** $c_1x_1 + \cdots + c_nx_n$ to maximize (or minimize).
3. **Linear constraints**, each an inequality ($\le$ or $\ge$) or equality ($=$) between a linear combination of the variables and a constant.

"Linear" means every appearance of a variable is just that variable times a constant, added to others. No x_1^2, no x_1x_2, no $\sqrt{x_1}$.

Common trap

The single most common modeling mistake is sneaking in a *nonlinearity* without noticing. "The profit per chair drops once I sell more than 10" is a nonlinear objective. "I can build

chairs *or* tables but not both" is a logical `or`, which is also not linear (it needs integer 0/1 variables – see Section 28.6). Watch for products of variables ("revenue = price × quantity" when *both* are decisions) – that is the classic trap. If your model has any of these, it is not a plain LP, and the gentle theory of this chapter does not directly apply.

Why this matters

This template fits a very wide range of real decisions. Oil refineries blend crude streams into gasoline grades to hit octane targets at least cost – an LP. Airlines assign crews and aircraft to routes – LPs (with integers) at industrial scale. Hospitals schedule nurses, farms choose crop mixes, data centers route traffic, portfolios balance risk. A seventy-year-old method is still in constant use because "maximize a linear thing subject to linear limits" describes the basic structure of a great many planning problems.

28.1.1 A second model: shortest paths as an LP

To convince you LP is a *language*, let us encode something you already know – single-source shortest paths – as a linear program. Suppose you want the shortest-path distances d_v from a source s in a graph with edge weights $w(u, v)$. The trick is to ask for the distances that are as *large* as possible while still respecting the triangle inequality on every edge.

$$\begin{aligned} \text{maximize} \quad & \sum_v d_v \\ \text{subject to} \quad & d_v - d_u \le w(u,v) \quad \text{for every edge } (u,v), \\ & d_s = 0. \end{aligned}$$

The constraint $d_v \le d_u + w(u, v)$ is exactly the relaxation condition from the shortest-paths chapter. Maximizing the total pushes each d_v up until some constraint stops it – and the value where it gets stuck is precisely the true shortest-path distance. We will not prove that here, but file it away: a problem you solved with Dijkstra is *also* an LP. This generality is what makes LP useful.

28.2 Standard and Slack Forms

If we want general algorithms, "any LP" has to mean something precise. Linear programs can be written in many surface forms – maximizing or minimizing, mixing $\le$, $\ge$, and $=$ constraints, or allowing unconstrained variables. Two canonical forms standardize all of them.

28.2.1 Standard form

Formal

A linear program is in **standard form** if it is a *maximization* over nonnegative variables subject to $\le$ constraints:

$$\begin{aligned} \text{maximize} \quad & \sum_{j=1}^{n} c_j x_j \\ \text{subject to} \quad & \sum_{j=1}^{n} a_{ij} x_j \le b_i \quad (i = 1, \dots, m), \\ & x_j \ge 0 \quad (j = 1, \dots, n). \end{aligned}$$

In matrix shorthand, with $c, x \in \mathbb{R}^n$, $b \in \mathbb{R}^m$, and A an $m \times n$ matrix: *maximize* $c^\top x$ *subject to* $Ax \le b$ *and* $x \ge 0$.

Our furniture problem is already in standard form, with

$$c = \begin{pmatrix} 3 \\ 5 \end{pmatrix}, \quad A = \begin{pmatrix} 1 & 2 \\ 2 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 8 \\ 12 \end{pmatrix}.$$

Every LP can be massaged into standard form with a few mechanical moves. Keep this little phrasebook:

- **Minimize $\to$ maximize.** To minimize $c^\top x$, instead maximize $-c^\top x$ and negate the answer at the end.
- **$\ge$ constraint $\to$ $\le$.** Multiply both sides by -1: the constraint $\sum_j a_{ij} x_j \ge b_i$ becomes $\sum_j (-a_{ij}) x_j \le -b_i$.
- **Equality $\to$ two inequalities.** Replace $\sum_j a_{ij} x_j = b_i$ by the pair $\le b_i$ and $\ge b_i$.
- **Free (unbounded) variable.** If some x_j is allowed to be negative, split it: write $x_j = x_j' - x_j''$ with $x_j', x_j'' \ge 0$. Any real value is some nonnegative minus some nonnegative.

Example 28.1. Convert "minimize $2x_1 - 3x_2$ subject to $x_1 + x_2 = 4$, $x_1 \ge 1$, with x_2 free" to standard form. Maximize $-2x_1 + 3x_2$. Split $x_2 = x_2' - x_2''$. The equality becomes $x_1 + x_2' - x_2'' \le 4$ and $-x_1 - x_2' + x_2'' \le -4$. The bound $x_1 \ge 1$ becomes $-x_1 \le -1$. Now every variable (x_1, x_2', x_2'') is ≥ 0 and every constraint is $\le$. Done.

28.2.2 Slack form

Standard form is the right shape for stating theory and duality. But the simplex algorithm prefers a cousin in which every inequality has been turned into an *equality* by introducing a brand-new nonnegative variable that soaks up the difference. That extra variable is called a *slack variable*, because it measures how much slack – unused resource – remains in the constraint.

Intuition

Geometrically, a constraint like $\sum_j a_{ij} x_j \le b_i$ divides space into two halves separated by the boundary plane $\sum_j a_{ij} x_j = b_i$. If your current production plan x uses strictly less than b_i,

you sit strictly inside the legal half-space. The slack variable

$$x_{n+i} = b_i - \sum_{j=1}^{n} a_{ij} x_j$$

measures your remaining buffer to that boundary wall. Requiring $x_{n+i} \geq 0$ is mathematically identical to the original inequality, but it turns the inequality into a simple linear equation. When $x_{n+i} = 0$, you are standing directly on the boundary hyperplane – the constraint is *tight* (or *active*). When $x_{n+i} > 0$, you have breathing room.

To convert a standard-form LP into slack form systematically:

1. For each constraint $i \in \{1, \ldots, m\}$, create a fresh variable x_{n+i} and define it as the difference between the bound b_i and the left-hand side $\sum_{j=1}^{n} a_{ij} x_j$.
2. Express each basic variable $x_{n+1}, \ldots, x_{n+m}$ solely in terms of the original (*nonbasic*) variables $x_1, \ldots, x_n$.
3. Write the objective z explicitly as an offset v (initially 0) plus a linear combination of the nonbasic variables.
4. Require all variables – both original and slack – to be nonnegative: $x_k \geq 0$ for all $k \in \{1, \ldots, n+m\}$.

Doing this for both constraints of our furniture LP:

$$\begin{aligned} z &= 0 + 3x_1 + 5x_2 \\ x_3 &= 8 - x_1 - 2x_2 \qquad\qquad x_1, x_2, x_3, x_4 \geq 0. \\ x_4 &= 12 - 2x_1 - 2x_2, \end{aligned}$$

Focus

Slack form writes the objective z and each slack variable on the left, as a function of the others on the right, plus a constant. The variables that sit alone on the left are called *basic*; the ones on the right are *nonbasic*. A slack variable being zero means its constraint is *tight* (the resource is fully used); positive slack means room to spare. The simplex method keeps track of its progress through this split into basic and nonbasic variables.

A key geometric consequence: if you set all n nonbasic variables to 0, you can read off the m basic variables instantly. Here, setting $x_1 = x_2 = 0$ gives $x_3 = 8$, $x_4 = 12$, and $z = 0$. In n-dimensional space, setting n variables to zero pins down the intersection of n bounding hyperplanes – a single sharp corner (vertex) of the polytope! Here that vertex is the origin $(0, 0)$, representing the "build nothing" plan – a valid starting point that we can now systematically climb away from.

28.3 The Geometry of an LP

Before solving the system algebraically, let us plot the furniture constraints in the plane so we can see the feasible region directly.

Each variable is an axis. With two variables we live in the plane, with x_1 (chairs) horizontal and x_2 (tables) vertical. Each constraint is a half of the plane:

- $x_1 \geq 0$: everything to the right of the vertical axis.
- $x_2 \geq 0$: everything above the horizontal axis.
- $x_1 + 2x_2 \leq 8$: everything on or below the line through $(8, 0)$ and $(0, 4)$.
- $2x_1 + 2x_2 \leq 12$, i.e. $x_1 + x_2 \leq 6$: on or below the line through $(6, 0)$ and $(0, 6)$.

The points satisfying *all* the constraints form the shaded region in Figure 28.1, called the *feasible region*. Because it is an intersection of half-planes, it is a convex polygon (in higher dimensions, a convex *polytope*).

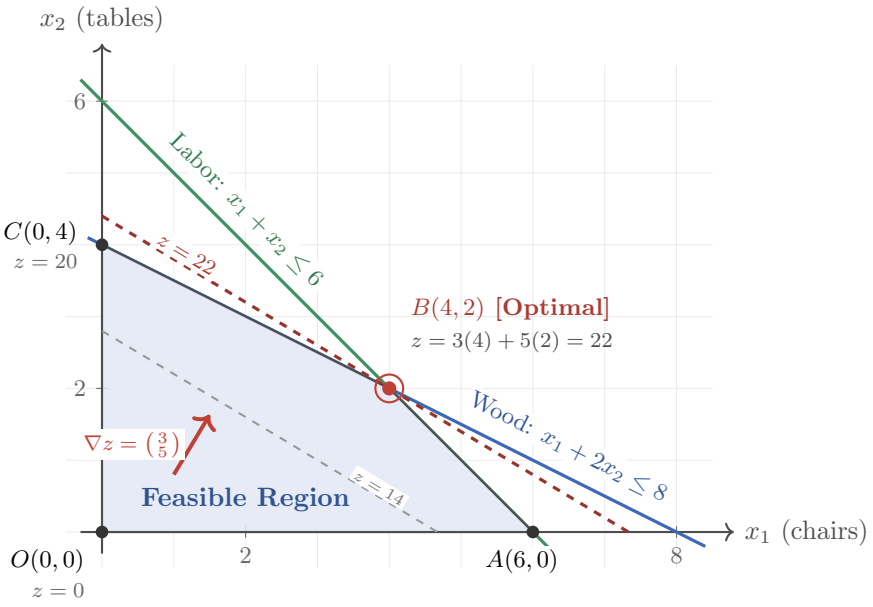


Figure 28.1: The geometry of the furniture linear program. The feasible region (shaded blue) is bounded by four half-planes: nonnegativity ($x_1 \geq 0, x_2 \geq 0$), the wood constraint ($x_1 + 2x_2 \leq 8$, blue boundary), and the labor constraint ($x_1 + x_2 \leq 6$, green boundary). Dashed lines trace iso-profit contours $z = 3x_1 + 5x_2 = \text{const}$, advancing perpendicular to the objective gradient $\vec{c} = (3, 5)$. The maximum achievable profit touches the feasible polygon at the extreme corner $B(4, 2)$ with value $z = 22$.

The four corners are $O = (0, 0)$, $A = (6, 0)$, $B = (4, 2)$, and $C = (0, 4)$. (Corner B is where the two constraint lines cross; solve $x_1 + 2x_2 = 8$ and $x_1 + x_2 = 6$ simultaneously: subtracting gives $x_2 = 2$, then $x_1 = 4$.)

Now the objective. The function $z = 3x_1 + 5x_2$ has the same value along any line of slope $-3/5$; these are the dashed *iso-profit* lines. Increasing z slides that line in the direction of the gradient $(3, 5)$ – up and to the right. To maximize z you push the iso-profit line as far in the $(3, 5)$ direction as you can while still touching the feasible region. The last point it touches is the optimum.

Let us just evaluate z at all four corners:

$$z(O) = 0, \quad z(A) = 18, \quad z(B) = 3 \cdot 4 + 5 \cdot 2 = 22, \quad z(C) = 5 \cdot 4 = 20.$$

The winner is $B = (4, 2)$ with profit $\boxed{22}$: build 4 chairs and 2 tables.

Intuition

Notice the optimum landed on a *corner*, not in the middle of an edge or the interior. This always happens: an iso-profit line, sliding outward, leaves the region last at a vertex (or, in a tie, along a whole edge – but then a vertex of that edge is still optimal). So you never need to search the infinite interior. **You only ever need to check the corners.** A continuous problem with infinitely many feasible points collapses to a finite search over vertices. The simplex method is built on this observation.

Formal

The feasible region of an LP is a convex set: if x and y are both feasible then so is every point $\lambda x + (1 - \lambda)y$ on the segment between them, for $0 \leq \lambda \leq 1$. (Each constraint is preserved under averaging because it is linear.) A **vertex** (or *basic feasible solution*) is a feasible point that is *not* the midpoint of any segment lying in the region – a true corner. The fundamental theorem of linear programming says: if an LP has an optimal solution, then some *vertex* is optimal.

Common trap

"Just check the corners" sounds like a finished algorithm, but it hides a trap: the number of vertices can be *exponential* in the number of variables and constraints. An LP in n variables with m constraints can have on the order of $\binom{m+n}{n}$ vertices. Enumerating them all is hopeless for large problems. What the simplex method adds is a rule for walking from vertex to neighboring vertex, always uphill, so that in practice you reach the best one after visiting only a small fraction of them.

A few more geometric vocabulary words you will meet:

- **Infeasible.** The constraints contradict each other and the feasible region is empty. (E.g. $x \leq 1$ and $x \geq 2$.) There is nothing to optimize.
- **Unbounded.** The region runs off to infinity in a direction where the objective keeps improving, so there is no finite maximum. (E.g. maximize x_1 with only $x_1 \geq 0$.)
- **Bounded and feasible.** The good case: a finite optimum exists, on a vertex.

28.4 The Simplex Method, by Example

Now we make "walk uphill from corner to corner" mechanical. The simplex method, due to George Dantzig in 1947, does exactly this, using slack form as its ledger. We will run it on the furniture LP and *watch* it travel from O to B.

28.4.1 The idea in one paragraph

Start at a known vertex (the origin, where all original variables are 0 and the slacks hold all the resource). Look at the objective written in terms of the current nonbasic variables. If some nonbasic

variable has a *positive* coefficient, increasing it would raise z – so we want to bring it into the basis. We increase it as much as we can until some basic variable hits 0 (a constraint goes tight); that variable leaves the basis. Swap them, rewrite everything, and repeat. Each swap moves us to an adjacent vertex with no worse (usually strictly better) objective value. When no nonbasic variable has a positive coefficient, no local move helps – and by convexity, we are at the global optimum.

Focus

Two pieces of jargon for the swap:

- The variable we increase (positive objective coefficient) is the **entering variable** – it enters the basis.
- The basic variable that hits 0 first, limiting how far we can go, is the **leaving variable** – it leaves the basis.

One entering, one leaving, per iteration. The rewrite that re-expresses everything in terms of the new basis is called a **pivot**, by analogy with Gaussian elimination.

28.4.2 Iteration 0: the starting dictionary

We begin in slack form, which when written this way is often called a *dictionary*:

$$\begin{aligned} z &= \quad 3x_1 + 5x_2 \\ x_3 &= 8 - x_1 - 2x_2 \\ x_4 &= 12 - 2x_1 - 2x_2. \end{aligned}$$

Basic variables: x_3, x_4. Nonbasic: x_1, x_2 (set to 0). Current vertex: $(x_1, x_2) = (0, 0)$, with $z = 0$. This is corner O.

Choose entering variable. Both x_1 (coefficient 3) and x_2 (coefficient 5) would increase z. A common rule is to pick the largest coefficient, so let x_2 enter.

Choose leaving variable (ratio test). As we raise x_2 from 0 (keeping $x_1 = 0$), how far can we go before a basic variable goes negative? Look at each basic equation:

$$x_3 = 8 - 2x_2 \geq 0 \;\Rightarrow\; x_2 \leq \tfrac{8}{2} = 4, \qquad x_4 = 12 - 2x_2 \geq 0 \;\Rightarrow\; x_2 \leq \tfrac{12}{2} = 6.$$

The tightest bound wins: x_2 can rise to 4, at which point x_3 hits 0. So x_2 enters and x_3 leaves. This is the *ratio test*: divide each constant by the (positive) coefficient of the entering variable and take the minimum.

28.4.3 Iteration 1: the pivot

Solve the binding equation $x_3 = 8 - x_1 - 2x_2$ for the entering variable x_2:

$$x_2 = 4 - \tfrac{1}{2}x_1 - \tfrac{1}{2}x_3.$$

Now substitute this everywhere x_2 appeared. In the objective:

$$z = 3x_1 + 5\big(4 - \tfrac{1}{2}x_1 - \tfrac{1}{2}x_3\big) = 20 + \tfrac{1}{2}x_1 - \tfrac{5}{2}x_3.$$

In the other basic equation:

$$x_4 = 12 - 2x_1 - 2\big(4 - \tfrac{1}{2}x_1 - \tfrac{1}{2}x_3\big) = 4 - x_1 + x_3.$$

So the new dictionary is

$$\begin{aligned} z &= 20 + \tfrac{1}{2}x_1 - \tfrac{5}{2}x_3 \\ x_2 &= 4 - \tfrac{1}{2}x_1 - \tfrac{1}{2}x_3 \\ x_4 &= 4 - x_1 + x_3. \end{aligned}$$

Basic: x_2, x_4. Nonbasic: x_1, x_3 (both 0). Vertex: $(x_1, x_2) = (0, 4)$ – that is corner C – with $z = 20$. We climbed from 0 to 20 in one step.

Are we done? The objective row is $z = 20 + \frac{1}{2}x_1 - \frac{5}{2}x_3$. The coefficient on x_1 is $+\frac{1}{2} > 0$, so raising x_1 still helps. Not done. Let x_1 enter.

Ratio test for x_1:

$$x_2 = 4 - \tfrac{1}{2}x_1 \geq 0 \Rightarrow x_1 \leq \tfrac{4}{1/2} = 8, \qquad x_4 = 4 - x_1 \geq 0 \Rightarrow x_1 \leq \tfrac{4}{1} = 4.$$

Minimum is 4, achieved by the x_4 row, so x_4 leaves.

28.4.4 Iteration 2: the second pivot

Solve the binding equation $x_4 = 4 - x_1 + x_3$ for x_1:

$$x_1 = 4 - x_4 + x_3.$$

Substitute into the objective:

$$z = 20 + \tfrac{1}{2}(4 - x_4 + x_3) - \tfrac{5}{2}x_3 = 22 - 2x_3 - \tfrac{1}{2}x_4.$$

And into the x_2 row:

$$x_2 = 4 - \tfrac{1}{2}(4 - x_4 + x_3) - \tfrac{1}{2}x_3 = 2 - x_3 + \tfrac{1}{2}x_4.$$

New dictionary:

$$\begin{aligned} z &= 22 - 2x_3 - \tfrac{1}{2}x_4 \\ x_1 &= 4 + x_3 - x_4 \\ x_2 &= 2 - x_3 + \tfrac{1}{2}x_4. \end{aligned}$$

Basic: x_1, x_2. Nonbasic: x_3, x_4 (both 0). Vertex: $(x_1, x_2) = (4, 2)$ – corner B – with $z = 22$.

Stop? Every coefficient in the objective row is now *negative* (-2 and $-\frac{1}{2}$). Increasing either nonbasic variable would only lower z. There is no improving move. **We are optimal:** build 4 chairs and 2 tables for \$22, exactly matching what the corner-by-corner geometry told us.

Figure 28.2 draws the sequence of pivots on the feasible polygon: two pivots, two edges, climbing $0 \to 20 \to 22$, alongside the algebraic ledger of dictionary states.

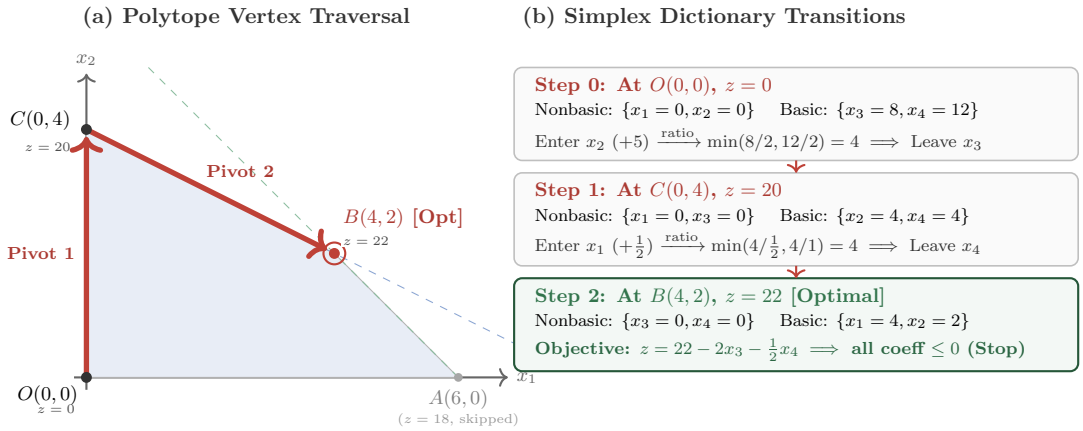


Figure 28.2: The simplex walk in geometry and algebra. (a) Starting at the origin $O(0,0)$ where both decision variables are nonbasic, Pivot 1 enters x_2 and slides along edge OC until constraint x_3 binds at $C(0,4)$, lifting profit from 0 to 20. Pivot 2 enters x_1 and traverses edge CB until x_4 binds at $B(4,2)$, reaching the maximum profit 22. Vertex A is bypassed entirely. (b) The corresponding dictionary ledger tracking entering/leaving variables, ratio tests, and the termination condition.

Algorithm lab

Trace the path on Figure 28.2: simplex walked $O \to C \to B$, climbing $0 \to 20 \to 22$. It never visited A. Now *you* re-run it but break the first tie differently: at iteration 0 let x_1 enter instead of x_2 ("largest coefficient" is just one rule; "first positive" is another). Do the ratio test, pivot, and keep going. You should travel $O \to A \to B$ and still land at $z = 22$. The two paths reach the same optimum, as convexity guarantees. Write out all three dictionaries yourself.

28.4.5 Simplex as pseudocode

The example contains the whole algorithm. Here it is in skeleton form, operating on a slack-form dictionary with basic set B and nonbasic set N.

Algorithm 123: SIMPLEX (one phase, on a feasible dictionary)

Input: slack-form dictionary: objective row $z = v + \sum_{j \in N} \bar{c}_j x_j$ and rows $x_i = \bar{b}_i - \sum_{j \in N} \bar{a}_{ij} x_j$ for $i \in B$

Output: an optimal basic feasible solution, or report "unbounded"

while *some* $\bar{c}_e > 0$ *for* $e \in N$ **do**
  pick entering index $e \in N$ with $\bar{c}_e > 0$;
  `// ratio test: how far can` x_e `rise?`
  if $\bar{a}_{ie} \le 0$ *for all* $i \in B$ **then**
    return "unbounded" ; `//` x_e `rises forever,` $z \to \infty$
  pick leaving index $\ell \in B$ minimizing $\bar{b}_i / \bar{a}_{ie}$ over i with $\bar{a}_{ie} > 0$;
  `// pivot: swap` x_e `into the basis,` x_ℓ `out`
  rewrite row ℓ to express x_e, then substitute into all other rows and the objective;
  update $B \leftarrow (B \setminus \{\ell\}) \cup \{e\}$ and $N \leftarrow (N \setminus \{e\}) \cup \{\ell\}$;
return the basic solution: $x_j = 0$ for $j \in N$, $x_i = \bar{b}_i$ for $i \in B$;

Focus

The loop makes three decisions:

1. **Optimality test.** If no nonbasic coefficient $\bar{c}_e$ is positive, stop – you are optimal.
2. **Entering rule.** Among positive $\bar{c}_e$, pick one (largest coefficient, or Bland's rule, etc.).
3. **Ratio test / leaving rule.** The entering variable rises until the first basic variable hits 0; that one leaves. If *nothing* stops it, the LP is unbounded.

28.4.6 Two loose ends: starting and cycling

We glossed over two things that deserve to be named.

Getting an initial feasible vertex. We were lucky: the origin was feasible (all $b_i \ge 0$), so the slacks gave a ready basic feasible solution. When some $b_i < 0$, the origin is infeasible and cannot serve as the starting basis. The fix is a preliminary "Phase I" LP: introduce an auxiliary variable that measures total infeasibility and minimize it with simplex itself. If the minimum is 0, you have found a genuine feasible vertex to start the real ("Phase II") optimization from; if it is positive, the original LP is infeasible. So simplex bootstraps itself.

Cycling. The ratio test can tie, producing a *degenerate* pivot that changes the basis but not the vertex or the objective (you move 0 distance). In rare crafted cases a sequence of degenerate pivots can loop forever, returning to a basis it already saw. The cure is a tie-breaking discipline called *Bland's rule*: among allowed entering and leaving candidates, always choose the one with the smallest index. It can be proved that Bland's rule never cycles, so simplex always terminates.

History and connection

Leonid Kantorovich formulated linear programming in 1939 to optimize production, earning the 1975 Nobel Memorial Prize in Economic Sciences. George Dantzig independently developed the general linear programming model and the simplex method in 1947 while working on Air Force planning problems; the word "programming" was military planning lingo from that

era. There is a famous tale that as a graduate student Dantzig arrived late to class, copied two "homework" problems off the board, and solved them – only to learn they were open problems in statistics. The simplex method is in **CLRS Chapter 29**, specifically Section 29.3 (the algorithm), Section 29.4 (duality), and Section 29.5 (the initial basic feasible solution / Phase I). CLRS develops the same pivot-on-a-slack-form machinery we just traced; read it right after this chapter and it will feel familiar. A notable fact, covered in CLRS's notes, is that simplex can take exponentially many steps on adversarial inputs (the Klee–Minty cube) yet is fast in practice (explained decades later by Spielman and Teng's smoothed analysis); the first *provably* polynomial methods were Khachiyan's ellipsoid algorithm (1979) and Karmarkar's interior-point method (1984).

Cost summary

Simplex cost. Each pivot is a Gaussian-elimination-style rewrite of an m-row dictionary in n variables, costing $O(mn)$ arithmetic per iteration (or $O(m^2)$ in the revised, matrix form). The number of pivots is the wild card: typically $O(m)$ to $O(m+n)$ in practice, giving excellent real-world speed, but worst-case *exponential* for every known deterministic pivot rule. By contrast, interior-point methods solve an LP to accuracy ε in a provably polynomial number of iterations – roughly $O(\sqrt{n}\,\log(1/\varepsilon))$ Newton steps, each a linear solve. In short: simplex is fast in practice and ugly in the worst case; interior-point is polynomial always and competitive in practice. Solvers ship both.

28.5 Duality: Every LP Has a Shadow

Every linear program (the *primal*) has a companion linear program called its *dual*, formed from the exact same coefficients rearranged. The two are locked together so tightly that solving one solves the other, and the dual reveals something the primal never explicitly asked for: the *value of each scarce resource*.

28.5.1 Where does the dual come from?

Go back to the furniture LP and ask a direct question: *without* solving it, can we prove an *upper bound* on the best possible profit? Take the labor constraint $2x_1 + 2x_2 \le 12$ and multiply it by $\frac{5}{2}$:

$$5x_1 + 5x_2 \le 30.$$

Since $x_1 \ge 0$, certainly $3x_1 \le 5x_1$, so the profit obeys

$$z = 3x_1 + 5x_2 \le 5x_1 + 5x_2 \le 30.$$

We just proved, with one line of algebra, that no plan earns more than \$30 – *without ever solving the LP*. Can we do better? Try combining the two constraints. Pick nonnegative multipliers y_1 (for wood) and y_2 (for labor):

$$y_1(x_1 + 2x_2) + y_2(2x_1 + 2x_2) \le 8y_1 + 12y_2.$$

The left side is $(y_1 + 2y_2)x_1 + (2y_1 + 2y_2)x_2$. If we choose the y's so that this dominates the objective coefficient-by-coefficient – that is,

$$y_1 + 2y_2 \geq 3 \quad \text{(coeff. of } x_1\text{)}, \qquad 2y_1 + 2y_2 \geq 5 \quad \text{(coeff. of } x_2\text{)},$$

then, because all $x_j \geq 0$,

$$z = 3x_1 + 5x_2 \leq (y_1 + 2y_2)x_1 + (2y_1 + 2y_2)x_2 \leq 8y_1 + 12y_2.$$

So *any* nonnegative y_1, y_2 satisfying those two inequalities gives an upper bound $8y_1 + 12y_2$ on the profit. Naturally, we want the *tightest* such bound – the smallest $8y_1 + 12y_2$. And that is itself a linear program!

Focus

The **dual** of our furniture LP is:

$$\begin{aligned} \text{minimize} \quad & 8y_1 + 12y_2 \\ \text{subject to} \quad & y_1 + 2y_2 \geq 3 \\ & 2y_1 + 2y_2 \geq 5 \\ & y_1, y_2 \geq 0. \end{aligned}$$

One dual variable per primal *constraint*; one dual constraint per primal *variable*; the objective and right-hand side swap roles; max becomes min; $\leq$ becomes $\geq$. The dual is the search for the best provable upper bound.

The mechanical recipe, for an LP in standard form (max $c^\top x$ s.t. $Ax \leq b$, $x \geq 0$):

Primal (max)	**Dual (min)**
maximize $c^\top x$	minimize $b^\top y$
m constraints $Ax \leq b$	m variables $y \geq 0$
n variables $x \geq 0$	n constraints $A^\top y \geq c$
right-hand side b	objective b
objective c	right-hand side c

In matrix form: the dual of "max $c^\top x$ s.t. $Ax \leq b$, $x \geq 0$" is "min $b^\top y$ s.t. $A^\top y \geq c$, $y \geq 0$." Transpose the matrix, swap b and c, flip the inequalities, flip max/min. The construction is symmetric: the dual of the dual is the primal again.

28.5.2 Weak and strong duality

Theorem 28.1 (Weak duality)**.** *If x is any feasible solution of the primal (a max LP) and y is any feasible solution of the dual (a min LP), then $c^\top x \leq b^\top y$. Every primal value is below every dual value.*

We essentially proved this above: the chain $c^\top x \leq (A^\top y)^\top x = y^\top (Ax) \leq y^\top b$ uses only $x \geq 0$, $A^\top y \geq c$, $y \geq 0$, and $Ax \leq b$. It is just bookkeeping with inequalities, but it sandwiches the unknown optimum from both sides.

Intuition

Picture the primal values (feasible profits) as numbers piling up from below and the dual values (provable upper bounds) coming down from above. Weak duality says the two piles *never cross*: every bound from above is at least every achievable profit from below. The question that remains is whether they actually *meet*. If a profit and a bound are ever equal, that profit must be optimal (nothing can beat its own upper bound) and that bound must be tight. Strong duality states that whenever the primal has a finite optimum, the two bounds meet with zero gap.

Figure 28.3 pictures the squeeze. The achievable primal profits cannot rise above the line of provable dual bounds, and at the shared optimum the two sides touch at the single value 22.

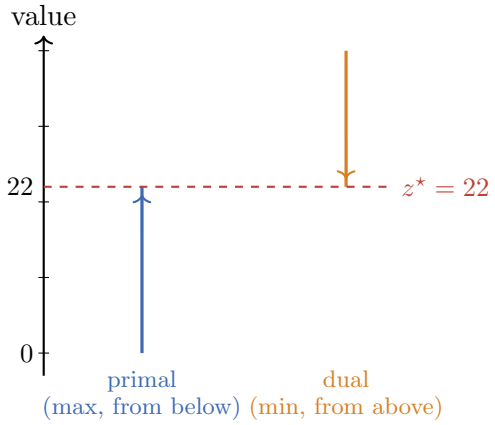


Figure 28.3: Weak and strong duality as a squeeze. Every feasible primal value (blue, pushing up) stays at or below every feasible dual value (orange, pressing down); weak duality forbids them from crossing. Strong duality says the gap closes exactly at the common optimum $z^\star = 22$.

Theorem 28.2 (Strong duality)**.** *If the primal LP has a finite optimum, then so does the dual, and their optimal values are* equal*:* $\max c^\top x = \min b^\top y$.

This is not obvious and does not hold for general (nonlinear) optimization. For LPs it is a theorem – in fact simplex proves it constructively: when simplex halts, the negated objective coefficients on the slack variables turn out to be exactly an optimal dual solution. Let us verify on the furniture problem. The primal optimum was 22. Solve the dual: minimize $8y_1 + 12y_2$ s.t. $y_1 + 2y_2 \geq 3$, $2y_1 + 2y_2 \geq 5$, $y \geq 0$. The optimum sits where both constraints are tight: $y_1 + 2y_2 = 3$ and $2y_1 + 2y_2 = 5$. Subtracting, $y_1 = 2$; then $2 + 2y_2 = 3$ gives $y_2 = \frac{1}{2}$. Dual value: $8(2) + 12(\frac{1}{2}) = 16 + 6 = 22$. The two optima are equal, as strong duality says.

Common trap

A frequent confusion: students expect the dual optimum to be *bigger* than the primal because "it's an upper bound." At the optimum they are *equal*; the dual is bigger only at *suboptimal* feasible points. Weak duality gives the inequality everywhere; strong duality collapses it to equality precisely at the joint optimum. Also: strong duality can fail if the primal is infeasible or unbounded – then the meeting-in-the-middle story needs the careful case analysis CLRS Section 29.4 provides.

28.5.3 What the dual variables mean: shadow prices

The dual values $y_1 = 2$ and $y_2 = \frac{1}{2}$ have a concrete economic meaning: they are *shadow prices*. Specifically, y_i represents the rate at which the optimal profit would increase if you were granted one additional unit of resource i.

Check it. We had 8 board-feet of wood; $y_1 = 2$ predicts that a 9th board-foot is worth \$2 of extra profit. Indeed, re-solving with the wood limit raised to 9 moves the optimum to $(3, 3)$ with $z = 24$ – up by \$2, exactly y_1. The labor shadow price $y_2 = \frac{1}{2}$ says an extra hour of labor is worth fifty cents at the margin.

Why this matters

Shadow prices make LP especially useful to managers. Solve the plan once and the dual *automatically* tells you what each bottleneck resource is worth – where to invest, what overtime is worth paying for, which contract to renegotiate. A resource with a *zero* shadow price has slack to spare; buying more of it is wasted money. Along with a plan, the LP gives you the *prices of your constraints*. This is the same idea that underlies market prices in competitive-equilibrium economics, and it is why Dantzig's method and Koopmans's economics are closely related (Kantorovich and Koopmans won the 1975 Nobel in economics for related work).

Formal

Complementary slackness. At a primal-dual optimum, for each i and j:

$$y_i > 0 \;\Rightarrow\; (\text{primal constraint } i \text{ is tight}), \qquad x_j > 0 \;\Rightarrow\; (\text{dual constraint } j \text{ is tight}).$$

Equivalently: a resource with leftover slack ($Ax < b$ in row i) has shadow price $y_i = 0$; and you produce a positive amount of product j ($x_j > 0$) only if its dual constraint binds (it "pays for itself"). In the furniture optimum both resources were fully used, so both $y_i > 0$; and we built both products, so both dual constraints were tight – consistent with the solution we found.

28.6 LP Relaxation as a Design Tool

We close with the reason an algorithms book cares about LP beyond "it solves LPs." Linear programming is a *design tool* for attacking problems that are not themselves LPs – especially hard combinatorial ones.

The trouble usually is integrality. Real decisions are often whole numbers or yes/no: you ship a truck or you don't; you assign a worker to a shift or you don't; you pick a vertex into a cover or you leave it out. Add the constraint "$x_j \in \{0, 1\}$" (or "$x_j \in \mathbb{Z}$") to an LP and you get an *integer linear program* (ILP). This one extra requirement makes the problem much harder: ILP is NP-hard in general (Chapter 33), and the corner-walking of simplex no longer finds integer corners.

The standard workaround is *LP relaxation*: drop the integrality requirement. Replace "$x_j \in \{0, 1\}$" with the interval "$0 \le x_j \le 1$," solve the resulting ordinary LP (fast!), and use its answer as a guide.

Intuition

The relaxed LP is more permissive – it allows fractional, "half a truck" solutions that the integer problem forbids. So the relaxed feasible region *contains* all the integer points and then some. For a minimization problem, allowing more options can only lower (or equal) the best value, so

$$\text{LP optimum} \;\leq\; \text{ILP optimum}.$$

The relaxation gives you a *bound* for free, and a fractional solution to round. That bound drives two important algorithmic techniques in the next chapters.

Figure 28.4 makes the containment concrete. The shaded LP polytope holds every legal integer point (the dots) but also a continuum of fractional points between them; the LP optimum can sit at a fractional corner the ILP could never reach.

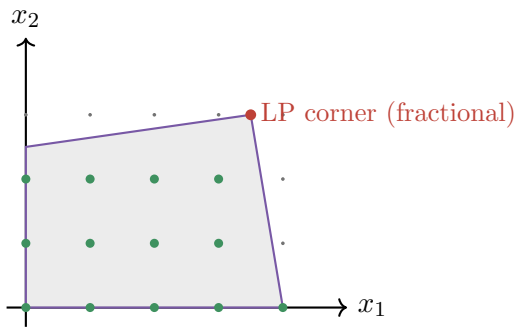


Figure 28.4: LP relaxation. The relaxed polytope (shaded, purple boundary) contains all the integer feasible points (green dots) and the fractional points between them. The relaxed optimum may land on a fractional corner (red), which is why its value bounds the integer optimum but its coordinates may need rounding.

Use 1: bounding for exact search. Branch-and-bound solves ILPs exactly by exploring a tree of subproblems. At each node it solves the LP relaxation; if the relaxed optimum is already worse than the best integer solution found so far, the entire subtree can be pruned – no integer point down there can beat what we have. The relaxation's bound does the pruning. This is how industrial ILP solvers (CPLEX, Gurobi) solve problems with millions of variables.

Use 2: rounding for approximation. When exact is too slow, solve the relaxation, then *round* the fractional solution to a nearby integer one, and prove the rounding doesn't cost too much. Consider Vertex Cover: choose a smallest set of vertices touching every edge. The ILP has $x_v \in \{0, 1\}$ and a constraint $x_u + x_v \geq 1$ for each edge (u, v). Relax to $0 \leq x_v \leq 1$, solve the LP, and round: put v in the cover iff $x_v \geq \frac{1}{2}$.

Focus

Why the rounding is safe and good: every edge constraint $x_u + x_v \geq 1$ forces at least one of x_u, x_v to be $\geq \frac{1}{2}$, so the rounded set still covers every edge (still feasible). And we at most *double* each variable (from $\geq \frac{1}{2}$ up to 1), so the rounded cost is at most $2 \times$ (LP optimum) $\leq$ $2 \times$ (ILP optimum). That is a **2-approximation**: a polynomial-time algorithm guaranteed

within a factor of 2 of optimal, with the LP relaxation supplying both the rounding recipe and the proof. We meet this style again in Chapter 34.

Walking question

Take this on a walk. The shortest-path-as-an-LP from Section 28.1 and the maximum-flow problem (which can also be written as an LP) both have a special structural property: when capacities or weights are integers, their LP relaxations *automatically* have integer optimal vertices – you never need to round. Why should some combinatorial LPs hand you whole numbers for free while Vertex Cover does not? Think about what is special about the constraint matrices of flow and shortest-path problems. (The keyword, for after your walk, is *total unimodularity*: matrices whose square submatrices all have determinant 0 or ± 1 guarantee integer vertices. Network-flow matrices are totally unimodular; the vertex-cover matrix is not.)

Focus

LP relaxation links continuous optimization to combinatorial problems. It connects directly to integer programming, NP-completeness (Chapter 33, CLRS Chapter 34), and approximation algorithms (Chapter 34, CLRS Chapter 35), where LP-rounding and primal-dual methods are central design strategies. CLRS Chapter 29 covers pure LP: standard and slack forms (29.1), formulating problems (29.2), simplex (29.3), duality (29.4), and the initial solution (29.5).

28.7 Summary

Starting from a two-variable workshop problem, we developed the core machinery of linear optimization:

- An **LP** maximizes a linear objective subject to linear constraints. Modeling means naming decision variables and writing the objective and constraints in their terms – staying strictly linear.
- **Standard form** (max, $\leq$, $x \geq 0$) is the canonical shape; **slack form** turns every inequality into an equality with a slack variable, splitting variables into basic and nonbasic.
- **Geometry**: the feasible region is a convex polytope, the optimum lives at a vertex, and you only ever need to check corners.
- The **simplex method** walks from vertex to adjacent vertex, always uphill, via entering/leaving pivots, stopping when no objective coefficient is positive. Fast in practice, exponential in the worst case.
- **Duality** pairs every LP with a shadow LP. Weak duality sandwiches the optimum; strong duality says primal and dual optima are equal; the dual variables are shadow prices that value each scarce resource.
- **LP relaxation** drops integrality to turn hard integer problems into solvable LPs, giving bounds for branch-and-bound and rounding recipes for approximation, which leads into the book's final chapters.

Linear programming optimizes linear forms over flat geometric polytopes. In the next chapter, we turn from straight boundaries to curves: polynomials, their dual coefficient and point-value representations, and the fast Fourier transform that multiplies degree-n polynomials in near-linear time.

Exercises

Exercise 28.1 (Model it)**.** A bakery makes muffins and cookies. A muffin earns \$2 and uses 3 oz flour and 1 egg; a cookie earns \$1 and uses 1 oz flour and 1 egg. There are 30 oz of flour and 12 eggs. Write the LP that maximizes revenue. Identify the decision variables, objective, and constraints.

Exercise 28.2 (Corners by hand)**.** For the bakery LP above, draw the feasible region in the muffin–cookie plane, list all its vertices, evaluate the revenue at each, and read off the optimal production plan. Confirm the optimum is at a corner.

Exercise 28.3 (Into standard form)**.** Convert to standard form: "minimize $4x_1 - x_2 + 2x_3$ subject to $x_1 - x_2 \geq 3$, $x_2 + x_3 = 5$, $x_1 \geq 0$, $x_2 \geq 0$, x_3 free." Show every transformation step.

Exercise 28.4 (Slack form and the origin)**.** Put the furniture LP's constraints into slack form, then write the basic feasible solution you get by setting both original variables to 0. Which constraints are tight there? Now do the same after raising the wood limit from 8 to 10 – does the origin remain feasible?

Exercise 28.5 (Run simplex)**.** Solve by simplex, showing every dictionary: "maximize $x_1 + 2x_2$ subject to $x_1 + x_2 \leq 4$, $x_1 + 3x_2 \leq 6$, $x_1, x_2 \geq 0$." Report the optimal vertex, the optimal value, and the path of vertices simplex visited.

Exercise 28.6 (Spot the unbounded LP)**.** Try to run simplex on "maximize $2x_1 + x_2$ subject to $x_1 - x_2 \leq 1$, $x_1, x_2 \geq 0$." At some iteration the ratio test will find no positive coefficient in the entering column. Explain why, and explain geometrically why this LP has no finite maximum.

Exercise 28.7 (Write the dual)**.** Write the dual of the bakery LP from Exercise 28.1. Identify what each dual variable would mean economically (the shadow price of which resource?).

Exercise 28.8 (Verify strong duality)**.** Solve the dual of the furniture LP (minimize $8y_1 + 12y_2$ subject to $y_1 + 2y_2 \geq 3$, $2y_1 + 2y_2 \geq 5$, $y \geq 0$) by checking its vertices. Confirm the optimal dual value equals the primal optimum of 22, and state which primal constraints the nonzero y_i correspond to.

Exercise 28.9 (Shadow price prediction)**.** Using the furniture shadow prices $y_1 = 2$ (wood) and $y_2 = \frac{1}{2}$ (labor), predict the new optimal profit if labor rises from 12 to 14 hours. Then re-solve the LP from scratch and check your prediction. For roughly how large an increase do you expect the shadow price to stay valid? (Hint: think about when the optimal corner changes.)

Exercise 28.10 (Relax and round)**.** Take the triangle graph with vertices $\{a, b, c\}$ and edges ab, bc, ca. Write the Vertex Cover ILP, then its LP relaxation. Show that $x_a = x_b = x_c = \frac{1}{2}$ is an optimal *fractional* solution with value $\frac{3}{2}$. Round by the "$x_v \geq \frac{1}{2}$" rule and report the resulting integer cover. Compare its size to the true optimum (which is 2) and to the $2 \times \frac{3}{2} = 3$ bound the analysis promises.

Exercise 28.11 (Challenge: shortest path is an LP)**.** Take a 4-vertex graph $s \to a \to t$ and $s \to t$ with weights $w(s,a) = 2$, $w(a,t) = 3$, $w(s,t) = 6$. Write the shortest-path LP from Section 28.1 (maximize $\sum_v d_v$ subject to $d_v - d_u \le w(u,v)$ and $d_s = 0$). Solve it and check that the d_v you get are the true shortest-path distances. Why does *maximizing* produce the *shortest* distances?

Exercise 28.12 (Challenge: dual of the dual)**.** Starting from the standard-form dual "minimize $b^\top y$ subject to $A^\top y \ge c$, $y \ge 0$," first convert it into a maximization in standard form, then take *its* dual using the recipe in Section 28.5. Show you recover the original primal "maximize $c^\top x$ subject to $Ax \le b$, $x \ge 0$." Conclude that dualizing twice is the identity.

Chapter 29

Polynomials and the FFT

In Chapter 28, we bounded and optimized linear functions whose straight boundaries create flat polyhedral geometries. When we move beyond flat boundaries to curved relationships, the natural objects of study are polynomials. Even their most basic operation, multiplication, can be done much faster than the obvious method suggests.

Multiplying two polynomials of degree bound n feels like it should cost a fixed amount of work — you learned the "multiply every term by every term" method in school, doing about n^2 coefficient multiplications. For $n = 1{,}000{,}000$ that is a trillion operations. For centuries, multiplication seemed intrinsically quadratic.

That assumption was wrong. The divide-and-conquer algorithm that beats it — the *Fast Fourier Transform*, or FFT — multiplies degree-n polynomials in $\Theta(n \log n)$ time. The key idea is to switch representations: instead of listing a polynomial's coefficients, we evaluate it at carefully chosen complex roots of unity, where multiplication takes only linear time.

Focus

By the end of this chapter you should be able to: (1) describe a polynomial in *coefficient* form and in *point-value* form, and say why each is good at a different job; (2) explain why multiplying in point-value form takes only linear time; (3) define the roots of unity and list the three algebraic facts about them that make the FFT work; (4) derive the FFT as a divide-and-conquer algorithm and prove its $\Theta(n \log n)$ running time from a recurrence; and (5) assemble the whole pipeline that multiplies two polynomials in $\Theta(n \log n)$ time.

29.1 Two ways to describe a polynomial

A polynomial of degree bound n (meaning degree at most $n - 1$) is the object

$$A(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1} = \sum_{j=0}^{n-1} a_j x^j.$$

The numbers $a_0, a_1, \ldots, a_{n-1}$ are its *coefficients*. This is the representation you have used your whole life, and it is so familiar that you may never have noticed it is a *choice*. We will call the list $(a_0, a_1, \ldots, a_{n-1})$ the *coefficient representation* of A.

Intuition

A polynomial is a function: feed it a number x, get a number $A(x)$ back. The coefficient list is one way to pin down which function it is. But there is a different way to pin down the same function: list some of the *values it actually produces*. "At $x = 0$ it outputs 3; at $x = 1$ it outputs 7; at $x = 2$ it outputs 17." If you give enough such observations, they nail down the function just as tightly as the coefficients do.

29.1.1 Point-value form

Here is the second representation. A *point-value representation* of a polynomial A of degree bound n is a set of n point-value pairs

$$\{(x_0, y_0), (x_1, y_1), \ldots, (x_{n-1}, y_{n-1})\}$$

where the x_k are all distinct and $y_k = A(x_k)$ for each k. So instead of the recipe, you record what the polynomial *evaluates to* at n chosen inputs.

Why exactly n points, no more and no fewer? Because of a familiar geometric fact generalized to arbitrary degree: through any two points there is exactly one line ($\deg \le 1$, so 2 coefficients); through any three points in general position there is exactly one parabola ($\deg \le 2$, so 3 coefficients); and in general, n points determine a unique polynomial of degree bound n.

Theorem 29.1 (Uniqueness of interpolation)**.** *Let $(x_0, y_0), \ldots, (x_{n-1}, y_{n-1})$ be any n point-value pairs with all x_k distinct. There is a* unique *polynomial A of degree bound n such that $A(x_k) = y_k$ for all k.*

Formal

Why it is true. Asking for $A(x_k) = y_k$ for $k = 0, \ldots, n-1$ is asking the unknown coefficients $a_0, \ldots, a_{n-1}$ to satisfy the linear system

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & & & & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}.$$

The matrix on the left is the *Vandermonde matrix* $V(x_0, \ldots, x_{n-1})$. Its determinant equals $\prod_{j<k}(x_k - x_j)$, which is nonzero exactly when all the x_k are distinct. A square linear system with a nonzero determinant has exactly one solution — so the coefficients exist and are unique. That is the whole proof.

The two representations are therefore *equivalent*: each one determines the same polynomial, and we may convert back and forth.

- **Evaluation** goes coefficient → point-value: pick the points x_k, compute $y_k = A(x_k)$.
- **Interpolation** goes point-value → coefficient: recover the a_j from the pairs (x_k, y_k). (This is solving the Vandermonde system, or running Lagrange's formula.)

Focus

Hold these two pictures side by side for the whole chapter:

coefficient form $(a_0, \ldots, a_{n-1})$ $\underset{\text{interpolate}}{\overset{\text{evaluate}}{\rightleftharpoons}}$ **point-value form** $\{(x_k, y_k)\}$.

The FFT is a fast way to move in either direction along this double arrow when the x_k are chosen well.

29.1.2 A small worked conversion

Take $A(x) = 1 + 2x + 3x^2$ (so $n = 3$, coefficients $(1, 2, 3)$) and evaluate it at $x_0 = 0$, $x_1 = 1$, $x_2 = 2$.

$$A(0) = 1, \qquad A(1) = 1 + 2 + 3 = 6, \qquad A(2) = 1 + 4 + 12 = 17.$$

So a point-value representation is $\{(0, 1), (1, 6), (2, 17)\}$. Three coefficients in, three pairs out — both forms carry exactly the same information about the same parabola. If somebody hands you those three pairs and you did not know the coefficients, Theorem 29.1 promises you could recover $(1, 2, 3)$ and nothing else.

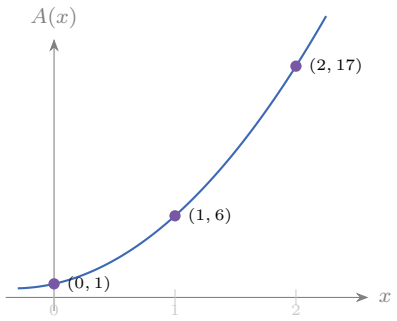


Figure 29.1: The equivalence of coefficient and point-value representations for $A(x) = 1 + 2x + 3x^2$. The three point-value pairs $(0, 1)$, $(1, 6)$, and $(2, 17)$ uniquely determine the blue parabola through polynomial interpolation.

The three purple dots are the point-value pairs; the blue curve is the unique parabola threaded through them. Move any dot and you get a different parabola — but fix all three and the curve has no freedom left, which is exactly what Theorem 29.1 asserts.

29.2 The operation we actually care about: multiplication

Why bother with two representations at all? The payoff shows up the moment we multiply. Suppose we want the *product* of two polynomials,

$$A(x) = \sum_{j=0}^{n-1} a_j x^j, \qquad B(x) = \sum_{j=0}^{n-1} b_j x^j,$$

giving a polynomial $C(x) = A(x)B(x)$ of degree bound $2n - 1$. In coefficient form, its coefficients are the *convolution* of the two coefficient lists:

$$c_k = \sum_{j=0}^{k} a_j \, b_{k-j}, \qquad k = 0, 1, \ldots, 2n-2.$$

Intuition

Convolution is just "collect every way to make x^k." To get the coefficient of x^k in the product, you pair up a term $a_j x^j$ from A with the term $b_{k-j}x^{k-j}$ from B, because their exponents add to k. You sum over all such pairs. Computing one c_k costs about k multiplications; summing over all k costs $\Theta(n^2)$. That is the schoolbook cost, and it is exactly what we want to beat.

Now consider *point-value* form. Suppose we already know A and B at the *same* set of points $x_0, \ldots, x_{m-1}$:

$$A : \{(x_k, A(x_k))\}, \qquad B : \{(x_k, B(x_k))\}.$$

Then the value of the product at any one point is just the product of the two values, because $C(x_k) = A(x_k)\,B(x_k)$ — multiplication of *functions* is multiplication of their *outputs*, pointwise. So

$$C : \{(x_k,\ A(x_k)\,B(x_k))\}.$$

Focus

Point-value multiplication takes linear time. If both polynomials are sampled at the same points, multiplying them is just multiplying the y-values one at a time: m scalar multiplications, total cost $\Theta(m)$. Linear! Compare that to the $\Theta(n^2)$ convolution in coefficient form.

There is one bookkeeping subtlety. The product C has degree bound $2n - 1$, so by Theorem 29.1 it needs at least $2n - 1$ points to be pinned down. So before sampling we treat A and B as degree-bound-$2n$ polynomials by padding their coefficient lists with zeros, and we evaluate both at $2n$ points. Then pointwise multiplication gives a valid point-value form of C.

Figure 29.2 captures this three-step multiplication strategy:

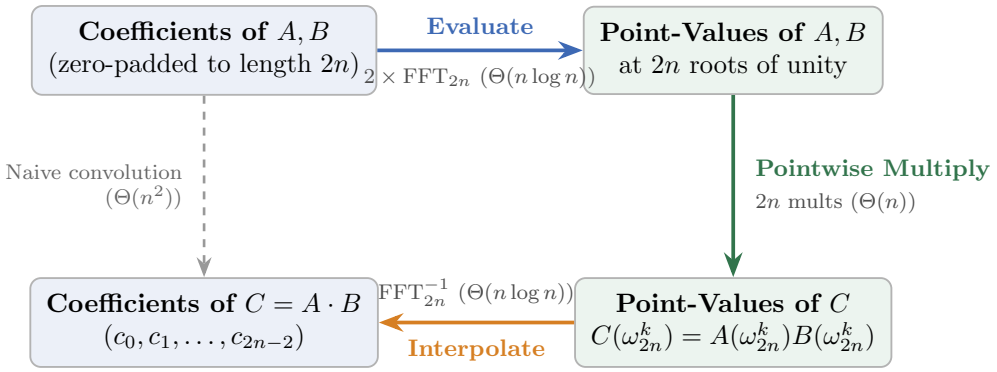


Figure 29.2: The three-step polynomial multiplication pipeline using the FFT. Both polynomials are evaluated at $2n$ complex roots of unity via forward FFTs in $\Theta(n \log n)$ time, multiplied pointwise in $\Theta(n)$ time, and interpolated back into coefficient form via the inverse FFT in $\Theta(n \log n)$ time.

The middle step is already linear. All the cost is in the side steps: *evaluate* (go right) and *interpolate* (come back). Done naively, each of those is $\Theta(n^2)$ — evaluating one polynomial at n points by Horner's rule is $\Theta(n)$ per point, $\Theta(n^2)$ for all points — and we have gained nothing. The FFT makes those two side trips cost $\Theta(n \log n)$. It does this by choosing, instead of arbitrary points x_k, points whose squares collapse into a set half the size.

29.3 Choosing the points: roots of unity

The choice of evaluation points is what makes the algorithm fast.

Intuition

Divide-and-conquer wants to split a problem of size n into two problems of size $n/2$ that we can solve and cheaply recombine. To evaluate A at n points, we would love to split A into two smaller polynomials and evaluate *those* at $n/2$ points each. For that to save work, the $n/2$ smaller points must somehow be "shared" between the two halves — evaluating the big polynomial at a point and its negative should reuse the same sub-computation. Ordinary points like $0, 1, 2, 3, \ldots$ have no such hidden relationships. In the *complex* plane, however, points evenly spaced around the unit circle halve in count under squaring *at every level of recursion*, all the way down to size 1. Those are the *roots of unity*.

29.3.1 Complex roots of unity

Before analyzing the algorithm, we review the basics. The complex numbers $\mathbb{C}$ extend ordinary real numbers by introducing the imaginary unit $i = \sqrt{-1}$, so that every point in the 2D plane can be written as $z = x + iy$. In polar coordinates, a point at distance r from the origin making angle θ with the positive horizontal axis is written $z = r(\cos\theta + i\sin\theta)$. Euler's formula gives this a compact and memorable form:

$$e^{i\theta} = \cos\theta + i\sin\theta.$$

Geometrically, **multiplying two complex numbers multiplies their lengths and adds their angles**:

$$(r_1 e^{i\theta_1}) \cdot (r_2 e^{i\theta_2}) = (r_1 r_2)\, e^{i(\theta_1+\theta_2)}.$$

In particular, multiplying by a number on the unit circle ($r = 1$) does not stretch or shrink anything; it is a *pure rotation* by angle θ.

Now consider solving the equation $z^n = 1$. A solution is called a **complex nth root of unity**. Since 1 has length 1 and angle 0 (or any integer multiple of 2π), any root $z = re^{i\theta}$ must have $r^n = 1 \implies r = 1$, and $n\theta = 2\pi k \implies \theta = 2\pi k/n$ for integer k. Setting $k = 1$ gives the *principal* nth root of unity:

$$\omega_n = e^{2\pi i/n} = \cos\left(\frac{2\pi}{n}\right) + i\sin\left(\frac{2\pi}{n}\right).$$

Successive powers $\omega_n^k = (\omega_n)^k = e^{2\pi ik/n}$ correspond to taking k successive rotation steps of angle $2\pi/n$ ($360°/n$) around the unit circle. For $k = 0, 1, \ldots, n-1$, these produce exactly n distinct, evenly spaced vertices of a regular n-sided polygon inscribed in the unit circle.

Figure 29.3 shows the eight 8th roots of unity, and how squaring them accomplishes the halving step.

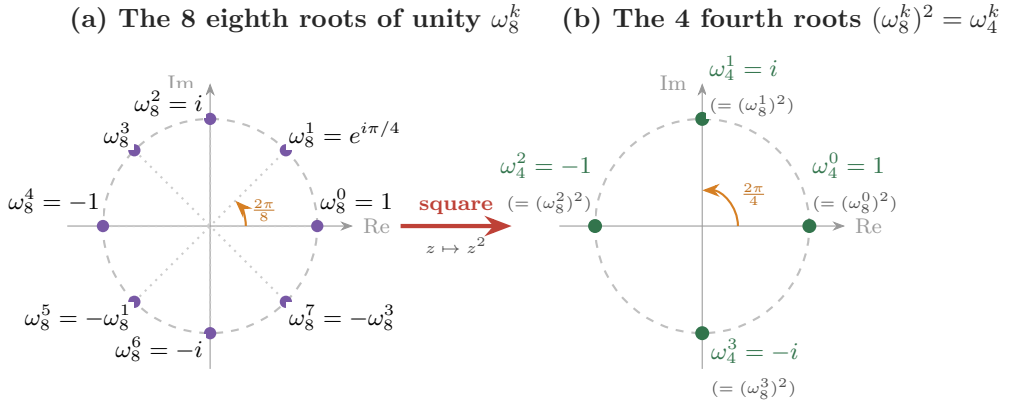


Figure 29.3: Roots of unity on the complex unit circle and the Halving Lemma. (a) The 8 eighth roots of unity $\omega_8^k = e^{2\pi ik/8}$ form a regular octagon in the complex plane, pairing antipodally via $\omega_8^{k+4} = -\omega_8^k$. (b) Squaring each point doubles its angle, mapping opposite antipodal pairs $(\omega_8^k)^2 = (\omega_8^{k+4})^2 = \omega_4^k$ onto the 4 fourth roots of unity, collapsing the evaluation problem to half its size.

Notice they come in *antipodal pairs*: ω_8^0 and $\omega_8^4 = -1$ are opposite; so are ω_8^1 and ω_8^5; and so on. Divide-and-conquer will use this antipodal pairing.

29.3.2 Three facts that make everything work

The FFT relies on three small algebraic facts. Assume throughout that n is even, and in fact a power of two.

Lemma 29.1 (Cancellation)**.** *For any integers $n > 0$, $d > 0$, and $k \geq 0$, $\omega_{dn}^{dk} = \omega_n^k$. In particular $\omega_n^{k+n/2} = -\omega_n^k$ (taking $d = 2$ together with $\omega_2^1 = -1$).*

Formal

Why. $\omega_{dn}^{dk} = \left(e^{2\pi i/(dn)}\right)^{dk} = e^{2\pi i\, dk/(dn)} = e^{2\pi i\, k/n} = \omega_n^k$. The exponents just cancel. For the second statement, $\omega_n^{k+n/2} = \omega_n^k\, \omega_n^{n/2}$, and $\omega_n^{n/2} = e^{2\pi i\,(n/2)/n} = e^{\pi i} = -1$. So evaluating at a root and at its antipode differ only by a sign — this is the "share the work between a point and its negative" phenomenon we hoped for.

Lemma 29.2 (Halving)**.** *If $n > 0$ is even, then the squares of the n complex nth roots of unity are the $n/2$ complex $(n/2)$th roots of unity,* each appearing exactly twice.

Formal

Why. Square the kth root: $\left(\omega_n^k\right)^2 = \omega_n^{2k} = \omega_{n/2}^k$ by cancellation (with $d = 2$, replacing n by $n/2$). As k runs $0, 1, \ldots, n-1$, the value $\omega_{n/2}^k$ cycles through all $n/2$ of the $(n/2)$th roots, and because ω_n^k and $\omega_n^{k+n/2}$ square to the same thing (their squares differ by $\omega_n^n = 1$), each value is hit exactly twice.

Intuition

The halving lemma is what makes the FFT fast. It says: if I want to evaluate a polynomial at all n roots of unity, the *squares* of those points form a problem half the size — the $n/2$ roots of unity — and the squares are what a divide step naturally produces. So a size-n evaluation reduces to a size-$n/2$ evaluation. Repeating this halving gives the $\log n$ levels of recursion.

Lemma 29.3 (Summation)**.** *For any integer k not divisible by n, $\sum_{j=0}^{n-1} (\omega_n^k)^j = 0$. (When k is divisible by n, every term is 1 and the sum is n.)*

Formal

Why. This is a geometric series with ratio ω_n^k:

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} = \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} = \frac{1^k - 1}{\omega_n^k - 1} = 0,$$

valid because $\omega_n^k \neq 1$ when $n \nmid k$, so we are not dividing by zero. The summation lemma is what makes *interpolation* (the trip back) work; we will use it when we invert the FFT.

29.4 The Discrete Fourier Transform

Now we can name the object we are computing. Evaluate A at the n roots of unity; the resulting list of values is important enough to have its own name.

Definition 29.1 (DFT)**.** Let A have coefficient vector $(a_0, \ldots, a_{n-1})$. Its *Discrete Fourier Transform* is the vector $(y_0, \ldots, y_{n-1})$ where

$$y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \, \omega_n^{kj}, \qquad k = 0, 1, \ldots, n-1.$$

We write $\mathbf{y} = \mathrm{DFT}_n(\mathbf{a})$.

So the DFT is just "evaluate at the roots of unity," packaged. Computed directly from the definition — n values, each a sum of n terms — it costs $\Theta(n^2)$. The *Fast* Fourier Transform is an algorithm that produces the very same vector $\mathbf{y}$ in $\Theta(n \log n)$ time.

Why this matters

The DFT is one of the most widely used computations. Sampled sound, the pixels streaming off a camera sensor, the radio waves hitting a cellphone tower, the seismograph trace of an earthquake — all of these are sequences of numbers, and asking "which frequencies are present, and how strong?" is exactly asking for their DFT. Without a *fast* way to compute it, digital audio, image compression (JPEG), wireless communication (OFDM in Wi-Fi and 4G/5G), and medical imaging would all be impractical. Gilbert Strang has called the FFT "the most

important numerical algorithm of our lifetime."

29.5 The FFT: divide and conquer on the coefficients

We want $A(\omega_n^k)$ for all k, where $A(x) = a_0 + a_1 x + \cdots + a_{n-1}x^{n-1}$ and n is a power of two. Split A into its *even-indexed* and *odd-indexed* coefficients:

$$A^{[0]}(x) = a_0 + a_2 x + a_4 x^2 + \cdots + a_{n-2}x^{n/2-1},$$
$$A^{[1]}(x) = a_1 + a_3 x + a_5 x^2 + \cdots + a_{n-1}x^{n/2-1}.$$

Each of these is a polynomial of degree bound $n/2$ — half the size. And they reassemble the original by

$$A(x) = A^{[0]}(x^2) + x\,A^{[1]}(x^2).$$

Intuition

Group the even-power terms: $a_0 + a_2x^2 + a_4x^4 + \cdots$ is exactly $A^{[0]}$ evaluated at x^2. Group the odd-power terms: $a_1x + a_3x^3 + \cdots = x(a_1 + a_3x^2 + \cdots) = x\,A^{[1]}(x^2)$. Add them, you recover $A(x)$. So to evaluate A at a point x, it suffices to evaluate the two half-size polynomials at x^2 and combine with one multiply and one add.

Now plug in the roots of unity for x. To get all of $A(\omega_n^0), \ldots, A(\omega_n^{n-1})$, we need $A^{[0]}$ and $A^{[1]}$ evaluated at the *squares* $\left(\omega_n^k\right)^2$. But by the halving lemma those squares are exactly the $n/2$ roots $\omega_{n/2}^k$ — and there are only $n/2$ distinct ones! So:

Focus

Evaluating A at the n roots of unity reduces to evaluating $A^{[0]}$ and $A^{[1]}$ — each half the degree — at the $n/2$ roots of unity. Two subproblems of half the size. That is a clean divide-and-conquer split, and the halving lemma is what guarantees the recursion lands back on the same kind of problem.

29.5.1 The combine step, made explicit

Suppose the two recursive calls have returned. Let $y_k^{[0]} = A^{[0]}(\omega_{n/2}^k)$ and $y_k^{[1]} = A^{[1]}(\omega_{n/2}^k)$ for $k = 0, \ldots, n/2 - 1$. We want $y_k = A(\omega_n^k)$ for $k = 0, \ldots, n-1$.

For k in the *lower* half, $0 \le k < n/2$, plug $x = \omega_n^k$ into the identity and use $\left(\omega_n^k\right)^2 = \omega_{n/2}^k$:

$$y_k = A^{[0]}(\omega_{n/2}^k) + \omega_n^k\,A^{[1]}(\omega_{n/2}^k) = y_k^{[0]} + \omega_n^k\,y_k^{[1]}.$$

For the *upper* half, look at index $k + n/2$. This is where the cancellation lemma is used. We have $\left(\omega_n^{k+n/2}\right)^2 = \omega_{n/2}^k$ (squaring removes the antipodal sign), so $A^{[0]}$ and $A^{[1]}$ are evaluated at the *same* point as before; and $\omega_n^{k+n/2} = -\omega_n^k$. Therefore

$$y_{k+n/2} = A^{[0]}(\omega_{n/2}^k) - \omega_n^k\,A^{[1]}(\omega_{n/2}^k) = y_k^{[0]} - \omega_n^k\,y_k^{[1]}.$$

Intuition

The quantity $\omega_n^k\, y_k^{[1]}$ — called the *twiddle factor* times the odd value — is computed once, then *added* to give the lower-half answer and *subtracted* to give the upper-half answer. This add/subtract pair is called the "butterfly," and it is the reason we get n output values from only $n/2$ subproblem values plus $n/2$ multiplications.

Figure 29.4 shows this basic computational cell:

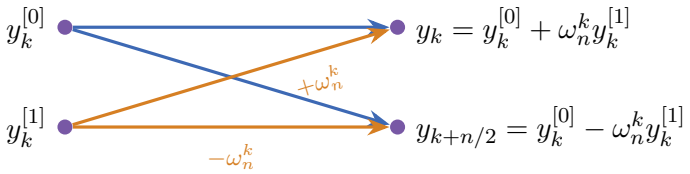


Figure 29.4: The elementary butterfly unit of the FFT. Multiplying the subproblem value $y_k^{[1]}$ by the twiddle factor ω_n^k once provides the additive contribution for the lower-half output y_k and the subtractive contribution for the upper-half output $y_{k+n/2}$.

29.5.2 The algorithm

Algorithm 124: RECURSIVE-FFT($\mathbf{a}$)

Input: A coefficient vector $\mathbf{a} = (a_0, \ldots, a_{n-1})$ with n a power of 2.
Output: $\mathbf{y} = \mathrm{DFT}_n(\mathbf{a})$, i.e. $y_k = A(\omega_n^k)$.

```
if n = 1 then
    return a;                                   // a constant:  A(ω_1^0) = a_0
ω_n ← e^{2πi/n};   ω ← 1;
a^[0] ← (a_0, a_2, ..., a_{n-2});
a^[1] ← (a_1, a_3, ..., a_{n-1});
y^[0] ← RECURSIVE-FFT(a^[0]);
y^[1] ← RECURSIVE-FFT(a^[1]);
for k ← 0 to n/2 − 1 do
    t ← ω · y_k^[1];                            // twiddle × odd value
    y_k ← y_k^[0] + t;
    y_{k+n/2} ← y_k^[0] − t;
    ω ← ω · ω_n;                                // advance to next root:  ω = ω_n^k
return y;
```

The loop maintains $\omega = \omega_n^k$ as an invariant, updating it by a single multiplication each pass rather than recomputing a power from scratch. The base case $n = 1$ returns the single coefficient unchanged, because a degree-0 polynomial $A(x) = a_0$ evaluates to a_0 everywhere, in particular at $\omega_1^0 = 1$.

29.5.3 Correctness, gently

Formal

Claim. RECURSIVE-FFT(**a**) returns $y_k = A(\omega_n^k)$ for every k.
Induction on n (a power of 2). *Base case* $n = 1$: the only root is $\omega_1^0 = 1$, and $A(1) = a_0$, which is what we return. *Inductive step:* assume the two recursive calls are correct on inputs of size $n/2$, so $y_k^{[0]} = A^{[0]}(\omega_{n/2}^k)$ and $y_k^{[1]} = A^{[1]}(\omega_{n/2}^k)$. The combine step computes, for $0 \le k < n/2$,

$$y_k = y_k^{[0]} + \omega_n^k y_k^{[1]}, \qquad y_{k+n/2} = y_k^{[0]} - \omega_n^k y_k^{[1]}.$$

We showed above, using the identity $A(x) = A^{[0]}(x^2) + xA^{[1]}(x^2)$ and the halving/cancellation lemmas, that these equal $A(\omega_n^k)$ and $A(\omega_n^{k+n/2})$ respectively. As k ranges over $0, \dots, n/2 - 1$, the indices k and $k + n/2$ cover all of $0, \dots, n - 1$. So every output is correct. By induction the algorithm is correct for all powers of two. □

29.5.4 Running time

Each call does the two recursive calls on size $n/2$, plus a loop of $n/2$ iterations doing constant work each, plus building the even/odd lists in $\Theta(n)$. So

$$T(n) = 2\,T(n/2) + \Theta(n).$$

You have met this recurrence before — it is merge sort's recurrence. By the master theorem (case 2, since $n^{\log_2 2} = n$ matches the $\Theta(n)$ work), or by examining the recursion tree in Figure 29.5:

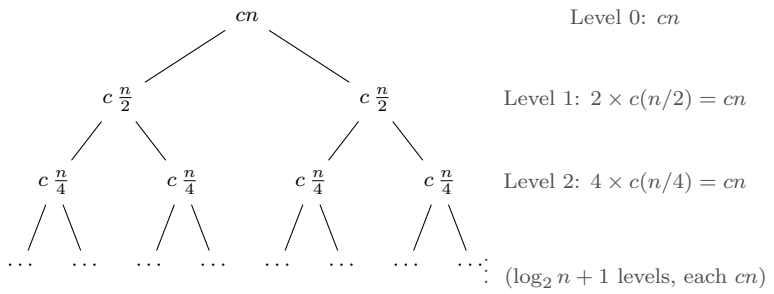


Figure 29.5: The recursion tree for the divide-and-conquer FFT recurrence $T(n) = 2T(n/2) + cn$. Each level incurs total work cn, and there are $\log_2 n + 1$ levels in the tree, giving total time $\Theta(n \log n)$.

Every level of the tree does a total of cn work — the subproblems shrink but there are proportionally more of them — and there are $\log_2 n + 1$ levels, so

$$T(n) = \Theta(n \log n).$$

We have evaluated a polynomial at n points in $n \log n$ time.

Cost summary

Task	Coefficient form	with the FFT
Evaluate at n roots of unity (DFT)	$\Theta(n^2)$	$\Theta(n \log n)$
Interpolate from n roots (inverse DFT)	$\Theta(n^2)$	$\Theta(n \log n)$
Multiply two degree-n polynomials	$\Theta(n^2)$	$\Theta(n \log n)$
Pointwise multiply n values	$\Theta(n)$	$\Theta(n)$

29.6 Coming back: the inverse FFT

We can travel coefficient $\to$ point-value fast. To finish the multiplication pipeline we need the return trip, point-value $\to$ coefficient, just as fast. It turns out the inverse DFT is *almost the same algorithm.*

Write the DFT as a matrix-vector product $\mathbf{y} = V_n \mathbf{a}$, where V_n is the Vandermonde matrix at the roots of unity: its (k, j) entry is ω_n^{kj}. Inverting the DFT means solving $\mathbf{a} = V_n^{-1}\mathbf{y}$, and the summation lemma gives us the inverse outright.

Theorem 29.2 (Inverse DFT)**.** *The inverse of V_n has (k, j) entry $\frac{1}{n}\omega_n^{-kj}$. Equivalently,*

$$a_j = \frac{1}{n}\sum_{k=0}^{n-1} y_k\, \omega_n^{-kj}, \qquad j = 0, \ldots, n-1.$$

Formal

Why. Multiply the candidate inverse W (with $W_{jk} = \frac{1}{n}\omega_n^{-kj}$) by V_n and look at the (j, j') entry of the product:

$$[WV_n]_{jj'} = \sum_{k=0}^{n-1} \frac{1}{n}\,\omega_n^{-kj}\,\omega_n^{kj'} = \frac{1}{n}\sum_{k=0}^{n-1} \omega_n^{k(j'-j)}.$$

By the summation lemma (Lemma 29.3) the inner sum is n when $j = j'$ and 0 otherwise. So $[WV_n]_{jj'}$ is 1 on the diagonal and 0 off it: $WV_n = I$. Hence $W = V_n^{-1}$. $\square$

Put the inverse formula next to the forward one:

$$y_k = \sum_j a_j\, \omega_n^{kj} \qquad \text{vs.} \qquad a_j = \frac{1}{n}\sum_k y_k\, \omega_n^{-kj}.$$

They differ in exactly two cosmetic ways: replace ω_n by ω_n^{-1}, and divide the final answer by n. So we can compute the inverse DFT with the *same* RECURSIVE-FFT code: feed it $\mathbf{y}$, use ω_n^{-1} in place of ω_n, and divide every output by n. The cost is therefore the same $\Theta(n \log n)$.

Walking question

Take a walk and convince yourself that ω_n^{-1} is itself a primitive nth root of unity — it is just ω_n traversed clockwise instead of counterclockwise. Then ask: does the halving lemma still

hold for ω_n^{-1}? (It must, or the same code could not work.) Why? Hint: the roots of unity as a *set* do not care which direction you number them.

29.7 The whole pipeline: fast polynomial multiplication

Now we can assemble the algorithm promised at the start of the chapter.

Algorithm 125: POLY-MULTIPLY($\mathbf{a}, \mathbf{b}$)

Input: Coefficient vectors of A, B, each of degree bound n.
Output: Coefficient vector of $C = A \cdot B$, degree bound $2n - 1$.

Let m be the smallest power of 2 with $m \geq 2n$;
Pad $\mathbf{a}$ and $\mathbf{b}$ with zeros up to length m;
$\mathbf{y}_A \leftarrow$ RECURSIVE-FFT($\mathbf{a}$); // evaluate A at m roots
$\mathbf{y}_B \leftarrow$ RECURSIVE-FFT($\mathbf{b}$); // evaluate B at m roots
for $k \leftarrow 0$ **to** $m - 1$ **do**
    $\mathbf{y}_C[k] \leftarrow \mathbf{y}_A[k] \cdot \mathbf{y}_B[k]$; // pointwise multiply
$\mathbf{c} \leftarrow$ INVERSE-FFT($\mathbf{y}_C$); // interpolate back
return $\mathbf{c}$ (real parts, rounded if inputs were integers);

Focus

Three FFTs ($\Theta(m \log m)$), one pointwise loop ($\Theta(m)$), and $m = \Theta(n)$. Total: $\Theta(n \log n)$. We multiply two polynomials — equivalently, convolve two sequences — in $n \log n$ time. This beats the schoolbook n^2 bound.

29.7.1 A complete numerical trace

Let us multiply $A(x) = 1 + 2x$ and $B(x) = 3 + 4x$. By hand, $C(x) = 3 + 10x + 8x^2$, so we are hoping to recover coefficients $(3, 10, 8)$. The product has degree bound 3, so we pad to $m = 4$ and use the 4th roots of unity

$$\omega_4^0 = 1, \quad \omega_4^1 = i, \quad \omega_4^2 = -1, \quad \omega_4^3 = -i.$$

Padded coefficients: $\mathbf{a} = (1, 2, 0, 0)$, $\mathbf{b} = (3, 4, 0, 0)$.

Step 1: evaluate A. $A(x) = 1 + 2x$ (the padding zeros do nothing), so

$$y_A = \big(A(1), A(i), A(-1), A(-i)\big) = (3,\ 1 + 2i,\ -1,\ 1 - 2i).$$

Step 1b: evaluate B. $B(x) = 3 + 4x$, so

$$y_B = (7,\ 3 + 4i,\ -1,\ 3 - 4i).$$

Step 2: pointwise multiply.

$$y_C = \big(3 \cdot 7,\ (1 + 2i)(3 + 4i),\ (-1)(-1),\ (1 - 2i)(3 - 4i)\big).$$

Work out the middle two: $(1 + 2i)(3 + 4i) = 3 + 4i + 6i + 8i^2 = 3 + 10i - 8 = -5 + 10i$, and its conjugate $(1 - 2i)(3 - 4i) = -5 - 10i$. So

$$y_C = (21,\ -5 + 10i,\ 1,\ -5 - 10i).$$

Step 3: inverse DFT. Use $a_j = \frac{1}{4}\sum_k y_{C,k}\,\omega_4^{-kj}$ with $\omega_4^{-1} = -i$. For example, $c_0 = \frac{1}{4}(21 + (-5 + 10i) + 1 + (-5 - 10i)) = \frac{1}{4}(12) = 3$. And

$$c_1 = \tfrac{1}{4}\big(21 \cdot 1 + (-5 + 10i)(-i) + 1 \cdot (-1) + (-5 - 10i)(i)\big).$$

Compute the pieces: $(-5 + 10i)(-i) = 5i + 10 = 10 + 5i$; $(-5 - 10i)(i) = -5i + 10 = 10 - 5i$. Sum: $21 + (10 + 5i) - 1 + (10 - 5i) = 40$, so $c_1 = 40/4 = 10$. A similar computation gives $c_2 = 8$ and $c_3 = 0$.

Algorithm lab

Trace it yourself. You just watched $(1 + 2x)(3 + 4x)$ come out as $3 + 10x + 8x^2$ through the FFT machinery. Now do $A(x) = 1 + x + x^2$ times $B(x) = 1 - x$ by the same recipe.

1. The product has degree 3, so pad to $m = 4$. Write the padded vectors.
2. Evaluate both at $1, i, -1, -i$. (You already know $A(i) = 1 + i + i^2 = i$.)
3. Multiply the four value-pairs.
4. Run the inverse formula $c_j = \frac{1}{4}\sum_k y_{C,k}(-i)^{kj}$.
5. Check against the schoolbook answer $(1 + x + x^2)(1 - x) = 1 - x^3$, i.e. coefficients $(1, 0, 0, -1)$. Did the imaginary parts cancel to (almost) zero? They must, since real inputs give a real product.

Common trap

Five classic mistakes.

- **Forgetting to pad.** If you sample A and B at only n points, you get n values of C, but C has degree bound $2n - 1$ and needs $2n$ values. Your interpolation will silently produce the wrong ("wrapped-around") answer. Always pad to length $\geq 2n$, rounded up to a power of two.
- **Using $+\omega_n$ instead of ω_n^{-1} on the way back**, or **forgetting the** $1/n$. Either one corrupts the inverse transform.
- **Assuming the imaginary parts are exactly zero.** With floating-point arithmetic the recovered coefficients carry tiny imaginary residue like $10^{-15}i$; take real parts and (for integer problems) round.
- **Thinking the FFT needs n prime, or n anything other than a power of two.** The radix-2 FFT in this chapter needs n to be a power of two; pad up to one. (More general FFTs exist, but start here.)
- **Confusing degree with degree bound.** A polynomial of *degree* d has *degree bound* $d + 1$, i.e. $d + 1$ coefficients. Off-by-one errors here cause the worst padding bugs.

29.8 Why the same code runs forward and backward

The symmetry deserves one more pass, because it is easy to use the inverse FFT mechanically without seeing *why* it works out so neatly.

Intuition

The forward DFT mixes the coefficients into values by "winding" them around the circle counterclockwise (multiplying by $\omega_n = e^{+2\pi i/n}$). The inverse unwinds them by going clockwise ($\omega_n^{-1} = e^{-2\pi i/n}$) and then dividing out the factor n that the winding introduced. Because both directions use roots of unity — and the roots of unity are the *same set* whether you number them clockwise or counter — the halving and butterfly structure is identical. The FFT runs equally well in reverse; you only change the root ($\omega \to \omega^{-1}$) and apply a final scale ($\div n$).

Why this matters

This forward/backward symmetry is why the FFT is central to signal processing. To filter a sound (remove hiss, boost bass, cancel an echo) you FFT into the frequency domain, tweak the frequency values, and inverse-FFT back to a waveform — and both legs cost $n \log n$. Convolution in the time domain becomes *pointwise multiplication* in the frequency domain, which is the same fact we used to multiply polynomials. Audio plug-ins, noise-cancelling headphones, MRI reconstruction, and the modulation scheme in every Wi-Fi router all depend on this $n \log n$ algorithm.

History and connection

Carl Friedrich Gauss used essentially the divide-and-conquer FFT around 1805 to calculate the orbits of asteroids Ceres and Pallas, two years before Joseph Fourier's work was published; his Latin notes lay unnoticed in his collected works for over a century. In 1965, James Cooley and John Tukey independently rediscovered the algorithm to process seismic sensor data for detecting nuclear tests. This material is **CLRS Chapter 30** ("Polynomials and the FFT"): Section 30.1 covers the two representations and the multiplication strategy, Section 30.2 develops the DFT and RECURSIVE-FFT, and Section 30.3 treats efficient implementations (the iterative, in-place FFT with bit-reversal). In 2019, David Harvey and Joris van der Hoeven showed that the FFT family reaches the integer multiplication bound $O(n \log n)$. For the number theory behind doing FFTs in modular arithmetic — the Number-Theoretic Transform, which avoids floating-point errors — see Chapter 30.

29.9 A note on making it iterative and fast in practice

The recursive code is the clearest way to *understand* the FFT, but production implementations are usually *iterative* and operate in place. Two ideas drive this.

First, *bit-reversal*. If you trace which input coefficient ends up feeding each leaf of the recursion, you find that coefficient a_j lands in the slot whose index is the binary digits of j *reversed*. For $n = 8$, a_1 (001_2) lands at position $100_2 = 4$, a_3 (011_2) lands at $110_2 = 6$, and so on. Permute the input by bit-reversal once at the start, and the recursion's even/odd splitting is already laid out for you.

Second, *building up* instead of down. After bit-reversal, combine adjacent pairs with butterflies into transforms of size 2, then size 4, then size 8, ... up to size n — $\log_2 n$ passes, each touching all n elements with $n/2$ butterflies. No recursion stack, no array allocation per call, excellent cache behavior. The running time is still $\Theta(n \log n)$, with a much smaller constant.

Figure 29.6 illustrates the complete iterative FFT butterfly network for $n = 8$. The inputs enter on the left in bit-reversed order; three sequential stages of butterflies combine subproblems of size 2, then 4, then 8, advancing rightward into the final Discrete Fourier Transform values $y_0, \ldots, y_7$.

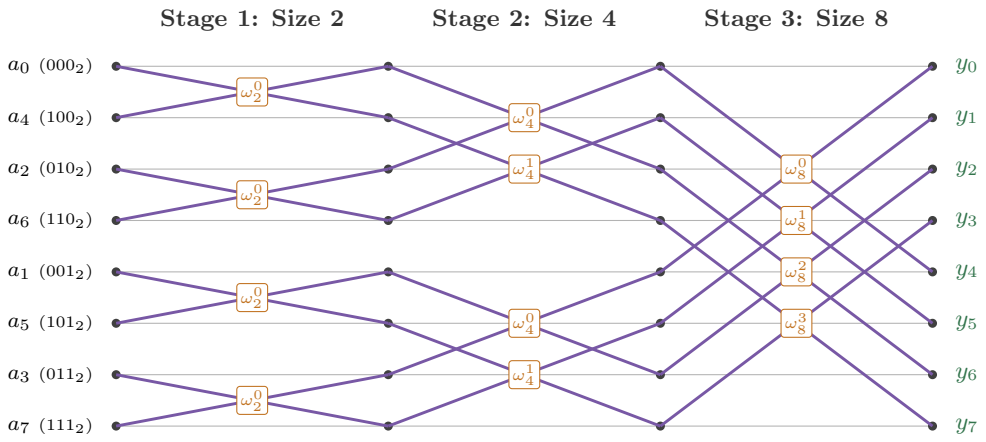


Figure 29.6: The iterative 8-point FFT butterfly network. Input coefficients enter on the left in bit-reversed order $(a_0, a_4, a_2, a_6, a_1, a_5, a_3, a_7)$. Three successive butterfly stages combine subtransforms of size 2, 4, and 8 using twiddle factors ω_n^k, yielding the DFT outputs $y_0, \ldots, y_7$ in natural order in $\Theta(n \log n)$ time with zero recursion overhead.

Cost summary

Recursive vs. iterative FFT. Both are $\Theta(n \log n)$. The iterative, in-place version (CLRS 30.3) wins by a constant factor: it avoids per-call allocation, computes twiddle factors with one multiply per use, and streams through memory in a cache-friendly pattern. For the asymptotics in this book either is fine; for real software, write the iterative one.

Pause

If this chapter went by fast, here is the one paragraph to keep. A polynomial can be written by its coefficients *or* by its values at chosen points; multiplication is slow (n^2) in coefficients but fast (n) in values. Converting between the two forms is normally n^2, but if you choose the points to be the nth roots of unity — evenly spaced on the unit circle — then a divide-and-conquer split on even/odd coefficients turns the conversion into $n \log n$. That conversion is the FFT; running it forward, multiplying pointwise, and running it backward multiplies polynomials in $\Theta(n \log n)$.

Polynomial multiplication via the FFT relied on complex roots of unity and floating-point arithmetic. In Chapter 30, we turn from continuous rotations in the complex plane to the discrete structures of number theory: modular arithmetic, greatest common divisors, primality testing, and the algebraic mechanisms that secure public-key cryptography.

29.10 Exercises

Exercise 29.1. Convert $A(x) = 2 - x + x^2$ from coefficient form to a point-value representation using the points $x = -1, 0, 1$. Then, pretending you only had those three pairs, reconstruct the polynomial with Lagrange interpolation and confirm you get the original coefficients back.

Exercise 29.2. Write out the convolution sum for c_k when $A = (1, 2, 3)$ and $B = (4, 5, 6)$, computing all five coefficients $c_0, \ldots, c_4$ of the product by hand. How many scalar multiplications did the schoolbook method use? Now state the count for general degree bound n.

Exercise 29.3. List the six 6th roots of unity in the form $a + bi$ (use exact values where you can, decimals otherwise). Verify directly that their sum is 0, illustrating the summation lemma.

Exercise 29.4. Compute the squares of all eight 8th roots of unity and confirm by hand that the results are exactly the four 4th roots of unity, each appearing twice (the halving lemma).

Exercise 29.5. Run RECURSIVE-FFT on the input $\mathbf{a} = (1, 2, 3, 4)$ ($n = 4$). Show the even/odd split, the two size-2 recursive results, and the butterfly combine. Your answer should be the four values $A(1), A(i), A(-1), A(-i)$. Check by evaluating $A(x) = 1 + 2x + 3x^2 + 4x^3$ directly.

Exercise 29.6. Take the output vector you computed in Exercise 29.5 and run the inverse DFT on it (use $\omega_4^{-1} = -i$ and divide by 4). You should recover $(1, 2, 3, 4)$. Show at least the computations for a_0 and a_2.

Exercise 29.7. Use the full POLY-MULTIPLY pipeline to compute $(1 + x + x^2)(2 + 3x)$. Pad to the right power of two, do the two forward FFTs, multiply pointwise, and inverse-transform. Confirm the coefficients match the schoolbook product.

Exercise 29.8. For $n = 8$, write the bit-reversal permutation: for each index $0, \ldots, 7$ give its 3-bit binary, reverse the bits, and report the destination index. Which indices are fixed points (map to themselves)?

Exercise 29.9. The recurrence for RECURSIVE-FFT is $T(n) = 2T(n/2) + cn$. Solve it two ways: (a) by drawing the recursion tree and summing the per-level work; (b) by the master theorem, identifying which case applies and why. Confirm both give $\Theta(n \log n)$.

Exercise 29.10. *(Challenge.)* Explain how to multiply two n-digit integers in $\Theta(n \log n)$ time using the FFT. (Hint: an integer $\overline{d_{n-1} \cdots d_1 d_0}$ in base 10 is the polynomial $\sum d_j 10^j$ evaluated at $x = 10$. Multiply the polynomials, then handle carries.) What goes wrong if you forget to pad, and what is the role of carrying after the convolution?

Exercise 29.11. *(Reflection.)* The summation lemma says the roots of unity sum to zero for any k not a multiple of n. Explain geometrically why this must be true by thinking of the roots as vectors pointing from the origin, equally spaced around the circle. (What is the "center of mass" of evenly spaced points on a circle?)

Chapter 30

Number-Theoretic Algorithms

In Chapter 29, we manipulated polynomials and signals using complex roots of unity. Here we turn from continuous rotations to discrete arithmetic on whole numbers: which integers divide which, what is left over when they do not, and how arithmetic behaves when we retain only the remainder. This is *number theory*, the mathematical foundation of modern cryptography.

For most of this book, data items have been treated as opaque tokens. In this chapter we open integers up to inspect their internal structure. We begin with Euclid's greatest common divisor algorithm, extend it to solve modular linear equations and compute multiplicative inverses, use repeated squaring for fast modular exponentiation, and combine these tools into the Miller–Rabin randomized primality test and the RSA cryptosystem.

Throughout, we track both arithmetic operations and bit-level costs for β-bit operands, since what makes these algorithms useful is that they stay efficient even when numbers span hundreds of digits.

30.1 Divisibility, GCD, and the Idea of a Remainder

Let us begin where every number-theory course begins, with the relation of one integer *dividing* another.

Definition 30.1 (Divisibility)**.** For integers a and d with $d \neq 0$, we say d *divides* a, written $d \mid a$, if there is an integer k with $a = k\,d$. We call d a *divisor* of a and a a *multiple* of d. If d does not divide a we write $d \nmid a$.

So $3 \mid 12$ because $12 = 4 \cdot 3$, but $3 \nmid 13$. Every integer d divides 0 (take $k = 0$), and 1 divides everything.

Everything that follows depends on one more fact: when d does *not* evenly divide a, there is a unique leftover.

Theorem 30.1 (Division theorem)**.** *For any integer a and any positive integer n, there exist unique integers q (the* quotient*) and r (the* remainder*) such that*

$$a = q\,n + r \qquad \text{and} \qquad 0 \le r < n.$$

We write $q = \lfloor a/n \rfloor$ and $r = a \bmod n$.

You have known this since grade school as "long division with a remainder." The remainder $a \bmod n$ is always dragged into the range $\{0, 1, \ldots, n-1\}$, even when a is negative: $(-7) \bmod 3 = 2$, because $-7 = (-3) \cdot 3 + 2$. Hold onto that; sign conventions for mod trip up almost everyone at least once.

30.1.1 The greatest common divisor

Definition 30.2 (GCD)**.** The *greatest common divisor* of two integers a and b, not both zero, written $\gcd(a, b)$, is the largest integer that divides both. We set $\gcd(0, 0) = 0$ by convention. If $\gcd(a, b) = 1$ we say a and b are *relatively prime* (or *coprime*).

How would you compute $\gcd(48, 36)$ if you had never heard of Euclid? You might factor: $48 = 2^4 \cdot 3$ and $36 = 2^2 \cdot 3^2$, take the smaller power of each shared prime, $2^2 \cdot 3 = 12$. That is correct, but it does not scale: *factoring large numbers is believed to be hard*. Nobody knows a fast way to factor a 600-digit number, and the security of RSA leans on exactly that difficulty. So if computing gcd required factoring, we would be stuck. The surprise is that you can compute gcd without factoring at all, and very fast, using an idea about 2300 years old.

Intuition

Picture a floor that is 48 by 36 inches, and you want to tile it with the largest possible square tiles, all the same size, no cutting. The side length of that biggest tile is exactly $\gcd(48, 36)$. Here is Euclid's insight as a physical act. Lay down as many 36×36 squares along the long side as you can – you fit one, with a 12×36 strip left over. Now the leftover strip is itself a smaller tiling problem: tile a 36×12 region. The key fact is that the biggest tile for the original floor is the same as the biggest tile for the leftover strip. The problem got smaller but the answer did not change. Keep peeling off the leftover and you eventually reach a strip that tiles perfectly – and that final side length is your answer.

Figure 30.1 draws the first peel of that process for a 48×36 floor: one 36×36 square fits along the long side, leaving a 12×36 strip that becomes the next, smaller tiling problem.

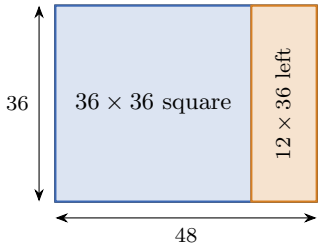


Figure 30.1: One step of Euclid as tiling: $\gcd(48, 36) = \gcd(36, 12)$. The blue 36×36 square is the largest that fits along the long side; the orange 12×36 leftover is the next subproblem, with the same answer.

The picture rests on one short lemma.

Theorem 30.2 (Euclid's reduction)**.** *For any integer a and positive integer b,*

$$\gcd(a, b) = \gcd(b,\ a \bmod b).$$

Formal

Proof. Write $a = qb + r$ with $r = a \bmod b$. We show the two pairs (a, b) and (b, r) have *exactly the same set* of common divisors; then in particular their greatest common divisors coincide. Suppose $d \mid a$ and $d \mid b$. Then $d \mid (a - qb) = r$, so d divides both b and r. Conversely, suppose $d \mid b$ and $d \mid r$. Then $d \mid (qb + r) = a$, so d divides both a and b. The common-divisor sets are equal, hence so are their maxima. □

Euclid's algorithm simply applies this fact over and over. We replace the pair (a, b) by the smaller pair $(b, a \bmod b)$ and repeat. The second coordinate strictly shrinks each round and stays non-negative, so we must reach 0. When the second coordinate is 0, the first coordinate is the answer, because $\gcd(g, 0) = g$.

Algorithm 126: EUCLID(a, b) – greatest common divisor

Input: Non-negative integers a, b, not both zero.
Output: $\gcd(a, b)$.

```
while b ≠ 0 do
    r ← a mod b;
    a ← b;
    b ← r;
return a;
```

Example 30.1 (Tracing $\gcd(252, 198)$)**.** Each line replaces (a, b) with $(b,\ a \bmod b)$.

a	b	$q = \lfloor a/b \rfloor$	$r = a \bmod b$
252	198	1	54
198	54	3	36
54	36	1	18
36	18	2	0
18	0	—	—

The loop stops when $b = 0$, returning $a = 18$. And indeed $252 = 14 \cdot 18$ and $198 = 11 \cdot 18$, with $\gcd(14, 11) = 1$, so 18 is right.

30.1.2 How fast is it? Enter Fibonacci

Each iteration is cheap, so the only question is: how many iterations can there be? The answer ties this ancient algorithm to a sequence you have met before.

Theorem 30.3 (Lamé's theorem, informal)**.** *If $a > b \geq 0$ and the call* EUCLID(a, b) *performs $k \geq 1$ recursive reductions, then $a \geq F_{k+2}$, where F_i is the i-th Fibonacci number. Equivalently, the number of reductions is $O(\log b)$.*

Intuition

Why Fibonacci? Each reduction step replaces (a, b) with $(b, a \bmod b)$. To make the algorithm work as *hard* as possible – the most steps for the smallest inputs – you want the remainder

$a \bmod b$ to be as large as it can be while still being a genuine reduction, and you want the quotient to be the smallest possible, namely 1. A quotient of 1 at every single step means $a = b + r$, i.e. each number is the *sum* of the next two. That recurrence, "each term is the sum of the previous two," is the definition of the Fibonacci numbers. So consecutive Fibonacci numbers are the worst case: $\gcd(F_{n+1}, F_n)$ takes the maximum number of steps.

Because F_k grows like ϕ^k with $\phi = (1+\sqrt{5})/2 \approx 1.618$, the condition $a \geq F_{k+2}$ forces $k = O(\log a)$. Concretely, the number of divisions is at most about $\log_\phi a \approx 1.44 \log_2 a$ – under five divisions per decimal digit. That is why Euclid's algorithm is fast even on huge inputs: a 600-digit number triggers only a few thousand divisions.

Cost summary

Euclid's algorithm. On inputs $a > b \geq 0$:

- *Number of divisions:* $O(\log b)$, worst case at consecutive Fibonacci numbers.
- *Arithmetic operations:* $O(\log b)$ modulo operations.
- *Bit complexity:* each division of β-bit numbers costs $O(\beta^2)$ with schoolbook arithmetic, giving an easy $O(\beta^3)$ bound; a sharper accounting (the quotients shrink the numbers) brings the true cost down to $O(\beta^2)$, and fast multiplication improves it further.

Common trap

Do *not* "optimize" Euclid by factoring first. Factoring is exponentially slower than the GCD itself for large numbers. Euclid's method is fast precisely because it avoids factoring entirely. Also: the algorithm needs $a \bmod b$, not $a - b$ – repeated subtraction is correct but can take a/b steps in a single "level," which is catastrophic when, say, $a = 10^9$ and $b = 1$.

30.2 The Extended Euclidean Algorithm

Euclid's algorithm tells you *what* $\gcd(a, b)$ is. With a little extra bookkeeping it also expresses $\gcd(a, b)$ as a combination of a and b, and that extended version is the most useful tool in this chapter.

Theorem 30.4 (Bézout's identity)**.** *For any integers a and b, not both zero, there exist integers x and y with*

$$ax + by = \gcd(a, b).$$

In fact, $\gcd(a, b)$ *is the* smallest *positive integer expressible as an integer combination $ax + by$.*

Intuition

Geometrically, consider all integers you can reach on the number line by taking integer combinations of step sizes a and b:

$$L = \{ax + by : x, y \in \mathbb{Z}\}.$$

This set forms a discrete regular grid (a one-dimensional sub-lattice or ideal) on the number line. Because adding or subtracting any two numbers in L produces another number in L, the entire lattice consists precisely of all integer multiples of its smallest positive element d. Any common divisor of a and b must divide every number in L, including d; conversely, since $a = a \cdot 1 + b \cdot 0 \in L$ and $b = a \cdot 0 + b \cdot 1 \in L$, that minimal positive generator d must divide both a and b. Therefore, d is exactly $\gcd(a, b)$.
Bézout's identity says that the greatest common divisor is also an integer linear combination of a and b: you can always express 18 as a combination of 252 and 198, and indeed $252 \cdot 4 + 198 \cdot (-5) = 1008 - 990 = 18$. The integers x and y are the *Bézout coefficients*, and extracting them is the job of the extended Euclidean algorithm.

Here is how the algorithm extracts them while running Euclid: write each remainder as a combination of the original a and b. At the start, $a = 1 \cdot a + 0 \cdot b$ and $b = 0 \cdot a + 1 \cdot b$. Every later remainder is built by subtracting a multiple of one previous line from another: $r = a - qb$. Since both inputs to that subtraction are already combinations of the original a and b, so is the result. The combinations ride along for free on top of the ordinary GCD computation. When you reach the GCD, you have its combination too.

The cleanest way to compute the coefficients is the recursive form. Suppose the recursive call on $(b, a \bmod b)$ has already handed back a triple (d, x', y') with $bx' + (a \bmod b)y' = d$. We want (d, x, y) with $ax + by = d$. Substitute $a \bmod b = a - \lfloor a/b \rfloor\, b$:

$$d = bx' + (a - \lfloor a/b \rfloor\, b)\, y' = a\, y' + b\big(x' - \lfloor a/b \rfloor\, y'\big).$$

Matching coefficients, $x = y'$ and $y = x' - \lfloor a/b \rfloor\, y'$.

Algorithm 127: EXTENDED-EUCLID(a, b)

Input: Non-negative integers a, b.
Output: Triple (d, x, y) with $d = \gcd(a, b)$ and $ax + by = d$.

```
if b = 0 then
    return (a, 1, 0);
(d, x', y') ← EXTENDED-EUCLID(b, a mod b);
(d, x, y) ← (d, y', x' − ⌊a/b⌋ · y');
return (d, x, y);
```

Example 30.2 (Coefficients for $\gcd(252, 198)$)**.** We recurse down to the base case, then unwind, building the coefficients on the way back up. The downward pass is exactly Example 30.1; here is the upward pass, each line using $x = y'$ and $y = x' - \lfloor a/b \rfloor y'$.

a	b	$\lfloor a/b \rfloor$	d	x	y
18	0	—	18	1	0
36	18	2	18	0	1
54	36	1	18	1	-1
198	54	3	18	-1	4
252	198	1	18	4	-5

The top line of the original call returns $(18, 4, -5)$, and we verify $252 \cdot 4 + 198 \cdot (-5) = 1008 - 990 = 18$. The Bézout coefficients took the same number of steps as the plain GCD.

Figure 30.2 illustrates the two-way dataflow of the extended Euclidean algorithm: the downward pass discovers the quotients and remainders via ordinary division, while the upward pass reassembles the Bézout coefficients from the bottom up.

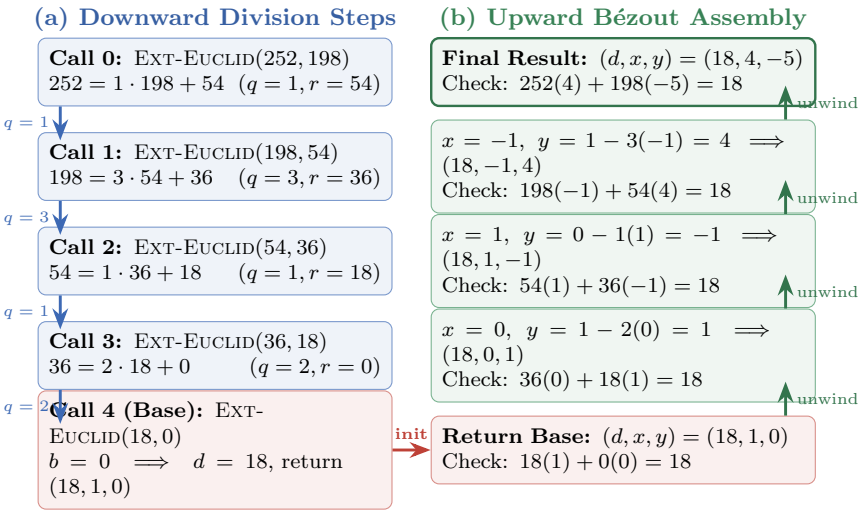


Figure 30.2: The Extended Euclidean algorithm on $(252, 198)$. (a) The downward pass executes standard Euclidean division $a = qb + r$, strictly reducing inputs down to the base case $(18, 0)$. (b) The upward pass unwinds via $x = y'$ and $y = x' - qy'$, producing the Bézout linear combination $252(4) + 198(-5) = 18$ at the root with no extra division steps.

Focus

The extended algorithm gives you, for free, an integer x such that $ax \equiv \gcd(a, b)$. When $\gcd(a, b) = 1$, that x is the *multiplicative inverse* of a modulo b. Nearly every major tool in the rest of this chapter – solving modular linear equations, the Chinese Remainder Theorem, and RSA decryption – builds directly on computing modular inverses. If you remember one algorithm from this chapter, make it Extended-Euclid.

30.3 Modular Arithmetic: Computing on a Clock

A clock face is a familiar example. Seven hours after 9 o'clock is not 16 o'clock; it is 4, because we wrap around at 12. We threw away the "how many times around" information and kept only the position on the dial. Modular arithmetic does exactly this, with n positions instead of 12. Figure 30.3 shows the wrap-around for $9 + 7 \equiv 4 \pmod{12}$.

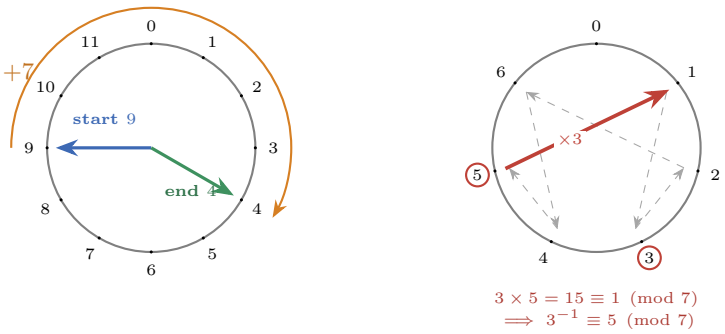


Figure 30.3: Modular arithmetic on cyclic dials. (a) Modular addition in $\mathbb{Z}_{12}$: stepping forward 7 units from 9 wraps past the origin and lands on 4, discarding full rotations. (b) Modular multiplication and inverses in $\mathbb{Z}_7$: multiplying by 3 permutes the nonzero residues in a cyclic orbit $1 \to 3 \to 2 \to 6 \to 4 \to 5 \to 1$. Because $3 \times 5 = 15 \equiv 1 \pmod 7$, the residues 3 and 5 are multiplicative inverses of each other.

Definition 30.3 (Congruence modulo n)**.** For a positive integer n, we say a is *congruent to b modulo n*, written $a \equiv b \pmod n$, if $n \mid (a-b)$; equivalently, a and b leave the same remainder when divided by n.

So $9+7 = 16 \equiv 4 \pmod{12}$. Congruence is an *equivalence relation* – reflexive, symmetric, transitive – and it carves the integers into n buckets called *residue classes*. The class of a is written $[a]_n$ and collects every integer that lands on the same dial position as a. We name the set of classes $\mathbb{Z}_n = \{[0]_n, [1]_n, \ldots, [n-1]_n\}$, and in practice we just use the representatives $\{0, 1, \ldots, n-1\}$.

The key property, which justifies all our algorithms, is that congruence *respects* addition and multiplication.

Theorem 30.5 (Arithmetic respects congruence)**.** *If $a \equiv a' \pmod n$ and $b \equiv b' \pmod n$, then*

$$a + b \equiv a' + b' \pmod n \qquad \text{and} \qquad a\,b \equiv a'\,b' \pmod n.$$

In practice this means *you may reduce mod n at any time.* To compute $(123 \cdot 456) \bmod 7$ you do not need the product 56088. Reduce first: $123 \equiv 4$ and $456 \equiv 1 \pmod 7$, so the answer is $4 \cdot 1 = 4 \bmod 7$. This "reduce early, reduce often" habit is what keeps modular algorithms from ever building giant intermediate numbers.

Common trap

Addition and multiplication carry over to the clock perfectly. *Division does not.* You cannot in general "cancel" a common factor: from $6 \equiv 0 \pmod 6$ you cannot conclude $2 \equiv 0 \pmod 6$. Cancelling a from both sides of a congruence is legal only when $\gcd(a, n) = 1$. Getting this wrong is the classic modular-arithmetic blunder, and untangling exactly when division *is* allowed is the subject of the next section.

30.3.1 Units and inverses

On the clock $\mathbb{Z}_n$, the element 1 acts as a multiplicative identity. An element a has a *multiplicative inverse* if there is some a^{-1} with $a \cdot a^{-1} \equiv 1 \pmod{n}$. When such an inverse exists we call a a *unit*. Inverses are what let us "divide": dividing by a means multiplying by a^{-1}.

Theorem 30.6 (Existence of inverses)**.** *The element a has a multiplicative inverse modulo n if and only if $\gcd(a, n) = 1$. When it exists, the inverse is unique modulo n.*

Intuition

This follows immediately from Bézout's identity. When $\gcd(a, n) = 1$, the extended Euclidean algorithm returns integers x, y with $ax + ny = 1$. Reading that equation modulo n drops the ny term (a multiple of n) and leaves $ax \equiv 1 \pmod{n}$, so $x \bmod n$ *is* the inverse. Conversely, if $\gcd(a, n) = g > 1$, then every multiple of a is a multiple of g, so ax can never be $\equiv 1$; no inverse exists.

The set of units modulo n is written $\mathbb{Z}_n^* = \{a : 1 \le a \le n-1,\ \gcd(a, n) = 1\}$, and its size has a name: $\phi(n)$, *Euler's totient*. For a prime p, every nonzero element is a unit, so $\phi(p) = p - 1$. For a product of two distinct primes, $\phi(pq) = (p-1)(q-1)$ – a fact we will lean on hard when we build RSA.

Example 30.3 (Inverting 5 modulo 12)**.** Since $\gcd(5, 12) = 1$, an inverse exists. Run extended Euclid on $(5, 12)$ or $(12, 5)$: we find $5 \cdot 5 + 12 \cdot (-2) = 25 - 24 = 1$, so $5^{-1} \equiv 5 \pmod{12}$. Check: $5 \cdot 5 = 25 \equiv 1 \pmod{12}$. (Some elements are their own inverse; 5 here is one of them.) By contrast 6 has *no* inverse mod 12 because $\gcd(6, 12) = 6 \neq 1$: $6 \cdot k$ runs through $6, 0, 6, 0, \ldots$ and never hits 1.

30.4 Solving Modular Linear Equations

We can now solve the general modular linear congruence: find all x satisfying

$$ax \equiv b \pmod{n}.$$

This is the modular cousin of the line $ax = b$. The number of solutions is governed entirely by $\gcd(a, n)$.

Theorem 30.7 (Solutions of $ax \equiv b \pmod{n}$)**.** *Let $g = \gcd(a, n)$. The congruence $ax \equiv b \pmod{n}$ has a solution if and only if $g \mid b$. When $g \mid b$, there are exactly g solutions modulo n, and they are evenly spaced n/g apart.*

Intuition

Why $g \mid b$? As x ranges over all integers, $ax \bmod n$ only ever lands on multiples of g – that is exactly what Bézout told us, that $ax + ny$ can only produce multiples of g. So if b is not a multiple of g, the target is simply unreachable, like trying to land on an odd number while only ever taking even-sized steps. When b *is* reachable, the symmetry of the clock means the solutions repeat in a regular pattern, and there turn out to be exactly g of them.

The recipe to find them: use extended Euclid to get x', y' with $ax' + ny' = g$. Scale by b/g to get one solution $x_0 = x' \cdot (b/g) \bmod n$. The full solution set is then $x_0 + k\,(n/g)$ for $k = 0, 1, \ldots, g - 1$.

Algorithm 128: MODULAR-LINEAR-SOLVER(a, b, n)

Input: Integers a, b and modulus $n > 0$.
Output: All solutions of $ax \equiv b \pmod{n}$, or report none.

```
(g, x', y') ← EXTENDED-EUCLID(a mod n, n);
if g ∤ b then
    return "no solution";
x_0 ← x' · (b/g) mod n;
for k ← 0 to g − 1 do
    output (x_0 + k · (n/g)) mod n;
```

Example 30.4 (Solving $14x \equiv 30 \pmod{100}$)**.** Here $g = \gcd(14, 100) = 2$, and $2 \mid 30$, so solutions exist – exactly 2 of them. Extended Euclid gives $14 \cdot (-7) + 100 \cdot 1 = 2$, so $x' = -7$. Scale by $b/g = 15$: $x_0 = (-7) \cdot 15 \bmod 100 = -105 \bmod 100 = 95$. The two solutions, spaced $n/g = 50$ apart, are

$$x \equiv 95 \quad \text{and} \quad x \equiv 95 + 50 = 145 \equiv 45 \pmod{100}.$$

Check $x = 45$: $14 \cdot 45 = 630 = 6 \cdot 100 + 30 \equiv 30 \pmod{100}$. And $x = 95$: $14 \cdot 95 = 1330 = 13 \cdot 100 + 30 \equiv 30$. Both land.

When $\gcd(a, n) = 1$ there is a single solution, $x \equiv a^{-1}b$, and this special case – one equation, one answer – is the one we use most often.

30.5 The Chinese Remainder Theorem

The next result goes back to an ancient puzzle. A 3rd-century Chinese text asks: *there is a number which leaves remainder* 2 *when divided by* 3*, remainder* 3 *when divided by* 5*, and remainder* 2 *when divided by* 7 *– what is it?* The Chinese Remainder Theorem (CRT) says such a number always exists, is unique modulo $3 \cdot 5 \cdot 7 = 105$, and tells you how to find it.

Theorem 30.8 (Chinese Remainder Theorem)**.** *Let $n_1, n_2, \ldots, n_k$ be* pairwise coprime *positive integers and let $N = n_1 n_2 \cdots n_k$. Then for any target remainders $a_1, \ldots, a_k$ the system*

$$x \equiv a_1 \pmod{n_1}, \quad x \equiv a_2 \pmod{n_2}, \quad \ldots, \quad x \equiv a_k \pmod{n_k}$$

has a unique solution x modulo N.

Intuition

The simplest way to understand CRT is as a *change of coordinates*. A single number x in the range $0 \le x < N$ can be described in two completely equivalent ways: by its ordinary value, or by the tuple of its remainders $(x \bmod n_1, \ldots, x \bmod n_k)$. CRT says this tuple is a perfect ID badge: no two numbers in $[0, N)$ share a badge, and every possible badge belongs to exactly one number. So computing with big numbers mod N is the same as computing with several small numbers in parallel, one per coordinate – and addition and multiplication act coordinate by coordinate. It is the integer version of breaking a vector into its components.

Figure 30.4 draws the bijection for the small case $N = 3 \cdot 5 = 15$: each of the 15 numbers gets a unique two-coordinate badge $(x \bmod 3, x \bmod 5)$, and every cell of the 3×5 grid is filled exactly once.

$x \bmod 3$ \ $x \bmod 5$	0	1	2	3	4
0	0	6	12	3	9
1	10	1	7	13	4
2	5	11	2	8	14

Figure 30.4: CRT as a change of coordinates, $N = 15$. Row $= x \bmod 3$, column $= x \bmod 5$. Every number $0 \le x < 15$ lands in a distinct cell, and every cell is occupied: the map $x \mapsto (x \bmod 3,\ x \bmod 5)$ is a bijection. The highlighted cell $(2, 3)$ holds the unique $x = 8$.

30.5.1 Constructing the solution

For each i, let $M_i = N/n_i$ – the product of *all the other* moduli. The point of M_i is that it is divisible by every n_j except n_i, so it is invisible (zero) in every coordinate but the i-th. Because the moduli are pairwise coprime, $\gcd(M_i, n_i) = 1$, so M_i has an inverse $M_i^{-1} \pmod{n_i}$. Define $c_i = M_i \cdot (M_i^{-1} \bmod n_i)$. Then $c_i \equiv 1 \pmod{n_i}$ and $c_i \equiv 0 \pmod{n_j}$ for $j \neq i$ – an "indicator" for coordinate i. The answer is the weighted sum

$$x \equiv \sum_{i=1}^{k} a_i \, c_i \pmod{N}.$$

Each term contributes a_i to its own coordinate and 0 to every other, so the whole sum has exactly the remainders we ordered.

Algorithm 129: CRT$(a_1, \ldots, a_k;\ n_1, \ldots, n_k)$

Input: Remainders a_i and pairwise-coprime moduli n_i.
Output: The unique $x \in [0, N)$ solving all congruences, $N = \prod n_i$.

```
N ← n_1 n_2 ··· n_k;
x ← 0;
for i ← 1 to k do
    M_i ← N / n_i;
    (g, s, t) ← Extended-Euclid(M_i mod n_i, n_i);
    // s is M_i^{-1} mod n_i since g = 1
    x ← (x + a_i · M_i · s) mod N;
return x;
```

Example 30.5 (The ancient puzzle)**.** Solve $x \equiv 2 \pmod 3$, $x \equiv 3 \pmod 5$, $x \equiv 2 \pmod 7$. Here $N = 105$.

i	n_i	a_i	$M_i = N/n_i$	$M_i \bmod n_i$	$M_i^{-1} \bmod n_i$
1	3	2	35	2	2
2	5	3	21	1	1
3	7	2	15	1	1

Now sum the weighted terms:

$$x \equiv 2 \cdot 35 \cdot 2 \ + \ 3 \cdot 21 \cdot 1 \ + \ 2 \cdot 15 \cdot 1 = 140 + 63 + 30 = 233 \equiv 233 - 2 \cdot 105 = 23 \pmod{105}.$$

So $x = 23$. Verify: $23 = 7 \cdot 3 + 2 \equiv 2 \pmod 3$; $23 = 4 \cdot 5 + 3 \equiv 3 \pmod 5$; $23 = 3 \cdot 7 + 2 \equiv 2 \pmod 7$. All three hold.

Why this matters

The Chinese Remainder Theorem also speeds up RSA decryption by a factor of roughly four: instead of one exponentiation modulo the big number $N = pq$, you do two exponentiations modulo the half-size primes p and q and recombine with CRT. Since modular exponentiation cost grows faster than linearly in the bit length, two half-size jobs beat one full-size job. CRT also underlies fast big-integer arithmetic, error-correcting codes, and hash-table tricks. All of these use the same idea of computing in parallel coordinates.

30.6 Modular Exponentiation

RSA needs to compute things like $a^b \bmod n$ where a, b, and n are all hundreds of digits long. Two naive approaches both fail. Multiplying a by itself b times takes b multiplications – but b might be 2^{1024}, more multiplications than there are atoms in the universe. And computing the full a^b first and reducing at the end is worse still: a^b has astronomically many digits and would never fit in memory. We need an algorithm that is fast in the number of multiplications *and* keeps every intermediate value small.

Both problems are solved by one idea: *repeated squaring*, also called *binary exponentiation*.

Intuition

To compute a^{13}, do not multiply thirteen copies of a. Instead notice that 13 in binary is 1101, i.e. $13 = 8 + 4 + 1$, so $a^{13} = a^8 \cdot a^4 \cdot a^1$. And the powers a^1, a^2, a^4, a^8 form a chain where each is the *square* of the previous: $a^2 = (a^1)^2$, $a^4 = (a^2)^2$, $a^8 = (a^4)^2$. So you compute the chain by repeated squaring – just a handful of multiplications – and multiply in the ones that correspond to a 1-bit of the exponent. The bits of the exponent are a recipe, read one at a time. An exponent with β bits needs only about β squarings, not 2^β multiplications.

The clean way to implement this scans the bits of b from the *most* significant down to the least. Maintain a running result r, initially 1. For each bit, square r (this "shifts" the partial exponent left by one binary place); if the current bit is 1, also multiply by a. Reduce mod n after every operation, so nothing ever exceeds n^2 in size.

Algorithm 130: MODULAR-POWER(a, b, n) – computes $a^b \bmod n$

Input: Base a, exponent $b \geq 0$ with bits $b_{t-1}b_{t-2}\cdots b_1 b_0$, modulus $n > 1$.
Output: $a^b \bmod n$.

```
r ← 1;
for i ← t − 1 downto 0 do
    r ← (r · r) mod n;
    if b_i = 1 then
        r ← (r · a) mod n;
return r;
```

Example 30.6 (Computing $7^{13} \bmod 11$)**.** The exponent $13 = 1101_2$, so we scan bits $1, 1, 0, 1$ from the top. Start $r = 1$.

bit b_i	operation	r after square	r after multiply
1	sq, mul	$1^2 = 1$	$1 \cdot 7 = 7$
1	sq, mul	$7^2 = 49 \equiv 5$	$5 \cdot 7 = 35 \equiv 2$
0	sq	$2^2 = 4$	(skip) 4
1	sq, mul	$4^2 = 16 \equiv 5$	$5 \cdot 7 = 35 \equiv 2$

Figure 30.5 shows how the bits of the exponent drive the square/multiply chain: every bit triggers a square, and a 1-bit triggers an extra multiply by the base.

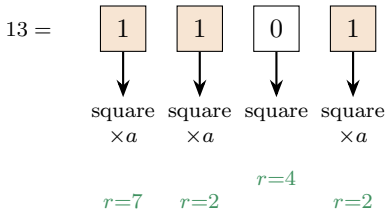


Figure 30.5: Bits of the exponent $13 = 1101_2$ as a recipe for $7^{13} \bmod 11$, read left to right. Each bit squares the running value r; each shaded 1-bit also multiplies by the base 7. The final $r = 2$ is the answer.

All reductions are mod 11. The result is $7^{13} \equiv 2 \pmod{11}$. (Sanity check by Fermat, coming next: $7^{10} \equiv 1$, so $7^{13} \equiv 7^3 = 343 = 31 \cdot 11 + 2 \equiv 2$. Agreed.) Notice every intermediate value stayed below $121 = 11^2$; we never built a giant number.

Cost summary

Modular exponentiation. For a β-bit exponent b and modulus n:

- *Multiplications:* between β and 2β modular multiplications (one square per bit, plus one multiply per 1-bit).
- *Intermediate size:* every value stays in $[0, n)$, so each multiply is of β-bit numbers, costing $O(\beta^2)$ bit operations naively.
- *Total bit cost:* $O(\beta^3)$ with schoolbook multiplication.

Contrast the naive b multiplications, which is $\Theta(2^\beta)$; for exponents of cryptographic size the naive method would never finish.

Walking question

Take MODULAR-POWER on a walk. Convince yourself of its loop invariant: just before processing bit b_i, the value of r equals $a^{(\text{bits seen so far})} \bmod n$, where "bits seen so far" is the number formed by the high-order bits $b_{t-1} \cdots b_{i+1}$ already consumed. Squaring doubles that prefix value (appends a 0 bit); a conditional multiply by a turns the new last bit into a 1. When the loop ends, every bit has been folded in, and $r = a^b \bmod n$. Why is squaring exactly "append a zero bit" to the exponent?

30.6.1 Fermat's and Euler's theorems

Two classical facts about exponents underpin both the primality test and RSA.

Theorem 30.9 (Fermat's little theorem)**.** *If p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$.*

Theorem 30.10 (Euler's theorem)**.** *If $\gcd(a, n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod{n}$, where ϕ is Euler's totient.*

Euler's theorem generalizes Fermat's (take $n = p$ prime, so $\phi(p) = p - 1$). The intuition for both: the units mod n form a finite group under multiplication, and in any finite group, raising any element to the size of the group lands you back at the identity. We will use Euler's theorem directly to prove RSA decryption inverts encryption.

30.7 The RSA Public-Key Cryptosystem

RSA, named for Rivest, Shamir, and Adleman, lets two strangers who have never met exchange a secret over an open channel. The key idea is that the encryption key can be published, while only the holder of a separate private key can decrypt. This is *public-key* cryptography, and before 1977 most people thought it was impossible.

Intuition

Think of a padlock that anyone can snap shut but only you have the key to open. You manufacture thousands of these open padlocks and scatter them everywhere – a mailbox at every street corner. Anyone can grab one, lock a box with it, and send you the box; nobody but you can reopen it, because only you hold the key. The public key is the open padlock; the private key is the key in your pocket. The mathematical question is: how do you build a "lock" that is easy to snap shut (encrypt) but, without the secret, ruinously hard to open (decrypt)? RSA's answer is the gap between multiplying primes (easy) and factoring their product (hard).

30.7.1 Setting up the keys

1. Pick two large random primes p and q (in practice, each several hundred digits). Compute $n = pq$.
2. Compute $\phi(n) = (p - 1)(q - 1)$.
3. Choose a public exponent e with $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$. A common choice is $e = 65537$.
4. Compute the private exponent $d = e^{-1} \bmod \phi(n)$ using extended Euclid – this is the one step that needs the secret factorization.

The *public key* is the pair (e, n). The *private key* is d (and implicitly the factors p, q). The primes p and q, and $\phi(n)$, are destroyed or hidden after setup.

30.7.2 Encrypting and decrypting

A message is encoded as a number m with $0 \le m < n$. Then:

$$\text{encrypt:} \quad c = m^e \bmod n, \qquad\qquad \text{decrypt:} \quad m = c^d \bmod n.$$

Figure 30.6 traces a single message across the channel.

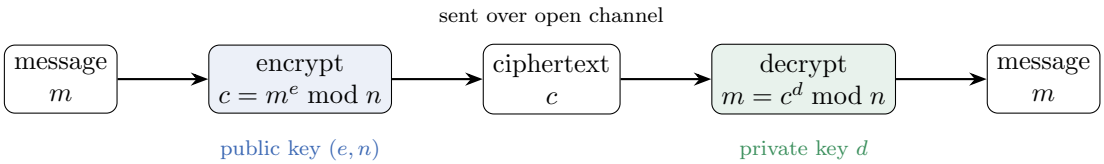


Figure 30.6: The RSA round trip. Anyone can encrypt with the published (e, n); only the holder of the private d can turn the ciphertext back into m. Both arrows are a single call to Modular-Power.

Both are just Modular-Power. Encryption uses only the public (e, n); decryption needs the private d. The whole security rests on this asymmetry: to recover d from the public (e, n) you would need $\phi(n) = (p-1)(q-1)$, which requires knowing p and q – that is, factoring n. For large n, nobody knows how to do that in any reasonable time.

Theorem 30.11 (RSA correctness)**.** *With keys set up as above, decryption inverts encryption:* $(m^e)^d \equiv m \pmod n$ *for every* m *with* $0 \le m < n$.

Formal

Proof sketch. By construction $ed \equiv 1 \pmod{\phi(n)}$, so $ed = 1 + k\,\phi(n)$ for some integer k. Hence

$$m^{ed} = m^{1+k\phi(n)} = m \cdot \left(m^{\phi(n)}\right)^k.$$

When $\gcd(m, n) = 1$, Euler's theorem (Theorem 30.10) gives $m^{\phi(n)} \equiv 1 \pmod n$, so the whole second factor is 1 and $m^{ed} \equiv m \pmod n$. The remaining cases, where m shares a factor with n, are handled by applying Fermat's little theorem modulo p and modulo q separately and stitching the two results back together with the Chinese Remainder Theorem; the congruence $m^{ed} \equiv m$ holds modulo both primes, hence modulo their product n. □

Example 30.7 (A toy RSA, end to end)**.** Use tiny primes so we can do everything by hand. Let $p = 5$, $q = 11$, so $n = 55$ and $\phi(n) = 4 \cdot 10 = 40$. Pick $e = 3$; check $\gcd(3, 40) = 1$, good. Find $d = 3^{-1} \bmod 40$: extended Euclid gives $3 \cdot 27 = 81 = 2 \cdot 40 + 1$, so $d = 27$.

Encrypt the message $m = 7$:

$$c = 7^3 \bmod 55 = 343 \bmod 55 = 343 - 6 \cdot 55 = 13.$$

Decrypt $c = 13$ with $d = 27$:

$$m = 13^{27} \bmod 55.$$

Compute by repeated squaring mod 55: $13^2 = 169 \equiv 4$, $13^4 \equiv 16$, $13^8 \equiv 256 \equiv 36$, $13^{16} \equiv 36^2 = 1296 \equiv 31$. Since $27 = 16 + 8 + 2 + 1$,

$$13^{27} \equiv 31 \cdot 36 \cdot 4 \cdot 13 \pmod{55}.$$

Multiply step by step mod 55: $31 \cdot 36 = 1116 \equiv 1116 - 20 \cdot 55 = 16$; $16 \cdot 4 = 64 \equiv 9$; $9 \cdot 13 = 117 \equiv 117 - 2 \cdot 55 = 7$. We recover $m = 7$. The round trip worked.

Why this matters

This toy is trivial to break – anyone can factor $55 = 5 \cdot 11$. Real RSA uses n with around 2048 to 4096 bits, where the best known factoring algorithms would run for longer than the age of the universe. Every time your browser shows a padlock icon, a key exchange related to ideas in this chapter just happened. The entire scheme is built from the elementary tools we developed by hand: GCD, modular inverses, modular exponentiation, and Euler's theorem.

Common trap

Textbook RSA, exactly as described, is *not* safe to deploy. It is deterministic (the same message always encrypts to the same ciphertext, leaking information), it is malleable, and small messages with small e can be attacked directly. Real systems add randomized padding (OAEP) and use RSA only to wrap a symmetric key. Learn the math here; never ship the textbook version.

30.8 Primality Testing

RSA needs large primes, hundreds of digits each. So we need to answer, quickly: *is this 300-digit number prime?* Trial division – checking every divisor up to $\sqrt{n}$ – would require about 10^{150} operations for a 300-digit number, which is hopeless. We need something exponentially faster, and Fermat's little theorem, used *in reverse*, provides it.

30.8.1 The Fermat test and its flaw

Fermat's little theorem says: if p is prime and $\gcd(a, p) = 1$, then $a^{p-1} \equiv 1 \pmod{p}$. Turn it into a test. To check whether n is prime, pick a random "witness" a and compute $a^{n-1} \bmod n$ by repeated squaring. If the result is *not* 1, then n is *definitely composite*, since Fermat's little theorem rules this out for every prime. If the result *is* 1, then n is *probably prime*, and we gain confidence by trying more witnesses.

Common trap

The word "probably" matters. Passing for a single base does not prove primality: some composites pass for specific bases (making such a base a *Fermat liar*), and *Carmichael numbers* (the smallest is $561 = 3 \cdot 11 \cdot 17$) satisfy $a^{n-1} \equiv 1 \pmod{n}$ for *every* a coprime to n, fooling the plain Fermat test on every coprime base. Carmichael numbers are why we need a stronger test.

30.8.2 Miller–Rabin: a better witness

Miller–Rabin patches the hole with one extra observation. If n is an odd prime, then in $\mathbb{Z}_n$ the only square roots of 1 are $+1$ and -1. (In a field, $x^2 - 1 = (x-1)(x+1) = 0$ forces $x = \pm 1$; composites can

have *nontrivial* square roots of 1, which immediately prove compositeness.) Miller–Rabin watches the squaring chain for an illegal square root of 1.

Write $n-1 = 2^s \cdot u$ with u odd – just pull out all the factors of two. Now look at the sequence

$$a^u,\ \ a^{2u},\ \ a^{4u},\ \ \ldots,\ \ a^{2^s u} = a^{n-1} \pmod{n},$$

each term the square of the one before it. If n is prime, this sequence must either start at 1, or hit -1 at some point before the end (and stay 1 thereafter). If instead the sequence reaches 1 *without* the step just before it being -1, then we have found a nontrivial square root of 1, proving that n is composite. We call such a base a a *witness to compositeness*.

Algorithm 131: MILLER–RABIN-WITNESS(a, n) – does a prove n composite?

Input: Base a with $2 \le a \le n-2$; odd $n > 2$.
Output: TRUE if a proves n composite; FALSE if n passes for a.

```
write n − 1 = 2^s · u with u odd;
x ← MODULAR-POWER(a, u, n);
if x = 1 or x = n − 1 then
    return FALSE;
for j ← 1 to s − 1 do
    x ← (x · x) mod n;
    if x = n − 1 then
        return FALSE;
return TRUE;
```

To test n, run this witness check for several random bases a. If any base returns TRUE, n is composite, with certainty. If all of them return FALSE, declare n "probably prime."

Theorem 30.12 (Miller–Rabin error bound)**.** *If n is an odd composite, then at least $3/4$ of the bases $a \in \{2, \ldots, n-2\}$ are witnesses to its compositeness. Hence a single random base fails to detect a composite with probability at most $1/4$, and k independent random bases reduce the failure probability to at most 4^{-k}.*

Intuition

The 4^{-k} decay is what makes this practical. With $k = 50$ random bases the chance of wrongly calling a composite "prime" is below $4^{-50} \approx 10^{-30}$ – vastly smaller than the chance of a cosmic ray flipping a bit in your computer mid-calculation. For all engineering purposes that is certainty. And unlike the Fermat test, no composite – not even a Carmichael number – can fool a $3/4$ fraction of bases, since Theorem 30.12 holds for every odd composite. We accept a tiny error probability in exchange for a huge speedup.

Example 30.8 (Catching $n = 561$, the smallest Carmichael number)**.** The Fermat test fails on 561, so let us watch Miller–Rabin succeed. Here $n - 1 = 560 = 2^4 \cdot 35$, so $s = 4$ and $u = 35$. Take base $a = 2$. First $2^{35} \bmod 561$. By repeated squaring one finds $2^{35} \equiv 263 \pmod{561}$. That is neither 1 nor 560, so we begin squaring:

$$263^2 \equiv 166, \quad 166^2 \equiv 67, \quad 67^2 \equiv 1 \pmod{561}.$$

The sequence hit 1, but the term just before it was 67, *not* $560 \equiv -1$. We have caught a nontrivial square root of 1: $67^2 \equiv 1$ yet $67 \neq \pm 1$. So $a = 2$ is a witness, and 561 is declared composite –

correctly, since $561 = 3 \cdot 11 \cdot 17$. The extra square-root check is exactly what the plain Fermat test was missing.

Figure 30.7 contrasts the two endings of the squaring chain. A prime is allowed to reach 1 only by passing through -1 first; the composite 561 jumps directly to 1 from 67, a nontrivial square root of 1 that proves 561 is composite.

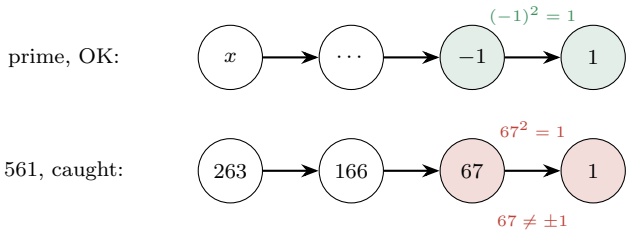


Figure 30.7: Miller–Rabin's square-root check. Each arrow squares (mod n). A prime can only arrive at 1 from -1 (top). The composite 561 reaches 1 directly from 67, a nontrivial square root of 1 that proves 561 is composite (bottom).

Cost summary

Miller–Rabin primality test. For an n with $\beta = \log_2 n$ bits, using k random bases:

- Each base runs one modular exponentiation plus up to $s \leq \beta$ squarings: $O(\beta)$ modular multiplications, i.e. $O(\beta^3)$ bit operations naively.
- Total: $O(k\,\beta^3)$ bit operations.
- *Error:* at most 4^{-k} of declaring a composite prime; never errs on a true prime.

This is fast enough to generate 2048-bit primes in milliseconds: pick a random odd number, run Miller–Rabin, repeat until one passes. The prime number theorem guarantees primes are dense enough that you do not wait long.

History and connection

This material maps to **CLRS Chapter 31**, "Number-Theoretic Algorithms." Euclid's algorithm appears in Book VII of the *Elements* (c. 300 BCE), making it the oldest nontrivial algorithm still in daily use. The Chinese Remainder Theorem traces to Sunzi's *Mathematical Classic* (3rd–5th century CE). Fermat stated his little theorem in 1640; Euler proved and generalized it in 1736 with the totient function. Miller's 1976 deterministic test (conditional on the Extended Riemann Hypothesis) was made unconditional and randomized by Rabin in 1980. RSA was published by Rivest, Shamir, and Adleman in 1978; in 1997, British intelligence agency GCHQ revealed that Clifford Cocks, James Ellis, and Malcolm Williamson had discovered equivalent concepts earlier under classified cover. In 2002, Agrawal, Kayal, and Saxena discovered AKS, the first deterministic polynomial-time primality test, though randomized Miller–Rabin remains faster in practice. CLRS develops all of this with more group theory than we have; read it next for the rigorous treatment of $\mathbb{Z}_n^*$ as a group and the full proof of the Miller–Rabin error bound.

Algorithm lab

Trace it yourself. Pick up paper and do these by hand before moving on.

1. Run EUCLID$(89, 55)$ – consecutive Fibonacci numbers – and count the divisions. You should see the maximum number of steps for inputs this size, every quotient equal to 1 until the very last.
2. Run EXTENDED-EUCLID$(17, 23)$ and confirm the coefficients give $17x + 23y = 1$. Then read off $17^{-1} \bmod 23$ from your x.
3. Compute $3^{20} \bmod 100$ with MODULAR-POWER, scanning the bits of $20 = 10100_2$. Keep every intermediate value below 100^2.
4. Solve the system $x \equiv 1 \pmod 4$, $x \equiv 2 \pmod 9$, $x \equiv 3 \pmod{25}$ with CRT. (The moduli are pairwise coprime; $N = 900$.)
5. Run MILLER–RABIN-WITNESS on $n = 221 = 13 \cdot 17$ with base $a = 5$ and watch it return TRUE.

30.9 Putting It Together

Every tool in this chapter grew out of a single recurrence from 300 BCE. At the bottom sits $\gcd(a, b) = \gcd(b, a \bmod b)$, from which comes Euclid's algorithm. Tracking the quotients along the way yields Bézout's coefficients and modular inverses; inverses solve modular linear equations and drive the Chinese Remainder Theorem. Alongside them, repeated squaring gives fast modular exponentiation, Euler's theorem gives the cycle length of modular powers, and combining modular inverses with Euler's theorem produces RSA – while adding the square-root-of-one check gives Miller–Rabin to find the primes RSA needs.

Focus

The algorithm used most in this chapter is *the extended Euclidean algorithm.* Modular inverses, linear congruences, CRT reconstruction, and RSA key generation all reduce directly to computing Bézout coefficients with that one routine. Master EXTENDED-EUCLID and MODULAR-POWER, understand why each is fast, and you understand the computational core of classical cryptography.

The modular arithmetic and prime-modulus hash constructions developed here do not end with cryptosystems. In Chapter 31, we turn from numbers to text: string matching, pattern search, and the Rabin–Karp algorithm, which uses modular hashing to find patterns in long texts in linear expected time.

Exercises

Exercise 30.1 (Warmup: trace Euclid)**.** Compute $\gcd(1071, 462)$ by hand using EUCLID, writing out the quotient and remainder at each step. How many divisions did it take? Then state gcd as a product of shared prime factors to double-check.

Exercise 30.2 (Bézout by hand)**.** Run EXTENDED-EUCLID$(240, 46)$. Report the triple (d, x, y) and verify $240x + 46y = d$. Is the solution (x, y) unique? If not, describe the full family of integer solutions.

Exercise 30.3 (Inverses exist or do not)**.** For each pair, decide whether a has an inverse modulo n, and compute it when it exists: (a) $a = 7, n = 26$; (b) $a = 8, n = 12$; (c) $a = 10, n = 21$. Explain each answer in terms of $\gcd(a, n)$.

Exercise 30.4 (Solving congruences)**.** Find all solutions modulo n, or explain why none exist: (a) $6x \equiv 9 \pmod{15}$; (b) $6x \equiv 8 \pmod{15}$; (c) $35x \equiv 14 \pmod{84}$. For each, first compute $g = \gcd(a, n)$ and use the divisibility test before hunting for solutions.

Exercise 30.5 (Chinese Remainder reconstruction)**.** A basket of eggs breaks into groups with 1 left over when counted by 3, 2 left over by 5, and 6 left over by 7. What is the smallest possible number of eggs? Use the CRT construction and show your M_i and inverses.

Exercise 30.6 (Fast exponentiation)**.** Compute $5^{117} \bmod 19$ using MODULAR-POWER. Write 117 in binary, list the squaring/multiplying steps, and keep every intermediate value reduced mod 19. Then verify your answer using Fermat's little theorem to shrink the exponent first.

Exercise 30.7 (RSA round trip)**.** Let $p = 7$, $q = 13$, so $n = 91$ and $\phi(n) = 72$. Choose $e = 5$. (a) Confirm $\gcd(5, 72) = 1$ and compute the private exponent $d = 5^{-1} \bmod 72$. (b) Encrypt $m = 10$. (c) Decrypt your ciphertext and confirm you recover 10.

Exercise 30.8 (Why the square-root test matters)**.** The number $341 = 11 \cdot 31$ passes the Fermat test for base $a = 2$ (indeed $2^{340} \equiv 1 \pmod{341}$), so the Fermat test calls it "probably prime." Run MILLER–RABIN-WITNESS$(2, 341)$ instead. Does base 2 expose 341 as composite, or does 341 slip past base 2 as well? (You will need $n - 1 = 340 = 2^2 \cdot 85$.)

Exercise 30.9 (Worst case of Euclid)**.** Prove that EUCLID(F_{k+1}, F_k) performs exactly $k - 1$ division steps, where F_i is the i-th Fibonacci number. (Hint: show by induction that each reduction turns the pair (F_{i+1}, F_i) into (F_i, F_{i-1}) because the quotient is always 1.)

Exercise 30.10 (Challenge: CRT speedup for RSA)**.** Explain, with a rough operation count, why decrypting $c^d \bmod n$ by computing $c^{d \bmod (p-1)} \bmod p$ and $c^{d \bmod (q-1)} \bmod q$ separately and recombining with CRT is about four times faster than the direct computation modulo $n = pq$. (Hint: modular multiplication of β-bit numbers costs $\Theta(\beta^2)$ naively, and p, q each have about half the bits of n.)

Exercise 30.11 (Challenge: a flawed inverse routine)**.** A student computes modular inverses by the formula $a^{-1} \equiv a^{n-2} \pmod{n}$, citing Fermat. (a) For which n is this formula valid? (b) Give a specific a, n where it produces a wrong answer, and explain which hypothesis of Fermat's little theorem fails. (c) State the correct generalization using Euler's theorem, and say why EXTENDED-EUCLID is preferable in practice anyway.

Chapter 31

String Matching

In Chapter 30, we explored the algebraic structure of modular arithmetic, prime numbers, and public-key cryptography. Now we turn from arithmetic on numbers to processing text.

Open any document on your computer and press the keys that mean "find." Type a word, and every occurrence lights up at once. You have just asked a machine one of the oldest questions in text processing: *where, inside this long string, does that short string appear?* It feels instant, but behind it is an algorithm, and usually not the obvious one. The obvious method can be very slow on bad inputs; the better methods, of which there are several, avoid that slowdown, and this chapter explains how.

We begin with the naive sliding-window approach, examine how Rabin–Karp converts substring comparisons into rolling modular hashes, construct deterministic finite automata that track match progress, and finish with the Knuth–Morris–Pratt algorithm, whose prefix function achieves optimal linear time without ever backtracking in the text.

31.1 Setting the stage

History and connection

In the late 1960s and early 1970s, computing was escaping the rigid world of batch-processed punched cards and entering the era of interactive time-sharing terminals (such as the DEC PDP-10 and SDS 940). Programmers were suddenly writing and editing large source files directly on screen using early text editors like `QED` and `TECO`. They needed a way to search for strings in real time. But the physical machinery was unforgiving: text was stored on magnetic core memory, rotating drums, or sequential magnetic tape. Backtracking—rewinding the tape or sliding an internal buffer pointer backward to re-read characters—was slow, and on tape it was mechanically costly.

The naive search algorithm, which simply resets and shifts by one character whenever a match fails, performs badly on repetitive text: like searching for $\mathtt{a}^{m-1}\mathtt{b}$ inside $\mathtt{a}^{n}$, it repeatedly scans $m-1$ matching characters only to fail on the last letter, rewinding the pointer and restarting from scratch. These $\Theta(nm)$ operations could stall an interactive time-shared mainframe.

The solution came from practice and theory at about the same time. In 1970, James H.

Morris was writing a text editor at UC Berkeley and grew exasperated by this rewinding behavior; he devised a clever way to keep the text pointer moving forward by exploiting the pattern's internal structure. Meanwhile at Stanford, Donald Knuth was studying a theoretical result newly proved by Stephen Cook (1970): Cook had demonstrated that any two-way deterministic pushdown automaton could be simulated on a random-access machine in linear time. When Knuth's student Vaughan Pratt observed that string matching was precisely a two-way automaton problem, Knuth set out to turn Cook's abstract theorem into an explicit, practical algorithm. Knuth reduced the transition table to an m-element integer array—the *prefix function*—and proved that exact matching could be performed in strictly linear $\Theta(n)$ time with *zero* backward movement of the text pointer. Knuth, Morris, and Pratt published the algorithm jointly in 1977.

That same year, Robert S. Boyer and J. Strother Moore published a different approach. By aligning the pattern and checking characters *from right to left*, a single mismatch against a character that does not appear anywhere in the pattern allows the entire pattern window to skip forward by its full length m in one step. This gives *sublinear* average-case search times of $O(n/m)$ on natural language texts. A decade later, in 1987, Michael O. Rabin and Richard M. Karp introduced an algebraic alternative: the rolling fingerprint hash, treating substrings as base-d polynomials modulo a prime q, computing window updates in $O(1)$ arithmetic steps. While exact single-pattern matching is solved in optimal linear time, an active research area is compressed-index pattern matching on massive, highly repetitive genomic collections: querying petabyte-scale pan-genomes directly in compressed space without decompression, using Wheeler graphs and r-index data structures.

Let us fix notation once and carefully, because string matching is a place where sloppy indexing causes real pain.

We are given a *text* T, an array of n characters $T[1\,..\,n]$, and a *pattern* P, an array of m characters $P[1\,..\,m]$, with $m \le n$. The characters are drawn from a finite *alphabet* Σ. For English text Σ might be the 26 letters plus punctuation and a space; for DNA it is just $\{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{T}\}$; for raw bytes it is the 256 values 0 through 255. We write $|\Sigma|$ for the alphabet size.

Definition 31.1 (Valid shift)**.** We say the pattern P *occurs with shift* s in the text T if $0 \le s \le n-m$ and

$$T[s+1\,..\,s+m] = P[1\,..\,m],$$

that is, $T[s+j] = P[j]$ for every $j = 1, 2, \ldots, m$. When this holds we call s a *valid shift*; otherwise s is an *invalid shift*. The *string-matching problem* is to find all valid shifts.

The shift s is best read as "how far the pattern has slid to the right before it lines up." Shift 0 means the pattern sits at the very front of the text. If `abcab` is the text and `ab` is the pattern, then $s = 0$ and $s = 3$ are valid, because the pattern lines up at positions 1 and 4.

Intuition

Picture two strips of paper. The long one is the text, laid flat on the table. The short one is the pattern, sitting on top of it. You slide the pattern strip to the right, one notch at a time, and at each resting place you peer down through the layers and check: do all m characters underneath match the m characters on top? Each resting place is a *shift*. There are exactly $n - m + 1$ of them, from $s = 0$ (flush left) to $s = n - m$ (flush right). String matching is the discipline of checking those resting places *without doing more peering than you have to.*

Figure 31.1 draws the strips for a tiny example: the pattern sliding across the text, one shift at a time.

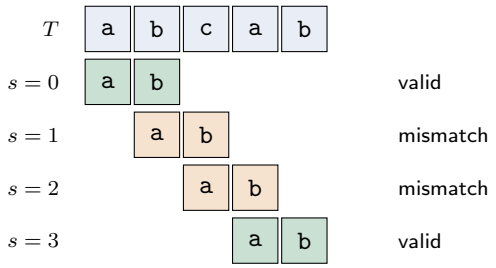


Figure 31.1: The pattern $P =$ `ab` slid across $T =$ `abcab`. Each row is one shift s. There are $n - m + 1 = 4$ candidate shifts; the green rows ($s = 0$ and $s = 3$) line up exactly, the orange ones do not. The faster algorithms in this chapter check these rows without re-reading the text underneath.

Focus

Two numbers govern every analysis in this chapter: n, the text length, and m, the pattern length. There are $n - m + 1$ candidate shifts. The goal is to keep the cost per shift small and, more importantly, to avoid redundant comparisons by reusing what earlier alignments already revealed.

We also need some notation for strings. We use Σ^* for the set of all finite strings over Σ, including the empty string ε. If w is a string, $|w|$ is its length. The *concatenation* of x and y, written xy, is x followed by y, of length $|x| + |y|$. A string w is a *prefix* of x, written $w \sqsubset x$, if $x = wy$ for some string y – w is a chunk off the front. It is a *suffix*, written $w \sqsupset x$, if $x = yw$ for some y – a chunk off the back. Thus `ab` $\sqsubset$ `abcab` and `cab` $\sqsupset$ `abcab`. Prefix and suffix are the only two relations we need for the fastest algorithms in this chapter, so we fix them firmly now.

31.2 The naive matcher: just try every shift

The simplest algorithm tries every shift. Slide the pattern to each of the $n - m + 1$ shifts. At each one, compare characters left to right until either they all match (report the shift) or one of them disagrees (give up, slide one notch right, start over).

Algorithm 132: NAIVE-STRING-MATCHER

Input: Text $T[1\,..\,n]$, pattern $P[1\,..\,m]$.
Output: Every valid shift s.

```
1 for s ← 0 to n − m do
2     j ← 1;
3     while j ≤ m and T[s + j] = P[j] do
4         j ← j + 1;
5     if j = m + 1 then
6         report "P occurs with shift s";
```

The inner **while** marches j forward as long as characters agree. If it makes it all the way to $j = m+1$, every one of the m characters matched, so s is valid. Otherwise it bailed early.

Algorithm lab

Let us trace it. Text $T =$ `a b a b a b c a` (so $n = 8$) and pattern $P =$ `a b a b c` (so $m = 5$). The candidate shifts run from 0 to 3.
Shift $s = 0$: compare $T[1] =$ `a` vs $P[1] =$ `a` (ok), $T[2] =$ `b` vs $P[2] =$ `b` (ok), $T[3] =$ `a` vs $P[3] =$ `a` (ok), $T[4] =$ `b` vs $P[4] =$ `b` (ok), $T[5] =$ `a` vs $P[5] =$ `c` – *mismatch*. Four comparisons matched, the fifth failed.
Shift $s = 1$: compare $T[2] =$ `b` vs $P[1] =$ `a` – *mismatch* on the very first character. One comparison.
Shift $s = 2$: $T[3] =$ `a` vs `a` (ok), $T[4] =$ `b` vs `b` (ok), $T[5] =$ `a` vs `a` (ok), $T[6] =$ `b` vs `b` (ok), $T[7] =$ `c` vs `c` (ok). All five match! Report shift $s = 2$.
Shift $s = 3$: $T[4] =$ `b` vs $P[1] =$ `a` – mismatch.
The only valid shift is $s = 2$, and indeed $T[3\,..\,7] =$ `ababc` $= P$. Notice the wasteful part: at shift 0 we worked through four matching characters before the fifth failed, and then at shift 1 we threw all of that away and started from scratch. The rest of this chapter is about eliminating that waste.

31.2.1 How slow can the naive matcher be?

Each shift costs at most m comparisons in the inner loop, and there are $n - m + 1$ shifts, so the total is at most $(n - m + 1)\,m$ comparisons. In the worst case this bound is tight. Take a text of n copies of the letter `a` and a pattern of m `a`'s followed by one `b`, e.g. $T =$ `aaaaaa` and $P =$ `aab`. At every shift the inner loop matches all the leading `a`'s and only fails on the final `b`, so each of the $n - m + 1$ shifts burns the full m comparisons.

Cost summary

Naive string matching uses $O((n - m + 1)\,m)$ character comparisons, which is $O(nm)$ in the worst case. It uses $O(1)$ extra space and needs no preprocessing of the pattern. Its average case on random text is actually decent – close to $O(n)$ – because mismatches usually come fast. But the worst case is quadratic, and the algorithms below improve on it.

Common trap

The naive matcher's flaw is not that it checks every shift – in the worst case every shift really might be valid, so you cannot avoid *looking* at each. The flaw is that it re-reads text characters it has already examined. At shift 0 it read $T[1\,..\,5]$; at shift 1 it re-read $T[2]$; and so on. A single text character can be inspected up to m times. The fast algorithms all follow one rule: *look at each text character a bounded number of times* – ideally just once.

Why this matters

Why care about beating $O(nm)$? Because n is often very large. A genome is three billion characters. A web-scale text index runs to terabytes. A log file you are grepping might be gigabytes. If your pattern is a few dozen characters and your matcher is quadratic in the bad case, an adversary – or just unlucky data – can turn a one-second search into an hour. At that scale, linear-time matching decides whether a tool finishes or hangs.

31.3 Rabin–Karp: matching by fingerprint

A different approach starts from a simple observation. Comparing two strings of length m takes up to m character comparisons, but comparing two *numbers* takes one operation. What if we could turn each length-m window of text into a single number, turn the pattern into a single number, and just compare numbers?

That is the Rabin–Karp idea. We assign to every string a numeric *fingerprint*, computed so that equal strings always get equal fingerprints. Then, instead of comparing the pattern against each text window character by character, we compare fingerprints – one integer comparison per window.

31.3.1 Treating a string as a number

Suppose for a moment the alphabet is the ten digits $\{0, 1, \ldots, 9\}$, so $|\Sigma| = 10$. Then a string *is* a number written in base 10. The pattern $P = \texttt{31415}$ has the value

$$p = 3 \cdot 10^4 + 1 \cdot 10^3 + 4 \cdot 10^2 + 1 \cdot 10^1 + 5 \cdot 10^0 = 31415.$$

For a general alphabet, treat each character as a "digit" in base $d = |\Sigma|$, and the same construction gives every length-m string a value between 0 and $d^m - 1$. Let t_s denote the value of the length-m text window starting after shift s, i.e. of $T[s+1\,..\,s+m]$. Then s is a valid shift exactly when $t_s = p$ – provided we computed both values correctly.

This raises two problems, and Rabin–Karp has a fix for each.

Problem one: the numbers are astronomically large. A pattern of length $m = 20$ over bytes has a value up to 256^{20}, a 48-digit number. Exact arithmetic on m-byte integers takes $\Theta(m)$ time rather than $O(1)$, undoing the speedup. *Fix:* do all arithmetic modulo a prime q. Choose q small enough that dq fits in one machine word, so every operation is genuinely $O(1)$. Now p and each t_s are reduced mod q to a small number we call a *fingerprint*.

Problem two: computing each t_s from scratch costs m operations, which buys us nothing. *Fix:* compute t_{s+1} from t_s in *constant* time with a *rolling* update. Slide the window one step right: you drop the high-order character $T[s+1]$ and append the new low-order character $T[s+m+1]$. In base d that is

$$t_{s+1} = d\left(t_s - T[s+1] \cdot d^{m-1}\right) + T[s+m+1].$$

Read it left to right: subtract off the contribution of the leaving character ($T[s+1]$, which sat in the top place worth d^{m-1}), shift everything up one place by multiplying by d, then add the entering character in the ones place. A car odometer rolls over the same way. This constant-time update is what makes the method fast. Figure 31.2 shows one such step on the decimal example.

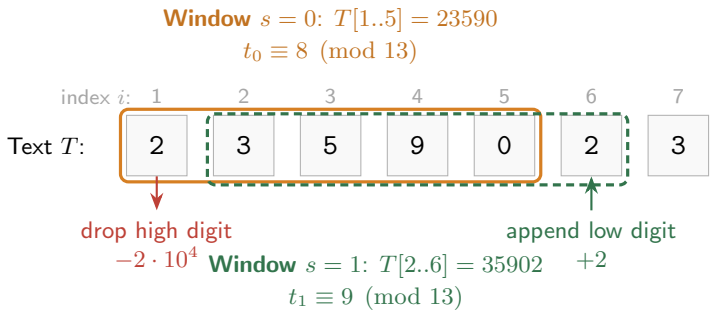


Figure 31.2: One rolling step on $T = 235902\ldots$ with $m = 5$, $d = 10$, $q = 13$. The window slides from $T[1\,..\,5]$ (orange) to $T[2\,..\,6]$ (green): the high digit $T[1] = 2$ leaves the top place (worth 10^4), the remainder shifts left ($\times 10$), and the new digit $T[6] = 2$ enters the units place. The new fingerprint $t_1 = \big(10(t_0 - T[1] \cdot h) + T[6]\big) \bmod 13$ costs $O(1)$ arithmetic operations rather than $O(m)$ character scans.

Intuition

A fingerprint is a tiny summary of a long string – like the last few digits of a credit card number, or a checksum. If two strings are equal, their fingerprints are equal, guaranteed. The converse is *not* guaranteed: two different strings can collide on the same fingerprint, the way two people can share a birthday. So a fingerprint match is a *strong hint*, not a proof. Rabin–Karp uses fingerprints to skip the expensive character-by-character check almost everywhere, and only does the real check on the rare windows whose fingerprints actually match the pattern's.

31.3.2 Modular arithmetic and the spurious-hit check

Working modulo q means t_s is really $t_s \bmod q$. The valid-shift test becomes: if $t_s \equiv p \pmod{q}$, the window is a *candidate*. But congruence mod q does not imply equality of the underlying strings – two different windows can be congruent mod q by coincidence. We call such a window a *spurious hit*. So whenever the fingerprints match, we must *verify* by comparing the actual characters $T[s+1\,..\,s+m]$ against $P[1\,..\,m]$. If they truly match, report s; if not, it was a spurious hit and we move on.

Algorithm 133: RABIN-KARP-MATCHER

Input: Text $T[1\,..\,n]$, pattern $P[1\,..\,m]$, radix $d = |\Sigma|$, prime q.
Output: Every valid shift s.

```
1  h ← d^{m−1} mod q ;                                  // value of the top place
2  p ← 0;   t ← 0;
3  for i ← 1 to m do
4  |  p ← (d · p + P[i]) mod q ;                        // Horner: fingerprint of P
5  |  t ← (d · t + T[i]) mod q ;                        // fingerprint of T[1..m]
6  for s ← 0 to n − m do
7  |  if p = t then
8  |  |  if P[1..m] = T[s + 1..s + m] then
9  |  |  |  report "P occurs with shift s";
10 |  if s < n − m then
11 |  |  t ← (d (t − T[s + 1] · h) + T[s + m + 1]) mod q;
```

The first loop is *Horner's rule*: building a number digit by digit, each step multiplying the running total by the base and adding the next digit, all mod q. The second loop walks the windows, comparing fingerprints, verifying hits, and rolling the window forward. (In real code, after subtracting $T[s+1]\cdot h$ the value can go negative; add q before taking the mod to keep it in range.)

Algorithm lab

Let us do the classic decimal example. Text $T =$ `2 3 5 9 0 2 3 1 4 1 5 2 6 7 3 9 9 2 1` ($n = 19$), pattern $P =$ `3 1 4 1 5` ($m = 5$), radix $d = 10$, prime $q = 13$.
First the pattern's fingerprint. By Horner mod 13: $31415 \bmod 13$. Compute $31415 = 13 \cdot 2416 + 7$, so $p = 7$.
Now the first window $T[1\,..\,5] =$ `23590`: $23590 = 13 \cdot 1814 + 8$, so $t_0 = 8$. Since $8 \neq 7$, no candidate.
Roll forward. The high digit leaving is $T[1] = 2$, the place value $h = 10^4 \bmod 13$. Since $10^4 = 10000 = 13{\cdot}769{+}3$, we have $h = 3$. Each roll: $t' = (10\,(t{-}T[\text{leave}]{\cdot}3){+}T[\text{enter}]) \bmod 13$.
The window `31415` appears as $T[7\,..\,11]$, i.e. at shift $s = 6$. Its true value $31415 \bmod 13 = 7 = p$, so the fingerprints match: a candidate. We verify $T[7\,..\,11] =$ `31415` $= P$ – a real hit. Report $s = 6$.
Watch for a spurious hit. The window `67399` has value $67399 = 13 \cdot 5184 + 7$, so its fingerprint is also $7 = p$! The fingerprints match, but `67399` $\neq$ `31415`. Character-by-character verification catches the spurious collision, so we do *not* report a match. That window was a spurious hit – two strings sharing a fingerprint by coincidence, exactly the collision the intuition box warned about.

31.3.3 Why it is usually fast, and why it can be slow

Preprocessing – computing p, t_0, and h – is $\Theta(m)$. The main loop runs $n-m+1$ times; each iteration does $O(1)$ fingerprint work plus, only on a hit, an $O(m)$ verification. So the running time hinges entirely on how often fingerprints match.

In the worst case fingerprints match every time. A worst-case input is T all `a`'s, P all `a`'s: every

window equals the pattern, every shift is a real hit, and we pay $\Theta(m)$ to verify each, for $\Theta((n-m+1)m)$ – right back to naive's quadratic. But that input is degenerate. On "ordinary" data, with a well-chosen prime q, spurious hits are rare. If we model the fingerprints as roughly uniform over the q residues, the chance a given invalid shift looks like a hit is about $1/q$. With at most n shifts, the expected number of spurious hits is about n/q, and the expected verification work is $O((n/q)\,m)$. If q is chosen larger than m (easy, since q can be nearly word-sized), this is $O(n)$, and the whole algorithm runs in expected

$$O(n) + O(v \cdot m),$$

where v is the number of valid shifts – linear, plus the unavoidable cost of confirming the real occurrences.

Cost summary

Rabin–Karp. Preprocessing $\Theta(m)$. Worst-case matching $\Theta((n-m+1)m)$ – no better than naive, attained when fingerprints collide constantly. Expected matching $O(n+vm)$ on data where collisions are rare and q is large, which for a small fixed number of occurrences is $O(n)$. Extra space $O(1)$.

Focus

Beyond single-pattern search, fingerprinting scales naturally: because a fingerprint is just an integer, you can keep a *set* of target fingerprints in a hash table and search for many patterns simultaneously in one pass, or extend the rolling-hash to two dimensions to locate subimages inside large bitmaps. The same rolling hash powers synchronization utilities like `rsync` and content-defined storage deduplication. CLRS covers Rabin–Karp in Section 32.2.

Walking question

Rabin–Karp is fast only if fingerprints rarely collide. But an adversary who *knows your prime* q can craft a text where every window collides, dragging you back to $O(nm)$. How would you defend against such an adversary? (Hint: what if q, or the radix, were chosen at random and kept secret – how does that change the analysis, and what does it cost you?) This is the same randomization idea used in quicksort and hashing.

31.4 String matching with finite automata

The next idea drops probability entirely and returns to a worst-case guarantee. It also achieves the goal set out in the trap box above: *read each text character exactly once, never backing up.* The price is some work up front – we compile the pattern into a little machine.

31.4.1 What a string-matching automaton is

A *finite automaton* is a tiny processor with a finite set of *states*. It starts in a designated start state, then reads the text one character at a time, and each character makes it jump to a new state according to a fixed *transition function* δ. Some states are marked *accepting*. For string matching we build the machine so that, after reading $T[1\,..\,i]$, its current state is precisely *how much of the*

pattern we have matched so far – a number from 0 to m. State m is accepting: reaching it means the last m characters we read were exactly the pattern, so we have just found an occurrence ending at position i.

Definition 31.2 (String-matching automaton)**.** For a pattern $P[1\,..\,m]$ the matching automaton has state set $Q = \{0, 1, \ldots, m\}$, start state 0, single accepting state m, and transition function $\delta(q, a)$ defined for a state q and a character $a \in \Sigma$ by

$$\delta(q, a) = \text{the length of the longest prefix of } P \text{ that is also a suffix of } P_q\, a,$$

where $P_q = P[1\,..\,q]$ is the length-q prefix of the pattern and $P_q\, a$ means P_q with the character a appended.

In words, the state q means "the last q characters I read were $P[1\,..\,q]$." Now a new character a arrives. The string I have matched, with a stuck on the end, is $P_q\, a$. The new state must answer: *after seeing a, what is the longest prefix of P I can still claim to have just matched at the end?* That is the longest prefix of P that is a suffix of $P_q\, a$. If a happened to be exactly $P[q+1]$, then $P_q\, a = P[1\,..\,q+1]$ is itself a prefix of P, and we advance to state $q+1$. If a was wrong, we cannot advance – but we might not have to fall all the way to 0, because a shorter prefix of P might still match at the tail. The automaton precomputes the correct landing state for every (q, a) pair.

Intuition

Think of climbing a ladder with m rungs, where rung q means "I have matched the first q characters of the pattern." Each text character you read either moves you up one rung (you read the character the pattern wanted next) or knocks you down to some lower rung (you read something else). The interesting part is exactly *how far down* a wrong character knocks you. It does not necessarily send you back to the ground, because the characters you just read might form the start of a fresh match. The transition function δ has the right landing rung stored for every situation, so the machine never hesitates.

Once δ is built, scanning is simple.

Algorithm 134: FINITE-AUTOMATON-MATCHER

Input: Text $T[1\,..\,n]$, transition function δ, pattern length m.
Output: Every valid shift s.

```
q ← 0 ;                                   // start state:  nothing matched yet
for i ← 1 to n do
    q ← δ(q, T[i]) ;                      // one table lookup, no backtracking
    if q = m then
        report "P occurs with shift s = i − m";
```

The whole body is one lookup per text character and a check for the accepting state. There is no inner loop, and the pointer i never moves backward. This is the structure of `grep` on a fixed string, and of the lexical analyzer in every compiler.

31.4.2 A worked machine

Let $P =$ `ababaca`, so $m = 7$, over the alphabet $\Sigma = \{\mathtt{a}, \mathtt{b}, \mathtt{c}\}$. The states are 0 through 7. We compute $\delta(q, a)$ for each by the definition: the longest prefix of P that is a suffix of $P_q a$.

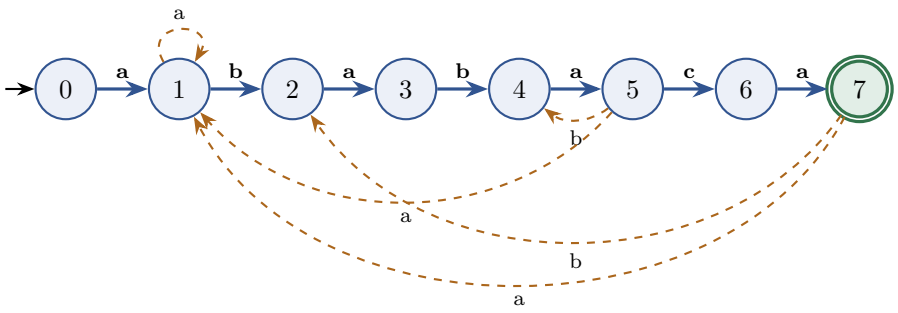
Take $\delta(5, \texttt{b})$. State 5 means we have matched $P_5 = \texttt{ababa}$. Append `b`: we get `ababab`. What is the longest prefix of `abababaca` that is a suffix of `ababab`? Check suffixes longest-first: `ababab`? not a prefix of P. `babab`? no. `abab`? yes – `abab` is a prefix of P (it is P_4). So $\delta(5, \texttt{b}) = 4$. We read a wrong character and only slipped from rung 5 to rung 4, not to the ground.

Take $\delta(5, \texttt{a})$. Append `a` to `ababa`: get `ababaa`. Longest prefix of P that is a suffix? `a` works (P_1); `aa` is not a prefix of P. So $\delta(5, \texttt{a}) = 1$.

And the happy case $\delta(5, \texttt{c})$. Append `c`: `ababac`, which is P_6 itself – a prefix of P. So $\delta(5, \texttt{c}) = 6$: we advance. Here is the full table:

state q	$\delta(q, \texttt{a})$	$\delta(q, \texttt{b})$	$\delta(q, \texttt{c})$
0	1	0	0
1	1	2	0
2	3	0	0
3	1	4	0
4	5	0	0
5	1	4	6
6	7	0	0
7	1	2	0

Here is the same machine as a state diagram. The forward spine (solid blue) advances character by character along the pattern $P = \texttt{abababaca}$. The curved fallback transitions (dashed orange) show the non-trivial retreats when a mismatch occurs, illustrating how the machine preserves the longest matching prefix without ever rescanning earlier text characters.



Algorithm lab

Let us run the machine on $T = \texttt{a b a b a b a c a b a}$ ($n = 11$). Start in state $q = 0$. Read each character and look up δ.

i	$T[i]$	new state $q = \delta(q, T[i])$	note
1	a	$\delta(0, a) = 1$	
2	b	$\delta(1, b) = 2$	
3	a	$\delta(2, a) = 3$	
4	b	$\delta(3, b) = 4$	
5	a	$\delta(4, a) = 5$	
6	b	$\delta(5, b) = 4$	wrong char, slip to 4
7	a	$\delta(4, a) = 5$	
8	c	$\delta(5, c) = 6$	
9	a	$\delta(6, a) = 7$	**accept!** shift $s = 9 - 7 = 2$
10	b	$\delta(7, b) = 2$	
11	a	$\delta(2, a) = 3$	

At $i = 9$ the machine entered state $7 = m$, so the pattern `ababaca` occurs with shift 2 – check: $T[3\,..\,9] =$ `ababaca` $= P$. Notice especially row $i = 6$: a stray `b` where the pattern wanted `c`. The naive matcher would have restarted; the automaton simply slipped from state 5 to state 4 and kept reading T forward, never re-examining a single character.

31.4.3 Cost, and the one catch

Scanning is cheap: the loop runs n times with one table lookup each, so matching is $\Theta(n)$ – guaranteed, no probability, no backtracking. The catch is building δ. The table has $m + 1$ states times $|\Sigma|$ characters $= (m + 1)|\Sigma|$ entries, and the straightforward way to compute each entry tests prefixes against suffixes, costing up to $O(m)$ per entry, for $O(m^2|\Sigma|)$ preprocessing. (A cleverer construction, closely related to the prefix function of the next section, brings this down to $O(m|\Sigma|)$.)

Cost summary

Finite-automaton matcher. Matching $\Theta(n)$ in the worst case as well as on average. Preprocessing $O(m^2|\Sigma|)$ by the direct construction, $O(m|\Sigma|)$ by the more structured one. Extra space $O(m|\Sigma|)$ for the transition table. The matching phase reads each text character exactly once and never backtracks.

Common trap

The automaton's transition table is $\Theta(m|\Sigma|)$ in size. For DNA ($|\Sigma| = 4$) or ASCII this is compact. But for Unicode, where $|\Sigma|$ can be over a million, the table becomes enormous – it can dwarf the text you are searching. This dependence on $|\Sigma|$ is the automaton's primary drawback, and it is exactly the weakness KMP, our final algorithm, was designed to remove.

31.5 The Knuth–Morris–Pratt algorithm

KMP achieves the same $\Theta(n)$ scan as the automaton, with the same no-backtracking guarantee, but its preprocessing is $\Theta(m)$ and *entirely independent of the alphabet size*. It needs no transition table

at all. Instead it carries one small array, the *prefix function*, which captures the pattern's internal self-similarity. You can think of the prefix function as the automaton's table compressed down to what actually matters.

31.5.1 The insight: a mismatch already tells you a lot

Return once more to the wasteful moment in the naive matcher. We were matching P against the text, we had matched q characters, and then character $q+1$ disagreed. The naive matcher slides the pattern one step and restarts from the beginning. But consider what we already *know*: the last q text characters were exactly $P[1\,..\,q]$. That is real information about the text, paid for with q comparisons, and KMP keeps it.

Here is the question KMP asks. We matched $P[1\,..\,q]$ and then failed. We must slide the pattern forward – but how far can we slide it without missing a possible match? A match cannot start anywhere that would contradict the q characters we already verified. The pattern, after sliding, must agree with the tail of $P[1\,..\,q]$ wherever they now overlap. In other words: we want the longest *proper* prefix of $P[1\,..\,q]$ that is also a suffix of $P[1\,..\,q]$. That length tells us the new state to drop to – *and it depends only on the pattern, not on the text at all.*

Definition 31.3 (Prefix function)**.** The *prefix function* of a pattern $P[1\,..\,m]$ is the array $\pi[1\,..\,m]$ where

$$\pi[q] = \text{the length of the longest proper prefix of } P[1\,..\,q] \text{ that is also a suffix of } P[1\,..\,q].$$

"Proper" means strictly shorter than $P[1\,..\,q]$ itself (so we never pick the whole thing). When no proper prefix is also a suffix, $\pi[q] = 0$.

Intuition

The prefix function measures how much the pattern *repeats its own beginning* inside itself. Suppose you have matched the first q characters and then hit a mismatch. Those q characters are $P[1\,..\,q]$. If the *end* of that chunk happens to repeat the *beginning* of the pattern – say the last 3 characters of what you matched are also the first 3 characters of P – then you have effectively already matched 3 characters of a *fresh* attempt, for free, without re-reading any text. So you do not restart from zero; you restart from 3. The prefix function precomputes, for every q, exactly how big that free head start is.

31.5.2 Computing the prefix function by hand

Let $P =$ `ababaca` again, $m = 7$. We fill $\pi[q]$ for $q = 1, \ldots, 7$.

- $\pi[1]$: $P[1\,..\,1] =$ `a`. A proper prefix is strictly shorter, so the only candidate is the empty string. $\pi[1] = 0$.
- $\pi[2]$: $P[1\,..\,2] =$ `ab`. Proper prefixes: `a`. Is `a` a suffix of `ab`? No (the suffix of length 1 is `b`). $\pi[2] = 0$.
- $\pi[3]$: `aba`. Proper prefixes `ab`, `a`. Is `ab` a suffix? The length-2 suffix is `ba`; no. Is `a` a suffix? The length-1 suffix is `a`; yes. $\pi[3] = 1$.

- $\pi[4]$: `abab`. Is `aba` a suffix? Length-3 suffix is `bab`; no. Is `ab` a suffix? Length-2 suffix is `ab`; yes. $\pi[4] = 2$.
- $\pi[5]$: `ababa`. Is `abab` a suffix? Length-4 suffix is `baba`; no. Is `aba` a suffix? Length-3 suffix `aba`; yes. $\pi[5] = 3$.
- $\pi[6]$: `ababac`. Length-5 suffix `babac`? no. Length-4 `abac`? no. Length-3 `bac`? no. Length-2 `ac`? no. Length-1 `c`? no. $\pi[6] = 0$.
- $\pi[7]$: `abababaca`. Down the suffixes… only `a` (length 1) is a prefix and a suffix. $\pi[7] = 1$.

q	1	2	3	4	5	6	7
$P[q]$	a	b	a	b	a	c	a
$\pi[q]$	0	0	1	2	3	0	1

Read $\pi[5] = 3$ aloud: "if I have matched `ababa` and then fail, the last 3 characters `aba` are also the first 3, so I restart from a head start of 3." The automaton needed a whole row of its table to express what this single number does. Figure 31.3 draws this overlap: the longest proper prefix of `ababa` that is also a suffix.

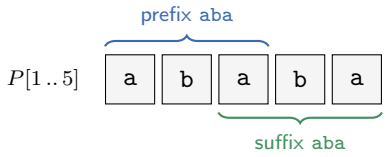


Figure 31.3: Why $\pi[5] = 3$ for $P =$ `ababaca`. Inside $P[1..5] =$ `ababa`, the prefix `aba` (blue, positions 1–3) equals the suffix `aba` (green, positions 3–5); they even overlap at position 3. That length-3 agreement is the free head start KMP keeps after a mismatch at state 5.

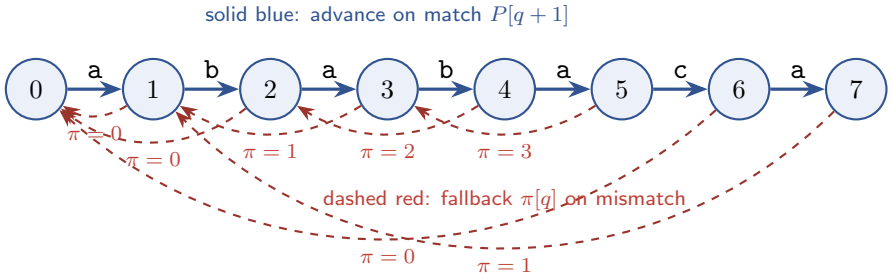


Figure 31.4: The KMP prefix function π viewed as state failure links. When matching at state $q = 5$ ($P[1..5] =$ `ababa`) fails to match the next character `c`, the algorithm retreats along the dashed link to $\pi[5] = 3$, where the prefix `aba` is guaranteed to match the text so far. The text pointer i never steps backward.

31.5.3 Computing the prefix function efficiently

Filling π by brute force, as above, costs $O(m^2)$. There is a linear-time method, and it uses the prefix function to compute *itself*. Process q from 2 upward, carrying a variable $k = \pi[q-1]$, the length

matched so far. To extend to position q, we want to match $P[q]$ against $P[k+1]$. If they agree, the matched length grows: $k \leftarrow k+1$. If they disagree, we fall back to the next-shorter candidate, which is precisely $\pi[k]$ – the prefix function calling on itself – and try again, until either we match or k hits 0.

Algorithm 135: COMPUTE-PREFIX-FUNCTION

Input: Pattern $P[1\,..\,m]$.
Output: The prefix-function array $\pi[1\,..\,m]$.

```
1 π[1] ← 0;
2 k ← 0 ;                                  // length of the current longest prefix-suffix
3 for q ← 2 to m do
4 |   while k > 0 and P[k + 1] ≠ P[q] do
5 |   |   k ← π[k] ;                       // fall back to the next-best candidate
6 |   if P[k + 1] = P[q] then
7 |   |   k ← k + 1 ;                      // extend the match by one
8 |   π[q] ← k;
9 return π;
```

31.5.4 KMP itself

With π in hand, matching works like the automaton but without its table. We scan the text with a pointer i that never retreats, and a state q counting how much of the pattern is currently matched. On a mismatch we do not restart – we consult $\pi[q]$ to find the right fallback state, exactly as COMPUTE-PREFIX-FUNCTION fell back on itself.

Algorithm 136: KMP-MATCHER

Input: Text $T[1\,..\,n]$, pattern $P[1\,..\,m]$.
Output: Every valid shift s.

```
1 π ← Compute-Prefix-Function(P);
2 q ← 0 ;                                  // number of pattern characters matched so far
3 for i ← 1 to n do
4 |   while q > 0 and P[q + 1] ≠ T[i] do
5 |   |   q ← π[q] ;                       // mismatch: slide using the prefix function
6 |   if P[q + 1] = T[i] then
7 |   |   q ← q + 1 ;                      // match: advance the state
8 |   if q = m then
9 |   |   report "P occurs with shift s = i − m";
10|   |   q ← π[q] ;                       // look for the next, possibly overlapping, match
```

The two loops have the same structure. COMPUTE-PREFIX-FUNCTION matches the pattern against *itself*; KMP-MATCHER matches the pattern against the *text*. Both advance on agreement and both consult π on disagreement. After a full match we set $q \leftarrow \pi[m]$ so that overlapping occurrences (think of pattern `aa` in text `aaa`) are not missed.

Algorithm lab

Run KMP with $P =$ `ababaca` (and $\pi = [0, 0, 1, 2, 3, 0, 1]$ from above) on $T =$ `a b a b a b a c a b a` ($n = 11$). Start $q = 0$.

i	$T[i]$	action	q after
1	a	$P[1]$ =a matches; advance	1
2	b	$P[2]$ =b matches; advance	2
3	a	$P[3]$ =a matches; advance	3
4	b	$P[4]$ =b matches; advance	4
5	a	$P[5]$ =a matches; advance	5
6	b	$P[6]$ =c $\neq$ b: fall back $q \leftarrow \pi[5] = 3$; now $P[4]$ =b matches; advance	4
7	a	$P[5]$ =a matches; advance	5
8	c	$P[6]$ =c matches; advance	6
9	a	$P[7]$ =a matches; advance $\rightarrow 7 = m$: **report** $s = 2$; set $q \leftarrow \pi[7] = 1$	1
10	b	$P[2]$ =b matches; advance	2
11	a	$P[3]$ =a matches; advance	3

Study row $i = 6$. We had matched 5 characters (`ababa`) and the text gave us `b` where the pattern wanted `c`. Instead of backing up, KMP consulted $\pi[5] = 3$: the prefix `aba` of what we matched is also its suffix, so we keep a head start of 3 and immediately try $P[4] =$ `b` against the same $T[6] =$ `b` – it matches, and we are at state 4 without having moved i backward at all.

31.5.5 Why KMP is linear: an amortized argument

The worry with KMP is the inner **while** loop. Could it spin many times per character and blow up the cost? No: an amortized argument that tracks the state q like a bank balance shows that the total work is linear.

Focus

Watch the value of q across the whole run. Each iteration of the outer **for** loop increases q by *at most* 1 (the single "advance" line). Each iteration of the inner **while** loop *strictly decreases* q (since $\pi[q] < q$ always – the prefix is *proper*). And q never goes below 0. So q is a counter that rises by at most 1 per text character and can never fall below zero.

Formal

Use the *potential method*. Let the potential be the current value of q, which starts at 0 and is always ≥ 0. Across the n iterations of the outer loop, the total *increase* in q is at most n (at most 1 per iteration). Every inner-**while** iteration decreases q by at least 1. Since total decrease cannot exceed total increase plus the starting value, the inner loop executes at most n times *over the entire algorithm* – not per character, but in total. Hence the matching phase does $O(n)$ work. Computing π is the same argument over P, giving $O(m)$. Together, KMP runs in $\Theta(n + m) = \Theta(n)$ time.

This is the same accounting argument used for the dynamic array and the union-find structure: a

single operation may be expensive, but expensive steps must be "paid for" by cheap ones that came before, so the average over the whole run is small. Here, every fall-back must have been earned by an earlier advance.

Figure 31.5 plots q across the trace from the lab above. Every up-step is one advance; every down-step is a fall-back. Because q can only fall as far as it has previously climbed, the total descent over the whole run is bounded by the total ascent, which is at most n.

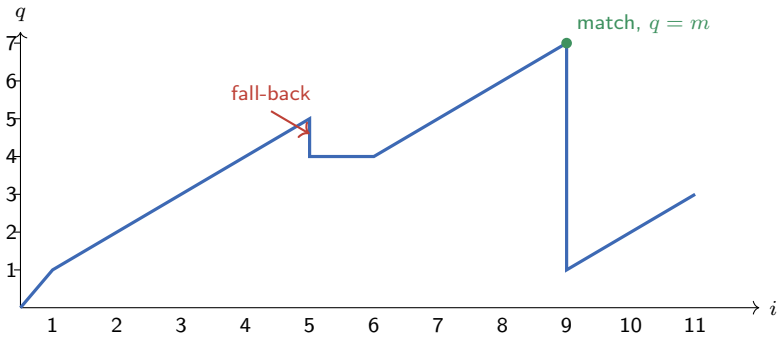


Figure 31.5: The state q during the KMP run on $T =$ `abababacaba` with $P =$ `ababaca`. Diagonal rises are advances (at most one per text character); vertical drops are fall-backs through π (at $i = 6$, and the reset to $\pi[7] = 1$ after the match at $i = 9$). Total drop $\leq$ total rise $\leq n$, so the inner loop fires $O(n)$ times in all.

Cost summary

Knuth–Morris–Pratt. Preprocessing $\Theta(m)$, and – unlike the automaton – *independent of* $|\Sigma|$. Matching $\Theta(n)$ in the worst case. Total $\Theta(n + m)$. Extra space $\Theta(m)$ for the array π. Each text character is examined a small *amortized* constant number of times; the pointer i never moves backward.

Focus

Knuth–Morris–Pratt keeps only the part of the full automaton it actually needs: rather than storing transitions for every character in the alphabet, the prefix function π records the single best fallback state on mismatch. The same *failure function* idea appears in parsing and automata theory, and it led directly to the Aho–Corasick algorithm for matching dictionaries of multiple patterns simultaneously. CLRS treats finite-automaton matching in Section 32.3 and KMP in Section 32.4; the prefix function there is named exactly as we have named it here (π).

Common trap

A common confusion: people think the inner **while** loop makes KMP quadratic because "it has a loop inside a loop." It does not. The number of nested-loop iterations is bounded *globally*, not per outer step, by the potential argument. Counting loops by their nesting depth, rather than by how many times they truly run, is a common analysis error, and KMP is the standard example.

31.6 Putting the four side by side

We have gone from the obvious method to an optimal one. Here is the comparison side by side.

Algorithm	Preprocessing	Matching	Notes
Naive	0	$O((n-m+1)m)$	no extra space; re-reads text
Rabin–Karp	$\Theta(m)$	$O(n+vm)$ expected	worst case $O((n-m+1)m)$
Finite automaton	$O(m\|\Sigma\|)$	$\Theta(n)$	big table; $\|\Sigma\|$-dependent
KMP	$\Theta(m)$	$\Theta(n)$	small array; $\|\Sigma\|$-independent

Here v is the number of valid shifts. Naive is simplest but re-reads text. Rabin–Karp escapes re-reading on *average* by gambling on fingerprints. The automaton escapes it *always* but pays in a fat, alphabet-sized table. KMP keeps the automaton's worst-case linear guarantee while shrinking the table down to a single m-element array that does not care how big the alphabet is. That progression – obvious, then probabilistic, then guaranteed-but-heavy, then guaranteed-and-light – recurs all over algorithm design.

Why this matters

Exact string matchers run inside everyday tools: KMP and Boyer–Moore variants are used in `grep`, your editor's find command, DNA aligners scanning genomes, network intrusion detectors comparing packets against attack signatures at line rate, and the tokenizers of compilers. Regular-expression engines are built on the automaton view. And Rabin–Karp's rolling hash is used by file-syncing and deduplication tools that decide which blocks of a multi-gigabyte file actually changed. The ideas in this short chapter underlie much of everyday text processing.

31.6.1 From text streams to genomic search

Avoiding backtracking, together with the other ideas in this chapter, made several applications practical:

- **True streaming search without buffering.** Because KMP inspects each character of the text exactly once and never rewinds the text pointer, it can operate over infinite, non-seekable data streams. Network intrusion detection systems (like Snort) inspect packet payloads at line rate without caching gigabytes of traffic in RAM.
- **Sublinear skipping in practice.** The Boyer–Moore algorithm (1977) showed that scanning the pattern from right to left allows skipping long stretches of text without reading them at all. On English prose, it routinely achieves an effective running time of $O(n/m)$ character reads, and GNU `grep` uses a variant of it for fixed strings.
- **Multi-pattern scaling.** Generalizing the finite-automaton view of Section 31.4 to a tree of patterns yields the *Aho–Corasick algorithm* (1975). It matches an entire dictionary of k patterns simultaneously in a single linear pass over the text, with running time $O(n + m_{\text{total}} + z)$ where z is the number of matches.
- **Algebraic fingerprinting and chunking.** Rabin–Karp's rolling hash has uses well beyond string matching, most notably in content-defined chunking (e.g., the `rsync` protocol and backup

deduplication systems), where files are divided into variable-length blocks at fingerprint boundaries to synchronize differences across the internet with minimal bandwidth.

31.6.2 Practical trade-offs and engineering limits

These algorithms also have practical limits:

- **Fragility to noise and mutations.** The algorithms in this chapter are strictly *exact*. Change a single character, delete a comma, or insert a single DNA base pair, and KMP, Boyer–Moore, and Rabin–Karp report zero matches. In biological sequencing or optical character recognition, approximate matching via dynamic programming (edit distance and local alignment) becomes necessary.
- **Preprocessing penalties on tiny inputs.** Preprocessing a pattern costs $\Theta(m)$ for KMP and $\Theta(m|\Sigma|)$ for a finite automaton. If you are searching for a short word in a tiny snippet of text, the naive matcher is often faster in wall-clock time because it pays zero initialization overhead.
- **Memory footprint on large alphabets.** The transition table δ of a string-matching automaton requires $\Theta(m|\Sigma|)$ entries. For 16-bit or 32-bit Unicode alphabets, this table easily swells to tens of megabytes, thrashing CPU caches. KMP's prefix function π circumvents this by paying an occasional amortized fall-back loop, independent of alphabet size.
- **Adversarial collisions in Rabin–Karp.** Rabin–Karp achieves its fast average time only when hash collisions are rare. If an adversary knows the chosen prime modulus q, they can construct texts where every shift produces a spurious collision, degrading the algorithm to its $\Theta(nm)$ worst case.

Pause

The main lesson of this chapter: the fast string matchers all win by *never re-reading the text* – they invest a little preprocessing of the pattern so that, when a match breaks down, they already know exactly how far to slide without going back. Rabin–Karp buys that with a fingerprint; the automaton with a table; KMP with the prefix function.

In this chapter, we tracked exact patterns across one-dimensional sequences of discrete characters. In Chapter 32, we step from discrete strings to continuous space: computational geometry, where objects are points, segments, and polygons in the plane, and the fundamental questions concern spatial orientation, convex hulls, and geometric intersections.

31.7 Exercises

Exercise 31.1 (Warmup: trace the naive matcher)**.** Run Naive-String-Matcher on the text $T =$ `aaabaaaab` and pattern $P =$ `aaab`. List every shift, the number of character comparisons it makes at that shift, and which shifts are valid. What is the total comparison count?

Exercise 31.2 (The quadratic worst case)**.** Exhibit a text and pattern, each parameterized by n and m, on which the naive matcher makes exactly $(n - m + 1)\,m$ character comparisons. Argue briefly why no shift can be skipped on your input.

Exercise 31.3 (Rolling hash by hand)**.** With radix $d = 10$ and prime $q = 11$, compute the Rabin–Karp fingerprint of the pattern $P =$ `2718` (so $m = 4$), and the fingerprint of the first length-4 window `9271` of $T =$ `927183`. Then use the rolling update to obtain the fingerprint of the second window `2718` and the third window `7183`. Show every modular step, and say which window is a real match for P.

Exercise 31.4 (Spurious hits)**.** For $d = 10$ and $q = 13$, find two distinct 3-digit strings that have the same Rabin–Karp fingerprint. Explain why the verification step in RABIN-KARP-MATCHER is not optional, using your example.

Exercise 31.5 (Choosing the prime)**.** Suppose your text has length $n = 10^6$ and you want the *expected* number of spurious hits to be at most 1. Using the estimate that the expected number of spurious hits is about n/q, what is the smallest prime q that suffices? How large does the machine word need to be so that $d \cdot q$ fits, for byte data ($d = 256$)?

Exercise 31.6 (Compute a prefix function)**.** Compute the prefix function π for $P =$ `aabaabaaa` by hand, showing the longest prefix-that-is-a-suffix at each position. Then verify your answer by running COMPUTE-PREFIX-FUNCTION and listing the value of k at the end of each iteration of the outer loop.

Exercise 31.7 (KMP trace)**.** For the pattern $P =$ `ababaca` with prefix table $\pi = [0, 0, 1, 2, 3, 0, 1]$, trace KMP-MATCHER on $T =$ `abacababacaba`. Make a table with columns i, $T[i]$, the value of q before, any fall-backs taken, and q after. Report all valid shifts.

Exercise 31.8 (Building one automaton transition)**.** For the pattern $P =$ `aabab`, compute $\delta(4, \texttt{a})$ and $\delta(4, \texttt{b})$ from the definition (longest prefix of P that is a suffix of $P_4\, a$). Recall $P_4 =$ `aaba`. Show the suffixes you test.

Exercise 31.9 (Why q never overshoots)**.** Prove the invariant that drives KMP's correctness: at the top of the outer **for** loop in KMP-MATCHER, q equals the length of the longest prefix of P that is a suffix of $T[1\,..\,i-1]$. (Hint: induct on i, using the definition of the prefix function for the mismatch case.)

Exercise 31.10 (Amortized counting)**.** Consider the pattern $P =$ `aaaaab` ($m = 6$) and the text of n `a`'s. Trace how the state q rises and how the inner **while** loop fires across the scan. Confirm by direct count that the *total* number of inner-loop iterations over the whole text is at most n, matching the potential argument, even though some individual characters trigger several fall-backs.

Exercise 31.11 (Challenge: two-dimensional matching)**.** You are given an $n \times n$ grid of characters and an $m \times m$ pattern grid, and you want every position where the small grid sits inside the big one. Describe how to combine the row-wise rolling hash of Rabin–Karp with a second rolling hash down the columns to solve this in expected $O(n^2)$ time after $O(m^2)$ preprocessing. What is the analogue of a spurious hit here?

Exercise 31.12 (Challenge: shrinking the automaton table)**.** The direct automaton construction costs $O(m^2|\Sigma|)$. Explain how the prefix function π lets you compute the transition $\delta(q, a)$ for any state q and character a on the fly in amortized $O(1)$, so that you never store the full $\Theta(m|\Sigma|)$ table. (Hint: $\delta(q, a)$ either advances to $q+1$ when $P[q+1] = a$, or behaves like $\delta(\pi[q], a)$ otherwise – which is exactly the inner loop of KMP.)

Chapter 32

Computational Geometry

In Chapter 31, we searched for patterns across one-dimensional strings of characters, where order was linear. Much of the data we care about, however, lives in continuous physical space: pixels on a screen, cities on a map, atoms in a protein, or robots navigating obstacles. Each of those is a point, and the natural questions about them are geometric: which two points are closest? does this road cross that river? what is the smallest fence enclosing this herd?

To a human eye these questions look easy: glance at a scatter of dots and you spot the two nearest ones or trace their bounding perimeter. But an algorithm sees only coordinate pairs. Our main tool for answering them quickly and exactly is the 2D *cross product*, which turns the direction of a turn into the sign of a single number. Using the cross product as our orientation test, we determine segment intersections without division, trace convex hulls via Graham's scan and Jarvis's march, and locate the closest pair of points in near-linear time.

History and connection

Until the mid-1970s, geometric software consisted largely of isolated analytic formulas and ad-hoc graphics routines. That changed when Ronald Graham at Bell Laboratories designed Graham's scan in 1972 to minimize the physical pen travel of automated drafting plotters by computing the convex hull in $\Theta(n \log n)$ time. In 1973, R. A. Jarvis introduced gift-wrapping, an output-sensitive method running in $O(nh)$ time. Michael Shamos and Dan Hoey unified the discipline in Shamos's 1975 Yale doctoral dissertation, proving that computing a convex hull requires $\Omega(n \log n)$ operations by reducing sorting to convex hulls, and introducing divide-and-conquer closest-pair detection and Voronoi diagrams. When algorithms were first implemented with standard floating-point arithmetic, rounding errors caused orientation tests to violate transitivity (p left of q, q left of r, but r left of p), leading to infinite loops or self-intersecting hulls. This prompted the development of exact geometric computation, including Jonathan Shewchuk's adaptive floating-point filters and Herbert Edelsbrunner's Simulation of Simplicity.

32.1 The cross product: the only tool you really need

Everything starts with one question about *three* points p_0, p_1, p_2. Walk from p_0 straight to p_1, then turn to head toward p_2. Do you turn *left*, turn *right*, or keep going *straight*? Every algorithm in this

chapter is built on this question.

Intuition

You are driving. You reach p_1 and you want to get to p_2. If the steering wheel goes counterclockwise (left), the three points make a left turn. If it goes clockwise (right), a right turn. If you do not touch the wheel at all, the three points are *collinear* – they sit on one straight line. Almost every geometric algorithm in this chapter works by asking "left, right, or straight?" over and over.

How do we compute the turn direction from raw coordinates, without a steering wheel? We use *vectors* and the *cross product*. Treat each point as a pair $p = (p_x, p_y)$. From p_0 form two arrows:

$$\vec{u} = p_1 - p_0 = (p_{1x} - p_{0x},\ p_{1y} - p_{0y}), \qquad \vec{v} = p_2 - p_0 = (p_{2x} - p_{0x},\ p_{2y} - p_{0y}).$$

The *two-dimensional cross product* of $\vec{u}$ and $\vec{v}$ is the determinant of the 2×2 matrix whose columns are the two vectors:

$$\vec{u} \times \vec{v} \ = \ \det \begin{pmatrix} u_x & v_x \\ u_y & v_y \end{pmatrix} \ = \ u_x v_y - u_y v_x.$$

Two multiplications and a subtraction, and everything we need lives in its *sign*.

Formal

Let $\vec{u} = (u_x, u_y)$ and $\vec{v} = (v_x, v_y)$. Define $\vec{u} \times \vec{v} = u_x v_y - u_y v_x$. Then:

- if $\vec{u} \times \vec{v} > 0$, then $\vec{v}$ is counterclockwise from $\vec{u}$ (a left turn);
- if $\vec{u} \times \vec{v} < 0$, then $\vec{v}$ is clockwise from $\vec{u}$ (a right turn);
- if $\vec{u} \times \vec{v} = 0$, then $\vec{u}$ and $\vec{v}$ are *parallel* (collinear through p_0).

The magnitude $|\vec{u} \times \vec{v}|$ equals the area of the parallelogram spanned by $\vec{u}$ and $\vec{v}$, so half of it is the area of the triangle $p_0 p_1 p_2$.

Why does the sign encode the turn? Geometrically, the 2×2 determinant $u_x v_y - u_y v_x$ computes the *signed area* of the parallelogram. If you curl the fingers of your right hand from $\vec{u}$ toward $\vec{v}$ through the smaller angle, your thumb points out of the page (positive z-axis) when the turn is counterclockwise, and into the page (negative z-axis) when clockwise. Sweep the arrow $\vec{u}$ counterclockwise toward $\vec{v}$ and the swept region has positive area; sweep it clockwise and the area is negative; if the two arrows point along the same line, the area is zero. The sign of the area is exactly the direction of rotation. The test needs no trigonometry, inverse tangents, or square roots, only multiplication and subtraction, which are exact on integers.

Focus

The cross product replaces "looking at the picture." Where your eye sees a left turn, the machine computes a positive number. Hold this equivalence in your head:

$$\text{sign of } (u_x v_y - u_y v_x) \quad \longleftrightarrow \quad \text{left / straight / right.}$$

Everything else in this chapter comes from asking this question, sometimes thousands of times.

32.1.1 The Orientation primitive

Package this as a reusable function. Given three points it returns one of three labels, encoded as $+1$ (counterclockwise / left), -1 (clockwise / right), and 0 (collinear).

Algorithm 137: ORIENTATION(p_0, p_1, p_2)

Input: Three points p_0, p_1, p_2 in the plane.
Output: $+1$ if $p_0p_1p_2$ make a counterclockwise (left) turn, -1 if clockwise (right), 0 if collinear.

```
c ← (p1x − p0x)(p2y − p0y) − (p1y − p0y)(p2x − p0x);
if c > 0 then
    return +1
if c < 0 then
    return −1
return 0;
```

Trace it on actual numbers. Take $p_0 = (0, 0)$, $p_1 = (4, 0)$, and three candidates for p_2.

Example 32.1. With $p_0 = (0, 0)$ and $p_1 = (4, 0)$ (so $\vec{u} = (4, 0)$, pointing east):

- $p_2 = (2, 3)$ (north of the line): $c = (4)(3) - (0)(2) = 12 > 0$, a **left turn**. Yes – to get from heading east to pointing at a spot above the line, you turn left. ✓
- $p_2 = (2, -3)$ (south of the line): $c = (4)(-3) - (0)(2) = -12 < 0$, a **right turn**. ✓
- $p_2 = (7, 0)$ (straight ahead): $c = (4)(0) - (0)(7) = 0$, **collinear**. ✓

Figure 32.1 illustrates both the determinant area geometry and the orientation turn test of Example 32.1.

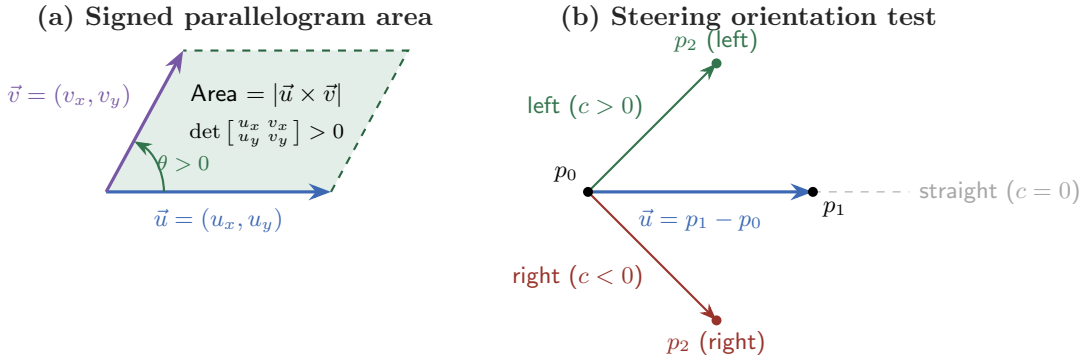


Figure 32.1: The 2D cross product as orientation test. **(a)** The cross product $\vec{u} \times \vec{v} = u_x v_y - u_y v_x$ computes the signed area of the parallelogram spanned by $\vec{u}$ and $\vec{v}$; counterclockwise sweep gives a positive determinant. **(b)** Walking from p_0 to p_1 and looking toward p_2: a positive sign indicates a left turn, negative a right turn, and zero collinearity.

Common trap

Two traps bite beginners here. First, **do not compute the slope** $(p_{1y} - p_{0y})/(p_{1x} - p_{0x})$ to compare directions. Division invites division-by-zero (vertical lines!) and floating-point noise. The cross product uses only $\times$ and $-$, so on *integer* inputs it is *exact* – no rounding, ever. Second, **order matters**: $\textsc{Orientation}(p_0, p_1, p_2)$ and $\textsc{Orientation}(p_0, p_2, p_1)$ have opposite signs. Swapping the last two arguments flips left and right, because you are now asking about the reverse turn.

Why this matters

This one sign computation shows up all over. A GPS deciding which side of a road you are on, a CAD program checking whether you clicked inside a polygon, a video game testing whether a bullet's path crosses a wall, a chip-design tool checking that wires do not overlap – all of them rest on the orientation test. It plays the role in geometry that the comparison "<" plays in sorting.

32.2 Do two segments intersect?

Suppose you have two line segments, $\overline{p_1p_2}$ and $\overline{p_3p_4}$. Do they cross? Not the infinite lines through them – the finite segments, with their endpoints. This is the test a game engine runs to see if your character walked through a wall.

The naive approach solves for the intersection point of the two lines with algebra, then checks whether that point falls inside both segments. It works, but it needs division, it stumbles on parallel and vertical lines, and floating-point error makes the "is the point inside?" check shaky. There is a cleaner route that uses *only orientation tests* – exact integer arithmetic, no division.

Intuition

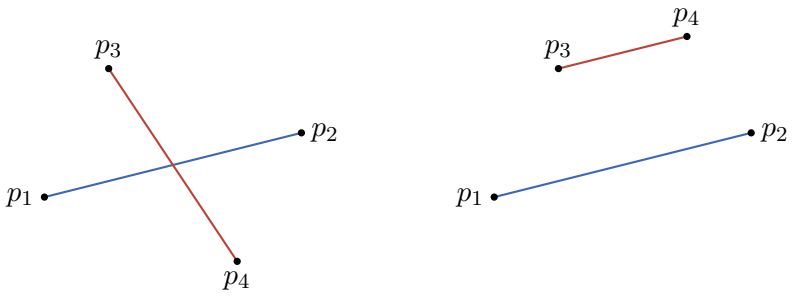
The key idea: segment $\overline{p_1p_2}$ crosses segment $\overline{p_3p_4}$ exactly when each segment "straddles" the line through the other. "Straddle" means the two endpoints sit on *opposite sides*. Look at p_3 and p_4: if one is to the left of the line p_1p_2 and the other is to the right, then $\overline{p_3p_4}$ pokes across that line. When *both* segments straddle each other's lines, they must cross; if either one fails to straddle, they miss.

"Which side of the line p_1p_2 is the point p_3 on?" is precisely $\textsc{Orientation}(p_1, p_2, p_3)$, so the whole test is four orientation calls. Define

$$d_1 = \textsc{Orientation}(p_3, p_4, p_1), \quad d_2 = \textsc{Orientation}(p_3, p_4, p_2),$$

$$d_3 = \textsc{Orientation}(p_1, p_2, p_3), \quad d_4 = \textsc{Orientation}(p_1, p_2, p_4).$$

The points p_1 and p_2 are on opposite sides of line p_3p_4 exactly when d_1 and d_2 have *opposite signs*; symmetrically for d_3, d_4 and the line p_1p_2.



they cross no crossing: p_3, p_4 same side

One subtlety remains: what if an endpoint lands *exactly* on the other segment? Then some $d_i = 0$, and we must check whether that collinear point actually lies *within* the bounds of the segment rather than out on its extension. A small helper, OnSegment, handles it: assuming p_i is known collinear with $\overline{p_j p_k}$, it checks that p_i's coordinates fall inside the bounding box of that segment.

Algorithm 138: SegmentsIntersect(p_1, p_2, p_3, p_4)

Input: Four points defining segments $\overline{p_1 p_2}$ and $\overline{p_3 p_4}$.
Output: TRUE if the segments intersect (including touching), else FALSE.

```
d1 ← Orientation(p3, p4, p1);
d2 ← Orientation(p3, p4, p2);
d3 ← Orientation(p1, p2, p3);
d4 ← Orientation(p1, p2, p4);
if d1·d2 < 0 and d3·d4 < 0 then
    return TRUE ;                                   // proper crossing
if d1 = 0 and OnSegment(p3, p4, p1) then
    return TRUE
if d2 = 0 and OnSegment(p3, p4, p2) then
    return TRUE
if d3 = 0 and OnSegment(p1, p2, p3) then
    return TRUE
if d4 = 0 and OnSegment(p1, p2, p4) then
    return TRUE
return FALSE;
```

Example 32.2 works the four orientation calls end to end. The products $d_1 d_2 < 0$ and $d_3 d_4 < 0$ are a compact way to say "the two signs differ." If they differ *and* neither is zero, the segments straddle properly and cross. The remaining four lines handle the boundary cases where an endpoint touches.

Example 32.2. Take $\overline{p_1 p_2}$ with $p_1 = (0, 0)$, $p_2 = (4, 1)$, and $\overline{p_3 p_4}$ with $p_3 = (1, 2)$, $p_4 = (3, -1)$ (the crossing picture above). Compute:

- $d_1 = \text{Orientation}(p_3, p_4, p_1)$: $\vec{u} = p_4 - p_3 = (2, -3)$, $\vec{v} = p_1 - p_3 = (-1, -2)$, $c = (2)(-2) - (-3)(-1) = -4 - 3 = -7 < 0$, so $d_1 = -1$.
- $d_2 = \text{Orientation}(p_3, p_4, p_2)$: $\vec{v} = p_2 - p_3 = (3, -1)$, $c = (2)(-1) - (-3)(3) = -2 + 9 = 7 > 0$, so $d_2 = +1$.
- $d_1 d_2 = -1 < 0$: p_1, p_2 straddle line $p_3 p_4$. ✓

$d_3 = \textsc{Orientation}(p_1, p_2, p_3)$: $\vec{u} = (4, 1)$, $\vec{v} = (1, 2)$, $c = 8 - 1 = 7 > 0$, so $d_3 = +1$. And d_4: $\vec{v} = p_4 - p_1 = (3, -1)$, $c = (4)(-1) - (1)(3) = -7 < 0$, so $d_4 = -1$. Then $d_3 d_4 = -1 < 0$. Both straddles hold, so the segments **intersect**. ✓

Cost summary

SegmentsIntersect runs in $\Theta(1)$ time: a fixed number of orientation tests, each $\Theta(1)$. On integer coordinates it is *exact* – it never rounds.

32.2.1 Testing any pair: the sweep-line algorithm

Now suppose you are given n segments and want to know whether *any* two of them intersect. Checking all $\binom{n}{2}$ pairs costs $\Theta(n^2)$. When $n = 100{,}000$, n^2 is ten billion comparisons. We can do much better using a *sweep-line* algorithm (Shamos–Hoey), which runs in $\Theta(n \log n)$.

Intuition

Imagine dragging a vertical line L across the plane from left to right. At any moment, L cuts through some subset of the segments. These active segments have a natural vertical order along L, from highest to lowest. The key geometric invariant is that two segments cannot cross unless they become *adjacent* in this vertical ordering. So instead of checking all pairs, we only ever test segments that are immediate vertical neighbors on the sweep line!

We maintain two data structures during the sweep:

1. **Event queue** Q: a priority queue holding the $2n$ endpoints, sorted by x-coordinate from left to right.

2. **Sweep-line status** T: a balanced binary search tree (such as a red-black tree) holding the segments currently intersecting the sweep line L, ordered by their y-coordinates at the current position of L.

When the sweep line reaches a *left endpoint* of segment s, we insert s into T and test s against its immediate neighbors directly above and below in T using SegmentsIntersect. When it reaches a *right endpoint*, we remove s from T and test whether its former above-neighbor and below-neighbor intersect each other. If any neighbor test returns TRUE, we stop and report an intersection.

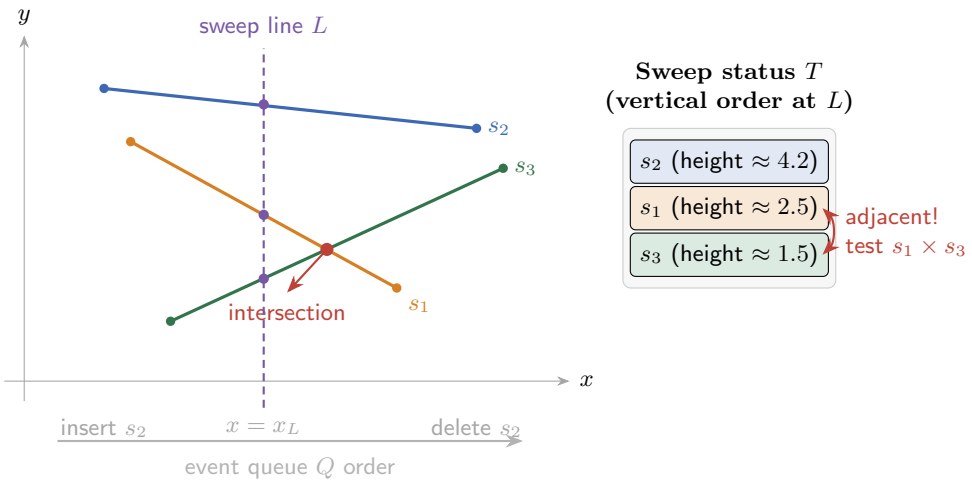


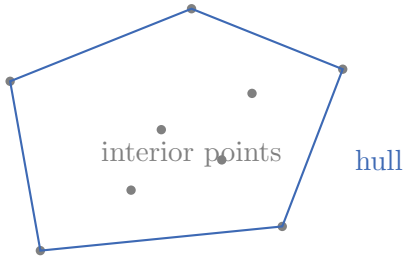
Figure 32.2: The sweep-line line segment intersection algorithm. The vertical line L advances left to right through event points stored in Q. Active segments intersecting L are ordered by height in a balanced search tree T. Segments s_1 and s_3 are vertical neighbors in T, triggering SegmentsIntersect(s_1, s_3) and detecting their crossing in $\Theta(n \log n)$ total time.

32.3 The convex hull

Given n points scattered in the plane, the *convex hull* is the smallest convex polygon that contains all of them.

Intuition

Hammer a nail into the table at each point. Stretch a rubber band wide open around the whole cluster and let go. It snaps tight and clings to the outermost nails, ignoring everything tucked inside. The taut rubber band is the convex hull. "No dents": between any two band-nails the band goes perfectly straight, never caving inward. Equivalently, the hull is convex – the segment between any two points of the polygon stays inside it.



The hull points are exactly the "extreme" ones; every other point is interior and irrelevant to the boundary. We will see two algorithms, both built entirely on the orientation test.

Why this matters

Convex hulls turn up wherever a "smallest enclosing shape" is useful: collision detection in physics engines and games (convex shapes are cheap to test), finding the diameter of a point set, path planning for robots (hug the hull to avoid obstacles), pattern recognition, and as a preprocessing step for dozens of other geometric algorithms. A phone that crops a photo to "the region with all the faces" can compute that region as the hull of the detected faces.

32.3.1 Graham's scan

Graham's scan sorts the points by angle around an anchor, then walks through them keeping a "stack" of hull vertices, popping any point that would create a right turn (a dent). It runs in $\Theta(n \log n)$, dominated by the sort.

Step 1 – pick the anchor. Choose p_0, the point with the lowest y-coordinate (break ties by lowest x). This point is *guaranteed* to be on the hull – nothing sits below it, so the rubber band must touch it.

Step 2 – sort by polar angle. Sort the remaining points by the angle the ray $p_0 \to p$ makes with the positive x-axis, increasing. No angles actually get computed. To decide whether p_a comes before p_b, ask Orientation(p_0, p_a, p_b); if it is $+1$ (counterclockwise), p_a has the smaller angle. The cross product sorts by angle with no trig function in sight. (Ties at equal angle: keep the farther point, or handle collinear points with a small rule.)

Step 3 – the scan. Now walk the sorted points, maintaining a stack of the hull-so-far. Push p_0 and the first two sorted points. Then for each subsequent point p_i: while the top two stack entries together with p_i make a *non-left* turn (right turn or collinear), *pop* the top – it was a dent, a mistake we now correct. Then push p_i. When you have consumed every point, the stack *is* the hull, in counterclockwise order.

Algorithm 139: GrahamScan(P)

Input: A set P of $n \geq 3$ points.
Output: The vertices of the convex hull of P in counterclockwise order.

```
p_0 ← point of P with lowest y (ties: lowest x);
sort the other points by polar angle around p_0, using Orientation as the comparator; call
  them p_1, ..., p_m;
S ← empty stack;
Push(S, p_0);;
Push(S, p_1);;
Push(S, p_2);
for i ← 3 to m do
    while |S| ≥ 2 and Orientation(NextToTop(S), Top(S), p_i) ≤ 0 do
        Pop(S) ;                          // the turn is not left:  pop the dent
    Push(S, p_i);
return S;
```

Algorithm lab

Run it on a small set. Anchor $p_0 = (0, 0)$, and after sorting by angle suppose the points are

$$p_1 = (3, 1),\ p_2 = (5, 3),\ p_3 = (2, 2),\ p_4 = (0, 4).$$

Start the stack as $[p_0, p_1, p_2]$ (bottom to top).

- **Consider $p_3 = (2, 2)$.** Check the turn $p_1 \to p_2 \to p_3$: $\vec{u} = p_2 - p_1 = (2, 2)$, $\vec{v} = p_3 - p_1 = (-1, 1)$, $c = (2)(1) - (2)(-1) = 4 > 0$, a *left* turn. Left is good – no pop. Push p_3. Stack: $[p_0, p_1, p_2, p_3]$. (At this moment p_3 looks like a hull vertex; the next point will reveal that it is not.)
- **Consider $p_4 = (0, 4)$.** Check $p_2 \to p_3 \to p_4$: $\vec{u} = p_3 - p_2 = (-3, -1)$, $\vec{v} = p_4 - p_2 = (-5, 1)$, $c = (-3)(1) - (-1)(-5) = -3 - 5 = -8 < 0$, a *right* turn. Right is a dent – *pop* p_3. Stack: $[p_0, p_1, p_2]$.
- Re-check $p_1 \to p_2 \to p_4$: $\vec{u} = p_2 - p_1 = (2, 2)$, $\vec{v} = p_4 - p_1 = (-3, 3)$, $c = (2)(3) - (2)(-3) = 6 + 6 = 12 > 0$, a left turn. Good – push p_4. Stack: $[p_0, p_1, p_2, p_4]$.

The hull is $p_0, p_1, p_2, p_4 = (0, 0), (3, 1), (5, 3), (0, 4)$, and $p_3 = (2, 2)$ was correctly discarded as interior. Figure 32.3 traces this sequence step by step, showing how the non-left turn at p_3 is detected and the dent is popped from the stack.

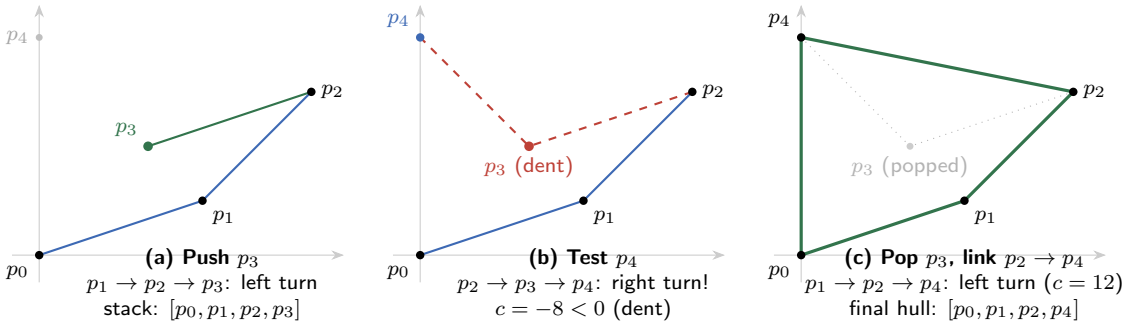


Figure 32.3: Graham's scan popping an interior dent. **(a)** Point p_3 is initially pushed onto the stack after a valid left turn. **(b)** Arriving at p_4, the triple $p_2 \to p_3 \to p_4$ creates a clockwise right turn ($c = -8 < 0$), revealing p_3 as a dent. **(c)** Popping p_3 restores the left turn $p_1 \to p_2 \to p_4$ ($c = 12 > 0$), yielding the convex boundary.

Why does popping on a non-left turn give the right answer? The argument runs through a loop invariant.

Focus

Loop invariant (Graham's scan). At the top of each `for` iteration, the stack S holds, from bottom to top, the vertices of the convex hull of the points considered so far, in counterclockwise order, with every consecutive triple making a left turn.

Initialization. Three non-collinear points form a triangle, which is its own convex hull. *Maintenance.* When we add p_i, the `while` loop pops every vertex that would create a right or straight turn – exactly the vertices that p_i now "hides" from the boundary, the dents. After popping, the top triple turns

left, so the invariant is restored when we push p_i. The points sit in angle order, so p_i is the next counterclockwise vertex. *Termination.* After the last point, S is the hull of all of P. Every step of the induction uses only the orientation sign.

Cost summary

Graham's scan. The sort costs $\Theta(n \log n)$. The scan looks like it could be $\Theta(n^2)$ because of the inner `while`, but each point is *pushed once and popped at most once* over the whole run – an amortized argument (Chapter 17). So the scan itself is $\Theta(n)$, and the total is

$$T(n) = \underbrace{\Theta(n \log n)}_{\text{sort}} + \underbrace{\Theta(n)}_{\text{scan}} = \Theta(n \log n).$$

The sort dominates. This is asymptotically optimal for comparison-based hull algorithms.

Common trap

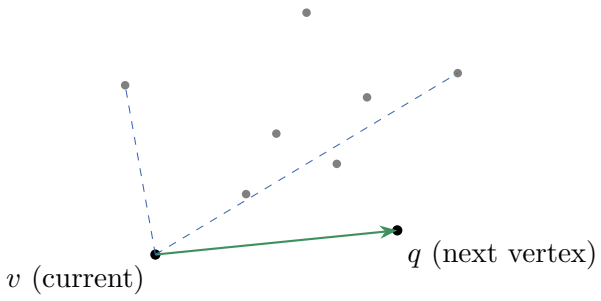
Two classic mistakes. (1) Forgetting to pop in a `while` loop – one pop is not enough; adding a far-out point can invalidate *several* previous vertices, so you must keep popping until the turn is left. (2) Mishandling collinear points. With ≤ 0 in the test, collinear points ($c = 0$) get popped, which usually you want (a vertex in the middle of a straight edge is not a true corner). If your application needs them *kept*, change the test to < 0. Be deliberate about which you mean.

32.3.2 Jarvis's march (gift wrapping)

A second algorithm is slower in the worst case but very intuitive, and faster when the hull has few vertices. It is called *Jarvis's march*, or *gift wrapping*, and it does exactly what your hand does when you wrap a present: it walks around the outside, always turning as little as possible.

Intuition

Start at the lowest point – definitely on the hull. Now imagine sweeping a ray counterclockwise until it bumps into the very first point. That point is the next hull vertex; the ray "wrapped" onto it. Stand there and sweep again, and again, each time pivoting to the next point, until you come all the way back to the start. You have wrapped a string around the outside of the point cloud.



The dashed rays in the figure show v "seeing" the rest of the cloud; the solid green ray $v \to q$ is the most clockwise of all of them, so q is the next hull vertex.

How do you find "the next point" without sweeping a physical ray? With the orientation test, again. From the current hull vertex v, you want the point q such that *every other point* lies to the right of the ray $v \to q$ – equivalently, q is the most clockwise point as seen from v. A linear scan finds it: start with any candidate and replace it whenever $\textsc{Orientation}(v, \text{candidate}, r) = -1$ (point r is even more clockwise).

Algorithm 140: JarvisMarch(P)

Input: A set P of $n \geq 3$ points.
Output: The convex hull of P in counterclockwise order.

```
start ← leftmost point of P (ties: lowest y) ;          // guaranteed on the hull
v ← start;
H ← empty list;
repeat
    append v to H;
    q ← any point of P with q ≠ v;
    for each point r ∈ P do
        if Orientation(v, q, r) = −1 then
            q ← r ;                          // r is more clockwise; better next vertex
    v ← q;
until v = start;
return H;
```

Cost summary

Jarvis's march. Each pass around the inner `for` loop is $\Theta(n)$ and produces exactly one hull vertex. If the hull has h vertices, the total cost is $\Theta(nh)$ – this is an *output-sensitive* bound. If only a handful of points are on the hull (h small, say a constant), Jarvis runs in $\Theta(n)$, beating Graham. But in the worst case *all* n points lie on the hull (think of points on a circle), so $h = n$ and Jarvis degrades to $\Theta(n^2)$. Graham's $\Theta(n \log n)$ is the safer default; Jarvis shines when you know the hull is small.

Walking question

Take a coin out of your pocket and put points only on its rim – a perfect circle. How many points are on the convex hull? *All of them.* Now you understand why Jarvis's march hits its $\Theta(n^2)$ worst case here: $h = n$. Next, scatter n points uniformly inside a square. Roughly how many end up on the hull? (The answer grows like $\log n$, which is why Jarvis is often surprisingly fast in practice.)

32.4 The closest pair of points

The last problem: given n points, find the two whose Euclidean distance is smallest. A radar system might run this to spot the two aircraft in greatest danger of colliding.

Brute force is obvious: compute all $\binom{n}{2}$ pairwise distances and keep the minimum, at a cost of $\Theta(n^2)$. Can we do better? Distances feel harder than the comparisons that sorting needed, yet divide and conquer (Chapter 4) gets us to $\Theta(n \log n)$.

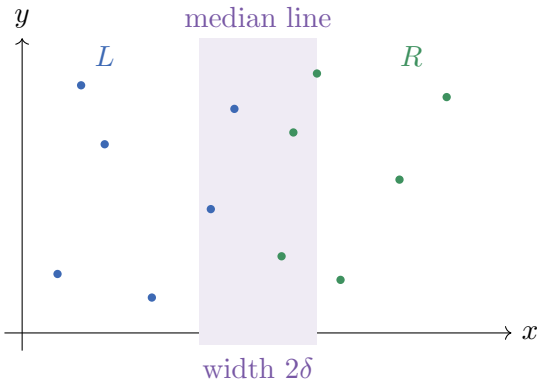
Intuition

Split the points into a left half and a right half with a vertical line. Find the closest pair on the left (recursively) and the closest pair on the right (recursively). Let δ be the smaller of those two distances. The true closest pair is either entirely on the left, entirely on the right, *or* it straddles the line – one point on each side. The first two cases are already covered by δ. All the difficulty is in the third case: checking straddling pairs *fast*.

32.4.1 The setup and the strip

First, presort: make a copy of the points sorted by x (call it X) and another sorted by y (call it Y). This happens *once*, up front, in $\Theta(n \log n)$, and the sorted orders stay consistent through the recursion. Inside the recursion, draw a vertical line at the median x, splitting the points into left half L and right half R of size $n/2$ each. Recurse to get δ_L and δ_R, and set $\delta = \min(\delta_L, \delta_R)$.

Now the straddling pairs. A straddling pair closer than δ must have both points within horizontal distance δ of the dividing line; otherwise they are already more than δ apart in x alone. So only the points in a vertical *strip* of width 2δ centered on the line can matter.



The difficulty is that the strip could contain *all* n points, if they all cluster near the line. Comparing every pair inside the strip would cost $\Theta(n^2)$ again and lose the speedup, so the strip step needs a cheaper method.

32.4.2 Why seven neighbors suffice

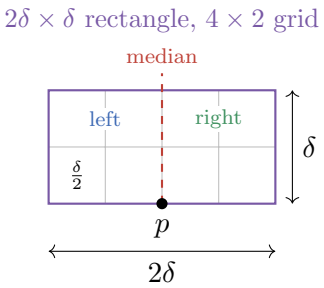
The following fact makes the strip step cheap.

Focus

The key geometric fact. Sort the strip points by their y-coordinate. Then for any point in the strip, you only need to compare it against the next 7 points in that y-order. Any straddling pair closer than δ must appear within 7 positions of each other in the y-sorted

strip.

Why 7? Picture the strip as a 2δ-wide vertical band. Take any point p in it. Any point that could be within δ of p must lie in a $2\delta \times \delta$ rectangle just above p (within δ in y, and at most 2δ across in x, with the median line running down its middle). Now split that rectangle into a grid of squares of side $\delta/2$: it fits exactly $4 \times 2 = 8$ such squares, four entirely on the left side of the line and four entirely on the right. The key observation is that *each little square can hold at most one point.* The two recursive calls already guarantee that any two points on the *same* side are at least δ apart, each square sits wholly on one side, and the diagonal of a $\delta/2$-square is only $\delta/\sqrt{2} < \delta$ – two points in one square would be closer than δ, a contradiction. So at most 8 points sit in the rectangle, one of which is p itself, leaving at most 7 others to check.



Formal

Claim. Let p and q be points in the strip with q above p in y-order and $\text{dist}(p, q) < \delta$. Then q is among the next 7 points after p in the y-sorted strip.

Proof sketch. Such a q lies in the $2\delta \times \delta$ rectangle above p, which straddles the median line: its left half is a $\delta \times \delta$ square sitting on the left side, its right half a $\delta \times \delta$ square on the right side. Tile each half with four $\frac{\delta}{2} \times \frac{\delta}{2}$ squares, eight in all. Each little square has diameter $\frac{\delta}{\sqrt{2}} < \delta$, and every square lies entirely on one side of the line, so any two points inside the same square are on the same side and would be closer than δ – contradicting that the per-side minimum is exactly δ. Hence each square holds at most one input point, so at most 8 points occupy the rectangle, p included, leaving at most 7 candidates after p. □

A constant number of comparisons per point, at most 7, converts the strip step from $\Theta(n^2)$ into $\Theta(n)$. That bound is what makes the whole algorithm work.

32.4.3 The algorithm and its recurrence

Algorithm 141: ClosestPair(X, Y)

Input: Points sorted by x in array X and by y in array Y (same set).
Output: The smallest distance between any two of the points.

```
if |X| ≤ 3 then
    return brute-force minimum distance
split X at the median x into left half X_L and right half X_R;
let ℓ be the median x-coordinate;
partition Y into Y_L, Y_R preserving y-order (one linear pass);
δ_L ← ClosestPair(X_L, Y_L);
δ_R ← ClosestPair(X_R, Y_R);
δ ← min(δ_L, δ_R);
build S ← points of Y with |x − ℓ| < δ ;          // strip, still y-sorted
for i ← 1 to |S| do
    for j ← i + 1 to min(i + 7, |S|) do
        δ ← min(δ, dist(S[i], S[j]));
return δ;
```

The one subtlety that makes this $\Theta(n \log n)$ rather than $\Theta(n \log^2 n)$: we must *not* re-sort by y inside each recursive call. Instead, we pass the already-y-sorted array Y down and split it into Y_L, Y_R in a single linear pass each level (just like the merge step of merge sort, run in reverse). With that care, each level of recursion does $\Theta(n)$ non-recursive work.

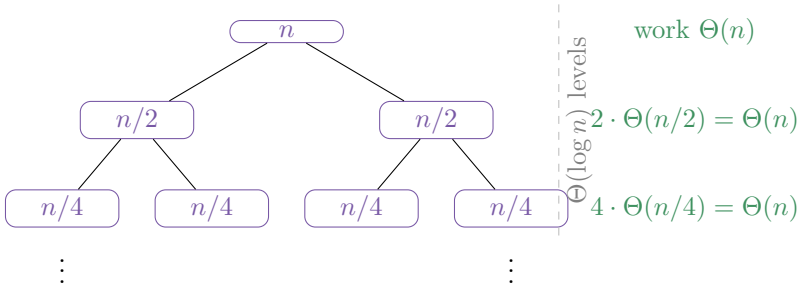
Cost summary

The recurrence. One presort costs $\Theta(n \log n)$. Inside the recursion, each call splits ($\Theta(n)$), recurses on two halves, and does the strip scan with its at-most-7 comparisons ($\Theta(n)$). So

$$T(n) = 2\,T(n/2) + \Theta(n).$$

By the master theorem (Chapter 4), case 2, this solves to $T(n) = \Theta(n \log n)$. Add the one-time presort – also $\Theta(n \log n)$ – and the total stays $\Theta(n \log n)$. We beat the $\Theta(n^2)$ brute force, and we did it with the *same* recurrence as merge sort.

The recursion tree makes the $\Theta(n \log n)$ visible: every level does $\Theta(n)$ total work (the split plus the at-most-7 strip scan), and there are $\Theta(\log n)$ levels.



Summing $\Theta(n)$ across $\Theta(\log n)$ levels gives $\Theta(n \log n)$ – precisely the shape of the merge-sort tree.

Common trap

The number-one bug in closest-pair implementations is re-sorting the strip (or the whole set) by y *inside* every recursive call. That sneaks an extra $\log n$ factor in and gives $\Theta(n \log^2 n)$ – still better than n^2, but not optimal, and on an exam it loses points. The fix is to thread the y-sorted order through the recursion by splitting, never re-sorting. A second subtle point: use the strip's δ *from the recursive calls*, and keep shrinking it as you find closer straddling pairs – but always loop the at-most-7 window, not a window that depends on the shrinking δ.

Walking question

Convince yourself the constant really is bounded. Take the $2\delta \times \delta$ rectangle and try to cram in more than 8 points such that no two on the same side are closer than δ. You will fail – the squares simply run out of room. This "packing" style of argument, where geometry forces a constant, recurs all over computational geometry. Where else have you seen "the space can only hold so many"? (Hint: hashing, and the pigeonhole principle.)

32.5 Looking back

Every algorithm in this chapter rests on the sign of one cross product.

Focus

What to carry forward.

- The *cross product* $u_x v_y - u_y v_x$ turns "which way did we turn?" into the sign of one number – exact on integers, no trig, no division. It is the "<" of geometry.
- *Segment intersection* is four orientation tests: each segment must straddle the other's line.
- *Convex hull*: Graham's scan sorts then walks, popping right turns, in $\Theta(n \log n)$. Jarvis's march gift-wraps in $\Theta(nh)$ – great when the hull is small, quadratic when it is not.
- *Closest pair*: divide by a vertical line, recurse, and patch the middle strip using the seven-neighbors fact, for $\Theta(n \log n)$.
- The general method: translate the picture into signs and packing arguments, and the algorithms follow almost directly.

32.5.1 From planar primitives to spatial computing

Exact geometric predicates like these are used throughout industry:

- **Computer-Aided Design and Manufacturing (CAD/CAM).** Modern aerospace, automotive, and architectural engineering rests on solid modeling kernels that rely on exact orientation tests to compute boolean operations (unions and intersections) of mechanical parts.

- **Geographic Information Systems (GIS).** Mapping engines and satellite analytics overlay millions of boundary polygons, route vehicles around terrain obstacles, and index spatial features using the geometric predicates developed here.
- **VLSI physical design.** Chip design-rule checking runs sweep-line segment intersection over billions of overlapping transistor traces to guarantee that electrical circuits never short-circuit.
- **Robotics and autonomous vehicles.** Real-time collision detection and motion planning model obstacles as convex polyhedra, continuously running orientation and closest-pair queries to steer robotic arms and self-driving vehicles safely through physical environments.

32.5.2 Higher dimensions and geometric boundaries

The planar algorithms do not carry over cleanly to higher dimensions or to non-Euclidean spaces:

- **The curse of dimensionality.** In the plane, the convex hull of n points has at most n vertices and n edges. But by McMullen's Upper Bound Theorem (1970), the convex hull of n points in d dimensions can have $\Theta(n^{\lfloor d/2 \rfloor})$ faces: $\Theta(n^2)$ faces in $d = 4$ and $\Theta(n^5)$ in $d = 10$. The simple angular sort of Graham's scan fails, and computing high-dimensional hulls becomes combinatorially explosive.
- **Exactness overhead.** Running exact arithmetic predicates or adaptive floating-point filter cascades prevents topological inconsistencies and crashes at the expense of constant-factor runtime compared to unchecked hardware arithmetic.
- **Non-Euclidean geometries.** Planar cross products assume flat Euclidean space $\mathbb{R}^2$. On spherical surfaces (such as planetary flight paths) or hyperbolic spaces, straight lines bend into geodesics, requiring non-Euclidean metric adaptations.

Reading note

For the rigorous versions of all four algorithms, read **CLRS Chapter 33** next: 33.1 (line-segment properties), 33.2 (the sweep-line "any segments intersect"), 33.3 (Graham's scan and Jarvis's march), and 33.4 (closest pair). CLRS proves the seven-neighbors bound carefully and handles every degenerate case (collinear points, coincident points, vertical lines) that we have waved past. With the pictures from this chapter in mind, those proofs should be easier to follow.

Planar convex hulls and closest pairs take only $\Theta(n \log n)$ time, but some similar-looking spatial problems—such as finding the shortest tour through n points (the Euclidean Traveling Salesperson Problem) or connecting points with minimal wire length via Steiner junctions—have no known polynomial-time algorithm. Chapter 33 explains why, introducing NP-completeness, polynomial reductions, and the P versus NP question.

Exercises

Exercise 32.1. Compute ORIENTATION(p_0, p_1, p_2) by hand for $p_0 = (1, 1)$, $p_1 = (4, 3)$, $p_2 = (2, 5)$. Is the turn left, right, or straight? Then swap p_1 and p_2 and recompute; explain why the sign flipped.

Exercise 32.2. The magnitude of the cross product is twice the area of triangle $p_0p_1p_2$. Use this to find the area of the triangle with vertices $(0,0)$, $(5,0)$, $(2,4)$. Check against the schoolbook formula $\frac{1}{2} \cdot \text{base} \cdot \text{height}$.

Exercise 32.3. Do segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ intersect for $p_1 = (0,0), p_2 = (4,4), p_3 = (0,4), p_4 = (2,2)$? Notice p_4 lies *on* the first segment. Walk through SegmentsIntersect and identify which branch fires.

Exercise 32.4. Explain in two or three sentences why the orientation test is preferred over computing and comparing slopes. Give a concrete pair of segments where the slope approach would divide by zero but the cross product handles it cleanly.

Exercise 32.5. Run Graham's scan by hand on the points $(0,0), (2,0), (4,1), (2,2), (1,1), (3,1)$. Identify the anchor, the angular order, and which points get popped. Draw the final hull.

Exercise 32.6. For a set of n points in "convex position" (all on the hull, e.g. on a circle), exactly how many times does Jarvis's march execute its inner `for` loop in total? Express your answer in terms of n and confirm it gives $\Theta(n^2)$.

Exercise 32.7. In Graham's scan we pop on a non-left turn using the test $\textsc{Orientation}(\cdot, \cdot, p_i) \leq 0$. Suppose three consecutive hull candidates are exactly collinear. With ≤ 0, is the middle one kept or popped? What changes if you use < 0 instead? Describe an application that would want each behavior.

Exercise 32.8. In the closest-pair strip step we compare each point with the next 7 in y-order. Redo the packing argument with $\delta/2$-squares but suppose someone claims you only need to check the next 5 points. Construct (or argue the impossibility of) a configuration that would break the 5-bound, and conclude why 7 is the safe constant.

Exercise 32.9. The closest-pair recurrence is $T(n) = 2T(n/2) + \Theta(n)$. Solve it from scratch with a recursion tree: how many levels, how much work per level, and what is the total? Then explain precisely where an accidental re-sort would change the $\Theta(n)$ term and what the new solution would be.

Exercise 32.10. *(Challenge.)* The cross product of two 3D vectors is itself a vector, not a scalar. Look up the formula and explain how, in 3D, you would test whether four points are coplanar (the analogue of three points being collinear). Which arithmetic operation plays the role of our 2D sign test? *(Hint: the scalar triple product, a 3×3 determinant.)*

Chapter 33

NP-Completeness

In Chapter 32, we solved geometric problems such as planar convex hulls and closest pairs in $\Theta(n \log n)$ time. Some similar-looking spatial problems, such as finding the shortest tour visiting n cities, have no known polynomial-time algorithm.

Almost every chapter of this book has been a success story. You wanted to sort a list, and we found a way to do it in $\Theta(n \log n)$. You wanted the shortest path through a city, and Dijkstra delivered it in near-linear time. You wanted to match jobs to machines, route flow through a network, multiply two huge integers quickly – and each time, after some cleverness, we walked away with an algorithm whose running time grew like a polynomial in the input size: n, or n^2, or n^3. Polynomial bounds are our working standard of tractability: double the input and an n^2 algorithm slows down by a factor of four.

This chapter studies the other side of that boundary: problems for which no polynomial-time algorithm is known, yet which all reduce to one another in polynomial time. We formalize the boundary between polynomial-time solvability (P) and polynomial-time verifiability (NP), build reductions that translate problem instances without loss of truth, and follow the chain of NP-completeness proofs that starts from Cook's satisfiability theorem.

History and connection

The roots of the P versus NP question reach back to a 1956 letter from Kurt Gödel to John von Neumann, who was dying of cancer in hospital. Gödel asked whether finding a mathematical proof could be mechanized in polynomial time: specifically, whether deciding if a formula of length n has a proof of length k could be computed by a Turing machine in k or k^2 steps. If such an algorithm existed, Gödel observed, "the mental work of a mathematician could be completely replaced by an automaton." Von Neumann died without replying, and Gödel's letter lay forgotten in archives for three decades.

In the late 1960s, computing pioneer Jack Edmonds proposed that polynomial-time algorithms ($O(n^c)$) characterize "good" (tractable) algorithms, contrasting them with exponential brute-force searches. In 1971, Stephen Cook at the University of Toronto published *The Complexity of Theorem-Proving Procedures*, formally defining the classes P and NP and proving that Boolean Satisfiability (SAT) is universal: every problem in NP polynomial-time reduces to SAT. Working independently in Moscow under Andrei Kolmogorov, Leonid Levin discovered the exact same phenomenon, delayed by Soviet academic bureaucracy until 1973. This

foundation is the *Cook–Levin Theorem*.
In 1972, Richard Karp at UC Berkeley proved that 21 classic combinatorial problems across graph theory, scheduling, and number theory are all NP-complete. This material maps directly to **CLRS Chapter 34**: formal definitions of P, NP, and verification are in §34.1–34.2, Cook's theorem is in §34.3, and the classic reductions developed in this chapter mirror §34.4–34.5. In 2000, the Clay Mathematics Institute designated P versus NP a Millennium Prize Problem with a one-million-dollar bounty. Decades of attempts to prove P $\neq$ NP have encountered three structural barriers: relativization (Baker, Gill, Solovay, 1975), natural proofs (Razborov and Rudich, 1994), and algebrization (Aaronson and Wigderson, 2008), showing that standard circuit and diagonalization methods cannot separate the classes. Whether P = NP remains the foremost open question in theoretical computer science.

33.1 Decision problems: shrinking the question

Before we can talk about "hard," we have to agree on what kind of question we are asking. Most of the problems in this book have been *optimization* problems: find the *shortest* path, the *maximum* flow, the *largest* matching. But the theory of NP-completeness is cleanest when we ask *yes/no* questions, called *decision problems*.

The restriction costs us little. Any optimization problem has a decision twin that is essentially just as hard. Instead of asking "what is the shortest path from s to t?" we ask "is there a path from s to t of length *at most* k?" and we get to pick k. If you could answer the yes/no version quickly for every k, you could binary-search on k to find the true optimum quickly too. So the decision version is no easier than the optimization version – and if we can show the *decision* version is hard, the optimization version is at least as hard.

Intuition

A decision problem is a question with only two possible answers: YES or NO. "Is this number prime?" "Does this graph have a cycle visiting every vertex?" "Can these numbers be split into two equal-sum piles?" We reduce the output to a single bit because that bit already carries the difficulty: finding the actual path, pile, or cycle is rarely harder than knowing whether one exists.

Formally, we encode every input as a finite string of symbols (think: the bytes in a file). A decision problem is then just a set L of strings – the strings that should answer YES. We call such a set a *language*, and we say an algorithm *decides* L if, given any string x, it halts and outputs YES exactly when $x \in L$ and NO otherwise. The word "language" is borrowed from formal logic; do not overthink it. A language is a yes/no problem.

Focus

Three things to hold onto. (1) We study *decision* problems – yes/no – but this costs us nothing, because every optimization problem has a decision twin of the same difficulty. (2) Inputs are *strings*, and the input *size* n is the length of that string. (3) "Efficient" will always mean "polynomial in n," i.e. running time $O(n^c)$ for some constant c.

A subtle point we will keep returning to: the input size n is the number of *symbols* it takes to write the input down, not the magnitude of the numbers in it. The integer one billion is a big *number* but a tiny *input* – about 30 bits. This distinction looks pedantic now but will matter a great deal when we reach subset-sum.

33.2 Computability Foundations: Turing Machines and the Undecidability of the Halting Problem

The central question of this chapter is one of efficiency: is every problem whose solution can be checked quickly (NP) also solvable quickly (P)?

Before we ask whether a problem can be solved *fast*, we have to ask an older question: can it be solved *at all* by any finite algorithm in any amount of time?

In 1900, David Hilbert presented his program to place all of mathematics on a secure formal foundation. Among his challenges was the *Entscheidungsproblem* (the "decision problem"): does there exist an effective mechanical procedure that, given any mathematical assertion in first-order logic, can decide in a finite sequence of deterministic steps whether that assertion is universally valid?

Hilbert took for granted that such an algorithm must exist; mathematicians had only to find it. But to prove that an algorithm *cannot* exist for a task, one must first define precisely what an algorithm is, so that a statement about all possible algorithms can be proved.

33.2.1 Alan Turing's 1936 model: the pencil-and-paper clerk

In 1936, a 24-year-old Cambridge mathematician named Alan Turing published *On Computable Numbers, with an Application to the Entscheidungsproblem*. Rather than starting with abstract symbolic logic, Turing began with a concrete physical thought experiment: what does a human clerk actually do when performing a calculation with pencil and paper?

A human calculator sits at a desk with a squared notebook. At any moment:

1. The clerk focuses attention on a single square of the paper.
2. The clerk alters the mark in that square by writing or erasing a symbol from a finite alphabet of symbols.
3. The clerk moves attention to an adjacent square, left or right.
4. The clerk's internal "state of mind" changes based on what was just read and the current instructions being followed.
5. The number of distinct states of mind, like the alphabet of symbols, is strictly finite.

Turing abstracted this clerk into a mechanical device known ever since as the **Turing Machine** (Figure 33.1):

- **An infinite tape:** A one-dimensional paper tape divided into discrete squares (cells) extending indefinitely in both directions. Each cell can hold a single symbol from a finite alphabet Γ, containing a special blank symbol $\sqcup$.

- **A read/write head:** A physical mechanism positioned over exactly one tape cell at any given moment, capable of reading the current symbol, overwriting it with a new symbol, and shifting one cell left (L) or right (R).
- **A finite state control:** A controller that resides in one of a finite collection of internal states $Q = \{q_0, q_1, \ldots, q_{\text{accept}}, q_{\text{reject}}\}$, starting at initial state q_0.
- **A transition function:** A deterministic lookup table

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\},$$

  which dictates the machine's next action: given the current state $q \in Q$ and the symbol $s \in \Gamma$ currently beneath the head, $\delta(q, s) = (q', s', D)$ specifies the new state q', the replacement symbol s' to write into the cell, and the direction $D \in \{L, R\}$ to move the head.

If the machine reaches q_{accept}, it halts and outputs YES. If it reaches q_{reject}, it halts and outputs NO. There is also a third possibility: the head may shuttle back and forth along the infinite tape forever, never entering an accepting or rejecting state. In that case, the machine *never halts*.

Focus

The Church–Turing Thesis. Any algorithmic procedure that can be carried out by any physical computer, modern supercomputer, human clerk, or futuristic quantum architecture can be simulated by a standard Turing machine.
The Church–Turing thesis is not a mathematical theorem that can be formally proven, because "intuitive mechanical procedure" is an informal concept. Rather, it is a working hypothesis backed by strong evidence: decades of independent formulations—Alonzo Church's λ-calculus, Emil Post's production systems, Markov algorithms, and modern random-access machines (RAM)—all proved to compute precisely the same class of functions. If the thesis holds, Turing machines mark the limit of what any algorithm can compute.

33.2.2 Code as data: the Universal Turing Machine

Turing's second idea underlies the design of modern stored-program computers.

Because the state set Q, the alphabet Γ, and the transition table δ of any specific Turing machine M are finite, the entire blueprint of machine M can be written down as a finite string of text. We can encode that blueprint into a standard binary representation, denoted $\langle M \rangle$.

Every algorithm can be encoded directly as a finite string of bits.

This led Turing to construct a **Universal Turing Machine** U. The machine U is a single, fixed Turing machine whose tape is initialized with two inputs: the encoded description $\langle M \rangle$ of some target machine, and an input string x. Machine U reads the encoded transition table of M and faithfully simulates the execution of M on x, step by mechanical step. If M halts on x and accepts, U accepts; if M halts and rejects, U rejects; if M loops forever on x, U loops forever.

Every software compiler, Python interpreter, and hardware CPU you have ever used is an engineering realization of Turing's Universal Turing Machine: a fixed piece of machinery that reads arbitrary software programs as data and executes them.

33.2.3 The Halting Problem and Turing's diagonalization proof

Once we realize that programs can take other programs as inputs, a natural question arises: can a program inspect another program and decide what it will do?

Every programmer has watched their terminal freeze on an unintentional infinite loop:

```
while n > 1:
  if n % 2 == 0:
    n = n // 2
  else:
    n = 3 * n + 1
```

Does this simple loop terminate for every positive starting integer n? This is the Collatz conjecture, unsolved for nearly a century. If we had an automated tool that could examine the source code of any program and determine whether it halts, we could settle open mathematical conjectures simply by writing five lines of code and asking the analyzer whether the loop halts.

We formulate this as a decision problem, the **Halting Problem**:

$$\textsc{Halt} = \{(\langle P \rangle, x) : \text{program } P \text{ halts when run on input } x\}.$$

Can there exist an algorithm $\textsc{Halts}(\langle P \rangle, x)$ that takes any program description $\langle P \rangle$ and input x, always terminates in a finite number of steps, and correctly returns $\textsc{True}$ if $P(x)$ halts and $\textsc{False}$ if $P(x)$ loops forever?

Turing proved that no such algorithm can exist.

Theorem 33.1 (Undecidability of the Halting Problem, Turing 1936)**.** *The Halting Problem* $\textsc{Halt}$ *is undecidable: there is no algorithm that decides whether an arbitrary program P halts on arbitrary input x.*

Proof. We proceed by contradiction. Suppose there exists an algorithm $\textsc{Halts}(\langle P \rangle, x)$ that decides $\textsc{Halt}$. By definition, $\textsc{Halts}$ always terminates in finite time and satisfies:

$$\textsc{Halts}(\langle P \rangle, x) = \begin{cases} \textsc{True} & \text{if } P(x) \text{ halts,} \\ \textsc{False} & \text{if } P(x) \text{ loops forever.} \end{cases}$$

Because $\textsc{Halts}$ is a valid algorithm, we can use it as a subroutine to construct a new program, which we call the contrarian machine D. The program D takes a single program description $\langle P \rangle$ as input and executes the following instructions:

```
def D(P_code):
  if Halts(P_code, P_code) == True:
    while True:
      pass  # loop forever
  else:
    return True  # halt immediately
```

Examine the behavior of D. When given the code $\langle P \rangle$, D asks $\textsc{Halts}$: "Does program P halt when executed on its own source code as input?"

- If $\textsc{Halts}(\langle P \rangle, \langle P \rangle)$ reports $\textsc{True}$ (P halts on its own code), D deliberately enters an infinite `while True` loop and never terminates.

- If HALTS($\langle P \rangle, \langle P \rangle$) reports FALSE ($P$ loops forever on its own code), D halts immediately and returns TRUE.

D is a finite, unambiguous piece of software. Therefore, D itself can be compiled into a valid program description $\langle D \rangle$.

Now feed machine D its own source code as input, and evaluate $D(\langle D \rangle)$. Does $D(\langle D \rangle)$ halt?

There are only two possible outcomes, and both lead to a contradiction:

1. **Suppose $D(\langle D \rangle)$ halts.** By the definition of HALTS, the call HALTS($\langle D \rangle, \langle D \rangle$) must evaluate to TRUE. But looking at the source code of D, when HALTS returns TRUE, D enters the `while True` branch and runs forever without halting. Thus, if $D(\langle D \rangle)$ halts, it does not halt.
2. **Suppose $D(\langle D \rangle)$ loops forever.** By the definition of HALTS, the call HALTS($\langle D \rangle, \langle D \rangle$) must evaluate to FALSE. But looking at the source code of D, when HALTS returns FALSE, D takes the `else` branch and halts immediately. Thus, if $D(\langle D \rangle)$ does not halt, it halts.

In both cases, we arrive at the logical impossibility:

$$D(\langle D \rangle) \text{ halts} \iff D(\langle D \rangle) \text{ does not halt.}$$

The only assumption made throughout this entire chain of deduction was that the halting decider HALTS existed. That assumption must be false. No algorithm can decide the Halting Problem. □
□

33.2.4 Cantor's diagonal and Rice's Theorem

The self-reference in $D(\langle D \rangle)$ is a computational version of Georg Cantor's 1891 diagonalization argument, illustrated in Figure 33.1(a).

Imagine constructing an infinite matrix whose rows index all possible programs $P_1, P_2, P_3, \ldots$ (ordered lexicographically by their binary encoding) and whose columns index all possible inputs $\langle P_1 \rangle, \langle P_2 \rangle, \langle P_3 \rangle, \ldots$. Cell (i, j) in this matrix records whether program P_i halts on input $\langle P_j \rangle$, taking value H for Halts or L for Loops.

The diagonal of this matrix consists of the self-evaluations:

$$(P_1, \langle P_1 \rangle), \quad (P_2, \langle P_2 \rangle), \quad (P_3, \langle P_3 \rangle), \quad \ldots$$

The contrarian machine D is designed to invert the diagonal: for every row i, $D(\langle P_i \rangle)$ does the opposite of cell (i, i). Because D's behavior differs from program P_i on the specific input $\langle P_i \rangle$, D differs from every row in the entire infinite table by at least one entry. Yet if HALTS existed, D would be a valid computable program, so its code $\langle D \rangle$ must appear as some row k in the table. Cell (k, k) would then have to be both H and L simultaneously, an impossible contradiction.

One might hope that the Halting Problem is a quirky edge case. Perhaps we cannot decide whether a program halts, but we can decide other properties?

In 1953, Gordon Rice showed that this hope fails for every non-trivial property of a program's behavior.

Theorem 33.2 (Rice's Theorem, 1953)**.** *Let P be any non-trivial semantic property of programs. Then the problem of deciding whether an arbitrary program computes a language with property P is undecidable.*

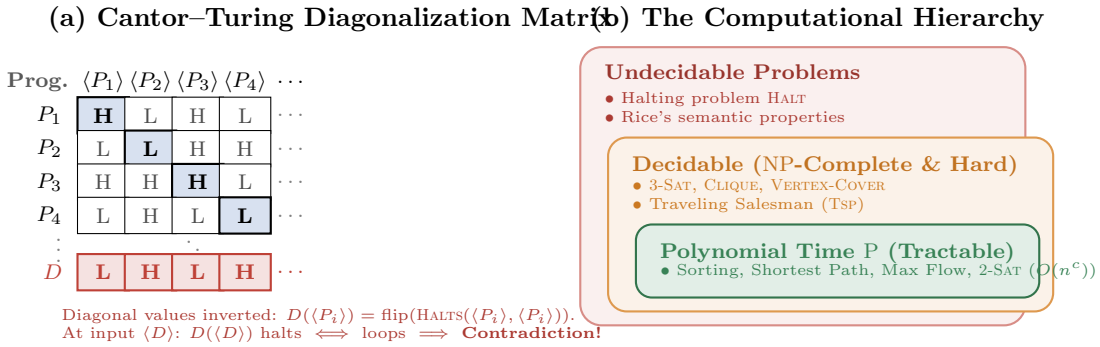


Figure 33.1: Foundations of computability and computational complexity. (a) Turing's diagonalization matrix where rows index programs P_i and columns index inputs $\langle P_j \rangle$ with entries H (Halts) and L (Loops). The contrarian machine D systematically flips every diagonal entry. When evaluated on its own description $\langle D \rangle$, D must halt if and only if it loops, demonstrating that no general halting decider can exist. (b) The structural hierarchy of computational problems, partitioned into undecidable problems (beyond algorithmic reach), decidable but intractable problems (such as NP-complete problems), and efficiently solvable problems (P).

A property of programs is called *semantic* if it depends only on the input-output behavior of the program (the language it recognizes), rather than its syntactic presentation. A property is *non-trivial* if at least one computable program satisfies it and at least one computable program does not.

Rice's Theorem has a direct consequence for software engineering: **no static analyzer, compiler optimizer, or security auditor can ever be completely automated,** 100% **accurate, and guaranteed to terminate on all inputs.** Questions such as:

- Does this function ever access uninitialized memory?
- Is this branch of code dead (unreachable)?
- Does this program produce the same output as a reference implementation?
- Does this software contain a security vulnerability?

are all undecidable semantic properties. Every production compiler, linter, and static verification tool must make approximations: either accepting false alarms (warning about code that is safe) or missing certain bugs. Perfect automated program verification is mathematically impossible.

33.2.5 The hierarchy: from the impossible to the intractable

Figure 33.1(b) organizes problems into three nested tiers:

1. **Undecidable problems:** Problems for which no algorithm can guarantee both correctness and termination on all inputs. The Halting Problem and all non-trivial semantic questions live here.
2. **Decidable problems:** Problems for which algorithms exist that always terminate with the correct answer in finite time.
3. **Tractable problems (P):** Problems within the decidable world that can be solved within polynomial time $O(n^c)$.

From here on we consider only decidable problems. Decidable does not mean practical: some decidable problems appear to require exponential search, while others can be solved in polynomial time. We start with the class P.

33.3 The class P: problems you can actually solve

The class P is the home of the well-behaved problems – the ones this book has mostly been about.

Formal

The class P. A decision problem (language) L is in P if there is an algorithm and a constant c such that, on every input x of length n, the algorithm halts within $O(n^c)$ steps and correctly outputs whether $x \in L$. In words: P is the class of problems solvable in *polynomial time*.

Sorting, shortest paths, maximum flow, primality testing, matrix multiplication, string matching, computing a minimum spanning tree – all in P. When we say a problem is "tractable" or "efficiently solvable," we mean it lives in P.

Why polynomials, and not, say, "running time under a million steps"? Because polynomials are *robust*. They are closed under addition, multiplication, and composition: if you call a polynomial-time subroutine a polynomial number of times, the whole thing is still polynomial. That closure lets us compose modular subroutines while preserving tractability: a polynomial of a polynomial is still a polynomial, whereas substituting $m = 2^n$ into 2^m yields 2^{2^n}, a doubly exponential blowup.

Why this matters

The line between polynomial and exponential matters in practice: at scale it separates what can be computed from what cannot. Suppose each "step" takes a nanosecond. An n^2 algorithm on a million-element input finishes in about a millisecond. A 2^n algorithm on a mere 70-element input would run for longer than the age of the universe. No improvement in hardware or engineering closes a gap like that. When a problem genuinely needs exponential time, faster code will not help; you have to change the question.

33.4 The class NP: problems you can check

The central idea of this chapter is familiar from everyday life: *solving* a puzzle can be hard even when *checking* a solved puzzle is easy.

Think of a finished Sudoku grid. Filling it in might have taken you an hour of agonizing. But if I hand you a completed grid and ask "is this a valid solution?", you can verify it in seconds: scan each row, each column, each box, confirm 1 through 9 appear once. The *checking* is fast even though the *solving* was slow.

The class NP captures exactly this phenomenon: problems where, if the answer is YES, there is a short piece of evidence – a *certificate*, or *witness* – that lets you *verify* the YES quickly.

Intuition

NP is the class of problems where a lucky guess can be checked fast. Imagine an untrusted prover who, whenever the true answer is YES, hands you a short candidate proof – the filled-in Sudoku, the satisfying assignment, the actual route. You do not have to trust the prover. You just have to be able to *check* its hint in polynomial time. If every YES instance has such a checkable hint, the problem is in NP.

Formal

The class NP. A language L is in NP if there is a polynomial-time *verifier* algorithm V and a constant c such that for every string x:

$$x \in L \iff \text{there exists a certificate } y \text{ with } |y| \le |x|^c \text{ and } V(x, y) = \text{YES}.$$

The verifier runs in time polynomial in $|x|$. The asymmetry matters: a YES answer must have *some* certificate that convinces V, while a NO answer must have *no* certificate that fools V.

The name NP stands for *nondeterministic polynomial time*, which is the older, equivalent definition: a theoretical machine allowed to branch on finitely many choices at each step and accept if at least one computation path accepts. The deterministic verifier view is equivalent and more direct, so we will use it throughout, while the name NP preserves that historical origin.

Common trap

NP does *not* stand for "non-polynomial." This is the single most common misunderstanding in the subject. NP is the class of efficiently *checkable* problems, and it *contains* P. If you can *solve* a problem in polynomial time, you can certainly *check* a claimed answer in polynomial time (just ignore the certificate and solve it yourself). So P $\subseteq$ NP. The mystery is whether the containment is strict.

Here is a concrete certificate for a problem we will use throughout the chapter.

Example 33.1 (HAMILTONIAN CYCLE is in NP)**.** A *Hamiltonian cycle* in a graph is a closed tour that visits every vertex *exactly once*. The decision problem HAM-CYCLE asks: "does this graph G have a Hamiltonian cycle?" Finding one seems to require searching through up to $(n-1)!$ possible vertex orderings – astronomically many. But *checking* requires only a single verification pass. The certificate is just a list of the vertices in tour order, say $v_1, v_2, \ldots, v_n, v_1$. To verify: confirm the list contains each vertex once (a sort, $O(n \log n)$), and confirm each consecutive pair is joined by an edge ($O(n)$ lookups). Polynomial. So HAM-CYCLE $\in$ NP, even though we know no polynomial *solver*.

Walking question

Take a walk and turn this over: why is the certificate asymmetry so important? For a YES instance, one good tour is all you need to show. But to be *sure* a graph has *no* Hamiltonian cycle, what would you hand someone as proof? There is no obvious short certificate of a NO – you would seemingly have to rule out all $(n-1)!$ tours. That nagging asymmetry between "easy to confirm a yes" and "hard to confirm a no" sits at the center of the P versus NP question.

33.4.1 The million-dollar question

We have $\mathrm{P} \subseteq \mathrm{NP}$. The question that has hung over computer science since 1971 is whether they are actually *equal*:

$$\mathrm{P} \stackrel{?}{=} \mathrm{NP}.$$

Is every problem whose solutions can be *checked* quickly also a problem that can be *solved* quickly? Intuition screams *no* – surely solving is harder than checking, surely there is no shortcut around the factorial search for a Hamiltonian cycle. But nobody has proved it. A proof either way is worth a million-dollar Clay Millennium Prize and would have major consequences for mathematics and cryptography. The overwhelming consensus is $\mathrm{P} \neq \mathrm{NP}$, and this chapter explains much of the evidence behind that belief: thousands of important problems have been shown to stand or fall together.

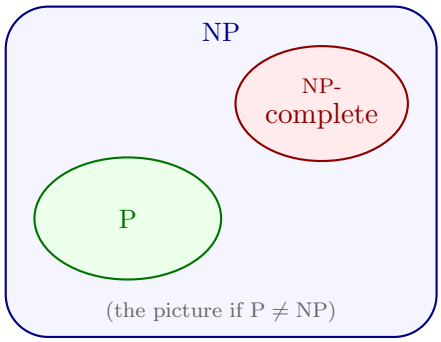


Figure 33.2: The conjectured complexity structure. P sits inside NP. The NP-complete problems are the "hardest" members of NP, pushed up against the far wall. If P = NP, the whole picture collapses to a single blob.

33.5 Reductions: "no harder than"

We now need a way to *compare* the difficulty of two problems without solving either of them. The tool is a *reduction*: a way of transforming one problem into another so cleanly that a solver for the second hands you a solver for the first.

Intuition

A reduction is a translator. Suppose you face problem A but you only own a machine that solves problem B. If you can rephrase any instance of A as an equivalent instance of B – preserving the yes/no answer – then you can solve A by translating and feeding it to your B-machine. The translation says: "A is *no harder than* B, give or take the cost of translating." If the translation itself is cheap (polynomial), then A piggybacks on B entirely.

Formal

Polynomial-time reduction. A language A *reduces in polynomial time* to a language B, written $A \leq_{\mathrm{P}} B$, if there is a polynomial-time computable function f (the reduction) such

that for every input x:

$$x \in A \iff f(x) \in B.$$

That is, f maps YES-instances of A to YES-instances of B, and NO to NO, and f runs in polynomial time.

The direction of the inequality trips everyone up at first, so let us nail it down with the two facts that make reductions useful.

Lemma 33.1 (Reductions transfer easiness and hardness)**.** *Suppose $A \leq_{\mathrm{P}} B$. Then:*

1. *If $B \in \mathrm{P}$, then $A \in \mathrm{P}$.* *(Easiness flows* backward*: if the target is easy, so is the source.)*
2. *If $A \notin \mathrm{P}$, then $B \notin \mathrm{P}$.* *(Hardness flows* forward*: if the source is hard, so is the target.)*

Proof. For (1): to decide whether $x \in A$, compute $f(x)$ in polynomial time, then run the polynomial-time decider for B on $f(x)$. Because $x \in A \iff f(x) \in B$, this answers correctly. The total time is polynomial (translate) plus polynomial (decide B), composed on an input that has grown only polynomially – still polynomial. So $A \in \mathrm{P}$.

Statement (2) is just the contrapositive of (1): if $A \notin \mathrm{P}$ then B cannot be in P, for if it were, (1) would force $A \in \mathrm{P}$. □

Common trap

Reductions point "uphill toward hardness." To prove a *new* problem B is hard, you reduce a *known-hard* problem A *into* B: $A \leq_{\mathrm{P}} B$. Students constantly do it backward – reducing the new problem into a known one – which proves the new problem is *easy*, the opposite of what they wanted. Memorize the slogan: *to show B is hard, turn a hard A into B.* The hard problem you already trust is the *source*; the new suspect is the *target*.

33.6 NP-completeness: the hardest problems in NP

We can now state the central definition. An NP-complete problem is a problem in NP that is *at least as hard as every other problem in* NP – everything reduces to it.

Formal

NP-completeness. A language L is *NP-complete* if:

1. $L \in \mathrm{NP}$ (it is checkable), and
2. for *every* language $A \in \mathrm{NP}$, we have $A \leq_{\mathrm{P}} L$ (everything in NP reduces to L).

A problem satisfying only condition (2) – hard for all of NP, but not necessarily in NP itself – is called *NP-hard*.

Condition (2) is a strong requirement: it says that *any* problem in NP can be encoded as an instance of L. If you had a polynomial algorithm for one NP-complete problem, you would have one for *all* of NP, and P would equal NP.

Focus

The central consequence:

$$\text{If any NP-complete problem is in P, then P} = \text{NP}.$$

The NP-complete problems stand or fall together: a polynomial algorithm for one gives polynomial algorithms for all. Conversely, the widespread failure to crack *any* of them, despite decades of effort by brilliant people, is the strongest practical evidence that $\text{P} \neq \text{NP}$.

But condition (2) quantifies over *every* problem in NP – an infinite, open-ended collection. How could you ever prove such a thing for even a single L? Cook and Levin proved it directly for SAT in 1971, and once that single problem was established, single reductions sufficed for every problem after it.

33.6.1 The first domino: Cook–Levin

Theorem 33.3 (Cook–Levin, 1971)**.** *The problem* SAT – *"is this Boolean formula satisfiable?" – is NP-complete.*

We will not grind through the full proof (it is technical, and CLRS §34.3 does it carefully), but the core construction is easy to describe. A SAT instance is a logical formula built from Boolean variables $x_1, \ldots, x_n$ using AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). The formula is *satisfiable* if there is some assignment of TRUE/FALSE to the variables that makes the whole thing evaluate to TRUE.

Why is *every* NP problem reducible to SAT? Because a Boolean formula can describe any computation of bounded length. Take any problem $A \in \text{NP}$ with its polynomial-time verifier V. The verifier is ultimately a circuit of logic gates churning through the input x and a guessed certificate y. Cook's reduction builds, from V and x, a giant Boolean formula whose variables encode "the certificate y" and "the entire step-by- step computation of V on (x, y)." Clauses of the formula enforce that the computation is *legal* (each step follows the verifier's rules) and that it *accepts*. The formula is satisfiable precisely when there exists a certificate y that makes V accept – i.e. precisely when $x \in A$. Because V runs in polynomial time, the formula has polynomial size, so the translation is polynomial.

Intuition

Think of SAT as the assembly language of NP. Any verification procedure, no matter how it is described, can be "compiled down" into a Boolean formula that simulates it. Satisfiability of that formula *is* the existence of a valid certificate. That is why SAT can stand in for every other problem in NP at once: logic is universal enough to mimic any computation.

33.6.2 The chain reaction: how to prove a problem NP-complete

Cook–Levin gives us our *first* NP-complete problem the hard way – by arguing about all of NP directly. Every problem after that is proved NP-complete the *easy* way, by a single reduction from a problem we already trust. Here is the recipe, and you will use it for the rest of the chapter.

Focus

To prove a new problem L is NP-complete:

1. Show $L \in$ NP: describe a short certificate and a polynomial-time check for it.
2. Pick a problem K *already known* to be NP-complete.
3. Give a polynomial-time reduction $K \leq_{\mathrm{P}} L$: a translation turning any instance of K into an instance of L with the *same* yes/no answer.

Step 3 is the creative one. The translation only *transfers* hardness; it relies on K being a previously established NP-complete problem.

Why does this work? Because reductions *compose*. If K is NP-complete then every $A \in$ NP satisfies $A \leq_{\mathrm{P}} K$. If we also show $K \leq_{\mathrm{P}} L$, then for every $A \in$ NP we can chain the translations, $A \leq_{\mathrm{P}} K \leq_{\mathrm{P}} L$, so $A \leq_{\mathrm{P}} L$. Combined with $L \in$ NP, that is exactly the definition of NP-complete. So a single new reduction is enough to make L NP-complete.

Lemma 33.2 (Reductions compose)**.** *If $A \leq_{\mathrm{P}} B$ and $B \leq_{\mathrm{P}} C$, then $A \leq_{\mathrm{P}} C$.*

Proof. Let f reduce A to B and g reduce B to C. Then $x \in A \iff f(x) \in B \iff g(f(x)) \in C$, so $g \circ f$ reduces A to C. Both f and g are polynomial-time and $f(x)$ has polynomial length, so computing $g(f(x))$ is polynomial-time. Hence $g \circ f$ is a valid polynomial-time reduction. □

The rest of the chapter walks the historical tree of reductions. Each arrow below is a single proof we will either sketch or do in full.

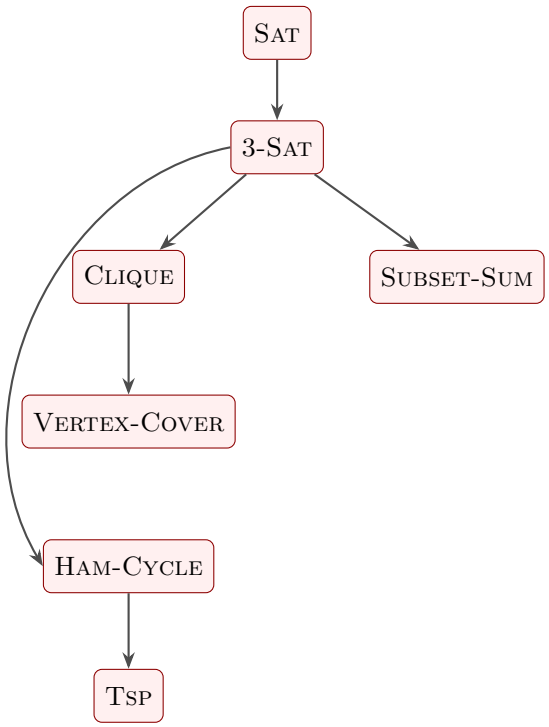


Figure 33.3: The reduction tree of this chapter. An arrow $X \to Y$ means "we reduce X to Y," proving Y NP-complete given that X already is. Everything ultimately roots at Sat via Cook–Levin.

33.7 From SAT to 3-SAT: taming the formula

Our first reduction tidies SAT into a rigid, regular shape that is far easier to build other reductions *from*. A formula is in *conjunctive normal form* (CNF) if it is an AND of *clauses*, each clause an OR of *literals*, where a literal is a variable or its negation. The problem 3-SAT restricts every clause to *exactly three* literals:

$$(x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (\neg x_1 \vee x_3 \vee x_4) \ \wedge \ (x_2 \vee \neg x_3 \vee \neg x_4).$$

3-SAT asks whether such a 3-clause formula has a satisfying assignment.

Theorem 33.4. 3-SAT *is NP-complete.*

Membership in NP is immediate: a satisfying assignment is a certificate of length n, and evaluating the formula on it takes time linear in the formula's size. *Hardness* follows from SAT $\leq_{\mathrm{P}}$ 3-SAT. We convert an arbitrary formula into an equisatisfiable 3-CNF formula in three mechanical steps: push negations inward, distribute to reach CNF, then *split long clauses* and *pad short ones* so every clause has exactly three literals.

The clever move is splitting a long clause. A clause like $(\ell_1 \vee \ell_2 \vee \ell_3 \vee \ell_4 \vee \ell_5)$ becomes a *chain* of 3-clauses linked by fresh "connector" variables y_i:

$$(\ell_1 \vee \ell_2 \vee y_1) \wedge (\neg y_1 \vee \ell_3 \vee y_2) \wedge (\neg y_2 \vee \ell_4 \vee \ell_5).$$

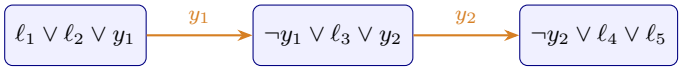


Figure 33.4: Splitting the long clause $(\ell_1 \vee \cdots \vee \ell_5)$ into a chain of 3-clauses. The fresh connector variables y_1, y_2 (orange) thread the clauses together. If every ℓ_j is false, satisfying the first clause forces y_1 true, which forces y_2 true, which leaves the last clause unsatisfiable – so the chain is satisfiable exactly when some original ℓ_j is true.

If some original ℓ_j is true, you can set the connectors so every new clause is satisfied. If *every* ℓ_j is false, then to satisfy the first clause you must set y_1 true, which forces (via $\neg y_1$) the second to need y_2 true, which forces the last clause to fail – there is no escape (Figure 33.4). So the chain is satisfiable exactly when the original long clause was. Short clauses (one or two literals) are padded by duplicating a literal or adding dummy variables. The whole conversion is polynomial in size, so SAT $\leq_{\mathrm{P}}$ 3-SAT. □

Why this matters

3-SAT is the usual starting point for NP-completeness proofs. Its structure – a fixed pile of variables, a fixed pile of three-literal clauses – is just rigid enough to "wire up" into other problems, like building a circuit out of a uniform stock of components. Almost every hardness proof in this chapter and the literature beyond starts from 3-SAT, not raw SAT. When you want to show your scheduling, layout, or game problem is hard, 3-SAT is the hammer you reach for first.

33.8 Clique: hardness becomes a graph

Now we move from logic to graph theory. A *clique* in an undirected graph is a set of vertices that are *all* pairwise adjacent. The problem Clique asks: "does graph G contain a clique of size at least k?"

Theorem 33.5. Clique *is NP-complete.*

In NP*:* the certificate is the set of k vertices; checking that all $\binom{k}{2}$ pairs are edges takes $O(k^2)$ time. For hardness we show 3-SAT $\leq_P$ Clique, a reduction that turns "can I satisfy this formula?" into "can I find a clique?"

The construction. Take a 3-CNF formula ϕ with m clauses $C_1, \ldots, C_m$. Build a graph G as follows. For each clause, create one vertex per literal – so $3m$ vertices total, arranged in m groups of three. Now add an edge between two literal-vertices ℓ (in clause C_i) and ℓ' (in clause C_j) *if and only if*:

- they are in *different* clauses ($i \neq j$), and
- they are *not contradictory*, i.e. ℓ' is not the negation of ℓ.

We deliberately put *no* edges inside a clause. Set the target $k = m$, the number of clauses.

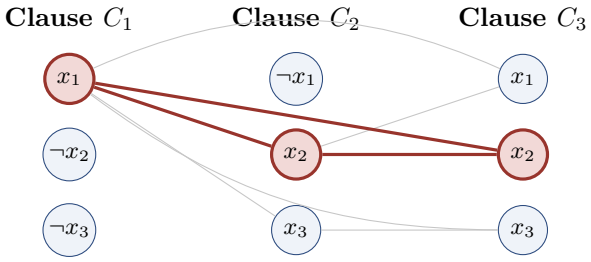


Figure 33.5: The 3-Sat-to-Clique gadget for $\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$. Edges join non-contradictory literals in different clauses (only a few cross-edges are drawn, to keep the picture readable). The highlighted red vertices $x_1 \in C_1$, $x_2 \in C_2$, and $x_2 \in C_3$ form a 3-clique corresponding to the satisfying assignment $x_1 = x_2 =$ true: exactly one true literal picked per clause, mutually consistent across all clauses.

Why it works. Suppose ϕ is satisfiable. Pick a satisfying assignment, and in each clause select *one* literal that the assignment makes true. That gives m literal-vertices, one per clause. Any two of them are in different clauses (so eligible for an edge) and are both true under the same assignment (so they cannot be a variable and its negation – that would be contradictory). Hence every pair is joined: these m vertices form a clique of size $m = k$.

Conversely, suppose G has a clique of size m. Since there are no edges inside a clause, the clique can include at most one vertex per clause – and to reach size m it must include *exactly* one from each. Because clique-vertices are pairwise non-contradictory, we can consistently set every chosen literal to true (no variable is asked to be both true and false). That assignment makes one literal true in every clause, satisfying ϕ.

So ϕ is satisfiable $\iff$ G has an m-clique. The graph has $3m$ vertices and is built in time polynomial in $|\phi|$. That completes 3-SAT $\leq_P$ Clique, and Clique is NP-complete. $\square$

Intuition

The reduction is a *translation of meaning*. "Pick one true literal in each clause" became "pick one vertex in each group." "The picks must be logically consistent" became "the picks must be pairwise connected." The geometry of the graph is engineered so that the only large cliques are exactly the consistent literal-selections. This pattern – encode the constraints of one problem as the adjacency structure of another – drives every reduction in this chapter.

33.9 Vertex Cover: the same coin, flipped

A *vertex cover* of a graph is a set of vertices that *touches every edge*: every edge has at least one endpoint in the set. Vertex-Cover asks: "is there a vertex cover of size at most k?" The direction has flipped – we want a *small* cover, whereas Clique wanted a *large* clique.

Theorem 33.6. Vertex-Cover *is NP-complete.*

In NP*:* the cover is the certificate; scan every edge and check an endpoint is included, in $O(|E|)$ time. For hardness we reduce from Clique – and the reduction takes a single complementation step, resting on one structural fact.

Lemma 33.3 (Clique–cover duality)**.** *Let G have n vertices, and let $\bar{G}$ be its* complement *(same vertices; an edge in $\bar{G}$ exactly where G has none). Then S is a clique of size k in G if and only if $V \setminus S$ is a vertex cover of size $n - k$ in $\bar{G}$.*

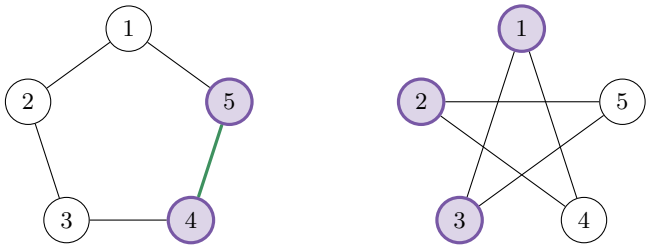
Proof. S is a clique in G means every pair inside S is an edge of G, i.e. no pair inside S is an edge of $\bar{G}$. So in $\bar{G}$, every edge has at least one endpoint *outside* S – that is, in $V \setminus S$. Thus $V \setminus S$ covers all edges of $\bar{G}$. Each step is an "if and only if," so the equivalence holds, and $|V \setminus S| = n - k$. □

The reduction. Given a Clique instance (G, k), output the Vertex-Cover instance $(\bar{G}, n - k)$. Computing the complement is $O(n^2)$, certainly polynomial. By the lemma, G has a k-clique $\iff$ $\bar{G}$ has a vertex cover of size $n - k$. So Clique $\leq_P$ Vertex -Cover, and Vertex-Cover is NP-complete. □

Example 33.2 (A traced instance)**.** Let G be the 5-cycle on vertices $\{1, 2, 3, 4, 5\}$ with edges $12, 23, 34, 45, 51$, and ask for a clique of size $k = 2$. The complement $\bar{G}$ is the *other* 5-cycle, with edges $13, 24, 35, 41, 52$. The duality predicts a vertex cover of size $n - k = 5 - 2 = 3$ in $\bar{G}$. Indeed $\{1, 2, 3\}$ covers $\bar{G}$: edge 13 hits 1, 24 hits 2, 35 hits 3, 41 hits 1, 52 hits 2. And $\{4, 5\}$ – the complement of that cover – is a clique in G (the edge 45). The bookkeeping matches perfectly (Figure 33.6).

Common trap

Do not confuse *small* and *large*. Clique and Independent-Set look for the *largest* possible structure; Vertex-Cover looks for the *smallest*. The duality lemma shows they are the same problem viewed from opposite ends – maximizing a clique is minimizing the leftover cover. When you read a hardness claim, always note whether the target wants a max or a min; a careless flip turns a correct reduction into nonsense.



$G = C_5$: clique $\{4, 5\}$ $\bar{G}$: cover $\{1, 2, 3\}$

Figure 33.6: Clique–cover duality on the 5-cycle of Example 33.2. The size-2 clique $\{4, 5\}$ in G (left, green edge) is exactly the complement of the size-3 vertex cover $\{1, 2, 3\}$ in $\bar{G}$ (right, shaded). Every edge of $\bar{G}$ touches a shaded vertex; the two unshaded vertices $4, 5$ are mutually adjacent in G.

33.9.1 The direct gadget reduction: 3-SAT to Vertex Cover

While the complement reduction from CLIQUE is short, there is another reduction from 3-SAT directly to VERTEX-COVER that every computer scientist should see once. It uses the classic technique of *component gadgets* wired together.

Given a 3-CNF formula ϕ with n variables $x_1, \ldots, x_n$ and m clauses $C_1, \ldots, C_m$:

1. **Variable gadgets**: For each variable x_i, construct an edge connecting two vertices labeled x_i and $\neg x_i$. To cover this edge, any vertex cover must pick at least one of these two vertices (n vertices across all n variables).

2. **Clause gadgets**: For each clause $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$, construct a triangle of three vertices labeled by its literals. To cover the three edges of a triangle, any vertex cover must pick at least *two* vertices ($2m$ vertices across all m clauses).

3. **Wiring edges**: For each clause C_j, connect each of its three literal vertices to the corresponding literal vertex in its variable gadget.

We ask: does the resulting graph G have a vertex cover of size at most $k = n + 2m$?

Intuition

Count the vertices: any vertex cover requires at least n vertices to cover the n variable edges, and at least $2m$ vertices to cover the m disjoint clause triangles. To reach the budget $k = n+2m$, the cover must take *exactly* one vertex per variable (encoding a consistent assignment) and *exactly* two vertices per clause triangle. The third vertex in each clause triangle is left uncovered from below, so its dashed wiring edge *must* be covered from above by the chosen variable vertex! Thus every clause contains at least one true literal.

33.10 Hamiltonian Cycle and the Traveling Salesman

We met HAM-CYCLE in Example 33.1: does a graph have a closed tour visiting every vertex exactly once? It too is NP-complete; the reduction (from 3-SAT, via intricate "gadgets" that simulate variables and clauses with little chains of vertices) is genuinely involved, so we will state the result

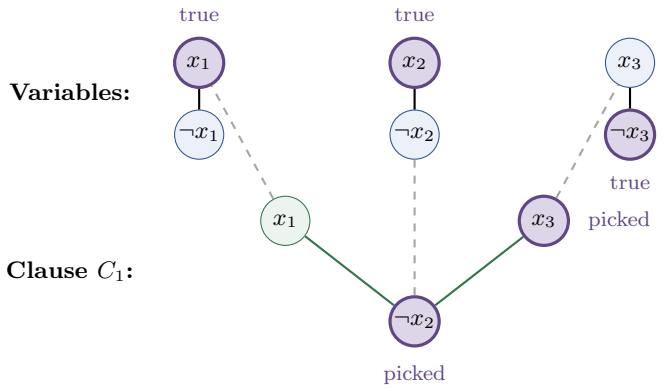


Figure 33.7: The direct 3-SAT-to-VERTEX-COVER gadget reduction for clause $C_1 = (x_1 \vee \neg x_2 \vee x_3)$. Variable gadgets (top) force at least 1 vertex per variable into any cover. Clause triangles (bottom) force at least 2 vertices per clause. Setting $x_1 = \text{TRUE}$, $x_2 = \text{TRUE}$, $x_3 = \text{FALSE}$ satisfies C_1 through literal x_1; its dashed wiring edge is covered by x_1 in the variable gadget, allowing the remaining two clause vertices ($\neg x_2$ and x_3) to cover the triangle and all remaining wires. Total cover size is exactly $k = n + 2m = 3 + 2(1) = 5$.

and instead spend our energy on its famous corollary.

Theorem 33.7. HAM-CYCLE *is NP-complete.*

The reduction's flavor is worth a sentence. One builds, for each variable, a long two-way "ladder" that any Hamiltonian cycle must traverse either left-to-right or right-to-left – and that binary choice *is* the variable's truth value. Small clause-gadgets are wired into the ladders so that the cycle can complete its tour if and only if every clause is satisfied by the chosen traversal directions. The variables' two-way streets encode TRUE/FALSE; the clauses act as gates that the tour can only pass through when satisfied. It is fiddly, but it is the same idea as the CLIQUE reduction – encode logic as structure – scaled up.

33.10.1 TSP: putting numbers on the edges

Now for one of the most famous problems in all of computing. The *traveling salesman* must visit a set of cities and return home, and wants the *cheapest* possible round trip. As a decision problem, TSP asks: given cities, pairwise distances, and a budget B, is there a tour visiting all cities exactly once with total length at most B?

Theorem 33.8. TSP *is NP-complete.*

In NP*:* a tour is the certificate; sum its edge lengths and compare to B in $O(n)$ time. For hardness we reduce from HAM-CYCLE, and this reduction is short. It also shows hardness carrying over from a purely structural problem to a numeric optimization problem.

The reduction. Given a graph G on n vertices, build a TSP instance whose "cities" are the vertices of G, with the *complete* distance matrix

$$d(u, v) = \begin{cases} 1 & \text{if } (u, v) \text{ is an edge of } G, \\ 2 & \text{if } (u, v) \text{ is } \textit{not} \text{ an edge of } G, \end{cases}$$

and set the budget $B = n$. The matrix is built in $O(n^2)$ time.

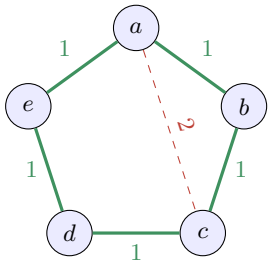


Figure 33.8: The HAM-CYCLE-to-TSP reduction. Real edges of G get distance 1 (solid green); every non-edge gets distance 2 (dashed red, one shown). A tour of $n = 5$ cities uses 5 edges, so the only way to hit the budget $B = 5$ is to ride entirely on cost-1 edges – that is, on a Hamiltonian cycle of G. A single cost-2 edge would push the total to $6 > B$.

Why it works (Figure 33.8)**.** A TSP tour visits all n cities and uses exactly n edges (it is a cycle). Each edge costs 1 or 2, so the cheapest conceivable tour costs n – achieved *only* if every single edge it uses costs 1, i.e. every consecutive pair of cities is a genuine edge of G. But a tour visiting every vertex once using only real edges of G is precisely a Hamiltonian cycle! So:

$$G \text{ has a Hamiltonian cycle} \iff \text{there is a tour of length} \le n = B.$$

If G has a Hamiltonian cycle, follow it: cost exactly $n \le B$. If the TSP instance has a tour of length $\le n$, every edge it uses must cost 1 (any cost-2 edge would push the total above n), so the tour rides only on edges of G – a Hamiltonian cycle. Hence HAM-CYCLE $\le_P$ TSP, and TSP is NP-complete. □

Why this matters

TSP is the public face of NP-completeness: delivery routing, circuit- board drilling, DNA sequencing, and telescope scheduling all take this form. Its NP-completeness does *not* mean it is hopeless – real solvers route thousands of cities daily. It means there is (almost certainly) no algorithm that is fast *and* exact *and* general all at once. In practice you give up one of those: settle for "almost optimal" (approximation, Chapter 34), or exploit special structure (cities in the plane behave better), or let a solver grind on a particular instance. Proving a problem NP-complete does not mean giving up on solving it in practice; it tells you to stop searching for an exact worst-case polynomial algorithm on arbitrary inputs and instead adopt one of those strategies.

33.11 Subset-Sum: when small numbers hide big inputs

The last problem in this chapter does not look like a graph problem at all, and it illustrates a subtle point about input size. SUBSET-SUM: given a multiset of positive integers $\{a_1, \ldots, a_n\}$ and a target t, is there a subset that sums *exactly* to t?

Theorem 33.9. SUBSET-SUM *is NP-complete.*

In NP*:* the certificate is the subset; add up its elements and compare to t, in $O(n)$ additions. Hardness follows from 3-SAT $\le_P$ SUBSET-SUM via a clever number-encoding: each variable and

each clause gets its own "decimal column," and we craft numbers so that hitting the target requires choosing one truth value per variable while filling every clause's column to a fixed amount – forcing each clause to be satisfied. The arithmetic of carries is rigged to never interfere across columns. (CLRS §34.5.5 builds it digit by digit; the construction is worth reading once you trust the pattern.)

Common trap

"But SUBSET-SUM has an easy dynamic program!" – a fair objection, and the reason for this section. In Chapter 15 you built a table of size $n \times t$ and filled it in $O(nt)$ time. So is SUBSET-SUM in P? *No.* The catch is that t is a *number*, but the input *size* is the number of *bits* needed to write everything down, and t takes only about $\log_2 t$ bits. So $O(nt)$ is $O(n \cdot 2^{\log_2 t})$ – *exponential* in the input length (Figure 33.9). Writing t in binary is cheap; counting up to t is not.

Here the warning from the first section becomes concrete: input size counts *symbols*, not magnitudes. An algorithm whose running time is polynomial in the numeric *value* of the inputs (here, in t) – but exponential in their bit-length – is called *pseudo-polynomial*. It can be genuinely useful when the numbers are small, but it does *not* place the problem in P, and it does not threaten P $\neq$ NP.

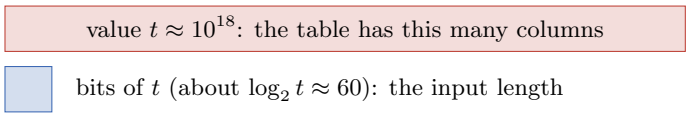


Figure 33.9: Why the $O(nt)$ table is not polynomial. The work is proportional to the *value* t (long red bar), but the input is only the *bits* of t (short blue bar), about $\log_2 t$ of them. Each extra bit roughly *doubles* t, so the running time is exponential in the input length even though it looks linear in t.

Formal

Pseudo-polynomial vs. polynomial. An algorithm is *polynomial-time* if its running time is bounded by a polynomial in the input *length* (the number of bits/symbols). It is merely *pseudo-polynomial* if its running time is polynomial in the input length *and* in the largest numeric value appearing in the input. The $O(nt)$ table for SUBSET-SUM is pseudo-polynomial: polynomial in n and in the *value* t, but exponential in the *bits* of t.

Walking question

On your next walk, consider this question: why does the bit-length convention feel right? Imagine an "algorithm" that solves a problem by counting from 1 up to the input number N. Its running time is $O(N)$ – linear in the value, surely "fast"? But to make N a hundred times bigger you add only about seven bits to the input, while the running time grows a hundredfold. Any sensible measure of efficiency must punish that. Charging by bits, not by magnitude, is what makes "polynomial time" line up with our gut sense of "scales gracefully."

33.12 A worked reduction, end to end

Reductions are a skill learned by practice. Let us run one all the way through on a tiny instance. We start from a concrete 3-SAT formula and reduce it to CLIQUE, then read the answer back.

Algorithm lab

Take the formula

$$\phi = (x_1 \vee x_2 \vee \neg x_3) \ \wedge \ (\neg x_1 \vee \neg x_2 \vee x_3) \ \wedge \ (x_1 \vee \neg x_2 \vee x_3).$$

Step 1 – build the graph. It has 3 clauses, so 9 vertices in 3 groups:

$$C_1 : \{x_1, x_2, \neg x_3\}, \quad C_2 : \{\neg x_1, \neg x_2, x_3\}, \quad C_3 : \{x_1, \neg x_2, x_3\}.$$

Connect every pair from *different* groups unless they contradict (one is the negation of the other). For instance $x_1 \in C_1$ connects to $\neg x_2, x_3 \in C_2$ but *not* to $\neg x_1 \in C_2$.
Step 2 – hunt for a 3-clique. Try the trio $\{x_1(C_1), x_3(C_2), x_1(C_3)\}$. Different groups? Yes. Pairwise non-contradictory? x_1 vs. x_3: fine. x_1 vs. x_1: same literal, not a contradiction, fine. x_3 vs. x_1: fine. All three pairs are edges – a 3-clique!
Step 3 – read off the assignment. The clique used the literals x_1, x_3, x_1, so set $x_1 = \text{TRUE}$ and $x_3 = \text{TRUE}$; x_2 is unconstrained, say FALSE. Check ϕ: clause 1 has x_1 true; clause 2 has x_3 true; clause 3 has x_1 true. All satisfied. The clique we found *is* a satisfying assignment, exactly as the reduction promised.

Algorithm 142: Generic NP-completeness proof template

Input: A decision problem L you suspect is NP-complete
Output: A proof that L is NP-complete (or a discovery that it is easy)

Step A (membership). Exhibit a certificate y for YES-instances and a polynomial-time verifier $V(x, y)$;
if *no short, checkable certificate exists* **then**
    L may be harder than NP; stop and reconsider;
Step B (choose a source). Select a known NP-complete problem K whose structure resembles L (often 3-SAT);
Step C (reduce). Design f mapping each instance w of K to an instance $f(w)$ of L;
Step D (correctness). Prove $w \in K \iff f(w) \in L$ (both directions);
Step E (efficiency). Verify f runs in time polynomial in $|w|$;
return "L is NP-complete" (Steps A and C–E together);

33.13 A Plain-English Guide to P, NP, and Related Complexity Classes

The central question of P versus NP is easy to state, but after more than half a century its terminology still confuses newcomers and experienced software engineers alike.

People routinely mix up NP, NP-complete, and NP-hard. Many assume NP stands for "not polynomial." Others wonder why finding the shortest traveling-salesman tour is treated differently from

deciding whether a tour of length at most B exists, or why integer factoring is so central to public-key cryptography if it is not known to be NP-complete.

Before looking at how engineers handle hardness in practice, let us define each of these classes in plain English.

33.13.1 Why the names are confusing (and how to never mix them up again)

The vocabulary of complexity theory grew historically across logic, discrete mathematics, and early computer architecture. Table 33.1 and the narrative below contrast all six terms that arise in these discussions.

Term	Plain-English Meaning	Intuitive Metaphor	Canonical Examples
P	Solvable from scratch in polynomial time $O(n^c)$.	Calculating a driving route on a GPS unit.	Shortest paths (Dijkstra), sorting, max flow, primality testing (AKS).
NP	Verifiable in polynomial time when handed a hint.	Checking whether a filled-in Sudoku grid is correct.	All of P, plus 3-Sat, Subset-Sum, Clique, factoring.
co-NP	Refutable in polynomial time with a single counterexample.	Proving an argument is invalid by finding one flaw.	Tautology (formula true on all inputs), No-Hamilton-Cycle.
NP-**Complete**	The hardest problems in NP; every problem in NP reduces to them.	A master key that can be shaped to open any door in a building.	3-Sat, Vertex-Cover, Clique, Ham-Cycle, Tsp (decision).
NP-**Hard**	At least as hard as NP-complete; not necessarily in NP.	A task so demanding it might not even have checkable answers.	Tsp optimization (find minimum tour), the Halting Problem.
NP-**Intermediate**	Inside NP, not in P, yet likely not NP-complete.	A problem that resists fast solution without every NP problem reducing to it.	Integer Factoring, Discrete Logarithm, Graph Isomorphism.

Table 33.1: The six primary terms in the P versus NP vocabulary, translated into plain English.

Let us walk through each term carefully to understand why these boundaries exist:

1. P: Easy to solve from scratch

A problem lives in P if an ordinary deterministic computer can start from a cold input of size n and calculate the exact answer within $O(n^c)$ clock ticks, with no hints or lucky guesses. Sorting a list, computing minimum spanning trees, finding shortest routes, and determining whether a number is prime (settled by Agrawal, Kayal, and Saxena in 2002 via the polynomial AKS primality test) all live comfortably in P.

2. NP: Easy to check when handed a hint

A common misunderstanding is that NP means "not polynomial"; it does not. NP stands for *nondeterministic polynomial time*, a historical name for a simple concept: problems where verifying

a proposed answer takes polynomial time.

Every problem in P is automatically in NP: if you can solve something from scratch in polynomial time, you can verify a claimed answer by ignoring the certificate and solving it yourself. The mystery of NP comes from the problems that appear to be much harder to solve than to check:

- Solving a 500-piece jigsaw puzzle might take you an entire weekend of trial and error; verifying a finished puzzle takes ten seconds of scanning the seams.
- Factoring a 600-digit number $N = p \times q$ defeats supercomputers; verifying that $p \times q = N$ requires one multiplication pass.
- Finding a formal proof for a mathematical conjecture can consume centuries; verifying that a formal proof in Lean or Coq contains no syntax or logic errors runs in seconds on a laptop.

3. co-NP: Easy to disprove with a counterexample

co-NP is the mirror image of NP. Recall the asymmetry of NP: for a YES instance, there is a short certificate that convinces you (the Hamiltonian tour, the satisfying assignment). But for a NO instance ("this graph has no Hamiltonian cycle"), you seemingly have to rule out all factorial possibilities.

co-NP swaps the roles of YES and NO. A problem is in co-NP if a NO answer has a short, polynomial-time checkable certificate.

Consider the TAUTOLOGY problem: given a Boolean formula ϕ, is ϕ true for *every possible truth assignment*?

- If the answer is NO, proving it is easy: hand the verifier a single assignment that evaluates ϕ to FALSE. That single counterexample is verified in linear time. Thus, TAUTOLOGY is in co-NP.
- If the answer is YES, how do you convince someone that *no counterexample exists* anywhere among the 2^n possibilities? There is no known short witness.

Complexity theorists believe that NP $\neq$ co-NP. If they were equal, every problem with a short proof of YES would also have a short proof of NO.

4. NP-Complete: The hardest problems in NP

An NP-complete problem is a problem inside NP with a universal translation property: *every single problem in* NP *can be transformed into it in polynomial time.*

They are the hardest problems in NP. As we have seen throughout this chapter, 3-SAT, CLIQUE, VERTEX-COVER, HAMILTONIAN-CYCLE, and SUBSET-SUM are all polynomial-time equivalent. If an engineer discovers a polynomial-time algorithm for any one of them, that single breakthrough would immediately solve every problem in NP in polynomial time, establishing P = NP.

5. NP-Hard: At least as hard as NP-complete, maybe not even checkable

This is the second common point of confusion. What is the difference between NP-complete and NP-hard?

- To be NP**-complete**, a problem must satisfy two conditions: it must be in NP, and it must be NP-hard.

- To be NP**-hard**, a problem only needs to be at least as hard as NP-complete problems. It does *not* have to be in NP, and it does *not* even have to be a yes/no decision problem!

Consider two concrete examples of problems that are NP-hard but *not* in NP:

1. **The Optimization Version of Traveling Salesman (Tsp-Opt):** "Given a list of cities and pairwise distances, find the tour of *absolute minimum* total length." Why is this not in NP? Because if someone hands you a tour of length 412, how do you verify in polynomial time that *no tour of length* 411 *exists*? To verify optimality, you would need to rule out all competing tours – a co-NP task, not an NP task. Because it is not a decision problem with a polynomial certificate, it is not in NP. But it is NP-hard: if you could find the optimal tour instantly, you could immediately answer the yes/no decision version.

2. **The Halting Problem (Halt):** As proved in Theorem 33.1, deciding whether an arbitrary program halts is undecidable. Yet Halt is NP-hard! Any NP problem, such as 3-Sat, reduces to Halt: given a formula ϕ, construct a program that enumerates all assignments and halts if and only if it finds one that satisfies ϕ. The program halts if and only if ϕ is satisfiable. Thus, Halt is NP-hard, even though it is not computable at all.

6. NP-Intermediate: The mysterious middle ground

If P $\neq$ NP, is every problem in NP either in P or NP-complete?

In 1975, Richard Ladner proved that the answer is **no**: if P $\neq$ NP, there exists an infinite hierarchy of problems in NP that are provably not in P, yet provably not NP-complete. These problems are designated NP**-intermediate**.

Ladner's proof uses an artificial diagonal construction, but some natural problems are strong candidates for this intermediate class:

- **Integer Factoring:** Given an integer N, find its prime factors. Factoring is in NP because checking that factors multiply to N takes $O((\log N)^2)$ time. But Factoring is also in co-NP: by the Fundamental Theorem of Arithmetic, prime factorizations are unique. A list of claimed prime factors along with primality certificates proves both what the factors are and that no other factors exist. Because Factoring lives in NP $\cap$ co-NP, complexity theorists strongly believe it is *not* NP-complete. If Factoring were NP-complete, it would imply NP = co-NP, an eventuality considered almost as unlikely as P = NP. RSA cryptography relies on this combination: no known classical algorithm factors in polynomial time, yet the number-theoretic structure of factoring is what makes public-key encryption work.

- **Graph Isomorphism:** Given two graphs G_1 and G_2, can the vertices of G_1 be relabeled to produce G_2? In 2015, László Babai proved that Graph Isomorphism can be solved in quasi-polynomial time ($2^{O((\log n)^c)}$). Because $2^{(\log n)^c}$ grows far slower than the $2^{\Omega(n)}$ time expected for NP-complete problems, Graph Isomorphism is almost certainly intermediate rather than NP-complete.

Figure 33.10 illustrates how these classes nest under the standard assumption that P $\neq$ NP.

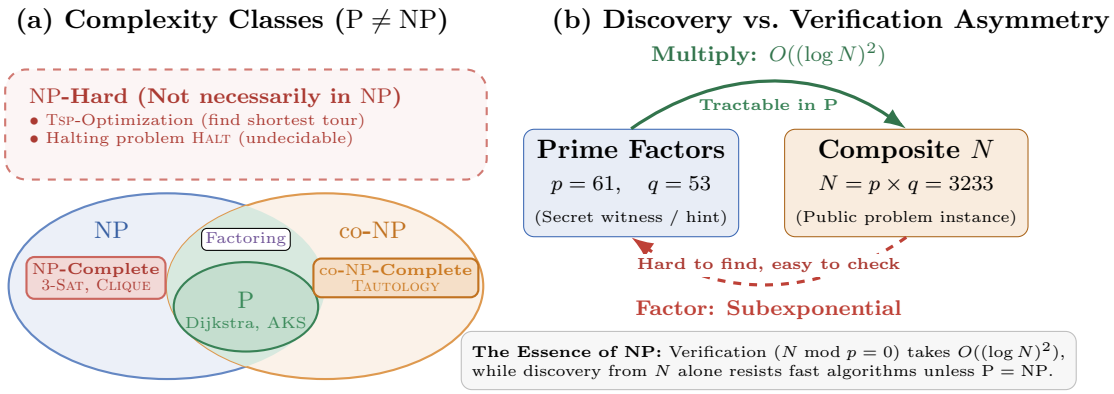


Figure 33.10: The taxonomy of computational complexity and the core asymmetry of verification. (a) Under the standard consensus that P $\neq$ NP, the complexity classes cleanly separate: problems in P are solvable in polynomial time; NP contains problems whose solutions are verifiable in polynomial time; co-NP contains problems whose counterexamples are verifiable in polynomial time; NP-Complete problems are the hardest problems in NP; NP-Intermediate problems like Factoring sit in NP $\cap$ co-NP outside P without being NP-Complete; and NP-Hard encompasses problems at least as hard as 3-SAT, including optimization and undecidable questions. (b) The gap between easy verification (in P) and difficult search (NP), exemplified by integer multiplication versus factoring.

33.13.2 What is actually at stake: a day in a P = NP world

The P versus NP question draws interest far outside theoretical computer science because, informally, it asks whether finding a solution is fundamentally harder than checking one.

Consider what would happen if a researcher proved P = NP by exhibiting a practical, low-degree polynomial-time algorithm for 3-SAT:

- **Public-key and symmetric cryptography break.** Modern digital security – from HTTPS encryption and banking transactions to SSH keys and digital signatures – rests entirely on the assumption that certain mathematical tasks are easy to perform in the forward direction but intractable to reverse without a secret key. Finding an AES-256 key that matches a given plaintext and ciphertext is an NP search problem: verifying a candidate key takes microseconds. If P = NP, cracking any symmetric or asymmetric cipher reduces to solving a polynomial-size SAT instance in minutes.
- **Mathematical theorem-proving is automated.** Is a famous mathematical conjecture of length n provable in fewer than M steps? An algorithm cannot easily guess the proof by brute force. But checking whether a candidate sequence of deductions constitutes a valid proof is a purely mechanical, polynomial-time verification pass. If P = NP, an algorithm could generate formal mathematical proofs of length M as fast as a computer checks them. As Kurt Gödel realized in his 1956 letter to John von Neumann, mathematical research itself would change fundamentally.
- **Scientific and industrial optimization becomes exact.** Designing a synthetic protein that folds to minimize free energy, routing hundreds of thousands of delivery vehicles with optimal fuel consumption, and floorplanning billions of transistors on a microprocessor without thermal bottlenecks are all NP search problems. A fast SAT solver would solve each of them to global mathematical optimality.

As computer scientist Scott Aaronson noted, if P = NP, then anyone who can appreciate a beautiful melody could compose like Mozart, and anyone who can follow a mathematical argument could discover breakthroughs like Gauss. The everyday experience that appreciating art or verifying a proof is far easier than creating it is a common informal reason to believe that P $\neq$ NP.

33.13.3 Why proving P $\neq$ NP is so hard: the three mathematical barriers

If virtually every computer scientist believes P $\neq$ NP, why has nobody managed to prove it in more than fifty years?

Consider the fundamental asymmetry of the challenge:

- To prove P = NP, you only need to design **one** algorithm: an ingenious polynomial-time method for 3-SAT.
- To prove P $\neq$ NP, you must prove an impossibility theorem about **every possible algorithm**.

You must prove that no algorithm – no matter how unexpected its mechanics – can ever solve 3-SAT in polynomial time.

This is formidable because computer science is full of algorithms that perform surprising mathematical shortcuts. Naive matrix multiplication requires 8 recursive multiplications of half-size blocks, leading to $O(n^3)$ time; Volker Strassen (1969) showed that an unexpected algebraic cancellation computes the product with only 7 multiplications, dropping the exponent below 3. Multiplying two degree-n polynomials by convolution appears to require $O(n^2)$ coefficient operations; the Fast Fourier Transform (Cooley and Tukey, 1965) collapses the cost to $O(n \log n)$ by converting into the frequency domain. To prove P $\neq$ NP, one must prove that *no such algebraic shortcut exists* for any of the thousands of NP-complete problems.

Whenever complexity theorists tried to prove lower bounds, their techniques ran into three barriers:

1. **The Relativization Barrier (Baker, Gill, and Solovay, 1975):** Early impossibility results in computability (like the Halting Problem) relied on simulation and diagonalization. Baker, Gill, and Solovay proved that these techniques are "relativizing"—they hold unchanged even if every machine is granted free, instantaneous access to an external black-box oracle A. They then constructed two oracles: an oracle A where $\mathrm{P}^A = \mathrm{NP}^A$, and an oracle B where $\mathrm{P}^B \neq \mathrm{NP}^B$. Therefore, no proof technique that relativizes can resolve P versus NP.

2. **The Natural Proofs Barrier (Razborov and Rudich, 1997):** Researchers next turned to circuit complexity, attempting to prove that small Boolean circuits cannot compute functions like CLIQUE. Alexander Razborov and Steven Rudich demonstrated that almost all known circuit lower bound methods fall into a category they termed "natural proofs" (identifying a constructive combinatorial property common to random functions but rare in small circuits). They proved that if strong pseudorandom function generators exist (the foundation of modern cryptography), then no natural proof can ever separate P from NP. In other words, if strong cryptography exists, natural proofs cannot establish the kind of hardness it relies on.

3. **The Algebrization Barrier (Aaronson and Wigderson, 2008):** In the 1990s, non-relativizing algebraic techniques successfully proved that IP = PSPACE. Scott Aaronson and Avi Wigderson proved that even when algebraic oracle extensions are permitted ("algebrization"), the barrier re-emerges: there exist algebraic oracles where the classes coincide and

others where they separate.

Any proof that separates P from NP must be simultaneously non-relativizing, non-natural, and non-algebrizing. No known technique meets all three conditions.

33.14 What to do when your problem is NP-complete

Suppose you have just proved – or found in a textbook – that the problem on your desk is NP-complete. What now? You do *not* give up, and you do *not* keep hunting for a fast exact general algorithm (most researchers expect that search to fail). Instead you relax one of the three requirements "fast, exact, general."

- **Drop exactness – approximate.** Accept a provably near-optimal answer. A good Vertex-Cover heuristic gets within a factor of 2 of optimal in linear time. This is the subject of the next chapter.
- **Drop generality – exploit structure.** Real instances are not adversarial. TSP on points in the plane, 2-SAT (clauses of size two – actually in P!), trees instead of general graphs: special cases routinely escape the hardness.
- **Drop speed in the worst case – search smartly.** Branch-and-bound, SAT solvers, and integer-programming engines often solve real instances with thousands of variables, even though some adversarial input could in principle make them crawl.
- **Drop determinism – randomize or use heuristics.** Simulated annealing, genetic algorithms, and local search give no guarantees but often deliver excellent answers fast.

Focus

The practical lesson of this chapter: *before you spend months on a perfect algorithm, check whether the problem is NP-complete.* If it is, skip the search for an exact fast algorithm and spend the time on approximation, special cases, or smart search instead.

33.14.1 The practical dividend of hardness

The discovery of NP-completeness changed the daily practice of computer science:

- **A way to recognize unrealistic goals.** Before Karp and Cook, software engineers and researchers frequently spent years of futile effort attempting to invent polynomial-time algorithms for problems like Traveling Salesman or Boolean Satisfiability, assuming failure was merely their own lack of mathematical cleverness. NP-completeness gave engineers a clear test: proving a problem is NP-complete shows that finding an exact polynomial algorithm is as hard as resolving the entire P versus NP question.
- **The SAT revolution in verification.** Cook's proof showed that any polynomial-time verification can be encoded in Boolean logic. Decades later, engineers turned the same idea into an industrial tool. Modern Conflict-Driven Clause Learning (CDCL) SAT and SMT solvers routinely compile microprocessor circuits, operating system drivers, and cryptographic protocols into millions of Boolean clauses, verifying hardware and software automatically.

- **Connections across fields.** NP-completeness showed that many hard problems from graph theory, number theory, compiler register allocation, molecular biology, and operations research are polynomial-time equivalent: an efficient algorithm for one would give efficient algorithms for all of them.

33.14.2 Coping with worst-case limits

NP-completeness also has practical limits:

- **It is a worst-case statement.** NP-completeness is an asymptotic, worst-case verdict. It asserts only that there exists *some* pathological family of instances on which every algorithm stumbles. It says nothing about average-case instances or real-world distributions. In practice, industrial instances of 3-SAT, TSP, and KNAPSACK with thousands or millions of variables are routinely solved to optimality in seconds because real-world problem instances possess low treewidth, modularity, or geometric structure.

- **Polynomial reductions can hide large constants.** A reduction is declared efficient as long as its runtime is $O(n^c)$. A reduction that transforms an instance of size n into an instance of size n^{10} or requires $10^{10}n^2$ operations is theoretically in P, but practically slow.

- **The unproven foundation.** Despite overwhelming consensus and a Clay Millennium Prize, P $\neq$ NP remains rigorously unproven. Decades of attempts to prove lower bounds using circuit complexity and diagonalization have run into deep structural barriers.

Cost summary

A compact map of the chapter.

- Undecidable: problems beyond algorithmic reach; the Halting Problem is uncomputable by Turing diagonalization ($D(\langle D \rangle)$), and Rice's theorem extends this to all non-trivial semantic properties.
- P: solvable in time $O(n^c)$. "Tractable."
- NP: YES-instances have a polynomial-size certificate checkable in polynomial time. Contains P.
- $A \leq_{\mathrm{P}} B$: a polynomial-time f with $x \in A \iff f(x) \in B$. Hardness flows forward, easiness backward.
- NP-complete: in NP *and* every NP problem reduces to it. One in P $\Rightarrow$ all of NP in P.
- Cook–Levin: SAT is the first NP-complete problem. All others follow by single reductions: SAT $\leq$ 3-SAT $\leq$ {CLIQUE, SUBSET-SUM}, CLIQUE $\leq$ VERTEX-COVER, 3-SAT $\leq$ HAM-CYCLE $\leq$ TSP.
- Pseudo-polynomial: polynomial in input *value*, not input *length*. SUBSET-SUM's $O(nt)$ table is this – not a proof that the problem is in P.

A closing thought

NP-completeness also has an encouraging side. It shows that thousands of stubborn problems across logic, graphs, scheduling, biology, and number theory are not an unrelated collection: they are all polynomial-time translations of a single underlying search problem. Solve the salesman problem efficiently and you solve them all; if one of them is truly hard, they all are. What we do not know is therefore well organized: it comes down to the single question $\mathrm{P} \stackrel{?}{=} \mathrm{NP}$.

When an exact polynomial algorithm is out of reach, we can trade optimality for efficiency. In Chapter 34, we turn to approximation algorithms, designing algorithms that run in guaranteed polynomial time while provably bounding the distance between their solutions and the true optimum.

33.15 Exercises

Exercise 33.1 (Certificates, warmup)**.** For each problem, state a short certificate for YES-instances and the polynomial-time check that verifies it: (a) COMPOSITE ("is N not prime?"); (b) GRAPH-3-COLORING ("can G be properly colored with 3 colors?"); (c) SUBSET-SUM.

Exercise 33.2 (The direction of reductions)**.** Your classmate proves NEW-PROBLEM $\leq_{\mathrm{P}}$ 3-SAT and concludes NEW-PROBLEM is NP-complete. Explain precisely what is wrong, and state what this reduction *does* legitimately tell you about NEW-PROBLEM.

Exercise 33.3 (Optimization to decision)**.** Describe carefully how a polynomial-time decider for the *decision* version of TSP ("is there a tour of length $\leq B$?") could be used to find the exact length of the *optimal* tour in polynomial time. (Hint: binary search on B.) Why does this argument show the decision and optimization versions are equally hard?

Exercise 33.4 (Tracing 3-SAT to Clique)**.** Apply the 3-SAT-to-CLIQUE reduction to $\phi = (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$. Draw the 6-vertex graph, list its edges, find a 2-clique, and read off a satisfying assignment. Is there any 3-clique? Explain why or why not.

Exercise 33.5 (Clique–cover duality by hand)**.** Let G be the complete graph K_4 (every pair of 4 vertices adjacent). What is the largest clique in G? Using Lemma 33.3, what does that say about the smallest vertex cover of $\bar{G}$? Verify directly by describing $\bar{G}$.

Exercise 33.6 (Independent sets)**.** An *independent set* is a set of vertices with *no* edges between them. Prove that S is an independent set of G if and only if $V \setminus S$ is a vertex cover of G. Conclude that INDEPENDENT-SET is NP-complete by a one-line reduction from VERTEX-COVER.

Exercise 33.7 (Hamiltonian path vs. cycle)**.** The HAM-PATH problem asks for a path (not necessarily closed) visiting every vertex exactly once. Give a polynomial-time reduction HAM-CYCLE $\leq_{\mathrm{P}}$ HAM-PATH. (Hint: pick an edge of the cycle to "cut," or add a gadget vertex. Think about what turns a path into a cycle.)

Exercise 33.8 (Why exactly three?)**.** We reduced SAT to 3-SAT. Suppose instead you restrict to 2-SAT: every clause has exactly two literals. It turns out 2-SAT is in P. Where in the long-clause-splitting argument of Theorem 33.4 would a "split into 2-clauses" attempt break down? (You need not solve 2-SAT; just locate the obstruction.)

Exercise 33.9 (Pseudo-polynomial, concretely)**.** The Subset-Sum table runs in $O(nt)$ time. (a) For $n = 100$ numbers each around 10^{18}, estimate t and the table size; is this practical? (b) Explain in one sentence why this does not contradict NP-completeness. (c) Give an instance family where the $O(nt)$ algorithm *is* fast, and say what is special about it.

Exercise 33.10 (Build your own reduction – challenge)**.** The Set-Cover problem: given a universe U of elements, a collection of subsets $S_1, \ldots, S_m \subseteq U$, and a number k, can k of the subsets cover all of U? Prove Set-Cover is NP-complete by reducing Vertex-Cover to it. (Hint: let the universe be the edges of the graph, and let each vertex contribute the set of edges it touches.)

Exercise 33.11 (A reduction that proves membership in P, not hardness – challenge)**.** Consider 2-Coloring ("can G be properly 2-colored?"). A student tries to prove it NP-complete by reducing 3-Coloring to it. Without doing the reduction, argue why any such attempt must fail, assuming P $\neq$ NP. (Hint: 2-Coloring is exactly the bipartiteness test from Chapter 22 – what complexity class is it in?)

Exercise 33.12 (The halting contrarian by hand)**.** Recall the contrarian program D from the proof of Theorem 33.1. Suppose a computer scientist claims to have implemented an analyzer `Halts(P, x)` in 500 lines of Python that handles all programs. (a) Write out the exact 6-line Python script for $D(\langle P \rangle)$ that uses this analyzer. (b) Walk through the exact step-by-step evaluation of $D(\langle D \rangle)$, tracing the execution path if the analyzer returns `True`, and tracing the path if it returns `False`. (c) Conclude why this contradiction implies that no static analyzer can ever be both sound (never miss an infinite loop) and complete (never falsely flag a terminating program) while always halting.

Exercise 33.13 (Rice's Theorem in compiler optimization)**.** A compiler engineer attempts to build an optimization pass that automatically replaces any function computing the identity function $f(n) = n$ with a direct return instruction `return n`. (a) Define what it means for a property of programs to be *semantic* versus *syntactic*. (b) Prove that "computes the identity function" is a non-trivial semantic property. (c) Using Rice's Theorem (Theorem 33.2), explain why no compiler optimization pass can ever identify *all* functions that compute the identity function without either looping indefinitely on some inputs or using conservative heuristics that miss equivalent implementations.

Chapter 34

Approximation Algorithms

In Chapter 33, we mapped the boundary of computational hardness: thousands of practical problems—from vertex cover to the traveling salesperson—are NP-complete, resisting exact polynomial-time algorithms. If anybody ever finds a polynomial-time algorithm that solves one of them exactly on every input, they will have solved all of them, collected a million-dollar prize, and rewritten computer science. Most experts bet that no such algorithm exists.

So where does that leave the engineer who has to route delivery trucks tomorrow morning? You cannot wait for 2^n steps. You cannot tell your team that the problem is NP-hard and walk away: the trucks still need routes.

This chapter covers one principled response to intractability: instead of demanding the *exact* optimum, we settle for an answer that is provably *close* to optimal, computed in guaranteed polynomial time. We formalize the approximation ratio, design constant-factor algorithms for vertex cover and metric TSP using matchings and spanning trees, analyze logarithmic bounds for greedy set cover, and culminate in fully polynomial-time approximation schemes (FPTAS).

History and connection

In the wake of Richard Karp's 1972 proof that dozens of foundational optimization problems were NP-complete, industrial computer science confronted an urgent question: if exact polynomial solutions are intractable, can we salvage provable performance guarantees? In 1974, David S. Johnson published *Approximation algorithms for combinatorial problems*, establishing the modern hierarchy of approximation: constant-factor ratios, logarithmic degradations, and polynomial-time approximation schemes (PTAS). In 1976, Nicos Christofides at Carnegie Mellon designed his celebrated 3/2-approximation algorithm for metric TSP by computing a minimum-weight perfect matching on the odd-degree vertices of a minimum spanning tree using Edmonds' blossom algorithm. In the 1990s, the PCP Theorem (Arora, Lund, Motwani, Sudan, Szegedy, 1992) and Johan Håstad's 2001 breakthrough established tight inapproximability bounds: for MAX-3-SAT, approximating beyond $7/8 + \varepsilon$ is NP-hard, meaning random coin-flipping achieves the best possible ratio unless P = NP. Prominent open questions include whether metric TSP can be approximated to 4/3, and Subhash Khot's 2002 Unique Games Conjecture, which implies that a 2-approximation for vertex cover cannot be improved in polynomial time.

34.1 The one number that matters: the approximation ratio

First we need a precise meaning for "close to optimal."

Fix an optimization problem – say, find the *smallest* vertex cover, or the *shortest* tour. Every input instance I has some true optimal cost, which we will write $\mathrm{OPT}(I)$, or just C^* when the instance is clear. We do not know C^*; if we could compute it quickly the problem would not be NP-hard. Our algorithm, run on I, produces some feasible solution of cost C. The whole question of this chapter is: *how far can C be from C^*?*

Definition 34.1 (Approximation ratio)**.** An algorithm A is a *ρ-approximation algorithm* (or "achieves approximation ratio ρ") if for *every* input I the solution it returns, of cost C, satisfies

$$\max\left(\frac{C}{C^*},\ \frac{C^*}{C}\right) \le \rho.$$

We always have $\rho \ge 1$. For a minimization problem this reduces to $C \le \rho\, C^*$; for a maximization problem it reduces to $C \ge C^*/\rho$.

The two-sided max in the definition is just a clever way to write one formula that works for both kinds of problem. For *minimization* you worry about C being too *big*, so $C^*/C \le 1$ and the binding term is C/C^*. For *maximization* you worry about C being too *small*, so $C/C^* \le 1$ and the binding term is C^*/C. Either way ρ measures the worst-case blow-up, and $\rho = 1$ would mean "always exactly optimal."

Intuition

Think of ρ as a *warranty*. A 2-approximation for vertex cover is a promise stamped on the box: "whatever graph you feed me, the cover I hand back is at most twice the size of the perfect cover – guaranteed, no matter how adversarial your input." You do not get the best answer. You get a hard ceiling on how bad the answer can be. For a problem nobody can solve exactly in reasonable time, a hard ceiling is very valuable.

One point here confuses almost everyone the first time.

Common trap

To prove $C \le 2\,C^*$ you seem to need to know C^* – but C^* is exactly the thing you *cannot* compute! How can you bound your distance to a number you will never see?
The resolution drives every proof in this chapter: you never compute C^* directly. Instead, you find a quantity L that you *can* compute or reason about, prove that $L \le C^*$ (so L is a *lower bound* on the optimum), and separately prove that your answer satisfies $C \le 2L$. Chaining these,

$$C \ \le\ 2L \ \le\ 2\,C^*,$$

and you have your guarantee *without ever knowing C^**. Almost every proof in this chapter has exactly this shape: *find a lower bound the optimum cannot beat, then relate your output to that lower bound.*

Figure 34.1 draws this sandwich on a number line. It is the skeleton of nearly every proof to come.

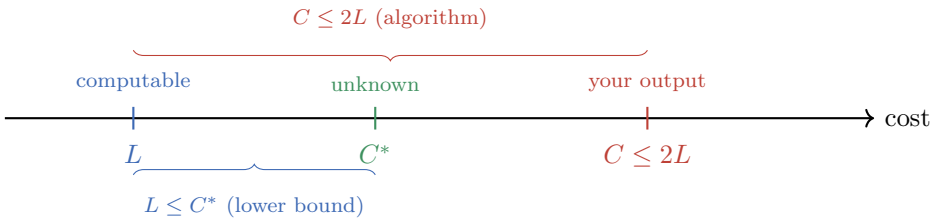


Figure 34.1: The lower-bound sandwich for a minimization problem. You never see C^* (green). You compute a lower bound L (blue) that the optimum cannot fall below, and you prove your output C (red) is at most $2L$. The two brackets together force $C \le 2L \le 2C^*$ without ever measuring the gap to C^* directly.

Focus

Hold onto three things:

- ρ is a worst-case guarantee over *all* inputs, not an average.
- Smaller ρ is better; $\rho = 1$ means exact.
- The trick to proving a ratio is to sandwich the optimum between a computable lower bound and your output. You bound your distance to C^* *through* a proxy you can actually touch.

A few problems are so friendly that you can drive ρ as close to 1 as you are willing to pay for. A family of algorithms A_ε, one for each $\varepsilon > 0$, where A_ε is a $(1+\varepsilon)$-approximation, is called a *polynomial-time approximation scheme* (PTAS) if each A_ε runs in time polynomial in the input size n. If the running time is *also* polynomial in $1/\varepsilon$, it is a *fully polynomial-time approximation scheme* (FPTAS) – the gold standard. We will build one for subset-sum at the end of the chapter. Other problems are so hostile that, unless P = NP, no constant ratio is possible at all – general TSP without the triangle inequality is the classic example, as we will see.

Focus

The hierarchy of approximation guarantees. Not all NP-hard problems behave identically under approximation. Some (like subset-sum and knapsack) admit fully polynomial-time approximation schemes (FPTAS), achieving $(1 + \varepsilon)$ optimality for any $\varepsilon > 0$ in polynomial time. Others (like metric TSP and vertex cover) admit constant-factor approximations, while set cover degrades logarithmically with input size. At the opposite extreme, general TSP without the triangle inequality cannot be approximated within any polynomial ratio unless P = NP. This material corresponds to **CLRS Chapter 35**: the vertex-cover, TSP, set-cover, MAX-3-CNF, and subset-sum results developed here line up with sections 35.1 through 35.5 respectively. Vazirani's *Approximation Algorithms* and Williamson–Shmoys' *The Design of Approximation Algorithms* provide comprehensive advanced treatments.

34.2 Vertex cover: a 2-approximation from a silly idea

A *vertex cover* of an undirected graph $G = (V, E)$ is a set of vertices $S \subseteq V$ such that every edge has at least one endpoint in S. Think of vertices as guards and edges as corridors; a vertex cover

is a placement of guards so that every corridor is watched from at least one end. The *minimum vertex cover problem* asks for the smallest such S. It is NP-hard, so an exact polynomial algorithm is (almost certainly) out of reach.

Here is a heuristic that looks naive but still comes with a guarantee. Repeat the following: pick *any* edge that is not yet covered, throw *both* of its endpoints into your cover, and cross off every edge touching either endpoint. Keep going until no uncovered edges remain.

Algorithm 143: APPROX-VERTEX-COVER(G)

Input: An undirected graph $G = (V, E)$.
Output: A vertex cover S with $|S| \le 2\,|S^*|$.

```
S ← ∅;
E' ← E ;                                   // the still-uncovered edges
while E' ≠ ∅ do
    pick any edge (u, v) ∈ E';
    S ← S ∪ {u, v};
    remove from E' every edge incident to u or to v;
return S;
```

The algorithm is greedy and makes no attempt to choose *which* edge to grab, and it runs in linear time $O(V + E)$ with the right adjacency structures. Let us watch it work.

Example 34.1 (A traced run)**.** Take the graph on six vertices $\{a, b, c, d, e, f\}$ with edges

$$\{a\text{–}b,\ b\text{–}c,\ c\text{–}d,\ c\text{–}e,\ d\text{–}e,\ d\text{–}f,\ e\text{–}f\}.$$

Suppose the algorithm grabs edges in this order.

- Pick b–c. Add b, c to S. This covers a–b, b–c, c–d, c–e. Remaining: d–e, d–f, e–f.
- Pick d–e. Add d, e to S. This covers d–e, d–f, e–f. Remaining: none.

Output $S = \{b, c, d, e\}$, of size 4. What is the true minimum here? It is $\{b, d, e\}$, of size 3, and you can check each edge against it: a–b hits b; b–c hits b; c–d hits d; c–e hits e; d–e hits both; d–f hits d; e–f hits e. Every edge is covered, so $\{b, d, e\}$ is a valid cover, and a short search confirms no cover of size 2 exists (in fact $\{b, d, e\}$ is the *only* cover of size 3). So the algorithm returned 4, the optimum is 3, and $4 \le 2 \cdot 3$ – comfortably inside the warranty.

The picture below shows the key idea; with it, the proof is short.

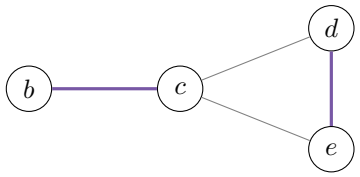


Figure 34.2: The two *purple* edges are the ones the algorithm picked. They share no endpoint – a *matching*. Any cover must spend at least one vertex on each of them, so $|S^*| \ge 2$, while the algorithm spends exactly 2 vertices per picked edge.

Theorem 34.1 (Vertex cover 2-approximation)**.** APPROX-VERTEX-COVER *returns a vertex cover S with $|S| \le 2\,|S^*|$, where S^* is a minimum vertex cover. So it is a 2-approximation algorithm.*

Proof. First, S *is* a vertex cover: the loop only stops when E' is empty, and an edge leaves E' exactly when one of its endpoints enters S. So every edge has an endpoint in S. Good – the output is feasible.

Now the ratio. Let A be the *set of edges* the algorithm picked in the "pick any edge" line (one per iteration). Here is the key observation: *no two edges in A share an endpoint.* Why? When we pick (u, v), we immediately delete every edge touching u or v from E', so no future pick can be incident to u or v. A set of edges with no shared endpoints is called a *matching*. So A is a matching.

Two consequences fall out. First, since each picked edge contributes its two fresh endpoints, $|S| = 2|A|$ exactly. Second – and this is the lower bound from the trap box – *any* vertex cover, including the optimal S^*, must contain at least one endpoint of every edge in A. Because the edges of A are disjoint, those endpoints are all distinct, so S^* needs at least $|A|$ vertices just to cover the matching:

$$|S^*| \;\geq\; |A|.$$

That is our lower bound $L = |A| \leq C^*$. Chain it:

$$|S| \;=\; 2|A| \;\leq\; 2\,|S^*|. \qquad \square$$

The proof follows the recipe exactly. We never computed $|S^*|$. We found a computable quantity ($|A|$, the matching size) that the optimum cannot undercut, and we showed our output is exactly twice it.

Common trap

A tempting "improvement": instead of grabbing both endpoints of an edge, greedily grab the single vertex of *highest degree*, remove its edges, and repeat. It sounds smarter – and on most graphs it does better! But its worst-case ratio is *not* 2; it degrades like $\Theta(\log n)$. The smarter-looking rule loses the guarantee. The "silly" both-endpoints rule is the one with the clean constant guarantee.

Walking question

The factor of 2 comes from spending *two* vertices to cover edges the optimum might cover with *one*. Can you cook up a graph where the algorithm genuinely returns a cover of size $2|S^*|$ – where the bound is *tight*, rather than merely an upper bound? (Hint: a single edge already gives a ratio of 2 if the optimum is 1. Now chain many disjoint edges together.) On your walk, decide whether *any* algorithm could beat ratio 2 here. It is a famous open question whether $2 - \delta$ is possible.

34.3 The traveling salesman with a triangle inequality

A salesman must visit n cities and return home, minimizing total travel. Formally: given a complete graph with a cost $c(u, v) \geq 0$ on each edge, find the cheapest *Hamiltonian cycle* – a tour visiting every vertex exactly once. This is the *traveling salesman problem* (TSP), and it is the poster child of NP-hardness.

In full generality TSP is hopeless to approximate: unless P = NP, no polynomial algorithm achieves *any* constant ratio. (The reason: a good approximation could be used to detect Hamiltonian cycles exactly, which is NP-hard.) So we add one mild, extremely common assumption.

Definition 34.2 (Triangle inequality)**.** The costs satisfy the *triangle inequality* if for all vertices u, v, w,

$$c(u, w) \;\le\; c(u, v) + c(v, w).$$

In words: a direct trip never costs more than a detour through some other city. Any cost that comes from actual distances in a map – Euclidean, driving time, anything "metric" – obeys this automatically. With this assumption, we get a simple 2-approximation built from minimum spanning trees, which you already know well.

Intuition

A *minimum spanning tree* (MST) is the cheapest way to wire all the cities together so they are connected – no cycle, just a skeleton. Because it connects every vertex at minimum total weight, its cost is a *lower bound* on any tour: deleting one edge from any tour leaves a spanning tree. We then turn that skeleton into a valid tour by *walking around it*, and the triangle inequality keeps the shortcuts from increasing the cost. The MST costs no more than the optimal tour, and converting it into a legal tour at most doubles its weight.

Algorithm 144: Approx-TSP-Tour(G, c)

Input: Complete graph G with metric costs c; a root vertex r.
Output: A Hamiltonian cycle of cost $\le 2\,C^*$.

1 compute a minimum spanning tree T of G rooted at r (e.g. Prim);
2 let $H = \langle v_1, v_2, \ldots, v_n \rangle$ be the order in which a *preorder* walk of T first visits the vertices;
3 **return** the cycle that visits the vertices in the order H and returns to v_1;

The idea has three steps. (1) Build the MST T. (2) Imagine walking around the outside of T – a "full walk" that goes down each branch and back up, traversing every tree edge exactly *twice*, costing $2\,c(T)$. (3) That walk repeats vertices, which is illegal for a tour. So *shortcut*: list the vertices in the order the walk *first* meets them (preorder) and go directly between consecutive ones. Shortcutting can only help, thanks to the triangle inequality.

Theorem 34.2 (Metric TSP 2-approximation)**.** *If c satisfies the triangle inequality,* Approx-TSP-Tour *returns a tour of cost at most $2\,C^*$, where C^* is the cost of an optimal tour.*

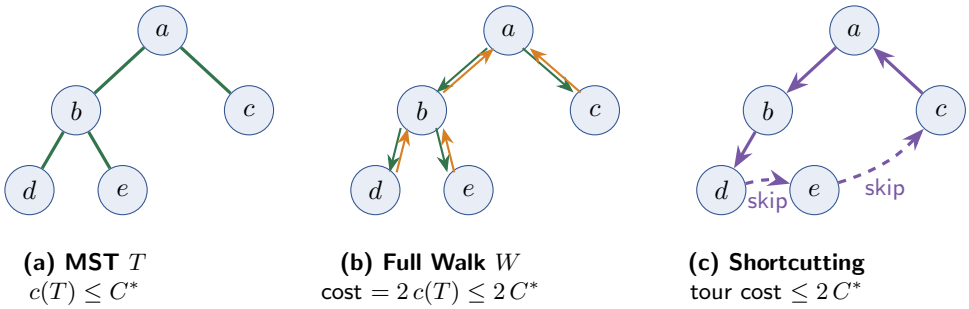
Proof. Step 1 – the MST is a lower bound. Let H^* be an optimal tour, of cost C^*. Delete any single edge from H^* and you are left with a path through all n vertices, which is a spanning tree. Since T is a *minimum* spanning tree and edge costs are nonnegative,

$$c(T) \;\le\; c(H^* \text{ minus one edge}) \;\le\; c(H^*) \;=\; C^*.$$

There is our lower bound: $c(T) \le C^*$.

Step 2 – the full walk costs $2\,c(T)$. Walking down into every branch of T and back up traverses each of the $n - 1$ tree edges exactly twice, for a total of $2\,c(T)$. Call this multiset of moves W.

Step 3 – shortcutting does not increase cost. The walk W visits some vertices more than once. Our tour follows the *first-visit* (preorder) order, so wherever W would revisit an already-seen vertex, our



(a) MST T
$c(T) \le C^*$

(b) Full Walk W
cost $= 2\,c(T) \le 2\,C^*$

(c) Shortcutting
tour cost $\le 2\,C^*$

Figure 34.3: The 2-approximation for metric TSP. **(a)** The minimum spanning tree T satisfies $c(T) \le C^*$ because deleting an edge from an optimal tour leaves a spanning tree. **(b)** Traversing every tree edge twice yields a full walk $W = \langle a, b, d, b, e, b, a, c, a \rangle$ of cost $2\,c(T)$. **(c)** Shortcutting to visited vertices in preorder sequence $\langle a, b, d, e, c, a \rangle$ replaces paths with direct edges; by the triangle inequality, direct shortcuts cannot increase the cost, giving a tour of cost at most $2\,C^*$.

tour skips straight to the next *new* vertex. Each such skip replaces a path $v \rightsquigarrow w$ in W by the direct edge (v, w); by the triangle inequality applied repeatedly, the direct edge costs no more than the path it replaces. Hence the tour's cost is at most the walk's cost.

Chaining the three steps:

$$\text{tour cost} \;\le\; c(W) \;=\; 2\,c(T) \;\le\; 2\,C^*. \qquad \square$$

The proof uses the same recipe: a computable lower bound ($c(T)$, the MST) beneath the optimum, and an output bounded by twice it.

Why this matters

Metric TSP arises directly in vehicle routing, circuit board drilling, DNA fragment assembly, warehouse pick-paths, and the order in which a telescope slews between targets. A guaranteed-within-double tour you can compute in $O(E \log V)$ (the cost of an MST) is the kind of thing that runs inside logistics software shipping millions of packages a day. And the better 1.5-approximation below is more than academic – variants of it are what real solvers warm-start from.

34.3.1 Christofides: squeezing the ratio down to 1.5

The factor of 2 came entirely from *doubling* the tree edges. That was crude. The point of doubling was to make every vertex have *even* degree, because a connected graph where every vertex has even degree contains an *Eulerian circuit* – a closed walk using each edge exactly once – which we can then shortcut into a tour. But doubling *all* edges is overkill; we only need to fix the vertices whose degree in T is currently *odd*.

Christofides' insight (1976): there is always an even number of odd-degree vertices in T. Add a minimum-weight *perfect matching* M on just those odd-degree vertices. Now $T \cup M$ has all even degrees, admits an Eulerian circuit, and shortcuts to a tour. The clever part is bounding $c(M)$: the odd vertices lie on the optimal tour in some cyclic order, which splits into two alternating matchings;

the cheaper of the two costs at most $C^*/2$, and M is the minimum, so $c(M) \le C^*/2$. Therefore

$$\text{tour cost} \;\le\; c(T) + c(M) \;\le\; C^* + \tfrac{1}{2}C^* \;=\; \tfrac{3}{2}\,C^*.$$

Figure 34.4 contrasts the two repairs. Doubling (left) copies *every* tree edge to make all degrees even – safe but wasteful. Christofides (right) only adds a matching on the odd-degree vertices, a much cheaper way to reach the same "all degrees even" state that admits an Eulerian circuit.



doubling: every edge twice matching on odd vertices $\{c, d\}$

Figure 34.4: Two ways to make every degree even so an Eulerian circuit exists. Left: double all green tree edges (cost $2\,c(T)$). Right (Christofides): the only odd-degree vertices of this tree are the leaves c and d; add the one red matching edge c–d, costing at most $C^*/2$. Both reach all-even degrees, but the matching is far cheaper, dropping the ratio from 2 to 3/2.

This 1.5-approximation stood as the best known for metric TSP from 1976 until a tiny improvement to $1.5 - 10^{-36}$ in 2020, which shows how hard further progress has been.

Cost summary

APPROX-TSP-TOUR: build MST in $O(E \log V)$, preorder walk in $O(V)$; overall $O(E \log V)$, ratio 2. Christofides: dominated by the minimum-weight perfect matching, $O(V^3)$, ratio 3/2. General (non-metric) TSP: no constant-ratio polynomial algorithm exists unless P = NP.

34.4 Set cover: when the best you can do is logarithmic

Now a problem where no constant ratio is known to be achievable, and where the natural greedy algorithm's error grows – gently – with the input.

You are given a universe X of n elements and a family $\mathcal{F} = \{S_1, \ldots, S_m\}$ of subsets of X whose union is all of X. A *set cover* is a subfamily $\mathcal{C} \subseteq \mathcal{F}$ whose union is still all of X. You want the *fewest* sets. Vertex cover is the special case where each element is an edge and each set is "the edges touching one vertex," so set cover is at least as hard – NP-hard.

The greedy algorithm is exactly what your gut suggests: repeatedly grab the set that covers the most *still-uncovered* elements.

Algorithm 145: GREEDY-SET-COVER$(X, \mathcal{F})$

Input: Universe X, family $\mathcal{F}$ covering X.
Output: A cover $\mathcal{C}$.

```
U ← X ;                                   // the still-uncovered elements
C ← ∅;
while U ≠ ∅ do
    pick S ∈ F maximizing |S ∩ U|;
    U ← U \ S;
    C ← C ∪ {S};
return C;
```

Example 34.2 (A traced run)**.** Let $X = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$ and

$$S_1 = \{1, 2, 3, 4, 5, 6\}, \quad S_2 = \{5, 6, 7, 8, 9\}, \quad S_3 = \{1, 4, 7, 10\},$$

$$S_4 = \{2, 5, 8, 11\}, \quad S_5 = \{3, 6, 9, 12\}.$$

Greedy picks S_1 first (6 new elements – the most). Uncovered now $\{7, 8, 9, 10, 11, 12\}$. Next S_2 covers $\{7, 8, 9\}$, three new; S_3 covers $\{7, 10\}$, two; S_4 covers $\{8, 11\}$; S_5 covers $\{9, 12\}$. Greedy takes S_2. Uncovered $\{10, 11, 12\}$. Now S_3, S_4, S_5 each cover exactly one remaining element, so greedy needs all three. Result: $\{S_1, S_2, S_3, S_4, S_5\}$, size 5.

But the *optimal* cover is $\{S_3, S_4, S_5\}$ of size 3 – those three sets partition all twelve elements! Greedy picked the large overlapping sets S_1 and S_2 early, leaving three disjoint singletons to cover individually. This instance, generalized, is the classic near-worst case showing the $\ln n$ factor is real, not an artifact of loose analysis.

Figure 34.5 lays the twelve elements out as a grid so you can *see* the trap. The optimal cover $\{S_3, S_4, S_5\}$ is the three green *rows*, which tile the grid perfectly. Greedy ignores that structure and grabs the two fat sets S_1, S_2 (the shaded left-hand columns) first, after which it is stuck mopping up the leftovers one fragment at a time.

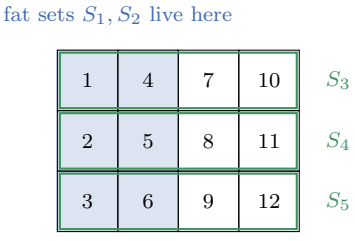


Figure 34.5: Set-cover trap as a grid. The optimum is the three green rows $S_3 = \{1, 4, 7, 10\}$, $S_4 = \{2, 5, 8, 11\}$, $S_5 = \{3, 6, 9, 12\}$, which partition all twelve elements into a clean 3-set cover. Greedy instead seizes the fat sets $S_1 = \{1, \ldots, 6\}$ and $S_2 = \{5, \ldots, 9\}$ (concentrated in the shaded left-hand columns) because they cover the most at first, then must clean up with all of S_3, S_4, S_5 anyway – five sets where three would do.

Theorem 34.3 (Greedy set cover ratio)**.** GREEDY-SET-COVER *returns a cover of size at most* $H(d) \cdot |\mathcal{C}^*|$*, where* $d = \max_i |S_i|$ *is the size of the largest set,* $\mathcal{C}^*$ *is an optimal cover, and* $H(d) = \sum_{k=1}^{d} 1/k \le \ln d + 1$ *is the* d*-th harmonic number. In particular it is an* $(\ln n + 1)$*-approximation.*

Proof idea (the charging argument). This proof uses a different device: *charging*. When greedy picks a set S that newly covers t elements, it pays a cost of 1 (one set). Spread that cost evenly over the newly covered elements: charge each of those t elements a cost of $1/t$. Then the total number of sets greedy picks equals the total charge over all elements: $|\mathcal{C}| = \sum_{x \in X} \text{cost}(x)$.

Now bound the total charge by looking through the lens of the *optimal* cover. Take any optimal set $S^* \in \mathcal{C}^*$ with its elements ordered by when greedy covered them. When greedy was about to cover the k-th element of S^*, that set S^* still had at least $|S^*| - k + 1$ uncovered elements available to greedy – so the set greedy actually chose covered *at least* that many, meaning the per-element charge was at most $1/(|S^*| - k + 1)$. Summing over the elements of S^* telescopes into a harmonic number $H(|S^*|) \le H(d)$. Since the optimal sets cover all of X, summing over $\mathcal{C}^*$ gives

$$|\mathcal{C}| \;=\; \sum_{x} \text{cost}(x) \;\le\; \sum_{S^* \in \mathcal{C}^*} H(d) \;=\; H(d)\,|\mathcal{C}^*|. \qquad \square$$

Focus

Set cover teaches two new lessons. First, not every problem admits a constant ratio; sometimes the true answer is $\Theta(\log n)$, and – by a deep result built on the PCP theorem – that is provably the *best possible* unless P = NP. Second, the *charging argument* is a reusable proof technique: distribute the algorithm's cost onto the elements, then re-collect it through the optimum's structure.

34.5 MAX-3-CNF: how a coin flip guarantees 7/8

The next algorithm does not look at the input at all: it simply *flips coins*.

A *3-CNF formula* is an AND of *clauses*, each clause an OR of exactly three *literals*, where a literal is a variable or its negation, and – to keep things clean – the three literals in a clause involve three *distinct* variables. Example:

$$(x_1 \vee \neg x_2 \vee x_3) \;\wedge\; (\neg x_1 \vee x_2 \vee x_4) \;\wedge\; (x_2 \vee \neg x_3 \vee \neg x_4).$$

Deciding whether *all* clauses can be satisfied at once is 3-SAT, the canonical NP-complete problem. The optimization cousin, *MAX-3-CNF*, asks: over all truth assignments, what is the maximum number of clauses you can make true? This is a *maximization* problem, so our ratio guarantee will read $C \ge C^*/\rho$.

Here is the algorithm in its entirety: set each variable to `true` or `false` independently with probability $1/2$ each. The algorithm never reads the formula.

Theorem 34.4 (Randomized MAX-3-CNF)**.** *Given a 3-CNF formula with m clauses (each over three distinct variables), the random assignment satisfies,* in expectation, *$\frac{7}{8}m$ clauses. Since the optimum can satisfy at most m, this is a randomized $8/7$-approximation.*

Proof. Fix a clause C_j, say $(\ell_1 \vee \ell_2 \vee \ell_3)$ on three distinct variables. The only way the clause comes out *false* is if all three literals are false simultaneously. Each literal is false with probability $1/2$, and because the three variables are distinct, the three events are *independent*, so

$$\Pr[C_j \text{ unsatisfied}] = \left(\tfrac{1}{2}\right)^3 = \tfrac{1}{8}, \qquad \Pr[C_j \text{ satisfied}] = 1 - \tfrac{1}{8} = \tfrac{7}{8}.$$

Define an indicator $Y_j = 1$ if C_j is satisfied, else 0. Then $\mathbb{E}[Y_j] = \Pr[C_j \text{ satisfied}] = 7/8$. Let $Y = \sum_j Y_j$ be the number of satisfied clauses. By *linearity of expectation* – which needs no independence between clauses at all –

$$\mathbb{E}[Y] = \sum_{j=1}^{m} \mathbb{E}[Y_j] = \tfrac{7}{8}m.$$

The optimum $C^* \le m$, so $\mathbb{E}[Y] = \frac{7}{8}m \ge \frac{7}{8}C^*$, i.e. $C^*/\mathbb{E}[Y] \le 8/7$. □

Intuition

Why does blind luck do so well? Each clause is hard to falsify: you must get *all three* of its literals wrong at once, a 1/8 event, so a random assignment satisfies it seven times out of eight. Linearity of expectation does the real work here – it lets us add up the per-clause 7/8 probabilities even though the clauses overlap in messy, dependent ways. We never untangle the dependence; we just sum expectations.

Remark 34.1 (From expectation to certainty). "In expectation" might feel like a dodge – what if your particular coin flips land badly? Two fixes. (1) Run it a few times and keep the best; since the average is $\frac{7}{8}m$, some run hits at least $\frac{7}{8}m$, and the probability of repeatedly falling short shrinks geometrically. (2) More elegantly, *derandomize* by the *method of conditional expectations*: fix the variables one at a time, each time choosing the value that keeps the conditional expected number of satisfied clauses at least $\frac{7}{8}m$. This is always possible (one of the two choices must be at least the average) and yields a *deterministic* assignment satisfying at least $\frac{7}{8}m$ clauses, no luck required.

Walking question

The 7/8 ratio is notable: Håstad proved in 2001 that for MAX-3-SAT no polynomial algorithm beats $7/8 + \delta$ unless P = NP. So a coin flip is, up to an arbitrarily small margin, the *best possible* polynomial-time algorithm. On your walk, consider that the best efficient strategy for this NP-hard problem ignores the formula entirely. When is randomness a feature rather than a fallback?

34.6 Subset-sum: an FPTAS, the gold standard

We close with the most refined possibility: a single dial ε you can turn to make the answer as close to optimal as you wish – 1%, 0.01%, anything – with the running time growing only *polynomially* in $1/\varepsilon$. That is a *fully polynomial-time approximation scheme*, and subset-sum is the cleanest place to meet one.

The problem. Given positive integers $x_1, \dots, x_n$ and a target t, find a subset whose sum is as large as possible *without exceeding* t. (Think: a knapsack where every item's value equals its weight, and the sack holds t.) NP-hard in general.

The exact-but-slow core. Build the set P_i of *all* subset sums achievable using the first i items and not exceeding t. Start with $P_0 = \{0\}$; to get P_i, take every sum s in P_{i-1} and also offer $s + x_i$, keeping only values $\le t$. The answer is the largest element of P_n. Correct – but $|P_i|$ can double each step, blowing up to 2^n sums. Exponential.

The fix: trimming. The lists explode because they hold many sums that are nearly equal. So after each step, *trim*: walk the sorted list and delete any value that is within a factor $(1+\delta)$ of a value you are keeping. Formally, you keep a sparse "representative" list where consecutive kept values differ by more than a factor $1+\delta$. Trimming loses a little accuracy but keeps the lists short, because there are only so many powers of $(1+\delta)$ between 1 and t.

Algorithm 146: APPROX-SUBSET-SUM$(x_1, \ldots, x_n,\ t,\ \varepsilon)$

Input: Positive integers x_i, target t, accuracy $\varepsilon \in (0, 1)$.
Output: A sum $z^* \le t$ with $z^* \ge (1-\varepsilon)\,C^*$.

$L_0 \leftarrow \langle 0 \rangle$;
$\delta \leftarrow \varepsilon/(2n)$;
for $i \leftarrow 1$ **to** n **do**
  $L_i \leftarrow \textsc{Merge}(L_{i-1},\ L_{i-1} + x_i)$; `// add` x_i `to each, merge sorted`
  $L_i \leftarrow \textsc{Trim}(L_i,\ \delta)$; `// drop near-duplicates within` $1+\delta$
  remove from L_i every element $> t$;
return the largest element z^* of L_n;

$\textsc{Trim}(L, \delta)$ scans the sorted L keeping a running "last kept" value last; it keeps the next element y only if $y > \text{last} \cdot (1+\delta)$, then updates last $\leftarrow y$. So every dropped element sits within a $(1+\delta)$ factor of a kept one.

Example 34.3 (Trimming in action)**.** Suppose after a merge the sorted list is

$$\langle 10,\ 11,\ 12,\ 15,\ 20,\ 21,\ 22,\ 23,\ 24,\ 29 \rangle, \qquad \delta = 0.1.$$

Trim keeps 10, then checks 11: since $10 \cdot 1.1 = 11$ and $11 \not> 11$, drop 11; check 12: $12 > 11$, keep it; $12 \cdot 1.1 = 13.2$, so 15 is kept; $15 \cdot 1.1 = 16.5$, so 20 is kept; $20 \cdot 1.1 = 22$, so 21 and 22 are dropped, 23 kept; $23 \cdot 1.1 = 25.3$, so 24 is dropped, 29 kept. Result: $\langle 10, 12, 15, 20, 23, 29 \rangle$. The ten-element list became six, and every deleted value is within 10% of a survivor. Over many steps this is what stops exponential blow-up. Figure 34.6 shows the same trim on a number line.

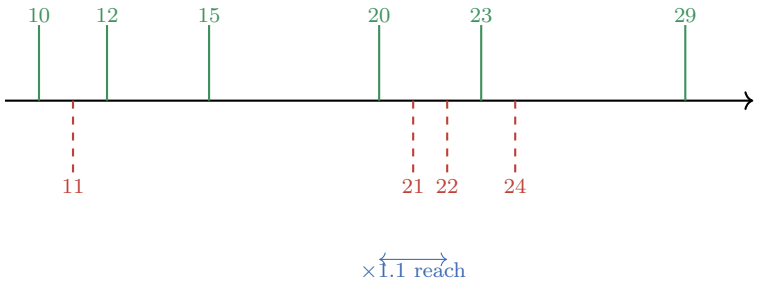


Figure 34.6: Trimming with $\delta = 0.1$. Green ticks (above) are kept; red dashed ticks (below) are dropped because each lies within a factor $1+\delta = 1.1$ of the previous kept value. For instance, keeping 20 shadows out everything up to $20 \cdot 1.1 = 22$, so 21 and 22 vanish and the next survivor is 23. The kept values are spread by more than a factor 1.1 apart, which is what keeps the list short.

Theorem 34.5 (Subset-sum FPTAS)**.** *With $\delta = \varepsilon/(2n)$,* APPROX-SUBSET-SUM *returns z^* with $(1-\varepsilon)\,C^* \le z^* \le C^*$, and runs in time polynomial in both n and $1/\varepsilon$. Hence it is an FPTAS.*

Why it is accurate. The key invariant: *for every achievable sum $y \le t$ that the exact algorithm would*

have in P_i, the trimmed list L_i contains some representative y' with

$$\frac{y}{(1+\delta)^i} \;\le\; y' \;\le\; y.$$

Each trim can shrink a value's nearest representative by at most a factor $1+\delta$, and we trim once per item, so after n items the accumulated loss is at most $(1+\delta)^n$. Apply the invariant to $y = C^*$, the true optimum: the returned z^* satisfies $z^* \ge C^*/(1+\delta)^n$. Now bound the denominator using $1+\delta \le e^\delta$:

$$(1+\delta)^n \le e^{\delta n} = e^{\varepsilon/2} \le 1+\varepsilon,$$

where the last step holds for $\varepsilon \in (0,1]$ (since $e^u \le 1+2u$ for $u \in [0,1/2]$). Therefore

$$z^* \;\ge\; \frac{C^*}{1+\varepsilon} \;\ge\; (1-\varepsilon)\,C^*.$$

And $z^* \le C^*$ because every value in every list is a genuine subset sum that never exceeds t. So the answer is sandwiched as claimed. □

Why it is fast. After trimming, consecutive kept values in L_i differ by more than a factor $1+\delta$. All positive values lie in $[1,t]$, so the number of kept values (plus the element 0) is at most

$$\log_{1+\delta} t + 2 \;=\; \frac{\ln t}{\ln(1+\delta)} + 2 \;\le\; \frac{2\ln t}{\delta} + 2,$$

using $\ln(1+\delta) \ge \delta/2$ for small δ. Substituting $\delta = \varepsilon/(2n)$ makes each list length $O\big(\frac{n \ln t}{\varepsilon}\big)$. Each of the n iterations does work proportional to the list length, so the total is $O\big(\frac{n^2 \ln t}{\varepsilon}\big)$ – polynomial in n, in $\log t$ (the *number of bits* of the input), and in $1/\varepsilon$. That triple "poly" is exactly the FPTAS promise. □

Cost summary

Approx-Subset-Sum: each list has $O\big(\frac{n}{\varepsilon}\ln t\big)$ entries; total time $O\big(\frac{n^2}{\varepsilon}\ln t\big)$. Turning ε down from 0.1 to 0.001 costs a factor of 100 in time – expensive, but *polynomial*, never exponential. That is what "fully" means in FPTAS: $1/\varepsilon$ lives in the polynomial, not the exponent.

Algorithm lab

Trace Approx-Subset-Sum by hand on $x = \langle 1,4,5\rangle$, $t = 10$, $\varepsilon = 0.4$ (so $\delta = 0.4/6 \approx 0.0667$).

1. $L_0 = \langle 0\rangle$.
2. Item 1: merge gives $\langle 0,1\rangle$; trim and cap at 10 keep both. $L_1 = \langle 0,1\rangle$.
3. Item 4: merge $\langle 0,1\rangle$ with $\langle 4,5\rangle$ gives $\langle 0,1,4,5\rangle$; trimming changes nothing ($5/4 > 1.0667$). $L_2 = \langle 0,1,4,5\rangle$.
4. Item 5: merge with $\langle 5,6,9,10\rangle$ gives $\langle 0,1,4,5,6,9,10\rangle$; the two copies of 5 collapse; trim and cap keep the spread-out values. L_3 contains 10.

The true optimum is $1+4+5 = 10 = t$, and the algorithm returns 10. Now redo it with $t = 9$: the optimum is $5+4 = 9$, and watch whether trimming ever costs you the exact answer. Convince yourself the returned value never drops below $(1-\varepsilon)\,C^*$.

34.7 Stepping back: a field of guarantees

We have seen five problems and five different ways to prove "close enough," but the proofs share one idea.

- **Vertex cover** and **metric TSP**: a *constant* ratio of 2, each proved by sandwiching the optimum against a structure you can compute – a maximal matching, an MST.
- **Set cover**: only a *logarithmic* ratio $\Theta(\log n)$ is possible, proved by a charging argument, and that logarithm is provably the floor.
- **MAX-3-CNF**: a *coin flip* achieves a 7/8 fraction in expectation, via linearity of expectation, and 7/8 is essentially optimal.
- **Subset-sum**: a *tunable* $(1+\varepsilon)$ ratio via an FPTAS, proved by tracking accumulated rounding error against the optimum.

Every single proof obeyed the rule from the very first trap box: *find a quantity you can compute that the optimum cannot beat, then relate your output to that quantity.* Matching size, MST cost, harmonic charges, expected satisfied clauses, accumulated trim error – all proxies for the unreachable C^*. That is the one idea to carry out of this chapter and into CLRS Chapter 35.

Why this matters

Many of these ratios are not just the limits of current techniques. The PCP theorem and its descendants prove that $\ln n$ for set cover and 7/8 for MAX-3-SAT cannot be improved unless P = NP, while $(2-\delta)$-approximation for vertex cover is ruled out under the Unique Games Conjecture. Approximation algorithms thus map the boundary of what efficient computation can and cannot achieve.

34.7.1 The trap of uncertified heuristics and non-metric spaces

Several intuitive heuristics look sensible but have poor worst-case guarantees:

- **The nearest-neighbor fallacy in TSP.** The most intuitive greedy heuristic for TSP is to start at a city, always travel to the nearest unvisited city, and finally return home. Even in Euclidean metric spaces, this greedy rule can leave isolated cities until the end, forcing long return trips. Its approximation ratio is $\Omega(\log n)$—it gets strictly worse as the input grows, unlike Christofides' guaranteed 1.5.
- **Non-metric costs.** Inexperienced practitioners often attempt to run metric approximation algorithms on general cost graphs where the triangle inequality does not hold. But without the triangle inequality, TSP cannot be approximated within *any* polynomial-time ratio $\rho \geq 1$ (not even $\rho = 2^n$) unless P = NP. A single violated metric assumption turns a tractable approximation problem into an intractable one.

34.7.2 Practical engineering with certified bounds

Approximation algorithms matter in practice in several ways:

- **Worst-case quality guarantees.** In telecommunications backbone design, circuit layout, and supply-chain logistics, systems cannot rely on uncertified heuristics that might produce erratic routes. Approximation algorithms guarantee worst-case performance bounds that hold on every input.
- **Linear programming relaxations and randomized rounding.** Work on approximation ratios produced two standard tools: relaxing discrete 0-1 integer programs into continuous linear programs (solvable in polynomial time via Chapter 28) and rounding the fractional coordinates back to integers using probabilistic coin flips.
- **Sensitivity to real-world edge costs.** Metric algorithms depend strictly on the triangle inequality $c(u, v) \leq c(u, w) + c(w, v)$ and symmetry $c(u, v) = c(v, u)$. Real delivery networks have one-way streets, congestion delays, and turn penalties that disrupt metric symmetry, requiring engineering adjustments.
- **Inapproximability plateaus.** For many foundational problems, known algorithms have reached provable mathematical ceilings. Under the Unique Games Conjecture, no polynomial-time algorithm can achieve an approximation ratio better than 2 for Vertex-Cover—the matching heuristic of Section 34.2 achieves the best bound theoretically attainable in polynomial time.

Approximation algorithms accept bounded suboptimality to recover polynomial-time guarantees. When an algorithm must handle very large workloads, another option is to divide the work across multiple processing cores. In Chapter 35, we study parallel algorithms: dynamic multithreading, the work and span laws, and the limits of concurrent execution.

34.8 Exercises

Exercise 34.1 (Tight vertex cover)**.** Construct a graph on which Approx-Vertex-Cover returns a cover of size *exactly* $2\,|S^*|$. (Hint: k disjoint edges. What is the minimum cover? What does the algorithm pick?) Then argue that on a star graph $K_{1,k}$ – one center joined to k leaves – the algorithm can return a cover of size 2 while the optimum is 1, so the ratio is 2 even here.

Exercise 34.2 (Order matters?)**.** Approx-Vertex-Cover says "pick any edge." Show that two different choices of edge order can yield covers of *different* sizes on the same graph, yet both still satisfy $|S| \leq 2|S^*|$. Find a small graph exhibiting this.

Exercise 34.3 (Why metric matters)**.** Give a four-city instance of TSP that *violates* the triangle inequality and on which Approx-TSP-Tour returns a tour far worse than twice optimal. (You only need one edge that is wildly cheaper than the detour implies.) Explain in one sentence why the proof of Theorem 34.2 breaks.

Exercise 34.4 (Preorder cost)**.** For the MST in Figure 34.3, write out the full walk and the preorder tour explicitly. Assuming unit-cost tree edges and a metric, give a concrete upper bound on the tour cost in terms of $c(T)$, and check it against the theorem.

Exercise 34.5 (Greedy set cover lower bound)**.** Generalize Example 34.2: design an instance on n elements where the optimum is small (say 2 sets) but greedy is forced to take $\Theta(\log n)$ sets. Describe the construction; you do not need to write out every set.

Exercise 34.6 (MAX-2-CNF coin flip)**.** Repeat the MAX-3-CNF analysis for clauses of exactly *two* distinct literals. What fraction of clauses does a uniformly random assignment satisfy in expectation? State the resulting approximation ratio.

Exercise 34.7 (Conditional expectations)**.** On the three-clause formula in the MAX-3-CNF section, run the method of conditional expectations by hand: decide x_1, then x_2, then x_3, x_4, each time choosing the value that keeps the conditional expected number of satisfied clauses highest. Verify your final assignment satisfies all three clauses (so this formula is fully satisfiable).

Exercise 34.8 (FPTAS error tracking)**.** In Approx-Subset-Sum we set $\delta = \varepsilon/(2n)$. Suppose instead a careless implementer sets $\delta = \varepsilon$ (no division by n). Show that the accuracy guarantee can fail: bound the worst-case loss factor $(1+\varepsilon)^n$ and explain why it is *not* within $(1-\varepsilon)$ of optimal for large n.

Exercise 34.9 (Trim invariant)**.** Prove the invariant used in Theorem 34.5: after i rounds, for every exact achievable sum $y \le t$ there is a kept value $y' \in L_i$ with $y/(1+\delta)^i \le y' \le y$. Use induction on i; the inductive step combines "merging preserves representatives" with "one trim loses at most a factor $1+\delta$."

Exercise 34.10 (Scaling instead of trimming, challenge)**.** There is an alternate FPTAS for the 0/1 knapsack that *scales item values* down by a common factor and then solves exactly by dynamic programming. Look up or reconstruct the idea: if values are divided by $K = \varepsilon\, v_{\max}/n$ and rounded down, argue that the DP runs in time polynomial in n and $1/\varepsilon$ and loses at most an ε fraction of value. Compare its spirit to the trimming approach here.

Chapter 35

Parallel and Multithreaded Algorithms

In the last chapter we tackled computational limits by trading exactness for speed, settling for provably good approximations when NP-hard problems refused to yield. Even polynomial-time algorithms become a bottleneck when an input outgrows what a single processor can execute sequentially. Every algorithm in this book up to now has assumed a sequential model: one processor executing one instruction stream, one step after another. That model matched the machines of the last century, where a single core performed one operation per cycle. Modern hardware, however, is parallel: a phone has eight cores, a workstation has sixty-four, and a data center packs hundreds of thousands. If an algorithm runs strictly sequentially, every core beyond the first sits idle unless the computation exposes independent work.

This chapter shows how to design algorithms that use many processors at once. You already know the most important idea. Divide-and-conquer *naturally* produces independent subproblems — "sort the left half" and "sort the right half" have nothing to say to each other until the very end — and independent work is exactly the kind of work that can be done in parallel. Much of what follows is the divide-and-conquer you already know, now allowed to run its branches side by side.

Parallelism comes with a new bookkeeping problem, though. When work happens at the same time, "how long does this take?" splits into two different questions, and we need two different cost measures to answer them. Most of this chapter is about reasoning with both numbers at once.

Focus

By the end of this chapter you should be able to: (1) describe an algorithm in the *fork-join* model using **spawn**, **sync**, and **parallel for**; (2) compute its *work* T_1 and its *span* T_∞, and read off its *parallelism* T_1/T_∞; (3) bound how fast it runs on P processors using the work and span laws and Brent's theorem; (4) parallelize loops, matrix multiplication, and merge sort, and analyze each by solving a recurrence; and (5) avoid the classic traps — counting work as if it were time, and forgetting that a *race* can silently corrupt your answer.

History and connection

This material corresponds to CLRS (4th edition) Chapter 26, "Parallel Algorithms." For nearly four decades, software enjoyed a free lunch. Under Dennard scaling—the observation that as transistors shrink, their power density stays roughly constant—and Moore's law, single-

threaded CPU clock frequencies doubled every eighteen months, rising from 1 MHz in the early 1970s to over 3 GHz by the early 2000s. Programs ran twice as fast without changing a single line of code. Around 2004, chips hit the "Power Wall": they could no longer dissipate the heat generated by pushing clock frequencies higher without melting. Chipmakers abruptly stopped chasing raw gigahertz and pivoted to putting multiple processing cores on every die.
The theoretical foundations were laid long before this. In 1967, Gene Amdahl formulated *Amdahl's law*, proving that if an algorithm has even a tiny fraction s of serial work, no number of processors can ever speed up the computation by more than $1/s$. In 1974, Richard Brent proved his landmark scheduling theorem: any computation DAG with total work W and longest dependency path (span) D can be scheduled on P processors in time $T_P \leq W/P + D$. Brent's bound shows that the critical path is a limit that adding hardware cannot remove.
The 1980s and 1990s saw fierce architectural debates between massive SIMD (Single Instruction, Multiple Data) machines like Danny Hillis's 65,536-processor Connection Machine CM-2 and MIMD (Multiple Instruction, Multiple Data) shared-memory clusters. While SIMD excelled at uniform vector arithmetic, branch divergence (idle cores whenever threads took different code paths) crippled general algorithms. In 1990, Guy Blelloch introduced the modern *work-span model* on the CM-2, showing that parallel prefix scans compute all prefix sums in $O(n)$ work and $O(\log n)$ span. In 1995, Robert Blumofe and Charles Leiserson at MIT introduced the *Cilk* runtime. Early multithreaded systems choked on a single centralized task queue; Blumofe and Leiserson solved this with decentralized, randomized *work-stealing*, where idle processors steal tasks from the tails of busy processors' double-ended queues. They proved work-stealing achieves Brent's near-optimal bound with strictly bounded stack memory. Today, fork-join scheduling underpins runtimes from Intel oneTBB to Java's ForkJoinPool and Go's runtime, while modern GPU architectures blend SIMD vector lanes with multithreaded scheduling. The frontier remains open: scheduling algorithms that preserve cache locality on deep non-uniform memory access (NUMA) hierarchies without drowning in communication latency.

35.1 Two ways for work to happen at once

There are two kinds of "doing things at the same time," and it helps to keep them apart.

Intuition

Parallelism is many workers doing the *same job faster* by splitting it up. Ten people painting one fence finish in a tenth of the time. This chapter is about parallelism: we have a result to compute, and we want it sooner by spreading the work across processors.
Concurrency is many independent activities that happen to be alive at once — a download, a clock, a spell-checker — coordinated so they do not step on each other. Concurrency is about *managing* simultaneity; parallelism is about *exploiting* it for speed. They share machinery, but their goals are different.

We will also make a clean simplifying assumption, the one CLRS makes, that keeps the analysis tractable: we imagine a friendly scheduler that we never have to program. We describe *what may run in parallel*, and an unseen runtime system decides *which core runs what, and when*. This separation lets us reason about an algorithm's potential speed without drowning in the details of any particular

machine.

35.2 The fork-join model

We need a way to *write down* an algorithm that says "these two pieces may run at the same time." We use three keywords. They are deliberately few.

Focus

spawn — placed before a procedure call, it says: "you *may* run this call in parallel with the statements that come after it." The caller does *not* wait; it keeps going. (It is permission, not a command — the scheduler is free to run it serially if no core is free.)
sync — "wait here until every call I have spawned so far has finished." This is the join. After a sync, every spawned result is available.
parallel for — a loop whose iterations may all run at the same time because they do not depend on one another.

The name *fork-join* is the shape these make. At a **spawn** the thread of control *forks* into two strands; at a **sync** the strands *join* back into one. Every procedure includes an implicit sync at its end: a procedure never returns until all the work it spawned has completed. That single rule keeps the model sane — you can never accidentally use a result before the worker that produced it is done.

Here is the canonical first example, computed two ways. Recall the Fibonacci numbers $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$. The doubly-recursive program below is a *terrible* way to compute Fibonacci (it does exponential work), but it is the perfect specimen for studying parallel structure, because its two recursive calls are independent.

Algorithm 147: P-FIB(n)

Input: A nonnegative integer n.
Output: The nth Fibonacci number F_n.

```
1 if n ≤ 1 then
2     return n
3 x ← spawn P-FIB(n − 1);          // may run in parallel
4 y ← P-FIB(n − 2);                // the caller does this meanwhile
5 sync;                            // wait for the spawned call
6 return x + y;
```

We **spawn** the computation of F_{n-1} and store its eventual result in x. We do *not* wait. The caller immediately marches on and computes F_{n-2} into y itself. Only at the **sync** do we pause to let the spawned strand catch up. After the sync, both x and y hold real values, and we may add them. Drop the sync and we might read x before it was written — a bug we discuss shortly.

Common trap

spawn does not mean "start a thread now and run it." It means "it is *permitted* to run this in parallel." On a one-core machine, or when every core is busy, the scheduler may simply run the spawned call serially, exactly as an ordinary call. This is why a correct fork-join program must produce the same answer whether or not anything actually runs in parallel. If

your answer depends on parallelism actually happening, the program has a bug.

35.2.1 The computation as a graph

The cleanest way to think about a fork-join computation is as a directed graph that the execution traces out. Each vertex is a *strand*: a maximal run of instructions with no spawn, sync, or return in the middle. An edge $u \to v$ means "u must finish before v may begin." A spawn creates a vertex with two out-edges (the spawned strand and the continuation); a sync creates a vertex with several in-edges that all must arrive before it fires. Because work cannot depend on itself, this graph is a *directed acyclic graph* (a DAG); Figure 35.1 shows the smallest interesting one.

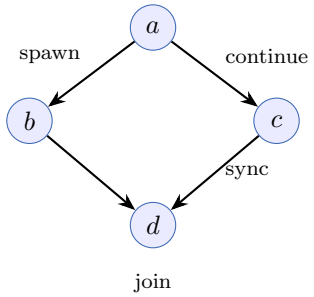


Figure 35.1: One spawn-and-sync as a DAG. Strand a forks into the spawned strand b and the continuation c; both must finish before the join strand d may run. The graph encodes *dependencies*, not a schedule.

This graph is the object we measure. Everything in the analysis below is a statement about it, and two numbers describe it completely for our purposes.

35.3 Work and span: the two numbers that matter

There are exactly two questions you can ask about a parallel computation that do not depend on the messy details of a real machine, and each gives a number.

Intuition

Work T_1: how much *total* computation is there? Equivalently, how long would it take on *one* processor, which must do every strand one after another. It is the sum of the running times of all the strands — the total area of the job, ignoring that some of it could overlap.
Span T_∞: how long is the *longest chain of dependencies*? Equivalently, how long would it take on *infinitely many* processors, where the only thing that can ever slow you down is waiting for something that genuinely had to happen first. It is the length of the longest path through the DAG — the "critical path."

The subscripts are a memory aid: T_1 is the time on 1 processor, and T_∞ is the time on ∞ processors. The work is the *area*; the span is the *depth*. Hold both pictures: a wide, shallow computation has lots of work but a short span (very parallelizable); a tall, skinny one has a long span no matter how many processors you throw at it.

Walking question

Picture a relay race versus a choir. In a relay, runner two cannot start until runner one finishes: the work (total distance) equals the span (longest chain), so extra runners standing around do not help — the dependencies are a single line. In a choir, a hundred singers all sing at once: the work is a hundred voice-parts, but the span is just the length of *one* song. As you walk, classify everyday tasks as "relay-shaped" or "choir-shaped." Which kind benefits from more hands? That is the work–span distinction.

35.3.1 The two laws

These two numbers immediately bound the running time on a real machine with P processors, which we write T_P. Two laws do the work, and both are obvious once stated.

Formal

The work law. P processors can do at most P units of work per time step, so to finish T_1 work takes at least T_1/P steps:

$$T_P \geq \frac{T_1}{P}.$$

The span law. Adding processors can never make you faster than the critical path, which must be walked one strand after another no matter what:

$$T_P \geq T_\infty.$$

Read the work law as "you cannot finish faster than sharing the total work perfectly," and the span law as "you cannot finish faster than the longest unavoidable wait." The actual time T_P is squeezed from below by both. A good parallel algorithm is one where these two lower bounds are also roughly achievable — which is exactly what Brent's theorem, coming up, will promise.

35.3.2 Speedup and parallelism

With both laws in hand, we can define speedup and parallelism.

Definition 35.1 (Speedup and parallelism)**.** The *speedup* of a computation on P processors is the ratio T_1/T_P — how many times faster the P-processor run is than the serial run. The *parallelism* of the computation is

$$\frac{T_1}{T_\infty},$$

the work divided by the span. It is a single number attached to the algorithm itself, with no P in it.

By the work law, $T_1/T_P \leq P$: with P processors you can never go more than P times faster. When the speedup equals P we call it *perfect* (or *linear*) speedup. By the span law, $T_1/T_P \leq T_1/T_\infty$: the speedup can never exceed the parallelism, no matter how many processors you buy. So parallelism is a ceiling. If an algorithm has parallelism 1000, then putting more than about 1000 processors on it does not help, because there is not enough independent work to feed them. We say a computation has *slack* when $P \ll T_1/T_\infty$, meaning there is comfortably more parallelism available than processors to use it, and in that case good schedulers achieve near-perfect speedup.

Focus

Three things to remember:

- **Work** is total area; you minimize it to avoid *wasting* computation.
- **Span** is critical-path depth; you minimize it to avoid *serializing* computation.
- **Parallelism** = work / span is the most processors you can ever usefully employ.

The goal is to keep work low (so you are not slower than the serial program) while *also* keeping span low (so the parallelism is high). These two goals can conflict, and most of the algorithm design in this chapter is about balancing them.

35.3.3 Worked example: the work and span of P-Fib

Let us compute the two numbers for P-FIB(n). Spawn, sync, and the bookkeeping cost only constant time, so the recurrences look just like the serial ones — except that *span* gets to take the longer of the two recursive branches instead of their sum, because those branches run in parallel.

Work $T_1(n)$. On one processor, spawn is just an ordinary call, so the work is exactly the serial running time of double-recursive Fibonacci:

$$T_1(n) = T_1(n-1) + T_1(n-2) + \Theta(1).$$

This is the Fibonacci recurrence itself (plus constants), so $T_1(n) = \Theta(F_n) = \Theta(\phi^n)$ where $\phi = (1+\sqrt{5})/2 \approx 1.618$. The work is exponential. (Again: this is a bad way to compute Fibonacci; we are using it only to study structure.)

Span $T_\infty(n)$. The two recursive calls run *in parallel*, so the critical path goes through whichever one is longer, not through their sum. Since $T_\infty(n-1) \geq T_\infty(n-2)$, the longer branch is the $(n-1)$ one:

$$T_\infty(n) = \max\big(T_\infty(n-1),\, T_\infty(n-2)\big) + \Theta(1) = T_\infty(n-1) + \Theta(1) = \Theta(n).$$

So the span is merely *linear*. The critical path is the chain $n \to n-1 \to n-2 \to \cdots \to 1$, one step per level.

Parallelism.

$$\frac{T_1(n)}{T_\infty(n)} = \frac{\Theta(\phi^n)}{\Theta(n)} = \Theta\left(\frac{\phi^n}{n}\right).$$

This grows enormously with n. For $n = 30$ it is already in the millions. There is a vast amount of independent work here — a side effect of all that wasteful recomputation. High parallelism and low total work are *different* virtues, and an algorithm can have one without the other.

Algorithm lab

Trace P-FIB(4) by hand and build its DAG. The call $P(4)$ spawns $P(3)$ and runs $P(2)$; $P(3)$ spawns $P(2)$ and runs $P(1)$; each $P(2)$ spawns $P(1)$ and runs $P(0)$. Count the calls (that is your work, up to constants): the number of calls is $2F_{n+1} - 1$, which for $n = 4$ is $2 \cdot 5 - 1 = 9$ calls. Now find the longest dependency chain from the top call down to a leaf and back up through the adds: it has length proportional to $n = 4$, not to 9. Seeing "9 calls but a depth-4 critical path" in front of you (Figure 35.2) makes the work–span gap concrete.

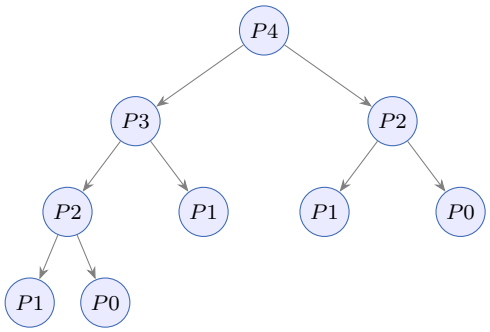


Figure 35.2: The call tree of P-FIB(4): 9 calls in all (that is the work, up to constants), but the longest root-to-leaf chain $P4 \to P3 \to P2 \to P1$ has only 4 nodes. The critical path runs down this spine of left-most (largest-argument) children and back up through the additions, so the span is $\Theta(n)$ while the work is $\Theta(\phi^n)$.

35.4 Scheduling: turning the two numbers into a real running time

We have lower bounds ($T_P \geq T_1/P$ and $T_P \geq T_\infty$). Can we *achieve* something close to them on a real machine, with a scheduler that does not know the future? We can, and a simple greedy rule is enough.

Definition 35.2 (Greedy scheduler)**.** A *greedy* scheduler, at every time step, assigns as many ready strands to processors as it can. A step is *complete* if all P processors are kept busy; it is *incomplete* if fewer than P ready strands exist and some processors idle. Greedy simply never leaves a processor idle when there is ready work for it.

Theorem 35.1 (Greedy scheduling bound)**.** *On P processors, a greedy scheduler runs any computation with work T_1 and span T_∞ in time*

$$T_P \leq \frac{T_1}{P} + T_\infty.$$

Intuition

Classify each time step as *complete* (all P processors busy) or *incomplete* (some idle). There cannot be many complete steps, because each one chews through P units of work and there are only T_1 units total — so at most T_1/P complete steps. There cannot be many incomplete steps either: every incomplete step must make progress along the critical path (it ran every ready strand, so the next strand on the longest remaining path became ready), and the critical path has only T_∞ strands — so at most T_∞ incomplete steps. Add the two buckets: total steps $\leq T_1/P + T_\infty$.

Formal

Proof sketch. Count complete steps. Each uses P work, and total work is T_1, so there are at most $\lfloor T_1/P \rfloor$ of them. Count incomplete steps. Consider the subgraph of strands not yet executed; its longest path starts with a ready strand. On an incomplete step the scheduler runs *all* ready strands (there were fewer than P), so in particular it runs the first strand of that longest path, shortening the remaining critical path by one. The remaining critical path

starts at length T_∞ and cannot drop below 0, so there are at most T_∞ incomplete steps. Every step is one or the other, giving $T_P \le T_1/P + T_\infty$. ■

Figure 35.3 visualizes the two kinds of steps on a concrete DAG with $T_1 = 10$ strands and critical path length $T_\infty = 4$.

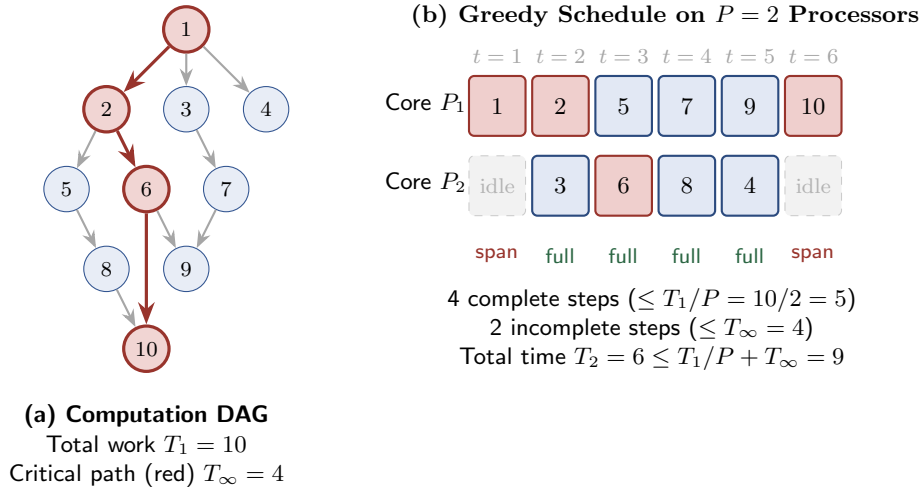


Figure 35.3: The greedy scheduling principle. **(a)** A computation DAG with work $T_1 = 10$ strands and critical path span $T_\infty = 4$ (red spine $1 \to 2 \to 6 \to 10$). **(b)** Execution on $P = 2$ processors. Complete steps ($t = 2, 3, 4, 5$) keep all P cores busy; incomplete steps ($t = 1, 6$) advance the critical path. Total time is bounded by $T_P \le T_1/P + T_\infty$.

This bound, in this clean form, is essentially Graham's and Brent's, and it is the central theorem of the chapter: *a greedy schedule never takes longer than the work term plus the span term.* Compare it to the lower bound $T_P \ge \max(T_1/P, T_\infty)$. The upper and lower bounds differ by at most a factor of 2, so greedy is always within a factor of 2 of optimal. We can do even better when there is slack.

Corollary 35.1 (Near-perfect speedup with slack)**.** *If $P \ll T_1/T_\infty$ (the parallelism comfortably exceeds the processor count), then $T_\infty \ll T_1/P$, so the span term in Theorem 35.1 is negligible and*

$$T_P \approx \frac{T_1}{P}, \qquad \textit{speedup } \frac{T_1}{T_P} \approx P.$$

That is, when there is plenty of slack, a greedy scheduler delivers near-perfect linear speedup.

Why this matters

This is why the work–span model is the model real systems use. A programmer writing in Cilk, Intel TBB, Java's fork-join framework, OpenMP tasks, or Rust's Rayon does *not* schedule threads by hand. They express the spawn/sync structure and let a provably-good greedy (work-stealing) scheduler do the rest. The two numbers you compute on paper — work and span — are exactly the two numbers that predict how the program will scale when the hardware team doubles the core count next year. In effect, you are computing how the running time falls as cores are added.

35.5 Analyzing parallel loops

Most parallel code in practice parallelizes loops rather than binary recursion. A **parallel for** declares that all iterations are independent and may run at once. But "run all n at once" is not literally true on real hardware, and even in the model it hides a cost we must account for. The runtime does not literally fork n strands in one step; it forks them with a *recursive binary splitting* of the index range, like a balanced tournament bracket. The analysis depends on that hidden tree.

Consider the simplest possible parallel loop: add two vectors, $C[i] \leftarrow A[i] + B[i]$ for $i = 1 \ldots n$.

Algorithm 148: VECTOR-ADD(A, B, C, n)

Input: Arrays A, B of length n and an output array C.
Output: $C[i] = A[i] + B[i]$ for all i.

```
1 parallel for i ← 1 to n;
2     C[i] ← A[i] + B[i];
```

Work. There are n iterations, each doing $\Theta(1)$ work, plus the overhead of the splitting tree. The splitting tree that forks n iterations has $\Theta(n)$ leaves and hence $\Theta(n)$ internal nodes, so the bookkeeping is also $\Theta(n)$. Total work $T_1(n) = \Theta(n)$. Good — the same as the serial loop.

Span. If the loop body has span $\Theta(1)$, the span of the whole loop is *not* $\Theta(1)$. To reach n independent iterations the runtime must spawn them through a balanced binary tree of depth $\Theta(\log n)$ (Figure 35.4). You walk *down* the tree (forking) and back *up* (joining), so

$$T_\infty(n) = \Theta(\log n) + \Theta(1) = \Theta(\log n).$$

The $\log n$ is the unavoidable price of distributing the work; the trailing $\Theta(1)$ is the body. If the body itself had span s, the loop's span would be $\Theta(\log n) + s$.

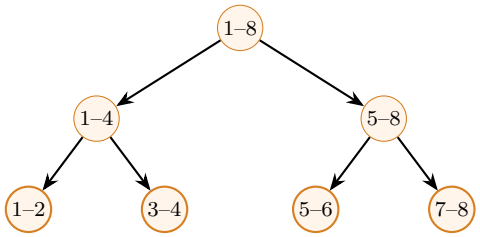


Figure 35.4: How a **parallel for** over 8 iterations is really executed: a balanced splitting tree (shown two levels deep) of depth $\Theta(\log n)$. The leaves are the iterations. Walking down and back up this tree is why the span carries a $\log n$, even when each iteration is $\Theta(1)$.

Parallelism. $T_1/T_\infty = \Theta(n)/\Theta(\log n) = \Theta(n/\log n)$, which is large. For a million-element vector that is on the order of 50,000 — you would need fifty thousand processors before the parallelism ran dry. Parallel loops over big arrays are about as good as parallelism gets.

Common trap

Do not write $T_\infty = \Theta(1)$ for a parallel loop just because each iteration is constant time. The *logarithmic* cost of forking the iterations is real and shows up in the span every single time. Forgetting it is the most common mistake in span analysis. The rule of thumb: *a parallel for adds* $\log(\textit{number of iterations})$ *to the span.*

Walking question

Why a *tree* and not a single fork into n? Imagine one strand trying to spawn a million iterations by itself, one spawn at a time. That spawning loop is serial — it takes a million steps just to hand out the work, so the span would be $\Theta(n)$ and the parallelism would collapse to $\Theta(1)$. The balanced tree lets the spawning itself be parallel: each node spawns only two children, so after $\log n$ levels all n leaves exist. On your walk, convince yourself that *distributing* work is itself work that must be parallelized, or it becomes the bottleneck.

35.5.1 Parallel prefix sums (scan): up-sweep and down-sweep

Consider one of the most fundamental primitives in parallel algorithm design: given an array $A[1 \,..\, n]$, compute the array $S[1 \,..\, n]$ of *prefix sums*, where $S[i] = \sum_{j=1}^{i} A[j]$. The standard serial loop computes $S[i] = S[i-1] + A[i]$, which requires $T_1 = \Theta(n)$ work, but creates an unbroken dependency chain of length $T_\infty = \Theta(n)$ — offering zero parallelism! Because each sum depends directly on the preceding prefix, it appears hopelessly sequential.

We can achieve optimal work and logarithmic span using the **Blelloch scan** algorithm. It proceeds in two balanced-tree sweeps:

1. **Up-sweep (reduce)**: Build a binary sum tree bottom-up. At each level, pairs of adjacent elements are added in parallel. After $\log_2 n$ levels, the root holds the sum of all elements. Work is $\Theta(n)$, span is $\Theta(\log n)$.

2. **Down-sweep (distribute)**: Replace the root with 0 (the additive identity). At each step, a parent holding value v sends v to its left child, and sends $v +$ (saved left partial sum) to its right child. After $\log_2 n$ levels, the leaves hold the exact prefix sums!

Figure 35.5 illustrates both sweeps on an 8-element array.

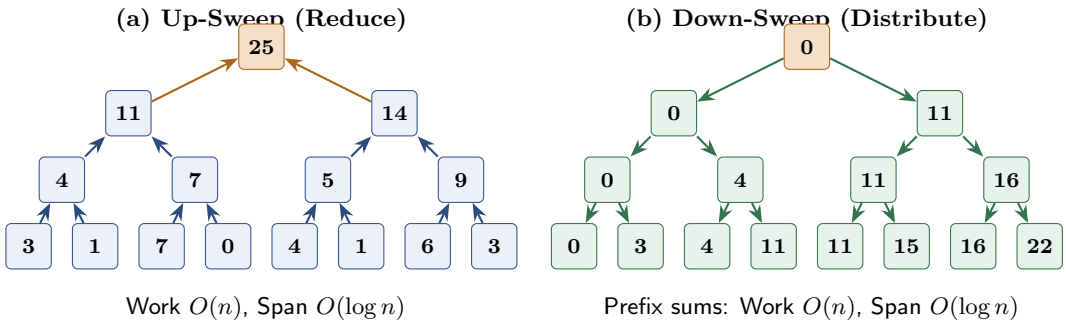


Figure 35.5: Parallel prefix sums (Blelloch scan) on 8 elements. **(a)** The up-sweep builds partial sums in a balanced reduction tree. **(b)** The down-sweep passes prefixes downward: each left child receives the parent's prefix, while each right child receives the parent's prefix plus the left child's stored partial sum. Total work is $\Theta(n)$ and span is $\Theta(\log n)$, yielding massive parallelism $\Theta(n/\log n)$.

35.6 A new hazard: races

Sequential code cannot have a race. Parallel code can, and a race is the most insidious bug in this chapter because the program usually *works* — until, one run in a thousand, it silently gives a wrong answer.

Definition 35.3 (Determinacy race)**.** A *determinacy race* occurs when two strands that are *not* ordered by the dependency DAG both access the same memory location and at least one of them writes to it. Because the scheduler may interleave the two strands in either order, the result can differ from run to run.

Common trap

Here is the classic failure mode. Suppose you tried to sum an array in parallel by having every iteration add into a shared variable:

$$\textbf{parallel for } i \leftarrow 1 \textbf{ to } n: \qquad total \leftarrow total + A[i]$$

The statement $total \leftarrow total + A[i]$ is really three steps: *read total*, *add* $A[i]$, *write total*. If two iterations read the old value *before* either writes, one of the additions is lost. Run it on $A = [1, 1, 1, 1]$ and you might get 4, or 3, or 2, depending on timing. The two writing strands are unordered in the DAG, both touch *total*, and at least one writes — the definition of a race. The fix is to make the updates not collide: split the array, sum the halves into *separate* variables in parallel, sync, then add the two partial sums. That is a parallel *reduction* (Figure 35.6), and it has span $\Theta(\log n)$ with no race.

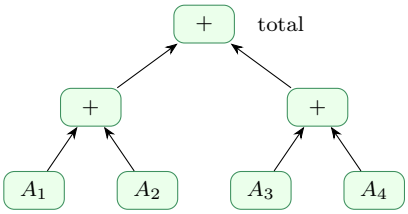


Figure 35.6: Race-free parallel summation (a *reduction*). Pairs of partial sums combine up a balanced tree; siblings are computed in parallel and each "+" node writes its own cell, so no two parallel strands ever touch the same location. The tree has depth $\Theta(\log n)$, giving span $\Theta(\log n)$ with work $\Theta(n)$.

The model gives us a clean way to stay safe: *a fork-join program is race-free if every pair of strands that touch the same location, one of them writing, is ordered by a spawn/sync dependency.* Put differently, never let two *parallel* strands write the same cell, or write-and-read the same cell. In all the algorithms below we are careful to give each parallel branch its *own* disjoint piece of memory to write into — the left half writes the left half of the output, the right half writes the right half — so no two parallel strands ever collide. That discipline is what makes the model's clean analysis legitimate.

35.7 Parallel matrix multiplication

Now we apply everything to a real, useful problem. Multiplying two $n \times n$ matrices, $C = A \cdot B$, sits at the center of scientific computing, graphics, and machine learning, and it parallelizes well because

every output entry can be computed independently.

35.7.1 The straightforward version

Recall the definition $C[i][j] = \sum_{k=1}^{n} A[i][k]\,B[k][j]$. The n^2 output entries do not depend on each other, so compute them all in parallel.

Algorithm 149: P-MATRIX-MULTIPLY(A, B, C, n)

Input: $n \times n$ matrices A, B; output matrix C.
Output: $C = A \cdot B$.

```
1 parallel for i ← 1 to n;
2     parallel for j ← 1 to n;
3         C[i][j] ← 0;
4         for k ← 1 to n do
5             C[i][j] ← C[i][j] + A[i][k] · B[k][j]
```

Notice the inner k-loop is an *ordinary* (serial) for, not a parallel one. That is deliberate: it accumulates into the single cell $C[i][j]$, so making it parallel would create exactly the summation race from the last section. Each (i, j) pair owns its own output cell, so the two outer parallel loops are perfectly safe.

Work. Every cell does $\Theta(n)$ multiply-adds, and there are n^2 cells, so $T_1(n) = \Theta(n^3)$ — identical to the serial algorithm. We waste nothing.

Span. The outer parallel-for contributes $\Theta(\log n)$; the nested parallel-for inside it contributes another $\Theta(\log n)$; and the serial inner loop is a chain of length $\Theta(n)$. The span is the sum of these along the critical path:

$$T_\infty(n) = \Theta(\log n) + \Theta(\log n) + \Theta(n) = \Theta(n).$$

The serial k-loop dominates — its length-n chain is the critical path.

Parallelism. $T_1/T_\infty = \Theta(n^3)/\Theta(n) = \Theta(n^2)$. For $n = 1000$ that is a parallelism of a million. This simple algorithm already scales well. Its only weakness is the $\Theta(n)$ span coming from the serial dot product, and we can shorten that by parallelizing the summation too, with a divide-and-conquer multiply.

35.7.2 Divide-and-conquer multiply, for a shorter span

Split each $n \times n$ matrix into four $n/2 \times n/2$ blocks:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}.$$

Block matrix multiplication gives eight smaller products:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \quad C_{12} = A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \quad C_{22} = A_{21}B_{12} + A_{22}B_{22}. \end{aligned}$$

The eight recursive multiplies are independent (each writes into a scratch area), so spawn all eight; sync; then add the pairs in parallel, as mapped out in Figure 35.7.

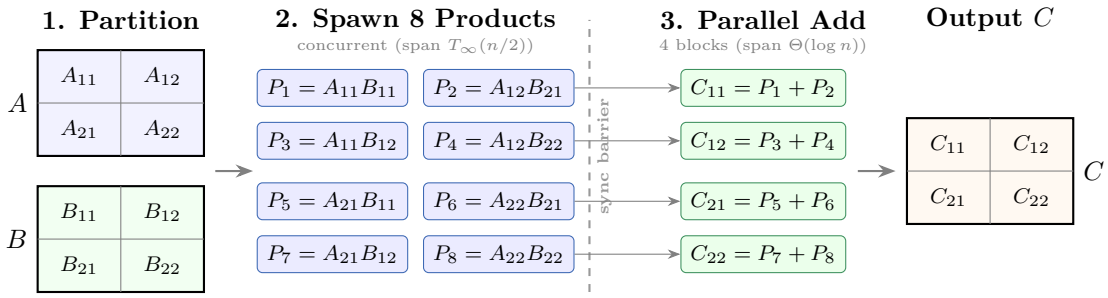


Figure 35.7: Divide-and-conquer parallel matrix multiplication. The inputs A and B are divided into 2×2 blocks. The eight recursive submatrix products $P_1, \ldots, P_8$ are mutually independent and spawned in parallel, incurring only $T_\infty(n/2)$ span. After synchronization, the four block sums C_{ij} are added in parallel with span $\Theta(\log n)$, keeping the total critical-path span at $\Theta(\log^2 n)$.

Work. Eight recursive multiplies of size $n/2$, plus $\Theta(n^2)$ for the four matrix additions:

$$T_1(n) = 8\,T_1(n/2) + \Theta(n^2).$$

By the master theorem, $a = 8$, $b = 2$, $\log_b a = 3$, and $n^{\log_b a} = n^3$ dominates n^2, so $T_1(n) = \Theta(n^3)$ — still cubic, as expected.

Span. The eight multiplies run in parallel, so the span takes *one* recursive call's span, not eight. The block additions can be done with parallel fors over n^2 entries, contributing $\Theta(\log n)$:

$$T_\infty(n) = T_\infty(n/2) + \Theta(\log n).$$

Unrolling: $\Theta(\log n) + \Theta(\log(n/2)) + \cdots$ over $\log n$ levels, each level's term at most $\Theta(\log n)$, giving $T_\infty(n) = \Theta(\log^2 n)$.

Parallelism. $T_1/T_\infty = \Theta(n^3)/\Theta(\log^2 n) = \Theta(n^3/\log^2 n)$. This is much larger than the $\Theta(n^2)$ of the simple version — the span dropped from $\Theta(n)$ to $\Theta(\log^2 n)$ by removing that one serial summation. The work is unchanged and the span is much shorter, which is exactly the design goal.

Cost summary

Parallel matrix multiply, $n \times n$.

Method	Work T_1	Span T_∞	Parallelism
Triple-loop (serial k)	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
Divide-and-conquer (8-way)	$\Theta(n^3)$	$\Theta(\log^2 n)$	$\Theta(n^3/\log^2 n)$

Both do the same total work. The divide-and-conquer version has the same work and a much shorter critical path, hence far more parallelism.

35.8 Parallel merge sort

Sorting is a useful case study because the obvious parallelization is only moderately good, and fixing it shows an important point: *sometimes you must parallelize the combine step too.*

35.8.1 The naive parallelization, and its disappointing span

Merge sort divides, sorts the two halves, and merges. The two halves are independent — spawn them.

Algorithm 150: P-MERGE-SORT(A, lo, hi)

Input: Array A and bounds $lo \le hi$.
Output: $A[lo \dots hi]$ sorted in place.

```
1 if lo ≥ hi then
2     return
3 mid ← ⌊(lo + hi)/2⌋;
4 spawn P-MERGE-SORT(A, lo, mid);          // sort left half in parallel
5 P-MERGE-SORT(A, mid + 1, hi);            // sort right half here
6 sync;
7 MERGE(A, lo, mid, hi);                   // serial merge: the bottleneck
```

Work. Two recursive sorts of size $n/2$ plus a serial merge of $\Theta(n)$:

$$T_1(n) = 2\,T_1(n/2) + \Theta(n) = \Theta(n \log n),$$

by the master theorem ($a = b = 2$, $\log_b a = 1$, the n^1 and the merge tie, so a log factor appears). Same as serial merge sort — no work wasted.

Span. The two recursive sorts run in parallel, so the span uses one of them; but the MERGE is serial, contributing $\Theta(n)$:

$$T_\infty(n) = T_\infty(n/2) + \Theta(n) = \Theta(n).$$

That last recurrence solves to $\Theta(n)$ because the $\Theta(n)$ merge at the *top* level dominates: $n + n/2 + n/4 + \cdots = \Theta(n)$.

Parallelism. $T_1/T_\infty = \Theta(n \log n)/\Theta(n) = \Theta(\log n)$. That is *terrible*. A parallelism of only $\log n$ means even for a billion elements you can usefully employ only about 30 processors. The cause is the serial MERGE: its $\Theta(n)$ span limits the whole algorithm. To get real parallelism, we must learn to *merge in parallel*.

35.8.2 Merging two sorted arrays in parallel

How do you merge $X[1 \dots n_1]$ and $Y[1 \dots n_2]$ (both sorted) in parallel? The serial merge walks a single pointer down the output, which is inherently a chain of length n. We need a divide-and-conquer merge instead.

Intuition

The pivot trick. Take the *median* element of the larger array, say $X[q]$ where q is the middle index of X. Everything in the merged output that is less than $X[q]$ comes from the front of X *and* the front of Y; we find where $X[q]$ belongs in Y by *binary search*, splitting Y at some position q'. Now we have two completely independent merge problems — merge X's left part with Y's left part, and X's right part with Y's right part — and we know exactly where each result goes in the output. Spawn both. No race: the left merge writes the left output, the right merge writes the right output. Figure 35.8 shows the split.

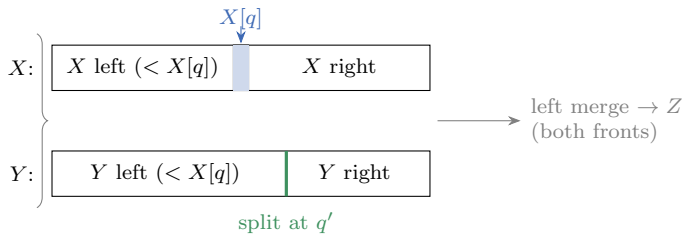


Figure 35.8: The P-MERGE pivot trick. The median $X[q]$ of the longer array X is located inside Y by binary search (position q'). Everything to the left of both splits (less than $X[q]$) becomes one independent merge subproblem; the two right parts become the other. The two subproblems write into disjoint output regions, so they may be spawned without a race.

Algorithm 151: P-MERGE$(X, p_1, r_1, Y, p_2, r_2, Z, p_3)$

Input: Sorted ranges $X[p_1 \ldots r_1]$, $Y[p_2 \ldots r_2]$; output array Z starting at index p_3.
Output: The merged sorted sequence written into $Z[p_3 \ldots]$.

```
1  n1 ← r1 − p1 + 1;   n2 ← r2 − p2 + 1;
2  if n1 < n2 then
3      swap the two input ranges so that X is the longer one;
4  if n1 = 0 then
5      return ;                                          // both ranges empty
6  q ← ⌊(p1 + r1)/2⌋;                                   // median of the longer range X
7  q′ ← BINARY-SEARCH(X[q], Y, p2, r2);   // first index in [p2, r2 + 1] with Y[q′] ≥ X[q]
8  q3 ← p3 + (q − p1) + (q′ − p2);                       // output position for X[q]
9  Z[q3] ← X[q];
10 spawn P-MERGE(X, p1, q − 1, Y, p2, q′ − 1, Z, p3);                  // left halves
11 P-MERGE(X, q + 1, r1, Y, q′, r2, Z, q3 + 1);                        // right halves
12 sync;
```

Here BINARY-SEARCH(x, Y, p_2, r_2) returns the first index q' in the range $[p_2, r_2 + 1]$ with $Y[q'] \geq x$; if no such element exists it returns $r_2 + 1$. The elements $Y[p_2 \ldots q' - 1]$ are exactly those strictly below $X[q]$, so $q' - p_2$ of Y's elements precede $X[q]$ in the output. Adding the $q - p_1$ elements of X that precede $X[q]$ gives its output slot q_3. The left recursive call then merges everything below $X[q]$ (the front of both ranges), the right call merges everything at or above it, and the two write into disjoint output regions $Z[p_3 \ldots q_3 - 1]$ and $Z[q_3 + 1 \ldots]$ — so they cannot race.

Span of P-Merge. Each call does a binary search ($\Theta(\log n)$) and then recurses on two independent subproblems in parallel. By always pivoting on the median of the *larger* array, we guarantee each recursive call handles at most $3/4$ of the total elements. To see why, let X be the longer range, so $|X| \geq n/2$ and $|Y| \leq n/2$. Splitting X at its median sends at most $|X|/2$ of its elements to either side, and the worst case dumps *all* of Y onto one side. So a recursive call gets at most $|X|/2 + |Y| = (n - |Y|)/2 + |Y| = n/2 + |Y|/2 \leq n/2 + n/4 = 3n/4$. So

$$T_\infty^{\text{merge}}(n) = T_\infty^{\text{merge}}(3n/4) + \Theta(\log n) = \Theta(\log^2 n).$$

The recursion has depth $\log_{4/3} n = \Theta(\log n)$, and each level costs $\Theta(\log n)$ for its binary search, multiplying to $\Theta(\log^2 n)$.

Work of P-Merge. One can show (it takes a little care because the split is uneven) that the work

is still $T_1^{\text{merge}}(n) = \Theta(n)$ — the binary searches sum to only $\Theta(n)$ across the whole recursion, the same as the serial merge up to constants.

35.8.3 Putting it together: a genuinely parallel merge sort

Now replace the serial MERGE in P-MERGE-SORT with P-MERGE. (A clean implementation sorts into an auxiliary array so the parallel merge has separate input and output regions and stays race-free.) The recurrences become:

Work. Same shape, $T_1(n) = 2\,T_1(n/2) + \Theta(n) = \Theta(n \log n)$. The parallel merge does the same total work as the serial one, so the work is unchanged. Good — we are not paying for the speedup in wasted computation.

Span. Now the merge contributes only $\Theta(\log^2 n)$, not $\Theta(n)$:

$$T_\infty(n) = T_\infty(n/2) + \Theta(\log^2 n).$$

Unrolling over $\log n$ levels, the per-level cost grows like $\log^2$ of the current size, and the sum is dominated by the top levels; it solves to $T_\infty(n) = \Theta(\log^3 n)$.

Parallelism. $T_1/T_\infty = \Theta(n \log n)/\Theta(\log^3 n) = \Theta(n/\log^2 n)$. Compare this with the naive version's $\Theta(\log n)$. We went from "about 30 useful processors for a billion elements" to "about a million" (for $n = 10^9$, $\log_2 n \approx 30$, so $n/\log^2 n \approx 10^9/900 \approx 10^6$). *Same work, span slashed from* $\Theta(n)$ *to* $\Theta(\log^3 n)$.

Cost summary

Parallel merge sort, n elements.

Version	Work T_1	Span T_∞	Parallelism
Serial merge	$\Theta(n \log n)$	$\Theta(n)$	$\Theta(\log n)$
Parallel merge	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n/\log^2 n)$

The single change — merging in parallel instead of serially — raises the parallelism by roughly a factor of $n/\log^3 n$. The lesson: *a serial combine step can be the entire bottleneck, and parallelizing it is where the largest gains are.*

Focus

The same pattern appeared twice (matrix multiply, then merge sort):

1. Parallelize the obvious independent parts (the recursive calls). Work stays the same; span improves only if the *combine* step is cheap.
2. Discover that a leftover *serial* step (the dot product; the merge) dominates the span and chokes the parallelism.
3. Parallelize *that* step too, with its own divide-and-conquer. The span drops, the parallelism rises, and the work is unchanged.

Whenever a parallel algorithm disappoints, look for the serial step hiding in the combine. It is almost always there.

35.9 Brent's theorem, and the right way to think about it

The last result ties the model together and tells you, before you ever run anything, roughly how fast your algorithm will go on any machine you might buy. We already proved the working form (Theorem 35.1); now we name it and look at what it means.

Theorem 35.2 (Brent's theorem)**.** *Any computation with work T_1 and span T_∞ can be executed on P processors in time*

$$T_P \le \frac{T_1 - T_\infty}{P} + T_\infty.$$

This is a slightly sharper cousin of the greedy bound (the T_∞ work on the critical path is counted once, on the span side, rather than being divided again by P). For our purposes the looser $T_P \le T_1/P + T_\infty$ says everything important, and we will reason with it.

Intuition

Brent's bound has two terms:

- The term T_1/P is the *work term*: if you had no dependencies at all, P processors would split the T_1 work perfectly and finish in T_1/P.
- The term T_∞ is the *span term*: the unavoidable serial spine that no number of processors can shorten.

For small P, the work term T_1/P is large and dominates — you are *work-bound*, and buying more processors helps a lot (speedup is nearly linear). As P grows, T_1/P shrinks until it sinks below T_∞; past that point you are *span-bound*, and buying more processors does almost nothing. The crossover happens right around $P \approx T_1/T_\infty$ — which is exactly the *parallelism*. That is the precise sense in which parallelism is "the number of processors worth buying." Figure 35.9 draws the two regimes.

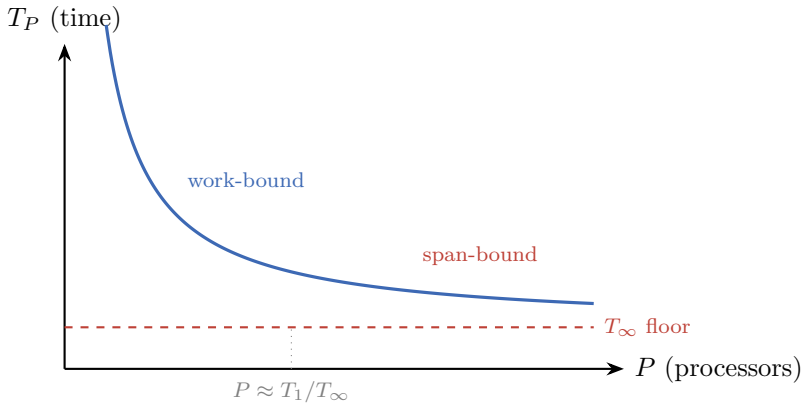


Figure 35.9: Running time T_P versus processor count P under Brent's bound. While the work term T_1/P dominates (left), time falls like $1/P$ — nearly linear speedup. Once it sinks to the span floor T_∞ (right), more processors stop helping. The knee sits at the parallelism, $P \approx T_1/T_\infty$.

Why this matters

This is why chip vendors and cloud providers care about an algorithm's span as well as its work. Two algorithms with identical work $\Theta(n \log n)$ can have wildly different spans ($\Theta(n)$ versus $\Theta(\log^3 n)$ for our two merge sorts), and that difference *is* the difference between a program that stops scaling at 30 cores and one that keeps scaling to millions. When someone says "this code does not scale," they almost always mean its span is too large relative to its work. You now have the vocabulary to diagnose and fix exactly that.

Remark 35.1 (A reminder about what these bounds hide). The model deliberately ignores real costs that matter in practice: the time to *communicate* data between cores, the contention for shared memory and caches, and the overhead of the scheduler itself. A computation that is good in work and span can still be slow if every strand fights over the same cache line, or if the spawned pieces are so tiny that scheduling them costs more than running them (which is why real implementations stop recursing and switch to serial code below some grain size). The work–span model tells you about an algorithm's *inherent* parallelism — a necessary condition for scaling, and the right first thing to analyze — but it is only a starting point.

35.10 Summary

You learned to describe computations with three keywords (**spawn**, **sync**, **parallel for**) and to view the result as a directed acyclic graph (DAG) of strands. You measure that DAG with two fundamental numbers: **work** T_1 (total operations, the serial running time) and **span** T_∞ (critical-path depth, the time on an infinite number of processors). Their ratio is the **parallelism** T_1/T_∞, the maximum number of processors the algorithm can fruitfully keep busy.

The core ideas to carry forward:

- **The work and span laws.** $T_P \geq T_1/P$ and $T_P \geq T_\infty$. You can never beat the total work divided by the processors, and you can never beat the critical path.
- **Greedy scheduling and Brent's bound.** A simple greedy scheduler guarantees $T_P \leq T_1/P + T_\infty$, remaining within a factor of two of optimal. When parallelism comfortably exceeds processor count ($P \ll T_1/T_\infty$), the span term becomes negligible and speedup is nearly linear.
- **Parallel loops cost a logarithm.** A **parallel for** splits recursively like a tournament bracket, adding $\Theta(\log n)$ to the span.
- **Races and memory hazards.** Parallelism introduces nondeterminacy when concurrent strands access the same memory location and at least one writes. Avoid races by privatizing state or using tree reductions.
- **Parallelize the combine step.** In divide-and-conquer, spawning subproblems is easy. The real bottleneck is usually the serial combine phase; parallelizing the combine (as in parallel merge or prefix scan) is what turns merge sort's $\Theta(n)$ span into $\Theta(\log^3 n)$.
- **Hardware limits in practice.** Real multicore execution faces Amdahl's law for any unparallelized serial fraction, cache-coherence traffic (false sharing) across shared cache lines, and memory bus saturation when cores demand more bandwidth than DRAM can deliver.

The parallel matrix multiplication and tree reductions built here are core operations in modern

machine learning, which we examine next in Chapter 36. After that, we turn to hardware: how caches, pipelines, and memory hierarchies execute these DAGs (Chapter 37), how optimizing compilers map high-level algorithms down to machine instructions (Chapter 38), and how quantum computers use superposition as a different kind of parallelism (Chapter 39).

Exercises

Exercise 35.1 (Warmup: read the two numbers)**.** For each, state the work, span, and parallelism in Θ notation. (a) A single **parallel for** of n iterations, each $\Theta(1)$. (b) A serial loop of n iterations, each $\Theta(1)$. (c) A balanced binary tree of n leaves where every node does $\Theta(1)$ work and a node's two children are spawned in parallel.

Exercise 35.2 (Span by inspection)**.** Consider P-Fib(5). Without computing Fibonacci, draw the dependency DAG, mark a longest path from the root to a leaf and back, and confirm its length is $\Theta(n)$ with $n = 5$. How many total calls (strands) are there? Relate your two counts to T_1 and T_∞.

Exercise 35.3 (The forgotten logarithm)**.** A student claims a **parallel for** of n constant-time iterations has span $\Theta(1)$ "because all iterations run at once." Explain precisely why the true span is $\Theta(\log n)$, referring to the splitting tree. Then give the span of two *nested* parallel fors, each of length n, with a $\Theta(1)$ body.

Exercise 35.4 (Spot the race)**.** You want the maximum of an array $A[1 \ldots n]$. A colleague writes: **parallel for** i: **if** $A[i] > m$ **then** $m \leftarrow A[i]$, where m starts at $-\infty$. Exhibit a concrete input and interleaving on two processors that returns the wrong answer. Then describe a race-free parallel algorithm for the maximum, and give its work and span.

Exercise 35.5 (Greedy bound arithmetic)**.** A computation has $T_1 = 10^9$ and $T_\infty = 10^4$. (a) What is its parallelism? (b) Use $T_P \le T_1/P + T_\infty$ to estimate T_P for $P = 100$, $P = 10^4$, and $P = 10^6$. (c) For which of these P is the speedup close to linear, and why does it stop being so?

Exercise 35.6 (Why the inner loop stays serial)**.** In P-Matrix-Multiply, explain what goes wrong if you make the inner k-loop a **parallel for**. Then propose a *race-free* way to parallelize the dot product anyway (hint: a parallel reduction / tree of partial sums) and state the span it would give the inner loop.

Exercise 35.7 (Block multiply recurrences)**.** For the divide-and-conquer matrix multiply, derive $T_1(n) = 8T_1(n/2) + \Theta(n^2)$ and $T_\infty(n) = T_\infty(n/2) + \Theta(\log n)$ from the pseudocode structure, explaining where each term comes from. Solve both (master theorem for the work, unrolling for the span) and state the parallelism.

Exercise 35.8 (The merge is the bottleneck)**.** For naive P-Merge-Sort (serial merge), solve $T_\infty(n) = T_\infty(n/2) + \Theta(n)$ carefully, showing the geometric sum $n + n/2 + n/4 + \cdots = \Theta(n)$. Then explain in one sentence why this single $\Theta(n)$ term, and not the recursion, is what caps the parallelism at $\Theta(\log n)$.

Exercise 35.9 (Why pivot on the larger array)**.** In P-Merge, we always take the median of the *longer* of the two input ranges. Prove that with this choice each recursive call receives at most $3/4$ of the total elements. Then explain why pivoting on the *shorter* array could fail to give a good span.

Exercise 35.10 (Challenge: parallel prefix sums)**.** The *prefix sums* (scan) of $A[1 \ldots n]$ is the array S with $S[i] = A[1] + A[2] + \cdots + A[i]$. The obvious serial loop has span $\Theta(n)$. Design a parallel

algorithm with work $\Theta(n)$ and span $\Theta(\log n)$. (Hint: an up-sweep that builds partial sums in a tree, then a down-sweep that pushes prefixes back down. Describe both passes and give a recurrence for each measure.)

Exercise 35.11 (Challenge: when parallelism cannot help)**.** Give an example of a natural computation whose span equals its work, $T_\infty = \Theta(T_1)$, so its parallelism is $\Theta(1)$ and no number of processors speeds it up. Explain what feature of the problem forces the long dependency chain. (Think about a computation where each step truly needs the previous step's result.)

Exercise 35.12 (Challenge: grain size)**.** Real implementations stop spawning and switch to serial code once subproblems are smaller than a *grain size* g. Argue informally that coarsening a parallel loop to chunks of size g changes its span to $\Theta(g + \log(n/g))$ while keeping work $\Theta(n)$. For what range of g is the parallelism still $\Theta(n/\log n)$ up to constants, and why might a real system deliberately choose a larger g anyway?

Chapter 36

Machine-Learning Algorithms

In Chapter 35 we learned how to distribute computation across many cores at once, parallelizing loops, matrix multiplications, and divide-and-conquer combines. Throughout those chapters, we wrote algorithms that *know* what to do: you told sorting how to compare, you told Dijkstra how edges relax, and you told dynamic programming exactly which subproblems to solve. Every line of pseudocode was a rule you supplied.

This chapter takes a different approach. Here we will write algorithms that do *not* start out knowing what to do. Instead, we hand them a pile of examples and a way to measure how badly they are doing, and we let them *adjust themselves* until they do well. The rule is no longer something you write down by hand; it is something the algorithm *finds*.

The marketing around "machine learning" can make this sound mysterious, but the three core ideas of this chapter – clustering, gradient descent, and the perceptron – are ordinary algorithms in the exact sense of everything you have already read. They have inputs, outputs, loops, invariants, and running times. The only genuinely new idea is that the *objective* – the thing we are optimizing – is now a quantity defined by data, and "solving the problem" means "making that quantity small."

Focus

The main idea of the chapter: *learning is optimization over data.* You define a number that measures "how wrong am I" (a **loss** or **cost**), and then you run an algorithm that pushes that number down. Everything else – k-means, gradient descent, the perceptron – is a specific choice of *what* number and *how* to push it down.

History and connection

The idea of machines that learn from examples rather than fixed instructions goes back to the early years of postwar computing. In 1957, Frank Rosenblatt at the Cornell Aeronautical Laboratory built the *Mark I Perceptron*, an early attempt to instantiate biological neural computation in custom electronic hardware. The machine was an imposing room-sized apparatus: a camera with a 20×20 grid of cadmium-sulfide photocells served as the retina, connected to hundreds of motor-driven potentiometers (variable resistors that adjust electrical current) that physically rotated to adjust their analog weights during training. Rosenblatt proved the Perceptron Convergence Theorem (formalized by Albert Novikoff in 1962), establishing that

if a linearly separable boundary exists, the machine is guaranteed to find it in a finite number of mistake updates.
That same year, in 1957, Stuart Lloyd at Bell Telephone Laboratories was grappling with pulse-code modulation (PCM)—converting continuous analog human voice waveforms into discrete digital pulses for transmission over telephone wires. To minimize distortion, Lloyd needed to quantize continuous signal voltages into a finite set of representative voltage levels. He proposed an alternating algorithm: partition the voltage range into nearest-neighbor cells, then update each code level to the centroid (the average value) of its cell. That memorandum became the classic k-means algorithm (formally published in 1982), which remains the standard method for unsupervised clustering across science and industry.
Closely related to both the perceptron and k-means is an optimization method first described by Augustin-Louis Cauchy in 1847: the method of steepest descent, taking small steps opposite the gradient to minimize a function. Yet the field was nearly extinguished in 1969 when Marvin Minsky and Seymour Papert published *Perceptrons*, mathematically proving that a single-layer perceptron cannot even learn the simple non-linear XOR (exclusive OR) function. The resulting disillusionment triggered the first "AI Winter," freezing neural network research funding for nearly two decades until David Rumelhart, Geoffrey Hinton, and Ronald Williams demonstrated in 1986 that multi-layer networks trained by backpropagation (calculus's chain rule applied to layers of weights) easily learn non-linear representations. Today, learning algorithms drive automated speech, vision, and scientific modeling, though open theoretical questions remain: why heavily overparameterized deep networks generalize so well instead of merely memorizing their training data, and how to guarantee safety on non-convex loss surfaces riddled with saddle points.

36.1 Learning as optimization: the one idea

Let us make the slogan concrete before we touch any specific method.

Imagine you are given a bunch of data points and a *model*: a function f_θ with some adjustable knobs collected into a vector θ (the Greek letter "theta," our standard name for "the parameters"). Turning the knobs changes the function. For each setting of θ you can ask: how well does f_θ fit the data? You answer that with a single number, the **cost** (or **loss**) $J(\theta)$. Low cost means good fit; high cost means bad fit.

Intuition

Picture J as a curved surface. The horizontal directions are the knob settings θ; the height above each point is the cost $J(\theta)$. A *good* model is a *low* point on this surface – a valley. Learning is just *walking downhill*. You start somewhere, look around for a downward direction, take a step, and repeat until the ground is flat under your feet. That flat spot is your trained model.

Formal

A **supervised learning problem** is specified by:

- a dataset of n examples $(x^{(1)}, y^{(1)}), \ldots, (x^{(n)}, y^{(n)})$, where each $x^{(i)} \in \mathbb{R}^d$ is an *input* (a

vector of d *features*) and each $y^{(i)}$ is a *label*;

- a model family $\{f_\theta : \theta \in \mathbb{R}^p\}$, indexed by a parameter vector θ;
- a per-example loss $\ell(f_\theta(x), y) \geq 0$.

The training objective is the average loss over the data,

$$J(\theta) \;=\; \frac{1}{n}\sum_{i=1}^{n} \ell\big(f_\theta(x^{(i)}),\, y^{(i)}\big),$$

and *to train* means to (approximately) solve $\theta^\star = \arg\min_\theta J(\theta)$. In *unsupervised* problems (like clustering) there are no labels y; the cost is defined purely in terms of the inputs.

That is the whole framework, visualized as a closed optimization loop in Figure 36.1. Now we just need (1) ways to write down useful costs, and (2) algorithms to minimize them. We will meet both in three flavors.

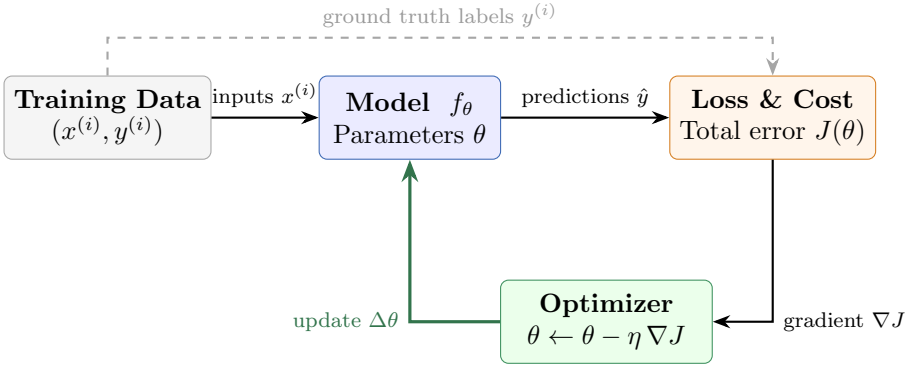


Figure 36.1: The supervised machine learning loop. Inputs $x^{(i)}$ flow forward through the parameterized model f_θ to produce predictions $\hat{y}^{(i)}$. The loss evaluates error against true labels $y^{(i)}$ and aggregates into total cost $J(\theta)$. An optimizer computes gradients $\nabla J(\theta)$ and feeds updated parameters back into the model, driving iterative descent.

This connects to earlier chapters: greedy algorithms made a locally best choice each step; gradient descent makes a locally downhill choice each step. Dynamic programming reused subproblem answers; here we reuse the *current* parameters as the starting point for the next improvement. Optimization has appeared throughout the book.

36.2 Clustering with k-means

We begin with an *unsupervised* problem because it shows the "define-a-cost-then-minimize-it" pattern at its cleanest.

You are handed n points in space and no labels at all – just the points. You suspect they fall into natural groups. Maybe they are customers and the groups are buying habits; maybe they are pixels and the groups are colors. You want to discover k groups (**clusters**) and a representative point for each (a **centroid**).

Intuition

k-means answers two linked questions:

1. *If I knew the centroids*, which cluster does each point belong to? Easy: assign each point to its *nearest* centroid.
2. *If I knew the assignments*, where should each centroid sit? Easy: at the *average* (the "mean") of its assigned points.

Neither is known up front, so we guess, then alternate: re-assign, re-center, re-assign, re-center. Each round can only make things tidier. That alternation is the whole algorithm.

Figure 36.2 traces this alternating optimization on three natural clusters, showing the centroids shifting to cluster means and the Voronoi cell boundaries settling into place.

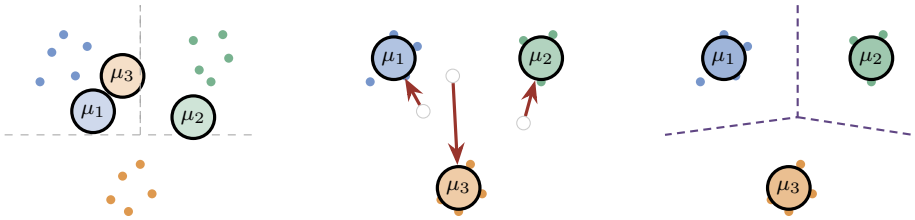


Figure 36.2: Lloyd's k-means clustering algorithm in geometric space. **(a)** Centroids μ_1, μ_2, μ_3 are initialized arbitrarily, partitioning the points into Voronoi cells. **(b)** In the update step, each centroid slides to the mean (center of mass) of its assigned cluster. **(c)** In the assignment step, points rejoin the nearest updated centroid; boundaries settle cleanly between natural clusters.

36.2.1 The cost that k-means minimizes

What does "tidier" mean precisely? We measure how spread out each cluster is. Let $\mu_1, \ldots, \mu_k$ be the centroids and let $c(i) \in \{1, \ldots, k\}$ be the cluster index assigned to point $x^{(i)}$. The cost is the total squared distance from points to their own centroid:

$$J(c, \mu) \;=\; \sum_{i=1}^{n} \left\| x^{(i)} - \mu_{c(i)} \right\|^2.$$

Here $\|v\|^2 = v_1^2 + v_2^2 + \cdots + v_d^2$ is the squared length of the vector v. This number is small exactly when every point sits close to its centroid – which is what we mean by good clusters.

Focus

k-means uses **squared** distance, not plain distance. That is not an arbitrary choice. Squared distance is what makes step 2 – "put the centroid at the mean" – exactly optimal, as we prove below. Different distance choices give different algorithms (e.g. the *k-medoids* or *k-medians* variants).

36.2.2 Lloyd's algorithm

The standard procedure is **Lloyd's algorithm**. It just alternates the two easy steps until nothing changes.

Algorithm 152: k-MEANS (Lloyd's algorithm)

Input: points $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$; number of clusters k
Output: centroids $\mu_1, \dots, \mu_k$ and an assignment $c : \{1..n\} \to \{1..k\}$

initialize $\mu_1, \dots, \mu_k$ (e.g. pick k distinct points at random);
repeat
  `// Assignment step: each point joins its nearest centroid`
  for $i \leftarrow 1$ **to** n **do**
    $c(i) \leftarrow \arg\min_{j \in \{1..k\}} \left\| x^{(i)} - \mu_j \right\|^2$;
  `// Update step: each centroid moves to the mean of its members`
  for $j \leftarrow 1$ **to** k **do**
    $S_j \leftarrow \{\, i : c(i) = j \,\}$;
    if $S_j \neq \emptyset$ **then** $\mu_j \leftarrow \dfrac{1}{|S_j|} \sum_{i \in S_j} x^{(i)}$;
until *no assignment changed*;
return (μ, c);

36.2.3 A fully worked numeric run

Let us cluster six points on the line ($d = 1$, so the math is light) into $k = 2$ groups. The points are

$$x = 1, \;\; 2, \;\; 4, \;\; 7, \;\; 8, \;\; 9.$$

By eye there are two clusters, $\{1, 2, 4\}$ and $\{7, 8, 9\}$. Watch the algorithm discover them.

Initialize. Suppose our random pick lands on the two centroids $\mu_1 = 2$ and $\mu_2 = 8$ (the lab below traces an unfavorable initial pair so you can see how initialization affects the outcome).

Round 1, assignment. For each point pick the nearer centroid (distance, not squared, gives the same winner):

point	$\lvert x - 2 \rvert$	$\lvert x - 8 \rvert$	nearer	cluster
1	1	7	μ_1	1
2	0	6	μ_1	1
4	2	4	μ_1	1
7	5	1	μ_2	2
8	6	0	μ_2	2
9	7	1	μ_2	2

So $S_1 = \{1, 2, 4\}$ and $S_2 = \{7, 8, 9\}$.

Round 1, update.

$$\mu_1 = \frac{1 + 2 + 4}{3} = \frac{7}{3} \approx 2.33, \qquad \mu_2 = \frac{7 + 8 + 9}{3} = 8.$$

Round 2, assignment. Re-check every point against the new centroids 2.33 and 8. Point 4 is at distance 1.67 from μ_1 and 4 from μ_2, so it stays in cluster 1. Every point keeps its cluster: *no assignment changed*. The loop stops.

Final cost.

$$J = \underbrace{(1-\tfrac{7}{3})^2 + (2-\tfrac{7}{3})^2 + (4-\tfrac{7}{3})^2}_{\text{cluster 1}} + \underbrace{(7-8)^2 + (8-8)^2 + (9-8)^2}_{\text{cluster 2}} = \tfrac{14}{3} + 2 \approx 6.67.$$

The algorithm recovered exactly the clustering you would have drawn by hand, and it did so in two passes.



36.2.4 Why does it stop? (Correctness as a decreasing potential)

How do we know Lloyd's algorithm does not loop forever, ping-ponging between configurations? Because the cost J acts like a *potential* that can only go down, and there are only finitely many possible assignments.

Lemma 36.1 (Each step does not increase the cost)**.** *In Lloyd's algorithm, both the assignment step and the update step leave $J(c, \mu)$ unchanged or make it smaller.*

Focus

The proof rests on two "easy optimization" facts:

- **Assignment step.** With the centroids μ held *fixed*, $J = \sum_i \|x^{(i)} - \mu_{c(i)}\|^2$ is minimized over the choice of $c(i)$ by sending each point to its nearest centroid – the terms do not interact, so you minimize each one separately.
- **Update step.** With the assignment c held *fixed*, the best single point to represent a set S under *squared* distance is its mean. This is the fact below.

Lemma 36.2 (The mean minimizes squared distance)**.** *For any finite set of points $\{z_1, \dots, z_m\} \subset \mathbb{R}^d$, the function $g(\mu) = \sum_{t=1}^{m} \|z_t - \mu\|^2$ is minimized at $\mu^\star = \frac{1}{m}\sum_t z_t$, the mean.*

Proof. Work one coordinate at a time; the coordinates separate because the squared norm is a sum over coordinates. In one dimension we minimize $g(\mu) = \sum_t (z_t - \mu)^2$. This is a parabola opening upward in μ. Take the derivative and set it to zero:

$$g'(\mu) = \sum_{t=1}^{m} 2(\mu - z_t) = 2\Big(m\mu - \sum_t z_t\Big) = 0 \quad \Longrightarrow \quad \mu^\star = \frac{1}{m}\sum_t z_t.$$

Since $g''(\mu) = 2m > 0$, this critical point is the unique minimum. Doing this in every coordinate gives the vector mean. □

Proof of Lemma 36.1. The assignment step minimizes J over c with μ fixed, so it cannot raise J. The update step replaces each centroid by the mean of its cluster, which by Lemma 36.2 minimizes that cluster's contribution to J with c fixed, so it cannot raise J either. □

Theorem 36.1 (Lloyd's algorithm terminates)**.** *Lloyd's algorithm halts after finitely many rounds.*

Proof. There are at most k^n distinct assignments c. Each round either changes the assignment – in which case, by Lemma 36.1, the cost strictly drops (an assignment changes only when a point finds a strictly closer centroid) – or it does not, in which case the loop stops. Because the cost strictly decreases on every productive round and the cost is determined by the assignment, no assignment can ever recur. With finitely many assignments, the process must reach a round where nothing changes. □

Common trap

Termination is *not* the same as optimality. Lloyd's algorithm finds a *local* minimum – a configuration no single re-assignment or re-centering can improve – but possibly not the *global* best clustering. A bad random start can strand it in a poor local minimum. In fact, finding the globally optimal k-means clustering is NP-hard. This is the same greedy/local-search limitation you saw in earlier chapters.

Algorithm lab

Trace a bad start. Re-run the six-point example but initialize $\mu_1 = 1$, $\mu_2 = 2$ (both centroids inside the left clump). Do the assignment step (point 1 to μ_1; everything ≥ 2 to μ_2, since it is closer to 2 than to 1), recompute the means, and keep going. Where does it settle? Compare its final cost to the 6.67 we got above. Then try $\mu_1 = 2$, $\mu_2 = 4$. You will see why practitioners run k-means several times from different starts and keep the lowest-cost result. This restart trick, when the starts are chosen cleverly, is called **k-means++**.

Cost summary

Cost of one round of Lloyd's algorithm. The assignment step compares each of n points against each of k centroids, each comparison touching d coordinates: $\Theta(nkd)$. The update step sums each point into one centroid: $\Theta(nd)$. So a round is $\Theta(nkd)$, dominated by assignment. If the algorithm runs for T rounds, the total is $\Theta(Tnkd)$. In the worst case T can be exponential, but in practice T is small (often tens of rounds), so k-means is treated as a near-linear-time algorithm. Memory is $\Theta(nd + kd)$.

Why this matters

k-means is everywhere because it is cheap and intuitive: compressing an image to k colors (*color quantization*), grouping customers for marketing, summarizing documents, initializing more elaborate models, and compressing vectors for fast nearest-neighbor search. When an engineer says "let us just cluster it and see," they almost always mean run k-means.

36.3 Gradient descent: walking downhill on purpose

k-means minimized its cost by a clever alternation that happened to have a closed-form best move at each step. Most learning problems are not so lucky: there is no formula for the best θ, only a cost surface $J(\theta)$ we can evaluate and *differentiate*. For those, the standard tool is **gradient descent**.

36.3.1 The gradient is the uphill arrow

Recall from calculus that the derivative $J'(\theta)$ of a one-variable function tells you its slope: if $J'(\theta) > 0$ the function is rising as θ increases, so to go *down* you should *decrease* θ. To descend, step *against* the sign of the derivative.

In many variables the derivative becomes the **gradient**, written $\nabla J(\theta)$ – the vector of partial derivatives, one per knob:

$$\nabla J(\theta) = \left(\frac{\partial J}{\partial \theta_1}, \frac{\partial J}{\partial \theta_2}, \ldots, \frac{\partial J}{\partial \theta_p} \right).$$

Intuition

At any point on the cost surface, the gradient $\nabla J(\theta)$ points in the direction of *steepest ascent* – the compass bearing that climbs fastest. You want to go down, so you step in the *opposite* direction, $-\nabla J(\theta)$. "Gradient descent" literally means "repeatedly step a little bit in the most-downhill direction."

36.3.2 The update rule and the learning rate

You take steps of size controlled by a number $\eta > 0$ (Greek "eta"), the **learning rate** or **step size**:

$$\theta \leftarrow \theta - \eta \nabla J(\theta).$$

You repeat this until the gradient is nearly zero (the ground is flat) or you run out of patience.

Algorithm 153: Gradient-Descent

Input: differentiable cost $J : \mathbb{R}^p \to \mathbb{R}$; learning rate $\eta > 0$; iterations T
Output: a parameter vector θ with (we hope) small $J(\theta)$

initialize θ (e.g. all zeros or small random values);
for $t \leftarrow 1$ **to** T **do**
  compute the gradient $g \leftarrow \nabla J(\theta)$;
  $\theta \leftarrow \theta - \eta\, g$;
  if $\|g\|$ *is tiny* **then break** `// ground is flat: stop;`
return θ;

Common trap

The learning rate η is the number beginners get wrong most often.

- Too **small**: you inch downhill and it takes forever (and may stall before reaching the valley within your iteration budget).

- Too **large**: you overshoot the valley, land on the far slope *higher* than you started, overshoot back, and *diverge* – the cost explodes to infinity.

Choosing η means finding the stable middle range between those two regimes. We will see the divergence concretely below.

36.3.3 A fully worked one-variable run

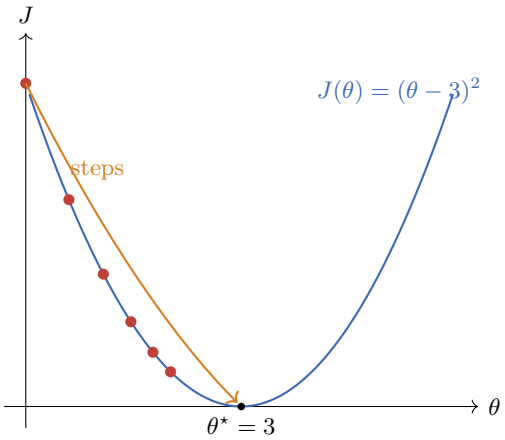
Take the simplest non-trivial cost, a parabola:

$$J(\theta) = (\theta - 3)^2, \qquad J'(\theta) = 2(\theta - 3).$$

The minimum occurs at $\theta = 3$ where $J = 0$. Let us pretend we do not know that and let gradient descent find it. Start at $\theta_0 = 0$ with $\eta = 0.1$. The update is $\theta \leftarrow \theta - 0.1 \cdot 2(\theta - 3) = \theta - 0.2(\theta - 3) = 0.8\,\theta + 0.6$.

step t	θ_t	$J'(\theta_t) = 2(\theta_t - 3)$	$J(\theta_t)$
0	0.000	−6.000	9.000
1	0.600	−4.800	5.760
2	1.080	−3.840	3.686
3	1.464	−3.072	2.359
4	1.771	−2.458	1.510
5	2.017	−1.966	0.966
⋮	⋮	⋮	⋮
10	2.678	−0.644	0.104
20	2.965	−0.069	0.0012

Every step, the derivative is negative (we are to the left of the minimum, slope going down-to-the-right), so the update *adds* to θ, nudging us rightward toward 3. The cost falls steadily. The steps shrink as we approach, because the gradient itself shrinks near the bottom – so gradient descent automatically takes smaller steps near a minimum. After 20 steps we are at 2.965, very close to the minimum.



Algorithm lab

Make it diverge. Re-run the same parabola from $\theta_0 = 0$ but set $\eta = 1.1$. Now the update is $\theta \leftarrow \theta - 1.1 \cdot 2(\theta - 3) = -1.2\,\theta + 6.6$. Compute $\theta_1, \theta_2, \theta_3$. You will get 6.600, then -1.320, then 8.184 – the iterates fling *outward*, alternating sides of 3 and getting worse each time. The cost blows up. This is what "η too large" looks like. Find by experiment the largest η that still converges. (For $J = (\theta - 3)^2$ the boundary is $\eta = 1$; convergence needs $\eta < 1$.)

36.3.4 Convexity: when downhill is enough

Gradient descent found the bottom of the parabola because a parabola has exactly one valley. A cost function with a single bowl shape – no false valleys, no plateaus that trap you – is called **convex**. On convex costs, "keep walking downhill" is guaranteed to reach the true global minimum.

Formal

A function $J : \mathbb{R}^p \to \mathbb{R}$ is **convex** if for all θ, θ' and all $\lambda \in [0, 1]$,

$$J\big(\lambda\theta + (1-\lambda)\theta'\big) \;\le\; \lambda J(\theta) + (1-\lambda) J(\theta').$$

Geometrically: the straight line segment joining any two points on the graph never dips *below* the graph. Such a function has no local minimum that is not also global. The squared-error costs of k-means' update step and of linear models (next section) are convex in their parameters; this is why they are easy to optimize.

Figure 36.3 makes the contrast visible. On the left, a convex bowl: the chord between any two points stays *above* the curve, and wherever you start, downhill leads to the single global minimum. On the right, a wavy non-convex cost: the chord dips below the curve in places, and which valley you fall into now depends on where you begin.

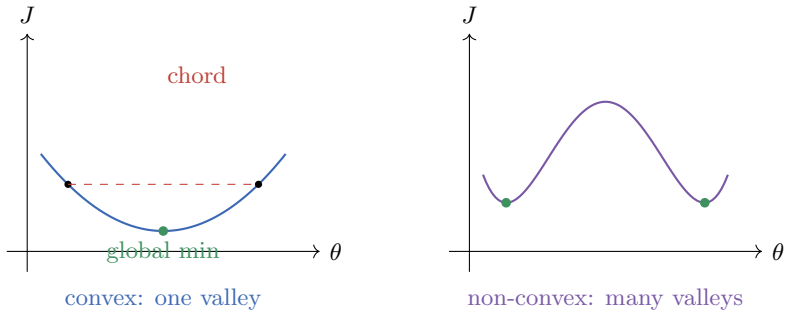


Figure 36.3: Convex versus non-convex cost surfaces. On a convex cost (left) every chord lies above the curve and downhill always reaches the single global minimum. On a non-convex cost (right) the two marked valleys are different local minima; the starting point decides which one gradient descent finds.

Walking question

Consider this question while you stretch your legs. Convexity guarantees a single global valley, so gradient descent on a convex cost cannot get stuck in a bad local minimum. Yet the modern deep-learning models you hear about are *not* convex – their loss surfaces are complicated, full of valleys. They still work surprisingly well. So: what does "learning succeeded" really mean if we never reach the global minimum? Is a "good enough" valley the real goal? Consider the possibility that for these problems, *finding any deep-enough valley* is the primary objective, and the global minimum may not even be desirable (it might just memorize the data).

36.3.5 Batch vs. stochastic: the great speedup

When the cost is an average over n data points, $J(\theta) = \frac{1}{n}\sum_i \ell_i(\theta)$, the gradient is also an average: $\nabla J(\theta) = \frac{1}{n}\sum_i \nabla \ell_i(\theta)$. Computing it exactly means touching all n examples for *every single step*. With millions of examples that is expensive.

Intuition

You do not need the exact downhill direction – you only need a direction that is *downhill on average*. So estimate the gradient from a small random sample of the data (even a single example!) and step using that noisy estimate. Each step is now cheap, so you take many more of them. The noise can even help you escape shallow local minima. This is **stochastic gradient descent (SGD)**, the method behind essentially all modern large-scale learning.

Figure 36.4 shows the trade in one picture, looking down on the cost surface from above (the closed curves are contours of equal cost, like a topographic map). Batch descent takes a few smooth, expensive steps straight for the minimum. SGD jitters along a noisier path – each step points only roughly downhill – but every step is so cheap that it covers far more ground per unit of work.

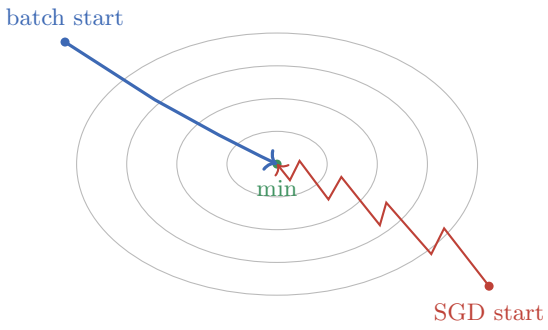


Figure 36.4: Two paths down the same bowl, drawn as a contour map. Batch gradient descent (blue) takes a few exact steps; stochastic gradient descent (red) takes many cheap, noisy steps. Per step SGD wanders, but per unit of computation it usually reaches the valley faster on large datasets.

Cost summary

Let one gradient term $\nabla \ell_i$ cost $\Theta(d)$ to evaluate.

- **Batch** gradient descent: $\Theta(nd)$ per step, T steps, $\Theta(Tnd)$ total. Each step is an *exact*

downhill move.

- **Stochastic** gradient descent with a minibatch of size b: $\Theta(bd)$ per step. With $b = 1$ that is $\Theta(d)$ per step – independent of dataset size n. The trade is noisier steps, so you usually need more of them, but each is so cheap that SGD wins decisively at scale.

36.4 The perceptron: learning a straight-line boundary

Now we put the pieces together on a *supervised* problem and meet one of the earliest learning algorithms. The **perceptron** (Rosenblatt, 1958) is a learning rule for telling two classes apart with a flat boundary. It is the ancestor of every neural network, and it is simple enough to trace by hand.

36.4.1 The setup: a linear classifier

Each example is a feature vector $x \in \mathbb{R}^d$ with a label $y \in \{-1, +1\}$ (think "cat vs. not-cat," "spam vs. not-spam"). The model is a weight vector $w \in \mathbb{R}^d$ and a bias $b \in \mathbb{R}$. It predicts

$$\hat{y}(x) = \operatorname{sign}\big(w \cdot x + b\big), \qquad w \cdot x = \sum_{j=1}^{d} w_j x_j.$$

The set of points where $w \cdot x + b = 0$ is a flat boundary – a line in 2D, a plane in 3D, a **hyperplane** in general. Points on one side get $+1$, the other side -1. Training means *finding* a (w, b) whose boundary puts the two classes on opposite sides.

Focus

A convenient trick removes the bias from the bookkeeping: glue a constant 1 onto every input, $\tilde{x} = (x_1, \ldots, x_d, 1)$, and fold b into the weights as the last coordinate, $\tilde{w} = (w_1, \ldots, w_d, b)$. Then $w \cdot x + b = \tilde{w} \cdot \tilde{x}$, and we only ever update one vector. We use this trick in the trace below.

36.4.2 The learning rule

The perceptron's update is very simple. Loop through the examples. When the current weights classify an example correctly, do nothing. When they get one *wrong*, nudge the weights toward fixing that example:

$$\text{if } y^{(i)}\,(w \cdot x^{(i)}) \le 0 \;\text{ (a mistake)}, \qquad w \leftarrow w + y^{(i)}\,x^{(i)}.$$

Algorithm 154: PERCEPTRON (with the bias folded into w)

Input: examples $(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})$ with $y^{(i)} \in \{-1, +1\}$; max passes T
Output: weight vector w separating the classes (if one exists)

```
w ← 0;
for pass ← 1 to T do
    mistakes ← 0;
    for i ← 1 to n do
        if y^(i) (w · x^(i)) ≤ 0 then
            w ← w + y^(i) x^(i);
            mistakes ← mistakes + 1;
    if mistakes = 0 then break                    // a full clean pass: done;
return w;
```

Intuition

Why does adding $y^{(i)}x^{(i)}$ fix a mistake? Suppose the true label is $+1$ but $w \cdot x^{(i)}$ came out ≤ 0 (wrong). We add $x^{(i)}$ to w. The new score on that same point becomes $(w + x^{(i)}) \cdot x^{(i)} = w \cdot x^{(i)} + \|x^{(i)}\|^2$, which is *larger* than before – pushed in the $+$ direction, exactly where we wanted it. For a -1 example we subtract $x^{(i)}$ and the score drops. Every update moves the boundary so the offending point is "more correct."

36.4.3 A fully worked trace

Let us separate two classes in the plane. Using the bias trick, each input is $(x_1, x_2, 1)$ and $w = (w_1, w_2, w_3)$. Four examples:

i	$x^{(i)} = (x_1, x_2, 1)$	$y^{(i)}$
1	(2, 1, 1)	$+1$
2	(0, -1, 1)	-1
3	(-1, 0, 1)	-1
4	(1, 1, 1)	$+1$

Start $w = (0, 0, 0)$. With $w = \mathbf{0}$ the score is 0, and $y \cdot 0 = 0 \le 0$, so the very first point counts as a mistake.

Example 1 $(2, 1, 1)$, $y = +1$: score $w \cdot x = 0 \le 0$ – mistake. Update $w \leftarrow (0, 0, 0) + (+1)(2, 1, 1) = (2, 1, 1)$.

Example 2 $(0, -1, 1)$, $y = -1$: score $(2, 1, 1) \cdot (0, -1, 1) = 0 - 1 + 1 = 0$, and $y \cdot 0 = 0 \le 0$ – mistake. Update $w \leftarrow (2, 1, 1) + (-1)(0, -1, 1) = (2, 2, 0)$.

Example 3 $(-1, 0, 1)$, $y = -1$: score $(2, 2, 0) \cdot (-1, 0, 1) = -2 + 0 + 0 = -2$, and $y \cdot (-2) = (-1)(-2) = 2 > 0$ – correct! No update.

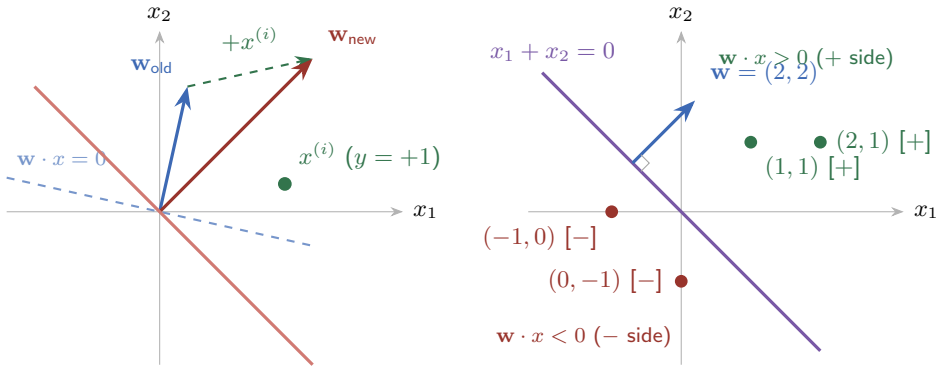
Example 4 $(1, 1, 1)$, $y = +1$: score $(2, 2, 0) \cdot (1, 1, 1) = 2 + 2 + 0 = 4$, and $y \cdot 4 = 4 > 0$ – correct! No update.

That ends pass 1 with $w = (2, 2, 0)$ and two mistakes. Pass 2: re-check all four.

i	$x^{(i)}$	$y^{(i)}$	score $w \cdot x$	$y \cdot$ score
1	$(2, 1, 1)$	$+1$	$4 + 2 + 0 = 6$	$6 > 0$ correct
2	$(0, -1, 1)$	-1	$0 - 2 + 0 = -2$	$2 > 0$ correct
3	$(-1, 0, 1)$	-1	$-2 + 0 + 0 = -2$	$2 > 0$ correct
4	$(1, 1, 1)$	$+1$	$2 + 2 + 0 = 4$	$4 > 0$ correct

Zero mistakes on a full pass – we stop. The final classifier is $w = (2, 2, 0)$, i.e. predict $\text{sign}(2x_1 + 2x_2)$, whose boundary is the line $x_1 + x_2 = 0$. Check it against the data: it really does put the two $+$ points on one side and the two $-$ points on the other.

Figure 36.5 illustrates both the geometric vector update and the final converged separating boundary.



(a) Weight Vector Update
Adding $y^{(i)}x^{(i)}$ tilts $\mathbf{w}$
Hyperplane rotates toward $x^{(i)}$

(b) Converged Decision Boundary
Zero errors on all 4 points
Normal $\mathbf{w}$ points into positive class

Figure 36.5: The perceptron decision boundary and update geometry. **(a)** When a positive example $x^{(i)}$ is misclassified, the update $\mathbf{w} \leftarrow \mathbf{w} + x^{(i)}$ tilts the weight vector toward $x^{(i)}$, rotating the orthogonal hyperplane decision boundary $\mathbf{w} \cdot x = 0$ until $x^{(i)}$ lands on the positive side. **(b)** The converged classifier on the four trace points: boundary line $x_1 + x_2 = 0$ perfectly separates the positive points from the negative points, with normal vector $\mathbf{w} = (2, 2)$ pointing into the positive half-space.

Notice the weight vector $w = (2, 2)$ (dropping the bias coordinate) points *perpendicular* to the boundary, toward the positive side. That is no accident: w is always the normal direction of the hyperplane it defines.

36.4.4 Why it works: the perceptron convergence theorem

The key fact, and the reason the perceptron caused such excitement, is that if the data *can* be separated by a hyperplane at all, this simple update rule is *guaranteed* to find one – in a bounded number of mistakes that does not depend on how many examples there are.

Formal

The data is **linearly separable with margin** $\gamma > 0$ if there is a unit weight vector $w^\star$ ($\|w^\star\| = 1$) such that every example satisfies $y^{(i)}(w^\star \cdot x^{(i)}) \geq \gamma$. Let $R = \max_i \|x^{(i)}\|$ be the radius of the data.

Theorem 36.2 (Perceptron convergence; Novikoff, 1962)**.** *If the data is linearly separable with margin γ and radius R, the perceptron algorithm makes at most $\big(R/\gamma\big)^2$ mistakes before it classifies every example correctly (and then halts).*

Proof. We track two quantities about w as mistakes accumulate; let w_k be the weights after the k-th mistake, starting from $w_0 = \mathbf{0}$.

Lower bound (the weights line up with $w^\star$). On a mistake using example (x, y) we set $w_k = w_{k-1} + y\,x$. Dot with $w^\star$:

$$w^\star \cdot w_k = w^\star \cdot w_{k-1} + y\,(w^\star \cdot x) \;\geq\; w^\star \cdot w_{k-1} + \gamma,$$

using the margin condition. After k mistakes, $w^\star \cdot w_k \geq k\gamma$.

Upper bound (the weights cannot grow too fast). Compute the squared length after the update:

$$\|w_k\|^2 = \|w_{k-1}\|^2 + 2y\,(w_{k-1} \cdot x) + \|x\|^2.$$

Because we only update on a *mistake*, we have $y\,(w_{k-1} \cdot x) \leq 0$, and $\|x\|^2 \leq R^2$, so $\|w_k\|^2 \leq \|w_{k-1}\|^2 + R^2$. After k mistakes, $\|w_k\|^2 \leq kR^2$.

Squeeze. By the Cauchy–Schwarz inequality (and $\|w^\star\| = 1$), $w^\star \cdot w_k \leq \|w_k\|$. Combine the two bounds:

$$k\gamma \;\leq\; w^\star \cdot w_k \;\leq\; \|w_k\| \;\leq\; \sqrt{k}\,R.$$

Thus $k\gamma \leq \sqrt{k}\,R$, i.e. $\sqrt{k} \leq R/\gamma$, giving $k \leq (R/\gamma)^2$. The number of mistakes is bounded, so after finitely many the algorithm makes a clean pass and stops. □

Focus

The bound $(R/\gamma)^2$ says nothing about n, the number of examples, and nothing about d, the dimension. It depends only on the *geometry*: how big the data is (R) versus how comfortably it can be split (γ). Wide margin ⇒ few mistakes ⇒ fast learning. The margin later became the central quantity of *support vector machines*, the dominant classifiers of the 1990s and 2000s.

Common trap

The convergence theorem assumes the data *is* separable. If it is not – if no straight boundary can split the classes – the perceptron never makes a clean pass and loops until it hits T, with the weights never settling. Historically, Minsky and Papert (1969) pointed out that even the tiny XOR function is not linearly separable, so a single perceptron *cannot* learn it. That critique froze the field for years. The fix – stacking perceptrons into layers and training them with gradient descent – is the neural network, and it is why Section 36.3 matters so much.

36.4.5 The perceptron as gradient descent

The two halves of this chapter are connected. The perceptron update *is* a stochastic gradient step on a particular cost. Define the per-example **perceptron loss**

$$\ell(w; x, y) = \max\big(0,\ -y\,(w \cdot x)\big),$$

which is 0 when the example is classified correctly (with positive score) and grows linearly with how wrong you are otherwise. Its gradient with respect to w is 0 on a correct example and $-y\,x$ on a mistake. So a stochastic gradient *descent* step with learning rate 1, $w \leftarrow w - \eta\,\nabla\ell = w + y\,x$ on mistakes, is exactly Rosenblatt's rule. Thus the 1958 perceptron update is stochastic subgradient descent applied to hinge-at-zero loss.

Why this matters

Everything in this chapter is used in systems you rely on daily. A spam filter is a linear classifier trained with SGD. The recommendation feed that clusters users and items leans on k-means-style grouping. A large language model is a very large stack of differentiable layers whose billions of parameters are tuned by exactly the gradient-descent loop in Section 36.3 – just with a more elaborate J and a lot more hardware. The basic loop is the same in all of them: define a cost, take the gradient, step downhill, repeat.

36.5 Summary

The methods in this chapter connect to much of the earlier material:

- **Greedy and local search.** Lloyd's algorithm is alternating minimization: each step takes the locally best assignment or center, eventually settling into a local minimum.
- **Loop invariants and potentials.** We proved k-means halts using a strictly decreasing potential J, the same potential argument used for amortized analysis. We proved perceptron convergence with a two-sided squeeze on weight vector growth.
- **Recurrences and cost analysis.** We derived per-round costs $\Theta(nkd)$ and $\Theta(Tnd)$ from loop structure and matrix operations.
- **Linear algebra and geometry.** Dot products, hyperplanes, margins, and projections are the native geometric language of data and parameters.
- **Tradeoffs and limits.** Real-world learning balances optimization against generalization. Non-convex loss surfaces can trap descent in poor local valleys; unregularized models risk memorizing noise rather than patterns; and statistical models lack the hard worst-case correctness guarantees of classical algorithms.

In theory, training a machine-learning model is pure continuous optimization over linear algebra. In practice, computing gradients across billions of parameters strains physical hardware to its limits. Arithmetic throughput is rarely the true bottleneck; the limiting factor is moving bits across the memory bus, cache hierarchies, and DRAM banks. To understand why algorithms run fast or slow on real hardware, we need to look at the machine itself. In Chapter 37, we step from algorithmic models into systems architecture and memory physics, before exploring how optimizing compilers

bridge source code to machine instructions (Chapter 38) and how quantum mechanics changes what can be computed efficiently (Chapter 39).

Pause

The one idea to remember is this loop: *(1) write a number that measures wrongness; (2) find the direction that makes it smaller; (3) take a small step; (4) repeat.* Clustering, classification, regression, and modern neural networks are all variations on those four lines. Algorithms that learn are still algorithms; what drives them is a cost function.

Exercises

Exercise 36.1 (Warmup: assign and update)**.** Run one full round of Lloyd's algorithm by hand on the one-dimensional points $0, 1, 10, 11, 12$ with $k = 2$ and initial centroids $\mu_1 = 0$, $\mu_2 = 1$. Show the assignment, then the updated centroids. Run a second round. Does the assignment change? Report the final clustering and its cost J.

Exercise 36.2 (The mean, again)**.** Prove that for points $z_1, \ldots, z_m$ on the real line, the value μ that minimizes $\sum_t |z_t - \mu|$ (note: *absolute* value, not squared) is a *median*, not the mean. Explain in one sentence why k-means therefore uses the mean while k-medians uses the median.

Exercise 36.3 (Gradient descent on a parabola)**.** For $J(\theta) = (\theta - 5)^2$ with start $\theta_0 = 0$ and learning rate $\eta = 0.25$, write the closed-form update $\theta_{t+1} = a\theta_t + c$ and compute $\theta_1, \theta_2, \theta_3$. Then find, by solving the recurrence, a formula for θ_t and confirm $\theta_t \to 5$.

Exercise 36.4 (The edge of stability)**.** For $J(\theta) = (\theta - c)^2$, show that the gradient-descent update can be written $\theta_{t+1} - c = (1 - 2\eta)(\theta_t - c)$. Conclude that the iterates converge to c if and only if $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$, and that $\eta = 0.5$ jumps to the exact minimum in one step. Explain the behavior at $\eta = 1$.

Exercise 36.5 (Trace the perceptron)**.** Using the bias trick (append a 1 to each input), run the perceptron from $w = \mathbf{0}$ on: $(1, 0; +1)$, $(0, 1; +1)$, $(-1, -1; -1)$ (here the semicolon separates features from label). Show every score and every update. How many mistakes occur before a clean pass? Draw the final boundary.

Exercise 36.6 (Why $y\,x$?)**.** Suppose example (x, y) with $y = +1$ is currently misclassified, so $w \cdot x \le 0$. After the update $w' = w + x$, show the new score increased by exactly $\|x\|^2$. Then show that for a $y = -1$ mistake the update $w' = w - x$ *decreases* the score by $\|x\|^2$. Why does this not guarantee the point is now correctly classified after a single update?

Exercise 36.7 (Margin and mistakes)**.** Two datasets are each linearly separable. Dataset A has radius $R = 2$ and margin $\gamma = 1$; dataset B has radius $R = 10$ and margin $\gamma = 0.5$. Use Theorem 36.2 to bound the number of perceptron mistakes on each. Which dataset is "harder" for the perceptron, and by what factor?

Exercise 36.8 (XOR is not linearly separable)**.** The XOR data is $(0, 0; -1)$, $(1, 1; -1)$, $(1, 0; +1)$, $(0, 1; +1)$. Suppose a separating $w = (w_1, w_2)$ and bias b existed. Write the four inequalities $y^{(i)}(w \cdot x^{(i)} + b) > 0$ and derive a contradiction. Conclude that no single perceptron can learn XOR.

Exercise 36.9 (Batch vs. stochastic cost)**.** A dataset has $n = 10^6$ examples in $d = 100$ dimensions. Estimate the number of multiply-add operations for one *batch* gradient step versus one *stochastic* step with minibatch size $b = 32$. If a fixed compute budget allows 10^9 multiply-adds, roughly how many steps of each can you afford?

Exercise 36.10 (Challenge: a non-convex trap)**.** Consider $J(\theta) = \theta^4 - 4\theta^2 + \theta$. Find $J'(\theta)$ and argue (by examining sign changes) that J has two local minima. Pick two starting points for gradient descent that you predict will converge to *different* minima, and explain your reasoning geometrically. This is a simple example of why neural-network training depends on initialization.

Exercise 36.11 (Challenge: k-means is a coordinate descent)**.** Explain why Lloyd's algorithm can be viewed as *alternating minimization*: it minimizes $J(c, \mu)$ over c with μ fixed, then over μ with c fixed, forever alternating. Using this view, give a one-paragraph argument for why the cost sequence $J_0 \geq J_1 \geq J_2 \geq \cdots$ is monotone and bounded below by 0, hence convergent. Why does convergence of the *cost* not by itself prove the *assignment* stops changing?

Chapter 37

Modern Systems Architecture: From the CPU Pipeline to DRAM Cycles

In Chapter 36, we saw that training machine-learning models strains hardware by moving billions of parameters across memory. While asymptotic analysis counts operations, execution speed on real silicon is governed by physical memory movement. Consider two simple programs that perform the exact same task. Both take an ordered collection of $n = 10^7$ sixty-four-bit integers and compute their sum. Both execute exactly n additions in a single pass from start to finish. Under the classical Random Access Machine (RAM) model introduced in early chapters, both algorithms execute $\Theta(n)$ operations, and one might expect them to finish in virtually identical running times.

```
// Program A: Contiguous flat array
int64_t sum_array(const int64_t* arr, size_t n) {
    int64_t total = 0;
    for (size_t i = 0; i < n; ++i) total += arr[i];
    return total;
}
// Program B: Pointer-linked list
struct Node { int64_t val; Node* next; };
int64_t sum_list(const Node* head) {
    int64_t total = 0;
    for (const Node* curr = head; curr != nullptr; curr = curr->next) total += curr->val;
    return total;
}
```

When compiled with identical optimization flags and measured on modern hardware, Program A completes in roughly 3 milliseconds. Program B requires approximately 180 milliseconds—a slowdown of 60×. If the nodes of the linked list were allocated haphazardly across a fragmented heap, the disparity can exceed 100×.

Why does an algorithm with the exact same nominal step count run sixty to one hundred times slower simply because its numbers are separated by pointers rather than arranged in consecutive memory locations? The classical RAM model makes an assumption that was broadly reasonable in the early 1970s: that reaching any word in memory requires a fixed, uniform unit of time. On modern hardware, that assumption is badly wrong.

Why this matters

The main bottleneck in contemporary computing is the time and energy needed to move numbers from DRAM capacitors into the processor's registers, not the speed at which the processor adds them.

In 1975, a single CPU arithmetic instruction and a read from main memory took roughly the same duration—about 500 nanoseconds. Over the subsequent four decades, transistor scaling, architectural pipelining, and frequency increases sped up CPU arithmetic by more than 10,000×. Clock frequencies reached several gigahertz, bringing the time for a single register-level addition down to roughly 0.25 nanoseconds (less than one clock cycle on a 4 GHz core).

Main memory, however, faced an entirely different set of physical constraints. A memory fetch requires driving electrical signals through silicon packages, pin connectors, and copper traces on a motherboard, reaching dynamic memory cells whose charge must be sensed, amplified, and restored. While DRAM capacity exploded by a factor of millions, the latency of a random DRAM read improved by barely a factor of ten, dropping from roughly 500 ns down to 50–80 ns.

This growing gap is known as the *Memory Wall*. When a modern processor requests a word from memory that does not already reside in on-chip cache, it must stall for roughly 200 to 300 clock cycles. During that stall, a contemporary four-wide out-of-order processor could have executed nearly one thousand independent arithmetic instructions. An algorithm whose memory access pattern ignores hardware structure will spend more than ninety-eight percent of its execution time completely idle, waiting for the memory bus.

History and connection

The structural tension between computation and memory dates to the earliest days of electronic computing. In June 1945, John von Neumann authored the foundational *First Draft of a Report on the EDVAC*, formalizing an architecture where both executable instructions and program data inhabit a single unified storage space and pass across a shared communication bus. While revolutionary in its flexibility, this arrangement forced all arithmetic to queue behind memory transfers, a structural ceiling that John Backus later christened the *von Neumann bottleneck* during his 1977 Turing Award lecture.

By 1965, the widening performance gap between processing logic and magnetic core memory prompted British computing pioneer Maurice Wilkes to propose a radical design: placing a small, auxiliary memory built from high-speed transistors directly between the central processor and main storage. Wilkes termed this buffer a "slave memory"—the direct predecessor of the modern processor cache. Three years later, in 1968, Robert Dennard at the IBM Thomas J. Watson Research Center invented the one-transistor dynamic random-access memory (DRAM) cell. By replacing complex multi-transistor latch circuits with a single microscopic capacitor and a lone access gate, Dennard achieved orders of magnitude higher bit density at low cost, establishing DRAM as the universal medium for system memory.

As processor clock rates escalated into gigahertz frequencies during the 1990s, microarchitects realized that passive caching alone was insufficient to bridge the Memory Wall. Chip designers adopted silicon implementations of Robert Tomasulo's dynamic scheduling algorithm, giving rise to out-of-order superscalar execution. Processors began inspecting hundreds of upcoming instructions in flight, predicting conditional branches speculatively, and reordering independent arithmetic operations to hide hundreds of cycles of DRAM latency. In 2007,

Ulrich Drepper published his 114-page paper *What Every Programmer Should Know About Memory*, warning programmers that hardware had become so deeply tiered that algorithms designed without explicit attention to cache hierarchies, memory buses, and line boundaries were leaving most of the hardware's performance unused.
The physical properties of memory also affect system reliability and security. Microscopic DRAM storage capacitors packed nanometers apart suffer from electromagnetic interference, giving rise to *Rowhammer* attacks, where repeatedly activating adjacent rows induces electrical disturbance that flips bits in neighboring physical memory without write privileges. Concurrently, speculative execution across out-of-order pipelines exposed subtle timing side channels such as *Spectre* and *Meltdown*, where the transient presence of data in CPU caches leaks cryptographic secrets across security boundaries. To supply modern machine learning accelerators with the necessary memory throughput, engineers have embraced High Bandwidth Memory (HBM), stacking dynamic memory dies vertically with through-silicon vias directly atop silicon interposers alongside the processing cores.

37.1 Inside the Silicon: 6-Transistor SRAM versus 1T1C DRAM

To understand why the memory hierarchy is organized as it is, we must look at how physical silicon stores a single binary digit. Physically, a bit is a charge or voltage that the circuit must hold against leakage. Modern computers use two radically different circuit designs for random-access memory: Static RAM (SRAM) and Dynamic RAM (DRAM).

37.1.1 Static RAM (SRAM): The Fast, Expensive Bistable Seesaw

Static RAM is the technology used for CPU registers and the L1, L2, and L3 on-chip caches. An SRAM cell achieves speed and stability by employing a closed feedback loop composed of two cross-coupled CMOS inverters.

Each CMOS inverter consists of two complementary transistors: a p-channel transistor connected to the supply voltage (V_{DD}) to pull the output high, and an n-channel transistor connected to ground (0 V) to pull the output low. When two such inverters are wired in a loop—the output of the first driving the input of the second, and the output of the second driving the input of the first—the circuit possesses two stable equilibrium states:

1. State '1': Node Q is held at V_{DD}, forcing the second inverter to pull $\overline{Q}$ to 0 V, which in turn keeps Q clamped to V_{DD}.
2. State '0': Node Q is held at 0 V, forcing $\overline{Q}$ to V_{DD}, which actively keeps Q at ground.

Intuition

Picture two people sitting on a seesaw with a mechanical locking pin. As long as the circuit has power, the active transistors act like muscles holding the board locked firmly at one side or the other. It will not drift into the middle. To read the position, you simply look at which side is up; sensing the state does not disturb the locked seesaw in the slightest.

To read from or write to this loop, two n-channel access transistors controlled by a horizontal *Wordline* (WL) connect nodes Q and $\overline{Q}$ to vertical differential *Bitlines* (BL and $\overline{BL}$). This brings the transistor count to six per bit, known as a **6T SRAM cell**.

When the Wordline activates, the access transistors turn on. The internal inverters immediately drive current onto the vertical bitlines:

- **Extreme Speed**: Active transistors drive output lines directly, presenting values in under a nanosecond (< 1 ns), matching gigahertz clock cycles.
- **Non-Destructive Reads**: Reading leaves stored charges fully intact. No refresh or restoration is required as long as power is maintained.
- **Large Silicon Footprint and Static Leakage**: Six transistors per bit consume 0.02–0.04 μm^2 of area. Fabricating 32 gigabytes of SRAM would require trillions of transistors and dozens of watts of static leakage power.

37.1.2 Dynamic RAM (DRAM): Density via Dennard's 1T1C Simplification

In 1968, Robert Dennard observed that if one is willing to accept operational complexity, the six-transistor latch can be stripped down to a single switch and a microscopic storage reservoir: **one access transistor and one capacitor** (a **1T1C cell**).

The capacitor is a vertical cylinder etched into silicon with a capacitance C_s of only 20 to 30 femtofarads (10^{-15} F). At 1.2 V, it holds:

$$Q = C_s \cdot V \approx (25 \times 10^{-15}\text{ F}) \times (1.2\text{ V}) = 3.0 \times 10^{-14}\text{ Coulombs} \approx 180{,}000\text{ electrons.}$$

A 1T1C cell requires only 6 to 8 F^2 of silicon area—over ten times smaller than SRAM. This density enables gigabytes of memory per die, but introduces two physical penalties.

Formal

Physical Penalty 1: Destructive Reads and Charge Sharing
The vertical bitline wire spans thousands of cells, creating a parasitic capacitance $C_{\text{bitline}} \approx$ 200–300 fF (10× larger than C_s). Precharging the bitline to $V_{DD}/2$ and opening the access transistor equalizes charge:

$$\Delta V = \frac{C_s}{C_s + C_{\text{bitline}}} \left(V_{\text{cell}} - \frac{V_{DD}}{2} \right) \approx \pm 25 \text{ to } 50 \text{ millivolts.}$$

This charge sharing empties the capacitor, destroying the bit. A differential *Sense Amplifier* at the bitline detects this millivolt deflection, amplifies it to full rail voltage (0 V or V_{DD}), and drives that amplified potential back into the opened capacitor to restore its charge before the row closes.

Formal

Physical Penalty 2: Charge Leakage and Periodic Refresh (t_{REFI})
Dielectric insulation is atomic in thickness. Electrons leak out via quantum tunneling and junction leakage within 64 milliseconds. The memory controller must issue refresh commands every $t_{\text{REFI}} \approx 7.8\,\mu$s, opening and restoring rows and temporarily locking out normal read/write

commands.

Figure 37.1 illustrates the physical anatomy of both cells.

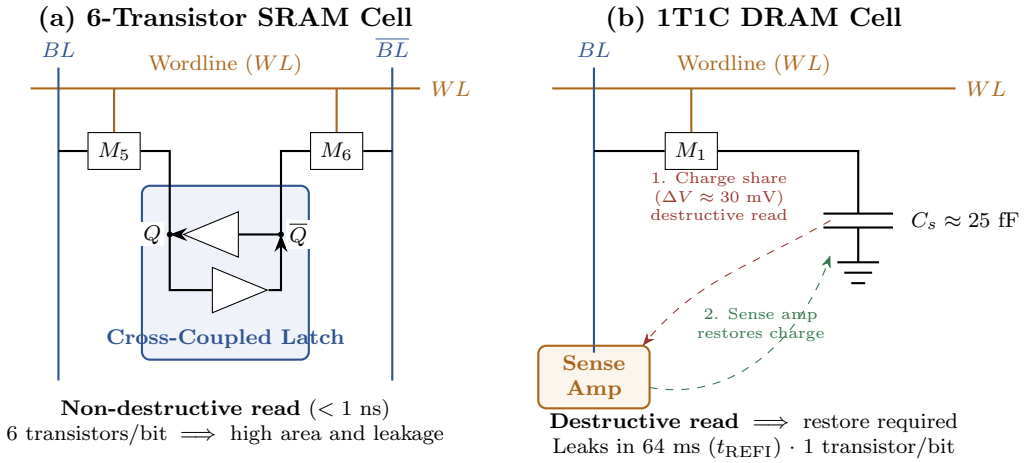


Figure 37.1: The physical storage of a bit. (a) A 6-transistor SRAM cell maintains a bistable voltage state using cross-coupled inverters, providing sub-nanosecond, non-destructive access at the expense of high silicon footprint and power. (b) A 1T1C DRAM cell stores data as microscopic charge in a 25-femtofarad capacitor, yielding high density but requiring destructive charge sharing, sense-amplifier restoration, and periodic refresh.

37.2 Anatomy of a DRAM Cycle: Banks, Row Buffers, and Latency Numbers

Internally, a DRAM chip is not a flat sequence of addressable bytes; its cells are arranged in banks, rows, and columns, and access time depends on where a request falls.

37.2.1 Matrix Organization: Rows, Columns, and the Row Buffer

A DRAM chip is organized internally into multiple independent sub-arrays called **Banks** (typically 16 in DDR4, 32 in DDR5) to allow interleaved parallel requests. Inside each bank sits a 2D matrix of 1T1C cells:

- **Rows**: Typically 65,536 rows (2^{16}).
- **Columns**: Each row contains 8,192 bitlines (1 KiB = 8,192 bits).

At the bottom of this matrix sits an array of 8,192 differential Sense Amplifiers. Activating a row dumps all 8,192 charges onto bitlines; the sense amplifiers latch the entire 1 KiB of data, acting as a high-speed internal cache called the **Row Buffer** (or "open page"). Only *one row per bank* can be open in the Row Buffer at any moment.

37.2.2 The Three Scenarios: Hit, Empty, and Conflict

When reading a 64-byte block of memory from a bank, hardware encounters one of three states:

1. **Row Buffer Hit (Open Page)**: The target address falls within the row already open in the Sense Amplifiers. The controller sends a **Column Address Strobe** ($\overline{\text{CAS}}$) command. The column multiplexer selects the 64-byte slice and streams it across the bus. The delay is purely the **CAS Latency** (t_{CL}):
$$T_{\text{hit}} = t_{\text{CL}} \approx 14 \text{ nanoseconds.}$$
2. **Row Buffer Empty (Precharged)**: No row is open; bitlines are equalized to $V_{DD}/2$. The controller sends an **Activate** (ACT) command with the row address, waits the **Row-to-Column Delay** (t_{RCD}) while cells latch into sense amplifiers, then sends $\overline{\text{CAS}}$ and waits t_{CL}:
$$T_{\text{empty}} = t_{\text{RCD}} + t_{\text{CL}} \approx 14 \text{ ns} + 14 \text{ ns} = 28 \text{ nanoseconds.}$$
3. **Row Buffer Conflict (Row Miss)**: Row B is requested, but Row A is open. The controller must execute three sequential steps:
   - *Precharge (PRE)*: Deactivate Row A and equalize bitlines to $V_{DD}/2$, taking the **Row Precharge Time** ($t_{\text{RP}} \approx 14$ ns).
   - *Activate (ACT)*: Open Row B and latch into sense amplifiers, taking $t_{\text{RCD}} \approx 14$ ns.
   - *Column Read ($\overline{\text{CAS}}$)*: Read the column slice, taking $t_{\text{CL}} \approx 14$ ns.

   Total latency is the sum of all three phases:
$$T_{\text{conflict}} = t_{\text{RP}} + t_{\text{RCD}} + t_{\text{CL}} \approx 14 \text{ ns} + 14 \text{ ns} + 14 \text{ ns} = 42 \text{ to } 55 \text{ nanoseconds.}$$

37.2.3 Reading Memory Timings on Hardware Labels

When RAM is sold as DDR4-3200 CL16-18-18-38, the numbers specify physical delays:

- **DDR4-3200**: I/O clock of 1,600 MHz (3,200 MT/s). Clock period $t_{\text{CK}} = 0.625$ ns.
- **CL16**: $t_{\text{CL}} = 16 \text{ cycles} \times 0.625 \text{ ns} = 10.0$ ns.
- **tRCD 18**: $t_{\text{RCD}} = 18 \text{ cycles} \times 0.625 \text{ ns} = 11.25$ ns.
- **tRP 18**: $t_{\text{RP}} = 18 \text{ cycles} \times 0.625 \text{ ns} = 11.25$ ns.
- **tRAS 38**: Row Active Time (38 cycles ≈ 23.75 ns), the minimum duration a row must stay open to fully recharge its capacitors.

Walking question

Tracing the Memory Bus on a Real Memory Access Trace
Let us trace the cycle-by-cycle physical behavior on a DDR4-3200 bus (0.625 ns per clock cycle) for two algorithms reading 128 consecutive 64-bit integers:

1. **Array Access**: All 128 integers fit inside 1,024 bytes = 1 KiB. Assuming the bank was initially precharged:
   - At cycle 0: Controller sends ACT. It waits $t_{\text{RCD}} = 18$ cycles (11.25 ns).
   - At cycle 18: Controller sends $\overline{\text{CAS}}$ for column 0. It waits $t_{\text{CL}} = 16$ cycles (10 ns).

- At cycle 34: Data burst begins. A 64-byte line transfers in 4 bus cycles (8 half-cycle beats).
- For the remaining 15 cache lines (120 integers), the row buffer is *already open*. The controller pipelines back-to-back $\overline{\text{CAS}}$ commands, streaming subsequent cache lines every 4 cycles without paying t_{RP} or t_{RCD} again!

2. **Pointer Chasing**: The 128 integers reside in 128 independently allocated nodes scattered across different heap rows. Every single access targets a different row in the same bank:
   - Each hop pays $t_{\text{RP}}(18\text{ cyc}) + t_{\text{RCD}}(18\text{ cyc}) + t_{\text{CL}}(16\text{ cyc}) + \text{Burst}(4\text{ cyc}) = 56$ cycles (35 ns).
   - For 128 elements, total bus time exceeds 7,168 cycles (4.48 μs), compared to fewer than 100 cycles for the contiguous array!

Focus

The Algorithmic Consequence of Row Buffers
A 1 KiB Row Buffer holds sixteen 64-byte cache lines (128 eight-byte integers). Scanning a contiguous array incurs one activation followed by fifteen consecutive *Row Buffer Hits* costing only t_{CL}. Pointer-chasing in a linked list or binary tree jumps to arbitrary rows in arbitrary banks, causing **Row Buffer Conflicts** ($t_{\text{RP}} + t_{\text{RCD}} + t_{\text{CL}}$) on nearly every hop.

Figure 37.2 compares the internal bank geometry and access timelines.

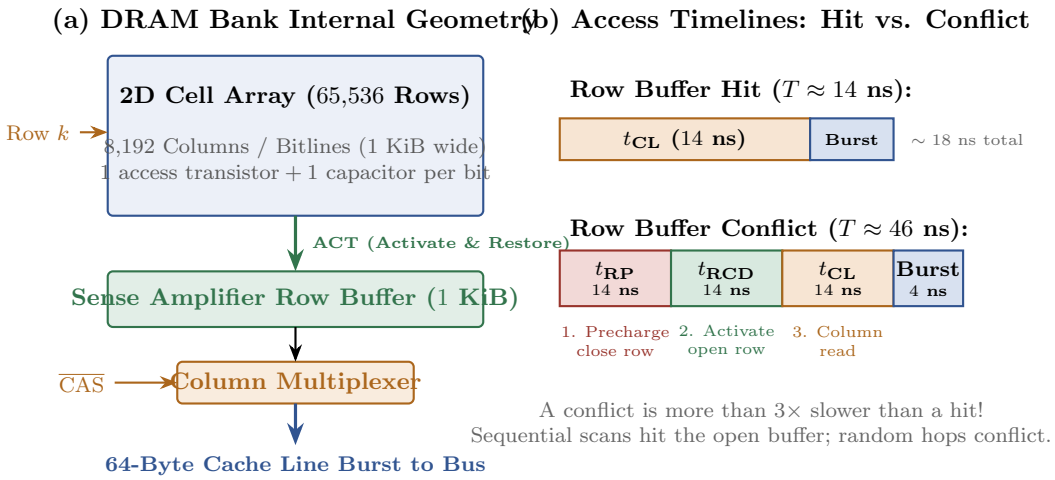


Figure 37.2: DRAM internal bank geometry and access latencies. (a) An activated row fills the 1 KiB Sense Amplifier Row Buffer, after which individual 64-byte chunks are extracted via column multiplexing. (b) A Row Buffer Hit pays only the CAS latency t_{CL}, while a Row Buffer Conflict must first close the active row (t_{RP}), activate the requested row (t_{RCD}), and then read the column (t_{CL}), more than tripling latency.

37.3 The CPU Cache Hierarchy and Working-Set Curves

Because trips to physical DRAM are so punishing, modern processors place a hierarchy of SRAM caches between registers and external memory.

37.3.1 The Hierarchy Levels: L1, L2, and L3

A contemporary x86 or ARM CPU core features three cache levels:

- **L1 Data Cache (L1d)**: Private to each core, 32–48 KiB, latency $\sim$ **4** clock cycles ($\approx$ 1 ns).
- **L2 Cache**: Private to each core, 512 KiB to 2 MiB, latency $\sim$ **12–14** clock cycles ($\approx$ 3–4 ns).
- **L3 Cache (Last-Level Cache)**: Shared across all cores, 16–128 MiB, latency $\sim$ **40–50 clock cycles** ($\approx$ 10–12 ns).
- **Main DRAM**: External off-chip, tens of gigabytes, latency $\sim$ **200–300 clock cycles** ($\approx$ 50–80 ns).

37.3.2 The 64-Byte Cache Line: The Quantum of Memory Movement

Data moves exclusively in aligned **64-Byte Cache Lines** (512 bits). Reading an 8-byte integer at `0x1000` automatically loads all 64 bytes spanning from `0x1000` to `0x103F`. The next seven consecutive integers are already present in L1 cache, accessible in a single cycle. Figure 37.3 contrasts how this 64-byte movement transforms the execution of the two summation algorithms introduced at the start of this chapter.

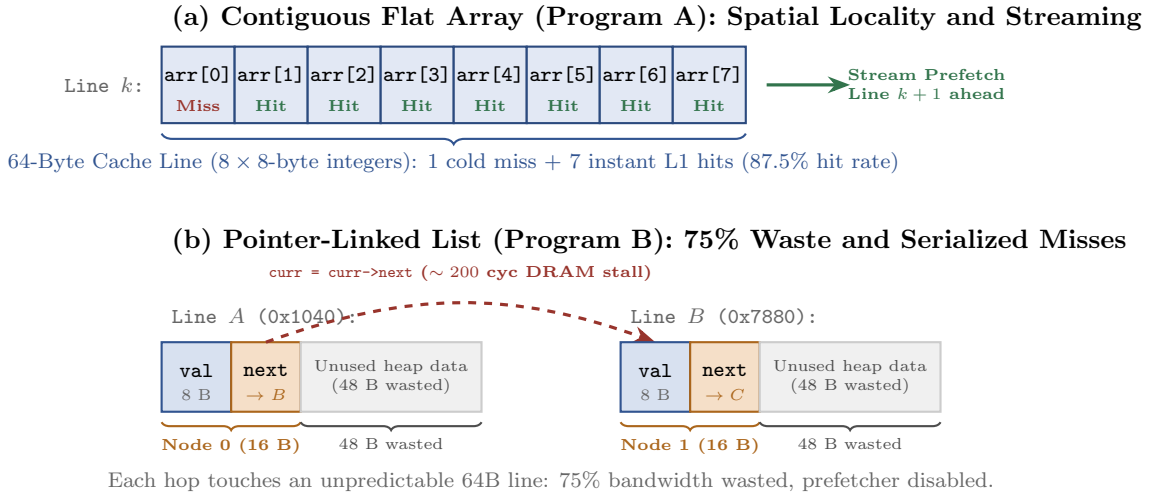


Figure 37.3: Cache line quantization and spatial locality. (a) In a contiguous array, loading a 64-byte cache line brings in eight 64-bit integers; the first access incurs a cache miss while the remaining seven hit in L1 in a single cycle, and the hardware prefetcher streams subsequent lines. (b) In a pointer-linked list, each 16-byte node resides on an independent cache line, discarding 48 bytes of transferred bandwidth and forcing serialized $\sim$ 200-cycle DRAM stalls on every pointer dereference.

As shown in Figure 37.3, Program A utilizes every byte transferred across the memory bus, turning

seven out of every eight accesses into instant single-cycle cache hits while the hardware prefetcher streams subsequent lines in the background. Program B discards three quarters of each fetched cache line and stalls the CPU on unpredictable pointer hops.

37.3.3 Set Associativity and the Power-of-Two Stride Disaster

Hardware implements W**-Way Set-Associative Caches**. An m-bit physical address is carved into:

$$\text{Physical Address: } [\ \underbrace{\text{Tag}}_{m-s-6 \text{ bits}} \ | \ \underbrace{\text{Set Index}}_{s \text{ bits}} \ | \ \underbrace{\text{Offset}}_{6 \text{ bits}} \ ]$$

where 6 bits select the byte in the 64-byte line, s bits select one of $S = 2^s$ sets, and the Tag is compared across all W ways simultaneously.

Common trap

Power-of-Two Stride Conflict Misses
Iterating down a column of a 1024×1024 float matrix (`matrix[i][j]`) jumps with a stride of 1024×4 bytes = 4,096 bytes = 2^{12} bytes. Every address in this column traversal has the exact same low 12 bits! In a 32 KiB 8-way cache (64 sets = 2^6 sets, indexed by bits 6–11), every access hashes to the exact same cache set. After 8 iterations, that single set is full, provoking a 100% cache miss rate while the remaining 63 sets sit completely empty.

37.3.4 Hardware Prefetchers and Ulrich Drepper's Working-Set Curves

Modern CPUs feature **Hardware Stream and Stride Prefetchers** that detect linear memory access and fetch lines $k+1, k+2, \ldots$ ahead of execution. In 2007, Ulrich Drepper demonstrated the effect of the cache hierarchy by measuring pointer chasing across a circular list of 64-byte nodes (`struct Node { Node* next; char pad[56]; }`):

- **Sequential Traversal**: Prefetchers detect the stride; latency stays flat at $\sim$ 4 cycles across all working-set sizes up to gigabytes!

- **Random Pointer Chasing**: Prefetchers cannot predict hops, producing a staircase: 4 cycles in L1 ($\leq$ 32 KiB), 14 cycles in L2 ($\leq$ 1 MiB), 45 cycles in L3 ($\leq$ 32 MiB), and soaring to 220 + cycles in DRAM.

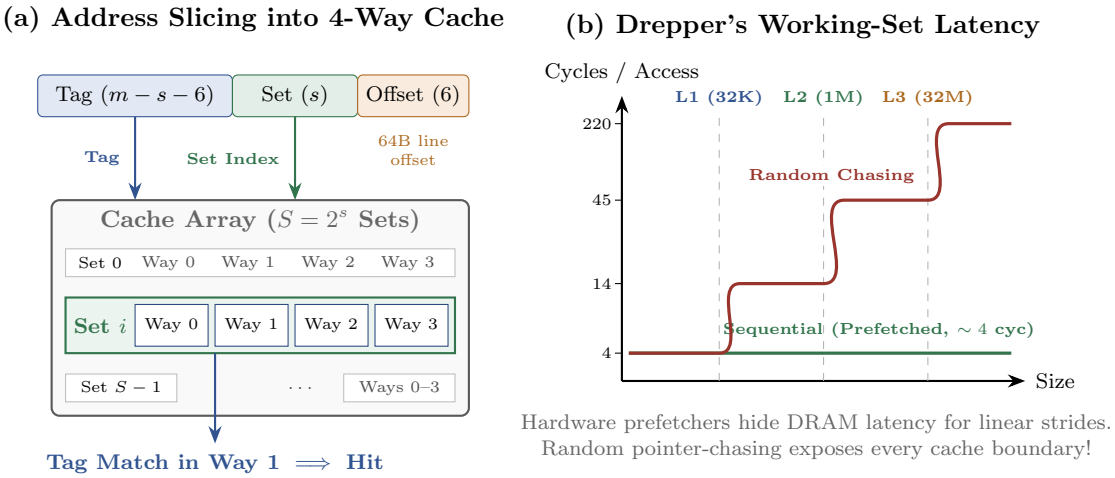


Figure 37.4: Cache organization and the physical latency staircase. (a) Physical address decomposition into Tag, Set Index, and Offset in a 4-way set-associative cache. (b) Ulrich Drepper's benchmark: sequential pointer chasing remains flat at ~ 4 cycles across all working-set sizes due to hardware stream prefetchers, whereas random pointer chasing climbs a staircase through L1, L2, L3, and DRAM.

Connection to other fields

From Cache Lines to Cache-Oblivious Algorithms

In Chapter 21, we study the van Emde Boas layout and cache-oblivious algorithms. A traditional cache-conscious algorithm tunes its tile size B directly to the hardware's 64-byte cache line or 4 KiB page size. By contrast, cache-oblivious algorithms achieve optimal cache transfers across *all* levels of the memory hierarchy simultaneously—L1, L2, L3, DRAM, and disk—without knowing the cache line size B or cache capacity M in advance! Understanding that memory transfers are quantized into blocks of size B allows us to design recursive divide-and-conquer algorithms (such as recursive matrix multiplication and Funnelsort) whose data layout automatically matches whichever hardware cache boundary is encountered.

37.4 Virtual Memory, Multi-Level Page Tables, and the TLB

Operating systems provide processes with private isolated address spaces via **Virtual Memory**, mapping virtual addresses to physical RAM frames.

37.4.1 The x86-64 4-Level Page Table Walk

Memory is sliced into 4 KiB pages (2^{12} bytes). A 48-bit Virtual Address is translated via a 4-level radix tree rooted at CPU register `CR3`:

$$\text{48-Bit Virtual Address: } [\underbrace{\text{PML4}}_{9\text{ bits}} \mid \underbrace{\text{PDP}}_{9\text{ bits}} \mid \underbrace{\text{PD}}_{9\text{ bits}} \mid \underbrace{\text{PT}}_{9\text{ bits}} \mid \underbrace{\text{Offset}}_{12\text{ bits}}]$$

Each table level holds 512 eight-byte entries (4 KiB). If a translation is not cached, the MMU performs a **Page Table Walk**, executing up to **four consecutive memory reads** ($\sim$ 240 ns) before accessing data.

Walking question

Translating a Virtual Address Step-by-Step
Consider the 48-bit virtual address `0x7FFF_DEAD_BEEF` in binary:

- Bits 47–39 (`0x0FF`): Selects entry 255 of the PML4 table specified by `CR3`. The entry holds the physical base address of a Page Directory Pointer Table.
- Bits 38–30 (`0x1FE`): Selects entry 510 of the PDP table, yielding the physical base of a Page Directory Table.
- Bits 29–21 (`0x15A`): Selects entry 346 of the PD table, yielding the physical base of a Page Table.
- Bits 20–12 (`0x1AD`): Selects entry 429 of the PT table, yielding the 36-bit *Physical Frame Number* (PFN).
- Bits 11–0 (`0xEEF`): Slices into the 4 KiB physical page at byte offset `0xEEF`.

Four pointers are read sequentially from memory. If any entry is missing, the CPU raises a hardware *Page Fault* exception (`#PF`), suspending user code while the OS kernel allocates physical backing storage.

37.4.2 The TLB and Huge Pages

The **Translation Lookaside Buffer (TLB)** caches virtual-to-physical translations on-chip, resolving hits in **1 clock cycle**. However, a typical L2 TLB holds only $\sim$ 1,500 entries:

$$\text{TLB Reach} = 1{,}500 \times 4 \text{ KiB} = 6 \text{ Megabytes}.$$

Modern application

Huge Pages in High-Performance Systems and Databases
When traversing a 32-gigabyte database index, the TLB covers under 0.02% of the memory, causing continuous TLB misses. Modern operating systems support **Huge Pages**:

- **2 MiB Huge Pages**: Translation terminates at the Page Directory level, expanding a 1,500-entry TLB reach to $1{,}500 \times 2$ MiB = 3.0 Gigabytes.
- **1 GiB Gigantic Pages**: Translation terminates at the PDP level, allowing a handful of entries to cover an entire server's memory space.

Enabling Huge Pages eliminates multi-level page table walks for large working sets.

37.5 Inside the Core and Across Cores: Pipeline Execution and MESI False Sharing

Inside the CPU, instructions execute out of order, while multicore systems enforce consistency using hardware coherence protocols.

37.5.1 Out-of-Order Superscalar Execution and Dependency Chains

Modern cores are speculative **Out-of-Order (OoO) Superscalar Engines**:

1. **Fetch and Predict**: 16–32 bytes/cycle fetched; branches predicted speculatively.
2. **Decode**: x86 instructions decomposed into RISC-like micro-ops (μops).
3. **Register Renaming**: Maps architectural registers to 180+ physical registers, dissolving Write-After-Read (WAR) and Write-After-Write (WAW) hazards.
4. **Reservation Stations and Reorder Buffer (ROB)**: Instructions wait for operands and dispatch out of order across 6–8 execution ports (IPC ≤ 4).

Independent operations (like multiple accumulators `s0 += a[i]; s1 += a[i+1]`) execute 4-wide per cycle. Pointer dereferences (`curr = curr->next`) form a **Serializing Pointer-Dependency Chain**, stalling the execution window for hundreds of cycles on cache misses.

37.5.2 Branch Prediction Penalties and Branchless Code

When data is unpredictable (e.g. Quicksort partition `if (arr[i] < pivot)`), branch predictors mispredict roughly 50% of the time. Each misprediction triggers a **Pipeline Flush**, wasting 15 to 20 clock cycles. Branchless code converts control dependencies into data dependencies using conditional moves:

```
pIndex += (arr[i] < pivot); // Branchless: zero pipeline flushes
```

37.5.3 Multicore Coherence and the MESI False Sharing Trap

Cores maintain coherent private caches via the **MESI Protocol**:

- **Modified (M)**: Exclusive, dirty (newer than RAM), writable.
- **Exclusive (E)**: Exclusive, clean (matches RAM), writable.
- **Shared (S)**: Shared across multiple cores, read-only.
- **Invalid (I)**: Contains no valid data; triggers cache miss.

Common trap

False Sharing Across Cores
When two threads update independent counters sharing the same 64-byte line (`int64_t cnt[2]`), each write invalidates the other core's cache. The line bounces back and forth across the interconnect via Read-For-Ownership (RFO) requests, making two parallel threads

run 10× slower than a single thread. **Fix**: Isolate variables onto distinct cache lines with `alignas(64)`.

37.5.4 Store Buffers, Memory Ordering, and Memory Barriers

Even with the MESI protocol, waiting for an RFO invalidation across the bus would stall the execution core on every store instruction. To decouple execution from coherence delays, hardware inserts a private FIFO **Store Buffer** between each core and its L1 cache.

When a store executes, the core writes the data into its Store Buffer and continues executing immediately. The Store Buffer drains asynchronously into L1 cache as invalidation acknowledgments arrive.

While the local core reads its own buffered writes via *Store-to-Load Forwarding*, other cores cannot observe writes sitting in an external core's private Store Buffer. This gives rise to **Store-Load Reordering** in x86 Total Store Order (TSO):

```
1 // Thread 0: x = 1; r1 = y;        // Thread 1: y = 1; r2 = x;
```

It is physically possible on real x86 hardware for both threads to observe $r_1 = 0$ and $r_2 = 0$, because both writes are buffered locally and have not yet reached coherent cache memory! To guarantee cross-core visibility in concurrent lock-free data structures, software must issue hardware **Memory Barriers** (`MFENCE` on x86, `DMB` on ARM, or `std::atomic_thread_fence` in C++), forcing the core to pause until its Store Buffer has completely drained into coherent cache lines.

Figure 37.5 contrasts the pipeline efficiency of independent versus serialized operations and illustrates how private store buffers cause store-load reordering in multicore execution.

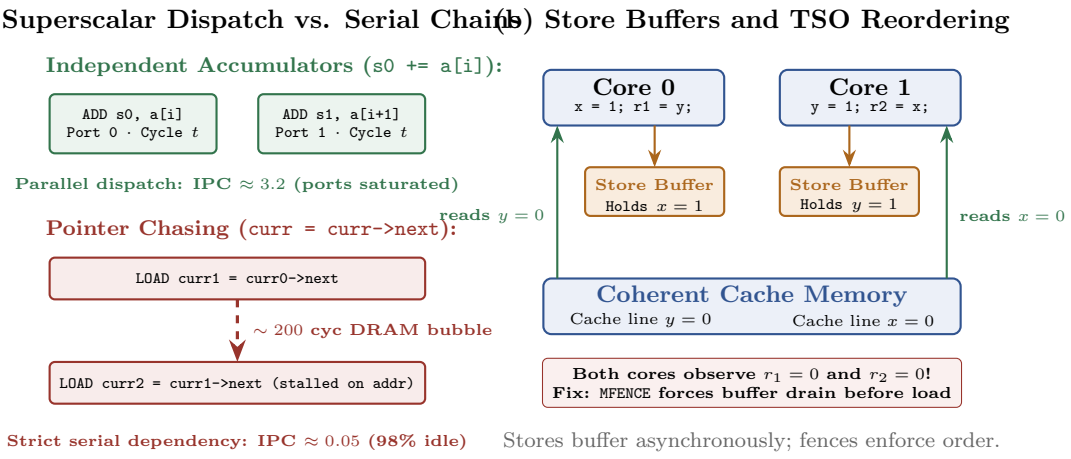


Figure 37.5: Superscalar instruction execution and memory consistency. (a) Independent accumulator additions saturate out-of-order execution ports without delay, whereas serialized pointer-chasing stalls the pipeline for hundreds of cycles waiting for memory addresses. (b) Private FIFO store buffers allow execution to continue before writes reach coherent cache, allowing external cores to observe stale values ($r_1 = 0, r_2 = 0$) unless synchronized with memory fences.

As shown in Figure 37.5(a), independent operations saturate the execution ports, whereas pointer dereferences create an unbreakable sequential dependency that starves execution units during memory stalls. In Figure 37.5(b), the private FIFO store buffer permits local execution to proceed past stores before they become globally visible, necessitating explicit memory fences in concurrent algorithms.

Algorithm lab

Measuring Memory and Pipeline Stalls with Hardware Performance Counters
Modern processors contain Hardware Performance Counters (PMUs). On Linux, you can directly observe the effects described in this chapter using `perf stat`:

```
$ perf stat -e cycles,instructions,cache-misses,branch-misses ./sum_array
  3,105,420,110  cycles                    #  3.91 GHz
 10,024,512,890  instructions              #  3.23 insn per cycle (IPC)
        156,210  cache-misses              #  0.015% of all refs
         12,410  branch-misses             #  0.02% of all branches

$ perf stat -e cycles,instructions,cache-misses,branch-misses ./sum_list
185,412,090,300  cycles                    #  3.89 GHz
 10,031,102,400  instructions              #  0.05 insn per cycle (IPC)
  9,985,140,210  cache-misses              # 99.54% of all refs
         14,200  branch-misses             #  0.03% of all branches
```

The instruction counts are virtually identical ($\sim 10^{10}$). But `sum_array` retires 3.23 instructions per cycle with almost zero cache misses, while `sum_list` drops to 0.05 instructions per cycle because the execution pipeline spends 98% of its clock cycles stalled waiting for DRAM cache misses!

Figure 37.6 illustrates address translation and false sharing.

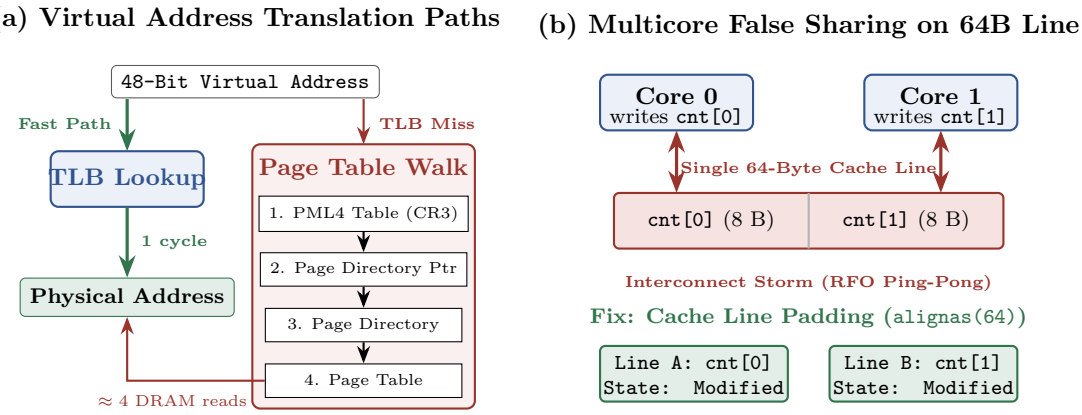


Figure 37.6: Virtual memory translation and multicore cache coherence. (a) A virtual address is checked against the TLB in 1 cycle; a TLB miss forces a 4-level page table walk in DRAM. (b) False sharing occurs when two independent variables share the same 64-byte cache line, causing constant invalidation ping-pongs across the interconnect; aligning data with `alignas(64)` isolates them into distinct lines.

37.6 Summary: Hardware-Conscious Algorithm Design

The algorithms studied throughout this book are analyzed asymptotically, but their actual speed depends on the hardware:

- **SRAM vs. DRAM**: 6T SRAM provides sub-nanosecond non-destructive access at high cost/power. 1T1C DRAM stores bits cheaply in capacitors, requiring destructive charge sharing, sense-amp restoration, and periodic refresh (t_{REFI}).
- **DRAM Banks and Row Buffers**: Row Buffer Hits cost only CAS latency ($t_{\text{CL}} \approx 14$ ns). Conflicts triple access time ($t_{\text{RP}} + t_{\text{RCD}} + t_{\text{CL}} \approx 42$–$55$ ns) to restore and precharge before activation.
- **64-Byte Cache Lines**: Memory moves exclusively in aligned 64-byte chunks. Sequential access exploits spatial locality and hardware stream prefetching.
- **Set Associativity and Strides**: Power-of-two strides in matrices collapse all accesses into a single cache set, provoking catastrophic conflict misses.
- **Virtual Memory and TLB Reach**: Translating virtual addresses requires a 4-level page table walk in memory. Huge Pages (2 MiB / 1 GiB) prevent repeated walks.
- **Pipelines and False Sharing**: Out-of-order cores stall on serializing pointer chains. Random branches flush 15–20 pipeline stages. Co-locating independent variables on a single 64-byte line triggers false sharing, degrading scalability.

Understanding hardware pipelines, cache lines, and register files is essential, yet algorithm designers rarely target machine opcodes by hand. Instead, we write high-level pseudocode or source languages and rely on an optimizing compiler to transform abstract logic into machine instructions. How does a compiler translate nested loops, allocate a finite register set using graph coloring, and schedule instructions to keep out-of-order pipelines fed? We examine how compilers bridge that gap next in Chapter 38.

Exercises

Exercise 37.1 (Quicksort Partitioning: Branchy vs. Branchless Selection)**.** Consider Lomuto partitioning on an array of $n = 10^7$ uniformly distributed, random 64-bit integers.

1. What is the theoretical probability that a randomly chosen element is smaller than the chosen pivot, assuming the pivot is the median? What is the expected branch misprediction rate of the condition `if (arr[i] < pivot)` on an idealized two-level branch predictor?
2. If a branch misprediction incurs a pipeline flush penalty of 18 clock cycles, calculate the total CPU cycles wasted purely on branch mispredictions during one full partition pass over n elements.
3. Write a branchless C++ implementation of Lomuto partitioning using conditional moves or bitwise operations (`pIndex += (arr[i] < pivot)`). Explain how this avoids pipeline flushes and what new data dependency constraint it introduces to the out-of-order execution engine.

Exercise 37.2 (Matrix Multiplication Loop Interchange and Cache Lines)**.** Consider multiplying two $N \times N$ dense matrices of single-precision floating point numbers (4 bytes each), $C = A \times B$, where $N = 2048$. Matrices are stored in contiguous row-major order. Assume a CPU with a 32 KiB,

8-way set-associative L1 data cache with 64-byte cache lines, using an LRU eviction policy. Assume no prefetching occurs.

1. Analyze the naive (i, j, k) loop ordering:

```
for (int i = 0; i < N; ++i)
    for (int j = 0; j < N; ++j)
        for (int k = 0; k < N; ++k)
            C[i][j] += A[i][k] * B[k][j];
```

   Determine the memory access stride of $B[k][j]$ across the innermost loop over k. How many cache misses does each inner loop iteration incur on matrix B?

2. Analyze the interchanged (i, k, j) loop ordering:

```
for (int i = 0; i < N; ++i)
    for (int k = 0; k < N; ++k)
        for (int j = 0; j < N; ++j)
            C[i][j] += A[i][k] * B[k][j];
```

   Determine the memory access strides of $A[i][k]$, $B[k][j]$, and $C[i][j]$ across the innermost loop over j. What is the theoretical cache miss rate per inner loop iteration?

3. Estimate the ratio of total memory traffic to DRAM between the two loop orderings.

Exercise 37.3 (B-Tree Node Sizing: Cache Lines, Memory Pages, and TLB Reach)**.** In Chapter 18, we proved that B-tree search cost is dominated by node visits, and that node sizes should match secondary storage block sizes.

1. Suppose an in-memory B-tree is designed to index 10^9 keys entirely within 64 gigabytes of main DRAM. If each node is sized to exactly one 64-byte cache line, what is the fanout of the tree assuming 8-byte keys and 8-byte pointers? What is the height of the tree?
2. Now suppose the node size is increased to 4 KiB (matching a standard operating system page). Compare the total number of DRAM visits required to locate a key in the 64-byte node tree versus the 4 KiB node tree.
3. Explain the trade-off: why does the 4 KiB node tree reduce DRAM row activations, and how does the use of 2 MiB Huge Pages alter the TLB miss rate during the traversal from the root to the leaf?

Exercise 37.4 (Cache-Conscious Hash Tables: Open Addressing vs. Chaining)**.** In Chapter 11, we demonstrated that hash tables with separate chaining and open addressing with linear probing both achieve $O(1)$ expected lookup time.

1. In separate chaining, each bucket contains a pointer to a linked list of dynamically allocated nodes. Trace the physical memory operations performed during a lookup of a non-existent key when the hash bucket is non-empty. How many cache lines and potential DRAM row buffer conflicts are touched?
2. In open addressing with linear probing, all keys reside in a flat contiguous array. If the load factor is $\alpha = 0.5$, how many keys on average are inspected during an unsuccessful search? How many 64-byte cache lines does this inspection touch if each key-value slot is 16 bytes?
3. Why does linear probing suffer severe degradation as $\alpha \to 1.0$ (primary clustering), and how does Robin Hood hashing preserve cache friendliness while bounding maximum probe sequence lengths?

Exercise 37.5 (Quantifying False Sharing in Parallel Prefix Sums)**.** A parallel reduction across $P = 8$ cores assigns each core n/P elements to sum into a global array of partial results:

```
int64_t partial_sums[8]; // Thread p writes exclusively to partial_sums[p]
```

1. If each thread updates its assigned slot `partial_sums[p]` on every addition for 10^7 iterations, calculate the total number of MESI invalidation requests transmitted across the processor interconnect.
2. If each interconnect invalidation and transfer transaction requires 40 nanoseconds, what is the total wall-clock time spent purely in coherence stalls?
3. Rewrite the data structure using `alignas(64)` to guarantee that each partial sum occupies its own unique cache line. Verify that the memory footprint changes from 64 bytes to 512 bytes, and explain why this eliminates interconnect saturation.

Chapter 38

Compilers: From Human Text to Optimizing Silicon

In Chapter 37, we saw how CPU pipelines, cache hierarchies, and DRAM bus cycles dictate real execution speed. But software engineers rarely write raw binary machine instructions or manually coordinate pipeline registers. Programs are written with variable names, arithmetic expressions, and control structures such as loops. The processor that runs them has none of these. It executes binary instructions on billions of transistors switching several billion times a second, and it knows only how to fetch a word from memory, decode its bit pattern into control signals, and write an arithmetic result into a register.

How does source text become a sequence of such machine instructions?

The program that performs this translation is a compiler. A modern optimizing compiler is a five-stage pipeline of data structures and graph algorithms, and earlier chapters of this book have already covered the main algorithm behind each stage:

- **Stage 1: Lexing** scans raw character streams into token sequences using the *Finite Automata* of Chapter 31.
- **Stage 2: Parsing** turns flat tokens into an unambiguous hierarchical *Tree* (Chapters 10 and 12) through recursive descent and precedence climbing.
- **Stage 3: Semantic Analysis** checks types and resolves variable scopes using an environment stack of *Hash Tables* (Chapter 11).
- **Stage 4: Intermediate Representation & Optimization** lowers trees into three-address code, builds a *Control-Flow Graph* of basic blocks, renames variables into *Static Single Assignment (SSA)* form to turn data dependencies into an explicit *Directed Acyclic Graph* (Chapter 22), eliminates duplicate computations via hash tables, and hoists loop invariants.
- **Stage 5: Code Generation & Register Allocation** maps an infinite supply of virtual variables down to a handful of physical CPU registers by solving the *Graph Coloring Problem* (Chapter 33) via Kempe's degree-peeling heuristic, and selects optimal instruction sequences using *Dynamic Programming* (Chapter 15).

We will follow a single four-line program throughout this chapter:

```
sum = 0
```

```
while (i < n) {
    sum = sum + a[i] * 2
    i = i + 1
}
```

We will trace these four lines by hand from their raw ASCII bytes on disk, through tokens, through an abstract syntax tree, through scoped type lowering, through control-flow graphs and SSA ϕ-functions, through four classical optimizing transformations, and finally through liveness analysis and graph-coloring register allocation down to a six-instruction register-only x86-64 loop.

38.1 History and Overview

History and connection

In 1952, Grace Murray Hopper was working at Remington Rand on the UNIVAC I. Programs were written directly in octal machine code, with programmers looking up operation codes and memory addresses in thick binders. Hopper realized that programmers spent days re-deriving routine operations: evaluating sines, taking square roots, or copying memory blocks. She built the A-0 system, which took a sequence of call descriptors, searched a magnetic tape library of subroutines, adjusted absolute addresses, and linked them into an executable routine. Hopper coined the word "compiler" because her program compiled subroutines from a library. When she presented it, contemporaries scoffed, insisting that machines were calculating engines designed for arithmetic, not writing programs.

Between 1954 and 1957, John Backus led a small team at IBM to develop FORTRAN (Formula Translating System) for the IBM 704. In an era where mainframes cost millions of dollars and CPU time was billed by the minute, hand-written assembly was considered indispensable. Skeptics insisted that machine-generated code would run at least five times slower than human assembly, making high-level languages an unaffordable extravagance. Backus understood that unless FORTRAN's output rivaled handcrafted assembly in execution speed, the project would be abandoned. This performance mandate forced Backus's team to invent compiler optimization from the ground up: they created the first algorithms for loop nest analysis, execution frequency weighting, and index register allocation. When FORTRAN was delivered, its emitted code ran with such efficiency that programmers rapidly left manual assembly behind.

The theoretical foundations took shape in the late 1950s. In 1956, Noam Chomsky formulated his hierarchy of formal grammars, showing that regular languages (recognized by finite automata) are strictly less expressive than context-free languages (which require a stack). When John Backus designed the grammar of ALGOL 58, he introduced a formal metalanguage, which Peter Naur refined for the landmark ALGOL 60 report into Backus–Naur Form (BNF). BNF replaced ambiguous English prose specifications with mathematically rigorous substitution rules, making parsing a systematic, grammar-driven task.

During the 1960s and 1970s at IBM Research, Frances Allen and John Cocke created the architectural framework of modern optimizing compilers. Allen formalized the control-flow graph, introduced interval analysis and dominator trees, and built the mathematical theory of dataflow equations. Her 1970 paper "Control Flow Analysis" and her joint work with Cocke established the classical repertoire of loop optimizations—including loop-invariant code motion

and induction variable strength reduction—that every modern compiler performs. Allen was later awarded the ACM A.M. Turing Award for this work.

Yet for three decades, register allocation remained an ad-hoc heuristic. A CPU possesses only a handful of hardware registers, while programs manipulate scores of variables. In 1981, Gregory Chaitin at IBM showed that assigning K physical registers to variables without interference is mathematically isomorphic to the graph vertex coloring problem investigated by Alfred Kempe a century earlier. While graph coloring is NP-complete, Chaitin showed that a degree-peeling heuristic colors real-world program graphs near-optimally in linear time, giving compilers their first principled register allocator.

In 1989, Ron Cytron, Jeanne Ferrante, Barry Rosen, Mark Wegman, and Kenneth Zadeck introduced Static Single Assignment (SSA) form. By placing ϕ-nodes at control-flow joins, SSA ensured that every variable is assigned at exactly one syntactic site. This decoupled values from memory cells, turning tangled dataflow into clean directed acyclic graphs and accelerating optimization passes from quadratic to linear time. In 2004, Chris Lattner and Vikram Adve at UIUC unveiled LLVM, establishing a language-independent, typed SSA intermediate representation. Today, LLVM, GCC, and just-in-time engines such as Google's V8 and PyTorch's `torch.compile` build directly upon this foundation.

Focus

A compiler lowers representations through five distinct transformations:

$$\text{Characters} \xrightarrow[\text{Lex}]{\text{FA}} \text{Tokens} \xrightarrow[\text{Parse}]{\text{CFG}} \text{AST} \xrightarrow[\text{Check}]{\text{Types}} \text{SSA IR} \xrightarrow[\text{Optimize}]{\text{Graphs}} \text{SSA IR} \xrightarrow[\text{Backend}]{\text{Coloring}} \text{Assembly}$$

Each stage drops surface syntax and makes more of the program's structure explicit.

Figure 38.1 illustrates this three-tiered architecture. By interposing a shared intermediate representation between languages and targets, modern compilers solve the classic $N \times M$ proliferation problem: N language frontends compile into one shared IR, which undergoes machine-independent optimization before M backends emit native machine code.

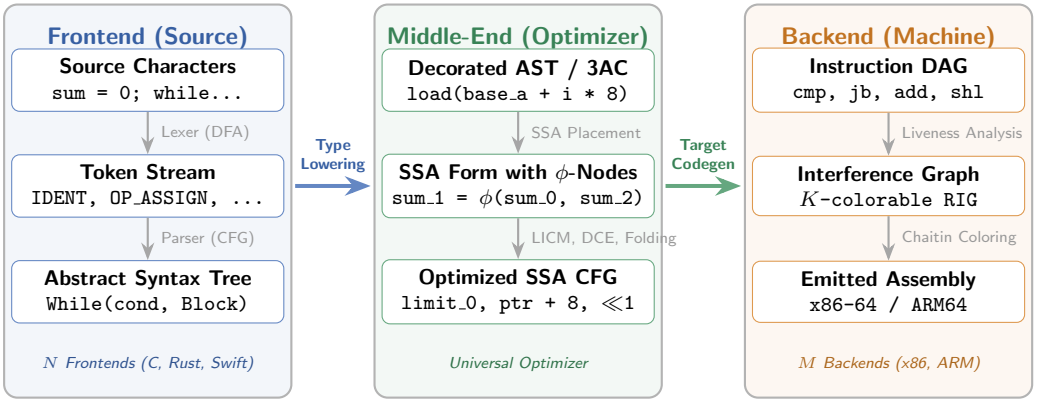


Figure 38.1: The three-tiered pipeline of modern optimizing compilers. The source-specific Frontend parses characters into a typed AST; the target-independent Middle-End optimizes code in SSA form via loop-invariant motion, strength reduction, and dead code elimination; and the hardware-specific Backend constructs a register interference graph to assign bounded physical registers via graph coloring.

38.2 Stage 1: Lexing — From Character Streams to Tokens via Finite Automata

The input to a compiler is a file on disk: a flat byte array of ASCII or UTF-8 characters. When your editor saves our running program, the file contains:

`s`, `u`, `m`, ` `, `=`, ` `, `0`, `\n`, `w`, `h`, `i`, `l`, `e`, ...

Why can the parser not read this character array directly?

If the parser operated on raw characters, every single grammar rule would have to account for optional whitespace, tabs, line breaks, and comments between every letter of every keyword and identifier. In our running example, the grammar would need rules describing that `'s'` followed by `'u'` followed by `'m'` forms the identifier `sum`, and would have to permit arbitrary spaces between `s` and `u`. Such an approach leads to a combinatorial explosion of grammatical rules.

The compiler solves this by delegating character scanning to Stage 1: the *lexical analyzer* or *lexer*. The lexer scans the raw byte stream, discarding whitespace and comments and grouping contiguous characters into atomic units called **tokens**.

Definition 38.1 (Token)**.** A *token* is a pair (type, attribute), where type is an enumerated category representing a syntactic role (such as an identifier, keyword, or operator), and attribute is the optional payload (such as the string text of an identifier or the numerical value of a literal).

Table 38.1 displays the stream of tokens produced by the lexer for our running program.

Snippet	Token type	Attribute	Snippet	Token type	Attribute
sum	IDENT	"sum"	a	IDENT	"a"
=	OP_ASSIGN	—	[	LBRACKET	—
0	INT_LIT	0	i	IDENT	"i"
while	KW_WHILE	—	]	RBRACKET	—
(	LPAREN	—	*	OP_STAR	—
i	IDENT	"i"	2	INT_LIT	2
<	OP_LT	—	i	IDENT	"i"
n	IDENT	"n"	=	OP_ASSIGN	—
)	RPAREN	—	i	IDENT	"i"
{	LBRACE	—	+	OP_PLUS	—
sum	IDENT	"sum"	1	INT_LIT	1
=	OP_ASSIGN	—	}	RBRACE	—
sum	IDENT	"sum"	+	OP_PLUS	—

Table 38.1: The token stream produced by lexical analysis on our running example.

38.2.1 The Maximal Munch Principle

Tokenizing introduces boundary ambiguities. When encountering `<=`, does the lexer emit `OP_LE`, or `OP_LT` followed by `OP_ASSIGN`? When reading `while1`, is it the keyword `while` followed by 1, or a single identifier?

Lexers resolve this with the **Maximal Munch** principle (the *longest match rule*):

Formal

Maximal Munch Rule: The lexer consumes the longest prefix of remaining input that matches a valid token specification. When the longest match fits multiple patterns (for example, `while` matches both keyword and identifier rules), reserved keyword rules take precedence over general identifier rules.

Under Maximal Munch, `<=` is recognized as a single `OP_LE` token because two characters form a longer match than one. Similarly, `while1` is treated as an identifier.

38.2.2 Scanning via Deterministic Finite Automata

To execute Maximal Munch in strictly linear time, compilers compile regular expressions into a **Deterministic Finite Automaton (DFA)** (Chapter 31). A DFA consists of a state set Q, an alphabet Σ, a transition function $\delta : Q \times \Sigma \to Q$, a start state q_0, and accepting states $F \subseteq Q$.

Algorithm 155: DFA-Lexer-Next-Token

Input: Character stream S, start index p, transition table δ, accepting map F.
Output: Next token T, updated position p.

$q \leftarrow q_0$; last_accepting_state $\leftarrow \bot$; last_pos $\leftarrow p$; scan_pos $\leftarrow p$;
while *scan_pos* $\leq |S|$ ***and*** $\delta(q, S[\textit{scan_pos}]) \neq \bot$ **do**
  $q \leftarrow \delta(q, S[\text{scan_pos}])$;
  scan_pos $\leftarrow$ scan_pos $+ 1$;
  if $q \in F$ **then**
    last_accepting_state $\leftarrow q$; last_pos $\leftarrow$ scan_pos;
if *last_accepting_state* $= \bot$ **then**
  return Error: Unrecognized character at position p;
$T \leftarrow$ ConstructToken(last_accepting_state, $S[p \,..\, \text{last_pos} - 1]$);
return $(T, \text{last_pos})$;

Because the transition table $\delta[q, c]$ is precomputed in an array, each character triggers an $O(1)$ state lookup. The entire source file of n bytes is tokenized in $O(n)$ time with zero backtracking.

38.3 Stage 2: Parsing — From Flat Tokens to an Abstract Syntax Tree

The lexer gives us a flat sequence of tokens. But computer programs are intrinsically hierarchical: loops enclose blocks, blocks contain statements, and statements enclose nested expressions.

38.3.1 Why Finite Automata Cannot Parse Code

Why not use finite automata to parse programming languages?

The limitation is mathematical. A finite automaton has a bounded number of states: $|Q| = m$.

Consider matching nested parentheses:

$$\underbrace{((((\ldots(}_{k \text{ times}} a + b \underbrace{)\ldots))))}_{k \text{ times}}$$

To verify that the count of closing parentheses matches the opening ones, the machine must store the current nesting depth. If the nesting depth k exceeds the state count m, the pigeonhole principle forces the automaton to enter some state q at least twice during the opening sequence. Having entered a cycle, the automaton cannot distinguish k open parentheses from $k + p$ open parentheses. It cannot verify that the closing parentheses match.

Parsing nested syntax requires unbounded memory: a pushdown automaton with a LIFO stack. The mathematical formalism for pushdown automata is the **Context-Free Grammar (CFG)**.

Definition 38.2 (Context-Free Grammar)**.** A *Context-Free Grammar* is a 4-tuple $G = (V, \Sigma, R, S)$, where V is a finite set of nonterminal symbols, Σ is the set of terminal tokens, R is a set of production rules of the form $A \to \alpha$ with $A \in V$ and $\alpha \in (V \cup \Sigma)^*$, and $S \in V$ is the start symbol.

38.3.2 The Trap of Grammar Ambiguity

Consider a naive expression grammar:

$$E \;\to\; E + E \;\mid\; E * E \;\mid\; \texttt{id} \;\mid\; \texttt{num}$$

Now consider line 3 of our running program: `sum+a[i]*2`. Treating `a[i]` as x, this grammar allows two distinct parse trees:

1. **Tree 1 (Addition first):** Apply $E \to E * E$ first, producing $(E + E) * E$, which computes $(\texttt{sum} + x) * 2$.
2. **Tree 2 (Multiplication first):** Apply $E \to E + E$ first, producing $E + (E * E)$, which computes $\texttt{sum} + (x * 2)$.

Common trap

A grammar allowing two or more distinct parse trees for the same input string is **ambiguous**. For $3 + 4 * 2$, Tree 1 evaluates to $(3 + 4) * 2 = 14$, while Tree 2 evaluates to $3 + (4 * 2) = 11$. A compiler cannot leave operator precedence to chance.

38.3.3 Resolving Ambiguity: Recursive Descent and Operator Precedence Climbing

Compilers eliminate ambiguity by stratifying grammar rules into explicit precedence tiers:

$$\begin{aligned}
E &\to E + T \;\mid\; T && \text{(Expressions: lowest precedence, addition)} \\
T &\to T * F \;\mid\; F && \text{(Terms: medium precedence, multiplication)} \\
F &\to \texttt{id} \;\mid\; \texttt{num} \;\mid\; \texttt{id}[E] \;\mid\; (E) && \text{(Factors: highest precedence, primary atoms)}
\end{aligned}$$

In this stratified grammar, Tree 2 is the unique valid derivation: any multiplication is parsed inside T, forcing $T * F$ to become a child of $E + T$.

In modern production compilers, this hierarchy is implemented via **Recursive Descent** or Vaughan Pratt's 1973 **Operator Precedence Climbing**. In Pratt parsing, every operator token has a numerical *binding power*. When parsing `sum + a[i] * 2`, the parser reads `+` (binding power 10). When the recursive sub-call encounters `*` (binding power 20), the higher binding power ($20 > 10$) pulls `a[i]` and `2` into the multiplication subtree before the addition operator can close.

38.3.4 The Abstract Syntax Tree (AST)

Concrete parse trees retain punctuation tokens such as parentheses and semicolons. Once grouping has been established, the compiler discards punctuation and constructs an **Abstract Syntax Tree (AST)**. Interior nodes represent operations (`While`, `Assign`, `BinOp`), and leaves represent operands. Figure 38.2 illustrates the pipeline from raw characters to tokens, and finally to an AST.

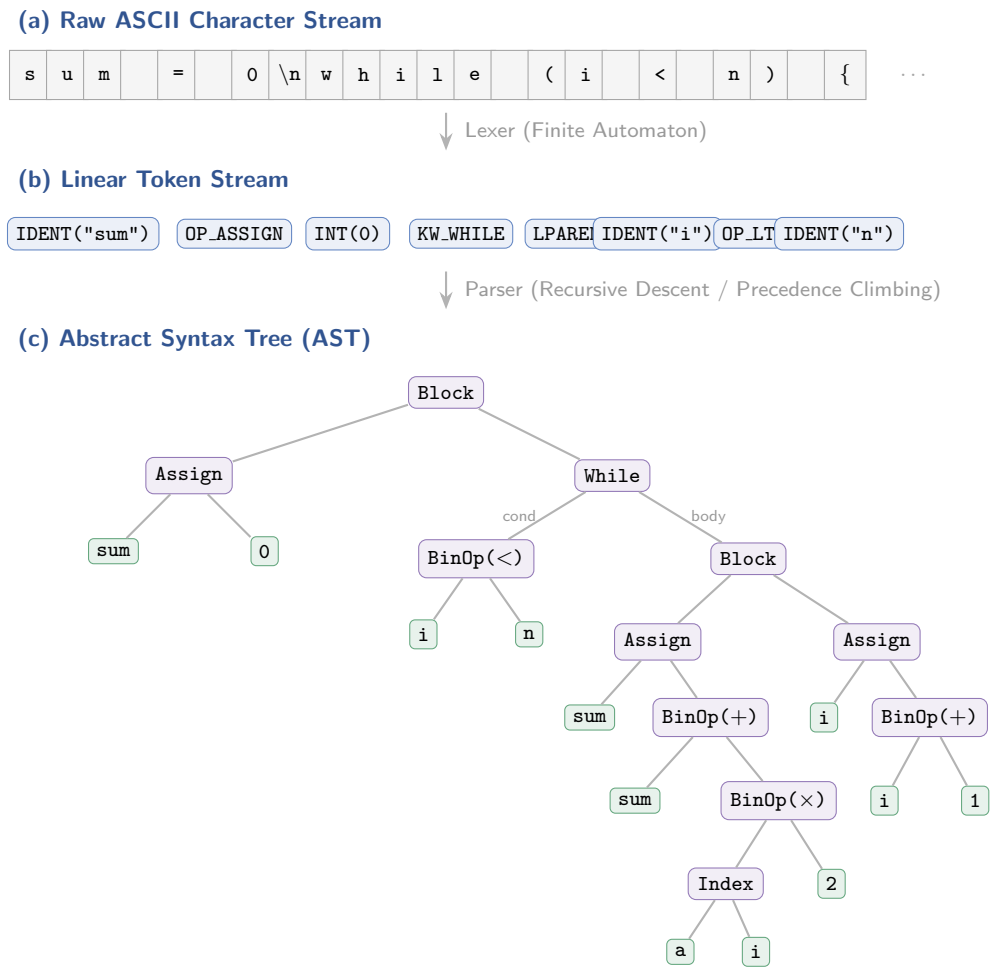


Figure 38.2: Tracing our running example across the compiler frontend: (a) raw character stream, (b) token stream after lexing, and (c) the resulting Abstract Syntax Tree (AST). Notice how operator precedence pushes BinOp(×) strictly deeper than BinOp(+), eliminating typographical ambiguity without parentheses.

38.4 Stage 3: Semantic Analysis — Scoped Symbol Tables and Type Lowering

The AST in Figure 38.2 captures structure, but structure alone does not guarantee validity. Consider the syntactically valid statement:

```
x = y + 2
```

The parser accepts this statement without complaint. Yet at compile time, we must know: Has `y` been declared? What is its type? If `y` is an open network socket or a file handle, adding 2 to it is invalid.

Stage 3, called **Semantic Analysis**, walks the AST to enforce semantic validity and resolve variable bindings.

38.4.1 Scoped Symbol Tables as a Stack of Hash Tables

To track variable bindings across nested lexical blocks, compilers maintain a **Symbol Table** using a stack of the **Hash Tables** introduced in Chapter 11:

- **Entering a scope:** When the AST walker enters a block delimited by `{ ...}`, it pushes a fresh, empty hash table onto the scope stack. Each table retains a pointer to its enclosing parent table.

- **Declaring a symbol:** Variable declarations insert the identifier and its metadata (type, bit-width, memory offset) into the top hash table.

- **Resolving a symbol:** When an identifier appears in an expression, the compiler queries the top hash table in $O(1)$ expected time. If absent, it traverses parent pointers up to the global table. Inner variables naturally *shadow* outer variables of the same name.

- **Exiting a scope:** Leaving a block pops the top table from the stack, instantly de-allocating local variables from visibility.

Figure 38.3 visualizes this stack discipline for our running example: nested syntactic blocks map directly to stacked hash tables connected by parent pointers.

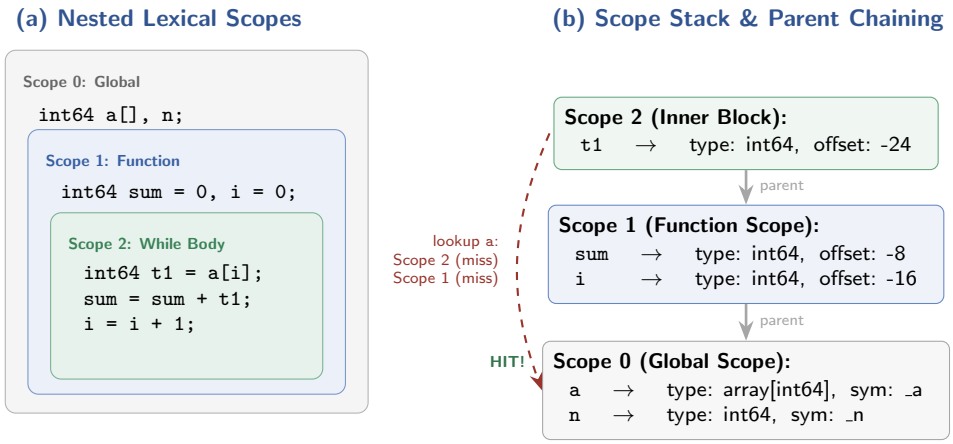


Figure 38.3: Scoped symbol tables organized as a stack of hash tables with enclosing parent pointers. In panel (a), nested blocks establish a lexical scope hierarchy. In panel (b), entering a scope pushes a new table with a parent link; identifier lookups probe the local table first and traverse parent pointers upward until hitting the binding or reaching the global scope.

38.4.2 Type Checking and Lowering

Once identifiers are bound, the compiler annotates each AST node with its static type. In our running example:

$$\texttt{sum} : \text{int64}, \quad \texttt{i} : \text{int64}, \quad \texttt{n} : \text{int64}, \quad \texttt{a} : \text{array}[\text{int64}].$$

The semantic analyzer verifies that `i` $<$ `n` compares two 64-bit integers and that array elements match the type of `sum`.

Next comes **Type Lowering**. Modern silicon has no instruction that takes an abstract array reference and returns an element. In hardware, array `a` is simply a memory address pointer base_a. Because each 64-bit integer requires 8 bytes of storage, element i sits at byte address:

$$\text{address}(a[i]) = \text{base}_a + i \times 8.$$

The semantic analyzer lowers $\text{Index}(a, i)$ into an explicit pointer dereference:

$$\text{Index}(a, i) \implies \text{load}\big(\text{base}_a + i \times 8\big).$$

By making array indexing arithmetic explicit, the compiler exposes the hidden multiplication by 8 and the memory fetch to subsequent optimization passes.

38.5 Stage 4: Intermediate Representation, Control-Flow Graphs, and SSA Form

Translating M source languages to N computer architectures directly requires $M \times N$ compilers. Production compilers avoid this by compiling frontends to a common **Intermediate Representation (IR)**. With a universal IR, we need only M frontends and N backends ($M + N$ components), and all optimizations run on the shared IR.

38.5.1 Three-Address Code and the Control-Flow Graph

Modern IRs express computation as **Three-Address Code (3AC)**, where nested expressions are broken into flat instructions with at most one operator: $x \leftarrow y$ op z. To model control flow, instructions are grouped into **Basic Blocks** connected in a **Control-Flow Graph (CFG)** (Chapter 22).

Definition 38.3 (Basic Block and CFG)**.** A *Basic Block* is a maximal sequence of consecutive instructions where control enters exclusively at the first instruction and leaves exclusively at the last instruction. A *Control-Flow Graph* is a directed graph $G = (V, E)$ where vertices V are basic blocks and directed edges $(B_1, B_2) \in E$ represent possible branch or fallthrough transfers.

38.5.2 The Mutable Variable Dilemma and Static Single Assignment (SSA)

In high-level languages, variables are mutable memory cells. In our loop, `sum` is updated on every iteration. This mutability obscures dataflow: to determine what value `sum` holds at an instruction, an optimizer must trace all incoming execution paths.

In 1989, Cytron et al. introduced the standard representation for modern optimizers: **Static Single Assignment (SSA)** form.

Formal

The SSA Property: A program is in *Static Single Assignment form* if and only if every variable is assigned a value at most once in the program text.

Instead of overwriting variables, each assignment creates a uniquely numbered, immutable version: $\text{sum}_0, \text{sum}_1, \text{sum}_2$. Each variable becomes an immutable value, and definition-to-use relationships form explicit edges in a Directed Acyclic Graph (DAG).

38.5.3 The ϕ (Phi) Function at Control-Flow Joins

At the top of our `while` loop, `sum` has value $\text{sum}_0 = 0$ on iteration 0 (from the entry block), but has value sum_2 on iteration $k \geq 1$ (from the loop body). Because an SSA variable cannot be assigned twice, how does the loop header refer to `sum`?

SSA introduces a ϕ **(phi) function**:

$$\text{sum}_1 = \phi\big(B_{\text{entry}} : \text{sum}_0,\ \ B_{\text{body}} : \text{sum}_2\big).$$

A ϕ-node is not a runtime instruction; microprocessors spend zero clock cycles executing it. It is a compile-time marker that instructs the optimizer: *"If execution arrived from B_{entry}, use sum_0; if execution arrived from B_{body}, use sum_2."*

38.5.4 Four Classic Optimizations Simplified by SSA

Because SSA turns variable updates into explicit def-use graph edges, program optimizations become clean graph rewrites:

1. **Constant Propagation and Folding:** Constant operands are evaluated at compile time. Constant multiplications such as $t_2 \times 2$ undergo *strength reduction* into single-cycle bitwise shifts: $t_2 \ll 1$.

2. **Dead Code Elimination (DCE):** In SSA, every variable tracks its list of uses. If an SSA variable has zero uses and its defining instruction has no side effects, the instruction is deleted in $O(1)$ operations.

3. **Common Subexpression Elimination (CSE) / Global Value Numbering (GVN):** Expressions are hashed into a Chapter 11 Hash Table keyed by $(\text{operator}, \text{val}_1, \text{val}_2)$. Duplicate computations are replaced with the precomputed value ID.

4. **Loop-Invariant Code Motion (LICM) and Strength Reduction:** In our loop, i_1 is an induction variable stepping by 1. The address $\text{base}_a + i_1 \times 8$ therefore steps by exactly $1 \times 8 = 8$ bytes per iteration. The compiler replaces the loop multiplication with a pointer variable ptr that increments by 8:

$$\text{ptr}_0 = \text{base}_a, \quad \text{ptr}_2 = \text{ptr}_1 + 8.$$

   It then hoists the termination limit $\text{limit}_0 = \text{base}_a + (n_0 \ll 3)$ into the entry block. The loop condition becomes $\text{ptr}_1 < \text{limit}_0$, and i is eliminated by DCE!

Figure 38.4 shows the CFG before and after these transformations.

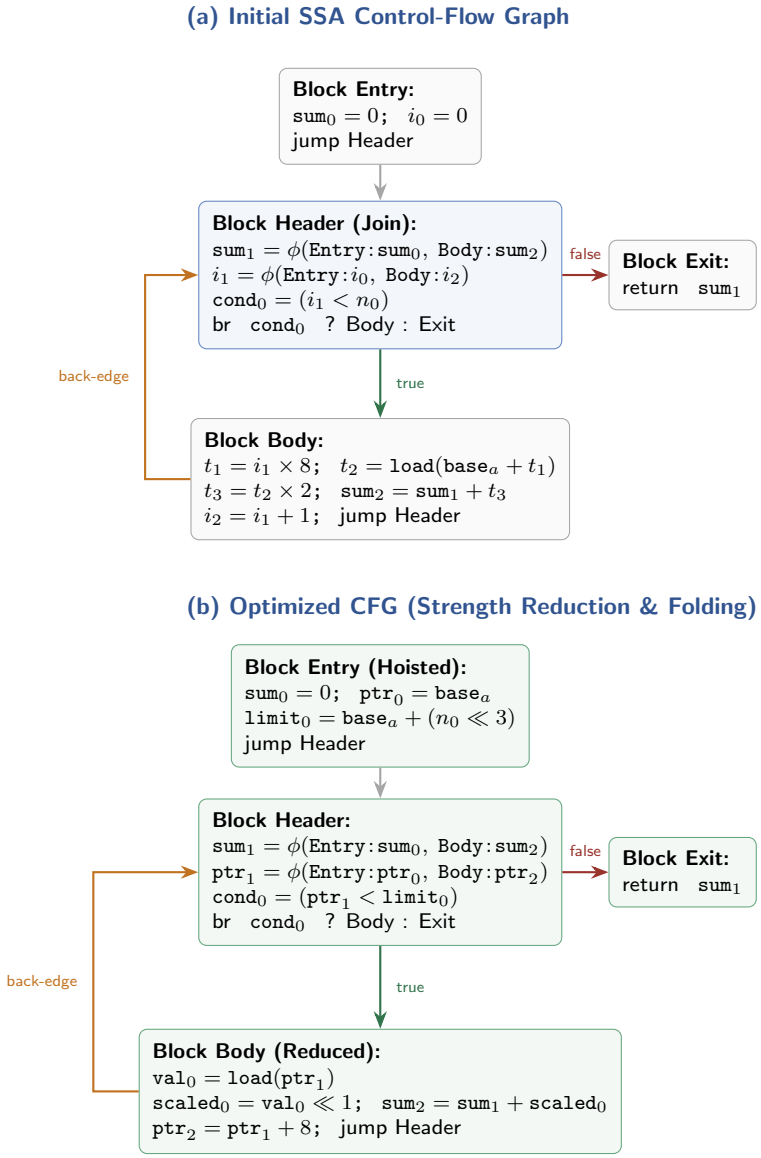


Figure 38.4: The Control-Flow Graph (CFG) in Static Single Assignment (SSA) form: (a) initial SSA form with ϕ-nodes at the loop header join point, and (b) after hoisting limit_0, reducing $i \times 8$ to pointer increment $\text{ptr} + 8$, and folding $t_2 \times 2 \rightarrow \text{val}_0 \ll 1$. All multiplications have been removed from the inner loop.

38.6 Stage 5: The Backend — Instruction Selection, Liveness, and Register Allocation

The optimizer leaves our loop with an infinite supply of virtual variables. But microprocessors have a strictly bounded bank of physical registers (16 on x86-64, 31 on ARM64). Register accesses cost zero extra clock cycles; spilling a variable to stack RAM incurs memory latency.

The backend maps virtual variables to K physical registers without allowing concurrently live values to overwrite each other.

38.6.1 Liveness Analysis: Working Backward Through Time

To assign registers safely, the compiler computes when each variable is active:

Definition 38.4 (Live Variable)**.** A variable v is *live* at program point p if there exists an execution path from p that reads v before any redefinition of v.

Liveness is computed **backward** from loop exit to entry:

$$\text{LiveIn}[B] = \text{Use}[B] \cup \big(\text{LiveOut}[B] \setminus \text{Def}[B]\big).$$

Tracing the five instructions of the optimized loop body:

1. **Insn 1:** val = load(ptr) — Live in: $\{\text{sum}, \text{ptr}, \text{limit}\}$. Live out: $\{\text{sum}, \text{ptr}, \text{limit}, \text{val}\}$.
2. **Insn 2:** scaled = val $\ll 1$ — Reads val; defines scaled. Variable val is never read again and **dies**! Live out: $\{\text{sum}, \text{ptr}, \text{limit}, \text{scaled}\}$.
3. **Insn 3:** sum = sum + scaled — Reads scaled; variable scaled **dies**! Live out: $\{\text{sum}, \text{ptr}, \text{limit}\}$.
4. **Insn 4:** ptr = ptr + 8 — Updates ptr. Live out: $\{\text{sum}, \text{ptr}, \text{limit}\}$.
5. **Insn 5:** cmp ptr, limit; jb loop — Compares ptr and limit; jumps to header.

Here val dies at instruction 2 and scaled is born at instruction 2. Their live ranges **never overlap in time**.

38.6.2 The Register Interference Graph and Chaitin's Graph Coloring

Definition 38.5 (Register Interference Graph)**.** The *Register Interference Graph (RIG)* is an undirected graph $G = (V, E)$ where vertices V are program variables, and an edge $(u, v) \in E$ indicates that u and v are simultaneously live at some program point.

In 1981, Gregory Chaitin proved that assigning K physical registers to variables without conflict is isomorphic to K**-Vertex Coloring** (Chapter 33). Although graph coloring is NP-complete, compilers use the **Chaitin–Briggs degree-peeling heuristic**:

1. **Simplify:** Find a vertex v with $\deg(v) < K$. Push v onto a stack and remove it from the graph. Because v has at most $K - 1$ neighbors, even if all neighbors receive distinct colors, at least one color among $\{1, \ldots, K\}$ is guaranteed to remain free for v!
2. **Spill:** If all vertices have degree $\geq K$, select a spill candidate for stack storage and continue peeling.
3. **Select:** Pop vertices from the stack and assign each vertex any color unused by its already-colored neighbors.

Tracing on our loop with $K = 4$ registers (R_0, R_1, R_2, R_3):

- val and scaled have deg $= 3 < 4$.
- There is **no edge** between val and scaled because their live intervals are disjoint.
- Simplify peels scaled, then val, then the rest.

- The Select phase colors: sum $\mapsto R_0$, ptr $\mapsto R_1$, limit $\mapsto R_2$, val $\mapsto R_3$, and scaled $\mapsto R_3$.

Variables val and scaled share physical register R_3 with zero conflicts! Figure 38.5 illustrates the live ranges and colored graph.

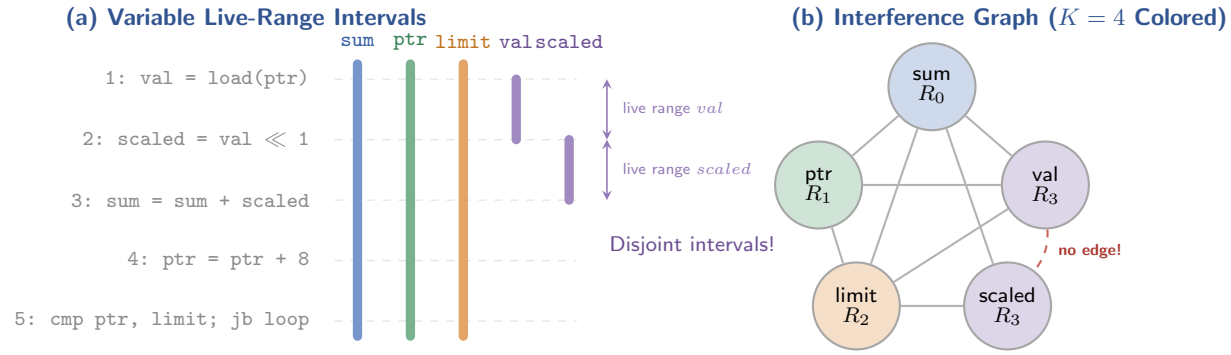


Figure 38.5: Register allocation via Graph Coloring: (a) Live-range intervals across the five instructions of the optimized loop. Because val dies at instruction 2 and scaled is born at instruction 2, their live ranges are strictly disjoint. (b) The Register Interference Graph. Because val and scaled do not interfere, they share physical register R_3. The graph is validly 4-colored using registers R_0, R_1, R_2, R_3 with zero memory spills.

38.6.3 The Emitted Machine Code

Mapping registers $R_0 \to$ `%rax`, $R_1 \to$ `%rsi`, $R_2 \to$ `%rdx`, and $R_3 \to$ `%rcx`, the compiler emits x86-64 machine assembly:

```
.Lloop:
    movq    (%rsi), %rcx      # rcx (val) = load 8 bytes from ptr (rsi)
    shlq    $1, %rcx          # rcx (scaled) = rcx << 1
    addq    %rcx, %rax        # rax (sum) += scaled
    addq    $8, %rsi          # rsi (ptr) += 8
    cmpq    %rdx, %rsi        # compare ptr against limit (rdx)
    jb      .Lloop            # if ptr < limit, branch to .Lloop
```

A naive compiler emits over 18 stack memory accesses per iteration, stalling the execution pipeline. The optimizing compiler delivers a six-instruction inner loop (five fused micro-ops) with zero stack spills and pure register execution.

38.7 Summary: The Pipeline of Transformations

Table 38.2 maps each compilation stage to its data structures, algorithms, and textbook chapter foundations.

Stage	Input	Output	Algorithmic Engine	Book Chapter
1. Lexing	ASCII bytes	Token stream	Deterministic Finite Automata	Ch. 31
2. Parsing	Token stream	AST	Recursive Descent / Pratt Parsing	Ch. 10, 12
3. Semantics	AST	Typed AST	Scoped Symbol Tables / Type Lowering	Ch. 11
4. IR / SSA	Typed AST	SSA CFG	Def-Use DAGs / Value Numbering	Ch. 11, 22
5. Backend	SSA CFG	Assembly	Liveness Analysis / Graph Coloring	Ch. 15, 33

Table 38.2: The five stages of a modern optimizing compiler.

Across parallel algorithms, machine learning, systems architecture, and compiler design, every algorithm we studied shared a classical premise: bits are strictly 0 or 1, gates compute Boolean functions, and execution moves through discrete physical states. In Chapter 39, we turn to a different physical model: quantum computing, where qubits exist in superposition and quantum interference enables algorithms like Grover's search and Shor's factoring.

Exercises

Exercise 38.1 (Tracing an AST with Operator Precedence)**.** Consider the arithmetic expression:

$$x = a + b * c - d/e.$$

Assuming standard operator precedence $(*, / > +, -)$ and left-associativity for operators of equal precedence, draw the unambiguous Abstract Syntax Tree produced by a Pratt parser. Which operation forms the root of the expression tree?

Exercise 38.2 (Constructing SSA Form for Branching Control Flow)**.** Consider the following program with an `if-else` branch:

```
x = 10;
if (condition) {
    x = x + 5;
    y = x * 2;
} else {
    x = x - 3;
    y = 0;
}
z = x + y;
```

1. Draw the Control-Flow Graph showing the four basic blocks: `Entry`, `Then`, `Else`, and `Join`.
2. Convert the program into Static Single Assignment (SSA) form by introducing subscripted variable versions $(x_0, x_1, \dots)$.
3. Write down the exact ϕ-functions required in the `Join` block for variables x and y.

Exercise 38.3 (Register Allocation via Graph 3-Coloring)**.** A compiler optimizes a loop containing five temporary variables: u, v, w, x, y. Liveness analysis reveals the following symmetric interference pairs:

$$(u, v), \quad (u, w), \quad (v, w), \quad (v, x), \quad (w, x), \quad (w, y), \quad (x, y).$$

Suppose the CPU has $K = 3$ physical registers: $\{R_0, R_1, R_2\}$.

1. Compute the degree of each vertex in the interference graph.
2. Apply the Chaitin–Briggs `Simplify` heuristic step by step. Which vertex can be peeled first? Show the order in which vertices are pushed onto the stack.
3. Pop the vertices in the `Select` phase and assign registers from $\{R_0, R_1, R_2\}$ without conflict. Can the graph be colored with zero spills?

Exercise 38.4 (Induction Variable Strength Reduction on 2D Arrays)**.** In row-major order, an array element $M[i][j]$ in an $N \times N$ matrix of 64-bit integers sits at byte address:

$$\text{address}(M[i][j]) = \text{base}_M + (i \times N + j) \times 8.$$

Consider an inner loop where i is fixed and j runs from 0 to $N - 1$:

```
j = 0;
while (j < N) {
    sum = sum + M[i][j];
    j = j + 1;
}
```

Explain how an optimizing compiler applies loop-invariant code motion and induction variable strength reduction to replace the two multiplications ($i \times N$ and $(i \times N + j) \times 8$) with a single pointer increment inside the loop. Write the optimized SSA pseudocode for the loop body.

Exercise 38.5 (The Cost of Register Spilling)**.** Suppose a microprocessor takes 0 extra cycles to read a physical register, 4 cycles to read from L1 data cache, and 200 cycles on an L1 cache miss that reads DRAM memory.

1. If a loop body executes 10^7 times and spills two variables to the stack, compute the minimum latency penalty (in clock cycles) incurred by the spill assuming all stack accesses hit the L1 cache.
2. Why do compiler register allocators use loop nest depth as a primary weighting factor when selecting which variable to spill?

Chapter 39

Quantum Computing: Amplitudes, Interference, and What Qubits Actually Do

In Chapter 38, we traced how optimizing compilers translate high-level algorithms into machine instructions, managing pipelines, registers, and silicon gates where bits are strictly 0 or 1. A popular claim in science journalism goes: "A classical bit is either 0 or 1, whereas a quantum bit can be 0 and 1 at the same time, so an n-qubit computer tests all 2^n possible solutions in parallel simultaneously!"

This slogan is misleading because it leaves out the most important rule of quantum mechanics: **measurement**. Suppose you prepare an n-qubit quantum register in an equal blend of all 2^n possible n-bit strings, and you measure it. You do not see all 2^n answers. You obtain **exactly one bitstring**, chosen completely at random with probability $1/2^n$.

If all a quantum computer could do was prepare a massive superposition and pick a random item upon measurement, it would act only as an expensive random-number generator. You could simulate that on your laptop in two lines of code using a pseudo-random number generator, with zero quantum hardware.

What a quantum computer actually exploits is a mathematical difference between classical probabilities and quantum amplitudes:

- **Classical probabilities are non-negative numbers ($p \geq 0$).** If an algorithm has two distinct computational pathways that lead to the same incorrect output, the total probability of failing is $p_1 + p_2$. Because probabilities can never be negative, adding an alternative path to an error always increases (or at best maintains) your chance of failure.
- **Quantum states are described by probability amplitudes.** Amplitudes can be positive, negative, or complex numbers ($\alpha \in \mathbb{C}$). If a quantum algorithm provides two computational pathways that lead to the *same incorrect output*, but one path arrives with amplitude $+a$ and the other path arrives with amplitude $-a$, the two amplitudes add linearly:

  $$+a + (-a) = 0.$$

  The two paths **destructively interfere and cancel each other out to exact zero**.

Designing a quantum algorithm means choosing unitary transformations so that computational paths

leading to wrong answers interfere destructively and cancel, while paths leading to the right answer interfere constructively and reinforce. When you measure the machine at the end, the surviving amplitude is concentrated almost entirely on the solution.

History and connection

The idea that quantum mechanics might offer more computational power than classical machines came from a problem in physics simulation. In 1981, at a joint MIT–IBM conference on the physics of computation, Richard Feynman pointed out that simulating a system of n interacting quantum particles on a classical digital computer appears to require exponential resources. Because each quantum particle adds a degree of freedom, describing the state of n entangled particles requires tracking 2^n continuous complex numbers. Simulating just a few hundred electrons demands more memory than there are atoms in the observable universe. Feynman proposed the reverse: instead of simulating quantum physics on classical logic gates, build a computing device whose physical elements obey quantum laws, allowing nature to carry out the exponential linear algebra. Concurrently, Paul Benioff proved that a quantum mechanical Hamiltonian could simulate a classical reversible Turing machine.

In 1985, David Deutsch at Oxford laid the mathematical foundations of the field by defining the Universal Quantum Computer, proving that a quantum machine could evaluate functions in superposition. In 1992, Deutsch and Richard Jozsa formulated the first quantum algorithm demonstrating a provable, deterministic separation between quantum and classical query complexity. A year later, Ethan Bernstein and Umesh Vazirani formalized the complexity class BQP (Bounded-Error Quantum Polynomial Time) and presented an oracle problem establishing an exponential quantum speedup under randomized query models.

For a decade, quantum computing remained largely a theoretical curiosity among physicists and complexity theorists. That changed abruptly in 1994. Peter Shor, working at AT&T Bell Laboratories in New Jersey, announced polynomial-time quantum algorithms for two famous mathematical problems: prime factorization and discrete logarithms. Because the world's deployed public-key cryptographic infrastructure (RSA, Diffie-Hellman, and elliptic curves) depended on the presumed intractability of these two problems, Shor's algorithm turned quantum computing from an academic topic into a field of major practical and commercial interest. In 1996, Lov Grover, also at Bell Labs, discovered an algorithm for searching an unsorted database of N items in $\Theta(\sqrt{N})$ operations, establishing a quadratic speedup for unstructured search that was soon proven by Bennett, Bernstein, Brassard, and Vazirani to be optimal for generic black-box search.

Early critics, including Rolf Landauer and William Unruh, raised severe physical skepticism: quantum amplitudes are continuous numbers, so analog noise, stray thermal photons, and environmental decoherence would inevitably scramble the delicate phases before any non-trivial algorithm could complete. Unlike digital bits, which snap back to clean voltage rails, analog states seemed doomed to continuous drift. This skepticism was overcome when Peter Shor (1995) and Alexei Kitaev (1997) invented quantum error correction and topological surface codes. They proved the Quantum Threshold Theorem: if physical error rates per gate can be driven below a constant threshold (roughly 1%), quantum errors can be actively detected and corrected through discrete syndrome measurements without ever measuring or collapsing the underlying quantum superposition. Today, laboratories worldwide compete to build fault-tolerant hardware using superconducting transmon circuits, trapped atomic ions, neutral atoms trapped in optical tweezers, and integrated photonic waveguides.

39.1 A Single Qubit as a 2D Unit Vector

To understand quantum computing, you do not need quantum field theory, wave-particle duality, or Hamiltonian operators. All you need is the linear algebra of unit vectors in a two-dimensional vector space.

39.1.1 Unwrapping Dirac's notation

In classical computing, a bit stores a scalar value from the discrete set $\{0, 1\}$. In quantum computing, the fundamental unit of information is a **qubit** (quantum bit). A qubit lives in a two-dimensional vector space over the field of complex numbers $\mathbb{C}^2$.

Physicists use Paul Dirac's notation, writing a vertical ket $|\cdot\rangle$ to denote a column vector. The two standard basis vectors of $\mathbb{R}^2$ (or $\mathbb{C}^2$) are given special labels:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Here $|0\rangle$ is not the number zero but the unit vector pointing along the horizontal x-axis. Likewise, $|1\rangle$ is the unit vector pointing along the vertical y-axis. Any state of a single qubit is simply a linear combination of these two orthogonal basis vectors:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}.$$

The coefficients α and β are called **probability amplitudes**.

39.1.2 Real amplitudes on the 2D unit circle

Before we introduce complex numbers, let us build our intuition in the familiar Cartesian plane $\mathbb{R}^2$, where α and β are ordinary real numbers.

In $\mathbb{R}^2$, a qubit state $|\psi\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ is an arrow originating at the origin. There is one constraint: the state must be a **unit vector**. By the Pythagorean theorem, its Euclidean length squared must equal 1:

$$\||\psi\rangle\|^2 = \alpha^2 + \beta^2 = 1.$$

A single qubit with real amplitudes is simply a point on the circumference of the unit circle in the plane! For example, the state with equal positive amplitudes is located at an angle of 45°:

$$|+\rangle = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}.$$

The state with opposite signs is located at an angle of $-45°$ (315°):

$$|-\rangle = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}.$$

Both vectors have length $\sqrt{(1/\sqrt{2})^2 + (\pm 1/\sqrt{2})^2} = \sqrt{1/2 + 1/2} = 1$.

39.1.3 Measurement and the Born Rule

What happens when we observe a qubit? In classical mechanics, looking at an object reveals its state without altering it. In quantum mechanics, observing a system forces it to make a discrete, non-deterministic choice. This interaction is called **measurement**.

Definition 39.1 (The Born Rule in $\mathbb{R}^2$)**.** Let a qubit be in state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ with $\alpha^2 + \beta^2 = 1$. When measured in the standard basis $\{|0\rangle, |1\rangle\}$:

1. The outcome is 0 with probability $\Pr(0) = \alpha^2$. If observed, the state collapses instantaneously onto $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$.
2. The outcome is 1 with probability $\Pr(1) = \beta^2$. If observed, the state collapses instantaneously onto $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

Geometrically, α is the horizontal projection (shadow) of the unit arrow onto the $|0\rangle$-axis, and β is the vertical projection (shadow) onto the $|1\rangle$-axis. The probability of collapsing onto an axis is the **squared length of that shadow**!

Because the state arrow has unit length, the Pythagorean theorem guarantees that the probabilities sum to 1:

$$\Pr(0) + \Pr(1) = \alpha^2 + \beta^2 = 1.$$

If amplitudes are generalized to complex numbers $\alpha, \beta \in \mathbb{C}$, the exact same geometry holds in $\mathbb{C}^2$: the probability of observing outcome 0 is $|\alpha|^2$, the probability of observing outcome 1 is $|\beta|^2$, and the normalization condition $|\alpha|^2 + |\beta|^2 = 1$ ensures that total probability is conserved.

Common trap

A common trap is confusing the probability amplitude α with the probability $|\alpha|^2$. If $\alpha = -1/\sqrt{2}$, the probability of measuring 0 is $(-1/\sqrt{2})^2 = +1/2$. The minus sign has zero effect on the immediate measurement statistics of that single isolated qubit. But as the next section shows, the minus sign is still stored in the state, and it changes the outcome once further gates are applied.

39.2 Quantum Gates as Rotations and Reflections

How does a quantum computer perform a computation? In classical computing, logic gates like `AND`, `OR`, and `NOT` transform input bits into output bits. In quantum computing, a computation is performed by applying a linear operator to the state vector.

39.2.1 Conservation of probability and unitary matrices

Every valid quantum gate must transform unit vectors into unit vectors. If a state $|\psi\rangle$ satisfies $\||\psi\rangle\| = 1$, and we apply a matrix U, the resulting vector $U|\psi\rangle$ must also satisfy $\|U|\psi\rangle\| = 1$. Total probability must remain 100% under all circumstances.

In real vector spaces $\mathbb{R}^n$, matrices that preserve vector lengths are the **orthogonal matrices** ($U^\mathsf{T}U = I$). In complex vector spaces $\mathbb{C}^n$, matrices that preserve vector lengths are the **unitary matrices**:

$$U^\dagger U = I,$$

where $U^\dagger = (U^\mathsf{T})^*$ denotes the conjugate transpose (the adjoint).

This single geometric requirement has an immediate consequence: **every isolated quantum computation is reversible**. Because $U^\dagger U = I$, the matrix U is always invertible, with its inverse given by its conjugate transpose: $U^{-1} = U^\dagger$. A quantum gate never erases information.

39.2.2 Elementary quantum gates

Let us inspect the three foundational single-qubit gates:

1. **The Pauli-X Gate (Quantum NOT):**

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

   Applying X to the basis states swaps them:

$$X|0\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix} = |1\rangle, \qquad X|1\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle.$$

   Geometrically, X is a reflection across the diagonal line $y = x$.

2. **The Pauli-Z Gate (Phase Flip):**

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

   Applying Z leaves $|0\rangle$ unchanged and flips the sign of $|1\rangle$:

$$Z|0\rangle = |0\rangle, \qquad Z|1\rangle = -|1\rangle.$$

   Geometrically, Z is a reflection across the horizontal $|0\rangle$-axis.

3. **The Hadamard Gate (H):**

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

   Notice that H is symmetric ($H^\mathsf{T} = H$) and orthogonal ($H^\mathsf{T}H = I$), which means $H^2 = I$: the Hadamard gate is its own inverse. Geometrically, H is a reflection across the line at angle 22.5° ($\pi/8$), swapping $|0\rangle$ with $|+\rangle$ and $|1\rangle$ with $|-\rangle$.

39.2.3 Tracing the Hadamard gate by hand: The Two-Coin-Flip experiment

The Hadamard gate is the basic tool for creating superposition and interference. To see why, let us walk through a thought experiment comparing a classical random process with a quantum circuit.

Imagine a classical machine that flips a fair coin. We initialize the coin to Heads (0). The flipper scrambles it so that the state becomes Heads (0) with probability 1/2 and Tails (1) with probability

1/2. Now we run the flipper a second time. What is the outcome? A random coin flipped twice is still random: $\Pr(0) = \frac{1}{2} \times \frac{1}{2} + \frac{1}{2} \times \frac{1}{2} = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$.

Now let us execute the exact same sequence on a quantum computer using the Hadamard gate.

Step 1: The first Hadamard gate. We initialize our qubit to $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$. We apply H:

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle = |+\rangle.$$

If we choose to measure the qubit right now, what do we observe?

- Outcome 0 occurs with probability $(1/\sqrt{2})^2 = 1/2$.
- Outcome 1 occurs with probability $(1/\sqrt{2})^2 = 1/2$.

The measurement statistics are identical to a fair coin flip!

What if we had initialized the qubit to $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$? Applying H yields:

$$H|1\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix} = \frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle = |-\rangle.$$

If measured right now, the outcome is 0 with probability $(1/\sqrt{2})^2 = 1/2$, and 1 with probability $(-1/\sqrt{2})^2 = 1/2$. To a direct measurement, $|+\rangle$ and $|-\rangle$ look completely indistinguishable: both produce 50%/50% coin flips.

Step 2: The second Hadamard gate without measurement. Now comes the demonstration of quantum interference. Suppose we start with $|0\rangle$, apply H, do **not** measure, and immediately apply H a second time.

Let us trace every individual computational path by distributing the linear operator:

$$\begin{aligned} H(H|0\rangle) &= H\left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) \\ &= \frac{1}{\sqrt{2}}H|0\rangle + \frac{1}{\sqrt{2}}H|1\rangle \\ &= \frac{1}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) + \frac{1}{\sqrt{2}}\left(\frac{1}{\sqrt{2}}|0\rangle - \frac{1}{\sqrt{2}}|1\rangle\right) \\ &= \left(\frac{1}{2}|0\rangle + \frac{1}{2}|1\rangle\right) + \left(\frac{1}{2}|0\rangle - \frac{1}{2}|1\rangle\right) \\ &= \left(\frac{1}{2} + \frac{1}{2}\right)|0\rangle + \left(\frac{1}{2} - \frac{1}{2}\right)|1\rangle \\ &= 1|0\rangle + 0|1\rangle = |0\rangle. \end{aligned}$$

The amplitude of outcome $|1\rangle$ is $\frac{1}{2} - \frac{1}{2} = 0$. The path through the intermediate state $|0\rangle$ contributed $+1/2$ to the final state $|1\rangle$, while the path through the intermediate state $|1\rangle$ contributed $-1/2$ to the final state $|1\rangle$. The two pathways cancel exactly.

Meanwhile, both intermediate pathways contributed $+1/2$ to the final state $|0\rangle$, reinforcing each other to yield a total amplitude of 1, giving a measurement probability of $1^2 = 100\%$.

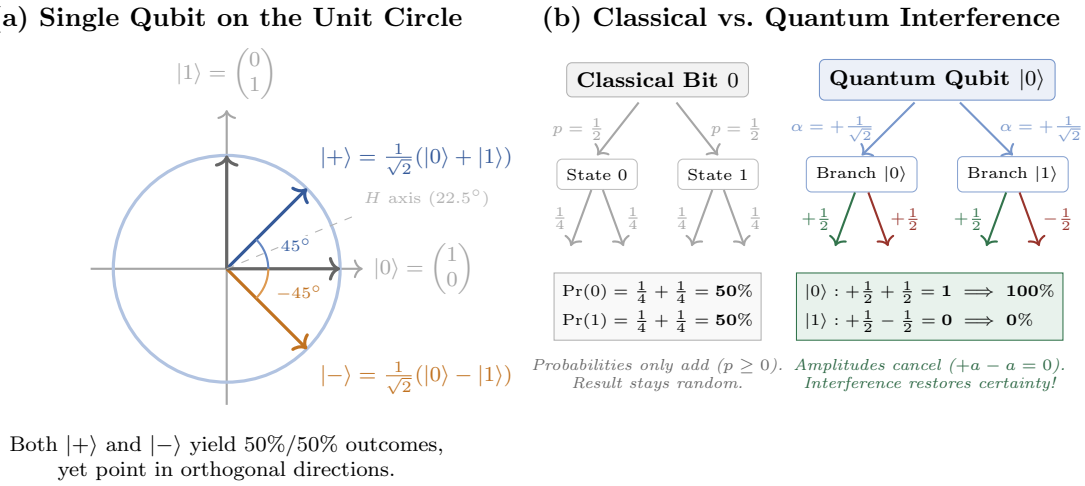


Figure 39.1: The foundational distinction of quantum computation. Panel (a) shows the single-qubit unit circle where $|0\rangle, |1\rangle$ are standard basis axes and $|+\rangle, |-\rangle$ are diagonal superpositions. Panel (b) compares two successive operations: two classical coin flips sum probabilities to maintain 50%/50% uncertainty, whereas two Hadamard gates cause destructive interference on $|1\rangle$ (canceling to 0) and constructive interference on $|0\rangle$ (reinforcing to 100%).

The second Hadamard gate acts as an **interferometer**. It takes a state that looked completely random under measurement and uses the hidden phase to funnel all amplitude back into the original definite state.

39.3 Multiple Qubits, Tensor Products, and Entanglement

A single qubit has two basis states, $|0\rangle$ and $|1\rangle$. What happens when we assemble a register of multiple qubits?

39.3.1 Two qubits: A 4D vector space

Two classical bits can be in four possible configurations: 00, 01, 10, or 11. A two-qubit quantum register has four corresponding standard basis vectors:

$$|00\rangle = \begin{pmatrix}1\\0\\0\\0\end{pmatrix}, \quad |01\rangle = \begin{pmatrix}0\\1\\0\\0\end{pmatrix}, \quad |10\rangle = \begin{pmatrix}0\\0\\1\\0\end{pmatrix}, \quad |11\rangle = \begin{pmatrix}0\\0\\0\\1\end{pmatrix}.$$

An arbitrary two-qubit state is a unit vector in $\mathbb{C}^4$:

$$|\psi\rangle = \alpha_{00}|00\rangle + \alpha_{01}|01\rangle + \alpha_{10}|10\rangle + \alpha_{11}|11\rangle,$$

with normalization condition:

$$|\alpha_{00}|^2 + |\alpha_{01}|^2 + |\alpha_{10}|^2 + |\alpha_{11}|^2 = 1.$$

When two independent qubits $|\psi_1\rangle = a|0\rangle + b|1\rangle$ and $|\psi_2\rangle = c|0\rangle + d|1\rangle$ are placed next to each other, their joint state is their **tensor product** (or Kronecker product):

$$|\psi_1\rangle \otimes |\psi_2\rangle = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle) = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle.$$

For n qubits, the state space is $\mathbb{C}^{2^n}$. A 50-qubit processor requires $2^{50} \approx 10^{15}$ complex amplitudes to specify its complete state; a 300-qubit processor requires 2^{300} amplitudes, exceeding the number of particles in the universe.

39.3.2 The CNOT Gate: Conditional interaction

To perform logic between qubits, we need multi-qubit gates. The most important two-qubit gate is the **Controlled-NOT (CNOT)** gate.

The CNOT gate acts on two qubits: a **control qubit** (the first qubit) and a **target qubit** (the second qubit). If the control qubit is 0, the target qubit is left alone. If the control qubit is 1, the target qubit is inverted (X is applied):

$$|00\rangle \mapsto |00\rangle, \quad |01\rangle \mapsto |01\rangle, \quad |10\rangle \mapsto |11\rangle, \quad |11\rangle \mapsto |10\rangle.$$

In matrix form acting on $\mathbb{C}^4$, CNOT is represented by:

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Notice that CNOT is a permutation matrix: its rows and columns are orthonormal, so it is unitary and reversible ($\text{CNOT}^\dagger = \text{CNOT}$ and $\text{CNOT}^2 = I$).

39.3.3 Creating the Bell State: An entangled pair

Now let us combine the Hadamard gate and the CNOT gate to produce the most famous state in quantum physics: the **Bell State** (or EPR pair).

We start with two qubits initialized to $|00\rangle$.

1. Apply H to the first qubit:

$$(H|0\rangle) \otimes |0\rangle = \left(\frac{1}{\sqrt{2}}|0\rangle + \frac{1}{\sqrt{2}}|1\rangle\right) \otimes |0\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|10\rangle.$$

2. Apply CNOT with qubit 1 as control and qubit 2 as target:
   - The branch $|00\rangle$ has control 0, so target stays 0: $|00\rangle \mapsto |00\rangle$.
   - The branch $|10\rangle$ has control 1, so target flips to 1: $|10\rangle \mapsto |11\rangle$.

The resulting two-qubit state is:

$$|\Phi^+\rangle = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle.$$

Figure 39.2 tracks this two-step entangling circuit and the resulting redistribution of amplitude across the two-qubit computational basis.

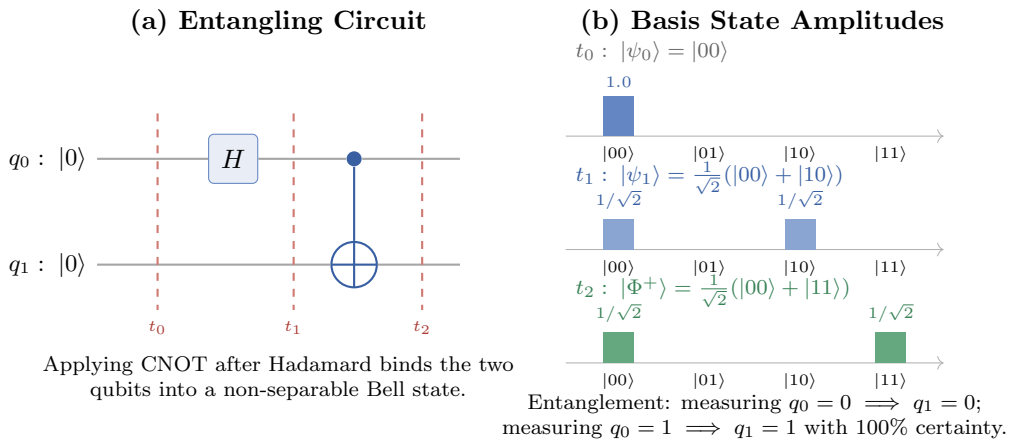


Figure 39.2: Creation of the maximally entangled Bell state $|\Phi^+\rangle$. In panel (a), qubit q_0 passes through a Hadamard gate before acting as the control for a CNOT targeting q_1. In panel (b), amplitude evolves across basis states: H creates an equal superposition of $|00\rangle$ and $|10\rangle$ (a separable product state); CNOT then flips the target of the second branch ($|10\rangle \mapsto |11\rangle$), producing an entangled pair whose measurement outcomes are 100% correlated.

This state is **entangled**. What does that word mean mathematically?

Definition 39.2 (Entanglement)**.** A multi-qubit state $|\psi\rangle \in \mathbb{C}^{2^n}$ is **separable** (or a product state) if it can be factored into individual single-qubit states: $|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle$. A state is **entangled** if no such factorization exists.

Let us prove in one algebraic check that $|\Phi^+\rangle$ is entangled. Suppose for contradiction that $|\Phi^+\rangle$ could be factored as:

$$|\Phi^+\rangle = (a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle) = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle.$$

Matching the coefficients of $|\Phi^+\rangle = \frac{1}{\sqrt{2}}|00\rangle + 0|01\rangle + 0|10\rangle + \frac{1}{\sqrt{2}}|11\rangle$ requires:

$$ac = \frac{1}{\sqrt{2}}, \qquad ad = 0, \qquad bc = 0, \qquad bd = \frac{1}{\sqrt{2}}.$$

Look at $ad = 0$. If $a = 0$, then $ac = 0$, which contradicts $ac = 1/\sqrt{2}$. Therefore $a \neq 0$, which forces $d = 0$. But if $d = 0$, then $bd = 0$, which contradicts $bd = 1/\sqrt{2}$.

The contradiction is immediate. The state $|\Phi^+\rangle$ **cannot be factored**. Neither individual qubit possesses a state of its own! The two-qubit system has a definite, pure state, but the individual components do not.

39.3.4 What entanglement does, and what it cannot do

If Alice takes qubit 1 to Alpha Centauri and Bob keeps qubit 2 on Earth, what happens when they measure?

- If Alice measures qubit 1 in the standard basis, she sees 0 with probability $(1/\sqrt{2})^2 = 50\%$, and 1 with probability $(1/\sqrt{2})^2 = 50\%$.
- If Alice saw 0, the state instantly collapses to $|00\rangle$. When Bob measures qubit 2, he is guaranteed to see 0 with 100% certainty!
- If Alice saw 1, the state collapses to $|11\rangle$, and Bob is guaranteed to see 1.

Their outcomes are 100% correlated, no matter how far apart they are. As John Bell proved in 1964, these correlations violate Bell's inequality, demonstrating that the universe cannot be explained by local classical hidden variables.

However, **entanglement cannot transmit information faster than light**. Notice that from Bob's local perspective, his qubit has probability 1/2 of being 0 and probability 1/2 of being 1. Alice cannot choose whether she gets 0 or 1; her outcome is fundamentally random. Bob cannot tell whether Alice has measured her qubit or not until Alice sends him a classical message (traveling at or below the speed of light) telling him what she observed. Entanglement provides correlated shared randomness, but it never violates special relativity.

39.4 Deutsch's Algorithm and the Phase-Kickback Trick

Now that we understand single-qubit interference and two-qubit gates, we can trace our first complete quantum algorithm: David Deutsch's 1985 algorithm.

39.4.1 The problem: Constant vs. balanced

Suppose you are given a black-box function $f : \{0,1\} \to \{0,1\}$ that takes a single bit as input and outputs a single bit. There are four such functions:

1. $f(0) = 0$ and $f(1) = 0$ (Constant)
2. $f(0) = 1$ and $f(1) = 1$ (Constant)
3. $f(0) = 0$ and $f(1) = 1$ (Balanced)
4. $f(0) = 1$ and $f(1) = 0$ (Balanced)

A function is **constant** if $f(0) = f(1)$, and **balanced** if $f(0) \neq f(1)$.

The classical query complexity: How many times must a classical computer evaluate f to determine with certainty whether f is constant or balanced? The answer is strictly **two**. If you evaluate $f(0)$ and obtain 0, you know nothing about whether f is constant or balanced; $f(1)$ could be 0 (constant) or 1 (balanced). You must evaluate both $f(0)$ and $f(1)$.

Can a quantum computer solve this problem in **one single query**?

39.4.2 The quantum oracle and Phase Kickback

In quantum mechanics, all gate operations must be reversible. If f is a constant function, the mapping $|x\rangle \mapsto |f(x)\rangle$ erases the input x and is not unitary. To make evaluation reversible, we

encode f into a two-qubit unitary oracle U_f:

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle,$$

where $\oplus$ denotes addition modulo 2 (bitwise XOR). The first qubit $|x\rangle$ is the input, and the second qubit $|y\rangle$ is the target scratchpad.

This setup gives a basic building block of quantum algorithms: **Phase Kickback**.

What happens if we prepare the target qubit in the state $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$? Let us calculate $U_f|x\rangle|-\rangle$ explicitly:

$$\begin{aligned} U_f|x\rangle|-\rangle &= U_f|x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \\ &= \frac{1}{\sqrt{2}}\Big(|x\rangle|0 \oplus f(x)\rangle - |x\rangle|1 \oplus f(x)\rangle\Big). \end{aligned}$$

Let us check the two possible values of $f(x)$:

- If $f(x) = 0$:
$$\frac{1}{\sqrt{2}}\Big(|x\rangle|0\rangle - |x\rangle|1\rangle\Big) = |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = +|x\rangle|-\rangle.$$
- If $f(x) = 1$:
$$\frac{1}{\sqrt{2}}\Big(|x\rangle|1\rangle - |x\rangle|0\rangle\Big) = -|x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) = -|x\rangle|-\rangle.$$

In both cases, we can write the output compactly as:

$$\boxed{U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle.}$$

The target qubit $|-\rangle$ emerged completely unchanged! But its presence caused the value $f(x)$ to be **kicked back** into the phase of the input qubit as a factor of $(-1)^{f(x)}$.

39.4.3 Tracing Deutsch's algorithm step-by-step

Here is the complete quantum circuit:

1. Initialize the register to $|\psi_0\rangle = |0\rangle|1\rangle$.
2. Apply Hadamard to both qubits:
$$|\psi_1\rangle = (H|0\rangle) \otimes (H|1\rangle) = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes |-\rangle.$$
3. Query the oracle U_f once:
$$\begin{aligned} |\psi_2\rangle &= U_f \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |-\rangle\right) \\ &= \frac{1}{\sqrt{2}}U_f|0\rangle|-\rangle + \frac{1}{\sqrt{2}}U_f|1\rangle|-\rangle \\ &= \frac{1}{\sqrt{2}}\Big((-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle\Big) \otimes |-\rangle. \end{aligned}$$

4. Factor out $(-1)^{f(0)}$:

$$|\psi_2\rangle = (-1)^{f(0)} \left(\frac{|0\rangle + (-1)^{f(0)\oplus f(1)}|1\rangle}{\sqrt{2}} \right) \otimes |-\rangle.$$

   Now look at the term $(-1)^{f(0)\oplus f(1)}$:

   - **If f is constant:** $f(0) = f(1)$, so $f(0) \oplus f(1) = 0$. The first qubit is:

$$\pm \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right) = \pm|+\rangle.$$

   - **If f is balanced:** $f(0) \neq f(1)$, so $f(0) \oplus f(1) = 1$. The first qubit is:

$$\pm \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) = \pm|-\rangle.$$

5. Apply Hadamard to the first qubit: Because $H|+\rangle = |0\rangle$ and $H|-\rangle = |1\rangle$, the first qubit becomes:

$$|\psi_3\rangle = \begin{cases} \pm|0\rangle & \text{if } f \text{ is constant,} \\ \pm|1\rangle & \text{if } f \text{ is balanced.} \end{cases}$$

6. Measure the first qubit: If the outcome is 0, f is constant. If the outcome is 1, f is balanced.

The algorithm determines whether f is constant or balanced with 100% certainty in **a single query**. Figure 39.3 illustrates the complete quantum circuit and the geometric interference mechanism that separates the two cases.

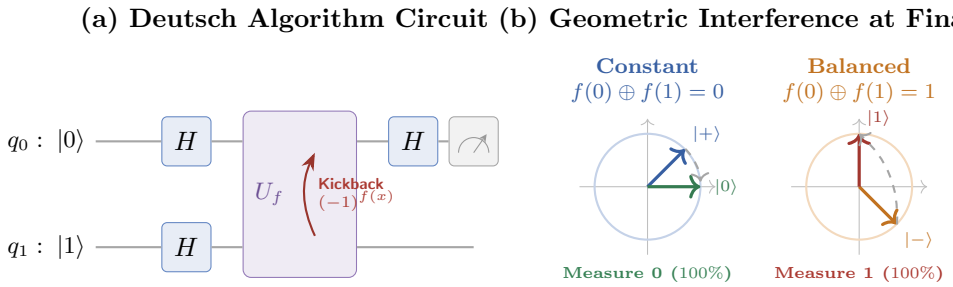


Figure 39.3: Deutsch's algorithm and the phase-kickback mechanism. In panel (a), the target qubit prepared in $|-\rangle$ remains unaltered while kicking back the relative phase factor $(-1)^{f(x)}$ into the query qubit. In panel (b), the final Hadamard gate transforms relative phase into constructive interference: $|+\rangle$ rotates to $|0\rangle$ for constant functions, while $|-\rangle$ rotates to $|1\rangle$ for balanced functions, answering the decision problem with certainty in one query.

Deutsch's algorithm never learns $f(0)$ or $f(1)$ individually. Instead, it uses quantum interference to evaluate the **global relational property** $f(0) \oplus f(1)$ directly, canceling the individual values while preserving their relative phase.

39.5 Grover's Search as 2D Plane Geometry

Deutsch's algorithm demonstrated a modest advantage (1 query vs. 2). In 1996, Lov Grover discovered that quantum interference could accelerate unstructured database search across $N = 2^n$ items, finding a unique marked item in $\Theta(\sqrt{N})$ steps where classical search requires $\Theta(N)$ steps.

Rather than expanding pages of algebraic sums, we can see Grover's search directly through plane geometry: each iteration is a **rotation in a two-dimensional plane**.

39.5.1 The 2D Grover plane

Suppose we have an unsorted database of $N = 2^n$ elements, represented by n-bit strings $x \in \{0,1\}^n$. Exactly one marked item $x^\star$ satisfies $f(x^\star) = 1$, while $f(x) = 0$ for all $x \neq x^\star$.

Instead of attempting to visualize a 2^n-dimensional space, notice that the entire algorithm takes place in a flat two-dimensional subspace spanned by just two vectors:

1. The target state: $|x^\star\rangle$.
2. The uniform superposition of all $N-1$ non-target states:

$$|w\rangle = \frac{1}{\sqrt{N-1}} \sum_{x \neq x^\star} |x\rangle.$$

Notice that $\langle w \mid x^\star \rangle = 0$: the two vectors are strictly orthogonal! We can treat $|w\rangle$ as our horizontal x-axis and $|x^\star\rangle$ as our vertical y-axis.

We initialize our algorithm in the uniform superposition over all N items:

$$|s\rangle = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle = \sqrt{\frac{N-1}{N}} |w\rangle + \frac{1}{\sqrt{N}} |x^\star\rangle.$$

Define the angle θ such that $\sin\theta = \frac{1}{\sqrt{N}}$ and $\cos\theta = \sqrt{\frac{N-1}{N}}$. Then:

$$|s\rangle = \cos\theta |w\rangle + \sin\theta |x^\star\rangle.$$

When N is large (say $N = 10^6$), $\sin\theta \approx 1/\sqrt{N} = 1/1000$, which means θ is a tiny angle. The initial vector $|s\rangle$ lies almost flat against the horizontal $|w\rangle$-axis. Our goal is to rotate this arrow until it points vertically along $|x^\star\rangle$.

39.5.2 Two reflections make a rotation

A famous theorem of high-school plane geometry states:

Theorem 39.1 (Composition of Reflections)**.** *Reflecting a vector across a line L_1, and then reflecting the result across a line L_2 that intersects L_1 at an angle θ, rotates the vector by an angle of 2θ in the direction from L_1 toward L_2.*

Grover's algorithm implements exactly this theorem! Each Grover iteration consists of two operations:

1. **Operation 1: The Oracle Reflection (U_f).** The oracle flips the sign of the target state while leaving all other states alone: $U_f = I - 2|x^\star\rangle\langle x^\star|$. In our 2D plane:

$$U_f|w\rangle = |w\rangle, \qquad U_f|x^\star\rangle = -|x^\star\rangle.$$

   Geometrically, this flips the vertical component of the vector while leaving the horizontal component unchanged. This is a **reflection across the horizontal axis** $|w\rangle$!

   When applied to our state $|s\rangle$ (at angle $+\theta$), the oracle reflects it to angle $-\theta$.

2. **Operation 2: The Diffusion Reflection (U_s).** The diffusion operator is defined by:

$$U_s = 2|s\rangle\langle s| - I.$$

   If you take a vector parallel to $|s\rangle$, $U_s|s\rangle = 2|s\rangle(1) - |s\rangle = +|s\rangle$. If you take a vector orthogonal to $|s\rangle$, $U_s|s_\perp\rangle = 2|s\rangle(0) - |s_\perp\rangle = -|s_\perp\rangle$. This is a **reflection across the line along** $|s\rangle$!

Now put the two reflections together:

- The state was at angle $+\theta$.
- Oracle reflection across line L_1 (horizontal axis at angle 0) sent it to angle $-\theta$.
- The angular distance from the new state ($-\theta$) to line L_2 (vector $|s\rangle$ at angle $+\theta$) is 2θ.
- Diffusion reflection across line L_2 reflects it across $|s\rangle$, placing it at angle:

$$\theta + 2\theta = 3\theta!$$

In one single Grover iteration $G = U_s U_f$, the state arrow has rotated by an angle of **exactly** 2θ toward the vertical target axis $|x^\star\rangle$!

39.5.3 How many steps to find the needle?

Every iteration advances the angle by 2θ. We started at angle θ. To point straight up at $|x^\star\rangle$, we need the angle to reach $\pi/2$ (90°). The remaining angular distance is:

$$\frac{\pi}{2} - \theta \approx \frac{\pi}{2}.$$

Because each iteration adds $2\theta \approx \frac{2}{\sqrt{N}}$ radians, the total number of iterations R required is:

$$R \approx \frac{\pi/2}{2\theta} \approx \frac{\pi/2}{2/\sqrt{N}} = \frac{\pi}{4}\sqrt{N}.$$

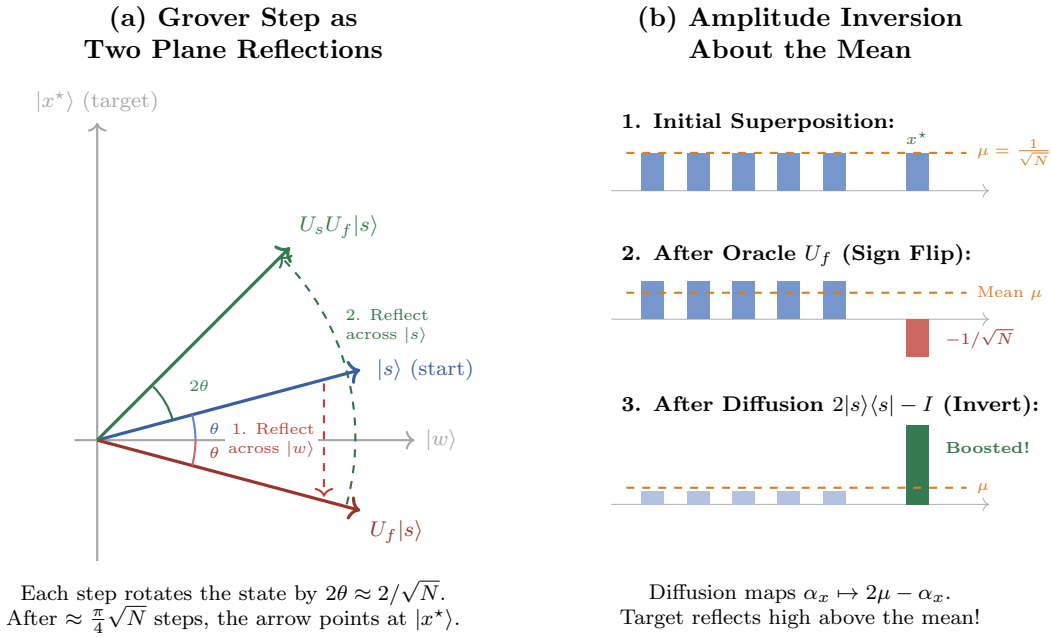


Figure 39.4: Grover's algorithm as geometric rotation and amplitude amplification. In panel (a), the oracle reflects the state across $|w\rangle$, and the diffusion operator reflects across $|s\rangle$, combining to produce an exact rotation of $2\theta \approx 2/\sqrt{N}$ per iteration. In panel (b), the algebraic diffusion operation $2|s\rangle\langle s| - I$ inverts every amplitude about the mean μ, amplifying the negative target amplitude into a tall positive peak.

Common trap

Unlike classical algorithms that improve monotonically the longer they run, Grover's algorithm is a **rotation**. If you continue applying iterations past $R \approx \frac{\pi}{4}\sqrt{N}$, the arrow rotates right past $|x^\star\rangle$ and points back toward the horizontal axis! The probability of success drops toward zero. You must stop the machine and measure at precisely the right number of steps.

39.6 Shor's Algorithm, BQP Complexity, and Fault Tolerance

Grover's search offers a quadratic speedup for unstructured problems. Can quantum computers achieve an **exponential speedup**?

39.6.1 Shor's algorithm and the Quantum Fourier Transform

In 1994, Peter Shor answered with integer factorization. Given a composite number $N = pq$, Shor's algorithm finds its prime factors in $O((\log N)^3)$ operations, whereas the best known classical method (the General Number Field Sieve) requires sub-exponential time $\exp\big(O((\log N)^{1/3}(\log\log N)^{2/3})\big)$.

Shor's breakthrough connects directly to the discrete Fourier transform and the FFT developed in Chapter 29. The algorithm proceeds in two stages:

1. **Classical Reduction to Period Finding:** Pick an integer a coprime to N $(1 < a < N)$.

The sequence of modular powers $f(x) = a^x \bmod N$ is periodic: there exists a smallest positive integer r (the order of a) such that $a^r \equiv 1 \pmod{N}$. If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then $(a^{r/2} - 1)(a^{r/2} + 1) = a^r - 1 \equiv 0 \pmod{N}$. By Euclid's algorithm, $\gcd(a^{r/2} - 1, N)$ reveals a non-trivial prime factor of N! Classically, finding r requires testing exponentially many values of x.

2. **Quantum Period Extraction via the QFT:** A quantum register is prepared in an equal superposition over inputs $x \in \{0, \ldots, Q-1\}$, and the modular exponentiation $a^x \bmod N$ is evaluated in a second register. Measuring the second register collapses the first register into a periodic spike train: superposition over all inputs $x_0, x_0 + r, x_0 + 2r, \ldots$ spaced by the unknown period r.

   Applying the **Quantum Fourier Transform (QFT)** turns this time-domain periodic train into frequency-domain interference peaks concentrated at integer multiples of Q/r. Measuring the output reveals a frequency peak, from which the period r is extracted in polynomial time using continued fractions.

Shor's algorithm achieves an exponential speedup because integer factorization has algebraic structure: the periodicity of modular exponentiation in an abelian group.

39.6.2 Does a quantum computer solve NP-complete problems?

A frequent question from computer scientists is:

> *"Can a quantum computer solve NP-complete problems (like 3-SAT, Traveling Salesperson, or Clique) in polynomial time?"*

The consensus among complexity theorists is: **almost certainly not**.

NP-complete problems do not possess the global periodic group structure that Shor's QFT exploits. An instance of 3-SAT with n variables has 2^n possible truth assignments, with no algebraic periodicity linking satisfying solutions. Against an unstructured combinatorial search space, the most effective generic quantum strategy known is Grover's search.

Grover's algorithm accelerates searching the 2^n assignments from $O(2^n)$ down to $O(2^{n/2})$. A square-root reduction is substantial, but $2^{n/2}$ is still an **exponential runtime**. For a 3-SAT instance with $n = 200$ boolean variables:

$$2^{n/2} = 2^{100} \approx 1.27 \times 10^{30} \text{ operations.}$$

A computer running a billion iterations every microsecond would still require tens of thousands of years to complete the search. Quantum computers do not eliminate exponential combinatorial growth.

39.6.3 The Complexity Class BQP: Where Quantum Sits

To formalize quantum computational power, Bernstein and Vazirani defined the complexity class BQP:

Definition 39.3 (BQP)**.** BQP (Bounded-Error Quantum Polynomial Time) is the class of decision problems solvable by a uniform family of polynomial-size quantum circuits with an error probability at most 1/3 on all inputs.

How does BQP compare to the classical complexity classes studied in Chapter 33?

- P $\subseteq$ BPP $\subseteq$ BQP $\subseteq$ PSPACE. Every problem efficiently solvable by classical computers (deterministic or randomized) is efficiently solvable by a quantum computer.
- **Factoring and Discrete Logarithms** are in BQP. Classically, they reside in NP $\cap$ co-NP, but are not believed to be in P.
- **Quantum Simulation** (simulating the time evolution of local quantum Hamiltonians) is in BQP, and is BQP-complete.
- BQP is **not** believed to contain NP-complete problems. Indeed, there are oracle worlds where BQP and NP are incomparable: BQP contains problems outside NP, and NP contains problems outside BQP.

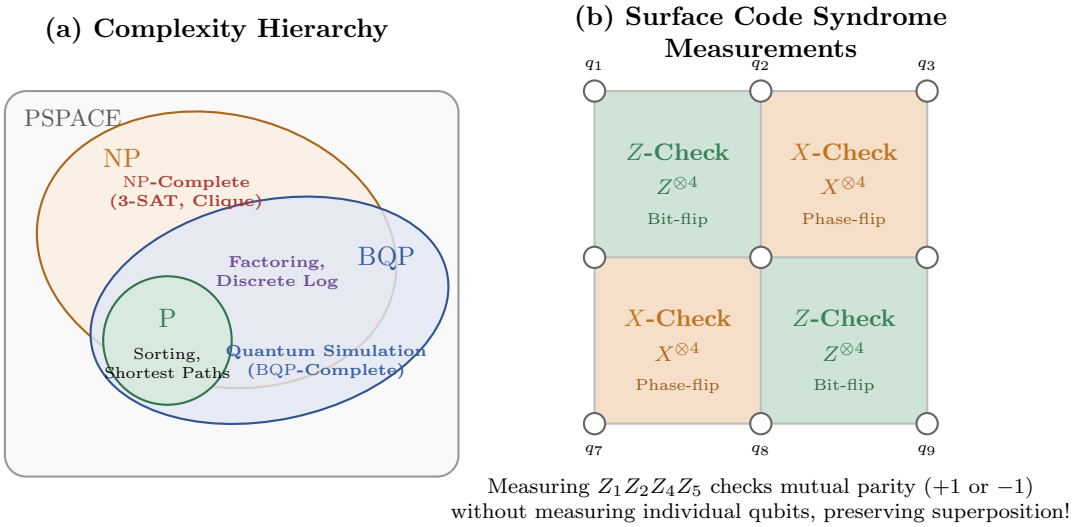


Figure 39.5: Quantum complexity classes and surface-code error correction. Panel (a) illustrates the complexity hierarchy: BQP encompasses P and contains problems like Factoring and Quantum Simulation, while NP-complete problems remain outside. Panel (b) depicts Kitaev's topological surface code, where physical data qubits reside on vertices and alternating plaquettes perform non-destructive parity checks (Z-syndromes for bit-flips, X-syndromes for phase-flips) to detect errors without collapsing the encoded state.

39.6.4 Quantum Error Correction and the No-Cloning Theorem

In classical computing, error correction is conceptually simple: if noise corrupts a bit, you can back up the data by making redundant copies ($0 \mapsto 000$). If one bit flips, majority voting restores the truth.

In quantum computing, this simple duplication is physically impossible:

Theorem 39.2 (The No-Cloning Theorem, Wootters & Zurek, 1982)**.** *There is no unitary operator U that can create an identical duplicate of an unknown arbitrary quantum state:*

$$U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle \quad \textit{for all } |\psi\rangle.$$

Proof. Suppose such a unitary operator U existed. Then U must clone the basis states:

$$U(|0\rangle|0\rangle) = |0\rangle|0\rangle, \qquad U(|1\rangle|0\rangle) = |1\rangle|1\rangle.$$

Now consider the superposition state $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$. Because U is a linear operator, applying U to $|+\rangle|0\rangle$ must act linearly:

$$U\left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \otimes |0\rangle\right) = \frac{1}{\sqrt{2}}U(|0\rangle|0\rangle) + \frac{1}{\sqrt{2}}U(|1\rangle|0\rangle) = \frac{1}{\sqrt{2}}|00\rangle + \frac{1}{\sqrt{2}}|11\rangle.$$

However, if U successfully cloned the state $|+\rangle$, the output would be:

$$|+\rangle|+\rangle = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) \otimes \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right) = \frac{1}{2}|00\rangle + \frac{1}{2}|01\rangle + \frac{1}{2}|10\rangle + \frac{1}{2}|11\rangle.$$

These two states are manifestly unequal. A linear operator cannot clone non-orthogonal states. □

If you cannot clone a qubit, and looking at a qubit collapses its superposition, how can quantum errors ever be corrected?

The insight of Peter Shor and Alexei Kitaev was to encode one **logical qubit** into an entangled state across multiple physical qubits. Consider the 3-qubit bit-flip code:

$$|0_L\rangle = |000\rangle, \qquad |1_L\rangle = |111\rangle.$$

An arbitrary logical state is $\alpha|000\rangle + \beta|111\rangle$.

Instead of measuring the individual qubits (which would reveal whether the state is 0 or 1 and destroy α and β), we measure **parity-check operators** using helper (ancilla) qubits:

- Measure the parity Z_1Z_2: returns $+1$ if qubits 1 and 2 agree, and -1 if they differ.
- Measure the parity Z_2Z_3: returns $+1$ if qubits 2 and 3 agree, and -1 if they differ.

If an environmental disturbance flips qubit 1 (X_1), the parities read ($Z_1Z_2 = -1, Z_2Z_3 = +1$). The syndrome identifies precisely which qubit flipped, allowing a corrective X_1 gate to restore the state, **without the measurement ever learning whether the data was $|000\rangle$ or $|111\rangle$!**

In modern architectures, this principle is scaled to two-dimensional grids of qubits called **surface codes** (Figure 39.5b). Data qubits sit on the vertices of a lattice, and interleaved measurement qubits constantly check local parities to detect bit-flips and phase-flips. By scaling the distance of the code, error rates can be made arbitrarily small, provided the physical error rate stays below the threshold.

39.7 Summary

Quantum computing replaces classical bits with state vectors in complex Hilbert spaces:

- **Amplitudes vs. Probabilities**: While classical probabilities are non-negative numbers that always accumulate, quantum amplitudes are complex numbers that can cancel destructively, eliminating false computational paths.

- **Unitary Evolution**: Quantum gates are represented by unitary matrices ($U^\dagger U = I$), which preserve vector norms, prevent information loss, and enforce physical reversibility.
- **Superposition and Measurement**: An n-qubit register lives in a 2^n-dimensional state space, but measurement irreversibly collapses the state, yielding a single classical bitstring with probability $|\alpha_x|^2$.
- **Quantum Algorithms**: Deutsch-Jozsa demonstrates deterministic oracle separation in a single query; Grover's algorithm achieves a quadratic speedup ($\Theta(\sqrt{N})$) for unstructured search via geometric amplitude rotations; and Shor's algorithm achieves exponential speedup for factoring and discrete logs by extracting group periods with the Quantum Fourier Transform.
- **Limits and Fault Tolerance**: The No-Cloning Theorem prevents copying unknown quantum states, but topological surface codes enable fault tolerance by measuring multi-qubit parity syndromes without disturbing data superpositions.

Why this matters

Quantum computing extends the classical model of computation rather than replacing it. Moving from nonnegative probabilities to signed and complex amplitudes lets unitary gates rotate a high-dimensional state vector so that amplitudes on wrong answers cancel while amplitudes on valid answers add up before a final measurement.

This chapter ends the main text, which ran from elementary sorting and dynamic programming to NP-completeness, parallel DAGs, machine learning, systems architecture, and quantum mechanics. The final part of this volume collects the mathematical tools used throughout in three reference appendices: Summations and Discrete Calculus (Appendix 40), Sets, Counting, and Probability (Appendix 41), and Matrices and Discrete Math (Appendix 42).

Exercises

Exercise 39.1. (Hadamard interference by hand.) Suppose a single qubit begins in the state $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$.

1. Compute the state vector after applying the Hadamard gate: $|\psi_1\rangle = H|1\rangle$.
2. What is the probability of observing 0 if $|\psi_1\rangle$ is measured? What is the probability of observing 1?
3. Compute the state vector after applying a second Hadamard gate: $|\psi_2\rangle = H|\psi_1\rangle$. Write down each of the four paths from the intermediate basis states to the final basis states, and verify that the paths to $|0\rangle$ cancel to exact zero while the paths to $|1\rangle$ reinforce.

Exercise 39.2. (Entanglement of the Bell states.) The four Bell states form an orthonormal basis for two qubits:

$$|\Phi^\pm\rangle = \frac{|00\rangle \pm |11\rangle}{\sqrt{2}}, \qquad |\Psi^\pm\rangle = \frac{|01\rangle \pm |10\rangle}{\sqrt{2}}.$$

1. Prove that $|\Psi^+\rangle$ cannot be factored as a product state $(\alpha_1|0\rangle + \beta_1|1\rangle) \otimes (\alpha_2|0\rangle + \beta_2|1\rangle)$.

2. Verify that the four Bell states are mutually orthogonal: show that $\langle \Phi^+ \mid \Phi^- \rangle = 0$ and $\langle \Phi^+ \mid \Psi^+ \rangle = 0$.

Exercise 39.3. (Deutsch's algorithm with an explicit balanced function.) Let $f : \{0,1\} \to \{0,1\}$ be the balanced function $f(x) = x \oplus 1$ (the classical NOT function, so $f(0) = 1$ and $f(1) = 0$). Trace Deutsch's algorithm step-by-step for this specific function:

1. Write down the state after initializing $|0\rangle|1\rangle$ and applying $H \otimes H$.
2. Calculate the state after applying the oracle U_f. Explicitly show how the phase kickback $(-1)^{f(x)}$ produces a minus sign on the $|0\rangle$ branch and a plus sign on the $|1\rangle$ branch of the first qubit.
3. Apply H to the first qubit and show that the final measured state is $|1\rangle$ with probability 1, confirming that f is balanced.

Exercise 39.4. (Grover's search on four items.) Consider a database of $N = 4$ items ($n = 2$ qubits), with marked target $x^\star = 11$.

1. What is the initial state $|s\rangle$ in terms of the standard basis vectors $|00\rangle, |01\rangle, |10\rangle, |11\rangle$?
2. Calculate the angle θ such that $\sin\theta = 1/\sqrt{N} = 1/2$. What is θ in degrees?
3. Recall that one Grover iteration rotates the vector by 2θ. Compute the angle of the state after exactly $R = 1$ Grover iteration.
4. Show that for $N = 4$, exactly **one single Grover iteration** rotates the state to an angle of 90° ($\pi/2$), finding the target with 100% certainty without any approximations!
5. What happens to the success probability if you mistakenly run a second Grover iteration?

Exercise 39.5. (No-Cloning and inner product preservation.) An alternative proof of the No-Cloning Theorem proceeds by considering the inner product of two cloned states. Suppose a unitary operator U clones two distinct states $|\phi\rangle$ and $|\psi\rangle$:

$$U(|\phi\rangle|0\rangle) = |\phi\rangle|\phi\rangle, \qquad U(|\psi\rangle|0\rangle) = |\psi\rangle|\psi\rangle.$$

1. Using the fact that unitary operators preserve inner products, compute the inner product between the left-hand sides: $\langle U(\phi \otimes 0) \mid U(\psi \otimes 0)\rangle$.
2. Compute the inner product between the right-hand sides: $\langle \phi \otimes \phi \mid \psi \otimes \psi \rangle$.
3. Equate the two results to derive the equation $\langle \phi \mid \psi \rangle = (\langle \phi \mid \psi \rangle)^2$.
4. Conclude that cloning is only possible if $\langle \phi \mid \psi \rangle = 0$ (orthogonal states) or $|\langle \phi \mid \psi \rangle| = 1$ (identical states), proving that an unknown arbitrary quantum state cannot be cloned.

Part VIII

Mathematical Background

Chapter 40

Appendix: Summations

You have already met summations everywhere in this book. Every time you wrote "the loop runs once per element, and the i-th pass costs about i steps," and then asked "so how much in total?", you were really asking for a *summation*. The analysis of almost every algorithm bottoms out in adding a list of numbers together and asking how big the answer is.

This appendix is a reference. It collects the handful of sums that show up again and again — arithmetic, geometric, harmonic — proves the formulas you have been quoting, and then teaches you the three reliable *techniques* for sums you do not have a formula for: telescoping, splitting, and bounding by an integral. By the end you should be able to look at any recurrence in this book, expand it into a sum, and extract a clean Θ bound.

Focus

A summation is just repeated addition with a name. The real skill is recognizing the *shape* of a sum so you can replace a loop's worth of work with a single closed-form expression or a tight bound.

History and connection

This appendix corresponds to **CLRS, Appendix A ("Summations")**. For centuries, mathematicians knew how to sum simple arithmetic and geometric sequences, but evaluating arbitrary sums—such as the sum of inverse squares $\sum_{k=1}^{\infty} 1/k^2$ or the partial sums of the harmonic series $H_n = \sum_{k=1}^{n} 1/k$—seemed intractable without summing millions of terms by hand. In 1735, Leonhard Euler solved the Basel problem, proving that the infinite sum of inverse squares converges to exactly $\pi^2/6$. Shortly thereafter, Euler and Scottish mathematician Colin Maclaurin (1742) independently discovered the *Euler–Maclaurin summation formula*. By using Bernoulli numbers as bridge coefficients, Euler and Maclaurin showed how any smooth discrete sum can be expressed as a continuous integral plus an asymptotic series of derivative corrections, yielding the precise expansion $H_n = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + \dots$ where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant.
Yet throughout the 18th century, manipulating infinite sums without convergence criteria bred fierce controversies. In 1703, Guido Grandi argued that the alternating series $1-1+1-1+\dots$ equaled $1/2$, while Euler asserted that $1+2+4+\cdots = -1$ by setting $x = 2$ in $1/(1-x)$. In 1826, Niels Henrik Abel famously lamented that "divergent series are in the main the work of the

devil," urging Augustin-Louis Cauchy and others to establish strict limit-based foundations. When modern computer science emerged in the 1960s, Donald Knuth at Stanford realized that analyzing algorithms demanded turning discrete summation into an exact operational science. In *The Art of Computer Programming* and *Concrete Mathematics* (with Ronald Graham and Oren Patashnik), Knuth unified finite difference operators (Δ) and summation by parts into a rigorous *discrete calculus*. Today, finding closed forms for sums such as Apéry's constant $\zeta(3) = \sum 1/k^3$ remains an active frontier of number theory.

40.1 Notation: what $\sum$ actually says

Be very explicit here, because sloppy reading of $\sum$ causes more grief than the arithmetic itself. The expression

$$\sum_{k=1}^{n} a_k \;=\; a_1 + a_2 + a_3 + \cdots + a_n$$

means: "start the counter k at 1; for each integer value of k up to and including n, compute the *term* a_k; add all those terms together." The letter k is the *index of summation*, 1 is the *lower limit*, n is the *upper limit*, and a_k is the term, sometimes called the *summand*.

Three things trip people up, so pin them down now.

- **The index is a dummy.** $\sum_{k=1}^{n} a_k$ and $\sum_{i=1}^{n} a_i$ and $\sum_{j=1}^{n} a_j$ are the *same number*. The index name vanishes after you add; it never escapes the sum.
- **An empty sum is 0.** If the upper limit is below the lower limit, for example $\sum_{k=1}^{0} a_k$, there are no terms, and the convention is that the total is 0. (Likewise an empty *product* is 1.) This convention keeps formulas working at the boundary.
- **Count the terms.** The sum $\sum_{k=m}^{n}$ has exactly $n - m + 1$ terms, *not* $n - m$. From $k = 3$ to $k = 7$ inclusive is $7 - 3 + 1 = 5$ terms: $3, 4, 5, 6, 7$. Off-by-one errors in term-counting are the number-one source of wrong constants.

Common trap

"$\sum_{k=1}^{n} 1 = 1$." No. The summand happens to be the constant 1, but you still add it once per value of k, so $\sum_{k=1}^{n} 1 = n$. A constant summand c gives $\sum_{k=1}^{n} c = c\,n$. Always ask "how many terms?" before you ask "what is each term?".

40.1.1 Two properties you will use constantly

Summation is *linear*: you can pull constants out and split sums apart. For any constant c and any terms a_k, b_k,

$$\sum_{k=1}^{n} (c\,a_k + b_k) \;=\; c \sum_{k=1}^{n} a_k \;+\; \sum_{k=1}^{n} b_k.$$

Both halves follow from the commutativity and associativity of ordinary addition — you are allowed to reorder and regroup a finite list of numbers however you like. Linearity is what lets you turn a messy summand into a combination of sums you already know.

A second move, *re-indexing* (also called shifting), changes which value the counter starts at without changing the total. If you substitute $j = k - 1$, then when k runs from 1 to n, the new index j runs from 0 to $n - 1$:

$$\sum_{k=1}^{n} a_k \;=\; \sum_{j=0}^{n-1} a_{j+1}.$$

Same terms, relabelled. Re-indexing is the trick that makes telescoping and geometric-series manipulations work; we will lean on it below.

Example 40.1 (Linearity in action)**.** Suppose you need $\sum_{k=1}^{n}(3k+2)$. Rather than expanding by hand, split it:

$$\sum_{k=1}^{n}(3k+2) = 3\sum_{k=1}^{n} k + \sum_{k=1}^{n} 2 = 3 \cdot \frac{n(n+1)}{2} + 2n,$$

using the arithmetic-series formula we prove next. For $n = 4$ this is $3 \cdot 10 + 8 = 38$, and indeed $5 + 8 + 11 + 14 = 38$.

40.2 The arithmetic series

The most fundamental sum in algorithm analysis is $1 + 2 + 3 + \cdots + n$. It is what you get when a loop's inner work grows by one each pass — selection sort, insertion sort's worst case, the nested loops of a naive comparison routine. Its closed form is the one formula in this appendix you should be able to write in your sleep.

Intuition

Picture the numbers 1 through n written left to right, then the *same* numbers written right to left underneath:

$$\begin{array}{cccccc} 1 & 2 & 3 & \cdots & n-1 & n \\ n & n-1 & n-2 & \cdots & 2 & 1 \end{array}$$

Each *column* adds to $n+1$. There are n columns, so both rows together total $n(n+1)$. But the two rows are the same sum, so one row is half of that. This "fold the list in half" argument, traditionally attributed to a young Gauss, is already a complete proof.

Figure 40.1 gives the geometric equivalent of Gauss's pairing: two staircases of $1+2+\cdots+n$ square blocks snap together into a solid $n \times (n+1)$ rectangle of area $n(n+1)$, proving that each staircase occupies exactly half.

Theorem 40.1 (Sum of the first n integers)**.** *For every integer $n \geq 0$,*

$$\sum_{k=1}^{n} k \;=\; \frac{n(n+1)}{2}.$$

Proof. We argue by induction on n, which mirrors the way a loop builds the sum one term at a time. *Base case:* for $n = 0$ the sum is empty, equal to 0, and the formula gives $\frac{0 \cdot 1}{2} = 0$. They agree.

Inductive step: assume $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$ for some $n \geq 0$. Then add the next term $n + 1$:

$$\sum_{k=1}^{n+1} k = \Big(\sum_{k=1}^{n} k\Big) + (n+1) = \frac{n(n+1)}{2} + (n+1) = (n+1) \cdot \frac{n+2}{2} = \frac{(n+1)\big((n+1)+1\big)}{2}.$$

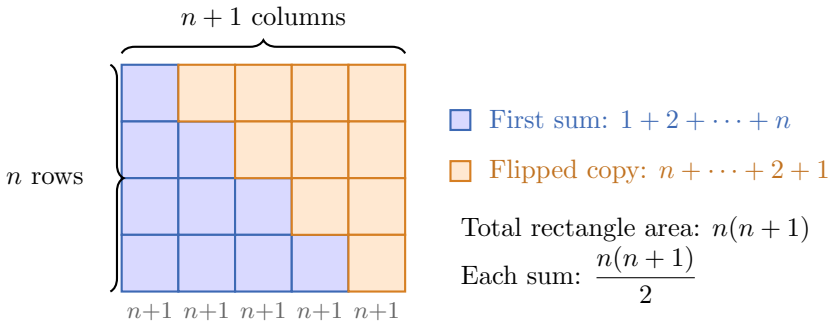


Figure 40.1: Geometric proof of the arithmetic series $\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$. Two identical staircases of blocks (here $n = 4$) interlock into an $n \times (n+1)$ rectangle of 20 blocks. Because the two copies tile the rectangle without gap or overlap, each staircase occupies exactly half the total area.

That is exactly the formula with n replaced by $n+1$, so by induction it holds for all $n \geq 0$. □

The same fold-in-half idea works for *any* arithmetic series — one whose terms step up by a constant difference d. If the first term is a and there are m terms, the last term is $a + (m-1)d$, and

$$\sum_{k=0}^{m-1} (a + kd) \;=\; m \cdot \underbrace{\frac{(\text{first}) + (\text{last})}{2}}_{\text{average term}} \;=\; \frac{m\big(2a + (m-1)d\big)}{2}.$$

Read it as "number of terms times the average of the first and last." That phrasing is easier to remember than the formula, and you can rebuild the formula from it.

Example 40.2 (Tracing a nested loop)**.** Consider this fragment, the loop structure of selection sort:

```
for i = 1 to n:
    for j = i to n:
        do one unit of work
```

The inner loop runs $n - i + 1$ times when the outer index is i. Total work:

$$\sum_{i=1}^{n} (n - i + 1) = \sum_{t=1}^{n} t = \frac{n(n+1)}{2},$$

where we re-indexed with $t = n - i + 1$ (as i goes $1 \to n$, t goes $n \to 1$, the same set of values). So the running time is $\Theta(n^2)$ — and now you know the leading constant is $\frac{1}{2}$, not 1.

Why this matters

Why care about the $\frac{1}{2}$? Because constants matter in practice. Two $\Theta(n^2)$ sorts can differ by $2\times$ in wall-clock time purely from whether the inner loop is $\sum i$ or $\sum 2i$. Asymptotic analysis rules out the slow algorithms; the constants help you choose among the ones that remain.

40.2.1 Sums of squares and higher powers

One step up in difficulty is the sum of squares, which appears whenever the inner cost itself grows quadratically (think of summing the comparison counts in some divide-and-combine routines):

$$\sum_{k=1}^{n} k^2 \;=\; \frac{n(n+1)(2n+1)}{6}.$$

You can prove it by the same induction; the bookkeeping is heavier but the idea is identical. The pattern continues: $\sum_{k=1}^{n} k^p$ is a polynomial in n of degree $p+1$ with leading term $\frac{n^{p+1}}{p+1}$. That single fact —

$$\sum_{k=1}^{n} k^p \;=\; \Theta\big(n^{p+1}\big) \quad \text{for any fixed } p \geq 0$$

— is usually all an algorithm analysis needs, and in Section 40.5 we will see *why* the $\frac{1}{p+1}$ leading constant has to be there, using only the area under a curve.

40.3 The geometric series

The second standard sum is the geometric series, where each term is a fixed ratio x times the one before: $1, x, x^2, x^3, \ldots$. This is the sum behind binary representations, doubling arrays, balanced-tree level counts, and the "merge" levels of merge sort. It behaves *completely differently* depending on whether $x < 1$ or $x > 1$, and understanding that split is the single most useful thing in this appendix.

Theorem 40.2 (Finite geometric series)**.** *For any real $x \neq 1$ and integer $n \geq 0$,*

$$\sum_{k=0}^{n} x^k \;=\; 1 + x + x^2 + \cdots + x^n \;=\; \frac{x^{n+1} - 1}{x - 1}.$$

Proof. The proof is a *telescoping* argument. Call the sum $S = \sum_{k=0}^{n} x^k$. Multiply both sides by x and re-index:

$$xS = \sum_{k=0}^{n} x^{k+1} = \sum_{j=1}^{n+1} x^j = x + x^2 + \cdots + x^{n+1}.$$

Now subtract. Almost every term cancels — the $x, x^2, \ldots, x^n$ appear in both S and xS:

$$xS - S = x^{n+1} - x^0 = x^{n+1} - 1, \qquad \text{so} \qquad S(x-1) = x^{n+1} - 1.$$

Divide by $x - 1$ (legal because $x \neq 1$) to finish. The trick was to create a *shifted copy* of the sum so that subtracting collapses it. □

Focus

Focus on the algebraic technique rather than the raw formula: *multiply by the ratio, shift, subtract, watch the middle cancel.* You will reuse it for sums like $\sum k\,x^k$ that have no formula on the tip of your tongue.

40.3.1 The two regimes: shrinking vs. growing

The behaviour of a geometric series depends on one question: is the ratio x less than 1 or greater than 1?

Shrinking ($0 < x < 1$). The terms get smaller and the sum *converges* even if you let $n \to \infty$. Take the limit of the formula: since $x^{n+1} \to 0$,

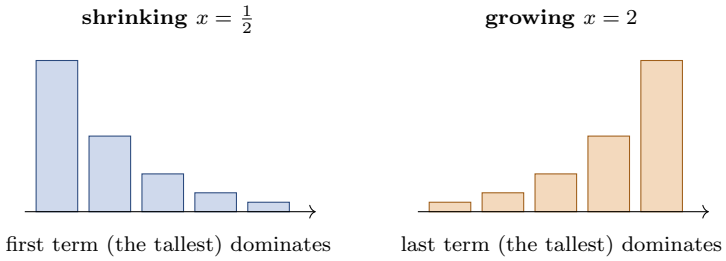
$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}.$$

The consequence for algorithms is that a shrinking geometric sum is *dominated by its first term*. For instance $\sum_{k=0}^{\infty}(1/2)^k = 2$, and the very first term is 1 — the entire infinite tail only adds another 1. So when costs halve at each level, the *top* level pays for everything, up to a constant factor.

Growing ($x > 1$). Now the terms blow up and the sum is *dominated by its last term*. From the formula,

$$\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1} \le \frac{x^{n+1}}{x-1} = x^n \cdot \frac{x}{x-1} = \Theta\big(x^n\big).$$

The last term x^n already is $\Theta(x^n)$, and everything below it adds only a constant multiple. So when costs double at each level, the *bottom* level pays for everything, up to a constant factor.



Each bar is one term x^k, drawn left to right as k increases. When $x < 1$ the bars shrink and the leftmost (first) bar towers over the rest; when $x > 1$ they grow and the rightmost (last) bar does. In both cases the tallest bar alone is a constant fraction of the whole sum — that is exactly what "dominated by one term, up to a constant factor" means.

Intuition

This "first term wins" / "last term wins" dichotomy is the exact mechanism behind the Master Theorem. A recursion tree has geometric per-level costs; whether the root or the leaves dominate is *exactly* the question of whether the ratio is below or above 1. If the per-level cost shrinks, the root wins (case akin to $\Theta(f(n))$); if it grows, the leaves win (case akin to $\Theta(n^{\log_b a})$); if it stays flat ($x = 1$), every level costs the same and you pay an extra factor of the number of levels, $\Theta(\log n)$.

Example 40.3 (Building a dynamic array)**.** A growable array that *doubles* its capacity copies $1, 2, 4, \ldots, n$ elements over its lifetime as it grows to size n. The total copy work is a *growing* geometric series:

$$1 + 2 + 4 + \cdots + n = \sum_{k=0}^{\log_2 n} 2^k = 2^{\log_2 n + 1} - 1 = 2n - 1.$$

Last term wins: the grand total is just under $2n$, even though you resized $\log_2 n$ times. Spread over n insertions, that is $O(1)$ *amortized* cost per insertion. The doubling strategy is cheap precisely because the geometric series is dominated by its final, biggest copy.

40.4 The harmonic series

The third standard sum is subtler, and its behaviour surprises most people at first. The *harmonic series* is

$$H_n = \sum_{k=1}^{n} \frac{1}{k} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}.$$

Each term shrinks toward 0, so you might guess — as with a shrinking geometric series — that the total settles to a finite limit. In fact H_n grows without bound, just *very* slowly, like $\ln n$.

Intuition

Group the terms in blocks whose sizes double: $\{1\}$, $\{\frac{1}{2}\}$, $\{\frac{1}{3}, \frac{1}{4}\}$, $\{\frac{1}{5}, \ldots, \frac{1}{8}\}$, and so on. In the block ending at $\frac{1}{2^m}$ there are 2^{m-1} terms, each at least $\frac{1}{2^m}$, so the block sums to at least $2^{m-1} \cdot \frac{1}{2^m} = \frac{1}{2}$. Every doubling block contributes at least another $\frac{1}{2}$. To get a total of T you need about $2T$ doubling blocks, i.e. about 2^{2T} terms. The sum grows, but you must go exponentially far to gain a constant, which is exactly what logarithmic growth means.

Theorem 40.3 (Harmonic growth)**.** *$H_n = \ln n + \Theta(1)$. More precisely, $\ln n < H_n \le \ln n + 1$, and in fact $H_n = \ln n + \gamma + o(1)$ where $\gamma \approx 0.5772$ is the Euler–Mascheroni constant. For algorithm analysis the bound that matters is*

$$H_n = \Theta(\log n).$$

We will prove the $\ln n < H_n \le \ln n + 1$ bounds cleanly in the next section using integrals — it is the cleanest possible illustration of the integral-bounding technique.

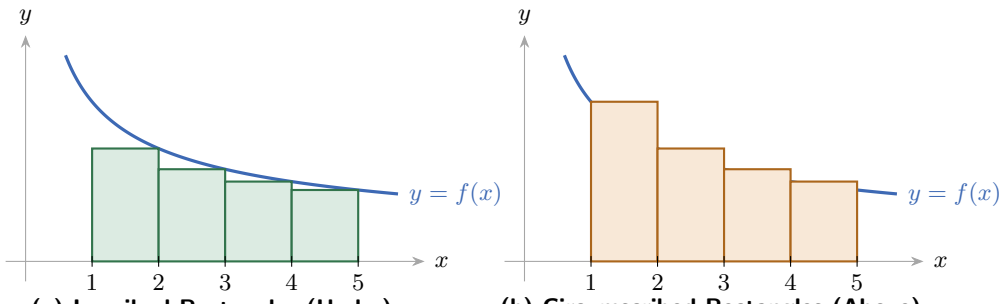
Why this matters

The harmonic series is the running time of the *average case* of quicksort and of building a randomized binary search tree, and it governs the "coupon collector" problem (how many random draws to see all n coupons: about $nH_n \approx n \ln n$). Whenever a cost looks like "$1 + \frac{1}{2} + \frac{1}{3} + \cdots$," you should immediately think $\Theta(\log n)$ and move on.

40.5 Bounding a sum by an integral

This is the most *general* tool in the appendix, and the one to use when no closed form exists. The idea is geometric: a sum $\sum f(k)$ is a row of rectangles, each of width 1 and height $f(k)$, and the integral $\int f(x)\,dx$ is the smooth area under the curve f. If you nest the rectangles against the curve correctly, the integral *traps* the sum from above and below.

Figure 40.2 illustrates the two bounding orientations for a monotonically decreasing function $f(x)$.



(a) Inscribed Rectangles (Under)
Height $f(k+1)$ on $[k, k+1]$
$\int_m^{n+1} f(x)\,dx \le \sum_{k=m}^{n} f(k)$

(b) Circumscribed Rectangles (Above)
Height $f(k)$ on $[k, k+1]$
$\sum_{k=m}^{n} f(k) \le \int_{m-1}^{n} f(x)\,dx$

Figure 40.2: Trapping a summation with integrals for a decreasing function $f(x)$. **(a)** Inscribed rectangles of height $f(k+1)$ on $[k, k+1]$ sit strictly beneath the curve, establishing $\int_m^{n+1} f(x)\,dx \le \sum_{k=m}^{n} f(k)$. **(b)** Circumscribed rectangles of height $f(k)$ on $[k, k+1]$ poke above the curve, establishing $\sum_{k=m}^{n} f(k) \le \int_{m-1}^{n} f(x)\,dx$.

The picture above shows how the geometry forces the bounds: when rectangles tuck completely under the curve, the integral exceeds their sum; when they circumscribe the curve from above, their sum exceeds the integral. Sliding the indices by one unit provides the matching bound from the opposite side. Trapping the rectangles between two slices of the curve is all the technique requires.

The rule has two halves, one for increasing functions and one for decreasing functions. State it once and you never have to rederive it.

Formal

Integral bounds. Let f be *monotone* on the relevant range.

- If f is *increasing*, then

$$\int_{m-1}^{n} f(x)\,dx \;\le\; \sum_{k=m}^{n} f(k) \;\le\; \int_{m}^{n+1} f(x)\,dx.$$

- If f is *decreasing*, then

$$\int_{m}^{n+1} f(x)\,dx \;\le\; \sum_{k=m}^{n} f(k) \;\le\; \int_{m-1}^{n} f(x)\,dx.$$

In words: *the rectangle of height $f(k)$ sits between the lower and upper slices of curve next to it.* You rarely need to remember which inequality is which — just draw the rectangles against the curve and read off the picture.

Example 40.4 (Why $\sum k^p = \Theta(n^{p+1})$)**.** Let $f(x) = x^p$ for a fixed $p \ge 0$; it is increasing, so

$$\int_{0}^{n} x^p\,dx \;\le\; \sum_{k=1}^{n} k^p \;\le\; \int_{1}^{n+1} x^p\,dx.$$

The left integral is $\frac{n^{p+1}}{p+1}$ and the right is $\frac{(n+1)^{p+1}-1}{p+1}$. Both are $\Theta(n^{p+1})$ with leading constant $\frac{1}{p+1}$, so

$$\sum_{k=1}^{n} k^p = \frac{n^{p+1}}{p+1} + \Theta(n^p) = \Theta\big(n^{p+1}\big).$$

No induction or guessing of the polynomial is needed; the area under x^p gives the answer directly.

Example 40.5 (Pinning down the harmonic series)**.** Now take $f(x) = 1/x$, which is decreasing. The decreasing rule gives

$$\int_1^{n+1} \frac{dx}{x} \;\le\; \sum_{k=1}^{n} \frac{1}{k} \;\le\; 1 + \int_1^{n} \frac{dx}{x}.$$

(The extra "1" on the right is the first term $f(1) = 1$, peeled off so the remaining terms line up under the decreasing curve from 1 to n.) Since $\int_1^t \frac{dx}{x} = \ln t$, we get

$$\ln(n+1) \;\le\; H_n \;\le\; 1 + \ln n.$$

So H_n is squeezed between $\ln n$ and $\ln n + 1$: it is $\ln n$ plus a number that never escapes the interval $[0, 1]$. That is the promised proof of Theorem 40.3: the staircase of terms stays within 1 of the logarithm curve.

Walking question

Take $f(x) = \ln x$ on a walk. Use the integral bounds to show that $\sum_{k=1}^{n} \ln k$ — which is $\ln(n!)$ — is $\Theta(n \ln n)$. This is the leading-order part of *Stirling's formula*, and it is the reason comparison sorting cannot beat $n \log n$. (Hint: $\int \ln x \, dx = x \ln x - x$.)

40.6 Telescoping: when the middle cancels

A *telescoping* sum is one whose terms are built as a difference $a_k - a_{k-1}$ (or $a_{k+1} - a_k$), so that consecutive pieces annihilate and only the two endpoints survive — like a collapsing spyglass. Formally,

$$\sum_{k=1}^{n} (a_k - a_{k-1}) \;=\; a_n - a_0.$$

You already saw a disguised telescope: the geometric-series proof subtracted S from xS and watched the middle collapse. Look for telescoping whenever the summand is a difference, and try to rewrite the summand as one when it is not.

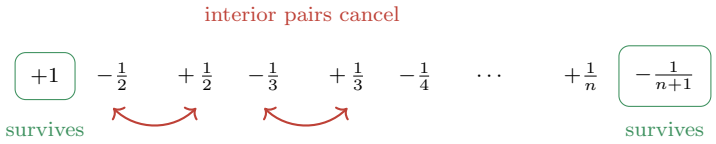
Example 40.6 (A partial-fractions telescope)**.** Evaluate $\displaystyle\sum_{k=1}^{n} \frac{1}{k(k+1)}$. The key observation is the partial-fraction split

$$\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1}.$$

So the sum telescopes:

$$\sum_{k=1}^{n} \left(\frac{1}{k} - \frac{1}{k+1} \right) = \left(1 - \tfrac{1}{2}\right) + \left(\tfrac{1}{2} - \tfrac{1}{3}\right) + \cdots + \left(\tfrac{1}{n} - \tfrac{1}{n+1}\right) = 1 - \frac{1}{n+1}.$$

Every interior fraction is created once with a $+$ and destroyed once with a $-$. The diagram below shows the bookkeeping: each $-\frac{1}{k+1}$ from one term pairs off against the $+\frac{1}{k+1}$ from the next, leaving only the very first $+1$ and the very last $-\frac{1}{n+1}$ unmatched.



The closed form $1 - \frac{1}{n+1}$ is exact, and it tends to 1 as $n \to \infty$. For $n = 3$: $\frac{1}{2} + \frac{1}{6} + \frac{1}{12} = \frac{9}{12} = \frac{3}{4}$, and the formula gives $1 - \frac{1}{4} = \frac{3}{4}$.

Common trap

Telescoping fails the moment a term does *not* have a matching opposite. If your summand is $a_{k+1} - a_{k-1}$ (a gap of two), the cancellation leaves *two* terms at each end, not one. Always write out the first three and last three terms before declaring victory — the surviving terms are exactly those that lack a partner.

40.7 A field guide to the recurrences in this book

All of these techniques come together when you unroll a recurrence into a sum. Here is the workflow, then the catalog.

Algorithm lab

The unroll-and-sum recipe.

1. Write $T(n)$ in terms of smaller inputs.
2. Substitute the recurrence into itself a few times until you see the pattern of per-level costs.
3. Express the total as a *sum over levels* (or over i).
4. Identify the sum's shape — arithmetic, geometric, or harmonic — and apply the matching formula or bound.

Almost every analysis in this book is an instance of this recipe.

40.7.1 Subtract-a-constant recurrences $\to$ arithmetic sums

A recurrence like $T(n) = T(n-1) + n$ (the cost of inserting into the back of a sorted list, summed over a build) unrolls into the arithmetic series:

$$T(n) = T(0) + \sum_{k=1}^{n} k = \frac{n(n+1)}{2} = \Theta(n^2).$$

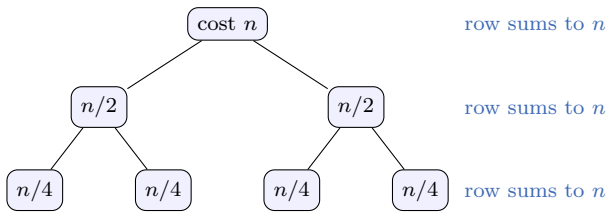
More generally $T(n) = T(n-1) + f(n)$ telescopes to $T(0) + \sum_{k=1}^{n} f(k)$, and you finish with the integral bound on $\sum f(k)$. Notice this is literally telescoping (Section 40.6) followed by a sum.

40.7.2 Halve-the-input recurrences → geometric sums

A recurrence like $T(n) = 2\,T(n/2) + n$ (merge sort) unrolls into a recursion tree with $\log_2 n$ levels, and the per-level cost is n at *every* level because the doubling number of subproblems exactly cancels their halving size. This is the flat case $x = 1$:

$$T(n) = \sum_{\text{level } i=0}^{\log_2 n} n = n\,(\log_2 n + 1) = \Theta(n \log n).$$

Change the combine cost and the per-level sum becomes a genuine geometric series. With $T(n) = 2\,T(n/2) + n^2$ the per-level cost *shrinks* geometrically ($x = \frac{1}{2}$): the root's n^2 dominates and $T(n) = \Theta(n^2)$. With $T(n) = 4\,T(n/2) + n$ it *grows* ($x = 2$): the leaves dominate and $T(n) = \Theta(n^2)$. First-term-wins versus last-term-wins, exactly as in Section 40.3.1.



The picture makes the flat case obvious: each row of the merge-sort recursion tree costs n, there are about $\log_2 n$ rows, so the sum is the constant n added $\log_2 n$ times.

40.7.3 Random-pivot and tree-build recurrences → harmonic sums

The expected number of comparisons in randomized quicksort satisfies a recurrence whose solution is governed by $\sum 1/k$. The clean way to see it: the elements of rank i and rank j (with $i < j$) are compared with probability $\frac{2}{j-i+1}$, and summing those probabilities over all pairs gives the expected total. Fix the smaller rank i and let the gap $g = j - i$ run from 1 up to at most $n-1$; the inner sum over gaps is then a harmonic series:

$$\sum_{i<j} \frac{2}{j-i+1} \;=\; \sum_{i=1}^{n} \sum_{j=i+1}^{n} \frac{2}{j-i+1} \;\le\; \sum_{i=1}^{n} 2\,H_n \;=\; 2n\,H_n \;=\; \Theta(n \log n),$$

using $H_n = \Theta(\log n)$ from Section 40.4. (The middle step replaces the inner sum $\sum_{g=1}^{n-i} \frac{2}{g+1}$, which is at most $2(H_n - 1) < 2H_n$, by the larger constant $2H_n$ to keep the bookkeeping simple.) Whenever a recurrence averages over a uniformly random split point, expect a harmonic series and therefore a $\log n$ factor.

Cost summary

The three shapes and their verdicts, all in one place:

Sum	Closed form / bound	Asymptotic
$\sum_{k=1}^{n} k$	$n(n+1)/2$	$\Theta(n^2)$
$\sum_{k=1}^{n} k^p$	$\approx n^{p+1}/(p+1)$	$\Theta(n^{p+1})$
$\sum_{k=0}^{n} x^k,\ x<1$	$\leq 1/(1-x)$	$\Theta(1)$ (first term wins)
$\sum_{k=0}^{n} x^k,\ x=1$	$n+1$	$\Theta(n)$
$\sum_{k=0}^{n} x^k,\ x>1$	$(x^{n+1}-1)/(x-1)$	$\Theta(x^n)$ (last term wins)
$\sum_{k=1}^{n} 1/k$	$\ln n + \Theta(1)$	$\Theta(\log n)$
$\sum_{k=1}^{n} \ln k$	$n \ln n - n + \Theta(\ln n)$	$\Theta(n \log n)$

40.8 Putting it together: a fully worked analysis

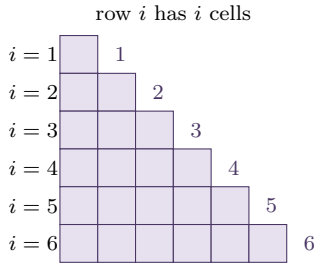
To close, we analyze a small algorithm from scratch, using only the tools above. Consider:

Algorithm 156: TriangleSums

Input: An integer $n \geq 1$.
Output: The value $\sum_{i=1}^{n} \sum_{j=1}^{i} j$.

```
1 total ← 0;
2 for i ← 1 to n do
3 |   for j ← 1 to i do
4 |   |   total ← total + j;
5 return total;
```

Counting the work. The body $total \leftarrow total + j$ runs once for each pair (i, j) with $1 \leq j \leq i \leq n$. Those pairs form a triangle: one cell in row $i = 1$, two in row $i = 2$, and so on, as the picture shows.



For a fixed i the inner loop runs i times, so the total number of additions is the number of cells in the triangle:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2),$$

an arithmetic series (Section 40.2). So the running time is $\Theta(n^2)$.

Computing the output. The returned value is a *nested* sum, and we peel it from the inside out. The inner sum is itself arithmetic:

$$\sum_{j=1}^{i} j = \frac{i(i+1)}{2}.$$

Substitute and use linearity plus the sum-of-squares formula (Section 40.2.1):

$$\sum_{i=1}^{n} \frac{i(i+1)}{2} = \frac{1}{2}\sum_{i=1}^{n}(i^2+i) = \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right) = \frac{n(n+1)(n+2)}{6}.$$

A sanity check. For $n = 3$ the algorithm computes $(1) + (1{+}2) + (1{+}2{+}3) = 1 + 3 + 6 = 10$, and the formula gives $\frac{3\cdot4\cdot5}{6} = 10$. The output is $\Theta(n^3)$ while the running time is only $\Theta(n^2)$ — a reminder that the *size of a number* and the *time to compute it* are different sums, and you must do both.

Pause

If the worked example felt like a lot, that is fine — it deliberately stacks three techniques (term-counting, the arithmetic formula, and the sum-of-squares formula) in one place. In real analysis you will usually need only one of them at a time. The general skill is the recipe: *name the shape, then apply the tool.*

40.9 Summary: Summation Reference and Bounds

When evaluating algorithm running times, use this reference checklist to classify and bound discrete sums:

- **Linearity**: $\sum_k (ca_k + b_k) = c\sum_k a_k + \sum_k b_k$. Constants pull out; sums split across additions.
- **Arithmetic Series**: $\sum_{k=1}^{n} k = \frac{n(n+1)}{2} = \Theta(n^2)$. General polynomial sums $\sum_{k=1}^{n} k^p = \Theta(n^{p+1})$.
- **Geometric Series**: $\sum_{k=0}^{n} x^k = \frac{x^{n+1}-1}{x-1}$. When ratio $|x| < 1$, the infinite sum converges to $\frac{1}{1-x} = \Theta(1)$ (the first term dominates). When $x > 1$, the sum is $\Theta(x^n)$ (the last term dominates).
- **Harmonic Series**: $H_n = \sum_{k=1}^{n} \frac{1}{k} = \ln n + \gamma + O(1/n) = \Theta(\log n)$.
- **Approximation by Integrals**: For any monotonically increasing function $f(x)$:

$$\int_{m-1}^{n} f(x)\,dx \;\le\; \sum_{k=m}^{n} f(k) \;\le\; \int_{m}^{n+1} f(x)\,dx.$$

- **Bounding Traps**: Avoid replacing non-constant terms with the maximum term when summands vary over orders of magnitude (e.g., bounding $\sum 1/k$ by $\sum 1 = n$ gives an exponential over-estimate).

We turn next to sets, relations, combinatorics, and probability in Appendix 41, where these exact series are used to count permutations, evaluate expectations, and bound tail probabilities.

Exercises

Exercise 40.1 (Term counting warmup)**.** How many terms are in $\sum_{k=7}^{30} k$? Evaluate the sum two ways: directly with the arithmetic formula on the shifted range, and as $\sum_{k=1}^{30} k - \sum_{k=1}^{6} k$. Confirm the answers agree.

Exercise 40.2 (Constant summands)**.** Evaluate $\sum_{k=1}^{n} 5$ and $\sum_{k=3}^{n} 5$ as closed forms in n. Then explain in one sentence why $\sum_{k=1}^{n} c \neq c$ for $n > 1$.

Exercise 40.3 (Linearity practice)**.** Find a closed form for $\sum_{k=1}^{n}(4k-3)$. Check your formula at $n = 1$ and $n = 4$ by direct addition.

Exercise 40.4 (Geometric regimes)**.** Compute $\sum_{k=0}^{10} 3^k$ exactly using the finite geometric formula, and $\sum_{k=0}^{\infty}(1/3)^k$ exactly using the infinite formula. For each, state whether the first or the last term dominates.

Exercise 40.5 (Engineer a telescope)**.** Verify the identity $\frac{1}{k(k+2)} = \frac{1}{2}\left(\frac{1}{k} - \frac{1}{k+2}\right)$, and use it to evaluate $\sum_{k=1}^{n} \frac{1}{k(k+2)}$ in closed form. (Caution: the gap is two, so *two* terms survive at each end — write out the first and last few terms.)

Exercise 40.6 (Integral bound from scratch)**.** Using the integral bounds with $f(x) = \sqrt{x}$, prove that $\sum_{k=1}^{n} \sqrt{k} = \Theta(n^{3/2})$ and identify the leading constant.

Exercise 40.7 (A scaled harmonic sum)**.** A program does $\sum_{k=1}^{n} \frac{n}{k}$ units of work (each pass does n/k work). Show this is $\Theta(n \log n)$, and explain which series from this appendix you used.

Exercise 40.8 (Recurrence to sum)**.** Solve $T(n) = T(n-1) + 2n - 1$ with $T(0) = 0$ by unrolling into a sum. (Hint: the answer is a familiar perfect square — recognize the sum of the first n odd numbers.)

Exercise 40.9 (Which level wins?)**.** For each recurrence, decide whether the recursion tree's per-level cost shrinks, stays flat, or grows, and give the resulting Θ: (a) $T(n) = 2T(n/2) + 1$, (b) $T(n) = 2T(n/2) + n$, (c) $T(n) = 3T(n/2) + n$. Tie each answer back to the geometric-series regime that decides it.

Exercise 40.10 (Stirling on a walk, written up)**.** Carry out the Walking-question prompt rigorously: use the integral bounds on $f(x) = \ln x$ to prove $n \ln n - n < \ln(n!) \leq (n+1)\ln(n+1) - n$, and conclude $\ln(n!) = \Theta(n \log n)$. State one consequence for sorting lower bounds.

Exercise 40.11 (Challenge: an arithmetic–geometric sum)**.** Evaluate $\sum_{k=1}^{n} k\,2^k$ in closed form by adapting the "multiply-by-the-ratio-and-subtract" telescoping trick from the geometric-series proof. (Hint: let S be the sum, consider $2S - S$, and you will be left with a plain geometric series to finish.)

Exercise 40.12 (Challenge: bounding without a formula)**.** There is no elementary closed form for $\sum_{k=1}^{n} \frac{1}{k^2}$, yet it *converges*. Use the telescoping bound $\frac{1}{k^2} \leq \frac{1}{k(k-1)} = \frac{1}{k-1} - \frac{1}{k}$ (for $k \geq 2$) to prove that $\sum_{k=1}^{n} \frac{1}{k^2} < 2$ for every n. Why does this confirm the sum is $\Theta(1)$, unlike the harmonic series?

Chapter 41

Appendix: Sets, Counting, and Probability

Building on the summation techniques established in Appendix 40, this appendix serves as the mathematical reference for sets, relations, counting, and probability. The rest of the book leans on it constantly — every time we say "there are $n!$ orderings," "the expected running time is $O(n \log n)$," or "the probability of a bad pivot is small," we are quietly using the language built here. CLRS folds this into its appendices and expects you to already know it; we build it from the ground up, in plain words.

You can read this front to back, or keep it open as a reference when later chapters say "recall expectation" or "by linearity." By the end you will be able to count the configurations an algorithm explores and reason about chance when an algorithm flips coins.

History and connection

This material corresponds to Appendices B (Sets, Relations, Functions) and C (Counting and Probability) of CLRS. The mathematics of chance began with a gambling question. In the summer of 1654, the French nobleman and amateur gambler Antoine Gombaud, known as the Chevalier de Méré, posed a classical puzzle to Blaise Pascal: the *Problem of Points*. If two players are forced to abandon a game of dice before finishing, how should the pot of money be divided fairly based on the current score? Pascal began an epistolary exchange with Pierre de Fermat in Toulouse. Fermat solved the puzzle by listing all possible future roll sequences, while Pascal solved it recursively using his newly minted arithmetic triangle of binomial coefficients. In doing so, they created the mathematical theory of probability.

A century later, the question inverted: rather than calculating the chance of an outcome from known rules, how do we deduce the hidden cause behind an observed event? In 1763, the English nonconformist minister Thomas Bayes wrote an essay (presented posthumously to the Royal Society by Richard Price) establishing inverse probability. In 1774, Pierre-Simon Laplace independently discovered the same principle, developing what we now call *Bayes' Theorem* and applying it to celestial mechanics and jury verdicts.

Throughout the 18th and 19th centuries, naive probability sparked fierce controversies over subtle fallacies—such as Jean d'Alembert's 1754 assertion that two coin tosses have only three outcomes (HH, TT, and one of each) rather than four equally likely permutations. In Saint

Petersburg, Pafnuty Chebyshev (1867) and his student Andrey Markov (1900) proved that even when an exact probability distribution is completely unknown, one can derive rigorous *tail bounds* from its first two moments alone—its mean (Markov's inequality) and its variance (Chebyshev's inequality). In 1901, Bertrand Russell exposed paradoxes in naive set theory (the set of all sets that do not contain themselves), precipitating formal axiomatization. Finally, in 1933, Andrey Kolmogorov unified probability with measure theory into the rigorous axiomatic foundation used across modern computer science. The frontier remains active: proving tight concentration inequalities for high-dimensional dependent random variables in machine learning and randomized algorithms.

41.1 Sets: the language everything is written in

Intuition

A *set* is just a collection of distinct things, with no order and no repetition. Think of it as a bag where you only care *which* items are inside, not how many copies there are or what order you pulled them out. The bag $\{3, 1, 2\}$ is the very same bag as $\{1, 2, 3\}$ and as $\{1, 1, 2, 3\}$ — the duplicate and the reordering simply do not register.

We write $x \in A$ to say "x is an element of A" and $x \notin A$ for the negation. The set with nothing in it is the *empty set*, written $\emptyset$ or $\{\}$. Some sets you already know:

$$\mathbb{N} = \{0, 1, 2, 3, \dots\}, \qquad \mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}, \qquad \mathbb{R} = \text{the real numbers.}$$

(We follow the convention that $\mathbb{N}$ includes 0; some authors start at 1. When it matters, we will say so.)

A set can be described by *listing* its elements, $\{2, 4, 6, 8\}$, or by giving a *rule*, the set-builder notation

$$\{\, x \in \mathbb{N} : x \text{ is even and } x \leq 8 \,\}.$$

Read the colon as "such that." Both descriptions name the same four-element set.

41.1.1 Size, subsets, and the empty set

The number of elements in a finite set A is its *cardinality*, written $|A|$. So $|\{2, 4, 6, 8\}| = 4$ and $|\emptyset| = 0$. Cardinality is the quantity this chapter teaches you to compute above all others, because the running time of an algorithm is almost always "do a fixed amount of work for each element of some set," and so the time is "|that set| times a constant."

We say A is a *subset* of B, written $A \subseteq B$, when every element of A is also an element of B. The empty set is a subset of *every* set (there is no element of $\emptyset$ to check, so the condition holds vacuously), and every set is a subset of itself. If $A \subseteq B$ but $A \neq B$ we call A a *proper* subset and write $A \subset B$.

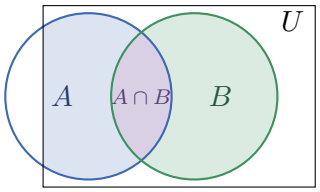
Common trap

Do not confuse $\in$ with $\subseteq$. The number 3 is an *element* of $\{1, 2, 3\}$, so $3 \in \{1, 2, 3\}$. The one-element set $\{3\}$ is a *subset*, so $\{3\} \subseteq \{1, 2, 3\}$. Writing $3 \subseteq \{1, 2, 3\}$ or $\{3\} \in \{1, 2, 3\}$ is a type error — the same kind of mistake as adding a pointer to an integer. Keep "is one of the things inside" ($\in$) separate from "is a smaller bag inside" ($\subseteq$).

41.1.2 Operations on sets

Given two sets A and B, we combine them in four standard ways. Picture two overlapping circles — a Venn diagram — as you read each one.

- **Union** $A \cup B$: everything in A, or in B, or in both.
- **Intersection** $A \cap B$: only the things in *both*.
- **Difference** $A \setminus B$: the things in A that are *not* in B.
- **Complement** $\overline{A}$: everything (in some agreed-upon universe U) that is *not* in A. So $\overline{A} = U \setminus A$.



Two sets with empty intersection, $A \cap B = \emptyset$, are called *disjoint*: their circles do not overlap. Disjointness is what makes counting clean, as the next section shows.

Formal

De Morgan's laws. For any sets in a universe U,

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \qquad \overline{A \cap B} = \overline{A} \cup \overline{B}.$$

In words: "not (in A or B)" means "not in A and not in B," and "not (in both)" means "missing from at least one." These mirror the logic laws $\neg(p \vee q) = \neg p \wedge \neg q$ exactly, because membership in a set A is equivalent to the predicate $x \in A$.

41.1.3 Ordered pairs, tuples, and the Cartesian product

Sets forget order. But algorithms care about order all the time — a coordinate (x, y) is not the same point as (y, x). So we also need *ordered* structure.

An *ordered pair* (a, b) remembers which came first. Two pairs are equal only when both slots match: $(a, b) = (c, d)$ exactly when $a = c$ and $b = d$. More generally an *n-tuple* $(a_1, a_2, \ldots, a_n)$ is an ordered list of n slots.

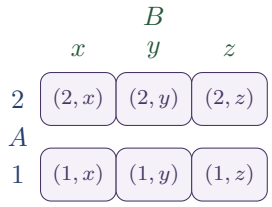
The *Cartesian product* $A \times B$ is the set of *all* ordered pairs with first slot from A and second from B:

$$A \times B = \{\, (a, b) : a \in A,\ b \in B \,\}.$$

Example 41.1 (A small product)**.** Let $A = \{1, 2\}$ and $B = \{x, y, z\}$. Then

$$A \times B = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\},$$

which has $2 \times 3 = 6$ elements. The grid you would draw to list them — 2 rows, 3 columns — is exactly why $|A \times B| = |A| \cdot |B|$. The same picture underlies the product rule in the counting section.



The $|A| = 2$ rows times the $|B| = 3$ columns give one cell per pair: a $2 \times 3 = 6$ rectangle of outcomes.

41.2 Relations and functions

41.2.1 Relations

A *relation* from A to B is any subset $R \subseteq A \times B$. In other words, a relation is a rule that, for each pair (a, b), says either "yes, these are related" (the pair is in R) or "no" (it is not). When $(a, b) \in R$ we often write $a\,R\,b$. The familiar $\leq$ on numbers is a relation: the pair $(3, 5)$ is in it because $3 \leq 5$; the pair $(5, 3)$ is not.

A relation R on a single set A (meaning $R \subseteq A \times A$) can have nice properties:

- **Reflexive**: $a\,R\,a$ for every a. (Everything relates to itself.)
- **Symmetric**: whenever $a\,R\,b$, also $b\,R\,a$.
- **Transitive**: whenever $a\,R\,b$ and $b\,R\,c$, also $a\,R\,c$.

A relation that is reflexive, symmetric, and transitive all at once is an *equivalence relation*. It carves A into disjoint clumps — *equivalence classes* — where everything in a clump is related to everything else in that clump and to nothing outside it. "Has the same remainder mod 7" is an equivalence relation on $\mathbb{Z}$; it sorts the integers into 7 classes. Union-find (Chapter 19) maintains exactly these equivalence classes: its job is to track which clump each element belongs to.

41.2.2 Functions

Intuition

A *function* is a machine: feed it one input, it gives you back exactly one output. The essential word is *exactly one*. A vending machine that sometimes returns chips and sometimes a soda for the same button-press is not a function; a machine that returns nothing for some buttons is not (a total) function either. One input in, one output out, every time.

Formally a function $f : A \to B$ is a special relation: for each $a \in A$ there is exactly one $b \in B$ with $(a, b) \in f$, and we write $b = f(a)$. The set A is the *domain* (allowed inputs), B is the *codomain* (the type of the outputs), and the *image* (or range) is the set of values actually hit, $\{\, f(a) : a \in A \,\}$.

Three properties matter constantly in algorithms:

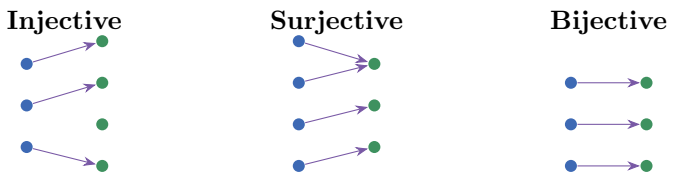
- **Injective** (one-to-one): different inputs give different outputs. If $f(a_1) = f(a_2)$ then $a_1 = a_2$. No collisions.
- **Surjective** (onto): every element of the codomain is hit by something. The image equals the whole codomain.
- **Bijective**: both injective and surjective. A perfect pairing — every input matched to a unique output and vice versa.

Focus

Bijections are a central tool in counting. If you can build a bijection between two finite sets, you have *proved* they are the same size without counting either one. "Show $|A| = |B|$" becomes "pair them up perfectly." The next section uses this idea, and it returns whenever a hash table claims a one-to-one slot assignment or a sorting argument matches inputs to outputs.

Example 41.2 (Injective but not surjective)**.** Let $f : \mathbb{N} \to \mathbb{N}$ be $f(n) = 2n$. Different inputs clearly give different outputs, so f is injective. But the image is $\{0, 2, 4, 6, \dots\}$ — the odd numbers are never produced — so f is not surjective. A function can be one without the other.

The three properties are easiest to see as pictures of arrows from domain to codomain. Injective forbids two arrows landing on the same target; surjective demands every target be hit; a bijection is both at once — a perfect pairing with no target missed and none doubled.



In the left picture a codomain dot is left untouched (not surjective); in the middle, two arrows share a target (not injective); only the right picture is both.

41.3 Counting without enumeration

Counting is where set language pays off. The question is always the same shape: "how many objects of such-and-such kind are there?" Listing them all and tallying is hopeless — there are often astronomically many — so the skill is finding the answer *without* listing.

Everything rests on two rules.

Formal

Sum rule. If a set S is split into disjoint pieces $S_1, S_2, \ldots, S_k$ (no overlap, together they cover all of S), then

$$|S| = |S_1| + |S_2| + \cdots + |S_k|.$$

Product rule. If an object is built by making a sequence of k independent choices, where the i-th choice has n_i options *regardless* of the earlier choices, then the number of objects is

$$n_1 \cdot n_2 \cdots n_k.$$

The product rule is exactly the grid from Example 41.1 stretched to k dimensions. Want to count 3-letter strings over a 26-letter alphabet? Three choices, 26 options each, independent: $26^3 = 17{,}576$. Want license plates of the form letter-letter-digit-digit-digit? $26 \cdot 26 \cdot 10 \cdot 10 \cdot 10 = 676{,}000$.

Algorithm lab

Trace the product rule by hand. Count the binary strings of length 3. Choice 1: $\{0, 1\}$. Choice 2: $\{0, 1\}$. Choice 3: $\{0, 1\}$. The product rule predicts $2^3 = 8$. Now actually write them out: $000, 001, 010, 011, 100, 101, 110, 111$. Eight, as predicted. The first half all start with 0, the second half with 1 — that is the sum rule $(4 + 4)$ inside the product rule. Most counting arguments combine the two rules.

41.3.1 Permutations: counting orderings

A *permutation* is an arrangement of objects in a row, where order matters and nothing repeats. How many ways can you line up n distinct objects?

Reason with the product rule. The first slot: n choices. The second slot: $n - 1$ remaining objects. The third: $n - 2$. And so on down to the last slot with 1 choice. Multiply:

$$n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1 = n!,$$

read "n factorial." By convention $0! = 1$ (there is exactly one way to arrange nothing: do nothing).

Example 41.3 (Three books on a shelf)**.** Three distinct books $\{A, B, C\}$ arranged in a row: $3! = 6$ ways. List them to be sure: $ABC, ACB, BAC, BCA, CAB, CBA$. Six. The first letter has 3 choices, then 2, then the last is forced — $3 \cdot 2 \cdot 1 = 6$.

Sometimes you only fill k of the n slots — arrange k objects chosen from n, order mattering. This is the *k-permutation*, and the same product-rule argument (now stopping after k factors) gives

$$P(n, k) = n \cdot (n-1) \cdots (n-k+1) = \frac{n!}{(n-k)!}.$$

For instance, the number of ways to award gold, silver, and bronze among 8 runners is $P(8,3) = 8 \cdot 7 \cdot 6 = 336$.

Why this matters

Why does an algorithms book care about $n!$? Because $n!$ is the number of possible orderings of n items, and *sorting* is the act of finding the one correct ordering. Any comparison-based sorting algorithm must, in effect, distinguish among all $n!$ possibilities, and a decision tree that distinguishes $n!$ leaves needs depth at least $\log_2(n!) = \Theta(n \log n)$. This counting fact gives the $\Omega(n \log n)$ lower bound for comparison sorting.

41.3.2 Combinations: counting selections

Now order does *not* matter. How many ways can you choose a k-element subset from n distinct objects? This count is written $\binom{n}{k}$, read "n choose k," and it is the most important number in this chapter.

To derive it, first count *ordered* selections of k objects: that is $P(n,k) = \frac{n!}{(n-k)!}$. But every unordered subset of size k has been counted once for each of its $k!$ orderings. So to get the unordered count we divide out the overcounting:

$$\binom{n}{k} = \frac{P(n,k)}{k!} = \frac{n!}{k!\,(n-k)!}.$$

This "count with order, then divide by the orderings" move is one of the most useful techniques in combinatorics, visualized in Figure 41.1. Spotting and dividing out overcounting solves a large share of counting problems.

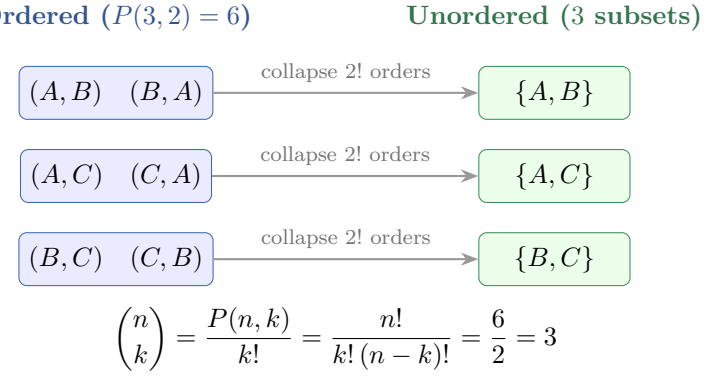


Figure 41.1: Why combinations divide out $k!$. Choosing $k = 2$ items from $\{A, B, C\}$ yields $P(3,2) = 6$ ordered sequences. When order no longer matters, every group of $k! = 2! = 2$ sequences collapses into a single subset. Dividing total permutations by the overcounting factor $k!$ gives the number of combinations $\binom{3}{2} = 3$.

Example 41.4 (Choosing a committee)**.** From 5 people, choose a committee of 2 (no roles, so order is irrelevant):

$$\binom{5}{2} = \frac{5!}{2!\,3!} = \frac{120}{2 \cdot 6} = 10.$$

Sanity check against the ordered count: there are $5 \cdot 4 = 20$ ways to pick a first then a second person, but each committee $\{a, b\}$ was counted twice (as ab and as ba), so $20/2! = 10$. The two routes agree.

Common trap

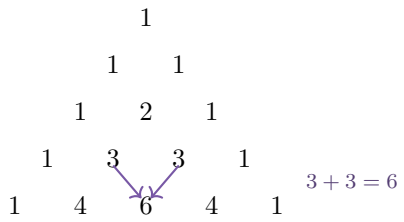
A common mistake is to use a permutation when a combination is called for, or vice versa. The deciding question is one sentence: *does rearranging the chosen items give a different outcome?* Gold/silver/bronze — yes, order matters, use $P(n,k)$. A handful of lottery numbers, a poker hand, a subset of array indices — no, order is irrelevant, use $\binom{n}{k}$. Ask the question before you reach for a formula.

41.3.3 Identities to understand rather than memorize

A few facts about $\binom{n}{k}$ come up so often that they are worth understanding by sight rather than by rote.

Symmetry. $\binom{n}{k} = \binom{n}{n-k}$. Choosing the k you keep is the same act as choosing the $n-k$ you leave behind; pairing each subset with its complement is a bijection, so no algebra is needed.

Pascal's rule. $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$. Fix one special element, say the last. A k-subset either contains it ($\binom{n-1}{k-1}$ ways to fill the rest) or does not ($\binom{n-1}{k}$ ways). Those two cases are disjoint and cover everything — the sum rule in action. This is also the recurrence that builds Pascal's triangle, where each entry is the sum of the two above it:



Sum of a row. $\displaystyle\sum_{k=0}^{n} \binom{n}{k} = 2^n$. The left side counts subsets of every size; the right side counts *all* subsets directly, since each of the n elements is independently in or out (product rule, $2 \cdot 2 \cdots 2$). Two ways to count the same thing must agree.

41.3.4 The binomial theorem

These counts combine in one formula. When you expand $(x+y)^n$, you multiply out n copies of $(x+y)$ and collect terms. Each term picks either x or y from each factor; a term with $x^k y^{n-k}$ comes from choosing x in exactly k of the n factors — and there are $\binom{n}{k}$ ways to choose which. Hence

Formal

Binomial theorem.

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k \, y^{n-k}.$$

The numbers $\binom{n}{k}$ are exactly the coefficients — which is why they are called *binomial* coefficients.

Setting $x = y = 1$ recovers $\sum_k \binom{n}{k} = 2^n$ instantly; setting $x = 1$, $y = -1$ gives $\sum_k (-1)^{n-k}\binom{n}{k} = 0$, a handy alternating identity.

Walking question

On your next walk, prove this to yourself without algebra: why is the number of *lattice paths* from the bottom-left to the top-right corner of an $a \times b$ grid, moving only right or up, equal to $\binom{a+b}{a}$? Hint: every path is a sequence of a "right" steps and b "up" steps; a path is fully determined by *which* of the $a + b$ moves are the rights. Counting paths becomes counting subsets. The same idea, encoding a complicated object as a string and then counting strings, appears often in algorithm design.

41.4 Probability: reasoning about chance

Randomized algorithms flip coins. Quicksort picks a random pivot; hashing scatters keys with a random function; sampling estimates a quantity from a few draws. To analyze them we need a precise language for "how likely" and "on average." That language is probability, and it is built directly on the counting you just learned.

41.4.1 Sample spaces and events

Intuition

Picture every possible outcome of a random experiment laid out as points. That collection is the *sample space* Ω. An *event* is just a subset of Ω — a question you can ask whose answer is yes or no, like "did the die show an even number?" Probability is then a way of assigning a weight to each event so the weights behave sensibly.

A *probability* is a function Pr assigning to each event $A \subseteq \Omega$ a number $\Pr(A)$, obeying three axioms.

Formal

Axioms of probability (Kolmogorov).

1. $\Pr(A) \geq 0$ for every event A. (No negative chances.)
2. $\Pr(\Omega) = 1$. (*Something* happens, with certainty.)
3. If A and B are disjoint ($A \cap B = \emptyset$), then $\Pr(A \cup B) = \Pr(A) + \Pr(B)$. (Disjoint chances add.)

From these three, useful consequences fall out with a line or two of reasoning:

$$\Pr(\overline{A}) = 1 - \Pr(A), \qquad \Pr(\emptyset) = 0, \qquad \Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B).$$

That last one — *inclusion–exclusion* — corrects for the overlap you would otherwise double-count, exactly mirroring the Venn diagram from Section 41.1.

When all $|\Omega|$ outcomes are *equally likely* (a fair die, a uniformly random pivot), probability reduces to counting:

$$\Pr(A) = \frac{|A|}{|\Omega|} = \frac{\text{favorable outcomes}}{\text{total outcomes}}.$$

This is why the counting section came first: in the equally-likely world, *probability is just a ratio of two counts*.

Example 41.5 (A pair of dice)**.** Roll two fair six-sided dice. The sample space is the 36 ordered pairs (i, j) with $1 \le i, j \le 6$, all equally likely. What is $\Pr(\text{sum} = 7)$? Count the favorable pairs: $(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)$ — six of them. So

$$\Pr(\text{sum} = 7) = \frac{6}{36} = \frac{1}{6}.$$

And $\Pr(\text{sum} = 12)$? Only $(6,6)$ works, so $\frac{1}{36}$. Once the outcomes are counted, the probability is a single division.

second die j

first die i						
6,1	6,2	6,3	6,4	6,5	6,6
5,1	5,2	5,3	5,4	5,5	5,6
4,1	4,2	4,3	4,4	4,5	4,6
3,1	3,2	3,3	3,4	3,5	3,6
2,1	2,2	2,3	2,4	2,5	2,6
1,1	1,2	1,3	1,4	1,5	1,6

Each of the 36 cells is one equally-likely outcome (i, j); the six shaded cells on the anti-diagonal are exactly the rolls with $i + j = 7$, giving $6/36 = 1/6$.

41.4.2 Conditional probability

New information changes the odds. Once you learn that an event B has occurred, the sample space effectively shrinks to B, and you re-measure everything inside it.

Formal

The *conditional probability* of A given B (with $\Pr(B) > 0$) is

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

Intuitively, you are restricting attention to the world where B happened (divide by $\Pr(B)$ to renormalize so that world has total weight 1), and asking how much of that world also has A (the numerator $\Pr(A \cap B)$).

Example 41.6 (Dice, with a hint)**.** Roll two fair dice. Let A be "the sum is 7" and B be "the first die is 3." We have $\Pr(B) = \frac{6}{36} = \frac{1}{6}$. The pairs in $A \cap B$: only $(3, 4)$ gives both a leading 3 and a sum of 7, so $\Pr(A \cap B) = \frac{1}{36}$. Thus

$$\Pr(A \mid B) = \frac{1/36}{1/6} = \frac{1}{6}.$$

Knowing the first die showed 3 left the chance of a 7 unchanged at $\frac{1}{6}$ — a hint of the independence idea coming next.

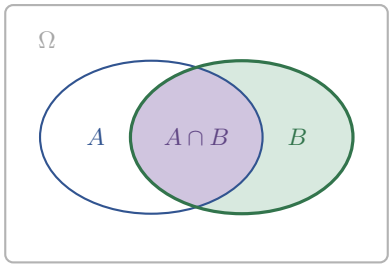
Rearranging the definition gives the *chain rule* $\Pr(A \cap B) = \Pr(A \mid B)\,\Pr(B)$, and summing conditional pieces over a partition $B_1, \ldots, B_k$ of Ω gives the *law of total probability*

$$\Pr(A) = \sum_{i=1}^{k} \Pr(A \mid B_i)\,\Pr(B_i),$$

which is how you compute a probability by splitting into cases — the probabilistic counterpart of the sum rule. Flipping the conditioning around yields *Bayes' rule*,

$$\Pr(B \mid A) = \frac{\Pr(A \mid B)\,\Pr(B)}{\Pr(A)} = \frac{\Pr(A \mid B)\,\Pr(B)}{\Pr(A \mid B)\,\Pr(B) + \Pr(A \mid B^c)\,\Pr(B^c)},$$

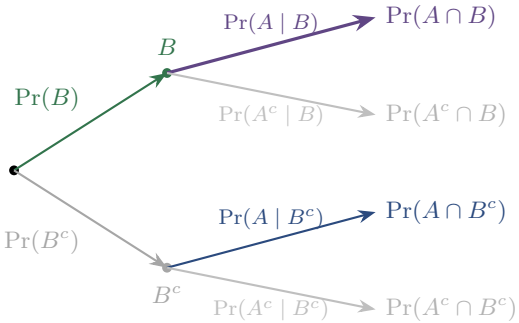
the basis of every "update my belief given evidence" calculation. Figure 41.2 visualizes both the Venn restriction and the branching tree of possibilities.



(a) Universe Shrinks to B

Conditioning on B renormalizes by $\Pr(B)$:

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}$$



(b) Probability Tree & Bayes Inversion

Total prob: $\Pr(A) = \Pr(A \cap B) + \Pr(A \cap B^c)$

Bayes' rule: $\Pr(B \mid A) = \dfrac{\Pr(A \cap B)}{\Pr(A)}$

Figure 41.2: Conditional probability and Bayes' rule. **(a)** Once event B is known to occur, the effective sample space contracts to B. The probability of A is the ratio of the overlap $A \cap B$ to the area of B. **(b)** The probability tree traces the paths yielding evidence A. Summing the two paths gives the Law of Total Probability; dividing the upper path by the sum gives Bayes' rule for posterior belief $\Pr(B \mid A)$.

41.4.3 Independence

Intuition

Two events are *independent* when learning one tells you nothing about the other. Knowing the coin came up heads says nothing about tomorrow's weather.

Formally, A and B are *independent* when

$$\Pr(A \cap B) = \Pr(A)\ \Pr(B),$$

equivalently $\Pr(A \mid B) = \Pr(A)$ whenever $\Pr(B) > 0$ (conditioning on B does not change the probability of A). In Example 41.6 the sum-is-7 event was independent of the first die's value — a small surprise, but the formula confirms it.

Common trap

Independence is a property of the *numbers*, not a feeling. "These seem unrelated" is not a proof. You must check $\Pr(A \cap B) = \Pr(A)\Pr(B)$. And beware: events can be *pairwise* independent yet not *mutually* independent — the three-way relationship can fail even when every pair checks out. When an analysis needs full independence (as in some hashing arguments), pairwise is not enough, and assuming it without checking is a common source of broken proofs.

41.5 Random variables and expectation

41.5.1 What a random variable is

Intuition

A *random variable* is a number that depends on the random outcome — a measurement you read off after the experiment runs. "The number of comparisons quicksort makes" is a random variable: it has no single fixed value, because it depends on which random pivots came up.

Formally a (discrete) random variable X is a function $X : \Omega \to \mathbb{R}$. For each value v it can take, the probability $\Pr(X = v)$ is shorthand for the probability of the event $\{\, \omega \in \Omega : X(\omega) = v \,\}$. The list of these probabilities, over all values, is the *distribution* of X.

41.5.2 Expectation: the average value

The *expected value* (or mean) of X is the average of its values, each weighted by how likely it is:

$$\mathbb{E}[X] = \sum_{v} v \cdot \Pr(X = v),$$

the sum running over all values X can take. Think of it as the center of mass of the distribution — the balance point if you placed weight $\Pr(X = v)$ at position v on a ruler.

Example 41.7 (Expected value of one die)**.** For a fair die, $X \in \{1, 2, 3, 4, 5, 6\}$ each with probability $\frac{1}{6}$:

$$\mathbb{E}[X] = \frac{1}{6}(1 + 2 + 3 + 4 + 5 + 6) = \frac{21}{6} = 3.5.$$

Notice the average, 3.5, is a value the die can never actually show. That is fine: expectation is a long-run average, not a prediction of any single roll.

41.5.3 Linearity of expectation

This is the most useful single fact in randomized algorithm analysis.

Formal

Linearity of expectation. For *any* random variables X and Y and constants a, b,

$$\mathbb{E}[aX + bY] = a\,\mathbb{E}[X] + b\,\mathbb{E}[Y].$$

This holds *whether or not X and Y are independent.* It extends to any finite sum: $\mathbb{E}[X_1 + \cdots + X_n] = \mathbb{E}[X_1] + \cdots + \mathbb{E}[X_n]$.

The "no independence required" clause is what makes linearity so effective. You can break a complicated total into simple pieces, take each piece's expectation in isolation, and add, never once worrying about how the pieces interact.

Focus

The standard recipe: to find $\mathbb{E}[X]$ where X counts how many times something happens, write $X = X_1 + X_2 + \cdots + X_n$ where each X_i is an *indicator* — a 0/1 random variable that is 1 exactly when the i-th thing happens. The key identity is

$$\mathbb{E}[X_i] = 0 \cdot \Pr(X_i = 0) + 1 \cdot \Pr(X_i = 1) = \Pr(X_i = 1).$$

The expectation of an indicator is just the probability of the event it indicates. So by linearity, $\mathbb{E}[X] = \sum_i \Pr(i\text{-th event happens})$. A hard average becomes a sum of easy probabilities. This is the technique behind quicksort's expected-comparison count, hashing's expected chain length, and dozens more.

Example 41.8 (Expected number of fixed points)**.** Shuffle n cards uniformly at random. How many cards land in their original position on average? Let X be that count, and let $X_i = 1$ if card i stays put. Card i is equally likely to land in any of the n positions, so $\Pr(X_i = 1) = \frac{1}{n}$, hence $\mathbb{E}[X_i] = \frac{1}{n}$. The cards' fates are tangled (if one is fixed, the others are constrained), but linearity still applies:

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{n} = n \cdot \frac{1}{n} = 1.$$

On average, exactly *one* card stays in place, whatever the size of the deck. We never needed the distribution of X, only n simple indicators.

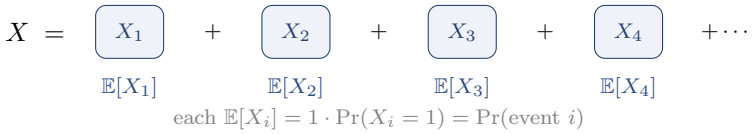
Algorithm lab

Verify Example 41.8 by brute force for $n = 3$. The $3! = 6$ permutations of $(1, 2, 3)$, with their fixed-point counts:

$$123 \to 3, \quad 132 \to 1, \quad 213 \to 1,$$
$$231 \to 0, \quad 312 \to 0, \quad 321 \to 1.$$

Average $= (3+1+1+0+0+1)/6 = 6/6 = 1$. Exactly the predicted answer, and you can see why no shortcut would have been faster than linearity once n grows.

The picture below shows the indicator recipe. The total X splits into a row of simple 0/1 flags, each easy to average on its own; linearity lets us sum those averages even though the flags are correlated.



41.5.4 Variance, briefly

Expectation tells you the center; *variance* tells you the spread. It is the expected squared distance from the mean,

$$\mathrm{Var}[X] = \mathbb{E}\big[(X - \mathbb{E}[X])^2\big] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2,$$

and its square root $\sqrt{\mathrm{Var}[X]}$ is the *standard deviation*. Unlike expectation, variance is *not* linear in general — but it does add for *independent* variables: $\mathrm{Var}[X+Y] = \mathrm{Var}[X] + \mathrm{Var}[Y]$ when X, Y are independent. Variance is what you reach for when you want to show that a randomized algorithm performs well not merely on average, but with high concentration around that mean.

41.6 Useful inequalities

Often you cannot compute a probability exactly — but you can *bound* it, and a bound is usually all an algorithm analysis needs ("the failure probability is at most $1/n^2$" is plenty). Three inequalities cover most needs.

41.6.1 Markov's inequality

Formal

Markov's inequality. If $X \geq 0$ and $t > 0$, then

$$\Pr(X \geq t) \leq \frac{\mathbb{E}[X]}{t}.$$

The intuition: a nonnegative quantity cannot often be far above its average, because those rare large values would, by themselves, drag the average up past where it actually sits. Equivalently, taking $t = c\,\mathbb{E}[X]$: a nonnegative random variable is at least c times its mean with probability at most $1/c$. It uses only nonnegativity and the mean, which is why it applies so widely and why it is often loose.

Example 41.9 (A quick Markov bound)**.** An algorithm runs in expected time $\mathbb{E}[T] = 100$ steps. What can we say about the chance it takes 1000 or more? With $t = 1000$,

$$\Pr(T \geq 1000) \leq \frac{100}{1000} = \frac{1}{10}.$$

So at most a 10% chance of being ten times over budget — derived from the mean alone, knowing nothing else about T.



Markov says the probability mass to the right of t is capped: a nonnegative variable with mean $\mathbb{E}[X]$ cannot pile too much weight far above its average without dragging the average up past where it sits.

41.6.2 Chebyshev's inequality

When you also know the variance, you can bound deviations on *both* sides of the mean, and tightly.

Formal

Chebyshev's inequality. For any random variable X with finite variance, and any $t > 0$,

$$\Pr\big(|X - \mathbb{E}[X]| \geq t\big) \leq \frac{\operatorname{Var}[X]}{t^2}.$$

It is just Markov applied to the nonnegative variable $(X - \mathbb{E}[X])^2$. Consequently, the probability of straying k standard deviations from the mean is at most $1/k^2$ — small spread means X clusters tightly around $\mathbb{E}[X]$, which is the concentration guarantee mentioned in the discussion of variance.

41.6.3 The union bound

Formal

Union bound (Boole's inequality). For any events $A_1, \ldots, A_n$,

$$\Pr\Big(\bigcup_{i=1}^{n} A_i\Big) \leq \sum_{i=1}^{n} \Pr(A_i).$$

The union bound is simple and widely used. If each of n "bad" events is individually unlikely, the chance that *any* of them happens is at most the sum of their probabilities. It is the inclusion–exclusion identity with the (subtracted) overlaps thrown away — which can only make the right side bigger, so it is always a valid upper bound.

Example 41.10 (Union bound in action)**.** Suppose a randomized data structure has n operations, and each individually fails with probability at most $1/n^2$. Then the probability that *some* operation fails is at most

$$\sum_{i=1}^{n} \frac{1}{n^2} = n \cdot \frac{1}{n^2} = \frac{1}{n}.$$

So with probability at least $1 - 1/n$ — "with high probability" — the entire run is flawless. This pattern, bound each rare event then union them, is how most high-probability guarantees in this book are proved.

41.7 Summary: Sets, Counting, and Probability Reference

When analyzing randomized algorithms, counting configurations, or establishing average-case bounds, use this reference checklist:

- **Sets and Relations**: Sets model dynamic collections and universes; relations model graph edges, equivalences, and partial orders. An equivalence relation partitions a set into disjoint equivalence classes.

- **Permutations and Combinations**: Order matters in permutations ($P(n,k) = \frac{n!}{(n-k)!}$); order does not matter in combinations ($\binom{n}{k} = \frac{n!}{k!(n-k)!}$). Total subsets equal 2^n.

- **Discrete Probability and Bayes**: Probabilities of disjoint events add; independent probabilities multiply ($\Pr(A \cap B) = \Pr(A)\Pr(B)$). Conditional probability updates beliefs via Bayes' theorem: $\Pr(A \mid B) = \frac{\Pr(B|A)\Pr(A)}{\Pr(B)}$.

- **Linearity of Expectation**: $\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ holds unconditionally, without requiring independence. Decomposing complex variables into sums of 0/1 indicator variables is the standard method for randomized algorithm analysis.

- **Tail Bounds**: When full distributions are unknown, Markov's inequality bounds non-negative tails via the mean ($\Pr(X \geq t) \leq \mathbb{E}[X]/t$), while Chebyshev's inequality bounds deviations via the variance ($\Pr(|X - \mathbb{E}[X]| \geq t) \leq \mathrm{Var}[X]/t^2$). Boole's union bound $\Pr(\bigcup A_i) \leq \sum \Pr(A_i)$ bounds the probability that any of several failure events occurs.

Our final appendix turns to linear algebra and discrete mathematics in Appendix 42. There, we formalize vectors, matrices, dot products, determinants, and graph representations—the linear-algebraic language underlying graph reachability, Markov chain transitions, and high-dimensional algorithmics.

Cost summary

The counting and probability facts you will reuse most:

Quantity	Formula
Subsets of an n-set	2^n
Orderings of n items	$n!$
Ordered k from n	$P(n,k) = n!/(n-k)!$
Unordered k from n	$\binom{n}{k} = n!\,/\,(k!\,(n-k)!)$
Equally-likely probability	$\Pr(A) = \lvert A\rvert/\lvert\Omega\rvert$
Conditional probability	$\Pr(A \mid B) = \Pr(A \cap B)/\Pr(B)$
Expectation	$\mathbb{E}[X] = \sum_v v\,\Pr(X = v)$
Linearity	$\mathbb{E}[X+Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ (always)
Indicator expectation	$\mathbb{E}[X_i] = \Pr(\text{event } i)$
Markov	$\Pr(X \geq t) \leq \mathbb{E}[X]/t$
Chebyshev	$\Pr(\lvert X - \mathbb{E}[X]\rvert \geq t) \leq \mathrm{Var}[X]/t^2$
Union bound	$\Pr(\bigcup A_i) \leq \sum \Pr(A_i)$

Exercises

Exercise 41.1 (Warmup: set arithmetic)**.** Let $A = \{1,2,3,4,5\}$ and $B = \{2,4,6\}$ with universe $U = \{1,2,\ldots,8\}$. Compute $A \cup B$, $A \cap B$, $A \setminus B$, $\overline{A}$, and $\lvert A \times B\rvert$. Then verify De Morgan's law $\overline{A \cup B} = \overline{A} \cap \overline{B}$ by computing both sides.

Exercise 41.2 (Type-checking $\in$ versus $\subseteq$)**.** For the set $S = \{1, \{2\}, \{1,2\}\}$, decide whether each statement is true or false, and explain in one sentence: (a) $1 \in S$; (b) $\{2\} \in S$; (c) $2 \in S$; (d) $\{1\} \subseteq S$; (e) $\{\{2\}\} \subseteq S$.

Exercise 41.3 (Permutation versus combination)**.** A standard deck has 52 cards. (a) How many ways are there to deal a 5-card poker hand, where order does not matter? (b) How many ways to deal 5 cards in order to five seated players, one each, where who gets which card matters? Express each as a product/factorial expression and as a number, and state which is larger and why.

Exercise 41.4 (Counting strings)**.** How many strings of length 4 over the alphabet $\{a,b,c\}$ contain at least one a? (Hint: it is far easier to count the complement — strings with *no* a — and subtract from the total. This complement trick is worth internalizing.)

Exercise 41.5 (Pascal by hand)**.** Use Pascal's rule $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ to build the row of $\binom{6}{k}$ for $k = 0,1,\ldots,6$, starting from the row for $n = 5$ $(1,5,10,10,5,1)$. Then check that your row sums to $2^6 = 64$.

Exercise 41.6 (Conditional dice)**.** Roll two fair dice. Given that the sum is 8, what is the probability that at least one die shows a 5? Identify the conditioning event B, list the outcomes in B, then list those that also satisfy A, and compute $\Pr(A \mid B)$ directly.

Exercise 41.7 (Independence check)**.** Roll two fair dice. Let A be "the first die is even" and B be "the sum is 7." Compute $\Pr(A)$, $\Pr(B)$, and $\Pr(A \cap B)$, and determine whether A and B are independent. Does the answer match your intuition?

Exercise 41.8 (Linearity to the rescue)**.** You roll a fair die n times. Let X be the number of times a 6 appears. Using indicator variables X_i (one per roll) and linearity of expectation, find $\mathbb{E}[X]$. Then, separately, find the expected number of times a 6 is *immediately followed by another* 6 among the $n-1$ adjacent pairs.

Exercise 41.9 (Expected maximum of two dice)**.** Roll two fair dice and let M be the larger of the two values (the maximum). Write out the distribution of M (the probability $\Pr(M = v)$ for $v = 1, \ldots, 6$) and compute $\mathbb{E}[M]$. (Hint: $\Pr(M \le v) = (v/6)^2$, since both dice must be at most v; turn that into $\Pr(M = v)$.)

Exercise 41.10 (Choosing the right inequality)**.** A randomized algorithm has expected running time $\mathbb{E}[T] = 50$ and variance $\mathrm{Var}[T] = 25$. (a) Use Markov's inequality to bound $\Pr(T \ge 200)$. (b) Use Chebyshev's inequality to bound $\Pr(|T - 50| \ge 30)$. (c) In one or two sentences, explain why Chebyshev gave a more informative bound here, and what extra information made that possible.

Exercise 41.11 (Union bound design)**.** You hash n keys into a table. For a fixed pair of distinct keys, suppose the probability they collide (land in the same slot) is exactly $1/m$, where m is the table size. Using the union bound over all pairs, show that the probability of *any* collision is at most $\binom{n}{2}/m$. Then determine how large m must be, as a function of n, to make this bound at most $1/2$. (This is the birthday-paradox threshold, and it explains why hash tables need spare capacity.)

Exercise 41.12 (Challenge: a counting bijection)**.** Prove that the number of ways to write a positive integer n as an ordered sum of positive integers (a *composition*, e.g. $4 = 1 + 3 = 3 + 1 = 2 + 2 = 1 + 1 + 2 = \cdots$) is exactly 2^{n-1}. (Hint: line up n dots in a row; between each adjacent pair of dots there is a gap, and you either "cut" there or not. Set up a bijection between compositions and subsets of the $n - 1$ gaps, then count the subsets.)

Chapter 42

Appendix: Matrices and Discrete Math

Following the sets, counting, and probability of Appendix 41, this final appendix gathers linear algebra and discrete mathematics: matrices, graphs, logarithms, and factorials. Throughout the book we used these objects as if you already knew them. Here we build each from the ground up: a matrix is a grid of numbers with bookkeeping rules; a graph is dots and lines; a logarithm answers "how many times did I halve?" For each idea we attach a concrete example and state the rules clearly so you can trust them.

Focus

This appendix maps to CLRS Appendices C and D (and a little of A and B). When CLRS writes "recall from Appendix D that matrix multiplication is associative," this is the chapter that covers it.

History and connection

For centuries, mathematicians solved systems of linear equations using ad-hoc substitutions, viewing numbers strictly as isolated scalars. The leap to treating an entire rectangular grid of numbers as a unified algebraic object came in 1858 from the English mathematician Arthur Cayley in *A Memoir on the Theory of Matrices*. Cayley defined matrix addition, scalar multiplication, matrix multiplication, and inverses, proving that matrices form a closed associative algebra. Yet Cayley's non-commutative multiplication ($AB \neq BA$) initially provoked controversy among traditionalists accustomed to commutative arithmetic, until Werner Heisenberg's 1925 matrix mechanics showed that non-commuting matrices are the natural way to describe quantum observables.

Half a century earlier, systematic elimination had already solved a pressing problem in astronomy. On New Year's Day in 1801, Italian astronomer Giuseppe Piazzi discovered the dwarf planet Ceres, tracking it for only forty days before it was lost in the glare of the Sun. A twenty-four-year-old Carl Friedrich Gauss took up the challenge: developing least squares and systematic row elimination—what we now call *Gaussian elimination*—Gauss solved the massive linear system governing Ceres's orbit, enabling astronomers to spot the planet exactly where he predicted in December 1801. In his 1809 treatise *Theoria Motus*, Gauss codified the

elimination method that remains the foundational numerical algorithm of scientific computing. A central geometric quantity in matrix algebra is the *determinant*. Rather than an abstract polynomial, the determinant $\det(A)$ measures the factor by which a transformation A scales volumes in space. A determinant of zero means space has been flattened into a lower dimension—a plane collapsed onto a line or point—irreversibly destroying information and explaining why singular matrices cannot be inverted. For over a century, matrix multiplication was assumed to require $\Theta(n^3)$ operations, until Volker Strassen's 1969 discovery of an $O(n^{2.807})$ algorithm disproved that assumption. Today, discovering the true exponent ω of matrix multiplication remains a major open problem in theoretical computer science.

42.1 Matrices: grids of numbers that *do* something

42.1.1 What a matrix is

A *matrix* is a rectangular grid of numbers. We describe its shape by saying "m by n," written $m \times n$, meaning m rows and n columns. Here is a 2×3 matrix (two rows, three columns):

$$A = \begin{pmatrix} 2 & 0 & -1 \\ 5 & 3 & 4 \end{pmatrix}.$$

The number in row i, column j is called the (i, j) *entry* and written A_{ij} (or a_{ij}). So $A_{11} = 2$, $A_{13} = -1$, and $A_{23} = 4$. Always read the row index first, then the column. That ordering is a convention you must keep straight, because A_{23} and A_{32} are different requests entirely (and in our example A_{32} does not even exist – there is no third row).

Intuition

Picture a spreadsheet. A matrix is exactly a spreadsheet of numbers with the column headers and row labels stripped away, leaving just the cells. Everything we do to matrices is something you could imagine doing to a spreadsheet: add two sheets cell by cell, scale every cell, or combine two sheets with a more elaborate row-meets-column rule. Hold onto the spreadsheet picture and you will rarely get lost.

A matrix with the same number of rows and columns ($m = n$) is called *square*. A matrix with a single column ($n = 1$) is a *column vector*; one with a single row ($m = 1$) is a *row vector*. We will write vectors as lowercase letters, like x, and matrices as uppercase, like A.

Focus

Three numbers fully pin down the "shape situation" of any matrix product, and keeping them visible saves you from most mistakes: the matrix is $m \times n$, the row index i runs $1 \le i \le m$, and the column index j runs $1 \le j \le n$. Write the shape next to every matrix while you are learning.

42.1.2 Adding matrices and scaling them

Two matrices of the *same shape* are added entry by entry. If A and B are both $m \times n$, then $A + B$ is the $m \times n$ matrix whose (i, j) entry is $A_{ij} + B_{ij}$. For example:

$$\begin{pmatrix} 2 & 0 \\ 5 & 3 \end{pmatrix} + \begin{pmatrix} 1 & 4 \\ -2 & 1 \end{pmatrix} = \begin{pmatrix} 2+1 & 0+4 \\ 5-2 & 3+1 \end{pmatrix} = \begin{pmatrix} 3 & 4 \\ 3 & 4 \end{pmatrix}.$$

If the shapes do not match, the sum is undefined – there is no row to pair with a missing row. *Scaling* (multiplying by a single number c, called a *scalar*) multiplies every entry by c:

$$3 \cdot \begin{pmatrix} 2 & 0 \\ 5 & 3 \end{pmatrix} = \begin{pmatrix} 6 & 0 \\ 15 & 9 \end{pmatrix}.$$

Addition is commutative ($A+B = B+A$) and associative ($(A+B)+C = A+(B+C)$) for the same reason ordinary number addition is: it all happens entry by entry, and the entries are just numbers.

42.1.3 The matrix product

Matrix *multiplication* is the one rule that looks strange the first time and obvious the tenth time. To multiply A by B and form $C = AB$, the shapes must *interlock*: the number of columns of A must equal the number of rows of B. If A is $m \times n$ and B is $n \times p$, then $C = AB$ is $m \times p$, and

$$C_{ij} \;=\; \sum_{k=1}^{n} A_{ik}\, B_{kj}.$$

In words: the (i, j) entry of the product is the *dot product* of row i of A with column j of B. You march across the row and down the column in lockstep, multiply the paired numbers, and add up the results.

Intuition

The shared dimension n is the "handshake." Row i of A reaches out with n fingers; column j of B reaches back with n fingers; each finger meets its partner, they multiply, and you total the handshake. If the finger counts do not match, no handshake is possible – that is exactly why the inner dimensions must agree.

Let us do one slowly. Take

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 3 \end{pmatrix} \quad (2 \times 2), \qquad B = \begin{pmatrix} 4 & 5 \\ 6 & 7 \end{pmatrix} \quad (2 \times 2).$$

The inner dimensions match ($2 = 2$), so $C = AB$ is 2×2. Compute each entry as a row-times-column handshake:

$$\begin{aligned} C_{11} &= (1)(4) + (2)(6) = 4 + 12 = 16, \\ C_{12} &= (1)(5) + (2)(7) = 5 + 14 = 19, \\ C_{21} &= (0)(4) + (3)(6) = 0 + 18 = 18, \\ C_{22} &= (0)(5) + (3)(7) = 0 + 21 = 21. \end{aligned}$$

So

$$AB = \begin{pmatrix} 16 & 19 \\ 18 & 21 \end{pmatrix}.$$

Algorithm lab

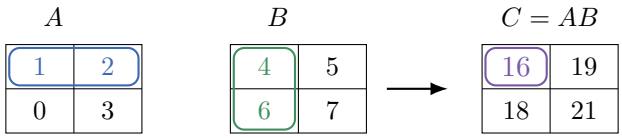
Trace it yourself. Compute BA for the same two matrices and compare it with AB above. You should get $BA = \left(\begin{smallmatrix} 4 & 23 \\ 6 & 33 \end{smallmatrix}\right)$, which is *not* equal to AB: the order in which you multiply matrices matters. We will turn this observation into a rule below.

Common trap

The single most common matrix mistake is assuming $AB = BA$. **Matrix multiplication is not commutative.** It is also easy to forget the shape rule and try to multiply, say, a 2×3 by a 2×2: the inner dimensions (3 and 2) disagree, so the product does not exist. Always check the handshake before you compute a single entry.

42.1.4 A picture of the handshake

The diagram below shows how the highlighted row of A and the highlighted column of B produce a single entry of C.



$$C_{11} = (1)(4) + (2)(6) = 16$$

The blue row of A shakes hands with the green column of B: $(1)(4) + (2)(6) = 16$, which lands in the purple top-left cell of C. Every other entry is computed the same way from a different row and column.

42.1.5 The identity matrix and the transpose

The *identity matrix* I_n is the $n \times n$ square matrix with 1's down the main diagonal (the entries I_{ii}) and 0's everywhere else. For example,

$$I_3 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

It earns its name: for any $m \times n$ matrix A we have $I_m A = A$ and $A I_n = A$. Multiplying by I changes nothing, exactly as multiplying a number by 1 changes nothing. The identity is the "do-nothing" matrix.

The *transpose* of A, written A^{T}, flips the matrix across its main diagonal: row i becomes column i. Formally $(A^{\mathsf{T}})_{ij} = A_{ji}$. If A is $m \times n$, then A^{T} is $n \times m$:

$$A = \begin{pmatrix} 2 & 0 & -1 \\ 5 & 3 & 4 \end{pmatrix} \quad \Longrightarrow \quad A^{\mathsf{T}} = \begin{pmatrix} 2 & 5 \\ 0 & 3 \\ -1 & 4 \end{pmatrix}.$$

As Figure 42.1(a) highlights, row 1 of A swings down to become column 1 of A^{T}, and row 2 becomes column 2. When a matrix equals its own transpose ($A = A^{\mathsf{T}}$, forcing it to be square), it is called *symmetric* (Figure 42.1(b)). In a symmetric matrix, the main diagonal acts as a mirror: each entry above matches the corresponding entry below, $S_{ij} = S_{ji}$. Symmetric matrices show up constantly in graph algorithms, notably as adjacency matrices of undirected graphs where an edge between u and v implies an edge between v and u.

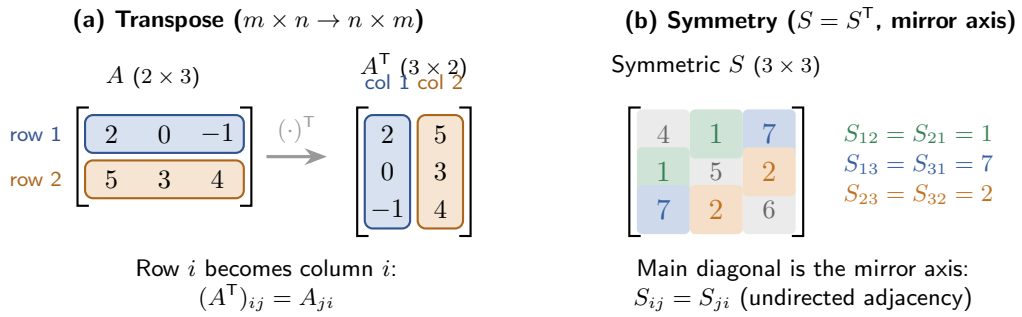


Figure 42.1: Matrix transposition and symmetry. **(a)** The transpose A^{T} swaps row and column indices ($(A^{\mathsf{T}})_{ij} = A_{ji}$), converting each row of an $m \times n$ matrix into the corresponding column of an $n \times m$ matrix. **(b)** A matrix is symmetric if $S = S^{\mathsf{T}}$, meaning entries are mirrored across the main diagonal ($S_{ij} = S_{ji}$). Adjacency matrices of undirected graphs are symmetric because an edge between u and v connects both ways.

A handy fact you will meet again: the transpose of a product reverses the order, $(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}}$.

42.1.6 The algebra rules

Here is the short list of laws you may assume in the rest of the book. They are not difficult to prove – each follows from pushing the sum $\sum_k A_{ik}B_{kj}$ around – but they are worth seeing collected in one place.

Formal

For matrices of compatible shapes and any scalar c:

- **Associative:** $(AB)C = A(BC)$.
- **Distributive:** $A(B + C) = AB + AC$ and $(A + B)C = AC + BC$.
- **Scalar slide:** $c(AB) = (cA)B = A(cB)$.
- **Identity:** $IA = A = AI$ (for square A).
- **Transpose of a product:** $(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}}$.
- **NOT commutative:** in general $AB \neq BA$.

The associative law is used in several chapters. It says that when you multiply a chain $A_1A_2 \cdots A_r$, the *answer* does not depend on how you parenthesize the multiplications – only the *cost* of computing it does. This difference in cost is what makes the matrix-chain multiplication problem (a classic dynamic-programming example) interesting.

Why this matters

Why carry matrices into an algorithms book at all? Because they describe computations done *in bulk*. A single matrix multiply applies one linear transformation to a whole batch of vectors at once; this underlies 3D graphics pipelines, the layers of a neural network, Google's original PageRank computation, and the fast all-pairs-shortest-paths trick that reuses matrix multiplication over a funny "min-plus" arithmetic. When you can phrase a computation as a matrix product, decades of fast multiplication algorithms (and fast hardware) become available to you.

42.1.7 The cost of multiplying matrices

How expensive is one matrix multiply? Take the square case, A and B both $n \times n$, so $C = AB$ is also $n \times n$. The product has n^2 entries. Each entry C_{ij} is a handshake of length n: n multiplications and $n - 1$ additions. So the total arithmetic is

$$\underbrace{n^2}_{\text{entries}} \times \underbrace{(2n-1)}_{\text{ops each}} = 2n^3 - n^2 = \Theta(n^3).$$

The naive algorithm is three nested loops, and counting them gives the same answer directly.

Algorithm 157: Naive square matrix multiplication

Input: $n \times n$ matrices A and B
Output: $C = AB$

```
for i ← 1 to n do
    for j ← 1 to n do
        s ← 0;
        for k ← 1 to n do
            s ← s + A_ik · B_kj;
        C_ij ← s;
return C;
```

The body $s \leftarrow s + A_{ik}B_{kj}$ runs exactly $n \cdot n \cdot n = n^3$ times, once for each triple (i, j, k), so the running time is $\Theta(n^3)$.

Cost summary

For two $n \times n$ matrices, the naive algorithm above costs:

- Time: $\Theta(n^3)$ arithmetic operations.
- Extra space: $\Theta(n^2)$ to store the output (or $\Theta(1)$ extra beyond the output, since each C_{ij} is computed independently).

Strassen's algorithm (CLRS Chapter 4) beats this with $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$ by trading some multiplications for additions via divide and conquer. The current theoretical record is below $n^{2.372}$, though nobody uses those galactic algorithms in practice.

Walking question

On your walk, picture the three nested loops as a $n \times n \times n$ cube of little "multiply-and-add" events. The naive algorithm visits every one of the n^3 cells. Strassen's insight is that those cells are not all independent – clever combinations let you skip about an eighth of the work at each level of recursion. Can you see, intuitively, why *recursion* (splitting each matrix into four quadrants) is the natural place to look for such savings?

42.2 A pinch of linear algebra

You do not need a full linear-algebra course to read this book, but a few facts recur. We collect them here with the lightest possible touch – enough to recognize the words, not to pass a final exam.

42.2.1 Vectors, linear combinations, and independence

A *vector* in this appendix is just a column matrix of n numbers, an arrow's worth of data. Given vectors $v_1, \dots, v_k$ and scalars $c_1, \dots, c_k$, the expression

$$c_1 v_1 + c_2 v_2 + \cdots + c_k v_k$$

is called a *linear combination* of them. It is the most natural way to "mix" vectors: scale each one, then add. The set of *all* linear combinations of a fixed collection of vectors is called their *span*.

The vectors are *linearly independent* if the only way to make the zero vector as a linear combination is the boring way, with every coefficient zero. If some non-trivial combination collapses to zero, the vectors are *dependent* – one of them is redundant, expressible from the others. For example, $(1, 0)$, $(0, 1)$, and $(2, 3)$ are dependent because $2(1, 0) + 3(0, 1) - (2, 3) = (0, 0)$ with not-all-zero coefficients.

Intuition

Independence means "no freeloaders." Each vector contributes a genuinely new direction that the others cannot reproduce by mixing. Dependence means at least one vector is a freeloader – you could delete it and lose nothing, because the remaining vectors can already reach it.

42.2.2 Determinant and inverse, briefly

For a square matrix A, the *determinant* $\det(A)$ is a single number computed from all the entries that tells you, among other things, whether A is *invertible*. For the 2×2 case the formula is short and worth memorizing:

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc.$$

A square matrix A has an *inverse* A^{-1} – a matrix with $AA^{-1} = A^{-1}A = I$ – exactly when $\det(A) \neq 0$. The inverse is the matrix analogue of a reciprocal: A^{-1} undoes whatever A does. For the 2×2 case, when $ad - bc \neq 0$,

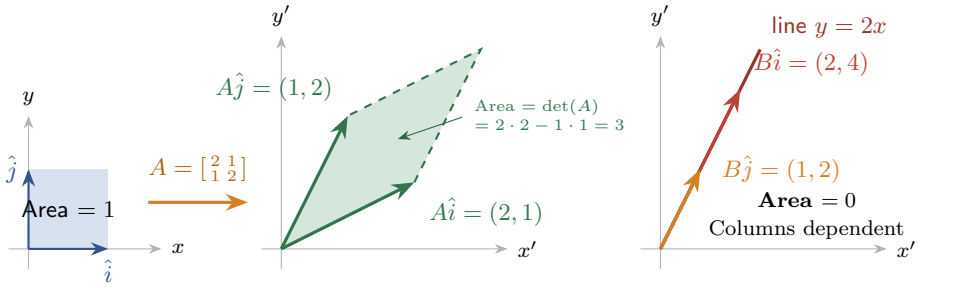
$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Let us check one. With $A = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 1 \end{smallmatrix}\right)$ we have $\det(A) = 2 \cdot 1 - 1 \cdot 1 = 1$, so $A^{-1} = \left(\begin{smallmatrix} 1 & -1 \\ -1 & 2 \end{smallmatrix}\right)$. Multiply to confirm:

$$\begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} = \begin{pmatrix} 2-1 & -2+2 \\ 1-1 & -1+2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I. \checkmark$$

Common trap

A square matrix with $\det(A) = 0$ is called *singular* and has *no* inverse at all – you cannot "divide" by it. This is the matrix world's version of dividing by zero, and the determinant tells you when it happens. Whenever an algorithm hands you a matrix it intends to invert (solving linear systems, least-squares fitting), a zero determinant is the place where things break.



(a) Invertible ($\det(A) = 3 \neq 0$)
Area scales by factor $\det(A)$
Inverse A^{-1} restores original square

(b) Singular ($\det(B) = 0$)
2D area squashed to 1D line
Irreversible: no inverse B^{-1} exists

Figure 42.2: The geometric meaning of the determinant. **(a)** A linear transformation maps the unit square to the parallelogram spanned by its columns $A\hat{\imath}$ and $A\hat{\jmath}$. The determinant $\det(A) = ad - bc$ is the factor by which areas scale. **(b)** When $\det(B) = 0$, the columns are linearly dependent; 2D area collapses into a 1D line, destroying dimensionality. Because a flattened line cannot be uniquely unpacked into a plane, singular matrices cannot be inverted.

42.3 Graphs as formal objects

So much of this book – shortest paths, spanning trees, network flow, topological sorting – lives on *graphs*. The main chapters use graphs operationally; here we pin down the definitions precisely, so that when a theorem says "let $G = (V, E)$ be a connected, undirected graph," every word has already been defined.

42.3.1 The definition

Definition 42.1 (Graph)**.** A *graph* is a pair $G = (V, E)$, where V is a finite set of *vertices* (also called *nodes*) and E is a set of *edges*. In a *directed* graph each edge is an ordered pair (u, v) with $u, v \in V$, drawn as an arrow from u to v. In an *undirected* graph each edge is an unordered pair $\{u, v\}$, drawn as a plain line between u and v.

That is the whole formal content: a set of dots and a set of connections between them. Everything else – paths, cycles, connectivity, weights – is built on top of this skeleton. We usually write $|V| = n$

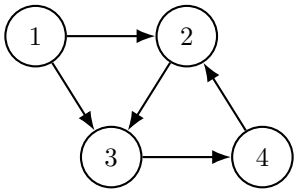
for the number of vertices and $|E| = m$ for the number of edges, and inside asymptotic notation it is traditional to drop the bars and just write V and E, so that "$O(V + E)$" means "$O(|V| + |E|)$."

Intuition

An undirected edge is a two-way street: if you can walk from u to v, you can walk back. A directed edge is a one-way street: the arrow (u, v) lets you go from u to v but says nothing about going back. Almost every modeling decision in graph algorithms starts with the question, "are my relationships symmetric?" Friendship on a typical social network is undirected; "follows" on a microblog is directed.

Here is a small directed graph $G = (V, E)$ with vertex set $V = \{1, 2, 3, 4\}$ and edge set

$$E = \{(1, 2),\ (1, 3),\ (2, 3),\ (3, 4),\ (4, 2)\}.$$



42.3.2 The vocabulary you actually need

A few terms come up on nearly every page of the graph chapters:

- **Adjacent / neighbor.** Vertex v is adjacent to u if the edge between them exists. In a directed graph we are careful about direction.
- **Degree.** In an undirected graph, the *degree* of a vertex is the number of edges touching it. In a directed graph we split this into *in-degree* (arrows coming in) and *out-degree* (arrows going out).
- **Path.** A *path* is a sequence of vertices $v_0, v_1, \ldots, v_\ell$ where each consecutive pair is joined by an edge. Its *length* is the number of edges, ℓ. A path is *simple* if it repeats no vertex.
- **Cycle.** A *cycle* is a path that starts and ends at the same vertex (with at least one edge). A graph with no cycles is *acyclic*; a directed acyclic graph gets the famous acronym *DAG*.
- **Connected.** An undirected graph is *connected* if there is a path between every pair of vertices. The maximal connected pieces are its *connected components*.
- **Weighted.** A *weight* function $w : E \to \mathbb{R}$ attaches a number to each edge – a distance, a cost, a capacity. Shortest-path and minimum-spanning-tree algorithms live on weighted graphs.

Formal

Handshake lemma. In any undirected graph, the sum of all vertex degrees equals twice the number of edges:

$$\sum_{v \in V} \deg(v) = 2\,|E|.$$

Why: each edge has two endpoints, so it contributes exactly 1 to the degree of each of its two

vertices – 2 in total. Summing over all edges counts each edge's contribution twice, once per endpoint. A pleasant corollary: the number of vertices with *odd* degree is always even.

42.3.3 Two ways to store a graph

Algorithms do not see your hand-drawn picture; they see a data structure. There are two standard representations, and choosing between them is a recurring real-world decision.

Adjacency matrix. Number the vertices $1, \ldots, n$ and build the $n \times n$ matrix M where $M_{ij} = 1$ if the edge (i, j) exists and $M_{ij} = 0$ otherwise. (For weighted graphs, store the weight instead of 1.) For our directed example above:

$$M = \begin{array}{c|cccc} & 1 & 2 & 3 & 4 \\ \hline 1 & 0 & 1 & 1 & 0 \\ 2 & 0 & 0 & 1 & 0 \\ 3 & 0 & 0 & 0 & 1 \\ 4 & 0 & 1 & 0 & 0 \end{array}$$

Reading row 1: vertex 1 points to 2 and 3, matching the edges $(1, 2)$ and $(1, 3)$. For an *undirected* graph the matrix is symmetric ($M = M^{\mathsf{T}}$), because the edge $\{u, v\}$ marks both M_{uv} and M_{vu}.

Adjacency list. For each vertex, store a list of its out-neighbors. Our example becomes:

$$1 \to [2, 3], \quad 2 \to [3], \quad 3 \to [4], \quad 4 \to [2].$$

Pictured as actual linked lists, one row per vertex, the structure looks like Figure 42.3: the leftmost column is the array of vertices, and each vertex points along a chain of its out-neighbors.

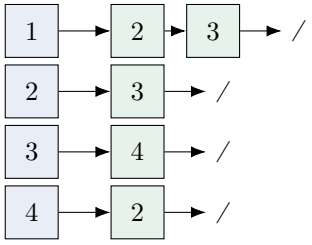


Figure 42.3: The adjacency list for the directed example. Each vertex (blue) heads a chain of its out-neighbors (green); "/" marks the end of a chain. Only the five edges that exist take up space – contrast the 16-cell matrix above.

Intuition

The matrix is a *grid*: instant "is there an edge from i to j?" lookups, but it always reserves n^2 cells whether the graph is dense or nearly empty. The list is a *phone book*: it stores only the edges that exist, so it is lean for sparse graphs, but answering "is i joined to j?" means scanning i's list. Dense graph, lots of edge queries: matrix. Sparse graph, lots of neighbor sweeps: list.

Cost summary

Let $n = |V|$ and $m = |E|$.

	Adjacency matrix	**Adjacency list**
Space	$\Theta(n^2)$	$\Theta(n+m)$
Test edge (u, v)?	$\Theta(1)$	$\Theta(\deg u)$
List neighbors of u	$\Theta(n)$	$\Theta(\deg u)$
Iterate all edges	$\Theta(n^2)$	$\Theta(n+m)$

Most textbook graph algorithms (BFS, DFS, Dijkstra, Prim) assume the adjacency *list*, because their running times like $O(V + E)$ only emerge when iterating neighbors is cheap.

Why this matters

The matrix view has a further use: the (i, j) entry of M^k (the adjacency matrix raised to the kth power) counts the number of distinct walks of length exactly k from vertex i to vertex j. So a question about *paths* turns into a question about *matrix powers*, and matrix exponentiation by squaring answers reachability and counting questions fast. This connects the matrix material in this appendix to the graph material.

Why does it work? The defining sum $(M^2)_{ij} = \sum_k M_{ik} M_{kj}$ adds a 1 for exactly those middle vertices k with both an edge $i \to k$ and an edge $k \to j$ – that is, for every two-edge route $i \to k \to j$. For our example, $(M^2)_{14} = 1$ because the single route $1 \to 3 \to 4$ exists (Figure 42.4), and you can read that very handshake off row 1 and column 4 of M.

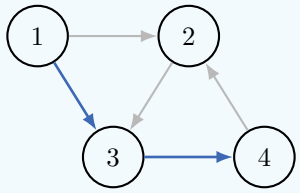


Figure 42.4: The unique length-2 walk from 1 to 4, namely $1 \to 3 \to 4$ (highlighted), is exactly what $(M^2)_{14} = 1$ records.

42.4 Logarithms, factorials, and identities you will reuse

The last toolbox is the analytic one: the functions that show up in every running time. We treat logarithms first because they count how many times a problem can be halved and measure the height of balanced binary trees, then factorials and a few summation identities.

42.4.1 What a logarithm really is

Start with the friendly definition: $\log_b x$ is the answer to the question "*to what power must I raise b to get x?*" So

$$\log_b x = y \quad \Longleftrightarrow \quad b^y = x.$$

The number b is the *base*. Because $2^3 = 8$, we have $\log_2 8 = 3$. Because $10^2 = 100$, we have $\log_{10} 100 = 2$. In other words, the logarithm is exponentiation read backwards.

Intuition

For base 2 – the base that dominates this book – there is an even homelier reading: $\log_2 x$ is roughly "how many times can I halve x before I reach 1?" Halve 16: you get $8, 4, 2, 1$ – four halvings (Figure 42.5), and indeed $\log_2 16 = 4$. Every time an algorithm cuts its problem in half (binary search, merge sort, balanced trees), a $\log_2 n$ slips into the running time, and that "how many halvings" picture is exactly why.

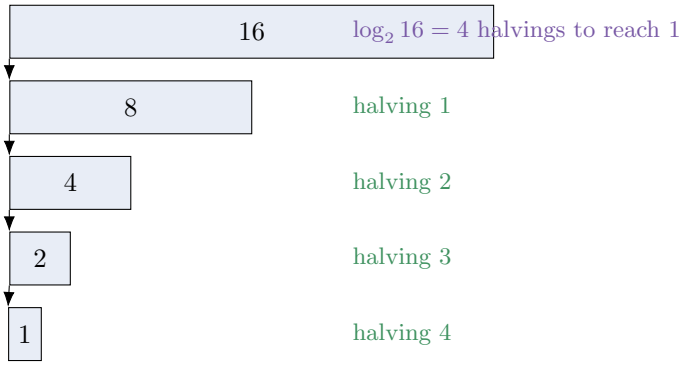


Figure 42.5: Reading $\log_2 16$ as repeated halving: $16 \to 8 \to 4 \to 2 \to 1$ is four steps, and indeed $\log_2 16 = 4$. Every "cut the problem in half" algorithm takes about this many steps.

The logarithm grows very slowly. To make $\log_2 n$ reach 10 you need $n = 1024$; to reach 20 you need $n \approx 10^6$; to reach 30 you need $n \approx 10^9$. This is why an $O(n \log n)$ sort feels almost as fast as a linear scan, and why an $O(\log n)$ search through a billion items takes about thirty comparisons.

42.4.2 The identities, with quick justifications

You will use these constantly. Each one is a direct consequence of the exponent rules, since logs and exponents are inverses.

Formal

For positive reals and a valid base $b > 0, b \neq 1$:

$$\begin{aligned}
\log_b(xy) &= \log_b x + \log_b y && \text{(a product becomes a sum)} \\
\log_b(x/y) &= \log_b x - \log_b y && \text{(a quotient becomes a difference)} \\
\log_b(x^k) &= k \log_b x && \text{(an exponent slides out front)} \\
\log_b 1 = 0, &\qquad \log_b b = 1 && \\
b^{\log_b x} = x, &\qquad \log_b(b^k) = k && \text{(inverse functions)} \\
\log_b x &= \frac{\log_a x}{\log_a b} && \text{(change of base)}
\end{aligned}$$

Here is why the product rule holds. Write $x = b^p$ and $y = b^q$, which is just saying $p = \log_b x$ and $q = \log_b y$. Then $xy = b^p \cdot b^q = b^{p+q}$, and reading that backwards gives $\log_b(xy) = p + q = \log_b x + \log_b y$. Every other identity falls out the same way, by translating to exponents and using the exponent laws.

The *change-of-base* formula matters most in an algorithms book. It says all logarithms are the same up to a constant multiplier, because $\log_b x = \log_a x / \log_a b$ and $\log_a b$ is a fixed constant once a and b are chosen. That is why we are cavalier and write "$O(\log n)$" without naming a base: switching from $\log_2$ to $\log_{10}$ to ln only multiplies by a constant, and big-O absorbs constant factors.

Common trap

Constants matter *outside* big-O, though. "Time $= 3\log_2 n$ seconds" and "time $= 3\ln n$ seconds" are genuinely different stopwatch readings – the second is about 1.44 times longer, since $\log_2 n = \ln n / \ln 2$ and $1/\ln 2 \approx 1.4427$. The base is free to ignore only when it lives behind an O, Θ, or Ω.

42.4.3 Factorials and how fast they explode

The *factorial* of a non-negative integer n is

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 2 \cdot 1, \qquad \text{with the convention } 0! = 1.$$

It counts the number of ways to arrange n distinct objects in a row: n choices for the first slot, $n-1$ for the next, and so on. Watch it run away from you:

n	1	2	3	4	5	6	7	10
$n!$	1	2	6	24	120	720	5040	3 628 800

By $n = 20$ the factorial has passed 2×10^{18}. This growth is why brute-force "try every permutation" algorithms (the naive traveling salesman, for instance) are hopeless beyond tiny inputs: 20! arrangements would take longer to enumerate than the age of the universe.

Intuition

Factorial growth is what limits brute force. Exponential growth (2^n) is already a disaster, but factorial growth is *worse than any fixed exponential*: $n!$ eventually outruns c^n for every constant c, because each new factor n is bigger than any fixed c. When you see $n!$ in a running time, read it as "this only works for n in the low twenties, ever."

How big is $n!$ exactly? *Stirling's approximation* gives a close estimate:

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n.$$

Taking logs (which is what we usually want for complexity), it tells us

$$\log(n!) = \Theta(n \log n).$$

This fact, $\log(n!) = \Theta(n \log n)$, is the basis of the well-known lower bound that *any* comparison-based sorting algorithm needs $\Omega(n \log n)$ comparisons: there are $n!$ possible orderings to distinguish, a decision tree distinguishing them needs $\log_2(n!)$ levels, and $\log_2(n!) = \Theta(n \log n)$.

42.4.4 Two summation identities worth memorizing

When you analyze loops, sums fall out, and two of them are everywhere.

Formal

Arithmetic series (the cost of nested loops where the inner count grows linearly):

$$1 + 2 + 3 + \cdots + n = \sum_{k=1}^{n} k = \frac{n(n+1)}{2} = \Theta(n^2).$$

Geometric series (the cost of recursion that shrinks by a factor each level), for $r \neq 1$:

$$1 + r + r^2 + \cdots + r^n = \sum_{k=0}^{n} r^k = \frac{r^{n+1} - 1}{r - 1}.$$

When $|r| < 1$ the infinite version converges: $\sum_{k=0}^{\infty} r^k = \dfrac{1}{1-r}$.

The arithmetic series has a one-line proof you can show a child, attributed (maybe apocryphally) to a young Gauss. Write the sum forwards and backwards and add column by column:

$$\begin{array}{rccccc} S = & 1 & + & 2 & +\cdots+ & n \\ S = & n & + & (n{-}1) & +\cdots+ & 1 \\ \hline 2S = & (n{+}1) & + & (n{+}1) & +\cdots+ & (n{+}1) \end{array}$$

There are n columns, each summing to $n+1$, so $2S = n(n+1)$ and $S = n(n+1)/2$. This is why a double loop where the inner loop runs $1, 2, 3, \ldots, n$ times costs $\Theta(n^2)$, the pattern behind selection sort, insertion sort's worst case, and naive nested scans.

Algorithm lab

Trace the geometric sum yourself for the case that matters most in this book, $r = 1/2$:

$$1 + \tfrac{1}{2} + \tfrac{1}{4} + \tfrac{1}{8} + \cdots = \frac{1}{1 - \frac{1}{2}} = 2.$$

Now connect it to algorithms. When a recursion does $\Theta(n)$ work at the top, $\Theta(n/2)$ across the next level, $\Theta(n/4)$ below that, and so on, the *total* work is $n(1 + \frac{1}{2} + \frac{1}{4} + \cdots) < 2n = \Theta(n)$. The geometric series is precisely why such "halving" recursions stay linear instead of blowing up. You will meet this exact computation when analyzing building a heap and several divide-and-conquer recurrences.

The geometric collapse is easiest to believe as a picture. Stack the per-level work as bars whose heights halve each time, and watch the running total creep toward $2n$ without ever reaching it (Figure 42.6).

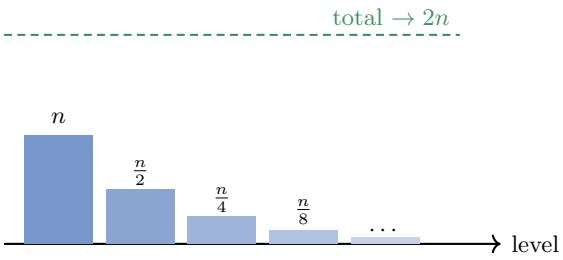


Figure 42.6: Per-level work in a "halving" recursion. Each bar is half the height of the one before, so the heights sum to $n(1 + \frac{1}{2} + \frac{1}{4} + \cdots) = 2n$. The dashed line is the ceiling the running total approaches but never crosses.

42.5 Putting the toolbox together

These tools are closely connected. The adjacency *matrix* let us turn a question about graph *paths* into a question about matrix *powers*. Computing those powers fast uses *exponentiation by squaring*, whose efficiency rests on a *logarithm* – you square $\log_2 k$ times to reach the kth power. Counting the arrangements a sorting algorithm must distinguish brought in *factorials*, and pinning down how big a factorial is brought back the *logarithm* through Stirling. The discrete and linear-algebraic tools are used together throughout the analysis of algorithms.

Focus

The four things to carry out of this appendix:

1. A matrix is a spreadsheet; multiplication is row-meets-column handshakes, costs $\Theta(n^3)$ naively, and is *not* commutative.
2. A graph is (V, E), stored as a matrix ($\Theta(n^2)$, fast edge tests) or a list ($\Theta(n + m)$, fast neighbor sweeps).
3. $\log_b x$ answers "b to what power is x?"; it grows very slowly, turns products into sums, and is base-independent under big-O.
4. $n!$ explodes faster than any exponential, and $\log(n!) = \Theta(n \log n)$ gives the comparison-sort lower bound.

This appendix completes the mathematical background for *Algorithms From the Ground Up*. The algorithms in the forty-two chapters of this book—from elementary sorting, binary search trees, and dynamic programming, through network flows, NP-completeness, parallel multithreading, machine learning, systems architecture, compilers, and quantum algorithms—are analyzed with the same tools: summations, counting and probability, and linear algebra.

Exercises

Exercise 42.1 (Warmup: shapes)**.** For each pair, decide whether the product AB exists and, if so, give the shape of the result: (a) A is 3×4, B is 4×2; (b) A is 2×5, B is 2×5; (c) A is $1 \times n$, B is

$n \times 1$; (d) A is $n \times 1$, B is $1 \times n$. For (c) and (d), what kind of object is the result?

Exercise 42.2 (Multiply by hand)**.** Let $A = \left(\begin{smallmatrix} 2 & -1 \\ 0 & 3 \end{smallmatrix}\right)$ and $B = \left(\begin{smallmatrix} 1 & 4 \\ 2 & 1 \end{smallmatrix}\right)$. Compute both AB and BA entry by entry and confirm they differ. Then compute A^{T} and verify $(AB)^{\mathsf{T}} = B^{\mathsf{T}}A^{\mathsf{T}}$.

Exercise 42.3 (Identity and inverse)**.** Verify that $I_2 A = A$ for the matrix A of the previous exercise. Then compute $\det(B)$ and, if it is nonzero, write down B^{-1} using the 2×2 formula and check that $BB^{-1} = I_2$.

Exercise 42.4 (Degrees and the handshake)**.** Draw any undirected graph on 5 vertices with exactly 6 edges. Write down the degree of each vertex, sum them, and confirm the sum equals $2 \times 6 = 12$. Then explain in one sentence why a graph in which every one of the 5 vertices has odd degree is impossible.

Exercise 42.5 (Two representations)**.** For the directed graph with $V = \{a, b, c, d\}$ and edge set

$$E = \{(a, b),\ (b, c),\ (c, a),\ (c, d)\},$$

write both the 4×4 adjacency matrix and the adjacency lists. Which representation uses less space here, and why?

Exercise 42.6 (Walks via matrix powers)**.** Using the adjacency matrix M of the graph in the previous exercise, compute M^2 and interpret the (a, c) entry: how many length-2 walks go from a to c? Trace such a walk in the picture to confirm.

Exercise 42.7 (Log gymnastics)**.** Without a calculator, evaluate: (a) $\log_2 64$; (b) $\log_2(1/8)$; (c) $\log_2(32 \cdot 4)$ using the product rule; (d) $\log_{10} 10^7$; (e) $\log_2 1000$ to the nearest integer, given $\log_{10} 2 \approx 0.301$ and the change-of-base formula.

Exercise 42.8 (Factorial intuition)**.** (a) Compute $8!$ by hand. (b) Using $0! = 1$, explain why $n! = n \cdot (n-1)!$ is a sensible recursive definition all the way down to $n = 1$. (c) Roughly how many permutations does $13!$ give, and why does that doom a brute-force ordering of just 13 tasks?

Exercise 42.9 (Sum it up)**.** Use the arithmetic-series formula to find $1 + 2 + \cdots + 100$. Then a loop runs its body once on the first iteration, twice on the second, $\ldots$, n times on the nth. Give a closed form for the total number of body executions and its Θ class.

Exercise 42.10 (Geometric collapse)**.** Evaluate the finite sum $1 + 3 + 9 + 27 + 81$ with the geometric formula ($r = 3$), then check by direct addition. Separately, argue that $1 + \frac{1}{3} + \frac{1}{9} + \cdots$ converges and find its value.

Exercise 42.11 (Challenge: cost of a matrix chain)**.** You must compute the product $A_1 A_2 A_3$ where A_1 is 10×100, A_2 is 100×5, and A_3 is 5×50. Using the rule that multiplying a $p \times q$ matrix by a $q \times r$ matrix costs $p \cdot q \cdot r$ scalar multiplications, compute the total cost of $(A_1 A_2)A_3$ versus $A_1(A_2 A_3)$. Which parenthesization is cheaper, and by how much? (This is the seed of the matrix-chain dynamic-programming problem.)

Exercise 42.12 (Challenge: Stirling and sorting)**.** Stirling gives $\log_2(n!) = \Theta(n \log n)$. A comparison sort can be modeled as a binary decision tree whose leaves are the $n!$ possible orderings. Explain why such a tree must have height at least $\log_2(n!)$, and conclude that every comparison-based sort makes $\Omega(n \log n)$ comparisons in the worst case.